TogetherWeGo: complete source code

FBLA Mobile Application Development 2026-27, “Together We Go: Group Trip Planner”. Expo SDK 57 / React Native / TypeScript, with a small Node sync server. 345 files, 35,246 lines. Photos and fonts are listed in ATTRIBUTIONS.md and are not printed.

1. README.md
2. ATTRIBUTIONS.md
3. package.json
4. app.json
5. eas.json
6. tsconfig.json
7. eslint.config.js
8. jest.config.js
9. jest.setup.js
10. src/app/(tabs)/_layout.tsx
11. src/app/(tabs)/budget.tsx
12. src/app/(tabs)/chat.tsx
13. src/app/(tabs)/explore.tsx
14. src/app/(tabs)/itinerary.tsx
15. src/app/(tabs)/profile.tsx
16. src/app/_layout.tsx
17. src/app/about.tsx
18. src/app/airport.tsx
19. src/app/bucket-list.tsx
20. src/app/crew.tsx
21. src/app/currency.tsx
22. src/app/destination/[id].tsx
23. src/app/destinations.tsx
24. src/app/documents.tsx
25. src/app/emergency.tsx
26. src/app/flights.tsx
27. src/app/forgot-password.tsx
28. src/app/help.tsx
29. src/app/how-it-works.tsx
30. src/app/index.tsx
31. src/app/join.tsx
32. src/app/map.tsx
33. src/app/nearby.tsx
34. src/app/new-trip.tsx
35. src/app/notifications.tsx
36. src/app/onboarding.tsx
37. src/app/packing.tsx
38. src/app/panorama.tsx
39. src/app/personalize.tsx
40. src/app/photos.tsx
41. src/app/phrasebook.tsx
42. src/app/quiz.tsx
43. src/app/recap.tsx
44. src/app/settings.tsx
45. src/app/sign-in.tsx
46. src/app/sign-up.tsx
47. src/app/survey.tsx
48. src/app/trips.tsx

49. src/app/weather.tsx
50. src/components/CurrencyConverter.tsx
51. src/components/CurrencyPickerSheet.tsx
52. src/components/GuidedTour.tsx
53. src/components/Logo.tsx
54. src/components/QrCode.tsx
55. src/components/ReminderService.tsx
56. src/components/ShareSheet.tsx
57. src/components/SkylineHero.tsx
58. src/components/TabBar.tsx
59. src/components/auth/DemoTicket.tsx
60. src/components/auth/GoogleSheet.tsx
61. src/components/auth/JoinCodeSheet.tsx
62. src/components/auth/__tests__/logic.test.ts
63. src/components/auth/kit.tsx
64. src/components/auth/legal.ts
65. src/components/auth/logic.ts
66. src/components/auth/slides.ts
67. src/components/budget/AddExpenseSheet.tsx
68. src/components/budget/AmountField.tsx
69. src/components/budget/BalancesCard.tsx
70. src/components/budget/BigAmount.tsx
71. src/components/budget/BudgetCapSheet.tsx
72. src/components/budget/DailyPlanner.tsx
73. src/components/budget/ExpenseDetailSheet.tsx
74. src/components/budget/ExpensesSection.tsx
75. src/components/budget/MoreTools.tsx
76. src/components/budget/SettleSheet.tsx
77. src/components/budget/SpendSummary.tsx
78. src/components/budget/__tests__/logic.test.ts
79. src/components/budget/logic.ts
80. src/components/budget/styles.ts
81. src/components/chat/ChatBubble.tsx
82. src/components/chat/ChatHeader.tsx
83. src/components/chat/Composer.tsx
84. src/components/chat/EmptyChat.tsx
85. src/components/chat/__tests__/logic.test.ts
86. src/components/chat/logic.ts
87. src/components/crew/AddMemberSheet.tsx
88. src/components/crew/ChecklistSection.tsx
89. src/components/crew/InviteCard.tsx
90. src/components/crew/InviteQrSheet.tsx
91. src/components/crew/MembersSection.tsx
92. src/components/crew/NewPollSheet.tsx
93. src/components/crew/PollsSection.tsx
94. src/components/crew/RoleSheet.tsx
95. src/components/crew/__tests__/crew.test.ts
96. src/components/crew/barcode.ts
97. src/components/crew/polls.ts
98. src/components/documents/DocFileBody.tsx
99. src/components/documents/DocRow.tsx
100. src/components/explore.tsx
101. src/components/flights/AddFlightSheet.tsx
102. src/components/flights/FlightTicket.tsx
103. src/components/flights/__tests__/flightForm.test.ts

104. src/components/flights/flightForm.ts
105. src/components/home/BudgetSection.tsx
106. src/components/home/CrewSection.tsx
107. src/components/home/DiscoverSection.tsx
108. src/components/home/HomeHero.tsx
109. src/components/home/MonoAmount.tsx
110. src/components/home/NearbySection.tsx
111. src/components/home/PicksSection.tsx
112. src/components/home/PlaySection.tsx
113. src/components/home/Podium.tsx
114. src/components/home/RadiusSlider.tsx
115. src/components/home/TriviaSlip.tsx
116. src/components/home/UpNextSection.tsx
117. src/components/home/VoteCard.tsx
118. src/components/home/WeatherSection.tsx
119. src/components/home/__tests__/logic.test.ts
120. src/components/home/__tests__/picks.test.ts
121. src/components/home/logic.ts
122. src/components/home/picks.ts
123. src/components/home/useHomeContext.ts
124. src/components/join/QrScanner.tsx
125. src/components/map/OfflineRadar.tsx
126. src/components/map/PinBadge.tsx
127. src/components/map/TripMap.tsx
128. src/components/map/TripMap.web.tsx
129. src/components/map/types.ts
130. src/components/more/AccountTicket.tsx
131. src/components/more/NearbyFilters.tsx
132. src/components/more/Tag.tsx
133. src/components/more/WeatherBoard.tsx
134. src/components/more/__tests__/groups.test.ts
135. src/components/more/groups.ts
136. src/components/panorama/Chrome.tsx
137. src/components/panorama/Scene.tsx
138. src/components/panorama/__tests__/nativeDriver.test.tsx
139. src/components/panorama/shapes.ts
140. src/components/personalize/Building.tsx
141. src/components/personalize/__tests__/build.test.ts
142. src/components/personalize/__tests__/look.test.ts
143. src/components/personalize/applyLook.ts
144. src/components/personalize/build.ts
145. src/components/personalize/kit.tsx
146. src/components/personalize/look.ts
147. src/components/personalize/options.ts
148. src/components/personalize/palette.ts
149. src/components/personalize/parts.tsx
150. src/components/personalize/steps/StepInterests.tsx
151. src/components/personalize/steps/StepLayout.tsx
152. src/components/personalize/steps/StepLook.tsx
153. src/components/personalize/steps/StepReveal.tsx
154. src/components/personalize/steps/StepVibe.tsx
155. src/components/personalize/steps/StepWelcome.tsx
156. src/components/photos/PhotoTile.tsx
157. src/components/photos/PhotoViewer.tsx
158. src/components/pickers.tsx

159. src/components/play/BucketTicket.tsx
160. src/components/play/PackingParts.tsx
161. src/components/play/QuizParts.tsx
162. src/components/play/RecapPoster.tsx
163. src/components/play/SurveyParts.tsx
164. src/components/play/__tests__/bucket.test.ts
165. src/components/play/__tests__/packingLogic.test.ts
166. src/components/play/__tests__/quizLogic.test.ts
167. src/components/play/__tests__/recapLogic.test.ts
168. src/components/play/__tests__/recommend.test.ts
169. src/components/play/bucket.ts
170. src/components/play/packingLogic.ts
171. src/components/play/quizLogic.ts
172. src/components/play/recapLogic.ts
173. src/components/play/recommend.ts
174. src/components/settings/AccessibilityCard.tsx
175. src/components/settings/AssistantCard.tsx
176. src/components/settings/DataCard.tsx
177. src/components/settings/DemoCard.tsx
178. src/components/settings/Group.tsx
179. src/components/settings/HomeLayoutCard.tsx
180. src/components/settings/LookCard.tsx
181. src/components/settings/NotificationsCard.tsx
182. src/components/settings/ProfileCard.tsx
183. src/components/settings/__tests__/blocks.test.ts
184. src/components/settings/blocks.ts
185. src/components/sync/SyncBadge.tsx
186. src/components/sync/SyncSettingsCard.tsx
187. src/components/sync/SyncTripCard.tsx
188. src/components/trip/ActivityCard.tsx
189. src/components/trip/ActivityParts.tsx
190. src/components/trip/AddActivitySheet.tsx
191. src/components/trip/DayTabs.tsx
192. src/components/trip/MemberPickerSheet.tsx
193. src/components/trip/NoTrip.tsx
194. src/components/trip/PlaceSearch.tsx
195. src/components/trip/TripMenuSheet.tsx
196. src/components/trip/VoteCard.tsx
197. src/components/trip/__tests__/addForm.test.ts
198. src/components/trip/__tests__/plan.test.ts
199. src/components/trip/addForm.ts
200. src/components/trip/meta.ts
201. src/components/trip/plan.ts
202. src/components/ui/Button.tsx
203. src/components/ui/Confetti.tsx
204. src/components/ui/Donut.tsx
205. src/components/ui/Fab.tsx
206. src/components/ui/Fold.tsx
207. src/components/ui/Input.tsx
208. src/components/ui/Plate.tsx
209. src/components/ui/Press.tsx
210. src/components/ui/RollingNumber.tsx
211. src/components/ui/Rule.tsx
212. src/components/ui/Screen.tsx
213. src/components/ui/Segmented.tsx

214. src/components/ui/Sheet.tsx
215. src/components/ui/Skeleton.tsx
216. src/components/ui/Slider.tsx
217. src/components/ui/SplitFlap.tsx
218. src/components/ui/StaggerText.tsx
219. src/components/ui/Stamp.tsx
220. src/components/ui/T.tsx
221. src/components/ui/Ticket.tsx
222. src/components/ui/__tests__/confetti.test.ts
223. src/components/ui/bits.tsx
224. src/components/ui/feedback.tsx
225. src/components/ui/motion.tsx
226. src/hooks/__tests__/useDraft.test.tsx
227. src/hooks/__tests__/usePlaceSearch.test.tsx
228. src/hooks/__tests__/useReduceMotion.test.ts
229. src/hooks/useDraft.ts
230. src/hooks/usePlaceSearch.ts
231. src/hooks/useReduceMotion.ts
232. src/hooks/useTrip.ts
233. src/i18n/index.ts
234. src/i18n/strings.ts
235. src/services/__tests__/assistant.test.ts
236. src/services/__tests__/placeSearch.test.ts
237. src/services/__tests__/pretendFriends.test.ts
238. src/services/__tests__/triviaReminder.test.ts
239. src/services/assistant.ts
240. src/services/currency.ts
241. src/services/files.ts
242. src/services/location.ts
243. src/services/notifications.ts
244. src/services/placeSearch.ts
245. src/services/pretendFriends.ts
246. src/services/print.ts
247. src/services/secrets.ts
248. src/services/share.ts
249. src/services/weather.ts
250. src/store/__tests__/auth.test.ts
251. src/store/__tests__/migrate.test.ts
252. src/store/__tests__/trips.test.ts
253. src/store/seed.ts
254. src/store/selectors.ts
255. src/store/slices/app.ts
256. src/store/slices/auth.ts
257. src/store/slices/budget.ts
258. src/store/slices/demo.ts
259. src/store/slices/files.ts
260. src/store/slices/groupTools.ts
261. src/store/slices/helpers.ts
262. src/store/slices/itinerary.ts
263. src/store/slices/social.ts
264. src/store/slices/trips.ts
265. src/store/slices/types.ts
266. src/store/store.ts
267. src/store/types.ts
268. src/store/useAppStore.ts

269. src/sync/__tests__/blobs.test.ts
270. src/sync/__tests__/crdt.test.ts
271. src/sync/__tests__/e2e.test.ts
272. src/sync/__tests__/engine.test.ts
273. src/sync/__tests__/invite.test.ts
274. src/sync/__tests__/secureRandom.test.ts
275. src/sync/__tests__/transport.test.ts
276. src/sync/actions.ts
277. src/sync/adapter.ts
278. src/sync/blobCache.ts
279. src/sync/blobs.ts
280. src/sync/crdt.ts
281. src/sync/engine.ts
282. src/sync/fileio.ts
283. src/sync/hlc.ts
284. src/sync/invite.ts
285. src/sync/notify.ts
286. src/sync/prefs.ts
287. src/sync/runtime.ts
288. src/sync/schema.ts
289. src/sync/secureRandom.ts
290. src/sync/status.ts
291. src/sync/storage.ts
292. src/sync/testing.ts
293. src/sync/timers.ts
294. src/sync/transport.ts
295. src/sync/useFile.ts
296. src/sync/view.ts
297. src/theme/__tests__/palettes.test.ts
298. src/theme/index.ts
299. src/theme/palettes.ts
300. src/utils/__tests__/guess.test.ts
301. src/utils/__tests__/money.test.ts
302. src/utils/__tests__/password.test.ts
303. src/utils/__tests__/points.test.ts
304. src/utils/__tests__/qr.test.ts
305. src/utils/__tests__/time.test.ts
306. src/utils/__tests__/trivia.test.ts
307. src/utils/__tests__/validation.test.ts
308. src/utils/contrast.ts
309. src/utils/format.ts
310. src/utils/geo.ts
311. src/utils/guess.ts
312. src/utils/money.ts
313. src/utils/password.ts
314. src/utils/points.ts
315. src/utils/qr.ts
316. src/utils/time.ts
317. src/utils/trivia.ts
318. src/utils/validation.ts
319. server/__tests__/server.test.js
320. server/index.js
321. server/store.js
322. e2e/README.md
323. e2e/auth.js

324. e2e/lib.js
325. e2e/motion.js
326. e2e/nesting.js
327. e2e/overflow.js
328. e2e/places.js
329. e2e/run.js
330. e2e/serve.js
331. e2e/shoot.js
332. e2e/targets.js
333. e2e/twophones.js
334. e2e/verify.js
335. e2e/verify2.js
336. e2e/verify3.js
337. e2e/verify4.js
338. scripts/source-pdf.js
339. docs/BUILDING.md
340. docs/DEMO_SCRIPT.md
341. docs/DESIGN.md
342. docs/JUDGE_CHECKLIST.md
343. docs/PLANNING.md
344. docs/RATING_SHEET.md
345. docs/SYNC.md

README.md

```
# TogetherWeGo ✈️ — Group Trip Planner

**FBLA Mobile Application Development 2026–27 · Topic: “Together We Go: Group Trip Planner”**

TogetherWeGo helps a group of friends or family plan, budget and enjoy a shared trip — from the first idea, through the trip itself, to the final recap. It runs natively on **iPhone and Android** (Expo / React Native, TypeScript) and keeps working **offline**, which matters at a conference venue with unreliable wifi.

<p>
<img src="docs/screenshots/03-home-night.jpg" width="170" alt="Home in the dark mode: the trip as a boarding pass with a flip-board countdown"/>
<img src="docs/screenshots/04-home-day.jpg" width="170" alt="Home in the light mode"/>
<img src="docs/screenshots/05-plan.jpg" width="170" alt="The Plan: a timeline with LOCKED IN stamps"/>
<img src="docs/screenshots/06-vote.jpg" width="170" alt="Vote tab with a live tally and Lock in"/>
<img src="docs/screenshots/07-budget.jpg" width="170" alt="Budget: money left, state stamp, who owes who"/>
<img src="docs/screenshots/08-chat-scout-ai.jpg" width="170" alt="Chat: Crew and Scout AI"/>
<img src="docs/screenshots/09-more.jpg" width="170" alt="More: a boarding pass and luggage-tag tiles"/>
<img src="docs/screenshots/10-crew.jpg" width="170" alt="Crew: the invitation ticket with code, barcode and QR"/>
<img src="docs/screenshots/11-flights.jpg" width="170" alt="Flights as boarding passes with each airport’s own clock"/>
<img src="docs/screenshots/12-packing.jpg" width="170" alt="Packing list with progress"/>
<img src="docs/screenshots/13-recap.jpg" width="170" alt="Trip recap: a passport-page poster you can share"/>
<img src="docs/screenshots/22-quiz.jpg" width="170" alt="Travel quiz with a podium"/>
<img src="docs/screenshots/23-bucket-list.jpg" width="170" alt="Bucket list tickets"/>
<img src="docs/screenshots/25-add-activity.jpg" width="170" alt="Add an activity: search a place, see how far it is"/>
<img src="docs/screenshots/26-settings.jpg" width="170" alt="Settings: the basics up front, the rest under Advanced"/>
<img src="docs/screenshots/15-make-it-yours.jpg" width="170" alt="Make it yours: pick a look and watch the screen repaint"/>
<img src="docs/screenshots/19-360-positano.jpg" width="170" alt="360° view of Positano"/>
<img src="docs/screenshots/20-360-paris.jpg" width="170" alt="360° view of Paris"/>
<img src="docs/screenshots/01-onboarding.jpg" width="170" alt="First-run slides"/>
<img src="docs/screenshots/02-sign-in.jpg" width="170" alt="Sign in, with a demo account ticket"/>
</p>

---

## 1. Run it on your phone (5 minutes)

**You need:** a laptop with [Node.js 20+](https://nodejs.org) and **Expo Go** on your phone. This project uses **Expo SDK 57**, so it must be the SDK 57 build of Expo Go:

* **Android:** install it from [expo.dev/go](https://expo.dev/go) (pick SDK 57; the Play Store copy can lag behind a new SDK).
* **iPhone:** Apple's App Store copy of Expo Go stops at SDK 54, so build your own with `npx eas-cli@latest go` and install it through TestFlight. That needs the paid Apple Developer Program, and each teammate's Apple ID must be added as an *internal tester* (App Store Connect → Expo Go → TestFlight). Expo's walkthrough: [Set up your environment → iOS device with Expo Go](https://docs.expo.dev/get-started/set-up-your-environment/).
* **No Apple account?** Use an Android phone, or the web build on the iPhone ([docs/BUILDING.md](docs/BUILDING.md)).

```bash
cd together-we-go
npm install
```
```

```
npx expo start            # same wifi for phone + laptop
# or, on school / conference wifi that blocks devices from seeing each other:
npx expo start --tunnel
```

Scan the QR code (iPhone: Camera app · Android: Expo Go → *Scan QR code*).

On an iPhone, sign in first. Expo Go only opens a project from your laptop when the laptop and the phone use the *same* Expo accoun
t: run `npx expo login` on the laptop (check with `npx expo whoami`), then tap the account icon (top right) in Expo Go and sign in with
that account. Android phones do not need this.

A teammate cannot open the app?
One running `npx expo start` can serve several phones. Match what the teammate's phone says:

The phone shows	Fix
You need to be signed in to Expo Go and Expo CLI… or *You're signed in to Expo CLI as … and to Expo Go as …* (iPhone only)	Use the
same Expo account in both places: `npx expo whoami` on the laptop, account icon in Expo Go, then tap *Try again* (restart `npx expo star
t` if you changed the laptop's account). A teammate who wants their own account runs the app from their own laptop instead: `git clone`,
`npm install`, `npx expo login`, `npx expo start`.
Project is incompatible with this version of Expo Go or *requires a newer version of Expo Go*	Wrong Expo Go version: install the S
DK 57 build (see above).
It keeps spinning, or *Could not connect to the server* / *Network request failed*	Same wifi as the laptop (a phone hotspot is safes
t). Windows: allow Node.js on **private** networks in the firewall prompt. Turn VPNs off. iPhone: *Settings → Expo Go → Local Network* o
n. Not on the same wifi at all? Start with `npx expo start --tunnel`.
Scanning does nothing	iPhone: scan with the Camera app. In Expo Go choose *Enter URL manually* and type the `exp://…` address printe
d under the QR code.

Demo login (also shown on the sign-in screen — tap the box to fill it in):

Email	Password
`maya.chen@togetherwego.app`	`TravelTogether2026!`

The sample trip is generated **relative to today**, so “Amalfi Squad” (4 friends, 6 days in Positano, Italy) always starts in 3 days; a
second 10-day “Tokyo & Kyoto Explorer” trip, a Bali plan and a finished Barcelona trip show multi-trip management. *Settings → Demo time
line* rebuilds it as **upcoming**, **in progress (Day 2)** or **just finished** to show every stage.

Plan together on different phones
Friends can each use their own phone and see the same trip, live. Start the small sync server on your computer in a **second terminal**:

```bash
npm run server        # prints the address your phones will use, e.g. http://192.168.1.20:8787
```

Then on phone A: *More → Crew & roles → Share this trip across phones*. On phone B: *Join a trip* and type the invite code (or scan its
QR code). Chat, votes, the plan, expenses, photos, documents and receipts now update on both, and catch up after being offline. During d
evelopment the phones find the server by themselves; otherwise type its address in *Settings → Sync across phones*. Details, the merge r
ule and the limits are in **docs/SYNC.md**.

For the competition: run without Expo Go or venue wifi
Expo Go needs the laptop's dev server. For an installed copy, build an Android `.apk` with EAS (free account) or use Expo Go over a phon
e hotspot. Steps, the iPhone options and what each choice costs are in **docs/BUILDING.md**. Once installed, everyth
ing except live weather, live exchange rates, map tiles and the optional Claude AI works offline — and those fall back to cached or esti
mated data automatically.

Other scripts: `npm test` (unit tests), `npm run e2e` (browser tests, see [`e2e/README.md`](e2e/README.md)), `npm run web` (browser prev
iew), `npx tsc --noEmit` (type-check), `npm run lint`.

---

2. What it does — mapped to the brief

Brief area	Feature in TogetherWeGo	Where
Collaboration	Trips with auto-generated invite codes (`AML-2026`), join by code, deep link or **QR code**	Home code pill · Mor
e → Crew & roles · *Join a trip*
	Live sync across phones: chat, votes, plan, expenses, checklist, photos, documents and receipts, with offline catch-up	More →
Crew & roles → *Share this trip across phones* · docs/SYNC.md
	Roles: Organizer, Treasurer, Navigator, Food Lead, Photographer… with descriptions	More → Crew & roles
	Shared checklist with assignees	More → Crew & roles
	Voting: 👍/👎 on activities (majority auto-confirms) + multi-option polls	Plan → Vote tab (progress bar + **Lock in**) · More → Cr
ew & roles (polls)
Communication	Group trip chat with typing indicators	Chat tab
	Trip Assistant (AI chatbot) — answers from live trip data: budget, who owes who, weather, food nearby, next activity, flights, t
ime zones, currency, phrases, emergencies. Works offline; optional Claude mode with your own key	Chat tab (✨ button or quick chips)
Scheduling	Day-by-day itinerary/calendar with timeline, status pills, assignees	Plan tab
	Members upload flight tickets (photo/PDF); everyone’s arrivals & departures in one list	Flights
	Time zones: times stored as exact instants; toggle trip time ↔ your phone’s time	More → Flights & arrivals
	Trip map with itinerary pins, places and airports + **offline radar map** (no tiles needed)	Trip map
	Live reminders: tap 🔔 on an activity → real phone notification before it starts (10/30/60/120 min) + live countdown on Home	A
ctivity card · Settings
Budgeting	Group budget cap, spent so far, per-person, % used	Budget tab
	Spending breakdown by category (Wanderlog-style bar)	Budget tab
	Currency converter (live rates, cached offline) + **home ↔ local currency view**	Budget tab · Currency
	Expenses in any currency, split between chosen members, **settle-up with fewest payments**	Budget tab

	Daily spending planner + “budget finds” within budget & radius	Budget tab
	Printable planner (print or save as PDF)	Plan → printer button
Organization	Offline document vault (PDFs, photos, notes)	More → Offline Docs
	Weather (7-day forecast, best outdoor day, packing tips)	Weather
	Choose your own radius → popular places nearby	More → Explore Nearby (also an optional block on Home)
	Photo dump with likes, captions, per-day filter, share to socials	Photo dump
	Activity filters (adventure, sightseeing, food…) **and** time filters (under 1 h, 1–2 h, 2–4 h, half day, full day)	Plan → Filter · Nearby
	Search for a place while adding an activity: matches appear as you type, each with how far it is from you (saved places work offline; the rest comes from OpenStreetMap)	Plan → +
Other	Interactive travel quiz with timer, streak bonus and group leaderboard	Travel quiz
	6 languages: English, Español, Français, 日本語, हिन्दी, 中文	Settings → Language
	Bucket list with inspiration	Bucket list
	Tracks location → nearest airport, distance, drive time, directions	Emergency → Nearest airport
	Recommendation survey → ranked destinations with photography	For you
	Offline map	Trip map → Offline
Trip management	Multiple trips, switch / create / edit / leave / delete, planning → active → completed	My trips

Extras to stand out
- **360° city views** — drag (or use the arrows / auto-rotate) to spin all the way around 10 popular cities: Positano, Tokyo, Paris, New York, London, Dubai, Sydney, Singapore, Rio and Rome. Three parallax skyline layers, famous landmarks (tap a landmark chip to turn to it and read a fact), a compass readout and a **day / night toggle**: at night the windows light up, the landmarks glow, cars stream along the street and stars and a moon come out. Open it from Home (360° chip), a destination page or More → 360° City Views.
- **Skyline hero that follows the clock** — the Home photo switches between a sunny day look (sun glare, drifting clouds) and a blue-hour night look (stars, moon) based on the local time in the trip’s city, with a sun/moon chip to preview the other half of the day.
- **Lock in voting** — Plan → Vote shows each proposal’s vote count, a progress bar towards a majority and a **Lock in** button (organizer only) that confirms it and turns its reminder on.
- **Scout AI tab** — the group chat stays human; the Trip Assistant has its own tab with quick questions (budget, food nearby, weather, who owes who…).
- **Today card** — once the trip starts, Home shows “Today · Day 2 of 6” with a tick-off list.
- **Who’s bringing what** — the packing list assigns every item to a person, with a “Split the rest fairly” button.
- **Receipt photos** on expenses, and a shareable **story card** for the trip recap.
- **First-run tour** and a **How it works** page (architecture, security, validation and a rubric map) inside the app.
- **Make it yours** — a short personalisation flow (Welcome, Look, Interests, Layout, Vibe, then a "Building your trip HQ" reveal): 6 themes (Limoncello, Tomato, Lagoon, Bubblegum, Grape, Ink) in **light, dark or follow-my-phone**, a photo-tile interest grid that feeds *Picked for you*, three Home layouts (Planner, Money, Party) and a celebration setting (Chill, Lively, Max). Choosing a look repaints the survey screen live; everything can be changed later in Settings.
- **“Picked for you”** — Home ranks destinations against your interests and budget style and shows a match %.
- **Motion everywhere** — animated home hero (a plane flies a dotted route past drifting clouds and a pulsing sun), staggered card entrances, a tab highlight that glides between tabs, Ken Burns zoom on onboarding photos, floating logo, pulsing live-reminder bell. One *Animations* switch turns it all off for people who prefer less motion.
- **Trip recap** — stats (days, km between stops, votes, photos), highlights, final balances, rating, shareable summary.
- **Emergency & safety** — local police/ambulance numbers (tap to call), share live location, hospital & embassy finder.
- **Phrasebook** with **text-to-speech** pronunciation (Japanese, French, Spanish, Italian, Korean, Thai, Portuguese).
- **Smart packing list** — suggestions from the destination and the forecast (e.g. “IC card (Suica)”, umbrella when rain is likely).
- **Demo timeline** switch to show the whole trip lifecycle live (Settings → Advanced).
- **Kept simple** — forms ask for the few things that matter (adding an activity is: where, which day, what time) and fold the rest under *More options*; More is one grid of big buttons; Settings keeps the technical switches under *Advanced*.
- **Social media integration** — direct share into WhatsApp, Instagram, X, Facebook, Messenger, Telegram, Messages, Email + the native share sheet; photos and the **recap story card share as pictures**; invites are a code, a link and a **QR code** that a friend can scan from inside the app (or with their camera), and the link carries the sync server so their phone connects by itself. See *Honest limitations* for what needs developer accounts.
- Accessibility: screen-reader labels on every control, adjustable text size, haptics toggle, high-contrast palette, and the phone's **Reduce Motion** setting is respected (no animations, no auto-rotating 360° view).

Judges: [`docs/RATING_SHEET.md`](docs/RATING_SHEET.md) lists every line of the FBLA rating sheet with where it is shown. The 7-minute script is in [`docs/DEMO_SCRIPT.md`](docs/DEMO_SCRIPT.md).

---

3. How it’s built

```
src/
  app/            Screens (Expo Router — every file is a route)
    (tabs)/       Home · Plan · Budget · Chat · More
    *.tsx         Feature screens (flights, map, weather, photos, …)
  components/     Reusable UI kit (ui/), trip widgets (trip/), map (native + web)
  store/          Zustand store: data model, actions, selectors, demo seed
  services/       Side effects: weather, currency, notifications, assistant, files, print, share, location
  data/           Offline datasets: 32 destinations, curated places, 100+ airports, phrases, quiz…
  i18n/           Translations (6 languages) + useT() hook
  utils/          Validation, time zones, money maths, passwords, QR codes, formatting
  sync/           Phone-to-phone sync: merge rules, engine, files, invites (see docs/SYNC.md)
server/           The sync server (plain Node, no dependencies) and its tests
e2e/              Browser tests that drive the web build, including a two-phone sync run
docs/             Planning, rating-sheet map, demo script, sync, building, judge checklist
```

- **Architecture:** layered MVVM-style. Screens (views) read state through selector hooks (view-model) and call store actions; services wrap every external API/device capability so screens never talk to the network directly. One-way data flow.
- **State & storage:** Zustand + Immer, persisted to AsyncStorage → the whole app works offline and survives restarts.
- **Navigation:** Expo Router with `Stack.Protected` guards (signed-out users can only see onboarding/auth).
- **Platform-specific code:** `TripMap.tsx` (react-native-maps) vs `TripMap.web.tsx` (OpenStreetMap embed).

- **Theming:** `src/theme/palettes.ts` holds the 6 themes in light and dark (plain data, unit-tested so body text reaches 4.5:1 contrast everywhere) and `src/theme/index.ts` the other design tokens. The saved choice is read synchronously before any screen's styles are created, so every `StyleSheet` is built with the right colours; changing the look reloads the app. See `docs/DESIGN.md` for the design system and how each Law of UX is applied.
- **Animations:** `src/components/ui/motion.tsx` (FadeIn, Float, Pulse, looping values) on React Native's `Animated` API with the native driver on iOS/Android.
- **React Compiler** enabled; the code passes the compiler's lint rules (`npm run lint` → 0 problems) and `tsc --noEmit` with `strict: true`.
- **Small files.** The store is split into slices (`src/store/slices`), and the big screens into components (`src/components/home`, `crew`, `settings`, `chat`, `photos`, …). No screen file is over 300 lines.

Testing
- `npm test` runs **unit tests** for the money maths (exact settle-up), validation, time zones, password hashing (against the official RFC test vector), the store, the assistant (including the Claude request), invites and QR codes, and the whole sync stack (merge convergence on random runs, the real server over real HTTP).
- `npm run e2e` runs **browser tests** against the web build: every main flow, a sideways-overflow audit of every screen at three sizes, a check that no button sits inside another button, Reduce Motion, and two phones syncing through the real server. See [`docs/PLANNING.md`](docs/PLANNING.md) §9 for what each covers.
- Not covered: the parts that need a physical phone (native file upload and download, the camera QR scanner, "share as picture", notifications). They are built into the iPhone bundle but were not run on a device here.

Data handling & security
- Passwords are **salted and stretched with PBKDF2-HMAC-SHA256** (`@noble/hashes`, rounds stored in the hash, old hashes upgraded at next sign-in) — never stored in plain text. Covered by unit tests, including the official RFC test vector.
- "Remember me" off → the session is not persisted.
- The optional Anthropic API key lives in the **iOS Keychain / Android Keystore** (expo-secure-store), never in the app database.
- Documents are copied into the app's private storage so they stay available offline.
- Network calls have timeouts and fall back to cached or bundled data.
- **Sync privacy:** your Scout AI chat, other people's email addresses and flight ticket files never leave your phone. A shared trip is kept on the sync server you run yourself, behind a 24-character random id that only the invite code reveals (code look-ups are rate limited). There are no accounts and the traffic is plain HTTP, so run the server only on a network you trust.

Input validation (syntactic + semantic)
Email format, password strength meter, matching passwords, names (letters only), money amounts (`25` / `25.50`, positive, sane max), trip dates (end after start, ≤ 60 days, not in the past), invite-code format `ABC-1234`, flight numbers (`NH 175`), IATA codes checked against the airport database, arrival-after-departure across time zones, seat format, overlapping-activity warning, duplicate poll options, file-size limits.

---

4. Libraries, sources & licences
See **ATTRIBUTIONS.md** for every library, API, font, icon set and photo source. Planning documents: **docs/PLANNING.md**. Presentation script: **docs/DEMO_SCRIPT.md**.

5. Honest limitations
- **Sync needs a server you run.** Phones share a trip through `npm run server` on a computer they can reach (same wifi, a hotspot, or a typed address). There is no hosted service, no accounts, and the traffic is plain HTTP on your own network. See [docs/SYNC.md](docs/SYNC.md#limits-the-honest-list). Without a server the app still works fully on one phone (invite codes then join the demo directory, try `SEO-5521`).
- **Last writer wins per field.** Two people changing the *same* field at the same moment keep the later change. Chat messages, votes, expenses and photos are separate items, so they never clash.
- **Social media:** sharing uses the phone's share sheet and the apps' own link formats. Signing in with, or posting to, Instagram/Facebook/X *from inside the app* needs developer accounts and app review (OAuth credentials) that a student project cannot hold, so it is not simulated.
- **Not run on a physical phone:** on-device file upload/download, the camera QR scanner and "share as picture" (see Testing).
- The 360° city views are illustrated scenes drawn in code (SVG), not photographs, so they work offline and need no image licences. The Home hero's day/night effect is a bundled photo with code-drawn light, stars and clouds.
- "Continue with Google" is a simulated account chooser (it says so on screen); a store build would use Google OAuth (expo-auth-session).
- Claude AI mode requires the user's own Anthropic API key; without it the built-in offline assistant answers.
- Standalone Android builds need a Google Maps API key for the native map (Expo Go already includes one); the offline radar map needs nothing.
- Passwords live in accounts on the phone, hashed with PBKDF2 (10,000 rounds: a count chosen so sign-in stays quick in a phone's JavaScript engine, where a server would use 600,000 or Argon2id).

ATTRIBUTIONS.md

Attributions, sources & licences

TogetherWeGo was written for the FBLA Mobile Application Development event (2026–27). This file documents every third-party library, data source, font, icon set and image used, as required by the rating sheet's *Documentation and copyright compliance* item.

Starter template
The project was scaffolded with `npx create-expo-app` (default template, MIT licence — see `LICENSE`, © 650 Industries / Expo). The template's example screens were removed; all screens, components, data and services in `src/` are original work for this project.

Libraries (all open source)
Package	Version	Licence
`expo` (SDK 57) and Expo modules: `expo-router`, `expo-notifications`, `expo-location`, `expo-image`, `expo-image-picker`, `expo-document-picker`, `expo-file-system`, `expo-print`, `expo-sharing`, `expo-speech`, `expo-crypto`, `expo-secure-store`, `expo-camera` (scanning invite QR codes), `expo-build-properties`, `expo-clipboard`, `expo-haptics`, `expo-linear-gradient`, `expo-linking`, `expo-localization`, `expo-asset`, `expo-font`, `expo-splash-screen`, `expo-status-bar`, `expo-system-ui`, `expo-constants`	57.x	MIT
`react`, `react-dom`	19.2.3	MIT
`react-native`	0.86.3	MIT
`react-native-web`	0.21.3	MIT

`react-native-maps`	1.27.2	MIT
`react-native-svg` (spending donut chart)	15.x	MIT
`react-native-safe-area-context`, `react-native-screens`, `react-native-gesture-handler`, `react-native-reanimated`, `react-native-worklets`	–	MIT
`@react-native-async-storage/async-storage`	2.2.0	MIT
`zustand`	5.0.15	MIT
`immer`	11.1.21	MIT
`@noble/hashes` (PBKDF2-SHA256 password hashing; audited, no dependencies)	2.4.0	MIT
`qrcode-generator` (draws the invite QR code; Kazuhiko Arase)	1.5.x	MIT
`react-native-view-shot` (turns the recap story card into a picture to share)	5.1.0	MIT
`jest`, `jest-expo`, `@types/jest` (unit tests, development only)	29.x / 57.x	MIT
`jsqr` (reads a QR code back, so the tests prove the codes can be scanned; development only)	1.4.0	Apache-2.0
`playwright-core` (browser tests, development only)	1.x	Apache-2.0

“QR Code” is a registered trademark of DENSO WAVE INCORPORATED. The sync server in `server/` uses only Node.js built-in modules.
| `@expo/vector-icons` (Ionicons, Material Community Icons) | 15.1.1 | MIT |
| `@expo-google-fonts/fraunces` (Fraunces by Undercase Type), `@expo-google-fonts/space-grotesk` (Space Grotesk by Florian Karsten), `@expo-google-fonts/space-mono` (Space Mono by Colophon Foundry) | 0.4.x | MIT (package) / SIL Open Font License 1.1 (fonts) |

APIs & data
Data	Source	Terms
Weather forecasts	[Open-Meteo](https://open-meteo.com)	Free, no key, CC BY 4.0 attribution
Exchange rates	[ExchangeRate-API open endpoint](https://www.exchangerate-api.com/docs/free) (`open.er-api.com`)	Free with attribution; cached for 6 h
Maps (phone)	Apple Maps (iOS) / Google Maps (Android) via react-native-maps	Platform map terms
Maps (web preview)	[OpenStreetMap](https://www.openstreetmap.org/copyright) embed	© OpenStreetMap contributors, ODbL
Place search ("Where do you want to go?")	[Photon](https://photon.komoot.io) by komoot, searching [OpenStreetMap](https://www.openstreetmap.org/copyright) data	Free, no key; © OpenStreetMap contributors, ODbL. Only the words typed and a map point rounded to about 1 km are sent; the distances are worked out on the phone
Airport coordinates & codes	Public IATA codes and coordinates (compiled by hand, 100+ airports)	Factual data
Emergency numbers	Official tourism / government pages, e.g. [JNTO emergency numbers](https://www.jnto.go.jp/safety-tips/eng/emergency-numbers.html)	Factual data
Destination & place facts	Summarised in our own words from public tourism information	Original text
AI chatbot (optional)	[Anthropic Claude API](https://docs.claude.com) — only when a user adds their own API key	Anthropic usage policies

The built-in offline Trip Assistant is original rule-based code (`src/services/assistant.ts`) that reads the trip’s own data.

Photography
All photos are from [Unsplash](https://unsplash.com) and used under the [Unsplash License](https://unsplash.com/license) (free for commercial and non-commercial use, no permission needed). They are bundled in `assets/images/photos/` (resized/compressed) so the app looks complete offline. Source URLs:

File	Source
`amalfi.jpg`	https://images.unsplash.com/photo-1612698093158-e07ac200d44e
`tokyo.jpg`	https://images.unsplash.com/photo-1536098561742-ca998e48cbcc
`kyoto.jpg`	https://images.unsplash.com/photo-1493976040374-85c8e12f0c0e
`paris.jpg`	https://images.unsplash.com/photo-1502602898657-3e91760cbb34
`bali.jpg`	https://images.unsplash.com/photo-1537996194471-e657df975ab4
`bangkok.jpg`	https://images.unsplash.com/photo-1563492065599-3520f775eeed
`singapore.jpg`	https://images.unsplash.com/photo-1525625293386-3f8f99389edd
`cape-town.jpg`	https://images.unsplash.com/photo-1580060839134-75a5edca2e99
`new-york.jpg`	https://images.unsplash.com/photo-1534430480872-3498386e7856
`santorini.jpg`	https://images.unsplash.com/photo-1613395877344-13d4a8e0d49e
`rome.jpg`	https://images.unsplash.com/photo-1552832230-c0197dd311b5
`london.jpg`	https://images.unsplash.com/photo-1513635269975-59663e0ac1ad
`iceland.jpg`	https://images.unsplash.com/photo-1504893524553-b855bce32c67
`swiss-alps.jpg`	https://images.unsplash.com/photo-1530122037265-a5f1f91d3b99
`maldives.jpg`	https://images.unsplash.com/photo-1514282401047-d79a71a590e8
`machu-picchu.jpg`	https://images.unsplash.com/photo-1526392060635-9d6019884377
`ha-long-bay.jpg`	https://images.unsplash.com/photo-1528127269322-539801943592
`dubai.jpg`	https://images.unsplash.com/photo-1512453979798-5ea266f8880c
`barcelona.jpg`	https://images.unsplash.com/photo-1583422409516-2895a77efded
`sydney.jpg`	https://images.unsplash.com/photo-1506973035872-a4ec16b8e8d9
`seoul.jpg`	https://images.unsplash.com/photo-1517154421773-0529f29ea451
`istanbul.jpg`	https://images.unsplash.com/photo-1541432901042-2d8bd64b4a9b
`amsterdam.jpg`	https://images.unsplash.com/photo-1534351590666-13e3e96b5017
`prague.jpg`	https://images.unsplash.com/photo-1519677100203-a0e668c92439
`lisbon.jpg`	https://images.unsplash.com/photo-1585208798174-6cedd86e019a
`banff.jpg`	https://images.unsplash.com/photo-1561134643-668f9057cce4
`marrakech.jpg`	https://images.unsplash.com/photo-1597212618440-806262de4f6b
`rio.jpg`	https://images.unsplash.com/photo-1483729558449-99ef09a8c325
`mexico-city.jpg`	https://images.unsplash.com/photo-1512813195386-6cf811ad3542
`grand-canyon.jpg`	https://images.unsplash.com/photo-1615551043360-33de8b5f410c
`cinque-terre.jpg`	https://images.unsplash.com/photo-1499678329028-101435549a4e
`venice.jpg`	https://images.unsplash.com/photo-1514890547357-a9ee288728e0
`mount-fuji.jpg`	https://images.unsplash.com/photo-1528164344705-47542687000d
`onboard-1.jpg`	https://images.unsplash.com/photo-1501555088652-021faa106b9b
`onboard-2.jpg`	https://images.unsplash.com/photo-1469854523086-cc02fe5d8800
`onboard-3.jpg`	https://images.unsplash.com/photo-1507525428034-b723cf961d3e
`lagoon.jpg`	https://images.unsplash.com/photo-1506929562872-bb421503ef21
`shibuya.jpg`	https://images.unsplash.com/photo-1542051841857-5f90071e7989
`akihabara.jpg`	https://images.unsplash.com/photo-1540959733332-eab4deabeeaf
`fushimi-inari.jpg`	https://images.unsplash.com/photo-1478436127897-769e1b3f0f36

`omoide-yokocho.jpg`	https://images.unsplash.com/photo-1554797589-7241bb691973
`lantern-alley.jpg`	https://images.unsplash.com/photo-1528360983277-13d401cdc186
`fuji-blossom.jpg`	https://images.unsplash.com/photo-1490806843957-31f4c9a91c65
`tokyo-skyline.jpg`	https://images.unsplash.com/photo-1513407030348-c983a97b98d8
`kyoto-skyline.jpg`	https://images.unsplash.com/photo-1545569341-9eb8b30979d9
`sushi.jpg`	https://images.unsplash.com/photo-1579871494447-9811cf80d66c
`ramen.jpg`	https://images.unsplash.com/photo-1569718212165-3a8278d5f624
`planning.jpg`	https://images.unsplash.com/photo-1488646953014-85cb44e25828
`avatar-maya.jpg`	https://images.unsplash.com/photo-1494790108377-be9c29b29330
`avatar-diego.jpg`	https://images.unsplash.com/photo-1507003211169-0a1dd7228f2d
`avatar-priya.jpg`	https://images.unsplash.com/photo-1534528741775-53994a69daeb
`avatar-sam.jpg`	https://images.unsplash.com/photo-1500648767791-00dcc994a43e
`avatar-ana.jpg`	https://images.unsplash.com/photo-1438761681033-6461ffad8d80
`avatar-noah.jpg`	https://images.unsplash.com/photo-1506794778202-cad84cf45f1d

Portrait photos are used only as avatars for the fictional demo group (Maya, Diego, Priya, Sam, Ana, Noah). All demo people, trips, chats and bookings are fictional sample data.

App icon & logo
Original design for this project: forest-green tile, the Material Community Icons “airplane-takeoff” glyph (Apache 2.0 / MIT via `@expo/vector-icons`) and three dots representing the travel group. Rendered to PNG for the app icon, Android adaptive icon and splash screen.

Design reference
The visual style (cream background, forest-green accents, pill navigation, card layouts) follows the reference prototype video provided by the team.

Developer tooling
- **Ponytail** skills (`.claude/skills/ponytail*`, from [DietrichGebert/ponytail](https://github.com/DietrichGebert/ponytail) @ 9cc65d0, MIT licence) — coding-style instructions for the Claude Code assistant used while building the app. Not part of the app itself.
- The 360° city scenes (`src/components/panorama`, `src/data/panoramas.ts`) are original code-drawn illustrations of public landmarks. Landmark facts were written from public sources (e.g. official tower and building heights).

package.json

```
{
  "name": "together-we-go",
  "main": "expo-router/entry",
  "version": "1.0.0",
  "dependencies": {
    "@expo-google-fonts/fraunces": "^0.4.1",
    "@expo-google-fonts/space-grotesk": "^0.4.1",
    "@expo-google-fonts/space-mono": "^0.4.2",
    "@expo/vector-icons": "^15.0.2",
    "@noble/hashes": "^2.4.0",
    "@react-native-async-storage/async-storage": "2.2.0",
    "expo": "~57.0.30",
    "expo-asset": "~57.0.18",
    "expo-blur": "~57.0.3",
    "expo-build-properties": "~57.0.22",
    "expo-camera": "~57.0.6",
    "expo-clipboard": "~57.0.2",
    "expo-constants": "~57.0.20",
    "expo-crypto": "~57.0.4",
    "expo-document-picker": "~57.0.3",
    "expo-file-system": "~57.0.7",
    "expo-font": "~57.0.4",
    "expo-haptics": "~57.0.3",
    "expo-image": "~57.0.5",
    "expo-image-picker": "~57.0.20",
    "expo-linear-gradient": "~57.0.2",
    "expo-linking": "~57.0.13",
    "expo-localization": "~57.0.2",
    "expo-location": "~57.0.21",
    "expo-notifications": "~57.0.23",
    "expo-print": "~57.0.2",
    "expo-router": "~57.0.28",
    "expo-secure-store": "~57.0.4",
    "expo-sharing": "~57.0.23",
    "expo-speech": "~57.0.3",
    "expo-splash-screen": "~57.0.10",
    "expo-status-bar": "~57.0.1",
    "expo-system-ui": "~57.0.4",
    "immer": "^11.1.21",
    "qrcode-generator": "^1.5.2",
    "react": "19.2.3",
    "react-dom": "19.2.3",
    "react-native": "0.86.3",
    "react-native-gesture-handler": "~2.32.0",
    "react-native-maps": "1.27.2",
    "react-native-reanimated": "4.5.1",
    "react-native-safe-area-context": "~5.7.0",
    "react-native-screens": "~4.26.0",
    "react-native-svg": "15.15.4",
    "react-native-view-shot": "5.1.0",
    "react-native-web": "~0.21.0",
    "react-native-worklets": "0.10.1",
```

```
    "zustand": "^5.0.15"
  },
  "devDependencies": {
    "@types/jest": "29.5.14",
    "@types/react": "~19.2.2",
    "eslint": "^9.0.0",
    "eslint-config-expo": "~57.0.2",
    "jest": "~29.7.0",
    "jest-expo": "~57.0.6",
    "jsqr": "^1.4.0",
    "playwright-core": "^1.56.1",
    "typescript": "~6.0.3"
  },
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "server": "node server/index.js",
    "lint": "expo lint",
    "test": "jest",
    "e2e": "node e2e/run.js",
    "source-pdf": "node scripts/source-pdf.js"
  },
  "private": true
}
```

app.json

```
{
  "expo": {
    "name": "TogetherWeGo",
    "slug": "together-we-go",
    "description": "Plan, budget and enjoy group trips together.",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "togetherwego",
    "userInterfaceStyle": "automatic",
    "backgroundColor": "#F4EFE4",
    "ios": {
      "bundleIdentifier": "com.togetherwego.app",
      "supportsTablet": true,
      "infoPlist": {
        "NSAppTransportSecurity": {
          "NSAllowsLocalNetworking": true
        },
        "NSLocalNetworkUsageDescription": "TogetherWeGo connects to your group's sync server on your wifi so friends see the same trip."
      }
    },
    "android": {
      "package": "com.togetherwego.app",
      "adaptiveIcon": {
        "backgroundColor": "#16130E",
        "foregroundImage": "./assets/images/android-icon-foreground.png",
        "backgroundImage": "./assets/images/android-icon-background.png",
        "monochromeImage": "./assets/images/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false
    },
    "web": {
      "output": "single",
      "favicon": "./assets/images/favicon.png",
      "name": "TogetherWeGo",
      "themeColor": "#16130E",
      "backgroundColor": "#F4EFE4"
    },
    "plugins": [
      "expo-router",
      [
        "expo-splash-screen",
        {
          "backgroundColor": "#F4EFE4",
          "image": "./assets/images/splash-icon.png",
          "imageWidth": 160,
          "dark": {
            "backgroundColor": "#121212"
          }
        }
      ],
      [
        "expo-location",
        {
          "locationWhenInUsePermission": "TogetherWeGo uses your location to show how far places are from you, places nearby, the closes
t airport and to share your location in an emergency."
        }
```

```
      ],
      [
        "expo-image-picker",
        {
          "photosPermission": "TogetherWeGo lets you add trip photos and flight tickets from your library.",
          "cameraPermission": "TogetherWeGo uses the camera to take photos for the group photo dump and to scan the invite QR code from
a friend."
        }
      ],
      [
        "expo-notifications",
        {
          "color": "#16130E"
        }
      ],
      [
        "expo-camera",
        {
          "cameraPermission": "TogetherWeGo uses the camera to take photos for the group photo dump and to scan the invite QR code from
a friend.",
          "recordAudioAndroid": false
        }
      ],
      "expo-sharing",
      "expo-localization",
      "expo-secure-store",
      "expo-asset",
      "expo-font",
      [
        "expo-build-properties",
        {
          "android": {
            "usesCleartextTraffic": true
          }
        }
      ]
    ],
    "experiments": {
      "typedRoutes": true,
      "reactCompiler": true
    }
  }
}
```

eas.json

```
{
  "cli": {
    "version": ">= 16.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "preview": {
      "distribution": "internal",
      "android": {
        "buildType": "apk"
      }
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {}
  }
}
```

tsconfig.json

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "paths": {
      "@/*": [
        "./src/*"
      ],
      "@/assets/*": [
        "./assets/*"
      ]
    },
    "types": [
      "jest",
      "node"
    ]
  },
```

```
  "include": [
    "**/*.ts",
    "**/*.tsx",
    ".expo/types/**/*.ts",
    "expo-env.d.ts"
  ],
  "exclude": [
    "node_modules",
    "dist",
    "server"
  ]
}
```

eslint.config.js

```
// https://docs.expo.dev/guides/using-eslint/
const { defineConfig } = require('eslint/config');
const expoConfig = require("eslint-config-expo/flat");

const node = ['require', 'module', 'exports', 'process', 'Buffer', '__dirname', 'console', 'setTimeout', 'clearTimeout', 'URL', 'fetch',
'document', 'window', 'localStorage', 'Image', 'btoa', 'XMLSerializer', 'getComputedStyle'];
const jest = ['describe', 'it', 'expect', 'beforeEach', 'afterEach', 'beforeAll', 'afterAll', 'jest'];

module.exports = defineConfig([
  expoConfig,
  {
    ignores: ["dist/*", "e2e/.dist/**", "e2e/shots/**", "server/data/**"],
  },
  {
    // the sync server, the browser tests and the helper scripts are plain Node (CommonJS), not part of the app bundle
    files: ["server/**/*.js", "e2e/**/*.js", "scripts/**/*.js", "jest.config.js", "jest.setup.js"],
    languageOptions: {
      sourceType: 'commonjs',
      globals: Object.fromEntries([...node, ...jest].map((name) => [name, 'readonly'])),
    },
  },
]);
```

jest.config.js

```
// Unit tests run with jest-expo so React Native modules are mocked the same way Expo does it.
const preset = require('jest-expo/jest-preset');

module.exports = {
  preset: 'jest-expo',
  setupFiles: ['<rootDir>/jest.setup.js'],
  // images live in ./assets (like tsconfig's paths), everything else under ./src
  moduleNameMapper: { '^@/assets/(.*)$': '<rootDir>/assets/$1', '^@/(.*)$': '<rootDir>/src/$1' },
  // immer and zustand ship ES modules, so they must go through Babel like the Expo packages do.
  transformIgnorePatterns: [
    preset.transformIgnorePatterns[0].replace('standard-navigation))', 'standard-navigation|immer|zustand|@noble))'),
    ...preset.transformIgnorePatterns.slice(1),
  ],
  testPathIgnorePatterns: ['/node_modules/', '/dist/'],
};
```

jest.setup.js

```
// AsyncStorage has no native module under Jest, so use the in-memory mock that ships with it.
jest.mock('@react-native-async-storage/async-storage', () => require('@react-native-async-storage/async-storage/jest/async-storage-moc
k'));

// expo-crypto needs its native module; stand in with Node's crypto so salts are real random values.
jest.mock('expo-crypto', () => ({
  randomUUID: () => require('crypto').randomUUID(),
  getRandomBytes: (n) => new Uint8Array(require('crypto').randomBytes(n)),
}));
```

src/app/(tabs)/_layout.tsx

```
import { Tabs } from 'expo-router/js-tabs';

import { TabBar } from '@/components/TabBar';
import { colors } from '@/theme';

export default function TabsLayout() {
  return (
    <Tabs
      initialRouteName="explore"
      tabBar={(props) => <TabBar {...props} />}
      screenOptions={{ headerShown: false, sceneStyle: { backgroundColor: colors.bg } }}>
      <Tabs.Screen name="explore" />
      <Tabs.Screen name="itinerary" />
      <Tabs.Screen name="budget" />
      <Tabs.Screen name="chat" />
```

```
      <Tabs.Screen name="profile" />
    </Tabs>
  );
}
```

src/app/(tabs)/budget.tsx

```
import { useState } from 'react';
import { View } from 'react-native';

import { AddExpenseSheet } from '@/components/budget/AddExpenseSheet';
import { BalancesCard } from '@/components/budget/BalancesCard';
import { BudgetCapSheet } from '@/components/budget/BudgetCapSheet';
import { DailyPlanner } from '@/components/budget/DailyPlanner';
import { ExpenseDetailSheet } from '@/components/budget/ExpenseDetailSheet';
import { ExpensesSection } from '@/components/budget/ExpensesSection';
import { MoreTools } from '@/components/budget/MoreTools';
import { SettleSheet } from '@/components/budget/SettleSheet';
import { SpendSummary } from '@/components/budget/SpendSummary';
import { CurrencyConverter } from '@/components/CurrencyConverter';
import { NoTrip } from '@/components/trip/NoTrip';
import { Fab } from '@/components/ui/Fab';
import { Screen, Header } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { SectionHeader } from '@/components/ui/bits';
import { destinationById } from '@/data/destinations';
import { useRatesLoader } from '@/hooks/useTrip';
import { useT } from '@/i18n';
import { convert, useRates } from '@/services/currency';
import type { Expense } from '@/store/types';
import { totalSpent, useActiveTrip, useAppStore } from '@/store/useAppStore';
import { space } from '@/theme';
import { money } from '@/utils/format';
import { daysBetween, todayInTz } from '@/utils/time';

export default function Budget() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <BudgetInner key={trip.id} />;
}

/**
 * The Budget tab. From the top: a summary ink pass (what is left, how it is going, where it went), who owes who with a Settle up butto
n,
 * the expenses as a receipt, and the rarely used tools folded away. The one main action is the Expense button in the corner.
 * It owns the few things the parts share (home or local currency, which sheet is open) and hands the rest to components/budget.
 */
function BudgetInner() {
  const t = useT();
  useRatesLoader();
  const trip = useActiveTrip()!;
  const rates = useRates();
  const home = useAppStore((s) => s.settings.homeCurrency);
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const dest = destinationById(trip.destinationId);
  const local = dest?.currency ?? 'EUR';

  const [view, setView] = useState<'home' | 'local'>('home');
  const [conv, setConv] = useState<[string, string]>([home, local === home ? 'EUR' : local]);
  const [detail, setDetail] = useState<Expense | null>(null);
  const [settleOpen, setSettleOpen] = useState(false);
  const [addOpen, setAddOpen] = useState(false);
  const [capOpen, setCapOpen] = useState(false);
  /** True once a settle-up on this screen cleared the last balance, so the SETTLED stamp slams on (never on the first draw). */
  const [justSettled, setJustSettled] = useState(false);
  const [today] = useState(() => todayInTz(dest?.tz ?? 'UTC'));

  const viewCode = view === 'local' && home !== local ? local : home;
  const fmt = (usd: number) => money(convert(usd, 'USD', viewCode, rates), viewCode);

  const spent = totalSpent(trip);
  const catTotals: Record<string, number> = {};
  trip.expenses.filter((e) => !e.settlement).forEach((e) => (catTotals[e.category] = (catTotals[e.category] ?? 0) + e.amount));
  const byCat = Object.entries(catTotals).sort((a, b) => b[1] - a[1]);
  const tripLen = daysBetween(trip.startDate, trip.endDate) + 1;
  const daysLeft = Math.max(1, Math.min(tripLen, daysBetween(today, trip.endDate) + 1));

  return (
    <Screen
      bottomInset={30}
      header={<Header back={false} kicker={trip.name} title={t('budget.title')} />}
      fab={<Fab label="Expense" accessibilityLabel="Add expense" testID="add-expense-open" onPress={() => setAddOpen(true)} />}>
      {/* Only worth a switch when the two currencies differ (a trip in the home country has one). */}
      {home !== local ? (
        <View style={{ marginBottom: space.xxl }}>
          <Segmented
```

```
            options={[
              { key: 'home', label: `Home · ${home}` },
              { key: 'local', label: `Local · ${local}` },
            ]}
            value={view}
            onChange={setView}
          />
        </View>
      ) : null}

      <View>
        <SpendSummary byCat={byCat} spent={spent} budget={trip.budget} remaining={trip.budget - spent} per={spent / Math.max(1, trip.mem
bers.length)} daysLeft={daysLeft} fmt={fmt} onEditBudget={() => setCapOpen(true)} />
      </View>

      {/* With nothing logged there is nothing to split: the empty Expenses block below is the invitation, so this one stays out of the
way. */}
      {trip.expenses.some((e) => !e.settlement) ? (
        <View style={{ marginTop: space.section }}>
          <BalancesCard trip={trip} userId={userId} fmt={fmt} onSettle={() => setSettleOpen(true)} slam={justSettled} />
        </View>
      ) : null}

      <View style={{ marginTop: space.section }}>
        <ExpensesSection trip={trip} fmt={fmt} today={today} onOpen={setDetail} onAdd={() => setAddOpen(true)} />
      </View>

      <View style={{ marginTop: space.section }}>
        <MoreTools>
          <View>
            <SectionHeader title="Converter" />
            <CurrencyConverter from={conv[0]} to={conv[1]} onChange={(a, b) => setConv([a, b])} />
          </View>
          <DailyPlanner trip={trip} />
        </MoreTools>
      </View>

      <AddExpenseSheet visible={addOpen} onClose={() => setAddOpen(false)} trip={trip} userId={userId} />
      <ExpenseDetailSheet key={detail?.id ?? 'none'} detail={detail} onClose={() => setDetail(null)} trip={trip} fmt={fmt} />
      <SettleSheet visible={settleOpen} onClose={() => setSettleOpen(false)} trip={trip} onAllSettled={() => setJustSettled(true)} />
      <BudgetCapSheet visible={capOpen} onClose={() => setCapOpen(false)} trip={trip} />
    </Screen>
  );
}
```

src/app/(tabs)/chat.tsx

```
import { useFocusEffect } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { FlatList, KeyboardAvoidingView, Platform, ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { ChatBubble } from '@/components/chat/ChatBubble';
import { ChatHeader, type ChatTab } from '@/components/chat/ChatHeader';
import { Composer } from '@/components/chat/Composer';
import { EmptyChat } from '@/components/chat/EmptyChat';
import { NoTrip } from '@/components/trip/NoTrip';
import { Chip } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { tap } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { answerLocally, askClaude, type AssistantContext } from '@/services/assistant';
import { friendReply, planFriendReply } from '@/services/pretendFriends';
import { getApiKey } from '@/services/secrets';
import { useSyncStatus } from '@/sync/status';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { ChatMessage, Trip } from '@/store/types';
import { MAX_WIDTH, colors } from '@/theme';

const QUICK = ['chat.q.budget', 'chat.q.food', 'chat.q.weather', 'chat.q.next'];

const inAiThread = (m: ChatMessage) => m.channel === 'ai' || m.ai === true;

export default function Chat() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <ChatInner key={trip.id} />;
}

function ChatInner() {
  const t = useT();
  const trip = useActiveTrip()!;
  const insets = useSafeAreaInsets();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const send = useAppStore((s) => s.sendMessage);
```

```
  const aiMode = useAppStore((s) => s.settings.aiMode);
  const markKindRead = useAppStore((s) => s.markKindRead);
  const synced = useSyncStatus((s) => s.shared.includes(trip.id)); // real friends are on the other end: no pretend replies
  const [text, setText] = useState('');
  const [tab, setTab] = useState<ChatTab>('crew');
  const [typing, setTyping] = useState<string | null>(null);
  const list = useRef<FlatList<ChatMessage>>(null);
  const timers = useRef<ReturnType<typeof setTimeout>[]>([]);

  useFocusEffect(
    useCallback(() => {
      markKindRead('chat');
    }, [markKindRead]),
  );
  useEffect(() => () => timers.current.forEach(clearTimeout), []);

  const later = (fn: () => void, ms: number) => timers.current.push(setTimeout(fn, ms));

  const askAssistant = async (question: string, history: ChatMessage[]) => {
    setTyping('assistant');
    const s = useAppStore.getState();
    const live = s.trips.find((x) => x.id === trip.id) as Trip;
    const ctx: AssistantContext = { trip: live, userId, weather: s.weather[live.destinationId], rates: s.rates, homeCurrency: s.setting
s.homeCurrency };
    let reply = '';
    let source: 'local' | 'claude' = 'local';
    if (aiMode === 'claude') {
      const key = await getApiKey();
      if (key) {
        try {
          reply = await askClaude(key, question, history, ctx);
          source = 'claude';
        } catch {
          toast('Claude is unavailable — answered offline instead', { tone: 'warn', icon: 'cloud-offline-outline' });
        }
      }
    }
    if (!reply) {
      await new Promise((r) => setTimeout(r, 650));
      reply = answerLocally(question, ctx);
    }
    setTyping(null);
    send(trip.id, { authorId: 'assistant', ai: true, aiSource: source, channel: 'ai', text: reply });
    tap();
  };

  const submit = (raw?: string) => {
    const msg = (raw ?? text).trim();
    if (!msg) return;
    if (msg.length > 1000) return toast('Messages can be up to 1000 characters', { tone: 'warn' });
    setText('');
    if (tab === 'ai') {
      send(trip.id, { authorId: userId, text: msg, channel: 'ai' });
      askAssistant(msg, trip.messages.filter(inAiThread));
      return;
    }
    send(trip.id, { authorId: userId, text: msg });
    const plan = synced ? null : planFriendReply(trip.members.filter((m) => m.id !== userId && !m.invited));
    if (plan) {
      later(() => setTyping(plan.friend.id), 900);
      later(() => {
        setTyping(null);
        send(trip.id, { authorId: plan.friend.id, text: friendReply(msg) });
      }, plan.delay);
    }
  };

  const isAi = tab === 'ai';
  const data = trip.messages.filter((m) => inAiThread(m) === isAi);
  const typingFor = typing === 'assistant' ? (isAi ? 'Scout AI' : null) : isAi ? null : typing ? trip.members.find((m) => m.id === typin
g)?.name.split(' ')[0] : null;

  return (
    <View style={[styles.root, { paddingTop: insets.top }]}>
      <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={styles.column}>
        <ChatHeader trip={trip} tab={tab} onTab={setTab} synced={synced} aiMode={aiMode} />

        <FlatList
          key={tab}
          ref={list}
          data={data}
          keyExtractor={(m) => m.id}
          renderItem={({ item, index }) => <ChatBubble item={item} prev={data[index - 1]} next={data[index + 1]} trip={trip} userId={use
rId} />}
          contentContainerStyle={{ paddingHorizontal: 20, paddingTop: 4, paddingBottom: 16, flexGrow: 1 }}
          onContentSizeChange={() => list.current?.scrollToEnd({ animated: true })}
```

```
          onLayout={() => list.current?.scrollToEnd({ animated: false })}
          keyboardShouldPersistTaps="handled"
          ListEmptyComponent={<EmptyChat ai={isAi} onSay={isAi ? undefined : submit} />}
          ListFooterComponent={
            typingFor ? (
              <View style={styles.typing}>
                <T variant="kicker" color={colors.textMuted}>
                  {t('chat.typing', { name: typingFor })}
                </T>
              </View>
            ) : null
          }
        />

        {isAi ? (
          <ScrollView horizontal showsHorizontalScrollIndicator={false} keyboardShouldPersistTaps="handled" contentContainerStyle={{ ga
p: 8, paddingHorizontal: 20, paddingBottom: 10 }} style={{ flexGrow: 0 }}>
            {QUICK.map((k) => (
              <Chip key={k} small label={t(k)} onPress={() => submit(t(k))} />
            ))}
            <Chip small label="Who owes who?" onPress={() => submit('Who owes who?')} />
            <Chip small label="Convert 50 USD" onPress={() => submit('Convert 50 USD')} />
          </ScrollView>
        ) : null}

        <Composer value={text} onChange={setText} onSend={() => submit()} placeholder={isAi ? 'Ask Scout anything…' : 'Message the cre
w…'} />
      </KeyboardAvoidingView>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  typing: { paddingLeft: 38, paddingTop: 12 },
});
```

src/app/(tabs)/explore.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Clipboard from 'expo-clipboard';
import { router } from 'expo-router';
import { useEffect, useState, type ReactNode } from 'react';
import { Animated, StyleSheet, View, useWindowDimensions } from 'react-native';

import { GuidedTour } from '@/components/GuidedTour';
import { BudgetSection } from '@/components/home/BudgetSection';
import { CrewSection } from '@/components/home/CrewSection';
import { DiscoverSection } from '@/components/home/DiscoverSection';
import { HomeHero } from '@/components/home/HomeHero';
import { blockHasContent, celebrateKey, shouldCelebrateOneDay } from '@/components/home/logic';
import { NearbySection } from '@/components/home/NearbySection';
import { PicksSection } from '@/components/home/PicksSection';
import { pickedForYou } from '@/components/home/picks';
import { PlaySection } from '@/components/home/PlaySection';
import { UpNextSection } from '@/components/home/UpNextSection';
import { hasTrip, useHomeContext } from '@/components/home/useHomeContext';
import { WeatherSection } from '@/components/home/WeatherSection';
import { TripMenuSheet } from '@/components/trip/TripMenuSheet';
import { Button } from '@/components/ui/Button';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { FadeIn, useScrollY } from '@/components/ui/motion';
import { Press, tap } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { visibleSections } from '@/data/layouts';
import { useT } from '@/i18n';
import type { HomeSection } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, space } from '@/theme';
import { greetingFor } from '@/utils/time';

/**
 * Home: the trip's photo and boarding pass, a one-line greeting, then the blocks the person chose, in the order they chose
 * (see data/layouts). Each block is its own component in components/home; a block with nothing to show is left out.
 */
export default function Home() {
  const t = useT();
  const ctx = useHomeContext();
  const { trip, dest, status, now, today, tripLen, daysToGo, dayNo, personal, userName } = ctx;
  const unread = useAppStore((s) => s.notifications.filter((n) => !n.read).length);
  const fired = useAppStore((s) => s.firedReminders);
  const markReminderFired = useAppStore((s) => s.markReminderFired);
  const setToured = useAppStore((s) => s.setToured);
  const { width } = useWindowDimensions();
```

```
  const { y, onScroll } = useScrollY();
  const [menu, setMenu] = useState(false);

  const colW = Math.min(width, MAX_WIDTH) - 2 * space.gutter;
  const firstName = personal.nickname || userName.split(' ')[0] || 'traveller';
  const picks = personal.done ? pickedForYou(personal) : [];
  const blocks = visibleSections(personal).filter((k) => blockHasContent(k, { hasTrip: !!trip, status, hasPicks: picks.length > 0, hasDestination: !!dest }));

  // One day before the trip: a small celebration, once (it is remembered, so it does not repeat on every visit that day).
  const oneDay = !!trip && shouldCelebrateOneDay(status, daysToGo, fired, trip.id, today);
  const tripId = trip?.id;
  useEffect(() => {
    if (!oneDay || !tripId) return;
    const id = setTimeout(() => {
      markReminderFired(celebrateKey(tripId, today));
      celebrate('One day to go');
    }, 1200);
    return () => clearTimeout(id);
  }, [oneDay, tripId, today, markReminderFired]);

  const copyCode = async () => {
    if (!trip) return;
    await Clipboard.setStringAsync(trip.inviteCode).catch(() => {});
    tap('success');
    toast(`Invite code ${trip.inviteCode} copied`, { icon: 'copy-outline' });
  };

  const render = (k: HomeSection): ReactNode => {
    const withTrip = hasTrip(ctx) ? ctx : null;
    switch (k) {
      case 'upnext':
        return withTrip ? <UpNextSection ctx={withTrip} /> : null;
      case 'weather':
        return withTrip ? <WeatherSection ctx={withTrip} /> : null;
      case 'budget':
        return withTrip ? <BudgetSection ctx={withTrip} /> : null;
      case 'crew':
        return withTrip ? <CrewSection ctx={withTrip} onInvite={copyCode} /> : null;
      case 'play':
        return <PlaySection ctx={ctx} />;
      case 'forYou':
        return <PicksSection picks={picks} />;
      case 'nearby':
        return trip && dest ? <NearbySection trip={trip} dest={dest} /> : null;
      case 'discover':
        return <DiscoverSection dest={dest} colW={colW} />;
    }
  };

  return (
    <View style={styles.root}>
      <Animated.ScrollView style={styles.scroll} showsVerticalScrollIndicator={false} contentContainerStyle={{ paddingBottom: 40 }} onScroll={onScroll} scrollEventThrottle={16}>
        <View style={styles.column}>
          <HomeHero trip={trip} dest={trip ? dest : undefined} tz={ctx.tz} status={status} daysToGo={daysToGo} dayNo={dayNo} tripLen={tripLen} unread={unread} scrollY={y} onMenu={() => setMenu(true)} onCopyCode={copyCode} />

          <View style={styles.page}>
            <FadeIn delay={120} distance={10}>
              <T variant="title">
                {t(greetingFor(new Date(now).getHours()))}, <T variant="title" weight="bold">{firstName}</T>
              </T>
            </FadeIn>

            {!trip ? (
              <FadeIn delay={180} distance={12} style={styles.block}>
                <View style={{ gap: 6 }}>
                  <T variant="h2">Plan your first group trip</T>
                  <T variant="small" color={colors.textSecondary}>
                    Create a trip, invite friends with a code and start planning together.
                  </T>
                </View>
                <View style={styles.pair}>
                  <Button label="Create trip" icon="add" size="md" onPress={() => router.push('/new-trip')} style={{ flex: 1 }} />
                  <Button label="Join a trip" icon="people-outline" size="md" variant="secondary" onPress={() => router.push('/join')} style={{ flex: 1 }} />
                </View>
              </FadeIn>
            ) : null}

            {blocks.map((k, i) => (
              <FadeIn key={k} delay={220 + i * 70} distance={14} duration={380} style={styles.block}>
                {render(k)}
              </FadeIn>
            ))}
```

```
            <Press onPress={() => router.push('/settings')} style={styles.customize} accessibilityLabel="Customize this page">
              <Ionicons name="options-outline" size={18} color={colors.ink} />
              <T variant="small" weight="bold" style={{ textDecorationLine: 'underline' }}>
                Customize this page
              </T>
            </Press>
          </View>
        </View>
      </Animated.ScrollView>

      {trip ? <TripMenuSheet visible={menu} onClose={() => setMenu(false)} trip={trip} /> : null}
      <GuidedTour visible={!personal.toured && !!trip} onClose={() => setToured(true)} />
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  // width 100% so the page is exactly as wide as the phone (the root centres its children, which would otherwise let a wide row stretch
the page sideways)
  scroll: { width: '100%' },
  column: { width: '100%', maxWidth: MAX_WIDTH },
  page: { paddingHorizontal: space.gutter, paddingTop: 20 },
  block: { marginTop: space.section, gap: 16 },
  pair: { flexDirection: 'row', gap: 12 },
  customize: { alignSelf: 'flex-start', flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 48, marginTop: space.section },
});
```

src/app/(tabs)/itinerary.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useIsFocused, useLocalSearchParams } from 'expo-router';
import { useEffect, useState, type ReactNode } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { ActivityCard } from '@/components/trip/ActivityCard';
import { AddActivitySheet } from '@/components/trip/AddActivitySheet';
import { DayTabs } from '@/components/trip/DayTabs';
import { MemberPickerSheet } from '@/components/trip/MemberPickerSheet';
import { activityFilters, typeIcon } from '@/components/trip/meta';
import { NoTrip } from '@/components/trip/NoTrip';
import { byTime, groupByDay, newlyLocked, sameStatuses, statusMap } from '@/components/trip/plan';
import { Button } from '@/components/ui/Button';
import { Chip, EmptyState, IconButton } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { Fab } from '@/components/ui/Fab';
import { Press } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { Header, Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useT } from '@/i18n';
import { printPlanner } from '@/services/print';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Activity, ActivityType } from '@/store/types';
import { colors, space } from '@/theme';
import { LENGTH_FILTERS, matchesLength, type LengthFilter } from '@/utils/format';
import { dayLabel, rangeLabel, todayInTz, tripDays, tzAbbrev, utcOffsetLabel, weekdayShort } from '@/utils/time';

export default function Plan() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <PlanInner key={trip.id} />;
}

/** How long a just-locked item stays on the Vote tab with its stamp, so the moment is seen. */
const LINGER_MS = 3800;

function PlanInner() {
  const t = useT();
  const trip = useActiveTrip()!;
  const params = useLocalSearchParams<{ tab?: string }>();
  const focused = useIsFocused();
  const toggleAssign = useAppStore((s) => s.toggleAssign);
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const days = tripDays(trip.startDate, trip.endDate);
  const today = todayInTz(tz);

  const [tab, setTab] = useState<'itinerary' | 'vote'>(params.tab === 'vote' ? 'vote' : 'itinerary');
  const [seenTab, setSeenTab] = useState(params.tab);
  // Home’s “Vote” tile navigates here with ?tab=vote while this screen is already mounted.
```

```
  if (params.tab !== seenTab) {
    setSeenTab(params.tab);
    if (params.tab) setTab(params.tab === 'vote' ? 'vote' : 'itinerary');
  }
  const [day, setDay] = useState<'all' | string>('all');
  const [cat, setCat] = useState<'all' | ActivityType>('all');
  const [len, setLen] = useState<LengthFilter>('any');
  // The kind and length filters stay folded away until asked for (or while one is on, so it is never a hidden reason for an empty lis
t).
  const [filtersOpen, setFiltersOpen] = useState(false);
  const filtering = cat !== 'all' || len !== 'any';
  const showFilters = filtersOpen || filtering;
  const [addOpen, setAddOpen] = useState(false);
  const [editing, setEditing] = useState<Activity | null>(null);
  const [assigning, setAssigning] = useState<Activity | null>(null);

  // A vote tips an item over (or the organizer locks it in): notice the change, celebrate once and keep the card on the Vote tab for a
moment.
  const [seen, setSeen] = useState(() => statusMap(trip.activities));
  const [fresh, setFresh] = useState<{ id: string; title: string }[]>([]);
  const nowStatus = statusMap(trip.activities);
  if (!sameStatuses(seen, nowStatus)) {
    setSeen(nowStatus);
    const flipped = newlyLocked(seen, trip.activities);
    // not while another tab is showing: the moment would be missed
    if (flipped.length && focused) setFresh((f) => [...f, ...flipped.map((a) => ({ id: a.id, title: a.title }))]);
  }
  useEffect(() => {
    if (!fresh.length) return;
    celebrate(`Locked in: ${fresh[fresh.length - 1].title}`);
    const timer = setTimeout(() => setFresh([]), LINGER_MS);
    return () => clearTimeout(timer);
  }, [fresh]);

  const selectedDay = day === 'all' || days.includes(day) ? day : 'all';
  const matches = (a: Activity) => (selectedDay === 'all' || a.date === selectedDay) && (cat === 'all' || a.type === cat) && matchesLeng
th(a.durationMin, len);
  const voting = trip.activities.filter((a) => a.status === 'voting');
  const ballots = trip.activities.filter((a) => a.status === 'voting' || fresh.some((f) => f.id === a.id)).sort(byTime);
  const groups = groupByDay(trip.activities.filter((a) => a.status !== 'voting' && matches(a)), days);
  const anyPlanned = trip.activities.some((a) => a.status !== 'voting');
  // an empty day can still have ideas waiting for a vote, which are on the other tab
  const waiting = selectedDay === 'all' ? 0 : voting.filter((a) => a.date === selectedDay).length;
  const assigningLive = assigning ? trip.activities.find((a) => a.id === assigning.id) ?? null : null;

  const openAdd = () => {
    setEditing(null);
    setAddOpen(true);
  };
  const clearFilters = () => {
    setCat('all');
    setLen('any');
    setFiltersOpen(false);
  };
  const print = async () => {
    try {
      await printPlanner(trip);
    } catch {
      toast('Allow pop-ups to print the planner', { tone: 'warn' });
    }
  };

  const onVoteTab = tab === 'vote';
  const planEmpty = !onVoteTab && groups.length === 0;
  const voteEmpty = onVoteTab && ballots.length === 0;
  // an empty list brings its own Add button, so the floating one steps aside (one main action per screen)
  const showFab = !planEmpty && !voteEmpty;
  const dayNo = selectedDay === 'all' ? 0 : days.indexOf(selectedDay) + 1;

  // where each day's rows start in the whole list, so only the first few rows slide in
  const starts: number[] = [];
  groups.reduce((sum, g) => {
    starts.push(sum);
    return sum + g.items.length;
  }, 0);
  return (
    <Screen
      padded={false}
      bottomInset={30}
      fab={showFab ? <Fab icon="add" label="Add" accessibilityLabel="Add activity" testID="add-activity" onPress={openAdd} /> : undefine
d}>
      <Header
        back={false}
        kicker={dest ? `${dest.city}, ${dest.country}` : trip.name}
        title="The plan"
        subtitle={`${rangeLabel(trip.startDate, trip.endDate)} · ${days.length} ${days.length === 1 ? 'day' : 'days'}`}
```

```
        right={<IconButton icon="print-outline" label="Print planner" onPress={print} />}
      />

      <View style={styles.body}>
        <Segmented
          value={tab}
          onChange={setTab}
          options={[
            { key: 'itinerary', label: 'Itinerary' },
            { key: 'vote', label: `Vote (${voting.length})` },
          ]}
        />

        {!onVoteTab ? (
          <>
            <View style={styles.tabs}>
              <DayTabs days={days} value={selectedDay} onChange={setDay} today={days.includes(today) ? today : undefined} withAll />
            </View>

            <View style={styles.tzRow}>
              <T variant="kicker" color={colors.textMuted} style={{ flex: 1 }}>
                Times in {dest?.city ?? 'local'} · {tzAbbrev(tz)} {utcOffsetLabel(tz)}
              </T>
              <Press
                onPress={() => (filtering ? clearFilters() : setFiltersOpen((o) => !o))}
                style={styles.filterBtn}
                accessibilityLabel={filtering ? 'Clear filters' : showFilters ? 'Hide filters' : 'Show filters'}
                accessibilityState={{ expanded: showFilters }}>
                <Ionicons name={filtering ? 'close' : 'options-outline'} size={18} color={colors.ink} />
                <T variant="kicker" color={colors.ink}>
                  {filtering ? 'Clear' : 'Filter'}
                </T>
              </Press>
            </View>

            {showFilters ? (
              <View style={styles.filters}>
                <ChipRow>
                  {activityFilters.map((f) => (
                    <Chip key={f.key} small icon={f.key === 'all' ? undefined : typeIcon(f.key)} label={t(f.labelKey)} active={cat ===
f.key} onPress={() => setCat(f.key)} style={styles.chip} />
                  ))}
                </ChipRow>
                <ChipRow>
                  {LENGTH_FILTERS.map((f) => (
                    <Chip key={f.key} small icon={f.key === 'any' ? undefined : 'timer-outline'} label={t(f.labelKey)} active={len ===
f.key} onPress={() => setLen(f.key)} style={styles.chip} />
                  ))}
                </ChipRow>
              </View>
            ) : null}
          </>
        ) : null}

        <View style={{ marginTop: onVoteTab ? 24 : 20 }}>
          {onVoteTab ? (
            voteEmpty ? (
              <EmptyState icon="thumbs-up-outline" title="Nothing to vote on" body="Put an idea to the crew. More than half saying yes l
ocks it in.">
                <Button label="Suggest something" icon="add" onPress={openAdd} />
              </EmptyState>
            ) : (
              ballots.map((a, i) => <ActivityCard key={a.id} trip={trip} activity={a} mode="vote" index={i} justLocked={a.status !== 'vo
ting'} onEdit={onEdit(setEditing, setAddOpen)} onAssign={setAssigning} />)
            )
          ) : planEmpty ? (
            <EmptyState
              icon={filtering && anyPlanned ? 'funnel-outline' : 'calendar-outline'}
              title={filtering && anyPlanned ? t('itin.noMatch') : dayNo ? `Nothing planned for Day ${dayNo}` : 'Nothing planned yet'}
              body={filtering && anyPlanned ? 'Try a different kind or length.' : waiting ? `${waiting} ${waiting === 1 ? 'idea is' : 'i
deas are'} waiting on the Vote tab.` : 'Add the first thing and the crew will see it here.'}>
              {filtering && anyPlanned ? <Button label="Show everything" variant="secondary" onPress={clearFilters} /> : <Button label
="Add activity" icon="add" onPress={openAdd} />}
            </EmptyState>
          ) : (
            groups.map((g, gi) => (
              <View key={g.date} style={gi ? { marginTop: space.section } : undefined}>
                <View style={styles.dayHead}>
                  <View style={styles.dayTitle}>
                    <T variant="headline" accessibilityRole="header">
                      {g.dayNo ? `Day ${g.dayNo}` : dayLabel(g.date)}
                    </T>
                    {g.date === today ? <Stamp label="Today" tone="accent" /> : null}
                    <T variant="kicker" color={colors.textSecondary} style={{ flex: 1, textAlign: 'right' }} numberOfLines={1}>
                      {weekdayShort(g.date)} {dayLabel(g.date)}
                    </T>
```

```
                  </View>
                  <Rule />
                </View>
                {g.items.map((a, i) => (
                  <ActivityCard key={a.id} trip={trip} activity={a} index={starts[gi] + i} last={i === g.items.length - 1} onEdit={onEdi
t(setEditing, setAddOpen)} onAssign={setAssigning} />
                ))}
              </View>
            ))
          )}
        </View>
      </View>

      <AddActivitySheet visible={addOpen} onClose={() => setAddOpen(false)} trip={trip} date={selectedDay === 'all' ? days[0] : selected
Day} editing={editing} voting={onVoteTab} />
      <MemberPickerSheet visible={!!assigning} onClose={() => setAssigning(null)} members={trip.members} selected={assigningLive?.assign
ed ?? []} onToggle={(id) => assigning && toggleAssign(trip.id, assigning.id, id)} />
    </Screen>
  );
}

/** Opens Add activity filled in for one that exists (the sheet is the same for adding and editing). */
const onEdit = (setEditing: (a: Activity) => void, setOpen: (o: boolean) => void) => (a: Activity) => {
  setEditing(a);
  setOpen(true);
};

/** A row of chips that scrolls sideways and runs to the screen edge. */
function ChipRow({ children }: { children: ReactNode }) {
  return (
    <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
      {children}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  body: { paddingHorizontal: 20 },
  tabs: { marginTop: 16 },
  tzRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 4, minHeight: 44 },
  filterBtn: { flexDirection: 'row', alignItems: 'center', gap: 6, minHeight: 44, paddingLeft: 12 },
  filters: { gap: 8, marginBottom: 4 },
  chipScroll: { marginHorizontal: -20, flexGrow: 0 },
  chips: { gap: 8, paddingHorizontal: 20 },
  // the shortest label ("All") would otherwise be narrower than a fingertip
  chip: { minWidth: 44, justifyContent: 'center' },
  dayHead: { gap: 8, marginBottom: 18 },
  dayTitle: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 36 },
});
```

src/app/(tabs)/profile.tsx

```
import { router } from 'expo-router';
import { Fragment } from 'react';
import { StyleSheet, View } from 'react-native';

import { AccountTicket } from '@/components/more/AccountTicket';
import { Tag, WideTag } from '@/components/more/Tag';
import { SectionHeader, type IconName } from '@/components/ui/bits';
import { Header, Screen } from '@/components/ui/Screen';
import { destinationById } from '@/data/destinations';
import { useT } from '@/i18n';
import { useActiveTrip, useCurrentUser, useMyTrips } from '@/store/useAppStore';
import { space } from '@/theme';

/** One tag. `title` is the full name a screen reader says; `label` is the shorter print on the tag. */
type Item = { icon: IconName; title: string; label?: string; /** Spoken by screen readers, not shown. */ hint: string; href: string };

const groups: { title: string; items: Item[]; /** A wide tag under the squares. */ wide?: Item & { sub: string } }[] = [
  {
    title: 'Your trip',
    items: [
      { icon: 'people-outline', title: 'Crew & roles', hint: 'Invite code, roles, votes and checklist', href: '/crew' },
      { icon: 'airplane-outline', title: 'Flights & arrivals', label: 'Flights', hint: 'Everyone’s arrival and departure times', href:
'/flights' },
      { icon: 'briefcase-outline', title: 'Packing list', hint: 'Smart suggestions', href: '/packing' },
      { icon: 'folder-open-outline', title: 'Offline Docs', hint: 'Passports and bookings, no wifi needed', href: '/documents' },
      { icon: 'camera-outline', title: 'Photo Dump', hint: 'Everyone’s shots in one place', href: '/photos' },
      { icon: 'trophy-outline', title: 'Trip recap', hint: 'Stats, highlights and share card', href: '/recap' },
    ],
  },
  {
    title: 'Around town',
    items: [
      { icon: 'map-outline', title: 'Trip Map', hint: 'Pins and the nearest airport', href: '/map' },
      { icon: 'compass-outline', title: 'Explore Nearby', hint: 'Food and sights by radius', href: '/nearby' },
```

```
      { icon: 'globe-outline', title: '360° City Views', hint: 'Day and night skylines with the lights on', href: '/panorama' },
      { icon: 'partly-sunny-outline', title: 'Weather', hint: '7-day forecast', href: '/weather' },
      { icon: 'cash-outline', title: 'Currency', hint: 'Live converter', href: '/currency' },
      { icon: 'language-outline', title: 'Phrasebook', hint: 'Speak the basics', href: '/phrasebook' },
    ],
    wide: { icon: 'medkit-outline', title: 'Emergency', sub: 'Local numbers and safety', hint: 'Local numbers and safety', href: '/emerg
ency' },
  },
  {
    title: 'Just for fun',
    items: [
      { icon: 'star-outline', title: 'Bucket List', hint: 'Must-dos for the trip', href: '/bucket-list' },
      { icon: 'game-controller-outline', title: 'Trip Quiz', hint: 'Test the crew', href: '/quiz' },
      { icon: 'telescope-outline', title: 'Find a trip', hint: 'Answer five questions, get a dream destination', href: '/survey' },
    ],
  },
  {
    title: 'The app',
    items: [
      { icon: 'settings-outline', title: 'Settings', hint: 'Profile, language, reminders and theme', href: '/settings' },
      { icon: 'help-circle-outline', title: 'Help & FAQ', hint: 'A tour of the app and answers to common questions', href: '/help' },
      { icon: 'information-circle-outline', title: 'About', hint: 'Credits and sources', href: '/about' },
    ],
  },
];

const go = (href: string) => router.push(href as never);

/** Three to a row; a short last row is padded with empty cells so every tag keeps the same width. */
function rows<T>(items: T[]): (T | null)[][] {
  const out: (T | null)[][] = [];
  for (let i = 0; i < items.length; i += 3) {
    const row: (T | null)[] = items.slice(i, i + 3);
    while (row.length < 3) row.push(null);
    out.push(row);
  }
  return out;
}

/**
 * The More tab: your boarding pass, then four sheets of luggage tags (Your trip, Around town, Just for fun, The app), three to a row,
 * each a big square you can hit. No group has more than six tags.
 */
export default function More() {
  const t = useT();
  const trip = useActiveTrip();
  const user = useCurrentUser();
  const myTrips = useMyTrips();
  const dest = trip ? destinationById(trip.destinationId) : undefined;

  return (
    <Screen bottomInset={30} header={<Header back={false} kicker={trip?.name} title={t('nav.profile')} />}>
      <AccountTicket user={user} destCity={dest?.city} homeCity={user?.homeCity} tripCount={myTrips.length} />

      {groups.map((g) => (
        <View key={g.title} style={{ marginTop: space.section }}>
          <SectionHeader title={g.title} />
          <View style={styles.grid}>
            {rows(g.items).map((row, i) => (
              <View key={i} style={styles.row}>
                {row.map((r, j) => (
                  <Fragment key={j}>{r ? <Tag icon={r.icon} label={r.label ?? r.title} name={r.title} hint={r.hint} onPress={() => go(r.
href)} /> : <View style={{ flex: 1 }} />}</Fragment>
                ))}
              </View>
            ))}
            {g.wide ? <WideTag icon={g.wide.icon} label={g.wide.title} sub={g.wide.sub} hint={g.wide.hint} onPress={() => go(g.wide!.hre
f)} /> : null}
          </View>
        </View>
      ))}
    </Screen>
  );
}

const styles = StyleSheet.create({
  grid: { gap: space.sm },
  row: { flexDirection: 'row', gap: space.sm },
});
```

src/app/_layout.tsx

```
import { Fraunces_700Bold, Fraunces_900Black } from '@expo-google-fonts/fraunces';
import { SpaceGrotesk_400Regular, SpaceGrotesk_500Medium, SpaceGrotesk_600SemiBold, SpaceGrotesk_700Bold } from '@expo-google-fonts/spac
e-grotesk';
import { SpaceMono_400Regular, SpaceMono_700Bold, useFonts } from '@expo-google-fonts/space-mono';
```

```
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect } from 'react';
import { SafeAreaProvider } from 'react-native-safe-area-context';

import { ReminderService } from '@/components/ReminderService';
import { ConfettiHost, celebrate } from '@/components/ui/Confetti';
import { DialogHost, ToastHost } from '@/components/ui/feedback';
import { useWatchOsReduceMotion } from '@/hooks/useReduceMotion';
import { initNotifications } from '@/services/notifications';
import { startSync } from '@/sync/runtime';
import { useAppStore } from '@/store/useAppStore';
import { colors, isDark } from '@/theme';

SplashScreen.preventAutoHideAsync().catch(() => {});

export default function RootLayout() {
  const [fontsLoaded, fontError] = useFonts({ Fraunces_700Bold, Fraunces_900Black, SpaceGrotesk_400Regular, SpaceGrotesk_500Medium, Spac
eGrotesk_600SemiBold, SpaceGrotesk_700Bold, SpaceMono_400Regular, SpaceMono_700Bold });
  const hydrated = useAppStore((s) => s.hydrated);
  const seeded = useAppStore((s) => s.seeded);
  const signedIn = useAppStore((s) => !!s.session);
  const ensureSeed = useAppStore((s) => s.ensureSeed);

  useWatchOsReduceMotion();
  useEffect(() => {
    initNotifications();
  }, []);

  useEffect(() => {
    if (hydrated) ensureSeed();
  }, [hydrated, ensureSeed]);

  const ready = (fontsLoaded || !!fontError) && hydrated && seeded;
  useEffect(() => {
    if (!ready) return;
    SplashScreen.hideAsync().catch(() => {});
    void startSync(); // phone-to-phone sync for trips that were shared (does nothing until you share or join one)
    // Back from the reload that applies a new theme: say so, then forget the one-off session.
    const { restyled, setRestyled } = useAppStore.getState();
    if (restyled) {
      setRestyled(false);
      setTimeout(() => celebrate('Your new look is live. Change it anytime in Settings'), 600);
    }
  }, [ready]);

  if (!ready) return null;

  return (
    <SafeAreaProvider>
      <StatusBar style={isDark ? 'light' : 'dark'} />
      <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: colors.bg }, animation: 'slide_from_right' }}>
        <Stack.Screen name="index" />
        <Stack.Protected guard={!signedIn}>
          <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
          <Stack.Screen name="sign-in" options={{ animation: 'fade' }} />
          <Stack.Screen name="sign-up" />
          <Stack.Screen name="forgot-password" />
        </Stack.Protected>
        <Stack.Protected guard={signedIn}>
          <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />
          <Stack.Screen name="destination/[id]" />
          <Stack.Screen name="destinations" />
          <Stack.Screen name="nearby" />
          <Stack.Screen name="trips" />
          <Stack.Screen name="new-trip" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
          <Stack.Screen name="join" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
          <Stack.Screen name="flights" />
          <Stack.Screen name="crew" />
          <Stack.Screen name="how-it-works" />
          <Stack.Screen name="panorama" options={{ animation: 'fade' }} />
          <Stack.Screen name="map" />
          <Stack.Screen name="weather" />
          <Stack.Screen name="photos" />
          <Stack.Screen name="bucket-list" />
          <Stack.Screen name="quiz" />
          <Stack.Screen name="survey" />
          <Stack.Screen name="documents" />
          <Stack.Screen name="packing" />
          <Stack.Screen name="currency" />
          <Stack.Screen name="notifications" />
          <Stack.Screen name="settings" />
          <Stack.Screen name="emergency" />
          <Stack.Screen name="phrasebook" />
          <Stack.Screen name="recap" />
```

```
          <Stack.Screen name="airport" />
          <Stack.Screen name="help" />
          <Stack.Screen name="about" />
          <Stack.Screen name="personalize" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
        </Stack.Protected>
      </Stack>
      {signedIn ? <ReminderService /> : null}
      <ConfettiHost />
      <ToastHost />
      <DialogHost />
    </SafeAreaProvider>
  );
}
```

src/app/about.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { SectionHeader } from '@/components/ui/bits';
import { Plate } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { colors, hair, line, radius, space } from '@/theme';

const SOURCES: [string, string, string][] = [
  ['Weather', 'Open-Meteo forecast API (CC BY 4.0)', 'https://open-meteo.com'],
  ['Exchange rates', 'ExchangeRate-API open access endpoint', 'https://www.exchangerate-api.com/docs/free'],
  ['Maps', 'Apple Maps / Google Maps via react-native-maps; OpenStreetMap on web (ODbL)', 'https://www.openstreetmap.org/copyright'],
  ['Photography', 'Unsplash. Free to use under the Unsplash License (see ATTRIBUTIONS.md)', 'https://unsplash.com/license'],
  ['Fonts', 'Fraunces, Space Grotesk and Space Mono. SIL Open Font License', 'https://fonts.google.com'],
  ['Icons', 'Ionicons & Material Community Icons (MIT) via @expo/vector-icons', 'https://icons.expo.fyi'],
  ['Emergency numbers', 'Official tourism and government sources per country (e.g. JNTO for Japan)', 'https://www.jnto.go.jp/safety-tip
s/eng/emergency-numbers.html'],
  ['AI (optional)', 'Anthropic Claude API. Only when the user adds their own key', 'https://docs.claude.com'],
];

const STACK = [
  'Expo SDK 57 + React Native 0.86 (iOS & Android, TypeScript)',
  'Expo Router: file-based navigation with protected routes',
  'Zustand + Immer for state; AsyncStorage for offline persistence',
  '@noble/hashes (PBKDF2 password hashing), expo-crypto (random salts), expo-secure-store (API key)',
  'expo-notifications, expo-location, expo-image-picker, expo-document-picker',
  'expo-print, expo-sharing, expo-speech, expo-file-system, expo-camera, react-native-maps',
  'qrcode-generator (invite QR codes, MIT), react-native-view-shot (share the recap as a picture)',
  'A small Node sync server in server/ (no dependencies) keeps friends’ phones in step; Jest + Playwright for tests',
];

/** The link under a source: the address in mono with an arrow, a whole 44 point row you can press. */
function Link({ url }: { url: string }) {
  return (
    <Press onPress={() => Linking.openURL(url)} style={styles.link} scaleTo={0.99} accessibilityRole="link" accessibilityLabel={`Open
${url}`}>
      <T variant="kicker" color={colors.accentText} numberOfLines={1} style={{ flex: 1, textTransform: 'none' }}>
        {url.replace(/^https?:\/\//, '')}
      </T>
      <Ionicons name="open-outline" size={16} color={colors.ink} />
    </Press>
  );
}

/** Credits and sources: what the app is made of, where its data comes from, and the licences that allow it. */
export default function About() {
  return (
    <Screen header={<Header kicker="TogetherWeGo" title="About" subtitle="Credits, sources and how it’s built" backLabel="More" />}>
      <View style={styles.brand}>
        <Plate offset={4} corner={radius.card}>
          <View style={styles.mark}>
            <Ionicons name="airplane" size={30} color={colors.onAccent} />
          </View>
        </Plate>
        <View style={{ flex: 1, gap: 2 }}>
          <T variant="headline">TogetherWeGo</T>
          <T variant="kicker" color={colors.textSecondary}>
            Group trip planner · v1.0
          </T>
          <T variant="caption" color={colors.textSecondary}>
            Made for FBLA Mobile Application Development 2026–27
          </T>
        </View>
      </View>
```

```
      <View style={{ marginTop: space.section }}>
        <SectionHeader title="How it’s built" />
        <T variant="small" color={colors.textSecondary}>
          A layered, MVVM-style design: screens (views) in src/app, reusable components in src/components, a single store (model and vie
w-model selectors) in src/store, and side-effect services (weather, currency, notifications, AI, files) in src/services. Data flows one
way: screens call store actions, state updates, the UI draws again.
        </T>
        <View style={{ marginTop: space.lg }}>
          {STACK.map((s, i) => (
            <View key={s} style={[styles.stack, i === 0 && { borderTopWidth: 0 }]}>
              <View style={styles.bullet} />
              <T variant="small" style={{ flex: 1 }}>
                {s}
              </T>
            </View>
          ))}
        </View>
        <Button label="How it works" icon="construct-outline" variant="secondary" size="md" style={{ marginTop: space.lg }} onPress={()
=> router.push('/how-it-works')} accessibilityHint="Architecture, sync, security and where to find every feature" />
      </View>

      <View style={{ marginTop: space.section }}>
        <SectionHeader title="Sources and licences" />
        {SOURCES.map(([k, v, url], i) => (
          <View key={k} style={[styles.source, i === 0 && { borderTopWidth: 0, paddingTop: 0 }]}>
            <T variant="kicker" color={colors.textSecondary}>
              {k}
            </T>
            <T variant="small">{v}</T>
            <Link url={url} />
          </View>
        ))}
      </View>

      <T variant="caption" color={colors.textMuted} style={{ marginTop: space.section }}>
        Demo people, trips and chats are fictional sample data. Destination facts are summarised from public tourism information.
      </T>
    </Screen>
  );
}

const styles = StyleSheet.create({
  brand: { flexDirection: 'row', alignItems: 'center', gap: space.lg },
  mark: { width: 68, height: 68, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.accent,
alignItems: 'center', justifyContent: 'center' },
  stack: { flexDirection: 'row', alignItems: 'flex-start', gap: space.md, paddingVertical: space.sm, borderTopWidth: hair, borderTopColo
r: colors.border },
  bullet: { width: 7, height: 7, marginTop: 7, backgroundColor: colors.ink },
  source: { gap: 2, paddingVertical: space.md, borderTopWidth: hair, borderTopColor: colors.border },
  link: { flexDirection: 'row', alignItems: 'center', gap: space.sm, minHeight: 44 },
});
```

src/app/airport.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Chip, IconButton, SectionHeader } from '@/components/ui/bits';
import { Header, Screen } from '@/components/ui/Screen';
import { Skeleton } from '@/components/ui/Skeleton';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { destinationById } from '@/data/destinations';
import { useNow } from '@/hooks/useTrip';
import { getLocation, type LocationResult } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, hair, line, radius, space } from '@/theme';
import { durationLabel } from '@/utils/format';
import { bearingDeg, compassPoint, driveMinutes, kmLabel, nearestAirports } from '@/utils/geo';
import { partsInTz, time12, tzAbbrev } from '@/utils/time';

/** "~25m", or "~19h 14m" for an airport a day's drive away. */
const drive = (km: number) => `~${durationLabel(driveMinutes(km))}`;
const directions = (a: { lat: number; lng: number }) => Linking.openURL(`https://www.google.com/maps/dir/?api=1&destination=${a.la
t},${a.lng}`);

/** A small caps label over a mono value: one column of the ticket stub. */
function Stat({ label, value, icon }: { label: string; value: string; icon: 'navigate-outline' | 'car-outline' | 'time-outline' }) {
  return (
    <View style={{ flex: 1, gap: 2 }} accessible accessibilityLabel={`${label}: ${value}`}>
      <View style={{ flexDirection: 'row', alignItems: 'center', gap: 4 }}>
        <Ionicons name={icon} size={13} color={colors.textSecondary} />
```

```
        <T variant="kicker" color={colors.textSecondary}>
          {label}
        </T>
      </View>
      <T variant="numeralSm" style={{ fontSize: 17, lineHeight: 22 }}>
        {value}
      </T>
    </View>
  );
}

/** Tracks your location and finds the closest airports (works offline: the airport list is bundled). */
export default function AirportScreen() {
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const [from, setFrom] = useState<'trip' | 'me'>('trip');
  const [loc, setLoc] = useState<LocationResult | null>(null);
  const [loading, setLoading] = useState(false);
  const now = useNow(60000);

  const locateMe = async () => {
    setFrom('me');
    if (loc) return;
    setLoading(true);
    setLoc(await getLocation(dest));
    setLoading(false);
  };

  const origin = from === 'me' && loc?.source === 'device' ? loc : { lat: dest.lat, lng: dest.lng };
  const list = nearestAirports(origin, 6);
  const [first, ...rest] = list;

  return (
    <Screen header={<Header kicker="Around town" title="Nearest airport" subtitle={from === 'me' ? 'From your current location' : `From
central ${dest.city}`} backLabel="More" />}>
      <View style={styles.chips}>
        <Chip icon="business-outline" label={`${dest.city} centre`} active={from === 'trip'} onPress={() => setFrom('trip')} />
        <Chip icon="locate-outline" label="My location" active={from === 'me'} onPress={locateMe} />
      </View>

      {from === 'me' && loc?.source === 'fallback' ? (
        <View style={styles.notice} accessibilityRole="alert">
          <Ionicons name="location-outline" size={20} color={colors.warn} />
          <T variant="small" color={colors.warn} style={{ flex: 1 }}>
            Location permission is off, so this uses central {dest.city}. Turn on location in your phone settings to use GPS.
          </T>
        </View>
      ) : null}

      {loading ? (
        <View style={{ gap: space.md }} accessibilityLabel="Finding you">
          <Skeleton height={330} />
          <Skeleton height={72} />
          <Skeleton height={72} />
        </View>
      ) : (
        <>
          <Ticket
            style={{ marginTop: space.xs }}
            stub={
              <View style={styles.stub}>
                <View style={styles.stats}>
                  <Stat icon="navigate-outline" label="Distance" value={`${kmLabel(first.km)} ${compassPoint(bearingDeg(origin, first))}
`} />
                  <Stat icon="car-outline" label="Drive" value={drive(first.km)} />
                  <Stat icon="time-outline" label="There" value={`${time12(partsInTz(now, first.tz).hhmm)}`} />
                </View>
                <Button label="Directions" icon="navigate" size="md" onPress={() => directions(first)} accessibilityHint={`Opens maps to
${first.name}`} />
              </View>
            }>
            <View style={styles.main}>
              <View style={styles.top}>
                <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
                  Nearest · {tzAbbrev(first.tz)}
                </T>
                <Stamp label="Closest" tone="accent" />
              </View>
              <T variant="hero" accessibilityRole="header">
                {first.iata}
              </T>
              <View>
                <T variant="title" weight="bold">
                  {first.name}
                </T>
                <T variant="small" color={colors.textSecondary}>
```

```
                {first.city}, {first.country}
              </T>
            </View>
          </View>
        </Ticket>

        {rest.length ? (
          <View style={{ marginTop: space.section }}>
            <SectionHeader title="Other airports" />
            {rest.map((a, i) => (
              <View key={a.iata} style={[styles.row, i === 0 && { borderTopWidth: 0 }]}>
                <View style={styles.code} accessible accessibilityLabel={`Airport code ${a.iata}`}>
                  <T variant="numeralSm" style={{ fontSize: 18, lineHeight: 24 }}>
                    {a.iata}
                  </T>
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="bodySm" weight="semibold">
                    {a.name}
                  </T>
                  <T variant="caption" color={colors.textSecondary}>
                    {a.city}, {a.country}
                  </T>
                  <T variant="kicker" color={colors.textSecondary} style={{ marginTop: 3 }}>
                    {kmLabel(a.km)} {compassPoint(bearingDeg(origin, a))} · {drive(a.km)}
                  </T>
                </View>
                <IconButton icon="navigate-outline" label={`Directions to ${a.name}`} onPress={() => directions(a)} />
              </View>
            ))}
          </View>
        ) : null}
      </>
    )}
  </Screen>
 );
}

const styles = StyleSheet.create({
  chips: { flexDirection: 'row', flexWrap: 'wrap', gap: space.sm, marginBottom: space.xl },
  notice: { flexDirection: 'row', alignItems: 'center', gap: space.md, padding: space.md, marginBottom: space.lg, borderRadius: radius.c
ard, borderWidth: line, borderColor: colors.warn, backgroundColor: colors.warnSoft },
  main: { padding: 16, gap: space.md },
  top: { flexDirection: 'row', alignItems: 'center', gap: space.sm },
  stub: { paddingHorizontal: 16, paddingTop: 4, paddingBottom: 16, gap: space.lg },
  stats: { flexDirection: 'row', gap: space.md },
  row: { flexDirection: 'row', alignItems: 'flex-start', gap: space.md, paddingVertical: space.lg, borderTopWidth: hair, borderTopColor:
colors.border },
  code: { minWidth: 64, height: 44, paddingHorizontal: 8, borderRadius: radius.chip, borderWidth: line, borderColor: colors.ink, backgro
undColor: colors.card, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/bucket-list.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { BucketTicket } from '@/components/play/BucketTicket';
import { Button } from '@/components/ui/Button';
import { EmptyState, ProgressBar, SectionHeader } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { Fab } from '@/components/ui/Fab';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById, destinations } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useDraft } from '@/hooks/useDraft';
import { useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';

export default function BucketList() {
  const bucket = useAppStore((s) => s.bucket);
  const favorites = useAppStore((s) => s.favorites);
  const add = useAppStore((s) => s.addBucket);
  const toggle = useAppStore((s) => s.toggleBucket);
  const del = useAppStore((s) => s.deleteBucket);
  const insets = useSafeAreaInsets();
```

```
  const [adding, setAdding] = useState(false);
  const [text, setText, clearText] = useDraft('bucket-add', '');
  const [err, setErr] = useState<string | null>(null);
  const [slam, setSlam] = useState<string | null>(null);
  const done = bucket.filter((b) => b.done).length;
  const ideas = destinations
    .filter((d) => !bucket.some((b) => b.destinationId === d.id))
    .sort((a, b) => Number(favorites.includes(b.id)) - Number(favorites.includes(a.id)) || b.rating - a.rating)
    .slice(0, 8);

  const submit = () => {
    const t = text.trim();
    if (t.length < 3) return setErr('Write a few words about your dream');
    if (t.length > 80) return setErr('Keep it under 80 characters');
    add(t);
    clearText();
    setErr(null);
    setAdding(false);
    tap('success');
    toast('Added to your bucket list');
  };

  const tick = (id: string, wasDone: boolean) => {
    toggle(id);
    if (wasDone) return;
    setSlam(id);
    tap('success');
    // the last dream ticked off is a real achievement (only with a few on the list)
    if (bucket.length >= 3 && done + 1 === bucket.length) celebrate('Bucket list complete');
  };

  return (
    <Screen
      header={<Header title="Bucket list" subtitle="Places and moments for one day" />}
      fab={bucket.length ? <Fab icon="add" label="Add" accessibilityLabel="Add to bucket list" onPress={() => setAdding(true)} /> : unde
fined}
      fabBottom={insets.bottom + 16}>
      {bucket.length ? (
        <View style={styles.progress} accessible accessibilityLabel={`${done} of ${bucket.length} ticked off`}>
          <View style={styles.count}>
            <T variant="title" importantForAccessibility="no">
              <T variant="numeralSm">{done}</T> of <T variant="numeralSm">{bucket.length}</T> ticked off
            </T>
            {done === bucket.length ? <Stamp label="All done" tone="positive" icon="checkmark" rotate={0} /> : null}
          </View>
          <ProgressBar value={done / bucket.length} color={done === bucket.length ? colors.positive : colors.accent} />
        </View>
      ) : (
        <EmptyState icon="flag-outline" title="No bucket list yet" body="Add the places and moments you want to share with your crew one
day.">
          <Button label="Add a place" icon="add" onPress={() => setAdding(true)} />
        </EmptyState>
      )}

      {bucket.length ? (
        <View style={styles.list}>
          {bucket.map((b, i) => {
            const d = b.destinationId ? destinationById(b.destinationId) : undefined;
            return (
              <BucketTicket
                key={b.id}
                item={b}
                dest={d}
                index={i}
                slam={slam === b.id}
                onToggle={() => tick(b.id, b.done)}
                onOpen={() => (d ? router.push({ pathname: '/destination/[id]', params: { id: d.id } }) : tick(b.id, b.done))}
                onRemove={() => confirmAction('Remove from bucket list?', b.text, () => del(b.id), { confirmLabel: 'Remove', destructiv
e: true })}
              />
            );
          })}
        </View>
      ) : null}

      {ideas.length ? (
        <View style={{ marginTop: 32 }}>
          <SectionHeader title={bucket.length ? 'More ideas' : 'Ideas to start'} />
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.ideas} style={{ marginHorizontal:
-20 }}>
            {ideas.map((d) => (
              <Press
                key={d.id}
                onPress={() => {
                  add(`Visit ${d.city}, ${d.country}`, d.id);
                  tap('success');
```

```
                  toast(`${d.city} added`);
                }}
                scaleTo={0.97}
                style={styles.idea}
                accessibilityLabel={`Add ${d.city} to bucket list`}>
                <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" accessibilityIgnoresInvertColors
/>
                <View style={styles.plus}>
                  <Ionicons name="add" size={22} color={colors.onAccent} />
                </View>
                <View style={styles.ideaTag}>
                  <T variant="kicker" color={colors.ink} numberOfLines={1}>
                    {d.city}
                  </T>
                </View>
              </Press>
            ))}
          </ScrollView>
        </View>
      ) : null}

      <Sheet
        visible={adding}
        onClose={() => {
          setAdding(false);
          setErr(null);
        }}
        title="Add to your list"
        subtitle="A place, a meal, a moment: anything you want to do together"
        footer={<Button label="Add to list" icon="add" onPress={submit} />}>
        <Input
          label="Your dream"
          accessibilityLabel="New bucket list item"
          placeholder="Someday I want to…"
          value={text}
          onChangeText={(v) => {
            setText(v);
            setErr(null);
          }}
          error={err}
          onSubmitEditing={submit}
          returnKeyType="done"
          maxLength={90}
        />
      </Sheet>
    </Screen>
  );
}

const styles = StyleSheet.create({
  progress: { gap: 10, marginBottom: 24 },
  count: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12, minHeight: 28 },
  list: { gap: 18 },
  ideas: { gap: 14, paddingHorizontal: 20, paddingBottom: 6 },
  idea: { width: 132, height: 168, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, overflow: 'hidden', background
Color: colors.bgAlt },
  plus: { position: 'absolute', top: 8, right: 8, width: 34, height: 34, borderRadius: radius.chip, borderWidth: line, borderColor: colo
rs.ink, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
  ideaTag: { position: 'absolute', left: 8, bottom: 8, maxWidth: 114, paddingHorizontal: 8, paddingVertical: 4, backgroundColor: colors.
card, borderWidth: line, borderColor: colors.ink, borderRadius: radius.xs },
});
```

src/app/crew.tsx

```
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { AddMemberSheet } from '@/components/crew/AddMemberSheet';
import { ChecklistSection } from '@/components/crew/ChecklistSection';
import { InviteCard } from '@/components/crew/InviteCard';
import { MembersSection } from '@/components/crew/MembersSection';
import { NewPollSheet } from '@/components/crew/NewPollSheet';
import { PollsSection } from '@/components/crew/PollsSection';
import { RoleSheet } from '@/components/crew/RoleSheet';
import { SyncTripCard } from '@/components/sync/SyncTripCard';
import { MemberPickerSheet } from '@/components/trip/MemberPickerSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { AvatarStack, Card, ListRow } from '@/components/ui/bits';
import { toast, confirmAction } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { inviteMessage } from '@/services/share';
import { useActiveTrip, useAppStore, useMyTrips } from '@/store/useAppStore';
import type { Member } from '@/store/types';
import { space } from '@/theme';
```

```
export default function Crew() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <Hub key={trip.id} />;
}

/** The crew: the invitation, who is on the trip, polls, the shared checklist, and sharing the trip across phones. */
function Hub() {
  const trip = useActiveTrip()!;
  const myTrips = useMyTrips();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  // Actions are stable references, so read them without subscribing to the whole store.
  const s = useAppStore.getState();
  const owner = trip.ownerId === userId;

  const [shareOpen, setShareOpen] = useState(false);
  const [roleFor, setRoleFor] = useState<Member | null>(null);
  const [addOpen, setAddOpen] = useState(false);
  const [pollOpen, setPollOpen] = useState(false);
  const [assignTaskId, setAssignTaskId] = useState<string | null>(null);

  const assignee = trip.tasks.find((k) => k.id === assignTaskId)?.assignee;

  return (
    <Screen
      header={<Header kicker={trip.name} title="The crew" backLabel="More" right={<AvatarStack people={trip.members} max={2} size={32} /
>} />}
      bottomInset={30}>
      <InviteCard trip={trip} owner={owner} onShare={() => setShareOpen(true)} />

      <View style={styles.block}>
        <MembersSection trip={trip} userId={userId} onPick={setRoleFor} onAdd={() => setAddOpen(true)} />
      </View>
      <View style={styles.block}>
        <PollsSection trip={trip} userId={userId} owner={owner} onNew={() => setPollOpen(true)} />
      </View>
      <View style={styles.block}>
        <ChecklistSection trip={trip} onAssign={setAssignTaskId} />
      </View>
      <View style={[styles.block, { marginTop: space.section - 14 }]}>
        <SyncTripCard trip={trip} />
      </View>
      <Card padded={false} style={[styles.switch, { paddingHorizontal: 14 }]}>
        <ListRow icon="swap-horizontal" label="Switch trip" sub={myTrips.length > 1 ? `${myTrips.length} trips` : 'All your trips, or st
art a new one'} onPress={() => router.push('/trips')} />
      </Card>

      <ShareSheet visible={shareOpen} onClose={() => setShareOpen(false)} title="Invite your group" message={inviteMessage(trip.name, tr
ip.inviteCode)} preview={inviteMessage(trip.name, trip.inviteCode)} />

      <RoleSheet
        member={roleFor}
        canEdit={owner || roleFor?.id === userId}
        canRemove={owner && roleFor?.id !== trip.ownerId}
        onClose={() => setRoleFor(null)}
        onPick={(role) => {
          if (roleFor) {
            s.setRole(trip.id, roleFor.id, role);
            toast(`${roleFor.name.split(' ')[0]} is now ${role}`);
          }
          setRoleFor(null);
        }}
        onRemove={() => {
          const m = roleFor!;
          setRoleFor(null);
          confirmAction(`Remove ${m.name}?`, 'They'll lose access to this trip.', () => {
            s.removeMember(trip.id, m.id);
            toast(`${m.name.split(' ')[0]} removed`);
          }, { destructive: true, confirmLabel: 'Remove' });
        }}
      />
      <AddMemberSheet
        visible={addOpen}
        onClose={() => setAddOpen(false)}
        onAdd={(n, e) => s.addMember(trip.id, n, e)}
        onShare={() => {
          setAddOpen(false);
          setTimeout(() => setShareOpen(true), 250);
        }}
      />
      <NewPollSheet visible={pollOpen} onClose={() => setPollOpen(false)} onCreate={(q, o) => s.createPoll(trip.id, q, o)} />
      <MemberPickerSheet
        visible={!!assignTaskId}
        single
        title="Assign task"
```

```
        onClose={() => setAssignTaskId(null)}
        members={trip.members}
        selected={assignee ? [assignee] : []}
        onToggle={(id) => assignTaskId && s.assignTask(trip.id, assignTaskId, id)}
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  block: { marginTop: space.section },
  switch: { marginTop: space.section },
});
```

src/app/currency.tsx

```
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { CurrencyConverter } from '@/components/CurrencyConverter';
import { Card, SectionHeader } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useRatesLoader } from '@/hooks/useTrip';
import { convert, fetchRates, useRates } from '@/services/currency';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, hair, space } from '@/theme';
import { money, timeAgo } from '@/utils/format';

const QUICK = [1, 5, 10, 20, 50, 100, 500, 1000];

/** The converter on its own page: the converter standing on the paper, then a short table of everyday amounts for the same pair. */
export default function Currency() {
  useRatesLoader();
  const trip = useActiveTrip();
  const home = useAppStore((s) => s.settings.homeCurrency);
  const rates = useRates();
  const local = destinationById(trip?.destinationId ?? 'tokyo')?.currency ?? 'JPY';
  const [pair, setPair] = useState<[string, string]>([home, local === home ? 'EUR' : local]);
  const [refreshing, setRefreshing] = useState(false);

  return (
    <Screen
      header={<Header title="Currency" kicker={rates.live ? `Live rates · updated ${timeAgo(rates.fetchedAt)}` : 'Offline reference rate
s'} />}
      onRefresh={async () => {
        setRefreshing(true);
        const r = await fetchRates(true);
        setRefreshing(false);
        toast(r.live ? 'Rates updated' : 'Offline — using saved rates', { tone: r.live ? 'ok' : 'warn', icon: r.live ? 'checkmark-circl
e' : 'cloud-offline-outline' });
      }}
      refreshing={refreshing}>
      <Card plated>
        <CurrencyConverter from={pair[0]} to={pair[1]} onChange={(a, b) => setPair([a, b])} />
      </Card>

      <View style={{ marginTop: space.section }}>
        <SectionHeader title="Quick reference" />
        <T variant="kicker" color={colors.textSecondary} style={{ marginBottom: 4 }}>
          {pair[0]} to {pair[1]}
        </T>
        {QUICK.map((n, i) => (
          <View key={n} style={[styles.row, i > 0 && styles.divider]}>
            <T variant="numeralSm">{money(n, pair[0])}</T>
            <T variant="numeralSm" color={colors.textSecondary}>
              {money(convert(n, pair[0], pair[1], rates), pair[1], { decimals: true })}
            </T>
          </View>
        ))}
      </View>
      <T variant="caption" color={colors.textMuted} style={{ marginTop: 20 }}>
        Rates from open.er-api.com, cached on your phone so the converter keeps working offline. Your home currency ({home}) can be chan
ged in Settings.
      </T>
    </Screen>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', minHeight: 48, paddingVertical: 8 },
  divider: { borderTopWidth: hair, borderTopColor: colors.border },
});
```

src/app/destination/[id].tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router, useLocalSearchParams } from 'expo-router';
import { useState, type ReactNode } from 'react';
import { Animated, StyleSheet, View, useWindowDimensions } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { AddToTripSheet, FavStar } from '@/components/explore';
import { placeCategoryIcon } from '@/components/trip/meta';
import { Button } from '@/components/ui/Button';
import { Card, EmptyState, SectionHeader } from '@/components/ui/bits';
import { useMotion, useScrollY } from '@/components/ui/motion';
import { PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { Header, Screen } from '@/components/ui/Screen';
import { Skeleton } from '@/components/ui/Skeleton';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { panoById } from '@/data/panoramas';
import { placesFor } from '@/data/places';
import { currencyByCode } from '@/data/reference';
import { useNow, useWeather } from '@/hooks/useTrip';
import { shareText } from '@/services/share';
import { weatherInfo } from '@/services/weather';
import { MAX_WIDTH, colors, hair, line, radius, space } from '@/theme';
import { durationLabel, money, priceTier } from '@/utils/format';
import { distanceKm, kmLabel, nearestAirports } from '@/utils/geo';
import { deviceTimeZone, hoursAhead, partsInTz, time12, utcOffsetLabel } from '@/utils/time';

/** How far the photo slides while the page scrolls (it is that much taller than its frame, so no gap shows). */
const PARALLAX = 60;
const HERO = 380;

/** 9,400 becomes "9.4K": a count that fits under a number in a narrow cell. */
const compactCount = (n: number) => (n >= 1000 ? `${(n / 1000).toFixed(1).replace(/\.0$/, '')}K` : String(n));

/** An outlined square on the photo, like the controls on Home. */
function Control({ onPress, label, children }: { onPress: () => void; label: string; children: ReactNode }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} style={styles.control}>
      {children}
    </PlatePress>
  );
}

/** One row of "At a glance": a small caps label on the left, the value on the right, a hair rule above. */
function Fact({ label, children, sub, first }: { label: string; children: ReactNode; sub?: string; first?: boolean }) {
  return (
    <View style={[styles.fact, first && styles.first]}>
      <T variant="kicker" color={colors.textSecondary} style={{ width: 96 }}>
        {label}
      </T>
      <View style={{ flex: 1, alignItems: 'flex-end' }}>
        {children}
        {sub ? (
          <T variant="caption" color={colors.textSecondary} style={{ textAlign: 'right' }}>
            {sub}
          </T>
        ) : null}
      </View>
    </View>
  );
}

export default function DestinationScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const d = destinationById(String(id));
  const insets = useSafeAreaInsets();
  const { width } = useWindowDimensions();
  const motion = useMotion();
  const { y, onScroll } = useScrollY();
  const weather = useWeather(d?.id);
  const [adding, setAdding] = useState<string | null>(null);
  const now = useNow(60000);
  if (!d)
    return (
      <Screen header={<Header title="Destination" />}>
        <EmptyState icon="alert-circle-outline" title="Destination not found" />
      </Screen>
    );
```

```
  const places = placesFor(d.id)
    .map((p) => ({ p, km: distanceKm(d, p) }))
    .sort((a, b) => b.p.rating - a.p.rating)
    .slice(0, 6);
  const air = nearestAirports(d, 1)[0];
  const diff = hoursAhead(d.tz, deviceTimeZone());
  const cur = currencyByCode(d.currency);
  const heroH = HERO + insets.top;
  const best = d.bestMonths.split(',').map((s) => s.trim());
  const topPick = d.tags.includes('best');
  const tags = d.tags.filter((t) => t !== 'best');
  const slide = motion ? y.interpolate({ inputRange: [0, heroH], outputRange: [0, PARALLAX], extrapolate: 'clamp' }) : 0;
  // pulled past the top, the photo grows instead of showing a gap
  const grow = motion ? y.interpolate({ inputRange: [-240, 0], outputRange: [1.6, 1], extrapolate: 'clamp' }) : 1;
  const barOpacity = y.interpolate({ inputRange: [heroH - 150, heroH - 90], outputRange: [0, 1], extrapolate: 'clamp' });
  const narrow = width < 350;
  const factSize = narrow ? 15 : 18;

  return (
    <View style={styles.root}>
      <View style={styles.column}>
        <Animated.ScrollView onScroll={onScroll} scrollEventThrottle={16} showsVerticalScrollIndicator={false} contentContainerStyle={{
paddingBottom: insets.bottom + 120 }}>
          <View style={{ height: heroH, overflow: 'hidden', backgroundColor: colors.ink }}>
            <Animated.View style={{ position: 'absolute', left: 0, right: 0, top: -PARALLAX, height: heroH + PARALLAX, transform: [{ tra
nslateY: slide }, { scale: grow }] }}>
              <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={250} accessibilityIgno
resInvertColors />
            </Animated.View>
            {/* the one scrim: it keeps the controls above and the name below readable on any photo */}
            <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0.45)', 'rgba(0,0,0,0)', 'rgba(0,0,0,0.72)']} locations={[0, 0.35,
1]} style={StyleSheet.absoluteFill} />
            <View style={styles.heroText} pointerEvents="none">
              <T variant="kicker" color={colors.white}>
                {d.country}
              </T>
              <T variant="display" color={colors.white} numberOfLines={2} accessibilityRole="header" style={{ fontSize: d.city.length >
10 ? 40 : 56, lineHeight: d.city.length > 10 ? 44 : 60 }}>
                {d.city}
              </T>
            </View>
          </View>

          <View style={styles.body}>
            {/* the quick facts: an outlined strip lying across the lower edge of the photo */}
            <Card plated padded={false}>
              <View style={styles.strip}>
                <View style={styles.cell} accessible accessibilityLabel={`Rated ${d.rating} from ${d.reviews.toLocaleString()} reviews`}
>
                  <T variant="kicker" color={colors.textSecondary}>
                    Rating
                  </T>
                  <View style={styles.valueRow}>
                    <Ionicons name="star" size={factSize - 2} color={colors.ink} />
                    <T variant="numeralSm" style={{ fontSize: factSize + 3, lineHeight: factSize + 9 }}>
                      {d.rating.toFixed(1)}
                    </T>
                  </View>
                  <T variant="micro" color={colors.textSecondary} numberOfLines={1}>
                    {compactCount(d.reviews)}
                    {narrow ? '' : ' reviews'}
                  </T>
                </View>
                <View style={styles.cutline} />
                <View style={styles.cell} accessible accessibilityLabel={`Price level ${d.priceLevel} of 4, about ${money(d.dailyCost)}
a day each`}>
                  <T variant="kicker" color={colors.textSecondary}>
                    Price
                  </T>
                  <T variant="numeralSm" style={{ fontSize: factSize + 3, lineHeight: factSize + 9 }}>
                    {'$'.repeat(d.priceLevel)}
                    <T variant="numeralSm" color={colors.borderStrong} style={{ fontSize: factSize + 3, lineHeight: factSize + 9 }}>
                      {'$'.repeat(4 - d.priceLevel)}
                    </T>
                  </T>
                  <T variant="micro" color={colors.textSecondary} numberOfLines={1}>
                    ~{money(d.dailyCost)} a day
                  </T>
                </View>
                <View style={styles.cutline} />
                <View style={styles.cell} accessible accessibilityLabel={`Best time to go: ${d.bestMonths}`}>
                  <T variant="kicker" color={colors.textSecondary}>
                    Best
                  </T>
                  <T variant="numeralSm" numberOfLines={1} style={{ fontSize: factSize, lineHeight: factSize + 9 }}>
```

```
                  {best[0]}
                </T>
                <T variant="micro" color={colors.textSecondary} numberOfLines={1}>
                  {best[1] ? `and ${best[1]}` : 'all year'}
                </T>
              </View>
            </View>
          </Card>

          <View style={styles.about}>
            {topPick || tags.length ? (
              <View style={styles.tagRow}>
                {topPick ? <Stamp label="Top pick" tone="accent" /> : null}
                <T variant="kicker" color={colors.textSecondary} style={{ flexShrink: 1 }}>
                  {tags.slice(0, 4).join(' · ')}
                </T>
              </View>
            ) : null}
            <T variant="body">{d.blurb}</T>
          </View>

          <View style={{ marginTop: space.section }}>
            <SectionHeader title="Weather" action="Details" onAction={() => router.push({ pathname: '/weather', params: { id: d.id }
})} />
            {weather ? (
              <Press onPress={() => router.push({ pathname: '/weather', params: { id: d.id } })} style={styles.weather} scaleTo={0.99}
accessibilityLabel={`Weather in ${d.city}: ${weather.current.temp} degrees, ${weatherInfo(weather.current.code).label}. Open the forecas
t`}>
                <View style={styles.weatherIcon}>
                  <Ionicons name={weatherInfo(weather.current.code).icon} size={28} color={colors.ink} />
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="numeralSm">{weather.current.temp}°C</T>
                  <T variant="small" color={colors.textSecondary}>
                    {weatherInfo(weather.current.code).label}
                    {weather.live ? '' : ' (estimate)'}
                  </T>
                </View>
                <View style={{ alignItems: 'flex-end' }}>
                  <T variant="kicker" color={colors.textSecondary}>
                    Next 3 days
                  </T>
                  <T variant="small" weight="bold" style={{ fontVariant: ['tabular-nums'] }}>
                    {weather.daily.slice(1, 4).map((x) => `${x.max}°`).join('  ')}
                  </T>
                </View>
              </Press>
            ) : (
              <Skeleton height={72} />
            )}
          </View>

          <View style={{ marginTop: space.section }}>
            <SectionHeader title="At a glance" />
            <View>
              <Fact label="Currency" first>
                <T variant="small" weight="bold">
                  {d.currency}
                  {cur ? ` (${cur.symbol})` : ''}
                </T>
              </Fact>
              <Fact label="Language">
                <T variant="small" weight="bold">
                  {d.language}
                </T>
              </Fact>
              <Fact label="Local time" sub={diff === 0 ? 'Same time zone as you' : `${Math.abs(diff)}h ${diff > 0 ? 'ahead of' : 'behi
nd'} you`}>
                <T variant="small" weight="bold" style={{ fontVariant: ['tabular-nums'] }}>
                  {time12(partsInTz(now, d.tz).hhmm)} · {utcOffsetLabel(d.tz)}
                </T>
              </Fact>
              <Fact label="Airport">
                <T variant="small" weight="bold" style={{ fontVariant: ['tabular-nums'] }}>
                  {air.iata} · {kmLabel(air.km)}
                </T>
              </Fact>
            </View>
          </View>

          <View style={{ marginTop: space.section }}>
            <SectionHeader title="Top things to do" />
            <View>
              {places.map(({ p, km }, i) => (
                <Press key={p.id} onPress={() => setAdding(d.id)} style={[styles.place, i === 0 && styles.first]} scaleTo={0.99} acces
sibilityLabel={`${p.name}. Plan a trip to ${d.city}`}>
```

```
                  <View style={styles.placeIcon}>
                    <Ionicons name={placeCategoryIcon[p.category]} size={20} color={colors.ink} />
                  </View>
                  <View style={{ flex: 1, gap: 2 }}>
                    <T variant="bodySm" weight="semibold">
                      {p.name}
                    </T>
                    <T variant="caption" color={colors.textSecondary} numberOfLines={2}>
                      {p.blurb}
                    </T>
                    <T variant="kicker" color={colors.textSecondary} style={{ marginTop: 2 }}>
                      {p.rating.toFixed(1)} stars · {durationLabel(p.durationMin)} · {priceTier(p.cost)} · {kmLabel(km)}
                    </T>
                  </View>
                </Press>
              ))}
            </View>
          </View>
        </View>
      </Animated.ScrollView>

      {/* once the photo has scrolled away, a paper bar with the city's name takes its place behind the controls */}
      <Animated.View pointerEvents="none" style={[styles.bar, { height: insets.top + 64, opacity: barOpacity }]}>
        <View style={styles.barTitle}>
          <T variant="title" weight="bold" numberOfLines={1}>
            {d.city}
          </T>
        </View>
      </Animated.View>

      {/* controls stay put while the page scrolls under them */}
      <View style={[styles.top, { top: insets.top + 10 }]} pointerEvents="box-none">
        <Control onPress={() => (router.canGoBack() ? router.back() : router.replace('/(tabs)/explore'))} label="Go back">
          <Ionicons name="arrow-back" size={20} color={colors.ink} />
        </Control>
        <View style={{ flex: 1 }} />
        {panoById(d.id) ? (
          <Control onPress={() => router.push({ pathname: '/panorama', params: { city: d.id } })} label={`360° view of ${d.city}`}>
            <Ionicons name="globe-outline" size={20} color={colors.ink} />
          </Control>
        ) : null}
        <Control onPress={() => shareText(`${d.city}, ${d.country}. ${d.blurb} Found on TogetherWeGo.`)} label={`Share ${d.city}`}>
          <Ionicons name="share-social-outline" size={20} color={colors.ink} />
        </Control>
        <FavStar id={d.id} />
      </View>

      <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 12) }]}>
        <Rule />
        <View style={styles.footerBody}>
          <Button label="Plan a trip here" icon="airplane-outline" onPress={() => setAdding(d.id)} />
        </View>
      </View>
    </View>
    <AddToTripSheet destinationId={adding} onClose={() => setAdding(null)} />
  </View>
 );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  top: { position: 'absolute', left: 16, right: 16, flexDirection: 'row', alignItems: 'center', gap: 10 },
  control: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
d, alignItems: 'center', justifyContent: 'center' },
  heroText: { position: 'absolute', left: 20, right: 20, bottom: 58, gap: 2 },
  body: { paddingHorizontal: 20, marginTop: -34 },
  strip: { flexDirection: 'row', alignItems: 'stretch' },
  cell: { flex: 1, paddingVertical: 12, paddingHorizontal: 10, gap: 2, minWidth: 0 },
  valueRow: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  cutline: { width: line, backgroundColor: colors.ink },
  about: { marginTop: space.xxl, gap: space.md },
  tagRow: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', columnGap: 10, rowGap: 6 },
  weather: { flexDirection: 'row', alignItems: 'center', gap: 14, minHeight: 72, padding: 12, borderRadius: radius.card, borderWidth: li
ne, borderColor: colors.ink, backgroundColor: colors.card },
  weatherIcon: { width: 48, height: 48, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: color
s.accentSoft, alignItems: 'center', justifyContent: 'center' },
  fact: { flexDirection: 'row', alignItems: 'flex-start', gap: 12, minHeight: 48, paddingVertical: 12, borderTopWidth: hair, borderTopCo
lor: colors.border },
  place: { flexDirection: 'row', alignItems: 'flex-start', gap: 12, minHeight: 56, paddingVertical: 14, borderTopWidth: hair, borderTopC
olor: colors.border },
  first: { borderTopWidth: 0 },
  barTitle: { height: 44, marginBottom: 10, marginLeft: 76, marginRight: 176, justifyContent: 'center' },
  bar: { position: 'absolute', top: 0, left: 0, right: 0, backgroundColor: colors.bg, borderBottomWidth: line, borderBottomColor: color
s.ink, justifyContent: 'flex-end' },
  placeIcon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.c
```

```
ard, alignItems: 'center', justifyContent: 'center' },
  footer: { position: 'absolute', left: 0, right: 0, bottom: 0, backgroundColor: colors.bg },
  footerBody: { paddingHorizontal: 20, paddingTop: 14, paddingBottom: 4 },
});
```

src/app/destinations.tsx

```
import { useLocalSearchParams } from 'expo-router';
import { useMemo, useState } from 'react';
import { ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';

import { PlaceCard } from '@/components/explore';
import { Chip, EmptyState } from '@/components/ui/bits';
import { Input } from '@/components/ui/Input';
import { Header, Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { destinations, exploreCategories, type DestinationTag } from '@/data/destinations';
import { useT } from '@/i18n';
import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, space } from '@/theme';
import { money } from '@/utils/format';

type Sort = 'rating' | 'cheap' | 'name';

/** The first range of "May–Jun, Sep–Oct": enough for a caption under a photo. */
const firstRange = (months: string) => months.split(',')[0].trim();

/**
 * Every destination as a photo: a search box, the interests as chips, a sort, then two columns of photo cards. Under each photo the
 * daily cost and the best months in mono, the two facts people choose by.
 */
export default function Destinations() {
  const t = useT();
  const params = useLocalSearchParams<{ q?: string; cat?: string }>();
  const { width } = useWindowDimensions();
  const favorites = useAppStore((s) => s.favorites);
  const [q, setQ] = useState(params.q ?? '');
  const [cat, setCat] = useState<'all' | DestinationTag | 'fav'>((params.cat as DestinationTag) ?? 'all');
  const [sort, setSort] = useState<Sort>('rating');
  const colW = Math.min(width, MAX_WIDTH) - 40;
  const cardW = Math.floor((colW - 12) / 2);

  const list = useMemo(() => {
    const s = q.trim().toLowerCase();
    return destinations
      .filter((d) => (cat === 'all' ? true : cat === 'fav' ? favorites.includes(d.id) : d.tags.includes(cat)))
      .filter((d) => !s || `${d.city} ${d.country} ${d.highlights.join(' ')}`.toLowerCase().includes(s))
      .sort((a, b) => (sort === 'rating' ? b.rating - a.rating : sort === 'cheap' ? a.dailyCost - b.dailyCost : a.city.localeCompare(b.c
ity)));
  }, [q, cat, sort, favorites]);

  return (
    <Screen header={<Header kicker="Explore" title="Destinations" subtitle={`${list.length} ${list.length === 1 ? 'place' : 'places'} to
explore`} />}>
      <Input icon="search" value={q} onChangeText={setQ} placeholder={t('explore.search')} accessibilityLabel="Search destinations" retu
rnKeyType="search" autoCorrect={false} />

      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
        <Chip label="Favourites" icon="star-outline" active={cat === 'fav'} onPress={() => setCat(cat === 'fav' ? 'all' : 'fav')} />
        {exploreCategories.map((c) => (
          <Chip key={c.key} label={t(c.labelKey)} image={c.image} active={cat === c.key} onPress={() => setCat(c.key)} />
        ))}
      </ScrollView>

      <View style={styles.sort}>
        <T variant="kicker" color={colors.textSecondary} style={{ marginBottom: 6 }}>
          Sort by
        </T>
        <Segmented
          value={sort}
          onChange={setSort}
          options={[
            { key: 'rating', label: 'Top rated' },
            { key: 'cheap', label: 'Cheapest' },
            { key: 'name', label: 'A to Z' },
          ]}
        />
      </View>

      {list.length ? (
        <View style={styles.grid}>
          {list.map((d) => (
            <View key={d.id} style={{ width: cardW, gap: 6 }}>
              <PlaceCard d={d} height={176} />
              <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
```

```
              {money(d.dailyCost)}/day · {firstRange(d.bestMonths)}
            </T>
          </View>
        ))}
      </View>
    ) : (
      <EmptyState icon="search-outline" title="No matches" body={cat === 'fav' ? 'Tap the star on any place to save it here.' : 'Try a
nother search or category.'} />
    )}
  </Screen>
 );
}

const styles = StyleSheet.create({
  // the chips run to the screen edges so they scroll under the margin, not into it
  chipScroll: { marginHorizontal: -20, marginTop: space.lg, flexGrow: 0 },
  chips: { gap: space.sm, paddingHorizontal: 20, paddingRight: 28 },
  sort: { marginTop: space.lg, marginBottom: space.xxl },
  grid: { flexDirection: 'row', flexWrap: 'wrap', columnGap: 12, rowGap: space.lg },
});
```

src/app/documents.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import * as Sharing from 'expo-sharing';
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { Platform, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { DocFileBody } from '@/components/documents/DocFileBody';
import { DocRow, DOC_ICON, docKind } from '@/components/documents/DocRow';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Card, Divider, EmptyState, type IconName } from '@/components/ui/bits';
import { Fab } from '@/components/ui/Fab';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { PlatePress } from '@/components/ui/Plate';
import { tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { useDraft } from '@/hooks/useDraft';
import { FileTooLargeError, deleteLocalFile, pickDocument, pickImage, sizeLabel } from '@/services/files';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { TripDoc } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { timeAgo } from '@/utils/format';
import { firstError, maxLen, required } from '@/utils/validation';

/** What can go in the vault, as the three big rows of the Add sheet. The label is the button's name. */
const KINDS: { key: 'file' | 'photo' | 'note'; label: string; sub: string; icon: IconName }[] = [
  { key: 'file', label: 'File', sub: 'A PDF, ticket or booking', icon: DOC_ICON.pdf },
  { key: 'photo', label: 'Photo', sub: 'A picture of a passport or card', icon: DOC_ICON.image },
  { key: 'note', label: 'Note', sub: 'Addresses and booking refs', icon: DOC_ICON.note },
];

/** Offline document vault: tickets, bookings, passport copies and notes. */
export default function Documents() {
  const params = useLocalSearchParams<{ open?: string }>();
  const trip = useActiveTrip();
  const insets = useSafeAreaInsets();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addDoc = useAppStore((s) => s.addDoc);
  const deleteDoc = useAppStore((s) => s.deleteDoc);
  const [open, setOpen] = useState<TripDoc | null>(() => (params.open ? trip?.docs.find((d) => d.id === params.open) ?? null : null));
  const [adding, setAdding] = useState(false);
  const [noteOpen, setNoteOpen] = useState(false);
  const [note, setNote, clearNote] = useDraft('doc-note', { name: '', note: '' });
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  if (!trip) return <NoTrip />;

  const upload = async (kind: 'file' | 'photo') => {
    setAdding(false);
    try {
      const f = kind === 'photo' ? await pickImage() : await pickDocument();
      if (!f) return;
      addDoc(trip.id, { name: f.name.replace(/\.[^.]+$/, ''), kind: f.kind, uri: f.uri, size: f.size, offline: true });
      tap('success');
      toast('Saved to the vault, available offline');
    } catch (e) {
      toast(e instanceof FileTooLargeError ? e.message : 'Couldn’t save that file', { tone: 'warn' });
    }
```

```
  };

  const choose = (k: 'file' | 'photo' | 'note') => {
    if (k === 'note') {
      setAdding(false);
      setNoteOpen(true);
    } else upload(k);
  };

  const openFile = async (uri: string, title: string) => {
    try {
      if (Platform.OS !== 'web' && (await Sharing.isAvailableAsync())) await Sharing.shareAsync(uri, { dialogTitle: title });
      else await Linking.openURL(uri);
    } catch {
      toast('No app available to open this file', { tone: 'warn' });
    }
  };

  const used = trip.docs.reduce((n, d) => n + (d.size ?? (d.note?.length ?? 0)), 0);
  const empty = trip.docs.length === 0;

  return (
    <Screen
      header={<Header title="Documents" subtitle={`${trip.docs.length} items · all available offline`} />}
      fab={empty ? undefined : <Fab icon="add" label="Add" accessibilityLabel="Add a document" onPress={() => setAdding(true)} />}
      fabBottom={insets.bottom + 16}>
      {empty ? (
        <EmptyState icon="folder-open-outline" title="No documents yet" body="Add tickets, bookings and passport copies so they’re alway
s on hand, even with no signal.">
          <Button label="Add a document" icon="add" onPress={() => setAdding(true)} />
        </EmptyState>
      ) : (
        <>
          <View style={styles.offline} accessible accessibilityLabel={`Works without wifi or data. ${sizeLabel(used) || '0 KB'} used on
this phone`}>
            <Ionicons name="cloud-done-outline" size={22} color={colors.ink} />
            <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }} importantForAccessibility="no">
              Works without wifi / {sizeLabel(used) || '0 KB'} used
            </T>
          </View>
          <Card padded={false} style={{ overflow: 'hidden' }}>
            {trip.docs.map((d, i) => (
              <View key={d.id}>
                {i > 0 ? <Divider /> : null}
                <DocRow doc={d} trip={trip} onPress={() => setOpen(d)} />
              </View>
            ))}
          </Card>
        </>
      )}

      <Sheet visible={adding} onClose={() => setAdding(false)} title="Add to the vault" subtitle="Saved on this phone, so it opens with
no signal">
        <View style={{ gap: 14, paddingBottom: 8 }}>
          {KINDS.map((k) => (
            <PlatePress key={k.key} onPress={() => choose(k.key)} accessibilityLabel={k.label} offset={3} corner={radius.button} style=
{styles.kind}>
              <View style={styles.kindIcon}>
                <Ionicons name={k.icon} size={22} color={colors.ink} />
              </View>
              <View style={{ flex: 1 }}>
                <T variant="title" weight="bold">
                  {k.label}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  {k.sub}
                </T>
              </View>
              <Ionicons name="arrow-forward" size={18} color={colors.ink} />
            </PlatePress>
          ))}
        </View>
      </Sheet>

      <Sheet visible={!!open} onClose={() => setOpen(null)} title={open?.name} subtitle={open ? `${docKind(open)} · added ${timeAgo(ope
n.addedAt)}` : undefined}>
        {open?.kind === 'note' ? (
          <View style={styles.note}>
            <T variant="bodySm" selectable>
              {open.note}
            </T>
          </View>
        ) : null}
        {open ? <DocFileBody doc={open} adderName={trip.members.find((m) => m.id === open.addedBy)?.name.split(' ')[0] ?? 'a friend'} on
Open={(uri) => openFile(uri, open.name)} /> : null}
        {open && (open.addedBy === userId || trip.ownerId === userId) ? (
```

```
          <View style={{ marginTop: 16 }}>
            <Button
              label="Delete"
              variant="danger"
              size="md"
              icon="trash-outline"
              onPress={() => {
                const d = open;
                setOpen(null);
                confirmAction('Delete document?', `“${d.name}” will be removed from the vault.`, () => {
                  deleteLocalFile(d.uri);
                  deleteDoc(trip.id, d.id);
                  toast('Document deleted');
                }, { destructive: true, confirmLabel: 'Delete' });
              }}
            />
          </View>
        ) : null}
      </Sheet>

      <Sheet
        visible={noteOpen}
        onClose={() => setNoteOpen(false)}
        title="New note"
        subtitle="Addresses, booking refs, allergy cards…"
        footer={
          <Button
            label="Save note"
            icon="checkmark"
            onPress={() => {
              const e = { name: firstError(note.name, required('a title'), maxLen(40, 'Title')), note: firstError(note.note, required('s
ome text'), maxLen(2000, 'Note')) };
              setErrors(e);
              if (e.name || e.note) return tap('warning');
              addDoc(trip.id, { name: note.name.trim(), kind: 'note', note: note.note.trim(), offline: true });
              clearNote();
              setNoteOpen(false);
              toast('Note saved offline');
            }}
          />
        }>
        <Input label="Title" value={note.name} onChangeText={(v) => setNote({ ...note, name: v })} error={errors.name} placeholder="e.g.
Hotel address in Japanese" />
        <Input label="Note" value={note.note} onChangeText={(v) => setNote({ ...note, note: v })} error={errors.note} multiline placehol
der="Write anything you’ll need offline…" style={{ minHeight: 140 }} />
      </Sheet>
    </Screen>
  );
}

const styles = StyleSheet.create({
  offline: { flexDirection: 'row', alignItems: 'center', gap: 10, marginBottom: 20, minHeight: 28 },
  kind: { flexDirection: 'row', alignItems: 'center', gap: 14, minHeight: 68, paddingHorizontal: 14, paddingVertical: 10, backgroundColo
r: colors.card, borderWidth: line, borderColor: colors.ink, borderRadius: radius.button },
  kindIcon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.ac
centSoft, alignItems: 'center', justifyContent: 'center' },
  note: { backgroundColor: colors.bgAlt, padding: 14, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink },
});
```

src/app/emergency.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { Button } from '@/components/ui/Button';
import { ListRow, SectionHeader, type IconName } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { PlatePress } from '@/components/ui/Plate';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { emergencyByCountry } from '@/data/reference';
import { getLocation } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, hair, line, radius, space } from '@/theme';

/** Safety screen: everything here works offline except the map searches. */
export default function Emergency() {
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const info = emergencyByCountry[dest.countryCode] ?? emergencyByCountry.DEFAULT;
  const [shareMsg, setShareMsg] = useState<string | null>(null);
```

```
  const call = (n: string) => Linking.openURL(`tel:${n}`).catch(() => toast(`Dial ${n} from your phone`, { tone: 'warn' }));

  const shareLocation = async () => {
    const loc = await getLocation(dest);
    if (loc.source !== 'device') toast('GPS unavailable. Sharing the trip centre instead', { tone: 'warn' });
    setShareMsg(`I need help. My location: https://maps.google.com/?q=${loc.lat.toFixed(5)},${loc.lng.toFixed(5)} (sent from TogetherWeGo, ${trip?.name ?? 'trip'})`);
  };

  const calls: [string, string, IconName][] = [
    ['Police', info.police, 'shield-outline'],
    ['Ambulance', info.ambulance, 'medkit-outline'],
    ['Fire', info.fire, 'flame-outline'],
  ];
  const help: { icon: IconName; label: string; sub: string; onPress: () => void; tint?: string }[] = [
    { icon: 'medical-outline', label: 'Find the nearest hospital', sub: 'Opens maps around you', tint: colors.red, onPress: () => Linking.openURL('https://www.google.com/maps/search/hospital+near+me') },
    { icon: 'flag-outline', label: 'Find my embassy', sub: `Embassies and consulates in ${dest.city}`, onPress: () => Linking.openURL(`https://www.google.com/maps/search/embassy+${encodeURIComponent(dest.city)}`) },
    { icon: 'document-lock-outline', label: 'Passport copies & insurance', sub: 'Your offline document vault', onPress: () => router.push('/documents') },
    { icon: 'language-outline', label: 'Emergency phrases', sub: `“Help!” and more in ${dest.language}`, onPress: () => router.push('/phrasebook') },
    { icon: 'airplane-outline', label: 'Nearest airport', sub: 'Closest airports and directions', onPress: () => router.push('/airport') },
  ];

  return (
    <Screen header={<Header kicker={`${dest.country} · works offline`} title="Emergency" subtitle="Tap a number to call it" backLabel="More" />}>
      <View style={styles.zone}>
        <View style={styles.zoneHead}>
          <Ionicons name="warning-outline" size={18} color={colors.red} />
          <T variant="kicker" color={colors.red}>
            Call in an emergency
          </T>
        </View>
        {calls.map(([label, num, icon]) => (
          <PlatePress key={label} onPress={() => call(num)} accessibilityLabel={`Call ${label}, ${num}`} color={colors.red} offset={4} corner={radius.card} style={styles.call}>
            <View style={styles.callIcon}>
              <Ionicons name={icon} size={26} color={colors.red} />
            </View>
            <View style={{ flex: 1 }}>
              <T variant="h3">{label}</T>
              <T variant="caption" color={colors.textSecondary}>
                Tap to call
              </T>
            </View>
            <T variant="numeral" style={styles.num}>
              {num}
            </T>
          </PlatePress>
        ))}
      </View>

      <Button label="Share my live location" icon="location" accessibilityHint="Sends your position to the group" style={{ marginTop: space.xl }} onPress={shareLocation} />

      <View style={{ marginTop: space.section }}>
        <SectionHeader title={`In ${dest.country}`} />
        {info.tips.map((tip, i) => (
          <View key={tip} style={[styles.tip, i === 0 && { borderTopWidth: 0 }]}>
            <Ionicons name="information-circle-outline" size={20} color={colors.ink} style={{ marginTop: 1 }} />
            <T variant="bodySm" style={{ flex: 1 }}>
              {tip}
            </T>
          </View>
        ))}
      </View>

      <View style={{ marginTop: space.section }}>
        <SectionHeader title="Find help" />
        {help.map((h, i) => (
          <View key={h.label} style={[styles.helpRow, i === 0 && { borderTopWidth: 0 }]}>
            <ListRow icon={h.icon} label={h.label} sub={h.sub} tint={h.tint} onPress={h.onPress} />
          </View>
        ))}
      </View>

      <ShareSheet visible={!!shareMsg} onClose={() => setShareMsg(null)} title="Send your location" message={shareMsg ?? ''} preview={shareMsg ?? ''} />
    </Screen>
  );
}
```

```
const styles = StyleSheet.create({
  // the red state area: everything you need in the first second
  zone: { gap: space.lg, padding: space.lg, paddingRight: space.lg + 4, borderRadius: radius.card, borderWidth: line, borderColor: color
s.red, backgroundColor: colors.redSoft },
  zoneHead: { flexDirection: 'row', alignItems: 'center', gap: space.sm },
  call: { flexDirection: 'row', alignItems: 'center', gap: space.md, minHeight: 76, paddingHorizontal: 12, paddingVertical: 10, borderRa
dius: radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
  callIcon: { width: 48, height: 48, borderRadius: radius.button, borderWidth: line, borderColor: colors.red, backgroundColor: colors.re
dSoft, alignItems: 'center', justifyContent: 'center' },
  num: { fontSize: 38, lineHeight: 42 },
  tip: { flexDirection: 'row', gap: space.md, paddingVertical: space.md, borderTopWidth: hair, borderTopColor: colors.border },
  helpRow: { borderTopWidth: hair, borderTopColor: colors.border },
});
```

src/app/flights.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { AddFlightSheet } from '@/components/flights/AddFlightSheet';
import { FlightTicket, zoneNoteFor } from '@/components/flights/FlightTicket';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, EmptyState } from '@/components/ui/bits';
import { Fab } from '@/components/ui/Fab';
import { confirmAction } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Flight, Member } from '@/store/types';
import { colors, line } from '@/theme';
import { durationLabel } from '@/utils/format';
import { dayLabel, deviceTimeZone, partsInTz, time12, tzAbbrev, utcOffsetLabel } from '@/utils/time';

type Person = { id: string; name: string; member?: Member; flights: Flight[] };

/** Flights: everyone's flights as boarding passes, grouped by person, and how long it takes the whole group to land. */
export default function Flights() {
  const trip = useActiveTrip();
  const insets = useSafeAreaInsets();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const showTripTime = useAppStore((s) => s.settings.showTripTime);
  const updateSettings = useAppStore((s) => s.updateSettings);
  const deleteFlight = useAppStore((s) => s.deleteFlight);
  const [adding, setAdding] = useState(false);
  if (!trip) return <NoTrip />;
  const dest = destinationById(trip.destinationId)!;
  const tz = showTripTime ? dest.tz : deviceTimeZone();
  const zoneName = showTripTime ? `${dest.city} time` : 'your time';
  const sorted = trip.flights.slice().sort((a, b) => (a.direction === 'arrival' ? a.arriveUtc : a.departUtc) - (b.direction === 'arriva
l' ? b.arriveUtc : b.departUtc));
  const arrivals = sorted.filter((f) => f.direction === 'arrival');
  const firstLand = arrivals[0];
  const lastLand = arrivals[arrivals.length - 1];

  const everyone: Person[] = trip.members.map((m) => ({ id: m.id, name: m.id === userId ? 'You' : m.name, member: m, flights: sorted.fil
ter((f) => f.memberId === m.id) }));
  const former = sorted.filter((f) => !trip.members.some((m) => m.id === f.memberId));
  if (former.length) everyone.push({ id: 'former', name: 'Former member', flights: former });
  const people = [...everyone.filter((p) => p.flights.length), ...everyone.filter((p) => !p.flights.length)];

  const when = (utc: number) => {
    const p = partsInTz(utc, tz);
    return `${time12(p.hhmm)}, ${dayLabel(p.ymd)}`;
  };
  const empty = sorted.length === 0;
  const firstId = people.find((p) => p.flights.length)?.flights[0]?.id;

  return (
    <Screen
      header={<Header title="Flights" subtitle={empty ? 'Who lands when, and where' : `${sorted.length} flight${sorted.length === 1 ? ''
: 's'} · ${trip.members.length} travellers`} />}
      fab={empty ? undefined : <Fab icon="add" label="Add flight" testID="add-flight" onPress={() => setAdding(true)} />}
      fabBottom={insets.bottom + 16}>
      {empty ? (
        <EmptyState icon="airplane-outline" title="No flights yet" body="Add your flights so everyone can see who lands when, and plan t
he first day around it.">
          <Button label="Add a flight" icon="add" testID="add-flight" onPress={() => setAdding(true)} />
        </EmptyState>
      ) : (
        <>
```

```
          <Segmented
            value={showTripTime ? 'trip' : 'mine'}
            onChange={(k) => updateSettings({ showTripTime: k === 'trip' })}
            options={[
              { key: 'trip', label: `${dest.city} time` },
              { key: 'mine', label: 'My time' },
            ]}
          />
          <T variant="kicker" color={colors.textSecondary} style={styles.zone}>
            Airport clocks on the passes / group times in {tzAbbrev(tz)} ({utcOffsetLabel(tz)})
          </T>

          {firstLand && lastLand ? (
            <Card style={styles.landing}>
              <Ionicons name="people-outline" size={22} color={colors.ink} />
              <View style={{ flex: 1, gap: 2 }}>
                <T variant="kicker" color={colors.textSecondary}>
                  Group landing
                </T>
                <T variant="bodySm" weight="semibold">
                  First in {when(firstLand.arriveUtc)}, last in {when(lastLand.arriveUtc)}.
                  {lastLand !== firstLand ? ` The group is complete ${durationLabel(Math.round((lastLand.arriveUtc - firstLand.arriveUt
c) / 60000))} after the first landing.` : ''}
                </T>
              </View>
            </Card>
          ) : null}

          {people.map((p) => (
            <View key={p.id} style={styles.person}>
              <View style={styles.who}>
                <Avatar name={p.member?.name ?? p.name} src={p.member?.avatar} size={36} />
                <T variant="h3" numberOfLines={1} style={{ flex: 1 }}>
                  {p.name}
                </T>
              </View>
              {p.flights.length ? (
                <View style={{ gap: 20 }}>
                  {p.flights.map((f) => (
                    <FlightTicket
                      key={f.id}
                      flight={f}
                      passenger={p.name === 'You' ? 'You' : p.name.split(' ')[0]}
                      plated={f.id === firstId}
                      zoneNote={zoneNoteFor(f, tz, zoneName)}
                      canDelete={f.memberId === userId || trip.ownerId === userId}
                      onDelete={() => confirmAction('Remove flight?', `${f.flightNo} will be removed from the trip.`, () => deleteFlight
(trip.id, f.id), { destructive: true, confirmLabel: 'Remove' })}
                    />
                  ))}
                </View>
              ) : (
                <View style={styles.none}>
                  <T variant="small" color={colors.textSecondary}>
                    No flight added yet
                  </T>
                </View>
              )}
            </View>
          ))}
        </>
      )}

      <AddFlightSheet visible={adding} onClose={() => setAdding(false)} />
    </Screen>
  );
}

const styles = StyleSheet.create({
  zone: { marginTop: 10 },
  landing: { flexDirection: 'row', alignItems: 'center', gap: 12, marginTop: 20, backgroundColor: colors.accentSoft },
  person: { marginTop: 32, gap: 14 },
  who: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  none: { minHeight: 56, justifyContent: 'center', paddingHorizontal: 14, borderRadius: 12, borderWidth: line, borderStyle: 'dashed', bo
rderColor: colors.textSecondary },
});
```

src/app/forgot-password.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useRef, useState } from 'react';
import { StyleSheet, View, type TextInput } from 'react-native';

import { AuthScreen, PasswordMeter, StepSegments, TextLink } from '@/components/auth/kit';
import { Button } from '@/components/ui/Button';
```

```
import { showDialog, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useDraft } from '@/hooks/useDraft';
import { useAppStore } from '@/store/useAppStore';
import { colors, fonts, line, radius } from '@/theme';
import { validateEmail, validatePassword } from '@/utils/validation';

const tick = (ok: boolean) => (ok ? <Ionicons name="checkmark-circle" size={20} color={colors.positive} accessibilityLabel="Looks good"
/> : undefined);

/** Two-step reset: ask for a 6-digit code, then set a new password. Works fully offline: this demo build shows the code on screen inste
ad of emailing it. */
export default function ForgotPassword() {
  const accountExists = useAppStore((s) => s.accountExists);
  const resetPassword = useAppStore((s) => s.resetPassword);
  const [step, setStep] = useState<1 | 2>(1);
  const [email, setEmail, clearEmail] = useDraft('reset-email', '');
  const [code, setCode] = useState('');
  const [sent, setSent] = useState('');
  const [pw, setPw] = useState('');
  const [shown, setShown] = useState({ email: false, code: false, pw: false });
  const [notFound, setNotFound] = useState<string | null>(null);
  const [wrong, setWrong] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const pwRef = useRef<TextInput>(null);

  const emailErr = validateEmail(email);
  const codeErr = !/^\d{6}$/.test(code) ? 'Enter the 6-digit code' : code !== sent ? 'That code doesn’t match' : null;
  const pwErr = validatePassword(pw);

  const sendCode = () => {
    setShown((s) => ({ ...s, email: true }));
    if (emailErr) return tap('warning');
    if (!accountExists(email)) {
      setNotFound('We couldn’t find an account with that email.');
      return tap('warning');
    }
    const c = String(Math.floor(100000 + Math.random() * 900000));
    setSent(c);
    setCode('');
    setShown({ email: true, code: false, pw: false });
    setNotFound(null);
    setStep(2);
    showDialog('Check your inbox', `We sent a 6-digit code to ${email.trim()}.\n\nDemo build: your code is ${c}`, [{ label: 'OK' }], 'ma
il-unread-outline');
  };

  const reset = async () => {
    setShown({ email: true, code: true, pw: true });
    if (codeErr || pwErr) return tap('warning');
    setLoading(true);
    const r = await resetPassword(email, pw);
    setLoading(false);
    if (!r.ok) {
      setWrong(r.error);
      return tap('warning');
    }
    tap('success');
    clearEmail();
    toast('Password updated. Sign in with your new password');
    router.back();
  };

  return (
    <AuthScreen top={0}>
      <View style={{ marginHorizontal: -20 }}>
        <Header kicker={`Step ${step} of 2`} title="Reset password" subtitle={step === 1 ? 'Verify your email' : 'Choose a new passwor
d'} />
      </View>
      <StepSegments now={step} total={2} />

      <T variant="body" color={colors.textSecondary} style={{ marginTop: 20 }}>
        {step === 1 ? 'Enter the email you signed up with and we will send you a reset code.' : `Enter the code sent to ${email.trim()}
and choose a new password.`}
      </T>

      {step === 1 ? (
        <View style={{ marginTop: 20, gap: 20 }}>
          <Input
            label="Email address"
            icon={(shown.email && emailErr) || notFound ? 'alert-circle' : 'mail-outline'}
            value={email}
            onChangeText={(v) => {
```

```
              setEmail(v);
              setNotFound(null);
            }}
            onBlur={() => email && setShown((s) => ({ ...s, email: true }))}
            keyboardType="email-address"
            autoCapitalize="none"
            autoCorrect={false}
            autoComplete="email"
            returnKeyType="go"
            onSubmitEditing={sendCode}
            error={notFound ?? (shown.email ? emailErr : null)}
            right={tick(!!email && !emailErr && !notFound)}
            placeholder="you@email.com"
          />
          <Button label="Send reset code" onPress={sendCode} icon="paper-plane-outline" />
        </View>
      ) : (
        <View style={{ marginTop: 20, gap: 16 }}>
          <View style={{ gap: 10 }}>
            <Input
              label="6-digit code"
              icon={(shown.code && codeErr) || wrong ? 'alert-circle' : 'key-outline'}
              value={code}
              onChangeText={(v) => {
                setCode(v.replace(/\D/g, '').slice(0, 6));
                setWrong(null);
              }}
              onBlur={() => code && setShown((s) => ({ ...s, code: true }))}
              keyboardType="number-pad"
              maxLength={6}
              returnKeyType="next"
              onSubmitEditing={() => pwRef.current?.focus()}
              error={wrong ?? (shown.code ? codeErr : null)}
              right={tick(!codeErr && !wrong)}
              style={{ fontFamily: fonts.mono, fontSize: 22, letterSpacing: 6 }}
              placeholder="000000"
              testID="reset-code"
            />
            <Press onPress={() => setCode(sent)} scaleTo={0.99} accessibilityLabel={`Fill in the demo code ${sent}`} style={styles.demo}
>
              <View style={{ flex: 1 }}>
                <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 10 }}>
                  Demo build: no email is sent
                </T>
                <T variant="numeralSm" style={{ fontSize: 18, lineHeight: 22 }}>
                  {sent}
                </T>
              </View>
              <T variant="kicker">Fill it in</T>
              <Ionicons name="arrow-forward" size={16} color={colors.ink} />
            </Press>
          </View>
          <View style={{ gap: 10 }}>
            <Input
              ref={pwRef}
              label="New password"
              icon={shown.pw && pwErr ? 'alert-circle' : 'lock-closed-outline'}
              value={pw}
              onChangeText={setPw}
              onBlur={() => pw && setShown((s) => ({ ...s, pw: true }))}
              secureToggle
              textContentType="newPassword"
              returnKeyType="go"
              onSubmitEditing={reset}
              error={shown.pw ? pwErr : null}
              hint="At least 8 characters, with a letter and a number"
              placeholder="At least 8 characters"
            />
            <PasswordMeter password={pw} />
          </View>
          <Button label="Update password" onPress={reset} loading={loading} />
          <View style={styles.links}>
            <TextLink label="Resend code" role="button" onPress={sendCode} />
            <TextLink
              label="Use a different email"
              role="button"
              onPress={() => {
                setStep(1);
                setWrong(null);
              }}
            />
          </View>
        </View>
      )}
    </AuthScreen>
  );
```

```
}

const styles = StyleSheet.create({
  demo: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 56, paddingHorizontal: 12, borderRadius: radius.chip, borderWi
dth: line, borderStyle: 'dashed', borderColor: colors.ink, backgroundColor: colors.accentSoft },
  links: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', justifyContent: 'space-between', columnGap: 16, marginTop: -4
},
});
```

src/app/help.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, type Href } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Card, SectionHeader } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { colors, hair, line, radius, space } from '@/theme';

const JOURNEY: { title: string; body: string; href: Href }[] = [
  { title: 'Create or join a trip', body: 'Pick a destination, dates and a group budget, or enter a friend’s invite code.', href: '/trip
s' },
  { title: 'Invite your group', body: 'More, then Crew & roles, then Share sends it straight to WhatsApp, Instagram, Messages and mor
e.', href: '/crew' },
  { title: 'Give everyone a role', body: 'Treasurer, Navigator, Food Lead… tap a member to assign who owns what.', href: '/crew' },
  { title: 'Build the plan', body: 'In the Plan tab, add activities per day, filter by type or length, and vote with “I’m in”. A majorit
y confirms.', href: '/(tabs)/itinerary' },
  { title: 'Add flights & tickets', body: 'Everyone’s arrivals show in one list, converted to the trip’s time zone automatically.', hre
f: '/flights' },
  { title: 'Track the budget', body: 'In the Budget tab, log expenses in any currency, see the breakdown and settle up with the fewest p
ayments.', href: '/(tabs)/budget' },
  { title: 'Chat & ask the assistant', body: 'In the Chat tab, message the group, or open the Scout AI tab to ask about budget, weather,
food or what’s next.', href: '/(tabs)/chat' },
  { title: 'Get live reminders', body: 'Tap the bell on any activity and everyone gets an alert before it starts.', href: '/settings' },
  { title: 'Wrap it up', body: 'Mark the trip complete for a shareable recap, final balances and the photo dump.', href: '/recap' },
];

const FAQ = [
  ['Does it work without internet?', 'Yes. Trips, chats, documents, the phrasebook, emergency numbers, the offline map and the Trip Assi
stant all work offline. Weather and currency rates refresh when you’re back online and are cached meanwhile.'],
  ['Where is my data stored?', 'On your phone, in TogetherWeGo’s private storage. Passwords are salted and hashed; an optional AI key go
es in the secure keychain.'],
  ['How do time zones work?', 'Itinerary times are in the destination’s local time. Flights are stored as exact moments, so they display
correctly in trip time or your own time. Toggle it in Flights.'],
  ['How does voting work?', 'Activities marked “Voting” become “Confirmed” once more than half the group taps “I’m in”. Polls in More le
t the group pick between options.'],
  ['Can I change the language?', 'Settings, then Language: English, Español, Français, 日本語, हिन्दी and 中文.'],
  ['How do I try the demo?', 'Sign in with the demo account on the sign-in screen. In Settings, Advanced, Demo timeline switches the sam
ple trip between upcoming, in progress and completed.'],
];

/** A tour of the app in nine steps, then the questions people ask. Every step opens the screen it talks about. */
export default function Help() {
  const [open, setOpen] = useState<number | null>(0);
  return (
    <Screen header={<Header kicker="TogetherWeGo" title="Help" subtitle="Your group trip, from first idea to final recap" backLabel="Mor
e" />}>
      <Card onPress={() => router.push('/how-it-works')} style={styles.how} accessibilityLabel="How it works">
        <View style={styles.howIcon}>
          <Ionicons name="construct-outline" size={22} color={colors.onAccent} />
        </View>
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold">
            How it works
          </T>
          <T variant="caption" color={colors.textSecondary}>
            How the app is built, how your data is kept safe, and where to find every feature
          </T>
        </View>
        <Ionicons name="arrow-forward" size={16} color={colors.ink} />
      </Card>

      <View style={{ marginTop: space.section }}>
        <SectionHeader title="The journey" />
        {JOURNEY.map((s, i) => (
          <Press key={s.title} onPress={() => router.push(s.href)} style={[styles.step, i === 0 && styles.first]} scaleTo={0.99} accessi
bilityLabel={`Step ${i + 1}. ${s.title}`}>
            <View style={styles.num}>
              <T variant="numeralSm" style={{ fontSize: 20, lineHeight: 26 }}>
                {i + 1}
              </T>
            </View>
            <View style={{ flex: 1, gap: 2 }}>
```

```
              <T variant="bodySm" weight="semibold">
                {s.title}
              </T>
              <T variant="caption" color={colors.textSecondary}>
                {s.body}
              </T>
            </View>
            <Ionicons name="arrow-forward" size={16} color={colors.ink} style={{ marginTop: 14 }} />
          </Press>
        ))}
      </View>

      <View style={{ marginTop: space.section }}>
        <SectionHeader title="Questions" />
        {FAQ.map(([q, a], i) => (
          <Press key={q} onPress={() => setOpen(open === i ? null : i)} style={[styles.faq, i === 0 && styles.first]} scaleTo={0.99} acc
essibilityLabel={q} accessibilityState={{ expanded: open === i }}>
            <View style={styles.q}>
              <T variant="bodySm" weight="semibold" style={{ flex: 1 }}>
                {q}
              </T>
              <Ionicons name={open === i ? 'remove' : 'add'} size={22} color={colors.ink} />
            </View>
            {open === i ? (
              <T variant="small" color={colors.textSecondary} style={{ marginTop: space.sm }}>
                {a}
              </T>
            ) : null}
          </Press>
        ))}
      </View>
    </Screen>
  );
}

const styles = StyleSheet.create({
  how: { flexDirection: 'row', alignItems: 'center', gap: space.md },
  howIcon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.acc
ent, alignItems: 'center', justifyContent: 'center' },
  step: { flexDirection: 'row', alignItems: 'flex-start', gap: space.md, paddingVertical: space.md, borderTopWidth: hair, borderTopColo
r: colors.border },
  first: { borderTopWidth: 0 },
  num: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, a
lignItems: 'center', justifyContent: 'center' },
  faq: { paddingVertical: space.md, minHeight: 56, justifyContent: 'center', borderTopWidth: hair, borderTopColor: colors.border },
  q: { flexDirection: 'row', alignItems: 'center', gap: space.md },
});
```

src/app/how-it-works.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, type Href } from 'expo-router';
import type { ReactNode } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { SectionHeader, type IconName } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, hair, line, radius, space } from '@/theme';

const LAYERS: { icon: IconName; title: string; where: string; body: string }[] = [
  { icon: 'phone-portrait-outline', title: 'Screens', where: 'src/app · src/components', body: 'Show the data and turn taps into action
s.' },
  { icon: 'funnel-outline', title: 'Selector hooks', where: 'useActiveTrip · balances · settlePlan', body: 'Work out exactly what each s
creen needs.' },
  { icon: 'server-outline', title: 'Zustand + Immer store', where: 'src/store', body: 'One source of truth. Saved to AsyncStorage on eve
ry change.' },
  { icon: 'sync-outline', title: 'Sync engine', where: 'src/sync · server', body: 'Keeps a shared trip identical on every phone, even af
ter being offline.' },
  { icon: 'cloud-outline', title: 'Services', where: 'src/services', body: 'Weather, currency, notifications, AI, files and sharing.' },
];

const SYNC: [IconName, string, string][] = [
  ['create-outline', 'Every change is a tiny note', 'Adding an expense or sending a message becomes a small stamped change, not a whole
copy of the trip.'],
  ['git-merge-outline', 'Newest change wins, per field', 'Two people can edit different things at once and both survive. Phones that swa
p their changes end up identical, in any order.'],
  ['cloud-offline-outline', 'Works with no signal', 'Changes wait on the phone and are sent when you are back online. Nothing is lost if
the app closes.'],
  ['images-outline', 'Photos and files travel too', 'Photos, documents and receipts are sent to the server separately, then saved on eve
ry phone so they open offline.'],
  ['server-outline', 'Your own server, no accounts', 'A small program on your computer (npm run server) passes changes between phones. A
trip is protected by a long random id that only the invite code reveals.'],
```

```
];

const SECURITY: [IconName, string, string][] = [
  ['key-outline', 'Passwords are never stored', 'Each one is salted and stretched with PBKDF2-SHA256 before it is saved, so even the sto
red copy can’t be read back.'],
  ['lock-closed-outline', 'API key in the keychain', 'The optional AI key lives in the iOS Keychain or Android Keystore (expo-secure-sto
re), not in the app database.'],
  ['cloud-offline-outline', 'Offline-first', 'Trips, chats, documents and maps are saved on the phone. Weather and exchange rates are ca
ched for when you lose signal.'],
  ['shield-checkmark-outline', 'Your data stays with you', 'Trips and chats are stored on the device. A trip you share is also kept on t
he sync server you chose. Only weather, exchange-rate and optional AI requests reach the wider internet.'],
  ['eye-off-outline', 'Private stays private', 'Your Scout AI chat, other people’s email addresses and your flight tickets are never sen
t to the sync server.'],
];

const VALIDATION: [string, string, string][] = [
  ['Email', 'Must look like name@mail.com', 'That email doesn’t look right'],
  ['Password', '8+ characters with a letter and a number', 'Use at least 8 characters'],
  ['Expense amount', 'Numbers only, above 0 and at most 1,000,000', 'Use numbers only, e.g. 25 or 25.50'],
  ['Trip dates', 'End on or after the start, up to 60 days', 'End date must be on or after the start date'],
  ['Invite code', 'Three letters, a dash, four digits', 'Codes look like ABC-1234'],
  ['Flight number', 'Airline code plus 1 to 4 digits', 'Flight numbers look like NH 175'],
  ['Packing item', '50 characters or fewer, no duplicates', 'That’s already on the list'],
];

const RUBRIC: { icon: IconName; area: string; where: string; href: Href }[] = [
  { icon: 'people-outline', area: 'Collaboration', where: 'More: members, roles, polls and checklist, plus live sync across phones. Pla
n: voting. Packing: who’s bringing what.', href: '/crew' },
  { icon: 'chatbubbles-outline', area: 'Communication', where: 'Chat tab: group chat and the Scout AI assistant.', href: '/(tabs)/chat'
},
  { icon: 'calendar-outline', area: 'Scheduling', where: 'Plan tab: day-by-day itinerary, reminders, flights and time zones.', href: '/
(tabs)/itinerary' },
  { icon: 'wallet-outline', area: 'Budgeting', where: 'Budget tab: expenses, split, settle up and currency converter.', href: '/(tabs)/b
udget' },
  { icon: 'folder-open-outline', area: 'Organization', where: 'More: offline docs, packing list, weather, photo dump and printable plann
er.', href: '/documents' },
  { icon: 'color-palette-outline', area: 'Other features', where: 'Themes and languages, trip quiz, bucket list, phrasebook and the reca
p story card.', href: '/personalize' },
  { icon: 'airplane-outline', area: 'Trip management', where: 'Home: switch trips. Create or join with an invite code. Recap when the tr
ip ends.', href: '/trips' },
];

const LIBRARIES = [
  'Expo SDK 57',
  'React Native 0.86',
  'TypeScript',
  'Expo Router',
  'Zustand',
  'Immer',
  'AsyncStorage',
  'expo-crypto',
  'expo-secure-store',
  'expo-notifications',
  'expo-location',
  'expo-image-picker',
  'expo-document-picker',
  'expo-file-system',
  'expo-print',
  'expo-sharing',
  'expo-camera',
  'react-native-view-shot',
  'qrcode-generator',
  '@noble/hashes',
  'expo-speech',
  'expo-haptics',
  'react-native-maps',
  'react-native-svg',
  'Reanimated',
  'Anthropic Messages API via fetch (optional)',
  'Jest',
];

/** One part of the page: a serif title over a rule, then its content. The gap between parts is the big one. */
function Section({ title, children, first }: { title: string; children: ReactNode; first?: boolean }) {
  return (
    <View style={{ marginTop: first ? 0 : space.section }}>
      <SectionHeader title={title} />
      {children}
    </View>
  );
}

/** A line icon, a bold title and a few words, one under the other with a hair rule between. */
function Rows({ items }: { items: [IconName, string, string][] }) {
  return (
    <View>
```

```
      {items.map(([icon, title, body], i) => (
        <View key={title} style={[styles.row, i === 0 && styles.first]}>
          <Ionicons name={icon} size={22} color={colors.ink} style={{ marginTop: 1 }} />
          <View style={{ flex: 1, gap: 2 }}>
            <T variant="bodySm" weight="bold">
              {title}
            </T>
            <T variant="small" color={colors.textSecondary}>
              {body}
            </T>
          </View>
        </View>
      ))}
    </View>
  );
}

/** How the app is built, drawn for the judges: layers, sync, security, validation, the rubric map and the libraries. */
export default function HowItWorks() {
  const setToured = useAppStore((s) => s.setToured);
  return (
    <Screen header={<Header kicker="TogetherWeGo" title="How it works" subtitle="How TogetherWeGo is built and where to find everything"
backLabel="More" />}>
      <Section title="Architecture" first>
        <T variant="small" color={colors.textSecondary}>
          A layered, MVVM-style design. Data flows one way: a screen calls a store action, the state updates, and the screen redraws.
        </T>
        <View style={{ marginTop: space.lg }}>
          {LAYERS.map((l, i) => (
            <View key={l.title}>
              <View style={styles.layer}>
                <View style={styles.layerIcon}>
                  <Ionicons name={l.icon} size={20} color={colors.ink} />
                </View>
                <View style={{ flex: 1, gap: 1 }}>
                  <T variant="bodySm" weight="bold">
                    {l.title}
                  </T>
                  <T variant="kicker" color={colors.textSecondary} style={{ textTransform: 'none' }}>
                    {l.where}
                  </T>
                  <T variant="small" color={colors.textSecondary}>
                    {l.body}
                  </T>
                </View>
              </View>
              {i < LAYERS.length - 1 ? <Ionicons name="arrow-down" size={16} color={colors.ink} style={{ alignSelf: 'center', marginVert
ical: 4 }} /> : null}
            </View>
          ))}
        </View>
      </Section>

      <Section title="Sync across phones">
        <T variant="small" color={colors.textSecondary} style={{ marginBottom: space.sm }}>
          Friends on different phones see the same plan, chat, votes and budget, live.
        </T>
        <Rows items={SYNC} />
      </Section>

      <Section title="Data and security">
        <Rows items={SECURITY} />
      </Section>

      <Section title="Input validation">
        <T variant="small" color={colors.textSecondary}>
          Every form checks the format and whether the value makes sense, then shows a plain message next to the field.
        </T>
        <View style={{ marginTop: space.sm }}>
          {VALIDATION.map(([field, rule, message], i) => (
            <View key={field} style={[styles.row, styles.col, i === 0 && styles.first]}>
              <T variant="bodySm" weight="bold">
                {field}
              </T>
              <T variant="small" color={colors.textSecondary}>
                {rule}
              </T>
              <View style={styles.msg}>
                <Ionicons name="alert-circle" size={14} color={colors.red} />
                <T variant="caption" color={colors.red} style={{ flexShrink: 1 }}>
                  {message}
                </T>
              </View>
            </View>
          ))}
        </View>
```

```
      </Section>

      <Section title="Rubric map">
        <T variant="small" color={colors.textSecondary} style={{ marginBottom: space.sm }}>
          Each competition area and where to find it. Tap a row to open it.
        </T>
        {RUBRIC.map((r, i) => (
          <Press key={r.area} onPress={() => router.push(r.href)} style={[styles.row, styles.press, i === 0 && styles.first]} scaleTo=
{0.99} accessibilityLabel={`${r.area}. ${r.where}`}>
            <Ionicons name={r.icon} size={22} color={colors.ink} style={{ marginTop: 1 }} />
            <View style={{ flex: 1, gap: 2 }}>
              <T variant="bodySm" weight="bold">
                {r.area}
              </T>
              <T variant="small" color={colors.textSecondary}>
                {r.where}
              </T>
            </View>
            <Ionicons name="arrow-forward" size={16} color={colors.ink} style={{ marginTop: 3 }} />
          </Press>
        ))}
      </Section>

      <Section title="Libraries used">
        <View style={styles.tags}>
          {LIBRARIES.map((l) => (
            <View key={l} style={styles.tag}>
              <T variant="caption" weight="semibold">
                {l}
              </T>
            </View>
          ))}
        </View>
      </Section>

      <Button
        label="Replay the app tour"
        icon="play-circle-outline"
        style={{ marginTop: space.section }}
        onPress={() => {
          setToured(false);
          router.navigate('/(tabs)/explore');
        }}
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'flex-start', gap: space.md, paddingVertical: space.md, borderTopWidth: hair, borderTopColor:
colors.border },
  first: { borderTopWidth: 0 },
  col: { flexDirection: 'column', alignItems: 'stretch', gap: 2 },
  press: { minHeight: 56 },
  layer: { flexDirection: 'row', alignItems: 'center', gap: space.md, padding: space.md, borderRadius: radius.card, borderWidth: line, b
orderColor: colors.ink, backgroundColor: colors.card },
  layerIcon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.b
gAlt, alignItems: 'center', justifyContent: 'center' },
  msg: { flexDirection: 'row', alignItems: 'center', gap: 6, marginTop: 6, paddingHorizontal: 10, paddingVertical: 6, borderRadius: radi
us.xs, borderWidth: 1.5, borderColor: colors.red, backgroundColor: colors.redSoft, alignSelf: 'flex-start', maxWidth: '100%' },
  tags: { flexDirection: 'row', flexWrap: 'wrap', gap: space.sm },
  tag: { paddingHorizontal: 10, paddingVertical: 6, borderRadius: radius.chip, borderWidth: 1.5, borderColor: colors.ink, backgroundColo
r: colors.card },
});
```

src/app/index.tsx

```
import { Redirect } from 'expo-router';

import { useAppStore } from '@/store/useAppStore';

/** Entry point: routes to onboarding, sign-in or the app depending on state. */
export default function Index() {
  const signedIn = useAppStore((s) => !!s.session);
  const hasOnboarded = useAppStore((s) => s.hasOnboarded);
  if (signedIn) return <Redirect href="/(tabs)/explore" />;
  return <Redirect href={hasOnboarded ? '/sign-in' : '/onboarding'} />;
}
```

src/app/join.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { Platform, StyleSheet, TextInput, View } from 'react-native';
```

```
import { QrScanner } from '@/components/join/QrScanner';
import { Button } from '@/components/ui/Button';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Plate } from '@/components/ui/Plate';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { joinTripAnywhere } from '@/sync/actions';
import { parseInviteUrl } from '@/sync/invite';
import { effectiveServerUrl, useSyncPrefs } from '@/sync/prefs';
import { normalizeServerUrl } from '@/sync/transport';
import { colors, fonts, line, radius, space } from '@/theme';
import { normalizeInvite, validateInvite } from '@/utils/validation';

/** Join a trip with an invite code (also opened by togetherwego://join?code=XXX-1234 links and by scanning the invite QR code). */
export default function Join() {
  const params = useLocalSearchParams<{ code?: string; server?: string }>();
  const [code, setCode] = useState(params.code ? normalizeInvite(String(params.code)) : '');
  // The sync server a friend's invite names. It is only used after this person agrees to it.
  const [inviteServer, setInviteServer] = useState<string | null>(params.server ? normalizeServerUrl(String(params.server)) : null);
  const [scanning, setScanning] = useState(false);
  const [focused, setFocused] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [busy, setBusy] = useState(false);
  useSyncPrefs((s) => s.serverUrl); // re-check when the address changes
  const newServer = inviteServer && inviteServer !== effectiveServerUrl() ? inviteServer : null;

  const join = async () => {
    setBusy(true);
    const r = await joinTripAnywhere(code);
    setBusy(false);
    if (!r.ok) {
      setError(r.error);
      return tap('warning');
    }
    tap('success');
    toast(r.synced ? 'You’re in. This trip now syncs live with your friends' : r.offlineCopy ? 'Joined the copy on this phone. The sync
server was not reachable.' : 'You’re in. Say hi in the trip chat');
    router.dismissAll?.();
    router.replace('/(tabs)/chat');
  };

  const submit = () => {
    if (busy) return;
    const e = validateInvite(code);
    if (e) {
      setError(e);
      return tap('warning');
    }
    if (!newServer) return void join();
    confirmAction(
      'Use your friend’s sync server?',
      `This invite shares the trip through ${newServer.replace(/^https?:\/\//, '')}. Its chat, plans and photos are kept on that compute
r, so only go on if you know who runs it.`,
      () => {
        useSyncPrefs.getState().setServerUrl(newServer);
        void join();
      },
      { confirmLabel: 'Use it and join' },
    );
  };

  const scanned = (text: string) => {
    setScanning(false);
    const found = parseInviteUrl(text);
    if (!found.code) return toast('That QR code is not a TogetherWeGo invite', { tone: 'warn', icon: 'qr-code-outline' });
    setCode(found.code);
    setError(null);
    if (found.server) setInviteServer(found.server);
    tap('success');
  };

  return (
    <Screen header={<Header kicker="Invitation" title="Join a trip" subtitle="Enter the code your friend shared" />} bottomInset={30}>
      <View style={styles.field}>
        <T variant="kicker" color={colors.ink} accessibilityRole="header">
          Invite code
        </T>
        <Plate offset={4} corner={radius.button} color={error ? colors.red : focused ? colors.accent : colors.plate}>
          <View style={[styles.codeBox, error && { backgroundColor: colors.redSoft, borderColor: colors.red }]}>
            <TextInput
              value={code}
              onChangeText={(v) => {
                setCode(normalizeInvite(v));
                setError(null);
```

```
            }}
            onFocus={() => setFocused(true)}
            onBlur={() => setFocused(false)}
            placeholder="ABC-1234"
            placeholderTextColor={colors.borderStrong}
            autoCapitalize="characters"
            autoCorrect={false}
            maxLength={8}
            style={styles.code}
            accessibilityLabel="Invite code"
            onSubmitEditing={submit}
            returnKeyType="go"
            testID="invite-code"
          />
        </View>
      </Plate>
      {error ? (
        <T variant="small" color={colors.red} accessibilityLiveRegion="assertive">
          {error}
        </T>
      ) : (
        <T variant="caption" color={colors.textMuted}>
          Three letters, a dash and four numbers. Ask the organizer to share it from the Crew page.
        </T>
      )}
      {newServer ? (
        <T variant="caption" color={colors.textSecondary}>
          This invite uses a sync server at {newServer.replace(/^https?:\/\//, '')}. You will be asked before it is used.
        </T>
      ) : null}
    </View>

    <View style={styles.actions}>
      <Button label="Join trip" icon="enter-outline" onPress={submit} loading={busy} testID="join-submit" />
      {Platform.OS !== 'web' ? <Button label="Scan a QR code" icon="qr-code-outline" variant="secondary" onPress={() => setScanning(tr
ue)} testID="join-scan" /> : null}
    </View>
    {Platform.OS !== 'web' ? <QrScanner visible={scanning} onClose={() => setScanning(false)} onScan={scanned} /> : null}

    <View style={styles.demo}>
      <T variant="kicker" color={colors.textSecondary}>
        No code yet? Try the demo
      </T>
      <Ticket
        plated={false}
        stub={
          <Press onPress={() => setCode('SEO-5521')} style={styles.stub} accessibilityLabel="Use demo code SEO-5521" scaleTo={0.99}>
            <View style={{ flex: 1 }}>
              <T variant="kicker" color={colors.textSecondary}>
                Demo code
              </T>
              <T variant="numeralSm">SEO-5521</T>
            </View>
            <T variant="kicker" color={colors.ink}>
              Fill it in
            </T>
            <Ionicons name="arrow-forward" size={16} color={colors.ink} />
          </Press>
        }>
        <View style={styles.ticketMain}>
          <T variant="h3">Seoul Food Crawl</T>
          <T variant="small" color={colors.textSecondary}>
            A trip that is waiting for new members.
          </T>
        </View>
      </Ticket>
    </View>
  </Screen>
);
}

const styles = StyleSheet.create({
  field: { gap: 8, marginTop: 4 },
  // the outline sits on a View around the input (an input is not a positioned box on the web, so a plate drawn behind it would paint ov
er it)
  codeBox: { borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, paddingHorizontal: 1
6 },
  code: {
    height: 72,
    fontFamily: fonts.mono,
    fontSize: 34,
    letterSpacing: 3,
    color: colors.text,
    ...(Platform.OS === 'web' ? ({ outlineStyle: 'none' } as object) : null),
  },
  actions: { gap: 14, marginTop: space.xxl },
```

```
  demo: { marginTop: space.section, gap: 10 },
  ticketMain: { padding: 16, gap: 4 },
  stub: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 56, paddingHorizontal: 16, paddingBottom: 10 },
});
```

src/app/map.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { router } from 'expo-router';
import { useMemo, useState } from 'react';
import { Platform, ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { OfflineRadar } from '@/components/map/OfflineRadar';
import { PinBadge } from '@/components/map/PinBadge';
import { TripMap } from '@/components/map/TripMap';
import type { MapPoint } from '@/components/map/types';
import { typeIcon } from '@/components/trip/meta';
import { NoTrip } from '@/components/trip/NoTrip';
import { Chip, IconButton } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Plate, PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Slider } from '@/components/ui/Slider';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { placesFor } from '@/data/places';
import { getLocation } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { MAX_WIDTH, colors, line, radius, space } from '@/theme';
import { durationLabel } from '@/utils/format';
import { distanceKm, kmLabel, nearestAirports } from '@/utils/geo';
import { dayLabel, time12, tripDays } from '@/utils/time';

const MODES = [
  { key: 'live', label: 'Map', icon: 'map-outline' },
  { key: 'offline', label: 'Offline', icon: 'cloud-offline-outline' },
] as const;

/**
 * The trip on a map. The chrome is paper: outlined squares for back and locate, a two-way switch between the street map and the offline radar,
 * chips for the days and the layers, and a bottom sheet with the picked pin and the search radius. Your plan is numbered 1, 2, 3 in the
 * order you will do it, places are small dots and airports carry a plane, so the kinds are told apart by shape, not by colour.
 */
export default function MapScreen() {
  const trip = useActiveTrip();
  const insets = useSafeAreaInsets();
  const [mode, setMode] = useState<'live' | 'offline'>('live');
  const [day, setDay] = useState<string>('all');
  const [layers, setLayers] = useState({ plan: true, places: true, airports: true });
  const [radiusKm, setRadiusKm] = useState(10);
  const [selected, setSelected] = useState<string | null>(null);
  const [me, setMe] = useState<{ lat: number; lng: number } | null>(null);
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const [center, setCenter] = useState({ lat: dest.lat, lng: dest.lng });

  const points = useMemo<MapPoint[]>(() => {
    if (!trip) return [];
    const out: MapPoint[] = [];
    if (layers.plan)
      trip.activities
        .filter((a) => a.lat && a.lng && (day === 'all' || a.date === day))
        .sort((a, b) => `${a.date} ${a.time}`.localeCompare(`${b.date} ${b.time}`))
        .forEach((a, i) => out.push({ id: a.id, lat: a.lat!, lng: a.lng!, title: a.title, subtitle: `${dayLabel(a.date)} · ${time12(a.time)} · ${durationLabel(a.durationMin)}`, color: colors.primary, kind: 'activity', label: String(i + 1), icon: typeIcon(a.type) }));
    if (layers.places) {
      const planned = new Set(trip.activities.map((a) => a.placeId));
      [trip.destinationId, ...trip.extraDestinationIds]
        .flatMap((id) => placesFor(id))
        .filter((p) => !planned.has(p.id))
        .forEach((p) => out.push({ id: p.id, lat: p.lat, lng: p.lng, title: p.name, subtitle: `${p.category} · ${p.rating.toFixed(1)} stars`, color: colors.textMuted, kind: 'place' }));
    }
    if (layers.airports) nearestAirports(center, 3).forEach((a) => out.push({ id: `air-${a.iata}`, lat: a.lat, lng: a.lng, title: `${a.iata} · ${a.name}`, subtitle: `${kmLabel(a.km)} away`, color: colors.blue, kind: 'airport', icon: 'airplane' }));
    if (me) out.push({ id: 'me', lat: me.lat, lng: me.lng, title: 'You are here', subtitle: '', color: colors.accent, kind: 'me' });
    return out;
  }, [trip, layers, day, center, me]);

  if (!trip) return <NoTrip />;
  const sel = points.find((p) => p.id === selected);
  const days = tripDays(trip.startDate, trip.endDate);

  const locate = async () => {
```

```
    const loc = await getLocation(dest);
    if (loc.source !== 'device') return toast('Location unavailable. Showing the trip centre', { tone: 'warn' });
    setMe(loc);
    if (distanceKm(loc, dest) < 80) setCenter(loc);
    else toast(`You're ${kmLabel(distanceKm(loc, dest))} from ${dest.city}`, { icon: 'navigate-outline' });
  };

  return (
    <View style={styles.root}>
      <View style={styles.mapWrap}>
        {mode === 'live' ? (
          <TripMap center={center} points={points} radiusKm={radiusKm} selectedId={selected} onSelect={setSelected} showsUser={!!me} />
        ) : (
          <OfflineRadar center={center} points={points} radiusKm={radiusKm} selectedId={selected} onSelect={setSelected} />
        )}
      </View>

      <View style={[styles.layer, { paddingTop: insets.top + 8, top: 0 }]} pointerEvents="box-none">
        <View style={styles.topRow}>
          <IconButton icon="arrow-back" label="Back" size={48} onPress={() => (router.canGoBack() ? router.back() : router.replace('/(ta
bs)/explore'))} />
          <Plate offset={3} corner={radius.button}>
            <View style={styles.seg}>
              {MODES.map((m) => {
                const on = mode === m.key;
                return (
                  <Press key={m.key} onPress={() => setMode(m.key)} style={[styles.segBtn, on && styles.segOn]} accessibilityLabel={m.la
bel} accessibilityState={{ selected: on }} scaleTo={0.97}>
                    <Ionicons name={m.icon} size={16} color={on ? colors.onPrimary : colors.ink} />
                    <T variant="small" weight="bold" color={on ? colors.onPrimary : colors.ink}>
                      {m.label}
                    </T>
                  </Press>
                );
              })}
            </View>
          </Plate>
          <IconButton icon="locate-outline" label="Show my location" size={48} onPress={locate} />
        </View>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} style={styles.chips} contentContainerStyle={styles.chipRow}>
          <Chip label="All days" active={day === 'all'} onPress={() => setDay('all')} />
          {days.map((d, i) => (
            <Chip key={d} label={`Day ${i + 1}`} active={day === d} onPress={() => setDay(d)} />
          ))}
        </ScrollView>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} style={styles.chips} contentContainerStyle={styles.chipRow}>
          <Chip icon="calendar-outline" label="Itinerary" active={layers.plan} onPress={() => setLayers((l) => ({ ...l, plan: !l.plan
}))} />
          <Chip icon="compass-outline" label="Places" active={layers.places} onPress={() => setLayers((l) => ({ ...l, places: !l.places
}))} />
          <Chip icon="airplane-outline" label="Airports" active={layers.airports} onPress={() => setLayers((l) => ({ ...l, airports: !l.
airports }))} />
        </ScrollView>
      </View>

      <View style={[styles.layer, { bottom: 0 }]} pointerEvents="box-none">
        <View style={[styles.sheet, { paddingBottom: insets.bottom + 14 }]}>
          {sel ? (
            <View style={styles.selRow}>
              <View style={styles.badge}>
                {sel.kind === 'activity' || sel.kind === 'place' ? <PinBadge p={sel} selected={false} /> : <Ionicons name={sel.kind ===
'airport' ? 'airplane' : 'locate'} size={22} color={colors.ink} />}
              </View>
              <View style={{ flex: 1 }}>
                <T variant="title" weight="bold" numberOfLines={1}>
                  {sel.title}
                </T>
                {sel.subtitle ? (
                  <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                    {sel.subtitle}
                  </T>
                ) : null}
                <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
                  {kmLabel(distanceKm(center, sel))} from centre
                </T>
              </View>
              <PlatePress onPress={() => Linking.openURL(`https://www.google.com/maps/dir/?api=1&destination=${sel.lat},${sel.lng}`)} ac
cessibilityLabel="Get directions" offset={3} corner={radius.button} color={colors.accent} style={styles.go}>
                <Ionicons name="navigate" size={20} color={colors.onPrimary} />
              </PlatePress>
            </View>
          ) : (
            <T variant="kicker" color={colors.textSecondary}>
              {points.length} pins · tap one for details{mode === 'offline' ? ' · works with no internet' : Platform.OS === 'web' ? ' ·
web preview uses OpenStreetMap' : ''}
            </T>
```

```
          )}
          <View>
            <View style={styles.radiusHead}>
              <T variant="kicker" color={colors.textSecondary}>
                Radius
              </T>
              <T variant="numeralSm">{radiusKm} km</T>
            </View>
            <Slider value={radiusKm} min={2} max={40} onChange={setRadiusKm} label="Map radius" formatValue={(v) => `${v} km`} />
          </View>
          {mode === 'live' && Platform.OS === 'web' && points.length ? (
            <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: space.sm }}>
              {points
                .filter((p) => p.kind !== 'place')
                .slice(0, 12)
                .map((p) => (
                  <Chip key={p.id} label={`${p.label ? `${p.label}. ` : ''}${p.title.slice(0, 22)}`} active={p.id === selected} onPress=
{() => setSelected(p.id)} />
                ))}
            </ScrollView>
          ) : null}
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg },
  mapWrap: { ...StyleSheet.absoluteFill },
  // each chrome layer is as wide as the screen, with its content kept to the phone column
  layer: { position: 'absolute', left: 0, right: 0, alignItems: 'center', gap: space.sm },
  topRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', width: '100%', maxWidth: MAX_WIDTH, paddingHori
zontal: 16 },
  // a centred column would size a row of chips to its content (wider than the screen); keep it as wide as the screen
  chips: { width: '100%', maxWidth: MAX_WIDTH, flexGrow: 0 },
  chipRow: { gap: space.sm, paddingHorizontal: 16, paddingRight: 24 },
  seg: { flexDirection: 'row', height: 48, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: col
ors.card, overflow: 'hidden' },
  segBtn: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, minWidth: 44, paddingHorizontal: 14 },
  segOn: { backgroundColor: colors.primary },
  sheet: { width: '100%', maxWidth: MAX_WIDTH, backgroundColor: colors.card, borderTopLeftRadius: radius.modal, borderTopRightRadius: ra
dius.modal, borderWidth: line, borderBottomWidth: 0, borderColor: colors.ink, paddingHorizontal: 16, paddingTop: 16, gap: space.md },
  selRow: { flexDirection: 'row', alignItems: 'center', gap: space.md },
  badge: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAl
t, alignItems: 'center', justifyContent: 'center' },
  go: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.primary,
alignItems: 'center', justifyContent: 'center' },
  radiusHead: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'baseline' },
});
```

src/app/nearby.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { useLocalSearchParams } from 'expo-router';
import { useMemo, useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { NearbyFilters, type NearbyFilterState } from '@/components/more/NearbyFilters';
import { AddActivitySheet, type ActivityDraft } from '@/components/trip/AddActivitySheet';
import { placeCategoryIcon, placeToActivityType } from '@/components/trip/meta';
import { Button } from '@/components/ui/Button';
import { EmptyState, IconButton } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { PlatePress } from '@/components/ui/Plate';
import { Header, Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { Skeleton } from '@/components/ui/Skeleton';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { placesFor, type Place } from '@/data/places';
import { useT } from '@/i18n';
import { getLocation } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, hair, line, radius, space } from '@/theme';
import { durationLabel, matchesLength, money, priceTier } from '@/utils/format';
import { bearingDeg, compassPoint, distanceKm, kmLabel, type LatLng } from '@/utils/geo';
import { todayInTz, tripDays } from '@/utils/time';

type Sort = 'distance' | 'rating' | 'price';

/** The Filter button: an outlined square-cornered button with a small accent count of the filters in use. */
function FilterButton({ count, onPress }: { count: number; onPress: () => void }) {
  return (
```

```
    <PlatePress onPress={onPress} accessibilityLabel={count ? `Filters, ${count} in use` : 'Filters'} offset={3} corner={radius.button}
style={styles.filterFace}>
      <Ionicons name="options-outline" size={18} color={colors.ink} />
      <T variant="small" weight="bold">
        Filter
      </T>
      {count ? (
        <View style={styles.count}>
          <T variant="kicker" color={colors.onAccent} style={{ fontSize: 11, lineHeight: 14 }}>
            {count}
          </T>
        </View>
      ) : null}
    </PlatePress>
  );
}

/** One place: the kind icon, the name, the distance in mono, a line about it, what it costs, and what you can do with it. */
function PlaceRow({ p, km, bearing, focus, planned, canAdd, onAdd }: { p: Place; km: number; bearing: string; focus: boolean; planned: b
oolean; canAdd: boolean; onAdd: () => void }) {
  const t = useT();
  return (
    <View style={[styles.row, focus && styles.focus]}>
      <View style={styles.head}>
        <View style={styles.icon}>
          <Ionicons name={placeCategoryIcon[p.category]} size={20} color={colors.ink} />
        </View>
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold">
            {p.name}
          </T>
          <T variant="caption" color={colors.textSecondary}>
            {p.area} · {p.rating.toFixed(1)} stars
          </T>
        </View>
        <View style={{ alignItems: 'flex-end' }} accessible accessibilityLabel={`${kmLabel(km)} ${bearing}`}>
          <T variant="numeralSm">{kmLabel(km)}</T>
          <T variant="kicker" color={colors.textSecondary}>
            {bearing}
          </T>
        </View>
      </View>
      <T variant="small" color={colors.textSecondary}>
        {p.blurb}
      </T>
      <View style={styles.meta}>
        <T variant="kicker" color={colors.textSecondary}>
          {t(`cat.${p.category}`)} · {durationLabel(p.durationMin)} · {p.cost ? `${priceTier(p.cost)} ~${money(p.cost)}` : 'Free'}
        </T>
        {planned ? <Stamp label="On plan" tone="positive" icon="checkmark" rotate={0} /> : null}
      </View>
      <View style={styles.actions}>
        {canAdd ? <Button label="Add to plan" icon="add" size="sm" variant="secondary" style={{ flex: 1 }} onPress={onAdd} /> : <View st
yle={{ flex: 1 }} />}
        <IconButton icon="navigate-outline" label={`Directions to ${p.name}`} onPress={() => Linking.openURL(`https://www.google.com/map
s/search/?api=1&query=${p.lat},${p.lng}`)} />
      </View>
    </View>
  );
}

/** Places around the middle of the trip's city (or around you), closest first. The many filters live behind one button. */
export default function Nearby() {
  const params = useLocalSearchParams<{ focus?: string; maxCost?: string }>();
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const defaultRadius = params.focus ? 25 : 6;
  const [center, setCenter] = useState<LatLng & { label: string }>({ lat: dest.lat, lng: dest.lng, label: `Central ${dest.city}` });
  const [f, setF] = useState<Omit<NearbyFilterState, 'fromMe'>>({ radiusKm: defaultRadius, cat: 'all', len: 'any', maxCost: params.maxCo
st ? Math.min(100, Number(params.maxCost)) : 100 });
  const [sort, setSort] = useState<Sort>(params.focus ? 'distance' : 'rating');
  const [locating, setLocating] = useState(false);
  const [filtersOpen, setFiltersOpen] = useState(false);
  const [draft, setDraft] = useState<ActivityDraft | null>(null);
  const fromMe = center.label === 'My location';

  const all = useMemo(() => (trip ? [trip.destinationId, ...trip.extraDestinationIds] : ['tokyo']).flatMap((id) => placesFor(id)), [tri
p]);
  const results = useMemo(
    () =>
      all
        .map((p) => ({ p, km: distanceKm(center, p) }))
        .filter(({ p, km }) => km <= f.radiusKm && (f.cat === 'all' || p.category === f.cat) && matchesLength(p.durationMin, f.len) &&
(f.maxCost >= 100 || p.cost <= f.maxCost))
        .sort((a, b) => (sort === 'distance' ? a.km - b.km : sort === 'rating' ? b.p.rating - a.p.rating : a.p.cost - b.p.cost)),
    [all, center, f, sort],
```

```
  );
  const planned = new Set(trip?.activities.map((a) => a.placeId).filter(Boolean));
  const inUse = [f.cat !== 'all', f.len !== 'any', f.maxCost < 100, fromMe, f.radiusKm !== defaultRadius].filter(Boolean).length;

  const toCentre = () => setCenter({ lat: dest.lat, lng: dest.lng, label: `Central ${dest.city}` });
  const locateMe = async () => {
    setLocating(true);
    const loc = await getLocation(dest);
    setLocating(false);
    if (loc.source === 'device') {
      const far = distanceKm(loc, dest) > 60;
      setCenter({ lat: loc.lat, lng: loc.lng, label: 'My location' });
      if (far) {
        setF((x) => ({ ...x, radiusKm: 25 }));
        toast(`You're ${kmLabel(distanceKm(loc, dest))} from ${dest.city}. Results will appear once you're there`, { tone: 'info', icon:
'navigate-outline' });
      } else toast('Using your current location');
    } else toast('Location unavailable. Using the trip centre', { tone: 'warn' });
  };
  const reset = () => {
    setF({ radiusKm: defaultRadius, cat: 'all', len: 'any', maxCost: 100 });
    toCentre();
  };

  const today = trip ? todayInTz(dest.tz) : '';
  const defaultDay = trip ? (tripDays(trip.startDate, trip.endDate).includes(today) ? today : trip.startDate) : '';

  return (
    <Screen header={<Header kicker="Around town" title="Nearby" subtitle={`${results.length} ${results.length === 1 ? 'place' : 'place
s'} · ${center.label}`} backLabel="More" />}>
      <Segmented
        value={sort}
        onChange={setSort}
        options={[
          { key: 'distance', label: 'Closest' },
          { key: 'rating', label: 'Top rated' },
          { key: 'price', label: 'Cheapest' },
        ]}
      />
      <View style={styles.bar}>
        <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
          Within {f.radiusKm} km
        </T>
        <FilterButton count={inUse} onPress={() => setFiltersOpen(true)} />
      </View>

      {locating ? (
        <View style={{ gap: space.md }} accessibilityLabel="Finding places near you">
          {[0, 1, 2].map((i) => (
            <Skeleton key={i} height={168} />
          ))}
        </View>
      ) : results.length ? (
        <View>
          {results.map(({ p, km }) => (
            <PlaceRow
              key={p.id}
              p={p}
              km={km}
              bearing={compassPoint(bearingDeg(center, p))}
              focus={params.focus === p.id}
              planned={planned.has(p.id)}
              canAdd={!!trip}
              onAdd={() => setDraft({ title: p.name, location: `${p.area}, ${destinationById(p.destinationId)?.city ?? ''}`, type: place
ToActivityType[p.category], durationMin: p.durationMin, costPerPerson: p.cost, lat: p.lat, lng: p.lng, placeId: p.id, notes: p.blurb })}
            />
          ))}
        </View>
      ) : (
        <EmptyState icon="compass-outline" title="Nothing matches yet" body="Widen the radius, raise the max price or pick another kind
of place.">
          <Button label="Reset filters" onPress={reset} />
        </EmptyState>
      )}

      <NearbyFilters
        visible={filtersOpen}
        onClose={() => setFiltersOpen(false)}
        state={{ ...f, fromMe }}
        onChange={(patch) => setF((x) => ({ ...x, ...patch }))}
        cityName={dest.city}
        locating={locating}
        onLocate={locateMe}
        onCentre={toCentre}
        onReset={reset}
        count={results.length}
```

```
      />
      {trip ? <AddActivitySheet visible={!!draft} onClose={() => setDraft(null)} trip={trip} date={defaultDay} draft={draft} /> : null}
    </Screen>
  );
}

const styles = StyleSheet.create({
  bar: { flexDirection: 'row', alignItems: 'center', gap: space.md, marginTop: space.lg, marginBottom: space.sm },
  filterFace: { flexDirection: 'row', alignItems: 'center', gap: 8, height: 44, paddingHorizontal: 14, borderRadius: radius.button, bord
erWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
  count: { minWidth: 22, height: 22, paddingHorizontal: 4, borderRadius: radius.xs, backgroundColor: colors.accent, borderWidth: 1.5, bo
rderColor: colors.ink, alignItems: 'center', justifyContent: 'center' },
  row: { gap: space.md, paddingVertical: space.lg, borderTopWidth: hair, borderTopColor: colors.border },
  focus: { backgroundColor: colors.accentSoft, borderWidth: line, borderColor: colors.ink, borderRadius: radius.card, padding: space.md,
marginVertical: space.sm },
  head: { flexDirection: 'row', alignItems: 'flex-start', gap: space.md },
  icon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card,
alignItems: 'center', justifyContent: 'center' },
  meta: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', columnGap: space.sm, rowGap: 6 },
  actions: { flexDirection: 'row', alignItems: 'center', gap: space.md },
});
```

src/app/new-trip.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router, useLocalSearchParams } from 'expo-router';
import { useMemo, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { DatePickerSheet } from '@/components/pickers';
import { Button } from '@/components/ui/Button';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById, destinations } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useDraft } from '@/hooks/useDraft';
import { useAppStore } from '@/store/useAppStore';
import { colors, line, radius, space } from '@/theme';
import { money } from '@/utils/format';
import { addDays, dayLabel, daysBetween, todayInTz, tzAbbrev, utcOffsetLabel } from '@/utils/time';
import { firstError, maxLen, parseAmount, required, validateAmount, validateTripDates } from '@/utils/validation';

/** Create a trip (or edit one with ?edit=<tripId>). A half-filled form is kept if the person leaves and comes back. */
export default function NewTrip() {
  const params = useLocalSearchParams<{ edit?: string; destination?: string }>();
  const editing = useAppStore((s) => (params.edit ? s.trips.find((t) => t.id === params.edit) : undefined));
  const createTrip = useAppStore((s) => s.createTrip);
  const updateTrip = useAppStore((s) => s.updateTrip);
  const today = todayInTz('UTC');

  const initialDest = editing?.destinationId ?? params.destination ?? 'tokyo';
  const key = editing ? `edit-trip-${editing.id}` : 'new-trip';
  const [destId, setDestId] = useState(initialDest);
  const [name, setName, clearName] = useDraft(`${key}-name`, editing?.name ?? '');
  const [start, setStart, clearStart] = useDraft(`${key}-start`, editing?.startDate ?? addDays(today, 30));
  const [end, setEnd, clearEnd] = useDraft(`${key}-end`, editing?.endDate ?? addDays(today, 36));
  const [budget, setBudget, clearBudget] = useDraft(`${key}-budget`, editing ? String(editing.budget) : '');
  const [q, setQ] = useState('');
  const [picker, setPicker] = useState<'start' | 'end' | null>(null);
  const [errors, setErrors] = useState<Record<string, string | null | undefined>>({});

  const dest = destinationById(destId)!;
  const days = Math.max(1, daysBetween(start, end) + 1);
  const suggested = Math.round(dest.dailyCost * days * 4 + 900 * 4);
  const filtered = useMemo(() => destinations.filter((d) => !q || `${d.city} ${d.country}`.toLowerCase().includes(q.toLowerCase())),
[q]);

  const save = () => {
    const dateErr = validateTripDates(start, end);
    const e = {
      name: firstError(name, required('a trip name'), maxLen(40, 'Trip name')),
      budget: validateAmount(budget, { min: 50, max: 1_000_000, label: 'a group budget' }),
      start: dateErr.start ?? (!editing && daysBetween(today, start) < -1 ? 'Start date can’t be in the past' : null),
      end: dateErr.end,
    };
    setErrors(e);
    if (Object.values(e).some(Boolean)) return tap('warning');
    const b = parseAmount(budget)!;
    if (editing) {
      updateTrip(editing.id, { name: name.trim(), startDate: start, endDate: end, budget: b, destinationId: destId, cover: destId !== ed
iting.destinationId ? dest.image : editing.cover });
      toast('Trip updated');
```

```
      [clearName, clearStart, clearEnd, clearBudget].forEach((c) => c());
      router.back();
    } else {
      createTrip({ name, destinationId: destId, startDate: start, endDate: end, budget: b });
      tap('success');
      toast('Trip created. Share the invite code with your group');
      [clearName, clearStart, clearEnd, clearBudget].forEach((c) => c());
      router.dismissAll?.();
      router.replace('/crew');
    }
  };

  return (
    <Screen header={<Header kicker="Group trip" title={editing ? 'Edit trip' : 'New trip'} subtitle={editing ? editing.name : 'Where is
the group headed?'} />} bottomInset={30}>
      <View style={{ gap: space.xxl }}>
        <View style={{ gap: 10 }}>
          <T variant="kicker" color={colors.ink}>
            Destination
          </T>
          <Input placeholder="Search destinations…" icon="search" value={q} onChangeText={setQ} accessibilityLabel="Search destinations"
/>
          {filtered.length ? (
            <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 12, paddingHorizontal: 20, paddi
ngBottom: 4 }} style={{ marginHorizontal: -20 }}>
              {filtered.map((d) => {
                const on = d.id === destId;
                return (
                  <Press key={d.id} onPress={() => setDestId(d.id)} scaleTo={0.97} style={[styles.dest, on && { backgroundColor: colors.
accent }]} accessibilityState={{ selected: on }} accessibilityLabel={`${d.city}, ${d.country}`}>
                    <Image source={imageSource(d.image)} style={styles.destImg} contentFit="cover" />
                    {on ? (
                      <View style={styles.check}>
                        <Ionicons name="checkmark" size={16} color={colors.onAccent} />
                      </View>
                    ) : null}
                    <View style={styles.destText}>
                      <T variant="caption" weight="bold" color={on ? colors.onAccent : colors.text} numberOfLines={1}>
                        {d.city}
                      </T>
                      <T variant="micro" color={on ? colors.onAccent : colors.textSecondary} numberOfLines={1}>
                        {d.country}
                      </T>
                    </View>
                  </Press>
                );
              })}
            </ScrollView>
          ) : (
            <T variant="small" color={colors.textSecondary}>
              No destination matches “{q}”.
            </T>
          )}
        </View>

        <Input label="Trip name" placeholder={`e.g. ${dest.city} with the crew`} value={name} onChangeText={setName} error={errors.name}
maxLength={45} />

        <View style={{ gap: 10 }}>
          <View style={{ flexDirection: 'row', gap: 12 }}>
            <Input label="Start date" value={dayLabel(start) + `, ${start.slice(0, 4)}`} onPressField={() => setPicker('start')} error=
{errors.start} containerStyle={{ flex: 1 }} />
            <Input label="End date" value={dayLabel(end) + `, ${end.slice(0, 4)}`} onPressField={() => setPicker('end')} error={errors.e
nd} containerStyle={{ flex: 1 }} />
          </View>
          <T variant="kicker" color={colors.textSecondary}>
            {days} {days === 1 ? 'day' : 'days'} / {dest.city} time, {tzAbbrev(dest.tz)} {utcOffsetLabel(dest.tz)}
          </T>
        </View>

        <View style={{ gap: 10 }}>
          <Input label="Group budget (USD)" placeholder={String(suggested)} value={budget} onChangeText={setBudget} keyboardType="decima
l-pad" error={errors.budget} hint={`Tip: about ${money(dest.dailyCost)} a person a day in ${dest.city}. For 4 people including flights,
around ${money(suggested)}.`} />
          {!budget ? <Button label={`Use ${money(suggested)}`} variant="secondary" size="md" full={false} style={{ alignSelf: 'flex-star
t' }} onPress={() => setBudget(String(suggested))} /> : null}
        </View>

        {!editing ? (
          <View style={styles.note}>
            <Ionicons name="key-outline" size={18} color={colors.ink} />
            <T variant="caption" color={colors.textSecondary} style={{ flex: 1 }}>
              You’ll get an invite code like <T variant="caption" weight="bold">{dest.city.slice(0, 3).toUpperCase()}-1234</T> to share,
so friends can join in a tap.
            </T>
          </View>
```

```
        ) : null}

        <Button label={editing ? 'Save changes' : 'Create trip'} icon={editing ? 'checkmark' : 'airplane-outline'} onPress={save} />
      </View>

      <DatePickerSheet visible={picker === 'start'} value={start} title="Start date" onClose={() => setPicker(null)} onChange={(v) => {
        setStart(v);
        if (daysBetween(v, end) < 0) setEnd(addDays(v, 5));
      }} />
      <DatePickerSheet visible={picker === 'end'} value={end} min={start} max={addDays(start, 60)} title="End date" onClose={() => setPi
cker(null)} onChange={setEnd} />
    </Screen>
  );
}

const styles = StyleSheet.create({
  dest: { width: 120, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, overflow: 'hi
dden' },
  destImg: { width: '100%', height: 84, borderBottomWidth: line, borderBottomColor: colors.ink },
  destText: { paddingHorizontal: 10, paddingVertical: 8, minHeight: 50 },
  check: { position: 'absolute', top: 8, right: 8, width: 26, height: 26, borderRadius: radius.xs, borderWidth: line, borderColor: color
s.ink, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
  note: { flexDirection: 'row', alignItems: 'center', gap: 12 },
});
```

src/app/notifications.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, type Href } from 'expo-router';
import { useEffect } from 'react';
import { StyleSheet, View } from 'react-native';

import { groupByDay } from '@/components/more/groups';
import { Button } from '@/components/ui/Button';
import { EmptyState, SectionHeader, type IconName } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { sendTestReminder } from '@/services/notifications';
import type { AppNotification } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';
import { colors, hair, line, radius, space } from '@/theme';
import { timeAgo } from '@/utils/format';

/** One line icon per kind of notification. Colour is kept for states, so every kind is ink; unread is the one that is lit. */
const ICONS: Record<AppNotification['kind'], IconName> = {
  reminder: 'alarm-outline',
  chat: 'chatbubble-ellipses-outline',
  vote: 'stats-chart-outline',
  expense: 'wallet-outline',
  system: 'information-circle-outline',
  member: 'person-add-outline',
  photo: 'images-outline',
};

function testReminder() {
  sendTestReminder(5);
  toast('Test reminder scheduled. Lock your phone to see it arrive', { icon: 'alarm-outline' });
}

/** Reminders, votes and chat mentions, grouped Today and Earlier. Unread ones have an accent icon. */
export default function Notifications() {
  const list = useAppStore((s) => s.notifications);
  const markAllRead = useAppStore((s) => s.markAllRead);
  const clear = useAppStore((s) => s.clearNotifications);
  const setActive = useAppStore((s) => s.setActiveTrip);

  // Mark everything read when leaving the screen.
  useEffect(() => () => markAllRead(), [markAllRead]);

  const unread = list.filter((n) => !n.read).length;
  const groups = groupByDay(list);

  return (
    <Screen header={<Header kicker="TogetherWeGo" title="Notifications" subtitle={unread ? `${unread} unread` : 'Nothing new'} backLabel
="Home" />}>
      {list.length ? (
        <>
          <View style={styles.bar}>
            <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
              {list.length} {list.length === 1 ? 'notification' : 'notifications'}
            </T>
            <Button
              label="Mark all read"
              icon="checkmark-done-outline"
```

```
              size="sm"
              variant="secondary"
              full={false}
              disabled={!unread}
              onPress={() => {
                markAllRead();
                toast('All marked as read', { icon: 'checkmark-done-outline' });
              }}
            />
          </View>

          {groups.map((g) => (
            <View key={g.label} style={{ marginTop: space.xl }}>
              <SectionHeader title={g.label} />
              {g.items.map((n, i) => (
                <Press
                  key={n.id}
                  onPress={() => {
                    if (n.tripId) setActive(n.tripId);
                    if (n.route) router.push((n.route === '/(tabs)/profile' ? '/crew' : n.route) as Href);
                  }}
                  style={[styles.item, i === 0 && styles.first]}
                  scaleTo={0.99}
                  accessibilityLabel={`${n.read ? '' : 'Unread. '}${n.title}. ${n.body}`}>
                  <View style={[styles.icon, !n.read && { backgroundColor: colors.accent }]}>
                    <Ionicons name={ICONS[n.kind]} size={20} color={n.read ? colors.ink : colors.onAccent} />
                  </View>
                  <View style={{ flex: 1, gap: 2 }}>
                    <T variant="bodySm" weight={n.read ? 'semibold' : 'bold'}>
                      {n.title}
                    </T>
                    <T variant="small" color={colors.textSecondary}>
                      {n.body}
                    </T>
                    <T variant="kicker" color={colors.textSecondary} style={{ marginTop: 2 }}>
                      {timeAgo(n.ts)}
                    </T>
                  </View>
                </Press>
              ))}
            </View>
          ))}

          <View style={styles.foot}>
            <Button label="Send a test reminder (5 sec)" icon="alarm-outline" variant="secondary" size="md" onPress={testReminder} />
            <Button
              label="Clear all notifications"
              icon="trash-outline"
              variant="danger"
              size="md"
              onPress={() => confirmAction('Clear all notifications?', 'This removes every notification from this list.', clear, { confi
rmLabel: 'Clear all', destructive: true })}
              />
            </View>
          </>
        ) : (
          <EmptyState icon="notifications-off-outline" title="You’re all caught up" body="Reminders, votes and chat mentions will show up
here.">
            <Button label="Send a test reminder (5 sec)" icon="alarm-outline" variant="secondary" onPress={testReminder} />
          </EmptyState>
        )}
      </Screen>
    );
}

const styles = StyleSheet.create({
  bar: { flexDirection: 'row', alignItems: 'center', gap: space.md },
  item: { flexDirection: 'row', alignItems: 'flex-start', gap: space.md, paddingVertical: space.md, minHeight: 64, borderTopWidth: hair,
borderTopColor: colors.border },
  first: { borderTopWidth: 0 },
  icon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card,
alignItems: 'center', justifyContent: 'center' },
  foot: { gap: space.xxxl, marginTop: space.section },
});
```

src/app/onboarding.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useRef, useState } from 'react';
import { Animated, FlatList, StyleSheet, View, useWindowDimensions, type NativeScrollEvent, type NativeSyntheticEvent } from 'react-nati
ve';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
```

```
import { StepSegments } from '@/components/auth/kit';
import { SLIDES, type Slide } from '@/components/auth/slides';
import { LogoMark } from '@/components/Logo';
import { Button } from '@/components/ui/Button';
import { FadeIn, useLoop } from '@/components/ui/motion';
import { PlatePress } from '@/components/ui/Plate';
import { Stamp } from '@/components/ui/Stamp';
import { StaggerText } from '@/components/ui/StaggerText';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { useT } from '@/i18n';
import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, line, radius } from '@/theme';

/** How much taller than the screen the photo is drawn, so it can be pulled up (Slide.lift) without showing a gap. */
const EXTRA = 0.2;

/** One full-bleed photo that zooms very slowly (the zoom stops when animations are off). */
function Photo({ slide, width, height, zoom }: { slide: Slide; width: number; height: number; zoom: Animated.Value }) {
  return (
    <View style={{ width, height, overflow: 'hidden', backgroundColor: colors.ink }}>
      <Animated.View
        style={{ position: 'absolute', left: 0, right: 0, top: -slide.lift * height, height: height * (1 + EXTRA), transform: [{ scale:
zoom.interpolate({ inputRange: [0, 1], outputRange: [1, 1.08] }) }] }}>
        <Image source={slide.image} style={StyleSheet.absoluteFill} contentFit="cover" transition={300} accessibilityIgnoresInvertColors
/>
      </Animated.View>
      {/* a soft scrim behind the controls at the top, so they stay readable on a bright sky */}
      <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0.28)', 'rgba(0,0,0,0)']} style={{ position: 'absolute', left: 0, right:
0, top: 0, height: 140 }} />
    </View>
  );
}

/**
 * First run: three full-bleed photos, one idea each, and a boarding pass that slides up over the lower part with the headline (it slide
s
 * in word by word), a step bar and one big black button in the thumb zone. Skip is a quiet outlined square at the top right.
 */
export default function Onboarding() {
  const t = useT();
  const { width: winW, height } = useWindowDimensions();
  const width = Math.min(winW, MAX_WIDTH);
  const insets = useSafeAreaInsets();
  const [index, setIndex] = useState(0);
  const list = useRef<FlatList<Slide>>(null);
  const complete = useAppStore((s) => s.completeOnboarding);
  const zoom = useLoop(18000, { pingPong: true, rest: 0 });
  const last = SLIDES.length - 1;
  const slide = SLIDES[index];

  const finish = () => {
    complete();
    router.replace('/sign-in');
  };
  const next = () => {
    if (index >= last) return finish();
    list.current?.scrollToIndex({ index: index + 1, animated: true });
    setIndex(index + 1);
  };
  const onScroll = (e: NativeSyntheticEvent<NativeScrollEvent>) => {
    const i = Math.min(last, Math.max(0, Math.round(e.nativeEvent.contentOffset.x / width)));
    if (i !== index) setIndex(i);
  };
  const narrow = width < 350;
  const titleSize = narrow ? { fontSize: 30, lineHeight: 34 } : { fontSize: 36, lineHeight: 40 };

  return (
    <View style={styles.root}>
      <View style={{ width, height, alignSelf: 'center' }}>
        <FlatList
          ref={list}
          data={SLIDES}
          horizontal
          pagingEnabled
          bounces={false}
          showsHorizontalScrollIndicator={false}
          onMomentumScrollEnd={onScroll}
          onScroll={onScroll}
          scrollEventThrottle={32}
          getItemLayout={(_, i) => ({ length: width, offset: width * i, index: i })}
          keyExtractor={(s) => s.key}
          renderItem={({ item }) => <Photo slide={item} width={width} height={height} zoom={zoom} />}
        />

        <View style={[styles.top, { top: insets.top + 12 }]}>
```

```
          <FadeIn from="down" distance={12}>
            <View style={styles.brand}>
              <LogoMark size={28} />
              <T variant="kicker" color={colors.ink}>
                TogetherWeGo
              </T>
            </View>
          </FadeIn>
          {index < last ? (
            <PlatePress onPress={finish} accessibilityLabel={t('common.skip')} offset={3} corner={radius.button} style={styles.skip}>
              <T variant="small" weight="bold" color={colors.ink}>
                {t('common.skip')}
              </T>
              <Ionicons name="arrow-forward" size={16} color={colors.ink} />
            </PlatePress>
          ) : null}
        </View>

        <View style={[styles.bottom, { paddingBottom: insets.bottom + 20 }]} pointerEvents="box-none">
          <FadeIn from="up" distance={80} duration={600} delay={250}>
            <Ticket
              stub={
                <View style={styles.stub}>
                  <StepSegments now={index + 1} total={SLIDES.length} />
                  <Button label={index === last ? t('common.getStarted') : t('common.next')} onPress={next} iconRight="arrow-forward" te
stID="onboarding-next" />
                </View>
              }>
              <View style={styles.main}>
                <View style={styles.kickRow}>
                  <T variant="kicker" color={colors.textSecondary}>
                    {`0${index + 1} / 0${SLIDES.length}`}
                  </T>
                  <Stamp key={slide.key} label={slide.stamp} tone="accent" slam />
                </View>
                <StaggerText key={slide.key} text={slide.title} variant="display" style={titleSize} />
                <FadeIn key={`b${slide.key}`} delay={280} distance={10}>
                  <T variant="small" color={colors.textSecondary}>
                    {slide.body}
                  </T>
                </FadeIn>
              </View>
            </Ticket>
          </FadeIn>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg },
  top: { position: 'absolute', left: 16, right: 16, flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  brand: { flexDirection: 'row', alignItems: 'center', gap: 8, height: 44, paddingHorizontal: 8, borderRadius: radius.button, borderWidt
h: line, borderColor: colors.ink, backgroundColor: colors.card },
  skip: { flexDirection: 'row', alignItems: 'center', gap: 6, minWidth: 44, height: 44, paddingHorizontal: 12, borderRadius: radius.butt
on, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, justifyContent: 'center' },
  bottom: { position: 'absolute', left: 20, right: 20, bottom: 0 },
  main: { padding: 16, gap: 10, minHeight: 168 },
  kickRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between' },
  stub: { paddingHorizontal: 16, paddingBottom: 12, paddingTop: 2, gap: 14 },
});
```

src/app/packing.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { PackRow } from '@/components/play/PackingParts';
import { completesList, needsSomeone as findNeeds, splitFairly } from '@/components/play/packingLogic';
import { MemberPickerSheet } from '@/components/trip/MemberPickerSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Chip, Divider, EmptyState, ProgressBar } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { PlatePress } from '@/components/ui/Plate';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { packingBase } from '@/data/reference';
```

```
import { useWeather } from '@/hooks/useTrip';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';

type Filter = 'all' | 'mine' | 'open';

const COUNTRY_EXTRAS: Record<string, string[]> = {
  JP: ['IC transit card (Suica / ICOCA)', 'Type A plug adapter', 'Coin purse — lots of cash & coins', 'Small towel (many restrooms lack
dryers)', 'Slip-on shoes for temples & ryokan'],
  FR: ['Type E plug adapter', 'Reusable shopping bag'],
  IT: ['Type L/F plug adapter', 'Scarf to cover shoulders in churches'],
  TH: ['Mosquito repellent', 'Light clothes covering knees for temples'],
  ID: ['Reef-safe sunscreen', 'Mosquito repellent'],
};

/** The shared packing list: how much is packed, who is bringing what, and every item in a section of its own. */
export default function Packing() {
  const trip = useActiveTrip();
  const me = useAppStore((s) => s.session?.userId);
  const toggle = useAppStore((s) => s.togglePacked);
  const assign = useAppStore((s) => s.assignPacking);
  const addItems = useAppStore((s) => s.addPackingItems);
  const del = useAppStore((s) => s.deletePackingItem);
  const markFired = useAppStore((s) => s.markReminderFired);
  const weather = useWeather(trip?.destinationId);
  const [text, setText] = useState('');
  const [cat, setCat] = useState('Extras');
  const [err, setErr] = useState<string | null>(null);
  const [filter, setFilter] = useState<Filter>('all');
  const [pickId, setPickId] = useState<string | null>(null);
  if (!trip) return <NoTrip />;

  const dest = destinationById(trip.destinationId);
  const total = trip.packing.length;
  const packed = trip.packing.filter((p) => p.packed).length;
  const memberIds = trip.members.map((m) => m.id);
  const bringer = (broughtBy?: string) => trip.members.find((m) => m.id === broughtBy);
  const needsSomeone = findNeeds(trip.packing, memberIds);
  const shown = trip.packing.filter((p) => (filter === 'mine' ? p.broughtBy === me : filter === 'open' ? !p.packed && !bringer(p.brought
By) : true));
  const groups = Array.from(new Set(shown.map((p) => p.category)));
  const have = new Set(trip.packing.map((p) => p.text.toLowerCase()));
  const picking = trip.packing.find((p) => p.id === pickId);
  const allPacked = total > 0 && packed === total;
  const suggestions = [
    ...(COUNTRY_EXTRAS[dest?.countryCode ?? ''] ?? []),
    ...(weather?.daily.some((d) => d.rain >= 50) ? ['Compact umbrella'] : []),
    ...(weather && Math.max(...weather.daily.map((d) => d.max)) >= 27 ? ['Sunglasses & hat'] : []),
    ...(weather && Math.min(...weather.daily.map((d) => d.min)) <= 10 ? ['Warm jacket'] : []),
  ].filter((s) => !have.has(s.toLowerCase()));

  const add = () => {
    const t = text.trim();
    if (!t) return setErr('Type an item to add');
    if (t.length > 50) return setErr('Keep items under 50 characters');
    if (have.has(t.toLowerCase())) return setErr('That’s already on the list');
    addItems(trip.id, [{ text: t, category: cat }]);
    setText('');
    setErr(null);
    tap('success');
  };

  // Packing the very last thing is the moment worth a celebration, once per trip (so un-ticking and ticking again stays quiet).
  const tick = (id: string, wasPacked: boolean) => {
    toggle(trip.id, id);
    const key = `packed:${trip.id}`;
    if (completesList(packed, total, wasPacked) && !(key in useAppStore.getState().firedReminders)) {
      markFired(key);
      celebrate('Everything is packed');
    }
  };

  const split = () => {
    const plan = splitFairly(trip.packing, memberIds);
    if (!plan.length) return;
    plan.forEach((p) => assign(trip.id, p.itemId, p.memberId));
    tap('success');
    toast(`Assigned ${plan.length} item${plan.length === 1 ? '' : 's'}`);
  };

  return (
    <Screen header={<Header title="Packing list" subtitle={`Shared with ${trip.members.length} travellers`} />}>
      {total ? (
        <View>
          <View style={styles.progress} accessible accessibilityLabel={`${packed} of ${total} packed`}>
            <View style={styles.count}>
```

```
              <T variant="title" importantForAccessibility="no">
                <T variant="numeral">{packed}</T> of <T variant="numeral">{total}</T> packed
              </T>
              {allPacked ? <Stamp label="All packed" tone="positive" icon="checkmark" rotate={-3} slam /> : null}
            </View>
            <ProgressBar value={packed / total} color={allPacked ? colors.positive : colors.accent} />
          </View>
          <View style={styles.crew}>
            {trip.members.map((m) => (
              <View key={m.id} style={styles.load} accessible accessibilityLabel={`${m.name.split(' ')[0]} is bringing ${trip.packing.fi
lter((p) => p.broughtBy === m.id).length}`}>
                <Avatar name={m.name} src={m.avatar} size={28} />
                <T variant="caption" weight="bold" importantForAccessibility="no">
                  {m.name.split(' ')[0]} · {trip.packing.filter((p) => p.broughtBy === m.id).length}
                </T>
              </View>
            ))}
            {needsSomeone.length ? (
              <View style={styles.load} accessible accessibilityLabel={`${needsSomeone.length} still need someone`}>
                <T variant="caption" weight="bold" color={colors.textSecondary} importantForAccessibility="no">
                  Anyone · {needsSomeone.length}
                </T>
              </View>
            ) : null}
          </View>
          <View style={{ marginTop: 16 }}>
            <Button label="Split the rest fairly" icon="shuffle-outline" size="md" variant="secondary" disabled={!needsSomeone.length} o
nPress={split} />
          </View>

          <View style={{ marginTop: 24 }}>
            <Segmented
              value={filter}
              onChange={setFilter}
              options={[
                { key: 'all', label: 'All' },
                { key: 'mine', label: 'Mine' },
                { key: 'open', label: 'Unassigned' },
              ]}
            />
          </View>
        </View>
      ) : (
        <EmptyState icon="bag-handle-outline" title="Nothing to pack yet" body="Start with the essentials, then add your own. The whole
crew shares this list.">
          <Button label="Start with the essentials" icon="list-outline" onPress={() => addItems(trip.id, packingBase.flatMap((g) => g.it
ems.map((text) => ({ text, category: g.category }))))} />
        </EmptyState>
      )}

      {groups.map((g) => {
        const rows = shown.filter((p) => p.category === g);
        return (
          <View key={g} style={{ marginTop: 28 }}>
            <View style={styles.groupHead}>
              <T variant="kicker" color={colors.ink} accessibilityRole="header">
                {g}
              </T>
              <T variant="kicker" color={colors.textSecondary}>
                {rows.filter((p) => p.packed).length} / {rows.length}
              </T>
            </View>
            <Card padded={false} style={{ overflow: 'hidden' }}>
              {rows.map((p, i) => (
                <View key={p.id}>
                  {i > 0 ? <Divider /> : null}
                  <PackRow item={p} who={bringer(p.broughtBy)} me={me} onToggle={() => tick(p.id, p.packed)} onWho={() => setPickId(p.i
d)} onRemove={() => del(trip.id, p.id)} />
                </View>
              ))}
            </Card>
          </View>
        );
      })}

      {total && !shown.length ? (
        <T variant="small" color={colors.textSecondary} center style={{ marginTop: 28 }}>
          {filter === 'mine' ? 'Nothing is assigned to you yet. Tap the person icon on an item to pick it up.' : 'Every item has someone
bringing it.'}
        </T>
      ) : null}

      {total ? (
        <View style={styles.addBlock}>
          {suggestions.length ? (
            <Card style={styles.tip}>
```

```
              <View style={styles.tipHead}>
                <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
                  Suggested for {dest?.city}
                </T>
                <Press
                  onPress={() => {
                    addItems(trip.id, suggestions.map((s) => ({ text: s, category: 'Suggested' })));
                    toast(`Added ${suggestions.length} items`);
                  }}
                  style={styles.addAll}
                  accessibilityLabel="Add all suggestions">
                  <T variant="kicker" color={colors.ink}>
                    Add all
                  </T>
                  <Ionicons name="add" size={16} color={colors.ink} />
                </Press>
              </View>
              <View style={styles.wrap}>
                {suggestions.map((s) => (
                  <Chip key={s} icon="add" label={s} onPress={() => addItems(trip.id, [{ text: s, category: 'Suggested' }])} />
                ))}
              </View>
            </Card>
          ) : null}
          <View style={styles.add}>
            <Input
              containerStyle={{ flex: 1 }}
              label="Add an item"
              accessibilityLabel="New packing item"
              placeholder="Add an item…"
              value={text}
              onChangeText={(v) => {
                setText(v);
                setErr(null);
              }}
              error={err}
              onSubmitEditing={add}
              returnKeyType="done"
            />
            <PlatePress onPress={add} accessibilityLabel="Add item" corner={radius.button} color={colors.accent} wrapperStyle={{ marginT
op: 21 }} style={styles.addBtn}>
              <Ionicons name="add" size={26} color={colors.onPrimary} />
            </PlatePress>
          </View>
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 8, paddingHorizontal: 20 }} style=
{{ marginHorizontal: -20, marginTop: 12 }}>
            {[...packingBase.map((g) => g.category), 'Weather', 'Suggested'].map((c) => (
              <Chip key={c} label={c} active={cat === c} onPress={() => setCat(c)} />
            ))}
          </ScrollView>
        </View>
      ) : null}

      <MemberPickerSheet
        visible={!!picking}
        onClose={() => setPickId(null)}
        members={trip.members}
        selected={picking?.broughtBy ? [picking.broughtBy] : []}
        onToggle={(id) => picking && assign(trip.id, picking.id, picking.broughtBy === id ? undefined : id)}
        title={picking ? `Who's bringing ${picking.text}?` : 'Who's bringing it?'}
        single
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  progress: { gap: 12 },
  count: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12, minHeight: 40 },
  crew: { flexDirection: 'row', flexWrap: 'wrap', columnGap: 16, rowGap: 8, marginTop: 16 },
  load: { flexDirection: 'row', alignItems: 'center', gap: 6 },
  tip: { marginBottom: 24, gap: 12 },
  tipHead: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 44 },
  addAll: { flexDirection: 'row', alignItems: 'center', gap: 2, minHeight: 44, paddingLeft: 10 },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  groupHead: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginBottom: 8 },
  addBlock: { marginTop: 32 },
  add: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
  addBtn: { width: 52, height: 52, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.primary, borderWidth: line, b
orderColor: colors.ink, borderRadius: radius.button },
});
```

src/app/panorama.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { LinearGradient } from 'expo-linear-gradient';
```

```
import { router, useFocusEffect, useLocalSearchParams } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { Animated, Easing, ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { StatusBar } from 'expo-status-bar';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { Pick, Readout, Square, Switch } from '@/components/panorama/Chrome';
import { headingOf, K, layout, PanoramaScene, posForHeading } from '@/components/panorama/Scene';
import { timeOfDayAt } from '@/components/SkylineHero';
import { toast } from '@/components/ui/feedback';
import { nativeDriver, useMotion } from '@/components/ui/motion';
import { tap } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { useNow } from '@/hooks/useTrip';
import { PANORAMAS, panoById } from '@/data/panoramas';
import { MAX_WIDTH, colors, line, radius } from '@/theme';

const WIND = ['N', 'NNE', 'NE', 'ENE', 'E', 'ESE', 'SE', 'SSE', 'S', 'SSW', 'SW', 'WSW', 'W', 'WNW', 'NW', 'NNW'];
const wind = (deg: number) => WIND[Math.round(deg / 22.5) % 16];
/** Smallest signed angle from a to b, in degrees. */
const angleDiff = (a: number, b: number) => ((((b - a) % 360) + 540) % 360) - 180;

export default function PanoramaScreen() {
  const { city, focus } = useLocalSearchParams<{ city?: string; focus?: string }>();
  const { width, height } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const S = Math.min(width, MAX_WIDTH);
  const H = height;
  const now = useNow(60000);

  const [cityId, setCityId] = useState(panoById(city)?.id ?? PANORAMAS[0].id);
  const pano = panoById(cityId) ?? PANORAMAS[0];
  const startLm = pano.landmarks.find((l) => l.id === focus) ?? pano.landmarks[0];
  const [forced, setForced] = useState<'day' | 'night' | null>(null);
  const [auto, setAuto] = useState(true);
  const motion = useMotion(); // off when "Animations" is off or the phone asks apps to reduce motion: no auto-rotation, no coasting, no
glides
  const autoOn = auto && motion;
  const [heading, setHeading] = useState(startLm.heading);
  const mode = forced ?? timeOfDayAt(pano.tz, now);

  const [pos] = useState(() => new Animated.Value(posForHeading(startLm.heading, S)));
  const [night] = useState(() => new Animated.Value(mode === 'night' ? 1 : 0));
  const posRef = useRef(posForHeading(startLm.heading, S));
  const velRef = useRef(0);
  const dragRef = useRef(false);
  const busyRef = useRef(false);
  const idleRef = useRef(1000);
  const autoRef = useRef(true);
  const motionRef = useRef(true);
  const gestureRef = useRef({ x: 0, pos: 0, last: 0 });

  // Keep the mirrors of state that the animation loop and gesture handlers read.
  useEffect(() => {
    motionRef.current = motion;
    autoRef.current = autoOn;
  }, [autoOn, motion]);
  useEffect(() => {
    const id = pos.addListener(({ value }) => {
      posRef.current = value;
    });
    return () => pos.removeListener(id);
  }, [pos]);

  // Day ↔ night cross-fade. `night` and `pos` are mixed with the scene's looping lights (which run on the phone's native driver),
  // and one value cannot be driven both ways: a JS-driven animation on it crashes ("moved to native earlier"). Use `nativeDriver` for b
oth.
  useEffect(() => {
    const anim = Animated.timing(night, { toValue: mode === 'night' ? 1 : 0, duration: motion ? 1200 : 0, easing: Easing.inOut(Easing.cu
bic), useNativeDriver: nativeDriver });
    anim.start();
    return () => anim.stop();
  }, [mode, night, motion]);

  // One ~30 fps loop: coasting after a flick, gentle auto-rotation when idle, and the compass readout.
  // It only runs while this screen is the one being looked at, so it costs nothing behind another screen.
  useFocusEffect(
    useCallback(() => {
      let tick = 0;
      const id = setInterval(() => {
        tick++;
        if (!dragRef.current && !busyRef.current) {
          velRef.current = motionRef.current ? velRef.current * 0.94 : 0;
          idleRef.current++;
          const idle = idleRef.current > 105;
          const step = velRef.current + (autoRef.current && idle && Math.abs(velRef.current) < 0.2 ? 0.45 : 0);
```

```
          if (Math.abs(step) > 0.01) pos.setValue(posRef.current + step);
        }
        if (tick % 4 === 0) setHeading(Math.round(headingOf(posRef.current, S)) % 360);
      }, 33);
      return () => clearInterval(id);
    }, [pos, S]),
  );

  const glide = (delta: number) => {
    idleRef.current = 0;
    velRef.current = 0;
    if (!motion) {
      pos.setValue(posRef.current + delta); // no sweeping animation when motion is off
      return;
    }
    busyRef.current = true;
    Animated.timing(pos, { toValue: posRef.current + delta, duration: 800, easing: Easing.out(Easing.cubic), useNativeDriver: nativeDriv
er }).start(() => {
      busyRef.current = false;
    });
  };
  const nudge = (dir: 1 | -1) => {
    tap();
    glide((dir * S * 0.45) / K.mid);
  };
  const lookAt = (deg: number) => {
    tap();
    const period = layout(S).far;
    const mid = layout(S).mid;
    const want = (angleDiff(headingOf(posRef.current, S), deg) / 360) * mid;
    glide((want / K.mid) % period);
  };
  const pick = (id: string) => {
    tap();
    const p = panoById(id);
    if (!p) return;
    setCityId(p.id);
    setForced(null);
    const h = p.landmarks[0].heading;
    setHeading(h);
    const target = posForHeading(h, S);
    posRef.current = target;
    velRef.current = 0;
    pos.setValue(target);
  };

  const near = pano.landmarks.find((l) => Math.abs(angleDiff(heading, l.heading)) < 13);
  const top = insets.top + 10;

  return (
    <View style={styles.root}>
      <StatusBar style="light" />
      <View
        style={{ width: S, height: H, overflow: 'hidden', backgroundColor: '#05070F' }}
        onStartShouldSetResponder={() => true}
        onMoveShouldSetResponder={() => true}
        onResponderGrant={(e) => {
          dragRef.current = true;
          busyRef.current = false;
          idleRef.current = 0;
          velRef.current = 0;
          gestureRef.current = { x: e.nativeEvent.pageX, pos: posRef.current, last: e.nativeEvent.pageX };
        }}
        onResponderMove={(e) => {
          const g = gestureRef.current;
          const x = e.nativeEvent.pageX;
          idleRef.current = 0;
          velRef.current = -(x - g.last) / K.mid;
          g.last = x;
          pos.setValue(g.pos - (x - g.x) / K.mid);
        }}
        onResponderRelease={() => {
          dragRef.current = false;
        }}
        onResponderTerminate={() => {
          dragRef.current = false;
        }}>
        <PanoramaScene key={pano.id} pano={pano} night={night} pos={pos} S={S} H={H} />
        <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0.62)', 'rgba(0,0,0,0)']} style={[styles.fade, { top: 0, height: 270
}]} />
        <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.72)']} style={[styles.fade, { bottom: 0, height: 36
0 }]} />

        {/* Top bar: outlined squares on the scene */}
        <View style={[styles.top, { top }]}>
          <Square icon="arrow-back" label="Go back" onPress={() => (router.canGoBack() ? router.back() : router.replace('/(tabs)/explor
```

```
e'))} />
          <View style={{ flex: 1 }} />
          <Switch
            role="switch"
            on={autoOn}
            icon={autoOn ? 'sync' : 'pause'}
            text={autoOn ? 'Auto' : 'Paused'}
            label="Auto rotate"
            onPress={() => (motion ? setAuto((a) => !a) : toast('Animations are off, so the view only moves when you drag it', { icon:
'accessibility-outline' }))}
          />
          <Switch
            icon={mode === 'night' ? 'moon' : 'sunny'}
            text={mode === 'night' ? 'Night' : 'Day'}
            label={mode === 'night' ? 'Night view. Switch to day' : 'Day view. Switch to night'}
            hint="Long press to follow local time"
            onPress={() => {
              tap();
              setForced(mode === 'night' ? 'day' : 'night');
            }}
            onLongPress={() => setForced(null)}
          />
        </View>

        {/* Title + compass */}
        <View style={[styles.titleBlock, { top: top + 62 }]} pointerEvents="none">
          <T variant="kicker" color={colors.white}>
            {pano.country}
          </T>
          <T variant="display" color={colors.white} style={styles.titleText} accessibilityRole="header">
            {pano.name}
          </T>
          <View style={{ marginTop: 6 }}>
            <Readout icon="compass-outline">
              360° · {wind(heading)} {heading}°
            </Readout>
          </View>
        </View>

        {/* Arrows */}
        <Square icon="chevron-back" label="Look left" onPress={() => nudge(-1)} style={[styles.arrow, { left: 10 }]} />
        <Square icon="chevron-forward" label="Look right" onPress={() => nudge(1)} style={[styles.arrow, { right: 10 }]} />

        {/* Bottom: callout, landmark jumps, city picker */}
        <View style={[styles.bottom, { paddingBottom: insets.bottom + 12 }]}>
          <View style={styles.callout} accessibilityLiveRegion="polite">
            {near ? (
              <>
                <T variant="title" weight="bold">
                  {near.name}
                </T>
                <T variant="small" color={colors.textSecondary}>
                  {near.fact}
                </T>
              </>
            ) : (
              <View style={{ flexDirection: 'row', alignItems: 'center', gap: 10 }}>
                <Ionicons name="hand-left-outline" size={20} color={colors.ink} />
                <T variant="small" weight="semibold" style={{ flex: 1 }}>
                  Drag to look around. It wraps all the way back to where you started.
                </T>
              </View>
            )}
          </View>
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.row} style={{ flexGrow: 0 }}>
            {pano.landmarks.map((l) => (
              <Pick key={l.id} text={l.name} icon="locate" label={`Look at ${l.name}`} on={near?.id === l.id} onPress={() => lookAt(l.he
ading)} />
            ))}
          </ScrollView>
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.row} style={{ flexGrow: 0 }}>
            {PANORAMAS.map((p) => (
              <Pick key={p.id} text={p.name} label={`${p.name} 360 degree view`} on={p.id === pano.id} onPress={() => pick(p.id)} />
            ))}
          </ScrollView>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: '#000', alignItems: 'center', overflow: 'hidden' },
  fade: { position: 'absolute', left: 0, right: 0 },
  top: { position: 'absolute', left: 14, right: 14, flexDirection: 'row', alignItems: 'center', gap: 10 },
  titleBlock: { position: 'absolute', left: 20, right: 20, gap: 2 },
```

```
  titleText: { fontSize: 52, lineHeight: 56 },
  arrow: { position: 'absolute', top: '46%' },
  bottom: { position: 'absolute', left: 0, right: 0, bottom: 0, gap: 12 },
  callout: { marginHorizontal: 16, padding: 14, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor:
colors.card, gap: 4, minHeight: 64, justifyContent: 'center' },
  row: { gap: 10, paddingHorizontal: 16, paddingRight: 24, paddingBottom: 4 },
});
```

src/app/personalize.tsx

```
import { router } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import { useMemo, useState } from 'react';
import { KeyboardAvoidingView, Platform, ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { Building } from '@/components/personalize/Building';
import { commitLook, currentLook } from '@/components/personalize/applyLook';
import { PalButton, PalIconButton, Tx } from '@/components/personalize/kit';
import { modeAfterPickingTheme } from '@/components/personalize/look';
import { STEP, STEPS } from '@/components/personalize/options';
import { paletteOf } from '@/components/personalize/palette';
import { PaletteVeil, StepBar } from '@/components/personalize/parts';
import { StepInterests } from '@/components/personalize/steps/StepInterests';
import { StepLayout } from '@/components/personalize/steps/StepLayout';
import { StepLook } from '@/components/personalize/steps/StepLook';
import { StepReveal } from '@/components/personalize/steps/StepReveal';
import { StepVibe } from '@/components/personalize/steps/StepVibe';
import { StepWelcome } from '@/components/personalize/steps/StepWelcome';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { FadeIn } from '@/components/ui/motion';
import { Press } from '@/components/ui/Press';
import { useDraft } from '@/hooks/useDraft';
import { ensurePermission, setTriviaReminder } from '@/services/notifications';
import type { Personal } from '@/store/types';
import { useAppStore, useCurrentUser } from '@/store/useAppStore';
import { MAX_WIDTH, line, type CornerStyle, type ModePref, type ThemeKey } from '@/theme';
import { luminance } from '@/utils/contrast';

/** Everything the person is choosing in this flow; kept for as long as the app is open so closing the screen by accident loses nothing.
*/
interface Draft {
  p: Personal;
  theme: ThemeKey;
  mode: ModePref;
  corners: CornerStyle;
}

const NO_PERMISSION = 'Allow notifications in your phone settings to get the reminder';

/**
 * "Make it yours": Welcome, Look, Interests, Layout, Vibe, then a boarding-pass summary and a short "Building your trip HQ" reveal.
 * One question per screen, every step skippable, and the whole screen repaints in the look being picked (it draws itself
 * with a palette object, not the app's own colours). Each step is a component in components/personalize/steps.
 */
export default function Personalize() {
  const insets = useSafeAreaInsets();
  const user = useCurrentUser();
  const saved = useAppStore((s) => s.personal);
  const updatePersonal = useAppStore((s) => s.updatePersonal);

  const [step, setStep] = useState(0);
  // True after a row on the summary was tapped: Continue and Back then return straight to the summary.
  const [fromSummary, setFromSummary] = useState(false);
  const [forward, setForward] = useState(true);
  const [building, setBuilding] = useState(false);

  // A new key whenever the saved answers change elsewhere (Settings), so an old draft never overrides a newer choice.
  const [draft, setDraft, clearDraft] = useDraft<Draft>(`personalize:${JSON.stringify(saved)}`, {
    p: { ...saved, sections: { ...saved.sections }, order: [...saved.order], interests: [...saved.interests] },
    ...currentLook(),
  });
  const { p, theme, mode, corners } = draft;

  const set = (patch: Partial<Personal>) => setDraft((d) => ({ ...d, p: { ...d.p, ...patch } }));
  const pickTheme = (k: ThemeKey) => setDraft((d) => ({ ...d, theme: k, mode: modeAfterPickingTheme(k, d.mode) }));
  const pickMode = (m: ModePref) => setDraft((d) => ({ ...d, mode: m }));

  const pal = useMemo(() => paletteOf(theme, mode), [theme, mode]);
  const firstName = user?.name.split(' ')[0] ?? 'traveller';
  const shownName = p.nickname.trim() || firstName;
  const onSummary = step === STEP.summary;
  const closing = step === 0 && !fromSummary;

  const exit = () => (router.canGoBack() ? router.back() : router.replace('/(tabs)/explore'));
```

```
  const toSummary = () => {
    setFromSummary(false);
    setForward(true);
    setStep(STEP.summary);
  };
  const edit = (to: number) => {
    setFromSummary(true);
    setForward(false);
    setStep(to);
  };
  const next = () => {
    if (fromSummary) return toSummary();
    setForward(true);
    setStep((s) => Math.min(STEP.summary, s + 1));
  };
  const back = () => {
    if (fromSummary) return toSummary();
    if (step === 0) return exit();
    setForward(false);
    setStep((s) => s - 1);
  };

  const onTrivia = async (on: boolean) => {
    set({ triviaReminder: on });
    if (!on || Platform.OS === 'web') return;
    if (!(await ensurePermission())) {
      set({ triviaReminder: false });
      toast(NO_PERMISSION, { tone: 'warn', icon: 'notifications-off-outline' });
    }
  };

  // The last button: save every answer, then run the short reveal (which puts the look on when it is done).
  const build = () => {
    const mine: Personal = { ...p, nickname: p.nickname.trim(), done: true };
    updatePersonal(mine);
    void setTriviaReminder(mine.triviaReminder).then((ok) => {
      if (ok) return;
      updatePersonal({ triviaReminder: false });
      toast(NO_PERMISSION, { tone: 'warn', icon: 'notifications-off-outline' });
    });
    setBuilding(true);
  };

  // After the reveal: put the look on. If it changes what is on screen the app restarts (see commitLook and the root layout), otherwise
go to Home.
  const finish = () => {
    void commitLook({ theme, mode, corners }).then((result) => {
      if (result === 'reloading') return;
      clearDraft();
      if (result === 'failed') {
        setBuilding(false);
        toast('Saved. Close and reopen the app to see your new look.', { icon: 'refresh' });
        router.replace('/(tabs)/explore');
        return;
      }
      router.replace('/(tabs)/explore');
      celebrate('All set. Change it anytime in Settings');
    });
  };

  const label = onSummary ? 'Apply my style' : fromSummary ? 'Back to summary' : step === 0 ? 'Let’s go' : 'Continue';

  return (
    <View style={[styles.root, { backgroundColor: pal.bg, paddingTop: insets.top }]}>
      {/* the phone's clock and battery follow the look being previewed, not the one the app was started in */}
      <StatusBar style={luminance(pal.bg) < 0.5 ? 'light' : 'dark'} />
      <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={styles.column}>
        <View style={styles.top}>
          <PalIconButton pal={pal} icon={closing ? 'close' : 'arrow-back'} label={closing ? 'Close' : 'Back'} onPress={back} />
          <View style={{ flex: 1 }}>
            <StepBar pal={pal} step={step} total={STEPS.length} />
          </View>
          {!onSummary ? (
            <Press onPress={toSummary} accessibilityLabel="Skip to the end" style={styles.skip}>
              <Tx pal={pal} variant="kicker">
                Skip
              </Tx>
            </Press>
          ) : (
            <View style={styles.skip} />
          )}
        </View>

        <ScrollView key={step} style={{ width: '100%' }} contentContainerStyle={styles.content} keyboardShouldPersistTaps="handled" show
sVerticalScrollIndicator={false}>
          <FadeIn from={forward ? 'left' : 'right'} distance={24} duration={260}>
```

```
            {step === STEP.welcome ? <StepWelcome pal={pal} p={p} set={set} firstName={firstName} onSubmit={next} /> : null}
            {step === STEP.look ? <StepLook pal={pal} theme={theme} mode={mode} onTheme={pickTheme} onMode={pickMode} /> : null}
            {step === STEP.interests ? <StepInterests pal={pal} p={p} set={set} /> : null}
            {step === STEP.layout ? <StepLayout pal={pal} p={p} set={set} /> : null}
            {step === STEP.vibe ? <StepVibe pal={pal} p={p} set={set} onTrivia={onTrivia} /> : null}
            {onSummary ? <StepReveal pal={pal} p={p} theme={theme} mode={mode} shownName={shownName} onEdit={edit} /> : null}
          </FadeIn>
        </ScrollView>

        <View style={[styles.footer, { paddingBottom: insets.bottom + 12, backgroundColor: pal.bg, borderTopColor: pal.ink }]}>
          <PalButton pal={pal} label={label} onPress={onSummary ? build : next} iconRight={onSummary ? undefined : 'arrow-forward'} />
        </View>
      </KeyboardAvoidingView>

      <PaletteVeil pal={pal} signature={`${theme}-${mode}`} />
      {building ? <Building pal={pal} onDone={finish} /> : null}
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, alignItems: 'center', overflow: 'hidden' },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  top: { flexDirection: 'row', alignItems: 'center', gap: 14, paddingHorizontal: 20, paddingTop: 8, paddingBottom: 12 },
  skip: { minWidth: 44, height: 44, alignItems: 'center', justifyContent: 'center' },
  content: { paddingHorizontal: 20, paddingTop: 12, paddingBottom: 32 },
  footer: { paddingHorizontal: 20, paddingTop: 14, borderTopWidth: line },
});
```

src/app/photos.tsx

```
import { Image } from 'expo-image';
import { useMemo, useState } from 'react';
import { Modal, Platform, ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { PhotoTile } from '@/components/photos/PhotoTile';
import { PhotoViewer } from '@/components/photos/PhotoViewer';
import { ShareSheet } from '@/components/ShareSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Chip, EmptyState } from '@/components/ui/bits';
import { Fab } from '@/components/ui/Fab';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { destinationById } from '@/data/destinations';
import { FileTooLargeError, pickImage } from '@/services/files';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Photo } from '@/store/types';
import { MAX_WIDTH, colors, line, radius } from '@/theme';
import { todayInTz, tripDays } from '@/utils/time';
import { maxLen } from '@/utils/validation';

type Filter = 'all' | 'mine' | 'liked' | string;

const GAP = 6;

/** The photo dump: every picture the crew took, in a tight grid of outlined prints. Tap one to open it, like it or share it. */
export default function Photos() {
  const trip = useActiveTrip();
  const { width } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addPhoto = useAppStore((s) => s.addPhoto);
  const like = useAppStore((s) => s.toggleLikePhoto);
  const remove = useAppStore((s) => s.deletePhoto);
  const [filter, setFilter] = useState<Filter>('all');
  const [viewing, setViewing] = useState<Photo | null>(null);
  const [pending, setPending] = useState<string | null>(null);
  const [caption, setCaption] = useState('');
  const [captionErr, setCaptionErr] = useState<string | null>(null);
  const [shareFor, setShareFor] = useState<Photo | null>(null);
  const [source, setSource] = useState(false);

  const list = useMemo(() => {
    if (!trip) return [];
    const p = trip.photos.slice();
    if (filter === 'mine') return p.filter((x) => x.by === userId);
    if (filter === 'liked') return p.sort((a, b) => b.likes.length - a.likes.length);
    if (filter !== 'all') return p.filter((x) => x.day === filter);
    return p;
  }, [trip, filter, userId]);
```

```
  if (!trip) return <NoTrip />;
  const colW = Math.min(width, MAX_WIDTH) - 40;
  const tile = Math.floor((colW - GAP * 2) / 3);
  const live = viewing ? trip.photos.find((p) => p.id === viewing.id) ?? null : null;
  const days = tripDays(trip.startDate, trip.endDate);

  const add = async (camera: boolean) => {
    setSource(false);
    try {
      const f = await pickImage(camera);
      if (f) {
        setPending(f.uri);
        setCaption('');
      }
    } catch (e) {
      toast(e instanceof FileTooLargeError ? e.message : e instanceof Error ? e.message : 'Couldn’t open photos', { tone: 'warn' });
    }
  };
  // On a phone the camera is one tap away; the web has only the file picker.
  const onAdd = () => (Platform.OS === 'web' ? add(false) : setSource(true));

  const today = todayInTz(destinationById(trip.destinationId)?.tz ?? 'UTC');
  const empty = trip.photos.length === 0;

  return (
    <Screen
      header={<Header title="Photo dump" subtitle={`${trip.photos.length} photos · ${trip.name}`} />}
      fab={empty ? undefined : <Fab icon="images-outline" label="Add photos" onPress={onAdd} />}
      fabBottom={insets.bottom + 16}>
      {empty ? (
        <EmptyState icon="images-outline" title="No photos yet" body="Add your best shots. Everyone on the trip can like and share the
m.">
          <Button label="Add photos" icon="images-outline" onPress={onAdd} />
        </EmptyState>
      ) : (
        <>
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={{ marginHorizontal:
-20, marginBottom: 20 }}>
            <Chip label="All" active={filter === 'all'} onPress={() => setFilter('all')} />
            <Chip label="Most liked" icon="heart-outline" active={filter === 'liked'} onPress={() => setFilter('liked')} />
            <Chip label="Mine" active={filter === 'mine'} onPress={() => setFilter('mine')} />
            {days.map((d, i) => (
              <Chip key={d} label={`Day ${i + 1}`} active={filter === d} onPress={() => setFilter(d)} />
            ))}
          </ScrollView>
          {list.length ? (
            <View style={styles.grid}>
              {list.map((p) => (
                <PhotoTile key={p.id} photo={p} size={tile} who={trip.members.find((x) => x.id === p.by)?.name.split(' ')[0] ?? 'a frien
d'} onPress={() => setViewing(p)} />
              ))}
            </View>
          ) : (
            <EmptyState icon="images-outline" title="Nothing here yet" body="No photos match this filter.">
              <Button label="Show all photos" variant="secondary" onPress={() => setFilter('all')} />
            </EmptyState>
          )}
        </>
      )}

      <Sheet visible={source} onClose={() => setSource(false)} title="Add a photo">
        <View style={{ gap: 12, paddingBottom: 8 }}>
          <Button label="Photo library" icon="images-outline" onPress={() => add(false)} />
          <Button label="Camera" icon="camera-outline" variant="secondary" onPress={() => add(true)} />
        </View>
      </Sheet>

      <Sheet
        visible={!!pending}
        onClose={() => setPending(null)}
        title="Add a caption"
        footer={
          <Button
            label="Post to photo dump"
            icon="cloud-upload-outline"
            onPress={() => {
              const e = maxLen(80, 'Caption')(caption);
              setCaptionErr(e);
              if (e || !pending) return;
              addPhoto(trip.id, { uri: pending, caption: caption.trim(), day: days.includes(today) ? today : trip.startDate });
              setPending(null);
              tap('success');
              toast('Photo added');
            }}
          />
        }>
```

```
        {pending ? <Image source={{ uri: pending }} style={styles.preview} contentFit="cover" accessibilityIgnoresInvertColors /> : nul
l}
        <Input placeholder="Say something about it…" value={caption} onChangeText={setCaption} error={captionErr} maxLength={90} />
      </Sheet>

      <Modal visible={!!live} transparent animationType="fade" onRequestClose={() => setViewing(null)}>
        {live ? (
          <PhotoViewer
            photo={live}
            trip={trip}
            userId={userId}
            onClose={() => setViewing(null)}
            onShare={() => setShareFor(live)}
            onLike={() => like(trip.id, live.id)}
            onDelete={() => {
              remove(trip.id, live.id);
              setViewing(null);
            }}
          />
        ) : null}
        <ShareSheet
          visible={!!shareFor}
          onClose={() => setShareFor(null)}
          title="Share this memory"
          message={`${shareFor?.caption ? `${shareFor.caption}. ` : ''}From our trip “${trip.name}”, planned with TogetherWeGo.`}
        />
      </Modal>
    </Screen>
  );
}

const styles = StyleSheet.create({
  chips: { gap: 8, paddingHorizontal: 20 },
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: GAP },
  preview: { width: '100%', height: 220, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink },
});
```

src/app/phrasebook.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Speech from 'expo-speech';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { Card, Chip, EmptyState } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { phrasebook } from '@/data/reference';
import { copy } from '@/services/share';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, line, radius, space } from '@/theme';

/** Offline phrasebook with native text-to-speech pronunciation. */
export default function Phrasebook() {
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo');
  const initial = dest && phrasebook[dest.language] ? dest.language : 'Japanese';
  const [lang, setLang] = useState(initial);
  const [q, setQ] = useState('');
  const [speaking, setSpeaking] = useState<string | null>(null);
  const book = phrasebook[lang];
  const list = book.phrases.filter((p) => !q || `${p.en} ${p.local} ${p.roman ?? ''}`.toLowerCase().includes(q.toLowerCase()));

  const say = (text: string) => {
    Speech.stop();
    setSpeaking(text);
    Speech.speak(text, {
      language: book.speechCode,
      rate: 0.85,
      onDone: () => setSpeaking(null),
      onStopped: () => setSpeaking(null),
      onError: () => {
        setSpeaking(null);
        toast(`Install the ${book.lang} voice in your phone settings to hear this`, { tone: 'warn' });
      },
    });
  };

  return (
    <Screen header={<Header kicker="Around town" title="Phrasebook" subtitle={`${book.phrases.length} ${book.lang} phrases. Tap the spea
ker to hear one.`} backLabel="More" />}>
```

```
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
        {Object.keys(phrasebook).map((l) => (
          <Chip key={l} label={l} active={l === lang} onPress={() => setLang(l)} />
        ))}
      </ScrollView>
      <Input icon="search" value={q} onChangeText={setQ} placeholder="Search phrases…" accessibilityLabel="Search phrases" returnKeyType
="search" autoCorrect={false} containerStyle={{ marginVertical: space.lg }} />
      {list.length ? (
        <View style={{ gap: space.md }}>
          {list.map((p) => {
            const on = speaking === p.local;
            return (
              <Card key={p.en} style={styles.card}>
                <View style={{ flex: 1, gap: 4 }}>
                  <T variant="small" color={colors.textSecondary}>
                    {p.en}
                  </T>
                  <T variant="h2" selectable>
                    {p.local}
                  </T>
                  {p.roman ? (
                    <T variant="small" color={colors.accentText} style={{ fontStyle: 'italic' }}>
                      {p.roman}
                    </T>
                  ) : null}
                </View>
                <View style={styles.buttons}>
                  <Press onPress={() => copy(p.local, 'Phrase copied')} style={styles.copy} accessibilityLabel={`Copy ${p.en}`}>
                    <Ionicons name="copy-outline" size={20} color={colors.ink} />
                  </Press>
                  <PlatePress onPress={() => say(p.local)} accessibilityLabel={`Hear ${p.en} in ${book.lang}`} offset={3} corner={radiu
s.button} style={[styles.speak, on && { backgroundColor: colors.accent }]}>
                    <Ionicons name={on ? 'volume-high' : 'volume-high-outline'} size={26} color={on ? colors.onAccent : colors.ink} />
                  </PlatePress>
                </View>
              </Card>
            );
          })}
        </View>
      ) : (
        <EmptyState icon="language-outline" title="No phrases match" body="Try a different word, or clear the search." />
      )}
    </Screen>
  );
}

const styles = StyleSheet.create({
  chipScroll: { marginHorizontal: -20, flexGrow: 0 },
  chips: { gap: space.sm, paddingHorizontal: 20, paddingRight: 28 },
  card: { flexDirection: 'row', alignItems: 'center', gap: space.md },
  buttons: { flexDirection: 'row', alignItems: 'center', gap: space.xs },
  speak: { width: 56, height: 56, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card,
alignItems: 'center', justifyContent: 'center' },
  copy: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/quiz.tsx

```
import { Image } from 'expo-image';
import { useEffect, useMemo, useState } from 'react';
import { StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { ordinal } from '@/components/home/logic';
import { ShareSheet } from '@/components/ShareSheet';
import { AnswerOption, Clock, Podium, StepBar, type OptionState } from '@/components/play/QuizParts';
import { ROUND, SECONDS, crewBoard, isNewBest, pointsFor, resultRows, roundTitle, shouldCelebrate, shuffle } from '@/components/play/qui
zLogic';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Divider, SectionHeader } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { tap } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { Header, Screen } from '@/components/ui/Screen';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { photos } from '@/data/images';
import { quizQuestions, type QuizQuestion } from '@/data/reference';
import { useActiveTrip, useAppStore, useCurrentUser } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';

type Phase = 'intro' | 'play' | 'done';

export default function Quiz() {
  const best = useAppStore((s) => s.quizBest);
```

```
const setBest = useAppStore((s) => s.setQuizBest);
const trip = useActiveTrip();
const user = useCurrentUser();
const insets = useSafeAreaInsets();
const [phase, setPhase] = useState<Phase>('intro');
const [qs, setQs] = useState<QuizQuestion[]>([]);
const [i, setI] = useState(0);
const [picked, setPicked] = useState<number | null>(null);
const [score, setScore] = useState(0);
const [gained, setGained] = useState(0);
const [correct, setCorrect] = useState(0);
const [streak, setStreak] = useState(0);
const [bestStreak, setBestStreak] = useState(0);
const [bestAtStart, setBestAtStart] = useState(0);
const [left, setLeft] = useState(SECONDS);
const [share, setShare] = useState(false);

const start = () => {
  setQs(shuffle(quizQuestions).slice(0, ROUND));
  setI(0);
  setScore(0);
  setGained(0);
  setCorrect(0);
  setStreak(0);
  setBestStreak(0);
  setBestAtStart(best);
  setPicked(null);
  setLeft(SECONDS);
  setPhase('play');
};

// Running out of time counts as answer -1 (derived, so no extra state to keep in step).
const answer = picked ?? (phase === 'play' && left <= 0 ? -1 : null);
const ticking = phase === 'play' && answer === null;

useEffect(() => {
  if (!ticking) return;
  const id = setInterval(() => setLeft((l) => Math.max(0, l - 1)), 1000);
  return () => clearInterval(id);
}, [ticking, i]);

const choose = (k: number) => {
  if (answer !== null) return;
  setPicked(k);
  if (k === qs[i].answer) {
    const pts = pointsFor(left, streak);
    setGained(pts);
    setScore((s) => s + pts);
    setCorrect((c) => c + 1);
    setStreak(streak + 1);
    setBestStreak((b) => Math.max(b, streak + 1));
    tap('success');
  } else {
    setGained(0);
    setStreak(0);
    tap('warning');
  }
};

const next = () => {
  if (answer === -1) {
    setGained(0);
    setStreak(0);
  }
  if (i + 1 >= qs.length) {
    setPhase('done');
    setBest(score);
    if (shouldCelebrate(correct, qs.length, score, bestAtStart)) celebrate();
    return;
  }
  setI(i + 1);
  setPicked(null);
  setGained(0);
  setLeft(SECONDS);
};

const me = useMemo(() => ({ id: user?.id, name: user?.name, avatar: user?.avatar }), [user]);
// The crew board: friends get a steady friendly score, you get your real one (your best before a round, this round after it).
const board = useMemo(() => crewBoard(trip?.members ?? [], me, phase === 'done' ? score : best), [trip, me, phase, score, best]);

if (phase === 'play') {
  const q = qs[i];
  const stateOf = (k: number): OptionState => {
    if (answer === null) return 'open';
    if (k === q.answer) return k === answer ? 'right' : 'reveal';
    return k === answer ? 'wrong' : 'dim';
```

```
    };
    const last = i + 1 >= qs.length;
    return (
      <Screen
        header={
          <Header
            kicker="Travel quiz"
            title={`Question ${i + 1}`}
            right={
              <View style={styles.points} accessible accessibilityLabel={`${score} points`}>
                <T variant="numeralSm" importantForAccessibility="no">
                  {score}
                </T>
                <T variant="kicker" color={colors.textSecondary} importantForAccessibility="no">
                  pts
                </T>
              </View>
            }
          />
        }
        footer={
          answer !== null ? (
            <View style={[styles.footer, { paddingBottom: insets.bottom + 12 }]}>
              <Button label={last ? 'See results' : 'Next question'} iconRight="arrow-forward" onPress={next} />
            </View>
          ) : undefined
        }>
        <StepBar index={i} total={qs.length} answered={answer !== null} />
        <View style={{ marginTop: 12 }}>
          <Clock left={left} />
        </View>
        <T variant="h1" style={{ marginTop: 16, marginBottom: 20 }} accessibilityRole="header">
          {q.q}
        </T>
        <View style={{ gap: 12 }}>
          {q.options.map((o, k) => (
            <AnswerOption key={o} letter={'ABCD'[k]} text={o} state={stateOf(k)} disabled={answer !== null} onPress={() => choose(k)} />
          ))}
        </View>
        {answer !== null ? (
          <View style={styles.fact} accessibilityLiveRegion="polite">
            <Rule />
            <View style={styles.factHead}>
              {answer === q.answer ? <Stamp label="Correct" tone="positive" icon="checkmark" slam /> : answer === -1 ? <Stamp label="Tim
e's up" tone="warn" icon="alarm-outline" slam /> : <Stamp label="Not quite" tone="danger" icon="close" slam />}
              {gained > 0 ? (
                <T variant="numeralSm" accessibilityLabel={`${gained} points`}>
                  +{gained}
                </T>
              ) : null}
              <View style={{ flex: 1 }} />
              {streak > 1 ? <Stamp label={`${streak} in a row`} tone="accent" rotate={2} /> : null}
            </View>
            <T variant="bodySm" color={colors.textSecondary}>
              {q.fact}
            </T>
          </View>
        ) : null}
      </Screen>
    );
  }

  if (phase === 'done') {
    const rows = resultRows(board, me, score, bestAtStart);
    const record = isNewBest(score, bestAtStart);
    const solo = rows.some((r) => r.id === 'round');
    const myRank = rows.findIndex((r) => r.me) + 1;
    return (
      <Screen header={<Header kicker="Travel quiz" title="Round complete" />}>
        <View style={styles.resultTop}>
          <View style={{ flex: 1 }}>
            <T variant="kicker" color={colors.textSecondary}>
              Your score
            </T>
            <T variant="hero" accessibilityLabel={`${score} points`}>
              {score}
            </T>
          </View>
          {record ? <Stamp label="New best" tone="accent" icon="star" slam rotate={-6} /> : null}
        </View>
        <T variant="headline" style={{ marginTop: 4 }}>
          {roundTitle(correct, qs.length)}
        </T>
        <T variant="small" color={colors.textSecondary} style={{ marginTop: 4 }}>
          {correct} of {qs.length} right · best streak {bestStreak}
          {!record && bestAtStart > 0 ? ` · your best is ${bestAtStart}` : ''}
```

```
        </T>

        <View style={styles.block}>
          <SectionHeader title={solo ? 'This round' : 'Crew podium'} />
          <Podium rows={rows} />
          {!solo && myRank > 3 ? (
            <View style={styles.mine} accessible accessibilityLabel={`You are ${ordinal(myRank)} with ${score} points`}>
              <Avatar name={me.name ?? 'You'} src={me.avatar} size={32} />
              <T variant="bodySm" weight="bold" color={colors.onAccent} style={{ flex: 1 }} importantForAccessibility="no">
                You are {ordinal(myRank)}
              </T>
              <T variant="numeralSm" color={colors.onAccent} style={{ fontSize: 17 }} importantForAccessibility="no">
                {score} pts
              </T>
            </View>
          ) : null}
        </View>

        <View style={{ gap: 12, marginTop: 32 }}>
          <Button label="Play again" icon="play" onPress={start} />
          <Button label="Challenge the group" icon="share-social-outline" variant="secondary" onPress={() => setShare(true)} />
        </View>
        <ShareSheet visible={share} onClose={() => setShare(false)} title="Share your score" message={`I scored ${score} points on the T
ogetherWeGo travel quiz. Think you can beat me?`} />
      </Screen>
    );
  }

  return (
    <Screen header={<Header kicker="Trip trivia" title="Travel quiz" subtitle="Test your globe-trotting knowledge" />}>
      <Ticket
        stub={
          <View style={styles.stub}>
            <Field value={`${ROUND}`} label="Questions" />
            <Field value={`${SECONDS}s`} label="Each" />
            <Field value="+2" label="Per streak" />
          </View>
        }>
        <View style={styles.photo}>
          <Image source={photos['fuji-blossom']} style={StyleSheet.absoluteFill} contentFit="cover" accessibilityIgnoresInvertColors />
          <View style={styles.tag}>
            <T variant="kicker" color={colors.ink}>
              Trip trivia
            </T>
          </View>
        </View>
        <View style={styles.main}>
          <T variant="kicker" color={colors.textSecondary}>
            Your best
          </T>
          <View style={styles.bestRow}>
            <T variant="hero" accessibilityLabel={`Your best score is ${best} points`}>
              {best}
            </T>
            <T variant="kicker" color={colors.textSecondary} style={{ paddingBottom: 12 }}>
              points
            </T>
          </View>
          <T variant="small" color={colors.textSecondary}>
            {best > 0 ? 'Faster answers and streaks earn bonus points.' : 'Play a round to set your first best. Faster answers and strea
ks earn bonus points.'}
          </T>
        </View>
      </Ticket>

      <View style={{ marginTop: 24 }}>
        <Button label="Play" icon="play" onPress={start} />
      </View>

      {board.length > 1 ? (
        <View style={styles.block}>
          <SectionHeader title="Crew best" />
          <Card padded={false} style={{ overflow: 'hidden' }}>
            {board.map((b, k) => (
              <View key={b.id}>
                {k > 0 ? <Divider /> : null}
                <View style={[styles.boardRow, b.me && { backgroundColor: colors.accent }]}>
                  <T variant="kicker" color={b.me ? colors.onAccent : colors.textSecondary} style={{ width: 26 }}>
                    {String(k + 1).padStart(2, '0')}
                  </T>
                  <Avatar name={b.name} src={b.avatar} size={32} />
                  <T variant="bodySm" weight="semibold" color={b.me ? colors.onAccent : colors.text} style={{ flex: 1 }} numberOfLines=
{1}>
                    {b.me ? 'You' : b.name.split(' ')[0]}
                  </T>
                  <T variant="numeralSm" color={b.me ? colors.onAccent : colors.text}>
```

```
                  {b.score}
                </T>
              </View>
            </View>
          ))}
        </Card>
      </View>
    ) : null}
  </Screen>
 );
}

/** One printed field on the stub of the ticket: a number in mono over a small caps label. */
function Field({ value, label }: { value: string; label: string }) {
 return (
  <View style={{ flex: 1 }}>
   <T variant="numeralSm">{value}</T>
   <T variant="kicker" color={colors.textSecondary}>
    {label}
   </T>
  </View>
 );
}

const CORNER = Math.max(0, radius.card - line);

const styles = StyleSheet.create({
 photo: { height: 124, borderTopLeftRadius: CORNER, borderTopRightRadius: CORNER, borderBottomWidth: line, borderBottomColor: colors.in
k, overflow: 'hidden', backgroundColor: colors.bgAlt },
 tag: { position: 'absolute', left: 12, bottom: 12, paddingHorizontal: 8, paddingVertical: 3, backgroundColor: colors.card, borderWidt
h: line, borderColor: colors.ink, borderRadius: radius.xs },
 main: { padding: 16, gap: 2 },
 bestRow: { flexDirection: 'row', alignItems: 'flex-end', gap: 10 },
 stub: { flexDirection: 'row', gap: 12, paddingHorizontal: 16, paddingBottom: 14, paddingTop: 2 },
 block: { marginTop: 32 },
 boardRow: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingHorizontal: 14, paddingVertical: 8 },
 points: { alignItems: 'flex-end', paddingBottom: 2 },
 footer: { paddingHorizontal: 20, paddingTop: 12, borderTopWidth: line, borderTopColor: colors.ink, backgroundColor: colors.bg },
 fact: { marginTop: 24, gap: 12 },
 factHead: { flexDirection: 'row', alignItems: 'center', gap: 12 },
 mine: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, marginTop: 14, paddingHorizontal: 10, borderRadius: radiu
s.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.accent },
 resultTop: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
});
```

src/app/recap.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useEffect, useRef, useState } from 'react';
import { Platform, StyleSheet, View, useWindowDimensions } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { RecapPoster } from '@/components/play/RecapPoster';
import { placesVisited, votesCast, yesVotes } from '@/components/play/recapLogic';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Divider, SectionHeader, type IconName } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { shareView } from '@/services/share';
import { settlePlan, totalSpent, tripStatus, useActiveTrip, useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, line, radius } from '@/theme';
import { money } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';
import { daysBetween, rangeLabel } from '@/utils/time';

const STATUS = {
 completed: { label: 'Trip complete', tone: 'accent' },
 active: { label: 'In progress', tone: 'positive' },
 planning: { label: 'Coming up', tone: 'ink' },
} as const;

/** End-of-trip summary: the "memories" stage of the planning, travelling, memories journey. A passport page you can share, then the num
bers behind it. */
export default function Recap() {
 const trip = useActiveTrip();
 const { width } = useWindowDimensions();
 const fired = useAppStore((s) => s.firedReminders);
 const markFired = useAppStore((s) => s.markReminderFired);
 const [share, setShare] = useState(false);
```

```
  const [rating, setRating] = useState(0);
  const card = useRef<View>(null);

  const tripId = trip?.id;
  const done = !!trip && tripStatus(trip) === 'completed';
  const seen = !!tripId && `recap:${tripId}` in fired;
  // The end of the trip is the peak: a small celebration the first time the finished recap is opened (and never again).
  useEffect(() => {
    if (!done || seen || !tripId) return;
    const id = setTimeout(() => {
      markFired(`recap:${tripId}`);
      celebrate('Trip complete');
    }, 900);
    return () => clearTimeout(id);
  }, [done, seen, tripId, markFired]);

  if (!trip) return <NoTrip />;

  const dest = destinationById(trip.destinationId)!;
  const status = tripStatus(trip);
  const days = daysBetween(trip.startDate, trip.endDate) + 1;
  const spent = totalSpent(trip);
  const stops = trip.activities.filter((a) => a.lat && a.lng).sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time));
  const km = stops.reduce((sum, a, i) => (i ? sum + distanceKm(stops[i - 1] as { lat: number; lng: number }, a as { lat: number; lng: nu
mber }) : 0), 0);
  const topPhoto = trip.photos.slice().sort((a, b) => b.likes.length - a.likes.length)[0];
  const topActivity = trip.activities.filter((a) => a.type !== 'transport' && a.type !== 'stay').sort((a, b) => yesVotes(b) - yesVotes
(a))[0];
  const byCat: Record<string, number> = {};
  trip.expenses.filter((e) => !e.settlement).forEach((e) => (byCat[e.category] = (byCat[e.category] ?? 0) + e.amount));
  const topCat = Object.entries(byCat).sort((a, b) => b[1] - a[1])[0];
  const photographer = trip.members.map((m) => ({ m, n: trip.photos.filter((p) => p.by === m.id).length })).sort((a, b) => b.n - a.n)
[0];
  const owed = settlePlan(trip);
  const places = placesVisited(trip.activities);
  const perPerson = spent / Math.max(1, trip.members.length);
  const posterW = Math.min(Math.min(width, MAX_WIDTH) - 40 - 4, 330);

  const ledger: [string, string][] = [
    [`${trip.members.length}`, 'travellers'],
    [`${trip.activities.length}`, 'activities'],
    [kmLabel(km), 'between stops'],
    [money(spent), 'spent together'],
    [`${trip.photos.length}`, 'photos'],
    [`${votesCast(trip)}`, 'votes cast'],
  ];

  const summary = `Our trip “${trip.name}”: ${days} days in ${dest.city} · ${trip.activities.length} activities · ${trip.photos.length}
photos · ${kmLabel(km)} explored · ${money(perPerson)} each. Planned together on TogetherWeGo.`;

  return (
    <Screen header={<Header kicker={trip.name} title="Trip recap" subtitle={status === 'completed' ? 'Your memories, wrapped up' : 'A pr
eview until the trip ends'} />}>
      <RecapPoster
        cardRef={card}
        width={posterW}
        image={trip.cover}
        trip={trip}
        place={`${dest.city}, ${dest.country}`}
        dates={rangeLabel(trip.startDate, trip.endDate)}
        status={STATUS[status]}
        stats={[
          [`${days}`, days === 1 ? 'day' : 'days'],
          [`${places.length}`, places.length === 1 ? 'place' : 'places'],
          [money(perPerson), 'spent each'],
        ]}
        places={places.slice(0, 4)}
        summary={summary}
      />

      <View style={styles.actions}>
        {Platform.OS !== 'web' ? <Button label="Share picture" icon="image-outline" onPress={() => shareView(card, 'Share your trip reca
p', summary)} style={{ flex: 1 }} testID="share-picture" /> : null}
        <Button label={Platform.OS === 'web' ? 'Share story card' : 'Share text'} icon="share-social-outline" variant={Platform.OS ===
'web' ? 'primary' : 'secondary'} onPress={() => setShare(true)} style={{ flex: 1 }} />
      </View>

      <View style={styles.block}>
        <SectionHeader title="The numbers" />
        <View style={styles.ledger}>
          {ledger.map(([value, label]) => (
            <View key={label} style={styles.cell} accessible accessibilityLabel={`${value} ${label}`}>
              <T variant="numeralSm" numberOfLines={1} importantForAccessibility="no">
                {value}
              </T>
              <T variant="kicker" color={colors.textSecondary} importantForAccessibility="no">
```

```
              {label}
            </T>
          </View>
        ))}
      </View>
    </View>

    <View style={styles.block}>
      <SectionHeader title="Highlights" />
      <Card padded={false} style={{ overflow: 'hidden' }}>
        {topActivity ? <Highlight icon="trophy-outline" title="Group favourite" body={`${topActivity.title}: ${yesVotes(topActivity)}
of ${trip.members.length} said “I’m in”`} /> : null}
        {topPhoto ? <Highlight icon="heart-outline" title="Most-liked photo" body={`“${topPhoto.caption || 'Untitled'}” · ${topPhoto.l
ikes.length} likes`} /> : null}
        {photographer && photographer.n ? <Highlight icon="camera-outline" title="Chief photographer" body={`${photographer.m.name.spl
it(' ')[0]} with ${photographer.n} photos`} /> : null}
        {topCat ? <Highlight icon="pie-chart-outline" title="Biggest spend" body={`${topCat[0]} · ${money(topCat[1])} (${Math.round((t
opCat[1] / Math.max(1, spent)) * 100)}%)`} /> : null}
        <Highlight icon="people-outline" title="The crew" body={trip.members.map((m) => `${m.name.split(' ')[0]} (${m.role})`).join(',
')} last />
      </Card>
    </View>

    <View style={styles.block}>
      <SectionHeader title="Final balances" action="Settle up" onAction={() => router.navigate('/(tabs)/budget')} />
      {owed.length ? (
        <Card padded={false} style={{ overflow: 'hidden' }}>
          {owed.slice(0, 4).map((p, i) => {
            const a = trip.members.find((m) => m.id === p.from);
            const b = trip.members.find((m) => m.id === p.to);
            return (
              <View key={`${p.from}${p.to}`}>
                {i > 0 ? <Divider /> : null}
                <View style={styles.owe}>
                  <Avatar name={a?.name ?? '?'} src={a?.avatar} size={32} />
                  <T variant="bodySm" weight="semibold" style={{ flex: 1 }} numberOfLines={1}>
                    {a?.name.split(' ')[0]} pays {b?.name.split(' ')[0]}
                  </T>
                  <T variant="numeralSm">{money(p.amount)}</T>
                </View>
              </View>
            );
          })}
        </Card>
      ) : (
        <View style={styles.settled}>
          <Stamp label="All settled" tone="positive" icon="checkmark" rotate={-3} />
          <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
            Nobody owes anybody.
          </T>
        </View>
      )}
    </View>

    <View style={styles.block}>
      <SectionHeader title="How was the trip?" />
      <View style={styles.stars} accessibilityRole="radiogroup">
        {[1, 2, 3, 4, 5].map((n) => (
          <Press
            key={n}
            onPress={() => {
              setRating(n);
              toast(n >= 4 ? 'Saved. Sounds like a great trip' : 'Thanks for the feedback');
            }}
            style={styles.star}
            accessibilityRole="radio"
            accessibilityState={{ checked: n === rating }}
            accessibilityLabel={`${n} star${n > 1 ? 's' : ''}`}>
            <Ionicons name={n <= rating ? 'star' : 'star-outline'} size={34} color={colors.ink} />
          </Press>
        ))}
      </View>
    </View>

    <Button label="Open the photo dump" icon="images-outline" variant="secondary" onPress={() => router.push('/photos')} style={{ marg
inTop: 32 }} />
    <ShareSheet visible={share} onClose={() => setShare(false)} title="Share your trip recap" message={summary} preview={summary} />
  </Screen>
 );
}

function Highlight({ icon, title, body, last }: { icon: IconName; title: string; body: string; last?: boolean }) {
 return (
  <View style={[styles.hl, !last && { borderBottomWidth: 1, borderBottomColor: colors.border }]}>
    <View style={styles.hlIcon}>
      <Ionicons name={icon} size={18} color={colors.ink} />
```

```
      </View>
      <View style={{ flex: 1, gap: 2 }}>
        <T variant="kicker" color={colors.textSecondary}>
          {title}
        </T>
        <T variant="bodySm" weight="semibold">
          {body}
        </T>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  actions: { flexDirection: 'row', gap: 12, marginTop: 24 },
  block: { marginTop: 32 },
  ledger: { flexDirection: 'row', flexWrap: 'wrap', rowGap: 18 },
  cell: { width: '50%', gap: 2, paddingRight: 8 },
  hl: { flexDirection: 'row', gap: 12, padding: 14 },
  hlIcon: { width: 38, height: 38, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAl
t, alignItems: 'center', justifyContent: 'center' },
  owe: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingHorizontal: 14, paddingVertical: 8 },
  settled: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  stars: { flexDirection: 'row', gap: 4 },
  star: { width: 48, height: 48, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/settings.tsx

```
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { CurrencyPickerSheet } from '@/components/CurrencyPickerSheet';
import { AccessibilityCard } from '@/components/settings/AccessibilityCard';
import { AssistantCard } from '@/components/settings/AssistantCard';
import { DataCard } from '@/components/settings/DataCard';
import { DemoCard } from '@/components/settings/DemoCard';
import { Group } from '@/components/settings/Group';
import { HomeLayoutCard } from '@/components/settings/HomeLayoutCard';
import { LookCard } from '@/components/settings/LookCard';
import { NotificationsCard } from '@/components/settings/NotificationsCard';
import { ProfileCard } from '@/components/settings/ProfileCard';
import { SyncSettingsCard } from '@/components/sync/SyncSettingsCard';
import { Button } from '@/components/ui/Button';
import { Fold } from '@/components/ui/Fold';
import { Card, Chip, Divider, ListRow, SectionHeader, ToggleRow } from '@/components/ui/bits';
import { confirmAction } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { currencyByCode } from '@/data/reference';
import { LANGUAGES, useT } from '@/i18n';
import { useAppStore } from '@/store/useAppStore';
import { colors, space } from '@/theme';

/**
 * Settings, everyday things first: you, the look, what Home shows, language and money, reminders, text and touch.
 * The technical ones (time zones, sync between phones, Claude AI, sample trips, reset) are folded under Advanced.
 * Each card is its own component in components/settings.
 */
export default function Settings() {
  const t = useT();
  const settings = useAppStore((s) => s.settings);
  const update = useAppStore((s) => s.updateSettings);
  const signOut = useAppStore((s) => s.signOut);
  const [curOpen, setCurOpen] = useState(false);

  return (
    <Screen header={<Header title={t('settings.title')} kicker="TogetherWeGo" subtitle="Your look, your Home, your reminders" />}>
      <ProfileCard />

      <SectionHeader title="Look and feel" style={styles.section} />
      <LookCard />
      <View style={{ marginTop: 8 }}>
        <ListRow icon="color-palette-outline" label="Make it yours again" sub="Redo the quick picks: look, interests, Home and vibe" onP
ress={() => router.push('/personalize')} />
      </View>

      <SectionHeader title="Home layout" style={styles.section} />
      <HomeLayoutCard />

      <SectionHeader title="Everyday" style={styles.section} />
      <View style={{ gap: 20 }}>
        <Group label={t('settings.language')}>
          <View style={styles.wrap}>
            {LANGUAGES.map((l) => (
```

```
              <Chip key={l.code} label={l.native} active={settings.language === l.code} onPress={() => update({ language: l.code })} />
            ))}
          </View>
        </Group>
        <Divider />
        <Group label="Money" gap={2}>
          <ListRow icon="cash-outline" label={t('settings.currency')} sub={`${settings.homeCurrency} · ${currencyByCode(settings.homeCur
rency)?.name ?? ''}`} onPress={() => setCurOpen(true)} />
        </Group>
        <Divider />
        <NotificationsCard />
        <Divider />
        <AccessibilityCard />
        <Divider />
        <Group label="Help" gap={2}>
          <ListRow icon="help-circle-outline" label="How to use TogetherWeGo" onPress={() => router.push('/help')} />
          <ListRow icon="information-circle-outline" label="About, credits & sources" onPress={() => router.push('/about')} />
        </Group>
      </View>

      <Fold title="Advanced" icon="options-outline" label="Advanced settings" summary="Time zones, sync between phones, Claude AI, sampl
e trips, reset">
        <Card style={{ marginTop: 14 }}>
          <ToggleRow icon="globe-outline" label="Plan times in trip time" sub="Off shows them in your phone’s time zone" value={setting
s.showTripTime} onChange={(v) => update({ showTripTime: v })} />
        </Card>
        <SyncSettingsCard />
        <AssistantCard />
        <DemoCard />
        <DataCard />
      </Fold>

      <Button
        label={t('settings.signOut')}
        variant="danger"
        icon="log-out-outline"
        style={{ marginTop: space.section }}
        onPress={() =>
          confirmAction(
            'Sign out?',
            'Your trips stay saved on this device.',
            () => {
              signOut();
              router.replace('/sign-in');
            },
            { confirmLabel: 'Sign out' },
          )
        }
      />
      <T variant="caption" color={colors.textMuted} style={{ marginTop: space.xl }}>
        TogetherWeGo v1.0 · Made for FBLA Mobile Application Development 2026–27
      </T>

      <CurrencyPickerSheet visible={curOpen} onClose={() => setCurOpen(false)} value={settings.homeCurrency} onPick={(c) => update({ hom
eCurrency: c })} title="Home currency" />
    </Screen>
  );
}

const styles = StyleSheet.create({
  section: { marginTop: space.section },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
});
```

src/app/sign-in.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Link, router, useLocalSearchParams } from 'expo-router';
import { useRef, useState } from 'react';
import { StyleSheet, View, type ScrollView, type TextInput } from 'react-native';

import { DemoTicket } from '@/components/auth/DemoTicket';
import { GoogleSheet } from '@/components/auth/GoogleSheet';
import { JoinCodeSheet } from '@/components/auth/JoinCodeSheet';
import { AuthScreen, InviteNote, LegalLinks, Notice, TextLink } from '@/components/auth/kit';
import { afterSignIn, calm, inviteFrom } from '@/components/auth/logic';
import { Logo } from '@/components/Logo';
import { Button } from '@/components/ui/Button';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { FadeIn } from '@/components/ui/motion';
import { Press, tap } from '@/components/ui/Press';
import { Divider, ToggleRow } from '@/components/ui/bits';
import { Rule } from '@/components/ui/Rule';
import { StaggerText } from '@/components/ui/StaggerText';
import { T } from '@/components/ui/T';
```

```
import { useT } from '@/i18n';
import { DEMO_EMAIL, DEMO_PASSWORD } from '@/store/seed';
import { useAppStore } from '@/store/useAppStore';
import { colors, line } from '@/theme';
import { validateEmail } from '@/utils/validation';

/**
 * The check-in desk. The logo, a serif welcome, two big fields, one black button, Google, and (for judges) the demo account as a
 * ticket. Fields say what is wrong with a colour, an icon and words once you have left them. Someone invited by a friend can enter the
 * code first: it is kept, and after signing in the trip's join page opens with it.
 */
export default function SignIn() {
  const t = useT();
  const params = useLocalSearchParams<{ code?: string }>();
  const signIn = useAppStore((s) => s.signIn);
  const signInWithGoogle = useAppStore((s) => s.signInWithGoogle);
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [remember, setRemember] = useState(true);
  const [shown, setShown] = useState({ email: false, password: false });
  const [formError, setFormError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [googleOpen, setGoogleOpen] = useState(false);
  const [joinOpen, setJoinOpen] = useState(false);
  const [invite, setInvite] = useState<string | null>(() => inviteFrom(params.code));
  const [filled, setFilled] = useState(false);
  const pwRef = useRef<TextInput>(null);
  const scroll = useRef<ScrollView>(null);

  const emailError = shown.email ? validateEmail(email) : null;
  const passwordError = shown.password && !password ? 'Enter your password' : null;

  const enter = () => {
    const next = afterSignIn(invite);
    router.replace(next.first);
    // the invite opens a moment later, once Home is on screen
    const then = next.then;
    if (then) setTimeout(() => router.push(then), 350);
  };

  const submit = async () => {
    setShown({ email: true, password: true });
    if (validateEmail(email) || !password) return tap('warning');
    setLoading(true);
    const res = await signIn(email, password, remember);
    setLoading(false);
    if (!res.ok) {
      setFormError(res.error);
      tap('warning');
      return;
    }
    tap('success');
    enter();
  };

  const fillDemo = () => {
    setEmail(DEMO_EMAIL);
    setPassword(DEMO_PASSWORD);
    setShown({ email: false, password: false });
    setFormError(null);
    setFilled(true);
    toast('Demo account filled in. Tap Sign In');
    scroll.current?.scrollTo({ y: 0, animated: true });
  };

  const edited = () => {
    setFormError(null);
    setFilled(false);
  };

  return (
    <AuthScreen scrollRef={scroll} top={12}>
      <Logo />
      <Rule style={{ marginTop: 16 }} />

      <View style={{ marginTop: 24, gap: 8 }}>
        <StaggerText text={calm(t('auth.welcome'))} variant="display" />
        <T variant="bodySm" color={colors.textSecondary}>
          {t('auth.welcomeSub')}
        </T>
      </View>

      {invite ? (
        <FadeIn from="up" distance={8} style={{ marginTop: 20 }}>
          <InviteNote code={invite} onRemove={() => setInvite(null)} />
        </FadeIn>
```

```
      ) : null}

      <View style={{ marginTop: 24, gap: 16 }}>
        <Input
          label={t('auth.email')}
          icon={emailError ? 'alert-circle' : 'mail-outline'}
          value={email}
          onChangeText={(v) => {
            setEmail(v);
            edited();
          }}
          onBlur={() => email && setShown((s) => ({ ...s, email: true }))}
          autoCapitalize="none"
          autoComplete="email"
          autoCorrect={false}
          keyboardType="email-address"
          textContentType="emailAddress"
          returnKeyType="next"
          onSubmitEditing={() => pwRef.current?.focus()}
          placeholder="name@email.com"
          error={emailError}
          testID="email"
        />
        <Input
          ref={pwRef}
          label={t('auth.password')}
          icon={passwordError ? 'alert-circle' : 'lock-closed-outline'}
          value={password}
          onChangeText={(v) => {
            setPassword(v);
            edited();
          }}
          onBlur={() => setShown((s) => ({ ...s, password: true }))}
          secureToggle
          autoComplete="password"
          textContentType="password"
          returnKeyType="go"
          onSubmitEditing={submit}
          placeholder="Your password"
          error={passwordError}
          testID="password"
        />
      </View>

      <View style={styles.row}>
        <View style={{ flexShrink: 1 }}>
          <ToggleRow label={t('auth.remember')} value={remember} onChange={setRemember} />
        </View>
        <TextLink label={t('auth.forgot')} onPress={() => router.push('/forgot-password')} />
      </View>

      {formError ? (
        <View style={{ marginBottom: 16 }}>
          <Notice>{formError}</Notice>
        </View>
      ) : null}

      <Button label={t('auth.signIn')} onPress={submit} loading={loading} testID="sign-in" />

      <View style={styles.or}>
        <Divider style={{ flex: 1 }} />
        <T variant="kicker" color={colors.textMuted}>
          {t('auth.or')}
        </T>
        <Divider style={{ flex: 1 }} />
      </View>

      <Button label={t('auth.google')} icon="logo-google" variant="secondary" size="md" onPress={() => setGoogleOpen(true)} />

      <View style={{ marginTop: 32 }}>
        <DemoTicket title={t('auth.demo')} onFill={fillDemo} filled={filled} />
      </View>

      <View style={styles.desk}>
        <Link href={{ pathname: '/sign-up', params: invite ? { code: invite } : {} }} asChild>
          <Press accessibilityRole="link" accessibilityLabel={`${t('auth.noAccount')} ${t('auth.signUp')}`} scaleTo={0.99} style={style
s.deskRow}>
            <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
              {t('auth.noAccount')}
            </T>
            <T variant="small" weight="bold">
              {t('auth.signUp')}
            </T>
            <Ionicons name="arrow-forward" size={16} color={colors.ink} />
          </Press>
        </Link>
```

```
        <Divider />
        <Press onPress={() => setJoinOpen(true)} accessibilityLabel="Join with a code" scaleTo={0.99} style={styles.deskRow}>
          <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
            Invited by a friend?
          </T>
          <T variant="small" weight="bold">
            Join with a code
          </T>
          <Ionicons name="arrow-forward" size={16} color={colors.ink} />
        </Press>
      </View>

      <LegalLinks style={{ marginTop: 8 }} />

      <GoogleSheet
        visible={googleOpen}
        onClose={() => setGoogleOpen(false)}
        onPick={async (mail, name) => {
          setGoogleOpen(false);
          await signInWithGoogle(mail, name);
          tap('success');
          enter();
        }}
      />
      <JoinCodeSheet
        visible={joinOpen}
        initial={invite}
        onClose={() => setJoinOpen(false)}
        onDone={(code) => {
          setInvite(code);
          setJoinOpen(false);
          tap('success');
          toast(`Invite ${code} saved. Sign in or create an account`);
          scroll.current?.scrollTo({ y: 0, animated: true });
        }}
      />
    </AuthScreen>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', justifyContent: 'space-between', columnGap: 12, marginTop: 4, mar
ginBottom: 12 },
  or: { flexDirection: 'row', alignItems: 'center', gap: 12, marginVertical: 20 },
  desk: { marginTop: 28, borderTopWidth: line, borderTopColor: colors.ink },
  deskRow: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 56 },
});
```

src/app/sign-up.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, useLocalSearchParams } from 'expo-router';
import { useRef, useState } from 'react';
import { View, type TextInput } from 'react-native';

import { AuthScreen, CheckRow, InviteNote, LegalLinks, PasswordMeter } from '@/components/auth/kit';
import { afterSignIn, inviteFrom } from '@/components/auth/logic';
import { Button } from '@/components/ui/Button';
import { Input } from '@/components/ui/Input';
import { tap } from '@/components/ui/Press';
import { Header } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useDraft } from '@/hooks/useDraft';
import { useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';
import { validateEmail, validateName, validatePassword } from '@/utils/validation';

type Field = 'name' | 'email' | 'password' | 'confirm' | 'agree';
const NONE: Record<Field, boolean> = { name: false, email: false, password: false, confirm: false, agree: false };

/** A green tick at the end of a field that is filled in correctly (with the field's own words, never colour alone). */
const tick = (ok: boolean) => (ok ? <Ionicons name="checkmark-circle" size={20} color={colors.positive} accessibilityLabel="Looks good"
/> : undefined);

/**
 * Create an account: four fields, one tick box, one black button. A field tells you what is wrong once you leave it (state colour, an
 * icon and words) and shows a tick when it is right. Name and email are kept as a draft if you leave and come back.
 */
export default function SignUp() {
  const params = useLocalSearchParams<{ code?: string }>();
  const invite = inviteFrom(params.code);
  const signUp = useAppStore((s) => s.signUp);
  const [name, setName, clearName] = useDraft('signup-name', '');
  const [email, setEmail, clearEmail] = useDraft('signup-email', '');
  const [password, setPassword] = useState('');
  const [confirm, setConfirm] = useState('');
```

```
const [agree, setAgree] = useState(false);
const [shown, setShown] = useState(NONE);
const [taken, setTaken] = useState<string | null>(null);
const [loading, setLoading] = useState(false);
const emailRef = useRef<TextInput>(null);
const pwRef = useRef<TextInput>(null);
const confirmRef = useRef<TextInput>(null);

const show = (f: Field) => setShown((s) => (s[f] ? s : { ...s, [f]: true }));
const nameErr = validateName(name);
const emailErr = validateEmail(email);
const passErr = validatePassword(password);
const confirmErr = !confirm ? 'Re-enter your password' : confirm !== password ? 'Passwords don’t match' : null;
const agreeErr = agree ? null : 'Please accept the terms to continue';

const submit = async () => {
  setShown({ name: true, email: true, password: true, confirm: true, agree: true });
  if (nameErr || emailErr || passErr || confirmErr || agreeErr) return tap('warning');
  setLoading(true);
  const res = await signUp(name, email, password);
  setLoading(false);
  if (!res.ok) {
    setTaken(res.error);
    return tap('warning');
  }
  tap('success');
  clearName();
  clearEmail();
  const next = afterSignIn(invite);
  router.replace(next.first);
  // Brand-new account: with an invite the trip comes first; otherwise offer the personalization survey straight away.
  setTimeout(() => router.push(next.then ?? '/personalize'), 350);
};

return (
  <AuthScreen top={0}>
    <View style={{ marginHorizontal: -20 }}>
      <Header kicker="New passenger" title="Create account" subtitle="Start planning trips with your crew" />
    </View>
    {invite ? (
      <View style={{ marginBottom: 20 }}>
        <InviteNote code={invite} />
      </View>
    ) : null}
    <View style={{ gap: 16 }}>
      <Input
        label="Full name"
        icon={shown.name && nameErr ? 'alert-circle' : 'person-outline'}
        value={name}
        onChangeText={setName}
        onBlur={() => name && show('name')}
        autoCapitalize="words"
        autoComplete="name"
        returnKeyType="next"
        onSubmitEditing={() => emailRef.current?.focus()}
        error={shown.name ? nameErr : null}
        right={tick(!!name && !nameErr)}
        placeholder="e.g. Alex Rivera"
      />
      <Input
        ref={emailRef}
        label="Email address"
        icon={(shown.email && emailErr) || taken ? 'alert-circle' : 'mail-outline'}
        value={email}
        onChangeText={(v) => {
          setEmail(v);
          setTaken(null);
        }}
        onBlur={() => email && show('email')}
        autoCapitalize="none"
        autoCorrect={false}
        keyboardType="email-address"
        autoComplete="email"
        textContentType="emailAddress"
        returnKeyType="next"
        onSubmitEditing={() => pwRef.current?.focus()}
        error={taken ?? (shown.email ? emailErr : null)}
        right={tick(!!email && !emailErr && !taken)}
        placeholder="you@email.com"
      />
      <View style={{ gap: 10 }}>
        <Input
          ref={pwRef}
          label="Password"
          icon={shown.password && passErr ? 'alert-circle' : 'lock-closed-outline'}
          value={password}
```

```
            onChangeText={setPassword}
            onBlur={() => password && show('password')}
            secureToggle
            textContentType="newPassword"
            returnKeyType="next"
            onSubmitEditing={() => confirmRef.current?.focus()}
            error={shown.password ? passErr : null}
            hint="At least 8 characters, with a letter and a number"
            placeholder="At least 8 characters"
          />
          <PasswordMeter password={password} />
        </View>
        <Input
          ref={confirmRef}
          label="Confirm password"
          icon={shown.confirm && confirmErr ? 'alert-circle' : 'lock-closed-outline'}
          value={confirm}
          onChangeText={setConfirm}
          onBlur={() => confirm && show('confirm')}
          secureToggle
          returnKeyType="go"
          onSubmitEditing={submit}
          error={shown.confirm ? confirmErr : null}
          right={tick(!!confirm && !confirmErr)}
          placeholder="Re-enter password"
        />
      </View>

      <View style={{ marginTop: 12 }}>
        <CheckRow
          checked={agree}
          onPress={() => {
            setAgree((a) => !a);
            show('agree');
          }}
          label="I agree to the Terms of Service and Privacy Policy"
          error={shown.agree ? agreeErr : null}>
          <T variant="small" color={colors.textSecondary}>
            I agree to the Terms of Service and Privacy Policy
          </T>
        </CheckRow>
        <LegalLinks />
      </View>

      <View style={{ marginTop: 12 }}>
        <Button label="Create account" onPress={submit} loading={loading} />
      </View>

      <View style={{ flexDirection: 'row', gap: 10, alignItems: 'center', marginTop: 20 }}>
        <Ionicons name="shield-checkmark-outline" size={20} color={colors.ink} />
        <T variant="caption" color={colors.textSecondary} style={{ flex: 1 }}>
          Your password is salted and hashed on this phone. It is never stored as plain text.
        </T>
      </View>
    </AuthScreen>
  );
}
```

src/app/survey.tsx

```
import { useState } from 'react';
import { View, useWindowDimensions } from 'react-native';

import { AddToTripSheet } from '@/components/explore';
import { MatchRow, OptionTile, TopPick } from '@/components/play/SurveyParts';
import { optionArt, recommend, sameAnswer, type Match } from '@/components/play/recommend';
import { Button } from '@/components/ui/Button';
import { Card, ProgressBar, SectionHeader } from '@/components/ui/bits';
import { tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { surveyQuestions } from '@/data/reference';
import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors } from '@/theme';

const GAP = 16;

/** The travel survey: one question per screen with big answer tiles, then the five destinations that fit best. */
export default function Survey() {
  const saved = useAppStore((s) => s.surveyResult);
  const save = useAppStore((s) => s.setSurveyResult);
  const { width } = useWindowDimensions();
  const [step, setStep] = useState(saved ? surveyQuestions.length : 0);
  const [answers, setAnswers] = useState<string[][]>([]);
  const [results, setResults] = useState<Match[]>(saved ? saved.map((id, i) => ({ id, match: 98 - i * 4 })) : []);
```

```
  const [adding, setAdding] = useState<string | null>(null);

  const pick = (tags: string[]) => {
    const next = [...answers.slice(0, step), tags];
    setAnswers(next);
    tap();
    if (step + 1 >= surveyQuestions.length) {
      const r = recommend(next);
      setResults(r);
      save(r.map((x) => x.id));
    }
    setStep(step + 1);
  };

  if (step >= surveyQuestions.length) {
    const [top, ...rest] = results;
    const topDest = top ? destinationById(top.id) : undefined;
    return (
      <Screen header={<Header kicker="Based on your travel style" title="Your matches" />}>
        <T variant="small" color={colors.textSecondary} style={{ marginBottom: 20 }}>
          We matched your answers against 32 destinations. Tap one to explore it, or add it to a trip.
        </T>
        {top && topDest ? <TopPick dest={topDest} match={top.match} onAdd={() => setAdding(topDest.id)} /> : null}
        {rest.length ? (
          <View style={{ marginTop: 32 }}>
            <SectionHeader title="Also good for you" />
            <Card padded={false} style={{ overflow: 'hidden' }}>
              {rest.map((r, i) => {
                const d = destinationById(r.id);
                return d ? <MatchRow key={r.id} dest={d} match={r.match} last={i === rest.length - 1} onAdd={() => setAdding(d.id)} /> :
null;
              })}
            </Card>
          </View>
        ) : null}
        <Button
          label="Retake survey"
          variant="secondary"
          icon="refresh-outline"
          style={{ marginTop: 32 }}
          onPress={() => {
            setAnswers([]);
            setStep(0);
          }}
        />
        <AddToTripSheet destinationId={adding} onClose={() => setAdding(null)} />
      </Screen>
    );
  }

  const q = surveyQuestions[step];
  // two tiles side by side in the phone-wide column (the plate under each tile adds 3 points)
  const tile = Math.floor((Math.min(width, MAX_WIDTH) - 40 - GAP) / 2) - 3;
  return (
    <Screen header={<Header kicker={`Question ${step + 1} of ${surveyQuestions.length}`} title="Find your next trip" />}>
      <ProgressBar value={step / surveyQuestions.length} />
      <T variant="headline" style={{ marginTop: 24, marginBottom: 20 }} accessibilityRole="header">
        {q.q}
      </T>
      <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: GAP - 3 }}>
        {q.options.map((o) => (
          <OptionTile key={o.label} label={o.label} art={optionArt(q, o.label)} width={tile} chosen={sameAnswer(answers[step], o.tags)}
onPress={() => pick(o.tags)} />
        ))}
      </View>
      {step > 0 ? <Button label="Back" variant="ghost" icon="arrow-back" onPress={() => setStep(step - 1)} style={{ marginTop: 24 }} />
: null}
    </Screen>
  );
}
```

src/app/trips.tsx

```
import { Image } from 'expo-image';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { Button } from '@/components/ui/Button';
import { AvatarStack, Card, EmptyState, SectionHeader } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Fab } from '@/components/ui/Fab';
import { Stamp } from '@/components/ui/Stamp';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
```

```
import { imageSource } from '@/data/images';
import { totalSpent, tripStatus, useAppStore, useMyTrips } from '@/store/useAppStore';
import type { Trip, TripStatus } from '@/store/types';
import { colors, line, radius, space } from '@/theme';
import { money } from '@/utils/format';
import { rangeLabel } from '@/utils/time';

const ORDER: { key: TripStatus; title: string }[] = [
  { key: 'active', title: 'In progress' },
  { key: 'planning', title: 'Upcoming' },
  { key: 'completed', title: 'Completed' },
];

/** Every trip the person is on, grouped by where it is in time. The trip you are looking at is stamped CURRENT and is the one card that
stands up. */
export default function Trips() {
  const trips = useMyTrips();
  const activeId = useAppStore((s) => s.activeTripId);
  const setActive = useAppStore((s) => s.setActiveTrip);
  const insets = useSafeAreaInsets();

  const open = (t: Trip) => {
    setActive(t.id);
    toast(`Switched to ${t.name}`);
    router.navigate('/(tabs)/itinerary');
  };

  return (
    <Screen
      header={<Header kicker="Passport" title="My trips" subtitle={`${trips.length} trip${trips.length === 1 ? '' : 's'}`} />}
      fab={trips.length ? <Fab icon="add" label="New trip" accessibilityLabel="New trip" onPress={() => router.push('/new-trip')} /> : u
ndefined}
      fabBottom={insets.bottom + 16}>
      {trips.length === 0 ? (
        <EmptyState icon="airplane-outline" title="No trips yet" body="Create one or join a friend’s trip with their invite code.">
          <View style={{ gap: 12 }}>
            <Button label="New trip" icon="add" onPress={() => router.push('/new-trip')} />
            <Button label="Join with code" icon="enter-outline" variant="secondary" onPress={() => router.push('/join')} />
          </View>
        </EmptyState>
      ) : (
        <Button label="Join with code" icon="enter-outline" variant="secondary" size="md" onPress={() => router.push('/join')} />
      )}
      {ORDER.map(({ key, title }) => {
        const list = trips.filter((t) => tripStatus(t) === key);
        if (!list.length) return null;
        return (
          <View key={key} style={{ marginTop: space.section }}>
            <SectionHeader title={title} />
            <View style={{ gap: 16 }}>
              {list.map((t) => {
                const d = destinationById(t.destinationId);
                const isActive = t.id === activeId;
                return (
                  <Card key={t.id} plated={isActive} padded={false} onPress={() => open(t)} accessibilityLabel={`${t.name}${isActive ?
', current trip' : ''}`}>
                    <View style={styles.row}>
                      <Image source={imageSource(t.cover)} style={styles.cover} contentFit="cover" accessibilityIgnoresInvertColors />
                      <View style={styles.text}>
                        {isActive ? <Stamp label="Current" tone="accent" icon="checkmark" rotate={0} /> : null}
                        <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
                          {d ? `${d.city}, ${d.country}` : 'Somewhere great'}
                        </T>
                        <T variant="h3" numberOfLines={2}>
                          {t.name}
                        </T>
                        <T variant="small" color={colors.textSecondary}>
                          {rangeLabel(t.startDate, t.endDate)}
                        </T>
                        <View style={styles.meta}>
                          <AvatarStack people={t.members} max={4} size={26} />
                          <T variant="kicker" color={colors.ink} numberOfLines={1} style={{ flexShrink: 1 }}>
                            {money(totalSpent(t))} / {money(t.budget)}
                          </T>
                        </View>
                      </View>
                    </View>
                  </Card>
                );
              })}
            </View>
          </View>
        );
      })}
    </Screen>
  );
```

```
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', gap: 14, padding: 12 },
  cover: { width: 92, borderRadius: radius.chip, borderWidth: line, borderColor: colors.ink, alignSelf: 'stretch', minHeight: 112 },
  text: { flex: 1, gap: 3, justifyContent: 'center' },
  meta: { flexDirection: 'row', alignItems: 'center', gap: 10, marginTop: 6, flexWrap: 'wrap' },
});
```

src/app/weather.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { WeatherBoard } from '@/components/more/WeatherBoard';
import { Button } from '@/components/ui/Button';
import { Card, Chip, SectionHeader } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { Skeleton } from '@/components/ui/Skeleton';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useWeather } from '@/hooks/useTrip';
import { cToF, fetchWeather, weatherInfo } from '@/services/weather';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, hair, line, radius, space } from '@/theme';
import { timeAgo } from '@/utils/format';
import { dayLabel, weekdayShort } from '@/utils/time';

/** Today: one huge mono number and the sky it belongs to, then humidity, wind and rain in three small columns. */
function Today({ w, unitF }: { w: NonNullable<ReturnType<typeof useWeather>>; unitF: boolean }) {
  const info = weatherInfo(w.current.code);
  const temp = unitF ? cToF(w.current.temp) : w.current.temp;
  return (
    <View>
      <T variant="kicker" color={colors.textSecondary}>
        Right now
      </T>
      <View style={styles.big} accessible accessibilityLabel={`${temp} degrees ${unitF ? 'Fahrenheit' : 'Celsius'}, ${info.label}`}>
        <T variant="hero" style={{ flexShrink: 1 }}>
          {temp}°
        </T>
        <Ionicons name={info.icon} size={76} color={colors.ink} />
      </View>
      <T variant="headline" accessibilityRole="header">
        {info.label}
      </T>
      <View style={styles.stats}>
        {[
          ['Humidity', `${w.current.humidity}`, '%'],
          ['Wind', `${w.current.wind}`, ' km/h'],
          ['Rain today', `${w.daily[0]?.rain ?? 0}`, '%'],
        ].map(([label, value, unit], i) => (
          <View key={label} style={[styles.stat, i > 0 && styles.statLine]} accessible accessibilityLabel={`${label} ${value}${unit}`}>
            <T variant="kicker" color={colors.textSecondary}>
              {label}
            </T>
            <T variant="numeralSm">
              {value}
              <T variant="kicker" color={colors.textSecondary}>
                {unit.toUpperCase()}
              </T>
            </T>
          </View>
        ))}
      </View>
    </View>
  );
}

/** The same shape as the real thing, so nothing jumps when the forecast arrives. */
function WeatherSkeleton() {
  return (
    <View style={{ gap: space.xxl }} accessibilityLabel="Loading the forecast">
      <View style={{ gap: space.md }}>
        <Skeleton width={90} height={14} />
        <Skeleton height={76} />
        <Skeleton width={200} height={28} />
      </View>
      <Skeleton height={430} />
    </View>
  );
}
```

```
export default function Weather() {
  const params = useLocalSearchParams<{ id?: string }>();
  const trip = useActiveTrip();
  const options = trip ? [trip.destinationId, ...trip.extraDestinationIds] : [];
  const [id, setId] = useState(params.id ?? trip?.destinationId ?? 'tokyo');
  const [unitF, setUnitF] = useState(false);
  const [loading, setLoading] = useState(false);
  const addPacking = useAppStore((s) => s.addPackingItems);
  const w = useWeather(id);
  const d = destinationById(id)!;
  const u = (c: number) => (unitF ? `${cToF(c)}°F` : `${c}°C`);
  const rainy = w?.daily.filter((x) => x.rain >= 50) ?? [];
  const best = w?.daily.slice().sort((a, b) => a.rain - b.rain || b.max - a.max)[0];
  const maxT = Math.max(...(w?.daily.map((x) => x.max) ?? [30]));
  const minT = Math.min(...(w?.daily.map((x) => x.min) ?? [0]));

  const tips: string[] = [];
  if (w) {
    if (rainy.length) tips.push('Compact umbrella or rain jacket');
    if (maxT >= 27) tips.push('Sunscreen, hat & refillable water bottle');
    if (minT <= 12) tips.push('Warm layer for evenings');
    if (minT <= 3) tips.push('Gloves & beanie');
    tips.push('Comfortable walking shoes');
  }

  return (
    <Screen
      header={<Header kicker={`${d.city}, ${d.country}`} title="Weather" subtitle={w ? `${w.live ? 'Live forecast' : 'Estimated (offlin
e)'} · updated ${timeAgo(w.fetchedAt)}` : 'Loading…'} backLabel="More" />}
      onRefresh={async () => {
        setLoading(true);
        const r = await fetchWeather(id, true);
        setLoading(false);
        toast(r?.live ? 'Forecast updated' : 'Offline. Showing the last saved forecast', { tone: r?.live ? 'ok' : 'warn' });
      }}
      refreshing={loading}>
      {options.length > 1 ? (
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
          {options.map((o) => (
            <Chip key={o} label={destinationById(o)?.city ?? o} active={o === id} onPress={() => setId(o)} />
          ))}
        </ScrollView>
      ) : null}

      {w ? (
        <>
          <Today w={w} unitF={unitF} />

          <View style={styles.statusRow}>
            {w.live ? <Stamp label="Live · Open-Meteo" tone="positive" icon="radio-button-on" rotate={0} /> : <Stamp label="Estimate" to
ne="warn" icon="cloud-offline-outline" rotate={0} />}
            <View style={{ flex: 1 }} />
            <Chip label="°C" active={!unitF} onPress={() => setUnitF(false)} />
            <Chip label="°F" active={unitF} onPress={() => setUnitF(true)} />
          </View>
          {!w.live ? (
            <T variant="caption" color={colors.textSecondary} style={{ marginTop: space.sm }}>
              No connection, so this is a typical week for {d.city}. Pull down to refresh when you are back online.
            </T>
          ) : null}

          <View style={{ marginTop: space.xl }}>
            <WeatherBoard daily={w.daily} city={d.city} unitF={unitF} />
          </View>

          {best ? (
            <Card style={styles.best}>
              <View style={styles.bestIcon}>
                <Ionicons name="sunny-outline" size={24} color={colors.ink} />
              </View>
              <View style={{ flex: 1 }}>
                <T variant="kicker" color={colors.textSecondary}>
                  Best outdoor day
                </T>
                <T variant="numeralSm">
                  {weekdayShort(best.date)} {dayLabel(best.date)}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  {best.rain}% rain, up to {u(best.max)}. Plan hikes and day trips here.
                </T>
              </View>
            </Card>
          ) : null}

          <View style={{ marginTop: space.section }}>
```

```
            <SectionHeader title="What to pack" />
            <View>
              {tips.map((t, i) => (
                <View key={t} style={[styles.tip, i === 0 && { borderTopWidth: 0 }]}>
                  <Ionicons name="checkmark" size={18} color={colors.ink} />
                  <T variant="bodySm" style={{ flex: 1 }}>
                    {t}
                  </T>
                </View>
              ))}
            </View>
            {trip ? (
              <Button
                label="Add these to the packing list"
                icon="briefcase-outline"
                style={{ marginTop: space.xl }}
                onPress={() => {
                  addPacking(trip.id, tips.map((text) => ({ text, category: 'Weather' })));
                  toast('Added to the shared packing list');
                }}
              />
            ) : null}
          </View>
        </>
      ) : (
        <WeatherSkeleton />
      )}
    </Screen>
  );
}

const styles = StyleSheet.create({
  chipScroll: { marginHorizontal: -20, marginBottom: space.xl, flexGrow: 0 },
  chips: { gap: space.sm, paddingHorizontal: 20, paddingRight: 28 },
  big: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: space.md, marginTop: 2, marginBottom: space.x
s },
  stats: { flexDirection: 'row', marginTop: space.lg, borderTopWidth: hair, borderTopColor: colors.border, paddingTop: space.md },
  stat: { flex: 1, gap: 2, paddingLeft: 0 },
  statLine: { paddingLeft: space.md, borderLeftWidth: hair, borderLeftColor: colors.border },
  statusRow: { flexDirection: 'row', alignItems: 'center', gap: space.sm, marginTop: space.xl },
  best: { flexDirection: 'row', alignItems: 'center', gap: space.md, marginTop: space.lg, backgroundColor: colors.accentSoft },
  bestIcon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.ca
rd, alignItems: 'center', justifyContent: 'center' },
  tip: { flexDirection: 'row', alignItems: 'center', gap: space.md, minHeight: 48, paddingVertical: space.sm, borderTopWidth: hair, bord
erTopColor: colors.border },
});
```

src/components/CurrencyConverter.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { Platform, StyleSheet, TextInput, View } from 'react-native';

import { convert, useRates } from '@/services/currency';
import { colors, fonts, line, radius } from '@/theme';
import { money } from '@/utils/format';
import { parseAmount } from '@/utils/validation';

import { CurrencyPickerSheet } from './CurrencyPickerSheet';
import { IconButton } from './ui/bits';
import { Press } from './ui/Press';
import { Stamp } from './ui/Stamp';
import { T } from './ui/T';

/**
 * Two-way converter used on the Budget tab and the Currency screen. Two outlined boxes like the lines of an exchange board: what you ha
ve
 * (type it) and what you get (a big mono number), each with its currency on the right, and a square swap button between them.
 */
export function CurrencyConverter({ from, to, onChange }: { from: string; to: string; onChange: (from: string, to: string) => void }) {
  const rates = useRates();
  const [amount, setAmount] = useState('50');
  const [picking, setPicking] = useState<'from' | 'to' | null>(null);
  const n = parseAmount(amount);
  const out = n === null ? null : convert(n, from, to, rates);
  const unit = convert(1, from, to, rates);
  const shown = out === null ? '—' : money(out, to, { decimals: true });

  const code = (which: 'from' | 'to', c: string) => (
    <Press onPress={() => setPicking(which)} style={styles.code} accessibilityLabel={`Change ${which} currency, currently ${c}`}>
      <T variant="kicker" color={colors.ink} style={{ fontSize: 14 }}>
        {c}
      </T>
      <Ionicons name="chevron-down" size={16} color={colors.ink} />
    </Press>
  );
```

```
  return (
    <View style={{ gap: 8 }}>
      <T variant="kicker" color={colors.textSecondary}>
        You have
      </T>
      <View style={[styles.box, n === null && !!amount && { borderColor: colors.red, backgroundColor: colors.redSoft }]}>
        <TextInput
          value={amount}
          onChangeText={(v) => setAmount(v.replace(/[^0-9.,]/g, ''))}
          keyboardType="decimal-pad"
          style={[styles.input, Platform.OS === 'web' && ({ outlineStyle: 'none' } as object)]}
          accessibilityLabel={`Amount in ${from}`}
          maxLength={12}
          placeholder="0"
          placeholderTextColor={colors.textMuted}
        />
        {code('from', from)}
      </View>

      <View style={styles.between}>
        <T variant="kicker" color={colors.textSecondary}>
          You get
        </T>
        <IconButton icon="swap-vertical" label="Swap currencies" onPress={() => onChange(to, from)} />
      </View>
      <View style={[styles.box, { backgroundColor: colors.bgAlt }]}>
        <T variant="numeral" numberOfLines={1} style={{ flex: 1, minWidth: 0, fontSize: shown.length > 11 ? 20 : shown.length > 9 ? 24 :
28 }}>
          {shown}
        </T>
        {code('to', to)}
      </View>

      {n === null && amount ? (
        <T variant="caption" color={colors.red}>
          Enter a number, e.g. 25 or 25.50
        </T>
      ) : null}
      <View style={styles.rateRow}>
        <T variant="caption" color={colors.textSecondary} style={{ flexShrink: 1 }}>
          1 {from} = {unit < 0.01 ? unit.toFixed(5) : unit.toFixed(unit > 100 ? 1 : 4)} {to}
        </T>
        <Stamp label={rates.live ? 'Live rate' : 'Offline rate'} tone={rates.live ? 'positive' : 'muted'} icon={rates.live ? 'radio-outl
ine' : 'cloud-offline-outline'} rotate={0} />
      </View>
      <CurrencyPickerSheet
        visible={!!picking}
        onClose={() => setPicking(null)}
        value={picking === 'from' ? from : to}
        onPick={(c) => (picking === 'from' ? onChange(c, to) : onChange(from, c))}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  box: { minHeight: 64, flexDirection: 'row', alignItems: 'center', borderRadius: radius.button, borderWidth: line, borderColor: colors.
ink, backgroundColor: colors.card, paddingLeft: 14, overflow: 'hidden' },
  // a text field cannot use the T component, so the mono face is named here
  input: { flex: 1, minWidth: 0, fontFamily: fonts.mono, fontSize: 28, color: colors.text, paddingVertical: 8 },
  code: { alignSelf: 'stretch', minWidth: 96, flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, paddingHoriz
ontal: 14, marginLeft: 8, borderLeftWidth: line, borderLeftColor: colors.ink, backgroundColor: colors.primarySoft },
  between: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', paddingTop: 6, paddingBottom: 4 },
  rateRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 10, paddingTop: 4 },
});
```

src/components/CurrencyPickerSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { currencies } from '@/data/reference';
import { colors, hair, line, radius } from '@/theme';

import { Input } from './ui/Input';
import { Press } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { T } from './ui/T';

/**
 * Pick a currency: search, then tap a row. A row is a code in mono, the name, and a symbol in a small square; the chosen one is filled
 * with the accent and has a check.
 */
export function CurrencyPickerSheet({ visible, onClose, value, onPick, title = 'Choose currency' }: { visible: boolean; onClose: () => v
```

```
oid; value: string; onPick: (code: string) => void; title?: string }) {
  const [q, setQ] = useState('');
  const needle = q.trim().toLowerCase();
  const list = currencies.filter((c) => !needle || c.code.toLowerCase().includes(needle) || c.name.toLowerCase().includes(needle));
  return (
    <Sheet visible={visible} onClose={onClose} title={title}>
      <Input icon="search-outline" value={q} onChangeText={setQ} placeholder="Search currency" autoCapitalize="none" accessibilityLabel
="Search currency" style={{ minHeight: 44 }} />
      <View style={styles.list}>
        {list.map((c, i) => {
          const on = c.code === value;
          return (
            <Press
              key={c.code}
              onPress={() => {
                onPick(c.code);
                onClose();
              }}
              accessibilityLabel={`${c.code}, ${c.name}`}
              accessibilityState={{ selected: on }}
              scaleTo={0.99}
              style={[styles.row, i > 0 && styles.divider, on && { backgroundColor: colors.accent }]}>
              <View style={[styles.sym, on && { backgroundColor: colors.card }]}>
                <T variant="kicker" color={colors.ink} style={{ fontSize: 12, letterSpacing: 0 }}>
                  {c.symbol.length > 3 ? c.code.slice(0, 1) : c.symbol}
                </T>
              </View>
              <T variant="kicker" color={on ? colors.onAccent : colors.ink} style={{ width: 44, fontSize: 14 }}>
                {c.code}
              </T>
              <T variant="small" color={on ? colors.onAccent : colors.textSecondary} style={{ flex: 1 }} numberOfLines={1}>
                {c.name}
              </T>
              {on ? <Ionicons name="checkmark" size={18} color={colors.onAccent} /> : null}
            </Press>
          );
        })}
        {!list.length ? (
          <T variant="small" color={colors.textSecondary} style={{ padding: 16 }}>
            No currency matches “{q}”.
          </T>
        ) : null}
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  list: { borderWidth: line, borderColor: colors.ink, borderRadius: radius.card, backgroundColor: colors.card, overflow: 'hidden' },
  row: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 52, paddingHorizontal: 12, paddingVertical: 6 },
  divider: { borderTopWidth: hair, borderTopColor: colors.border },
  sym: { minWidth: 34, height: 34, paddingHorizontal: 4, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, background
Color: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/GuidedTour.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useRef, useState } from 'react';
import { Animated, Modal, ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, line, radius } from '@/theme';

import { Button } from './ui/Button';
import type { IconName } from './ui/bits';
import { FadeIn, nativeDriver, useMotion } from './ui/motion';
import { Press } from './ui/Press';
import { T } from './ui/T';
import { Ticket } from './ui/Ticket';

const STEPS: { icon: IconName; tab: string; title: string; body: string; points: string[] }[] = [
  { icon: 'home-outline', tab: 'Home', title: 'Your trip at a glance', body: 'The moment you open the app you see the hero photo, invite
code, weather and your group budget.', points: ['Up next and countdown', 'Weather and local time', 'Budget progress'] },
  { icon: 'calendar-outline', tab: 'Plan', title: 'Plan it together', body: 'Build the itinerary day by day. Everyone votes “I’m in”, th
en the group locks the winners in.', points: ['Itinerary by day', 'Vote on ideas', 'Lock in the plan'] },
  { icon: 'wallet-outline', tab: 'Budget', title: 'Split it fairly', body: 'Log expenses in any currency and see exactly who owes who. S
nap a receipt so nothing gets lost.', points: ['Who owes who', 'Receipts and categories', 'Settle up in fewer payments'] },
  { icon: 'chatbubbles-outline', tab: 'Chat', title: 'Talk it out', body: 'Message the crew in one place, or ask Scout, the built-in AI,
about weather, food and what’s next.', points: ['Group chat', 'Scout AI assistant', 'Works offline'] },
  { icon: 'grid-outline', tab: 'More', title: 'Everything else', body: 'Offline documents, the trip map, a packing list, the photo dump,
a trip quiz and more live here.', points: ['Offline docs', 'Trip map', 'Packing, photos and quiz'] },
];

/** First-run walkthrough: five swipeable boarding passes, one per tab. Marks the tour as seen when it closes. */
```

```
export function GuidedTour({ visible, onClose }: { visible: boolean; onClose: () => void }) {
  const setToured = useAppStore((s) => s.setToured);
  const finish = () => {
    setToured(true);
    onClose();
  };
  return (
    <Modal visible={visible} animationType="fade" onRequestClose={finish} statusBarTranslucent>
      {visible ? <TourBody onFinish={finish} /> : null}
    </Modal>
  );
}

function TourBody({ onFinish }: { onFinish: () => void }) {
  const insets = useSafeAreaInsets();
  const { width } = useWindowDimensions();
  const motion = useMotion();
  const pageW = Math.min(width, MAX_WIDTH);
  const scroller = useRef<ScrollView>(null);
  const [index, setIndex] = useState(0);
  const [scrollX] = useState(() => new Animated.Value(0));
  // the height the pages may use, so each ticket sits in the middle of it
  const [pageH, setPageH] = useState(0);
  const last = index === STEPS.length - 1;

  const goTo = (i: number) => {
    setIndex(i);
    scroller.current?.scrollTo({ x: i * pageW, animated: motion });
  };

  return (
    <View style={[styles.root, { paddingTop: insets.top + 8, paddingBottom: Math.max(insets.bottom, 16) }]}>
      <View style={[styles.column, { width: pageW }]}>
        <View style={styles.top}>
          <T variant="kicker" color={colors.textSecondary} accessibilityLiveRegion="polite">
            Step {index + 1} of {STEPS.length}
          </T>
          {last ? null : (
            <Press onPress={onFinish} accessibilityLabel="Skip the tour" style={styles.skip}>
              <T variant="small" weight="bold" style={{ textDecorationLine: 'underline' }}>
                Skip
              </T>
            </Press>
          )}
        </View>

        <FadeIn style={{ flex: 1 }} duration={420} distance={16}>
          <Animated.ScrollView
            ref={scroller}
            horizontal
            pagingEnabled
            showsHorizontalScrollIndicator={false}
            scrollEventThrottle={16}
            onScroll={Animated.event([{ nativeEvent: { contentOffset: { x: scrollX } } }], {
              useNativeDriver: nativeDriver,
              listener: (e: { nativeEvent: { contentOffset: { x: number } } }) => setIndex(Math.max(0, Math.min(STEPS.length - 1, Math.r
ound(e.nativeEvent.contentOffset.x / pageW)))),
            })}
            onLayout={(e) => setPageH(e.nativeEvent.layout.height)}
            style={{ flex: 1 }}>
            {STEPS.map((s, i) => {
              const range = [(i - 1) * pageW, i * pageW, (i + 1) * pageW];
              const pop = motion ? { opacity: scrollX.interpolate({ inputRange: range, outputRange: [0.5, 1, 0.5], extrapolate: 'clamp'
}), transform: [{ scale: scrollX.interpolate({ inputRange: range, outputRange: [0.94, 1, 0.94], extrapolate: 'clamp' }) }] } : null;
              return (
                <View key={s.tab} style={[styles.page, { width: pageW, height: pageH || undefined }]} accessible accessibilityLabel={`St
ep ${i + 1} of ${STEPS.length}. ${s.tab}. ${s.title}. ${s.body}. ${s.points.join('. ')}`}>
                  <Animated.View style={pop}>
                    <Ticket
                      stub={
                        <View style={styles.points}>
                          {s.points.map((p) => (
                            <View key={p} style={styles.point}>
                              <View style={styles.tick}>
                                <Ionicons name="checkmark" size={14} color={colors.onAccent} />
                              </View>
                              <T variant="bodySm" weight="semibold" style={{ flex: 1 }}>
                                {p}
                              </T>
                            </View>
                          ))}
                        </View>
                      }>
                      <View style={styles.main}>
                        <View style={styles.row}>
                          <View style={styles.icon}>
```

```
                        <Ionicons name={s.icon} size={28} color={colors.onAccent} />
                      </View>
                      <View style={{ flex: 1 }}>
                        <T variant="kicker" color={colors.textSecondary}>
                          Tab
                        </T>
                        <T variant="h2">{s.tab}</T>
                      </View>
                    </View>
                    <T variant="display" style={{ fontSize: 34, lineHeight: 38 }}>
                      {s.title}
                    </T>
                    <T variant="body" color={colors.textSecondary}>
                      {s.body}
                    </T>
                  </View>
                </Ticket>
              </Animated.View>
            </View>
          );
        })}
      </Animated.ScrollView>
    </FadeIn>

    <View style={styles.steps} accessibilityRole="tablist">
      {STEPS.map((s, i) => (
        <Press key={s.tab} onPress={() => goTo(i)} haptic={false} scaleTo={1} style={styles.stepHit} accessibilityRole="tab" accessi
bilityState={{ selected: i === index }} accessibilityLabel={`Step ${i + 1} of ${STEPS.length}: ${s.tab}`}>
          <View style={[styles.step, i <= index && { backgroundColor: colors.accent }]} />
        </Press>
      ))}
    </View>
    <View style={{ paddingHorizontal: 20 }}>
      <Button label={last ? 'Got it' : 'Next'} iconRight={last ? 'checkmark' : 'arrow-forward'} onPress={() => (last ? onFinish() :
goTo(index + 1))} />
    </View>
  </View>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  column: { flex: 1 },
  top: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', paddingHorizontal: 20, height: 48 },
  skip: { minWidth: 64, height: 44, alignItems: 'flex-end', justifyContent: 'center' },
  page: { paddingHorizontal: 20, justifyContent: 'center' },
  main: { padding: 20, gap: 14 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 14 },
  icon: { width: 56, height: 56, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.accen
t, alignItems: 'center', justifyContent: 'center' },
  points: { gap: 12, paddingHorizontal: 20, paddingBottom: 18, paddingTop: 4 },
  point: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  tick: { width: 24, height: 24, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.accent, al
ignItems: 'center', justifyContent: 'center' },
  steps: { flexDirection: 'row', gap: 8, paddingHorizontal: 20, paddingTop: 8, paddingBottom: 4 },
  stepHit: { flex: 1, height: 44, justifyContent: 'center' },
  step: { height: 12, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
});
```

src/components/Logo.tsx

```
import { MaterialCommunityIcons } from '@expo/vector-icons';
import { View } from 'react-native';

import { colors, radius } from '@/theme';

import { T } from './ui/T';

/**
 * The brand mark: a square of ink (cream at night) with a take-off plane in paper, and one corner of the theme's accent, like a
 * ticket with its corner clipped. Every colour is a token, so it reads on the paper page, on a photo and in the dark mode.
 * The thin paper-coloured lines around the accent corner keep it apart from the ink in every theme.
 */
export function LogoMark({ size = 40 }: { size?: number }) {
  const corner = Math.round(size * 0.3);
  const gap = Math.max(1.5, Math.round(size / 20));
  return (
    <View
      accessibilityRole="image"
      accessibilityLabel="TogetherWeGo logo"
      style={{ width: size, height: size, borderRadius: Math.min(radius.xs, size * 0.2), backgroundColor: colors.primary, alignItems: 'c
enter', justifyContent: 'center', overflow: 'hidden' }}>
      <MaterialCommunityIcons name="airplane-takeoff" size={Math.round(size * 0.54)} color={colors.onPrimary} style={{ marginTop: size *
0.08, marginRight: size * 0.04 }} />
      <View
```

```
        style={{ position: 'absolute', top: 0, right: 0, width: corner + gap, height: corner + gap, backgroundColor: colors.accent, bord
erLeftWidth: gap, borderBottomWidth: gap, borderColor: colors.bg }}
      />
    </View>
  );
}

/** The mark and the name in the serif of the page titles (headline size, never smaller). `size` is the height of the mark. */
export function Logo({ size = 44 }: { size?: number }) {
  return (
    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 12 }}>
      <LogoMark size={size} />
      <T variant="headline" importantForAccessibility="no" style={{ flexShrink: 1 }}>
        TogetherWeGo
      </T>
    </View>
  );
}
```

src/components/QrCode.tsx

```
import { useMemo } from 'react';
import Svg, { Path, Rect } from 'react-native-svg';

import { colors } from '@/theme';
import { qrMatrix, qrPath } from '@/utils/qr';

const QUIET = 4; // squares of empty border all around; scanners need it to find the code

/** A QR code drawn with SVG. Always black on white, whatever the app theme, because that is what scanners read best. */
export function QrCode({ value, size = 220, accessibilityLabel }: { value: string; size?: number; accessibilityLabel?: string }) {
  const { total, path } = useMemo(() => {
    const matrix = qrMatrix(value);
    return { total: matrix.length + QUIET * 2, path: qrPath(matrix) };
  }, [value]);
  const px = Math.max(1, Math.floor(size / total)) * total; // a whole number of screen points per square keeps the edges sharp
  return (
    <Svg width={px} height={px} viewBox={`${-QUIET} ${-QUIET} ${total} ${total}`} accessibilityRole="image" accessibilityLabel={accessib
ilityLabel}>
      <Rect x={-QUIET} y={-QUIET} width={total} height={total} fill={colors.white} />
      <Path d={path} fill={colors.black} />
    </Svg>
  );
}
```

src/components/ReminderService.tsx

```
import { useEffect, useMemo } from 'react';
import { AppState } from 'react-native';

import { checkDueReminders, syncReminders } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';

/**
 * Invisible component that keeps reminders live:
 * - re-schedules OS notifications whenever reminder-relevant data changes
 * - polls every 20 s while the app is open to surface due reminders in-app
 */
export function ReminderService() {
  const trips = useAppStore((s) => s.trips);
  const lead = useAppStore((s) => s.settings.reminderLeadMin);
  const enabled = useAppStore((s) => s.settings.notificationsEnabled);
  const userId = useAppStore((s) => s.session?.userId);

  const signature = useMemo(
    () =>
      [lead, enabled, userId]
        .concat(trips.flatMap((t) => t.activities.filter((a) => a.reminder).map((a) => `${a.id}|${a.date}|${a.time}`)))
        .join(','),
    [trips, lead, enabled, userId],
  );

  useEffect(() => {
    const timer = setTimeout(() => {
      syncReminders().catch(() => {});
    }, 1200);
    return () => clearTimeout(timer);
  }, [signature]);

  useEffect(() => {
    checkDueReminders();
    const id = setInterval(checkDueReminders, 20000);
    const sub = AppState.addEventListener('change', (st) => st === 'active' && checkDueReminders());
    return () => {
      clearInterval(id);
      sub.remove();
```

```
    };
  }, []);

  return null;
}
```

src/components/ShareSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { copy, shareText, shareTo, socialTargets } from '@/services/share';
import { colors, line, radius } from '@/theme';

import { Button } from './ui/Button';
import { PlatePress } from './ui/Plate';
import { Sheet } from './ui/Sheet';
import { T } from './ui/T';

/**
 * Share picker: the message on a dashed paper slip, then a square tile for each app that can be opened straight from here
 * (the tiles are ink on paper like everything else; the icons say which app), then Copy and the phone's own share sheet.
 */
export function ShareSheet({ visible, onClose, title, message, preview }: { visible: boolean; onClose: () => void; title: string; messag
e: string; preview?: string }) {
  return (
    <Sheet visible={visible} onClose={onClose} title={title} subtitle="Send it straight to your group’s favourite app">
      {preview ? (
        <View style={styles.preview}>
          <T variant="small" color={colors.textSecondary}>
            {preview}
          </T>
        </View>
      ) : null}
      <View style={styles.grid}>
        {socialTargets.map((s) => (
          <View key={s.key} style={styles.item}>
            <PlatePress
              onPress={() => {
                onClose();
                shareTo(s.key, message);
              }}
              offset={3}
              corner={radius.button}
              style={styles.tile}
              accessibilityLabel={`Share via ${s.label}`}>
              <Ionicons name={s.icon} size={24} color={colors.ink} />
            </PlatePress>
            <T variant="kicker" color={colors.textSecondary} numberOfLines={1} importantForAccessibility="no">
              {s.label}
            </T>
          </View>
        ))}
      </View>
      <View style={{ flexDirection: 'row', gap: 12 }}>
        <Button label="Copy" variant="secondary" icon="copy-outline" size="md" onPress={() => copy(message)} style={{ flex: 1 }} />
        <Button
          label="More apps"
          icon="share-outline"
          size="md"
          onPress={() => {
            onClose();
            shareText(message);
          }}
          style={{ flex: 1 }}
        />
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  preview: { backgroundColor: colors.bgAlt, borderRadius: radius.chip, padding: 12, borderWidth: line, borderStyle: 'dashed', borderColo
r: colors.ink },
  grid: { flexDirection: 'row', flexWrap: 'wrap', rowGap: 18 },
  item: { width: '25%', alignItems: 'center', gap: 8 },
  tile: { width: 56, height: 56, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.card, borderWidth: line, border
Color: colors.ink, borderRadius: radius.button },
});
```

src/components/SkylineHero.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { useEffect, useState, type ReactNode } from 'react';
```

```
import { Animated, Easing, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { nativeDriver, useLoop, useMotion } from '@/components/ui/motion';
import { PlatePress } from '@/components/ui/Plate';
import { imageSource } from '@/data/images';
import { useNow } from '@/hooks/useTrip';
import { colors, line, radius } from '@/theme';
import { partsInTz } from '@/utils/time';

export type TimeOfDay = 'day' | 'night';

/** Hours 19:00–05:59 at the destination count as night. */
export function timeOfDayAt(tz: string, now: number): TimeOfDay {
  const h = Number(partsInTz(now, tz).hhmm.slice(0, 2));
  return h >= 19 || h < 6 ? 'night' : 'day';
}

/** Deterministic pseudo-random list so stars sit in the same place every render. */
const STARS = Array.from({ length: 30 }, (_, i) => {
  const a = Math.sin(i * 91.7) * 43758.5453;
  const b = Math.sin(i * 12.9898 + 4.1) * 24634.6345;
  return { x: (a - Math.floor(a)) * 100, y: (b - Math.floor(b)) * 52, size: 2 + ((i * 7) % 3), group: i % 3 };
});

/** How far the photo slides down while the page scrolls up (the photo is that much taller than its frame, so no gap shows). */
const PARALLAX = 44;
/** The slow zoom: from the photo as framed to 8% closer. */
const ZOOM = 1.08;

/**
 * The photo at the top of Home. It follows the day and night of the trip's city: by day a flat sun and drifting clouds; by
 * night a dark blue wash with square stars and a moon. The photo zooms very slowly (1.0 to 1.08) and, when `scrollY` is
 * given, slides against the scroll for depth. Both stop when animations are off. The outlined square at the right previews the other ha
lf of the day.
 * Children are drawn on top (the controls).
 */
export function SkylineHero({ image, tz, height, scrollY, children }: { image: string; tz: string; height: number; scrollY?: Animated.Va
lue; children?: ReactNode }) {
  const insets = useSafeAreaInsets();
  const motion = useMotion();
  const now = useNow(60000);
  const auto = timeOfDayAt(tz, now);
  const [forced, setForced] = useState<TimeOfDay | null>(null);
  const mode = forced ?? auto;
  const [night] = useState(() => new Animated.Value(auto === 'night' ? 1 : 0));

  useEffect(() => {
    const to = mode === 'night' ? 1 : 0;
    if (!motion) {
      night.setValue(to);
      return;
    }
    const anim = Animated.timing(night, { toValue: to, duration: 900, easing: Easing.inOut(Easing.cubic), useNativeDriver: nativeDriver
});
    anim.start();
    return () => anim.stop();
  }, [mode, night, motion]);

  const zoom = useLoop(20000, { pingPong: true, rest: 0 });
  const twinkle = [useLoop(1700, { pingPong: true, rest: 0.6 }), useLoop(2300, { pingPong: true, delay: 400, rest: 0.6 }), useLoop(2900,
{ pingPong: true, delay: 900, rest: 0.6 })];
  const cloudA = useLoop(30000, { rest: 0.2 });
  const cloudB = useLoop(44000, { delay: 3000, rest: 0.7 });

  const dayOpacity = night.interpolate({ inputRange: [0, 1], outputRange: [1, 0] });
  const slide = motion && scrollY ? scrollY.interpolate({ inputRange: [0, height], outputRange: [0, PARALLAX], extrapolate: 'clamp' }) :
0;

  return (
    <View style={{ height, overflow: 'hidden', backgroundColor: colors.ink }}>
      <Animated.View
        testID="hero-photo"
        style={{ position: 'absolute', left: 0, right: 0, top: -PARALLAX, height: height + PARALLAX, transform: [{ translateY: slide },
{ scale: zoom.interpolate({ inputRange: [0, 1], outputRange: [1, ZOOM] }) }] }}>
        <Image source={imageSource(image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={300} accessibilityIgnoresInver
tColors />
      </Animated.View>

      {/* Day: a flat sun and two clouds */}
      <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { opacity: dayOpacity }]}>
        <View style={styles.sun} />
        <Animated.View style={[styles.cloud, { top: height * 0.17, transform: [{ translateX: cloudA.interpolate({ inputRange: [0, 1], ou
tputRange: [-120, 520] }) }] }]}>
          <Ionicons name="cloud" size={60} color={colors.white} style={{ opacity: 0.55 }} />
        </Animated.View>
```

```
        <Animated.View style={[styles.cloud, { top: height * 0.31, transform: [{ translateX: cloudB.interpolate({ inputRange: [0, 1], ou
tputRange: [520, -140] }) }] }]}>
          <Ionicons name="cloud" size={42} color={colors.white} style={{ opacity: 0.4 }} />
        </Animated.View>
      </Animated.View>

      {/* Night: a dark blue wash (a flat overlay on the photo), square stars and a moon */}
      <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { opacity: night }]}>
        <View style={[StyleSheet.absoluteFill, { backgroundColor: 'rgba(6,10,42,0.62)' }]} />
        {STARS.map((s, i) => (
          <Animated.View key={i} style={[styles.star, { left: `${s.x}%`, top: `${s.y}%`, width: s.size, height: s.size, opacity: twinkle
[s.group].interpolate({ inputRange: [0, 1], outputRange: [0.3, 1] }) }]} />
        ))}
        <View style={styles.moon}>
          <View style={styles.moonBite} />
        </View>
      </Animated.View>

      {/* the one scrim: it keeps the controls above and the ticket below readable on any photo */}
      <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0.38)', 'rgba(0,0,0,0)', 'rgba(0,0,0,0.5)']} locations={[0, 0.4, 1]} sty
le={StyleSheet.absoluteFill} />

      {children}

      <PlatePress
        onPress={() => setForced(mode === 'night' ? 'day' : 'night')}
        onLongPress={() => setForced(null)}
        accessibilityLabel={mode === 'night' ? 'Showing night. Tap for day' : 'Showing day. Tap for night'}
        accessibilityHint="Long press to follow the local time again"
        offset={3}
        corner={radius.button}
        wrapperStyle={[styles.toggleWrap, { top: insets.top + 64 }]}
        style={styles.toggle}>
        <Ionicons name={mode === 'night' ? 'moon' : 'sunny'} size={20} color={colors.ink} />
        {forced ? null : <View style={styles.live} />}
      </PlatePress>
    </View>
  );
}

const styles = StyleSheet.create({
  sun: { position: 'absolute', right: 92, top: 84, width: 40, height: 40, borderRadius: 20, backgroundColor: colors.yellow, borderWidth:
line, borderColor: colors.black },
  cloud: { position: 'absolute', left: 0 },
  star: { position: 'absolute', backgroundColor: colors.white },
  // a flat crescent: a cream disc with a dark-blue disc biting into it
  moon: { position: 'absolute', right: 92, top: 84, width: 40, height: 40, borderRadius: 20, backgroundColor: colors.card, overflow: 'hi
dden', borderWidth: line, borderColor: colors.black },
  moonBite: { position: 'absolute', left: 14, top: -6, width: 34, height: 34, borderRadius: 17, backgroundColor: 'rgb(18,22,64)' },
  toggleWrap: { position: 'absolute', right: 19 },
  toggle: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
d, alignItems: 'center', justifyContent: 'center' },
  live: { position: 'absolute', top: -5, right: -5, width: 12, height: 12, backgroundColor: colors.accent, borderWidth: 2, borderColor:
colors.ink, borderRadius: 3 },
});
```

src/components/TabBar.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Tabs } from 'expo-router/js-tabs';
import { useEffect, useState, type ComponentProps } from 'react';
import { Animated, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { useT } from '@/i18n';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, line, radius } from '@/theme';

import type { IconName } from './ui/bits';
import { nativeDriver, useMotion } from './ui/motion';
import { Press } from './ui/Press';
import { T } from './ui/T';

type BottomTabBarProps = Parameters<NonNullable<ComponentProps<typeof Tabs>['tabBar']>>[0];

const TABS: Record<string, { label: string; icon: IconName; active: IconName }> = {
  explore: { label: 'nav.explore', icon: 'home-outline', active: 'home' },
  itinerary: { label: 'nav.itinerary', icon: 'calendar-outline', active: 'calendar' },
  budget: { label: 'nav.budget', icon: 'wallet-outline', active: 'wallet' },
  chat: { label: 'nav.chat', icon: 'chatbubble-outline', active: 'chatbubble' },
  profile: { label: 'nav.profile', icon: 'grid-outline', active: 'grid' },
};

/** Five tabs, Home first and More last (the ends of a row are the easiest to find). A highlighter-coloured sticker slides behind the ic
on of the tab you are on. */
export function TabBar({ state, navigation }: BottomTabBarProps) {
```

```
  const t = useT();
  const insets = useSafeAreaInsets();
  const trip = useActiveTrip();
  const lastSeen = useAppStore((s) => s.notifications.filter((n) => !n.read && n.kind === 'chat').length);
  const unreadChat = trip ? lastSeen : 0;
  const motion = useMotion();
  const visible = state.routes.filter((r) => TABS[r.name]);
  const activeSlot = Math.max(0, visible.findIndex((r) => r.key === state.routes[state.index]?.key));
  const [innerW, setInnerW] = useState(0);
  const [slide] = useState(() => new Animated.Value(activeSlot));
  const [pop] = useState(() => new Animated.Value(1));

  // The accent bar glides to the selected tab and the new icon does a little bounce.
  useEffect(() => {
    if (!motion) {
      slide.setValue(activeSlot);
      return;
    }
    Animated.spring(slide, { toValue: activeSlot, useNativeDriver: nativeDriver, friction: 8, tension: 90 }).start();
    pop.setValue(0.78);
    Animated.spring(pop, { toValue: 1, useNativeDriver: nativeDriver, friction: 4, tension: 160 }).start();
  }, [activeSlot, motion, slide, pop]);

  const slotW = innerW > 0 ? (innerW - 16) / Math.max(1, visible.length) : 0;
  const barW = 52;

  return (
    <View style={[styles.bar, { paddingBottom: Math.max(insets.bottom, 8) }]}>
      <View style={styles.inner} onLayout={(e) => setInnerW(e.nativeEvent.layout.width)}>
        {slotW > 0 ? (
          <Animated.View
            pointerEvents="none"
            style={[
              styles.indicator,
              { width: barW, left: 8 + (slotW - barW) / 2, transform: [{ translateX: slide.interpolate({ inputRange: [0, Math.max(1, vis
ible.length - 1)], outputRange: [0, slotW * Math.max(1, visible.length - 1)] }) }] },
            ]}
          />
        ) : null}
        {state.routes.map((route, index) => {
          const cfg = TABS[route.name];
          if (!cfg) return null;
          const focused = state.index === index;
          const onPress = () => {
            const event = navigation.emit({ type: 'tabPress', target: route.key, canPreventDefault: true });
            if (!focused && !event.defaultPrevented) navigation.navigate(route.name);
          };
          return (
            <Press
              key={route.key}
              onPress={onPress}
              accessibilityRole="tab"
              accessibilityState={{ selected: focused }}
              accessibilityLabel={t(cfg.label)}
              style={styles.item}
              scaleTo={0.92}>
              <Animated.View style={focused ? { transform: [{ scale: pop }] } : undefined}>
                <Ionicons name={focused ? cfg.active : cfg.icon} size={23} color={focused ? colors.onAccent : colors.textSecondary} />
                {route.name === 'chat' && unreadChat > 0 && !focused ? <View style={styles.dot} /> : null}
              </Animated.View>
              <T variant="kicker" color={focused ? colors.ink : colors.textSecondary} numberOfLines={1} style={{ fontSize: 10, letterSpa
cing: 0.8 }}>
                {t(cfg.label)}
              </T>
            </Press>
          );
        })}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  bar: { backgroundColor: colors.bg, borderTopWidth: line, borderTopColor: colors.ink, alignItems: 'center' },
  inner: { flexDirection: 'row', width: '100%', maxWidth: MAX_WIDTH, paddingHorizontal: 8, paddingTop: 6 },
  item: { flex: 1, alignItems: 'center', justifyContent: 'flex-start', gap: 6, height: 64, paddingTop: 9 },
  indicator: { position: 'absolute', top: 8, height: 34, borderRadius: radius.chip, backgroundColor: colors.accent, borderWidth: line, b
orderColor: colors.ink },
  dot: { position: 'absolute', top: -3, right: -5, width: 11, height: 11, borderRadius: 6, backgroundColor: colors.red, borderWidth: 2,
borderColor: colors.ink },
});
```

src/components/auth/DemoTicket.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View, useWindowDimensions } from 'react-native';
```

```
import { IconButton } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { copy } from '@/services/share';
import { DEMO_EMAIL, DEMO_PASSWORD } from '@/store/seed';
import { colors, line, radius } from '@/theme';

const ROWS = [
  { key: 'Email', value: DEMO_EMAIL },
  { key: 'Password', value: DEMO_PASSWORD },
] as const;

/**
 * The way in for judges and first-time visitors, drawn as a boarding pass with an accent-coloured main part: pressing the main part fil
ls the sign-in
 * form with Maya's account; under the tear line are the email and password in print, each with its own copy button. The copy buttons
 * sit beside the text, never inside another button (a button inside a button is invalid on the web).
 */
export function DemoTicket({ title, onFill, filled }: { title: string; onFill: () => void; filled?: boolean }) {
  // on a very narrow phone the email needs a smaller print to stay on one line
  const narrow = useWindowDimensions().width < 350;
  return (
    <Ticket
      stub={
        <View style={styles.stub}>
          {ROWS.map((r) => (
            <View key={r.key} style={styles.row}>
              <View style={{ flex: 1 }}>
                <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 10 }}>
                  {r.key}
                </T>
                <T variant="numeralSm" style={[styles.value, narrow && { fontSize: 12.5, letterSpacing: -0.5 }]} selectable>
                  {r.value}
                </T>
              </View>
              <IconButton icon="copy-outline" label={`Copy demo ${r.key.toLowerCase()}`} onPress={() => void copy(r.value, `${r.key} cop
ied`)} bordered={false} bg="transparent" />
            </View>
          ))}
        </View>
      }>
      <Press onPress={onFill} scaleTo={0.99} accessibilityLabel="Use the demo account" accessibilityHint="Fills in the email and passwor
d" style={styles.main}>
        <View style={{ flex: 1, gap: 6 }}>
          <View style={styles.top}>
            <T variant="kicker" color={colors.onAccent}>
              For judges and first looks
            </T>
          </View>
          <T variant="headline" color={colors.onAccent}>
            {title}
          </T>
          <T variant="small" color={colors.onAccent}>
            Maya’s Amalfi trip, already planned. One tap fills in the form.
          </T>
        </View>
        {filled ? (
          <View style={styles.done}>
            <Ionicons name="checkmark-circle" size={24} color={colors.onAccent} />
            <T variant="kicker" color={colors.onAccent} style={{ fontSize: 10 }}>
              Filled in
            </T>
          </View>
        ) : (
          <Ionicons name="arrow-forward" size={22} color={colors.onAccent} />
        )}
      </Press>
    </Ticket>
  );
}

const styles = StyleSheet.create({
  // the main part is the accent block (the highlighter); its top corners follow the ticket's outline
  main: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 16, minHeight: 56, backgroundColor: colors.accent, borderTopLeft
Radius: radius.card - line, borderTopRightRadius: radius.card - line },
  top: { flexDirection: 'row', alignItems: 'center' },
  done: { alignItems: 'center', gap: 2 },
  stub: { paddingLeft: 16, paddingRight: 6, paddingBottom: 8 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 56 },
  value: { fontSize: 14, lineHeight: 20, letterSpacing: -0.3 },
});
```

src/components/auth/GoogleSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Avatar, Card, Divider } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { Input } from '@/components/ui/Input';
import { Press } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { DEMO_EMAIL } from '@/store/seed';
import { colors, radius } from '@/theme';
import { validateEmail, validateName } from '@/utils/validation';

/** Simulated Google account chooser: a production build would use OAuth via expo-auth-session. */
export function GoogleSheet({ visible, onClose, onPick }: { visible: boolean; onClose: () => void; onPick: (email: string, name: string)
=> void }) {
  const [other, setOther] = useState(false);
  const [mail, setMail] = useState('');
  const [name, setName] = useState('');
  const [shown, setShown] = useState(false);
  const nameErr = shown ? validateName(name) : null;
  const mailErr = shown ? validateEmail(mail) : null;
  return (
    <Sheet visible={visible} onClose={onClose} title="Choose an account" subtitle="to continue to TogetherWeGo">
      <Card padded={false}>
        <Press onPress={() => onPick(DEMO_EMAIL, 'Maya Chen')} accessibilityLabel={`Maya Chen, ${DEMO_EMAIL}`} scaleTo={0.99} style={sty
les.row}>
          <Avatar name="Maya Chen" src="avatar-maya" size={40} />
          <View style={{ flex: 1 }}>
            <T variant="bodySm" weight="bold">
              Maya Chen
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {DEMO_EMAIL}
            </T>
          </View>
          <Ionicons name="arrow-forward" size={16} color={colors.ink} />
        </Press>
        <Divider />
        <Press onPress={() => setOther(true)} accessibilityLabel="Use another account" accessibilityState={{ expanded: other }} scaleTo=
{0.99} style={styles.row}>
          <View style={styles.plus}>
            <Ionicons name="person-add-outline" size={18} color={colors.ink} />
          </View>
          <T variant="bodySm" weight="bold" style={{ flex: 1 }}>
            Use another account
          </T>
          <Ionicons name={other ? 'remove' : 'add'} size={20} color={colors.ink} />
        </Press>
      </Card>
      {other ? (
        <View style={{ gap: 16 }}>
          <Input label="Name" value={name} onChangeText={setName} error={nameErr} autoCapitalize="words" icon={nameErr ? 'alert-circle'
: 'person-outline'} />
          <Input label="Google email" value={mail} onChangeText={setMail} error={mailErr} autoCapitalize="none" autoCorrect={false} keyb
oardType="email-address" icon={mailErr ? 'alert-circle' : 'mail-outline'} />
          <Button
            label="Continue"
            onPress={() => {
              setShown(true);
              if (!validateEmail(mail) && !validateName(name)) onPick(mail, name.trim());
            }}
          />
        </View>
      ) : null}
      <T variant="caption" color={colors.textMuted}>
        Demo sign-in: in a store build this opens Google’s secure OAuth screen.
      </T>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, minHeight: 64 },
  plus: { width: 40, height: 40, borderRadius: radius.button, borderWidth: 2, borderColor: colors.ink, borderStyle: 'dashed', alignItem
s: 'center', justifyContent: 'center' },
});
```

src/components/auth/JoinCodeSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
```

```
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Input } from '@/components/ui/Input';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { colors } from '@/theme';
import { normalizeInvite, validateInvite } from '@/utils/validation';

/**
 * "Join with a code" for someone who was invited and is not signed in yet. A trip needs a person in it, so this asks for the code
 * first and keeps it: after signing in (or creating an account) the app opens the trip's join page with the code already in.
 */
export function JoinCodeSheet({ visible, onClose, onDone, initial }: { visible: boolean; onClose: () => void; onDone: (code: string) =>
void; initial?: string | null }) {
  const [code, setCode] = useState(initial ?? '');
  const [shown, setShown] = useState(false);
  const error = shown ? validateInvite(code) : null;
  const ok = !validateInvite(code);

  const submit = () => {
    setShown(true);
    if (validateInvite(code)) return;
    onDone(code);
  };

  return (
    <Sheet visible={visible} onClose={onClose} title="Join with a code" subtitle="Enter the code your friend shared. You will sign in fi
rst, then land in the trip.">
      <Input
        label="Invite code"
        icon={error ? 'alert-circle' : 'ticket-outline'}
        value={code}
        onChangeText={(v) => setCode(normalizeInvite(v))}
        onBlur={() => code && setShown(true)}
        autoCapitalize="characters"
        autoCorrect={false}
        maxLength={8}
        placeholder="ABC-1234"
        returnKeyType="go"
        onSubmitEditing={submit}
        error={error}
        hint="Three letters, a dash and four numbers"
        right={ok ? <Ionicons name="checkmark-circle" size={20} color={colors.positive} /> : undefined}
        testID="join-code"
      />
      <View style={{ paddingBottom: 4 }}>
        <Button label="Continue" iconRight="arrow-forward" onPress={submit} testID="join-code-continue" />
      </View>
      <T variant="caption" color={colors.textMuted}>
        No account yet? You can create one on the next screen.
      </T>
    </Sheet>
  );
}
```

src/components/auth/__tests__/logic.test.ts

```
import { afterSignIn, calm, inviteFrom, meterView } from '../logic';

describe('calm', () => {
  it('drops exclamation marks and the Spanish opening mark', () => {
    expect(calm('Welcome back!')).toBe('Welcome back');
    expect(calm('¡Bienvenido de nuevo!')).toBe('Bienvenido de nuevo');
    expect(calm('おかえりなさい！')).toBe('おかえりなさい');
    expect(calm('Bon retour !')).toBe('Bon retour');
  });
  it('leaves plain text alone', () => {
    expect(calm('Sign in to continue.')).toBe('Sign in to continue.');
  });
});

describe('meterView', () => {
  it('shows nothing for an empty password', () => {
    expect(meterView('')).toBeNull();
  });
  it('is red and has at least one segment for a weak password', () => {
    expect(meterView('abc')).toMatchObject({ filled: 1, tone: 'danger', icon: 'alert-circle' });
  });
  it('is amber at "Okay" and green from "Strong"', () => {
    expect(meterView('Trip2027go')?.tone).toBe('warn');
    expect(meterView('Trip2027goNow!')?.tone).toBe('positive');
    expect(meterView('Trip2027goNow!')?.icon).toBe('checkmark-circle');
  });
});
```

```
describe('inviteFrom', () => {
  it('tidies a code', () => {
    expect(inviteFrom('seo5521')).toBe('SEO-5521');
    expect(inviteFrom(' AML-2026 ')).toBe('AML-2026');
  });
  it('rejects things that are not codes', () => {
    expect(inviteFrom('')).toBeNull();
    expect(inviteFrom(undefined)).toBeNull();
    expect(inviteFrom('hello')).toBeNull();
    expect(inviteFrom('AB-12')).toBeNull();
  });
});

describe('afterSignIn', () => {
  it('goes home when there is no invite', () => {
    expect(afterSignIn(null)).toEqual({ first: '/(tabs)/explore', then: null });
  });
  it('goes home and then to the invite', () => {
    expect(afterSignIn('SEO-5521')).toEqual({ first: '/(tabs)/explore', then: { pathname: '/join', params: { code: 'SEO-5521' } } });
  });
});
```

src/components/auth/kit.tsx

```
/** The small parts the first-run, sign-in, sign-up and reset screens share. Everything is drawn from tokens, so it reads in light and d
ark. */
import { Ionicons } from '@expo/vector-icons';
import type { ReactNode, RefObject } from 'react';
import { KeyboardAvoidingView, Platform, ScrollView, StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { showLegal } from './legal';
import { FadeIn } from '@/components/ui/motion';
import { Press } from '@/components/ui/Press';
import { Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import type { IconName } from '@/components/ui/bits';
import { colors, line, radius } from '@/theme';

import { meterView, type MeterTone } from './logic';

/**
 * The page of a form screen: paper, an optional header, and a scroll that keeps clear of the keyboard (on iPhone the page is lifted by
the
 * keyboard's height, as in Chat and Make it yours; on Android the window resizes).
 */
export function AuthScreen({ header, children, scrollRef, top = 8 }: { header?: ReactNode; children: ReactNode; scrollRef?: RefObject<Sc
rollView | null>; top?: number }) {
  return (
    <Screen scroll={false} padded={false} bottomInset={0} header={header}>
      <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={{ flex: 1 }}>
        <ScrollView
          ref={scrollRef}
          style={{ flex: 1 }}
          contentContainerStyle={{ paddingHorizontal: 20, paddingTop: top, paddingBottom: 32 }}
          keyboardShouldPersistTaps="handled"
          keyboardDismissMode="on-drag"
          showsVerticalScrollIndicator={false}>
          <FadeIn duration={320} distance={10}>
            {children}
          </FadeIn>
        </ScrollView>
      </KeyboardAvoidingView>
    </Screen>
  );
}

const TONE = {
  danger: { fg: colors.red, bg: colors.redSoft, border: colors.red },
  warn: { fg: colors.warn, bg: colors.warnSoft, border: colors.warn },
  positive: { fg: colors.positive, bg: colors.positiveSoft, border: colors.positive },
} as const;

/** A message with a state colour, an icon and words (never colour alone): the sign-in error, for example. */
export function Notice({ tone = 'danger', icon = 'alert-circle', children }: { tone?: keyof typeof TONE; icon?: IconName; children: Reac
tNode }) {
  const c = TONE[tone];
  return (
    <View accessibilityLiveRegion="assertive" accessibilityRole="alert" style={[styles.notice, { backgroundColor: c.bg, borderColor: c.b
order }]}>
      <Ionicons name={icon} size={20} color={c.fg} />
      <T variant="small" weight="semibold" color={c.fg} style={{ flex: 1 }}>
        {children}
      </T>
    </View>
  );
}
```

```
/** The invite code someone arrived with, kept until they are signed in: a dashed outline like the invite chip on a ticket. */
export function InviteNote({ code, onRemove }: { code: string; onRemove?: () => void }) {
  return (
    <View style={styles.invite}>
      <Ionicons name="ticket-outline" size={22} color={colors.ink} />
      <View style={{ flex: 1 }}>
        <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 10 }}>
          Invite code
        </T>
        <T variant="numeralSm" style={{ fontSize: 18, lineHeight: 22 }}>
          {code}
        </T>
        <T variant="caption" color={colors.textSecondary}>
          Sign in or create an account and you land in the trip.
        </T>
      </View>
      {onRemove ? (
        <Press onPress={onRemove} accessibilityLabel="Remove the invite code" style={styles.remove}>
          <Ionicons name="close" size={20} color={colors.ink} />
        </Press>
      ) : null}
    </View>
  );
}

/**
 * A tick box you can hit: a 56 high row, an outlined square that fills with the accent and shows a tick when chosen.
 * One control (a checkbox), so put no other button inside it.
 */
export function CheckRow({ checked, onPress, label, error, children }: { checked: boolean; onPress: () => void; label?: string; error?:
string | null; children: ReactNode }) {
  return (
    <View style={{ gap: 4 }}>
      <Press onPress={onPress} accessibilityRole="checkbox" accessibilityState={{ checked }} accessibilityLabel={label} scaleTo={0.99} s
tyle={styles.check}>
        <View style={[styles.box, checked && { backgroundColor: colors.accent }, !!error && { borderColor: colors.red }]}>
          {checked ? <Ionicons name="checkmark" size={18} color={colors.onAccent} /> : null}
        </View>
        <View style={{ flex: 1 }}>{children}</View>
      </Press>
      {error ? (
        <View style={styles.errorRow} accessibilityLiveRegion="polite">
          <Ionicons name="alert-circle" size={16} color={colors.red} />
          <T variant="caption" color={colors.red} style={{ flex: 1 }}>
            {error}
          </T>
        </View>
      ) : null}
    </View>
  );
}

/** Four segments that fill as the password gets stronger: red, amber, green, each with an icon and a word. */
export function PasswordMeter({ password }: { password: string }) {
  const v = meterView(password);
  if (!v) return null;
  const c = TONE[v.tone as MeterTone];
  return (
    <View style={{ gap: 6 }} accessibilityLabel={`Password strength: ${v.label}`}>
      <View style={styles.segRow}>
        {[0, 1, 2, 3].map((i) => (
          <View key={i} style={[styles.meterSeg, i < v.filled && { backgroundColor: c.fg }]} />
        ))}
      </View>
      <View style={styles.errorRow}>
        <Ionicons name={v.icon} size={16} color={c.fg} />
        <T variant="caption" weight="semibold" color={c.fg}>
          {v.label}
        </T>
      </View>
    </View>
  );
}

/** "Step 2 of 3" as a row of ticket stubs: the ones behind you are inked, the one you are on is the accent. */
export function StepSegments({ now, total, label = 'Step' }: { now: number; total: number; label?: string }) {
  return (
    <View accessibilityRole="progressbar" accessibilityLabel={`${label} ${now} of ${total}`} accessibilityValue={{ min: 1, max: total, n
ow }} style={styles.segRow}>
      {Array.from({ length: total }, (_, i) => (
        <View key={i} style={[styles.seg, { backgroundColor: i + 1 < now ? colors.ink : i + 1 === now ? colors.accent : 'transparent'
}]} />
      ))}
    </View>
  );
```

```
}

/** Terms and Privacy as two real links (each 44 high) that open the short notes. */
export function LegalLinks({ style }: { style?: StyleProp<ViewStyle> }) {
  return (
    <View style={[styles.legal, style]}>
      {(['Terms of Service', 'Privacy Policy'] as const).map((title) => (
        <Press key={title} onPress={() => showLegal(title)} accessibilityRole="link" accessibilityLabel={`Read the ${title}`} style={sty
les.legalLink}>
          <T variant="kicker" color={colors.textSecondary} style={{ textDecorationLine: 'underline' }}>
            {title}
          </T>
        </Press>
      ))}
    </View>
  );
}

/** A quiet underlined text link, 44 high: "Forgot password?". */
export function TextLink({ label, onPress, role = 'link', style }: { label: string; onPress?: () => void; role?: 'link' | 'button'; styl
e?: StyleProp<ViewStyle> }) {
  return (
    <Press onPress={onPress} accessibilityRole={role} accessibilityLabel={label} style={[styles.textLink, style]}>
      <T variant="small" weight="bold" style={{ textDecorationLine: 'underline' }}>
        {label}
      </T>
    </Press>
  );
}

const styles = StyleSheet.create({
  invite: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 12, borderRadius: radius.chip, borderWidth: line, borderStyle:
'dashed', borderColor: colors.ink, backgroundColor: colors.accentSoft },
  remove: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center' },
  notice: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingHorizontal: 14, paddingVertical: 12, minHeight: 48, borderRadiu
s: radius.button, borderWidth: line },
  check: { flexDirection: 'row', alignItems: 'center', gap: 14, minHeight: 56 },
  box: { width: 28, height: 28, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, align
Items: 'center', justifyContent: 'center' },
  errorRow: { flexDirection: 'row', alignItems: 'center', gap: 6 },
  segRow: { flexDirection: 'row', gap: 5 },
  meterSeg: { flex: 1, height: 12, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: 'transparent'
},
  seg: { flex: 1, height: 12, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink },
  legal: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', columnGap: 20 },
  legalLink: { minHeight: 44, justifyContent: 'center' },
  textLink: { minHeight: 44, minWidth: 44, justifyContent: 'center', alignItems: 'flex-start' },
});
```

src/components/auth/legal.ts

```
import { showDialog } from '@/components/ui/feedback';

/** The short Terms / Privacy notes shown from the sign-in and sign-up screens. */
export function showLegal(title: 'Terms of Service' | 'Privacy Policy') {
  showDialog(
    title,
    title === 'Privacy Policy'
      ? 'TogetherWeGo stores your trips on this device. Passwords are salted and hashed (PBKDF2); API keys go in the secure keychain. We
never sell data. Location is only used when you tap a nearby/airport feature.'
      : 'TogetherWeGo is a student project built for FBLA Mobile Application Development. Use it to plan trips with friends — be kind in
chats and only upload documents you own.',
    [{ label: 'Got it' }],
    'shield-checkmark-outline',
  );
}
```

src/components/auth/logic.ts

```
/** Small pure helpers for the first-run and sign-in screens (no React Native in here, so jest can check them). */
import { normalizeInvite, passwordStrength, validateInvite } from '@/utils/validation';

/**
 * A greeting without the shouting: drops a trailing "!" (or "!") and Spanish "¡", so "Welcome back!" reads "Welcome back".
 * The translations keep their exclamation marks; the new look does not use them.
 */
export function calm(text: string): string {
  return text.replace(/^¡/, '').replace(/[!!]+$/, '').trim();
}

export type MeterTone = 'danger' | 'warn' | 'positive';

/** How the password strength bar is drawn: how many of the 4 segments are filled, in which state colour, with which words. */
export function meterView(password: string): { filled: number; tone: MeterTone; label: string; icon: 'alert-circle' | 'checkmark-circle'
} | null {
  if (!password) return null;
```

```
  const { score, label } = passwordStrength(password);
  const filled = Math.max(1, score);
  if (score <= 1) return { filled, tone: 'danger', label, icon: 'alert-circle' };
  if (score === 2) return { filled, tone: 'warn', label, icon: 'alert-circle' };
  return { filled, tone: 'positive', label, icon: 'checkmark-circle' };
}

/** An invite code typed or passed in a link, tidied to ABC-1234, or null when it is not one. */
export function inviteFrom(raw: string | undefined | null): string | null {
  if (!raw) return null;
  const code = normalizeInvite(String(raw));
  return validateInvite(code) ? null : code;
}

/** Where to go once someone is signed in: the home tab, then (after a moment, so the tab is on screen) the invite they came with. */
export function afterSignIn(invite: string | null): { first: '/(tabs)/explore'; then: { pathname: '/join'; params: { code: string } } |
null } {
  return { first: '/(tabs)/explore', then: invite ? { pathname: '/join', params: { code: invite } } : null };
}
```

src/components/auth/slides.ts

```
import type { ImageSourcePropType } from 'react-native';

import { photos } from '@/data/images';

/**
 * The three first-run slides: one idea each. `lift` is how far (as a share of the screen height, 0 to 0.2) the photo is pulled up, so i
ts
 * subject stays above the boarding pass that lies over the lower part of the screen.
 */
export interface Slide {
  key: 'plan' | 'split' | 'share';
  image: ImageSourcePropType;
  stamp: string;
  title: string;
  body: string;
  lift: number;
}

export const SLIDES: Slide[] = [
  { key: 'plan', image: photos['onboard-1'], stamp: 'Plan', title: 'Plan together', body: 'Add ideas, vote on them, and the winners go i
nto one shared itinerary.', lift: 0.08 },
  { key: 'split', image: photos['onboard-2'], stamp: 'Split', title: 'Split the money', body: 'Log who paid for what. We work out who ow
es who, to the cent.', lift: 0.2 },
  { key: 'share', image: photos['onboard-3'], stamp: 'Share', title: 'Keep everyone in the loop', body: 'One chat, one checklist and one
invite code for the whole crew.', lift: 0.1 },
];
```

src/components/budget/AddExpenseSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useState } from 'react';
import { Platform, StyleSheet, View } from 'react-native';

import { expenseCategories } from '@/components/trip/meta';
import { Chip } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { toast } from '@/components/ui/feedback';
import { currencyByCode } from '@/data/reference';
import { destinationById } from '@/data/destinations';
import { useDraft } from '@/hooks/useDraft';
import { useT } from '@/i18n';
import { convert, useRates } from '@/services/currency';
import { deleteLocalFile, pickImage } from '@/services/files';
import type { ExpenseCategory, Trip } from '@/store/types';
import { totalSpent, useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';
import { money } from '@/utils/format';
import { guessExpenseCategory } from '@/utils/guess';
import { todayInTz } from '@/utils/time';
import { firstError, maxLen, parseAmount, required, validateAmount } from '@/utils/validation';

import { AmountField } from './AmountField';
import { EMPTY_DRAFT, activePayer, activeSplit, summaryLine, type ExpenseDraft } from './logic';

/**
 * Logging an expense in as few taps as possible: the amount first, as a big number; then what it was for. The category is guessed from
 * those words and shown as one chip; who paid and who shares it are one plain line ("Paid by you · split with everyone") with a Change
 * button for the rare case. A half-typed expense is kept (useDraft) if the sheet is closed, and cleared when it is added.
 */
```

```
export function AddExpenseSheet({ visible, onClose, trip, userId }: { visible: boolean; onClose: () => void; trip: Trip; userId: string
}) {
  const t = useT();
  const rates = useRates();
  const addExpense = useAppStore((s) => s.addExpense);
  const dest = destinationById(trip.destinationId);
  const local = dest?.currency ?? 'EUR';
  const memberIds = trip.members.map((m) => m.id);

  const [d, setD, clearDraft] = useDraft<ExpenseDraft>(`add-expense-${trip.id}`, EMPTY_DRAFT);
  const patch = (p: Partial<ExpenseDraft>) => setD((cur) => ({ ...cur, ...p }));
  const [more, setMore] = useState(false);
  const [attaching, setAttaching] = useState(false);
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  const paidBy = activePayer(d.paidBy, userId, memberIds);
  const split = activeSplit(d.split, memberIds);
  const payer = paidBy === userId ? 'you' : (trip.members.find((m) => m.id === paidBy)?.name.split(' ')[0] ?? 'someone');
  const currencies = ['USD', local].filter((c, i, a) => a.indexOf(c) === i);
  const cur = currencies.includes(d.cur) ? d.cur : 'USD';
  const symbol = cur === 'USD' ? '$' : (currencyByCode(cur)?.symbol ?? cur);
  const catIcon = expenseCategories.find((c) => c.key === d.cat)?.icon ?? 'ellipsis-horizontal';
  const typed = parseAmount(d.amount);

  const attach = async (camera: boolean) => {
    setAttaching(true);
    try {
      const f = await pickImage(camera);
      if (f) {
        deleteLocalFile(d.receipt ?? undefined);
        patch({ receipt: f.uri });
      }
    } catch (e) {
      toast(e instanceof Error ? e.message : 'Couldn’t open photos', { tone: 'warn' });
    } finally {
      setAttaching(false);
    }
  };

  const removeReceipt = () => {
    deleteLocalFile(d.receipt ?? undefined);
    patch({ receipt: null });
  };

  const submit = () => {
    const e = {
      title: firstError(d.title, required('what it was for'), maxLen(40, 'Description')),
      amount: validateAmount(d.amount, { max: 100000 }),
      split: split.length ? null : 'Pick at least one person to split with',
    };
    setErrors(e);
    if (Object.values(e).some(Boolean)) {
      if (e.split) setMore(true);
      return tap('warning');
    }
    const n = parseAmount(d.amount)!;
    const usd = cur === 'USD' ? n : convert(n, cur, 'USD', rates);
    const spent = totalSpent(trip);
    addExpense(trip.id, {
      title: d.title.trim(),
      amount: Math.round(usd * 100) / 100,
      category: d.cat,
      paidBy,
      splitBetween: split,
      date: todayInTz(dest?.tz ?? 'UTC'),
      original: cur === 'USD' ? undefined : { amount: n, currency: cur },
      receipt: d.receipt ?? undefined,
    });
    tap('success');
    if (trip.budget > 0 && spent <= trip.budget && spent + usd > trip.budget) toast(`Added ${money(usd)}. This puts the group over budge
t`, { tone: 'warn' });
    else toast(`Added ${money(usd)} · ${money(usd / split.length)} each`);
    clearDraft();
    setErrors({});
    setMore(false);
    onClose();
  };

  const pickCat = (c: ExpenseCategory) => patch({ cat: c, catTouched: true });
  /** Closing keeps what was typed (the draft) but not the red error messages. */
  const close = () => {
    setErrors({});
    onClose();
  };

  return (
```

```
    <Sheet visible={visible} onClose={close} title={t('budget.addExpense')} footer={<Button label={t('budget.add')} icon="add" onPress=
{submit} testID="add-expense" />}>
      <View style={{ gap: 16 }}>
        <View style={{ gap: 8 }}>
          <View style={styles.between}>
            <T variant="kicker">How much</T>
            {currencies.length > 1 ? (
              <View style={{ flexDirection: 'row', gap: 8 }}>
                {currencies.map((c) => (
                  <Chip key={c} small label={c} active={cur === c} onPress={() => patch({ cur: c })} />
                ))}
              </View>
            ) : null}
          </View>
          <AmountField value={d.amount} onChange={(v) => patch({ amount: v })} symbol={symbol} error={errors.amount} testID="expense-amo
unt" />
          {cur !== 'USD' && typed ? (
            <T variant="caption" color={colors.textSecondary}>
              ≈ {money(convert(typed, cur, 'USD', rates), 'USD', { decimals: true })} · saved in USD so the group can compare
            </T>
          ) : null}
        </View>

        <Input
          placeholder={t('budget.whatFor')}
          value={d.title}
          onChangeText={(text) => patch({ title: text, ...(d.catTouched ? null : { cat: guessExpenseCategory(text) }) })}
          error={errors.title}
          maxLength={45}
          style={{ minHeight: 44 }}
          testID="expense-title"
        />

        <View style={styles.summary}>
          <View style={styles.between}>
            {more ? <T variant="kicker">Category</T> : <Chip label={d.cat} icon={catIcon} active onPress={() => setMore(true)} />}
            <Press onPress={() => setMore((m) => !m)} style={styles.change} accessibilityLabel="Change who paid, the split or the catego
ry" accessibilityState={{ expanded: more }}>
              <T variant="small" weight="bold">
                {more ? 'Done' : 'Change'}
              </T>
              <Ionicons name={more ? 'chevron-up' : 'chevron-down'} size={16} color={colors.ink} />
            </Press>
          </View>
          {more ? (
            <View style={styles.wrap}>
              {expenseCategories.map((c) => (
                <Chip key={c.key} small label={c.key} icon={c.icon} active={d.cat === c.key} onPress={() => pickCat(c.key)} />
              ))}
            </View>
          ) : (
            <T variant="small" color={colors.textSecondary}>
              {summaryLine(payer, split, memberIds, userId)}
            </T>
          )}
        </View>

        {more ? (
          <View style={{ gap: 16 }}>
            <View style={{ gap: 8 }}>
              <T variant="kicker">Paid by</T>
              <View style={styles.wrap}>
                {trip.members.map((m) => (
                  <Chip key={m.id} small label={m.id === userId ? 'You' : m.name.split(' ')[0]} image={m.avatar} active={paidBy === m.i
d} onPress={() => patch({ paidBy: m.id })} />
                ))}
              </View>
            </View>
            <View style={{ gap: 8 }}>
              <View style={styles.between}>
                <T variant="kicker">Split between ({split.length})</T>
                <Press onPress={() => patch({ split: split.length === memberIds.length ? [userId] : null })} style={styles.change} acces
sibilityLabel={split.length === memberIds.length ? 'Split just for me' : 'Split with everyone'}>
                  <T variant="small" weight="bold">
                    {split.length === memberIds.length ? 'Just me' : 'Everyone'}
                  </T>
                </Press>
              </View>
              <View style={styles.wrap}>
                {trip.members.map((m) => (
                  <Chip
                    key={m.id}
                    small
                    label={m.id === userId ? 'You' : m.name.split(' ')[0]}
                    active={split.includes(m.id)}
                    onPress={() => patch({ split: split.includes(m.id) ? split.filter((x) => x !== m.id) : [...split, m.id] })}
```

```
                />
              ))}
            </View>
            {errors.split ? (
              <T variant="caption" color={colors.red}>
                {errors.split}
              </T>
            ) : null}
          </View>
        </View>
      ) : null}

      <View style={{ gap: 12 }}>
        <View style={styles.receiptRow}>
          <Button label="Attach receipt" icon="camera-outline" variant="soft" size="md" loading={attaching} onPress={() => attach(fals
e)} style={{ flex: 1 }} />
          {Platform.OS !== 'web' ? <Button label="Take photo" icon="scan-outline" variant="soft" size="md" disabled={attaching} onPres
s={() => attach(true)} style={{ flex: 1 }} /> : null}
        </View>
        {d.receipt ? (
          <View style={styles.receiptRow}>
            <Image source={{ uri: d.receipt }} style={styles.thumb} contentFit="cover" accessibilityLabel="Receipt photo" />
            <View style={{ flex: 1 }}>
              <T variant="small" color={colors.textSecondary}>
                Receipt attached. It is saved with the expense.
              </T>
              <Press onPress={removeReceipt} style={styles.change} accessibilityLabel="Remove receipt">
                <Ionicons name="trash-outline" size={16} color={colors.ink} />
                <T variant="small" weight="bold">
                  Remove
                </T>
              </Press>
            </View>
          </View>
        ) : null}
      </View>
    </View>
  </Sheet>
 );
}

const styles = StyleSheet.create({
  between: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 8 },
  summary: { gap: 8 },
  change: { flexDirection: 'row', alignItems: 'center', gap: 4, minHeight: 44, paddingHorizontal: 8, alignSelf: 'flex-start' },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  receiptRow: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  thumb: { width: 64, height: 64, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt },
});
```

src/components/budget/AmountField.tsx

```
import { useState } from 'react';
import { Platform, StyleSheet, TextInput, View } from 'react-native';

import { Plate } from '@/components/ui/Plate';
import { T } from '@/components/ui/T';
import { colors, fonts, line, radius } from '@/theme';

/**
 * The first thing in the Add expense sheet: the amount as a big mono number, like the total on a receipt. An ink outline, with the acce
nt
 * plate under it while you type. (A text field cannot use the T component, so the mono face is named here once.)
 */
export function AmountField({ value, onChange, symbol, error, testID, label = 'Amount', suffix, compact, onSubmit }: { value: string; on
Change: (v: string) => void; symbol: string; error?: string | null; testID?: string; /** What a screen reader calls the field. */ labe
l?: string; /** Small words after the number, like "per day". */ suffix?: string; compact?: boolean; onSubmit?: () => void }) {
  const [focused, setFocused] = useState(false);
  return (
    <View style={{ gap: 6 }}>
      <Plate offset={3} corner={radius.button} color={error ? colors.red : focused ? colors.accent : 'transparent'}>
        <View style={[styles.box, compact && styles.boxCompact, error ? { borderColor: colors.red, backgroundColor: colors.redSoft } : n
ull]}>
          <T variant={compact ? 'numeralSm' : 'numeral'} color={colors.textMuted} importantForAccessibility="no">
            {symbol}
          </T>
          <TextInput
            value={value}
            onChangeText={onChange}
            onSubmitEditing={onSubmit}
            placeholder="0"
            placeholderTextColor={colors.textMuted}
            keyboardType="decimal-pad"
            maxLength={10}
            accessibilityLabel={label}
            testID={testID}
```

```
            onFocus={() => setFocused(true)}
            onBlur={() => setFocused(false)}
            style={[styles.input, compact && styles.inputCompact, Platform.OS === 'web' && ({ outlineStyle: 'none' } as object)]}
          />
          {suffix ? (
            <T variant="small" color={colors.textSecondary}>
              {suffix}
            </T>
          ) : null}
        </View>
      </Plate>
      {error ? (
        <T variant="caption" color={colors.red} accessibilityLiveRegion="polite">
          {error}
        </T>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  box: { minHeight: 72, flexDirection: 'row', alignItems: 'center', gap: 8, paddingHorizontal: 16, borderRadius: radius.button, borderWi
dth: line, borderColor: colors.ink, backgroundColor: colors.card },
  boxCompact: { minHeight: 56, paddingHorizontal: 14 },
  input: { flex: 1, minWidth: 0, fontFamily: fonts.mono, fontSize: 40, color: colors.text, paddingVertical: 10 },
  inputCompact: { fontSize: 24, paddingVertical: 6 },
});
```

src/components/budget/BalancesCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { View } from 'react-native';

import { Avatar, type IconName } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { Rule } from '@/components/ui/Rule';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import type { Trip } from '@/store/types';
import { balances, settlePlan } from '@/store/useAppStore';
import { colors } from '@/theme';

import { balanceSentence, balanceView, isSettled, type BalanceTone } from './logic';
import { styles } from './styles';

const ICON: Record<BalanceTone, IconName> = { owes: 'arrow-up', gets: 'arrow-down', square: 'checkmark' };
const COLOR: Record<BalanceTone, string> = { owes: colors.red, gets: colors.positive, square: colors.textMuted };

/**
 * "Who owes who": one line per person on the paper, the amount in mono with an arrow (up = owes, down = gets back) and a word, never co
lour
 * alone. Under it the clear next step, "Settle up". When the last payment is made a SETTLED stamp slams on (`slam`), and it stays.
 */
export function BalancesCard({ trip, userId, fmt, onSettle, slam }: { trip: Trip; userId: string; fmt: (usd: number) => string; onSettl
e: () => void; slam?: boolean }) {
  const t = useT();
  const bal = balances(trip);
  const plan = settlePlan(trip);
  const settled = isSettled(trip.expenses.some((e) => !e.settlement), plan.length);
  return (
    <View>
      <View style={styles.sectionTop}>
        <T variant="headline" accessibilityRole="header" style={{ flexShrink: 1 }}>
          Who owes who
        </T>
        {settled ? <Stamp label="Settled" tone="positive" icon="checkmark-circle-outline" slam={slam} /> : null}
      </View>
      <Rule />

      <View>
        {trip.members.map((m, i) => {
          const isYou = m.id === userId;
          const name = isYou ? 'You' : m.name.split(' ')[0];
          const v = balanceView(bal[m.id] ?? 0);
          const color = COLOR[v.tone];
          return (
            <View key={m.id} style={[styles.ledgerRow, i > 0 && styles.ledgerDivider]} accessibilityLabel={balanceSentence(name, isYou,
bal[m.id] ?? 0, fmt)}>
              <Avatar name={m.name} src={m.avatar} size={40} />
              <View style={{ flex: 1, minWidth: 92 }}>
                <T variant="title" weight="bold" numberOfLines={1}>
                  {name}
                </T>
                {m.role === 'Treasurer' ? (
                  <T variant="kicker" color={colors.textSecondary} numberOfLines={1} style={{ fontSize: 10 }}>
```

```
                  Treasurer
                </T>
              ) : null}
            </View>
            <View style={{ flexDirection: 'row', alignItems: 'center', gap: 6, flexShrink: 1 }}>
              <Ionicons name={ICON[v.tone]} size={18} color={color} />
              {v.tone === 'square' ? (
                <T variant="small" weight="bold" color={color}>
                  all square
                </T>
              ) : (
                <T variant="small" weight="semibold" color={color} style={{ textAlign: 'right', flexShrink: 1 }}>
                  {v.word}{' '}
                  <T variant="numeralSm" color={color}>
                    {fmt(v.amount)}
                  </T>
                </T>
              )}
            </View>
          </View>
        );
      })}
    </View>

    {settled ? (
      <T variant="small" color={colors.textSecondary} style={{ marginTop: 6 }}>
        Everyone is square. Nobody owes anybody.
      </T>
    ) : plan.length ? (
      <View style={{ marginTop: 12 }}>
        <Button label={t('budget.settle')} size="md" variant="secondary" icon="swap-horizontal" onPress={onSettle} />
      </View>
    ) : null}
  </View>
 );
}
```

src/components/budget/BigAmount.tsx

```
import { Text } from 'react-native';

import { T } from '@/components/ui/T';
import { colors } from '@/theme';

import { amountSize } from './logic';

const isPoint = (ch: string | undefined) => ch === ',' || ch === '.';

/**
 * A big mono number. In a monospace face a comma takes a whole cell ("$1 , 650"), so punctuation is pulled in close to its digits.
 * It stays ONE piece of text (the nested spans only change spacing), so it reads and is found as "$1,650". The size shrinks for long am
ounts
 * unless `size` is given.
 */
export function BigAmount({ text, color = colors.text, size, testID }: { text: string; color?: string; size?: number; testID?: string })
{
  const s = size ?? amountSize(text.length);
  const chars = text.split('');
  return (
    <T variant="hero" color={color} style={{ fontSize: s, lineHeight: s + 4, flexShrink: 1 }} testID={testID}>
      {chars.map((ch, i) => (
        <Text key={i} style={isPoint(ch) || isPoint(chars[i + 1]) ? { letterSpacing: -s * 0.22 } : undefined}>
          {ch}
        </Text>
      ))}
    </T>
  );
}
```

src/components/budget/BudgetCapSheet.tsx

```
import { useState } from 'react';

import { Button } from '@/components/ui/Button';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { toast } from '@/components/ui/feedback';
import { tap } from '@/components/ui/Press';
import type { Trip } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';
import { money } from '@/utils/format';
import { parseAmount, validateAmount } from '@/utils/validation';

import { AmountField } from './AmountField';

/** Changes the group's budget: one amount in US dollars. Opened by tapping the budget on the summary. */
```

```
export function BudgetCapSheet({ visible, onClose, trip }: { visible: boolean; onClose: () => void; trip: Trip }) {
  const updateTrip = useAppStore((s) => s.updateTrip);
  // Until the person types, the field shows the trip's current budget (so a change from another phone shows up here too).
  const [typed, setTyped] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const text = typed ?? String(trip.budget);

  const close = () => {
    setTyped(null);
    setError(null);
    onClose();
  };
  const save = () => {
    const e = validateAmount(text, { min: 1, max: 1_000_000, label: 'a budget' });
    setError(e);
    if (e) return tap('warning');
    const v = parseAmount(text)!;
    updateTrip(trip.id, { budget: v });
    toast(`Budget set to ${money(v)}`);
    close();
  };

  return (
    <Sheet visible={visible} onClose={close} title="Group budget" footer={<Button label="Save budget" icon="checkmark" onPress={save} te
stID="budget-cap-save" />}>
      <T variant="kicker">Group budget cap in USD</T>
      <AmountField value={text} onChange={(v) => setTyped(v.replace(/[^0-9.,]/g, ''))} symbol="$" label="Group budget cap in USD" error=
{error} onSubmit={save} testID="budget-cap" />
    </Sheet>
  );
}
```

src/components/budget/DailyPlanner.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { placeCategoryIcon } from '@/components/trip/meta';
import { Card, SectionHeader } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { placesFor } from '@/data/places';
import { useT } from '@/i18n';
import type { Trip } from '@/store/types';
import { totalSpent } from '@/store/useAppStore';
import { categoryColors, colors, hair, line, radius } from '@/theme';
import { money, priceTier } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';
import { daysBetween } from '@/utils/time';
import { parseAmount } from '@/utils/validation';

import { AmountField } from './AmountField';
import { styles as shared } from './styles';

/** How a day's money is usually split, as shares of the daily amount. */
const SPLIT: [string, number][] = [
  ['Food', 0.35],
  ['Activities', 0.3],
  ['Transport', 0.15],
  ['Shopping', 0.12],
  ['Other', 0.08],
];

/** Daily spending planner: how much per person per day, does it fit what is left, and budget finds nearby. */
export function DailyPlanner({ trip }: { trip: Trip }) {
  const t = useT();
  const dest = destinationById(trip.destinationId);
  const [daily, setDaily] = useState(String(dest?.dailyCost ?? 120));
  const tripLen = daysBetween(trip.startDate, trip.endDate) + 1;
  const dailyN = parseAmount(daily) ?? 0;
  const remaining = trip.budget - totalSpent(trip);
  const plannedTotal = dailyN * tripLen * trip.members.length;
  const fits = plannedTotal <= remaining;
  const activityAllowance = dailyN * 0.3;
  const finds = dest
    ? [trip.destinationId, ...trip.extraDestinationIds]
        .flatMap((id) => placesFor(id))
        .filter((p) => p.cost <= activityAllowance)
        .map((p) => ({ p, km: distanceKm(dest, p) }))
        .filter((x) => x.km <= 8)
        .sort((a, b) => b.p.rating - a.p.rating)
        .slice(0, 3)
```

```
    : [];
  return (
    <View>
      <SectionHeader title={t('budget.planner')} />
      <Card style={{ gap: 16 }}>
        <T variant="small" color={colors.textSecondary}>
          How much does each person want to spend per day in {dest?.city}? Typical: {money(dest?.dailyCost ?? 0)}.
        </T>
        <AmountField compact symbol="$" suffix="per person, per day" label="Daily spend per person in USD" value={daily} onChange={(v) =
> setDaily(v.replace(/[^0-9.]/g, ''))} error={dailyN > 0 ? null : 'Enter a daily amount greater than 0'} />
        {dailyN > 0 ? (
          <>
            <View style={{ gap: 10 }}>
              <View accessibilityElementsHidden importantForAccessibility="no-hide-descendants" style={local.bar}>
                {SPLIT.map(([k, f]) => (
                  <View key={k} style={{ width: `${f * 100}%`, backgroundColor: categoryColors[k], borderRightWidth: line, borderRightCo
lor: colors.ink }} />
                ))}
              </View>
              <View style={local.legend}>
                {SPLIT.map(([k, f]) => (
                  <View key={k} style={local.legendItem}>
                    <View style={[local.swatch, { backgroundColor: categoryColors[k] }]} />
                    <T variant="caption" style={{ flex: 1 }} numberOfLines={1}>
                      {k}
                    </T>
                    <T variant="numeralSm" style={{ fontSize: 15, lineHeight: 20 }}>
                      {money(dailyN * f)}
                    </T>
                  </View>
                ))}
              </View>
            </View>

            <View style={{ gap: 8, borderTopWidth: hair, borderTopColor: colors.border, paddingTop: 14 }}>
              {fits ? <Stamp label="Fits" tone="positive" icon="checkmark-circle-outline" /> : <Stamp label="Over what is left" tone="wa
rn" icon="alert-circle-outline" />}
              <T variant="small">
                {tripLen} days × {trip.members.length} people = {money(plannedTotal)} on the ground ·{' '}
                {fits ? `fits the ${money(remaining)} left` : `${money(plannedTotal - remaining)} more than what’s left`}
              </T>
            </View>

            {finds.length ? (
              <View>
                <T variant="title" weight="bold" style={{ marginBottom: 4 }}>
                  Budget finds near {dest?.city} (up to {money(activityAllowance)} each)
                </T>
                {finds.map(({ p, km }, i) => (
                  <Press
                    key={p.id}
                    onPress={() => router.push({ pathname: '/nearby', params: { focus: p.id, maxCost: String(Math.round(activityAllowanc
e)) } })}
                    style={[shared.ledgerRow, { paddingVertical: 8, minHeight: 56 }, i > 0 && shared.ledgerDivider]}
                    accessibilityLabel={`${p.name}, ${priceTier(p.cost)}, ${kmLabel(km)}`}>
                    <View style={local.findIcon}>
                      <Ionicons name={placeCategoryIcon[p.category]} size={16} color={colors.ink} />
                    </View>
                    <T variant="small" weight="semibold" style={{ flex: 1 }} numberOfLines={1}>
                      {p.name}
                    </T>
                    <T variant="kicker" color={colors.textSecondary}>
                      {priceTier(p.cost)} · {kmLabel(km)}
                    </T>
                  </Press>
                ))}
              </View>
            ) : null}
          </>
        ) : null}
      </Card>
    </View>
  );
}

const local = StyleSheet.create({
  bar: { height: 18, flexDirection: 'row', borderWidth: line, borderColor: colors.ink, borderRadius: radius.xs, overflow: 'hidden' },
  legend: { flexDirection: 'row', flexWrap: 'wrap', columnGap: 20, rowGap: 6 },
  legendItem: { flexDirection: 'row', alignItems: 'center', gap: 8, width: '46%', minWidth: 132 },
  swatch: { width: 12, height: 12, borderWidth: 1.5, borderColor: colors.ink },
  findIcon: { width: 36, height: 36, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.ca
rd, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/budget/ExpenseDetailSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useState } from 'react';
import { ActivityIndicator, StyleSheet, View } from 'react-native';

import { Avatar, AvatarStack } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { Press } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { Sheet } from '@/components/ui/Sheet';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { confirmAction, toast } from '@/components/ui/feedback';
import { deleteLocalFile } from '@/services/files';
import type { Expense, Trip } from '@/store/types';
import { memberById, useAppStore } from '@/store/useAppStore';
import { useSharedFile } from '@/sync/useFile';
import { colors, line, radius } from '@/theme';
import { money } from '@/utils/format';
import { dayLabel } from '@/utils/time';

import { BigAmount } from './BigAmount';
import { styles } from './styles';

/** The receipt photo: this phone's own, or the copy from the friend who paid. Tap to enlarge. */
function Receipt({ expense }: { expense: Expense }) {
  const [big, setBig] = useState(false);
  const { uri, loading } = useSharedFile(expense.receipt, expense.receiptBlob);
  if (!uri && !loading) return null;
  return (
    <View style={{ gap: 8 }}>
      <T variant="kicker" color={colors.textSecondary}>
        Receipt
      </T>
      {uri ? (
        <Press onPress={() => setBig((b) => !b)} scaleTo={0.99} accessibilityLabel={big ? 'Shrink receipt' : 'Enlarge receipt'} style={{
alignSelf: big ? 'stretch' : 'flex-start' }}>
          <Image source={{ uri }} style={big ? local.big : local.small} contentFit={big ? 'contain' : 'cover'} accessibilityLabel="Recei
pt photo" />
        </Press>
      ) : (
        <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
          <ActivityIndicator color={colors.ink} />
          <T variant="small" color={colors.textSecondary}>
            Getting the receipt…
          </T>
        </View>
      )}
    </View>
  );
}

/**
 * One expense as a receipt: what it cost in a big mono number, who paid, who shares it, the photo of the paper receipt, and (apart from
 * everything else, at the bottom) Delete.
 */
export function ExpenseDetailSheet({ detail, onClose, trip, fmt }: { detail: Expense | null; onClose: () => void; trip: Trip; fmt: (usd:
number) => string }) {
  const deleteExpense = useAppStore((s) => s.deleteExpense);
  const payer = detail ? memberById(trip, detail.paidBy) : undefined;
  const sharers = detail ? detail.splitBetween.map((id) => memberById(trip, id)).filter((m): m is NonNullable<typeof m> => !!m) : [];
  return (
    <Sheet visible={!!detail} onClose={onClose} title={detail?.title} subtitle={detail ? `${detail.settlement ? 'Settlement' : detail.ca
tegory} · ${dayLabel(detail.date)}` : undefined}>
      {detail ? (
        <>
          <View style={styles.rowBetween}>
            <BigAmount text={fmt(detail.amount)} size={48} />
            {detail.original ? <Stamp label={`Paid as ${money(detail.original.amount, detail.original.currency)}`} tone="ink" /> : null}
          </View>
          <Rule weight={1} />
          <View style={styles.rowBetween}>
            <View style={{ flexDirection: 'row', alignItems: 'center', gap: 10, flexShrink: 1 }}>
              <Avatar name={payer?.name ?? '?'} src={payer?.avatar} size={36} />
              <View style={{ flexShrink: 1 }}>
                <T variant="kicker" color={colors.textSecondary}>
                  Paid by
                </T>
                <T variant="title" weight="bold" numberOfLines={1}>
                  {payer?.name ?? 'Someone'}
                </T>
              </View>
            </View>
            <Ionicons name="arrow-forward" size={18} color={colors.ink} />
```

```
            <View style={{ alignItems: 'flex-end', gap: 4 }}>
              <AvatarStack people={sharers} max={4} size={30} />
              <T variant="small" color={colors.textSecondary}>
                <T variant="numeralSm" style={{ fontSize: 17, lineHeight: 22 }}>
                  {fmt(detail.amount / Math.max(1, detail.splitBetween.length))}
                </T>{' '}
                each
              </T>
            </View>
          </View>
          <Receipt expense={detail} />
          <View style={{ marginTop: 12 }}>
            <Button
              label="Delete expense"
              variant="danger"
              icon="trash-outline"
              onPress={() => {
                const d = detail;
                onClose();
                confirmAction('Delete expense?', `${d.title} (${money(d.amount)}) will be removed.`, () => {
                  deleteExpense(trip.id, d.id);
                  deleteLocalFile(d.receipt);
                  toast('Expense deleted');
                }, { destructive: true, confirmLabel: 'Delete' });
              }}
            />
          </View>
        </>
      ) : null}
    </Sheet>
  );
}

const local = StyleSheet.create({
  small: { width: 112, height: 112, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt
},
  big: { width: '100%', height: 380, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt
},
});
```

src/components/budget/ExpensesSection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { View } from 'react-native';

import { expenseCategories } from '@/components/trip/meta';
import { EmptyState, SectionHeader, type IconName } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import type { Expense, Trip } from '@/store/types';
import { memberById } from '@/store/useAppStore';
import { colors } from '@/theme';
import { money } from '@/utils/format';

import { dayHeading, groupByDay } from './logic';
import { styles } from './styles';

/** How many expenses show before "Show older": a long trip stays quick to draw. */
const PAGE = 25;

const iconFor = (e: Expense): IconName => (e.settlement ? 'swap-horizontal' : (expenseCategories.find((c) => c.key === e.category)?.icon
?? 'ellipsis-horizontal'));

/**
 * The expenses as a receipt: grouped under day headings (with what the group spent that day), each line a category icon, what it was,
 * who paid, and the amount in mono. Tapping a line opens its details. With nothing logged yet it invites the first one.
 */
export function ExpensesSection({ trip, fmt, today, onOpen, onAdd }: { trip: Trip; fmt: (usd: number) => string; today: string; onOpen:
(e: Expense) => void; onAdd: () => void }) {
  const t = useT();
  const [limit, setLimit] = useState(PAGE);
  const { groups, hidden } = groupByDay(trip.expenses, limit);

  return (
    <View>
      <SectionHeader title={t('budget.expenses')} />
      {!trip.expenses.length ? (
        <EmptyState icon="receipt-outline" title="No expenses yet" body="Log the first one and the split is worked out for you.">
          <Button label="Log an expense" icon="add" onPress={onAdd} />
        </EmptyState>
      ) : (
        <View>
          {groups.map((g, gi) => (
```

```
            <View key={g.date} style={gi > 0 ? { marginTop: 20 } : undefined}>
              <View style={[styles.rowBetween, { paddingBottom: 6 }]}>
                <T variant="kicker" color={colors.ink} accessibilityRole="header">
                  {dayHeading(g.date, today)}
                </T>
                {g.total > 0 && g.items.length > 1 ? (
                  <T variant="kicker" color={colors.textSecondary}>
                    {fmt(g.total)}
                  </T>
                ) : null}
              </View>
              {g.items.map((e, i) => (
                <Press key={e.id} onPress={() => onOpen(e)} scaleTo={0.99} accessibilityLabel={`${e.title}, ${fmt(e.amount)}`} style={[s
tyles.ledgerRow, styles.ledgerDivider, i === 0 && { borderTopColor: colors.ink }, { paddingVertical: 10, minHeight: 64 }]}>
                  <View style={styles.iconBox}>
                    <Ionicons name={iconFor(e)} size={18} color={colors.ink} />
                  </View>
                  <View style={{ flex: 1 }}>
                    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 6 }}>
                      <T variant="bodySm" weight="bold" numberOfLines={2} style={{ flexShrink: 1 }}>
                        {e.title}
                      </T>
                      {e.receipt || e.receiptBlob ? <Ionicons name="receipt-outline" size={14} color={colors.textSecondary} accessibilit
yLabel="Has receipt" /> : null}
                    </View>
                    <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                      {e.settlement ? 'Settlement' : e.category} · {t('budget.paidBy', { name: memberById(trip, e.paidBy)?.name.split('
')[0] ?? '—' })}
                      {e.original ? ` · ${money(e.original.amount, e.original.currency)}` : ''}
                    </T>
                  </View>
                  <T variant="numeralSm" color={e.settlement ? colors.textSecondary : colors.text}>
                    {fmt(e.amount)}
                  </T>
                </Press>
              ))}
            </View>
          ))}
          {hidden > 0 ? (
            <View style={{ marginTop: 16 }}>
              <Button label={`Show ${Math.min(PAGE, hidden)} older`} variant="secondary" size="md" onPress={() => setLimit((l) => l + PA
GE)} />
            </View>
          ) : null}
        </View>
      )}
    </View>
  );
}
```

src/components/budget/MoreTools.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState, type ReactNode } from 'react';
import { StyleSheet, View } from 'react-native';

import { Press } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { T } from '@/components/ui/T';
import { colors } from '@/theme';

import { styles as shared } from './styles';

/**
 * The things most people use once or never (the currency converter, the daily planner) folded under one line, so the page stays short.
 * Open it and they unfold in place.
 */
export function MoreTools({ children }: { children: ReactNode }) {
  const [open, setOpen] = useState(false);
  return (
    <View>
      <Rule />
      <Press onPress={() => setOpen((o) => !o)} accessibilityLabel="More budget tools" accessibilityState={{ expanded: open }} scaleTo=
{0.99} style={[shared.ledgerRow, { minHeight: 64 }]}>
        <View style={shared.iconBox}>
          <Ionicons name="calculator-outline" size={20} color={colors.ink} />
        </View>
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold">
            More budget tools
          </T>
          <T variant="caption" color={colors.textSecondary}>
            Converter and daily planner
          </T>
        </View>
        <Ionicons name={open ? 'chevron-up' : 'chevron-down'} size={20} color={colors.ink} />
```

```
      </Press>
      {open ? <View style={styles.body}>{children}</View> : <Rule weight={1} style={{ backgroundColor: colors.border }} />}
    </View>
  );
}

const styles = StyleSheet.create({
  body: { gap: 28, paddingTop: 16 },
});
```

src/components/budget/SettleSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { View } from 'react-native';

import { Avatar, Card, EmptyState } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { celebrate } from '@/components/ui/Confetti';
import { tap } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { toast } from '@/components/ui/feedback';
import type { Trip } from '@/store/types';
import { memberById, settlePlan, useAppStore } from '@/store/useAppStore';
import { colors, hair } from '@/theme';
import { money } from '@/utils/format';

import { settleExplain } from './logic';

/**
 * "Settle up": says in plain words why these payments, then lists them (who pays whom, how much) each with a Mark paid button.
 * Marking the last one paid closes the sheet and celebrates, so the SETTLED stamp is what you see behind it.
 */
export function SettleSheet({ visible, onClose, trip, onAllSettled }: { visible: boolean; onClose: () => void; trip: Trip; onAllSettle
d?: () => void }) {
  const settleUp = useAppStore((s) => s.settleUp);
  const plan = settlePlan(trip);
  return (
    <Sheet visible={visible} onClose={onClose} title="Settle up">
      {plan.length ? (
        <>
          <T variant="small" color={colors.textSecondary}>
            {settleExplain(plan.length)}
          </T>
          <Card padded={false}>
            {plan.map((p, i) => {
              const from = memberById(trip, p.from);
              const to = memberById(trip, p.to);
              return (
                <View key={`${p.from}-${p.to}`} style={[{ padding: 14, gap: 10 }, i > 0 && { borderTopWidth: hair, borderTopColor: color
s.border }]}>
                  <View style={{ flexDirection: 'row', alignItems: 'center', gap: 10 }}>
                    <Avatar name={from?.name ?? '?'} src={from?.avatar} size={34} />
                    <Ionicons name="arrow-forward" size={16} color={colors.ink} />
                    <Avatar name={to?.name ?? '?'} src={to?.avatar} size={34} />
                    <T variant="small" weight="bold" style={{ flex: 1 }} numberOfLines={2}>
                      {from?.name.split(' ')[0]} pays {to?.name.split(' ')[0]}
                    </T>
                  </View>
                  <View style={{ flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12 }}>
                    <T variant="numeralSm">{money(p.amount, 'USD', { decimals: true })}</T>
                    <Button
                      label="Mark paid"
                      size="sm"
                      variant="secondary"
                      full={false}
                      onPress={() => {
                        settleUp(trip.id, p.from, p.to, p.amount);
                        if (plan.length === 1) {
                          // that was the last payment: everyone is square
                          celebrate('All settled up');
                          onAllSettled?.();
                          onClose();
                        } else {
                          tap('success');
                          toast('Payment recorded');
                        }
                      }}
                    />
                  </View>
                </View>
              );
            })}
          </Card>
        </>
      ) : (
```

```
        <EmptyState icon="checkmark-done-outline" title="All square" body="Everyone paid their share. Nobody owes anybody.">
          <Button label="Done" onPress={onClose} />
        </EmptyState>
      )}
    </Sheet>
  );
}
```

src/components/budget/SpendSummary.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Plate } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Stamp, type StampTone } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { colors, isDark, line, radius } from '@/theme';
import { categoryPalette } from '@/theme/palettes';
import { mix } from '@/utils/contrast';

import { BigAmount } from './BigAmount';
import { barShares, budgetState, leftLabel, type BudgetTone } from './logic';

/** The one loud object on the screen is an ink block, so everything on it is paper-coloured; secondary text is paper mixed back toward
ink. */
const dim = mix(colors.ink, colors.bg, 0.72);
const faint = mix(colors.ink, colors.bg, 0.35);
/** The category colours were picked for paper. On an ink block the other mode's set reads better (and "Stays" is no longer ink on ink).
*/
const onInk = categoryPalette(isDark ? 'light' : 'dark');

const STAMP: Record<BudgetTone, StampTone> = { ok: 'positive', warn: 'warn', over: 'danger', none: 'accent' };

const stubSize = (text: string) => (text.length <= 7 ? 28 : 22);

/**
 * The top of Budget: an ink boarding pass with an accent plate. The money still to spend is the huge number, the budget sits beside it
 * (tap it to change it), a bar shows where the spending went, and a stamp says in words how the budget is doing. Below the tear line:
 * what each person has spent and what the group can spend a day.
 */
export function SpendSummary({ byCat, spent, budget, remaining, per, daysLeft, fmt, onEditBudget }: {
  byCat: [string, number][];
  spent: number;
  budget: number;
  remaining: number;
  per: number;
  daysLeft: number;
  /** Formats a US-dollar amount in the currency the user is viewing. */
  fmt: (usd: number) => string;
  onEditBudget: () => void;
}) {
  const state = budgetState(spent, budget);
  const shares = barShares(byCat, budget);
  // The stamp slams when the state changes (a new expense tips the budget to Close or Over), never on the first draw.
  const [seen, setSeen] = useState(state.tone);
  const [slam, setSlam] = useState(false);
  if (state.tone !== seen) {
    setSeen(state.tone);
    setSlam(true);
  }
  const left = fmt(Math.abs(remaining));
  const daily = fmt(Math.max(0, remaining) / daysLeft);
  const perText = fmt(per);

  return (
    <Plate color={colors.accent} corner={radius.card}>
      <Ticket
        plated={false}
        tone="ink"
        stub={
          <View style={styles.stub}>
            <View style={styles.stubCol}>
              <T variant="kicker" color={dim}>
                Per person
              </T>
              <T variant="numeral" color={colors.bg} style={{ fontSize: stubSize(perText), lineHeight: 34 }} numberOfLines={1}>
                {perText}
              </T>
            </View>
            <View style={[styles.stubCol, styles.stubSplit]}>
              <T variant="kicker" color={dim}>
                Daily allowance
              </T>
```

```
              <T variant="numeral" color={colors.bg} style={{ fontSize: stubSize(daily), lineHeight: 34 }} numberOfLines={1}>
                {daily}
              </T>
              <T variant="caption" color={dim}>
                {daysLeft} {daysLeft === 1 ? 'day' : 'days'} left
              </T>
            </View>
          </View>
        }>
        <View style={styles.main}>
          <View style={styles.topRow}>
            <T variant="kicker" color={dim} style={{ flexShrink: 1 }}>
              {leftLabel(remaining)}
            </T>
            <Stamp key={state.tone} label={state.label} tone={STAMP[state.tone]} icon={state.icon} slam={slam} />
          </View>

          <View style={styles.numRow}>
            <BigAmount text={left} color={colors.bg} testID="budget-left" />
            <Press onPress={onEditBudget} accessibilityLabel={`Group budget ${fmt(budget)}. Change it`} testID="budget-cap-open" style=
{styles.cap} scaleTo={0.97}>
              <View>
                <T variant="kicker" color={dim}>
                  of
                </T>
                <T variant="numeralSm" color={colors.bg}>
                  {fmt(budget)}
                </T>
              </View>
              <Ionicons name="create-outline" size={16} color={colors.bg} />
            </Press>
          </View>

          <View style={{ gap: 10 }}>
            <T variant="kicker" color={dim}>
              {byCat.length ? `Spent ${fmt(spent)} · ${state.pct}%` : 'Nothing spent yet'}
            </T>
            <View accessibilityElementsHidden importantForAccessibility="no-hide-descendants" style={styles.bar}>
              {shares.map((s) => (
                <View key={s.key} style={{ width: `${s.share * 100}%`, backgroundColor: onInk[s.key] ?? dim, borderRightWidth: line, bor
derRightColor: colors.ink }} />
              ))}
            </View>
            {byCat.length ? (
              <View style={styles.legend}>
                {byCat.map(([c, v]) => (
                  <View key={c} style={styles.legendItem} accessibilityLabel={`${c} ${fmt(v)}`}>
                    <View style={[styles.swatch, { backgroundColor: onInk[c] ?? dim }]} />
                    <T variant="caption" color={colors.bg} numberOfLines={1} style={{ flex: 1 }}>
                      {c}
                    </T>
                    <T variant="numeralSm" color={colors.bg} style={{ fontSize: 15, lineHeight: 20 }}>
                      {fmt(v)}
                    </T>
                  </View>
                ))}
              </View>
            ) : null}
          </View>
        </View>
      </Ticket>
    </Plate>
  );
}

const styles = StyleSheet.create({
  main: { padding: 18, gap: 14 },
  topRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 8 },
  numRow: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'flex-end', columnGap: 12, rowGap: 2 },
  cap: { flexDirection: 'row', alignItems: 'center', gap: 6, minHeight: 44, paddingBottom: 6 },
  bar: { height: 24, flexDirection: 'row', borderWidth: line, borderColor: colors.bg, borderRadius: radius.xs, overflow: 'hidden', backg
roundColor: colors.ink },
  legend: { flexDirection: 'row', flexWrap: 'wrap', columnGap: 20, rowGap: 8 },
  legendItem: { flexDirection: 'row', alignItems: 'center', gap: 8, width: '46%', minWidth: 132 },
  swatch: { width: 12, height: 12, borderWidth: 1.5, borderColor: colors.bg },
  stub: { flexDirection: 'row', paddingHorizontal: 18, paddingBottom: 16, paddingTop: 2, gap: 16 },
  stubCol: { flex: 1, gap: 2 },
  stubSplit: { paddingLeft: 16, borderLeftWidth: 1, borderLeftColor: faint },
});
```

src/components/budget/__tests__/logic.test.ts

```
import type { Expense } from '@/store/types';

import {
  EMPTY_DRAFT,
```

```
  activePayer,
  activeSplit,
  amountSize,
  balanceSentence,
  balanceView,
  barShares,
  budgetState,
  dayHeading,
  draftHasContent,
  groupByDay,
  isSettled,
  leftLabel,
  settleExplain,
  summaryLine,
} from '../logic';

const exp = (over: Partial<Expense>): Expense => ({
  id: 'e',
  title: 'Dinner',
  amount: 10,
  category: 'Food',
  paidBy: 'a',
  splitBetween: ['a', 'b'],
  date: '2026-10-20',
  ...over,
});

describe('budgetState', () => {
  it('is on track under 80 percent', () => {
    expect(budgetState(0, 4000)).toMatchObject({ tone: 'ok', label: 'On track', pct: 0 });
    expect(budgetState(3199, 4000)).toMatchObject({ tone: 'ok', label: 'On track' });
  });
  it('is close from 80 to 100 percent, both ends included', () => {
    expect(budgetState(3200, 4000)).toMatchObject({ tone: 'warn', label: 'Close', pct: 80 });
    expect(budgetState(4000, 4000)).toMatchObject({ tone: 'warn', label: 'Close', pct: 100 });
  });
  it('is over above 100 percent', () => {
    expect(budgetState(4001, 4000)).toMatchObject({ tone: 'over', label: 'Over budget' });
    expect(budgetState(5000, 4000).pct).toBe(125);
  });
  it('has nothing to judge without a budget', () => {
    expect(budgetState(100, 0).tone).toBe('none');
    expect(budgetState(100, -5).label).toBe('No limit');
  });
  it('never goes below zero', () => {
    expect(budgetState(-20, 100).ratio).toBe(0);
  });
  it('always has an icon, so the state is never colour alone', () => {
    for (const spent of [0, 3500, 9000]) expect(budgetState(spent, 4000).icon).toMatch(/-outline$/);
  });
});

describe('leftLabel', () => {
  it('says what is left, or how far over', () => {
    expect(leftLabel(1914)).toBe('Left to spend');
    expect(leftLabel(0)).toBe('Left to spend');
    expect(leftLabel(-12)).toBe('Over by');
  });
});

describe('barShares', () => {
  it('makes the bar the budget while there is room', () => {
    const s = barShares([['Stays', 1000], ['Food', 500]], 2000);
    expect(s).toEqual([{ key: 'Stays', share: 0.5 }, { key: 'Food', share: 0.25 }]);
  });
  it('makes the bar the spending once it has gone past the budget, so it fills exactly', () => {
    const s = barShares([['Stays', 3000], ['Food', 1000]], 2000);
    expect(s.reduce((n, x) => n + x.share, 0)).toBeCloseTo(1);
  });
  it('skips empty categories and copes with no budget', () => {
    expect(barShares([['Food', 0]], 100)).toEqual([]);
    expect(barShares([], 0)).toEqual([]);
  });
});

describe('amountSize', () => {
  it('is biggest for short amounts and shrinks for long ones', () => {
    expect(amountSize(6)).toBe(56);
    expect(amountSize(10)).toBeLessThan(amountSize(8));
    expect(amountSize(40)).toBe(26);
    expect(amountSize(0)).toBe(56);
  });
});

describe('balanceView', () => {
  it('words an amount owed, an amount to get back and a square balance', () => {
```

```
    expect(balanceView(-382)).toEqual({ tone: 'owes', word: 'owes', amount: 382 });
    expect(balanceView(1129)).toEqual({ tone: 'gets', word: 'gets back', amount: 1129 });
    expect(balanceView(0)).toEqual({ tone: 'square', word: 'all square', amount: 0 });
  });
  it('treats anything under 50 cents as square, on both sides', () => {
    expect(balanceView(0.49).tone).toBe('square');
    expect(balanceView(-0.49).tone).toBe('square');
    expect(balanceView(0.5).tone).toBe('gets');
    expect(balanceView(-0.5).tone).toBe('owes');
  });
});

describe('balanceSentence', () => {
  const fmt = (n: number) => `$${n}`;
  it('speaks to you and about others', () => {
    expect(balanceSentence('You', true, -382, fmt)).toBe('You owe $382');
    expect(balanceSentence('Diego', false, -436, fmt)).toBe('Diego owes $436');
    expect(balanceSentence('Priya', false, 1129, fmt)).toBe('Priya gets back $1129');
    expect(balanceSentence('You', true, 90, fmt)).toBe('You get back $90');
    expect(balanceSentence('Sam', false, 0, fmt)).toBe('Sam is all square');
    expect(balanceSentence('You', true, 0.2, fmt)).toBe('You are all square');
  });
});

describe('isSettled', () => {
  it('needs real expenses and an empty plan, so a new trip is not "settled"', () => {
    expect(isSettled(true, 0)).toBe(true);
    expect(isSettled(true, 2)).toBe(false);
    expect(isSettled(false, 0)).toBe(false);
  });
});

describe('settleExplain', () => {
  it('counts the payments in plain words', () => {
    expect(settleExplain(3)).toMatch(/^3 payments clear every balance, the fewest possible\./);
    expect(settleExplain(3)).toMatch(/money only moves from people who owe to people who are owed/);
    expect(settleExplain(1)).toMatch(/^One payment clears every balance\./);
    expect(settleExplain(0)).toMatch(/Nobody owes anybody/);
  });
});

describe('groupByDay', () => {
  const list = [
    exp({ id: 'a', date: '2026-10-19', amount: 30 }),
    exp({ id: 'b', date: '2026-10-20', amount: 50 }),
    exp({ id: 'c', date: '2026-10-20', amount: 20 }),
    exp({ id: 'd', date: '2026-10-20', amount: 99, settlement: true }),
  ];
  it('puts the newest day first and keeps the logged order inside a day', () => {
    const { groups, hidden } = groupByDay(list);
    expect(groups.map((g) => g.date)).toEqual(['2026-10-20', '2026-10-19']);
    expect(groups[0].items.map((e) => e.id)).toEqual(['b', 'c', 'd']);
    expect(hidden).toBe(0);
  });
  it('leaves settle-up payments out of a day total', () => {
    expect(groupByDay(list).groups[0].total).toBe(70);
  });
  it('shows only the newest N and says how many are left out', () => {
    const { groups, hidden } = groupByDay(list, 2);
    expect(groups.flatMap((g) => g.items.map((e) => e.id))).toEqual(['b', 'c']);
    expect(hidden).toBe(2);
  });
  it('handles no expenses', () => {
    expect(groupByDay([])).toEqual({ groups: [], hidden: 0 });
  });
  it('does not change the list it is given', () => {
    const copy = list.map((e) => e.id);
    groupByDay(list);
    expect(list.map((e) => e.id)).toEqual(copy);
  });
});

describe('dayHeading', () => {
  it('names today and yesterday, and otherwise the weekday', () => {
    expect(dayHeading('2026-10-20', '2026-10-20')).toBe('Today · Oct 20');
    expect(dayHeading('2026-10-19', '2026-10-20')).toBe('Yesterday · Oct 19');
    expect(dayHeading('2026-10-13', '2026-10-20')).toBe('Tue · Oct 13');
  });
  it('knows yesterday across a month end', () => {
    expect(dayHeading('2026-09-30', '2026-10-01')).toBe('Yesterday · Sep 30');
  });
});

describe('the half-typed expense', () => {
  const members = ['m1', 'm2', 'm3'];
  it('starts empty and notices when something was typed', () => {
```

```
    expect(draftHasContent(EMPTY_DRAFT)).toBe(false);
    expect(draftHasContent({ ...EMPTY_DRAFT, amount: ' 12 ' })).toBe(true);
    expect(draftHasContent({ ...EMPTY_DRAFT, title: '   ' })).toBe(false);
    expect(draftHasContent({ ...EMPTY_DRAFT, receipt: 'file://r.jpg' })).toBe(true);
  });
  it('splits between everyone until someone is picked, and forgets people who left', () => {
    expect(activeSplit(null, members)).toEqual(members);
    expect(activeSplit(['m1', 'gone'], members)).toEqual(['m1']);
    expect(activeSplit([], members)).toEqual([]);
  });
  it('is paid by you unless the draft names someone still on the trip', () => {
    expect(activePayer(null, 'm1', members)).toBe('m1');
    expect(activePayer('m2', 'm1', members)).toBe('m2');
    expect(activePayer('gone', 'm1', members)).toBe('m1');
  });
  it('reads as one line', () => {
    expect(summaryLine('you', members, members, 'm1')).toBe('Paid by you · split with everyone');
    expect(summaryLine('Diego', ['m1', 'm2'], members, 'm1')).toBe('Paid by Diego · split between 2');
    expect(summaryLine('you', ['m1'], members, 'm1')).toBe('Paid by you · just for you');
  });
});
```

src/components/budget/logic.ts

```
/**
 * The Budget tab's small rules, with no React in them so they can be tested: what state the budget is in, how a balance is worded, how
 * expenses group under day headings, what a half-typed expense looks like, and how the settle-up plan is explained.
 */
import type { Expense, ExpenseCategory } from '@/store/types';
import { dayLabel, weekdayShort } from '@/utils/time';

// ---------------------------------------------------------------------------------------------
// How the budget is doing

export type BudgetTone = 'ok' | 'warn' | 'over' | 'none';

export interface BudgetState {
  tone: BudgetTone;
  /** The state in words, for the stamp: a colour is never the only signal. */
  label: string;
  /** Ionicons line icon for the stamp. */
  icon: 'checkmark-circle-outline' | 'alert-circle-outline' | 'warning-outline' | 'remove-circle-outline';
  /** spent / budget, 0 when there is no budget. Can be above 1. */
  ratio: number;
  /** Whole percent of the budget used. */
  pct: number;
}

/** Under 80% of the budget is on track, 80 to 100% is close, above 100% is over. With no budget set there is no state to judge. */
export function budgetState(spent: number, budget: number): BudgetState {
  if (!(budget > 0)) return { tone: 'none', label: 'No limit', icon: 'remove-circle-outline', ratio: 0, pct: 0 };
  const ratio = Math.max(0, spent) / budget;
  const pct = Math.round(ratio * 100);
  if (ratio > 1) return { tone: 'over', label: 'Over budget', icon: 'warning-outline', ratio, pct };
  if (ratio >= 0.8) return { tone: 'warn', label: 'Close', icon: 'alert-circle-outline', ratio, pct };
  return { tone: 'ok', label: 'On track', icon: 'checkmark-circle-outline', ratio, pct };
}

/** The small label above the big number: what is left, or how far over the group is. */
export const leftLabel = (remaining: number) => (remaining < 0 ? 'Over by' : 'Left to spend');

/** How wide each category is in the stacked bar, as a share (0 to 1) of the bar. The bar is the budget, unless spending has gone past i
t. */
export function barShares(parts: [string, number][], budget: number): { key: string; share: number }[] {
  const spent = parts.reduce((s, [, v]) => s + v, 0);
  const whole = Math.max(budget, spent);
  if (!(whole > 0)) return [];
  return parts.filter(([, v]) => v > 0).map(([key, v]) => ({ key, share: v / whole }));
}

/** The biggest the numeral can be (in points) for a text of this many characters, so a long amount shrinks instead of wrapping. */
export function amountSize(chars: number): number {
  return Math.min(56, Math.max(26, Math.floor(290 / (Math.max(1, chars) * 0.58))));
}

// ---------------------------------------------------------------------------------------------
// Who owes who

/** A balance under 50 cents counts as square: the same line the settle-up plan uses. */
export const SQUARE_BELOW = 0.5;

export type BalanceTone = 'owes' | 'gets' | 'square';

export interface BalanceView {
  tone: BalanceTone;
  /** What goes before the amount: "owes", "gets back", or "all square" (which has no amount). */
```

```
  word: 'owes' | 'gets back' | 'all square';
  /** Always positive; 0 when square. */
  amount: number;
}

/** A balance in dollars (positive = is owed money) turned into words and a signed meaning. */
export function balanceView(balance: number): BalanceView {
  if (Math.abs(balance) < SQUARE_BELOW) return { tone: 'square', word: 'all square', amount: 0 };
  return balance > 0 ? { tone: 'gets', word: 'gets back', amount: balance } : { tone: 'owes', word: 'owes', amount: -balance };
}

/** The whole sentence a screen reader says for a row: "You owe $382", "Priya gets back $1,129", "Sam is all square". */
export function balanceSentence(name: string, isYou: boolean, balance: number, fmt: (usd: number) => string): string {
  const v = balanceView(balance);
  if (v.tone === 'square') return isYou ? 'You are all square' : `${name} is all square`;
  if (v.tone === 'owes') return isYou ? `You owe ${fmt(v.amount)}` : `${name} owes ${fmt(v.amount)}`;
  return isYou ? `You get back ${fmt(v.amount)}` : `${name} gets back ${fmt(v.amount)}`;
}

/** True when there is something to settle: at least one real expense and nobody is more than a few cents away from square. */
export function isSettled(hasExpenses: boolean, planLength: number): boolean {
  return hasExpenses && planLength === 0;
}

/** Plain words for the settle-up sheet: why these payments, and why there are so few. */
export function settleExplain(payments: number): string {
  if (payments <= 0) return 'Everyone paid their share. Nobody owes anybody.';
  const count = payments === 1 ? 'One payment clears every balance.' : `${payments} payments clear every balance, the fewest possible.`;
  return `${count} We add up what each person paid and what they owe, so money only moves from people who owe to people who are owed.`;
}

// ---------------------------------------------------------------------------------------------
// Expenses under day headings

export interface DayGroup {
  date: string;
  items: Expense[];
  /** What the group spent that day (settle-up payments are not spending). */
  total: number;
}

/**
 * Newest day first; inside a day the order the expenses were logged in is kept (the store adds the newest to the front).
 * `limit` shows only the newest N expenses, and `hidden` says how many are left out.
 */
export function groupByDay(expenses: Expense[], limit = Infinity): { groups: DayGroup[]; hidden: number } {
  const sorted = expenses.map((e, i) => ({ e, i })).sort((a, b) => (a.e.date < b.e.date ? 1 : a.e.date > b.e.date ? -1 : a.i - b.i));
  const shown = sorted.slice(0, limit).map((x) => x.e);
  const groups: DayGroup[] = [];
  for (const e of shown) {
    const last = groups[groups.length - 1];
    if (last && last.date === e.date) {
      last.items.push(e);
      if (!e.settlement) last.total += e.amount;
    } else groups.push({ date: e.date, items: [e], total: e.settlement ? 0 : e.amount });
  }
  return { groups, hidden: Math.max(0, sorted.length - shown.length) };
}

/** "Today · Oct 20", "Yesterday · Oct 19", "Tue · Oct 13". */
export function dayHeading(date: string, today: string): string {
  const shortDate = dayLabel(date);
  if (date === today) return `Today · ${shortDate}`;
  const y = new Date(`${today}T00:00:00Z`);
  y.setUTCDate(y.getUTCDate() - 1);
  if (date === y.toISOString().slice(0, 10)) return `Yesterday · ${shortDate}`;
  return `${weekdayShort(date)} · ${shortDate}`;
}

// ---------------------------------------------------------------------------------------------
// The half-typed expense

/**
 * What the Add expense sheet keeps while it is closed. `paidBy: null` means "me" and `split: null` means "everyone", so a draft
 * never holds on to a person who has left the trip, and it follows the crew if people join before it is sent.
 */
export interface ExpenseDraft {
  title: string;
  amount: string;
  /** The currency the amount is typed in: USD or the destination's. */
  cur: string;
  cat: ExpenseCategory;
  /** True once the person picked a category themselves; until then it follows the words in the title. */
  catTouched: boolean;
  paidBy: string | null;
  split: string[] | null;
```

```
  /** Receipt photo uri. */
  receipt: string | null;
}

export const EMPTY_DRAFT: ExpenseDraft = { title: '', amount: '', cur: 'USD', cat: 'Other', catTouched: false, paidBy: null, split: nul
l, receipt: null };

/** Anything typed at all? (Decides whether a draft is worth mentioning.) */
export const draftHasContent = (d: ExpenseDraft) => !!(d.title.trim() || d.amount.trim() || d.receipt);

/** Who the expense is split between right now: everyone when untouched, otherwise the chosen people who are still on the trip. */
export function activeSplit(split: string[] | null, memberIds: string[]): string[] {
  return split === null ? memberIds : split.filter((id) => memberIds.includes(id));
}

/** Who paid right now: the person, unless the draft names someone who is no longer on the trip. */
export function activePayer(paidBy: string | null, userId: string, memberIds: string[]): string {
  return paidBy && memberIds.includes(paidBy) ? paidBy : userId;
}

/** "Paid by you · split with everyone", "Paid by Diego · split between 3", "Paid by you · just for you". */
export function summaryLine(payer: string, split: string[], memberIds: string[], userId: string): string {
  const everyone = split.length === memberIds.length;
  const who = everyone ? 'split with everyone' : split.length === 1 && split[0] === userId ? 'just for you' : `split between ${split.len
gth}`;
  return `Paid by ${payer} · ${who}`;
}
```

src/components/budget/styles.ts

```
import { StyleSheet } from 'react-native';

import { colors, hair, line, radius } from '@/theme';

/** Styles shared by the parts of the Budget screen. */
export const styles = StyleSheet.create({
  /** The title row of a block: a serif headline with something small on the right, above an ink rule. */
  sectionTop: { flexDirection: 'row', alignItems: 'flex-end', justifyContent: 'space-between', gap: 8, minHeight: 36, marginBottom: 8 },
  /** A line of a ledger: the next row gets a hairline above it. */
  ledgerRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingVertical: 12 },
  ledgerDivider: { borderTopWidth: hair, borderTopColor: colors.border },
  /** A square outlined icon, like the stamp box in a ticket's corner. */
  iconBox: { width: 40, height: 40, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
d, alignItems: 'center', justifyContent: 'center' },
  rowBetween: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 8 },
});
```

src/components/chat/ChatBubble.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { T } from '@/components/ui/T';
import { Avatar } from '@/components/ui/bits';
import type { ChatMessage, Trip } from '@/store/types';
import { colors, hair, line, radius } from '@/theme';

import { clock, continues, dayHeading, endsRun, startsDay, systemLine } from './logic';

/** Height the clock time takes under the last bubble of a run, so the face lines up with the bubble and not with the time. */
const CLOCK_SPACE = 19;

/** A thin line with a few words in the middle: a new day, or something that happened (Sam joined the trip). */
function Quiet({ children, strong }: { children: string; strong?: boolean }) {
  return (
    <View style={styles.quiet}>
      <View style={styles.hair} />
      <T variant="kicker" color={strong ? colors.ink : colors.textSecondary} center style={{ flexShrink: 1 }}>
        {children}
      </T>
      <View style={styles.hair} />
    </View>
  );
}

/**
 * One message. Yours are filled with ink, the others sit on paper in an outline, and each has one sharp corner where it points at
 * whoever said it. The name goes on the first bubble of a run, the face and the clock time on the last.
 */
export function ChatBubble({ item, prev, next, trip, userId }: { item: ChatMessage; prev: ChatMessage | undefined; next?: ChatMessage; t
rip: Trip; userId: string }) {
  const day = startsDay(item, prev) ? <Quiet strong>{dayHeading(item.ts)}</Quiet> : null;
  if (item.kind === 'system') {
    return (
      <>
        {day}
```

```
        <Quiet>{systemLine(item.text)}</Quiet>
      </>
    );
  }
  const mine = item.authorId === userId;
  const joined = continues(item, prev);
  const last = endsRun(item, next);
  const m = trip.members.find((x) => x.id === item.authorId);
  const name = item.ai ? 'Scout AI' : (m?.name.split(' ')[0] ?? 'Former member');
  const corner = Math.min(4, radius.card);
  return (
    <>
      {day}
      <View style={[styles.msgRow, mine && { flexDirection: 'row-reverse' }, joined && { marginTop: 4 }]}>
        {!mine ? (
          <View style={{ width: 30, marginBottom: last ? CLOCK_SPACE : 0 }}>
            {last ? (
              item.ai ? (
                <View style={styles.scout}>
                  <Ionicons name="compass-outline" size={18} color={colors.onAccent} />
                </View>
              ) : (
                <Avatar name={m?.name ?? '?'} src={m?.avatar} size={30} />
              )
            ) : null}
          </View>
        ) : null}
        <View style={{ maxWidth: '78%', alignItems: mine ? 'flex-end' : 'flex-start' }}>
          {!joined && !mine ? (
            <View style={styles.nameRow}>
              <T variant="kicker" color={colors.textSecondary}>
                {name}
              </T>
              {item.ai && item.aiSource === 'claude' ? (
                <T variant="kicker" color={colors.textMuted}>
                  · Claude
                </T>
              ) : null}
            </View>
          ) : null}
          <View style={[styles.bubble, mine ? styles.mine : item.ai ? styles.ai : styles.theirs, mine ? { borderBottomRightRadius: corne
r } : { borderBottomLeftRadius: corner }]}>
            <T variant="bodySm" color={mine ? colors.onPrimary : colors.text} selectable>
              {item.text}
            </T>
          </View>
          {last ? (
            <T variant="kicker" color={colors.textMuted} style={styles.clock}>
              {clock(item.ts)}
            </T>
          ) : null}
        </View>
      </View>
    </>
  );
}

const styles = StyleSheet.create({
  quiet: { flexDirection: 'row', alignItems: 'center', gap: 10, marginTop: 16, marginBottom: 4 },
  hair: { flex: 1, height: hair, backgroundColor: colors.borderStrong },
  msgRow: { flexDirection: 'row', alignItems: 'flex-end', gap: 8, marginTop: 14 },
  nameRow: { flexDirection: 'row', gap: 4, marginBottom: 4 },
  scout: { width: 30, height: 30, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.accent, a
lignItems: 'center', justifyContent: 'center' },
  bubble: { paddingHorizontal: 14, paddingVertical: 10, borderRadius: radius.card, borderWidth: line },
  mine: { backgroundColor: colors.primary, borderColor: colors.ink },
  theirs: { backgroundColor: colors.card, borderColor: colors.ink },
  ai: { backgroundColor: colors.accentSoft, borderColor: colors.ink },
  clock: { marginTop: 4 },
});
```

src/components/chat/ChatHeader.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { SyncBadge } from '@/components/sync/SyncBadge';
import { IconButton } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Header } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import type { Trip } from '@/store/types';
import { colors } from '@/theme';
```

```
export type ChatTab = 'crew' | 'ai';
const TABS = [
  { key: 'crew' as const, label: 'Crew' },
  { key: 'ai' as const, label: 'Scout AI' },
];

/** Trip name and the head count in small print, "Trip chat" as the title, the Crew / Scout AI switch, whether the trip is live-synced,
and (in Scout AI) which brain answers. */
export function ChatHeader({ trip, tab, onTab, synced, aiMode }: { trip: Trip; tab: ChatTab; onTab: (t: ChatTab) => void; synced: boolea
n; aiMode: 'offline' | 'claude' }) {
  const t = useT();
  return (
    <View>
      <Header back={false} kicker={`${trip.name} · ${t('chat.kicker', { n: trip.members.length })}`} title={t('chat.title')} right={<Ico
nButton icon="images-outline" label="Photo dump" onPress={() => router.push('/photos')} />} />
      <View style={styles.controls}>
        <Segmented options={TABS} value={tab} onChange={onTab} />
        {synced ? (
          <View style={styles.syncRow}>
            <SyncBadge tripId={trip.id} />
            <T variant="micro" color={colors.textSecondary}>
              Shared with your group
            </T>
          </View>
        ) : null}
        {tab === 'ai' ? (
          <Press onPress={() => router.push('/settings')} style={styles.mode} accessibilityLabel="Assistant settings" scaleTo={0.99}>
            <Ionicons name={aiMode === 'claude' ? 'cloud-outline' : 'cloud-offline-outline'} size={16} color={colors.ink} />
            <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
              {aiMode === 'claude' ? 'Answers by Claude' : 'Answers from trip data'}
            </T>
            <T variant="kicker" color={colors.ink}>
              Change
            </T>
            <Ionicons name="arrow-forward" size={14} color={colors.ink} />
          </Press>
        ) : null}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  controls: { paddingHorizontal: 20, gap: 4 },
  syncRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 8 },
  mode: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 44 },
});
```

src/components/chat/Composer.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { Platform, StyleSheet, TextInput, View } from 'react-native';

import { Plate, PlatePress } from '@/components/ui/Plate';
import { Rule } from '@/components/ui/Rule';
import { useAppStore } from '@/store/useAppStore';
import { colors, fonts, line, radius } from '@/theme';

const MIN = 48;
const MAX = 120;
const PAD = 12;

/** The message box and send button at the bottom of the chat: an outlined field and a plated square, on a ruled strip of paper. */
export function Composer({ value, onChange, onSend, placeholder }: { value: string; onChange: (v: string) => void; onSend: () => void; p
laceholder: string }) {
  const [focused, setFocused] = useState(false);
  const [grown, setHeight] = useState(MIN);
  const scale = useAppStore((s) => s.settings.textScale);
  const empty = !value.trim();
  // an empty box is one line tall again (after Send); a phone shrinks it by itself as lines are deleted, a browser only grows it
  if (!value && grown !== MIN) setHeight(MIN);
  const height = value ? grown : MIN;
  return (
    <View style={styles.wrap}>
      <Rule />
      <View style={styles.bar}>
        <View style={{ flex: 1 }}>
          <Plate offset={3} corner={radius.button} color={focused ? colors.accent : 'transparent'}>
            <View style={[styles.field, { height }]}>
              <TextInput
                value={value}
                onChangeText={onChange}
                onFocus={() => setFocused(true)}
                onBlur={() => setFocused(false)}
```

```
                placeholder={placeholder}
                placeholderTextColor={colors.textMuted}
                onContentSizeChange={(e) => setHeight(Math.max(MIN, Math.min(MAX, Math.ceil(e.nativeEvent.contentSize.height) + 2 * lin
e)))}
                style={[styles.input, { fontSize: 15 * scale }]}
                multiline
                maxLength={1000}
                // on the web a text area starts two rows tall; on a phone a fixed line count would stop it growing
                {...(Platform.OS === 'web' ? { numberOfLines: 1 } : null)}
                accessibilityLabel="Message"
                onSubmitEditing={onSend}
                submitBehavior="submit"
                returnKeyType="send"
                testID="chat-input"
              />
            </View>
          </Plate>
        </View>
        <PlatePress
          onPress={onSend}
          disabled={empty}
          accessibilityLabel="Send message"
          testID="chat-send"
          offset={3}
          corner={radius.button}
          color={colors.accent}
          wrapperStyle={empty && { opacity: 0.45 }}
          style={styles.send}>
          <Ionicons name="arrow-up" size={22} color={colors.onPrimary} />
        </PlatePress>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  wrap: { backgroundColor: colors.bg },
  bar: { flexDirection: 'row', alignItems: 'flex-end', gap: 12, paddingHorizontal: 20, paddingTop: 12, paddingBottom: 14 },
  // the outline sits on a View around the input (an input is not a positioned box on the web, so a plate drawn behind it would paint ov
er it)
  field: { borderRadius: radius.button, backgroundColor: colors.card, borderWidth: line, borderColor: colors.ink, overflow: 'hidden' },
  input: {
    flex: 1,
    paddingHorizontal: 12,
    paddingVertical: PAD,
    fontFamily: fonts.regular,
    color: colors.text,
    ...(Platform.OS === 'web' ? ({ outlineStyle: 'none' } as object) : null),
  },
  send: { width: 48, height: 48, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.primary, borderWidth: line, bor
derColor: colors.ink, borderRadius: radius.button },
});
```

src/components/chat/EmptyChat.tsx

```
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { EmptyState } from '@/components/ui/bits';

/** What an empty Crew chat or an empty Scout AI thread shows: an invitation to write the first line. */
export function EmptyChat({ ai, onSay }: { ai: boolean; onSay?: (text: string) => void }) {
  return ai ? (
    <EmptyState icon="compass-outline" title="Ask Scout" body="Scout knows your budget, the weather, food nearby, who owes who and your
flights. Ask anything, or tap a question below." />
  ) : (
    <EmptyState icon="chatbubbles-outline" title="Say hi to the crew" body="Messages here are shared with everyone on the trip. Switch t
o Scout AI to ask the assistant.">
      {onSay ? (
        <View style={{ alignItems: 'center' }}>
          <Button label="Say hi" icon="hand-left-outline" variant="secondary" size="md" full={false} onPress={() => onSay('Hi, crew')} /
>
        </View>
      ) : null}
    </EmptyState>
  );
}
```

src/components/chat/__tests__/logic.test.ts

```
import type { ChatMessage } from '@/store/types';

import { clock, continues, dayHeading, endsRun, localYmd, startsDay, systemLine } from '../logic';

const at = (d: number, h: number, m = 0) => new Date(2026, 9, d, h, m).getTime();
const msg = (authorId: string, ts: number, extra: Partial<ChatMessage> = {}): ChatMessage => ({ id: `${authorId}${ts}`, authorId, text:
```

```
'hi', ts, ...extra });

describe('chat times', () => {
  it('reads the clock in 12 hour form', () => {
    expect(clock(at(27, 15, 42))).toBe('3:42 PM');
    expect(clock(at(27, 0, 5))).toBe('12:05 AM');
    expect(clock(at(27, 12, 0))).toBe('12:00 PM');
  });

  it('names today and yesterday, otherwise the weekday and date', () => {
    const now = at(27, 18);
    expect(dayHeading(at(27, 9), now)).toBe('Today');
    expect(dayHeading(at(26, 23, 59), now)).toBe('Yesterday');
    expect(dayHeading(at(20, 9), now)).toBe('Tue Oct 20');
  });

  it('uses the local calendar date', () => {
    expect(localYmd(at(5, 23, 59))).toBe('2026-10-05');
  });
});

describe('chat runs', () => {
  it('starts a new day heading on the first message and when the date changes', () => {
    expect(startsDay(msg('a', at(26, 20)), undefined)).toBe(true);
    expect(startsDay(msg('a', at(27, 8)), msg('a', at(26, 20)))).toBe(true);
    expect(startsDay(msg('a', at(26, 21)), msg('a', at(26, 20)))).toBe(false);
  });

  it('groups one person within five minutes, not across people, long gaps, days or system lines', () => {
    const a = msg('a', at(27, 10, 0));
    expect(continues(msg('a', at(27, 10, 4)), a)).toBe(true);
    expect(continues(msg('a', at(27, 10, 6)), a)).toBe(false);
    expect(continues(msg('b', at(27, 10, 1)), a)).toBe(false);
    expect(continues(msg('a', at(27, 10, 1), { kind: 'system' }), a)).toBe(false);
    expect(continues(msg('a', at(28, 0, 1)), msg('a', at(27, 23, 59)))).toBe(false);
  });

  it('puts the clock time on the last bubble of a run', () => {
    const a = msg('a', at(27, 10, 0));
    const a2 = msg('a', at(27, 10, 2));
    expect(endsRun(a, a2)).toBe(false);
    expect(endsRun(a2, undefined)).toBe(true);
    expect(endsRun(a2, msg('b', at(27, 10, 3)))).toBe(true);
  });
});

describe('systemLine', () => {
  it('drops emoji the app itself wrote and tidies the spaces', () => {
    expect(systemLine('Sam joined the trip 👋')).toBe('Sam joined the trip');
    expect(systemLine('New poll: Where to eat? 🍝')).toBe('New poll: Where to eat?');
    expect(systemLine('Plain line')).toBe('Plain line');
  });
});
```

src/components/chat/logic.ts

```
import type { ChatMessage } from '@/store/types';
import { dayLabel, time12, weekdayShort } from '@/utils/time';

/** Messages from one person within five minutes of each other share one name and one clock time. */
export const GROUP_MS = 5 * 60000;

const pad = (n: number) => String(n).padStart(2, '0');

/** A moment as the phone's own calendar date, like "2026-10-27". */
export function localYmd(ts: number): string {
  const d = new Date(ts);
  return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
}

/** "3:42 PM" in the phone's own time zone. */
export function clock(ts: number): string {
  const d = new Date(ts);
  return time12(`${pad(d.getHours())}:${pad(d.getMinutes())}`);
}

/** The label over a new day in the chat: Today, Yesterday, or "Fri Oct 30". */
export function dayHeading(ts: number, now: number = Date.now()): string {
  const ymd = localYmd(ts);
  if (ymd === localYmd(now)) return 'Today';
  if (ymd === localYmd(now - 86400000)) return 'Yesterday';
  return `${weekdayShort(ymd)} ${dayLabel(ymd)}`;
}

/** Does this message start a new day compared with the one before it? */
export const startsDay = (m: ChatMessage, prev: ChatMessage | undefined): boolean => !prev || localYmd(m.ts) !== localYmd(prev.ts);
```

```
/** The same person speaking again straight away: the second bubble hangs under the first. */
export const continues = (m: ChatMessage, prev: ChatMessage | undefined): boolean =>
  !!prev && prev.kind !== 'system' && m.kind !== 'system' && prev.authorId === m.authorId && m.ts - prev.ts < GROUP_MS && !startsDay(m,
prev);

/** The last bubble of a run: the one that carries the clock time. */
export const endsRun = (m: ChatMessage, next: ChatMessage | undefined): boolean => !next || !continues(next, m);

/** System lines are written by the app, not typed by a person, so any emoji in them is dropped ("Sam joined the trip"). */
export function systemLine(text: string): string {
  return text
    .replace(/[\p{Extended_Pictographic}]/gu, '')
    .replace(/\s+/g, ' ')
    .trim();
}
```

src/components/crew/AddMemberSheet.tsx

```
import { useState } from 'react';

import { Button } from '@/components/ui/Button';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { tap } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { useDraft } from '@/hooks/useDraft';
import { validateEmail, validateName } from '@/utils/validation';

/** Add someone by name (they can still join with the invite code later). */
export function AddMemberSheet({ visible, onClose, onAdd, onShare }: { visible: boolean; onClose: () => void; onAdd: (name: string, emai
l?: string) => void; onShare: () => void }) {
  const [name, setName, clearName] = useDraft('member-name', '');
  const [email, setEmail, clearEmail] = useDraft('member-email', '');
  const [errors, setErrors] = useState<{ name?: string | null; email?: string | null }>({});
  return (
    <Sheet visible={visible} onClose={onClose} title="Add a member" subtitle="Add someone by name, or share the invite code so they can
join themselves">
      <Button label="Share invite code instead" variant="secondary" size="md" icon="paper-plane-outline" onPress={onShare} />
      <Input label="Name" value={name} onChangeText={setName} error={errors.name} icon="person-outline" autoCapitalize="words" />
      <Input label="Email (optional)" value={email} onChangeText={setEmail} error={errors.email} icon="mail-outline" autoCapitalize="non
e" keyboardType="email-address" />
      <Button
        label="Add member"
        icon="person-add-outline"
        onPress={() => {
          const e = { name: validateName(name), email: email.trim() ? validateEmail(email) : null };
          setErrors(e);
          if (e.name || e.email) return tap('warning');
          onAdd(name, email);
          toast(`${name.trim().split(' ')[0]} added to the trip`);
          clearName();
          clearEmail();
          onClose();
        }}
      />
    </Sheet>
  );
}
```

src/components/crew/ChecklistSection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { PlatePress } from '@/components/ui/Plate';
import { Press, tap } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Avatar, Card, Divider, EmptyState, ProgressBar, SectionHeader } from '@/components/ui/bits';
import { Input } from '@/components/ui/Input';
import { useT } from '@/i18n';
import { useAppStore } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { firstError, maxLen, required } from '@/utils/validation';

/** The shared to-do list: a count and a bar that fills, big square tick boxes, who is on each job, and a line to add one. */
export function ChecklistSection({ trip, onAssign }: { trip: Trip; onAssign: (taskId: string) => void }) {
  const t = useT();
  const s = useAppStore.getState();
  const [task, setTask] = useState('');
  const [taskErr, setTaskErr] = useState<string | null>(null);
  const done = trip.tasks.filter((k) => k.done).length;
  const total = trip.tasks.length;
```

```
  const addTask = () => {
    const e = firstError(task, required('a task'), maxLen(60, 'Task'));
    setTaskErr(e);
    if (e) return tap('warning');
    s.addTask(trip.id, task);
    setTask('');
    tap('success');
  };

  return (
    <View>
      <SectionHeader title={t('hub.checklist')} />
      {total ? (
        <View style={styles.progress} accessible accessibilityLabel={`${done} of ${total} done`}>
          <View style={styles.count}>
            <T variant="title" importantForAccessibility="no">
              <T variant="numeralSm">{done}</T> of <T variant="numeralSm">{total}</T> done
            </T>
            {done === total ? <Stamp label="All done" tone="positive" icon="checkmark" rotate={0} /> : null}
          </View>
          <ProgressBar value={done / total} color={done === total ? colors.positive : colors.accent} />
        </View>
      ) : (
        <EmptyState icon="checkbox-outline" title="Nothing to do yet" body="Add the first job for the crew below." />
      )}
      {total ? (
        <Card padded={false} style={{ overflow: 'hidden' }}>
          {trip.tasks.map((k, i) => {
            const who = trip.members.find((m) => m.id === k.assignee);
            return (
              <View key={k.id}>
                {i > 0 ? <Divider /> : null}
                <View style={styles.task}>
                  <Press onPress={() => s.toggleTask(trip.id, k.id)} accessibilityRole="checkbox" accessibilityState={{ checked: k.done
}} accessibilityLabel={k.text} style={styles.hit} scaleTo={0.92}>
                    <View style={[styles.box, k.done && { backgroundColor: colors.accent }]}>{k.done ? <Ionicons name="checkmark" size=
{20} color={colors.onAccent} /> : null}</View>
                  </Press>
                  <Press onPress={() => s.toggleTask(trip.id, k.id)} style={styles.text} haptic={false} accessibilityRole="none" focusab
le={false} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
                    <T variant="bodySm" color={k.done ? colors.textMuted : colors.text} style={k.done && { textDecorationLine: 'line-thr
ough' }}>
                      {k.text}
                    </T>
                  </Press>
                  <Press onPress={() => onAssign(k.id)} style={styles.hit} accessibilityLabel={`Assign ${k.text}. Now ${who ? who.name.s
plit(' ')[0] : 'everyone'}`}>
                    {who ? (
                      <Avatar name={who.name} src={who.avatar} size={32} />
                    ) : (
                      <View style={styles.everyone}>
                        <Ionicons name="people-outline" size={16} color={colors.textSecondary} />
                      </View>
                    )}
                  </Press>
                  <Press onPress={() => s.deleteTask(trip.id, k.id)} style={styles.hit} accessibilityLabel={`Delete task ${k.text}`}>
                    <Ionicons name="close" size={20} color={colors.textSecondary} />
                  </Press>
                </View>
              </View>
            );
          })}
        </Card>
      ) : null}

      <View style={styles.add}>
        <Input
          containerStyle={{ flex: 1 }}
          value={task}
          onChangeText={(v) => {
            setTask(v);
            if (taskErr) setTaskErr(null);
          }}
          placeholder={t('hub.addTask')}
          accessibilityLabel="New task"
          error={taskErr}
          onSubmitEditing={addTask}
          returnKeyType="done"
          maxLength={70}
        />
        <PlatePress onPress={addTask} accessibilityLabel="Add task" corner={radius.button} color={colors.accent} style={styles.addBtn}>
          <Ionicons name="add" size={26} color={colors.onPrimary} />
        </PlatePress>
      </View>
    </View>
```

```
  );
}

const styles = StyleSheet.create({
  progress: { gap: 10, marginBottom: 16 },
  count: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12 },
  task: { flexDirection: 'row', alignItems: 'center', paddingLeft: 4, paddingRight: 4, minHeight: 56 },
  hit: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center' },
  box: { width: 28, height: 28, borderRadius: radius.xs + 2, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, a
lignItems: 'center', justifyContent: 'center' },
  text: { flex: 1, minHeight: 44, justifyContent: 'center', paddingHorizontal: 6, paddingVertical: 6 },
  everyone: { width: 32, height: 32, borderRadius: 16, borderWidth: 1.5, borderStyle: 'dashed', borderColor: colors.textSecondary, align
Items: 'center', justifyContent: 'center' },
  add: { flexDirection: 'row', alignItems: 'flex-start', gap: 12, marginTop: 16 },
  addBtn: { width: 52, height: 52, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.primary, borderWidth: line, b
orderColor: colors.ink, borderRadius: radius.button },
});
```

src/components/crew/InviteCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useState } from 'react';
import { StyleSheet, View, useWindowDimensions } from 'react-native';

import { Button } from '@/components/ui/Button';
import { IconButton } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { destinationById } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useT } from '@/i18n';
import { copy } from '@/services/share';
import { useAppStore } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { rangeLabel } from '@/utils/time';

import { barcodeBars } from './barcode';
import { InviteQrSheet } from './InviteQrSheet';

/** A strip of vertical ink bars made from the invite code (decoration only, like the barcode on a boarding pass). */
function Barcode({ code }: { code: string }) {
  const bars = barcodeBars(code);
  return (
    <View accessibilityElementsHidden importantForAccessibility="no-hide-descendants" style={styles.barcode}>
      {bars.map((w, i) => (
        <View key={i} style={{ flex: w, backgroundColor: i % 2 === 0 ? colors.ink : 'transparent' }} />
      ))}
    </View>
  );
}

/**
 * The invitation, as a boarding pass: a photo of the place with a paper tag on it, the trip name and dates, and under the tear line
 * the join code in big mono print with a barcode, the QR button and the one black "Invite friends" button.
 */
export function InviteCard({ trip, owner, onShare }: { trip: Trip; owner: boolean; onShare: () => void }) {
  const t = useT();
  const [qrOpen, setQrOpen] = useState(false);
  const dest = destinationById(trip.destinationId);
  const n = trip.members.length;
  // on the narrowest phones the icon would squeeze "Invite friends" onto two lines
  const narrow = useWindowDimensions().width < 360;
  return (
    <>
      <Ticket
        style={styles.ticket}
        stub={
          <View style={styles.stub}>
            <View style={styles.stubTop}>
              <T variant="kicker" color={colors.textSecondary} accessibilityRole="header">
                {t('hub.invite')}
              </T>
              {owner ? (
                <Press
                  accessibilityLabel="New code"
                  style={styles.newCode}
                  onPress={() =>
                    confirmAction('New invite code?', 'The old code will stop working. Members already on the trip stay.', () => {
                      useAppStore.getState().regenerateInvite(trip.id);
                      toast('New invite code created');
                    })
```

```
                }>
                <Ionicons name="refresh-outline" size={15} color={colors.ink} />
                <T variant="kicker" color={colors.ink}>
                  New code
                </T>
              </Press>
            ) : null}
          </View>
          <View style={styles.codeRow}>
            <T variant="numeral" numberOfLines={1} style={styles.code} accessibilityLabel={`Invite code ${trip.inviteCode}`}>
              {trip.inviteCode}
            </T>
            <IconButton icon="copy-outline" label={`Copy invite code ${trip.inviteCode}`} onPress={() => copy(trip.inviteCode, `Invite
code ${trip.inviteCode} copied`)} />
          </View>
          <Barcode code={trip.inviteCode} />
          <T variant="caption" color={colors.textSecondary}>
            Friends type this in Join a trip, or scan the QR code.
          </T>
          <View style={styles.actions}>
            <PlatePress onPress={() => setQrOpen(true)} testID="invite-qr" accessibilityLabel="QR code" corner={radius.button} style=
{styles.qr}>
              <Ionicons name="qr-code-outline" size={24} color={colors.ink} />
            </PlatePress>
            <Button label="Invite friends" icon={narrow ? undefined : 'paper-plane-outline'} onPress={onShare} style={styles.invite} /
>
          </View>
        </View>
      }>
      <View style={styles.photo}>
        <Image source={imageSource(trip.cover)} style={StyleSheet.absoluteFill} contentFit="cover" accessibilityIgnoresInvertColors />
        <View style={styles.tag}>
          <T variant="kicker" color={colors.ink}>
            Invitation
          </T>
        </View>
      </View>
      <View style={styles.main}>
        <T variant="kicker" color={colors.textSecondary}>
          {dest ? `${dest.city}, ${dest.country}` : 'Somewhere great'}
        </T>
        <T variant="headline" numberOfLines={2} style={{ fontSize: 30, lineHeight: 34 }}>
          {trip.name}
        </T>
        <T variant="kicker" color={colors.textSecondary}>
          {rangeLabel(trip.startDate, trip.endDate)} / {n} {n === 1 ? 'traveller' : 'travellers'}
        </T>
      </View>
    </Ticket>
    <InviteQrSheet visible={qrOpen} onClose={() => setQrOpen(false)} trip={trip} />
  </>
);
}

const CORNER = Math.max(0, radius.card - line);

const styles = StyleSheet.create({
  ticket: { marginTop: 4 },
  photo: { height: 120, borderTopLeftRadius: CORNER, borderTopRightRadius: CORNER, borderBottomWidth: line, borderBottomColor: colors.in
k, overflow: 'hidden', backgroundColor: colors.bgAlt },
  tag: { position: 'absolute', left: 12, bottom: 12, paddingHorizontal: 8, paddingVertical: 3, backgroundColor: colors.card, borderWidt
h: line, borderColor: colors.ink, borderRadius: radius.xs },
  main: { padding: 16, gap: 6 },
  stub: { paddingHorizontal: 16, paddingBottom: 16, paddingTop: 2, gap: 12 },
  stubTop: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', minHeight: 44 },
  newCode: { flexDirection: 'row', alignItems: 'center', gap: 6, minHeight: 44, paddingLeft: 12 },
  codeRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12, marginTop: -8 },
  code: { flexShrink: 1, letterSpacing: 1 },
  barcode: { flexDirection: 'row', height: 40 },
  actions: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', gap: 12 },
  qr: { width: 56, height: 56, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.card, borderWidth: line, borderCo
lor: colors.ink, borderRadius: radius.button },
  invite: { flexGrow: 1, flexBasis: 150 },
});
```

src/components/crew/InviteQrSheet.tsx

```
import { StyleSheet, View, useWindowDimensions } from 'react-native';

import { QrCode } from '@/components/QrCode';
import { Plate } from '@/components/ui/Plate';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { inviteLink } from '@/services/share';
import type { Trip } from '@/store/types';
import { parseInviteUrl } from '@/sync/invite';
```

```
import { colors, line, radius } from '@/theme';

/** The invite as a QR code on a paper tile: a friend points their phone camera at it and the app opens on the join screen. */
export function InviteQrSheet({ visible, onClose, trip }: { visible: boolean; onClose: () => void; trip: Trip }) {
  const { width } = useWindowDimensions();
  const link = inviteLink(trip.inviteCode);
  const size = Math.min(260, width - 112);
  return (
    <Sheet visible={visible} onClose={onClose} title="Scan to join" subtitle={trip.name}>
      <View style={styles.center}>
        <Plate corner={radius.card}>
          <View style={styles.tile}>
            <QrCode value={link} size={size} accessibilityLabel={`QR code for invite ${trip.inviteCode}`} />
          </View>
        </Plate>
        <View style={styles.codeBlock}>
          <T variant="kicker" color={colors.textSecondary}>
            Invite code
          </T>
          <T variant="numeral" accessibilityLabel={`Invite code ${trip.inviteCode}`}>
            {trip.inviteCode}
          </T>
        </View>
        <T variant="small" color={colors.textSecondary} center>
          Friends point their phone camera here, or open TogetherWeGo, then Join a trip, then Scan, or type the code.
        </T>
        {parseInviteUrl(link).server ? (
          <T variant="caption" color={colors.textMuted} center>
            It also carries your sync server’s address, so their phone connects by itself.
          </T>
        ) : null}
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  center: { alignItems: 'center', gap: 16, paddingBottom: 8 },
  // the code inside is always black on white, whatever the theme, because that is what a camera reads best
  tile: { padding: 12, borderRadius: radius.card, backgroundColor: colors.white, borderWidth: line, borderColor: colors.ink },
  codeBlock: { alignItems: 'center', gap: 2 },
});
```

src/components/crew/MembersSection.tsx

```
import { StyleSheet, View } from 'react-native';

import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Avatar, Card, Divider, SectionHeader } from '@/components/ui/bits';
import { useT } from '@/i18n';
import type { Member, Trip } from '@/store/types';
import { colors } from '@/theme';

/**
 * Everyone on the trip, like the passenger list of a flight: a face, a name, a role in small print and a stamp that says
 * GOING or, for someone added by name who has not joined yet, INVITED. Tap a person to change their role.
 * The green dot on a face means "here now"; the app only knows that about you.
 */
export function MembersSection({ trip, userId, onPick, onAdd }: { trip: Trip; userId: string; onPick: (m: Member) => void; onAdd: () =>
void }) {
  const t = useT();
  return (
    <View>
      <SectionHeader title={t('hub.members')} action="+ Add" onAction={onAdd} />
      <Card padded={false} style={{ overflow: 'hidden' }}>
        {trip.members.map((m, i) => (
          <View key={m.id}>
            {i > 0 ? <Divider /> : null}
            <Press onPress={() => onPick(m)} style={styles.member} scaleTo={0.99} accessibilityLabel={`${m.name}, ${m.role}. Tap to chan
ge role`}>
              <Avatar name={m.name} src={m.avatar} size={44} dot={m.id === userId ? colors.positive : undefined} />
              <View style={styles.who}>
                <T variant="title" weight="bold" numberOfLines={2}>
                  {m.name}
                  {m.id === userId ? ' (you)' : ''}
                </T>
                <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
                  {m.role}
                </T>
                {m.invited || m.homeCity ? (
                  <T variant="caption" color={colors.textMuted} numberOfLines={1}>
                    {m.invited ? 'Hasn’t joined yet' : `From ${m.homeCity}`}
                  </T>
                ) : null}
```

```
              </View>
              {m.invited ? <Stamp label="Invited" tone="warn" rotate={0} /> : <Stamp label="Going" tone="positive" rotate={0} />}
            </Press>
          </View>
        ))}
      </Card>
    </View>
  );
}

const styles = StyleSheet.create({
  member: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12, minHeight: 68 },
  who: { flex: 1, gap: 1 },
});
```

src/components/crew/NewPollSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { PlatePress } from '@/components/ui/Plate';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { useDraft } from '@/hooks/useDraft';
import { colors, line, radius } from '@/theme';

/** Start a poll: a question and two to five answers. A half-written poll is kept if the sheet is closed by accident. */
export function NewPollSheet({ visible, onClose, onCreate }: { visible: boolean; onClose: () => void; onCreate: (q: string, opts: string
[]) => void }) {
  const [q, setQ, clearQ] = useDraft('poll-question', '');
  const [opts, setOpts, clearOpts] = useDraft('poll-options', ['', '']);
  const [err, setErr] = useState<string | null>(null);
  return (
    <Sheet visible={visible} onClose={onClose} title="New poll" subtitle="Let the group decide">
      <Input label="Question" value={q} onChangeText={setQ} placeholder="e.g. Where should we eat on Day 3?" maxLength={80} />
      <View style={{ gap: 10 }}>
        <T variant="kicker" color={colors.ink}>
          Answers
        </T>
        {opts.map((o, i) => (
          <View key={i} style={styles.option}>
            <Input
              containerStyle={{ flex: 1 }}
              value={o}
              onChangeText={(v) => setOpts((arr) => arr.map((x, j) => (j === i ? v : x)))}
              placeholder={`Answer ${i + 1}`}
              maxLength={50}
              accessibilityLabel={`Option ${i + 1}`}
            />
            {opts.length > 2 ? (
              <PlatePress onPress={() => setOpts((arr) => arr.filter((_, j) => j !== i))} accessibilityLabel={`Remove option ${i + 1}`}
offset={3} corner={radius.button} style={styles.remove}>
                <Ionicons name="close" size={20} color={colors.ink} />
              </PlatePress>
            ) : null}
          </View>
        ))}
        {opts.length < 5 ? <Button label="Add an answer" variant="secondary" size="md" icon="add" onPress={() => setOpts((a) => [...a,
'']))} /> : null}
      </View>
      {err ? (
        <T variant="caption" color={colors.red} accessibilityLiveRegion="polite">
          {err}
        </T>
      ) : null}
      <Button
        label="Start poll"
        icon="stats-chart-outline"
        onPress={() => {
          const clean = opts.map((o) => o.trim()).filter(Boolean);
          if (!q.trim()) return setErr('Write a question for the poll');
          if (clean.length < 2) return setErr('Add at least two options');
          if (new Set(clean.map((c) => c.toLowerCase())).size !== clean.length) return setErr('Options must be different');
          onCreate(q, clean);
          clearQ();
          clearOpts();
          setErr(null);
          toast('Poll started. The group has been notified');
          onClose();
        }}
      />
    </Sheet>
```

```
  );
}

const styles = StyleSheet.create({
  option: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
  remove: { width: 52, height: 52, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.card, borderWidth: line, bord
erColor: colors.ink, borderRadius: radius.button },
});
```

src/components/crew/PollsSection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { celebrate } from '@/components/ui/Confetti';
import { confirmAction } from '@/components/ui/feedback';
import { Press, tap } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { AvatarStack, Card, Divider, EmptyState, ProgressBar, SectionHeader } from '@/components/ui/bits';
import { useAppStore } from '@/store/useAppStore';
import type { Poll, Trip } from '@/store/types';
import { colors, line, radius, space } from '@/theme';

import { pollView } from './polls';

/** The group's polls: vote, close or delete, and start a new one. Each answer is an ink bar that grows as the votes come in. */
export function PollsSection({ trip, userId, owner, onNew }: { trip: Trip; userId: string; owner: boolean; onNew: () => void }) {
  return (
    <View>
      <SectionHeader title="Polls" action={trip.polls.length ? '+ New poll' : undefined} onAction={onNew} />
      {trip.polls.length ? (
        <View style={{ gap: 16 }}>
          {trip.polls.map((p) => (
            <PollCard key={p.id} poll={p} trip={trip} userId={userId} canManage={p.createdBy === userId || owner} />
          ))}
        </View>
      ) : (
        <EmptyState icon="stats-chart-outline" title="No polls yet" body="Put a question to the crew and watch the votes come in.">
          <Button label="New poll" icon="add" onPress={onNew} />
        </EmptyState>
      )}
    </View>
  );
}

function PollCard({ poll: p, trip, userId, canManage }: { poll: Poll; trip: Trip; userId: string; canManage: boolean }) {
  const s = useAppStore.getState();
  const view = pollView(p, userId, trip.members.map((m) => m.id));
  const voted = view.rows.some((r) => r.mine);
  // the stamp slams down on the poll the person has just closed, and only then
  const [slam, setSlam] = useState(false);

  const closeOrReopen = () => {
    const winner = view.rows.find((r) => r.id === view.winnerId);
    s.closePoll(trip.id, p.id);
    if (!p.closed && winner) {
      setSlam(true);
      celebrate(`Locked in: ${winner.text}`);
    }
  };

  return (
    <Card style={{ gap: 14 }}>
      <View style={styles.top}>
        <T variant="h3" style={{ flex: 1 }}>
          {p.question}
        </T>
        {view.stamp === 'locked' ? <Stamp label="Locked in" tone="positive" icon="lock-closed" slam={slam} /> : view.stamp === 'closed'
? <Stamp label="Closed" tone="muted" slam={slam} /> : <Stamp label="Open" tone="ink" rotate={0} />}
      </View>
      <T variant="kicker" color={colors.textSecondary} style={{ marginTop: -8 }}>
        {view.total} {view.total === 1 ? 'vote' : 'votes'}
      </T>

      <View style={{ gap: 4 }}>
        {view.rows.map((o) => {
          const winner = p.closed && o.id === view.winnerId;
          return (
            <Press
              key={o.id}
              disabled={p.closed}
              onPress={() => {
                s.votePoll(trip.id, p.id, o.id);
```

```
              tap();
            }}
            scaleTo={0.99}
            style={styles.option}
            accessibilityState={{ selected: o.mine, disabled: !!p.closed }}
            accessibilityLabel={`${o.text}, ${o.pct} percent${o.mine ? ', your vote' : ''}`}>
            <View style={styles.optionTop}>
              {p.closed ? (
                o.mine ? <Ionicons name="checkmark" size={18} color={colors.ink} /> : null
              ) : (
                <View style={[styles.box, o.mine && { backgroundColor: colors.accent }]}>{o.mine ? <Ionicons name="checkmark" size={1
6} color={colors.onAccent} /> : null}</View>
              )}
              <T variant="bodySm" weight={o.mine || winner ? 'bold' : 'regular'} style={{ flex: 1 }}>
                {o.text}
              </T>
              <T variant="numeralSm" accessibilityElementsHidden importantForAccessibility="no">
                {o.pct}%
              </T>
            </View>
            <ProgressBar value={o.pct / 100} color={winner ? colors.accent : colors.primary} track={colors.bgAlt} height={14} />
            {o.voters.length ? (
              <View style={styles.voters}>
                <AvatarStack people={o.voters.map((id) => trip.members.find((m) => m.id === id) ?? { name: 'Former member' })} max={6}
size={26} />
              </View>
            ) : null}
          </Press>
        );
      })}
    </View>

    <T variant="caption" color={colors.textMuted}>
      {p.closed ? (view.tie ? 'Voting closed. It was a tie.' : 'Voting closed') : voted ? 'Tap again to remove your vote' : 'Tap an an
swer to vote'}
    </T>

    {canManage ? (
      <>
        <Divider style={{ marginTop: -4 }} />
        <View style={styles.manage}>
          <Button label={p.closed ? 'Reopen' : 'Close poll'} variant="secondary" size="sm" full={false} icon={p.closed ? 'lock-open-ou
tline' : 'lock-closed-outline'} onPress={closeOrReopen} />
          <Press
            onPress={() => confirmAction('Delete poll?', p.question, () => s.deletePoll(trip.id, p.id), { destructive: true, confirmLa
bel: 'Delete' })}
            accessibilityLabel={`Delete poll ${p.question}`}
            style={styles.delete}>
            <Ionicons name="trash-outline" size={16} color={colors.red} />
            <T variant="kicker" color={colors.red}>
              Delete
            </T>
          </Press>
        </View>
      </>
    ) : null}
  </Card>
 );
}

const styles = StyleSheet.create({
 top: { flexDirection: 'row', alignItems: 'flex-start', justifyContent: 'space-between', gap: 12 },
 option: { gap: 8, paddingVertical: 8, minHeight: 56 },
 optionTop: { flexDirection: 'row', alignItems: 'center', gap: 10 },
 box: { width: 22, height: 22, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, align
Items: 'center', justifyContent: 'center' },
 voters: { flexDirection: 'row', paddingLeft: 0 },
 manage: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: space.md },
 delete: { flexDirection: 'row', alignItems: 'center', gap: 6, minHeight: 44, paddingHorizontal: 12 },
});
```

src/components/crew/RoleSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Press } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { ROLE_OPTIONS, roleDescriptions, type Role } from '@/data/reference';
import type { Member } from '@/store/types';
import { colors, line, radius } from '@/theme';

/** Pick a role for a member (or remove them from the trip, if you are the organizer). The chosen role is filled with the highlighter.
```

```
*/
export function RoleSheet({ member, canEdit, canRemove, onClose, onPick, onRemove }: { member: Member | null; canEdit: boolean; canRemov
e: boolean; onClose: () => void; onPick: (r: Role) => void; onRemove: () => void }) {
  return (
    <Sheet visible={!!member} onClose={onClose} title={member?.name} subtitle={canEdit ? 'Choose a role for this trip' : 'Only the organ
izer can change other people’s roles'}>
      <View style={{ gap: 10 }}>
        {ROLE_OPTIONS.map((r) => {
          const on = member?.role === r;
          return (
            <Press
              key={r}
              disabled={!canEdit}
              onPress={() => onPick(r)}
              scaleTo={0.99}
              style={[styles.roleRow, on && { backgroundColor: colors.accent }, !canEdit && !on && { opacity: 0.6 }]}
              accessibilityLabel={r}
              accessibilityState={{ selected: on, disabled: !canEdit }}>
              <View style={{ flex: 1, gap: 1 }}>
                <T variant="title" weight="bold" color={on ? colors.onAccent : colors.text}>
                  {r}
                </T>
                <T variant="caption" color={on ? colors.onAccent : colors.textSecondary}>
                  {roleDescriptions[r]}
                </T>
              </View>
              {on ? <Ionicons name="checkmark" size={20} color={colors.onAccent} /> : null}
            </Press>
          );
        })}
      </View>
      {canRemove ? (
        <>
          <Rule />
          <Button label="Remove from trip" variant="danger" size="md" icon="person-remove-outline" onPress={onRemove} />
        </>
      ) : null}
    </Sheet>
  );
}

const styles = StyleSheet.create({
  roleRow: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingHorizontal: 14, paddingVertical: 10, borderRadiu
s: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
});
```

src/components/crew/__tests__/crew.test.ts

```
import type { Poll } from '@/store/types';

import { barcodeBars } from '../barcode';
import { pollView } from '../polls';

const poll = (votes: string[][], closed = false): Poll => ({
  id: 'p1',
  question: 'Where to eat?',
  createdBy: 'a',
  createdAt: 0,
  closed,
  options: votes.map((v, i) => ({ id: `o${i}`, text: `Option ${i}`, votes: v })),
});

describe('pollView', () => {
  it('counts votes and rounds the percentages', () => {
    const v = pollView(poll([['a', 'b'], ['c'], []]), 'c');
    expect(v.total).toBe(3);
    expect(v.rows.map((r) => r.pct)).toEqual([67, 33, 0]);
    expect(v.rows.map((r) => r.mine)).toEqual([false, true, false]);
  });

  it('has no percentages and no winner before anyone votes', () => {
    const v = pollView(poll([[], []]), 'a');
    expect(v.rows.map((r) => r.pct)).toEqual([0, 0]);
    expect(v.winnerId).toBeNull();
    expect(v.tie).toBe(false);
  });

  it('names the winner only when one option is clearly ahead', () => {
    expect(pollView(poll([['a', 'b'], ['c']]), 'a').winnerId).toBe('o0');
    const level = pollView(poll([['a'], ['b']]), 'a');
    expect(level.winnerId).toBeNull();
    expect(level.tie).toBe(true);
  });

  it('says OPEN while voting, LOCKED IN with a winner and CLOSED without one', () => {
    expect(pollView(poll([['a'], []]), 'a').stamp).toBe('open');
```

```
    expect(pollView(poll([['a'], []], true), 'a').stamp).toBe('locked');
    expect(pollView(poll([['a'], ['b']], true), 'a').stamp).toBe('closed');
    expect(pollView(poll([[], []], true), 'a').stamp).toBe('closed');
  });

  it('lists voters in the order of the trip members', () => {
    const v = pollView(poll([['c', 'a', 'b']]), 'a', ['a', 'b', 'c']);
    expect(v.rows[0].voters).toEqual(['a', 'b', 'c']);
  });
});

describe('barcodeBars', () => {
  it('is the same strip for the same code and a different one for another code', () => {
    expect(barcodeBars('AML-2026')).toEqual(barcodeBars('AML-2026'));
    expect(barcodeBars('AML-2026')).not.toEqual(barcodeBars('SEO-5521'));
  });

  it('starts and ends on a bar (an odd number of parts) with weights from 1 to 4', () => {
    for (const n of [40, 41]) {
      const bars = barcodeBars('AML-2026', n);
      expect(bars.length % 2).toBe(1);
      expect(Math.min(...bars)).toBeGreaterThanOrEqual(1);
      expect(Math.max(...bars)).toBeLessThanOrEqual(4);
    }
  });

  it('has thin guard bars at both ends', () => {
    const bars = barcodeBars('XYZ-0000');
    expect(bars.slice(0, 3)).toEqual([1, 1, 1]);
    expect(bars.slice(-3)).toEqual([1, 1, 1]);
  });
});
```

src/components/crew/barcode.ts

```
/**
 * The strip of bars on the stub of the invitation. It is decoration only (hidden from screen readers), but it is made from the
 * invite code, so every trip prints its own strip and the same code always prints the same one.
 *
 * Returns a list of weights that alternate bar, gap, bar, gap... Each weight is 1 to 4. The strip always starts and ends on a bar,
 * and the first and last two bars are the same thin pair, like the guard bars of a real barcode.
 */
export function barcodeBars(code: string, count = 41): number[] {
  const n = count % 2 === 0 ? count + 1 : count; // bar, gap, bar ... bar: an odd number of parts
  let seed = 17;
  for (const ch of code) seed = (seed * 31 + ch.charCodeAt(0)) % 2147483647;
  const out: number[] = [];
  for (let i = 0; i < n; i++) {
    seed = (seed * 48271) % 2147483647;
    out.push(1 + (seed % 4));
  }
  // guard bars: thin, thin, gap, ... gap, thin, thin
  out[0] = 1;
  out[1] = 1;
  out[2] = 1;
  out[n - 1] = 1;
  out[n - 2] = 1;
  out[n - 3] = 1;
  return out;
}
```

src/components/crew/polls.ts

```
import type { Poll } from '@/store/types';

export interface PollRow {
  id: string;
  text: string;
  count: number;
  /** Whole percent of all votes cast (0 when nobody has voted). */
  pct: number;
  /** Who voted for it, in the order of the trip's member list. */
  voters: string[];
  mine: boolean;
}

export interface PollView {
  rows: PollRow[];
  total: number;
  /** The option with the most votes, or null when nobody voted or two options are level. */
  winnerId: string | null;
  /** The first place is shared by two or more options. */
  tie: boolean;
  /** What the stamp says: OPEN while voting, LOCKED IN when closed with a winner, CLOSED when closed without one. */
  stamp: 'open' | 'locked' | 'closed';
}
```

```
/** The numbers a poll card draws: counts, percentages, who voted, who is ahead and what state it is in. */
export function pollView(poll: Poll, userId: string, memberIds: string[] = []): PollView {
  const total = poll.options.reduce((n, o) => n + o.votes.length, 0);
  const rows: PollRow[] = poll.options.map((o) => ({
    id: o.id,
    text: o.text,
    count: o.votes.length,
    pct: total ? Math.round((o.votes.length / total) * 100) : 0,
    voters: [...o.votes].sort((a, b) => memberIds.indexOf(a) - memberIds.indexOf(b)),
    mine: o.votes.includes(userId),
  }));
  const top = Math.max(0, ...rows.map((r) => r.count));
  const leaders = top > 0 ? rows.filter((r) => r.count === top) : [];
  const winnerId = leaders.length === 1 ? leaders[0].id : null;
  return {
    rows,
    total,
    winnerId,
    tie: leaders.length > 1,
    stamp: !poll.closed ? 'open' : winnerId ? 'locked' : 'closed',
  };
}
```

src/components/documents/DocFileBody.tsx

```
import { Image } from 'expo-image';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Skeleton } from '@/components/ui/Skeleton';
import { T } from '@/components/ui/T';
import type { TripDoc } from '@/store/types';
import { useSharedFile } from '@/sync/useFile';
import { colors, line, radius } from '@/theme';

/** The file part of an opened document: a preview for pictures, an "Open / share" button, or why there is no file here. */
export function DocFileBody({ doc, adderName, onOpen }: { doc: TripDoc; adderName: string; onOpen: (uri: string) => void }) {
  const { uri, loading } = useSharedFile(doc.uri, doc.blob);
  if (doc.kind === 'note') return null;
  if (!uri) {
    return loading ? (
      <View style={{ gap: 10 }}>
        <Skeleton height={120} />
        <T variant="small" color={colors.textSecondary}>
          Getting the file from {adderName}…
        </T>
      </View>
    ) : (
      <T variant="small" color={colors.textSecondary}>
        This file is on {adderName}’s phone. Its name and notes sync; the file arrives once their phone has sent it to the sync server.
      </T>
    );
  }
  return (
    <>
      {doc.kind === 'image' ? <Image source={{ uri }} style={styles.preview} contentFit="contain" accessibilityIgnoresInvertColors /> :
null}
      <Button label="Open / share file" icon="open-outline" onPress={() => onOpen(uri)} />
    </>
  );
}

const styles = StyleSheet.create({
  preview: { width: '100%', height: 320, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.
bgAlt },
});
```

src/components/documents/DocRow.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { sizeLabel } from '@/services/files';
import type { Trip, TripDoc } from '@/store/types';
import { useSharedFile } from '@/sync/useFile';
import { colors, line, radius } from '@/theme';
import { timeAgo } from '@/utils/format';

/** One line icon for each kind of file, so the vault reads at a glance. */
export const DOC_ICON = { image: 'image-outline', note: 'create-outline', pdf: 'document-text-outline', file: 'attach-outline' } as cons
t;
export const docKind = (d: Pick<TripDoc, 'kind'>) => (d.kind === 'note' ? 'Note' : d.kind.toUpperCase());
```

```
/** One line in the document vault. The stamp says whether the file can be opened with no connection. */
export function DocRow({ doc, trip, onPress }: { doc: TripDoc; trip: Trip; onPress: () => void }) {
  const { uri, loading } = useSharedFile(doc.uri, doc.blob);
  const adder = trip.members.find((m) => m.id === doc.addedBy)?.name.split(' ')[0];
  return (
    <Press onPress={onPress} style={styles.row} scaleTo={0.99} accessibilityLabel={`Open ${doc.name}`}>
      <View style={styles.icon}>
        <Ionicons name={DOC_ICON[doc.kind]} size={22} color={colors.ink} />
      </View>
      <View style={{ flex: 1, gap: 4 }}>
        <T variant="title" weight="bold" numberOfLines={1}>
          {doc.name}
        </T>
        <View style={styles.meta}>
          <T variant="kicker" color={colors.textSecondary} numberOfLines={1} style={{ flexShrink: 1 }}>
            {docKind(doc)}
            {doc.size ? ` / ${sizeLabel(doc.size)}` : ''} / {timeAgo(doc.addedAt)}
            {adder ? ` / ${adder}` : ''}
          </T>
          {doc.kind === 'note' || uri ? (
            <Stamp label="Offline" tone="positive" rotate={0} />
          ) : loading ? (
            <Stamp label="Downloading" tone="muted" rotate={0} />
          ) : (
            <Stamp label={`On ${adder ?? 'a friend'}’s phone`} tone="warn" rotate={0} />
          )}
        </View>
      </View>
    </Press>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 68, paddingHorizontal: 12, paddingVertical: 10 },
  meta: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  icon: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt,
alignItems: 'center', justifyContent: 'center' },
});
```

src/components/explore.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { destinationById, type Destination } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';
import { money } from '@/utils/format';

import { ListRow } from './ui/bits';
import { toast } from './ui/feedback';
import { PlatePress } from './ui/Plate';
import { Press } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { Stamp } from './ui/Stamp';
import { T } from './ui/T';

/**
 * The favourite button: an outlined square (accent when saved) that sits NEXT TO a card's own button, never inside it.
 * It is always at least 44 x 44.
 */
export function FavStar({ id, size = 44 }: { id: string; size?: number }) {
  const fav = useAppStore((s) => s.favorites.includes(id));
  const toggle = useAppStore((s) => s.toggleFavorite);
  const face = Math.max(size, 44);
  return (
    <PlatePress
      onPress={() => {
        toggle(id);
        toast(fav ? 'Removed from favourites' : 'Saved to favourites', { icon: fav ? 'star-outline' : 'star' });
      }}
      accessibilityLabel={fav ? 'Remove favourite' : 'Save as favourite'}
      accessibilityState={{ selected: fav }}
      offset={3}
      corner={radius.button}
      style={[styles.star, { width: face, height: face, backgroundColor: fav ? colors.accent : colors.card }]}>
      <Ionicons name={fav ? 'star' : 'star-outline'} size={Math.round(face * 0.46)} color={fav ? colors.onAccent : colors.ink} />
    </PlatePress>
  );
}
```

```
/** A photo of a place in an ink outline, with the name over a dark scrim. */
export function PlaceCard({ d, style, height = 190 }: { d: Destination; style?: StyleProp<ViewStyle>; height?: number }) {
  return (
    // The star sits next to the card button, not inside it: on the web a button inside a button is invalid HTML.
    <View style={[styles.place, { height }, style]}>
      <Press onPress={() => router.push({ pathname: '/destination/[id]', params: { id: d.id } })} scaleTo={0.985} style={StyleSheet.abso
luteFill} accessibilityLabel={`${d.city}, ${d.country}, rated ${d.rating}`}>
        <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={200} />
        <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.78)']} locations={[0.4, 1]} style={StyleSheet.absol
uteFill} />
        <View style={styles.placeText} pointerEvents="none">
          <T variant="h3" color={colors.white} numberOfLines={1}>
            {d.city}
          </T>
          <View style={styles.row}>
            <T variant="kicker" color={colors.white} numberOfLines={1} style={{ flex: 1, fontSize: 10 }}>
              {d.country}
            </T>
            <Ionicons name="star" size={11} color={colors.white} />
            <T variant="kicker" color={colors.white} style={{ fontSize: 11 }}>
              {d.rating.toFixed(1)}
            </T>
          </View>
        </View>
      </Press>
      <View style={styles.starSlot}>
        <FavStar id={d.id} />
      </View>
    </View>
  );
}

export function AddToTripSheet({ destinationId, onClose }: { destinationId: string | null; onClose: () => void }) {
  const trip = useActiveTrip();
  const updateTrip = useAppStore((s) => s.updateTrip);
  const addBucket = useAppStore((s) => s.addBucket);
  const d = destinationId ? destinationById(destinationId) : undefined;
  if (!d) return <Sheet visible={false} onClose={onClose}>{null}</Sheet>;
  const already = trip && (trip.destinationId === d.id || trip.extraDestinationIds.includes(d.id));
  return (
    <Sheet visible={!!destinationId} onClose={onClose} title={`Add ${d.city}`} subtitle={`${d.country} · rated ${d.rating} · about ${mon
ey(d.dailyCost)} a day per person`}>
      {trip ? (
        <ListRow
          icon="add-circle-outline"
          label={already ? `Already part of ${trip.name}` : `Add as a stop on “${trip.name}”`}
          sub={already ? 'Its places show up in Nearby and the assistant' : 'Its places appear in Nearby, the map and assistant suggesti
ons'}
          onPress={() => {
            if (!already) {
              updateTrip(trip.id, { extraDestinationIds: [...trip.extraDestinationIds, d.id] });
              toast(`${d.city} added to ${trip.name}`);
            }
            onClose();
          }}
        />
      ) : null}
      <ListRow
        icon="airplane-outline"
        label="Start a new group trip here"
        sub="Pick dates, set a budget and invite friends"
        onPress={() => {
          onClose();
          setTimeout(() => router.push({ pathname: '/new-trip', params: { destination: d.id } }), 180);
        }}
      />
      <ListRow
        icon="flag-outline"
        label="Save to bucket list"
        onPress={() => {
          addBucket(`Visit ${d.city}, ${d.country}`, d.id);
          toast('Saved to your bucket list');
          onClose();
        }}
      />
      <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 8 }}>
        {d.highlights.slice(0, 4).map((h) => (
          <Stamp key={h} label={h} tone="muted" rotate={0} />
        ))}
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  star: { borderWidth: line, borderColor: colors.ink, alignItems: 'center', justifyContent: 'center' },
```

```
  starSlot: { position: 'absolute', top: 8, right: 11 },
  place: { borderRadius: radius.card, overflow: 'hidden', backgroundColor: colors.bgAlt, borderWidth: line, borderColor: colors.ink },
  placeText: { position: 'absolute', left: 12, right: 12, bottom: 10, gap: 2 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 4 },
});
```

src/components/flights/AddFlightSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { DatePickerSheet, TimePickerSheet } from '@/components/pickers';
import { Button } from '@/components/ui/Button';
import { Chip, ToggleRow } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Segmented } from '@/components/ui/Segmented';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { airportByIata } from '@/data/airports';
import { destinationById } from '@/data/destinations';
import { useDraft } from '@/hooks/useDraft';
import { FileTooLargeError, pickDocument, pickImage } from '@/services/files';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';
import { distanceKm, kmLabel } from '@/utils/geo';
import { dayLabel, time12 } from '@/utils/time';

import { blankFlight, checkFlight, withAirports, withDirection, type FlightForm } from './flightForm';

type Picker = null | 'depDate' | 'depTime' | 'arrDate' | 'arrTime';

/**
 * Adds a flight (a friend's, or yours): who, which way, airports, times, seat and an optional ticket file. Once both airports are known
the
 * app works out the times (you can change them), and a half-filled form is kept if the sheet is closed by accident.
 */
export function AddFlightSheet({ visible, onClose }: { visible: boolean; onClose: () => void }) {
  const trip = useActiveTrip()!;
  const dest = destinationById(trip.destinationId)!;
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addFlight = useAppStore((s) => s.addFlight);
  const addDoc = useAppStore((s) => s.addDoc);

  const [saved, setSaved, clearSaved] = useDraft<FlightForm | null>('add-flight', null);
  const form = saved && saved.tripId === trip.id ? saved : blankFlight(trip, dest.airport, userId);
  const set = (next: FlightForm) => setSaved(next);
  const patch = (p: Partial<FlightForm>) => set({ ...form, ...p });
  const [ticket, setTicket] = useState<{ uri: string; name: string; size?: number } | null>(null);
  const [saveDoc, setSaveDoc] = useState(true);
  const [picker, setPicker] = useState<Picker>(null);
  const [errors, setErrors] = useState<Record<string, string | null>>({});
  const clearError = (k: string) => setErrors((e) => (e[k] ? { ...e, [k]: null } : e));

  const attach = async (kind: 'photo' | 'file') => {
    try {
      const f = kind === 'photo' ? await pickImage() : await pickDocument();
      if (f) {
        setTicket(f);
        toast('Ticket attached');
      }
    } catch (e) {
      toast(e instanceof FileTooLargeError ? e.message : 'Couldn’t attach that file', { tone: 'warn' });
    }
  };

  const submit = () => {
    const c = checkFlight(form);
    setErrors(c.errors);
    if (!c.ok) return tap('warning');
    const name = trip.members.find((m) => m.id === form.memberId)?.name.split(' ')[0] ?? '';
    addFlight(trip.id, {
      memberId: form.memberId,
      airline: form.airline.trim(),
      flightNo: form.flightNo.trim().toUpperCase().replace(/^([A-Z0-9]{2})\s?/, '$1 '),
      from: form.from.trim().toUpperCase(),
      to: form.to.trim().toUpperCase(),
      departUtc: c.departUtc,
      arriveUtc: c.arriveUtc,
      direction: form.direction,
      seat: form.seat.trim().toUpperCase() || undefined,
      confirmation: form.conf.trim().toUpperCase() || undefined,
      ticketUri: ticket?.uri,
      ticketName: ticket?.name,
```

```
    });
    if (ticket && saveDoc) {
      addDoc(trip.id, { name: `${name}’s ticket · ${form.flightNo.toUpperCase()}`, kind: /\.pdf$/i.test(ticket.name) ? 'pdf' : 'image',
uri: ticket.uri, size: ticket.size, offline: true });
    }
    const B = airportByIata(form.to);
    const near = B ? distanceKm(B, dest) : 0;
    tap('success');
    toast(form.direction === 'arrival' && near > 150 ? `Flight saved. Note ${form.to.toUpperCase()} is ${kmLabel(near)} from ${dest.cit
y}` : 'Flight saved. The group can see it now');
    clearSaved();
    setTicket(null);
    setErrors({});
    onClose();
  };

  const A = airportByIata(form.from);
  const B = airportByIata(form.to);
  const touch = (p: Partial<FlightForm>) => patch({ ...p, timesTouched: true });
  return (
    <Sheet
      visible={visible}
      onClose={onClose}
      title="Add a flight"
      subtitle="Times are local to each airport. We convert them for everyone."
      footer={<Button label="Save flight" icon="airplane" onPress={submit} />}>
      <Segmented
        value={form.direction}
        onChange={(d) => set(withDirection(form, trip, d))}
        options={[
          { key: 'arrival', label: 'Arriving' },
          { key: 'departure', label: 'Departing' },
        ]}
      />
      <View style={{ gap: 6 }}>
        <T variant="kicker">Traveller</T>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 8, paddingRight: 8 }}>
          {trip.members.map((m) => (
            <Chip key={m.id} small label={m.name.split(' ')[0]} image={m.avatar} active={form.memberId === m.id} onPress={() => patch({
memberId: m.id })} />
          ))}
        </ScrollView>
      </View>
      <View style={styles.pair}>
        <Input
          label="Airline"
          placeholder="ANA"
          value={form.airline}
          onChangeText={(v) => {
            patch({ airline: v });
            clearError('airline');
          }}
          error={errors.airline}
          containerStyle={{ flex: 1 }}
        />
        <Input
          label="Flight no."
          placeholder="NH 175"
          value={form.flightNo}
          onChangeText={(v) => {
            patch({ flightNo: v });
            clearError('flightNo');
          }}
          autoCapitalize="characters"
          error={errors.flightNo}
          containerStyle={{ flex: 1 }}
        />
      </View>
      <View style={styles.pair}>
        <Input
          label="From"
          placeholder="JFK"
          value={form.from}
          onChangeText={(v) => {
            set(withAirports(form, trip, { from: v.toUpperCase().slice(0, 3) }));
            clearError('from');
          }}
          autoCapitalize="characters"
          error={errors.from}
          hint={A ? A.city : undefined}
          containerStyle={{ flex: 1 }}
        />
        <Input
          label="To"
          placeholder={dest.airport}
          value={form.to}
```

```
          onChangeText={(v) => {
            set(withAirports(form, trip, { to: v.toUpperCase().slice(0, 3) }));
            clearError('to');
          }}
          autoCapitalize="characters"
          error={errors.to}
          hint={B ? B.city : undefined}
          containerStyle={{ flex: 1 }}
        />
      </View>
      <View style={{ gap: 6 }}>
        <T variant="kicker">Departs{A ? ` (${A.city} time)` : ''}</T>
        <View style={styles.pair}>
          <Input accessibilityLabel="Departure date" value={`${dayLabel(form.depDate)}, ${form.depDate.slice(0, 4)}`} icon="calendar-out
line" onPressField={() => setPicker('depDate')} containerStyle={{ flex: 1.3 }} />
          <Input accessibilityLabel="Departure time" value={time12(form.depTime)} icon="time-outline" onPressField={() => setPicker('dep
Time')} containerStyle={{ flex: 1 }} />
        </View>
      </View>
      <View style={{ gap: 6 }}>
        <T variant="kicker">Arrives{B ? ` (${B.city} time)` : ''}</T>
        <View style={styles.pair}>
          <Input accessibilityLabel="Arrival date" value={`${dayLabel(form.arrDate)}, ${form.arrDate.slice(0, 4)}`} icon="calendar-outli
ne" onPressField={() => setPicker('arrDate')} containerStyle={{ flex: 1.3 }} />
          <Input accessibilityLabel="Arrival time" value={time12(form.arrTime)} icon="time-outline" onPressField={() => setPicker('arrTi
me')} containerStyle={{ flex: 1 }} />
        </View>
        {errors.arr ? (
          <T variant="caption" color={colors.red} accessibilityLiveRegion="polite">
            {errors.arr}
          </T>
        ) : A && B && !form.timesTouched ? (
          <T variant="caption" color={colors.textMuted}>
            Worked out from the distance. Change it if your ticket says otherwise.
          </T>
        ) : null}
      </View>
      <View style={styles.pair}>
        <Input
          label="Seat (optional)"
          placeholder="32A"
          value={form.seat}
          onChangeText={(v) => {
            patch({ seat: v });
            clearError('seat');
          }}
          autoCapitalize="characters"
          error={errors.seat}
          containerStyle={{ flex: 1 }}
        />
        <Input label="Confirmation (optional)" placeholder="ABC123" value={form.conf} onChangeText={(v) => patch({ conf: v })} autoCapit
alize="characters" maxLength={10} containerStyle={{ flex: 1 }} />
      </View>
      <View style={{ gap: 8 }}>
        <T variant="kicker">Ticket / boarding pass (optional)</T>
        {ticket ? (
          <View style={styles.attached}>
            <Ionicons name="document-attach-outline" size={20} color={colors.ink} />
            <T variant="small" weight="semibold" style={{ flex: 1 }} numberOfLines={1}>
              {ticket.name}
            </T>
            <Press onPress={() => setTicket(null)} accessibilityLabel="Remove attachment" style={styles.detach}>
              <Ionicons name="close" size={20} color={colors.ink} />
            </Press>
          </View>
        ) : (
          <View style={styles.pair}>
            <Button label="Photo" icon="image-outline" variant="secondary" size="md" onPress={() => attach('photo')} style={{ flex: 1 }}
/>
            <Button label="PDF / file" icon="document-outline" variant="secondary" size="md" onPress={() => attach('file')} style={{ fle
x: 1 }} />
          </View>
        )}
        {ticket ? <ToggleRow icon="cloud-done-outline" label="Also save to offline documents" value={saveDoc} onChange={setSaveDoc} /> :
null}
      </View>

      <DatePickerSheet visible={picker === 'depDate'} value={form.depDate} onClose={() => setPicker(null)} onChange={(v) => touch({ depD
ate: v })} title="Departure date" />
      <DatePickerSheet visible={picker === 'arrDate'} value={form.arrDate} onClose={() => setPicker(null)} onChange={(v) => touch({ arrD
ate: v })} title="Arrival date" />
      <TimePickerSheet visible={picker === 'depTime'} value={form.depTime} onClose={() => setPicker(null)} onChange={(v) => touch({ depT
ime: v })} />
      <TimePickerSheet visible={picker === 'arrTime'} value={form.arrTime} onClose={() => setPicker(null)} onChange={(v) => touch({ arrT
ime: v })} />
    </Sheet>
```

```
  );
}

const styles = StyleSheet.create({
  pair: { flexDirection: 'row', gap: 12, alignItems: 'flex-start' },
  attached: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingLeft: 12, minHeight: 52, borderRadius: radius.button, borderWi
dth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt },
  detach: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/flights/FlightTicket.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import * as Linking from 'expo-linking';
import { StyleSheet, View } from 'react-native';

import { barcodeBars } from '@/components/crew/barcode';
import { dateKicker, timeParts } from '@/components/home/logic';
import { Press } from '@/components/ui/Press';
import { Perforation } from '@/components/ui/Rule';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import { toast } from '@/components/ui/feedback';
import { airportByIata } from '@/data/airports';
import type { Flight } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { durationLabel } from '@/utils/format';
import { dayLabel, partsInTz, time12 } from '@/utils/time';

/** A time as a departures board prints it: the clock in mono and AM or PM small beside it, with the date in caps under it. */
function Clock({ hhmm, ymd, align }: { hhmm: string; ymd: string; align: 'left' | 'right' }) {
  const { clock, suffix } = timeParts(hhmm);
  return (
    <View style={{ alignItems: align === 'left' ? 'flex-start' : 'flex-end' }}>
      <View style={styles.clock}>
        <T variant="numeralSm">{clock}</T>
        <T variant="kicker" color={colors.textSecondary}>
          {suffix}
        </T>
      </View>
      <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 10 }}>
        {dateKicker(ymd)}
      </T>
    </View>
  );
}

/** One printed field on the stub: a small caps label over a value in mono. */
function Field({ label, value }: { label: string; value: string }) {
  return (
    <View style={{ flexShrink: 1 }}>
      <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 10 }}>
        {label}
      </T>
      <T variant="numeralSm" numberOfLines={1} style={{ fontSize: 18, lineHeight: 24 }}>
        {value}
      </T>
    </View>
  );
}

/**
 * One flight as a boarding pass. The main part has the two airport codes big in mono with each airport's own clock under it;
 * below the tear line are the seat, the confirmation and the flight time, a barcode made from the booking, and the ticket and remove bu
ttons.
 * `zoneNote` says what the time is in the zone the person chose, when that differs from the airport's clock.
 */
export function FlightTicket({ flight: f, passenger, plated, zoneNote, canDelete, onDelete }: { flight: Flight; passenger: string; plate
d: boolean; zoneNote?: string; canDelete: boolean; onDelete: () => void }) {
  const from = airportByIata(f.from);
  const to = airportByIata(f.to);
  const dep = from ? partsInTz(f.departUtc, from.tz) : null;
  const arr = to ? partsInTz(f.arriveUtc, to.tz) : null;
  const minutes = Math.round((f.arriveUtc - f.departUtc) / 60000);
  const arriving = f.direction === 'arrival';
  const isImage = !!f.ticketUri && (/\.(png|jpe?g|webp|heic)$/i.test(f.ticketName ?? '') || f.ticketUri.startsWith('data:image'));
  return (
    <Ticket
      plated={plated}
      stub={
        <View style={styles.stub}>
          <View style={styles.fieldsRow}>
            <View style={styles.fields}>
              <Field label="Flight time" value={durationLabel(minutes)} />
```

```
              {f.seat ? <Field label="Seat" value={f.seat} /> : null}
              {f.confirmation ? <Field label="Booking" value={f.confirmation} /> : null}
            </View>
            {canDelete ? (
              <Press onPress={onDelete} style={styles.remove} accessibilityLabel={`Remove flight ${f.flightNo}`}>
                <Ionicons name="trash-outline" size={20} color={colors.textSecondary} />
              </Press>
            ) : null}
          </View>
          <View style={styles.barcode} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
            {barcodeBars(`${f.id}${f.confirmation ?? f.flightNo}`).map((w, i) => (
              <View key={i} style={{ flex: w, backgroundColor: i % 2 === 0 ? colors.ink : 'transparent' }} />
            ))}
          </View>
          {f.ticketUri ? (
            <Press onPress={() => Linking.openURL(f.ticketUri!).catch(() => toast('Ticket saved offline in Documents'))} style={styles.t
icketBtn} accessibilityLabel={`Open ticket ${f.ticketName ?? ''}`}>
              {isImage ? <Image source={{ uri: f.ticketUri }} style={styles.thumb} accessibilityIgnoresInvertColors /> : <Ionicons name
="document-attach-outline" size={18} color={colors.ink} />}
              <T variant="small" weight="bold" numberOfLines={1} style={{ maxWidth: 180 }}>
                {f.ticketName ?? 'Ticket'}
              </T>
            </Press>
          ) : null}
        </View>
      }>
      <View style={styles.main}>
        <View style={styles.top}>
          <T variant="kicker" color={colors.textSecondary} numberOfLines={1} style={{ flex: 1 }}>
            {passenger} / {f.airline} {f.flightNo}
          </T>
          <Stamp label={arriving ? 'Arriving' : 'Departing'} tone={arriving ? 'accent' : 'ink'} rotate={-3} />
        </View>
        <View style={styles.route} accessible accessibilityLabel={`${f.flightNo}, ${from?.city ?? f.from} to ${to?.city ?? f.to}`}>
          <View style={styles.end}>
            <T variant="numeral" style={styles.code}>
              {f.from}
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {from?.city ?? ''}
            </T>
            {dep ? <Clock hhmm={dep.hhmm} ymd={dep.ymd} align="left" /> : null}
          </View>
          <View style={styles.mid}>
            <Perforation style={{ flex: 1 }} />
            <View style={styles.plane}>
              <Ionicons name="airplane" size={18} color={colors.ink} />
            </View>
            <Perforation style={{ flex: 1 }} />
          </View>
          <View style={[styles.end, { alignItems: 'flex-end' }]}>
            <T variant="numeral" style={styles.code}>
              {f.to}
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {to?.city ?? ''}
            </T>
            {arr ? <Clock hhmm={arr.hhmm} ymd={arr.ymd} align="right" /> : null}
          </View>
        </View>
        {zoneNote ? (
          <T variant="caption" color={colors.textSecondary}>
            {zoneNote}
          </T>
        ) : null}
      </View>
    </Ticket>
  );
}

/** What the trip-side time is in the zone the person chose, or nothing when it reads the same as the airport's clock. */
export function zoneNoteFor(f: Flight, tz: string, zoneName: string): string | undefined {
  const airport = airportByIata(f.direction === 'arrival' ? f.to : f.from);
  const utc = f.direction === 'arrival' ? f.arriveUtc : f.departUtc;
  if (!airport) return undefined;
  const there = partsInTz(utc, airport.tz);
  const chosen = partsInTz(utc, tz);
  if (there.hhmm === chosen.hhmm && there.ymd === chosen.ymd) return undefined;
  return `${f.direction === 'arrival' ? 'Lands' : 'Leaves'} ${time12(chosen.hhmm)} · ${dayLabel(chosen.ymd)}, ${zoneName}`;
}

const styles = StyleSheet.create({
  main: { padding: 16, gap: 14 },
  top: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  route: { flexDirection: 'row', alignItems: 'flex-start', gap: 8 },
  end: { gap: 4, flexShrink: 1 },
```

```
  code: { fontSize: 46, lineHeight: 50, letterSpacing: -1 },
  clock: { flexDirection: 'row', alignItems: 'flex-end', gap: 4 },
  mid: { flex: 1, minWidth: 24, flexDirection: 'row', alignItems: 'center', gap: 4, marginTop: 18 },
  plane: { width: 28, height: 28, alignItems: 'center', justifyContent: 'center' },
  stub: { paddingHorizontal: 16, paddingBottom: 14, paddingTop: 2, gap: 12 },
  fieldsRow: { flexDirection: 'row', alignItems: 'flex-start', gap: 8 },
  fields: { flex: 1, flexDirection: 'row', flexWrap: 'wrap', columnGap: 24, rowGap: 8 },
  barcode: { flexDirection: 'row', height: 34 },
  ticketBtn: { alignSelf: 'flex-start', flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 44, paddingHorizontal: 12, border
Radius: radius.chip, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt },
  thumb: { width: 22, height: 22, borderRadius: radius.xs },
  remove: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center', marginRight: -10, marginTop: -4 },
});
```

src/components/flights/__tests__/flightForm.test.ts

```
import { airportByIata } from '@/data/airports';

import { blankFlight, checkFlight, flightMinutes, suggestTimes, withAirports, withDirection, type FlightForm } from '../flightForm';

const trip = { id: 't1', startDate: '2026-10-30', endDate: '2026-11-04' };
const filled = (over: Partial<FlightForm> = {}): FlightForm => ({ ...blankFlight(trip, 'NAP', 'maya'), airline: 'Delta', flightNo: 'DL 9
9', ...over });

describe('how long a flight takes', () => {
  it('is between one and twenty hours', () => {
    expect(flightMinutes(10)).toBe(60);
    expect(flightMinutes(40000)).toBe(20 * 60);
  });
  it('gives about 8 to 9 hours from New York to Naples', () => {
    const km = 6400;
    expect(flightMinutes(km)).toBeGreaterThan(7.5 * 60);
    expect(flightMinutes(km)).toBeLessThan(10 * 60);
  });
});

describe('suggested times', () => {
  const BOS = airportByIata('BOS')!;
  const NAP = airportByIata('NAP')!;

  it('an arriving flight lands on the first day at 14:30 and leaves earlier, in the clock of the airport it leaves', () => {
    const s = suggestTimes('arrival', trip, BOS, NAP);
    expect(s.arrDate).toBe('2026-10-30');
    expect(s.arrTime).toBe('14:30');
    expect(s.depDate <= '2026-10-30').toBe(true);
    expect(s.depTime).toMatch(/^\d\d:\d[05]$/);
  });

  it('a departing flight leaves on the last day at 11:00', () => {
    const s = suggestTimes('departure', trip, NAP, BOS);
    expect(s.depDate).toBe('2026-11-04');
    expect(s.depTime).toBe('11:00');
  });

  it('always passes the checks, whatever the clocks do (the week the US and Europe change clocks on different days)', () => {
    for (const from of ['BOS', 'JFK', 'LAX', 'HND', 'SYD', 'LHR']) {
      const f = withAirports(filled({ from: '', to: 'NAP' }), trip, { from });
      expect(checkFlight(f).errors.arr).toBeUndefined();
    }
  });
});

describe('the form changes the times for you, until you change them yourself', () => {
  it('sets them when the second airport is known', () => {
    const f = withAirports(filled({ from: '' }), trip, { from: 'BOS' });
    expect(f.from).toBe('BOS');
    expect(f.depTime).not.toBe('20:00');
  });
  it('leaves them alone when they were set by hand', () => {
    const f = withAirports(filled({ timesTouched: true, depTime: '09:15' }), trip, { from: 'BOS' });
    expect(f.depTime).toBe('09:15');
  });
  it('waits while an airport code is unknown', () => {
    const f = withAirports(filled(), trip, { from: 'XXX' });
    expect(f.depTime).toBe('20:00');
  });
  it('swaps the airports when you switch to departing', () => {
    const f = withDirection(filled({ from: 'BOS', to: 'NAP' }), trip, 'departure');
    expect(f.direction).toBe('departure');
    expect(f.from).toBe('NAP');
    expect(f.to).toBe('BOS');
    expect(f.depDate).toBe('2026-11-04');
    expect(withDirection(f, trip, 'departure')).toBe(f);
  });
});
```

```
describe('checking a flight', () => {
  it('asks for what is missing', () => {
    const c = checkFlight(blankFlight(trip, 'NAP', 'maya'));
    expect(c.ok).toBe(false);
    expect(c.errors.airline).toBe('Enter the airline');
    expect(c.errors.flightNo).toBeTruthy();
    expect(c.errors.from).toBeTruthy();
  });
  it('rejects an airport it does not know and two airports that are the same', () => {
    expect(checkFlight(filled({ from: 'XXX' })).errors.from).toMatch(/recognise/);
    expect(checkFlight(filled({ from: 'NAP', to: 'NAP' })).errors.to).toMatch(/differ/);
  });
  it('rejects a bad seat', () => {
    expect(checkFlight(filled({ from: 'BOS', seat: '9' })).errors.seat).toBe('Seats look like 32A');
  });
  it('rejects a landing before the take-off and a flight that is far too long', () => {
    const early = checkFlight(filled({ from: 'BOS', depDate: '2026-10-30', depTime: '20:00', arrDate: '2026-10-30', arrTime: '08:00'
}));
    expect(early.errors.arr).toMatch(/after departure/);
    const long = checkFlight(filled({ from: 'BOS', depDate: '2026-10-27', depTime: '08:00', arrDate: '2026-10-30', arrTime: '14:30' }));
    expect(long.errors.arr).toMatch(/double-check/);
  });
  it('accepts a good flight and works out both instants', () => {
    const f = withAirports(filled({ from: '' }), trip, { from: 'BOS' });
    const c = checkFlight({ ...f, seat: '32A' });
    expect(c.ok).toBe(true);
    expect(c.arriveUtc).toBeGreaterThan(c.departUtc);
  });
});
```

src/components/flights/flightForm.ts

```
/**
 * The add-flight form as plain data, with no React in it so it can be tested: the blank form, the times the app works out for
 * the person, and the checks done before saving.
 */
import { airportByIata, type Airport } from '@/data/airports';
import { distanceKm } from '@/utils/geo';
import { addDays, partsInTz, zonedToUtc } from '@/utils/time';
import { maxLen, validateFlightNumber, validateIata } from '@/utils/validation';

export interface FlightForm {
  tripId: string;
  memberId: string;
  direction: 'arrival' | 'departure';
  airline: string;
  flightNo: string;
  from: string;
  to: string;
  depDate: string;
  depTime: string;
  arrDate: string;
  arrTime: string;
  seat: string;
  conf: string;
  /** The person has chosen a date or a time by hand: the airports no longer rewrite them. */
  timesTouched: boolean;
}

/** The longest flight the form accepts without asking the person to check the dates. */
export const MAX_FLIGHT_HOURS = 22;

export function blankFlight(trip: { id: string; startDate: string }, destAirport: string, memberId: string): FlightForm {
  return {
    tripId: trip.id,
    memberId,
    direction: 'arrival',
    airline: '',
    flightNo: '',
    from: '',
    to: destAirport,
    depDate: addDays(trip.startDate, -1),
    depTime: '20:00',
    arrDate: trip.startDate,
    arrTime: '14:30',
    seat: '',
    conf: '',
    timesTouched: false,
  };
}

/** About how long a flight of this distance takes, door to door on the runway: 40 minutes plus the distance at 820 km/h. Between one an
d twenty hours. */
export function flightMinutes(km: number): number {
  return Math.min(20 * 60, Math.max(60, Math.round(40 + (km / 820) * 60)));
}
```

```
const round5 = (utc: number) => Math.round(utc / 300000) * 300000;

/**
 * The dates and times to suggest once both airports are known. An arriving flight lands on the first day of the trip at 14:30 and
 * leaves as long before as the distance takes; a departing flight leaves on the last day at 11:00 and lands as long after.
 * Each time is in its own airport's clock, because that is how the form asks for them.
 */
export function suggestTimes(direction: FlightForm['direction'], trip: { startDate: string; endDate: string }, from: Airport, to: Airpor
t): Pick<FlightForm, 'depDate' | 'depTime' | 'arrDate' | 'arrTime'> {
  const minutes = flightMinutes(distanceKm(from, to));
  if (direction === 'arrival') {
    const arrive = zonedToUtc(trip.startDate, '14:30', to.tz);
    const dep = partsInTz(round5(arrive - minutes * 60000), from.tz);
    return { depDate: dep.ymd, depTime: dep.hhmm, arrDate: trip.startDate, arrTime: '14:30' };
  }
  const depart = zonedToUtc(trip.endDate, '11:00', from.tz);
  const arr = partsInTz(round5(depart + minutes * 60000), to.tz);
  return { depDate: trip.endDate, depTime: '11:00', arrDate: arr.ymd, arrTime: arr.hhmm };
}

/** Changes the airports and, unless the person has set the times by hand, lets the new airports set them. */
export function withAirports(form: FlightForm, trip: { startDate: string; endDate: string }, patch: Partial<Pick<FlightForm, 'from' | 't
o'>>): FlightForm {
  const next = { ...form, ...patch };
  const A = airportByIata(next.from);
  const B = airportByIata(next.to);
  if (next.timesTouched || !A || !B || A.iata === B.iata) return next;
  return { ...next, ...suggestTimes(next.direction, trip, A, B) };
}

/** Switches between arriving and departing: the two airports swap places (you fly from home to the trip, and back), and the times start
over. */
export function withDirection(form: FlightForm, trip: { startDate: string; endDate: string }, direction: FlightForm['direction']): Fligh
tForm {
  if (direction === form.direction) return form;
  const base: FlightForm = { ...form, direction, from: form.to, to: form.from, timesTouched: false };
  const A = airportByIata(base.from);
  const B = airportByIata(base.to);
  if (A && B && A.iata !== B.iata) return { ...base, ...suggestTimes(direction, trip, A, B) };
  return direction === 'arrival'
    ? { ...base, depDate: addDays(trip.startDate, -1), depTime: '20:00', arrDate: trip.startDate, arrTime: '14:30' }
    : { ...base, depDate: trip.endDate, depTime: '11:00', arrDate: trip.endDate, arrTime: '14:30' };
}

export interface FlightCheck {
  errors: Record<string, string | null>;
  departUtc: number;
  arriveUtc: number;
  ok: boolean;
}

/** Checks the form the way the sheet does when you press Save: every field, then the two times against each other. */
export function checkFlight(f: FlightForm): FlightCheck {
  const A = airportByIata(f.from);
  const B = airportByIata(f.to);
  const errors: Record<string, string | null> = {
    airline: f.airline.trim() ? maxLen(30, 'Airline')(f.airline) : 'Enter the airline',
    flightNo: validateFlightNumber(f.flightNo),
    from: validateIata(f.from, 'departure'),
    to: validateIata(f.to, 'arrival') ?? (f.from.trim().toUpperCase() === f.to.trim().toUpperCase() ? 'Departure and arrival airports mu
st differ' : null),
    seat: f.seat && !/^\d{1,3}[A-K]$/i.test(f.seat.trim()) ? 'Seats look like 32A' : null,
  };
  let departUtc = 0;
  let arriveUtc = 0;
  if (A && B && !errors.from && !errors.to) {
    departUtc = zonedToUtc(f.depDate, f.depTime, A.tz);
    arriveUtc = zonedToUtc(f.arrDate, f.arrTime, B.tz);
    const hrs = (arriveUtc - departUtc) / 3600000;
    if (hrs <= 0) errors.arr = 'Arrival must be after departure (times are local to each airport)';
    else if (hrs > MAX_FLIGHT_HOURS) errors.arr = `That’s a ${Math.round(hrs)}h flight — double-check the dates`;
  }
  return { errors, departUtc, arriveUtc, ok: !Object.values(errors).some(Boolean) };
}
```

src/components/home/BudgetSection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { ProgressBar, SectionHeader, type IconName } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Stamp, type StampTone } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
```

```
import { totalSpent } from '@/store/useAppStore';
import { colors } from '@/theme';
import { money } from '@/utils/format';

import { budgetView, oweLine, type BudgetTone } from './logic';
import { MonoAmount } from './MonoAmount';
import type { TripCtx } from './useHomeContext';

/** The bar is the accent while things are fine and turns amber, then red; the stamp always says the state in words. */
const TONE: Record<BudgetTone, { bar: string; stamp: StampTone }> = {
  ok: { bar: colors.accent, stamp: 'positive' },
  warn: { bar: colors.warn, stamp: 'warn' },
  over: { bar: colors.red, stamp: 'danger' },
};

const OWE_ICON: Record<'owe' | 'owed' | 'square' | 'other', IconName> = {
  owe: 'arrow-up-outline',
  owed: 'arrow-down-outline',
  square: 'checkmark-circle-outline',
  other: 'swap-horizontal',
};

/**
 * What the group has spent: one big mono number sitting open on the paper, the limit beside it, a chunky bar in the state's colour
 * with a stamp that says the state, and one line about who owes whom. The whole block opens the Budget tab.
 */
export function BudgetSection({ ctx }: { ctx: TripCtx }) {
  const { trip, userId } = ctx;
  const spent = totalSpent(trip);
  const v = budgetView(spent, trip.budget);
  const tone = TONE[v.tone];
  const owe = oweLine(trip, userId);
  const open = () => router.navigate('/(tabs)/budget');
  return (
    <View>
      <SectionHeader title="Group budget" action="Open" onAction={open} />
      <Press onPress={open} scaleTo={0.99} style={styles.body} accessibilityLabel={`Group budget. ${money(spent)} of ${money(trip.budge
t)} spent. ${v.word}, ${v.detail}. ${owe.text}. Open budget`}>
        <View style={styles.top}>
          <View style={styles.amount}>
            <MonoAmount text={money(spent)} size={56} />
            <T variant="small" color={colors.textSecondary} style={{ marginBottom: 8 }}>
              of {money(trip.budget)}
            </T>
          </View>
          <Stamp label={v.word} tone={tone.stamp} />
        </View>
        <ProgressBar value={v.ratio} color={tone.bar} height={18} />
        <T variant="kicker" color={colors.textSecondary}>
          {v.detail}
        </T>
        <View style={styles.owe}>
          <Ionicons name={OWE_ICON[owe.tone]} size={18} color={owe.tone === 'owed' || owe.tone === 'square' ? colors.positive : colors.i
nk} />
          <T variant="bodySm" weight="semibold" style={{ flex: 1 }}>
            {owe.text}
          </T>
          <Ionicons name="arrow-forward" size={16} color={colors.ink} />
        </View>
      </Press>
    </View>
  );
}

const styles = StyleSheet.create({
  body: { gap: 10 },
  top: { flexDirection: 'row', alignItems: 'flex-start', justifyContent: 'space-between', gap: 8 },
  amount: { flexDirection: 'row', alignItems: 'flex-end', flexWrap: 'wrap', columnGap: 8, flexShrink: 1 },
  owe: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, borderTopWidth: 1, borderTopColor: colors.border },
});
```

src/components/home/CrewSection.tsx

```
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { AvatarStack, Card, SectionHeader } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { colors } from '@/theme';

import { crewCounts } from './logic';
import type { TripCtx } from './useHomeContext';

/**
```

```
 * Who is coming: four faces and "+N", how many are going and how many are only invited, and the Invite button that copies the code.
 * The strip opens the crew; the button is its sibling (never inside it).
 */
export function CrewSection({ ctx, onInvite }: { ctx: TripCtx; onInvite: () => void }) {
  const { trip } = ctx;
  const { going, invited } = crewCounts(trip.members);
  return (
    <View>
      <SectionHeader title="Your crew" action="Open" onAction={() => router.push('/crew')} />
      <Card style={styles.card}>
        <Press
          onPress={() => router.push('/crew')}
          scaleTo={0.98}
          style={styles.strip}
          accessibilityLabel={`${going} going${invited ? `, ${invited} invited` : ''}. Open the crew`}>
          <AvatarStack people={trip.members} max={4} size={38} />
          <View style={styles.count}>
            <T variant="numeralSm">{going}</T>
            <T variant="small" weight="bold">
              Going
            </T>
            {invited ? (
              <T variant="small" color={colors.textSecondary}>
                · {invited} Invited
              </T>
            ) : null}
          </View>
        </Press>
        <Button label="Invite" icon="person-add-outline" size="md" full={false} onPress={onInvite} accessibilityHint={`Copies the invite
code ${trip.inviteCode}`} />
      </Card>
    </View>
  );
}

const styles = StyleSheet.create({
  card: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', justifyContent: 'space-between', columnGap: 12, rowGap: 12 },
  strip: { flexGrow: 1, flexShrink: 1, minHeight: 56, justifyContent: 'center', gap: 8 },
  count: { flexDirection: 'row', alignItems: 'baseline', gap: 6, flexWrap: 'wrap' },
});
```

src/components/home/DiscoverSection.tsx

```
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { PlaceCard } from '@/components/explore';
import { SectionHeader } from '@/components/ui/bits';
import { destinations, type Destination } from '@/data/destinations';

/** Four other places to look at, as photo cards in two columns. */
export function DiscoverSection({ dest, colW }: { dest: Destination | undefined; colW: number }) {
  return (
    <View>
      <SectionHeader title="Discover" action="See all" onAction={() => router.push('/destinations')} />
      <View style={styles.grid}>
        {destinations
          .filter((d) => d.id !== dest?.id)
          .slice(0, 4)
          .map((d) => (
            <PlaceCard key={d.id} d={d} style={{ width: (colW - 12) / 2 }} />
          ))}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 12 },
});
```

src/components/home/HomeHero.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { StyleSheet, View, type Animated } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { SkylineHero } from '@/components/SkylineHero';
import { AvatarStack } from '@/components/ui/bits';
import { FadeIn } from '@/components/ui/motion';
import { PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { SplitFlap } from '@/components/ui/SplitFlap';
import { Ticket } from '@/components/ui/Ticket';
```

```
import { T } from '@/components/ui/T';
import type { Destination } from '@/data/destinations';
import type { Trip } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { rangeLabel } from '@/utils/time';

import { countdownView, type HeroStatus } from './logic';

/** Words for the stamp in the corner of the ticket. */
const STAMP: Record<NonNullable<HeroStatus>, { label: string; tone: 'accent' | 'positive' | 'muted' }> = {
  planning: { label: 'Boarding soon', tone: 'accent' },
  active: { label: 'Live now', tone: 'positive' },
  completed: { label: 'Arrived', tone: 'muted' },
};

function Control({ onPress, label, children, wide }: { onPress: () => void; label: string; children: React.ReactNode; wide?: boolean })
{
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} style={[styles.control, wide && { padding
Horizontal: 12 }]}>
      {children}
    </PlatePress>
  );
}

/**
 * The top of Home: a photo of the place with square, outlined controls on it, and the trip as a boarding pass lying across
 * the photo's lower edge. The days to go flip like a departures board; below the tear line are the crew and the invite code.
 */
export function HomeHero({ trip, dest, tz, status, daysToGo, dayNo, tripLen, unread, scrollY, onMenu, onCopyCode }: { trip: Trip | undef
ined; dest: Destination | undefined; tz: string; status: HeroStatus; daysToGo: number; dayNo: number; tripLen: number; unread: number; /
** The page's scroll position, so the photo can drift against it. */ scrollY?: Animated.Value; onMenu: () => void; onCopyCode: () => voi
d }) {
  const insets = useSafeAreaInsets();
  const view = countdownView(status, daysToGo, dayNo, tripLen);
  const stamp = status ? STAMP[status] : null;
  return (
    <View>
      <SkylineHero image={dest?.image ?? 'amalfi'} tz={tz} height={300 + insets.top} scrollY={scrollY}>
        <View style={[styles.top, { top: insets.top + 10 }]}>
          <Control onPress={() => (trip ? onMenu() : router.push('/trips'))} label={trip ? `${trip.name}. Trip options and switching` :
'Choose a trip'} wide>
            <T variant="small" weight="bold" numberOfLines={1} style={{ maxWidth: 170 }}>
              {trip?.name ?? 'No trip yet'}
            </T>
            <Ionicons name="chevron-down" size={14} color={colors.ink} />
          </Control>
          <View style={{ flex: 1 }} />
          <Control onPress={() => router.push({ pathname: '/panorama', params: { city: dest?.id } })} label="Open the 360 degree city vi
ew" wide>
            <Ionicons name="globe-outline" size={16} color={colors.ink} />
            <T variant="kicker" color={colors.ink}>
              360°
            </T>
          </Control>
          <Control onPress={() => router.push('/notifications')} label={`Notifications, ${unread} unread`}>
            <Ionicons name="notifications-outline" size={19} color={colors.ink} />
            {unread > 0 ? (
              <View style={styles.badge}>
                <T variant="micro" weight="bold" color={colors.white} style={{ fontSize: 9.5 }}>
                  {unread > 9 ? '9+' : unread}
                </T>
              </View>
            ) : null}
          </Control>
        </View>
      </SkylineHero>

      <FadeIn delay={80} distance={20} style={styles.ticketWrap}>
        <Ticket
          stub={
            trip ? (
              <View style={styles.stub}>
                <AvatarStack people={trip.members} max={4} size={32} />
                <Press onPress={onCopyCode} style={styles.code} accessibilityLabel={`Invite code ${trip.inviteCode}. Tap to copy`}>
                  <View>
                    <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 9.5 }}>
                      Invite code
                    </T>
                    <T variant="numeralSm" style={{ fontSize: 17, lineHeight: 20 }}>
                      {trip.inviteCode}
                    </T>
                  </View>
                  <Ionicons name="copy-outline" size={18} color={colors.ink} />
                </Press>
              </View>
```

```
            ) : undefined
          }>
          <View style={styles.main}>
            <View style={styles.row}>
              <T variant="kicker" color={colors.textSecondary} style={{ flexGrow: 1, flexShrink: 1 }}>
                {dest ? `${dest.city}, ${dest.country}` : 'Somewhere great'}
              </T>
              {stamp ? <Stamp label={stamp.label} tone={stamp.tone} /> : null}
            </View>

            <View accessible accessibilityRole="header" accessibilityLabel={view.sentence} style={styles.count}>
              {view.kind === 'days' ? (
                <>
                  <SplitFlap value={view.number ?? 0} size={68} />
                  <T variant="headline" style={{ fontSize: 30, lineHeight: 34, marginLeft: 12 }}>
                    {view.words.join(' ')}
                  </T>
                </>
              ) : view.kind === 'day' ? (
                <>
                  <T variant="headline" style={{ fontSize: 30, lineHeight: 34, marginRight: 12 }}>
                    Day
                  </T>
                  <SplitFlap value={view.number ?? 1} size={68} />
                  <T variant="headline" style={{ fontSize: 30, lineHeight: 34, marginLeft: 12 }}>
                    {view.words.join(' ')}
                  </T>
                </>
              ) : (
                <T variant="display" style={{ fontSize: 44, lineHeight: 48 }}>
                  {view.words.join(' ')}
                </T>
              )}
            </View>

            {trip ? (
              <T variant="small" color={colors.textSecondary}>
                {rangeLabel(trip.startDate, trip.endDate)} · {tripLen} {tripLen === 1 ? 'day' : 'days'} · {trip.members.length} travelle
rs
              </T>
            ) : (
              <T variant="small" color={colors.textSecondary}>
                Create a trip or join one with a code.
              </T>
            )}
          </View>
        </Ticket>
      </FadeIn>
    </View>
  );
}

const styles = StyleSheet.create({
  top: { position: 'absolute', left: 16, right: 16, flexDirection: 'row', alignItems: 'center', gap: 10 },
  control: { flexDirection: 'row', alignItems: 'center', gap: 6, minWidth: 44, height: 44, justifyContent: 'center', paddingHorizontal:
0, borderRadius: radius.button, backgroundColor: colors.card, borderWidth: line, borderColor: colors.ink },
  badge: { position: 'absolute', top: -8, right: -8, minWidth: 20, height: 20, paddingHorizontal: 4, borderRadius: 10, backgroundColor:
colors.red, borderWidth: 2, borderColor: colors.ink, alignItems: 'center', justifyContent: 'center' },
  ticketWrap: { marginTop: -60, paddingHorizontal: 20 },
  main: { padding: 16, gap: 12 },
  row: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', columnGap: 8, rowGap: 6 },
  count: { flexDirection: 'row', alignItems: 'center', flexWrap: 'wrap', rowGap: 6 },
  stub: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', justifyContent: 'space-between', columnGap: 12, rowGap: 10, padd
ingHorizontal: 16, paddingBottom: 14, paddingTop: 2 },
  code: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 44, paddingHorizontal: 12, borderRadius: radius.chip, borderWi
dth: line, borderColor: colors.ink, borderStyle: 'dashed', backgroundColor: colors.accentSoft },
});
```

src/components/home/MonoAmount.tsx

```
import { Text } from 'react-native';

import { T } from '@/components/ui/T';

const isPoint = (ch: string | undefined) => ch === ',' || ch === '.';

/**
 * A big mono number whose commas and points are pulled in close to their neighbours. In a monospace face every character takes a full
 * cell, so "$2,086" otherwise reads as "$2 , 086". It stays ONE piece of text (nested spans only change the spacing), so it reads and
 * is found as "$2,086".
 */
export function MonoAmount({ text, size = 56 }: { text: string; size?: number }) {
  const chars = text.split('');
  return (
    <T variant="hero" style={{ fontSize: size, lineHeight: size + 4 }}>
      {chars.map((ch, i) => (
```

```
        <Text key={i} style={isPoint(ch) || isPoint(chars[i + 1]) ? { letterSpacing: -size * 0.22 } : undefined}>
          {ch}
        </Text>
      ))}
    </T>
  );
}
```

src/components/home/NearbySection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { placeCategoryIcon } from '@/components/trip/meta';
import { SectionHeader } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import type { Destination } from '@/data/destinations';
import { placesFor } from '@/data/places';
import type { Trip } from '@/store/types';
import { colors, hair, line, radius } from '@/theme';
import { lengthTag, money } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';

import { RadiusSlider } from './RadiusSlider';

/** Places close to the trip's centre, with a radius slider; the list is open on the page with hair dividers. */
export function NearbySection({ trip, dest }: { trip: Trip; dest: Destination }) {
  const [radiusKm, setRadiusKm] = useState(5);
  const nearby = [trip.destinationId, ...trip.extraDestinationIds]
    .flatMap((id) => placesFor(id))
    .map((p) => ({ p, km: distanceKm(dest, p) }))
    .filter((x) => x.km <= radiusKm)
    .sort((a, b) => a.km - b.km);
  return (
    <View>
      <SectionHeader title="Nearby" action="Filters" onAction={() => router.push('/nearby')} />
      <View style={styles.rowBetween}>
        <T variant="kicker" color={colors.textSecondary}>
          Search radius
        </T>
        <T variant="numeralSm">{radiusKm} km</T>
      </View>
      <RadiusSlider value={radiusKm} min={1} max={20} onChange={setRadiusKm} label="Search radius in kilometres" formatValue={(v) => `
${v} km`} />
      <T variant="caption" color={colors.textSecondary} style={{ marginTop: 8, marginBottom: 4 }}>
        {nearby.length} {nearby.length === 1 ? 'place' : 'places'} within {radiusKm} km of central {dest.city}
      </T>
      {nearby.slice(0, 4).map(({ p, km }) => (
        <Press key={p.id} onPress={() => router.push({ pathname: '/nearby', params: { focus: p.id } })} style={styles.nearRow} scaleTo=
{0.99} accessibilityLabel={`${p.name}, ${kmLabel(km)} away`}>
          <View style={styles.nearIcon}>
            <Ionicons name={placeCategoryIcon[p.category]} size={18} color={colors.ink} />
          </View>
          <View style={{ flex: 1 }}>
            <T variant="bodySm" weight="semibold" numberOfLines={1}>
              {p.name}
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {p.category[0].toUpperCase() + p.category.slice(1)} · {lengthTag(p.durationMin)} · {p.cost ? money(p.cost) : 'Free'}
            </T>
          </View>
          <T variant="kicker" color={colors.ink}>
            {kmLabel(km)}
          </T>
        </Press>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  rowBetween: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 8 },
  nearRow: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, borderTopWidth: hair, borderTopColor: colors.border },
  nearIcon: { width: 40, height: 40, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.ca
rd, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/home/PicksSection.tsx

```
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { ScrollView, StyleSheet, View } from 'react-native';
```

```
import { SectionHeader } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { imageSource } from '@/data/images';
import { colors, line, radius } from '@/theme';

import type { pickedForYou } from './picks';

/** "Picked for you": photo cards in an ink outline, each with a stamp for how well it matches, ranked from the survey answers. */
export function PicksSection({ picks }: { picks: ReturnType<typeof pickedForYou> }) {
  return (
    <View>
      <SectionHeader title="Picked for you" action="Edit" onAction={() => router.push('/personalize')} />
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 12, paddingHorizontal: 20 }} style={{
marginHorizontal: -20 }}>
        {picks.map(({ d, match, why }) => (
          <Press key={d.id} onPress={() => router.push({ pathname: '/destination/[id]', params: { id: d.id } })} scaleTo={0.985} style=
{styles.pick} accessibilityLabel={`${d.city}, ${match}% match. ${why}`}>
            <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={200} />
            <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.8)']} locations={[0.4, 1]} style={StyleSheet.ab
soluteFill} />
            <View style={styles.match} pointerEvents="none">
              <Stamp label={`${match}% match`} tone="accent" />
            </View>
            <View style={styles.text} pointerEvents="none">
              <T variant="h3" color={colors.white} numberOfLines={1}>
                {d.city}
              </T>
              <T variant="kicker" color={colors.white} numberOfLines={1} style={{ fontSize: 10 }}>
                {d.country}
              </T>
            </View>
          </Press>
        ))}
      </ScrollView>
    </View>
  );
}

const styles = StyleSheet.create({
  pick: { width: 164, height: 204, borderRadius: radius.card, overflow: 'hidden', borderWidth: line, borderColor: colors.ink, background
Color: colors.bgAlt },
  match: { position: 'absolute', top: 12, left: 10 },
  text: { position: 'absolute', left: 12, right: 12, bottom: 10, gap: 2 },
});
```

src/components/home/PlaySection.tsx

```
import { StyleSheet, View } from 'react-native';

import { SectionHeader } from '@/components/ui/bits';
import { Rule } from '@/components/ui/Rule';
import { T } from '@/components/ui/T';
import { colors } from '@/theme';
import { crewStandings } from '@/utils/points';

import { podiumView } from './logic';
import { Podium } from './Podium';
import { TriviaSlip } from './TriviaSlip';
import type { HomeCtx } from './useHomeContext';
import { hasTrip } from './useHomeContext';
import { VoteCard } from './VoteCard';

/**
 * Play: today's trivia, the crew-points podium and one thing to vote on. Each part shows only when it has something to show,
 * and they sit open on the page, told apart by a small label and a rule, not by three boxes.
 */
export function PlaySection({ ctx }: { ctx: HomeCtx }) {
  const trip = hasTrip(ctx) ? ctx.trip : undefined;
  const podium = trip ? podiumView(crewStandings(trip), ctx.userId) : null;
  return (
    <View>
      <SectionHeader title="Play" />
      <TriviaSlip today={ctx.today} />
      {podium ? (
        <View style={styles.part}>
          <Rule weight={1} />
          <T variant="kicker" color={colors.textSecondary}>
            Crew points
          </T>
          <Podium view={podium} userId={ctx.userId} />
        </View>
      ) : null}
      {trip ? <VoteCard ctx={{ ...ctx, trip }} /> : null}
```

```
    </View>
  );
}

const styles = StyleSheet.create({
  part: { marginTop: 24, gap: 12 },
});
```

src/components/home/Podium.tsx

```
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { Avatar } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';

import { ordinal, podiumSlots, type PodiumView } from './logic';

const first = (name: string) => name.split(' ')[0];

/**
 * Crew points as a podium of three ink blocks of different heights (second, first, third), each with a face, a name and the points in m
ono.
 * The winner's block is the accent. When you are not on the podium your own row sits underneath, highlighted. The whole thing opens the
crew.
 */
export function Podium({ view, userId }: { view: PodiumView; userId: string }) {
  const slots = podiumSlots(view.top);
  const summary = view.top.map((s) => `${ordinal(s.rank)} ${s.memberId === userId ? 'you' : first(s.name)}, ${s.points} points`).join('.
');
  return (
    <Press onPress={() => router.push('/crew')} scaleTo={0.99} style={{ gap: 12 }} accessibilityLabel={`Crew points. ${summary}${view.me
? `. You are ${ordinal(view.me.rank)} with ${view.me.points} points` : ''}. Open the crew`}>
      <View style={styles.row}>
        {slots.map((s) => {
          const winner = s.rank === 1;
          const me = s.memberId === userId;
          return (
            <View key={s.memberId} style={styles.col}>
              <Avatar name={s.name} src={s.avatar} size={winner ? 56 : 46} />
              <View style={[styles.name, me && { backgroundColor: colors.accent }]}>
                <T variant="caption" weight="bold" color={me ? colors.onAccent : colors.ink} numberOfLines={1}>
                  {me ? 'You' : first(s.name)}
                </T>
              </View>
              <View style={[styles.block, { height: s.height }, winner && { backgroundColor: colors.accent }]}>
                <T variant="numeralSm" color={winner ? colors.onAccent : colors.bg}>
                  {s.rank}
                </T>
                <T variant="kicker" color={winner ? colors.onAccent : colors.bg}>
                  {s.points} pts
                </T>
              </View>
            </View>
          );
        })}
      </View>
      {view.me ? (
        <View style={styles.mine}>
          <Avatar name={view.me.name} src={view.me.avatar} size={32} />
          <T variant="bodySm" weight="bold" color={colors.onAccent} style={{ flex: 1 }}>
            You are {ordinal(view.me.rank)}
          </T>
          <T variant="numeralSm" color={colors.onAccent} style={{ fontSize: 17 }}>
            {view.me.points} pts
          </T>
        </View>
      ) : null}
    </Press>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'flex-end', gap: 8 },
  col: { flex: 1, alignItems: 'center', gap: 6 },
  name: { alignSelf: 'center', maxWidth: '100%', paddingHorizontal: 6, borderRadius: 3 },
  block: { alignSelf: 'stretch', backgroundColor: colors.ink, borderWidth: line, borderColor: colors.ink, borderTopLeftRadius: radius.ch
ip, borderTopRightRadius: radius.chip, alignItems: 'center', paddingTop: 6, gap: 2 },
  mine: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, paddingHorizontal: 10, borderRadius: radius.button, border
Width: line, borderColor: colors.ink, backgroundColor: colors.accent },
});
```

src/components/home/RadiusSlider.tsx

```
import { Slider } from '@/components/ui/Slider';

/** The radius slider on Home is the shared ink-and-paper Slider. */
export const RadiusSlider = Slider;
```

src/components/home/TriviaSlip.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Card } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';
import { dailyQuestion } from '@/utils/trivia';

import { answerState, streakLabel } from './logic';

const KEYS = ['A', 'B', 'C', 'D'];

/**
 * Today's trivia question as a quiz slip: four big answer buttons. Once you answer, the right one is marked with a tick, a wrong pick
 * with a cross (colour AND icon), the fact appears, and a streak stamp lands. Only the first answer of the day counts, and a right answ
er celebrates.
 */
export function TriviaSlip({ today }: { today: string }) {
  const trivia = useAppStore((s) => s.trivia);
  const answerTrivia = useAppStore((s) => s.answerTrivia);
  const q = dailyQuestion(today);
  const answered = trivia.day === today;
  const picked = answered ? trivia.picked : null;
  const [justNow, setJustNow] = useState(false);
  const right = picked !== null && picked === q.answer;
  const streak = streakLabel(trivia.streak);

  const pick = (i: number) => {
    if (answered) return;
    const correct = i === q.answer;
    answerTrivia(today, i, correct);
    setJustNow(true);
    if (correct) celebrate();
  };

  return (
    <Card style={styles.card}>
      <View style={styles.head}>
        <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
          Today’s trivia
        </T>
        {streak && answered ? <Stamp label={streak} tone="accent" slam={justNow} /> : null}
      </View>
      <T variant="h3" accessibilityRole="header">
        {q.q}
      </T>
      <View style={{ gap: 8 }}>
        {q.options.map((o, i) => {
          const st = answerState(i, q.answer, picked);
          const good = st === 'right' || st === 'reveal';
          const bad = st === 'wrong';
          return (
            <Press
              key={o}
              onPress={() => pick(i)}
              disabled={answered}
              scaleTo={0.985}
              accessibilityLabel={`${KEYS[i]}. ${o}${st === 'right' ? '. Correct' : st === 'wrong' ? '. Not this one' : st === 'reveal'
? '. The right answer' : ''}`}
              accessibilityState={{ disabled: answered, selected: i === picked }}
              style={[styles.option, good && { borderColor: colors.positive, backgroundColor: st === 'right' ? colors.positiveSoft : col
ors.card }, bad && { borderColor: colors.red, backgroundColor: colors.redSoft }, st === 'dim' && { opacity: 0.5 }]}>
              <View style={[styles.key, good && { borderColor: colors.positive }, bad && { borderColor: colors.red }]}>
                <T variant="kicker" color={good ? colors.positive : bad ? colors.red : colors.ink} style={{ letterSpacing: 0 }}>
                  {KEYS[i]}
                </T>
              </View>
              <T variant="bodySm" weight="semibold" numberOfLines={2} style={{ flex: 1 }}>
                {o}
              </T>
              {good ? <Ionicons name="checkmark-circle" size={22} color={colors.positive} /> : bad ? <Ionicons name="close-circle" size=
```

```
{22} color={colors.red} /> : null}
            </Press>
          );
        })}
      </View>
      {answered ? (
        <View style={styles.result} accessibilityLiveRegion="polite">
          <View style={styles.resultHead}>
            <Ionicons name={right ? 'checkmark-circle' : 'close-circle'} size={20} color={right ? colors.positive : colors.red} />
            <T variant="title" weight="bold" color={right ? colors.positive : colors.red}>
              {right ? 'Correct' : 'Not quite'}
            </T>
          </View>
          <T variant="small" color={colors.textSecondary}>
            {q.fact}
          </T>
        </View>
      ) : null}
    </Card>
  );
}

const styles = StyleSheet.create({
  card: { gap: 12 },
  head: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 24 },
  option: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingHorizontal: 12, paddingVertical: 8, borderRadius:
radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
  key: { width: 30, height: 30, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, alignItems: 'center', justifyConten
t: 'center' },
  result: { gap: 4, paddingTop: 12, borderTopWidth: 1, borderTopColor: colors.border },
  resultHead: { flexDirection: 'row', alignItems: 'center', gap: 6 },
});
```

src/components/home/UpNextSection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { Linking, StyleSheet, View } from 'react-native';

import { typeIcon } from '@/components/trip/meta';
import { SectionHeader } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import type { Activity } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';
import { colors, hair, line, radius } from '@/theme';
import { time12 } from '@/utils/time';

import { checklistProgress, dateKicker, mapsUrl, nextPlanned, startsIn, startsSoon, timeParts, todaysChecklist, votingCount } from './lo
gic';
import type { TripCtx } from './useHomeContext';

const openPlan = () => router.navigate('/(tabs)/itinerary');

/**
 * Up next: the next planned thing as a ticket (what and where on the main part; the start time and a way to get there on the stub),
 * today's checklist while the trip is on, and a quiet row when something is waiting for votes. With nothing planned it invites you to P
lan.
 */
export function UpNextSection({ ctx }: { ctx: TripCtx }) {
  const { trip, status, tz, now, today, dayNo, tripLen } = ctx;
  const updateActivity = useAppStore((s) => s.updateActivity);
  const next = nextPlanned(trip.activities, tz, now);
  const todays = status === 'active' ? todaysChecklist(trip.activities, today) : [];
  const progress = checklistProgress(todays);
  const voting = votingCount(trip.activities);

  const openMaps = () => {
    if (next) Linking.openURL(mapsUrl(next.a)).catch(() => toast('Could not open maps on this device', { tone: 'warn' }));
  };

  return (
    <View>
      <SectionHeader title="Up next" action="See all" onAction={openPlan} />

      {next ? (
        <TicketOf
          a={next.a}
          ms={next.startUtc - now}
          onOpen={openPlan}
          onMaps={openMaps}
        />
```

```
      ) : (
        <View style={styles.invite}>
          <T variant="h3">Nothing planned yet</T>
          <T variant="small" color={colors.textSecondary}>
            Add the first thing to the plan and the crew can vote on it.
          </T>
          <Button label="Open the plan" icon="calendar-outline" variant="secondary" size="md" full={false} onPress={openPlan} />
        </View>
      )}

      {status === 'active' ? (
        <View style={{ marginTop: 20 }}>
          <View style={styles.listHead}>
            <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
              Today · Day {dayNo} of {tripLen}
            </T>
            <T variant="kicker" color={colors.ink}>
              {progress.done} / {progress.total} done
            </T>
          </View>
          {todays.length ? (
            todays.map((a) => (
              <Press
                key={a.id}
                onPress={() => updateActivity(trip.id, a.id, { done: !a.done })}
                style={styles.checkRow}
                scaleTo={0.99}
                accessibilityRole="checkbox"
                accessibilityState={{ checked: !!a.done }}
                accessibilityLabel={`${a.title} at ${time12(a.time)}`}>
                <View style={[styles.box, a.done && { backgroundColor: colors.accent }]}>{a.done ? <Ionicons name="checkmark" size={18}
color={colors.onAccent} /> : null}</View>
                <T variant="kicker" color={colors.textSecondary} style={{ width: 70 }}>
                  {time12(a.time)}
                </T>
                <T variant="bodySm" weight="semibold" numberOfLines={1} style={[{ flex: 1 }, a.done && { textDecorationLine: 'line-throu
gh', color: colors.textMuted }]}>
                  {a.title}
                </T>
              </Press>
            ))
          ) : (
            <T variant="small" color={colors.textSecondary} style={{ paddingVertical: 10 }}>
              A free day. Nothing on the plan.
            </T>
          )}
        </View>
      ) : null}

      {voting > 0 ? (
        <Press onPress={() => router.navigate({ pathname: '/(tabs)/itinerary', params: { tab: 'vote' } })} style={styles.voteRow} scaleT
o={0.99} accessibilityLabel={`${voting} ${voting === 1 ? 'thing' : 'things'} to vote on. Open the Vote tab`}>
          <Ionicons name="thumbs-up-outline" size={18} color={colors.ink} />
          <T variant="bodySm" weight="semibold" style={{ flex: 1 }}>
            {voting} {voting === 1 ? 'thing' : 'things'} to vote on
          </T>
          <Ionicons name="arrow-forward" size={16} color={colors.ink} />
        </Press>
      ) : null}
    </View>
  );
}

/** The ticket for one activity. The main part opens the plan; the stub holds the time and the maps button (siblings, never nested). */
function TicketOf({ a, ms, onOpen, onMaps }: { a: Activity; ms: number; onOpen: () => void; onMaps: () => void }) {
  const time = timeParts(a.time);
  const soon = startsSoon(ms);
  return (
    <Ticket
      plated={false}
      stub={
        <View style={styles.stub}>
          <View>
            <T variant="kicker" color={colors.textSecondary} style={{ fontSize: 9.5 }}>
              Starts
            </T>
            <View style={styles.clock} accessible accessibilityLabel={`Starts at ${time12(a.time)}`}>
              <T variant="numeral">{time.clock}</T>
              <T variant="kicker" color={colors.ink}>
                {time.suffix}
              </T>
            </View>
          </View>
          <Button label="Open in maps" icon="navigate-outline" variant="secondary" size="sm" full={false} onPress={onMaps} accessibility
Hint="Opens the place in your maps app" />
        </View>
```

```
      }>
      <Press onPress={onOpen} scaleTo={0.99} style={styles.main} accessibilityLabel={`Next: ${a.title}, ${a.location}. ${startsIn(ms)}.
Open the plan`}>
        <View style={styles.topRow}>
          <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }} numberOfLines={1}>
            {dateKicker(a.date)}
          </T>
          <Stamp label={startsIn(ms)} tone={soon ? 'accent' : 'ink'} />
        </View>
        <View style={styles.titleRow}>
          <View style={styles.kind}>
            <Ionicons name={typeIcon(a.type)} size={22} color={colors.ink} />
          </View>
          <View style={{ flex: 1, gap: 2 }}>
            <T variant="h2" numberOfLines={2}>
              {a.title}
            </T>
            <View style={styles.place}>
              <Ionicons name="location-outline" size={14} color={colors.textSecondary} />
              <T variant="small" color={colors.textSecondary} numberOfLines={1} style={{ flex: 1 }}>
                {a.location}
              </T>
            </View>
          </View>
        </View>
      </Press>
    </Ticket>
  );
}

const styles = StyleSheet.create({
  main: { padding: 16, gap: 14 },
  topRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  titleRow: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
  kind: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt,
alignItems: 'center', justifyContent: 'center' },
  place: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  stub: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', justifyContent: 'space-between', columnGap: 12, rowGap: 10, padd
ingHorizontal: 16, paddingBottom: 14, paddingTop: 2 },
  clock: { flexDirection: 'row', alignItems: 'baseline', gap: 6 },
  invite: { gap: 8, padding: 16, borderWidth: line, borderStyle: 'dashed', borderColor: colors.ink, borderRadius: radius.card, alignItem
s: 'flex-start' },
  listHead: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingBottom: 8, borderBottomWidth: line, borderBottomColor: colors.i
nk },
  checkRow: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, borderBottomWidth: hair, borderBottomColor: colors.bor
der },
  box: { width: 26, height: 26, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, align
Items: 'center', justifyContent: 'center' },
  voteRow: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, marginTop: 8, borderBottomWidth: hair, borderBottomColo
r: colors.border },
});
```

src/components/home/VoteCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { Rule } from '@/components/ui/Rule';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, line, radius } from '@/theme';
import { dayLabel, time12, weekdayShort } from '@/utils/time';

import { nextToVote, voteLine, voteLocks, voteTally } from './logic';
import type { TripCtx } from './useHomeContext';

/** One small square per yes vote the activity needs: filled with the accent when someone has said yes. */
function Ballot({ ups, needed }: { ups: number; needed: number }) {
  return (
    <View style={{ flexDirection: 'row', gap: 6 }} accessibilityElementsHidden importantForAccessibility="no">
      {Array.from({ length: Math.min(needed, 8) }, (_, i) => (
        <View key={i} style={[styles.box, i < ups && { backgroundColor: colors.accent }]}>
          {i < ups ? <Ionicons name="checkmark" size={13} color={colors.onAccent} /> : null}
        </View>
      ))}
    </View>
  );
}

/**
```

```
 * "Up for a vote": the first plan item still waiting for your vote, with big yes and no buttons and the live tally.
 * Your vote counts at once; if it is the one that makes a majority the item locks in with a stamp and a small celebration.
 * The item stays on screen, now settled, until you move on to the next one.
 */
export function VoteCard({ ctx }: { ctx: TripCtx }) {
  const { trip, userId } = ctx;
  const vote = useAppStore((s) => s.voteActivity);
  const [done, setDone] = useState<{ id: string; value: 1 | -1; locked: boolean } | null>(null);
  const next = nextToVote(trip.activities, userId);
  const shown = done ? trip.activities.find((a) => a.id === done.id) : next;
  if (!shown) return null;
  const n = trip.members.length;
  const tally = voteTally(shown, n);

  const cast = (value: 1 | -1) => {
    const locked = voteLocks(shown, userId, n, value);
    vote(trip.id, shown.id, value);
    setDone({ id: shown.id, value, locked });
    if (locked) celebrate(`Locked in: ${shown.title}`);
    else toast(value === 1 ? 'Your yes is counted' : 'Your no is counted', { icon: value === 1 ? 'thumbs-up' : 'thumbs-down' });
  };

  return (
    <View style={{ marginTop: 24, gap: 12 }}>
      <Rule weight={1} />
      <View style={styles.head}>
        <T variant="kicker" color={colors.textSecondary} style={{ flex: 1 }}>
          Up for a vote
        </T>
        {done ? <Stamp label={done.locked || shown.status !== 'voting' ? 'Locked in' : 'Vote counted'} tone={done.locked || shown.status
!== 'voting' ? 'positive' : 'ink'} icon={done.locked || shown.status !== 'voting' ? 'lock-closed' : 'checkmark'} slam /> : null}
      </View>
      <View style={{ gap: 2 }}>
        <T variant="h2" numberOfLines={2}>
          {shown.title}
        </T>
        <T variant="small" color={colors.textSecondary} numberOfLines={1}>
          {weekdayShort(shown.date)} {dayLabel(shown.date)} · {time12(shown.time)} · {shown.location}
        </T>
      </View>
      <View style={styles.tally}>
        <Ballot ups={tally.ups} needed={tally.needed} />
        <T variant="caption" color={colors.textSecondary} style={{ flex: 1 }}>
          {voteLine(shown, n)}
        </T>
      </View>
      {done ? (
        <View style={{ gap: 8 }}>
          <T variant="bodySm" weight="semibold">
            {done.value === 1 ? 'You voted yes.' : 'You voted no.'}
          </T>
          {next ? <Button label="Next vote" iconRight="arrow-forward" variant="secondary" size="md" full={false} onPress={() => setDone
(null)} /> : (
            <Button label="See the plan" iconRight="arrow-forward" variant="secondary" size="md" full={false} onPress={() => router.navi
gate('/(tabs)/itinerary')} />
          )}
        </View>
      ) : (
        <View style={styles.buttons}>
          <Button label="I'm in" icon="thumbs-up-outline" variant="accent" size="md" style={{ flex: 1 }} onPress={() => cast(1)} accessi
bilityHint={`Vote yes on ${shown.title}`} />
          <Button label="Not for me" icon="thumbs-down-outline" variant="secondary" size="md" style={{ flex: 1 }} onPress={() => cast(-
1)} accessibilityHint={`Vote no on ${shown.title}`} />
        </View>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  head: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 24 },
  tally: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  buttons: { flexDirection: 'row', gap: 12 },
  box: { width: 22, height: 22, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, align
Items: 'center', justifyContent: 'center' },
});
```

src/components/home/WeatherSection.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { ScrollView, StyleSheet, View } from 'react-native';

import { SectionHeader } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Skeleton } from '@/components/ui/Skeleton';
```

```
import { T } from '@/components/ui/T';
import { useWeather } from '@/hooks/useTrip';
import { weatherInfo } from '@/services/weather';
import { colors, isDark, line, radius } from '@/theme';
import { dayLabel, deviceTimeZone, weekdayShort } from '@/utils/time';

import { clockPair, typicalIcon, weatherView, type WeatherDay } from './logic';
import type { TripCtx } from './useHomeContext';

/** The board is black with paper-coloured print in the light mode; at night it is the dark card with cream print and a cream outline.
*/
const BOARD = isDark ? { bg: colors.card, fg: colors.ink } : { bg: colors.ink, fg: colors.bg };

/** Today's column is lit in the accent; in the Ink theme the accent is the board's own colour, so it is lit in paper instead. */
const LIT = colors.accent === BOARD.bg ? { bg: BOARD.fg, fg: BOARD.bg } : { bg: colors.accent, fg: colors.onAccent };

/** One trip day as a column of the board: the weekday, the date, an icon, the high and the low. Today is the highlighted column. */
function DayColumn({ d, today }: { d: WeatherDay; today: boolean }) {
  const icon = d.code === null ? typicalIcon(d.max) : weatherInfo(d.code).icon;
  const fg = today ? LIT.fg : BOARD.fg;
  return (
    <View style={[styles.col, today && { backgroundColor: LIT.bg }]} accessible accessibilityLabel={`${weekdayShort(d.date)} ${dayLabel
(d.date)}: high ${d.max}, low ${d.min}`}>
      <T variant="kicker" color={fg}>
        {weekdayShort(d.date)}
      </T>
      <T variant="kicker" color={fg} style={{ opacity: 0.7 }}>
        {Number(d.date.slice(8))}
      </T>
      <Ionicons name={icon} size={24} color={fg} style={{ marginVertical: 8 }} />
      <T variant="numeralSm" color={fg}>
        {d.max}°
      </T>
      <T variant="kicker" color={fg} style={{ opacity: 0.7, fontSize: 13, lineHeight: 18 }}>
        {d.min}°
      </T>
    </View>
  );
}

/** The same size as the board, so nothing jumps when the forecast arrives. */
function BoardSkeleton() {
  return (
    <View accessibilityLabel="Loading the forecast">
      <Skeleton height={188} corner={radius.card} />
    </View>
  );
}

/**
 * Weather as a departures board: a dark panel with one column per day of the trip (weekday, icon, high and low, today lit in
 * the accent), scrolling sideways when the trip is long, and the time there and at home in mono along the bottom.
 * Before the forecast reaches the trip it says "Typical weather", and says why.
 */
export function WeatherSection({ ctx }: { ctx: TripCtx }) {
  const { trip, dest, today, now, tz } = ctx;
  const snapshot = useWeather(dest?.id);
  const city = dest?.city ?? 'your destination';
  const view = snapshot ? weatherView({ startDate: trip.startDate, endDate: trip.endDate, today, lat: dest?.lat ?? 0, snapshot }) : nul
l;
  const clocks = clockPair(now, tz, deviceTimeZone());
  const open = () => router.push('/weather');
  return (
    <View>
      <SectionHeader title="Weather" action="Details" onAction={open} />
      {view ? (
        <>
          <Press onPress={open} scaleTo={0.99} style={styles.board} accessibilityLabel={`${view.title} in ${city}. Open the weather`}>
            <View style={styles.boardHead}>
              <View style={styles.bullet} />
              <T variant="kicker" color={BOARD.fg} numberOfLines={1} style={{ flex: 1 }}>
                {view.title} · {city}
              </T>
            </View>
            <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.cols}>
              {view.days.map((d) => (
                <DayColumn key={d.date} d={d} today={d.date === today} />
              ))}
            </ScrollView>
            <View style={styles.clocks}>
              <View accessible accessibilityLabel={`${clocks.there} in ${city}`}>
                <T variant="kicker" color={BOARD.fg} style={{ opacity: 0.7, fontSize: 9.5 }}>
                  {city}
                </T>
                <T variant="numeralSm" color={BOARD.fg} style={{ fontSize: 17, lineHeight: 22 }}>
                  {clocks.there}
```

```
              </T>
            </View>
            <View accessible accessibilityLabel={clocks.same ? `${clocks.home} at home, the same time` : `${clocks.home} at home`}>
              <T variant="kicker" color={BOARD.fg} style={{ opacity: 0.7, fontSize: 9.5, textAlign: 'right' }}>
                Home
              </T>
              <T variant="numeralSm" color={BOARD.fg} style={{ fontSize: 17, lineHeight: 22, textAlign: 'right' }}>
                {clocks.home}
              </T>
            </View>
          </View>
        </Press>
        {view.kind === 'typical' ? (
          <T variant="caption" color={colors.textSecondary} style={{ marginTop: 10 }}>
            {view.note}
          </T>
        ) : null}
      </>
    ) : (
      <BoardSkeleton />
    )}
  </View>
 );
}

const styles = StyleSheet.create({
  board: { backgroundColor: BOARD.bg, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, paddingTop: 14, paddingBott
om: 12, overflow: 'hidden' },
  boardHead: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingHorizontal: 14, paddingBottom: 12 },
  bullet: { width: 10, height: 10, backgroundColor: LIT.bg },
  cols: { flexGrow: 1, paddingHorizontal: 8, gap: 2 },
  col: { flex: 1, minWidth: 50, alignItems: 'center', paddingVertical: 10, borderRadius: radius.chip },
  clocks: { flexDirection: 'row', alignItems: 'flex-end', justifyContent: 'space-between', paddingHorizontal: 14, paddingTop: 12, margin
Top: 8, borderTopWidth: 1, borderTopColor: colors.textMuted },
});
```

src/components/home/__tests__/logic.test.ts

```
import type { Activity, Trip, WeatherSnapshot } from '@/store/types';
import { todayInTz, zonedToUtc } from '@/utils/time';

import {
  FORECAST_HORIZON_DAYS,
  answerState,
  blockHasContent,
  budgetView,
  celebrateKey,
  checklistProgress,
  clockPair,
  countdownView,
  crewCounts,
  crewCue,
  dateKicker,
  mapsUrl,
  nextPlanned,
  nextToVote,
  ordinal,
  oweLine,
  podiumSlots,
  podiumView,
  shouldCelebrateOneDay,
  startsIn,
  startsSoon,
  streakLabel,
  timeParts,
  todaysChecklist,
  typicalDay,
  typicalIcon,
  voteLine,
  voteLocks,
  voteTally,
  votingCount,
  weatherView,
} from '../logic';

const act = (over: Partial<Activity>): Activity => ({
  id: 'a',
  date: '2026-10-23',
  time: '10:00',
  title: 'Thing',
  location: 'Somewhere',
  type: 'food',
  durationMin: 60,
  costPerPerson: 0,
  status: 'confirmed',
  assigned: [],
```

```
  votes: {},
  reminder: false,
  createdBy: 'u1',
  ...over,
});

describe('the big headline', () => {
  it('counts the days to go, in the singular for one', () => {
    expect(countdownView('planning', 3, 0, 6)).toMatchObject({ kind: 'days', number: 3, words: ['Days', 'to go'], sentence: '3 days to g
o' });
    expect(countdownView('planning', 1, 0, 6)).toMatchObject({ number: 1, words: ['Day', 'to go'], sentence: '1 day to go' });
  });

  it('says TODAY on the first day of the trip', () => {
    expect(countdownView('active', 0, 1, 6)).toMatchObject({ kind: 'today', words: ['Today'] });
    expect(countdownView('planning', 0, 0, 6).kind).toBe('today');
  });

  it('says DAY 2 OF 5 while the trip is on', () => {
    expect(countdownView('active', -1, 2, 5)).toMatchObject({ kind: 'day', number: 2, prefix: 'Day', words: ['of 5'], sentence: 'Day 2 o
f 5' });
  });

  it('says TRIP COMPLETE when it is over and WHERE TO NEXT? with no trip', () => {
    expect(countdownView('completed', -9, 6, 6).words).toEqual(['Trip complete']);
    expect(countdownView(null, 0, 0, 0).words).toEqual(['Where to next?']);
  });
});

describe('celebrating "1 day to go" once', () => {
  it('fires on the last day before the trip, then not again', () => {
    expect(shouldCelebrateOneDay('planning', 1, {}, 't1', '2026-10-22')).toBe(true);
    const fired = { [celebrateKey('t1', '2026-10-22')]: Date.now() };
    expect(shouldCelebrateOneDay('planning', 1, fired, 't1', '2026-10-22')).toBe(false);
    expect(shouldCelebrateOneDay('planning', 1, fired, 't2', '2026-10-22')).toBe(true);
  });

  it('is quiet on every other day', () => {
    for (const days of [0, 2, 3, 30]) expect(shouldCelebrateOneDay('planning', days, {}, 't1', 'd')).toBe(false);
    expect(shouldCelebrateOneDay('active', 1, {}, 't1', 'd')).toBe(false);
    expect(shouldCelebrateOneDay(null, 1, {}, 't1', 'd')).toBe(false);
  });
});

describe('which blocks show', () => {
  const ctx = { hasTrip: true, status: 'planning' as const, hasPicks: true, hasDestination: true };
  it('hides the trip blocks when there is no trip, but keeps play and discover', () => {
    const none = { ...ctx, hasTrip: false, status: null };
    for (const k of ['upnext', 'weather', 'budget', 'crew', 'nearby'] as const) expect(blockHasContent(k, none)).toBe(false);
    expect(blockHasContent('play', none)).toBe(true);
    expect(blockHasContent('discover', none)).toBe(true);
  });

  it('drops up next and weather once the trip is over, keeps the budget and the crew', () => {
    const done = { ...ctx, status: 'completed' as const };
    expect(blockHasContent('upnext', done)).toBe(false);
    expect(blockHasContent('weather', done)).toBe(false);
    expect(blockHasContent('budget', done)).toBe(true);
    expect(blockHasContent('crew', done)).toBe(true);
  });

  it('shows Picked for you only when there are picks', () => {
    expect(blockHasContent('forYou', ctx)).toBe(true);
    expect(blockHasContent('forYou', { ...ctx, hasPicks: false })).toBe(false);
  });
});

describe('up next', () => {
  const tz = 'Europe/Rome';
  const noon = zonedToUtc('2026-10-23', '12:00', tz);

  it('picks the earliest planned activity that has not started', () => {
    const list = [act({ id: 'late', time: '19:00' }), act({ id: 'soon', time: '13:30' }), act({ id: 'past', time: '08:00' })];
    expect(nextPlanned(list, tz, noon)?.a.id).toBe('soon');
  });

  it('ignores things still being voted on and things already ticked off', () => {
    const list = [act({ id: 'vote', time: '12:30', status: 'voting' }), act({ id: 'done', time: '12:45', done: true }), act({ id: 'ok',
time: '15:00' })];
    expect(nextPlanned(list, tz, noon)?.a.id).toBe('ok');
  });

  it('keeps something that began under half an hour ago, then moves on', () => {
    const list = [act({ id: 'now', time: '11:45' }), act({ id: 'next', time: '16:00' })];
    expect(nextPlanned(list, tz, noon)?.a.id).toBe('now');
    expect(nextPlanned(list, tz, zonedToUtc('2026-10-23', '12:30', tz))?.a.id).toBe('next');
```

```
  });

  it('returns null when nothing is planned', () => {
    expect(nextPlanned([], tz, noon)).toBeNull();
    expect(nextPlanned([act({ status: 'voting' })], tz, noon)).toBeNull();
  });

  it('words the countdown', () => {
    expect(startsIn(3 * 86400000 + 13 * 3600000)).toBe('in 3d 13h');
    expect(startsIn(95 * 60000)).toBe('in 1h 35m');
    expect(startsIn(20000)).toBe('in 1m');
    expect(startsIn(0)).toBe('Now');
    expect(startsIn(-5000)).toBe('Now');
  });

  it('links to the exact spot when it is known, otherwise searches for the name', () => {
    expect(mapsUrl({ lat: 40.6, lng: 14.4, title: 'X', location: 'Y' })).toBe('https://www.google.com/maps/search/?api=1&query=40.6,14.
4');
    expect(mapsUrl({ title: 'Da Vincenzo', location: 'Viale Pasitea' })).toContain('query=Da%20Vincenzo%20Viale%20Pasitea');
  });

  it('builds today’s checklist in time order without voting items', () => {
    const list = [act({ id: 'b', time: '19:00' }), act({ id: 'a', time: '08:00' }), act({ id: 'v', time: '12:00', status: 'voting' }), a
ct({ id: 'other', date: '2026-10-24' })];
    expect(todaysChecklist(list, '2026-10-23').map((x) => x.id)).toEqual(['a', 'b']);
  });
});

describe('budget state', () => {
  it('is on track under 80%, close from 80% to 100%, over above 100%', () => {
    expect(budgetView(2086, 4000)).toMatchObject({ tone: 'ok', word: 'On track', pct: 52 });
    expect(budgetView(3199, 4000).tone).toBe('ok');
    expect(budgetView(3200, 4000)).toMatchObject({ tone: 'warn', word: 'Getting close' });
    expect(budgetView(4000, 4000).tone).toBe('warn');
    expect(budgetView(4001, 4000)).toMatchObject({ tone: 'over', word: 'Over budget' });
  });

  it('says it in words as well as colour', () => {
    expect(budgetView(2086, 4000).detail).toBe('52% used · $1,914 left');
    expect(budgetView(4250, 4000).detail).toBe('$250 over');
  });

  it('copes with no budget at all', () => {
    expect(budgetView(100, 0)).toMatchObject({ tone: 'ok', ratio: 0 });
  });
});

describe('who owes whom', () => {
  const members = [
    { id: 'me', name: 'Maya Chen', role: 'Member' },
    { id: 'p', name: 'Priya Patel', role: 'Member' },
    { id: 's', name: 'Sam Okafor', role: 'Member' },
  ] as Trip['members'];
  const exp = (paidBy: string, amount: number, extra: Partial<Trip['expenses'][number]> = {}) =>
    ({ id: `${paidBy}${amount}`, title: 'x', amount, category: 'Food', paidBy, splitBetween: ['me', 'p', 's'], date: '2026-10-01', ...ex
tra }) as Trip['expenses'][number];

  it('says nothing to split before any spending', () => {
    expect(oweLine({ members, expenses: [] }, 'me')).toEqual({ tone: 'square', text: 'Nothing to split yet' });
  });

  it('names the person you owe', () => {
    expect(oweLine({ members, expenses: [exp('p', 300)] }, 'me')).toEqual({ tone: 'owe', text: 'You owe Priya $100' });
  });

  it('says who owes you, or how many people do', () => {
    expect(oweLine({ members, expenses: [exp('me', 300, { splitBetween: ['me', 'p'] })] }, 'me')).toEqual({ tone: 'owed', text: 'Priya o
wes you $150' });
    expect(oweLine({ members, expenses: [exp('me', 300)] }, 'me').text).toBe('You’re owed $200 by 2 people');
  });

  it('is square once settled', () => {
    const settled = [exp('p', 300), exp('me', 100, { settlement: true, splitBetween: ['p'] })];
    // Maya paid Priya back $100, so Maya is square; Sam still owes Priya
    expect(oweLine({ members, expenses: settled }, 'me').tone).not.toBe('owe');
    const all = [exp('p', 300), exp('me', 100, { settlement: true, splitBetween: ['p'] }), exp('s', 100, { settlement: true, splitBetwee
n: ['p'] })];
    expect(oweLine({ members, expenses: all }, 'me')).toEqual({ tone: 'square', text: 'Everyone is square' });
  });
});

describe('crew cues', () => {
  it('marks a member added by name only as Invited, everyone else as Going', () => {
    expect(crewCue({ invited: true })).toBe('Invited');
    expect(crewCue({})).toBe('Going');
    expect(crewCounts([{}, {}, { invited: true }])).toEqual({ going: 2, invited: 1 });
```

```
  });
});

describe('ordinals', () => {
  it('writes 1st, 2nd, 3rd, 4th, 11th, 12th, 13th, 21st, 112th', () => {
    expect([1, 2, 3, 4, 11, 12, 13, 21, 22, 23, 112].map(ordinal)).toEqual(['1st', '2nd', '3rd', '4th', '11th', '12th', '13th', '21st',
'22nd', '23rd', '112th']);
  });
});

describe('the podium', () => {
  const s = (memberId: string, points: number) => ({ memberId, name: memberId.toUpperCase(), points });
  it('has nothing to show before anyone has points, or with one person', () => {
    expect(podiumView([s('a', 0), s('b', 0)], 'a')).toBeNull();
    expect(podiumView([s('a', 5)], 'a')).toBeNull();
  });

  it('lists the top three and no extra row when you are on the podium', () => {
    const v = podiumView([s('a', 9), s('b', 7), s('c', 3), s('d', 1)], 'b');
    expect(v?.top.map((x) => [x.memberId, x.rank])).toEqual([['a', 1], ['b', 2], ['c', 3]]);
    expect(v?.me).toBeNull();
  });

  it('adds your own row, with your rank, when you are outside the top three', () => {
    const v = podiumView([s('a', 9), s('b', 7), s('c', 3), s('d', 1)], 'd');
    expect(v?.me).toMatchObject({ memberId: 'd', rank: 4, points: 1 });
  });

  it('works with just two people', () => {
    expect(podiumView([s('a', 4), s('b', 1)], 'b')?.top).toHaveLength(2);
  });
});

describe('voting', () => {
  it('finds the first thing you have not voted on, earliest first', () => {
    const list = [
      act({ id: 'late', status: 'voting', date: '2026-10-26', time: '16:00' }),
      act({ id: 'voted', status: 'voting', date: '2026-10-26', time: '08:00', votes: { u1: 1 } }),
      act({ id: 'early', status: 'voting', date: '2026-10-26', time: '11:00' }),
      act({ id: 'plain' }),
    ];
    expect(nextToVote(list, 'u1')?.id).toBe('early');
    expect(nextToVote(list.filter((a) => a.id !== 'early' && a.id !== 'late'), 'u1')).toBeUndefined();
  });

  it('counts what is waiting for votes', () => {
    expect(votingCount([act({ status: 'voting' }), act({ status: 'voting' }), act({})])).toBe(2);
  });

  it('works out how many "in" votes lock it in (more than half)', () => {
    expect(voteTally({ votes: { a: 1, b: -1, c: 1 } }, 4)).toEqual({ ups: 2, downs: 1, needed: 3 });
    expect(voteTally({ votes: {} }, 5).needed).toBe(3);
  });
});

describe('trivia answers', () => {
  it('leaves every option open until an answer is picked', () => {
    expect([0, 1, 2, 3].map((i) => answerState(i, 2, null))).toEqual(['open', 'open', 'open', 'open']);
  });

  it('shows a right answer as right and dims the others', () => {
    expect([0, 1, 2, 3].map((i) => answerState(i, 2, 2))).toEqual(['dim', 'dim', 'right', 'dim']);
  });

  it('shows a wrong pick as wrong and reveals the right one', () => {
    expect([0, 1, 2, 3].map((i) => answerState(i, 2, 0))).toEqual(['wrong', 'dim', 'reveal', 'dim']);
  });
});

describe('weather', () => {
  const today = '2026-10-20';
  const snap = (live: boolean): WeatherSnapshot => ({
    destinationId: 'positano',
    fetchedAt: Date.now(),
    live,
    current: { temp: 20, code: 1, wind: 5, humidity: 50 },
    daily: Array.from({ length: 7 }, (_, i) => ({ date: `2026-10-${20 + i}`, max: 22 + i, min: 14, code: 1, rain: 10 })),
  });

  it('shows the live forecast for the trip days it covers', () => {
    const v = weatherView({ startDate: '2026-10-23', endDate: '2026-10-28', today, lat: 40.6, snapshot: snap(true) });
    expect(v.kind).toBe('forecast');
    expect(v.title).toBe('Forecast');
    expect(v.days.map((d) => d.date)).toEqual(['2026-10-23', '2026-10-24', '2026-10-25', '2026-10-26']);
    expect(v.days[0]).toMatchObject({ max: 25, min: 14, code: 1 });
  });
```

```
  it('only shows days from today when the trip has begun', () => {
    const v = weatherView({ startDate: '2026-10-18', endDate: '2026-10-23', today, lat: 40.6, snapshot: snap(true) });
    expect(v.days.map((d) => d.date)).toEqual(['2026-10-20', '2026-10-21', '2026-10-22', '2026-10-23']);
  });

  it(`says "Typical weather" when the trip is more than ${FORECAST_HORIZON_DAYS} days away`, () => {
    const v = weatherView({ startDate: '2026-11-15', endDate: '2026-11-20', today, lat: 40.6, snapshot: snap(true) });
    expect(v.kind).toBe('typical');
    expect(v.title).toBe('Typical weather');
    expect(v.note).toContain('November');
    expect(v.note).toContain(`${FORECAST_HORIZON_DAYS} days before`);
    expect(v.days).toHaveLength(6);
  });

  it('does not pass off an offline estimate as a forecast', () => {
    const v = weatherView({ startDate: '2026-10-23', endDate: '2026-10-28', today, lat: 40.6, snapshot: snap(false) });
    expect(v.kind).toBe('typical');
    expect(v.note).toContain('not available');
    expect(weatherView({ startDate: '2026-10-23', endDate: '2026-10-28', today, lat: 40.6, snapshot: undefined }).kind).toBe('typical');
  });

  it('shows at most seven days', () => {
    expect(weatherView({ startDate: '2027-03-01', endDate: '2027-03-20', today, lat: 10, snapshot: undefined }).days).toHaveLength(7);
  });

  it('has a sensible typical climate: warmer in summer, reversed in the south, warmer near the equator', () => {
    const jul = typicalDay(40.6, '2026-07-15');
    const jan = typicalDay(40.6, '2026-01-15');
    expect(jul.max).toBeGreaterThan(jan.max + 10);
    expect(jul.max).toBeGreaterThan(jul.min);
    expect(jul.max).toBeLessThan(35);
    expect(typicalDay(-33.9, '2026-01-15').max).toBeGreaterThan(typicalDay(-33.9, '2026-07-15').max);
    expect(typicalDay(-8.3, '2026-03-10').max).toBeGreaterThan(typicalDay(48.9, '2026-03-10').max);
    expect(Math.abs(typicalDay(-8.3, '2026-03-10').max - typicalDay(-8.3, '2026-09-10').max)).toBeLessThanOrEqual(4); // hardly any seas
ons near the equator
    expect(typicalDay(0, '2026-03-10').max).toBe(typicalDay(0, '2026-09-10').max);
  });

  it('picks an icon from the temperature', () => {
    expect([30, 20, 5].map(typicalIcon)).toEqual(['sunny', 'partly-sunny', 'cloud-outline']);
  });
});

describe('the clocks', () => {
  it('shows the time there and at home', () => {
    const at = Date.UTC(2026, 9, 20, 13, 41);
    expect(clockPair(at, 'Europe/Rome', 'America/New_York')).toEqual({ there: '3:41 PM', home: '9:41 AM', same: false });
  });

  it('says so when it is the same time', () => {
    const at = Date.UTC(2026, 9, 20, 13, 41);
    expect(clockPair(at, 'Europe/Rome', 'Europe/Paris').same).toBe(true);
  });

  it('uses the destination’s own date for "today"', () => {
    expect(todayInTz('Europe/Rome')).toMatch(/^\d{4}-\d{2}-\d{2}$/);
  });
});

describe('the ticket print', () => {
  it('splits a time into the clock and AM or PM', () => {
    expect(timeParts('19:30')).toEqual({ clock: '7:30', suffix: 'PM' });
    expect(timeParts('00:05')).toEqual({ clock: '12:05', suffix: 'AM' });
    expect(timeParts('12:00')).toEqual({ clock: '12:00', suffix: 'PM' });
  });

  it('writes the date line of a ticket', () => {
    expect(dateKicker('2026-10-23')).toBe('Fri 23 Oct');
    expect(dateKicker('2026-01-05')).toBe('Mon 5 Jan');
  });

  it('counts the ticked-off things of the day', () => {
    expect(checklistProgress([])).toEqual({ done: 0, total: 0 });
    expect(checklistProgress([{ done: true }, {}, { done: false }])).toEqual({ done: 1, total: 3 });
  });

  it('gives the countdown the accent only in the last three hours', () => {
    expect(startsSoon(2 * 3600000)).toBe(true);
    expect(startsSoon(0)).toBe(true);
    expect(startsSoon(3 * 3600000)).toBe(false);
    expect(startsSoon(3 * 86400000)).toBe(false);
  });
});

describe('a vote on Home', () => {
```

```
  const voting = (votes: Activity['votes']) => act({ status: 'voting', votes });

  it('words the tally', () => {
    expect(voteLine(voting({}), 5)).toBe('0 of 3 yes votes to lock it in');
    expect(voteLine(voting({ a: 1, b: 1 }), 5)).toBe('2 of 3 yes votes to lock it in');
    expect(voteLine(voting({ a: 1, b: 1, c: 1 }), 5)).toBe('3 yes, enough to lock it in');
  });

  it('locks in when this yes makes a majority, the same rule as the store', () => {
    expect(voteLocks(voting({ a: 1, b: 1 }), 'me', 5, 1)).toBe(true);
    expect(voteLocks(voting({ a: 1 }), 'me', 5, 1)).toBe(false);
    expect(voteLocks(voting({ a: 1, b: -1, c: 1 }), 'me', 4, 1)).toBe(true);
    expect(voteLocks(voting({ a: 1 }), 'me', 2, 1)).toBe(true);
  });

  it('never locks on a no, on something already decided, or on a vote that is taken back', () => {
    expect(voteLocks(voting({ a: 1, b: 1 }), 'me', 5, -1)).toBe(false);
    expect(voteLocks(act({ status: 'confirmed', votes: { a: 1, b: 1 } }), 'me', 5, 1)).toBe(false);
    expect(voteLocks(voting({ me: 1, a: 1 }), 'me', 3, 1)).toBe(false);
  });

  it('does not count the voter twice when they change their mind', () => {
    expect(voteLocks(voting({ me: -1, a: 1 }), 'me', 3, 1)).toBe(true);
    expect(voteLocks(voting({ me: -1, a: 1 }), 'me', 5, 1)).toBe(false);
  });
});

describe('the podium', () => {
  const ranked = (n: number) => Array.from({ length: n }, (_, i) => ({ memberId: `m${i}`, name: `P${i}`, points: 100 - i * 10, rank: i +
1 }));

  it('puts first in the middle, second on the left and third on the right', () => {
    expect(podiumSlots(ranked(3)).map((s) => s.rank)).toEqual([2, 1, 3]);
  });

  it('makes the winner the tallest and third the shortest', () => {
    const [second, first, third] = podiumSlots(ranked(3));
    expect(first.height).toBeGreaterThan(second.height);
    expect(second.height).toBeGreaterThan(third.height);
  });

  it('still works with only two people', () => {
    expect(podiumSlots(ranked(2)).map((s) => s.rank)).toEqual([2, 1]);
    expect(podiumSlots([])).toEqual([]);
  });
});

describe('the quiz streak', () => {
  it('has words only when there is a streak', () => {
    expect(streakLabel(0)).toBeNull();
    expect(streakLabel(1)).toBe('1 day streak');
    expect(streakLabel(4)).toBe('4 day streak');
  });
});
```

src/components/home/__tests__/picks.test.ts

```
import { destinations } from '@/data/destinations';
import { layoutAnswers } from '@/data/layouts';
import type { Personal } from '@/store/types';

import { pickedForYou } from '../picks';

const personal = (over: Partial<Personal>): Personal => ({
  done: true,
  nickname: '',
  emoji: '✈️',
  interests: [],
  travelStyle: 'balanced',
  budgetStyle: 'mid',
  ...layoutAnswers('planner'),
  effects: 'lively',
  triviaReminder: false,
  motion: true,
  ...over,
});

describe('picked for you', () => {
  it('shows nothing until an interest is chosen', () => {
    expect(pickedForYou(personal({ interests: [] }))).toEqual([]);
  });

  it('only suggests places that match a chosen interest, best match first', () => {
    const picks = pickedForYou(personal({ interests: ['beach'] }));
    expect(picks.length).toBeGreaterThan(0);
    expect(picks.length).toBeLessThanOrEqual(8);
```

```
    for (const { d } of picks) expect(d.tags).toContain('beach');
    const scores = picks.map((p) => p.match);
    expect([...scores].sort((a, b) => b - a)).toEqual(scores);
    for (const { match } of picks) {
      expect(match).toBeGreaterThanOrEqual(0);
      expect(match).toBeLessThanOrEqual(99);
    }
  });

  it('matching more of the chosen interests scores higher', () => {
    const [top] = pickedForYou(personal({ interests: ['beach', 'food', 'adventure'] }));
    const hits = (tags: string[]) => ['beach', 'food', 'adventure'].filter((i) => tags.includes(i)).length;
    const bestPossible = Math.max(...destinations.map((d) => hits(d.tags)));
    expect(hits(top.d.tags)).toBe(bestPossible);
  });

  it('prefers cheaper places for savers and pricier ones for treats', () => {
    const avgPrice = (style: Personal['budgetStyle']) => {
      const picks = pickedForYou(personal({ interests: ['food', 'beach', 'culture'], budgetStyle: style })).slice(0, 4);
      return picks.reduce((n, p) => n + p.d.priceLevel, 0) / picks.length;
    };
    expect(avgPrice('saver')).toBeLessThanOrEqual(avgPrice('treat'));
  });

  it('explains each pick in words (the interest and the country), without emoji', () => {
    const [first] = pickedForYou(personal({ interests: ['food'] }));
    expect(first.why).toContain('Food');
    expect(first.why).toContain(first.d.country);
    expect(first.why).not.toMatch(/\p{Extended_Pictographic}/u);
  });
});
```

src/components/home/logic.ts

```
/**
 * The small decisions Home makes, with no React in them so they can be tested: what the big countdown says,
 * whether the budget is fine, who owes whom, which activity is next, what is up for a vote, the podium,
 * which weather to show (and whether to call it a forecast or "typical").
 */
import type { Activity, HomeSection, Member, Trip, WeatherSnapshot } from '@/store/types';
import { money, countdownLabel } from '@/utils/format';
import { settlePlan } from '@/utils/money';
import type { Standing } from '@/utils/points';
import { dayLabel, daysBetween, partsInTz, time12, tripDays, weekdayShort, zonedToUtc } from '@/utils/time';

export type HeroStatus = 'planning' | 'active' | 'completed' | null;

// ---------------------------------------------------------------------------------------------
// The headline

export interface CountdownView {
  kind: 'days' | 'today' | 'day' | 'complete' | 'none';
  /** The big rolling number: days to go, or the day of the trip. */
  number?: number;
  /** Display-type words after the number (or alone): ["Days", "to go"], ["of 6"], ["Today"]. */
  words: string[];
  /** A word before the number ("Day" in DAY 2 OF 6). */
  prefix?: string;
  /** What a screen reader says. */
  sentence: string;
}

/**
 * What the big headline says. Planning: the days left ("1 DAY TO GO" in the singular). The first day of the trip:
 * "TODAY". After that: "DAY 2" with "OF 5". Finished: "TRIP COMPLETE". No trip yet: "WHERE TO NEXT?".
 */
export function countdownView(status: HeroStatus, daysToGo: number, dayNo: number, tripLen: number): CountdownView {
  if (status === null) return { kind: 'none', words: ['Where to next?'], sentence: 'Where to next?' };
  if (status === 'completed') return { kind: 'complete', words: ['Trip complete'], sentence: 'Trip complete' };
  if (status === 'planning' && daysToGo >= 1) {
    return { kind: 'days', number: daysToGo, words: [daysToGo === 1 ? 'Day' : 'Days', 'to go'], sentence: `${daysToGo} ${daysToGo === 1
? 'day' : 'days'} to go` };
  }
  if (status === 'active' && dayNo > 1) {
    return { kind: 'day', number: dayNo, prefix: 'Day', words: [`of ${tripLen}`], sentence: `Day ${dayNo} of ${tripLen}` };
  }
  return { kind: 'today', words: ['Today'], sentence: tripLen > 1 ? `Today is the first day of your ${tripLen} day trip` : 'Today is the
day' };
}

/** The key that remembers "1 day to go" was already celebrated for this trip on this day. */
export const celebrateKey = (tripId: string, day: string) => `celebrate:${tripId}:${day}`;

/** True once: on the last day before the trip, and not yet celebrated. */
export function shouldCelebrateOneDay(status: HeroStatus, daysToGo: number, fired: Record<string, number>, tripId: string, today: strin
g): boolean {
```

```
  return status === 'planning' && daysToGo === 1 && !(celebrateKey(tripId, today) in fired);
}

// ---------------------------------------------------------------------------------------------
// Which blocks have something to show

export interface BlockContext {
  hasTrip: boolean;
  status: HeroStatus;
  hasPicks: boolean;
  hasDestination: boolean;
}

/** A block with nothing to show is left out completely, so there is no empty gap on the page. */
export function blockHasContent(section: HomeSection, c: BlockContext): boolean {
  switch (section) {
    case 'upnext':
    case 'weather':
      return c.hasTrip && c.status !== 'completed';
    case 'budget':
    case 'crew':
      return c.hasTrip;
    case 'forYou':
      return c.hasPicks;
    case 'nearby':
      return c.hasTrip && c.hasDestination;
    case 'play':
    case 'discover':
      return true;
  }
}

// ---------------------------------------------------------------------------------------------
// Up next

/** The next activity that is really planned (not one still being voted on, not one already ticked off). */
export function nextPlanned(activities: Activity[], tz: string, now: number): { a: Activity; startUtc: number } | null {
  let best: { a: Activity; startUtc: number } | null = null;
  for (const a of activities) {
    if (a.status === 'voting' || a.done) continue;
    const startUtc = zonedToUtc(a.date, a.time, tz);
    if (startUtc > now - 30 * 60000 && (!best || startUtc < best.startUtc)) best = { a, startUtc };
  }
  return best;
}

/** "in 3d 13h", or "Now" once it has begun. */
export function startsIn(ms: number): string {
  if (ms <= 0) return 'Now';
  if (ms < 60000) return 'in 1m';
  return `in ${countdownLabel(ms)}`;
}

/** The "open in maps" link for an activity: exact coordinates when it has them, otherwise a search for its name. */
export function mapsUrl(a: Pick<Activity, 'lat' | 'lng' | 'title' | 'location'>): string {
  const q = a.lat && a.lng ? `${a.lat},${a.lng}` : encodeURIComponent(`${a.title} ${a.location}`);
  return `https://www.google.com/maps/search/?api=1&query=${q}`;
}

/** Today's checklist: the planned things on `today`, earliest first (things still being voted on are not part of the plan yet). */
export function todaysChecklist(activities: Activity[], today: string): Activity[] {
  return activities.filter((a) => a.date === today && a.status !== 'voting').sort((a, b) => a.time.localeCompare(b.time));
}

/** How many of today's things are ticked off. */
export function checklistProgress(items: Pick<Activity, 'done'>[]): { done: number; total: number } {
  return { done: items.filter((a) => a.done).length, total: items.length };
}

/** "7:30 PM" as the pieces a departures board prints separately: the clock and AM or PM. */
export function timeParts(hhmm: string): { clock: string; suffix: string } {
  const [clock, suffix] = time12(hhmm).split(' ');
  return { clock, suffix };
}

/** "FRI 23 OCT": the small caps line above a ticket (the kicker uppercases it, so this is plain text). */
export function dateKicker(ymd: string): string {
  return `${weekdayShort(ymd)} ${Number(ymd.slice(8))} ${dayLabel(ymd).split(' ')[0]}`;
}

/** True when the countdown is worth the accent: it starts within three hours (or has begun). */
export function startsSoon(ms: number): boolean {
  return ms < 3 * 3600000;
}

// ---------------------------------------------------------------------------------------------
```

```
// Budget

export type BudgetTone = 'ok' | 'warn' | 'over';

export interface BudgetView {
  /** 0 to well over 1. */
  ratio: number;
  pct: number;
  tone: BudgetTone;
  /** The state in words (a colour alone is never the signal). */
  word: string;
  detail: string;
  left: number;
}

/** Under 80% is fine, 80 to 100% is close, above 100% is over. */
export function budgetView(spent: number, budget: number): BudgetView {
  if (budget <= 0) return { ratio: 0, pct: 0, tone: 'ok', word: 'No limit set', detail: 'Add a group budget to track it', left: 0 };
  const ratio = spent / budget;
  const pct = Math.round(ratio * 100);
  const left = budget - spent;
  if (ratio > 1) return { ratio, pct, tone: 'over', word: 'Over budget', detail: `${money(spent - budget)} over`, left };
  if (ratio >= 0.8) return { ratio, pct, tone: 'warn', word: 'Getting close', detail: `${pct}% used · ${money(left)} left`, left };
  return { ratio, pct, tone: 'ok', word: 'On track', detail: `${pct}% used · ${money(left)} left`, left };
}

export interface OweLine {
  tone: 'owe' | 'owed' | 'square' | 'other';
  text: string;
}

const firstName = (members: Member[], id: string) => members.find((m) => m.id === id)?.name.split(' ')[0] ?? 'Someone';

/** One line about the money between friends: "You owe Priya $382", "Diego owes you $90", "Everyone is square". */
export function oweLine(trip: Pick<Trip, 'members' | 'expenses'>, userId: string): OweLine {
  if (!trip.expenses.some((e) => !e.settlement)) return { tone: 'square', text: 'Nothing to split yet' };
  const plan = settlePlan(trip);
  if (!plan.length) return { tone: 'square', text: 'Everyone is square' };
  const out = plan.filter((p) => p.from === userId);
  const into = plan.filter((p) => p.to === userId);
  const sum = (xs: typeof plan) => xs.reduce((n, p) => n + p.amount, 0);
  if (out.length === 1) return { tone: 'owe', text: `You owe ${firstName(trip.members, out[0].to)} ${money(out[0].amount)}` };
  if (out.length > 1) return { tone: 'owe', text: `You owe ${money(sum(out))} in ${out.length} payments` };
  if (into.length === 1) return { tone: 'owed', text: `${firstName(trip.members, into[0].from)} owes you ${money(into[0].amount)}` };
  if (into.length > 1) return { tone: 'owed', text: `You're owed ${money(sum(into))} by ${into.length} people` };
  return { tone: 'other', text: `${plan.length} ${plan.length === 1 ? 'payment settles' : 'payments settle'} everyone up` };
}

// ---------------------------------------------------------------------------------------
// Crew, podium, vote

export const crewCue = (m: Pick<Member, 'invited'>): 'Going' | 'Invited' => (m.invited ? 'Invited' : 'Going');

export function crewCounts(members: Pick<Member, 'invited'>[]): { going: number; invited: number } {
  const invited = members.filter((m) => m.invited).length;
  return { going: members.length - invited, invited };
}

export function ordinal(n: number): string {
  const rest = n % 100;
  if (rest >= 11 && rest <= 13) return `${n}th`;
  return `${n}${({ 1: 'st', 2: 'nd', 3: 'rd' } as Record<number, string>)[n % 10] ?? 'th'}`;
}

export interface Ranked extends Standing {
  rank: number;
}

export interface PodiumView {
  /** The top three, best first. */
  top: Ranked[];
  /** The signed-in person when they are not on the podium. */
  me: Ranked | null;
}

/** The podium needs at least two people and some points; otherwise there is nothing to show yet. */
export function podiumView(standings: Standing[], userId: string): PodiumView | null {
  if (standings.length < 2 || !standings.some((s) => s.points > 0)) return null;
  const ranked = standings.map((s, i) => ({ ...s, rank: i + 1 }));
  const mine = ranked.find((s) => s.memberId === userId);
  return { top: ranked.slice(0, 3), me: mine && mine.rank > 3 ? mine : null };
}

/** The first activity still being voted on that this person has not voted on, earliest first. */
export function nextToVote(activities: Activity[], userId: string): Activity | undefined {
  return activities
```

```
    .filter((a) => a.status === 'voting' && !(userId in a.votes))
    .sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time))[0];
}

export function votingCount(activities: Activity[]): number {
  return activities.filter((a) => a.status === 'voting').length;
}

export function voteTally(a: Pick<Activity, 'votes'>, memberCount: number) {
  const ups = Object.values(a.votes).filter((v) => v === 1).length;
  const downs = Object.values(a.votes).filter((v) => v === -1).length;
  return { ups, downs, needed: Math.floor(memberCount / 2) + 1 };
}

/** What the tally says under a vote: "1 of 3 yes votes needed to lock it in", or that it is already enough. */
export function voteLine(a: Pick<Activity, 'votes'>, memberCount: number): string {
  const { ups, needed } = voteTally(a, memberCount);
  return ups >= needed ? `${ups} yes, enough to lock it in` : `${ups} of ${needed} yes votes to lock it in`;
}

/** Would this vote lock the activity in? The store locks an item once more than half the group says yes. */
export function voteLocks(a: Pick<Activity, 'votes' | 'status'>, userId: string, memberCount: number, value: 1 | -1): boolean {
  if (a.status !== 'voting' || value !== 1 || a.votes[userId] === 1) return false;
  const others = Object.entries(a.votes).filter(([id, v]) => id !== userId && v === 1).length;
  return others + 1 > memberCount / 2;
}

/** How tall each step of the podium is, by place. */
export const PODIUM_HEIGHT: Record<1 | 2 | 3, number> = { 1: 120, 2: 92, 3: 68 };

export interface PodiumSlot extends Ranked {
  /** Height of this person's block. */
  height: number;
}

/** The podium left to right as on a real one: second, first, third (just second and first when only two people play). */
export function podiumSlots(top: Ranked[]): PodiumSlot[] {
  const slot = (r: Ranked): PodiumSlot => ({ ...r, height: PODIUM_HEIGHT[Math.min(3, r.rank) as 1 | 2 | 3] });
  const [first, second, third] = top;
  return [second, first, third].filter((r): r is Ranked => !!r).map(slot);
}

/** "3 day streak", or nothing when there is no streak. */
export function streakLabel(streak: number): string | null {
  return streak > 0 ? `${streak} day streak` : null;
}

/** How an answer button looks once today's question is answered. */
export function answerState(index: number, answer: number, picked: number | null): 'open' | 'right' | 'wrong' | 'reveal' | 'dim' {
  if (picked === null) return 'open';
  if (index === answer) return index === picked ? 'right' : 'reveal';
  return index === picked ? 'wrong' : 'dim';
}

// ---------------------------------------------------------------------------------------------
// Weather

export interface WeatherDay {
  date: string;
  max: number;
  min: number;
  /** A weather code from the forecast; typical days have none and pick an icon from the temperature. */
  code: number | null;
}

export interface WeatherView {
  kind: 'forecast' | 'typical';
  title: string;
  note: string;
  days: WeatherDay[];
}

/** How many days ahead the free forecast reaches in practice: past this the card says "Typical weather". */
export const FORECAST_HORIZON_DAYS = 10;

/**
 * A rough climate for a place and month: the yearly average falls with latitude, the seasons swing more far from the equator,
 * and the warmest month comes after the summer solstice. Close enough to say "usual temperatures" and not to call a forecast.
 */
export function typicalDay(lat: number, date: string): WeatherDay {
  const month = Number(date.slice(5, 7)) - 1;
  const mean = 28 - 0.0072 * lat * lat;
  const swing = Math.min(12, Math.abs(lat) * 0.22);
  const peak = lat >= 0 ? 6.8 : 0.8;
  const t = mean + swing * Math.cos(((month + 0.5 - peak) / 12) * 2 * Math.PI);
  return { date, max: Math.round(t + 4), min: Math.round(t - 4), code: null };
```

```
}

export function monthName(date: string): string {
  return ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'][Numb
er(date.slice(5, 7)) - 1] ?? '';
}

/**
 * What the weather card shows. A real forecast only for trip days the forecast really covers and only when it is live;
 * a trip further away than {@link FORECAST_HORIZON_DAYS}, or no connection, shows typical temperatures for the month, labelled as such.
 */
export function weatherView(input: { startDate: string; endDate: string; today: string; lat: number; snapshot: WeatherSnapshot | undefin
ed }): WeatherView {
  const { startDate, endDate, today, lat, snapshot } = input;
  const daysToGo = daysBetween(today, startDate);
  const days = tripDays(startDate, endDate).filter((d) => d >= today);
  if (daysToGo <= FORECAST_HORIZON_DAYS && snapshot?.live) {
    const covered = snapshot.daily.filter((d) => days.includes(d.date)).map((d) => ({ date: d.date, max: d.max, min: d.min, code: d.code
}));
    if (covered.length) return { kind: 'forecast', title: 'Forecast', note: 'Live forecast for your trip days', days: covered.slice(0,
7) };
  }
  const first = days[0] ?? startDate;
  const note =
    daysToGo > FORECAST_HORIZON_DAYS
      ? `Usual temperatures for ${monthName(first)}. The real forecast appears ${FORECAST_HORIZON_DAYS} days before you go.`
      : `Usual temperatures for ${monthName(first)}. The live forecast is not available right now.`;
  return { kind: 'typical', title: 'Typical weather', note, days: days.slice(0, 7).map((d) => typicalDay(lat, d)) };
}

/** An icon for a typical day (a real forecast has a weather code and uses that). */
export function typicalIcon(max: number): 'sunny' | 'partly-sunny' | 'cloud-outline' {
  if (max >= 28) return 'sunny';
  if (max >= 12) return 'partly-sunny';
  return 'cloud-outline';
}

/** The two clocks on the weather board: "3:41 PM" there and "9:41 AM" at home, and whether they are the same. */
export function clockPair(now: number, tz: string, homeTz: string): { there: string; home: string; same: boolean } {
  const there = time12(partsInTz(now, tz).hhmm);
  const home = time12(partsInTz(now, homeTz).hhmm);
  return { there, home, same: there === home };
}
```

src/components/home/picks.ts

```
import { destinations, type Destination } from '@/data/destinations';
import { INTERESTS } from '@/data/personal';
import type { Personal } from '@/store/types';

/** Ranks destinations against the “Make it yours” answers (interests + budget style). */
export function pickedForYou(p: Personal): { d: Destination; match: number; why: string }[] {
  if (!p.interests.length) return [];
  return destinations
    .map((d) => {
      const hits = INTERESTS.filter((i) => p.interests.includes(i.key) && d.tags.includes(i.key));
      const budgetFit = p.budgetStyle === 'saver' ? (d.priceLevel <= 2 ? 1 : 0) : p.budgetStyle === 'treat' ? (d.priceLevel >= 3 ? 1 :
0) : d.priceLevel <= 3 ? 1 : 0;
      const ratio = hits.length / Math.min(3, p.interests.length);
      const match = Math.min(99, Math.round(52 + 36 * Math.min(1, ratio) + 7 * budgetFit + (d.rating - 4.5) * 8));
      return { d, match, hits, why: `${hits.map((h) => h.label).join(', ')} · ${d.country}` };
    })
    .filter((x) => x.hits.length > 0)
    .sort((a, b) => b.match - a.match || b.d.rating - a.d.rating)
    .slice(0, 8);
}
```

src/components/home/useHomeContext.ts

```
import { destinationById, type Destination } from '@/data/destinations';
import { useNow } from '@/hooks/useTrip';
import type { Personal, Trip } from '@/store/types';
import { tripStatus, useActiveTrip, useAppStore, useCurrentUser } from '@/store/useAppStore';
import { daysBetween, partsInTz } from '@/utils/time';

import type { HeroStatus } from './logic';

/** What every Home block needs to know about the trip and the moment, worked out once. */
export interface HomeCtx {
  userId: string;
  userName: string;
  personal: Personal;
  trip: Trip | undefined;
  dest: Destination | undefined;
  /** The destination's time zone (everything about "today" on a trip is told in the place's own time). */
  tz: string;
```

```
  status: HeroStatus;
  now: number;
  /** Today's date at the destination. */
  today: string;
  tripLen: number;
  /** Days until the trip starts (0 or less once it has begun). */
  daysToGo: number;
  /** Which day of the trip it is (1 until it starts, then 1..tripLen). */
  dayNo: number;
}

/** A context for a person who has a trip (the blocks that need a trip take this). */
export type TripCtx = HomeCtx & { trip: Trip };

export const hasTrip = (c: HomeCtx): c is TripCtx => !!c.trip;

export function useHomeContext(): HomeCtx {
  const user = useCurrentUser();
  const trip = useActiveTrip();
  const personal = useAppStore((s) => s.personal);
  const now = useNow(30000);
  const dest = destinationById(trip?.destinationId ?? 'positano');
  const tz = dest?.tz ?? 'UTC';
  const today = partsInTz(now, tz).ymd;
  const tripLen = trip ? daysBetween(trip.startDate, trip.endDate) + 1 : 0;
  return {
    userId: user?.id ?? '',
    userName: user?.name ?? '',
    personal,
    trip,
    dest,
    tz,
    status: trip ? tripStatus(trip) : null,
    now,
    today,
    tripLen,
    daysToGo: trip ? Math.max(0, daysBetween(today, trip.startDate)) : 0,
    dayNo: trip ? Math.min(tripLen, Math.max(1, daysBetween(trip.startDate, today) + 1)) : 0,
  };
}
```

src/components/join/QrScanner.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { CameraView, useCameraPermissions } from 'expo-camera';
import { useState } from 'react';
import { Modal, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { Button } from '@/components/ui/Button';
import { PlatePress } from '@/components/ui/Plate';
import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';

/**
 * Full-screen camera that reads one QR code and hands its text back. The controls over the picture are outlined paper squares
 * (like the ones over the photo on Home); until the camera is allowed the screen is paper with a plain explanation.
 */
export function QrScanner({ visible, onClose, onScan }: { visible: boolean; onClose: () => void; onScan: (text: string) => void }) {
  const insets = useSafeAreaInsets();
  const [permission, requestPermission] = useCameraPermissions();
  const [done, setDone] = useState(false); // the camera reports the same code many times a second
  const blocked = !!permission && !permission.canAskAgain;

  return (
    <Modal visible={visible} animationType="slide" onRequestClose={onClose} onShow={() => setDone(false)}>
      <View style={styles.root}>
        {permission?.granted ? (
          <CameraView
            style={StyleSheet.absoluteFill}
            facing="back"
            barcodeScannerSettings={{ barcodeTypes: ['qr'] }}
            onBarcodeScanned={({ data }) => {
              if (done) return;
              setDone(true);
              onScan(data);
            }}
          />
        ) : (
          <View style={styles.ask}>
            <View style={styles.askIcon}>
              <Ionicons name="camera-outline" size={34} color={colors.ink} />
            </View>
            <T variant="headline" center>
              {blocked ? 'Camera access is off' : 'Allow the camera to scan'}
            </T>
```

```
            <T variant="small" color={colors.textSecondary} center>
              {blocked ? 'Turn it on for TogetherWeGo in your phone’s settings, or type the invite code instead.' : 'It is only used to
read the invite QR code your friend shows you.'}
            </T>
            {!blocked ? (
              <View style={{ alignSelf: 'stretch', marginTop: 8 }}>
                <Button label="Allow camera" icon="camera-outline" onPress={() => requestPermission()} />
              </View>
            ) : null}
          </View>
        )}
        <View style={[styles.top, { paddingTop: insets.top + 8 }]}>
          <PlatePress onPress={onClose} accessibilityLabel="Close scanner" offset={3} corner={radius.button} style={styles.close}>
            <Ionicons name="close" size={22} color={colors.ink} />
          </PlatePress>
        </View>
        {permission?.granted ? (
          <>
            <View style={styles.middle} pointerEvents="none">
              <View style={styles.frame} />
            </View>
            <View style={[styles.hint, { paddingBottom: insets.bottom + 24 }]} pointerEvents="none">
              <View style={styles.label}>
                <T variant="small" weight="bold" center>
                  Point the camera at your friend’s invite QR code
                </T>
              </View>
            </View>
          </>
        ) : null}
      </View>
    </Modal>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg },
  ask: { flex: 1, alignItems: 'center', justifyContent: 'center', gap: 12, padding: 32 },
  askIcon: { width: 84, height: 84, borderRadius: 42, borderWidth: line, borderStyle: 'dashed', borderColor: colors.ink, backgroundColo
r: colors.accentSoft, alignItems: 'center', justifyContent: 'center', marginBottom: 4 },
  top: { position: 'absolute', top: 0, left: 0, right: 0, paddingHorizontal: 16 },
  close: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.card, borderWidth: line, borde
rColor: colors.ink, borderRadius: radius.button, alignSelf: 'flex-start' },
  middle: { position: 'absolute', top: 0, bottom: 0, left: 0, right: 0, alignItems: 'center', justifyContent: 'center' },
  hint: { position: 'absolute', bottom: 0, left: 0, right: 0, alignItems: 'center', paddingHorizontal: 24 },
  // the frame over the camera picture is white: a white overlay on a photo is the one place a raw colour is fine
  frame: { width: 240, height: 240, borderRadius: radius.card, borderWidth: 4, borderColor: colors.white },
  label: { paddingHorizontal: 14, paddingVertical: 10, backgroundColor: colors.card, borderWidth: line, borderColor: colors.ink, borderR
adius: radius.card },
});
```

src/components/map/OfflineRadar.tsx

```
/**
 * Offline map: plots every stop around the trip centre using real distances
 * and bearings — no map tiles or internet needed. Handy on a plane or when
 * roaming data is off.
 */
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { colors, hair } from '@/theme';
import { projectKm } from '@/utils/geo';

import { PinBadge } from './PinBadge';
import type { MapPoint } from './types';

/** Every marker sits in a 44 point square, so it is an easy target even when it is drawn small. */
const HIT = 44;

export function OfflineRadar({ center, points, radiusKm, selectedId, onSelect }: { center: { lat: number; lng: number }; points: MapPoin
t[]; radiusKm: number; selectedId?: string | null; onSelect: (id: string) => void }) {
  const [size, setSize] = useState(300);
  const half = size / 2;
  const scale = half / radiusKm;
  const rings = [0.25, 0.5, 1].map((f) => Math.max(0.5, Math.round(radiusKm * f * 10) / 10));
  // the picked marker is drawn last, so it is never hidden under a neighbour
  const ordered = [...points].sort((a, b) => Number(a.id === selectedId) - Number(b.id === selectedId));

  return (
    <View style={styles.wrap} onLayout={(e) => setSize(Math.min(e.nativeEvent.layout.width, e.nativeEvent.layout.height))}>
      <View style={{ width: size, height: size }}>
        {rings.map((r) => (
          <View key={r} style={[styles.ring, { width: r * scale * 2, height: r * scale * 2, borderRadius: r * scale, left: half - r * sc
```

```
ale, top: half - r * scale }]}>
            <T variant="kicker" color={colors.textSecondary} style={{ position: 'absolute', top: -8, alignSelf: 'center', backgroundColo
r: colors.bgAlt, paddingHorizontal: 4, fontSize: 10 }}>
              {r} km
            </T>
          </View>
        ))}
        <View style={[styles.axis, { left: half, top: 0, width: hair, height: size }]} />
        <View style={[styles.axis, { top: half, left: 0, height: hair, width: size }]} />
        <T variant="kicker" color={colors.ink} style={{ position: 'absolute', top: 22, left: half + 6 }}>
          N
        </T>
        {ordered.map((p) => {
          const { x, y } = projectKm(center, p);
          if (Math.hypot(x, y) > radiusKm * 1.02) return null;
          const sel = p.id === selectedId;
          return (
            <Press key={p.id} onPress={() => onSelect(p.id)} accessibilityLabel={`${p.title}, ${p.subtitle}`} accessibilityState={{ sele
cted: sel }} scaleTo={0.9} style={[styles.hit, { left: half + x * scale - HIT / 2, top: half - y * scale - HIT / 2 }]}>
              <PinBadge p={p} selected={sel} />
            </Press>
          );
        })}
        <View pointerEvents="none" style={[styles.center, { left: half - 8, top: half - 8 }]} />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  wrap: { ...StyleSheet.absoluteFill, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  ring: { position: 'absolute', borderWidth: 1.5, borderColor: colors.textMuted, borderStyle: 'dashed' },
  axis: { position: 'absolute', backgroundColor: colors.borderStrong },
  hit: { position: 'absolute', width: HIT, height: HIT, alignItems: 'center', justifyContent: 'center' },
  // the middle of the search: a small ink cross-hair square
  center: { position: 'absolute', width: 16, height: 16, borderWidth: 2, borderColor: colors.ink, backgroundColor: colors.bgAlt, transfo
rm: [{ rotate: '45deg' }] },
});
```

src/components/map/PinBadge.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';

import type { MapPoint } from './types';

/** How big the drawn marker is: a stop is a numbered square, an airport a square with a plane, a place a small dot. */
export function pinSize(kind: MapPoint['kind'], selected: boolean) {
  const base = kind === 'place' ? 14 : kind === 'me' ? 18 : 26;
  return selected ? base + 8 : base;
}

/**
 * The look of one marker, the same on the offline radar and on the phone's own map. The cue is the shape, not the colour:
 * a numbered ink square is a stop in your plan, a square with a plane is an airport, a small outlined dot is a place worth a look,
 * and the accent dot is you. The one you picked is bigger and filled with the accent.
 */
export function PinBadge({ p, selected }: { p: MapPoint; selected: boolean }) {
  const size = pinSize(p.kind, selected);
  const fill = selected ? colors.accent : p.kind === 'activity' ? colors.primary : p.kind === 'me' ? colors.accent : colors.card;
  const ink = selected ? colors.onAccent : p.kind === 'activity' ? colors.onPrimary : colors.ink;
  const round = p.kind === 'place' || p.kind === 'me';
  return (
    <View style={[styles.base, { width: size, height: size, borderRadius: round ? size / 2 : radius.xs, backgroundColor: fill, borderCol
or: colors.ink, borderWidth: p.kind === 'me' ? 3 : line }]}>
      {p.kind === 'activity' && p.label ? (
        <T variant="kicker" color={ink} allowFontScaling={false} style={{ fontSize: selected ? 14 : 12, lineHeight: selected ? 18 : 16,
letterSpacing: 0 }}>
          {p.label}
        </T>
      ) : p.kind === 'airport' ? (
        <Ionicons name="airplane" size={selected ? 18 : 14} color={ink} />
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  base: { alignItems: 'center', justifyContent: 'center' },
});
```

src/components/map/TripMap.tsx

```
/** Native map (Apple Maps on iOS, Google Maps on Android) via react-native-maps. */
import { useEffect, useRef, useState } from 'react';
import { StyleSheet } from 'react-native';
import MapView, { Circle, Marker } from 'react-native-maps';

import { colors } from '@/theme';

import { PinBadge } from './PinBadge';
import type { TripMapProps } from './types';

export function TripMap({ center, points, radiusKm, selectedId, onSelect, showsUser }: TripMapProps) {
  const ref = useRef<MapView>(null);
  const delta = Math.max(0.04, (radiusKm / 111) * 2.6);

  // A custom marker is drawn once into a picture, so it must be allowed to redraw while it is new or has just been picked.
  const sig = `${points.map((p) => p.id).join(',')}|${selectedId ?? ''}`;
  const [settled, setSettled] = useState('');
  useEffect(() => {
    const id = setTimeout(() => setSettled(sig), 700);
    return () => clearTimeout(id);
  }, [sig]);

  useEffect(() => {
    ref.current?.animateToRegion({ latitude: center.lat, longitude: center.lng, latitudeDelta: delta, longitudeDelta: delta }, 400);
  }, [center.lat, center.lng, delta]);

  useEffect(() => {
    const p = points.find((x) => x.id === selectedId);
    if (p) ref.current?.animateToRegion({ latitude: p.lat, longitude: p.lng, latitudeDelta: 0.03, longitudeDelta: 0.03 }, 350);
  }, [selectedId, points]);

  return (
    <MapView
      ref={ref}
      style={StyleSheet.absoluteFill}
      initialRegion={{ latitude: center.lat, longitude: center.lng, latitudeDelta: delta, longitudeDelta: delta }}
      showsUserLocation={showsUser}
      showsCompass
      toolbarEnabled={false}>
      <Circle center={{ latitude: center.lat, longitude: center.lng }} radius={radiusKm * 1000} strokeColor={`${colors.ink}99`} fillColo
r={`${colors.accent}22`} strokeWidth={2} />
      {points.map((p) => (
        <Marker
          key={p.id}
          coordinate={{ latitude: p.lat, longitude: p.lng }}
          title={p.title}
          description={p.subtitle}
          anchor={{ x: 0.5, y: 0.5 }}
          onPress={() => onSelect(p.id)}
          zIndex={p.id === selectedId ? 2 : p.kind === 'activity' ? 1 : 0}
          tracksViewChanges={settled !== sig}>
          <PinBadge p={p} selected={p.id === selectedId} />
        </Marker>
      ))}
    </MapView>
  );
}
```

src/components/map/TripMap.web.tsx

```
/** Web preview: OpenStreetMap embed (react-native-maps is native-only). */
import { createElement } from 'react';
import { StyleSheet, View } from 'react-native';

import type { TripMapProps } from './types';

export function TripMap({ center, points, radiusKm, selectedId }: TripMapProps) {
  const focus = points.find((p) => p.id === selectedId) ?? center;
  const d = Math.max(0.02, (radiusKm / 111) * 1.3);
  const bbox = [focus.lng - d, focus.lat - d * 0.7, focus.lng + d, focus.lat + d * 0.7].join(',');
  const src = `https://www.openstreetmap.org/export/embed.html?bbox=${bbox}&layer=mapnik&marker=${focus.lat},${focus.lng}`;
  return (
    <View style={StyleSheet.absoluteFill}>
      {createElement('iframe', { src, title: 'Trip map', style: { border: 0, width: '100%', height: '100%' }, loading: 'lazy' })}
    </View>
  );
}
```

src/components/map/types.ts

```
import type { IconName } from '@/components/ui/bits';

export interface MapPoint {
```

```
  id: string;
  lat: number;
  lng: number;
  title: string;
  subtitle: string;
  /** The marker's colour for the phone's own map pins (ink for the plan, quieter for places, blue for airports). */
  color: string;
  kind: 'activity' | 'place' | 'airport' | 'me';
  /** The stop number of a planned activity, so the plan reads 1, 2, 3 on the map. */
  label?: string;
  /** A line icon for the kind (airports). */
  icon?: IconName;
}

export interface TripMapProps {
  center: { lat: number; lng: number };
  points: MapPoint[];
  radiusKm: number;
  selectedId?: string | null;
  onSelect: (id: string) => void;
  showsUser?: boolean;
}
```

src/components/more/AccountTicket.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { StyleSheet, View, useWindowDimensions } from 'react-native';

import { Avatar } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import type { Account } from '@/store/types';
import { colors } from '@/theme';

/**
 * The account line at the top of More, drawn as a boarding pass: PASSENGER, the name, FROM home TO the trip's city, and a stub
 * under the tear line that opens My trips (switch, create or join). Pressing the main part opens the profile and settings.
 */
export function AccountTicket({ user, destCity, homeCity, tripCount }: { user: Account | undefined; destCity?: string; homeCity?: strin
g; tripCount: number }) {
  // on a very narrow phone the name needs the room more than the arrow does
  const roomy = useWindowDimensions().width >= 350;
  const route = [homeCity, destCity].filter(Boolean).join('  →  ');
  return (
    <Ticket
      stub={
        <Press onPress={() => router.push('/trips')} style={styles.stub} scaleTo={0.99} accessibilityLabel="My trips" accessibilityHint
="Switch, create or join a trip">
          <Ionicons name="swap-horizontal" size={20} color={colors.ink} />
          <T variant="title" weight="bold" style={{ flex: 1 }}>
            My trips
          </T>
          <T variant="numeralSm">{tripCount}</T>
          <Ionicons name="arrow-forward" size={16} color={colors.ink} />
        </Press>
      }>
      <Press onPress={() => router.push('/settings')} style={styles.main} scaleTo={0.99} accessibilityLabel="Open your profile and setti
ngs">
        <Avatar name={user?.name ?? 'You'} src={user?.avatar} size={56} />
        <View style={{ flex: 1, gap: 2 }}>
          <T variant="kicker" color={colors.textSecondary}>
            Passenger
          </T>
          <T variant="headline" numberOfLines={1}>
            {user?.name ?? 'You'}
          </T>
          {route ? (
            <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
              {route}
            </T>
          ) : null}
        </View>
        {roomy ? <Ionicons name="arrow-forward" size={18} color={colors.ink} /> : null}
      </Press>
    </Ticket>
  );
}

const styles = StyleSheet.create({
  main: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 16 },
  stub: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingHorizontal: 16 },
});
```

src/components/more/NearbyFilters.tsx

```
import { StyleSheet, View } from 'react-native';

import { placeCategoryIcon } from '@/components/trip/meta';
import { Button } from '@/components/ui/Button';
import { Chip } from '@/components/ui/bits';
import { Sheet } from '@/components/ui/Sheet';
import { Slider } from '@/components/ui/Slider';
import { T } from '@/components/ui/T';
import type { PlaceCategory } from '@/data/places';
import { useT } from '@/i18n';
import { colors } from '@/theme';
import { LENGTH_FILTERS, money, type LengthFilter } from '@/utils/format';

export const CATS: ('all' | PlaceCategory)[] = ['all', 'food', 'sightseeing', 'adventure', 'culture', 'nightlife', 'shopping', 'relax'];

export interface NearbyFilterState {
  radiusKm: number;
  cat: 'all' | PlaceCategory;
  len: LengthFilter;
  maxCost: number;
  /** True when the search starts from the phone's own location rather than the middle of the city. */
  fromMe: boolean;
}

/** A small caps label on the left and its current value in mono on the right, over a control. */
function Row({ label, value, children }: { label: string; value?: string; children: React.ReactNode }) {
  return (
    <View style={{ gap: 6 }}>
      <View style={styles.between}>
        <T variant="kicker" color={colors.textSecondary} accessibilityRole="header">
          {label}
        </T>
        {value ? <T variant="numeralSm">{value}</T> : null}
      </View>
      {children}
    </View>
  );
}

/**
 * Everything that narrows Nearby, behind the one Filter button: radius, where to start from, the most you will pay, the kind of place a
nd
 * how long a visit takes. The button at the bottom says how many places are left, so it is also the way to close the sheet.
 */
export function NearbyFilters({ visible, onClose, state, onChange, cityName, locating, onLocate, onCentre, onReset, count }: { visible:
boolean; onClose: () => void; state: NearbyFilterState; onChange: (patch: Partial<NearbyFilterState>) => void; cityName: string; locatin
g: boolean; onLocate: () => void; onCentre: () => void; onReset: () => void; count: number }) {
  const t = useT();
  return (
    <Sheet
      visible={visible}
      onClose={onClose}
      title="Filters"
      footer={
        <View style={{ gap: 8 }}>
          <Button label={`Show ${count} ${count === 1 ? 'place' : 'places'}`} onPress={onClose} />
          <Button label="Reset filters" variant="ghost" size="md" onPress={onReset} />
        </View>
      }>
      <Row label="Search radius" value={`${state.radiusKm} km`}>
        <Slider value={state.radiusKm} min={1} max={25} onChange={(v) => onChange({ radiusKm: v })} label="Search radius" formatValue=
{(v) => `${v} km`} />
      </Row>
      <Row label="Start from">
        <View style={styles.wrap}>
          <Chip icon="business-outline" label={`${cityName} centre`} active={!state.fromMe} onPress={onCentre} />
          <Chip icon="locate-outline" label={locating ? 'Locating…' : 'My location'} active={state.fromMe} onPress={onLocate} />
        </View>
      </Row>
      <Row label="Max price per person" value={state.maxCost >= 100 ? 'Any' : state.maxCost === 0 ? 'Free only' : money(state.maxCost)}>
        <Slider value={state.maxCost} min={0} max={100} step={5} onChange={(v) => onChange({ maxCost: v })} label="Maximum price per per
son" formatValue={(v) => (v >= 100 ? 'Any' : `$${v}`)} />
      </Row>
      <Row label="Kind of place">
        <View style={styles.wrap}>
          {CATS.map((c) => (
            <Chip key={c} icon={c === 'all' ? undefined : placeCategoryIcon[c]} label={t(`cat.${c}`)} active={state.cat === c} onPress=
{() => onChange({ cat: c })} />
          ))}
        </View>
      </Row>
      <Row label="Visit length">
        <View style={styles.wrap}>
          {LENGTH_FILTERS.map((f) => (
```

```
            <Chip key={f.key} label={t(f.labelKey)} active={state.len === f.key} onPress={() => onChange({ len: f.key })} />
          ))}
        </View>
      </Row>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  between: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
});
```

src/components/more/Tag.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import type { IconName } from '@/components/ui/bits';
import { PlatePress } from '@/components/ui/Plate';
import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';

/**
 * One luggage tag on the More page: a paper square with an ink outline, a punched eyelet in the corner, a line icon and a short
 * label in mono caps. Pressing slides it onto its plate. `name` is what a screen reader says (the label can be shorter).
 */
export function Tag({ icon, label, name, hint, onPress }: { icon: IconName; label: string; name?: string; hint?: string; onPress: () =>
void }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={name ?? label} accessibilityHint={hint} corner={radius.card} wrapperStyle={styles.
wrap} style={styles.face}>
      <View pointerEvents="none" style={styles.eyelet}>
        <View style={styles.hole} />
      </View>
      <Ionicons name={icon} size={30} color={colors.ink} />
      <T variant="kicker" style={styles.label}>
        {label}
      </T>
    </PlatePress>
  );
}

/** The one wide tag, for the thing you hope not to need: Emergency. Its icon square is red, because red means a state. */
export function WideTag({ icon, label, sub, name, hint, onPress }: { icon: IconName; label: string; sub: string; name?: string; hint?: s
tring; onPress: () => void }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={name ?? label} accessibilityHint={hint} corner={radius.card} wrapperStyle={{ align
Self: 'stretch' }} style={styles.wide}>
      <View style={styles.alert}>
        <Ionicons name={icon} size={24} color={colors.red} />
      </View>
      <View style={{ flex: 1 }}>
        <T variant="title" weight="bold">
          {label}
        </T>
        <T variant="caption" color={colors.textSecondary}>
          {sub}
        </T>
      </View>
      <Ionicons name="arrow-forward" size={16} color={colors.ink} />
    </PlatePress>
  );
}

const styles = StyleSheet.create({
  wrap: { flex: 1 },
  wide: { flexDirection: 'row', alignItems: 'center', gap: 14, minHeight: 68, paddingHorizontal: 12, paddingVertical: 12, borderRadius:
radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
  alert: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.red, backgroundColor: colors.redSo
ft, alignItems: 'center', justifyContent: 'center' },
  face: { flex: 1, minHeight: 104, justifyContent: 'space-between', padding: 12, borderRadius: radius.card, borderWidth: line, borderCol
or: colors.ink, backgroundColor: colors.card },
  eyelet: { position: 'absolute', top: 10, right: 10, width: 16, height: 16, borderRadius: 8, backgroundColor: colors.ink, alignItems:
'center', justifyContent: 'center' },
  hole: { width: 8, height: 8, borderRadius: 4, backgroundColor: colors.bg },
  label: { fontSize: 10.5, lineHeight: 14, letterSpacing: 0.6 },
});
```

src/components/more/WeatherBoard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { T } from '@/components/ui/T';
import { cToF, weatherInfo } from '@/services/weather';
```

```
import type { WeatherSnapshot } from '@/store/types';
import { colors, isDark, line, radius } from '@/theme';
import { dayLabel, weekdayShort } from '@/utils/time';

/** The board is black with paper-coloured print in the light mode; at night it is the dark card with cream print. */
const BOARD = isDark ? { bg: colors.card, fg: colors.ink } : { bg: colors.ink, fg: colors.bg };
/** Today's row is lit in the accent (in the Ink theme the accent is the board's own colour, so it is lit in paper instead). */
const LIT = colors.accent === BOARD.bg ? { bg: BOARD.fg, fg: BOARD.bg } : { bg: colors.accent, fg: colors.onAccent };

/**
 * The week as a departures board: one row per day with the weekday and date, the sky, the chance of rain, the low, a bar from low to
 * high and the high, all in mono print. Today is the row that is lit.
 */
export function WeatherBoard({ daily, city, unitF }: { daily: WeatherSnapshot['daily']; city: string; unitF: boolean }) {
  const maxT = Math.max(...daily.map((x) => x.max));
  const minT = Math.min(...daily.map((x) => x.min));
  const span = Math.max(1, maxT - minT);
  const deg = (c: number) => `${unitF ? cToF(c) : c}°`;
  return (
    <View style={styles.board}>
      <View style={styles.head}>
        <View style={styles.bullet} />
        <T variant="kicker" color={BOARD.fg} numberOfLines={1} style={{ flex: 1 }}>
          {daily.length} days · {city}
        </T>
      </View>
      <View style={[styles.row, { paddingVertical: 0, minHeight: 22 }]} accessibilityElementsHidden importantForAccessibility="no">
        <T variant="kicker" color={BOARD.fg} style={[styles.day, styles.small]}>
          Day
        </T>
        <View style={styles.sky} />
        <T variant="kicker" color={BOARD.fg} style={[styles.rain, styles.small]}>
          Rain
        </T>
        <T variant="kicker" color={BOARD.fg} style={[styles.temp, styles.small]}>
          Low
        </T>
        <View style={{ flex: 1 }} />
        <T variant="kicker" color={BOARD.fg} style={[styles.temp, styles.small]}>
          High
        </T>
      </View>
      {daily.map((x, i) => {
        const today = i === 0;
        const fg = today ? LIT.fg : BOARD.fg;
        return (
          <View key={x.date} style={[styles.row, today && { backgroundColor: LIT.bg }]} accessible accessibilityLabel={`${today ? 'Toda
y' : weekdayShort(x.date)} ${dayLabel(x.date)}: ${weatherInfo(x.code).label}, ${x.rain} percent chance of rain, low ${deg(x.min)}, high
${deg(x.max)}`}>
            <View style={styles.day}>
              <T variant="kicker" color={fg}>
                {today ? 'Today' : weekdayShort(x.date)}
              </T>
              <T variant="kicker" color={fg} style={styles.dim}>
                {dayLabel(x.date)}
              </T>
            </View>
            <View style={styles.sky}>
              <Ionicons name={weatherInfo(x.code).icon} size={22} color={fg} />
            </View>
            <T variant="kicker" color={fg} style={styles.rain}>
              {x.rain}%
            </T>
            <T variant="numeralSm" color={fg} style={[styles.temp, styles.num]}>
              {deg(x.min)}
            </T>
            <View style={[styles.track, { backgroundColor: fg + '40' }]}>
              <View style={{ position: 'absolute', top: 0, bottom: 0, left: `${((x.min - minT) / span) * 100}%`, right: `${((maxT - x.ma
x) / span) * 100}%`, backgroundColor: fg }} />
            </View>
            <T variant="numeralSm" color={fg} style={[styles.temp, styles.num]}>
              {deg(x.max)}
            </T>
          </View>
        );
      })}
    </View>
  );
}

const styles = StyleSheet.create({
  board: { backgroundColor: BOARD.bg, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, padding: 8, paddingTop: 14
},
  head: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingHorizontal: 8, paddingBottom: 10 },
  bullet: { width: 10, height: 10, backgroundColor: LIT.bg },
  row: { flexDirection: 'row', alignItems: 'center', gap: 6, paddingHorizontal: 6, paddingVertical: 6, minHeight: 52, borderRadius: radi
```

```
us.chip },
  day: { width: 50 },
  sky: { width: 24, alignItems: 'center' },
  rain: { width: 34, textAlign: 'right' },
  temp: { width: 36, textAlign: 'right' },
  num: { fontSize: 16, lineHeight: 22 },
  small: { fontSize: 9.5, opacity: 0.7 },
  dim: { fontSize: 10, opacity: 0.7 },
  track: { flex: 1, height: 8 },
});
```

src/components/more/__tests__/groups.test.ts

```
import { groupByDay } from '../groups';

const at = (y: number, m: number, d: number, h: number, min = 0) => new Date(y, m - 1, d, h, min).getTime();
const now = at(2026, 10, 27, 15);

describe('groupByDay', () => {
  it('puts what happened since midnight under Today and the rest under Earlier, newest first', () => {
    const list = [
      { id: 'old', ts: at(2026, 10, 25, 9) },
      { id: 'late', ts: at(2026, 10, 27, 14, 30) },
      { id: 'early', ts: at(2026, 10, 27, 0, 5) },
      { id: 'yesterday', ts: at(2026, 10, 26, 23, 59) },
    ];
    const groups = groupByDay(list, now);
    expect(groups.map((g) => g.label)).toEqual(['Today', 'Earlier']);
    expect(groups[0].items.map((n) => n.id)).toEqual(['late', 'early']);
    expect(groups[1].items.map((n) => n.id)).toEqual(['yesterday', 'old']);
  });

  it('leaves out a group with nothing in it', () => {
    expect(groupByDay([{ ts: at(2026, 10, 27, 9) }], now).map((g) => g.label)).toEqual(['Today']);
    expect(groupByDay([{ ts: at(2026, 10, 1, 9) }], now).map((g) => g.label)).toEqual(['Earlier']);
    expect(groupByDay([], now)).toEqual([]);
  });

  it('does not change the list it is given', () => {
    const list = [{ ts: 1 }, { ts: 3 }, { ts: 2 }];
    groupByDay(list, 10);
    expect(list.map((n) => n.ts)).toEqual([1, 3, 2]);
  });
});
```

src/components/more/groups.ts

```
/** Splits notifications into "Today" and "Earlier" (by the phone's own calendar day), newest first. Empty groups are left out. */
export interface DayGroup<T> {
  label: 'Today' | 'Earlier';
  items: T[];
}

export function groupByDay<T extends { ts: number }>(items: T[], now = Date.now()): DayGroup<T>[] {
  const startOfToday = new Date(now);
  startOfToday.setHours(0, 0, 0, 0);
  const newestFirst = [...items].sort((a, b) => b.ts - a.ts);
  const today = newestFirst.filter((n) => n.ts >= startOfToday.getTime());
  const earlier = newestFirst.filter((n) => n.ts < startOfToday.getTime());
  const out: DayGroup<T>[] = [];
  if (today.length) out.push({ label: 'Today', items: today });
  if (earlier.length) out.push({ label: 'Earlier', items: earlier });
  return out;
}
```

src/components/panorama/Chrome.tsx

```
/**
 * The floating controls of the 360° screen, drawn in the paper-and-ink look: outlined squares on the scene, each at least 44 points.
 * These are only the controls. The scene and everything that moves live in Scene.tsx and panorama.tsx.
 */
import { Ionicons } from '@expo/vector-icons';
import type { ComponentProps, ReactNode } from 'react';
import { StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { PlatePress } from '@/components/ui/Plate';
import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';

type IconName = ComponentProps<typeof Ionicons>['name'];

/** An outlined square button with an icon only (back, look left, look right). */
export function Square({ icon, label, onPress, style }: { icon: IconName; label: string; onPress: () => void; style?: StyleProp<ViewStyl
e> }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} wrapperStyle={style} style={styles.face}>
```

```
      <Ionicons name={icon} size={20} color={colors.ink} />
    </PlatePress>
  );
}

/** An outlined button with an icon and a word. `on` fills it with the accent, like a chosen chip. */
export function Switch({ icon, text, label, hint, role = 'button', on, selected, onPress, onLongPress }: { icon: IconName; text: string;
label: string; hint?: string; role?: 'button' | 'switch'; on?: boolean; selected?: boolean; onPress: () => void; onLongPress?: () => voi
d }) {
  const fg = on ? colors.onAccent : colors.ink;
  return (
    <PlatePress
      onPress={onPress}
      onLongPress={onLongPress}
      accessibilityRole={role}
      accessibilityLabel={label}
      accessibilityHint={hint}
      accessibilityState={role === 'switch' ? { checked: !!on } : selected !== undefined ? { selected } : undefined}
      offset={3}
      corner={radius.button}
      style={[styles.face, styles.wide, on && { backgroundColor: colors.accent }]}>
      <Ionicons name={icon} size={16} color={fg} />
      <T variant="small" weight="bold" color={fg}>
        {text}
      </T>
    </PlatePress>
  );
}

/** One choice in a row of choices (a landmark to look at, a city to fly to): a rectangle that fills with the accent when it is the one.
*/
export function Pick({ text, icon, label, on, onPress }: { text: string; icon?: IconName; label: string; on: boolean; onPress: () => voi
d }) {
  const fg = on ? colors.onAccent : colors.ink;
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} accessibilityState={{ selected: on }} offset={3} corner={radius.chip} style
={[styles.pick, on && { backgroundColor: colors.accent }]}>
      {icon ? <Ionicons name={icon} size={15} color={fg} /> : null}
      <T variant="small" weight="bold" color={fg} numberOfLines={1}>
        {text}
      </T>
    </PlatePress>
  );
}

/** A small outlined readout that is not a button (the compass). */
export function Readout({ icon, children }: { icon: IconName; children: ReactNode }) {
  return (
    <View style={styles.readout}>
      <Ionicons name={icon} size={14} color={colors.ink} />
      <T variant="kicker" color={colors.ink}>
        {children}
      </T>
    </View>
  );
}

const styles = StyleSheet.create({
  face: { minWidth: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
d, alignItems: 'center', justifyContent: 'center' },
  wide: { flexDirection: 'row', gap: 6, paddingHorizontal: 12 },
  pick: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, minWidth: 44, height: 44, paddingHorizontal: 14,
borderRadius: radius.chip, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
  readout: { flexDirection: 'row', alignItems: 'center', gap: 6, alignSelf: 'flex-start', paddingHorizontal: 10, paddingVertical: 6, bor
derRadius: radius.chip, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
});
```

src/components/panorama/Scene.tsx

```
import { LinearGradient } from 'expo-linear-gradient';
import { memo, useId, useState, type ReactNode } from 'react';
import { Animated, StyleSheet, View } from 'react-native';
import Svg, { Circle, Defs, G, Path, Pattern, RadialGradient, Rect, Stop, Use } from 'react-native-svg';

import { useLoop } from '@/components/ui/motion';
import type { Landmark, Pano, Roof } from '@/data/panoramas';

import { LOOKS, SHAPES, type Part, type Tone } from './shapes';

/** Layer speeds relative to the far layer. Wider layers scroll faster, which gives the parallax. */
export const K = { far: 1, mid: 1.4, near: 1.9, sky: 0.5 } as const;

export function layout(S: number) {
  const base = 4 * S;
  return { far: base * K.far, mid: base * K.mid, near: base * K.near, sky: base * K.sky };
}
```

```
/** Compass heading (0–360°) at the centre of the screen for a scroll position. */
export function headingOf(pos: number, S: number) {
  const W = layout(S).mid;
  return (((((pos * K.mid + S / 2) % W) + W) % W) / W) * 360;
}

/** Scroll position that centres a given compass heading. */
export function posForHeading(deg: number, S: number) {
  return ((deg / 360) * layout(S).mid - S / 2) / K.mid;
}

/** x position of something at compass heading `deg` inside a layer (so it lines up with the mid layer). */
function xFor(layerK: number, W: number, deg: number, S: number) {
  return (((posForHeading(deg, S) * layerK + S / 2) % W) + W) % W;
}

function rng(seed: number) {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6d2b79f5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

function shade(hex: string, amt: number) {
  const n = parseInt(hex.slice(1), 16);
  const ch = (v: number) => Math.max(0, Math.min(255, Math.round(v + (amt >= 0 ? (255 - v) * amt : v * amt))));
  const r = ch((n >> 16) & 255);
  const g = ch((n >> 8) & 255);
  const b = ch(n & 255);
  return `#${((1 << 24) | (r << 16) | (g << 8) | b).toString(16).slice(1)}`;
}

interface Building {
  x: number;
  w: number;
  h: number;
  roof: Roof;
  v: number;
  amt: number;
}

function genBuildings(seed: number, W: number, wr: [number, number], hr: [number, number], gap: number, roofs: Roof[], towers: number,
H: number): Building[] {
  const rand = rng(seed);
  const out: Building[] = [];
  let x = 0;
  while (x < W) {
    let w = wr[0] + rand() * (wr[1] - wr[0]);
    if (W - x - w < wr[0]) w = W - x;
    const tall = rand() < towers;
    const h = H * (tall ? hr[1] * (0.6 + rand() * 0.4) : hr[0] + (hr[1] - hr[0]) * Math.pow(rand(), 1.8) * 0.7);
    out.push({ x, w: Math.max(4, w - gap), h, roof: roofs[Math.floor(rand() * roofs.length)], v: Math.floor(rand() * 4), amt: (rand() -
0.5) * 0.22 });
    x += w;
  }
  return out;
}

/** Four window grids with different lit patterns, so skylines don't look tiled. */
function WindowPatterns({ id, night, color, base }: { id: string; night: boolean; color: string; base: string }) {
  return (
    <>
      {[0, 1, 2, 3].map((v) => {
        const rand = rng(900 + v * 17);
        const cells: ReactNode[] = [];
        for (let i = 0; i < 4; i++)
          for (let j = 0; j < 4; j++) {
            const lit = rand() < 0.55;
            if (night && !lit) continue;
            cells.push(<Rect key={`${i}-${j}`} x={3 + i * 6} y={4 + j * 9} width={3} height={5} fill={night ? color : lit ? '#FFFFFF' :
base} opacity={night ? 0.95 : lit ? 0.4 : 0.55} />);
          }
        return (
          <Pattern key={v} id={`${id}${v}`} x={0} y={0} width={24} height={36} patternUnits="userSpaceOnUse">
            {cells}
          </Pattern>
        );
      })}
    </>
  );
}
```

```
function roofEl(b: Building, y: number, fill: string, night: boolean, key: string): ReactNode {
  const { x, w, h } = b;
  switch (b.roof) {
    case 'step':
      return <Rect key={key} x={x + w * 0.25} y={y - h * 0.08} width={w * 0.5} height={h * 0.08} fill={fill} />;
    case 'spire':
      return (
        <G key={key}>
          <Rect x={x + w / 2 - 0.8} y={y - h * 0.2} width={1.6} height={h * 0.2} fill={fill} />
          {night ? <Circle cx={x + w / 2} cy={y - h * 0.2} r={1.8} fill="#FF4B3E" /> : null}
        </G>
      );
    case 'slant':
      return <Path key={key} d={`M${x} ${y}L${x + w * 0.15} ${y - 4}L${x + w} ${y - 12}L${x + w} ${y}Z`} fill={fill} />;
    case 'gable':
      return <Path key={key} d={`M${x - 1} ${y}L${x + w / 2} ${y - w * 0.42}L${x + w + 1} ${y}Z`} fill={night ? shade('#8A4A3A', -0.6) :
'#B5614A'} />;
    case 'dome':
      return <Path key={key} d={`M${x + w * 0.1} ${y}A${w * 0.4} ${w * 0.4} 0 0 1 ${x + w * 0.9} ${y}Z`} fill={shade(fill, night ? 0.1 :
-0.1)} />;
    default:
      return null;
  }
}

function Landmarks({ list, layer, W, base, H, night, S }: { list: Landmark[]; layer: 'mid' | 'far'; W: number; base: number; H: number;
night: boolean; S: number }) {
  return (
    <>
      {list
        .filter((lm) => !!lm.far === (layer === 'far'))
        .map((lm) => {
          const look = LOOKS[lm.kind];
          const idx = night ? 1 : 0;
          const x = xFor(K[layer], W, lm.heading, S);
          const colour = (k: Tone) => look[k][idx];
          return (
            <G key={lm.id} transform={`translate(${x} ${base}) scale(${H * lm.size})`}>
              {night && look.glowR > 0 ? <Circle cx={0} cy={-0.5} r={look.glowR} fill={`url(#glow-${lm.kind})`} /> : null}
              {SHAPES[lm.kind].map((pt: Part, i) => {
                const f = colour(pt.k);
                if (pt.t === 'p') return <Path key={i} d={pt.d} fill={f} />;
                if (pt.t === 'r') return <Rect key={i} x={pt.x} y={pt.y} width={pt.w} height={pt.h} fill={f} />;
                if (pt.t === 'c') return <Circle key={i} cx={pt.x} cy={pt.y} r={pt.r} fill={f} />;
                if (pt.t === 'l') return <Path key={i} d={`M${pt.x1} ${pt.y1}L${pt.x2} ${pt.y2}`} stroke={f} strokeWidth={pt.w} />;
                return <Path key={i} d={pt.d} stroke={f} strokeWidth={pt.w} fill="none" />;
              })}
            </G>
          );
        })}
    </>
  );
}

function GlowDefs({ list }: { list: Landmark[] }) {
  return (
    <>
      {list.map((lm) => (
        <RadialGradient key={lm.id} id={`glow-${lm.kind}`} cx="50%" cy="50%" r="50%">
          <Stop offset="0" stopColor={LOOKS[lm.kind].glow} stopOpacity={0.55} />
          <Stop offset="1" stopColor={LOOKS[lm.kind].glow} stopOpacity={0} />
        </RadialGradient>
      ))}
    </>
  );
}

/** One scrolling layer: content is drawn once and repeated twice side by side so it can wrap around 360°. */
function Wrap({ W, H, defs, children }: { W: number; H: number; defs: ReactNode; children: ReactNode }) {
  const id = `strip${useId().replace(/\W/g, '')}`;
  return (
    <Svg width={W * 2} height={H} style={{ position: 'absolute', left: 0, top: 0 }}>
      <Defs>{defs}</Defs>
      <G id={id}>{children}</G>
      <Use href={`#${id}`} x={W} />
    </Svg>
  );
}

function skylineLayers(pano: Pano, S: number, H: number, night: boolean) {
  const W = layout(S);
  const i = night ? 1 : 0;
  const rand = rng(pano.seed * 7);
  const midBase = H * 0.58;
  const nearBase = H * 0.67;
```

```
  const farBase = H * 0.5;

  // ---- far layer: mountains or distant blocks
  const far: ReactNode[] = [];
  if (pano.mountains) {
    const f = rng(pano.seed + 3);
    const ph = [f() * 6, f() * 6, f() * 6];
    let d = `M0 ${farBase}`;
    for (let x = 0; x <= W.far; x += 12) {
      const t = (x / W.far) * Math.PI * 2;
      const amp =
        pano.mountains === 'hills'
          ? 0.05 + 0.03 * Math.sin(t * 3 + ph[0]) + 0.02 * Math.sin(t * 7 + ph[1])
          : pano.mountains === 'granite'
            ? 0.07 + 0.1 * Math.pow(Math.abs(Math.sin(t * 3 + ph[0])), 1.6) + 0.03 * Math.sin(t * 9 + ph[1])
            : 0.08 + 0.1 * Math.abs(Math.sin(t * 4 + ph[0])) + 0.04 * Math.sin(t * 11 + ph[2]);
      d += `L${x} ${farBase - H * amp}`;
    }
    d += `L${W.far} ${farBase}Z`;
    far.push(<Path key="mount" d={d} fill={pano.tone.far[i]} />);
  } else {
    genBuildings(pano.seed + 5, W.far, [12, 30], [0.03, 0.15], 2, ['flat', 'step', 'flat'], 0.1, H).forEach((b, k) =>
      far.push(<Rect key={`f${k}`} x={b.x} y={farBase - b.h} width={b.w} height={b.h} fill={shade(pano.tone.far[i], b.amt)} />),
    );
  }

  // ---- mid layer: the skyline
  const mid: ReactNode[] = [];
  const midDefs: ReactNode[] = [];
  if (pano.ground === 'slope') {
    const f = rng(pano.seed + 9);
    const ph = [f() * 6, f() * 6];
    const top = (x: number) => midBase - H * (0.13 + 0.06 * Math.sin((x / W.mid) * Math.PI * 2 * 2 + ph[0]) + 0.04 * Math.sin((x / W.mi
d) * Math.PI * 2 * 5 + ph[1]));
    let d = `M0 ${midBase}`;
    for (let x = 0; x <= W.mid; x += 12) d += `L${x} ${top(x)}`;
    d += `L${W.mid} ${midBase}Z`;
    mid.push(<Path key="hill" d={d} fill={pano.tone.mid[i]} />);
    const colours = pano.houses ?? ['#F2B8A2'];
    for (let row = 0; row < 2; row++) {
      let x = row * 11;
      let n = 0;
      while (x < W.mid - 40) {
        const w = 24 + rand() * 16;
        const h = H * (0.035 + rand() * 0.02);
        const y = Math.min(midBase - h, top(x + w / 2) + row * H * 0.035 + rand() * 6);
        const c = colours[Math.floor(rand() * colours.length)];
        const fill = night ? shade(c, -0.62) : c;
        mid.push(<Path key={`h${row}-${n}`} d={`M${x} ${y + h}L${x} ${y + 6}L${x + w / 2} ${y}L${x + w} ${y + 6}L${x + w} ${y + h}Z`} fi
ll={fill} />);
        if (night) mid.push(<Rect key={`w${row}-${n}`} x={x + w * 0.3} y={y + h * 0.45} width={4} height={5} fill={pano.lights} />);
        x += w + 3;
        n++;
      }
    }
  } else {
    midDefs.push(<WindowPatterns key="wp" id={night ? 'wmn' : 'wmd'} night={night} color={pano.lights} base={shade(pano.tone.mid[i], 0.2
5)} />);
    genBuildings(pano.seed, W.mid, pano.mid.w, pano.mid.h, pano.mid.gap, pano.mid.roofs, pano.mid.towers, H).forEach((b, k) => {
      const fill = shade(pano.tone.mid[i], b.amt);
      const y = midBase - b.h;
      mid.push(<Rect key={`m${k}`} x={b.x} y={y} width={b.w} height={b.h} fill={fill} />);
      if (b.w > 8 && b.h > 16) mid.push(<Rect key={`mw${k}`} x={b.x + 1} y={y + 2} width={b.w - 2} height={b.h - 2} fill={`url(#${night
? 'wmn' : 'wmd'}${b.v})`} />);
      mid.push(roofEl(b, y, fill, night, `mr${k}`));
      if (pano.ground === 'water')
        mid.push(<Rect key={`mr${k}f`} x={b.x} y={midBase} width={b.w} height={Math.min(H * 0.1, b.h * 0.45)} fill={night ? pano.lights
: '#FFFFFF'} opacity={night ? 0.12 : 0.1} />);
    });
  }

  // ---- near layer: low rooftops and street lamps
  const near: ReactNode[] = [];
  const nearDefs: ReactNode[] = [<WindowPatterns key="wp" id={night ? 'wnn' : 'wnd'} night={night} color={pano.lights} base={shade(pano.
tone.near[i], 0.2)} />];
  genBuildings(pano.seed + 1, W.near, [34, 78], [pano.near.h[0], pano.near.h[1]], 6, pano.houses ? ['gable', 'flat'] : ['flat', 'gable',
'step'], 0, H).forEach((b, k) => {
    const c = pano.houses ? pano.houses[k % pano.houses.length] : pano.tone.near[i];
    const fill = pano.houses ? (night ? shade(c, -0.68) : c) : shade(c, b.amt);
    const y = nearBase - b.h;
    near.push(
      <Rect key={`n${k}`} x={b.x} y={y} width={b.w} height={b.h} fill={fill} />,
      <Rect key={`nw${k}`} x={b.x + 2} y={y + 3} width={b.w - 4} height={b.h - 4} fill={`url(#${night ? 'wnn' : 'wnd'}${b.v})`} />,
      roofEl({ ...b, roof: b.roof === 'step' ? 'flat' : b.roof }, y, fill, night, `nr${k}`),
    );
  });
```

```
  for (let x = 40; x < W.near; x += 130 + rand() * 40) {
    near.push(<Rect key={`lp${x}`} x={x} y={nearBase - H * 0.075} width={2} height={H * 0.075} fill={night ? '#20243A' : '#4A505C'} />);
    near.push(night ? <Circle key={`lg${x}`} cx={x + 1} cy={nearBase - H * 0.078} r={13} fill={pano.lights} opacity={0.3} /> : null);
    near.push(<Circle key={`lb${x}`} cx={x + 1} cy={nearBase - H * 0.078} r={2.6} fill={night ? '#FFF1C9' : '#9AA3B0'} />);
  }
  // Foreground details: lane markings on streets, shimmer on water.
  if (pano.ground === 'road') {
    for (let x = 0; x < W.near; x += 64) near.push(<Rect key={`ln${x}`} x={x} y={H * 0.79} width={30} height={3} fill={night ? '#C9C3A0'
: '#E8E3C8'} opacity={night ? 0.3 : 0.7} />);
    near.push(<Rect key="edge" x={0} y={H * 0.72} width={W.near} height={2} fill={night ? '#2A2E48' : '#6B7280'} opacity={0.8} />);
  } else {
    for (let k = 0; k < 140; k++) near.push(<Rect key={`sh${k}`} x={rand() * W.near} y={H * (0.69 + rand() * 0.27)} width={14 + rand() *
40} height={1.6} rx={1} fill={night ? pano.lights : '#FFFFFF'} opacity={night ? 0.14 + rand() * 0.14 : 0.18 + rand() * 0.2} />);
  }
  return { far, mid, near, midDefs, nearDefs, W, farBase, midBase, nearBase };
}

function Skyline({ pano, S, H, night, pos }: { pano: Pano; S: number; H: number; night: boolean; pos: Animated.Value }) {
  const [built] = useState(() => skylineLayers(pano, S, H, night));
  const { W } = built;
  const glowDefs = <GlowDefs key="glow" list={pano.landmarks} />; // goes into a list of defs below, so it needs a key
  return (
    <>
      <Layer pos={pos} k={K.far} W={W.far} H={H} defs={null}>
        {built.far}
        <Landmarks list={pano.landmarks} layer="far" W={W.far} base={built.farBase} H={H} night={night} S={S} />
      </Layer>
      <Layer pos={pos} k={K.mid} W={W.mid} H={H} defs={[...built.midDefs, glowDefs]}>
        {built.mid}
        <Landmarks list={pano.landmarks} layer="mid" W={W.mid} base={built.midBase} H={H} night={night} S={S} />
      </Layer>
      <Layer pos={pos} k={K.near} W={W.near} H={H} defs={built.nearDefs}>
        {built.near}
      </Layer>
    </>
  );
}

/** A layer wrapper that scrolls with the shared position value. */
function Layer({ pos, k, W, H, defs, children }: { pos: Animated.Value; k: number; W: number; H: number; defs: ReactNode; children: Reac
tNode }) {
  const tx = Animated.multiply(Animated.modulo(Animated.multiply(pos, k), W), -1);
  return (
    <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { transform: [{ translateX: tx }] }]}>
      <Wrap W={W} H={H} defs={defs}>
        {children}
      </Wrap>
    </Animated.View>
  );
}

function SkyDecor({ pano, S, H, night }: { pano: Pano; S: number; H: number; night: boolean }) {
  const Ws = layout(S).sky;
  const [items] = useState(() => {
    const rand = rng(pano.seed * 13);
    const stars = Array.from({ length: 70 }, () => ({ x: rand() * Ws, y: rand() * H * 0.42, r: 0.6 + rand() * 1.4, o: 0.5 + rand() * 0.5
}));
    const clouds = Array.from({ length: 5 }, () => ({ x: rand() * Ws, y: H * (0.1 + rand() * 0.22), w: 70 + rand() * 90 }));
    return { stars, clouds };
  });
  const sunX = xFor(K.sky, Ws, 205, S);
  const moonX = xFor(K.sky, Ws, 150, S);
  return (
    <Wrap
      W={Ws}
      H={H}
      defs={
        <>
          <RadialGradient id="sun" cx="50%" cy="50%" r="50%">
            <Stop offset="0" stopColor="#FFE9A8" stopOpacity={0.9} />
            <Stop offset="1" stopColor="#FFE9A8" stopOpacity={0} />
          </RadialGradient>
          <RadialGradient id="moon" cx="50%" cy="50%" r="50%">
            <Stop offset="0" stopColor="#FFF4C7" stopOpacity={0.5} />
            <Stop offset="1" stopColor="#FFF4C7" stopOpacity={0} />
          </RadialGradient>
        </>
      }>
      {night ? (
        <>
          {items.stars.map((st, i) => (
            <Circle key={i} cx={st.x} cy={st.y} r={st.r} fill="#FFFFFF" opacity={st.o} />
          ))}
          <Circle cx={moonX} cy={H * 0.17} r={70} fill="url(#moon)" />
          <Circle cx={moonX} cy={H * 0.17} r={20} fill="#FFF4C7" />
          <Circle cx={moonX + 8} cy={H * 0.17 - 4} r={18} fill={pano.sky.nightTop} opacity={0.9} />
```

```
        </>
      ) : (
        <>
          <Circle cx={sunX} cy={H * 0.2} r={150} fill="url(#sun)" />
          <Circle cx={sunX} cy={H * 0.2} r={26} fill="#FFF3C4" />
          {items.clouds.map((c, i) => (
            <G key={i} opacity={0.75}>
              <Rect x={c.x} y={c.y} width={c.w} height={14} rx={7} fill="#FFFFFF" />
              <Rect x={c.x + c.w * 0.2} y={c.y - 9} width={c.w * 0.5} height={16} rx={8} fill="#FFFFFF" />
            </G>
          ))}
        </>
      )}
    </Wrap>
  );
}

function CarLights({ H, S, color }: { H: number; S: number; color: string }) {
  const a = useLoop(9000, { rest: 0 });
  const b = useLoop(12000, { rest: 0 });
  const dots = Array.from({ length: 18 }, (_, i) => i);
  const row = (key: string, v: Animated.Value, y: number, c: string, dir: 1 | -1) => (
    <Animated.View
      key={key}
      pointerEvents="none"
      style={{ position: 'absolute', top: y, left: 0, width: S * 2, height: 6, transform: [{ translateX: v.interpolate({ inputRange: [0,
1], outputRange: dir === 1 ? [-S, 0] : [0, -S] }) }] }}>
      {dots.map((i) => (
        <View key={i} style={{ position: 'absolute', left: (i * S * 2) / 18 + ((i * 37) % 23), width: 11, height: 4, borderRadius: 2, ba
ckgroundColor: c, opacity: 0.9 }} />
      ))}
    </Animated.View>
  );
  return (
    <>
      {row('w', a, H * 0.755, '#FFF6D8', -1)}
      {row('r', b, H * 0.8, '#FF5A4D', 1)}
      <View pointerEvents="none" style={{ position: 'absolute', left: 0, right: 0, top: H * 0.745, height: H * 0.08, backgroundColor: co
lor, opacity: 0.05 }} />
    </>
  );
}

function PanoramaSceneImpl({ pano, night, pos, S, H }: { pano: Pano; night: Animated.Value; pos: Animated.Value; S: number; H: number })
{
  const flicker = useLoop(2600, { pingPong: true, rest: 0.5 });
  const nightOp = Animated.multiply(night, flicker.interpolate({ inputRange: [0, 1], outputRange: [0.93, 1] }));
  const dayOp = night.interpolate({ inputRange: [0, 1], outputRange: [1, 0] });
  const water = pano.ground !== 'road';
  const ground = water ? { day: ['#5B97C9', '#2F6A9E'], night: ['#0B1235', '#04061A'] } : { day: ['#454C59', '#2B3039'], night: ['#0E112
2', '#05060F'] };
  const skyTx = Animated.multiply(Animated.modulo(Animated.multiply(pos, K.sky), layout(S).sky), -1);
  const sparkle = useLoop(3200, { pingPong: true, rest: 0.5 });
  return (
    <View style={StyleSheet.absoluteFill} pointerEvents="none">
      {/* Sky */}
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: dayOp }]}>
        <LinearGradient colors={[pano.sky.dayTop, pano.sky.dayHorizon]} locations={[0.05, 0.55]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: night }]}>
        <LinearGradient colors={[pano.sky.nightTop, pano.sky.nightHorizon]} locations={[0.05, 0.58]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: dayOp, transform: [{ translateX: skyTx }] }]}>
        <SkyDecor pano={pano} S={S} H={H} night={false} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: Animated.multiply(night, sparkle.interpolate({ inputRange: [0, 1], outp
utRange: [0.8, 1] })), transform: [{ translateX: skyTx }] }]}>
        <SkyDecor pano={pano} S={S} H={H} night />
      </Animated.View>

      {/* Ground / water */}
      <Animated.View style={[styles.ground, { top: H * 0.5, opacity: dayOp }]}>
        <LinearGradient colors={ground.day as [string, string]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[styles.ground, { top: H * 0.5, opacity: night }]}>
        <LinearGradient colors={ground.night as [string, string]} style={StyleSheet.absoluteFill} />
      </Animated.View>

      {/* Skyline: day set and night set cross-fade */}
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: dayOp }]}>
        <Skyline pano={pano} S={S} H={H} night={false} pos={pos} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: nightOp }]}>
        <Skyline pano={pano} S={S} H={H} night pos={pos} />
      </Animated.View>
```

```
      {/* Warm city glow above the skyline at night */}
      <Animated.View style={[styles.glow, { top: H * 0.3, height: H * 0.3, opacity: Animated.multiply(night, 0.5) }]}>
        <LinearGradient colors={['rgba(255,150,70,0)', pano.lights]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[styles.glow, { top: H * 0.3, height: H * 0.3, opacity: Animated.multiply(dayOp, 0.28) }]}>
        <LinearGradient colors={['rgba(255,255,255,0)', '#FFFFFF']} style={StyleSheet.absoluteFill} />
      </Animated.View>

      {!water ? (
        <Animated.View style={[StyleSheet.absoluteFill, { opacity: night }]}>
          <CarLights S={S} H={H} color={pano.lights} />
        </Animated.View>
      ) : null}
    </View>
  );
}

export const PanoramaScene = memo(PanoramaSceneImpl);

const styles = StyleSheet.create({
  ground: { position: 'absolute', left: 0, right: 0, bottom: 0 },
  glow: { position: 'absolute', left: 0, right: 0 },
});
```

src/components/panorama/__tests__/nativeDriver.test.tsx

```
/**
 * The 360° screen once crashed on a real phone with "Attempting to run JS driven animation on animated node that has been moved to
 * 'native' earlier": the scene mixes the day/night and position values with looping lights that run on the native driver, and one
 * value cannot be driven both ways. The web build ignores `useNativeDriver`, so only a test that turns the native driver on sees it.
 */
import { createElement, type ReactNode } from 'react';
import { act, create, type ReactTestRenderer } from 'react-test-renderer';

import PanoramaScreen from '@/app/panorama';

// (jest.mock calls are lifted above the imports.) Pretend the phone's native animation module exists, so `useNativeDriver: true` really
marks values as native.
jest.mock('react-native/Libraries/Animated/NativeAnimatedModule', () => ({
  __esModule: true,
  default: new Proxy({}, { get: (_t, name) => (name === 'then' ? undefined : jest.fn()) }),
}));
jest.mock('react-native/Libraries/ReactNative/RendererProxy', () => ({ ...jest.requireActual('react-native/Libraries/ReactNative/Rendere
rProxy'), findNodeHandle: () => 1 }));
jest.mock('react-native-safe-area-context', () => ({ useSafeAreaInsets: () => ({ top: 0, bottom: 0, left: 0, right: 0 }) }));
jest.mock('expo-router', () => ({
  useLocalSearchParams: () => ({}),
  // runs the screen's focus effect once, like a screen that is being looked at
  useFocusEffect: (effect: () => void | (() => void)) => jest.requireActual('react').useEffect(effect, [effect]),
  router: { canGoBack: () => true, back: jest.fn(), replace: jest.fn() },
}));

(globalThis as { IS_REACT_ACT_ENVIRONMENT?: boolean }).IS_REACT_ACT_ENVIRONMENT = true;

const press = async (root: ReactTestRenderer, label: string | RegExp) => {
  const hit = root.root.findAll((n) => typeof n.props.onPress === 'function' && (typeof label === 'string' ? n.props.accessibilityLabel
=== label : label.test(n.props.accessibilityLabel ?? '')))[0];
  expect(hit).toBeDefined();
  await act(async () => {
    hit.props.onPress();
  });
};

describe('360° screen on the native animation driver', () => {
  beforeEach(() => jest.useFakeTimers());
  afterEach(() => jest.useRealTimers());

  it('can switch day/night and look around while the lights loop natively', async () => {
    let root!: ReactTestRenderer;
    await act(async () => {
      root = create(createElement(PanoramaScreen as () => ReactNode), { createNodeMock: () => ({}) });
    });
    // let the looping lights start (they start on a timer after the first render)
    await act(async () => {
      jest.advanceTimersByTime(200);
    });
    await press(root, /Switch to (day|night)/); // restarts the day/night cross-fade on a value the lights already made native
    await press(root, 'Look right'); // glides the position value, which shares a style with the day/night value
    await press(root, 'Look left');
    await act(async () => {
      jest.advanceTimersByTime(2000);
    });
    act(() => root.unmount());
  });
});
```

src/components/panorama/shapes.ts

```
import type { LandmarkKind } from '@/data/panoramas';

/**
 * Landmark silhouettes in unit coordinates: x is centred, y is negative upwards, 1 = full height.
 * Tones: m = main body, a = accent, l = light (windows / lamps — glows at night).
 */
export type Tone = 'm' | 'a' | 'l';
export type Part =
  | { t: 'p'; d: string; k: Tone }
  | { t: 'r'; x: number; y: number; w: number; h: number; k: Tone }
  | { t: 'c'; x: number; y: number; r: number; k: Tone }
  | { t: 'l'; x1: number; y1: number; x2: number; y2: number; w: number; k: Tone }
  | { t: 's'; d: string; w: number; k: Tone };

const p = (d: string, k: Tone): Part => ({ t: 'p', d, k });
const r = (x: number, y: number, w: number, h: number, k: Tone): Part => ({ t: 'r', x, y, w, h, k });
const c = (x: number, y: number, rad: number, k: Tone): Part => ({ t: 'c', x, y, r: rad, k });
const l = (x1: number, y1: number, x2: number, y2: number, w: number, k: Tone): Part => ({ t: 'l', x1, y1, x2, y2, w, k });
const s = (d: string, w: number, k: Tone): Part => ({ t: 's', d, w, k });

export interface Look {
  /** [day, night] colour for each tone. */
  m: [string, string];
  a: [string, string];
  l: [string, string];
  /** Night glow colour. */
  glow: string;
  /** Glow radius in units. */
  glowR: number;
}

const shell = (x: number, w: number, hh: number) =>
  `M${x} -.06C${x} ${-0.06 - hh * 0.4} ${x + w * 0.4} ${-0.06 - hh} ${x + w * 0.8} ${-0.06 - hh}C${x + w * 0.9} ${-0.06 - hh * 0.6} ${x
+ w} ${-0.06 - hh * 0.2} ${x + w} -.06Z`;

function eye(): Part[] {
  const out: Part[] = [s('M-.5 -.5A.5 .5 0 1 0 .5 -.5A.5 .5 0 1 0 -.5 -.5', 0.012, 'a'), l(0, -0.5, -0.2, 0, 0.016, 'm'), l(0, -0.5, 0.
2, 0, 0.016, 'm')];
  for (let i = 0; i < 16; i++) {
    const a = (i / 16) * Math.PI * 2;
    const x = Math.cos(a) * 0.5;
    const y = -0.5 + Math.sin(a) * 0.5;
    out.push(l(0, -0.5, x, y, 0.003, 'm'), c(x, y, 0.017, 'l'));
  }
  return out;
}

function harbour(): Part[] {
  const out: Part[] = [s('M-1.1 0Q0 -1.5 1.1 0', 0.04, 'm'), r(-1.3, -0.12, 2.6, 0.03, 'm'), r(-1.3, -0.12, 2.6, 0.01, 'l'), r(-1.16, -
0.45, 0.12, 0.45, 'a'), r(1.04, -0.45, 0.12, 0.45, 'a')];
  for (let i = 1; i < 20; i++) {
    const t = i / 20;
    out.push(l(-1.1 + 2.2 * t, -3 * t * (1 - t), -1.1 + 2.2 * t, -0.12, 0.004, 'm'));
  }
  return out;
}

function operaParts(): Part[] {
  const shells: [number, number, number][] = [
    [-0.66, 0.3, 0.58],
    [-0.4, 0.28, 0.48],
    [-0.16, 0.26, 0.38],
    [0.08, 0.34, 0.64],
    [0.36, 0.32, 0.52],
    [0.62, 0.2, 0.3],
  ];
  return [r(-0.75, -0.06, 1.55, 0.06, 'a'), ...shells.map(([x, w, hh]) => p(shell(x, w, hh), 'm'))];
}

function colosseum(): Part[] {
  const out: Part[] = [p('M-.8 0L-.8-.4L-.62-.5L.5-.5L.8-.4L.8 0Z', 'm')];
  for (let row = 0; row < 3; row++) for (let i = 0; i < 15; i++) out.push(r(-0.76 + i * 0.105, -0.13 - row * 0.12, 0.05, 0.08, 'a'));
  return out;
}

function supertrees(): Part[] {
  const out: Part[] = [];
  for (const [cx, hh] of [
    [-0.5, 0.7],
    [0, 1],
    [0.5, 0.8],
  ] as const) {
    out.push(
      p(`M${cx - 0.03} 0L${cx - 0.008} ${-hh}L${cx + 0.008} ${-hh}L${cx + 0.03} 0Z`, 'm'),
```

```
      p(`M${cx - 0.24} ${-hh}Q${cx} ${-hh - 0.3} ${cx + 0.24} ${-hh}Z`, 'a'),
      c(cx, -hh - 0.07, 0.018, 'l'),
    );
  }
  return out;
}

export const SHAPES: Record<LandmarkKind, Part[]> = {
  eiffel: [
    p('M-.19 0L-.015-1L.015-1L.19 0L.13 0Q0 -.34 -.13 0Z', 'm'),
    r(-0.13, -0.405, 0.26, 0.016, 'm'),
    r(-0.075, -0.7, 0.15, 0.014, 'm'),
    r(-0.004, -1.07, 0.008, 0.07, 'm'),
    c(0, -0.41, 0.012, 'l'),
    c(0, -0.71, 0.01, 'l'),
    r(-0.004, -0.98, 0.008, 0.2, 'l'),
  ],
  tokyotower: [
    p('M-.14 0L-.03-.66L-.016-.78L-.006-1L.006-1L.016-.78L.03-.66L.14 0L.1 0Q0 -.22 -.1 0Z', 'm'),
    r(-0.065, -0.34, 0.13, 0.03, 'a'),
    r(-0.04, -0.62, 0.08, 0.025, 'a'),
    r(-0.12, -0.15, 0.24, 0.02, 'a'),
    c(0, -0.33, 0.012, 'l'),
  ],
  skytree: [p('M-.055 0L-.014-.6L.014-.6L.055 0Z', 'm'), r(-0.05, -0.64, 0.1, 0.03, 'a'), r(-0.032, -0.8, 0.064, 0.025, 'a'), r(-0.004,
-1, 0.008, 0.2, 'm'), r(-0.003, -0.6, 0.006, 0.4, 'l')],
  fuji: [p('M-1 0L-.12-.5L-.06-.52L0-.55L.06-.52L.12-.5L1 0Z', 'm'), p('M-.16-.43L-.12-.5L-.06-.52L0-.55L.06-.52L.12-.5L.16-.43L.09-.46
L.04-.4L-.02-.45L-.08-.39Z', 'a')],
  esb: [
    r(-0.15, -0.4, 0.3, 0.4, 'm'),
    r(-0.11, -0.62, 0.22, 0.22, 'm'),
    r(-0.075, -0.78, 0.15, 0.16, 'm'),
    r(-0.045, -0.88, 0.09, 0.1, 'a'),
    r(-0.022, -0.93, 0.044, 0.05, 'a'),
    r(-0.004, -1, 0.008, 0.07, 'a'),
    r(-0.03, -0.86, 0.06, 0.02, 'l'),
  ],
  wtc: [p('M-.075 0L-.075-.7L-.003-.86L.003-.86L.075-.7L.075 0Z', 'm'), r(-0.003, -1, 0.006, 0.15, 'a'), r(-0.003, -0.86, 0.006, 0.86,
'l')],
  chrysler: [r(-0.07, -0.66, 0.14, 0.66, 'm'), p('M-.07-.66Q-.07-.9 0-.95Q.07-.9 .07-.66Z', 'a'), r(-0.004, -1, 0.008, 0.06, 'a'), c(0,
-0.82, 0.014, 'l')],
  bigben: [
    r(-0.05, -0.78, 0.1, 0.78, 'm'),
    p('M-.065-.78L0-1L.065-.78Z', 'a'),
    r(-0.065, -0.8, 0.13, 0.03, 'a'),
    c(0, -0.62, 0.032, 'l'),
    r(0.05, -0.2, 0.7, 0.2, 'm'),
    r(0.2, -0.3, 0.05, 0.1, 'm'),
    r(0.45, -0.28, 0.04, 0.08, 'm'),
    p('M.2-.3L.225-.38L.25-.3Z', 'a'),
    r(0.1, -0.14, 0.6, 0.03, 'l'),
  ],
  eye: eye(),
  shard: [p('M-.07 0L0-1L.07 0Z', 'm'), l(0, -1, -0.03, 0, 0.004, 'a'), l(0, -1, 0.03, 0, 0.004, 'a'), l(0, -1, 0, 0, 0.006, 'l')],
  burj: [
    p('M-.07 0L-.05-.35L-.035-.5L-.03-.62L-.02-.78L-.012-.9L-.003-1L.003-1L.012-.9L.02-.78L.03-.62L.035-.5L.05-.35L.07 0Z', 'm'),
    l(0, 0, 0, -0.95, 0.005, 'l'),
    l(-0.03, -0.2, -0.015, -0.6, 0.003, 'a'),
    l(0.03, -0.2, 0.015, -0.6, 0.003, 'a'),
  ],
  burjalarab: [p('M-.2 0L-.2-.75Q-.02-1 .02-1Q.2-.45 .2 0Z', 'm'), r(-0.45, -0.03, 0.9, 0.03, 'a'), r(-0.012, -1.1, 0.012, 0.12, 'a'), l
(-0.2, -0.75, 0.02, -1, 0.012, 'l')],
  opera: operaParts(),
  harbour: harbour(),
  sydtower: [r(-0.012, -0.7, 0.024, 0.7, 'm'), p('M-.06-.7L.06-.7L.045-.8L-.045-.8Z', 'a'), r(-0.05, -0.83, 0.1, 0.05, 'a'), r(-0.004, -
1, 0.008, 0.17, 'm'), r(-0.045, -0.8, 0.09, 0.012, 'l')],
  mbs: [
    p('M-.52 0L-.34 0L-.22-1L-.4-1Z', 'm'),
    p('M-.09 0L.09 0L.09-1L-.09-1Z', 'm'),
    p('M.34 0L.52 0L.4-1L.22-1Z', 'm'),
    r(-0.66, -1.05, 1.32, 0.05, 'a'),
    p('M.66-1.05L.84-1.03L.66-1Z', 'a'),
    r(-0.66, -1, 1.32, 0.012, 'l'),
  ],
  supertrees: supertrees(),
  christ: [
    p('M-1 0Q-.4 -.15 -.2 -.6Q-.1 -.95 0 -.95Q.1 -.95 .2 -.6Q.5 -.15 1 0Z', 'm'),
    r(-0.016, -1, 0.032, 0.03, 'a'),
    r(-0.007, -1.12, 0.014, 0.12, 'a'),
    r(-0.06, -1.1, 0.12, 0.014, 'a'),
    c(0, -1.135, 0.009, 'a'),
  ],
  sugarloaf: [p('M-.5 0Q-.52 -.6 -.15 -.78Q.2 -.9 .4 -.5Q.52 -.2 .55 0Z', 'm'), p('M.45 0Q.6 -.35 .9 -.35Q1.3 -.3 1.5 0Z', 'm'), l(0.28,
-0.7, 1.0, -0.4, 0.004, 'a'), r(0.6, -0.6, 0.05, 0.04, 'l')],
  colosseum: colosseum(),
  stpeters: [
```

```
    r(-0.4, -0.28, 0.8, 0.28, 'm'),
    r(-0.16, -0.6, 0.32, 0.32, 'm'),
    p('M-.16-.6Q-.16-1 0-1Q.16-1 .16-.6Z', 'a'),
    r(-0.03, -1.12, 0.06, 0.12, 'm'),
    r(-0.004, -1.25, 0.008, 0.13, 'a'),
    r(-0.3, -0.2, 0.6, 0.02, 'l'),
  ],
  duomo: [
    r(-0.3, -0.5, 0.6, 0.5, 'm'),
    p('M-.22-.5Q-.22-.95 0-.95Q.22-.95 .22-.5Z', 'a'),
    l(-0.1, -0.5, -0.06, -0.93, 0.02, 'l'),
    l(0.1, -0.5, 0.06, -0.93, 0.02, 'l'),
    r(0.32, -0.78, 0.12, 0.78, 'm'),
    p('M.3-.78L.38-.92L.46-.78Z', 'a'),
    r(-0.24, -0.28, 0.12, 0.14, 'l'),
    r(0.12, -0.28, 0.12, 0.14, 'l'),
  ],
  arc: [p('M-.45 0L-.45-1L.45-1L.45 0L.18 0L.18-.55Q0 -.8 -.18-.55L-.18 0Z', 'm'), r(-0.45, -0.9, 0.9, 0.02, 'a'), r(-0.45, -1, 0.9, 0.0
5, 'a')],
  sacre: [
    r(-0.4, -0.35, 0.8, 0.35, 'm'),
    p('M-.15-.35Q-.15-.85 0-.9Q.15-.85 .15-.35Z', 'm'),
    p('M-.38-.35Q-.38-.6 -.28-.62Q-.18-.6 -.18-.35Z', 'm'),
    p('M.18-.35Q.18-.6 .28-.62Q.38-.6 .38-.35Z', 'm'),
    r(-0.012, -1, 0.024, 0.1, 'a'),
    r(-0.3, -0.2, 0.6, 0.02, 'l'),
  ],
};

const warm = (day: string, night: string): [string, string] => [day, night];

export const LOOKS: Record<LandmarkKind, Look> = {
  eiffel: { m: warm('#5B4B40', '#C98F2E'), a: warm('#6F5E50', '#E5B04C'), l: warm('#8C7A6A', '#FFE08A'), glow: '#FFC14D', glowR: 0.6 },
  tokyotower: { m: warm('#D9432B', '#FF7A3A'), a: warm('#F3F3F3', '#FFF1C9'), l: warm('#F3F3F3', '#FFF1C9'), glow: '#FF8A4A', glowR: 0.5
},
  skytree: { m: warm('#C9D2DC', '#6FB6FF'), a: warm('#EEF3F8', '#BFE4FF'), l: warm('#E8EEF5', '#CFEAFF'), glow: '#6FB6FF', glowR: 0.4 },
  fuji: { m: warm('#8C9DB8', '#1B2150'), a: warm('#FFFFFF', '#8FA4E0'), l: warm('#FFFFFF', '#8FA4E0'), glow: '#000000', glowR: 0 },
  esb: { m: warm('#B9B2A6', '#EAD9A8'), a: warm('#9C9488', '#FFD36B'), l: warm('#D7D1C6', '#FFF0B8'), glow: '#FFD36B', glowR: 0.5 },
  wtc: { m: warm('#8FA6C0', '#BFD7F5'), a: warm('#C9D5E2', '#FFFFFF'), l: warm('#DCE6F0', '#FFFFFF'), glow: '#BFD7F5', glowR: 0.45 },
  chrysler: { m: warm('#A9A39C', '#E6DFC8'), a: warm('#D5DAE0', '#F4F8FF'), l: warm('#F5F5F5', '#FFFFFF'), glow: '#F4F8FF', glowR: 0.45
},
  bigben: { m: warm('#B3A58E', '#D9B76E'), a: warm('#7D8B6E', '#F0D8A0'), l: warm('#E8E2D0', '#FFF6C9'), glow: '#FFD98A', glowR: 0.45 },
  eye: { m: warm('#8FA3B8', '#5EB6FF'), a: warm('#B5C7DA', '#8AD0FF'), l: warm('#FFFFFF', '#CFEFFF'), glow: '#5EB6FF', glowR: 0.7 },
  shard: { m: warm('#9DB6CE', '#CFE3FF'), a: warm('#C5D6E6', '#FFFFFF'), l: warm('#E6EEF6', '#FFFFFF'), glow: '#CFE3FF', glowR: 0.4 },
  burj: { m: warm('#B9C6D4', '#DDEBFF'), a: warm('#8EA2B8', '#7DB8FF'), l: warm('#E8F0F8', '#FFFFFF'), glow: '#9CC8FF', glowR: 0.4 },
  burjalarab: { m: warm('#F4F1EA', '#5A9BFF'), a: warm('#D9D5CB', '#B58BFF'), l: warm('#FFFFFF', '#E1D0FF'), glow: '#7B8CFF', glowR: 0.6
},
  opera: { m: warm('#F2EFE8', '#FFD9A0'), a: warm('#C9C3B6', '#C97F4A'), l: warm('#FFFFFF', '#FFE9C0'), glow: '#FFC878', glowR: 0.7 },
  harbour: { m: warm('#59616B', '#8EB8E8'), a: warm('#B59A7B', '#F4C27A'), l: warm('#FFFFFF', '#FFE08A'), glow: '#8EB8E8', glowR: 0.5 },
  sydtower: { m: warm('#9AA3AD', '#F4C95D'), a: warm('#C9A24B', '#FFE08A'), l: warm('#FFFFFF', '#FFF0B0'), glow: '#F4C95D', glowR: 0.4
},
  mbs: { m: warm('#B5BFC9', '#E6EEF8'), a: warm('#8FA0B2', '#FFB36B'), l: warm('#E8F2FF', '#FFE7A8'), glow: '#8FC8FF', glowR: 0.5 },
  supertrees: { m: warm('#4B6B52', '#2E4A3A'), a: warm('#6FA37A', '#C25BFF'), l: warm('#D0F0D8', '#FF8CF0'), glow: '#C25BFF', glowR: 0.7
},
  christ: { m: warm('#4D6B5A', '#10202A'), a: warm('#F2F2F2', '#FFF3C9'), l: warm('#FFFFFF', '#FFF3C9'), glow: '#FFF3C9', glowR: 0.1 },
  sugarloaf: { m: warm('#5E7467', '#16222B'), a: warm('#C9D2CC', '#FFD07A'), l: warm('#FFFFFF', '#FFD07A'), glow: '#FFD07A', glowR: 0 },
  colosseum: { m: warm('#C7A57C', '#6D4B33'), a: warm('#8E6E4A', '#FFC873'), l: warm('#E8D2B0', '#FFD89A'), glow: '#FFB45E', glowR: 0.5
},
  stpeters: { m: warm('#D9CDB6', '#F0CF8F'), a: warm('#B8A98B', '#FFE3A8'), l: warm('#F2EAD8', '#FFF0C8'), glow: '#FFD98A', glowR: 0.5
},
  duomo: { m: warm('#F2D5B0', '#FFCF8A'), a: warm('#E0A93C', '#FFE08A'), l: warm('#4E9E6B', '#7DFFB0'), glow: '#FFC878', glowR: 0.5 },
  arc: { m: warm('#C9BFA9', '#EBCB86'), a: warm('#A89E88', '#FFE2A0'), l: warm('#FFFFFF', '#FFF0C8'), glow: '#FFD98A', glowR: 0.5 },
  sacre: { m: warm('#F4EFE6', '#FFE9B8'), a: warm('#D9D2C4', '#FFF6D9'), l: warm('#FFFFFF', '#FFF6D9'), glow: '#FFE9B8', glowR: 0.5 },
};
```

src/components/personalize/Building.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useRef, useState } from 'react';
import { StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { FadeIn, Pulse, useMotion } from '@/components/ui/motion';
import { T } from '@/components/ui/T';
import { MAX_WIDTH, line, radius } from '@/theme';

import { PalBar, PalFlap, PalStamp, Tx } from './kit';
import { BUILD_DURATION, BUILD_HOLD, BUILD_TICK, buildPercent, linesDone } from './build';
import { BUILD_LINES } from './options';
import type { Pal } from './palette';
import { StepKicker } from './parts';

/**
 * "Building your trip HQ": a short reveal on the departures board. A number flips up to 100 while three status lines tick off,
 * then `onDone` puts the new look on. With animations off it is a still page with everything ticked, shown briefly.
 */
```

```
export function Building({ pal, onDone }: { pal: Pal; onDone: () => void }) {
  const insets = useSafeAreaInsets();
  const motion = useMotion();
  const [now, setNow] = useState(0);
  const pct = motion ? buildPercent(now) : 100;
  const done = linesDone(pct);
  const ready = pct >= 100;
  const finish = useRef(onDone);
  useEffect(() => {
    finish.current = onDone;
  }, [onDone]);

  useEffect(() => {
    if (!motion) {
      const t = setTimeout(() => finish.current(), 900);
      return () => clearTimeout(t);
    }
    let elapsed = 0;
    let end: ReturnType<typeof setTimeout> | undefined;
    const id = setInterval(() => {
      elapsed += BUILD_TICK;
      setNow(elapsed);
      if (elapsed >= BUILD_DURATION) {
        clearInterval(id);
        end = setTimeout(() => finish.current(), BUILD_HOLD);
      }
    }, BUILD_TICK);
    return () => {
      clearInterval(id);
      if (end) clearTimeout(end);
    };
  }, [motion]);

  return (
    <View style={[StyleSheet.absoluteFill, styles.root, { backgroundColor: pal.bg, paddingTop: insets.top, paddingBottom: insets.bottom
}]} accessibilityViewIsModal>
      <View style={styles.column}>
        <FadeIn duration={260} distance={10}>
          <View style={styles.stack}>
            <StepKicker pal={pal} text="Boarding" />
            <T variant="display" color={pal.text} accessibilityRole="header" accessibilityLiveRegion="polite">
              {ready ? 'Your trip HQ is ready' : 'Building your trip HQ'}
            </T>
            <View style={styles.percent} accessibilityRole="progressbar" accessibilityLabel="Building your trip HQ" accessibilityValue=
{{ min: 0, max: 100, now: pct }}>
              <PalFlap pal={pal} value={String(pct).padStart(3, '0')} size={76} />
              <T variant="numeral" color={pal.textSecondary}>
                %
              </T>
            </View>
            <PalBar pal={pal} value={pct / 100} />
            <View style={styles.lines}>
              {BUILD_LINES.map((text, i) => {
                const isDone = i < done;
                const current = i === done;
                return (
                  <View key={text} style={styles.line}>
                    {isDone ? (
                      <View style={[styles.box, { backgroundColor: pal.ink, borderColor: pal.ink }]}>
                        <Ionicons name="checkmark" size={16} color={pal.bg} />
                      </View>
                    ) : current ? (
                      <Pulse scale={1.12}>
                        <View style={[styles.box, { backgroundColor: pal.accent, borderColor: pal.ink }]} />
                      </Pulse>
                    ) : (
                      <View style={[styles.box, { borderColor: pal.borderStrong }]} />
                    )}
                    <Tx pal={pal} tone={isDone ? 'text' : current ? 'secondary' : 'muted'} variant="body" weight={isDone ? 'bold' : 'reg
ular'}>
                      {text}
                    </Tx>
                  </View>
                );
              })}
            </View>
            <View style={{ minHeight: 30 }}>{ready ? <PalStamp pal={pal} label="Ready" tone="accent" slam /> : null}</View>
          </View>
        </FadeIn>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { alignItems: 'center', zIndex: 50 },
```

```
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH, justifyContent: 'center', paddingHorizontal: 20 },
  stack: { gap: 20 },
  percent: { flexDirection: 'row', alignItems: 'flex-end', gap: 8 },
  lines: { gap: 12 },
  line: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 28 },
  box: { width: 24, height: 24, borderRadius: radius.xs, borderWidth: line, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/personalize/__tests__/build.test.ts

```
import { BUILD_DURATION, buildPercent, linesDone } from '../build';
import { BUILD_LINES } from '../options';

describe('the "Building your trip HQ" numbers', () => {
  it('starts at 0, ends at exactly 100 and never goes backwards', () => {
    expect(buildPercent(0)).toBe(0);
    expect(buildPercent(BUILD_DURATION)).toBe(100);
    expect(buildPercent(BUILD_DURATION * 5)).toBe(100);
    expect(buildPercent(-50)).toBe(0);
    let last = 0;
    for (let ms = 0; ms <= BUILD_DURATION; ms += 100) {
      const v = buildPercent(ms);
      expect(v).toBeGreaterThanOrEqual(last);
      last = v;
    }
  });

  it('ticks the status lines one third at a time, the last one only at 100', () => {
    expect(linesDone(0)).toBe(0);
    expect(linesDone(33)).toBe(1);
    expect(linesDone(66)).toBe(1);
    expect(linesDone(67)).toBe(2);
    expect(linesDone(99)).toBe(2);
    expect(linesDone(100)).toBe(BUILD_LINES.length);
  });
});
```

src/components/personalize/__tests__/look.test.ts

```
import { barcodeWidths, lookLabel, lookNeedsReload, modeAfterPickingTheme, styleCode, type Look } from '../look';

const now: Look & { resolved: 'light' | 'dark' } = { theme: 'amalfi', corners: 'rounded', mode: 'system', resolved: 'light' };

describe('choosing a look', () => {
  it('switches to the dark mode with Grape and leaves the mode alone for every other theme', () => {
    expect(modeAfterPickingTheme('midnight', 'system')).toBe('dark');
    expect(modeAfterPickingTheme('midnight', 'light')).toBe('dark');
    expect(modeAfterPickingTheme('sunset', 'dark')).toBe('dark');
    expect(modeAfterPickingTheme('amalfi', 'system')).toBe('system');
    expect(modeAfterPickingTheme('lagoon', 'light')).toBe('light');
  });

  it('does not restart the app when nothing that shows has changed', () => {
    expect(lookNeedsReload({ theme: 'amalfi', corners: 'rounded', mode: 'system' }, now, false)).toBe(false);
    // "light" on a phone that is in the light mode looks the same as "system"
    expect(lookNeedsReload({ theme: 'amalfi', corners: 'rounded', mode: 'light' }, now, false)).toBe(false);
  });

  it('restarts for a new theme, new corners, or a light or dark that changes what is shown', () => {
    expect(lookNeedsReload({ theme: 'sunset', corners: 'rounded', mode: 'system' }, now, false)).toBe(true);
    expect(lookNeedsReload({ theme: 'amalfi', corners: 'bubbly', mode: 'system' }, now, false)).toBe(true);
    expect(lookNeedsReload({ theme: 'amalfi', corners: 'rounded', mode: 'dark' }, now, false)).toBe(true);
    // "system" on a phone that has gone dark is a different look from the light one on screen
    expect(lookNeedsReload({ theme: 'amalfi', corners: 'rounded', mode: 'system' }, now, true)).toBe(true);
  });

  it('describes a look in a few words, with the theme names the person sees', () => {
    expect(lookLabel({ theme: 'midnight', mode: 'dark' })).toBe('Grape · Dark');
    expect(lookLabel({ theme: 'amalfi', mode: 'system' })).toBe('Limoncello · System');
  });
});

describe('the code printed on the last page', () => {
  it('is three short words in capitals', () => {
    expect(styleCode('amalfi', 'Planner', 'Lively')).toBe('LIM-PLA-LIV');
    expect(styleCode('classic', 'Custom', 'Max')).toBe('INK-CUS-MAX');
  });

  it('draws the same stripes for the same answers and different ones for different answers', () => {
    expect(barcodeWidths('LIM-PLA-LIV')).toEqual(barcodeWidths('LIM-PLA-LIV'));
    expect(barcodeWidths('LIM-PLA-LIV')).not.toEqual(barcodeWidths('TOM-MON-MAX'));
    expect(barcodeWidths('LIM-PLA-LIV')).toHaveLength(34);
    for (const w of barcodeWidths('LIM-PLA-LIV')) expect([1, 2, 3]).toContain(w);
  });
});
```

src/components/personalize/applyLook.ts

```
import { Appearance as OsAppearance } from 'react-native';

import { useAppStore } from '@/store/useAppStore';
import { appearance, applyAppearance, resolvedMode } from '@/theme';

import { lookNeedsReload, type Look } from './look';

/** A look that was saved without a restart (nothing visible changed), so screens opened later start from it, not from the one the app w
as drawn with. */
let saved: Look | null = null;

/** The look the person has chosen: the last one saved, or else the one the app was drawn with. */
export function currentLook(): Look {
  return saved ?? { theme: appearance.theme, mode: appearance.mode, corners: appearance.corners };
}

/** What happened when a look was put on: nothing visible changed, the app is restarting to show it, or the restart did not work. */
export type LookResult = 'same' | 'reloading' | 'failed';

/**
 * Saves a look and, if it changes what is on screen, restarts the app so every screen is drawn again with it.
 * Before a restart it sets the "restyled" flag (the root layout then says the new look is live, and the session survives the restart)
 * and waits a moment for the store to save to disk.
 */
export function commitLook(look: Look): Promise<LookResult> {
  const reload = lookNeedsReload(look, { theme: appearance.theme, mode: appearance.mode, corners: appearance.corners, resolved: resolved
Mode }, OsAppearance.getColorScheme() === 'dark');
  if (!reload) {
    applyAppearance(look); // only remembers the choice ("light" on a light phone and the like)
    saved = look;
    return Promise.resolve('same');
  }
  const { setRestyled } = useAppStore.getState();
  setRestyled(true);
  return new Promise((resolve) => {
    setTimeout(() => {
      if (applyAppearance(look)) return resolve('reloading');
      setRestyled(false);
      resolve('failed');
    }, 450);
  });
}
```

src/components/personalize/build.ts

```
/** The numbers behind the "Building your trip HQ" moment. Pure, so tests can check them. */
import { BUILD_LINES } from './options';

/** The number climbs for this long (ms), then holds briefly before the app is put on. The whole moment is about two seconds. */
export const BUILD_DURATION = 1800;
export const BUILD_TICK = 200;
export const BUILD_HOLD = 450;

/** How far along the build is after `ms`: quick at first, slowing toward 100 (a whole number from 0 to 100). */
export function buildPercent(ms: number): number {
  const t = Math.max(0, Math.min(1, ms / BUILD_DURATION));
  return Math.round(100 * (1 - (1 - t) ** 2));
}

/** How many of the status lines are ticked at a given percentage (a third each; the last one only at 100). */
export function linesDone(pct: number): number {
  return BUILD_LINES.filter((_, i) => pct >= Math.round(((i + 1) * 100) / BUILD_LINES.length)).length;
}
```

src/components/personalize/kit.tsx

```
/**
 * The survey's own building blocks. "Make it yours" repaints the whole screen in the look being picked (before it is applied),
 * so it cannot use the shared components, which read the app's colours once when they load. These draw the same things
 * (button with a plate, stamp, ticket, rule, segmented control) from a palette object (`pal`) instead.
 * In Settings, pass the app's own `colors` as `pal` and they look exactly like the shared ones.
 */
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useState, type ReactNode } from 'react';
import { Animated, Easing, StyleSheet, Text, View, type StyleProp, type ViewStyle } from 'react-native';

import type { IconName } from '@/components/ui/bits';
import { nativeDriver, useMotion } from '@/components/ui/motion';
import { Plate, PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Perforation } from '@/components/ui/Rule';
import { T, type TProps } from '@/components/ui/T';
import { fonts, line, radius } from '@/theme';
```

```
import { luminance } from '@/utils/contrast';

import type { Pal } from './palette';

type Tone = 'text' | 'secondary' | 'muted' | 'accent';

/** Text in the survey's palette (not the app's). */
export function Tx({ pal, tone = 'text', ...rest }: TProps & { pal: Pal; tone?: Tone }) {
  const color = tone === 'text' ? pal.text : tone === 'secondary' ? pal.textSecondary : tone === 'muted' ? pal.textMuted : pal.accentTex
t;
  return <T color={color} {...rest} />;
}

/** The thumb of a switch that is off: the lightest colour of the palette, so it shows against the track in both the light and the dark
mode. */
export function switchThumb(pal: Pal): string {
  return luminance(pal.card) > luminance(pal.ink) ? pal.card : pal.ink;
}

/** A solid ink line, like a newspaper rule. */
export function PalRule({ pal, weight = line, style }: { pal: Pal; weight?: number; style?: StyleProp<ViewStyle> }) {
  return <View accessibilityElementsHidden importantForAccessibility="no" style={[{ height: weight, backgroundColor: pal.ink }, style]}
/>;
}

/** The big button at the bottom of a step: ink (cream at night) with the accent as its plate, 56 high, full width. */
export function PalButton({
  pal,
  label,
  onPress,
  icon,
  iconRight,
  variant = 'primary',
  disabled,
  testID,
  accessibilityLabel,
}: {
  pal: Pal;
  label: string;
  onPress: () => void;
  icon?: IconName;
  iconRight?: IconName;
  variant?: 'primary' | 'secondary';
  disabled?: boolean;
  testID?: string;
  accessibilityLabel?: string;
}) {
  const primary = variant === 'primary';
  const fg = primary ? pal.onPrimary : pal.text;
  return (
    <PlatePress
      testID={testID}
      onPress={onPress}
      disabled={disabled}
      accessibilityLabel={accessibilityLabel ?? label}
      accessibilityState={{ disabled: !!disabled }}
      color={primary ? pal.accent : pal.plate}
      corner={radius.button}
      wrapperStyle={[{ alignSelf: 'stretch' }, disabled && { opacity: 0.45 }]}
      style={[styles.button, { backgroundColor: primary ? pal.primary : pal.card, borderColor: pal.ink }]}>
      {icon ? <Ionicons name={icon} size={19} color={fg} /> : null}
      <T variant="title" weight="bold" color={fg}>
        {label}
      </T>
      {iconRight ? <Ionicons name={iconRight} size={19} color={fg} /> : null}
    </PlatePress>
  );
}

/** A square outlined icon button (back, close) with its own small plate. 44 x 44. */
export function PalIconButton({ pal, icon, onPress, label }: { pal: Pal; icon: IconName; onPress: () => void; label: string }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} color={pal.plate} style={[styles.iconButt
on, { backgroundColor: pal.card, borderColor: pal.ink }]}>
      <Ionicons name={icon} size={20} color={pal.ink} />
    </PlatePress>
  );
}

/** A row of choices in one outline (Light / Dark / System): the chosen one is filled with ink. */
export function PalSegmented<K extends string>({
  pal,
  options,
  value,
  onChange,
  label,
```

```
}: {
  pal: Pal;
  options: { key: K; label: string; icon?: IconName }[];
  value: K;
  onChange: (k: K) => void;
  label?: string;
}) {
  return (
    <View accessibilityRole="radiogroup" accessibilityLabel={label} style={[styles.segTrack, { backgroundColor: pal.card, borderColor: p
al.ink }]}>
      {options.map((o) => {
        const on = o.key === value;
        const fg = on ? pal.onPrimary : pal.text;
        return (
          <Press
            key={o.key}
            onPress={() => onChange(o.key)}
            accessibilityRole="radio"
            accessibilityState={{ checked: on }}
            accessibilityLabel={o.label}
            scaleTo={0.98}
            style={[styles.segItem, on && { backgroundColor: pal.primary }]}>
            {o.icon ? <Ionicons name={o.icon} size={16} color={fg} /> : null}
            <T variant="small" weight="bold" color={fg}>
              {o.label}
            </T>
          </Press>
        );
      })}
    </View>
  );
}

/** A small square that shows a choice is made: the accent with an ink tick (a sticker), so it is not told by colour alone. */
export function CheckBadge({ pal, size = 26 }: { pal: Pal; size?: number }) {
  return (
    <View
      accessibilityElementsHidden
      importantForAccessibility="no-hide-descendants"
      style={{ width: size, height: size, borderRadius: radius.xs, backgroundColor: pal.accent, borderWidth: line, borderColor: pal.ink,
alignItems: 'center', justifyContent: 'center' }}>
      <Ionicons name="checkmark" size={Math.round(size * 0.66)} color={pal.onAccent} />
    </View>
  );
}

export type PalTone = 'ink' | 'accent' | 'positive' | 'muted';

/** A rubber stamp drawn in the survey's palette. With `slam` it lands with a thud. */
export function PalStamp({ pal, label, tone = 'ink', rotate = -3, slam }: { pal: Pal; label: string; tone?: PalTone; rotate?: number; sl
am?: boolean }) {
  const c =
    tone === 'accent'
      ? { fg: pal.onAccent, bg: pal.accent, line: pal.ink }
      : tone === 'positive'
        ? { fg: pal.positive, bg: pal.positiveSoft, line: pal.positive }
        : tone === 'muted'
          ? { fg: pal.textSecondary, bg: 'transparent', line: pal.borderStrong }
          : { fg: pal.ink, bg: 'transparent', line: pal.ink };
  const motion = useMotion();
  const [t] = useState(() => new Animated.Value(slam && motion ? 0 : 1));
  useEffect(() => {
    if (!slam || !motion) return;
    t.setValue(0);
    const a = Animated.timing(t, { toValue: 1, duration: 260, easing: Easing.out(Easing.back(2)), useNativeDriver: nativeDriver });
    a.start();
    return () => a.stop();
  }, [slam, motion, t]);
  return (
    <Animated.View
      accessible
      accessibilityLabel={label}
      style={[
        styles.stamp,
        { borderColor: c.line, backgroundColor: c.bg, opacity: t, transform: [{ rotate: `${rotate}deg` }, { scale: t.interpolate({ input
Range: [0, 1], outputRange: [1.9, 1] }) }] },
      ]}>
      <T variant="kicker" color={c.fg} importantForAccessibility="no">
        {label}
      </T>
    </Animated.View>
  );
}

const NOTCH = 22;
```

```
/** Half a punched hole in the left or right edge of a ticket. */
function Notch({ side, pal }: { side: 'left' | 'right'; pal: Pal }) {
  const edge = side === 'left' ? { left: -line } : { right: -line };
  const pos = side === 'left' ? { left: -NOTCH / 2 } : { right: -NOTCH / 2 };
  return (
    <View pointerEvents="none" style={[styles.notchBox, edge]}>
      <View style={[styles.ring, pos, { backgroundColor: pal.ink }]} />
      <View style={[styles.hole, side === 'left' ? { left: -NOTCH / 2 + 2.5 } : { right: -NOTCH / 2 + 2.5 }, { backgroundColor: pal.bg
}]} />
    </View>
  );
}

/** A boarding pass in the survey's palette: a main part and, optionally, a stub below a perforated tear line. */
export function PalTicket({ pal, children, stub, plated = true, style }: { pal: Pal; children: ReactNode; stub?: ReactNode; plated?: boo
lean; style?: StyleProp<ViewStyle> }) {
  const card = (
    <View style={[styles.ticket, { backgroundColor: pal.card, borderColor: pal.ink }]}>
      {children}
      {stub ? (
        <>
          <View style={styles.tear}>
            <Notch side="left" pal={pal} />
            <Perforation style={styles.dashes} color={pal.ink} />
            <Notch side="right" pal={pal} />
          </View>
          {stub}
        </>
      ) : null}
    </View>
  );
  return plated ? (
    <Plate corner={radius.card} color={pal.plate} style={style}>
      {card}
    </Plate>
  ) : (
    <View style={style}>{card}</View>
  );
}

/** One tile of a departures board: a digit, white on ink, that flips when it changes (still when animations are off). */
function FlapTile({ pal, ch, size }: { pal: Pal; ch: string; size: number }) {
  const motion = useMotion();
  const [state, setState] = useState({ cur: ch, prev: ch });
  const [t] = useState(() => new Animated.Value(1));
  if (state.cur !== ch) setState({ cur: ch, prev: state.cur });
  useEffect(() => {
    if (!motion || state.prev === state.cur) {
      t.setValue(1);
      return;
    }
    t.setValue(0);
    const a = Animated.timing(t, { toValue: 1, duration: 200, easing: Easing.inOut(Easing.cubic), useNativeDriver: nativeDriver });
    a.start();
    return () => a.stop();
  }, [state, motion, t]);

  const face = { fontFamily: fonts.mono, fontSize: Math.round(size * 0.74), lineHeight: size, color: pal.bg, textAlign: 'center' as cons
t, includeFontPadding: false };
  const flipping = motion && state.prev !== state.cur;
  return (
    <View style={[styles.tile, { width: Math.round(size * 0.68), height: size, borderRadius: Math.round(size * 0.1), backgroundColor: pa
l.ink }]}>
      <Animated.View
        style={[
          StyleSheet.absoluteFill,
          { opacity: t.interpolate({ inputRange: [0, 0.49, 0.5, 1], outputRange: [0, 0, 1, 1] }), transform: [{ scaleY: t.interpolate({
inputRange: [0, 0.5, 1], outputRange: [0, 0, 1] }) }] },
        ]}>
        <Text style={face} allowFontScaling={false}>
          {ch}
        </Text>
      </Animated.View>
      {flipping ? (
        <Animated.View
          style={[
            StyleSheet.absoluteFill,
            { opacity: t.interpolate({ inputRange: [0, 0.49, 0.5, 1], outputRange: [1, 1, 0, 0] }), transform: [{ scaleY: t.interpolate
({ inputRange: [0, 0.5, 1], outputRange: [1, 0, 0] }) }] },
          ]}>
          <Text style={face} allowFontScaling={false}>
            {state.prev}
          </Text>
        </Animated.View>
      ) : null}
      <View pointerEvents="none" style={[styles.hinge, { top: size / 2 - 1, backgroundColor: pal.bg }]} />
```

```
    </View>
  );
}

/** A number on departure-board tiles, in the survey's palette. */
export function PalFlap({ pal, value, size = 64 }: { pal: Pal; value: string; size?: number }) {
  return (
    <View accessible accessibilityLabel={value} style={{ flexDirection: 'row', gap: Math.max(3, Math.round(size / 16)) }}>
      {value.split('').map((ch, i, all) => (
        <FlapTile key={all.length - i} pal={pal} ch={ch} size={size} />
      ))}
    </View>
  );
}

/** A chunky bar in an ink outline that fills with the accent. */
export function PalBar({ pal, value, height = 14 }: { pal: Pal; value: number; height?: number }) {
  const pct = Math.max(0, Math.min(1, value));
  return (
    <View
      accessibilityRole="progressbar"
      accessibilityValue={{ min: 0, max: 100, now: Math.round(pct * 100) }}
      style={{ height, borderRadius: radius.xs, backgroundColor: pal.card, borderWidth: line, borderColor: pal.ink, overflow: 'hidden'
}}>
      <View style={{ width: `${pct * 100}%`, height: '100%', backgroundColor: pal.accent, borderRightWidth: pct > 0 && pct < 1 ? line :
0, borderColor: pal.ink }} />
    </View>
  );
}

const styles = StyleSheet.create({
  button: { height: 56, borderRadius: radius.button, borderWidth: line, flexDirection: 'row', alignItems: 'center', justifyContent: 'cen
ter', gap: 8, paddingHorizontal: 22 },
  iconButton: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, alignItems: 'center', justifyContent: 'center' },
  segTrack: { flexDirection: 'row', borderRadius: radius.button, borderWidth: line, padding: 3, gap: 3 },
  segItem: { flex: 1, height: 44, borderRadius: Math.max(2, radius.button - 3), flexDirection: 'row', alignItems: 'center', justifyConte
nt: 'center', gap: 6 },
  stamp: { alignSelf: 'flex-start', borderWidth: line, borderRadius: radius.xs, paddingHorizontal: 8, paddingVertical: 3 },
  ticket: { borderWidth: line, borderRadius: radius.card },
  tear: { height: NOTCH, justifyContent: 'center' },
  dashes: { marginHorizontal: NOTCH / 2 + 4 },
  notchBox: { position: 'absolute', top: 0, width: NOTCH / 2 + line, height: NOTCH, overflow: 'hidden' },
  ring: { position: 'absolute', top: 0, width: NOTCH, height: NOTCH, borderRadius: NOTCH },
  hole: { position: 'absolute', top: 2.5, width: NOTCH - 5, height: NOTCH - 5, borderRadius: NOTCH },
  tile: { overflow: 'hidden', justifyContent: 'center' },
  hinge: { position: 'absolute', left: 0, right: 0, height: 2, opacity: 0.55 },
});
```

src/components/personalize/look.ts

```
/** Pure helpers for choosing a look (theme, light or dark, corners). No React Native in here, so tests can check it. */
import { resolveMode, themes, type Mode, type ModePref, type ThemeKey } from '@/theme/palettes';

export type Corners = 'rounded' | 'bubbly' | 'sharp';

export interface Look {
  theme: ThemeKey;
  mode: ModePref;
  corners: Corners;
}

/** Picking Grape switches to the dark mode; every other theme leaves the mode as it is. The person can still change it back. */
export function modeAfterPickingTheme(theme: ThemeKey, mode: ModePref): ModePref {
  return themes[theme].mode ?? mode;
}

/** Does putting on `next` mean the app has to restart (a different theme, corner style, or light or dark that actually shows)? */
export function lookNeedsReload(next: Look, current: Look & { resolved: Mode }, systemIsDark: boolean): boolean {
  return next.theme !== current.theme || next.corners !== current.corners || resolveMode(next.mode, systemIsDark) !== current.resolved;
}

export const MODE_LABEL: Record<ModePref, string> = { system: 'System', light: 'Light', dark: 'Dark' };

/** "Limoncello · System", for a summary line. */
export function lookLabel(look: Pick<Look, 'theme' | 'mode'>): string {
  return `${themes[look.theme].name} · ${MODE_LABEL[look.mode]}`;
}

/** Three letters of a word in capitals ("Limoncello" gives "LIM"). */
const tag = (word: string) =>
  word
    .replace(/[^a-z]/gi, '')
    .slice(0, 3)
    .toUpperCase();

/** A short code for a set of answers, printed on the stub of the last page like a booking reference: "LIM-PLA-LIV". */
```

```
export function styleCode(theme: ThemeKey, layoutName: string, effectsName: string): string {
  return [themes[theme].name, layoutName, effectsName].map(tag).join('-');
}

/**
 * Widths (1 to 3) of the bars and gaps of a barcode drawn from a code, so the same answers always print the same stripes.
 * Bars and gaps alternate, starting with a bar.
 */
export function barcodeWidths(code: string, count = 34): number[] {
  let seed = 17;
  for (let i = 0; i < code.length; i++) seed = (seed * 31 + code.charCodeAt(i)) % 2147483647;
  return Array.from({ length: count }, () => {
    seed = (seed * 48271) % 2147483647;
    return 1 + (seed % 3);
  });
}
```

src/components/personalize/options.ts

```
import type { IconName } from '@/components/ui/bits';
import type { Effects, HomeSection } from '@/store/types';
import type { ModePref } from '@/theme/palettes';

/** Faces to pick from for the person's own avatar (the one place emoji are allowed: they are the person's choice, not decoration). */
export const EMOJIS = ['✈️', '🌴', '🏔️', '🍜', '📸', '🎒', '🌸', '🏄', '🚂', '🗺️', '🌙', '🦋'];

/** The numbered steps. The page after them is the summary with the build button. */
export const STEPS = ['Welcome', 'Look', 'Interests', 'Layout', 'Vibe'] as const;
export const STEP = { welcome: 0, look: 1, interests: 2, layout: 3, vibe: 4, summary: STEPS.length } as const;

export const MODES: { key: ModePref; label: string; icon: IconName }[] = [
  { key: 'light', label: 'Light', icon: 'sunny-outline' },
  { key: 'dark', label: 'Dark', icon: 'moon-outline' },
  { key: 'system', label: 'System', icon: 'phone-portrait-outline' },
];

/** The three Effects settings. `bars` is how many of the three intensity bars are lit. */
export const EFFECT_OPTIONS: { key: Effects; label: string; sub: string; bars: 1 | 2 | 3 }[] = [
  { key: 'chill', label: 'Chill', sub: 'No confetti, just a calm tick', bars: 1 },
  { key: 'lively', label: 'Lively', sub: 'A little confetti when it counts', bars: 2 },
  { key: 'max', label: 'Max', sub: 'Full party. Confetti everywhere', bars: 3 },
];

export const EFFECT_LABEL: Record<Effects, string> = { chill: 'Chill', lively: 'Lively', max: 'Max' };

/** What the little Home drawings say (the real names are in HOME_BLOCKS). */
export const BLOCK_SHORT: Record<HomeSection, string> = {
  upnext: 'Up next',
  weather: 'Weather',
  budget: 'Budget',
  crew: 'Crew',
  play: 'Play',
  forYou: 'For you',
  nearby: 'Nearby',
  discover: 'Discover',
};

/** The lines of the "Building your trip HQ" moment, in order. */
export const BUILD_LINES = ['Picking your colours', 'Arranging Home', 'Tuning the confetti'] as const;

export const BUDGET_OPTIONS: { key: 'saver' | 'mid' | 'treat'; label: string }[] = [
  { key: 'saver', label: 'Saver' },
  { key: 'mid', label: 'Mid-range' },
  { key: 'treat', label: 'Treat yo self' },
];
```

src/components/personalize/palette.ts

```
import { paletteOf, type Colors } from '@/theme';

/** The colours the survey draws itself with: those of the theme and mode being picked, not of the look the app has now. */
export type Pal = Colors;

export { paletteOf };
```

src/components/personalize/parts.tsx

```
/**
 * Building blocks of "Make it yours" (and of the Look card in Settings).
 * Everything here draws itself with a palette object (`pal`) instead of the global colours, so the survey can repaint itself
 * live in a theme that is not applied yet. In Settings, pass the app's own `colors` as `pal`.
 */
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useMemo, useState } from 'react';
import { Animated, StyleSheet, Switch, View } from 'react-native';
```

```
import type { IconName } from '@/components/ui/bits';
import { nativeDriver, useMotion } from '@/components/ui/motion';
import { Plate } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { LAYOUTS, layoutAnswers, visibleSections, type Preset } from '@/data/layouts';
import type { Effects, Personal } from '@/store/types';
import { hair, line, radius, themes, type ModePref, type ThemeKey } from '@/theme';

import { CheckBadge, PalSegmented, Tx, switchThumb } from './kit';
import { BLOCK_SHORT, MODES } from './options';
import { paletteOf, type Pal } from './palette';

/** Updates part of the survey answers. */
export type SetPersonal = (patch: Partial<Personal>) => void;

/** The small line over a step's title: a bullet in the accent, then "02 / 05 · Look" in mono caps. */
export function StepKicker({ pal, text }: { pal: Pal; text: string }) {
  return (
    <View style={styles.kickerRow}>
      <View style={[styles.bullet, { backgroundColor: pal.accent, borderColor: pal.ink }]} />
      <Tx pal={pal} tone="secondary" variant="kicker">
        {text}
      </Tx>
    </View>
  );
}

/** A step's title in heavy serif, with its kicker above and one line under it. */
export function StepHeading({ pal, kicker, title, sub }: { pal: Pal; kicker: string; title: string; sub: string }) {
  return (
    <View style={{ gap: 10 }}>
      <StepKicker pal={pal} text={kicker} />
      <T variant="display" color={pal.text} accessibilityRole="header">
        {title}
      </T>
      <Tx pal={pal} tone="secondary" variant="body">
        {sub}
      </Tx>
    </View>
  );
}

/** "Step 2 of 5" as five ticket stubs in a row: done ones are inked, the one you are on is the accent (goal-gradient: the end is in sig
ht). */
export function StepBar({ pal, step, total }: { pal: Pal; step: number; total: number }) {
  const now = Math.min(step + 1, total);
  return (
    <View accessibilityRole="progressbar" accessibilityLabel={`Step ${now} of ${total}`} accessibilityValue={{ min: 1, max: total, now
}} style={{ flexDirection: 'row', gap: 5 }}>
      {Array.from({ length: total }, (_, i) => (
        <View key={i} style={[styles.seg, { borderColor: pal.ink, backgroundColor: i < step || step >= total ? pal.ink : i === step ? pa
l.accent : 'transparent' }]} />
      ))}
    </View>
  );
}

/** Three dashes and a block: a tiny boarding pass drawn in one theme's own colours. */
export function MiniTicket({ tp }: { tp: Pal }) {
  return (
    <View accessibilityElementsHidden importantForAccessibility="no-hide-descendants" style={[styles.mini, { backgroundColor: tp.card, b
orderColor: tp.ink }]}>
      <View style={styles.miniRow}>
        <View style={{ flex: 1, gap: 4 }}>
          <View style={{ height: 5, width: '72%', backgroundColor: tp.ink }} />
          <View style={{ height: 5, width: '44%', backgroundColor: tp.textMuted }} />
        </View>
        <View style={[styles.miniStamp, { backgroundColor: tp.accent, borderColor: tp.ink }]} />
      </View>
      <View style={styles.miniDashes}>
        {Array.from({ length: 12 }, (_, i) => (
          <View key={i} style={{ width: 5, height: 2, backgroundColor: tp.ink }} />
        ))}
      </View>
      <View style={[styles.miniBar, { borderColor: tp.ink, backgroundColor: tp.card }]}>
        <View style={{ width: '58%', height: '100%', backgroundColor: tp.accent }} />
      </View>
    </View>
  );
}

/**
 * One theme as a swatch: a flat field of its accent colour with a tiny ticket lying on it, in that theme's own paper and ink,
 * and the name under it. The chosen one stands up on a plate and gets a tick (so it is not told by colour alone).
 */
```

```
export function ThemeTile({ pal, theme, mode, selected, onPress }: { pal: Pal; theme: ThemeKey; mode: ModePref; selected: boolean; onPre
ss: () => void }) {
  const tp = useMemo(() => paletteOf(theme, mode), [theme, mode]);
  const t = themes[theme];
  return (
    <Plate corner={radius.card} color={selected ? pal.plate : 'transparent'} style={{ flex: 1 }}>
      <Press
        onPress={onPress}
        accessibilityRole="radio"
        accessibilityState={{ checked: selected }}
        accessibilityLabel={`${t.name} theme`}
        accessibilityHint={t.vibe}
        scaleTo={0.97}
        style={[styles.themeTile, { borderColor: pal.ink, backgroundColor: pal.card }]}>
        <View style={[styles.themeField, { backgroundColor: tp.accent, borderBottomColor: pal.ink }]}>
          <MiniTicket tp={tp} />
        </View>
        <View style={styles.themeFoot}>
          <T variant="title" weight="bold" color={pal.text} numberOfLines={1} style={{ flex: 1 }}>
            {t.name}
          </T>
          {selected ? <CheckBadge pal={pal} size={24} /> : null}
        </View>
      </Press>
    </Plate>
  );
}

/** The six themes, two to a row. */
export function ThemeGrid({ pal, theme, mode, onPick }: { pal: Pal; theme: ThemeKey; mode: ModePref; onPick: (k: ThemeKey) => void }) {
  const keys = Object.keys(themes) as ThemeKey[];
  const rows: ThemeKey[][] = [];
  for (let i = 0; i < keys.length; i += 2) rows.push(keys.slice(i, i + 2));
  return (
    <View accessibilityRole="radiogroup" accessibilityLabel="Colour theme" style={{ gap: 10 }}>
      {rows.map((row) => (
        <View key={row.join()} style={{ flexDirection: 'row', gap: 10 }}>
          {row.map((k) => (
            <ThemeTile key={k} pal={pal} theme={k} mode={mode} selected={theme === k} onPress={() => onPick(k)} />
          ))}
        </View>
      ))}
    </View>
  );
}

/** A flat square of one theme's accent colour (for Settings): 44 x 44, with a tick when chosen. */
export function ThemeSwatch({ pal, theme, mode, selected, onPress }: { pal: Pal; theme: ThemeKey; mode: ModePref; selected: boolean; onP
ress: () => void }) {
  const tp = useMemo(() => paletteOf(theme, mode), [theme, mode]);
  const t = themes[theme];
  return (
    <Plate offset={3} corner={radius.button} color={selected ? pal.plate : 'transparent'}>
      <Press
        onPress={onPress}
        accessibilityRole="radio"
        accessibilityState={{ checked: selected }}
        accessibilityLabel={`${t.name} theme`}
        accessibilityHint={t.vibe}
        scaleTo={0.94}
        style={[styles.swatch, { backgroundColor: tp.accent, borderColor: pal.ink }]}>
        {selected ? <Ionicons name="checkmark" size={24} color={tp.onAccent} /> : null}
      </Press>
    </Plate>
  );
}

/** Light / Dark / System. */
export function ModePicker({ pal, value, onChange }: { pal: Pal; value: ModePref; onChange: (m: ModePref) => void }) {
  return <PalSegmented pal={pal} options={MODES} value={value} onChange={onChange} label="Light or dark" />;
}

/** A tiny drawing of the Home order: the boarding pass on top (always there), then the blocks. The first block is the loud one (accen
t). */
export function HomeWire({ pal, order, sections }: { pal: Pal; order: ReturnType<typeof layoutAnswers>['order']; sections: ReturnType<ty
peof layoutAnswers>['sections'] }) {
  const blocks = visibleSections({ order, sections }).slice(0, 4);
  return (
    <View style={styles.wire} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
      <View style={[styles.wireHero, { backgroundColor: pal.ink }]}>
        <View style={{ width: 22, height: 8, backgroundColor: pal.bg }} />
      </View>
      {blocks.map((k, i) => (
        <View key={k} style={[styles.wireBlock, { backgroundColor: i === 0 ? pal.accent : pal.card, borderColor: pal.ink }]}>
          <T variant="micro" weight="bold" color={i === 0 ? pal.onAccent : pal.text} numberOfLines={1}>
            {BLOCK_SHORT[k]}
```

```
          </T>
        </View>
      ))}
    </View>
  );
}

/** One ready-made Home layout, as a thumbnail of its order. The chosen one stands up on a plate, its name is highlighted and it has a t
ick. */
export function LayoutCard({ pal, preset, selected, onPress }: { pal: Pal; preset: Preset; selected: boolean; onPress: () => void }) {
  const l = LAYOUTS[preset];
  const a = useMemo(() => layoutAnswers(preset), [preset]);
  return (
    <Plate corner={radius.card} color={selected ? pal.plate : 'transparent'} style={{ flex: 1 }}>
      <Press
        onPress={onPress}
        accessibilityRole="radio"
        accessibilityState={{ checked: selected }}
        accessibilityLabel={`${l.name} layout`}
        accessibilityHint={l.blurb}
        scaleTo={0.97}
        style={[styles.layoutCard, { backgroundColor: pal.card, borderColor: pal.ink }]}>
        <HomeWire pal={pal} order={a.order} sections={a.sections} />
        {selected ? (
          <View style={styles.layoutTick}>
            <CheckBadge pal={pal} size={20} />
          </View>
        ) : null}
        <View style={[styles.layoutFoot, { borderTopColor: pal.ink, backgroundColor: selected ? pal.accent : pal.card }]}>
          <T variant="small" weight="bold" color={selected ? pal.onAccent : pal.text} numberOfLines={1}>
            {l.name}
          </T>
        </View>
      </Press>
    </Plate>
  );
}

/** Three bars that rise, with `lit` of them filled: how big the celebrations are, drawn instead of an emoji. */
export function Meter({ pal, lit }: { pal: Pal; lit: number }) {
  return (
    <View accessibilityElementsHidden importantForAccessibility="no-hide-descendants" style={styles.meter}>
      {[0, 1, 2].map((i) => (
        <View key={i} style={{ width: 9, height: 10 + i * 8, borderWidth: line, borderColor: pal.ink, backgroundColor: i < lit ? pal.acc
ent : 'transparent' }} />
      ))}
    </View>
  );
}

/** One Effects setting: its meter, a name and one line. The chosen one stands up on a plate with a tick. */
export function EffectRow({ pal, level, on, bars, label, sub, onPress }: { pal: Pal; level: Effects; on: boolean; bars: number; label: s
tring; sub: string; onPress: () => void }) {
  return (
    <Plate corner={radius.card} color={on ? pal.plate : 'transparent'}>
      <Press
        onPress={onPress}
        testID={`effects-${level}`}
        accessibilityRole="radio"
        accessibilityState={{ checked: on }}
        accessibilityLabel={label}
        accessibilityHint={sub}
        scaleTo={0.98}
        style={[styles.option, { backgroundColor: pal.card, borderColor: pal.ink }]}>
        <Meter pal={pal} lit={bars} />
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold" color={pal.text}>
            {label}
          </T>
          <Tx pal={pal} tone="secondary" variant="small">
            {sub}
          </Tx>
        </View>
        {on ? <CheckBadge pal={pal} /> : <View style={[styles.emptyBox, { borderColor: pal.textMuted }]} />}
      </Press>
    </Plate>
  );
}

/** A flat card with one switch (a daily reminder). */
export function SwitchRow({ pal, icon, label, sub, value, onChange }: { pal: Pal; icon: IconName; label: string; sub: string; value: boo
lean; onChange: (v: boolean) => void }) {
  return (
    <View style={[styles.switchRow, { backgroundColor: pal.card, borderColor: pal.ink }]}>
      <View style={[styles.switchIcon, { borderColor: pal.ink, backgroundColor: pal.bgAlt }]}>
        <Ionicons name={icon} size={20} color={pal.ink} />
```

```
      </View>
      <View style={{ flex: 1 }}>
        <T variant="title" weight="bold" color={pal.text}>
          {label}
        </T>
        <Tx pal={pal} tone="secondary" variant="small">
          {sub}
        </Tx>
      </View>
      <Switch
        accessibilityLabel={label}
        value={value}
        onValueChange={onChange}
        trackColor={{ false: pal.borderStrong, true: pal.ink }}
        thumbColor={switchThumb(pal)}
        {...({ activeThumbColor: pal.accent } as object)}
      />
    </View>
  );
}

/** One answer on the last page: a small caps label over the answer. Tapping the row opens the step where it can be changed. */
export function SummaryRow({ pal, label, value, onPress, last }: { pal: Pal; label: string; value: string; onPress: () => void; last?: b
oolean }) {
  return (
    <Press
      onPress={onPress}
      scaleTo={0.99}
      accessibilityLabel={`Change ${label.toLowerCase()}, now ${value}`}
      style={[styles.summary, !last && { borderBottomWidth: hair, borderBottomColor: pal.border }]}>
      <View style={{ flex: 1, gap: 2 }}>
        <Tx pal={pal} tone="secondary" variant="kicker">
          {label}
        </Tx>
        <T variant="title" weight="bold" color={pal.text} numberOfLines={2}>
          {value}
        </T>
      </View>
      <Ionicons name="create-outline" size={20} color={pal.ink} />
    </Press>
  );
}

/**
 * Hides the hard cut when the survey repaints in a new theme: the old page colour sits on top for an instant and fades away
 * (opacity only, 240 ms), so the screen dissolves into the new look instead of jumping. It never takes a tap.
 */
export function PaletteVeil({ pal, signature }: { pal: Pal; signature: string }) {
  const motion = useMotion();
  const [state, setState] = useState<{ cur: { sig: string; pal: Pal }; prev: Pal | null; n: number }>({ cur: { sig: signature, pal }, pr
ev: null, n: 0 });
  const [v] = useState(() => new Animated.Value(0));
  // when the look changes, keep the old colours around to fade them out
  if (state.cur.sig !== signature) setState({ cur: { sig: signature, pal }, prev: state.cur.pal, n: state.n + 1 });
  const { n } = state;

  useEffect(() => {
    if (!motion || n === 0) return;
    v.setValue(0.85);
    const a = Animated.timing(v, { toValue: 0, duration: 240, useNativeDriver: nativeDriver });
    a.start();
    return () => a.stop();
  }, [n, motion, v]);

  if (!motion || !state.prev) return null;
  return <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { opacity: v, backgroundColor: state.prev.bg }]} />;
}

const styles = StyleSheet.create({
  kickerRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  bullet: { width: 10, height: 10, borderWidth: 1.5 },
  seg: { flex: 1, height: 14, borderWidth: line, borderRadius: 2 },
  mini: { borderWidth: line, borderRadius: radius.xs + 2, padding: 6, gap: 5 },
  miniRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  miniStamp: { width: 20, height: 14, borderWidth: 1.5 },
  miniDashes: { flexDirection: 'row', justifyContent: 'space-between', overflow: 'hidden' },
  miniBar: { height: 7, borderWidth: 1.5, overflow: 'hidden' },
  themeTile: { borderWidth: line, borderRadius: radius.card, overflow: 'hidden' },
  themeField: { padding: 10, borderBottomWidth: line },
  themeFoot: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 44, paddingHorizontal: 10 },
  swatch: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, alignItems: 'center', justifyContent: 'center' },
  wire: { padding: 8, gap: 4 },
  wireHero: { height: 24, borderRadius: radius.xs, alignItems: 'flex-start', justifyContent: 'flex-end', padding: 4 },
  wireBlock: { height: 26, borderWidth: 1.5, borderRadius: radius.xs, justifyContent: 'center', paddingHorizontal: 6 },
  layoutCard: { borderWidth: line, borderRadius: radius.card, overflow: 'hidden' },
  layoutFoot: { alignItems: 'center', justifyContent: 'center', minHeight: 44, paddingHorizontal: 6, borderTopWidth: line },
```

```
  layoutTick: { position: 'absolute', top: 12, right: 12 },
  meter: { flexDirection: 'row', alignItems: 'flex-end', gap: 3, height: 34, width: 39 },
  option: { flexDirection: 'row', alignItems: 'center', gap: 14, paddingHorizontal: 16, paddingVertical: 14, minHeight: 76, borderRadiu
s: radius.card, borderWidth: line },
  emptyBox: { width: 26, height: 26, borderRadius: radius.xs, borderWidth: line },
  switchRow: { flexDirection: 'row', alignItems: 'center', gap: 14, paddingHorizontal: 16, paddingVertical: 14, minHeight: 76, borderRad
ius: radius.card, borderWidth: line },
  switchIcon: { width: 40, height: 40, borderRadius: radius.button, borderWidth: line, alignItems: 'center', justifyContent: 'center' },
  summary: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 64, paddingVertical: 10 },
});
```

src/components/personalize/steps/StepInterests.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { useState } from 'react';
import { Animated, StyleSheet, View } from 'react-native';

import { FadeIn, nativeDriver, useMotion } from '@/components/ui/motion';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { pickedForYou } from '@/components/home/picks';
import { photos } from '@/data/images';
import { INTERESTS, INTEREST_GOAL, interestHint } from '@/data/personal';
import type { Personal } from '@/store/types';
import { line, radius } from '@/theme';

import { CheckBadge, Tx } from '../kit';
import type { Pal } from '../palette';
import { StepHeading, type SetPersonal } from '../parts';

type Interest = (typeof INTERESTS)[number];

/** A photo tile. Chosen = a thick accent frame, a tick and a little bounce. The label sits on a scrim so it reads on any photo. */
function Tile({ pal, it, on, index, onPress }: { pal: Pal; it: Interest; on: boolean; index: number; onPress: () => void }) {
  const motion = useMotion();
  const [bounce] = useState(() => new Animated.Value(1));
  const press = () => {
    onPress();
    if (on || !motion) return;
    Animated.sequence([
      Animated.timing(bounce, { toValue: 1.06, duration: 90, useNativeDriver: nativeDriver }),
      Animated.spring(bounce, { toValue: 1, friction: 4, tension: 170, useNativeDriver: nativeDriver }),
    ]).start();
  };
  return (
    <FadeIn delay={Math.min(index, 5) * 40} distance={12} duration={240} style={{ width: '48%' }}>
      <Animated.View style={{ transform: [{ scale: bounce }] }}>
        <Press
          onPress={press}
          accessibilityRole="checkbox"
          accessibilityState={{ checked: on }}
          accessibilityLabel={it.label}
          accessibilityHint={it.sub}
          scaleTo={0.97}
          style={[styles.tile, { borderColor: pal.ink }]}>
          <Image source={photos[it.image]} style={StyleSheet.absoluteFill} contentFit="cover" />
          <LinearGradient colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.72)']} style={styles.scrim} />
          {on ? <View pointerEvents="none" style={[StyleSheet.absoluteFill, styles.frame, { borderColor: pal.accent }]} /> : null}
          <View style={styles.badge}>{on ? <CheckBadge pal={pal} size={28} /> : <View style={styles.badgeOff} />}</View>
          <View style={styles.label}>
            <Ionicons name={it.icon} size={19} color={pal.white} />
            <T variant="title" weight="bold" color={pal.white}>
              {it.label}
            </T>
          </View>
        </Press>
      </Animated.View>
    </FadeIn>
  );
}

/** Step 3: eight photo tiles. Pick 3 or more; picking none is fine and never blocks. */
export function StepInterests({ pal, p, set }: { pal: Pal; p: Personal; set: SetPersonal }) {
  const toggle = (k: string) => set({ interests: p.interests.includes(k) ? p.interests.filter((x) => x !== k) : [...p.interests, k] });
  const n = p.interests.length;
  // what the picks do: the first places "Picked for you" on Home would show right now
  const first = pickedForYou(p)
    .slice(0, 3)
    .map((x) => x.d.city);
  return (
    <View style={{ gap: 18 }}>
      <StepHeading pal={pal} kicker="03 / 05 · Interests" title="What do you love?" sub="We use these for Picked for you on Home and for
the daily trivia." />
      <View style={styles.hintRow} accessibilityLiveRegion="polite">
```

```
        <View style={{ flexDirection: 'row', gap: 5 }} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
          {Array.from({ length: INTEREST_GOAL }, (_, i) => (
            <View key={i} style={[styles.pip, { borderColor: pal.ink, backgroundColor: i < n ? pal.accent : 'transparent' }]} />
          ))}
        </View>
        <Tx pal={pal} tone="secondary" variant="kicker">
          {interestHint(n)}
        </Tx>
      </View>
      {first.length ? (
        <Tx pal={pal} tone="secondary" variant="small" accessibilityLiveRegion="polite" style={{ marginTop: -8 }}>
          On Home you would see {first.join(', ')}.
        </Tx>
      ) : null}
      <View style={styles.grid}>
        {INTERESTS.map((it, i) => (
          <Tile key={it.key} pal={pal} it={it} on={p.interests.includes(it.key)} index={i} onPress={() => toggle(it.key)} />
        ))}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  grid: { flexDirection: 'row', flexWrap: 'wrap', justifyContent: 'space-between', rowGap: 12 },
  tile: { height: 136, borderRadius: radius.card, borderWidth: line, overflow: 'hidden', justifyContent: 'flex-end' },
  scrim: { position: 'absolute', left: 0, right: 0, bottom: 0, height: 80 },
  frame: { borderWidth: 5, borderRadius: radius.card },
  badge: { position: 'absolute', top: 10, right: 10 },
  badgeOff: { width: 28, height: 28, borderRadius: radius.xs, borderWidth: line, borderColor: 'rgba(255,255,255,0.9)', backgroundColor:
'rgba(0,0,0,0.25)' },
  label: { flexDirection: 'row', alignItems: 'center', gap: 8, padding: 12 },
  hintRow: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 24 },
  pip: { width: 14, height: 14, borderWidth: line, borderRadius: 3 },
});
```

src/components/personalize/steps/StepLayout.tsx

```
import { View } from 'react-native';

import { FadeIn } from '@/components/ui/motion';
import { T } from '@/components/ui/T';
import { LAYOUTS, layoutAnswers, type Preset } from '@/data/layouts';
import type { Personal } from '@/store/types';

import { PalRule, Tx } from '../kit';
import type { Pal } from '../palette';
import { LayoutCard, StepHeading, type SetPersonal } from '../parts';

const PRESETS = Object.keys(LAYOUTS) as Preset[];

/** Step 4: three thumbnails of the Home order. Picking one sets the order, which blocks are on, and the layout. */
export function StepLayout({ pal, p, set }: { pal: Pal; p: Personal; set: SetPersonal }) {
  const chosen = p.layout !== 'custom' ? LAYOUTS[p.layout] : null;
  return (
    <View style={{ gap: 20 }}>
      <StepHeading pal={pal} kicker="04 / 05 · Layout" title="Shape your Home" sub="The boarding pass is always on top. Pick what comes
after it." />
      <View accessibilityRole="radiogroup" accessibilityLabel="Home layout" style={{ flexDirection: 'row', gap: 10 }}>
        {PRESETS.map((k, i) => (
          <FadeIn key={k} delay={i * 50} distance={12} duration={240} style={{ flex: 1 }}>
            <LayoutCard pal={pal} preset={k} selected={p.layout === k} onPress={() => set(layoutAnswers(k))} />
          </FadeIn>
        ))}
      </View>
      <View style={{ gap: 10 }} accessibilityLiveRegion="polite">
        <PalRule pal={pal} />
        <T variant="headline" color={pal.text}>
          {chosen ? chosen.name : 'Custom'}
        </T>
        <Tx pal={pal} tone="secondary" variant="body">
          {chosen ? chosen.blurb : 'Arranged by hand in Settings. Pick a layout to start over, or keep yours and tap Continue.'}
        </Tx>
        <Tx pal={pal} tone="muted" variant="caption">
          Hide or move any block later in Settings.
        </Tx>
      </View>
    </View>
  );
}
```

src/components/personalize/steps/StepLook.tsx

```
import { View } from 'react-native';

import { themes, type ModePref, type ThemeKey } from '@/theme';
```

```
import { Tx } from '../kit';
import type { Pal } from '../palette';
import { ModePicker, StepHeading, ThemeGrid } from '../parts';

/** Step 2: six themes and Light / Dark / System. Picking one repaints this whole screen, so the person sees the answer at once. */
export function StepLook({ pal, theme, mode, onTheme, onMode }: { pal: Pal; theme: ThemeKey; mode: ModePref; onTheme: (k: ThemeKey) => v
oid; onMode: (m: ModePref) => void }) {
  return (
    <View style={{ gap: 20 }}>
      <StepHeading pal={pal} kicker="02 / 05 · Look" title="Pick your look" sub="Tap one and watch this screen repaint." />
      <ThemeGrid pal={pal} theme={theme} mode={mode} onPick={onTheme} />
      <Tx pal={pal} tone="secondary" variant="small" accessibilityLiveRegion="polite">
        {themes[theme].name}: {themes[theme].vibe.toLowerCase()}.
      </Tx>
      <View style={{ gap: 8 }}>
        <Tx pal={pal} tone="secondary" variant="kicker">
          Light or dark
        </Tx>
        <ModePicker pal={pal} value={mode} onChange={onMode} />
        <Tx pal={pal} tone="muted" variant="caption">
          System follows your phone. Grape picks Dark.
        </Tx>
      </View>
    </View>
  );
}
```

src/components/personalize/steps/StepReveal.tsx

```
import { StyleSheet, View } from 'react-native';

import { StaggerText } from '@/components/ui/StaggerText';
import { LAYOUTS } from '@/data/layouts';
import { TRIVIA_REMINDER, interestNames } from '@/data/personal';
import type { Personal } from '@/store/types';
import type { ModePref, ThemeKey } from '@/theme';

import { PalStamp, PalTicket, Tx } from '../kit';
import { barcodeWidths, lookLabel, styleCode } from '../look';
import { EFFECT_LABEL, STEP } from '../options';
import type { Pal } from '../palette';
import { StepKicker, SummaryRow } from '../parts';

/** The stripes of a barcode, drawn from the code so the same answers always print the same ones. */
function Barcode({ pal, code }: { pal: Pal; code: string }) {
  return (
    <View accessibilityElementsHidden importantForAccessibility="no-hide-descendants" style={styles.barcode}>
      {barcodeWidths(code).map((w, i) => (
        <View key={i} style={{ width: w * 2, height: '100%', backgroundColor: i % 2 === 0 ? pal.ink : 'transparent' }} />
      ))}
    </View>
  );
}

/** The last page: every answer on a boarding pass (tap a row to change it), and the build button below. */
export function StepReveal({ pal, p, theme, mode, shownName, onEdit }: { pal: Pal; p: Personal; theme: ThemeKey; mode: ModePref; shownNa
me: string; onEdit: (step: number) => void }) {
  const layoutName = p.layout === 'custom' ? 'Custom' : LAYOUTS[p.layout].name;
  const code = styleCode(theme, layoutName, EFFECT_LABEL[p.effects]);
  const rows = [
    { label: 'Name', value: `${p.emoji}  ${shownName}`, step: STEP.welcome },
    { label: 'Look', value: lookLabel({ theme, mode }), step: STEP.look },
    { label: 'Interests', value: interestNames(p.interests), step: STEP.interests },
    { label: 'Home layout', value: layoutName, step: STEP.layout },
    { label: 'Effects', value: `${EFFECT_LABEL[p.effects]}${p.triviaReminder ? ` · trivia at ${TRIVIA_REMINDER.timeLabel}` : ''}`, step:
STEP.vibe },
  ] as const;
  return (
    <View style={{ gap: 22 }}>
      <View style={{ gap: 10 }}>
        <StepKicker pal={pal} text="Ready to board" />
        <StaggerText text={`All set, ${shownName}`} variant="display" color={pal.text} />
        <Tx pal={pal} tone="secondary" variant="body">
          Here is your pass. Tap any line to change it.
        </Tx>
      </View>
      <PalTicket
        pal={pal}
        stub={
          <View style={styles.stub}>
            <Barcode pal={pal} code={code} />
            <Tx pal={pal} variant="numeralSm">
              {code}
            </Tx>
          </View>
```

```
        }>
        <View style={styles.main}>
          <View style={styles.top}>
            <Tx pal={pal} tone="secondary" variant="kicker" style={{ flex: 1 }}>
              Crew pass
            </Tx>
            <PalStamp pal={pal} label="Ready" tone="accent" slam />
          </View>
          {rows.map((r, i) => (
            <SummaryRow key={r.label} pal={pal} label={r.label} value={r.value} onPress={() => onEdit(r.step)} last={i === rows.length -
1} />
          ))}
        </View>
      </PalTicket>
      <Tx pal={pal} tone="muted" variant="caption">
        Language, home currency and corner style are in Settings.
      </Tx>
    </View>
  );
}

const styles = StyleSheet.create({
  main: { paddingHorizontal: 16, paddingTop: 14 },
  top: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingBottom: 6 },
  stub: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 14, paddingHorizontal: 16, paddingBottom: 1
6, paddingTop: 2 },
  barcode: { flexDirection: 'row', height: 36, flexShrink: 1, overflow: 'hidden' },
});
```

src/components/personalize/steps/StepVibe.tsx

```
import { View } from 'react-native';

import { celebrate } from '@/components/ui/Confetti';
import { TRIVIA_REMINDER } from '@/data/personal';
import type { Personal } from '@/store/types';

import { Tx } from '../kit';
import { EFFECT_OPTIONS } from '../options';
import type { Pal } from '../palette';
import { EffectRow, StepHeading, SwitchRow, type SetPersonal } from '../parts';

/** Step 5: how big the celebrations are (tap to try one) and the daily trivia reminder. */
export function StepVibe({ pal, p, set, onTrivia }: { pal: Pal; p: Personal; set: SetPersonal; onTrivia: (on: boolean) => void }) {
  return (
    <View style={{ gap: 20 }}>
      <StepHeading pal={pal} kicker="05 / 05 · Vibe" title="Set the vibe" sub="How big should celebrations be? Tap one to try it." />
      <View accessibilityRole="radiogroup" accessibilityLabel="Effects" style={{ gap: 12 }}>
        {EFFECT_OPTIONS.map((o) => (
          <EffectRow
            key={o.key}
            pal={pal}
            level={o.key}
            on={p.effects === o.key}
            bars={o.bars}
            label={o.label}
            sub={o.sub}
            onPress={() => {
              set({ effects: o.key });
              // a preview of this setting (it is not saved until the end); Chill has no confetti, so it says so
              celebrate(o.key === 'chill' ? 'Chill: a calm tick, no confetti' : undefined, o.key);
            }}
          />
        ))}
      </View>
      <View style={{ gap: 8 }}>
        <SwitchRow
          pal={pal}
          icon="notifications-outline"
          label="Daily trivia reminder"
          sub={`One question a day at ${TRIVIA_REMINDER.timeLabel}. Keep your streak going.`}
          value={p.triviaReminder}
          onChange={onTrivia}
        />
        <Tx pal={pal} tone="muted" variant="caption">
          Off by default. You can change both in Settings.
        </Tx>
      </View>
    </View>
  );
}
```

src/components/personalize/steps/StepWelcome.tsx

```
import { useState } from 'react';
import { StyleSheet, TextInput, View } from 'react-native';
```

```
import { Press } from '@/components/ui/Press';
import { StaggerText } from '@/components/ui/StaggerText';
import { T } from '@/components/ui/T';
import type { Personal } from '@/store/types';
import { fonts, hair, line, radius } from '@/theme';

import { PalStamp, PalTicket, Tx } from '../kit';
import { EMOJIS } from '../options';
import type { Pal } from '../palette';
import { StepKicker, type SetPersonal } from '../parts';

/**
 * Step 1: the promise, then a boarding pass to fill in. The name is typed on the pass; the avatar (the one place emoji are allowed,
 * because the person picks it) is chosen on the stub.
 */
export function StepWelcome({ pal, p, set, firstName, onSubmit }: { pal: Pal; p: Personal; set: SetPersonal; firstName: string; onSubmi
t: () => void }) {
  const [focused, setFocused] = useState(false);
  return (
    <View style={{ gap: 24 }}>
      <View style={{ gap: 10 }}>
        <StepKicker pal={pal} text="01 / 05 · Welcome" />
        <StaggerText text="Let’s make it yours" variant="display" color={pal.text} />
        <Tx pal={pal} tone="secondary" variant="body">
          4 quick picks, about a minute. Skip any of them and change them later in Settings.
        </Tx>
      </View>

      <PalTicket
        pal={pal}
        stub={
          <View style={styles.stub}>
            <Tx pal={pal} tone="secondary" variant="kicker">
              Your avatar
            </Tx>
            <View style={styles.emojis} accessibilityRole="radiogroup" accessibilityLabel="Travel avatar">
              {EMOJIS.map((e) => {
                const on = p.emoji === e;
                return (
                  <Press
                    key={e}
                    onPress={() => set({ emoji: e })}
                    accessibilityRole="radio"
                    accessibilityState={{ checked: on }}
                    accessibilityLabel={`Avatar ${e}`}
                    scaleTo={0.9}
                    style={[styles.emoji, on ? { backgroundColor: pal.accent, borderColor: pal.ink, borderWidth: line } : { borderColor:
pal.border, borderWidth: hair }]}>
                    <T variant="numeralSm" importantForAccessibility="no">
                      {e}
                    </T>
                  </Press>
                );
              })}
            </View>
          </View>
        }>
        <View style={styles.main}>
          <View style={styles.topRow}>
            <Tx pal={pal} tone="secondary" variant="kicker" style={{ flex: 1 }}>
              Crew pass
            </Tx>
            <PalStamp pal={pal} label="About 1 min" tone="accent" />
          </View>
          <View style={styles.passenger}>
            <View style={[styles.avatar, { backgroundColor: pal.accent, borderColor: pal.ink }]} accessibilityElementsHidden importantFo
rAccessibility="no-hide-descendants">
              <T variant="numeral">{p.emoji}</T>
            </View>
            <View style={{ flex: 1, gap: 2 }}>
              <Tx pal={pal} tone="secondary" variant="kicker">
                Passenger
              </Tx>
              <TextInput
                value={p.nickname}
                onChangeText={(v) => set({ nickname: v })}
                onFocus={() => setFocused(true)}
                onBlur={() => setFocused(false)}
                onSubmitEditing={onSubmit}
                returnKeyType="next"
                placeholder={firstName}
                placeholderTextColor={pal.textMuted}
                maxLength={20}
                autoCapitalize="words"
                accessibilityLabel="Nickname"
```

```
              style={[styles.input, { color: pal.text, borderBottomColor: focused ? pal.accentText : pal.ink }]}
            />
          </View>
        </View>
        <Tx pal={pal} tone="muted" variant="caption">
          What should we call you? This shows on Home.
        </Tx>
      </View>
    </PalTicket>
  </View>
 );
}

const styles = StyleSheet.create({
  main: { padding: 16, gap: 14 },
  topRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  passenger: { flexDirection: 'row', alignItems: 'flex-end', gap: 14 },
  avatar: { width: 56, height: 56, borderRadius: radius.button, borderWidth: line, alignItems: 'center', justifyContent: 'center' },
  input: { minHeight: 44, borderBottomWidth: line, paddingVertical: 6, fontFamily: fonts.bold, fontSize: 24, ...({ outlineStyle: 'none'
} as object) },
  stub: { paddingHorizontal: 16, paddingBottom: 16, paddingTop: 2, gap: 10 },
  emojis: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  emoji: { width: 44, height: 44, borderRadius: radius.chip, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/photos/PhotoTile.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Press } from '@/components/ui/Press';
import { Skeleton } from '@/components/ui/Skeleton';
import { T } from '@/components/ui/T';
import { imageSource } from '@/data/images';
import type { Photo } from '@/store/types';
import { useSharedFile } from '@/sync/useFile';
import { colors, line, radius } from '@/theme';

/**
 * One square in the photo grid, in a 2 px ink outline like a print on a contact sheet. A grey block pulses where the picture will be un
til
 * it has loaded (or while a friend's copy downloads); a picture that is on a friend's phone shows its caption instead.
 */
export function PhotoTile({ photo, size, who, onPress }: { photo: Photo; size: number; who: string; onPress: () => void }) {
  const { uri, loading } = useSharedFile(photo.uri, photo.blob);
  const [loaded, setLoaded] = useState(false);
  return (
    <Press onPress={onPress} style={[styles.tile, { width: size, height: size }]} scaleTo={0.96} accessibilityLabel={photo.caption || 'P
hoto'}>
      {uri ? (
        <Image source={imageSource(uri)} style={styles.img} contentFit="cover" transition={150} onLoad={() => setLoaded(true)} accessibi
lityIgnoresInvertColors />
      ) : loading ? null : (
        <View style={[styles.img, styles.missing]}>
          <Ionicons name="image-outline" size={26} color={colors.textMuted} />
          <T variant="micro" color={colors.textSecondary} center numberOfLines={2}>
            {photo.caption || `Photo from ${who}`}
          </T>
        </View>
      )}
      {(uri && !loaded) || loading ? <Skeleton height={size} corner={0} style={StyleSheet.absoluteFill} /> : null}
      {photo.likes.length ? (
        <View style={styles.likes}>
          <Ionicons name="heart" size={11} color={colors.ink} />
          <T variant="kicker" color={colors.ink} style={{ fontSize: 10.5, lineHeight: 13 }}>
            {photo.likes.length}
          </T>
        </View>
      ) : null}
    </Press>
  );
}

const styles = StyleSheet.create({
  tile: { borderRadius: radius.chip, borderWidth: line, borderColor: colors.ink, overflow: 'hidden', backgroundColor: colors.bgAlt },
  img: { width: '100%', height: '100%' },
  missing: { alignItems: 'center', justifyContent: 'center', gap: 4, padding: 8 },
  likes: { position: 'absolute', bottom: 5, left: 5, flexDirection: 'row', alignItems: 'center', gap: 3, backgroundColor: colors.card, b
orderWidth: 1.5, borderColor: colors.ink, paddingHorizontal: 5, paddingVertical: 1, borderRadius: radius.xs },
});
```

src/components/photos/PhotoViewer.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { Button } from '@/components/ui/Button';
import { PlatePress } from '@/components/ui/Plate';
import { Skeleton } from '@/components/ui/Skeleton';
import { T } from '@/components/ui/T';
import { Avatar, type IconName } from '@/components/ui/bits';
import { confirmAction } from '@/components/ui/feedback';
import { imageSource } from '@/data/images';
import { deleteLocalFile } from '@/services/files';
import { sharePhoto } from '@/services/share';
import type { Photo, Trip } from '@/store/types';
import { useSharedFile } from '@/sync/useFile';
import { colors, line, radius } from '@/theme';
import { timeAgo } from '@/utils/format';
import { dayLabel } from '@/utils/time';

/** A square outlined control (44 points) that sits on the dark backdrop, like the controls on the Home photo. */
function Control({ icon, label, onPress, active }: { icon: IconName; label: string; onPress: () => void; active?: boolean }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} color={colors.accent} style={[styles.cont
rol, active && { backgroundColor: colors.accent }]}>
      <Ionicons name={icon} size={20} color={active ? colors.onAccent : colors.ink} />
    </PlatePress>
  );
}

/** The full-screen view of one photo: the picture as a print in a paper frame, who posted it, likes, share and (for your own photos) de
lete. */
export function PhotoViewer({ photo, trip, userId, onClose, onShare, onLike, onDelete }: { photo: Photo; trip: Trip; userId: string; onC
lose: () => void; onShare: () => void; onLike: () => void; onDelete: () => void }) {
  const insets = useSafeAreaInsets();
  const { uri, loading } = useSharedFile(photo.uri, photo.blob);
  const author = trip.members.find((x) => x.id === photo.by);
  const first = author?.name.split(' ')[0] ?? 'a friend';
  const liked = photo.likes.includes(userId);
  return (
    <View style={[styles.viewer, { paddingTop: insets.top + 12, paddingBottom: insets.bottom + 16 }]}>
      <View style={styles.top}>
        <Control icon="close" label="Close photo" onPress={onClose} />
        <View style={{ flexDirection: 'row', gap: 14 }}>
          <Control icon="share-social-outline" label="Share photo to social media" onPress={onShare} />
          {photo.by === userId ? (
            <Control
              icon="trash-outline"
              label="Delete photo"
              onPress={() =>
                confirmAction('Delete photo?', 'It will be removed from the photo dump for everyone.', () => {
                  deleteLocalFile(photo.uri);
                  onDelete();
                }, { destructive: true, confirmLabel: 'Delete' })
              }
            />
          ) : null}
        </View>
      </View>
      <View style={styles.print}>
        {uri ? (
          <Image source={imageSource(uri)} style={{ flex: 1, width: '100%', borderRadius: radius.card }} contentFit="contain" accessibil
ityIgnoresInvertColors />
        ) : loading ? (
          <Skeleton height={320} corner={radius.xs} />
        ) : (
          <View style={styles.away}>
            <Ionicons name="phone-portrait-outline" size={36} color={colors.ink} />
            <T variant="small" center>
              This photo is on {first}’s phone. Likes and captions still sync.
            </T>
          </View>
        )}
      </View>
      <View style={styles.panel}>
        <View style={styles.who}>
          <Avatar name={author?.name ?? '?'} src={author?.avatar} size={40} />
          <View style={{ flex: 1, gap: 2 }}>
            <T variant="title" weight="bold" numberOfLines={2}>
              {photo.caption || 'Untitled'}
            </T>
            <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
              {first} / {photo.day ? dayLabel(photo.day) : ''} / {timeAgo(photo.ts)}
            </T>
```

```
          </View>
          <PlatePress onPress={onLike} accessibilityLabel={liked ? 'Unlike' : 'Like'} accessibilityState={{ selected: liked }} offset=
{3} corner={radius.button} style={[styles.like, liked && { backgroundColor: colors.accent }]}>
            <Ionicons name={liked ? 'heart' : 'heart-outline'} size={20} color={liked ? colors.onAccent : colors.ink} />
            <T variant="numeralSm" color={liked ? colors.onAccent : colors.ink} style={{ fontSize: 17, lineHeight: 22 }}>
              {photo.likes.length}
            </T>
          </PlatePress>
        </View>
        {uri ? <Button label="Save & share" size="md" icon="download-outline" onPress={() => sharePhoto(uri, photo.caption)} /> : null}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  viewer: { flex: 1, backgroundColor: colors.black, paddingHorizontal: 20, gap: 16 },
  top: { flexDirection: 'row', justifyContent: 'space-between' },
  control: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
d, alignItems: 'center', justifyContent: 'center' },
  print: { flex: 1, justifyContent: 'center', overflow: 'hidden' },
  away: { alignItems: 'center', justifyContent: 'center', gap: 10, padding: 24, backgroundColor: colors.card, borderWidth: line, borderC
olor: colors.ink, borderRadius: radius.card },
  panel: { backgroundColor: colors.card, borderWidth: line, borderColor: colors.ink, borderRadius: radius.card, padding: 14, gap: 14 },
  who: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  like: { minWidth: 64, height: 44, flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, paddingHorizontal: 12,
borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
});
```

src/components/pickers.tsx

```
/** Cross-platform time & date pickers (identical on iOS, Android and web). */
import { Ionicons } from '@expo/vector-icons';
import { useMemo, useState } from 'react';
import { StyleSheet, View, useWindowDimensions } from 'react-native';

import { colors, line, radius } from '@/theme';
import { addDays, daysBetween, parseYMD, time12, ymdFromUtcDate } from '@/utils/time';

import { Button } from './ui/Button';
import { Chip } from './ui/bits';
import { PlatePress } from './ui/Plate';
import { Press } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { SplitFlap } from './ui/SplitFlap';
import { T } from './ui/T';

/** A square button with a plate: the arrows above and below the time, and the month arrows. */
function ArrowButton({ icon, label, onPress }: { icon: 'chevron-up' | 'chevron-down' | 'chevron-back' | 'chevron-forward'; label: strin
g; onPress: () => void }) {
  return (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} style={styles.arrow}>
      <Ionicons name={icon} size={22} color={colors.ink} />
    </PlatePress>
  );
}

/** Hours or minutes on departure-board tiles, with an arrow above and below to change them. */
function Stepper({ label, value, size, onUp, onDown }: { label: string; value: string; size: number; onUp: () => void; onDown: () => voi
d }) {
  return (
    <View style={styles.stepper}>
      <ArrowButton icon="chevron-up" label={`Increase ${label}`} onPress={onUp} />
      <View accessible accessibilityLabel={`${label} ${value}`} style={{ alignItems: 'center' }}>
        <SplitFlap value={value} size={size} />
      </View>
      <ArrowButton icon="chevron-down" label={`Decrease ${label}`} onPress={onDown} />
      <T variant="kicker" color={colors.textSecondary} style={{ alignSelf: 'center' }}>
        {label}
      </T>
    </View>
  );
}

export function TimePickerSheet({ visible, value, onClose, onChange }: { visible: boolean; value: string; onClose: () => void; onChange:
(v: string) => void }) {
  const [h, setH] = useState(10);
  const [m, setM] = useState(0);
  const { width } = useWindowDimensions();
  const size = width >= 380 ? 72 : width >= 340 ? 62 : 50;
  // Re-sync with the field each time the sheet opens (derived during render, no effect needed).
  const [openedWith, setOpenedWith] = useState<string | null>(null);
  const openKey = visible ? value : null;
  if (openKey !== openedWith) {
    setOpenedWith(openKey);
    if (visible) {
```

```
      const [hh, mm] = value.split(':').map(Number);
      setH(hh || 0);
      setM(mm || 0);
    }
  }
  const hhmm = `${String(h).padStart(2, '0')}:${String(m).padStart(2, '0')}`;
  const pm = h >= 12;
  const hour12 = h % 12 === 0 ? 12 : h % 12;
  const presets = ['07:00', '09:00', '12:30', '15:00', '18:00', '19:30', '21:00'];

  return (
    <Sheet
      visible={visible}
      onClose={onClose}
      title="Pick a time"
      subtitle={`Trip local time · ${time12(hhmm)}`}
      footer={
        <Button
          label="Set time"
          onPress={() => {
            onChange(hhmm);
            onClose();
          }}
        />
      }>
      <View style={styles.timeRow}>
        <Stepper label="hour" value={String(hour12).padStart(2, '0')} size={size} onUp={() => setH((x) => (x + 1) % 24)} onDown={() => s
etH((x) => (x + 23) % 24)} />
        <View style={styles.colon} accessibilityElementsHidden importantForAccessibility="no">
          <View style={styles.dot} />
          <View style={styles.dot} />
        </View>
        <Stepper label="minute" value={String(m).padStart(2, '0')} size={size} onUp={() => setM((x) => (x + 5) % 60)} onDown={() => setM
((x) => (x + 55) % 60)} />
        <View style={{ gap: 8, marginLeft: 4 }}>
          <Chip label="AM" active={!pm} onPress={() => pm && setH((x) => x - 12)} small />
          <Chip label="PM" active={pm} onPress={() => !pm && setH((x) => x + 12)} small />
        </View>
      </View>
      <View style={styles.wrap}>
        {presets.map((p) => (
          <Chip
            key={p}
            small
            label={time12(p)}
            active={p === hhmm}
            onPress={() => {
              const [a, b] = p.split(':').map(Number);
              setH(a);
              setM(b);
            }}
          />
        ))}
      </View>
    </Sheet>
  );
}

const WD = ['S', 'M', 'T', 'W', 'T', 'F', 'S'];
const MONTHS = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'Decembe
r'];

export function DatePickerSheet({
  visible,
  value,
  onClose,
  onChange,
  min,
  max,
  title = 'Pick a date',
}: {
  visible: boolean;
  value: string;
  onClose: () => void;
  onChange: (v: string) => void;
  min?: string;
  max?: string;
  title?: string;
}) {
  const [cursor, setCursor] = useState(value);
  const [openedWith, setOpenedWith] = useState<string | null>(null);
  const openKey = visible ? value : null;
  if (openKey !== openedWith) {
    setOpenedWith(openKey);
    if (visible) setCursor(value);
  }
```

```
  const { y, m } = parseYMD(cursor);
  const cells = useMemo(() => {
    const first = new Date(Date.UTC(y, m - 1, 1));
    const offset = first.getUTCDay();
    const daysInMonth = new Date(Date.UTC(y, m, 0)).getUTCDate();
    const out: (string | null)[] = Array(offset).fill(null);
    for (let d = 1; d <= daysInMonth; d++) out.push(ymdFromUtcDate(new Date(Date.UTC(y, m - 1, d))));
    while (out.length % 7) out.push(null);
    return out;
  }, [y, m]);

  const shiftMonth = (n: number) => setCursor(ymdFromUtcDate(new Date(Date.UTC(y, m - 1 + n, 1))));
  const disabled = (d: string) => (min && daysBetween(min, d) < 0) || (max && daysBetween(d, max) < 0);

  return (
    <Sheet visible={visible} onClose={onClose} title={title}>
      <View style={styles.monthRow}>
        <ArrowButton icon="chevron-back" label="Previous month" onPress={() => shiftMonth(-1)} />
        <View style={{ alignItems: 'center' }} accessible accessibilityLabel={`${MONTHS[m - 1]} ${y}`}>
          <T variant="headline" importantForAccessibility="no">
            {MONTHS[m - 1]}
          </T>
          <T variant="kicker" color={colors.textSecondary} importantForAccessibility="no">
            {y}
          </T>
        </View>
        <ArrowButton icon="chevron-forward" label="Next month" onPress={() => shiftMonth(1)} />
      </View>
      <View style={styles.grid}>
        {WD.map((w, i) => (
          <View key={`w${i}`} style={styles.head}>
            <T variant="kicker" color={colors.textSecondary}>
              {w}
            </T>
          </View>
        ))}
        {cells.map((d, i) =>
          d ? (
            <Press
              key={d}
              disabled={!!disabled(d)}
              onPress={() => {
                onChange(d);
                onClose();
              }}
              accessibilityLabel={d}
              accessibilityState={{ selected: d === value, disabled: !!disabled(d) }}
              scaleTo={0.94}
              style={[styles.cell, disabled(d) && { opacity: 0.3 }]}>
              <View style={[styles.day, d === value && styles.daySel]}>
                <T variant="numeralSm" color={d === value ? colors.onAccent : colors.text} importantForAccessibility="no">
                  {parseYMD(d).d}
                </T>
              </View>
            </Press>
          ) : (
            <View key={`e${i}`} style={styles.cell} />
          ),
        )}
      </View>
      <View style={styles.wrap}>
        <Chip small label="Today" onPress={() => setCursor(ymdFromUtcDate(new Date()))} />
        <Chip small label="+1 week" onPress={() => setCursor(addDays(cursor, 7))} />
        <Chip small label="+1 month" onPress={() => shiftMonth(1)} />
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  arrow: { height: 44, minWidth: 44, alignSelf: 'stretch', borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, back
groundColor: colors.card, alignItems: 'center', justifyContent: 'center' },
  timeRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 8, paddingVertical: 4 },
  stepper: { alignItems: 'stretch', gap: 8 },
  colon: { gap: 14, marginTop: -4 },
  dot: { width: 8, height: 8, backgroundColor: colors.ink },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  monthRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between' },
  grid: { flexDirection: 'row', flexWrap: 'wrap' },
  head: { width: `${100 / 7}%`, height: 28, alignItems: 'center', justifyContent: 'center' },
  cell: { width: `${100 / 7}%`, aspectRatio: 1, padding: 2 },
  day: { flex: 1, alignItems: 'center', justifyContent: 'center', borderRadius: radius.chip, borderWidth: line, borderColor: 'transparen
t' },
  daySel: { backgroundColor: colors.accent, borderColor: colors.ink },
});
```

src/components/play/BucketTicket.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useState } from 'react';
import { Animated, StyleSheet, View } from 'react-native';

import { Plate } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { nativeDriver, useMotion } from '@/components/ui/motion';
import type { Destination } from '@/data/destinations';
import { imageSource } from '@/data/images';
import type { BucketItem } from '@/store/types';
import { colors, hair, line, radius } from '@/theme';

import { tiltFor } from './bucket';

/**
 * One dream as a paper ticket stuck on a board: turned a couple of degrees (and straight while you press it), a stub with the tick
 * box on the left, the words in the middle, and a DONE stamp that slams down when it is ticked off.
 * The turn is decoration, so it is dropped when animations are off.
 */
export function BucketTicket({ item, dest, index, slam, onToggle, onOpen, onRemove }: { item: BucketItem; dest?: Destination; index: num
ber; slam: boolean; onToggle: () => void; onOpen: () => void; onRemove: () => void }) {
  const motion = useMotion();
  const [straight] = useState(() => new Animated.Value(0));
  const lift = (to: number) => {
    if (motion) Animated.timing(straight, { toValue: to, duration: 120, useNativeDriver: nativeDriver }).start();
  };
  const tilt = tiltFor(index);
  const done = item.done;
  return (
    <Animated.View style={motion ? { transform: [{ rotate: straight.interpolate({ inputRange: [0, 1], outputRange: [`${tilt}deg`, '0de
g'] }) }] } : undefined}>
      <Plate offset={3} corner={radius.card} color={done ? 'transparent' : colors.plate}>
        <View style={[styles.face, done && { backgroundColor: colors.bgAlt }]}>
          <Press
            onPress={onToggle}
            onPressIn={() => lift(1)}
            onPressOut={() => lift(0)}
            accessibilityRole="checkbox"
            accessibilityState={{ checked: done }}
            accessibilityLabel={item.text}
            scaleTo={0.94}
            style={[styles.stub, done && { backgroundColor: colors.accent }]}>
            <View style={[styles.box, done && { backgroundColor: colors.card }]}>{done ? <Ionicons name="checkmark" size={22} color={col
ors.ink} /> : null}</View>
          </Press>
          <Press onPress={onOpen} onPressIn={() => lift(1)} onPressOut={() => lift(0)} scaleTo={0.99} style={styles.body} accessibilityL
abel={dest ? `Open ${dest.city}` : `Tick off ${item.text}`}>
            {dest ? <Image source={imageSource(dest.image)} style={styles.thumb} contentFit="cover" accessibilityIgnoresInvertColors />
: null}
            <View style={{ flex: 1, gap: 4 }}>
              <T variant="title" weight="bold" color={done ? colors.textSecondary : colors.text} style={done && { textDecorationLine: 'l
ine-through' }}>
                {item.text}
              </T>
              {dest ? (
                <T variant="kicker" color={colors.textSecondary} numberOfLines={2}>
                  {dest.country} / {dest.bestMonths.split(',')[0]}
                </T>
              ) : null}
              {done ? <Stamp label="Done" tone="positive" icon="checkmark" slam={slam} rotate={-6} /> : null}
            </View>
          </Press>
          <Press onPress={onRemove} accessibilityLabel={`Remove ${item.text}`} style={styles.remove}>
            <Ionicons name="close" size={20} color={colors.textSecondary} />
          </Press>
        </View>
      </Plate>
    </Animated.View>
  );
}

const styles = StyleSheet.create({
  face: { flexDirection: 'row', alignItems: 'stretch', minHeight: 72, backgroundColor: colors.card, borderRadius: radius.card, borderWid
th: line, borderColor: colors.ink, overflow: 'hidden' },
  stub: { width: 60, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.bgAlt, borderRightWidth: line, borderRightC
olor: colors.ink },
  box: { width: 30, height: 30, borderRadius: radius.xs + 2, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, a
lignItems: 'center', justifyContent: 'center' },
  body: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 12, paddingVertical: 12, minHeight: 44 },
  thumb: { width: 46, height: 46, borderRadius: radius.xs, borderWidth: hair, borderColor: colors.ink },
```

```
  remove: { width: 44, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/play/PackingParts.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { Avatar } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import type { Member, PackingItem } from '@/store/types';
import { colors, line, radius } from '@/theme';

/**
 * One thing on the packing list: a big square tick box (accent and a tick when packed), the item, who is bringing it and a remove butto
n.
 * Every part you tap is at least 44 points.
 */
export function PackRow({ item, who, me, onToggle, onWho, onRemove }: { item: PackingItem; who?: Member; me?: string; onToggle: () => vo
id; onWho: () => void; onRemove: () => void }) {
  return (
    <View style={styles.row}>
      <Press onPress={onToggle} style={styles.main} accessibilityRole="checkbox" accessibilityState={{ checked: item.packed }} accessibi
lityLabel={item.text} scaleTo={0.98}>
        <View style={[styles.box, item.packed && { backgroundColor: colors.accent }]}>{item.packed ? <Ionicons name="checkmark" size={2
0} color={colors.onAccent} /> : null}</View>
        <T variant="bodySm" color={item.packed ? colors.textMuted : colors.text} style={[{ flex: 1 }, item.packed && { textDecorationLin
e: 'line-through' }]}>
          {item.text}
        </T>
      </Press>
      <Press onPress={onWho} style={styles.whoHit} accessibilityLabel={who ? `${item.text} is brought by ${who.name}. Change` : `Assign
${item.text}`}>
        {who ? (
          <View style={styles.who}>
            <Avatar name={who.name} src={who.avatar} size={24} />
            <T variant="caption" weight="bold" numberOfLines={1} style={{ maxWidth: 52 }}>
              {who.id === me ? 'Me' : who.name.split(' ')[0]}
            </T>
          </View>
        ) : (
          <Ionicons name="person-add-outline" size={20} color={colors.textSecondary} />
        )}
      </Press>
      <Press onPress={onRemove} style={styles.remove} accessibilityLabel={`Remove ${item.text}`}>
        <Ionicons name="close" size={20} color={colors.textSecondary} />
      </Press>
    </View>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', minHeight: 56, paddingLeft: 12 },
  main: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingVertical: 6 },
  box: { width: 28, height: 28, borderRadius: radius.xs + 2, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, a
lignItems: 'center', justifyContent: 'center' },
  whoHit: { minWidth: 44, height: 44, alignItems: 'center', justifyContent: 'center', paddingHorizontal: 2 },
  who: { flexDirection: 'row', alignItems: 'center', gap: 6, height: 32, paddingHorizontal: 4 },
  remove: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/play/QuizParts.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { Avatar, ProgressBar } from '@/components/ui/bits';
import { PlatePress } from '@/components/ui/Plate';
import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';

import { podiumSlots, SECONDS, type BoardRow } from './quizLogic';

/** What a row is called on the podium: "You" for you, "This round" / "Your best" when you play alone, a first name for a friend. */
const who = (r: BoardRow) => (r.id === 'round' || r.id === 'best' ? r.name : r.me ? 'You' : r.name.split(' ')[0]);

/** One short segment per question: ink for the ones answered, the accent for the one you are on, an empty box for the rest. */
export function StepBar({ index, total, answered }: { index: number; total: number; answered: boolean }) {
  return (
    <View accessible accessibilityRole="progressbar" accessibilityLabel={`Question ${index + 1} of ${total}`} accessibilityValue={{ min:
0, max: total, now: index + (answered ? 1 : 0) }} style={styles.steps}>
      {Array.from({ length: total }, (_, i) => {
        const done = i < index || (i === index && answered);
        return <View key={i} style={[styles.step, done && { backgroundColor: colors.ink }, i === index && !answered && { backgroundColo
r: colors.accent }]} />;
```

```
      ))}
    </View>
  );
}

/** The clock: the seconds left in mono and a bar that drains. The last five seconds turn red, with the alarm icon, so colour is not the
only signal. */
export function Clock({ left }: { left: number }) {
  const low = left <= 5;
  const color = low ? colors.red : colors.accent;
  return (
    <View style={styles.clock} accessible accessibilityLabel={`${Math.max(0, left)} seconds left`}>
      <Ionicons name={low ? 'alarm-outline' : 'timer-outline'} size={22} color={low ? colors.red : colors.ink} />
      <View style={{ flex: 1 }}>
        <ProgressBar value={Math.max(0, left) / SECONDS} color={color} height={14} />
      </View>
      <T variant="numeralSm" color={low ? colors.red : colors.ink} style={{ width: 62, textAlign: 'right' }} importantForAccessibility
="no">
        {Math.max(0, left)}s
      </T>
    </View>
  );
}

export type OptionState = 'open' | 'right' | 'reveal' | 'wrong' | 'dim';

/** A big answer button (60 points tall). Once the round is answered the right one gets a tick and the wrong pick a cross, in colour AND
icon. */
export function AnswerOption({ letter, text, state, onPress, disabled }: { letter: string; text: string; state: OptionState; onPress: ()
=> void; disabled: boolean }) {
  const good = state === 'right' || state === 'reveal';
  const bad = state === 'wrong';
  const tone = good ? colors.positive : bad ? colors.red : colors.ink;
  const label = `${letter}. ${text}${state === 'right' ? '. Correct' : state === 'wrong' ? '. Not this one' : state === 'reveal' ? '. Th
e right answer' : ''}`;
  return (
    <PlatePress
      onPress={onPress}
      disabled={disabled}
      accessibilityLabel={label}
      accessibilityState={{ disabled, selected: state === 'right' || state === 'wrong' }}
      offset={3}
      color={good ? colors.positive : bad ? colors.red : colors.plate}
      wrapperStyle={state === 'dim' ? { opacity: 0.5 } : undefined}
      style={[styles.option, { borderColor: tone, backgroundColor: good ? colors.positiveSoft : bad ? colors.redSoft : colors.card }]}>
      <View style={[styles.letter, { borderColor: tone, backgroundColor: good ? colors.positive : bad ? colors.red : 'transparent' }]}>
        <T variant="kicker" color={good || bad ? colors.white : colors.ink} style={{ letterSpacing: 0 }} importantForAccessibility="no">
          {letter}
        </T>
      </View>
      <T variant="bodySm" weight="semibold" style={{ flex: 1 }} importantForAccessibility="no">
        {text}
      </T>
      {good ? <Ionicons name="checkmark-circle" size={24} color={colors.positive} /> : bad ? <Ionicons name="close-circle" size={24} col
or={colors.red} /> : null}
    </PlatePress>
  );
}

/**
 * The podium: three ink blocks of different heights (second, first, third), each with a face, a name and the score in mono.
 * The winner's block is the accent and you are marked with an accent name tag.
 */
export function Podium({ rows }: { rows: BoardRow[] }) {
  const slots = podiumSlots(rows);
  return (
    <View style={styles.podium} accessible accessibilityLabel={`Podium. ${rows.slice(0, 3).map((r, i) => `${i + 1}. ${who(r)}, ${r.scor
e} points`).join('. ')}`}>
      {slots.map((s) => {
        const winner = s.rank === 1;
        return (
          <View key={s.id} style={styles.col}>
            <Avatar name={s.name} src={s.avatar} size={winner ? 56 : 46} />
            <View style={[styles.name, s.me && { backgroundColor: colors.accent }]}>
              <T variant="caption" weight="bold" color={s.me ? colors.onAccent : colors.ink} numberOfLines={1}>
                {who(s)}
              </T>
            </View>
            <View style={[styles.block, { height: s.height }, winner && { backgroundColor: colors.accent }]}>
              <T variant="numeralSm" color={winner ? colors.onAccent : colors.bg}>
                {s.score}
              </T>
              <T variant="kicker" color={winner ? colors.onAccent : colors.bg}>
                {s.rank === 1 ? '1st' : s.rank === 2 ? '2nd' : '3rd'}
              </T>
            </View>
```

```
          </View>
        );
      })}
    </View>
  );
}

const styles = StyleSheet.create({
  steps: { flexDirection: 'row', gap: 4 },
  step: { flex: 1, height: 10, borderRadius: 3, borderWidth: 2, borderColor: colors.ink, backgroundColor: colors.card },
  clock: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 44 },
  option: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 60, paddingHorizontal: 12, paddingVertical: 10, borderRadiu
s: radius.button, borderWidth: line },
  letter: { width: 34, height: 34, borderRadius: radius.xs, borderWidth: line, alignItems: 'center', justifyContent: 'center' },
  podium: { flexDirection: 'row', alignItems: 'flex-end', gap: 8 },
  col: { flex: 1, alignItems: 'center', gap: 6 },
  name: { alignSelf: 'center', maxWidth: '100%', paddingHorizontal: 6, borderRadius: 3 },
  block: { alignSelf: 'stretch', backgroundColor: colors.ink, borderWidth: line, borderColor: colors.ink, borderTopLeftRadius: radius.ch
ip, borderTopRightRadius: radius.chip, alignItems: 'center', paddingTop: 6, gap: 2 },
});
```

src/components/play/RecapPoster.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import type { RefObject } from 'react';
import { StyleSheet, View } from 'react-native';

import { barcodeBars } from '@/components/crew/barcode';
import { Plate } from '@/components/ui/Plate';
import { Rule } from '@/components/ui/Rule';
import { Stamp, type StampTone } from '@/components/ui/Stamp';
import { T, type TProps } from '@/components/ui/T';
import { imageSource } from '@/data/images';
import { colors, line, radius } from '@/theme';

import { shortName } from './recapLogic';

/** Text on the poster: a fixed graphic that is shared as a picture, so the phone's text size must not reflow it. */
function Fixed(props: TProps) {
  return <T allowFontScaling={false} {...props} />;
}

/** Each stamp lands a little crooked and a little to one side, like stamps in a passport. */
const TILT = [-4, 3, -2];
const SHIFT = [0, 36, 12];

/**
 * The trip as a passport page, and the picture that gets shared: a photo with a paper tag, the trip in serif, three numbers in mono,
 * a few stamps for the places visited and a barcode made from the invite code. Only the page itself is captured (not its plate).
 */
export function RecapPoster({ cardRef, width, image, trip, place, dates, status, stats, places, summary }: { cardRef: RefObject<View | n
ull>; width: number; image: string; trip: { name: string; inviteCode: string }; place: string; dates: string; status: { label: string; t
one: StampTone }; stats: [string, string][]; places: string[]; summary: string }) {
  return (
    <Plate offset={4} corner={radius.card} style={{ alignSelf: 'center' }}>
      <View ref={cardRef} collapsable={false} accessible accessibilityLabel={`Story card. ${summary}`} style={[styles.page, { width }]}>
        <View style={styles.photo}>
          <Image source={imageSource(image)} style={StyleSheet.absoluteFill} contentFit="cover" accessibilityIgnoresInvertColors />
          <View style={styles.tag}>
            <Fixed variant="kicker" color={colors.ink}>
              Trip recap
            </Fixed>
          </View>
          <View style={styles.status}>
            <Stamp label={status.label} tone={status.tone === 'ink' ? 'accent' : status.tone} rotate={-5} slam={status.tone === 'accen
t'} />
          </View>
        </View>
        <View style={styles.body}>
          <Fixed variant="kicker" color={colors.textSecondary} numberOfLines={1}>
            {place}
          </Fixed>
          <Fixed variant="headline" numberOfLines={3} style={{ fontSize: 32, lineHeight: 36 }}>
            {trip.name}
          </Fixed>
          <Fixed variant="kicker" color={colors.textSecondary}>
            {dates}
          </Fixed>
          <Rule style={{ marginTop: 6 }} />
          <View style={styles.stats}>
            {stats.map(([value, label]) => (
              <View key={label} style={{ flex: 1 }}>
                <Fixed variant="numeralSm" numberOfLines={1} adjustsFontSizeToFit>
                  {value}
                </Fixed>
```

```
                <Fixed variant="kicker" color={colors.textSecondary} style={{ fontSize: 9.5 }}>
                  {label}
                </Fixed>
              </View>
            ))}
          </View>
          {places.length ? (
            <>
              <Rule weight={1} style={{ backgroundColor: colors.border }} />
              <View style={styles.stamps}>
                {places.slice(0, 3).map((p, i) => (
                  <View key={p} style={{ marginLeft: SHIFT[i % SHIFT.length] }}>
                    <Stamp label={shortName(p, 24)} tone={i === 0 ? 'accent' : 'ink'} rotate={TILT[i % TILT.length]} />
                  </View>
                ))}
              </View>
            </>
          ) : null}
          <View style={styles.brand}>
            <Ionicons name="airplane" size={14} color={colors.ink} />
            <Fixed variant="kicker" color={colors.ink} style={{ flex: 1 }}>
              TogetherWeGo
            </Fixed>
          </View>
          <View style={styles.barcode} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
            {barcodeBars(trip.inviteCode, 49).map((w, i) => (
              <View key={i} style={{ flex: w, backgroundColor: i % 2 === 0 ? colors.ink : 'transparent' }} />
            ))}
          </View>
        </View>
      </View>
    </Plate>
  );
}

const CORNER = Math.max(0, radius.card - line);

const styles = StyleSheet.create({
  page: { backgroundColor: colors.card, borderWidth: line, borderColor: colors.ink, borderRadius: radius.card, overflow: 'hidden' },
  photo: { height: 190, borderBottomWidth: line, borderBottomColor: colors.ink, backgroundColor: colors.bgAlt, borderTopLeftRadius: CORN
ER, borderTopRightRadius: CORNER, overflow: 'hidden' },
  tag: { position: 'absolute', left: 12, top: 12, paddingHorizontal: 8, paddingVertical: 3, backgroundColor: colors.card, borderWidth: l
ine, borderColor: colors.ink, borderRadius: radius.xs },
  status: { position: 'absolute', right: 12, bottom: 12 },
  body: { padding: 16, gap: 8 },
  stats: { flexDirection: 'row', gap: 10, paddingVertical: 2 },
  stamps: { gap: 10, paddingVertical: 6 },
  brand: { flexDirection: 'row', alignItems: 'center', gap: 6, marginTop: 4 },
  barcode: { flexDirection: 'row', height: 28 },
});
```

src/components/play/SurveyParts.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { IconButton } from '@/components/ui/bits';
import { PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { Ticket } from '@/components/ui/Ticket';
import type { Destination } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { colors, hair, line, radius } from '@/theme';
import { money } from '@/utils/format';

import type { OptionArt } from './recommend';

const CORNER = Math.max(0, radius.card - line);
const open = (d: Destination) => router.push({ pathname: '/destination/[id]', params: { id: d.id } });

/**
 * A big answer tile with an ink outline: a photo, a line icon or a few dollar signs on top, and the answer on a label strip below.
 * The answer you gave before (when you come back to a question) is marked with the accent and a tick.
 */
export function OptionTile({ label, art, width, chosen, onPress }: { label: string; art: OptionArt; width: number; chosen: boolean; onPr
ess: () => void }) {
  return (
    <PlatePress
      onPress={onPress}
      testID="survey-option"
      accessibilityLabel={label}
```

```
      accessibilityState={{ selected: chosen }}
      offset={3}
      corner={radius.card}
      wrapperStyle={{ width: width + 3 }}
      style={[styles.tile, { width }]}>
      <View style={[styles.art, !('photo' in art) && { backgroundColor: colors.accentSoft }]}>
        {'photo' in art ? (
          <Image source={imageSource(art.photo)} style={StyleSheet.absoluteFill} contentFit="cover" accessibilityIgnoresInvertColors />
        ) : 'icon' in art ? (
          <Ionicons name={art.icon} size={44} color={colors.ink} />
        ) : (
          <T variant="numeral" color={colors.ink}>
            {art.glyph}
          </T>
        )}
      </View>
      <View style={[styles.strip, chosen && { backgroundColor: colors.accent }]}>
        <T variant="small" weight="bold" color={chosen ? colors.onAccent : colors.text} style={{ flex: 1 }} numberOfLines={2}>
          {label}
        </T>
        {chosen ? <Ionicons name="checkmark" size={18} color={colors.onAccent} /> : null}
      </View>
    </PlatePress>
  );
}

/** The best match as a boarding pass: the photo with the match stamped on it, the place in serif, and the way to add it to a trip on th
e stub. */
export function TopPick({ dest, match, onAdd }: { dest: Destination; match: number; onAdd: () => void }) {
  return (
    <Ticket
      stub={
        <View style={styles.stub}>
          <Button label="Add to a trip" icon="add" size="md" onPress={onAdd} />
        </View>
      }>
      {/* The Add button sits under the tear line, not inside the card button: on the web a button inside a button is invalid HTML. */}
      <Press onPress={() => open(dest)} scaleTo={0.99} accessibilityLabel={`${dest.city}, ${match}% match`} style={styles.pickPress}>
        <View style={styles.photo}>
          <Image source={imageSource(dest.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={250} accessibilityIgnor
esInvertColors />
          <View style={styles.stamp}>
            <Stamp label={`${match}% match`} tone="accent" rotate={-4} slam />
          </View>
          <View style={styles.tag}>
            <T variant="kicker" color={colors.ink}>
              Top pick for you
            </T>
          </View>
        </View>
        <View style={styles.main}>
          <T variant="kicker" color={colors.textSecondary}>
            {dest.country}
          </T>
          <T variant="headline" style={{ fontSize: 32, lineHeight: 36 }}>
            {dest.city}
          </T>
          <T variant="small" color={colors.textSecondary} numberOfLines={3}>
            {dest.blurb}
          </T>
          <View style={styles.facts}>
            <T variant="numeralSm">~{money(dest.dailyCost)}</T>
            <T variant="kicker" color={colors.textSecondary}>
              a day
            </T>
            <View style={{ flex: 1 }} />
            <Ionicons name="star" size={16} color={colors.ink} />
            <T variant="numeralSm">{dest.rating}</T>
          </View>
        </View>
      </Press>
    </Ticket>
  );
}

/** A runner-up: a small photo, the place, the price a day and the match as a stamp, with a separate button to add it to a trip. */
export function MatchRow({ dest, match, last, onAdd }: { dest: Destination; match: number; last?: boolean; onAdd: () => void }) {
  return (
    <View style={[styles.row, last && { borderBottomWidth: 0 }]}>
      <Press onPress={() => open(dest)} scaleTo={0.99} accessibilityLabel={`${dest.city}, ${match}% match`} style={styles.rowPress}>
        <Image source={imageSource(dest.image)} style={styles.thumb} contentFit="cover" accessibilityIgnoresInvertColors />
        <View style={{ flex: 1, gap: 2 }}>
          <T variant="title" weight="bold" numberOfLines={1}>
            {dest.city}
          </T>
          <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
```

```
            {dest.country} / ~{money(dest.dailyCost)}/day
          </T>
        </View>
        <View style={{ alignSelf: 'center' }}>
          <Stamp label={`${match}%`} tone="ink" rotate={0} />
        </View>
      </Press>
      <IconButton icon="add" label={`Add ${dest.city} to a trip`} onPress={onAdd} />
    </View>
  );
}

const styles = StyleSheet.create({
  tile: { backgroundColor: colors.card, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, overflow: 'hidden' },
  art: { height: 104, alignItems: 'center', justifyContent: 'center', borderBottomWidth: line, borderBottomColor: colors.ink, background
Color: colors.bgAlt },
  strip: { flexDirection: 'row', alignItems: 'center', gap: 6, minHeight: 56, paddingHorizontal: 10, paddingVertical: 8 },
  pickPress: { borderRadius: radius.card },
  photo: { height: 190, borderTopLeftRadius: CORNER, borderTopRightRadius: CORNER, borderBottomWidth: line, borderBottomColor: colors.in
k, overflow: 'hidden', backgroundColor: colors.bgAlt },
  stamp: { position: 'absolute', top: 14, right: 12 },
  tag: { position: 'absolute', left: 12, bottom: 12, paddingHorizontal: 8, paddingVertical: 3, backgroundColor: colors.card, borderWidt
h: line, borderColor: colors.ink, borderRadius: radius.xs },
  main: { padding: 16, gap: 6 },
  facts: { flexDirection: 'row', alignItems: 'center', gap: 6, marginTop: 4 },
  stub: { paddingHorizontal: 16, paddingBottom: 16, paddingTop: 2 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingLeft: 12, paddingRight: 10, paddingVertical: 10, borderBottomWidth:
hair, borderBottomColor: colors.border },
  rowPress: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56 },
  thumb: { width: 56, height: 56, borderRadius: radius.xs + 2, borderWidth: line, borderColor: colors.ink },
});
```

src/components/play/__tests__/bucket.test.ts

```
import { tiltFor } from '../bucket';

describe('bucket list tilt', () => {
  it('alternates left and right', () => {
    for (let i = 0; i < 12; i++) expect(Math.sign(tiltFor(i))).toBe(i % 2 === 0 ? -1 : 1);
  });
  it('never turns a ticket more than two degrees', () => {
    for (let i = -3; i < 20; i++) expect(Math.abs(tiltFor(i))).toBeLessThanOrEqual(2);
  });
});
```

src/components/play/__tests__/packingLogic.test.ts

```
import { completesList, needsSomeone, splitFairly } from '../packingLogic';

const item = (id: string, broughtBy?: string, packed = false) => ({ id, broughtBy, packed });

describe('packing list', () => {
  it('finds what nobody is bringing', () => {
    const list = [item('a'), item('b', 'm1'), item('c', 'gone'), item('d', undefined, true)];
    expect(needsSomeone(list, ['m1', 'm2']).map((p) => p.id)).toEqual(['a', 'c']);
  });

  it('hands out the rest so that everybody carries about the same', () => {
    const list = [item('a', 'm1'), item('b', 'm1'), item('c'), item('d'), item('e'), item('f')];
    const plan = splitFairly(list, ['m1', 'm2']);
    expect(plan).toHaveLength(4);
    const m1 = 2 + plan.filter((p) => p.memberId === 'm1').length;
    const m2 = plan.filter((p) => p.memberId === 'm2').length;
    expect(Math.abs(m1 - m2)).toBeLessThanOrEqual(1);
  });

  it('does nothing without a crew', () => {
    expect(splitFairly([item('a')], [])).toEqual([]);
  });

  it('knows when the last item is packed', () => {
    expect(completesList(4, 5, false)).toBe(true);
    expect(completesList(3, 5, false)).toBe(false);
    expect(completesList(4, 5, true)).toBe(false);
    expect(completesList(0, 1, false)).toBe(false);
  });
});
```

src/components/play/__tests__/quizLogic.test.ts

```
import { ROUND, SECONDS, crewBoard, crewScore, isGoodRound, isNewBest, podiumSlots, pointsFor, resultRows, roundTitle, shouldCelebrate,
shuffle } from '../quizLogic';

describe('quiz scoring', () => {
  it('pays 10, a speed bonus and a streak bonus', () => {
    expect(pointsFor(0, 0)).toBe(10);
```

```
    expect(pointsFor(20, 0)).toBe(15);
    expect(pointsFor(20, 3)).toBe(21);
    expect(pointsFor(-4, 0)).toBe(10);
  });

  it('counts 70% right as a good round', () => {
    expect(isGoodRound(6, 8)).toBe(true);
    expect(isGoodRound(5, 8)).toBe(false);
    expect(isGoodRound(0, 0)).toBe(false);
  });

  it('names the round from how many were right', () => {
    expect(roundTitle(8, ROUND)).toBe('Legendary navigator');
    expect(roundTitle(6, ROUND)).toBe('Seasoned traveller');
    expect(roundTitle(4, ROUND)).toBe('Getting there');
    expect(roundTitle(1, ROUND)).toBe('Every expert was once a tourist');
  });

  it('knows a new best only when the score really beats it', () => {
    expect(isNewBest(80, 60)).toBe(true);
    expect(isNewBest(60, 60)).toBe(false);
    expect(isNewBest(0, 0)).toBe(false);
  });

  it('celebrates a good round or a new best, and nothing else', () => {
    expect(shouldCelebrate(6, 8, 50, 100)).toBe(true);
    expect(shouldCelebrate(3, 8, 50, 40)).toBe(true);
    expect(shouldCelebrate(3, 8, 30, 40)).toBe(false);
  });
});

describe('shuffle', () => {
  it('keeps every item and does not change the original', () => {
    const a = [1, 2, 3, 4, 5, 6];
    const s = shuffle(a);
    expect(s.slice().sort()).toEqual(a);
    expect(a).toEqual([1, 2, 3, 4, 5, 6]);
  });
  it('is repeatable with a fixed generator', () => {
    expect(shuffle([1, 2, 3, 4], () => 0)).toEqual(shuffle([1, 2, 3, 4], () => 0));
  });
  it('a round fits the clock', () => {
    expect(ROUND * SECONDS).toBeLessThan(600);
  });
});

describe('the crew board and the podium', () => {
  const members = [
    { id: 'm', name: 'Maya Chen' },
    { id: 'd', name: 'Diego Alvarez' },
    { id: 'p', name: 'Priya Patel' },
    { id: 's', name: 'Sam Okafor' },
  ];

  it('gives the same crew score to the same name', () => {
    expect(crewScore('Diego Alvarez')).toBe(crewScore('Diego Alvarez'));
    expect(crewScore('Diego Alvarez')).toBeGreaterThanOrEqual(60);
    expect(crewScore('Diego Alvarez')).toBeLessThan(150);
  });

  it('sorts best first and marks you', () => {
    const board = crewBoard(members, { id: 'm', name: 'Maya Chen' }, 1000);
    expect(board).toHaveLength(4);
    expect(board[0]).toMatchObject({ id: 'm', me: true, score: 1000 });
    const low = crewBoard(members, { id: 'm', name: 'Maya Chen' }, 0);
    expect(low[low.length - 1]).toMatchObject({ id: 'm', score: 0 });
  });

  it('lays the podium out second, first, third', () => {
    const slots = podiumSlots(crewBoard(members, { id: 'm', name: 'Maya Chen' }, 1000));
    expect(slots.map((s) => s.rank)).toEqual([2, 1, 3]);
    expect(slots[1].height).toBeGreaterThan(slots[0].height);
    expect(slots[0].height).toBeGreaterThan(slots[2].height);
  });

  it('shows two blocks when only two play', () => {
    const slots = podiumSlots(crewBoard([members[0], members[1]], { id: 'm', name: 'Maya Chen' }, 1000));
    expect(slots.map((s) => s.rank)).toEqual([2, 1]);
  });

  it('on your own, the result is this round against your best', () => {
    const solo = crewBoard([members[0]], { id: 'm', name: 'Maya Chen' }, 90);
    const rows = resultRows(solo, { id: 'm', name: 'Maya Chen' }, 90, 120);
    expect(rows.map((r) => r.name)).toEqual(['Your best', 'This round']);
    expect(resultRows(solo, { id: 'm' }, 90, 0).map((r) => r.name)).toEqual(['This round']);
  });
```

```
  it('with a crew, the result is the crew board', () => {
    const board = crewBoard(members, { id: 'm', name: 'Maya Chen' }, 90);
    expect(resultRows(board, { id: 'm' }, 90, 10)).toBe(board);
  });
});
```

src/components/play/__tests__/recapLogic.test.ts

```
import type { Activity } from '@/store/types';

import { placesVisited, shortName, yesVotes } from '../recapLogic';

const act = (title: string, type: Activity['type'], date: string, time = '10:00'): Pick<Activity, 'title' | 'type' | 'date' | 'time'> =>
({ title, type, date, time });

describe('placesVisited', () => {
  it('leaves out getting there and sleeping, keeps the order of the trip, and names each place once', () => {
    const list = [
      act('Villa Margherita', 'stay', '2026-10-30'),
      act('Path of the Gods', 'adventure', '2026-10-31'),
      act('Da Adolfo', 'food', '2026-10-30', '13:00'),
      act('Ferry to Capri', 'transport', '2026-11-01'),
      act('da adolfo ', 'food', '2026-11-02'),
    ];
    expect(placesVisited(list)).toEqual(['Da Adolfo', 'Path of the Gods']);
  });
  it('is empty for an empty trip', () => {
    expect(placesVisited([])).toEqual([]);
  });
});

describe('shortName', () => {
  it('keeps short names and cuts long ones at a word', () => {
    expect(shortName('Da Adolfo')).toBe('Da Adolfo');
    expect(shortName('Path of the Gods hiking trail')).toBe('Path of the…');
    expect(shortName('Supercalifragilisticexpialidocious')).toHaveLength(16);
  });
});

describe('yesVotes', () => {
  it('counts only the yes votes', () => {
    expect(yesVotes({ votes: { a: 1, b: -1, c: 1 } })).toBe(2);
  });
});
```

src/components/play/__tests__/recommend.test.ts

```
import { surveyQuestions } from '@/data/reference';
import { destinations } from '@/data/destinations';
import { photos } from '@/data/images';

import { optionArt, recommend, sameAnswer } from '../recommend';

describe('recommend', () => {
  it('returns five different destinations with a match from 70 to 99, best first', () => {
    const r = recommend([['beach', 'romantic'], ['food'], ['$3'], ['beach'], ['romantic']]);
    expect(r).toHaveLength(5);
    expect(new Set(r.map((x) => x.id)).size).toBe(5);
    for (const x of r) {
      expect(x.match).toBeGreaterThanOrEqual(70);
      expect(x.match).toBeLessThanOrEqual(99);
      expect(destinations.some((d) => d.id === x.id)).toBe(true);
    }
    expect(r[0].match).toBeGreaterThanOrEqual(r[4].match);
  });

  it('works with no answers at all', () => {
    expect(recommend([])).toHaveLength(5);
  });
});

describe('option art', () => {
  it('gives every answer in the survey its own art', () => {
    for (const q of surveyQuestions) for (const o of q.options) expect(optionArt(q, o.label)).not.toEqual({ icon: 'compass-outline' });
  });
  it('only uses photos the app ships', () => {
    for (const q of surveyQuestions) for (const o of q.options) {
      const a = optionArt(q, o.label);
      if ('photo' in a) expect(Object.keys(photos)).toContain(a.photo);
    }
  });
  it('falls back to a compass', () => {
    expect(optionArt({ id: 'nope' }, 'nothing')).toEqual({ icon: 'compass-outline' });
  });
});
```

```
describe('sameAnswer', () => {
  it('matches the same tags in the same order', () => {
    expect(sameAnswer(['a', 'b'], ['a', 'b'])).toBe(true);
    expect(sameAnswer(['a'], ['a', 'b'])).toBe(false);
    expect(sameAnswer(undefined, ['a'])).toBe(false);
  });
});
```

src/components/play/bucket.ts

```
/** How far each ticket on the bucket list is turned, in degrees: alternating left and right, a couple of degrees at most (like paper st
uck on a board). */
const TILTS = [-2, 1.5, -1.2, 2, -1.8, 1];

export function tiltFor(index: number): number {
  return TILTS[((index % TILTS.length) + TILTS.length) % TILTS.length];
}
```

src/components/play/packingLogic.ts

```
/** The small decisions the packing list makes, with no React in them so they can be tested. */
import type { PackingItem } from '@/store/types';

type Item = Pick<PackingItem, 'id' | 'packed' | 'broughtBy'>;

/** The things nobody is bringing yet (and that are not packed already). */
export function needsSomeone(items: Item[], memberIds: string[]): Item[] {
  return items.filter((p) => !p.packed && !(p.broughtBy && memberIds.includes(p.broughtBy)));
}

/** Hands each unassigned item to whoever is carrying the least so far. Returns who gets what; changes nothing itself. */
export function splitFairly(items: Item[], memberIds: string[]): { itemId: string; memberId: string }[] {
  if (!memberIds.length) return [];
  const load: Record<string, number> = {};
  memberIds.forEach((id) => (load[id] = items.filter((p) => p.broughtBy === id).length));
  return needsSomeone(items, memberIds).map((p) => {
    const next = memberIds.reduce((a, b) => (load[b] < load[a] ? b : a));
    load[next] += 1;
    return { itemId: p.id, memberId: next };
  });
}

/** True when ticking this one item packs the very last thing on a list that has more than one item. */
export function completesList(packed: number, total: number, wasPacked: boolean): boolean {
  return !wasPacked && total > 1 && packed + 1 === total;
}
```

src/components/play/quizLogic.ts

```
/**
 * The small decisions the travel quiz makes, with no React in them so they can be tested: how a round is scored,
 * what a result is called, who is on the crew board, and how the podium is laid out.
 */

/** Questions per round and seconds to answer each. */
export const ROUND = 8;
export const SECONDS = 20;

/** A shuffled copy (Fisher-Yates). `rnd` can be replaced in tests. */
export function shuffle<T>(a: T[], rnd: () => number = Math.random): T[] {
  const x = a.slice();
  for (let i = x.length - 1; i > 0; i--) {
    const j = Math.floor(rnd() * (i + 1));
    [x[i], x[j]] = [x[j], x[i]];
  }
  return x;
}

/** A right answer is worth 10, plus a bonus for the seconds left (one point per four seconds), plus 2 for every answer in a row before
it. */
export function pointsFor(secondsLeft: number, streak: number): number {
  return 10 + Math.max(0, Math.round(secondsLeft / 4)) + streak * 2;
}

/** A good round is 70% right or better (6 of 8). */
export function isGoodRound(correct: number, total: number): boolean {
  return total > 0 && correct / total >= 0.7;
}

/** What the round is called, from how many were right. */
export function roundTitle(correct: number, total: number): string {
  const share = total > 0 ? correct / total : 0;
  if (share >= 0.9) return 'Legendary navigator';
  if (share >= 0.7) return 'Seasoned traveller';
  if (share >= 0.4) return 'Getting there';
```

```
  return 'Every expert was once a tourist';
}

/** Should the round celebrate? A good round, or a score that beats the best (a first round with at least one point counts as a new bes
t). */
export function shouldCelebrate(correct: number, total: number, score: number, bestBefore: number): boolean {
  return isGoodRound(correct, total) || isNewBest(score, bestBefore);
}

export function isNewBest(score: number, bestBefore: number): boolean {
  return score > 0 && score > bestBefore;
}

export interface BoardRow {
  id: string;
  name: string;
  avatar?: string;
  score: number;
  me?: boolean;
}

/** A friendly, stable score for a crew member who has not played on this phone (the same name always gives the same number). */
export function crewScore(name: string): number {
  return 60 + ((name.charCodeAt(0) * 7 + name.length * 13) % 90);
}

/** The crew board, best first: everyone else gets a stable score, you get your real one. */
export function crewBoard(members: { id: string; name: string; avatar?: string }[], me: { id?: string; name?: string; avatar?: string },
mine: number): BoardRow[] {
  const others: BoardRow[] = members.filter((m) => m.id !== me.id).map((m) => ({ id: m.id, name: m.name, avatar: m.avatar, score: crewSc
ore(m.name) }));
  const you: BoardRow = { id: me.id ?? 'me', name: me.name ?? 'You', avatar: me.avatar, score: mine, me: true };
  return [...others, you].sort((a, b) => b.score - a.score);
}

/** How tall each podium block is, by place. */
export const PODIUM_HEIGHT = { 1: 112, 2: 84, 3: 60 } as const;

export interface PodiumSlot extends BoardRow {
  rank: 1 | 2 | 3;
  height: number;
}

/** The top three left to right as on a real podium: second, first, third (just second and first when only two play). */
export function podiumSlots(rows: BoardRow[]): PodiumSlot[] {
  const top: PodiumSlot[] = rows.slice(0, 3).map((r, i) => ({ ...r, rank: (i + 1) as 1 | 2 | 3, height: PODIUM_HEIGHT[(i + 1) as 1 | 2 |
3] }));
  const [first, second, third] = top;
  return [second, first, third].filter((s) => !!s);
}

/**
 * The rows of the result podium. With a crew it is the crew board with this round as your score; on your own it is this round
 * against your best (the higher one stands first).
 */
export function resultRows(board: BoardRow[], me: { id?: string; name?: string; avatar?: string }, score: number, bestBefore: number): B
oardRow[] {
  if (board.length >= 2) return board;
  const round: BoardRow = { id: 'round', name: 'This round', avatar: me.avatar, score, me: true };
  const best: BoardRow = { id: 'best', name: 'Your best', avatar: me.avatar, score: bestBefore };
  return bestBefore > 0 ? [round, best].sort((a, b) => b.score - a.score) : [round];
}
```

src/components/play/recapLogic.ts

```
/** What the trip recap counts, with no React in it so it can be tested. */
import type { Activity, Trip } from '@/store/types';

/** The places the crew went to: planned activities that are not just getting there or sleeping, each name once, in the order of the tri
p. */
export function placesVisited(activities: Pick<Activity, 'title' | 'type' | 'date' | 'time'>[]): string[] {
  const seen = new Set<string>();
  const out: string[] = [];
  for (const a of activities.slice().sort((x, y) => (x.date + x.time).localeCompare(y.date + y.time))) {
    if (a.type === 'transport' || a.type === 'stay') continue;
    const key = a.title.trim().toLowerCase();
    if (!key || seen.has(key)) continue;
    seen.add(key);
    out.push(a.title.trim());
  }
  return out;
}

/** A name short enough for a stamp: cut at a word where it can be, with an ellipsis. */
export function shortName(name: string, max = 16): string {
  if (name.length <= max) return name;
```

```
  const cut = name.slice(0, max - 1);
  const space = cut.lastIndexOf(' ');
  return `${(space > 6 ? cut.slice(0, space) : cut).trimEnd()}…`;
}

/** The yes votes an activity has. */
export const yesVotes = (a: Pick<Activity, 'votes'>) => Object.values(a.votes).filter((v) => v === 1).length;

/** Every vote cast on the trip: on activities and in polls. */
export function votesCast(trip: Pick<Trip, 'activities' | 'polls'>): number {
  return trip.activities.reduce((n, a) => n + Object.keys(a.votes).length, 0) + trip.polls.reduce((n, p) => n + p.options.reduce((m, o)
=> m + o.votes.length, 0), 0);
}
```

src/components/play/recommend.ts

```
import { destinations } from '@/data/destinations';
import type { SurveyQuestion } from '@/data/reference';
import type { IconName } from '@/components/ui/bits';

export interface Match {
  id: string;
  match: number;
}

/** Scores every destination against the answers (tag matches + budget fit + rating) and returns the best five with a 70 to 99 match. */
export function recommend(answers: string[][]): Match[] {
  const tags = answers.flat();
  const budgetTag = tags.find((t) => t.startsWith('$'));
  const budget = budgetTag ? Number(budgetTag.slice(1)) : 2;
  const scored = destinations.map((d) => {
    let s = 0;
    for (const t of tags) if (!t.startsWith('$') && d.tags.includes(t as never)) s += 2;
    s -= Math.abs(d.priceLevel - budget) * 1.5;
    s += (d.rating - 4.5) * 4;
    return { id: d.id, s };
  });
  const max = Math.max(...scored.map((x) => x.s));
  const min = Math.min(...scored.map((x) => x.s));
  return scored
    .sort((a, b) => b.s - a.s)
    .slice(0, 5)
    .map((x) => ({ id: x.id, match: Math.round(70 + ((x.s - min) / Math.max(1, max - min)) * 29) }));
}

/** The picture on an answer tile: a bundled photo, a line icon, or a few characters in mono (the dollar signs of a budget). */
export type OptionArt = { photo: string } | { icon: IconName } | { glyph: string };

const ART: Record<string, OptionArt> = {
  'vibe:City lights': { photo: 'shibuya' },
  'vibe:Beach & sun': { photo: 'maldives' },
  'vibe:Mountains & trails': { photo: 'swiss-alps' },
  'vibe:History & culture': { photo: 'rome' },
  'food:It’s the whole point': { icon: 'restaurant-outline' },
  'food:Love a good meal': { icon: 'pizza-outline' },
  'food:Fuel for adventures': { icon: 'nutrition-outline' },
  'food:Not fussed': { icon: 'hand-left-outline' },
  'budget:Under $90': { glyph: '$' },
  'budget:$90 – $160': { glyph: '$$' },
  'budget:$160 – $230': { glyph: '$$$' },
  'budget:Treat ourselves': { glyph: '$$$$' },
  'pace:Packed schedule': { icon: 'flash-outline' },
  'pace:Balanced': { icon: 'scale-outline' },
  'pace:Slow & relaxed': { icon: 'leaf-outline' },
  'pace:Spontaneous': { icon: 'dice-outline' },
  'who:Friends': { icon: 'people-outline' },
  'who:Family with kids': { icon: 'home-outline' },
  'who:Partner': { icon: 'heart-outline' },
  'who:Classmates / club': { icon: 'school-outline' },
};

/** The art for one answer; an answer with no art of its own gets a plain compass. */
export function optionArt(q: Pick<SurveyQuestion, 'id'>, label: string): OptionArt {
  return ART[`${q.id}:${label}`] ?? { icon: 'compass-outline' };
}

/** True when `tags` is the answer already given for this step (so a tile can show it as chosen when you come back). */
export function sameAnswer(given: string[] | undefined, tags: string[]): boolean {
  return !!given && given.length === tags.length && given.every((t, i) => t === tags[i]);
}
```

src/components/settings/AccessibilityCard.tsx

```
import { StyleSheet, View } from 'react-native';

import { Chip, ToggleRow } from '@/components/ui/bits';
```

```
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { useOsReduceMotion } from '@/hooks/useReduceMotion';
import type { TextScale } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';

import { Group } from './Group';

const SCALES: { v: TextScale; l: string }[] = [
  { v: 1, l: 'Default' },
  { v: 1.12, l: 'Large' },
  { v: 1.25, l: 'Extra large' },
];

/** Bigger text, haptics and the animations switch. A group inside the Everyday card. */
export function AccessibilityCard() {
  const t = useT();
  const settings = useAppStore((s) => s.settings);
  const update = useAppStore((s) => s.updateSettings);
  const motion = useAppStore((s) => s.personal.motion);
  const updatePersonal = useAppStore((s) => s.updatePersonal);
  const osReduceMotion = useOsReduceMotion((s) => s.on);

  return (
    <Group label="Text and touch" gap={6}>
      <T variant="small" weight="semibold">
        {t('settings.textSize')}
      </T>
      <View style={styles.wrap}>
        {SCALES.map((s) => (
          <Chip key={s.v} label={s.l} active={settings.textScale === s.v} onPress={() => update({ textScale: s.v })} />
        ))}
      </View>
      <ToggleRow icon="phone-portrait-outline" label="Haptic feedback" sub="A small tick when you tap" value={settings.haptics} onChange
={(v) => update({ haptics: v })} />
      <ToggleRow
        icon="film-outline"
        label="Animations"
        sub={osReduceMotion ? 'Off for now: your phone’s Reduce Motion setting is on' : 'Flipping numbers, floating cards and smooth ent
rances'}
        value={motion && !osReduceMotion}
        onChange={(v) => updatePersonal({ motion: v })}
      />
    </Group>
  );
}

const styles = StyleSheet.create({
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
});
```

src/components/settings/AssistantCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card, Chip } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { CLAUDE_MODEL } from '@/services/assistant';
import { getApiKey, looksLikeApiKey, setApiKey } from '@/services/secrets';
import { useAppStore } from '@/store/useAppStore';
import { colors, hair, radius } from '@/theme';

/** Offline answers or Claude, and the optional API key (kept in the phone's secure keychain). */
export function AssistantCard() {
  const settings = useAppStore((s) => s.settings);
  const update = useAppStore((s) => s.updateSettings);
  const [key, setKey] = useState('');
  const [hasKey, setHasKey] = useState(false);
  const [keyErr, setKeyErr] = useState<string | null>(null);
  const [show, setShow] = useState(false);

  useEffect(() => {
    getApiKey().then((k) => setHasKey(!!k));
  }, []);

  const saveKey = async () => {
    if (!looksLikeApiKey(key)) return setKeyErr('Anthropic keys start with “sk-ant-”. Check you copied the whole key.');
    await setApiKey(key.trim());
    setKey('');
    setKeyErr(null);
```

```
    setHasKey(true);
    update({ aiMode: 'claude' });
    toast('Key saved securely. Claude mode is on');
  };

  return (
    <Card style={{ marginTop: 14, gap: 12 }}>
      <T variant="kicker" color={colors.textSecondary} accessibilityRole="header">
        Trip Assistant
      </T>
      <View style={styles.wrap}>
        <Chip icon="cloud-offline-outline" label="Offline (built-in)" active={settings.aiMode === 'offline'} onPress={() => update({ aiM
ode: 'offline' })} />
        <Chip
          icon="cloud-outline"
          label="Claude AI"
          active={settings.aiMode === 'claude'}
          onPress={() => {
            if (!hasKey) return toast('Add your Anthropic API key first', { tone: 'warn' });
            update({ aiMode: 'claude' });
          }}
        />
      </View>
      <T variant="caption" color={colors.textSecondary}>
        Offline mode answers from your trip data with no internet. Claude mode sends questions plus trip details to Anthropic’s {CLAUDE_
MODEL} model using your own key, which is stored in the phone’s
        secure keychain.
      </T>
      {hasKey ? (
        <View style={styles.keyRow}>
          <Ionicons name="key-outline" size={18} color={colors.ink} />
          <T variant="small" weight="semibold" style={{ flex: 1 }}>
            API key saved securely
          </T>
          <Press
            onPress={async () => {
              await setApiKey(null);
              setHasKey(false);
              update({ aiMode: 'offline' });
              toast('Key removed');
            }}
            accessibilityLabel="Remove the API key"
            style={styles.remove}>
            <T variant="small" weight="bold" color={colors.red}>
              Remove
            </T>
          </Press>
        </View>
      ) : (
        <>
          <Input
            placeholder="sk-ant-…"
            value={key}
            onChangeText={(v) => {
              setKey(v);
              setKeyErr(null);
            }}
            secureTextEntry={!show}
            right={
              <Press onPress={() => setShow((v) => !v)} accessibilityLabel={show ? 'Hide the key' : 'Show the key'} style={styles.eye}>
                <Ionicons name={show ? 'eye-off-outline' : 'eye-outline'} size={19} color={colors.textMuted} />
              </Press>
            }
            style={{ minHeight: 44 }}
            autoCapitalize="none"
            autoCorrect={false}
            error={keyErr}
            icon="key-outline"
            accessibilityLabel="Anthropic API key"
          />
          <Button label="Save key" size="md" variant="secondary" onPress={saveKey} disabled={!key} />
        </>
      )}
    </Card>
  );
}

const styles = StyleSheet.create({
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  keyRow: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
    paddingLeft: 12,
    minHeight: 52,
    borderRadius: radius.button,
```

```
    borderWidth: hair,
    borderColor: colors.borderStrong,
    backgroundColor: colors.bgAlt,
  },
  eye: { width: 44, height: 44, alignItems: 'center', justifyContent: 'center', marginRight: -10 },
  remove: { minWidth: 44, minHeight: 44, paddingHorizontal: 12, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/settings/DataCard.tsx

```
import { router } from 'expo-router';

import { Card, Divider, ListRow } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { shareText } from '@/services/share';
import { resetSync } from '@/sync/actions';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';

/** Export, clearing the sample trips and resetting the app. */
export function DataCard() {
  const trip = useActiveTrip();
  const resetEverything = useAppStore((s) => s.resetEverything);
  const clearDemoData = useAppStore((s) => s.clearDemoData);
  const ensureSeed = useAppStore((s) => s.ensureSeed);

  return (
    <Card style={{ marginTop: 14 }}>
      {trip ? (
        <>
          <ListRow
            icon="download-outline"
            label="Export this trip"
            sub="Share a JSON backup of the itinerary, budget and checklist"
            onPress={() => {
              const { messages, photos, docs, ...rest } = trip;
              void messages;
              void photos;
              void docs;
              shareText(JSON.stringify(rest, null, 1), `${trip.name} backup`);
            }}
          />
          <Divider />
        </>
      ) : null}
      <ListRow
        icon="leaf-outline"
        label="Clear the sample trips"
        sub="Start with a blank app. Your own trips and accounts stay"
        onPress={() =>
          confirmAction(
            'Clear the sample trips?',
            'Amalfi Squad and the other example trips are removed from this phone. Trips you made yourself are not touched.',
            () => {
              clearDemoData();
              toast('Sample trips cleared');
            },
            { destructive: true, confirmLabel: 'Clear' },
          )
        }
      />
      <Divider />
      <ListRow
        icon="refresh-circle-outline"
        tint={colors.red}
        label="Reset app data"
        sub="Deletes all trips and accounts on this device"
        onPress={() =>
          confirmAction(
            'Reset everything?',
            'All trips, chats and accounts on this device will be erased and the demo data restored.',
            async () => {
              await resetSync(); // also forget which trips were shared with friends
              resetEverything();
              await ensureSeed();
              router.replace('/sign-in');
            },
            { destructive: true, confirmLabel: 'Reset' },
          )
        }
      />
    </Card>
  );
}
```

src/components/settings/DemoCard.tsx

```
import { StyleSheet, View } from 'react-native';

import { Card, Chip } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';

/** Re-creates the sample trip as upcoming, in progress or finished. */
export function DemoCard() {
  const reseed = useAppStore((s) => s.reseedDemo);
  const timeline = useAppStore((s) => s.demoTimeline);

  return (
    <Card style={{ marginTop: 14, gap: 12 }}>
      <T variant="kicker" color={colors.textSecondary} accessibilityRole="header">
        Demo timeline
      </T>
      <T variant="caption" color={colors.textSecondary}>
        Re-create the sample Amalfi Squad trip as upcoming, in progress or finished to show every stage from planning through completio
n.
      </T>
      <View style={styles.wrap}>
        {(
          [
            ['upcoming', 'Starts in 3 days'],
            ['active', 'Day 2 in progress'],
            ['completed', 'Just finished'],
          ] as const
        ).map(([k, l]) => (
          <Chip
            key={k}
            small
            label={l}
            active={timeline === k}
            onPress={() =>
              confirmAction('Reset the demo trip?', `“Amalfi Squad” will be rebuilt as “${l}”. Your other trips are untouched.`, () => {
                reseed(k);
                toast('Demo trip updated');
              })
            }
          />
        ))}
      </View>
    </Card>
  );
}

const styles = StyleSheet.create({
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
});
```

src/components/settings/Group.tsx

```
import type { ReactNode } from 'react';
import { View } from 'react-native';

import { T } from '@/components/ui/T';
import { colors } from '@/theme';

/** One labelled group inside a card: a small caps label over its controls (Language, Reminders, Text and touch). */
export function Group({ label, children, gap = 10 }: { label: string; children: ReactNode; gap?: number }) {
  return (
    <View style={{ gap }}>
      <T variant="kicker" color={colors.textSecondary} accessibilityRole="header">
        {label}
      </T>
      {children}
    </View>
  );
}
```

src/components/settings/HomeLayoutCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, Switch, View } from 'react-native';

import { Card, Chip, Divider } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { HOME_BLOCKS, LAYOUTS, layoutAnswers, type Preset } from '@/data/layouts';
```

```
import { useAppStore } from '@/store/useAppStore';
import { colors, hair, radius } from '@/theme';

import { switchThumb } from '@/components/personalize/kit';
import { allBlocks, movedBlock, shownCount, switchedBlock } from './blocks';
import { Group } from './Group';

const PRESETS = Object.keys(LAYOUTS) as Preset[];

/** A square arrow button, 44 x 44, that moves a Home block up or down. */
function Move({ dir, label, disabled, onPress }: { dir: 'up' | 'down'; label: string; disabled: boolean; onPress: () => void }) {
  return (
    <Press onPress={onPress} disabled={disabled} accessibilityLabel={label} accessibilityState={{ disabled }} style={[styles.move, disab
led && { opacity: 0.3 }]}>
      <Ionicons name={dir === 'up' ? 'arrow-up' : 'arrow-down'} size={18} color={colors.ink} />
    </Press>
  );
}

/**
 * What Home shows and in what order. Three ready-made layouts (Planner, Money, Party) first, and the blocks one by one under
 * "Arrange blocks": a switch to hide each, and arrows to move it. Changing anything by hand makes the layout "Custom".
 */
export function HomeLayoutCard() {
  const order = useAppStore((s) => s.personal.order);
  const sections = useAppStore((s) => s.personal.sections);
  const layout = useAppStore((s) => s.personal.layout);
  const updatePersonal = useAppStore((s) => s.updatePersonal);
  const [open, setOpen] = useState(false);

  const answers = { order, sections };
  const all = allBlocks(order);
  const shown = shownCount(answers);
  const chosen = layout !== 'custom' ? LAYOUTS[layout] : null;

  return (
    <Card padded={false}>
      <View style={styles.top}>
        <Group label="Layout">
          <View style={styles.wrap} accessibilityRole="radiogroup" accessibilityLabel="Home layout">
            {PRESETS.map((k) => (
              <Chip key={k} label={LAYOUTS[k].name} active={layout === k} onPress={() => updatePersonal(layoutAnswers(k))} />
            ))}
            {layout === 'custom' ? <Stamp label="Custom" tone="muted" rotate={0} /> : null}
          </View>
          <T variant="caption" color={colors.textSecondary}>
            {chosen ? chosen.blurb : 'Arranged by hand. Pick a layout to start over.'}
          </T>
        </Group>
      </View>
      <Divider />
      <Press onPress={() => setOpen((o) => !o)} style={styles.head} scaleTo={0.99} accessibilityLabel="Arrange blocks" accessibilityStat
e={{ expanded: open }}>
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold">
            Arrange blocks
          </T>
          <T variant="caption" color={colors.textSecondary}>
            {shown} of {all.length} showing. The boarding pass is always first.
          </T>
        </View>
        <Ionicons name={open ? 'chevron-up' : 'chevron-down'} size={18} color={colors.ink} />
      </Press>
      {open ? (
        <View style={styles.list}>
          {all.map((k, i) => {
            const on = sections[k];
            const name = HOME_BLOCKS[k].label;
            return (
              <View key={k}>
                <Divider />
                <View style={styles.row}>
                  <T variant="kicker" color={colors.textMuted} style={{ width: 22 }}>
                    {String(i + 1).padStart(2, '0')}
                  </T>
                  <T variant="bodySm" weight="semibold" color={on ? colors.text : colors.textMuted} style={{ flex: 1 }} numberOfLines=
{2}>
                    {name}
                  </T>
                  <Move dir="up" label={`Move ${name} up`} disabled={i === 0} onPress={() => updatePersonal(movedBlock(answers, k, -1))}
/>
                  <Move dir="down" label={`Move ${name} down`} disabled={i === all.length - 1} onPress={() => updatePersonal(movedBlock
(answers, k, 1))} />
                  <Switch
                    accessibilityLabel={`Show ${name} on Home`}
                    value={on}
```

```
                    onValueChange={(v) => updatePersonal(switchedBlock(answers, k, v))}
                    trackColor={{ false: colors.borderStrong, true: colors.ink }}
                    thumbColor={switchThumb(colors)}
                    {...({ activeThumbColor: colors.accent } as object)}
                  />
                </View>
              </View>
            );
          })}
        </View>
      ) : null}
    </Card>
  );
}

const styles = StyleSheet.create({
  top: { padding: 16 },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', gap: 8 },
  head: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 16, paddingVertical: 12, minHeight: 64 },
  list: { paddingHorizontal: 16 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 6, minHeight: 56 },
  move: { width: 44, height: 44, borderRadius: radius.button, borderWidth: hair, borderColor: colors.borderStrong, alignItems: 'center',
justifyContent: 'center' },
});
```

src/components/settings/LookCard.tsx

```
import { useMemo, useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { commitLook, currentLook } from '@/components/personalize/applyLook';
import { modeAfterPickingTheme } from '@/components/personalize/look';
import { EFFECT_OPTIONS, MODES } from '@/components/personalize/options';
import { paletteOf } from '@/components/personalize/palette';
import { MiniTicket, ThemeSwatch } from '@/components/personalize/parts';
import { Button } from '@/components/ui/Button';
import { Card, Chip, Divider } from '@/components/ui/bits';
import { celebrate } from '@/components/ui/Confetti';
import { toast } from '@/components/ui/feedback';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, cornerStyles, line, radius, themes, THEME_KEYS, type CornerStyle, type ModePref, type ThemeKey } from '@/theme';

import { Group } from './Group';

const CORNERS = Object.keys(cornerStyles) as CornerStyle[];

/**
 * The look of the app: six themes, light or dark, corner style and how big the celebrations are.
 * Theme, mode and corners are chosen first and then put on with one button (the app restarts to redraw everything);
 * the small ticket shows what you would get. Effects take hold at once and play a preview.
 */
export function LookCard() {
  const effects = useAppStore((s) => s.personal.effects);
  const updatePersonal = useAppStore((s) => s.updatePersonal);
  // the look as chosen here, and the one that is on the phone right now
  const [start] = useState(currentLook);
  const [theme, setTheme] = useState<ThemeKey>(start.theme);
  const [mode, setMode] = useState<ModePref>(start.mode);
  const [corners, setCorners] = useState<CornerStyle>(start.corners);
  const [on, setOn] = useState(start);
  const [busy, setBusy] = useState(false);

  const tp = useMemo(() => paletteOf(theme, mode), [theme, mode]);
  const changed = theme !== on.theme || mode !== on.mode || corners !== on.corners;

  const apply = async () => {
    setBusy(true);
    const result = await commitLook({ theme, mode, corners });
    if (result === 'reloading') return; // the app is restarting
    setBusy(false);
    if (result === 'failed') toast('Saved. Close and reopen the app to see your new look.', { icon: 'refresh' });
    else toast('Look saved');
    setOn({ theme, mode, corners });
  };

  return (
    <Card plated>
      <View style={styles.stack}>
        <Group label="Theme">
          <View style={styles.swatches} accessibilityRole="radiogroup" accessibilityLabel="Colour theme">
            {THEME_KEYS.map((k) => (
              <ThemeSwatch
                key={k}
                pal={colors}
```

```
                theme={k}
                mode={mode}
                selected={theme === k}
                onPress={() => {
                  setTheme(k);
                  setMode((m) => modeAfterPickingTheme(k, m));
                }}
              />
            ))}
          </View>
          <T variant="small" color={colors.textSecondary} accessibilityLiveRegion="polite">
            {themes[theme].name}: {themes[theme].vibe.toLowerCase()}
          </T>
        </Group>

        <Group label="Light or dark">
          <Segmented options={MODES.map((m) => ({ key: m.key, label: m.label }))} value={mode} onChange={setMode} />
        </Group>

        <Group label="Corners">
          <View style={styles.wrap}>
            {CORNERS.map((c) => (
              <Chip key={c} label={`${cornerStyles[c].name} corners`} active={corners === c} onPress={() => setCorners(c)} />
            ))}
          </View>
        </Group>

        <View style={[styles.desk, { backgroundColor: tp.accent }]} accessibilityLabel={`Preview of ${themes[theme].name}`}>
          <MiniTicket tp={tp} />
        </View>
        {changed ? (
          <View style={{ gap: 8 }}>
            <Button label="Apply look" icon="color-palette-outline" size="md" loading={busy} onPress={apply} />
            <T variant="caption" color={colors.textMuted}>
              The app restarts to redraw everything in the new look.
            </T>
          </View>
        ) : null}

        <Divider />
        <Group label="Effects">
          <View style={styles.wrap} accessibilityRole="radiogroup" accessibilityLabel="Effects">
            {EFFECT_OPTIONS.map((o) => (
              <Chip
                key={o.key}
                label={o.label}
                active={effects === o.key}
                onPress={() => {
                  updatePersonal({ effects: o.key });
                  celebrate(o.key === 'chill' ? 'Chill: a calm tick, no confetti' : undefined, o.key);
                }}
              />
            ))}
          </View>
          <T variant="caption" color={colors.textSecondary}>
            {EFFECT_OPTIONS.find((o) => o.key === effects)?.sub}. Tap one to try it.
          </T>
        </Group>
      </View>
    </Card>
  );
}

const styles = StyleSheet.create({
  stack: { gap: 16 },
  swatches: { flexDirection: 'row', flexWrap: 'wrap', gap: 5 },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  desk: { padding: 12, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink },
});
```

src/components/settings/NotificationsCard.tsx

```
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Chip, ToggleRow } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { ensurePermission, sendTestReminder, setTriviaReminder } from '@/services/notifications';
import { TRIVIA_REMINDER } from '@/data/personal';
import { useAppStore } from '@/store/useAppStore';

import { Group } from './Group';

/** Reminder switch, how early to remind, the daily trivia nudge, and a test button. A group inside the Everyday card. */
```

```
export function NotificationsCard() {
  const t = useT();
  const settings = useAppStore((s) => s.settings);
  const update = useAppStore((s) => s.updateSettings);
  const trivia = useAppStore((s) => s.personal.triviaReminder);
  const updatePersonal = useAppStore((s) => s.updatePersonal);

  return (
    <Group label="Reminders" gap={6}>
      <ToggleRow
        icon="notifications-outline"
        label={t('settings.notifications')}
        sub="An alert before each activity that has a reminder on"
        value={settings.notificationsEnabled}
        onChange={async (v) => {
          update({ notificationsEnabled: v });
          if (v) {
            const ok = await ensurePermission();
            toast(ok ? 'Reminders on' : 'Allow notifications in your phone settings', { tone: ok ? 'ok' : 'warn' });
          }
        }}
      />
      <T variant="small" weight="semibold">
        {t('settings.lead')}
      </T>
      <View style={styles.wrap}>
        {[10, 30, 60, 120].map((m) => (
          <Chip key={m} small label={m < 60 ? `${m} min` : `${m / 60} hr`} active={settings.reminderLeadMin === m} onPress={() => update
({ reminderLeadMin: m })} />
        ))}
      </View>
      <ToggleRow
        icon="help-circle-outline"
        label="Daily trivia reminder"
        sub={`One question a day at ${TRIVIA_REMINDER.timeLabel}`}
        value={trivia}
        onChange={async (v) => {
          updatePersonal({ triviaReminder: v });
          const ok = await setTriviaReminder(v);
          if (!ok) {
            updatePersonal({ triviaReminder: false });
            toast('Allow notifications in your phone settings to get the reminder', { tone: 'warn', icon: 'notifications-off-outline'
});
          }
        }}
      />
      <Button
        label="Send a test reminder"
        variant="soft"
        size="md"
        icon="alarm-outline"
        onPress={() => {
          sendTestReminder(5);
          toast('Arriving in 5 seconds', { icon: 'alarm-outline' });
        }}
        style={{ marginTop: 6 }}
      />
    </Group>
  );
}

const styles = StyleSheet.create({
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
});
```

src/components/settings/ProfileCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Avatar, Card, Chip, Divider } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { pickImage } from '@/services/files';
import { useAppStore, useCurrentUser } from '@/store/useAppStore';
import { colors, line } from '@/theme';
import { maxLen, validateName } from '@/utils/validation';

const AVATARS = ['avatar-maya', 'avatar-diego', 'avatar-priya', 'avatar-sam', 'avatar-ana', 'avatar-noah'];

/** Who you are in one line; tap it to change the name, home city or avatar (a built-in one, a photo, or just initials). */
export function ProfileCard() {
```

```
  const user = useCurrentUser();
  const updateProfile = useAppStore((s) => s.updateProfile);
  const [name, setName] = useState(user?.name ?? '');
  const [city, setCity] = useState(user?.homeCity ?? '');
  const [errors, setErrors] = useState<Record<string, string | null>>({});
  const [open, setOpen] = useState(false);

  const saveProfile = () => {
    const e = { name: validateName(name), city: maxLen(40, 'City')(city) };
    setErrors(e);
    if (e.name || e.city) return tap('warning');
    updateProfile({ name: name.trim(), homeCity: city.trim() || undefined });
    toast('Profile saved');
    setOpen(false);
  };

  const pickAvatarPhoto = async () => {
    try {
      const f = await pickImage();
      if (f) {
        updateProfile({ avatar: f.uri });
        toast('Profile photo updated');
      }
    } catch {
      toast('Couldn’t open your photos', { tone: 'warn' });
    }
  };

  return (
    <Card padded={false}>
      <Press onPress={() => setOpen((o) => !o)} style={styles.summary} scaleTo={0.99} accessibilityLabel={open ? 'Close the profile edit
or' : 'Edit profile'} accessibilityState={{ expanded: open }}>
        <Avatar name={user?.name ?? 'You'} src={user?.avatar} size={48} />
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold" numberOfLines={1}>
            {user?.name ?? 'You'}
          </T>
          <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
            {[user?.email, user?.homeCity].filter(Boolean).join(' · ')}
          </T>
        </View>
        <T variant="kicker">{open ? 'Close' : 'Edit'}</T>
        <Ionicons name={open ? 'chevron-up' : 'chevron-down'} size={16} color={colors.ink} />
      </Press>
      {open ? (
        <View style={styles.body}>
          <Divider />
          <T variant="kicker" color={colors.textSecondary}>
            Picture
          </T>
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 8, paddingRight: 4 }}>
            {AVATARS.map((a) => {
              const on = user?.avatar === a;
              return (
                <Press
                  key={a}
                  onPress={() => updateProfile({ avatar: a })}
                  accessibilityLabel="Choose this avatar"
                  accessibilityState={{ selected: on }}
                  style={[styles.avatarOpt, on && { backgroundColor: colors.accent }]}>
                  <Avatar name="Avatar" src={a} size={36} />
                </Press>
              );
            })}
          </ScrollView>
          <View style={{ flexDirection: 'row', gap: 8 }}>
            <Chip small icon="image-outline" label="Upload photo" onPress={pickAvatarPhoto} />
            <Chip small icon="text-outline" label="Initials" onPress={() => updateProfile({ avatar: undefined })} />
          </View>
          <Input label="Name" value={name} onChangeText={setName} error={errors.name} icon="person-outline" style={styles.field} />
          <Input label="Home city" value={city} onChangeText={setCity} error={errors.city} icon="home-outline" placeholder="e.g. Middlet
own, DE" style={styles.field} />
          <T variant="caption" color={colors.textSecondary}>
            Signed in as {user?.email} · {user?.provider === 'google' ? 'Google' : 'email & password'}
          </T>
          <Button label="Save profile" size="md" onPress={saveProfile} />
        </View>
      ) : null}
    </Card>
  );
}

const styles = StyleSheet.create({
  summary: { flexDirection: 'row', alignItems: 'center', gap: 14, padding: 16, minHeight: 80 },
  body: { gap: 12, paddingHorizontal: 16, paddingBottom: 16 },
  field: { minHeight: 44 },
```

```
  avatarOpt: { width: 48, height: 48, borderRadius: 24, borderWidth: line, borderColor: colors.ink, alignItems: 'center', justifyConten
t: 'center' },
});
```

src/components/settings/__tests__/blocks.test.ts

```
import { layoutAnswers } from '@/data/layouts';

import { allBlocks, movedBlock, shownCount, switchedBlock } from '../blocks';

describe('the Home blocks list in Settings', () => {
  it('lists every block once, in the saved order, adding one a later version brought and dropping unknown names', () => {
    const planner = layoutAnswers('planner');
    expect(allBlocks(planner.order)).toEqual(planner.order);
    const without = planner.order.filter((k) => k !== 'crew');
    expect(allBlocks(without)).toEqual([...without, 'crew']);
    expect(allBlocks(['nonsense', ...planner.order])).toEqual(planner.order);
  });

  it('becomes Custom as soon as a block is moved or switched by hand', () => {
    const a = layoutAnswers('planner');
    expect(movedBlock(a, 'weather', -1).layout).toBe('custom');
    expect(switchedBlock(a, 'discover', false).layout).toBe('custom');
  });

  it('goes back to a ready-made layout when the changes are undone', () => {
    const a = layoutAnswers('money');
    const moved = movedBlock(a, 'upnext', 1);
    expect(moved.layout).toBe('custom');
    expect(movedBlock(moved, 'upnext', -1).layout).toBe('money');
    const off = switchedBlock(a, 'weather', false);
    expect(switchedBlock(off, 'weather', true).layout).toBe('money');
  });

  it('keeps the order when a block is only switched, and the switches when a block is only moved', () => {
    const a = layoutAnswers('planner');
    expect(switchedBlock(a, 'play', false).order).toEqual(a.order);
    expect(movedBlock(a, 'play', -1).sections).toEqual(a.sections);
  });

  it('stops at the ends of the list', () => {
    const a = layoutAnswers('planner');
    expect(movedBlock(a, a.order[0], -1).order).toEqual(a.order);
    expect(movedBlock(a, a.order[a.order.length - 1], 1).order).toEqual(a.order);
  });

  it('counts the blocks that are showing', () => {
    expect(shownCount(layoutAnswers('planner'))).toBe(7);
    expect(shownCount(layoutAnswers('money'))).toBe(6);
  });
});
```

src/components/settings/blocks.ts

```
/** Pure helpers for the Home blocks list in Settings. No React Native in here, so tests can check it. */
import { ALL_SECTIONS, HOME_BLOCKS, layoutOf, moveSection } from '@/data/layouts';
import type { HomeLayout, HomeSection } from '@/store/types';

export interface BlockAnswers {
  order: HomeSection[];
  sections: Record<HomeSection, boolean>;
}

/** Every Home block in the saved order: unknown names are dropped, and a block a later version added is put at the end. */
export function allBlocks(order: string[]): HomeSection[] {
  const known = order.filter((k): k is HomeSection => k in HOME_BLOCKS);
  return [...known, ...ALL_SECTIONS.filter((k) => !known.includes(k))];
}

/** The answers after moving one block up (-1) or down (1). The layout becomes 'custom' unless that happens to match a ready-made one.
*/
export function movedBlock(a: BlockAnswers, key: HomeSection, by: -1 | 1): BlockAnswers & { layout: HomeLayout } {
  const order = moveSection(allBlocks(a.order), key, by);
  return { order, sections: a.sections, layout: layoutOf({ order, sections: a.sections }) };
}

/** The answers after switching one block on or off. */
export function switchedBlock(a: BlockAnswers, key: HomeSection, on: boolean): BlockAnswers & { layout: HomeLayout } {
  const order = allBlocks(a.order);
  const sections = { ...a.sections, [key]: on };
  return { order, sections, layout: layoutOf({ order, sections }) };
}

/** How many blocks Home is showing. */
export function shownCount(a: BlockAnswers): number {
```

```
  return allBlocks(a.order).filter((k) => a.sections[k]).length;
}
```

src/components/sync/SyncBadge.tsx

```
import { Pill } from '@/components/ui/bits';
import { useSyncStatus } from '@/sync/status';

/** A small stamp that says whether a shared trip is up to date: "Live · 2 phones", "Offline · 3 waiting"... */
export function SyncBadge({ tripId }: { tripId: string }) {
  const shared = useSyncStatus((s) => s.shared.includes(tripId));
  const online = useSyncStatus((s) => s.online);
  const pending = useSyncStatus((s) => s.pending);
  const peers = useSyncStatus((s) => s.peers);
  if (!shared) return null;
  if (online === false) return <Pill label={pending ? `Offline · ${pending} waiting` : 'Offline'} tone="orange" />;
  if (online === null) return <Pill label="Connecting" tone="gray" />;
  return <Pill label={peers > 1 ? `Live · ${peers} phones` : 'Live'} tone="green" />;
}
```

src/components/sync/SyncSettingsCard.tsx

```
import { useState } from 'react';
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { testServer, type ConnectionTest } from '@/sync/actions';
import { SERVER_PORT, guessServerUrl, useSyncPrefs } from '@/sync/prefs';
import { useSyncStatus } from '@/sync/status';
import { normalizeServerUrl } from '@/sync/transport';
import { colors } from '@/theme';

/** In Settings: where the sync server is, a test button, and how syncing is going. */
export function SyncSettingsCard() {
  const saved = useSyncPrefs((s) => s.serverUrl);
  const setServerUrl = useSyncPrefs((s) => s.setServerUrl);
  const status = useSyncStatus();
  const guess = guessServerUrl();
  const [text, setText] = useState(saved);
  const [result, setResult] = useState<ConnectionTest | null>(null);
  const [testing, setTesting] = useState(false);

  const address = text.trim() || guess || '';

  const save = () => {
    const url = text.trim();
    if (url && !normalizeServerUrl(url)) return toast('That doesn’t look like an address', { tone: 'warn' });
    setServerUrl(url ? normalizeServerUrl(url)! : '');
    toast(url ? 'Sync address saved' : 'Using the automatic address');
  };

  const test = async () => {
    setTesting(true);
    setResult(await testServer(address));
    setTesting(false);
  };

  return (
    <Card style={{ gap: 12, marginTop: 14 }}>
      <T variant="kicker" color={colors.textSecondary} accessibilityRole="header">
        Sync across phones
      </T>
      <T variant="small" color={colors.textSecondary}>
        To plan together on different phones: on the computer that runs the app, start the sync server with{' '}
        <T variant="small" weight="bold">
          npm run server
        </T>
        , then make sure every phone has its address.
      </T>
      <Input
        label="Server address"
        value={text}
        onChangeText={(v) => {
          setText(v);
          setResult(null);
        }}
        onBlur={save}
        placeholder={guess ? guess.replace(/^https?:\/\//, '') : `192.168.1.20:${SERVER_PORT}`}
        autoCapitalize="none"
        autoCorrect={false}
        keyboardType="url"
```

```
        accessibilityLabel="Sync server address"
        testID="sync-address"
        style={{ minHeight: 44 }}
        hint={saved || !guess ? 'The address printed by the server.' : `Found automatically: ${guess}. Leave this blank to use it.`}
      />
      <View style={{ flexDirection: 'row', gap: 12 }}>
        <Button label="Test connection" icon="pulse-outline" variant="secondary" size="md" loading={testing} onPress={test} style={{ fle
x: 3 }} testID="sync-test" />
        <Button label="Save" icon="checkmark" size="md" onPress={save} style={{ flex: 2 }} />
      </View>
      {result ? (
        <View accessibilityLiveRegion="polite" testID="sync-result" style={{ gap: 6 }}>
          <Stamp label={result.ok ? 'Connected' : 'No answer'} tone={result.ok ? 'positive' : 'danger'} icon={result.ok ? 'checkmark' :
'close'} rotate={0} />
          <T variant="small" color={colors.textSecondary}>
            {result.ok ? `${result.trips} shared trip${result.trips === 1 ? '' : 's'} on this server` : result.message}
          </T>
        </View>
      ) : null}
      <T variant="caption" color={colors.textSecondary}>
        {status.shared.length
          ? `${status.shared.length} trip${status.shared.length === 1 ? ' is' : 's are'} syncing · ${status.online === false ? `offline,
${status.pending} change${status.pending === 1 ? '' : 's'} waiting` : status.online ? `online${status.peers > 1 ? `, ${status.peers} pho
nes` : ''}` : 'connecting…'}`
          : 'No trips are shared yet. Share one from More → Crew & roles.'}
      </T>
    </Card>
  );
}
```

src/components/sync/SyncTripCard.tsx

```
import { router } from 'expo-router';
import { useState } from 'react';
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { T } from '@/components/ui/T';
import { shareTrip, stopSharing } from '@/sync/actions';
import { effectiveServerUrl, useSyncPrefs } from '@/sync/prefs';
import { useSyncStatus } from '@/sync/status';
import { useAppStore } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { colors } from '@/theme';

import { SyncBadge } from './SyncBadge';

/** On the Crew screen: share this trip with friends' phones, or see that it is already live. */
export function SyncTripCard({ trip }: { trip: Trip }) {
  const shared = useSyncStatus((s) => s.shared.includes(trip.id));
  const error = useSyncStatus((s) => s.error);
  const [busy, setBusy] = useState(false);
  useSyncPrefs((s) => s.serverUrl); // re-check when the address changes
  const hasServer = !!effectiveServerUrl();

  const start = async () => {
    setBusy(true);
    let r = await shareTrip(trip.id);
    setBusy(false);
    if (r.ok) return toast('Sharing! Friends can now join with the invite code');
    if (r.reason === 'code-taken') {
      return confirmAction(
        'Invite code already in use',
        `Another shared trip already uses ${trip.inviteCode}. To join that trip, use “Join a trip” instead. To share this one, give it a
new code.`,
        async () => {
          useAppStore.getState().regenerateInvite(trip.id);
          setBusy(true);
          r = await shareTrip(trip.id);
          setBusy(false);
          toast(r.ok ? 'Sharing with a new invite code' : r.message, r.ok ? undefined : { tone: 'warn' });
        },
        { confirmLabel: 'New code and share' },
      );
    }
    toast(r.message, { tone: 'warn' });
  };

  return (
    <Card style={{ gap: 12, marginTop: 14 }}>
      <View style={{ flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 8 }}>
        <T variant="kicker" color={colors.textSecondary} accessibilityRole="header" style={{ flex: 1 }}>
          Sync across phones
        </T>
```

```
        <SyncBadge tripId={trip.id} />
      </View>
      {shared ? (
        <>
          <T variant="small" color={colors.textSecondary}>
            Chat, votes, the plan and expenses now update live on every phone in this trip. Changes made offline are sent when you are b
ack online.
          </T>
          {error ? (
            <T variant="caption" color={colors.warn}>
              {error}. Your changes are saved here and will sync when the server is back.
            </T>
          ) : null}
          <Button
            label="Stop syncing on this phone"
            variant="secondary"
            size="md"
            onPress={() =>
              confirmAction('Stop syncing?', 'This trip stays on this phone, but it will no longer update with your friends’ phones.', a
sync () => {
                await stopSharing(trip.id);
                toast('Syncing stopped for this trip');
              })
            }
          />
        </>
      ) : hasServer ? (
        <>
          <T variant="small" color={colors.textSecondary}>
            Share this trip so friends who join with the invite code see the same plan, chat and budget on their own phones.
          </T>
          <Button label="Share this trip across phones" icon="phone-portrait-outline" variant="secondary" loading={busy} onPress={start}
/>
        </>
      ) : (
        <>
          <T variant="small" color={colors.textSecondary}>
            Phones sync through a small server that runs on your computer. Set its address once and this trip can be shared.
          </T>
          <Button label="Set up sync" icon="settings-outline" variant="secondary" size="md" onPress={() => router.push('/settings')} />
        </>
      )}
    </Card>
  );
}
```

src/components/trip/ActivityCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { Platform, StyleSheet, View } from 'react-native';

import { AvatarStack, Chip, Divider } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { FadeIn } from '@/components/ui/motion';
import { Press, tap } from '@/components/ui/Press';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { ensurePermission } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';
import type { Activity, ActivityStatus, Trip } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { weekdayShort } from '@/utils/time';

import { ActionRow, DeleteRow, actionStyles, useActivityOps, useMetaLine } from './ActivityParts';
import { typeIcon } from './meta';
import { clockParts, voteTally } from './plan';
import { VoteCard } from './VoteCard';

interface Props {
  trip: Trip;
  activity: Activity;
  onEdit: (a: Activity) => void;
  onAssign: (a: Activity) => void;
  /** 'vote' is the ballot card of the Vote tab; 'plan' is a stop on the timeline. */
  mode?: 'plan' | 'vote';
  /** The last stop of its day: the rail ends here. */
  last?: boolean;
  /** Position in the list, so only the first few rows slide in. */
  index?: number;
  /** Vote card only: this item was just locked in, so it shows a stamp slamming down. */
  justLocked?: boolean;
}

/** Only the first few rows animate in, so a long day stays light. */
```

```
const STAGGER = 6;

export function ActivityCard({ mode = 'plan', ...rest }: Props) {
  return mode === 'vote' ? <VoteCard {...rest} /> : <Stop {...rest} />;
}

/** Locked in is the normal state, so it is printed in plain ink; the exceptions (pending, voting) are the ones with colour. */
const STATUS_STAMP: Record<ActivityStatus, { label: string; tone: 'ink' | 'warn' | 'accent' }> = {
  confirmed: { label: 'Locked in', tone: 'ink' },
  pending: { label: 'Pending', tone: 'warn' },
  voting: { label: 'Voting', tone: 'accent' },
};

/**
 * One stop on the timeline: the start time in mono on the left, a rail with a square stop on it (the kind of activity as a
 * line icon), then the title, the place and the facts. Tap it for the details; the reminder bell is its own button beside it.
 */
function Stop({ trip, activity: a, onEdit, onAssign, last, index = 0 }: Props) {
  const t = useT();
  const [open, setOpen] = useState(false);
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const vote = useAppStore((s) => s.voteActivity);
  const update = useAppStore((s) => s.updateActivity);
  const { directions, askDelete } = useActivityOps(trip, a);
  const meta = useMetaLine(trip, a);
  const tally = voteTally(a.votes, trip.members.length);
  const myVote = a.votes[userId];
  const yesPeople = Object.entries(a.votes)
    .filter(([, v]) => v === 1)
    .map(([id]) => trip.members.find((m) => m.id === id))
    .filter(Boolean) as Trip['members'];
  const { clock, suffix } = clockParts(a.time);
  const stamp = a.done ? { label: 'Done', tone: 'muted' as const } : STATUS_STAMP[a.status];

  const toggle = () => setOpen((o) => !o);

  const toggleReminder = async () => {
    const next = !a.reminder;
    if (next) {
      const ok = await ensurePermission();
      if (!ok && Platform.OS !== 'web') toast('Enable notifications in Settings to get alerts', { tone: 'warn', icon: 'notifications-off
-outline' });
    }
    update(trip.id, a.id, { reminder: next });
    tap();
    toast(next ? `Reminder on — we'll ping the group before ${a.title}` : 'Reminder turned off', { icon: next ? 'notifications' : 'notif
ications-off-outline' });
  };

  const setStatus = (s: ActivityStatus) => {
    if (s === a.status) return;
    update(trip.id, a.id, { status: s });
    toast(s === 'voting' ? 'Moved to the Vote tab' : `Marked as ${s}`);
  };

  const row = (
    <View style={[styles.stop, !last && { paddingBottom: 26 }]}>
      {/* The route line runs the whole height of the stop, behind the square stops and behind the opened details. */}
      {last ? null : <View pointerEvents="none" style={styles.line} />}
      <View style={styles.row}>
        <View style={styles.time} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
          <T variant="numeralSm" style={a.done ? { color: colors.textMuted } : undefined}>
            {clock}
          </T>
          <T variant="kicker" color={colors.textSecondary}>
            {suffix}
          </T>
        </View>

        <View style={[styles.node, a.status === 'pending' && !a.done && styles.nodePending, a.done && { backgroundColor: colors.ink }]}
accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
          <Ionicons name={a.done ? 'checkmark' : typeIcon(a.type)} size={18} color={a.done ? colors.bg : colors.ink} />
        </View>

        <Press
          onPress={toggle}
          scaleTo={0.99}
          style={styles.main}
          accessibilityLabel={`${a.title}, ${weekdayShort(a.date)} ${a.time}, ${tally.yes} of ${tally.total} in`}
          accessibilityHint="Shows details and actions"
          accessibilityState={{ expanded: open }}>
          <T variant="h3" numberOfLines={open ? undefined : 3} style={a.done ? { textDecorationLine: 'line-through', color: colors.textM
uted } : undefined}>
            {a.title}
          </T>
          <View style={styles.place}>
```

```
            <Ionicons name="location-outline" size={15} color={colors.textSecondary} />
            <T variant="small" color={colors.textSecondary} numberOfLines={1} style={{ flex: 1 }}>
              {a.location}
            </T>
          </View>
          <View style={styles.facts}>
            <Stamp label={stamp.label} tone={stamp.tone} icon={a.status === 'confirmed' && !a.done ? 'lock-closed' : undefined} rotate=
{0} />
            <T variant="kicker" color={colors.textSecondary} style={{ flexShrink: 1 }}>
              {meta}
            </T>
          </View>
        </Press>
        {/* The reminder belongs to the start time, so it sits under it. It comes after the stop in reading order and is never inside th
e tap area. */}
        <Press onPress={toggleReminder} accessibilityLabel={a.reminder ? 'Turn reminder off' : 'Turn reminder on'} style={styles.bell}>
          <Ionicons name={a.reminder ? 'notifications' : 'notifications-outline'} size={22} color={a.reminder ? colors.ink : colors.text
Muted} />
        </Press>
      </View>

      {open ? (
        <FadeIn distance={6} duration={200}>
          <View style={styles.details}>
            {a.notes ? (
              <T variant="small" color={colors.textSecondary}>
                {a.notes}
              </T>
            ) : null}
            <View style={styles.going}>
              {yesPeople.length ? <AvatarStack people={yesPeople} max={4} size={28} /> : null}
              <T variant="small" weight="semibold" color={colors.textSecondary}>
                {tally.yes} of {tally.total} in
              </T>
            </View>
            <View style={styles.pair}>
              <Chip label="I’m in" icon={myVote === 1 ? 'thumbs-up' : 'thumbs-up-outline'} active={myVote === 1} onPress={() => vote(tri
p.id, a.id, 1)} style={styles.choice} />
              <Chip label="Skip" icon={myVote === -1 ? 'thumbs-down' : 'thumbs-down-outline'} active={myVote === -1} onPress={() => vote
(trip.id, a.id, -1)} style={styles.choice} />
            </View>
            <Divider />
            <View style={actionStyles.grid}>
              <ActionRow icon="people-outline" label="Assign" onPress={() => onAssign(a)} />
              <ActionRow icon="navigate-outline" label="Directions" onPress={directions} />
              <ActionRow icon={a.done ? 'refresh-outline' : 'checkmark-done-outline'} label={a.done ? 'Undo' : 'Done'} onPress={() => up
date(trip.id, a.id, { done: !a.done })} />
              <ActionRow icon="create-outline" label="Edit" onPress={() => onEdit(a)} />
            </View>
            <Divider />
            <View style={{ gap: 8 }}>
              <T variant="kicker" color={colors.textSecondary}>
                Status
              </T>
              <View style={styles.pair}>
                <Chip small label="Locked in" active={a.status === 'confirmed'} onPress={() => setStatus('confirmed')} />
                <Chip small label={t('itin.voting')} active={a.status === 'voting'} onPress={() => setStatus('voting')} />
                <Chip small label={t('itin.pending')} active={a.status === 'pending'} onPress={() => setStatus('pending')} />
              </View>
            </View>
            <Divider />
            <DeleteRow title={a.title} onPress={askDelete} />
          </View>
        </FadeIn>
      ) : null}
    </View>
  );

  return index < STAGGER ? (
    <FadeIn delay={index * 45} distance={10} duration={320}>
      {row}
    </FadeIn>
  ) : (
    row
  );
}

const NODE = 36;
const TIME_W = 68;
const GAP = 8;

const styles = StyleSheet.create({
  stop: { position: 'relative' },
  row: { flexDirection: 'row', alignItems: 'flex-start', gap: GAP, minHeight: 96 },
  time: { width: TIME_W, alignItems: 'flex-end', paddingTop: 2 },
  node: { width: NODE, height: NODE, borderRadius: radius.chip, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
```

```
d, alignItems: 'center', justifyContent: 'center' },
  nodePending: { borderStyle: 'dashed' },
  line: { position: 'absolute', top: NODE / 2, bottom: 0, left: TIME_W + GAP + NODE / 2 - line / 2, width: line, backgroundColor: color
s.ink },
  main: { flex: 1, minWidth: 0, gap: 4, paddingTop: 4, minHeight: NODE },
  place: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  facts: { flexDirection: 'row', flexWrap: 'wrap', alignItems: 'center', columnGap: 8, rowGap: 4, marginTop: 4 },
  bell: { position: 'absolute', left: TIME_W - 44, top: 46, width: 44, height: 44, alignItems: 'flex-end', justifyContent: 'center', pad
dingRight: 1 },
  details: { marginTop: 8, padding: 14, gap: 12, backgroundColor: colors.card, borderRadius: radius.card, borderWidth: line, borderColo
r: colors.ink },
  going: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  pair: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  choice: { flexGrow: 1, flexBasis: '40%', justifyContent: 'center' },
});
```

src/components/trip/ActivityParts.tsx

```
/** Pieces the timeline row and the vote card share: the actions on an activity and the one-line facts under its title. */
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { StyleSheet } from 'react-native';

import type { IconName } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { convert, useRates } from '@/services/currency';
import type { Activity, Trip } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';
import { money } from '@/utils/format';

import { durationText } from './plan';

/** "1h 30m · €55 pp": how long and how much, in plain mono text (the kicker voice uppercases it). */
export function useMetaLine(trip: Trip, a: Activity): string {
  const rates = useRates();
  const currency = destinationById(trip.destinationId)?.currency ?? 'USD';
  const cost = a.costPerPerson > 0 ? `${money(convert(a.costPerPerson, 'USD', currency, rates), currency)} pp` : 'Free';
  return `${durationText(a.durationMin)} · ${cost}`;
}

/** What you can do to one activity: open it in maps, ask before deleting. */
export function useActivityOps(trip: Trip, a: Activity) {
  const remove = useAppStore((s) => s.deleteActivity);
  const directions = () => {
    const q = a.lat && a.lng ? `${a.lat},${a.lng}` : encodeURIComponent(`${a.title} ${a.location}`);
    Linking.openURL(`https://www.google.com/maps/search/?api=1&query=${q}`).catch(() => toast('Couldn’t open maps', { tone: 'warn' }));
  };
  const askDelete = () =>
    confirmAction(
      'Delete activity?',
      `“${a.title}” will be removed for everyone.`,
      () => {
        remove(trip.id, a.id);
        toast('Activity deleted');
      },
      { confirmLabel: 'Delete', destructive: true },
    );
  return { directions, askDelete };
}

/** One quiet action: a line icon and a word, 48 points tall. */
export function ActionRow({ icon, label, onPress }: { icon: IconName; label: string; onPress: () => void }) {
  return (
    <Press onPress={onPress} accessibilityLabel={label} style={styles.action} scaleTo={0.98}>
      <Ionicons name={icon} size={19} color={colors.ink} />
      <T variant="bodySm" weight="semibold">
        {label}
      </T>
    </Press>
  );
}

/** Delete: red words with a bin, at the bottom of the details and far from the buttons you press by habit. */
export function DeleteRow({ title, onPress }: { title: string; onPress: () => void }) {
  return (
    <Press onPress={onPress} accessibilityLabel={`Delete ${title}`} style={styles.action} scaleTo={0.98}>
      <Ionicons name="trash-outline" size={19} color={colors.red} />
      <T variant="bodySm" weight="semibold" color={colors.red}>
        Delete
      </T>
    </Press>
  );
```

```
}

export const actionStyles = StyleSheet.create({
  grid: { flexDirection: 'row', flexWrap: 'wrap', columnGap: 8 },
});

const styles = StyleSheet.create({
  action: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, flexBasis: '46%', flexGrow: 1 },
});
```

src/components/trip/AddActivitySheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useMemo, useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { TimePickerSheet } from '@/components/pickers';
import { Button } from '@/components/ui/Button';
import { Chip, ToggleRow } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Perforation } from '@/components/ui/Rule';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useDraft } from '@/hooks/useDraft';
import { useT } from '@/i18n';
import { ensurePermission } from '@/services/notifications';
import { guessType, type PlaceHit } from '@/services/placeSearch';
import { useAppStore } from '@/store/useAppStore';
import type { Activity, Trip } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { dayLabel, time12, tripDays } from '@/utils/time';
import { firstError, maxLen, parseAmount, required, validateAmount } from '@/utils/validation';

import { DayTabs } from './DayTabs';
import { activityTypes } from './meta';
import { PlaceSearch } from './PlaceSearch';
import { type ActivityDraft, type Form, blank, formFrom, hasExtras, mergeDraft } from './addForm';
import { DURATIONS, clockParts, durationText, stepDuration } from './plan';

export type { ActivityDraft };

interface Props {
  visible: boolean;
  onClose: () => void;
  trip: Trip;
  /** The day to start on (the one the Plan is showing). */
  date: string;
  editing?: Activity | null;
  draft?: ActivityDraft | null;
  /** Start with "put it to a group vote" on (the Vote tab asks for this). */
  voting?: boolean;
}

export function AddActivitySheet({ visible, onClose, trip, date, editing, draft, voting }: Props) {
  const t = useT();
  const add = useAppStore((s) => s.addActivity);
  const update = useAppStore((s) => s.updateActivity);
  const userId = useAppStore((s) => s.session?.userId ?? '');

  // A new activity is a draft that survives closing the sheet; editing one (or adding from a place) starts from that activity instead.
  const isNew = !editing && !draft;
  const [saved, setSaved, clearSaved] = useDraft<Form | null>('add-activity', null);
  const [local, setLocal] = useState<Form>(() => blank(trip.id));
  const form = isNew ? (saved && saved.tripId === trip.id ? saved : blank(trip.id)) : local;
  const patch = (p: Partial<Form>) => (isNew ? setSaved((cur) => mergeDraft(cur, trip.id, p)) : setLocal((cur) => ({ ...cur, ...p })));

  const { title, picked, location, type, time, duration, cost, notes, remind } = form;
  const day = form.day ?? date;
  const vote = form.vote ?? !!voting;

  const [more, setMore] = useState(false);
  const [timeOpen, setTimeOpen] = useState(false);
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  // When the sheet opens: fill the form for an edit, show the folded part if the draft uses it, forget old errors.
  const [openedFor, setOpenedFor] = useState<string | object | null>(null);
  const openKey = visible ? (editing?.id ?? draft ?? 'new') : null;
  if (openKey !== openedFor) {
    setOpenedFor(openKey);
    if (visible) {
      setErrors({});
      if (isNew) setMore(hasExtras(form));
      else {
```

```
        setLocal(formFrom(trip.id, date, editing, draft));
        setMore(!!editing);
      }
    }
  }

  const pick = (h: PlaceHit) => {
    tap();
    patch({
      picked: h,
      title: h.name,
      location: h.detail,
      ...(form.typeTouched ? {} : { type: h.type }),
      ...(h.durationMin ? { duration: DURATIONS.reduce((a, b) => (Math.abs(b - h.durationMin!) < Math.abs(a - h.durationMin!) ? b : a))
} : {}),
      ...(h.costPerPerson !== undefined ? { cost: h.costPerPerson ? String(h.costPerPerson) : '' } : {}),
    });
    setErrors((e) => ({ ...e, title: null }));
  };

  const typePlace = (text: string) =>
    patch({
      title: text,
      ...(picked && text !== picked.name ? { picked: null } : {}),
      ...(form.typeTouched ? {} : { type: guessType(text) }),
    });

  const days = tripDays(trip.startDate, trip.endDate);

  // Semantic check: warn about overlapping plans on the same day.
  const clash = useMemo(() => {
    const [h, m] = time.split(':').map(Number);
    const start = h * 60 + m;
    const end = start + duration;
    return trip.activities.find((a) => {
      if (a.date !== day || a.id === editing?.id) return false;
      const [ah, am] = a.time.split(':').map(Number);
      const s = ah * 60 + am;
      return start < s + a.durationMin && s < end;
    });
  }, [trip.activities, day, time, duration, editing?.id]);

  const submit = async () => {
    const e = {
      title: firstError(title, required('a place'), maxLen(60, 'The place')),
      location: maxLen(60, 'The address')(location),
      cost: cost.trim() ? validateAmount(cost, { min: 0, max: 10000 }) : null,
      notes: maxLen(280, 'Notes')(notes),
    };
    setErrors(e);
    if (Object.values(e).some(Boolean)) {
      if (e.location || e.cost || e.notes) setMore(true); // the problem is in a field that is folded away
      return tap('warning');
    }
    const payload = {
      date: day,
      time,
      title: title.trim(),
      location: location.trim() || picked?.detail || destinationById(trip.destinationId)?.city || '',
      type,
      durationMin: duration,
      costPerPerson: cost.trim() ? parseAmount(cost) ?? 0 : 0,
      notes: notes.trim() || undefined,
      status: vote ? ('voting' as const) : editing && editing.status !== 'voting' ? editing.status : ('confirmed' as const),
      reminder: remind,
    };
    const where = { lat: picked?.lat, lng: picked?.lng, placeId: picked?.placeId };
    if (remind) ensurePermission().catch(() => {});
    if (editing) {
      update(trip.id, editing.id, { ...payload, ...where });
      toast('Activity updated');
    } else {
      add(trip.id, { ...payload, assigned: [userId], ...where });
      toast(`Added to ${dayLabel(day)}${vote ? ' — group vote started' : ''}`);
      if (isNew) clearSaved();
    }
    tap('success');
    onClose();
  };

  const clock = clockParts(time);

  return (
    <Sheet
      visible={visible}
      onClose={onClose}
```

```
      title={editing ? 'Edit activity' : t('add.title')}
      subtitle={t('add.subtitle')}
      footer={<Button label={editing ? t('add.save') : vote ? 'Start the vote' : t('add.submit')} onPress={submit} testID="add-activity-
submit" />}>
      <PlaceSearch
        trip={trip}
        visible={visible}
        value={title}
        onChangeText={typePlace}
        picked={picked}
        onPick={pick}
        onClear={() => patch({ title: '', picked: null })}
        error={errors.title}
      />

      <View style={styles.group}>
        <T variant="kicker">Day</T>
        <DayTabs days={days} value={day} onChange={(d) => d !== 'all' && patch({ day: d })} edge={22} kind="radio" />
      </View>

      <View style={styles.group}>
        <T variant="kicker">{t('add.time')}</T>
        <Press onPress={() => setTimeOpen(true)} style={styles.field} accessibilityLabel={`${t('add.time')} ${time12(time)}. Change`} sc
aleTo={0.99}>
          <Ionicons name="time-outline" size={22} color={colors.ink} />
          <View style={styles.clock}>
            <T variant="numeralSm">{clock.clock}</T>
            <T variant="kicker" color={colors.textSecondary}>
              {clock.suffix}
            </T>
          </View>
          <View style={{ flex: 1 }} />
          <Ionicons name="create-outline" size={19} color={colors.textSecondary} />
        </Press>
      </View>

      <View style={styles.group}>
        <T variant="kicker">{t('add.duration')}</T>
        <View style={styles.stepper}>
          <Step icon="remove" label="Shorter" disabled={duration <= DURATIONS[0]} onPress={() => patch({ duration: stepDuration(duratio
n, -1) })} />
          <View style={styles.stepValue}>
            <T variant="numeralSm" accessibilityLabel={`Duration ${durationText(duration)}`}>
              {durationText(duration)}
            </T>
          </View>
          <Step icon="add" label="Longer" disabled={duration >= DURATIONS[DURATIONS.length - 1]} onPress={() => patch({ duration: stepDu
ration(duration, 1) })} />
        </View>
      </View>

      {clash ? (
        <View style={styles.warn} accessibilityLiveRegion="polite">
          <Ionicons name="warning-outline" size={18} color={colors.warn} />
          <T variant="small" color={colors.warn} style={{ flex: 1 }}>
            Overlaps with “{clash.title}” at {time12(clash.time)}. You can still add it.
          </T>
        </View>
      ) : null}

      <View style={[styles.vote, vote && { backgroundColor: colors.accentSoft }]}>
        <ToggleRow icon="thumbs-up-outline" label={t('add.vote')} sub="Everyone gets a vote; majority confirms it" value={vote} onChange
={(v) => patch({ vote: v })} />
      </View>

      <View style={{ paddingRight: 3 }}>
        <Perforation />
        <Press onPress={() => setMore((m) => !m)} style={styles.moreBtn} accessibilityLabel="More options" accessibilityState={{ expande
d: more }} scaleTo={0.99}>
          <T variant="kicker" style={{ flex: 1 }}>
            {more ? 'Fewer options' : 'More options'}
          </T>
          <Ionicons name={more ? 'chevron-up' : 'chevron-down'} size={18} color={colors.ink} />
        </Press>
      </View>

      {more ? (
        <>
          <View style={styles.group}>
            <T variant="kicker">{t('add.type')}</T>
            <View style={styles.wrap}>
              {activityTypes.map((a) => (
                <Chip key={a.key} label={a.label} icon={a.icon} active={type === a.key} onPress={() => patch({ type: a.key, typeTouched:
true })} />
              ))}
            </View>
```

```
          </View>
          <Input label="Address (optional)" placeholder="Street or area" icon="location-sharp" value={location} onChangeText={(v) => pat
ch({ location: v })} error={errors.location} maxLength={70} testID="activity-location" style={styles.typing} />
          <Input label={t('add.cost')} placeholder="0 = free" icon="cash-outline" value={cost} onChangeText={(v) => patch({ cost: v })}
keyboardType="decimal-pad" error={errors.cost} style={styles.typing} />
          <Input label={t('add.notes')} placeholder={t('add.notesPh')} value={notes} onChangeText={(v) => patch({ notes: v })} multiline
error={errors.notes} maxLength={300} />
          <ToggleRow icon="notifications-outline" label={t('add.remind')} value={remind} onChange={(v) => patch({ remind: v })} />
        </>
      ) : null}

      <TimePickerSheet visible={timeOpen} value={time} onClose={() => setTimeOpen(false)} onChange={(v) => patch({ time: v })} />
    </Sheet>
  );
}

/** One end of the duration stepper: a 56 point square with a plus or a minus. */
function Step({ icon, label, disabled, onPress }: { icon: 'add' | 'remove'; label: string; disabled: boolean; onPress: () => void }) {
  return (
    <Press onPress={onPress} disabled={disabled} accessibilityLabel={label} accessibilityState={{ disabled }} style={[styles.step, disab
led && { opacity: 0.35 }]} scaleTo={0.94}>
      <Ionicons name={icon} size={24} color={colors.ink} />
    </Press>
  );
}

const styles = StyleSheet.create({
  // the text box itself is as tall as a fingertip, not only the outlined field around it
  typing: { minHeight: 44 },
  // the Input component leaves 3 points on the right for its plate, so the other blocks do too and every edge lines up
  group: { gap: 8, paddingRight: 3 },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  field: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 56, paddingHorizontal: 14, borderRadius: radius.button, borde
rWidth: line, borderColor: colors.ink, backgroundColor: colors.card },
  clock: { flexDirection: 'row', alignItems: 'baseline', gap: 6 },
  stepper: { flexDirection: 'row', alignItems: 'center', height: 56, borderRadius: radius.button, borderWidth: line, borderColor: color
s.ink, backgroundColor: colors.card, overflow: 'hidden' },
  step: { width: 60, height: 56, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.bgAlt },
  stepValue: { flex: 1, height: 56, alignItems: 'center', justifyContent: 'center', borderLeftWidth: line, borderRightWidth: line, borde
rColor: colors.ink },
  warn: { marginRight: 3, flexDirection: 'row', gap: 10, alignItems: 'center', padding: 12, borderRadius: radius.button, borderWidth: li
ne, borderColor: colors.warn, backgroundColor: colors.warnSoft },
  vote: { marginRight: 3, paddingHorizontal: 12, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, backgroundColor:
colors.card },
  moreBtn: { flexDirection: 'row', alignItems: 'center', gap: 8, minHeight: 52 },
});
```

src/components/trip/DayTabs.tsx

```
import { useRef } from 'react';
import { ScrollView, StyleSheet, View, type LayoutChangeEvent } from 'react-native';

import { Plate } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { colors, line, radius } from '@/theme';
import { dayLabel, weekdayShort } from '@/utils/time';

const PLATE = 3;

/**
 * A row of day tabs cut like the tabs of a ticket folder: the weekday in small caps over the day of the month in mono.
 * The chosen day is filled with the accent and sits on a plate. Today's tab says TODAY. `withAll` adds the "All" tab first.
 * Wide enough to scroll sideways on a long trip; the chosen tab is scrolled into view when the row first appears.
 */
export function DayTabs<K extends string>({
  days,
  value,
  onChange,
  today,
  withAll,
  edge = 20,
  kind = 'tab',
}: {
  days: string[];
  /** A day (YYYY-MM-DD), or 'all' when `withAll` is on. */
  value: K | 'all';
  onChange: (day: K | 'all') => void;
  today?: string;
  withAll?: boolean;
  /** The page's side padding, so the row can run to the edge of the screen and still start on the gutter. */
  edge?: number;
  /** `tab` filters a list; `radio` picks one of several (the day in Add activity). */
  kind?: 'tab' | 'radio';
}) {
  const scroller = useRef<ScrollView>(null);
```

```
  const reveal = (e: LayoutChangeEvent) => scroller.current?.scrollTo({ x: Math.max(0, e.nativeEvent.layout.x - edge - 12), animated: fa
lse });
  return (
    <ScrollView
      ref={scroller}
      horizontal
      showsHorizontalScrollIndicator={false}
      accessibilityRole={kind === 'tab' ? 'tablist' : 'radiogroup'}
      style={{ marginHorizontal: -edge, flexGrow: 0 }}
      contentContainerStyle={{ paddingHorizontal: edge, gap: 6 }}>
      {withAll ? <Tab label="All days" on={value === 'all'} kind={kind} onPress={() => onChange('all')} onLayout={value === 'all' ? reve
al : undefined} top={null} bottom="All" /> : null}
      {days.map((d, i) => {
        const isToday = d === today;
        return (
          <Tab
            key={d}
            label={`Day ${i + 1}, ${weekdayShort(d)} ${dayLabel(d)}${isToday ? ', today' : ''}`}
            on={value === d}
            kind={kind}
            onPress={() => onChange(d as K)}
            onLayout={value === d ? reveal : undefined}
            top={isToday ? 'Today' : weekdayShort(d)}
            bottom={String(Number(d.slice(8)))}
          />
        );
      })}
    </ScrollView>
  );
}

function Tab({ label, on, kind, onPress, onLayout, top, bottom }: { label: string; on: boolean; kind: 'tab' | 'radio'; onPress: () => vo
id; onLayout?: (e: LayoutChangeEvent) => void; top: string | null; bottom: string }) {
  const fg = on ? colors.onAccent : colors.text;
  return (
    <View onLayout={onLayout}>
      <Plate offset={PLATE} corner={radius.chip} color={on ? colors.plate : 'transparent'}>
        <Press
          onPress={onPress}
          accessibilityRole={kind}
          accessibilityState={kind === 'tab' ? { selected: on } : { checked: on }}
          accessibilityLabel={label}
          scaleTo={0.97}
          style={[styles.tab, { backgroundColor: on ? colors.accent : colors.card }]}>
          {top ? (
            <T variant="kicker" color={on ? colors.onAccent : colors.textSecondary} style={styles.top} importantForAccessibility="no">
              {top}
            </T>
          ) : null}
          <T variant="numeralSm" color={fg} style={styles.num} importantForAccessibility="no">
            {bottom}
          </T>
        </Press>
      </Plate>
    </View>
  );
}

const styles = StyleSheet.create({
  tab: { minWidth: 56, height: 54, paddingHorizontal: 10, borderRadius: radius.chip, borderWidth: line, borderColor: colors.ink, alignIt
ems: 'center', justifyContent: 'center' },
  top: { lineHeight: 13 },
  num: { lineHeight: 26 },
});
```

src/components/trip/MemberPickerSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { Avatar, Card, Divider } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import type { Member } from '@/store/types';
import { colors, line, radius } from '@/theme';

/** Multi- or single-select list of trip members: one outlined list with a square tick box on each row. */
export function MemberPickerSheet({
  visible,
  onClose,
  members,
  selected,
  onToggle,
  title = 'Assign people',
  single,
```

```
}: {
  visible: boolean;
  onClose: () => void;
  members: Member[];
  selected: string[];
  onToggle: (id: string) => void;
  title?: string;
  single?: boolean;
}) {
  return (
    <Sheet visible={visible} onClose={onClose} title={title} subtitle={single ? 'Choose one person' : 'Tap to add or remove'}>
      <Card padded={false} style={{ overflow: 'hidden' }}>
        {members.map((m, i) => {
          const on = selected.includes(m.id);
          return (
            <View key={m.id}>
              {i > 0 ? <Divider /> : null}
              <Press
                onPress={() => {
                  onToggle(m.id);
                  if (single) onClose();
                }}
                style={styles.row}
                scaleTo={0.99}
                accessibilityRole="checkbox"
                accessibilityState={{ checked: on }}
                accessibilityLabel={`${m.name}, ${m.role}`}>
                <Avatar name={m.name} src={m.avatar} size={40} />
                <View style={{ flex: 1 }}>
                  <T variant="bodySm" weight="semibold">
                    {m.name}
                  </T>
                  <T variant="kicker" color={colors.textSecondary}>
                    {m.role}
                  </T>
                </View>
                <View style={[styles.box, on && { backgroundColor: colors.accent }]}>{on ? <Ionicons name="checkmark" size={18} color={c
olors.onAccent} /> : null}</View>
              </Press>
            </View>
          );
        })}
      </Card>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 64, paddingHorizontal: 14, paddingVertical: 8 },
  box: { width: 28, height: 28, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card, align
Items: 'center', justifyContent: 'center' },
});
```

src/components/trip/NoTrip.tsx

```
import { router } from 'expo-router';
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { EmptyState } from '@/components/ui/bits';
import { Screen } from '@/components/ui/Screen';

/** Shown on trip tabs when the signed-in user isn't part of any trip yet. */
export function NoTrip({ title = 'No trip selected' }: { title?: string }) {
  return (
    <Screen contentStyle={{ flexGrow: 1, justifyContent: 'center' }}>
      <EmptyState icon="airplane-outline" title={title} body="Create a trip and invite your group, or join a friend’s trip with their in
vite code.">
        <View style={{ gap: 10, alignSelf: 'stretch', marginTop: 12 }}>
          <Button label="Create a trip" icon="add" onPress={() => router.push('/new-trip')} />
          <Button label="Join with a code" icon="people-outline" variant="secondary" onPress={() => router.push('/join')} />
        </View>
      </EmptyState>
    </Screen>
  );
}
```

src/components/trip/PlaceSearch.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Input } from '@/components/ui/Input';
import { Press } from '@/components/ui/Press';
import { Skeleton } from '@/components/ui/Skeleton';
```

```
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useMyLocation, usePlaceSearch } from '@/hooks/usePlaceSearch';
import { normalizeQuery, type PlaceHit } from '@/services/placeSearch';
import type { Trip } from '@/store/types';
import { colors, hair, line, radius } from '@/theme';
import { distanceKm, kmLabel, type LatLng } from '@/utils/geo';

import { typeIcon } from './meta';

interface Props {
  trip: Trip;
  /** The sheet is open: this is when the phone is asked where it is. */
  visible: boolean;
  value: string;
  onChangeText: (text: string) => void;
  picked: PlaceHit | null;
  onPick: (hit: PlaceHit) => void;
  onClear: () => void;
  error?: string | null;
}

/** Type where you want to go; matching places appear below with how far each one is from you. */
export function PlaceSearch({ trip, visible, value, onChangeText, picked, onPick, onClear, error }: Props) {
  const dest = destinationById(trip.destinationId);
  const center: LatLng | null = dest ? { lat: dest.lat, lng: dest.lng } : null;
  const me = useMyLocation(center ?? { lat: 0, lng: 0 }, visible);
  const here = me.source === 'device';
  // At the trip: look around the phone. Planning from home: look around the trip.
  const onTrip = here && !!center && distanceKm(me, center) < 150;
  const [typed, setTyped] = useState(false);
  const [asTyped, setAsTyped] = useState(false);
  const open = typed && !picked && !asTyped && normalizeQuery(value).length >= 2;
  const { hits, loading, offline } = usePlaceSearch(value, {
    destinationIds: [trip.destinationId, ...trip.extraDestinationIds],
    around: onTrip ? me : center,
    enabled: visible && open,
  });

  // Distances start from the phone; without its location, from the middle of the trip's city.
  const from: LatLng | null = here ? me : center;
  const fromLabel = here ? 'from you' : dest ? `from ${dest.city}` : '';
  const away = (p: LatLng) => (from ? kmLabel(distanceKm(from, p)) : '');

  return (
    <View style={{ gap: 10 }}>
      <Input
        label="Where do you want to go?"
        placeholder="Search a place or type an idea"
        icon="search"
        value={value}
        onChangeText={(t) => {
          setTyped(true);
          setAsTyped(false);
          onChangeText(t);
        }}
        error={error}
        maxLength={70}
        autoCorrect={false}
        testID="activity-name"
        style={styles.typing}
        right={
          value ? (
            <Press
              onPress={() => {
                setTyped(false);
                setAsTyped(false);
                onClear();
              }}
              style={styles.clear}
              accessibilityLabel="Clear the place">
              <Ionicons name="close" size={20} color={colors.ink} />
            </Press>
          ) : null
        }
      />

      {picked ? (
        <View style={styles.picked}>
          <View style={styles.tick}>
            <Ionicons name="checkmark" size={14} color={colors.onAccent} />
          </View>
          <T variant="small" color={colors.textSecondary} style={{ flex: 1 }} numberOfLines={2}>
            {[picked.detail, from ? `${away(picked)} ${fromLabel}` : ''].filter(Boolean).join(' · ')}
          </T>
        </View>
```

```
      ) : null}

      {!picked && !open && !value ? (
        <T variant="small" color={colors.textMuted}>
          Try “museum”, “pizza” or the name of a place.
        </T>
      ) : null}

      {open ? (
        <View style={styles.list} accessibilityLiveRegion="polite">
          {hits.map((h, i) => (
            <Press key={h.id} onPress={() => onPick(h)} style={[styles.row, i > 0 && styles.divider]} accessibilityLabel={[h.name, h.det
ail, from ? `${away(h)} ${fromLabel}` : ''].filter(Boolean).join(', ')}>
              <View style={styles.icon}>
                <Ionicons name={typeIcon(h.type)} size={20} color={colors.ink} />
              </View>
              <View style={{ flex: 1 }}>
                <T variant="bodySm" weight="semibold" numberOfLines={1}>
                  {h.name}
                </T>
                {h.detail ? (
                  <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                    {h.detail}
                  </T>
                ) : null}
              </View>
              {from ? (
                <View style={{ alignItems: 'flex-end' }}>
                  <T variant="kicker" color={colors.ink} style={{ textTransform: 'none' }}>
                    {away(h)}
                  </T>
                  <T variant="micro" color={colors.textMuted}>
                    {fromLabel}
                  </T>
                </View>
              ) : null}
            </Press>
          ))}
          {loading ? (
            <View style={[styles.row, hits.length > 0 && styles.divider]} accessibilityLabel="Searching">
              <Skeleton width={40} height={40} corner={radius.button} />
              <View style={{ flex: 1, gap: 8 }}>
                <Skeleton width="62%" height={14} corner={radius.xs} />
                <Skeleton width="38%" height={11} corner={radius.xs} />
              </View>
            </View>
          ) : null}
          <Press onPress={() => setAsTyped(true)} style={[styles.row, (hits.length > 0 || loading) && styles.divider]} accessibilityLabe
l={`Use ${value.trim()} as typed`}>
            <View style={styles.icon}>
              <Ionicons name="create-outline" size={20} color={colors.ink} />
            </View>
            <T variant="bodySm" weight="semibold" style={{ flex: 1 }} numberOfLines={1}>
              Use “{value.trim()}”
            </T>
          </Press>
          {offline ? (
            <T variant="caption" color={colors.textMuted} style={styles.note}>
              No connection, so only saved places are shown.
            </T>
          ) : !loading ? (
            <T variant="micro" color={colors.textMuted} style={styles.note}>
              Places from OpenStreetMap · search by Photon
            </T>
          ) : null}
        </View>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  // the text box itself is as tall as a fingertip, not only the outlined field around it
  typing: { minHeight: 44 },
  clear: { width: 44, height: 44, marginRight: -10, alignItems: 'center', justifyContent: 'center' },
  picked: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  tick: { width: 22, height: 22, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.accent, al
ignItems: 'center', justifyContent: 'center' },
  list: { backgroundColor: colors.card, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, overflow: 'hidden' },
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, minHeight: 60, paddingVertical: 10, paddingHorizontal: 12 },
  divider: { borderTopWidth: hair, borderTopColor: colors.border },
  icon: { width: 40, height: 40, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt,
alignItems: 'center', justifyContent: 'center' },
  note: { paddingHorizontal: 12, paddingBottom: 10, paddingTop: 2 },
});
```

src/components/trip/TripMenuSheet.tsx

```
import { router } from 'expo-router';
import type { ReactNode } from 'react';
import { View } from 'react-native';

import { Divider, ListRow } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Rule } from '@/components/ui/Rule';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { printPlanner, sharePlannerPdf } from '@/services/print';
import { inviteMessage, shareText } from '@/services/share';
import { tripStatus, useAppStore } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { colors } from '@/theme';

/** A small caps label over a few rows, the rows divided by hairlines. */
function Group({ title, children }: { title: string; children: ReactNode[] }) {
  const rows = children.filter(Boolean);
  return (
    <View>
      <T variant="kicker" color={colors.textSecondary} style={{ marginBottom: 2 }}>
        {title}
      </T>
      {rows.map((row, i) => (
        <View key={i}>
          {i > 0 ? <Divider /> : null}
          {row}
        </View>
      ))}
    </View>
  );
}

/** Trip options in a bottom sheet, in three groups. Delete and leave sit alone at the bottom, away from everything else. */
export function TripMenuSheet({ visible, onClose, trip }: { visible: boolean; onClose: () => void; trip: Trip }) {
  const userId = useAppStore((s) => s.session?.userId);
  const updateTrip = useAppStore((s) => s.updateTrip);
  const leaveTrip = useAppStore((s) => s.leaveTrip);
  const deleteTrip = useAppStore((s) => s.deleteTrip);
  const owner = trip.ownerId === userId;
  const status = tripStatus(trip);
  const go = (href: Parameters<typeof router.push>[0]) => {
    onClose();
    setTimeout(() => router.push(href), 180);
  };

  return (
    <Sheet visible={visible} onClose={onClose} title={trip.name} subtitle="Trip options">
      <Group title="This trip">
        <ListRow icon="swap-horizontal" label="Switch or create trip" sub="All your group trips in one place" onPress={() => go('/trip
s')} />
        <ListRow icon="create-outline" label="Edit trip details" sub="Name, dates and group budget" onPress={() => go({ pathname: '/new-
trip', params: { edit: trip.id } })} />
        <ListRow icon="map-outline" label="Trip map" onPress={() => go('/map')} />
        <ListRow icon="airplane-outline" label="Flights & arrivals" onPress={() => go('/flights')} />
      </Group>

      <Group title="Share and print">
        <ListRow icon="share-social-outline" label="Share invite" sub={`Code ${trip.inviteCode}`} onPress={() => shareText(inviteMessage
(trip.name, trip.inviteCode))} />
        <ListRow
          icon="print-outline"
          label="Print planner"
          sub="A clean day-by-day printout with checkboxes"
          onPress={async () => {
            onClose();
            try {
              await printPlanner(trip);
            } catch {
              toast('Allow pop-ups to print the planner', { tone: 'warn' });
            }
          }}
        />
        <ListRow
          icon="document-text-outline"
          label="Save planner as PDF"
          sub="Share it to Files, email or the group chat"
          onPress={async () => {
            onClose();
            await sharePlannerPdf(trip).catch(() => toast('Couldn’t create the PDF', { tone: 'warn' }));
          }}
        />
      </Group>
```

```
      <Group title="Wrap up">
        <ListRow icon="trophy-outline" label="Trip recap" sub="Stats, highlights and memories" onPress={() => go('/recap')} />
        <ListRow
          icon={status === 'completed' ? 'refresh-outline' : 'flag-outline'}
          label={status === 'completed' ? 'Reopen trip' : 'Mark trip as complete'}
          sub={status === 'completed' ? 'Move it back to planning' : 'Wrap up, settle balances and see the recap'}
          onPress={() => {
            const done = status !== 'completed';
            updateTrip(trip.id, { completed: done });
            onClose();
            toast(done ? 'Trip completed. Here's your recap' : 'Trip reopened');
            if (done) setTimeout(() => router.push('/recap'), 250);
          }}
        />
      </Group>

      <View style={{ marginTop: 8 }}>
        <Rule weight={1} />
        {owner ? (
          <ListRow
            icon="trash-outline"
            tint={colors.red}
            label="Delete trip"
            sub="Removes it for everyone on this device"
            onPress={() => {
              onClose();
              confirmAction('Delete this trip?', `"${trip.name}" and all its plans, expenses and chats will be deleted. This can't be un
done.`, () => {
                deleteTrip(trip.id);
                toast('Trip deleted');
                router.navigate('/(tabs)/explore');
              }, { confirmLabel: 'Delete trip', destructive: true });
            }}
          />
        ) : (
          <ListRow
            icon="exit-outline"
            tint={colors.red}
            label="Leave trip"
            onPress={() => {
              onClose();
              confirmAction('Leave this trip?', 'You can rejoin later with the invite code.', () => {
                leaveTrip(trip.id);
                toast('You left the trip');
                router.navigate('/(tabs)/explore');
              }, { confirmLabel: 'Leave', destructive: true });
            }}
          />
        )}
      </View>
    </Sheet>
  );
}
```

src/components/trip/VoteCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { AvatarStack, Divider, ProgressBar } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { FadeIn } from '@/components/ui/motion';
import { PlatePress } from '@/components/ui/Plate';
import { Press } from '@/components/ui/Press';
import { Rule } from '@/components/ui/Rule';
import { Stamp } from '@/components/ui/Stamp';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import type { Activity, Trip } from '@/store/types';
import { colors, line, radius } from '@/theme';
import { dayLabel, time12, weekdayShort } from '@/utils/time';

import { ActionRow, DeleteRow, actionStyles, useActivityOps, useMetaLine } from './ActivityParts';
import { typeIcon } from './meta';
import { voteTally } from './plan';

interface Props {
  trip: Trip;
  activity: Activity;
  onEdit: (a: Activity) => void;
  onAssign: (a: Activity) => void;
  index?: number;
  /** Just locked in: the stamp slams down and the buttons make way for a line saying where it went. */
  justLocked?: boolean;
}
```

```
/**
 * A ballot on the Vote tab. An accent band on top says when it is, the live tally fills a bar toward the majority, and the crew
 * answers with two big buttons. The organizer gets a strip along the bottom to lock it in without waiting for the votes.
 */
export function VoteCard({ trip, activity: a, onEdit, onAssign, index = 0, justLocked }: Props) {
  const [open, setOpen] = useState(false);
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const vote = useAppStore((s) => s.voteActivity);
  const update = useAppStore((s) => s.updateActivity);
  const { directions, askDelete } = useActivityOps(trip, a);
  const meta = useMetaLine(trip, a);
  const tally = voteTally(a.votes, trip.members.length);
  const myVote = a.votes[userId];
  const isOwner = trip.ownerId === userId;
  const yesPeople = Object.entries(a.votes)
    .filter(([, v]) => v === 1)
    .map(([id]) => trip.members.find((m) => m.id === id))
    .filter(Boolean) as Trip['members'];

  const lockIn = () => {
    if (!isOwner) {
      const boss = trip.members.find((m) => m.id === trip.ownerId)?.name.split(' ')[0] ?? 'the organizer';
      return toast(`Only ${boss} (the organizer) can lock this in`, { tone: 'warn', icon: 'lock-closed-outline' });
    }
    // The Plan screen sees the status change and does the celebrating, so it also happens when a vote tips it over.
    update(trip.id, a.id, { status: 'confirmed', reminder: true });
  };

  const card = (
    <View style={styles.card}>
      <View style={styles.band}>
        <T variant="kicker" color={colors.onAccent} style={{ flex: 1 }} numberOfLines={1}>
          {weekdayShort(a.date)} {dayLabel(a.date)} · {time12(a.time)}
        </T>
        {justLocked ? (
          <Stamp label="Locked in" tone="positive" icon="lock-closed" slam />
        ) : (
          <T variant="kicker" color={colors.onAccent}>
            Voting
          </T>
        )}
      </View>

      <View style={styles.body}>
        <View style={styles.titleRow}>
          <View style={styles.kind} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
            <Ionicons name={typeIcon(a.type)} size={22} color={colors.ink} />
          </View>
          <View style={{ flex: 1, gap: 3 }}>
            <T variant="h2" numberOfLines={3}>
              {a.title}
            </T>
            <View style={styles.place}>
              <Ionicons name="location-outline" size={15} color={colors.textSecondary} />
              <T variant="small" color={colors.textSecondary} numberOfLines={1} style={{ flex: 1 }}>
                {a.location}
              </T>
            </View>
            <T variant="kicker" color={colors.textSecondary}>
              {meta}
            </T>
          </View>
        </View>

        <View style={{ gap: 8 }}>
          <View style={styles.tallyRow}>
            <T variant="title" accessibilityLabel={`${tally.yes} of ${tally.total} say yes. ${tally.needed} needed to lock it in`}>
              <T variant="numeralSm">{tally.yes}</T> of <T variant="numeralSm">{tally.total}</T> say yes
            </T>
            <T variant="kicker" color={colors.textSecondary}>
              {justLocked ? 'On the plan' : `${tally.needed} to lock in`}
            </T>
          </View>
          <ProgressBar value={tally.progress} />
          {yesPeople.length || tally.no ? (
            <View style={styles.who}>
              {yesPeople.length ? <AvatarStack people={yesPeople} max={5} size={28} /> : null}
              {tally.no ? (
                <T variant="small" color={colors.textSecondary}>
                  {tally.no} skipping
                </T>
              ) : null}
            </View>
          ) : null}
        </View>
```

```
        {justLocked ? (
          <View style={styles.moved}>
            <Ionicons name="calendar-outline" size={18} color={colors.ink} />
            <T variant="bodySm" weight="semibold" style={{ flex: 1 }}>
              It is on the plan now.
            </T>
          </View>
        ) : (
          <>
            <View style={styles.pair}>
              <Choice
                on={myVote === 1}
                tone="yes"
                icon={myVote === 1 ? 'thumbs-up' : 'thumbs-up-outline'}
                label="I’m in"
                accessibilityLabel={`I’m in. ${tally.yes} of ${tally.total} are in`}
                onPress={() => vote(trip.id, a.id, 1)}
              />
              <Choice on={myVote === -1} tone="no" icon={myVote === -1 ? 'thumbs-down' : 'thumbs-down-outline'} label="Skip" accessibili
tyLabel={myVote === -1 ? 'Skipping. Tap to undo' : 'Skip this'} onPress={() => vote(trip.id, a.id, -1)} />
            </View>

            <Press onPress={() => setOpen((o) => !o)} style={styles.more} accessibilityLabel={open ? 'Hide details' : 'Show details'} ac
cessibilityState={{ expanded: open }} scaleTo={0.99}>
              <T variant="kicker" color={colors.textSecondary}>
                {open ? 'Hide details' : 'Details'}
              </T>
              <Ionicons name={open ? 'chevron-up' : 'chevron-down'} size={16} color={colors.textSecondary} />
            </Press>
            {open ? (
              <View style={{ gap: 8 }}>
                {a.notes ? (
                  <T variant="small" color={colors.textSecondary}>
                    {a.notes}
                  </T>
                ) : null}
                <View style={actionStyles.grid}>
                  <ActionRow icon="people-outline" label="Assign" onPress={() => onAssign(a)} />
                  <ActionRow icon="navigate-outline" label="Directions" onPress={directions} />
                  <ActionRow icon="create-outline" label="Edit" onPress={() => onEdit(a)} />
                </View>
                <Divider />
                <DeleteRow title={a.title} onPress={askDelete} />
              </View>
            ) : null}
          </>
        )}
      </View>

      {isOwner && !justLocked ? (
        <>
          <Rule />
          <Press onPress={lockIn} style={styles.lock} accessibilityLabel={`Lock in ${a.title}`} scaleTo={0.99}>
            <Ionicons name="lock-closed-outline" size={19} color={colors.ink} />
            <T variant="title" weight="bold" style={{ flex: 1 }}>
              Lock in
            </T>
            <Ionicons name="arrow-forward" size={18} color={colors.ink} />
          </Press>
        </>
      ) : null}
    </View>
  );

  return index < 6 ? (
    <FadeIn delay={index * 60} distance={10} duration={320}>
      {card}
    </FadeIn>
  ) : (
    card
  );
}

/** One of the two answers: a big button that stays pressed (accent for yes, ink for skip) once chosen. */
function Choice({ on, tone, icon, label, accessibilityLabel, onPress }: { on: boolean; tone: 'yes' | 'no'; icon: 'thumbs-up' | 'thumbs-u
p-outline' | 'thumbs-down' | 'thumbs-down-outline'; label: string; accessibilityLabel: string; onPress: () => void }) {
  const bg = on ? (tone === 'yes' ? colors.accent : colors.primary) : colors.card;
  const fg = on ? (tone === 'yes' ? colors.onAccent : colors.onPrimary) : colors.ink;
  return (
    <PlatePress
      onPress={onPress}
      accessibilityLabel={accessibilityLabel}
      accessibilityState={{ selected: on }}
      offset={3}
      corner={radius.button}
```

```
      wrapperStyle={{ flex: 1 }}
      style={[styles.choice, { backgroundColor: bg }]}>
      <Ionicons name={icon} size={20} color={fg} />
      <T variant="title" weight="bold" color={fg}>
        {label}
      </T>
      {on ? <Ionicons name="checkmark" size={16} color={fg} /> : null}
    </PlatePress>
  );
}

const styles = StyleSheet.create({
  card: { backgroundColor: colors.card, borderRadius: radius.card, borderWidth: line, borderColor: colors.ink, overflow: 'hidden', margi
nBottom: 20 },
  band: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 44, paddingHorizontal: 14, paddingVertical: 8, backgroundColo
r: colors.accent, borderBottomWidth: line, borderBottomColor: colors.ink },
  body: { padding: 16, gap: 16 },
  titleRow: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
  kind: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.bgAlt,
alignItems: 'center', justifyContent: 'center' },
  place: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  tallyRow: { flexDirection: 'row', alignItems: 'baseline', justifyContent: 'space-between', gap: 8 },
  who: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  pair: { flexDirection: 'row', gap: 12 },
  choice: { height: 52, flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 8, borderRadius: radius.button, borde
rWidth: line, borderColor: colors.ink },
  more: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, minHeight: 44 },
  moved: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 48, paddingHorizontal: 12, borderRadius: radius.button, borde
rWidth: line, borderStyle: 'dashed', borderColor: colors.ink },
  lock: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 56, paddingHorizontal: 16, backgroundColor: colors.bgAlt },
});
```

src/components/trip/__tests__/addForm.test.ts

```
import type { Activity } from '@/store/types';

import { blank, formFrom, hasExtras, mergeDraft } from '../addForm';

const existing: Activity = {
  id: 'a1',
  date: '2026-10-24',
  time: '19:30',
  title: 'Dinner at Da Vincenzo',
  location: 'Viale Pasitea',
  type: 'food',
  durationMin: 120,
  costPerPerson: 55,
  status: 'voting',
  assigned: [],
  votes: {},
  notes: 'Window table',
  reminder: false,
  lat: 40.63,
  lng: 14.48,
  placeId: 'da-vincenzo',
  createdBy: 'm1',
};

describe('blank form', () => {
  it('starts at ten in the morning for an hour and a half, with the day and vote following the screen', () => {
    expect(blank('t1')).toMatchObject({ tripId: 't1', title: '', time: '10:00', duration: 90, day: null, vote: null, remind: true, picke
d: null });
  });
});

describe('formFrom', () => {
  it('fills the form from an activity being edited, with its own day and a vote switch matching its status', () => {
    const f = formFrom('t1', '2026-10-23', existing, null);
    expect(f).toMatchObject({ title: 'Dinner at Da Vincenzo', day: '2026-10-24', time: '19:30', duration: 120, cost: '55', notes: 'Windo
w table', vote: true, remind: false, type: 'food', typeTouched: true });
    expect(f.picked).toMatchObject({ lat: 40.63, lng: 14.48, placeId: 'da-vincenzo' });
  });
  it('fills the form from a place found elsewhere and starts on the given day', () => {
    const f = formFrom('t1', '2026-10-23', null, { title: 'Museo', location: 'Sorrento', lat: 40.6, lng: 14.3, type: 'culture', duration
Min: 90 });
    expect(f).toMatchObject({ title: 'Museo', location: 'Sorrento', day: '2026-10-23', duration: 90, type: 'culture', typeTouched: true,
vote: null });
    expect(f.picked?.name).toBe('Museo');
  });
  it('has no picked place without coordinates, and lets the type follow the words', () => {
    const f = formFrom('t1', '2026-10-23', null, { title: 'Gelato' });
    expect(f.picked).toBeNull();
    expect(f.typeTouched).toBe(false);
  });
  it('leaves cost empty for a free activity', () => {
    expect(formFrom('t1', '2026-10-23', { ...existing, costPerPerson: 0 }, null).cost).toBe('');
```

```
  });
});

describe('hasExtras', () => {
  it('is true when any folded field holds something', () => {
    expect(hasExtras(blank('t1'))).toBe(false);
    expect(hasExtras({ ...blank('t1'), notes: 'x' })).toBe(true);
    expect(hasExtras({ ...blank('t1'), cost: '5' })).toBe(true);
    expect(hasExtras({ ...blank('t1'), location: 'Via Roma' })).toBe(true);
    expect(hasExtras({ ...blank('t1'), typeTouched: true })).toBe(true);
  });
});

describe('mergeDraft', () => {
  it('adds a change to the saved draft', () => {
    const first = mergeDraft(null, 't1', { title: 'Gel' });
    const second = mergeDraft(first, 't1', { time: '15:00' });
    expect(second).toMatchObject({ tripId: 't1', title: 'Gel', time: '15:00' });
  });
  it('drops a draft that belongs to another trip', () => {
    const other = mergeDraft(null, 'other', { title: 'Old idea', notes: 'x' });
    const next = mergeDraft(other, 't1', { time: '15:00' });
    expect(next).toMatchObject({ tripId: 't1', title: '', notes: '', time: '15:00' });
  });
});
```

src/components/trip/__tests__/plan.test.ts

```
import type { Activity } from '@/store/types';

import { DURATIONS, clockParts, durationText, groupByDay, newlyLocked, sameStatuses, statusMap, stepDuration, voteTally } from '../pla
n';

const act = (id: string, date: string, time: string, over: Partial<Activity> = {}): Activity => ({
  id,
  date,
  time,
  title: id,
  location: 'Somewhere',
  type: 'food',
  durationMin: 60,
  costPerPerson: 0,
  status: 'confirmed',
  assigned: [],
  votes: {},
  reminder: false,
  createdBy: 'm1',
  ...over,
});

describe('clockParts', () => {
  it('splits a 24 hour time into a clock and AM or PM', () => {
    expect(clockParts('15:40')).toEqual({ clock: '3:40', suffix: 'PM' });
    expect(clockParts('00:05')).toEqual({ clock: '12:05', suffix: 'AM' });
    expect(clockParts('12:30')).toEqual({ clock: '12:30', suffix: 'PM' });
  });
});

describe('groupByDay', () => {
  const days = ['2026-10-23', '2026-10-24', '2026-10-25'];
  it('puts each activity under its day, in time order, and skips empty days', () => {
    const groups = groupByDay([act('c', '2026-10-25', '09:00'), act('b', '2026-10-23', '19:00'), act('a', '2026-10-23', '08:30')], day
s);
    expect(groups.map((g) => [g.date, g.dayNo, g.items.map((i) => i.id)])).toEqual([
      ['2026-10-23', 1, ['a', 'b']],
      ['2026-10-25', 3, ['c']],
    ]);
  });
  it('gives day 0 to a date outside the trip', () => {
    expect(groupByDay([act('x', '2026-11-01', '10:00')], days)[0].dayNo).toBe(0);
  });
  it('returns nothing for nothing', () => {
    expect(groupByDay([], days)).toEqual([]);
  });
});

describe('voteTally', () => {
  it('needs more than half the crew', () => {
    expect(voteTally({}, 4).needed).toBe(3);
    expect(voteTally({}, 5).needed).toBe(3);
    expect(voteTally({}, 1).needed).toBe(1);
  });
  it('counts yes and no and fills the bar toward the majority', () => {
    const t = voteTally({ a: 1, b: 1, c: -1 }, 4);
    expect(t).toMatchObject({ yes: 2, no: 1, total: 4, needed: 3 });
    expect(t.progress).toBeCloseTo(2 / 3);
```

```
  });
  it('never fills past the full bar', () => {
    expect(voteTally({ a: 1, b: 1, c: 1, d: 1 }, 4).progress).toBe(1);
  });
});

describe('newlyLocked', () => {
  const before = statusMap([act('a', '2026-10-23', '10:00', { status: 'voting' }), act('b', '2026-10-23', '11:00', { status: 'voting'
}), act('c', '2026-10-23', '12:00')]);
  it('finds the items that went from voting to confirmed', () => {
    const now = [act('a', '2026-10-23', '10:00'), act('b', '2026-10-23', '11:00', { status: 'voting' }), act('c', '2026-10-23', '12:0
0')];
    expect(newlyLocked(before, now).map((a) => a.id)).toEqual(['a']);
  });
  it('does not count items that were already confirmed, or that are new', () => {
    const now = [act('c', '2026-10-23', '12:00'), act('new', '2026-10-23', '13:00'), act('a', '2026-10-23', '10:00', { status: 'voting'
})];
    expect(newlyLocked(before, now)).toEqual([]);
  });
  it('does not count a move to pending', () => {
    expect(newlyLocked(before, [act('a', '2026-10-23', '10:00', { status: 'pending' })])).toEqual([]);
  });
  it('compares whole status maps', () => {
    expect(sameStatuses(before, { ...before })).toBe(true);
    expect(sameStatuses(before, { ...before, a: 'confirmed' })).toBe(false);
    expect(sameStatuses(before, { ...before, z: 'voting' })).toBe(false);
  });
});

describe('duration steps', () => {
  it('names the lengths', () => {
    expect(durationText(30)).toBe('30m');
    expect(durationText(60)).toBe('1h');
    expect(durationText(90)).toBe('1h 30m');
    expect(durationText(300)).toBe('Half day');
    expect(durationText(480)).toBe('Full day');
    expect(durationText(75)).toBe('1h 15m');
  });
  it('steps up and down and stops at the ends', () => {
    expect(stepDuration(90, 1)).toBe(120);
    expect(stepDuration(90, -1)).toBe(60);
    expect(stepDuration(480, 1)).toBe(480);
    expect(stepDuration(30, -1)).toBe(30);
  });
  it('steps from an odd length to the next one in that direction', () => {
    expect(stepDuration(75, 1)).toBe(90);
    expect(stepDuration(75, -1)).toBe(60);
    expect(stepDuration(1000, -1)).toBe(480);
  });
  it('keeps the list sorted', () => {
    expect([...DURATIONS]).toEqual([...DURATIONS].sort((a, b) => a - b));
  });
});
```

src/components/trip/addForm.ts

```
/**
 * What the Add activity form holds, and how it starts, with no React in it so it can be tested: a blank form, a form filled from an
 * activity being edited or from a place found elsewhere (Nearby), and how a change is merged into the saved draft.
 */
import type { PlaceHit } from '@/services/placeSearch';
import type { Activity, ActivityType } from '@/store/types';

/** A place found elsewhere (Nearby, Explore) handed to Add activity to start from. */
export interface ActivityDraft {
  title?: string;
  location?: string;
  type?: ActivityType;
  durationMin?: number;
  costPerPerson?: number;
  lat?: number;
  lng?: number;
  placeId?: string;
  notes?: string;
}

/** Everything the form holds. A new activity keeps it as a draft (see useDraft), so closing the sheet or leaving the tab loses nothing.
*/
export interface Form {
  /** The trip the draft belongs to: a draft for another trip is ignored. */
  tripId: string;
  title: string;
  picked: PlaceHit | null;
  location: string;
  type: ActivityType;
  /** Until the person picks a type themselves, it follows the place they chose or the words they typed. */
```

```
  typeTouched: boolean;
  /** null: follow the day the Plan is showing. */
  day: string | null;
  time: string;
  duration: number;
  cost: string;
  notes: string;
  /** null: follow `voting` (the Vote tab starts a vote by default). */
  vote: boolean | null;
  remind: boolean;
}

export const blank = (tripId: string): Form => ({ tripId, title: '', picked: null, location: '', type: 'sightseeing', typeTouched: fals
e, day: null, time: '10:00', duration: 90, cost: '', notes: '', vote: null, remind: true });

/** The form for an activity being edited, or filled from a place found elsewhere. */
export function formFrom(tripId: string, date: string, editing: Activity | null | undefined, draft: ActivityDraft | null | undefined): F
orm {
  const src = editing ?? draft ?? {};
  const hasPlace = src.lat !== undefined && src.lng !== undefined;
  return {
    tripId,
    title: src.title ?? '',
    picked: hasPlace ? { id: src.placeId ?? 'saved', name: src.title ?? '', detail: src.location ?? '', lat: src.lat!, lng: src.lng!, ty
pe: src.type ?? 'sightseeing', source: 'saved', placeId: src.placeId } : null,
    location: src.location ?? '',
    type: src.type ?? 'sightseeing',
    typeTouched: !!src.type,
    day: editing?.date ?? date,
    time: editing?.time ?? '10:00',
    duration: src.durationMin ?? 90,
    cost: src.costPerPerson ? String(src.costPerPerson) : '',
    notes: src.notes ?? '',
    vote: editing ? editing.status === 'voting' : null,
    remind: editing ? editing.reminder : true,
  };
}

/** True when the folded "More options" part holds something, so it opens by itself and nothing is hidden. */
export const hasExtras = (f: Form) => !!(f.location || f.cost || f.notes || f.typeTouched);

/** A change merged into the saved draft. A draft that belongs to another trip is dropped, not mixed in. */
export const mergeDraft = (saved: Form | null, tripId: string, patch: Partial<Form>): Form => ({ ...(saved && saved.tripId === tripId ?
saved : blank(tripId)), ...patch });
```

src/components/trip/meta.ts

```
import type { IconName } from '@/components/ui/bits';
import type { PlaceCategory } from '@/data/places';
import type { ActivityStatus, ActivityType, ExpenseCategory } from '@/store/types';
import { colors } from '@/theme';

/**
 * What kind of thing an activity is. Each kind has one line icon; the colours are the same ink on paper for every kind
 * (the app does not colour-code kinds: colour is kept for states).
 */
export const activityTypes: { key: ActivityType; label: string; icon: IconName; color: string; bg: string }[] = [
  { key: 'transport', label: 'Transport', icon: 'train-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'stay', label: 'Stay', icon: 'bed-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'food', label: 'Food', icon: 'restaurant-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'sightseeing', label: 'Sightseeing', icon: 'camera-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'culture', label: 'Culture', icon: 'business-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'adventure', label: 'Adventure', icon: 'bonfire-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'nightlife', label: 'Nightlife', icon: 'musical-notes-outline', color: colors.ink, bg: colors.bgAlt },
  { key: 'shopping', label: 'Shopping', icon: 'bag-handle-outline', color: colors.ink, bg: colors.bgAlt },
];

export const typeMeta = (t: ActivityType) => activityTypes.find((x) => x.key === t) ?? activityTypes[3];

export const statusMeta: Record<ActivityStatus, { tone: 'green' | 'orange' | 'blue'; key: string; dot: string }> = {
  confirmed: { tone: 'green', key: 'itin.confirmed', dot: colors.positive },
  voting: { tone: 'orange', key: 'itin.voting', dot: colors.orange },
  pending: { tone: 'blue', key: 'itin.pending', dot: colors.blue },
};

export { placeToActivityType } from '@/data/places';

export const placeCategoryIcon: Record<PlaceCategory, IconName> = {
  food: 'restaurant-outline',
  sightseeing: 'camera-outline',
  adventure: 'bonfire-outline',
  culture: 'business-outline',
  nightlife: 'musical-notes-outline',
  shopping: 'bag-handle-outline',
  relax: 'leaf-outline',
};
```

```
export const expenseCategories: { key: ExpenseCategory; icon: IconName }[] = [
  { key: 'Food', icon: 'restaurant-outline' },
  { key: 'Activities', icon: 'ticket-outline' },
  { key: 'Transport', icon: 'train-outline' },
  { key: 'Stays', icon: 'bed-outline' },
  { key: 'Flights', icon: 'airplane-outline' },
  { key: 'Shopping', icon: 'bag-handle-outline' },
  { key: 'Other', icon: 'ellipsis-horizontal' },
];

/** Filter chips for itinerary/nearby (the brief's "adventures, sightseeing, etc."). */
export const activityFilters: { key: 'all' | ActivityType; labelKey: string }[] = [
  { key: 'all', labelKey: 'cat.all' },
  { key: 'adventure', labelKey: 'cat.adventure' },
  { key: 'sightseeing', labelKey: 'cat.sightseeing' },
  { key: 'food', labelKey: 'cat.food' },
  { key: 'culture', labelKey: 'cat.culture' },
  { key: 'nightlife', labelKey: 'cat.nightlife' },
  { key: 'shopping', labelKey: 'cat.shopping' },
];

/** The line icon of an activity kind (use this, not emoji). */
export const typeIcon = (t: ActivityType): IconName => typeMeta(t).icon;
```

src/components/trip/plan.ts

```
/**
 * The small decisions the Plan screen makes, with no React in them so they can be tested: how a day's list is grouped,
 * what the clock on the rail says, how a vote stands, which items just locked in, and the duration steps of Add activity.
 */
import type { Activity, ActivityStatus } from '@/store/types';
import { time12 } from '@/utils/time';

/** "3:40" and "PM": the big mono clock on the rail and the small caps under it. */
export function clockParts(hhmm: string): { clock: string; suffix: string } {
  const [clock, suffix] = time12(hhmm).split(' ');
  return { clock, suffix };
}

export const byTime = (a: Activity, b: Activity) => (a.date + a.time).localeCompare(b.date + b.time);

export interface DayGroup {
  date: string;
  /** 1 for the first day of the trip; 0 when the date is outside the trip's days (the trip dates were edited). */
  dayNo: number;
  items: Activity[];
}

/** Activities in time order, grouped by day. Days with nothing on them are left out. */
export function groupByDay(activities: Activity[], days: string[]): DayGroup[] {
  const groups: DayGroup[] = [];
  for (const a of [...activities].sort(byTime)) {
    const last = groups[groups.length - 1];
    if (last && last.date === a.date) last.items.push(a);
    else groups.push({ date: a.date, dayNo: days.indexOf(a.date) + 1, items: [a] });
  }
  return groups;
}

export interface Tally {
  yes: number;
  no: number;
  total: number;
  /** Yes votes that make a majority (more than half the crew). */
  needed: number;
  /** How far the bar is filled: yes votes toward the majority, 0 to 1. */
  progress: number;
}

/** How a vote stands. The store confirms an item once more than half the crew says yes, so `needed` is that number. */
export function voteTally(votes: Activity['votes'], memberCount: number): Tally {
  const values = Object.values(votes);
  const yes = values.filter((v) => v === 1).length;
  const no = values.filter((v) => v === -1).length;
  const needed = Math.floor(memberCount / 2) + 1;
  return { yes, no, total: memberCount, needed, progress: Math.min(1, yes / needed) };
}

/** id to status, to compare one render with the next. */
export function statusMap(activities: Activity[]): Record<string, ActivityStatus> {
  const out: Record<string, ActivityStatus> = {};
  for (const a of activities) out[a.id] = a.status;
  return out;
}
```

```
export const sameStatuses = (a: Record<string, ActivityStatus>, b: Record<string, ActivityStatus>) => {
  const ka = Object.keys(a);
  return ka.length === Object.keys(b).length && ka.every((k) => a[k] === b[k]);
};

/**
 * The activities that were up for a vote and are now confirmed: the moment to stamp LOCKED IN and celebrate.
 * Anything new (not in `before`) never counts, so nothing celebrates on first render or when an item is added.
 */
export function newlyLocked(before: Record<string, ActivityStatus>, activities: Activity[]): Activity[] {
  return activities.filter((a) => before[a.id] === 'voting' && a.status === 'confirmed');
}

/** The lengths Add activity offers, in minutes. */
export const DURATIONS = [30, 60, 90, 120, 180, 300, 480] as const;

/** "30m", "1h", "1h 30m", "Half day", "Full day": the length of an activity as the plan and the duration stepper show it. */
export function durationText(min: number): string {
  if (min === 300) return 'Half day';
  if (min === 480) return 'Full day';
  if (min < 60) return `${min}m`;
  const h = Math.floor(min / 60);
  const m = min % 60;
  return m ? `${h}h ${m}m` : `${h}h`;
}

/** One step up or down the list of lengths from `min` (which may be an odd value of an edited activity). Stays at the ends. */
export function stepDuration(min: number, dir: 1 | -1): number {
  if (dir === 1) return DURATIONS.find((d) => d > min) ?? DURATIONS[DURATIONS.length - 1];
  return [...DURATIONS].reverse().find((d) => d < min) ?? DURATIONS[0];
}
```

src/components/ui/Button.tsx

```
import { Ionicons } from '@expo/vector-icons';
import type { ComponentProps } from 'react';
import { ActivityIndicator, StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { colors, line, radius } from '@/theme';

import { PlatePress } from './Plate';
import { Press } from './Press';
import { T } from './T';

type IconName = ComponentProps<typeof Ionicons>['name'];

export interface ButtonProps {
  label: string;
  onPress?: () => void;
  /** `primary` is ink and is for the one main action on a screen; `accent` is the highlighter-coloured loud one. */
  variant?: 'primary' | 'accent' | 'secondary' | 'soft' | 'ghost' | 'danger' | 'light';
  size?: 'lg' | 'md' | 'sm';
  icon?: IconName;
  iconRight?: IconName;
  loading?: boolean;
  disabled?: boolean;
  full?: boolean;
  style?: StyleProp<ViewStyle>;
  accessibilityHint?: string;
  testID?: string;
}

const palette = {
  primary: { bg: colors.primary, fg: colors.onPrimary, border: colors.ink, plate: true },
  accent: { bg: colors.accent, fg: colors.onAccent, border: colors.ink, plate: true },
  secondary: { bg: colors.card, fg: colors.text, border: colors.ink, plate: true },
  soft: { bg: colors.bgAlt, fg: colors.text, border: colors.ink, plate: true },
  ghost: { bg: 'transparent', fg: colors.text, border: 'transparent', plate: false },
  danger: { bg: colors.redSoft, fg: colors.red, border: colors.red, plate: true },
  light: { bg: 'rgba(255,255,255,0.16)', fg: colors.white, border: colors.white, plate: false },
};

/** Heights are all at least 44 points, so every button is an easy target. */
const HEIGHT = { lg: 56, md: 48, sm: 44 };

/** A printed button with a hard plate under it: it pushes down flush when pressed. */
export function Button({ label, onPress, variant = 'primary', size = 'lg', icon, iconRight, loading, disabled, full = true, style, acces
sibilityHint, testID }: ButtonProps) {
  const p = palette[variant];
  const off = disabled || loading;
  const face = [styles.base, { height: HEIGHT[size], backgroundColor: p.bg, borderColor: p.border, paddingHorizontal: size === 'sm' ? 16
: 22 }, !p.plate && { alignSelf: 'stretch' as const }];
  const content = loading ? (
    <ActivityIndicator color={p.fg} />
  ) : (
    <View style={styles.row}>
```

```
      {icon && <Ionicons name={icon} size={size === 'sm' ? 16 : 19} color={p.fg} />}
      <T variant={size === 'sm' ? 'small' : 'title'} weight="bold" color={p.fg}>
        {label}
      </T>
      {iconRight && <Ionicons name={iconRight} size={size === 'sm' ? 16 : 19} color={p.fg} />}
    </View>
  );
  const common = { testID, accessibilityLabel: label, accessibilityHint, accessibilityState: { disabled: !!off, busy: !!loading }, disab
led: off, onPress };
  if (!p.plate)
    return (
      <Press {...common} style={[face, full && { alignSelf: 'stretch' }, off && { opacity: 0.45 }, style]}>
        {content}
      </Press>
    );
  return (
    <PlatePress {...common} color={variant === 'danger' ? colors.red : variant === 'primary' ? colors.accent : colors.plate} wrapperStyl
e={[full && { alignSelf: 'stretch' }, off && { opacity: 0.45 }, style]} style={face}>
      {content}
    </PlatePress>
  );
}

const styles = StyleSheet.create({
  base: {
    borderRadius: radius.button,
    borderWidth: line,
    alignItems: 'center',
    justifyContent: 'center',
  },
  row: { flexDirection: 'row', alignItems: 'center', gap: 8 },
});
```

src/components/ui/Confetti.tsx

```
/**
 * Celebrations are rare on purpose: a quiz win, a poll or activity locking in, a budget settled, "1 day to go",
 * and the end of "Make it yours". `celebrate()` does a haptic tick plus, depending on the person's Effects
 * setting (Chill, Lively, Max) and the Reduce Motion switches, a burst of confetti.
 */
import { useEffect, useState } from 'react';
import { Animated, Easing, StyleSheet, View, useWindowDimensions } from 'react-native';
import { create } from 'zustand';

import { effectiveMotion, useOsReduceMotion } from '@/hooks/useReduceMotion';
import type { Effects } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';

import { toast } from './feedback';
import { nativeDriver } from './motion';
import { tap } from './Press';

export interface Piece {
  /** Where it starts across the width, 0 to 1. */
  x: number;
  /** Sideways drift as a fraction of the width. */
  drift: number;
  /** How far it rises before it falls, as a fraction of the height. */
  rise: number;
  /** How far it falls, as a fraction of the height. */
  fall: number;
  /** Turns while falling. */
  spin: number;
  /** When it starts, 0 to 0.2 of the whole burst. */
  delay: number;
  size: number;
  color: number;
  round: boolean;
}

/** The same pieces every time for a given count (a tiny seeded generator), so a burst can be tested. */
export function confettiPieces(count: number): Piece[] {
  let seed = 7919;
  const rnd = () => {
    seed = (seed * 16807) % 2147483647;
    return (seed - 1) / 2147483646;
  };
  return Array.from({ length: count }, () => ({
    x: 0.1 + rnd() * 0.8,
    drift: (rnd() - 0.5) * 0.5,
    rise: 0.12 + rnd() * 0.28,
    fall: 0.75 + rnd() * 0.35,
    spin: 1 + rnd() * 3,
    delay: rnd() * 0.2,
    size: 7 + rnd() * 7,
```

```
    color: Math.floor(rnd() * 5),
    round: rnd() > 0.65,
  }));
}

/** How many pieces each Effects setting throws (Chill throws none). */
export const PIECES = { chill: 0, lively: 36, max: 90 } as const;

interface Burst {
  id: number;
  count: number;
}
const useBurst = create<{ burst: Burst | null }>(() => ({ burst: null }));

/**
 * Celebrates. Always ticks the haptic; throws confetti unless Effects is Chill or animations are off.
 * If `message` is given it is shown as a toast too, so a calm setting still gets a clear "well done".
 * `level` shows a preview of an Effects setting that has not been saved yet.
 */
export function celebrate(message?: string, level?: Effects) {
  tap('success');
  const { personal } = useAppStore.getState();
  const count = PIECES[level ?? personal.effects] ?? PIECES.lively;
  const canMove = effectiveMotion(personal.motion, useOsReduceMotion.getState().on);
  if (count > 0 && canMove) useBurst.setState({ burst: { id: Date.now(), count } });
  if (message) toast(message, { icon: 'checkmark-circle' });
  return count > 0 && canMove;
}

/** Draws the confetti above everything. Render once, near the root. */
export function ConfettiHost() {
  const burst = useBurst((s) => s.burst);
  return burst ? <Rain key={burst.id} count={burst.count} /> : null;
}

function Rain({ count }: { count: number }) {
  const { width, height } = useWindowDimensions();
  const [t] = useState(() => new Animated.Value(0));
  const [pieces] = useState(() => confettiPieces(count));
  const palette = [colors.accent, colors.text, colors.accentText, colors.white, colors.yellow];

  useEffect(() => {
    const a = Animated.timing(t, { toValue: 1, duration: 2600, easing: Easing.linear, useNativeDriver: nativeDriver });
    a.start(({ finished }) => finished && useBurst.setState({ burst: null }));
    return () => a.stop();
  }, [t]);

  return (
    <View pointerEvents="none" style={styles.layer} accessibilityElementsHidden importantForAccessibility="no-hide-descendants">
      {pieces.map((p, i) => (
        <Animated.View
          key={i}
          style={{
            position: 'absolute',
            left: p.x * width,
            top: height * 0.35,
            width: p.size,
            height: p.round ? p.size : p.size * 1.7,
            borderRadius: p.round ? p.size : 2,
            backgroundColor: palette[p.color],
            opacity: t.interpolate({ inputRange: [0, p.delay, p.delay + 0.02, 0.8, 1], outputRange: [0, 0, 1, 1, 0] }),
            transform: [
              { translateX: t.interpolate({ inputRange: [0, 1], outputRange: [0, p.drift * width] }) },
              { translateY: t.interpolate({ inputRange: [0, p.delay + 0.25, 1], outputRange: [0, -p.rise * height, p.fall * height] })
},
              { rotate: t.interpolate({ inputRange: [0, 1], outputRange: ['0deg', `${Math.round(p.spin * 360)}deg`] }) },
            ],
          }}
        />
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  layer: { ...StyleSheet.absoluteFill as object, zIndex: 2000, elevation: 2000 },
});
```

src/components/ui/Donut.tsx

```
import { StyleSheet, View } from 'react-native';
import Svg, { Circle } from 'react-native-svg';

import { colors } from '@/theme';

import { T } from './T';
```

```
/** Ring chart of spending by category with the remaining budget in the middle. */
export function Donut({ parts, size = 132, stroke = 20, centerLabel, centerValue }: { parts: { value: number; color: string }[]; size?:
number; stroke?: number; centerLabel: string; centerValue: string }) {
  const r = (size - stroke) / 2;
  const c = 2 * Math.PI * r;
  const total = parts.reduce((s, p) => s + p.value, 0);
  let offset = 0;
  return (
    <View style={{ width: size, height: size }} accessibilityRole="image" accessibilityLabel={`${centerLabel} ${centerValue}`}>
      <Svg width={size} height={size} viewBox={`0 0 ${size} ${size}`} style={{ transform: [{ rotate: '-90deg' }] }}>
        <Circle cx={size / 2} cy={size / 2} r={r} stroke={colors.bgAlt} strokeWidth={stroke} fill="none" />
        {total > 0
          ? parts.map((p, i) => {
              const len = (p.value / total) * c;
              const el = <Circle key={i} cx={size / 2} cy={size / 2} r={r} stroke={p.color} strokeWidth={stroke} fill="none" strokeDasha
rray={`${Math.max(0, len - 2)} ${c}`} strokeDashoffset={-offset} />;
              offset += len;
              return el;
            })
          : null}
      </Svg>
      <View style={[StyleSheet.absoluteFill, { alignItems: 'center', justifyContent: 'center' }]}>
        <T variant="micro" color={colors.textSecondary} style={{ letterSpacing: 1 }}>
          {centerLabel}
        </T>
        <T variant="h3" weight="extrabold">
          {centerValue}
        </T>
      </View>
    </View>
  );
}
```

src/components/ui/Fab.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet } from 'react-native';

import { colors, line, radius } from '@/theme';

import type { IconName } from './bits';
import { PlatePress } from './Plate';
import { T } from './T';

/**
 * The one main "add" action of a screen: an accent-coloured block with an ink outline and a plate, in the corner where the
 * thumb rests. Give it a `label` and it grows wider. Pass it to `<Screen fab={…}>` so it floats above the content.
 */
export function Fab({ icon = 'add', label, onPress, accessibilityLabel, testID }: { icon?: IconName; label?: string; onPress: () => voi
d; accessibilityLabel?: string; testID?: string }) {
  return (
    <PlatePress onPress={onPress} testID={testID} accessibilityLabel={accessibilityLabel ?? label ?? 'Add'} corner={radius.button} style
={[styles.fab, label ? styles.wide : null]}>
      <Ionicons name={icon} size={26} color={colors.onAccent} />
      {label ? (
        <T variant="small" weight="bold" color={colors.onAccent}>
          {label}
        </T>
      ) : null}
    </PlatePress>
  );
}

const styles = StyleSheet.create({
  fab: { height: 56, minWidth: 56, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.acce
nt, alignItems: 'center', justifyContent: 'center', flexDirection: 'row', gap: 8 },
  wide: { paddingLeft: 16, paddingRight: 20 },
});
```

src/components/ui/Fold.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState, type ReactNode } from 'react';
import { StyleSheet, View } from 'react-native';

import { colors, line, radius } from '@/theme';

import { Card, type IconName } from './bits';
import { Press } from './Press';
import { T } from './T';

/**
 * Something most people never need, folded away under one line so the page stays short.
 * The cards inside bring their own top spacing.
 */
```

```
export function Fold({ title, summary, icon, label = title, children }: { title: string; summary: string; icon: IconName; /** What a scr
een reader says for the button. */ label?: string; children: ReactNode }) {
  const [open, setOpen] = useState(false);
  return (
    <View style={{ marginTop: 14 }}>
      <Card padded={false}>
        <Press onPress={() => setOpen((o) => !o)} style={styles.head} scaleTo={0.99} accessibilityLabel={label} accessibilityState={{ ex
panded: open }}>
          <View style={styles.icon}>
            <Ionicons name={icon} size={18} color={colors.ink} />
          </View>
          <View style={{ flex: 1 }}>
            <T variant="title" weight="bold">
              {title}
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={2}>
              {summary}
            </T>
          </View>
          <Ionicons name={open ? 'remove' : 'add'} size={22} color={colors.ink} />
        </Press>
      </Card>
      {open ? children : null}
    </View>
  );
}

const styles = StyleSheet.create({
  head: { flexDirection: 'row', alignItems: 'center', gap: 14, paddingHorizontal: 16, paddingVertical: 14, minHeight: 64 },
  icon: { width: 38, height: 38, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.card,
alignItems: 'center', justifyContent: 'center' },
});
```

src/components/ui/Input.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { forwardRef, useState, type ComponentProps, type ReactNode } from 'react';
import { Platform, StyleSheet, TextInput, View, type StyleProp, type TextInputProps, type ViewStyle } from 'react-native';

import { useAppStore } from '@/store/useAppStore';
import { colors, fonts, line, radius } from '@/theme';

import { Plate } from './Plate';
import { Press } from './Press';
import { T } from './T';

type IconName = ComponentProps<typeof Ionicons>['name'];

export interface InputProps extends TextInputProps {
  label?: string;
  /** Show the label inside the field (floating style from the sign-in screen). */
  inlineLabel?: boolean;
  icon?: IconName;
  error?: string | null;
  hint?: string;
  right?: ReactNode;
  secureToggle?: boolean;
  containerStyle?: StyleProp<ViewStyle>;
  onPressField?: () => void;
}

export const Input = forwardRef<TextInput, InputProps>(function Input(
  { label, inlineLabel, icon, error, hint, right, secureToggle, containerStyle, multiline, onPressField, style, secureTextEntry, onFocu
s, onBlur, ...rest },
  ref,
) {
  const [focused, setFocused] = useState(false);
  const [hidden, setHidden] = useState(true);
  const scale = useAppStore((s) => s.settings.textScale);
  const hasValue = !!rest.value;
  const borderColor = error ? colors.red : colors.ink;

  const field = (
    <Plate offset={3} corner={radius.button} color={error ? colors.red : focused ? colors.accent : 'transparent'}>
    <View
      style={[
        styles.field,
        multiline && styles.multiline,
        { borderColor },
        focused && !error && styles.focused,
        error && styles.errored,
      ]}>
      {icon && <Ionicons name={icon} size={18} color={error ? colors.red : colors.textMuted} style={{ marginRight: 10 }} />}
      <View style={{ flex: 1 }}>
        {inlineLabel && label && (hasValue || focused) ? (
          <T variant="micro" color={colors.textSecondary} style={{ marginBottom: 1 }}>
```

```
            {label}
          </T>
        ) : null}
        <TextInput
          ref={ref}
          placeholderTextColor={colors.textMuted}
          multiline={multiline}
          editable={!onPressField}
          pointerEvents={onPressField ? 'none' : 'auto'}
          secureTextEntry={secureToggle ? hidden : secureTextEntry}
          accessibilityLabel={label}
          {...rest}
          placeholder={inlineLabel && !hasValue && !focused ? label : rest.placeholder}
          onFocus={(e) => {
            setFocused(true);
            onFocus?.(e);
          }}
          onBlur={(e) => {
            setFocused(false);
            onBlur?.(e);
          }}
          style={[
            styles.input,
            { fontSize: 16 * scale, minHeight: 44 },
            multiline && { minHeight: 64, textAlignVertical: 'top' },
            Platform.OS === 'web' && ({ outlineStyle: 'none' } as object),
            style,
          ]}
        />
      </View>
      {secureToggle && (
        <Press accessibilityLabel={hidden ? 'Show password' : 'Hide password'} onPress={() => setHidden((h) => !h)} style={{ width: 44,
height: 44, alignItems: 'center', justifyContent: 'center', marginRight: -10 }}>
          <Ionicons name={hidden ? 'eye-outline' : 'eye-off-outline'} size={20} color={colors.ink} />
        </Press>
      )}
      {right}
    </View>
    </Plate>
  );

  return (
    <View style={[{ gap: 6 }, containerStyle]}>
      {label && !inlineLabel ? (
        <T variant="kicker" color={colors.ink}>
          {label}
        </T>
      ) : null}
      {onPressField ? (
        <Press onPress={onPressField} scaleTo={0.99} accessibilityLabel={label}>
          {field}
        </Press>
      ) : (
        field
      )}
      {error ? (
        <T variant="caption" color={colors.red} accessibilityLiveRegion="polite">
          {error}
        </T>
      ) : hint ? (
        <T variant="caption" color={colors.textMuted}>
          {hint}
        </T>
      ) : null}
    </View>
  );
});

const styles = StyleSheet.create({
  field: {
    minHeight: 52,
    borderRadius: radius.button,
    borderWidth: line,
    backgroundColor: colors.card,
    paddingHorizontal: 14,
    flexDirection: 'row',
    alignItems: 'center',
  },
  multiline: { alignItems: 'flex-start', paddingVertical: 12 },
  focused: { backgroundColor: colors.card },
  errored: { backgroundColor: colors.redSoft },
  input: {
    fontFamily: fonts.regular,
    color: colors.text,
    paddingVertical: Platform.OS === 'ios' ? 12 : 8,
```

```
  },
});
```

src/components/ui/Plate.tsx

```
import { useState, type ReactNode } from 'react';
import { Animated, Pressable, View, type PressableProps, type StyleProp, type ViewStyle } from 'react-native';

import { colors, plateOffset, radius } from '@/theme';

import { nativeDriver, useMotion } from './motion';
import { tap } from './Press';

/**
 * The hard "plate" under a thing: a solid block of ink (the accent at night) that sticks out to the right and below,
 * like a printed card sitting on a second card. It replaces the soft drop shadow.
 * The wrapper is `offset` wider and taller than what it holds, so layouts stay put.
 */
export function Plate({ children, offset = plateOffset, color = colors.plate, corner = radius.card, style }: { children: ReactNode; offs
et?: number; color?: string; corner?: number; style?: StyleProp<ViewStyle> }) {
  return (
    <View style={[{ paddingRight: offset, paddingBottom: offset }, style]}>
      <View pointerEvents="none" style={{ position: 'absolute', left: offset, top: offset, right: 0, bottom: 0, borderRadius: corner, ba
ckgroundColor: color }} />
      {children}
    </View>
  );
}

export interface PlatePressProps extends Omit<PressableProps, 'style'> {
  /** The face you press: its colour, outline, size and corner. */
  style?: StyleProp<ViewStyle>;
  /** Margins, flex and alignment belong here, on the wrapper around face and plate. */
  wrapperStyle?: StyleProp<ViewStyle>;
  offset?: number;
  color?: string;
  corner?: number;
  haptic?: boolean;
}

const AnimatedPressable = Animated.createAnimatedComponent(Pressable);

/**
 * A button that pushes down onto its plate: pressing slides the face right and down by the plate's offset until it
 * sits flush, and letting go springs it back. Only transform is animated (native driver), and with animations off
 * it simply jumps.
 */
export function PlatePress({ style, wrapperStyle, offset = plateOffset, color = colors.plate, corner = radius.button, haptic = true, onP
ress, onPressIn, onPressOut, accessibilityRole = 'button', ...rest }: PlatePressProps) {
  const motion = useMotion();
  const [t] = useState(() => new Animated.Value(0));
  const move = (to: number, spring: boolean) => {
    if (!motion) t.setValue(to);
    else if (spring) Animated.spring(t, { toValue: to, speed: 30, bounciness: 9, useNativeDriver: nativeDriver }).start();
    else Animated.timing(t, { toValue: to, duration: 70, useNativeDriver: nativeDriver }).start();
  };
  const state = rest.accessibilityState;
  return (
    <Plate offset={offset} color={color} corner={corner} style={wrapperStyle}>
      <AnimatedPressable
        accessibilityRole={accessibilityRole}
        aria-selected={state?.selected}
        aria-checked={state?.checked}
        {...rest}
        onPressIn={(e) => {
          move(1, false);
          onPressIn?.(e);
        }}
        onPressOut={(e) => {
          move(0, true);
          onPressOut?.(e);
        }}
        onPress={(e) => {
          if (haptic) tap();
          onPress?.(e);
        }}
        style={[style, { transform: [{ translateX: t.interpolate({ inputRange: [0, 1], outputRange: [0, offset] }) }, { translateY: t.in
terpolate({ inputRange: [0, 1], outputRange: [0, offset] }) }] }]}
      />
    </Plate>
  );
}
```

src/components/ui/Press.tsx

```
import * as Haptics from 'expo-haptics';
import { useState } from 'react';
import { Animated, Platform, Pressable, type LayoutChangeEvent, type PressableProps, type StyleProp, type ViewStyle } from 'react-nativ
e';

import { useAppStore } from '@/store/useAppStore';
import { touch } from '@/theme';

import { nativeDriver, useMotion } from './motion';

export function tap(kind: 'light' | 'medium' | 'success' | 'warning' = 'light') {
  if (Platform.OS === 'web' || !useAppStore.getState().settings.haptics) return;
  if (kind === 'success') Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success).catch(() => {});
  else if (kind === 'warning') Haptics.notificationAsync(Haptics.NotificationFeedbackType.Warning).catch(() => {});
  else Haptics.impactAsync(kind === 'medium' ? Haptics.ImpactFeedbackStyle.Medium : Haptics.ImpactFeedbackStyle.Light).catch(() => {});
}

export interface PressProps extends Omit<PressableProps, 'style'> {
  /** Don't put `transform` in here: the press-in squeeze owns it. */
  style?: StyleProp<ViewStyle>;
  /** Scale applied while pressed. */
  scaleTo?: number;
  haptic?: boolean;
}

const AnimatedPressable = Animated.createAnimatedComponent(Pressable);

/**
 * Pressable that squeezes to 96% when pressed and springs back with a little overshoot, with an optional
 * haptic tick. Anything smaller than 44 points gets an invisible hit area to make it 44 (Fitts's law).
 */
export function Press({ style, scaleTo = 0.96, haptic = true, onPress, onPressIn, onPressOut, onLayout, hitSlop, accessibilityRole = 'bu
tton', ...rest }: PressProps) {
  const state = rest.accessibilityState;
  const motion = useMotion();
  const [scale] = useState(() => new Animated.Value(1));
  const [slop, setSlop] = useState<{ top: number; bottom: number; left: number; right: number } | null>(null);

  const squeeze = (to: number, bounciness: number, speed: number) => {
    if (!motion) scale.setValue(to);
    else Animated.spring(scale, { toValue: to, speed, bounciness, useNativeDriver: nativeDriver }).start();
  };

  return (
    <AnimatedPressable
      accessibilityRole={accessibilityRole}
      // Mirror the state as ARIA props so web screen readers announce selection too.
      aria-selected={state?.selected}
      aria-checked={state?.checked}
      hitSlop={hitSlop ?? slop ?? undefined}
      {...rest}
      onLayout={(e: LayoutChangeEvent) => {
        onLayout?.(e);
        if (hitSlop !== undefined) return;
        const { width, height } = e.nativeEvent.layout;
        const v = Math.max(0, Math.ceil((touch.min - height) / 2));
        const h = Math.max(0, Math.ceil((touch.min - width) / 2));
        setSlop((prev) => (prev ? (prev.top === v && prev.left === h ? prev : { top: v, bottom: v, left: h, right: h }) : v || h ? { to
p: v, bottom: v, left: h, right: h } : prev));
      }}
      onPressIn={(e) => {
        if (scaleTo !== 1) squeeze(scaleTo, 0, 60);
        onPressIn?.(e);
      }}
      onPressOut={(e) => {
        if (scaleTo !== 1) squeeze(1, 12, 24);
        onPressOut?.(e);
      }}
      onPress={(e) => {
        if (haptic) tap();
        onPress?.(e);
      }}
      style={[style, { transform: [{ scale }] }]}
    />
  );
}
```

src/components/ui/RollingNumber.tsx

```
import { SplitFlap } from './SplitFlap';
import type { TProps } from './T';

/** Older name for the departures-board number: use SplitFlap in new code. */
```

```
export function RollingNumber({ value }: { value: number | string; variant?: TProps['variant']; color?: string }) {
  return <SplitFlap value={value} size={56} />;
}
```

src/components/ui/Rule.tsx

```
import { View, type StyleProp, type ViewStyle } from 'react-native';

import { colors, line } from '@/theme';

/** A solid ink line, like the rules in a newspaper: it divides one part of a page from the next. */
export function Rule({ style, weight = line }: { style?: StyleProp<ViewStyle>; weight?: number }) {
  return <View accessibilityElementsHidden importantForAccessibility="no" style={[{ height: weight, backgroundColor: colors.ink }, styl
e]} />;
}

/** A dashed line of tear-off dots, as between the two halves of a ticket. */
export function Perforation({ style, color = colors.ink }: { style?: StyleProp<ViewStyle>; color?: string }) {
  return (
    <View accessibilityElementsHidden importantForAccessibility="no" style={[{ flexDirection: 'row', height: 2, overflow: 'hidden', gap:
5 }, style]}>
      {Array.from({ length: 90 }, (_, i) => (
        <View key={i} style={{ width: 6, height: 2, backgroundColor: color, flexShrink: 0 }} />
      ))}
    </View>
  );
}
```

src/components/ui/Screen.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import type { ReactNode } from 'react';
import { RefreshControl, ScrollView, StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { MAX_WIDTH, colors, line, radius } from '@/theme';

import { PlatePress } from './Plate';
import { Rule } from './Rule';
import { FadeIn } from './motion';
import { T } from './T';

/** Page container: the paper (or night) colour, safe-area padding and a phone-width column on big screens. No gradients: the page is fl
at. */
export function Screen({
  children,
  scroll = true,
  padded = true,
  contentStyle,
  header,
  footer,
  fab,
  fabBottom = 20,
  bottomInset = 24,
  onRefresh,
  refreshing,
}: {
  children: ReactNode;
  scroll?: boolean;
  padded?: boolean;
  contentStyle?: StyleProp<ViewStyle>;
  header?: ReactNode;
  footer?: ReactNode;
  /** A floating action button (see Fab), bottom right. Room is left under the content so it never covers the last row. */
  fab?: ReactNode;
  /** Distance of the fab from the bottom edge. Tab screens use the default; a stack screen passes `insets.bottom + 16` to clear the hom
e indicator. */
  fabBottom?: number;
  bottomInset?: number;
  onRefresh?: () => void;
  refreshing?: boolean;
}) {
  const insets = useSafeAreaInsets();
  const inner = [padded && { paddingHorizontal: 20 }, { paddingBottom: bottomInset + (footer ? 0 : insets.bottom) + (fab ? 72 : 0) }, co
ntentStyle];
  return (
    <View style={[styles.root, { paddingTop: insets.top }]}>
      <View style={styles.column}>
        {header}
        {scroll ? (
          <ScrollView
            style={{ flex: 1 }}
            contentContainerStyle={inner}
            keyboardShouldPersistTaps="handled"
            showsVerticalScrollIndicator={false}
```

```
            refreshControl={onRefresh ? <RefreshControl refreshing={!!refreshing} onRefresh={onRefresh} tintColor={colors.ink} /> : unde
fined}>
            <FadeIn duration={320} distance={10}>
              {children}
            </FadeIn>
          </ScrollView>
        ) : (
          <View style={[{ flex: 1 }, inner]}>{children}</View>
        )}
        {fab ? (
          <View pointerEvents="box-none" style={[styles.fab, { bottom: footer ? 16 : fabBottom }]}>
            {fab}
          </View>
        ) : null}
        {footer}
      </View>
    </View>
  );
}

/**
 * Stack-screen header, like the top of a magazine page: a square back button, a mono kicker above, the title in heavy serif,
 * a subtitle, and an ink rule underneath. Optional right-hand actions.
 */
export function Header({ title, subtitle, kicker, right, onBack, back = true, backLabel = 'Back' }: { title: string; subtitle?: string;
kicker?: string; right?: ReactNode; onBack?: () => void; back?: boolean; backLabel?: string }) {
  return (
    <View style={styles.header}>
      {back ? (
        <PlatePress
          accessibilityLabel={`Go back to ${backLabel}`}
          offset={3}
          corner={radius.button}
          wrapperStyle={styles.back}
          style={styles.backFace}
          onPress={() => {
            if (onBack) onBack();
            else if (router.canGoBack()) router.back();
            else router.replace('/(tabs)/explore');
          }}>
          <Ionicons name="arrow-back" size={20} color={colors.ink} />
        </PlatePress>
      ) : null}
      <View style={{ flexDirection: 'row', alignItems: 'flex-end', gap: 12 }}>
        <View style={{ flex: 1 }}>
          {kicker ? (
            <View style={styles.kickerRow}>
              <View style={styles.bullet} />
              <T variant="kicker" color={colors.textSecondary}>
                {kicker}
              </T>
            </View>
          ) : null}
          <T variant="display" numberOfLines={2} accessibilityRole="header">
            {title}
          </T>
          {subtitle ? (
            <T variant="small" color={colors.textSecondary} numberOfLines={2} style={{ marginTop: 4 }}>
              {subtitle}
            </T>
          ) : null}
        </View>
        {right}
      </View>
      <Rule style={{ marginTop: 14 }} />
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center', overflow: 'hidden' },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  fab: { position: 'absolute', right: 20 },
  header: { paddingHorizontal: 20, paddingTop: 4, paddingBottom: 16, gap: 10 },
  back: { alignSelf: 'flex-start' },
  backFace: { width: 44, height: 44, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.ca
rd, alignItems: 'center', justifyContent: 'center' },
  kickerRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginBottom: 2 },
  bullet: { width: 10, height: 10, backgroundColor: colors.accent, borderWidth: 1.5, borderColor: colors.ink },
});
```

src/components/ui/Segmented.tsx

```
import { StyleSheet, View } from 'react-native';

import { colors, line, radius } from '@/theme';
```

```
import { Press } from './Press';
import { T } from './T';

/** A row of choices in one outline: the chosen one is filled with ink (Itinerary | Vote, Crew | Scout AI…). */
export function Segmented<K extends string>({ options, value, onChange }: { options: { key: K; label: string }[]; value: K; onChange:
(k: K) => void }) {
  return (
    <View style={styles.track} accessibilityRole="tablist">
      {options.map((o) => {
        const on = o.key === value;
        return (
          <Press key={o.key} onPress={() => onChange(o.key)} style={[styles.item, on && styles.on]} accessibilityRole="tab" accessibilit
yState={{ selected: on }} accessibilityLabel={o.label} scaleTo={0.98}>
            <T variant="small" weight="bold" color={on ? colors.onPrimary : colors.text}>
              {o.label}
            </T>
          </Press>
        );
      })}
    </View>
  );
}

const styles = StyleSheet.create({
  track: { flexDirection: 'row', backgroundColor: colors.card, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink,
padding: 3, gap: 3 },
  item: { flex: 1, height: 44, alignItems: 'center', justifyContent: 'center', borderRadius: radius.button - 3, flexDirection: 'row', ga
p: 6 },
  on: { backgroundColor: colors.primary },
});
```

src/components/ui/Sheet.tsx

```
import { useEffect, useState, type ReactNode } from 'react';
import {
  Animated,
  Easing,
  KeyboardAvoidingView,
  Modal,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  View,
  useWindowDimensions,
} from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { MAX_WIDTH, colors, line, radius, timing } from '@/theme';

import { T } from './T';

export interface SheetProps {
  visible: boolean;
  onClose: () => void;
  title?: string;
  subtitle?: string;
  children: ReactNode;
  footer?: ReactNode;
  /** Fraction of screen height the sheet may use. */
  maxHeight?: number;
  scroll?: boolean;
}

/** Bottom sheet with a drag-handle look, slide-up animation and a tappable backdrop. */
export function Sheet({ visible, onClose, title, subtitle, children, footer, maxHeight = 0.9, scroll = true }: SheetProps) {
  const { height } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const [mounted, setMounted] = useState(visible);
  const [anim] = useState(() => new Animated.Value(0));
  // Mount as soon as we're asked to show; unmount only after the close animation.
  if (visible && !mounted) setMounted(true);

  useEffect(() => {
    if (visible) {
      Animated.timing(anim, { toValue: 1, duration: timing.sheet, easing: Easing.out(Easing.cubic), useNativeDriver: Platform.OS !== 'we
b' }).start();
    } else if (mounted) {
      Animated.timing(anim, { toValue: 0, duration: timing.base, easing: Easing.in(Easing.cubic), useNativeDriver: Platform.OS !== 'web'
}).start(() =>
        setMounted(false),
      );
    }
  }, [visible, anim, mounted]);
```

```
  if (!mounted) return null;

  const translateY = anim.interpolate({ inputRange: [0, 1], outputRange: [height * 0.6, 0] });

  return (
    <Modal transparent visible={mounted} onRequestClose={onClose} statusBarTranslucent animationType="none">
      <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={{ flex: 1 }}>
        <Animated.View style={[StyleSheet.absoluteFill, { backgroundColor: colors.overlay, opacity: anim }]}>
          <Pressable style={{ flex: 1 }} onPress={onClose} accessibilityLabel="Close" accessibilityRole="button" />
        </Animated.View>
        <View style={styles.wrap} pointerEvents="box-none">
          <Animated.View
            accessibilityViewIsModal
            style={[styles.sheet, { maxHeight: height * maxHeight, paddingBottom: Math.max(insets.bottom, 16), transform: [{ translateY
}] }]}>
            <View style={styles.handle} />
            {title ? (
              <View style={{ paddingHorizontal: 22, paddingTop: 6, paddingBottom: 10 }}>
                <T variant="headline" accessibilityRole="header">
                  {title}
                </T>
                {subtitle ? (
                  <T variant="small" color={colors.textSecondary} style={{ marginTop: 2 }}>
                    {subtitle}
                  </T>
                ) : null}
              </View>
            ) : null}
            {scroll ? (
              <ScrollView
                keyboardShouldPersistTaps="handled"
                contentContainerStyle={{ paddingHorizontal: 22, paddingBottom: 12, gap: 16 }}
                showsVerticalScrollIndicator={false}>
                {children}
              </ScrollView>
            ) : (
              <View style={{ paddingHorizontal: 22, flexShrink: 1 }}>{children}</View>
            )}
            {footer ? <View style={{ paddingHorizontal: 22, paddingTop: 8 }}>{footer}</View> : null}
          </Animated.View>
        </View>
      </KeyboardAvoidingView>
    </Modal>
  );
}

const styles = StyleSheet.create({
  wrap: { flex: 1, justifyContent: 'flex-end', alignItems: 'center' },
  sheet: {
    width: '100%',
    maxWidth: MAX_WIDTH,
    backgroundColor: colors.raised,
    borderTopLeftRadius: radius.modal,
    borderTopRightRadius: radius.modal,
    borderWidth: line,
    borderBottomWidth: 0,
    borderColor: colors.ink,
  },
  handle: {
    alignSelf: 'center',
    width: 40,
    height: 5,
    borderRadius: 3,
    backgroundColor: colors.ink,
    marginTop: 10,
    marginBottom: 10,
  },
});
```

src/components/ui/Skeleton.tsx

```
import { Animated, type DimensionValue, type StyleProp, type ViewStyle } from 'react-native';

import { colors, radius } from '@/theme';

import { useLoop } from './motion';

/**
 * A grey block that pulses where content will appear. Give it the same size as the real thing so nothing jumps when the
 * data arrives. It sits still when animations are off.
 */
export function Skeleton({ width = '100%', height = 16, corner = radius.card, style }: { width?: DimensionValue; height?: number; corne
r?: number; style?: StyleProp<ViewStyle> }) {
  const v = useLoop(1100, { pingPong: true, rest: 0.4 });
  return (
    <Animated.View
```

```
      accessibilityElementsHidden
      importantForAccessibility="no-hide-descendants"
      style={[{ width, height, borderRadius: corner, backgroundColor: colors.primarySoft, opacity: v.interpolate({ inputRange: [0, 1], o
utputRange: [0.45, 1] }) }, style]}
    />
  );
}
```

src/components/ui/Slider.tsx

```
import { useState } from 'react';
import { StyleSheet, View, type GestureResponderEvent } from 'react-native';

import { colors, line, radius } from '@/theme';

import { T } from './T';

export interface SliderProps {
  value: number;
  min: number;
  max: number;
  step?: number;
  onChange: (v: number) => void;
  label: string;
  formatValue?: (v: number) => string;
}

const THUMB_W = 22;

/**
 * A chunky slider in the ink-and-paper look: an outlined track that fills with the accent and an ink block to drag.
 * Drag it, tap the track, or use a screen reader's increase and decrease. The whole 48 point strip is the control.
 */
export function Slider({ value, min, max, step = 1, onChange, label, formatValue }: SliderProps) {
  const [width, setWidth] = useState(1);
  const pct = (value - min) / (max - min);
  const travel = Math.max(1, width - THUMB_W);

  // the children ignore touches, so locationX is always measured on the strip itself
  const update = (e: GestureResponderEvent) => {
    const ratio = Math.max(0, Math.min(1, (e.nativeEvent.locationX - THUMB_W / 2) / travel));
    const next = Math.round((min + ratio * (max - min)) / step) * step;
    if (next !== value) onChange(next);
  };

  return (
    <View style={{ gap: 4 }}>
      <View
        accessible
        accessibilityRole="adjustable"
        accessibilityLabel={label}
        accessibilityValue={{ min, max, now: value, text: formatValue?.(value) }}
        accessibilityActions={[{ name: 'increment' }, { name: 'decrement' }]}
        onAccessibilityAction={(e) => {
          if (e.nativeEvent.actionName === 'increment') onChange(Math.min(max, value + step));
          if (e.nativeEvent.actionName === 'decrement') onChange(Math.max(min, value - step));
        }}
        style={styles.hit}
        onLayout={(e) => setWidth(e.nativeEvent.layout.width)}
        onStartShouldSetResponder={() => true}
        onMoveShouldSetResponder={() => true}
        onResponderTerminationRequest={() => false}
        onResponderGrant={update}
        onResponderMove={update}>
        <View pointerEvents="none" style={styles.track}>
          <View style={[styles.fill, { width: `${pct * 100}%` }]} />
        </View>
        <View pointerEvents="none" style={[styles.thumb, { left: pct * travel }]} />
      </View>
      {formatValue ? (
        <View style={styles.ends} accessibilityElementsHidden importantForAccessibility="no">
          <T variant="kicker" color={colors.textMuted}>
            {formatValue(min)}
          </T>
          <T variant="kicker" color={colors.textMuted}>
            {formatValue(max)}
          </T>
        </View>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  hit: { height: 48, justifyContent: 'center' },
  ends: { flexDirection: 'row', justifyContent: 'space-between' },
```

```
  track: { height: 16, marginHorizontal: THUMB_W / 2, borderRadius: radius.xs, borderWidth: line, borderColor: colors.ink, backgroundCol
or: colors.card, overflow: 'hidden' },
  fill: { height: '100%', backgroundColor: colors.accent, borderRightWidth: line, borderColor: colors.ink },
  thumb: { position: 'absolute', top: 7, width: THUMB_W, height: 34, borderRadius: radius.xs, backgroundColor: colors.ink, borderWidth:
line, borderColor: colors.ink },
});
```

src/components/ui/SplitFlap.tsx

```
import { useEffect, useState } from 'react';
import { Animated, Easing, StyleSheet, Text, View } from 'react-native';

import { colors, fonts } from '@/theme';

import { nativeDriver, useMotion } from './motion';

/** One digit on a black tile that flips like a departures board when it changes. */
function Tile({ ch, size }: { ch: string; size: number }) {
  const motion = useMotion();
  const [state, setState] = useState({ cur: ch, prev: ch });
  const [t] = useState(() => new Animated.Value(1));
  // when the character changes, keep the old one so it can flip away
  if (state.cur !== ch) setState({ cur: ch, prev: state.cur });
  useEffect(() => {
    if (!motion || state.prev === state.cur) {
      t.setValue(1);
      return;
    }
    t.setValue(0);
    const a = Animated.timing(t, { toValue: 1, duration: 420, easing: Easing.inOut(Easing.cubic), useNativeDriver: nativeDriver });
    a.start();
    return () => a.stop();
  }, [state, motion, t]);

  const font = Math.round(size * 0.74);
  const face = { fontFamily: fonts.mono, fontSize: font, lineHeight: size, color: colors.bg, textAlign: 'center' as const, includeFontPa
dding: false };
  const flipping = motion && state.prev !== state.cur;
  return (
    <View style={[styles.tile, { width: Math.round(size * 0.68), height: size, borderRadius: Math.round(size * 0.1) }]}>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: t.interpolate({ inputRange: [0, 0.49, 0.5, 1], outputRange: [0, 0, 1,
1] }), transform: [{ scaleY: t.interpolate({ inputRange: [0, 0.5, 1], outputRange: [0, 0, 1] }) }] }]}>
        <Text style={face} allowFontScaling={false}>
          {ch}
        </Text>
      </Animated.View>
      {flipping ? (
        <Animated.View style={[StyleSheet.absoluteFill, { opacity: t.interpolate({ inputRange: [0, 0.49, 0.5, 1], outputRange: [1, 1, 0,
0] }), transform: [{ scaleY: t.interpolate({ inputRange: [0, 0.5, 1], outputRange: [1, 0, 0] }) }] }]}>
          <Text style={face} allowFontScaling={false}>
            {state.prev}
          </Text>
        </Animated.View>
      ) : null}
      <View pointerEvents="none" style={[styles.hinge, { top: size / 2 - 1 }]} />
    </View>
  );
}

/**
 * A number on departure-board tiles: each digit is white-on-ink and flips to the next one when the number changes
 * (a ticking countdown). Still when animations are off. `size` is the height of a tile.
 */
export function SplitFlap({ value, size = 64 }: { value: number | string; size?: number }) {
  const chars = String(value).split('');
  return (
    <View accessible accessibilityLabel={String(value)} style={{ flexDirection: 'row', gap: Math.max(3, Math.round(size / 16)) }}>
      {chars.map((ch, i) =>
        /\d/.test(ch) ? (
          <Tile key={chars.length - i} ch={ch} size={size} />
        ) : (
          <Text key={chars.length - i} allowFontScaling={false} style={{ fontFamily: fonts.mono, fontSize: Math.round(size * 0.6), lineH
eight: size, color: colors.ink }}>
            {ch}
          </Text>
        ),
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  tile: { backgroundColor: colors.ink, overflow: 'hidden', justifyContent: 'center' },
  hinge: { position: 'absolute', left: 0, right: 0, height: 2, backgroundColor: colors.bg, opacity: 0.55 },
});
```

src/components/ui/StaggerText.tsx

```
import { View, type StyleProp, type TextStyle } from 'react-native';

import { FadeIn } from './motion';
import { T, type TProps } from './T';

/**
 * A headline whose words slide up and fade in one after another (about 50 ms apart). Plain text when animations are off.
 * The whole line is one accessible heading, and keeps real spaces so it can be read and searched as one sentence.
 */
export function StaggerText({ text, variant = 'display', color, delay = 0, step = 50, distance = 16, center, style }: { text: string; va
riant?: TProps['variant']; color?: string; delay?: number; step?: number; distance?: number; center?: boolean; style?: StyleProp<TextSty
le> }) {
  const words = text.split(' ');
  return (
    <View accessible accessibilityRole="header" accessibilityLabel={text} style={{ flexDirection: 'row', flexWrap: 'wrap', justifyConten
t: center ? 'center' : 'flex-start' }}>
      {words.map((w, i) => (
        <FadeIn key={`${w}${i}`} delay={delay + i * step} distance={distance} duration={380}>
          <T variant={variant} color={color} style={style} importantForAccessibility="no">
            {w}
            {i < words.length - 1 ? ' ' : ''}
          </T>
        </FadeIn>
      ))}
    </View>
  );
}
```

src/components/ui/Stamp.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useState } from 'react';
import { Animated, Easing, StyleSheet, View } from 'react-native';

import { colors, radius } from '@/theme';

import type { IconName } from './bits';
import { nativeDriver, useMotion } from './motion';
import { T } from './T';

export type StampTone = 'ink' | 'accent' | 'positive' | 'warn' | 'danger' | 'muted';

/** Colours of each tone: an outline and text in the tone, on a pale fill of it (`accent` is solid, like a highlighter). */
export function stampColors(tone: StampTone) {
  switch (tone) {
    case 'accent':
      return { fg: colors.onAccent, bg: colors.accent, line: colors.ink };
    case 'positive':
      return { fg: colors.positive, bg: colors.positiveSoft, line: colors.positive };
    case 'warn':
      return { fg: colors.warn, bg: colors.warnSoft, line: colors.warn };
    case 'danger':
      return { fg: colors.red, bg: colors.redSoft, line: colors.red };
    case 'muted':
      return { fg: colors.textSecondary, bg: 'transparent', line: colors.borderStrong };
    default:
      return { fg: colors.ink, bg: 'transparent', line: colors.ink };
  }
}

/**
 * A rubber stamp: a short word in mono capitals inside an outline, a little crooked on purpose.
 * It is the app's one kind of status label (GOING, LOCKED IN, SETTLED, OVER BUDGET). With `slam` it lands on screen with a thud.
 */
export function Stamp({ label, tone = 'ink', icon, rotate = -3, slam }: { label: string; tone?: StampTone; icon?: IconName; rotate?: num
ber; slam?: boolean }) {
  const c = stampColors(tone);
  const motion = useMotion();
  const [t] = useState(() => new Animated.Value(slam && motion ? 0 : 1));
  useEffect(() => {
    if (!slam || !motion) return;
    t.setValue(0);
    const a = Animated.timing(t, { toValue: 1, duration: 260, easing: Easing.out(Easing.back(2)), useNativeDriver: nativeDriver });
    a.start();
    return () => a.stop();
  }, [slam, motion, t]);
  return (
    <Animated.View
      accessible
      accessibilityLabel={label}
      style={[styles.stamp, { borderColor: c.line, backgroundColor: c.bg, opacity: t, transform: [{ rotate: `${rotate}deg` }, { scale:
t.interpolate({ inputRange: [0, 1], outputRange: [1.9, 1] }) }] }]}>
      <View style={styles.row}>
```

```
        {icon ? <Ionicons name={icon} size={12} color={c.fg} /> : null}
        <T variant="kicker" color={c.fg} style={{ fontSize: 11 }} importantForAccessibility="no">
          {label}
        </T>
      </View>
    </Animated.View>
  );
}

const styles = StyleSheet.create({
  stamp: { alignSelf: 'flex-start', borderWidth: 2, borderRadius: radius.xs, paddingHorizontal: 8, paddingVertical: 3 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 4 },
});
```

src/components/ui/T.tsx

```
import { StyleSheet, Text, type TextProps, type TextStyle } from 'react-native';

import { useAppStore } from '@/store/useAppStore';
import { colors, fonts } from '@/theme';

/**
 * Three voices, like a printed boarding pass:
 *  - Serif (Fraunces, heavy): `display` for page titles, `headline` for section titles. Sentence case, never for paragraphs, never belo
w 26 px.
 *  - Mono (Space Mono bold): `hero` and `numeral` / `numeralSm` for every number that matters (days, money, times, scores), and `kicker
`
 *    for the small caps labels (DAY 02 / 05, POSITANO).
 *  - Grotesk (Space Grotesk): `h1`, `h2`, `h3` and everything you read: `title`, `body`, `bodySm`, `small`, `caption`, `micro`.
 */
type Variant = 'hero' | 'display' | 'headline' | 'numeral' | 'numeralSm' | 'h1' | 'h2' | 'h3' | 'title' | 'body' | 'bodySm' | 'small' |
'caption' | 'kicker' | 'micro';
type Weight = 'regular' | 'medium' | 'semibold' | 'bold' | 'extrabold';

const variants: Record<Variant, TextStyle> = {
  hero: { fontSize: 72, lineHeight: 76, fontFamily: fonts.mono, letterSpacing: -2, fontVariant: ['tabular-nums'] },
  display: { fontSize: 40, lineHeight: 44, fontFamily: fonts.display, letterSpacing: -0.9 },
  headline: { fontSize: 26, lineHeight: 30, fontFamily: fonts.serif, letterSpacing: -0.4 },
  numeral: { fontSize: 34, lineHeight: 38, fontFamily: fonts.mono, letterSpacing: -1, fontVariant: ['tabular-nums'] },
  numeralSm: { fontSize: 22, lineHeight: 28, fontFamily: fonts.mono, letterSpacing: -0.5, fontVariant: ['tabular-nums'] },
  h1: { fontSize: 26, lineHeight: 32, fontFamily: fonts.bold, letterSpacing: -0.4 },
  h2: { fontSize: 21, lineHeight: 27, fontFamily: fonts.bold, letterSpacing: -0.3 },
  h3: { fontSize: 18, lineHeight: 24, fontFamily: fonts.bold, letterSpacing: -0.2 },
  title: { fontSize: 16, lineHeight: 22, fontFamily: fonts.semibold },
  body: { fontSize: 16, lineHeight: 23, fontFamily: fonts.regular },
  bodySm: { fontSize: 15, lineHeight: 21, fontFamily: fonts.regular },
  small: { fontSize: 14, lineHeight: 19, fontFamily: fonts.regular },
  caption: { fontSize: 12.5, lineHeight: 17, fontFamily: fonts.regular },
  kicker: { fontSize: 11, lineHeight: 15, fontFamily: fonts.mono, letterSpacing: 1.2, textTransform: 'uppercase' },
  micro: { fontSize: 11, lineHeight: 14, fontFamily: fonts.medium },
};

/** The serif and mono tiers are already large, so the person's text-size setting grows them only a little. */
const BIG: Variant[] = ['hero', 'display', 'headline', 'numeral', 'numeralSm'];

export interface TProps extends TextProps {
  variant?: Variant;
  weight?: Weight;
  color?: string;
  center?: boolean;
}

/** App text: applies the brand font, type scale and the user's text-size setting. */
export function T({ variant = 'body', weight, color, center, style, ...rest }: TProps) {
  const scale = useAppStore((s) => s.settings.textScale);
  const v = variants[variant];
  const k = BIG.includes(variant) ? 1 + (scale - 1) * 0.4 : scale;
  const scaled: TextStyle =
    k === 1
      ? {}
      : { fontSize: (v.fontSize ?? 15) * k, lineHeight: v.lineHeight ? v.lineHeight * k : undefined };
  return (
    <Text
      maxFontSizeMultiplier={1.4}
      {...rest}
      style={[
        styles.base,
        v,
        scaled,
        weight && { fontFamily: fonts[weight] },
        color ? { color } : null,
        center && { textAlign: 'center' },
        style,
      ]}
    />
  );
```

```
}

const styles = StyleSheet.create({
  base: { color: colors.text },
});
```

src/components/ui/Ticket.tsx

```
import type { ReactNode } from 'react';
import { StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { colors, line, plateOffset, radius } from '@/theme';

import { Plate } from './Plate';
import { Perforation } from './Rule';

const NOTCH = 22;

/** Half a punched hole, cut into the left or right edge of a ticket. */
function Notch({ side, holeColor }: { side: 'left' | 'right'; holeColor: string }) {
  const edge = side === 'left' ? { left: -line } : { right: -line };
  const pos = side === 'left' ? { left: -NOTCH / 2 } : { right: -NOTCH / 2 };
  return (
    <View pointerEvents="none" style={[styles.notchBox, edge]}>
      <View style={[styles.ring, pos]} />
      <View style={[styles.hole, side === 'left' ? { left: -NOTCH / 2 + 2.5 } : { right: -NOTCH / 2 + 2.5 }, { backgroundColor: holeColo
r }]} />
    </View>
  );
}

/**
 * A boarding-pass card: a main part and, optionally, a stub below a perforated tear line with a half-circle punched out of each edge.
 * `holeColor` is the colour of whatever the ticket lies on (the page, by default) so the punched holes look like holes.
 */
export function Ticket({ children, stub, plated = true, holeColor = colors.bg, tone, style }: { children: ReactNode; stub?: ReactNode; p
lated?: boolean; holeColor?: string; tone?: 'accent' | 'ink'; style?: StyleProp<ViewStyle> }) {
  const fill = tone === 'accent' ? colors.accent : tone === 'ink' ? colors.ink : colors.card;
  const card = (
    <View style={[styles.card, { backgroundColor: fill }]}>
      {children}
      {stub ? (
        <>
          <View style={styles.tear}>
            <Notch side="left" holeColor={holeColor} />
            <Perforation style={styles.dashes} color={tone === 'ink' ? colors.bg : tone === 'accent' ? colors.onAccent : colors.ink} />
            <Notch side="right" holeColor={holeColor} />
          </View>
          {stub}
        </>
      ) : null}
    </View>
  );
  return plated ? (
    <Plate offset={plateOffset} corner={radius.card} style={style}>
      {card}
    </Plate>
  ) : (
    <View style={style}>{card}</View>
  );
}

const styles = StyleSheet.create({
  card: { borderWidth: line, borderColor: colors.ink, borderRadius: radius.card },
  tear: { height: NOTCH, justifyContent: 'center', marginHorizontal: 0 },
  dashes: { marginHorizontal: NOTCH / 2 + 4 },
  notchBox: { position: 'absolute', top: 0, width: NOTCH / 2 + line, height: NOTCH, overflow: 'hidden' },
  ring: { position: 'absolute', top: 0, width: NOTCH, height: NOTCH, borderRadius: NOTCH, backgroundColor: colors.ink },
  hole: { position: 'absolute', top: 2.5, width: NOTCH - 5, height: NOTCH - 5, borderRadius: NOTCH },
});
```

src/components/ui/__tests__/confetti.test.ts

```
import { PIECES, confettiPieces } from '../Confetti';

describe('confetti', () => {
  it('throws the same pieces every time for the same count', () => {
    expect(confettiPieces(40)).toEqual(confettiPieces(40));
  });

  it('keeps every piece inside the screen and in sensible ranges', () => {
    for (const p of confettiPieces(PIECES.max)) {
      expect(p.x).toBeGreaterThanOrEqual(0.1);
      expect(p.x).toBeLessThanOrEqual(0.9);
      expect(p.delay).toBeLessThanOrEqual(0.2);
      expect(p.size).toBeGreaterThanOrEqual(7);
```

```
      expect(p.color).toBeGreaterThanOrEqual(0);
      expect(p.color).toBeLessThan(5);
      expect(p.fall).toBeGreaterThan(p.rise);
    }
  });

  it('throws nothing on Chill, a little on Lively and a lot on Max', () => {
    expect(PIECES.chill).toBe(0);
    expect(PIECES.lively).toBeGreaterThan(0);
    expect(PIECES.max).toBeGreaterThan(PIECES.lively);
  });
});
```

src/components/ui/bits.tsx

```
/** Small presentational building blocks shared by every screen. */
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import type { ComponentProps, ReactNode } from 'react';
import { StyleSheet, Switch, View, type StyleProp, type ViewStyle } from 'react-native';

import { imageSource } from '@/data/images';
import { colors, hair, isDark, line, radius } from '@/theme';
import { initials } from '@/utils/format';

import { Float } from './motion';
import { Plate, PlatePress } from './Plate';
import { Press } from './Press';
import { Rule } from './Rule';
import { Stamp, stampColors, type StampTone } from './Stamp';
import { T } from './T';

export type IconName = ComponentProps<typeof Ionicons>['name'];

/**
 * A card: a paper-white surface with a 2 px ink outline and 12 px corners. `plated` puts the hard plate under it, for the one
 * card on a screen that should stand up (the others lie flat). A pressable card squeezes a little when pressed.
 */
export function Card({ children, style, padded = true, onPress, accessibilityLabel, plated }: { children: ReactNode; style?: StyleProp<V
iewStyle>; padded?: boolean; onPress?: () => void; accessibilityLabel?: string; plated?: boolean }) {
  const s = [styles.card, padded && { padding: 16 }, style];
  if (plated) {
    const { margin, marginTop, marginBottom, marginLeft, marginRight, marginHorizontal, marginVertical, alignSelf, flex, width } = Style
Sheet.flatten(style) ?? {};
    const outer = { margin, marginTop, marginBottom, marginLeft, marginRight, marginHorizontal, marginVertical, alignSelf, flex, width
};
    const inner = [styles.card, padded && { padding: 16 }];
    return onPress ? (
      <PlatePress onPress={onPress} accessibilityLabel={accessibilityLabel} corner={radius.card} wrapperStyle={outer} style={inner}>
        {children}
      </PlatePress>
    ) : (
      <Plate corner={radius.card} style={outer}>
        <View style={inner}>{children}</View>
      </Plate>
    );
  }
  if (onPress)
    return (
      <Press onPress={onPress} style={s} scaleTo={0.985} accessibilityLabel={accessibilityLabel}>
        {children}
      </Press>
    );
  return <View style={s}>{children}</View>;
}

/** A small outlined rectangle for choosing. Chosen = filled with the accent, like a highlighter pass. */
export function Chip({
  label,
  active,
  onPress,
  icon,
  image,
  small,
  tone,
  style,
}: {
  label: string;
  active?: boolean;
  onPress?: () => void;
  icon?: IconName;
  image?: string;
  small?: boolean;
  tone?: 'soft';
  style?: StyleProp<ViewStyle>;
}) {
```

```
  const bg = active ? colors.accent : tone === 'soft' ? colors.bgAlt : colors.card;
  const fg = active ? colors.onAccent : colors.text;
  return (
    <Press
      onPress={onPress}
      accessibilityRole="button"
      accessibilityState={{ selected: !!active }}
      accessibilityLabel={label}
      scaleTo={0.97}
      style={[styles.chip, small && styles.chipSmall, { backgroundColor: bg, borderColor: colors.ink }, !!image && { paddingLeft: 5 }, s
tyle]}>
      {image ? <Image source={imageSource(image)} style={styles.chipImg} contentFit="cover" /> : null}
      {icon ? <Ionicons name={icon} size={small ? 14 : 16} color={fg} /> : null}
      <T variant={small ? 'caption' : 'small'} weight="bold" color={fg} numberOfLines={1}>
        {label}
      </T>
      {active ? <Ionicons name="checkmark" size={14} color={fg} /> : null}
    </Press>
  );
}

/** A person: a photo or initials in an ink ring. */
export function Avatar({ name, src, size = 34, ring, dot }: { name: string; src?: string; size?: number; ring?: string; /** A small stat
us dot, for example green for "here now". */ dot?: string }) {
  const tints = [colors.accentSoft, colors.bgAlt, colors.cardMuted, colors.primarySoft];
  const bg = tints[(name.charCodeAt(0) + name.length) % tints.length];
  return (
    <View accessibilityLabel={name} style={{ width: size, height: size }}>
      <View style={{ width: size, height: size, borderRadius: size / 2, backgroundColor: bg, alignItems: 'center', justifyContent: 'cent
er', overflow: 'hidden', borderWidth: size < 30 ? 1.5 : line, borderColor: ring ?? colors.ink }}>
        {src ? (
          <Image source={imageSource(src)} style={{ width: '100%', height: '100%' }} contentFit="cover" />
        ) : (
          <T variant="kicker" color={colors.ink} style={{ fontSize: size > 40 ? 15 : 10.5, letterSpacing: 0 }}>
            {initials(name)}
          </T>
        )}
      </View>
      {dot ? <View style={[styles.avatarDot, { backgroundColor: dot, width: Math.max(9, size * 0.28), height: Math.max(9, size * 0.28),
borderRadius: size }]} /> : null}
    </View>
  );
}

export function AvatarStack({ people, max = 3, size = 28 }: { people: { name: string; avatar?: string }[]; max?: number; size?: number
}) {
  const shown = people.slice(0, max);
  const extra = people.length - shown.length;
  return (
    <View style={{ flexDirection: 'row', alignItems: 'center' }}>
      {shown.map((p, i) => (
        <View key={`${p.name}${i}`} style={{ marginLeft: i === 0 ? 0 : -size * 0.28 }}>
          <Avatar name={p.name} src={p.avatar} size={size} />
        </View>
      ))}
      {extra > 0 && (
        <View style={{ marginLeft: -size * 0.28, width: size, height: size, borderRadius: size / 2, backgroundColor: colors.accent, bord
erWidth: size < 30 ? 1.5 : line, borderColor: colors.ink, alignItems: 'center', justifyContent: 'center' }}>
          <T variant="kicker" color={colors.onAccent} style={{ fontSize: 10.5, letterSpacing: 0 }}>
            +{extra}
          </T>
        </View>
      )}
    </View>
  );
}

/** A chunky bar in an ink outline. Pass a state colour (`colors.positive`, `colors.warn`, `colors.red`) only when it means something.
*/
export function ProgressBar({ value, color = colors.accent, track = colors.card, height = 14 }: { value: number; color?: string; track?:
string; height?: number }) {
  const pct = Math.max(0, Math.min(1, value));
  return (
    <View
      accessibilityRole="progressbar"
      accessibilityValue={{ min: 0, max: 100, now: Math.round(pct * 100) }}
      style={{ height, borderRadius: radius.xs, backgroundColor: track, borderWidth: line, borderColor: colors.ink, overflow: 'hidden'
}}>
      <View style={{ width: `${pct * 100}%`, height: '100%', backgroundColor: color, borderRightWidth: pct > 0 && pct < 1 ? line : 0, bo
rderColor: colors.ink }} />
    </View>
  );
}

/** A square-cornered icon button, 44 points by default, with its own small plate. */
export function IconButton({
```

```
  icon,
  onPress,
  label,
  size = 44,
  bg = colors.card,
  color = colors.ink,
  badge,
  bordered = true,
}: {
  icon: IconName;
  onPress?: () => void;
  label: string;
  size?: number;
  bg?: string;
  color?: string;
  badge?: number | boolean;
  bordered?: boolean;
}) {
  const face = { width: size, height: size, borderRadius: radius.button, backgroundColor: bg, alignItems: 'center' as const, justifyCont
ent: 'center' as const, borderWidth: bordered ? line : 0, borderColor: colors.ink };
  const inner = (
    <>
      <Ionicons name={icon} size={Math.round(size * 0.46)} color={color} />
      {badge ? (
        <View style={styles.badge}>
          {typeof badge === 'number' ? (
            <T variant="micro" weight="bold" color={colors.white} style={{ fontSize: 9.5 }}>
              {badge > 9 ? '9+' : badge}
            </T>
          ) : null}
        </View>
      ) : null}
    </>
  );
  return bordered ? (
    <PlatePress onPress={onPress} accessibilityLabel={label} offset={3} corner={radius.button} color={bg === colors.primary ? colors.acc
ent : colors.plate} style={face}>
      {inner}
    </PlatePress>
  ) : (
    <Press onPress={onPress} accessibilityLabel={label} style={face}>
      {inner}
    </Press>
  );
}

/** The title of a block of the page: a serif headline over an ink rule, with a quiet mono action on the right. */
export function SectionHeader({ title, action, onAction, style }: { title: string; action?: string; onAction?: () => void; style?: Style
Prop<ViewStyle> }) {
  return (
    <View style={[styles.sectionHeader, style]}>
      <View style={styles.sectionTop}>
        <T variant="headline" accessibilityRole="header" style={{ flexShrink: 1 }}>
          {title}
        </T>
        {action ? (
          <Press onPress={onAction} hitSlop={8} accessibilityLabel={action} style={styles.sectionAction}>
            <T variant="kicker" color={colors.ink}>
              {action}
            </T>
            <Ionicons name="arrow-forward" size={14} color={colors.ink} />
          </Press>
        ) : null}
      </View>
      <Rule />
    </View>
  );
}

/**
 * A small status tag, drawn like a stamp. Green, amber and red are for states only (under budget, near, over; going, maybe,
 * can't go). `yellow` is the theme accent; blue and purple are quiet info tints.
 */
export function Pill({ label, tone = 'green', icon }: { label: string; tone?: 'green' | 'orange' | 'blue' | 'red' | 'gray' | 'yellow' |
'purple'; icon?: IconName }) {
  const map: Record<string, StampTone> = { green: 'positive', orange: 'warn', red: 'danger', gray: 'muted', yellow: 'accent', blue: 'in
k', purple: 'ink' };
  return <Stamp label={label} tone={map[tone]} icon={icon} rotate={0} />;
}
export { stampColors };

/**
 * An empty list turned into an invitation: a big line icon in a dashed ring, a serif prompt, one line of help and one button.
 * (`art` is ignored: the app uses line icons, not emoji.)
 */
export function EmptyState({ icon, title, body, children }: { icon?: IconName; title: string; body?: string; children?: ReactNode; art?:
```

```
string }) {
  return (
    <View style={styles.empty}>
      <Float amount={6}>
        <View style={styles.emptyIcon}>
          <Ionicons name={icon ?? 'airplane-outline'} size={34} color={colors.ink} />
        </View>
      </Float>
      <T variant="headline" center accessibilityRole="header">
        {title}
      </T>
      {body ? (
        <T variant="small" color={colors.textSecondary} center>
          {body}
        </T>
      ) : null}
      {children ? <View style={{ alignSelf: 'stretch', marginTop: 8 }}>{children}</View> : null}
    </View>
  );
}

export function ToggleRow({ label, sub, value, onChange, icon }: { label: string; sub?: string; value: boolean; onChange: (v: boolean) =
> void; icon?: IconName }) {
  return (
    <View style={styles.toggleRow}>
      {icon ? (
        <View style={styles.rowIcon}>
          <Ionicons name={icon} size={18} color={colors.ink} />
        </View>
      ) : null}
      <View style={{ flex: 1 }}>
        <T variant="bodySm" weight="semibold">
          {label}
        </T>
        {sub ? (
          <T variant="caption" color={colors.textSecondary}>
            {sub}
          </T>
        ) : null}
      </View>
      <Switch
        accessibilityLabel={label}
        value={value}
        onValueChange={onChange}
        trackColor={{ false: colors.borderStrong, true: colors.ink }}
        thumbColor={isDark ? colors.ink : colors.card}
        {...({ activeThumbColor: colors.accent } as object)}
      />
    </View>
  );
}

export function ListRow({ icon, label, sub, onPress, right, tint = colors.ink }: { icon: IconName; label: string; sub?: string; onPres
s?: () => void; right?: ReactNode; tint?: string }) {
  return (
    <Press onPress={onPress} style={styles.listRow} scaleTo={0.99} accessibilityLabel={label}>
      <View style={[styles.rowIcon, tint !== colors.ink && { backgroundColor: `${tint}1F`, borderColor: tint }]}>
        <Ionicons name={icon} size={18} color={tint} />
      </View>
      <View style={{ flex: 1 }}>
        <T variant="bodySm" weight="semibold">
          {label}
        </T>
        {sub ? (
          <T variant="caption" color={colors.textSecondary} numberOfLines={2}>
            {sub}
          </T>
        ) : null}
      </View>
      {right ?? <Ionicons name="arrow-forward" size={16} color={colors.ink} />}
    </Press>
  );
}

export function Divider({ style }: { style?: StyleProp<ViewStyle> }) {
  return <View style={[{ height: hair, backgroundColor: colors.border }, style]} />;
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: colors.card,
    borderRadius: radius.card,
    borderWidth: line,
    borderColor: colors.ink,
  },
  chip: {
```

```
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'center',
    gap: 6,
    height: 44,
    minWidth: 44,
    paddingHorizontal: 14,
    borderRadius: radius.chip,
    borderWidth: line,
  },
  chipSmall: { paddingHorizontal: 11, gap: 5 },
  chipImg: { width: 30, height: 30, borderRadius: radius.xs },
  avatarDot: { position: 'absolute', right: -1, bottom: -1, borderWidth: 2, borderColor: colors.ink },
  badge: {
    position: 'absolute',
    top: -7,
    right: -7,
    minWidth: 20,
    height: 20,
    paddingHorizontal: 4,
    borderRadius: 10,
    backgroundColor: colors.red,
    alignItems: 'center',
    justifyContent: 'center',
    borderWidth: 2,
    borderColor: colors.ink,
  },
  sectionHeader: { marginBottom: 14, gap: 8 },
  sectionTop: { flexDirection: 'row', alignItems: 'flex-end', justifyContent: 'space-between', minHeight: 36 },
  sectionAction: { flexDirection: 'row', alignItems: 'center', gap: 4, minHeight: 44, paddingLeft: 10 },
  empty: { alignItems: 'center', gap: 10, paddingVertical: 40, paddingHorizontal: 24 },
  emptyIcon: {
    width: 84,
    height: 84,
    borderRadius: 42,
    borderWidth: line,
    borderStyle: 'dashed',
    borderColor: colors.ink,
    backgroundColor: colors.accentSoft,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: 4,
  },
  toggleRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingVertical: 10, minHeight: 56 },
  rowIcon: { width: 38, height: 38, borderRadius: radius.button, borderWidth: line, borderColor: colors.ink, backgroundColor: colors.car
d, alignItems: 'center', justifyContent: 'center' },
  listRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingVertical: 11, minHeight: 56 },
});
```

src/components/ui/feedback.tsx

```
/**
 * Global toast + dialog system. Works the same on iOS, Android and web
 * (React Native's Alert is a no-op on web, so we render our own).
 */
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useState } from 'react';
import { Animated, Modal, Platform, Pressable, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { create } from 'zustand';

import { colors, line, radius } from '@/theme';

import { Button } from './Button';
import { Plate } from './Plate';
import type { IconName } from './bits';
import { tap } from './Press';
import { T } from './T';

interface ToastState {
  toast: { id: number; text: string; icon: IconName; tone: 'ok' | 'info' | 'warn' } | null;
  show: (text: string, opts?: { icon?: IconName; tone?: 'ok' | 'info' | 'warn' }) => void;
  hide: () => void;
}

export const useToast = create<ToastState>((set) => ({
  toast: null,
  show: (text, opts) => set({ toast: { id: Date.now(), text, icon: opts?.icon ?? 'checkmark-circle', tone: opts?.tone ?? 'ok' } }),
  hide: () => set({ toast: null }),
}));

export const toast = (text: string, opts?: { icon?: IconName; tone?: 'ok' | 'info' | 'warn' }) => {
  if (opts?.tone === 'warn') tap('warning');
  useToast.getState().show(text, opts);
};
```

```
export function ToastHost() {
  const t = useToast((s) => s.toast);
  const hide = useToast((s) => s.hide);
  const insets = useSafeAreaInsets();
  const [anim] = useState(() => new Animated.Value(0));

  useEffect(() => {
    if (!t) return;
    anim.setValue(0);
    Animated.spring(anim, { toValue: 1, useNativeDriver: Platform.OS !== 'web', friction: 8 }).start();
    const timer = setTimeout(() => {
      Animated.timing(anim, { toValue: 0, duration: 180, useNativeDriver: Platform.OS !== 'web' }).start(() => hide());
    }, 2600);
    return () => clearTimeout(timer);
  }, [t, anim, hide]);

  if (!t) return null;
  // A toast is a slip of ink-coloured paper with a hard plate; a warning is the red one.
  const warn = t.tone === 'warn';
  const bg = warn ? colors.red : colors.primary;
  const fg = warn ? colors.white : colors.onPrimary;
  return (
    <Animated.View
      pointerEvents="box-none"
      style={[
        styles.toastWrap,
        { top: insets.top + 10, opacity: anim, transform: [{ translateY: anim.interpolate({ inputRange: [0, 1], outputRange: [-30, 0] })
}] },
      ]}>
      <Plate corner={radius.card} color={colors.accent} style={{ width: '100%', maxWidth: 480 }}>
        <Pressable onPress={hide} style={[styles.toast, { backgroundColor: bg }]} accessibilityLiveRegion="polite" accessibilityRole="al
ert">
          <Ionicons name={t.icon} size={20} color={warn ? fg : colors.accent} />
          <T variant="small" weight="bold" color={fg} style={{ flex: 1 }}>
            {t.text}
          </T>
        </Pressable>
      </Plate>
    </Animated.View>
  );
}

export interface DialogAction {
  label: string;
  style?: 'default' | 'cancel' | 'destructive';
  onPress?: () => void;
}

interface DialogState {
  dialog: { title: string; message?: string; actions: DialogAction[]; icon?: IconName } | null;
  open: (d: DialogState['dialog']) => void;
  close: () => void;
}

export const useDialog = create<DialogState>((set) => ({
  dialog: null,
  open: (d) => set({ dialog: d }),
  close: () => set({ dialog: null }),
}));

export function showDialog(title: string, message?: string, actions: DialogAction[] = [{ label: 'OK' }], icon?: IconName) {
  useDialog.getState().open({ title, message, actions, icon });
}

export function confirmAction(
  title: string,
  message: string,
  onConfirm: () => void,
  opts: { confirmLabel?: string; destructive?: boolean } = {},
) {
  showDialog(
    title,
    message,
    [
      { label: 'Cancel', style: 'cancel' },
      { label: opts.confirmLabel ?? 'Confirm', style: opts.destructive ? 'destructive' : 'default', onPress: onConfirm },
    ],
    opts.destructive ? 'warning-outline' : 'help-circle-outline',
  );
}

export function DialogHost() {
  const d = useDialog((s) => s.dialog);
  const close = useDialog((s) => s.close);
  if (!d) return null;
  return (
```

```
    <Modal transparent visible animationType="fade" onRequestClose={close} statusBarTranslucent>
      <View style={styles.dialogBackdrop}>
        <Plate corner={radius.modal} style={{ width: '100%', maxWidth: 372 }}>
        <View style={styles.dialog} accessibilityViewIsModal accessibilityRole="alert">
          {d.icon ? (
            <View style={styles.dialogIcon}>
              <Ionicons name={d.icon} size={26} color={colors.ink} />
            </View>
          ) : null}
          <T variant="headline" center>
            {d.title}
          </T>
          {d.message ? (
            <T variant="bodySm" color={colors.textSecondary} center style={{ marginTop: 6 }}>
              {d.message}
            </T>
          ) : null}
          <View style={{ gap: 8, marginTop: 18, alignSelf: 'stretch' }}>
            {d.actions.map((a) => (
              <Button
                key={a.label}
                label={a.label}
                size="md"
                variant={a.style === 'cancel' ? 'secondary' : a.style === 'destructive' ? 'danger' : 'primary'}
                onPress={() => {
                  close();
                  a.onPress?.();
                }}
              />
            ))}
          </View>
        </View>
        </Plate>
      </View>
    </Modal>
  );
}

const styles = StyleSheet.create({
  toastWrap: { position: 'absolute', left: 16, right: 16, alignItems: 'center', zIndex: 1000 },
  toast: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
    paddingHorizontal: 16,
    paddingVertical: 14,
    minHeight: 52,
    borderRadius: radius.card,
    borderWidth: line,
    borderColor: colors.ink,
    width: '100%',
  },
  dialogBackdrop: { flex: 1, backgroundColor: colors.overlay, alignItems: 'center', justifyContent: 'center', padding: 28 },
  dialog: {
    width: '100%',
    backgroundColor: colors.raised,
    borderRadius: radius.modal,
    borderWidth: line,
    borderColor: colors.ink,
    padding: 22,
    alignItems: 'center',
  },
  dialogIcon: {
    width: 54,
    height: 54,
    borderRadius: 27,
    borderWidth: line,
    borderStyle: 'dashed',
    borderColor: colors.ink,
    backgroundColor: colors.accentSoft,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: 10,
  },
});
```

src/components/ui/motion.tsx

```
/**
 * Small motion kit built on React Native's Animated API.
 * Every animation respects the "Animations" switch in Make it yours / Settings
 * (people who get motion-sick, or just want a calmer app, can turn it off).
 */
import { useEffect, useState, type ReactNode } from 'react';
import { Animated, Easing, Platform, type StyleProp, type ViewStyle } from 'react-native';
```

```
import { effectiveMotion, useOsReduceMotion } from '@/hooks/useReduceMotion';
import { useAppStore } from '@/store/useAppStore';

export const nativeDriver = Platform.OS !== 'web';

/** True when animations should run: the "Animations" switch is on and the phone's Reduce Motion setting is off. */
export function useMotion() {
  const wanted = useAppStore((s) => s.personal.motion);
  const osReduce = useOsReduceMotion((s) => s.on);
  return effectiveMotion(wanted, osReduce);
}

/**
 * A scroll position to build parallax from: pass `onScroll` to an `Animated.ScrollView` and use `y` in transforms.
 * `y` is only ever driven by the scroll view, so it can safely feed native-driver transforms.
 */
export function useScrollY() {
  const [state] = useState(() => {
    const y = new Animated.Value(0);
    return { y, onScroll: Animated.event([{ nativeEvent: { contentOffset: { y } } }], { useNativeDriver: nativeDriver }) };
  });
  return state;
}

/** How much celebration the person wants: chill (none), lively (a little confetti) or max. */
export function useEffects() {
  return useAppStore((s) => s.personal.effects);
}

/** Fades + slides its children in once, after `delay` ms. Use for staggered entrances. */
export function FadeIn({
  children,
  delay = 0,
  from = 'up',
  distance = 18,
  duration = 480,
  style,
}: {
  children: ReactNode;
  delay?: number;
  from?: 'up' | 'down' | 'left' | 'right' | 'scale';
  distance?: number;
  duration?: number;
  style?: StyleProp<ViewStyle>;
}) {
  const motion = useMotion();
  const [v] = useState(() => new Animated.Value(motion ? 0 : 1));
  useEffect(() => {
    if (!motion) {
      v.setValue(1); // motion was switched off (or the phone's setting arrived) after the first frame: show the content now
      return;
    }
    const anim = Animated.timing(v, {
      toValue: 1,
      duration,
      delay,
      easing: Easing.out(Easing.cubic),
      useNativeDriver: nativeDriver,
    });
    anim.start();
    return () => anim.stop();
  }, [v, delay, duration, motion]);

  const offset = v.interpolate({
    inputRange: [0, 1],
    outputRange: [from === 'down' || from === 'right' ? -distance : distance, 0],
  });
  const transform =
    from === 'scale'
      ? [
          {
            scale: v.interpolate({
              inputRange: [0, 1],
              outputRange: [0.86, 1],
            }),
          },
        ]
      : from === 'left' || from === 'right'
        ? [{ translateX: offset }]
        : [{ translateY: offset }];
  return <Animated.View style={[style, { opacity: v, transform }]}>{children}</Animated.View>;
}

/** A 0 → 1 value that loops forever (or sits at `rest` when motion is off). */
export function useLoop(duration: number, { delay = 0, rest = 0.5, pingPong = false }: { delay?: number; rest?: number; pingPong?: boole
an } = {}) {
```

```
  const motion = useMotion();
  const [v] = useState(() => new Animated.Value(motion ? 0 : rest));
  useEffect(() => {
    if (!motion) {
      v.setValue(rest);
      return;
    }
    v.setValue(0);
    const one = Animated.timing(v, {
      toValue: 1,
      duration,
      easing: pingPong ? Easing.inOut(Easing.sin) : Easing.linear,
      useNativeDriver: nativeDriver,
    });
    const seq = pingPong
      ? Animated.sequence([
          one,
          Animated.timing(v, {
            toValue: 0,
            duration,
            easing: Easing.inOut(Easing.sin),
            useNativeDriver: nativeDriver,
          }),
        ])
      : Animated.sequence([
          one,
          Animated.timing(v, {
            toValue: 0,
            duration: 0,
            useNativeDriver: nativeDriver,
          }),
        ]);
    const loop = Animated.loop(seq);
    const timer = setTimeout(() => loop.start(), delay);
    return () => {
      clearTimeout(timer);
      loop.stop();
    };
  }, [v, duration, delay, rest, pingPong, motion]);
  return v;
}

/** Gently bobs its children up and down. */
export function Float({ children, amount = 6, duration = 2200, delay = 0, style }: { children: ReactNode; amount?: number; duration?: nu
mber; delay?: number; style?: StyleProp<ViewStyle> }) {
  const v = useLoop(duration, { delay, pingPong: true });
  const translateY = v.interpolate({
    inputRange: [0, 1],
    outputRange: [-amount / 2, amount / 2],
  });
  return <Animated.View style={[style, { transform: [{ translateY }] }]}>{children}</Animated.View>;
}

/** Soft pulsing glow (scale + opacity) — great behind icons and "live" dots. */
export function Pulse({ children, scale = 1.12, duration = 1400, style }: { children?: ReactNode; scale?: number; duration?: number; sty
le?: StyleProp<ViewStyle> }) {
  const v = useLoop(duration, { pingPong: true, rest: 0 });
  return (
    <Animated.View
      style={[
        style,
        {
          opacity: v.interpolate({
            inputRange: [0, 1],
            outputRange: [1, 0.75],
          }),
          transform: [
            {
              scale: v.interpolate({
                inputRange: [0, 1],
                outputRange: [1, scale],
              }),
            },
          ],
        },
      ]}>
      {children}
    </Animated.View>
  );
}
```

src/hooks/__tests__/useDraft.test.tsx

```
import { act, create } from 'react-test-renderer';

import { resetDrafts, useDraft } from '../useDraft';
```

```
(globalThis as { IS_REACT_ACT_ENVIRONMENT?: boolean }).IS_REACT_ACT_ENVIRONMENT = true;

function setup(key: string) {
  const latest: { value: string; set: (v: string) => void; clear: () => void } = { value: '', set: () => {}, clear: () => {} };
  function Probe() {
    const [value, set, clear] = useDraft(key, '');
    latest.value = value;
    latest.set = set as (v: string) => void;
    latest.clear = clear;
    return null;
  }
  let tree!: ReturnType<typeof create>;
  act(() => {
    tree = create(<Probe />);
  });
  return { latest, tree };
}

describe('useDraft', () => {
  beforeEach(resetDrafts);

  it('gives back what was typed when the form is opened again', () => {
    const first = setup('add-activity');
    act(() => first.latest.set('Sunset boat trip'));
    act(() => first.tree.unmount());
    expect(setup('add-activity').latest.value).toBe('Sunset boat trip');
  });

  it('starts empty again once the form is submitted', () => {
    const first = setup('add-expense');
    act(() => first.latest.set('Pizza'));
    act(() => first.latest.clear());
    act(() => first.tree.unmount());
    expect(setup('add-expense').latest.value).toBe('');
  });

  it('keeps different forms apart', () => {
    const a = setup('a');
    act(() => a.latest.set('one'));
    expect(setup('b').latest.value).toBe('');
  });
});
```

src/hooks/__tests__/usePlaceSearch.test.tsx

```
import { act, create } from 'react-test-renderer';

import { searchOnline, type PlaceHit } from '@/services/placeSearch';

import { usePlaceSearch, type PlaceSearchState } from '../usePlaceSearch';

jest.mock('@/services/placeSearch', () => ({ ...jest.requireActual('@/services/placeSearch'), searchOnline: jest.fn() }));
const ask = searchOnline as jest.MockedFunction<typeof searchOnline>;

(globalThis as { IS_REACT_ACT_ENVIRONMENT?: boolean }).IS_REACT_ACT_ENVIRONMENT = true;

const online = (name: string, lat = 40.6288, lng = 14.3809): PlaceHit => ({ id: `online:${name}`, name, detail: 'Sorrento', lat, lng, ty
pe: 'culture', source: 'online' });

/** Renders the hook and keeps the latest answer in `latest.current`. */
function setup(text: string, enabled = true) {
  const latest: { current: PlaceSearchState } = { current: { hits: [], loading: false, offline: false } };
  function Probe(props: { text: string; enabled: boolean }) {
    latest.current = usePlaceSearch(props.text, { destinationIds: ['positano'], around: { lat: 40.63, lng: 14.49 }, enabled: props.enabl
ed });
    return null;
  }
  let tree!: ReturnType<typeof create>;
  act(() => {
    tree = create(<Probe text={text} enabled={enabled} />);
  });
  const set = (t: string, on = true) => act(() => tree.update(<Probe text={t} enabled={on} />));
  return { latest, set };
}

describe('usePlaceSearch', () => {
  beforeEach(() => {
    jest.useFakeTimers();
    ask.mockReset();
  });
  afterEach(() => jest.useRealTimers());

  it('shows saved places at once and the online ones when they arrive', async () => {
    ask.mockResolvedValue([online('Museo Correale')]);
    const { latest, set } = setup('');
```

```
    await set('da vincenzo');
    expect(latest.current.hits.map((h) => h.name)).toContain('Da Vincenzo'); // no waiting for the network
    expect(latest.current.loading).toBe(true);
    await act(async () => {
      jest.advanceTimersByTime(400);
    });
    expect(latest.current.loading).toBe(false);
    expect(ask).toHaveBeenCalledTimes(1);
  });

  it('asks once after typing stops, not once per letter', async () => {
    ask.mockResolvedValue([]);
    const { set } = setup('');
    for (const t of ['mu', 'mus', 'muse', 'museu', 'museum']) {
      await set(t);
      await act(async () => {
        jest.advanceTimersByTime(100);
      });
    }
    expect(ask).not.toHaveBeenCalled(); // still typing
    await act(async () => {
      jest.advanceTimersByTime(400);
    });
    expect(ask).toHaveBeenCalledTimes(1);
    expect(ask.mock.calls[0][0]).toBe('museum');
  });

  it('merges the online answer after the saved places, without duplicates', async () => {
    ask.mockResolvedValue([online('Museo Correale'), online('Da Vincenzo 1958', 40.6277, 14.4849)]); // the second one is the saved Da V
incenzo, 200 m away
    const { latest, set } = setup('');
    await set('vincenzo');
    await act(async () => {
      jest.advanceTimersByTime(400);
    });
    const names = latest.current.hits.map((h) => h.name);
    expect(names[0]).toBe('Da Vincenzo');
    expect(names.filter((n) => n === 'Da Vincenzo')).toHaveLength(1); // the online copy is the same place
    expect(names).toContain('Museo Correale');
  });

  it('says so when the online search cannot be reached, and keeps the saved places', async () => {
    ask.mockRejectedValue(new TypeError('Network request failed'));
    const { latest, set } = setup('');
    await set('santa maria');
    await act(async () => {
      jest.advanceTimersByTime(400);
    });
    expect(latest.current.offline).toBe(true);
    expect(latest.current.loading).toBe(false);
    expect(latest.current.hits[0].name).toBe('Santa Maria Assunta');
  });

  it('does nothing for one letter or while switched off', async () => {
    const { latest, set } = setup('');
    await set('d');
    await act(async () => {
      jest.advanceTimersByTime(1000);
    });
    expect(ask).not.toHaveBeenCalled();
    expect(latest.current).toEqual({ hits: [], loading: false, offline: false });
    await set('vincenzo', false);
    await act(async () => {
      jest.advanceTimersByTime(1000);
    });
    expect(ask).not.toHaveBeenCalled();
    expect(latest.current.hits).toEqual([]);
  });

  it('forgets an answer that comes back after the words changed', async () => {
    let finish: (hits: PlaceHit[]) => void = () => {};
    ask.mockImplementationOnce((_q, _c, o) => new Promise((resolve) => { finish = resolve; o?.signal?.addEventListener('abort', () => re
solve([])); }));
    ask.mockResolvedValue([online('Museum of Something Else')]);
    const { latest, set } = setup('');
    await set('first words');
    await act(async () => {
      jest.advanceTimersByTime(400);
    });
    await set('second words');
    await act(async () => {
      finish([online('Stale answer')]);
      jest.advanceTimersByTime(400);
    });
    expect(latest.current.hits.map((h) => h.name)).not.toContain('Stale answer');
```

```
  });
});
```

src/hooks/__tests__/useReduceMotion.test.ts

```
import { effectiveMotion } from '../useReduceMotion';

describe('animations', () => {
  it('run only when wanted and the phone does not ask for reduced motion', () => {
    expect(effectiveMotion(true, false)).toBe(true);
    expect(effectiveMotion(true, true)).toBe(false); // the phone's setting wins
    expect(effectiveMotion(false, false)).toBe(false);
    expect(effectiveMotion(false, true)).toBe(false);
  });
});
```

src/hooks/useDraft.ts

```
import { useState } from 'react';

/** Half-finished forms, kept for as long as the app is open, so closing a sheet by accident or switching tab loses nothing. */
const drafts = new Map<string, unknown>();

/**
 * Like `useState`, but the value is remembered under `key` after the form closes.
 * Call `clear()` when the form is submitted. Not saved to disk on purpose: a draft from last week is clutter.
 */
export function useDraft<T>(key: string, initial: T): [T, (next: T | ((cur: T) => T)) => void, () => void] {
  const [value, setValue] = useState<T>(() => (drafts.has(key) ? (drafts.get(key) as T) : initial));
  const set = (next: T | ((cur: T) => T)) =>
    setValue((cur) => {
      const v = typeof next === 'function' ? (next as (c: T) => T)(cur) : next;
      drafts.set(key, v);
      return v;
    });
  const clear = () => {
    drafts.delete(key);
    setValue(initial);
  };
  return [value, set, clear];
}

/** For tests. */
export const resetDrafts = () => drafts.clear();
```

src/hooks/usePlaceSearch.ts

```
import { useEffect, useMemo, useState } from 'react';

import { mergeHits, normalizeQuery, searchOnline, searchSaved, type PlaceHit } from '@/services/placeSearch';
import { getLocation, type LocationResult } from '@/services/location';
import type { LatLng } from '@/utils/geo';

/** Where the phone is (or `fallback` while that is unknown, denied or unavailable). Asks once the screen needs it. */
export function useMyLocation(fallback: LatLng, enabled: boolean): LocationResult {
  const [fix, setFix] = useState<LocationResult | null>(null);
  const { lat, lng } = fallback;
  useEffect(() => {
    if (!enabled) return;
    let live = true;
    getLocation({ lat, lng }).then((r) => live && setFix(r));
    return () => {
      live = false;
    };
  }, [enabled, lat, lng]);
  return fix ?? { lat, lng, source: 'fallback' };
}

export interface PlaceSearchState {
  hits: PlaceHit[];
  /** Waiting for the online answer (saved places are already in `hits`). */
  loading: boolean;
  /** The online search could not be reached, so only saved places are shown. */
  offline: boolean;
}

const WAIT_MS = 350;

/**
 * Matches for what is typed: the trip's saved places at once, then OpenStreetMap places a moment after typing stops.
 * `around` is where to look first (the trip, or the phone when it is at the trip).
 */
export function usePlaceSearch(text: string, o: { destinationIds: string[]; around: LatLng | null; enabled: boolean }): PlaceSearchState
{
  const q = normalizeQuery(text);
  const ids = o.destinationIds.join(',');
```

```
  const lat = o.around?.lat;
  const lng = o.around?.lng;
  const saved = useMemo(() => (q.length >= 2 ? searchSaved(q, ids.split(',').filter(Boolean)) : []), [q, ids]);
  const [done, setDone] = useState<{ q: string; hits: PlaceHit[]; offline: boolean } | null>(null);
  const active = o.enabled && q.length >= 2;

  useEffect(() => {
    if (!active) return;
    const stop = new AbortController();
    const timer = setTimeout(() => {
      searchOnline(q, lat === undefined || lng === undefined ? null : { lat, lng }, { signal: stop.signal }).then(
        (hits) => !stop.signal.aborted && setDone({ q, hits, offline: false }),
        () => !stop.signal.aborted && setDone({ q, hits: [], offline: true }),
      );
    }, WAIT_MS);
    return () => {
      clearTimeout(timer);
      stop.abort();
    };
  }, [active, q, lat, lng]);

  const answer = active && done?.q === q ? done : null;
  return {
    hits: active ? mergeHits(saved, answer?.hits ?? []) : [],
    loading: active && !answer,
    offline: !!answer?.offline,
  };
}
```

src/hooks/useReduceMotion.ts

```
import { useEffect } from 'react';
import { AccessibilityInfo } from 'react-native';
import { create } from 'zustand';

/** Whether the phone's own "Reduce Motion" accessibility setting is on. */
export const useOsReduceMotion = create<{ on: boolean }>(() => ({ on: false }));

/** Animations run only when the person wants them AND the phone is not asking apps to calm down. */
export const effectiveMotion = (wanted: boolean, osReduce: boolean) => wanted && !osReduce;

/** Call once near the root: reads the setting and follows changes while the app is open. */
export function useWatchOsReduceMotion() {
  useEffect(() => {
    let alive = true;
    AccessibilityInfo.isReduceMotionEnabled()
      .then((on) => alive && useOsReduceMotion.setState({ on }))
      .catch(() => {});
    const subscription = AccessibilityInfo.addEventListener('reduceMotionChanged', (on) => useOsReduceMotion.setState({ on }));
    return () => {
      alive = false;
      subscription.remove();
    };
  }, []);
}
```

src/hooks/useTrip.ts

```
import { useEffect, useState } from 'react';

import { destinationById } from '@/data/destinations';
import { fetchRates } from '@/services/currency';
import { fetchWeather } from '@/services/weather';
import { useAppStore } from '@/store/useAppStore';
import type { Trip, WeatherSnapshot } from '@/store/types';

/** Cached weather for a destination; refreshes in the background when online. */
export function useWeather(destinationId: string | undefined): WeatherSnapshot | undefined {
  const snap = useAppStore((s) => (destinationId ? s.weather[destinationId] : undefined));
  useEffect(() => {
    if (destinationId) fetchWeather(destinationId).catch(() => {});
  }, [destinationId]);
  return snap;
}

export function useTripDestination(trip: Trip | undefined) {
  return trip ? destinationById(trip.destinationId) : undefined;
}

/** Re-renders every `ms` so countdowns and "time ago" labels stay live. */
export function useNow(ms = 30000) {
  const [now, setNow] = useState(() => Date.now());
  useEffect(() => {
    const id = setInterval(() => setNow(Date.now()), ms);
    return () => clearInterval(id);
  }, [ms]);
```

```
  return now;
}

export function useRatesLoader() {
  useEffect(() => {
    fetchRates().catch(() => {});
  }, []);
}
```

src/i18n/index.ts

```
import { useCallback } from 'react';

import { useAppStore } from '@/store/useAppStore';

import { dictionaries, english, type LangCode, type StringKey } from './strings';

export { LANGUAGES } from './strings';
export type { LangCode, StringKey } from './strings';

export function translate(lang: LangCode, key: StringKey | string, vars?: Record<string, string | number>): string {
  const dict = dictionaries[lang] ?? {};
  let s: string = (dict as Record<string, string>)[key] ?? (english as Record<string, string>)[key] ?? key;
  if (vars) for (const [k, v] of Object.entries(vars)) s = s.replace(new RegExp(`\\{${k}\\}`, 'g'), String(v));
  return s;
}

/** `const t = useT(); t('nav.explore')` — re-renders when the language changes. */
export function useT() {
  const lang = useAppStore((s) => s.settings.language);
  return useCallback((key: StringKey | string, vars?: Record<string, string | number>) => translate(lang, key, vars), [lang]);
}
```

src/i18n/strings.ts

```
/* UI strings for the six supported languages. English is the source of truth;
   missing keys in other languages fall back to English. */

export const LANGUAGES = [
  { code: 'en', label: 'English', native: 'English', flag: '🇺🇸' },
  { code: 'es', label: 'Spanish', native: 'Español', flag: '🇪🇸' },
  { code: 'fr', label: 'French', native: 'Français', flag: '🇫🇷' },
  { code: 'ja', label: 'Japanese', native: '日本語', flag: '🇯🇵' },
  { code: 'hi', label: 'Hindi', native: 'हिन्दी', flag: '🇮🇳' },
  { code: 'zh', label: 'Chinese', native: '中文', flag: '🇨🇳' },
] as const;

export type LangCode = (typeof LANGUAGES)[number]['code'];

const en = {
  'nav.explore': 'Home',
  'nav.itinerary': 'Plan',
  'nav.budget': 'Budget',
  'nav.chat': 'Chat',
  'nav.profile': 'More',

  'greet.morning': 'Good morning',
  'greet.afternoon': 'Good afternoon',
  'greet.evening': 'Good evening',
  'greet.night': 'Hello night owl',

  'explore.title': 'Find Your Perfect\nDestination',
  'explore.subtitle': 'Plan, explore, and travel together.',
  'explore.search': 'Search a city or destination…',
  'explore.popular': 'Popular Places',
  'explore.featured': 'Featured Destination',
  'explore.seeAll': 'See all',
  'explore.matches': '{n} matches',
  'explore.nearby': 'Nearby within radius',
  'explore.activeTrip': 'Active trip',
  'explore.upcomingTrip': 'Upcoming trip',
  'explore.pastTrip': 'Completed trip',
  'explore.left': '{amount} left',
  'explore.members': '{n} members',
  'explore.reminder': 'Reminder',
  'explore.reminderSub': 'We'll ping everyone {n} min before.',
  'explore.nearestAirport': 'Nearest airport',
  'explore.addToTrip': 'Add to Trip',
  'explore.noResults': 'No destinations match “{q}”',
  'explore.tools': 'Trip tools',

  'cat.all': 'All',
  'cat.best': 'Best place',
  'cat.adventure': 'Adventures',
  'cat.food': 'Food',
  'cat.culture': 'Culture',
```

```
'cat.beach': 'Beaches',
'cat.nature': 'Nature',
'cat.nightlife': 'Nightlife',
'cat.sightseeing': 'Sightseeing',
'cat.shopping': 'Shopping',
'cat.relax': 'Relax',

'len.any': 'Any length',
'len.u1': 'Under 1 hour',
'len.1-2': '1–2 hours',
'len.2-4': '2–4 hours',
'len.half': 'Half day',
'len.full': 'Full day',

'itin.members': '{n} members',
'itin.noActivities': 'No activities yet',
'itin.noActivitiesSub': 'Tap the + button to add your\nfirst activity for this day.',
'itin.noMatch': 'No activities match these filters',
'itin.addActivity': 'Add Activity',
'itin.flights': 'Flights & arrivals',
'itin.flightsSub': 'Times shown in {zone}. Tap to switch time zone.',
'itin.addFlight': 'Add my flight',
'itin.assigned': '{n} assigned',
'itin.confirmed': 'Confirmed',
'itin.voting': 'Voting',
'itin.pending': 'Pending',
'itin.perPerson': '/person',

'add.title': 'Add activity',
'add.subtitle': 'Search for a place, then pick a day and time.',
'add.type': 'Activity Type',
'add.time': 'Time',
'add.name': 'Activity Name',
'add.namePh': 'e.g. Fushimi Inari Shrine visit',
'add.location': 'Location',
'add.locationPh': 'e.g. Fushimi, Kyoto',
'add.duration': 'Duration',
'add.cost': 'Cost per person (USD)',
'add.notes': 'Notes (optional)',
'add.notesPh': 'Any notes for the group…',
'add.vote': 'Put it to a group vote',
'add.remind': 'Remind everyone before it starts',
'add.submit': 'Add to the plan',
'add.save': 'Save changes',

'budget.kicker': 'Group spending',
'budget.title': 'Budget',
'budget.spent': 'Spent so far',
'budget.of': 'of {total} · {per} per person',
'budget.used': '{pct}% of the group budget used',
'budget.breakdown': 'Spending breakdown',
'budget.currency': 'Currency',
'budget.expenses': 'Expenses',
'budget.cap': 'Budget cap',
'budget.addExpense': 'Add an expense',
'budget.whatFor': 'What was it for?',
'budget.add': 'Add expense',
'budget.settle': 'Settle up',
'budget.planner': 'Daily spending planner',
'budget.paidBy': 'paid by {name}',

'chat.kicker': '{n} travellers',
'chat.title': 'Trip chat',
'chat.placeholder': 'Message your trip pals…',
'chat.askAi': 'Ask the Trip Assistant…',
'chat.assistant': 'Trip Assistant',
'chat.typing': '{name} is typing…',
'chat.q.budget': 'How’s the budget?',
'chat.q.food': 'Best food nearby?',
'chat.q.weather': 'What’s the weather?',
'chat.q.next': 'What’s next?',

'hub.kicker': 'Trip hub',
'hub.title': 'Your group',
'hub.invite': 'Invite code',
'hub.share': 'Share',
'hub.members': 'Members & roles',
'hub.vote': 'Vote on activities',
'hub.checklist': 'Checklist',
'hub.done': '{a}/{b} done',
'hub.addTask': 'Add a task',
'hub.documents': 'Documents',
'hub.documentsSub': 'Marked items stay readable without wifi or data.',
'hub.offline': 'Offline',
'hub.tools': 'More for this trip',
```

```
  'auth.welcome': 'Welcome back!',
  'auth.welcomeSub': 'Sign in to continue planning your adventures.',
  'auth.email': 'Email Address',
  'auth.password': 'Password',
  'auth.remember': 'Remember me',
  'auth.forgot': 'Forgot password?',
  'auth.signIn': 'Sign In',
  'auth.or': 'or continue with',
  'auth.google': 'Continue with Google',
  'auth.noAccount': 'Don’t have an account?',
  'auth.signUp': 'Sign Up',
  'auth.demo': 'Demo Account',
  'auth.terms': 'By continuing, you agree to our Terms of Service and Privacy Policy',

  'common.cancel': 'Cancel',
  'common.save': 'Save',
  'common.delete': 'Delete',
  'common.done': 'Done',
  'common.next': 'Next',
  'common.getStarted': 'Get Started',
  'common.skip': 'Skip',
  'common.back': 'Back',
  'common.edit': 'Edit',
  'common.close': 'Close',

  'settings.title': 'Settings',
  'settings.language': 'Language',
  'settings.currency': 'Home currency',
  'settings.notifications': 'Activity reminders',
  'settings.lead': 'Remind me before',
  'settings.textSize': 'Text size',
  'settings.signOut': 'Sign out',
};

export type StringKey = keyof typeof en;
type Dict = Partial<Record<StringKey, string>>;

const es: Dict = {
  'nav.explore': 'Inicio', 'nav.itinerary': 'Plan', 'nav.budget': 'Presupuesto', 'nav.chat': 'Chat', 'nav.profile': 'Más',
  'greet.morning': 'Buenos días', 'greet.afternoon': 'Buenas tardes', 'greet.evening': 'Buenas noches', 'greet.night': 'Hola, noctámbul
o',
  'explore.title': 'Encuentra tu\ndestino perfecto', 'explore.subtitle': 'Planea, explora y viaja en grupo.',
  'explore.search': 'Busca una ciudad o destino…', 'explore.popular': 'Lugares populares', 'explore.featured': 'Destino destacado',
  'explore.seeAll': 'Ver todo', 'explore.matches': '{n} resultados', 'explore.nearby': 'Cerca dentro del radio',
  'explore.activeTrip': 'Viaje activo', 'explore.upcomingTrip': 'Próximo viaje', 'explore.pastTrip': 'Viaje completado',
  'explore.left': 'Quedan {amount}', 'explore.members': '{n} miembros', 'explore.reminder': 'Recordatorio',
  'explore.reminderSub': 'Avisaremos a todos {n} min antes.', 'explore.nearestAirport': 'Aeropuerto más cercano',
  'explore.addToTrip': 'Añadir al viaje', 'explore.noResults': 'Ningún destino coincide con “{q}”', 'explore.tools': 'Herramientas',
  'cat.all': 'Todo', 'cat.best': 'Lo mejor', 'cat.adventure': 'Aventuras', 'cat.food': 'Comida', 'cat.culture': 'Cultura',
  'cat.beach': 'Playas', 'cat.nature': 'Naturaleza', 'cat.nightlife': 'Vida nocturna', 'cat.sightseeing': 'Turismo', 'cat.shopping': 'Co
mpras', 'cat.relax': 'Relax',
  'len.any': 'Cualquier duración', 'len.u1': 'Menos de 1 hora', 'len.1-2': '1–2 horas', 'len.2-4': '2–4 horas', 'len.half': 'Medio día',
'len.full': 'Día completo',
  'itin.members': '{n} miembros', 'itin.noActivities': 'Aún no hay actividades', 'itin.noActivitiesSub': 'Toca + para añadir la primera
\nactividad del día.',
  'itin.noMatch': 'Ninguna actividad coincide', 'itin.addActivity': 'Añadir actividad', 'itin.flights': 'Vuelos y llegadas',
  'itin.flightsSub': 'Horas en {zone}. Toca para cambiar la zona horaria.', 'itin.addFlight': 'Añadir mi vuelo', 'itin.assigned': '{n} a
signados',
  'itin.confirmed': 'Confirmado', 'itin.voting': 'Votación', 'itin.pending': 'Pendiente', 'itin.perPerson': '/persona',
  'add.title': 'Añadir actividad', 'add.subtitle': 'Completa los detalles de tu actividad.', 'add.type': 'Tipo de actividad', 'add.tim
e': 'Hora',
  'add.name': 'Nombre de la actividad', 'add.location': 'Ubicación', 'add.duration': 'Duración', 'add.cost': 'Coste por persona (USD)',
  'add.notes': 'Notas (opcional)', 'add.notesPh': 'Notas para el grupo…', 'add.vote': 'Someter a votación', 'add.remind': 'Recordar a to
dos antes',
  'add.submit': 'Añadir al itinerario', 'add.save': 'Guardar cambios',
  'budget.kicker': 'Gastos del grupo', 'budget.title': 'Presupuesto', 'budget.spent': 'Gastado hasta ahora', 'budget.of': 'de {total} ·
{per} por persona',
  'budget.used': '{pct}% del presupuesto usado', 'budget.breakdown': 'Desglose de gastos', 'budget.currency': 'Moneda', 'budget.expense
s': 'Gastos',
  'budget.cap': 'Límite', 'budget.addExpense': 'Añadir un gasto', 'budget.whatFor': '¿Para qué fue?', 'budget.add': 'Añadir gasto',
  'budget.settle': 'Saldar cuentas', 'budget.planner': 'Planificador diario', 'budget.paidBy': 'pagado por {name}',
  'chat.kicker': '{n} viajeros', 'chat.title': 'Chat del viaje', 'chat.placeholder': 'Escribe a tu grupo…', 'chat.askAi': 'Pregunta al a
sistente…',
  'chat.assistant': 'Asistente de viaje', 'chat.typing': '{name} está escribiendo…', 'chat.q.budget': '¿Cómo va el presupuesto?',
  'chat.q.food': '¿Buena comida cerca?', 'chat.q.weather': '¿Qué tiempo hace?', 'chat.q.next': '¿Qué sigue?',
  'hub.kicker': 'Centro del viaje', 'hub.title': 'Tu grupo', 'hub.invite': 'Código de invitación', 'hub.share': 'Compartir',
  'hub.members': 'Miembros y roles', 'hub.vote': 'Vota actividades', 'hub.checklist': 'Lista de tareas', 'hub.done': '{a}/{b} hechas',
  'hub.addTask': 'Añadir tarea', 'hub.documents': 'Documentos', 'hub.documentsSub': 'Disponibles sin wifi ni datos.', 'hub.offline': 'Si
n conexión', 'hub.tools': 'Más para este viaje',
  'auth.welcome': '¡Bienvenido de nuevo!', 'auth.welcomeSub': 'Inicia sesión para seguir planeando.', 'auth.email': 'Correo electrónic
o',
  'auth.password': 'Contraseña', 'auth.remember': 'Recordarme', 'auth.forgot': '¿Olvidaste tu contraseña?', 'auth.signIn': 'Iniciar sesi
ón',
  'auth.or': 'o continúa con', 'auth.google': 'Continuar con Google', 'auth.noAccount': '¿No tienes cuenta?', 'auth.signUp': 'Regístrat
e',
  'auth.demo': 'Cuenta de demostración', 'auth.terms': 'Al continuar, aceptas los Términos y la Política de privacidad',
```

```
  'common.cancel': 'Cancelar', 'common.save': 'Guardar', 'common.delete': 'Eliminar', 'common.done': 'Listo', 'common.next': 'Siguient
e',
  'common.getStarted': 'Empezar', 'common.skip': 'Omitir', 'common.back': 'Atrás', 'common.edit': 'Editar', 'common.close': 'Cerrar',
  'settings.title': 'Ajustes', 'settings.language': 'Idioma', 'settings.currency': 'Moneda local', 'settings.notifications': 'Recordator
ios',
  'settings.lead': 'Avisarme antes', 'settings.textSize': 'Tamaño del texto', 'settings.signOut': 'Cerrar sesión',
};

const fr: Dict = {
  'nav.explore': 'Accueil', 'nav.itinerary': 'Plan', 'nav.budget': 'Budget', 'nav.chat': 'Discussion', 'nav.profile': 'Plus',
  'greet.morning': 'Bonjour', 'greet.afternoon': 'Bon après-midi', 'greet.evening': 'Bonsoir', 'greet.night': 'Bonsoir, oiseau de nuit',
  'explore.title': 'Trouvez votre\ndestination idéale', 'explore.subtitle': 'Planifiez, explorez et voyagez ensemble.',
  'explore.search': 'Rechercher une ville ou destination…', 'explore.popular': 'Lieux populaires', 'explore.featured': 'Destination à la
une',
  'explore.seeAll': 'Tout voir', 'explore.matches': '{n} résultats', 'explore.nearby': 'À proximité dans le rayon',
  'explore.activeTrip': 'Voyage en cours', 'explore.upcomingTrip': 'Voyage à venir', 'explore.pastTrip': 'Voyage terminé',
  'explore.left': '{amount} restants', 'explore.members': '{n} membres', 'explore.reminder': 'Rappel', 'explore.reminderSub': 'On prévie
nt tout le monde {n} min avant.',
  'explore.nearestAirport': 'Aéroport le plus proche', 'explore.addToTrip': 'Ajouter au voyage', 'explore.noResults': 'Aucune destinatio
n pour « {q} »', 'explore.tools': 'Outils du voyage',
  'cat.all': 'Tout', 'cat.best': 'Incontournables', 'cat.adventure': 'Aventures', 'cat.food': 'Gastronomie', 'cat.culture': 'Culture',
  'cat.beach': 'Plages', 'cat.nature': 'Nature', 'cat.nightlife': 'Vie nocturne', 'cat.sightseeing': 'Visites', 'cat.shopping': 'Shoppin
g', 'cat.relax': 'Détente',
  'len.any': 'Toute durée', 'len.u1': 'Moins d’1 heure', 'len.1-2': '1–2 heures', 'len.2-4': '2–4 heures', 'len.half': 'Demi-journée',
'len.full': 'Journée entière',
  'itin.members': '{n} membres', 'itin.noActivities': 'Aucune activité', 'itin.noActivitiesSub': 'Touchez + pour ajouter la première\nac
tivité de la journée.',
  'itin.noMatch': 'Aucune activité ne correspond', 'itin.addActivity': 'Ajouter', 'itin.flights': 'Vols et arrivées',
  'itin.flightsSub': 'Heures affichées en {zone}. Touchez pour changer.', 'itin.addFlight': 'Ajouter mon vol', 'itin.assigned': '{n} ass
ignés',
  'itin.confirmed': 'Confirmé', 'itin.voting': 'Vote', 'itin.pending': 'En attente', 'itin.perPerson': '/pers.',
  'add.title': 'Ajouter une activité', 'add.subtitle': 'Renseignez les détails de l’activité.', 'add.type': 'Type d’activité', 'add.tim
e': 'Heure',
  'add.name': 'Nom de l’activité', 'add.location': 'Lieu', 'add.duration': 'Durée', 'add.cost': 'Coût par personne (USD)', 'add.notes':
'Notes (facultatif)',
  'add.notesPh': 'Des notes pour le groupe…', 'add.vote': 'Soumettre au vote', 'add.remind': 'Rappeler à tous avant le début', 'add.subm
it': 'Ajouter à l’itinéraire', 'add.save': 'Enregistrer',
  'budget.kicker': 'Dépenses du groupe', 'budget.title': 'Budget', 'budget.spent': 'Dépensé', 'budget.of': 'sur {total} · {per} par pers
onne',
  'budget.used': '{pct} % du budget utilisé', 'budget.breakdown': 'Répartition', 'budget.currency': 'Devise', 'budget.expenses': 'Dépens
es', 'budget.cap': 'Plafond',
  'budget.addExpense': 'Ajouter une dépense', 'budget.whatFor': 'Pour quoi ?', 'budget.add': 'Ajouter', 'budget.settle': 'Régler les com
ptes',
  'budget.planner': 'Planificateur quotidien', 'budget.paidBy': 'payé par {name}',
  'chat.kicker': '{n} voyageurs', 'chat.title': 'Discussion', 'chat.placeholder': 'Écrivez au groupe…', 'chat.askAi': 'Demandez à l’assi
stant…',
  'chat.assistant': 'Assistant voyage', 'chat.typing': '{name} écrit…', 'chat.q.budget': 'Où en est le budget ?', 'chat.q.food': 'Bonne
cuisine près d’ici ?',
  'chat.q.weather': 'Quel temps fait-il ?', 'chat.q.next': 'Et ensuite ?',
  'hub.kicker': 'Espace voyage', 'hub.title': 'Votre groupe', 'hub.invite': 'Code d’invitation', 'hub.share': 'Partager', 'hub.members':
'Membres et rôles',
  'hub.vote': 'Votez pour les activités', 'hub.checklist': 'Liste de tâches', 'hub.done': '{a}/{b} faites', 'hub.addTask': 'Ajouter une
tâche',
  'hub.documents': 'Documents', 'hub.documentsSub': 'Lisibles sans wifi ni données.', 'hub.offline': 'Hors ligne', 'hub.tools': 'Plus po
ur ce voyage',
  'auth.welcome': 'Bon retour !', 'auth.welcomeSub': 'Connectez-vous pour continuer à planifier.', 'auth.email': 'Adresse e-mail', 'aut
h.password': 'Mot de passe',
  'auth.remember': 'Se souvenir de moi', 'auth.forgot': 'Mot de passe oublié ?', 'auth.signIn': 'Se connecter', 'auth.or': 'ou continuer
avec',
  'auth.google': 'Continuer avec Google', 'auth.noAccount': 'Pas de compte ?', 'auth.signUp': 'S’inscrire', 'auth.demo': 'Compte de dém
o',
  'auth.terms': 'En continuant, vous acceptez nos Conditions et notre Politique de confidentialité',
  'common.cancel': 'Annuler', 'common.save': 'Enregistrer', 'common.delete': 'Supprimer', 'common.done': 'Terminé', 'common.next': 'Suiv
ant',
  'common.getStarted': 'Commencer', 'common.skip': 'Passer', 'common.back': 'Retour', 'common.edit': 'Modifier', 'common.close': 'Ferme
r',
  'settings.title': 'Réglages', 'settings.language': 'Langue', 'settings.currency': 'Devise principale', 'settings.notifications': 'Rapp
els d’activité',
  'settings.lead': 'Me rappeler avant', 'settings.textSize': 'Taille du texte', 'settings.signOut': 'Se déconnecter',
};

const ja: Dict = {
  'nav.explore': 'ホーム', 'nav.itinerary': '計画', 'nav.budget': '予算', 'nav.chat': 'チャット', 'nav.profile': 'その他',
  'greet.morning': 'おはようございます', 'greet.afternoon': 'こんにちは', 'greet.evening': 'こんばんは', 'greet.night': 'こんばんは',
  'explore.title': 'ぴったりの\n旅先を見つけよう', 'explore.subtitle': 'みんなで計画して、探して、旅しよう。', 'explore.search': '都市や旅先を検索…',
  'explore.popular': '人気の場所', 'explore.featured': '注目の旅先', 'explore.seeAll': 'すべて見る', 'explore.matches': '{n}件',
  'explore.nearby': '半径内のスポット', 'explore.activeTrip': '旅行中', 'explore.upcomingTrip': '次の旅行', 'explore.pastTrip': '完了した旅行',
  'explore.left': '残り{amount}', 'explore.members': '{n}人', 'explore.reminder': 'リマインダー', 'explore.reminderSub': '{n}分前に全員に通知
します。',
  'explore.nearestAirport': '最寄りの空港', 'explore.addToTrip': '旅行に追加', 'explore.noResults': '「{q}」に一致する旅先はありません', 'explor
e.tools': '旅のツール',
  'cat.all': 'すべて', 'cat.best': 'おすすめ', 'cat.adventure': 'アドベンチャー', 'cat.food': 'グルメ', 'cat.culture': '文化', 'cat.beach': 'ビ
ーチ',
  'cat.nature': '自然', 'cat.nightlife': 'ナイトライフ', 'cat.sightseeing': '観光', 'cat.shopping': 'ショッピング', 'cat.relax': 'リラックス',
  'len.any': 'すべての長さ', 'len.u1': '1時間未満', 'len.1-2': '1〜2時間', 'len.2-4': '2〜4時間', 'len.half': '半日', 'len.full': '1日',
  'itin.members': '{n}人', 'itin.noActivities': 'まだ予定はありません', 'itin.noActivitiesSub': '+ボタンでこの日の\n最初の予定を追加しましょう。',
```

```
  'itin.noMatch': '条件に合う予定はありません', 'itin.addActivity': '予定を追加', 'itin.flights': 'フライトと到着', 'itin.flightsSub': '{zone}で表示中。タップで切替。',
  'itin.addFlight': 'フライトを追加', 'itin.assigned': '{n}人担当', 'itin.confirmed': '確定', 'itin.voting': '投票中', 'itin.pending': '保留', 'itin.perPerson': '/人',
  'add.title': '予定を追加', 'add.subtitle': '新しい予定の詳細を入力してください。', 'add.type': '種類', 'add.time': '時間', 'add.name': '予定の名前',
  'add.location': '場所', 'add.duration': '所要時間', 'add.cost': '1人あたりの費用 (USD)', 'add.notes': 'メモ（任意）', 'add.notesPh': 'グループへのメモ…',
  'add.vote': 'グループ投票にする', 'add.remind': '開始前に全員に通知', 'add.submit': '旅程に追加', 'add.save': '変更を保存',
  'budget.kicker': 'グループの支出', 'budget.title': '予算', 'budget.spent': 'これまでの支出', 'budget.of': '{total}中 · 1人あたり{per}',
  'budget.used': 'グループ予算の{pct}%を使用', 'budget.breakdown': '支出の内訳', 'budget.currency': '通貨', 'budget.expenses': '支出', 'budget.cap': '予算上限',
  'budget.addExpense': '支出を追加', 'budget.whatFor': '何の支出？', 'budget.add': '支出を追加', 'budget.settle': '精算', 'budget.planner': '1日の予算プランナー', 'budget.paidBy': '{name}が支払い',
  'chat.kicker': '旅行者{n}人', 'chat.title': '旅のチャット', 'chat.placeholder': '旅の仲間にメッセージ…', 'chat.askAi': 'アシスタントに質問…',
  'chat.assistant': '旅のアシスタント', 'chat.typing': '{name}が入力中…', 'chat.q.budget': '予算はどう？', 'chat.q.food': '近くのおすすめグルメは？',
  'chat.q.weather': '天気は？', 'chat.q.next': '次の予定は？',
  'hub.kicker': '旅のハブ', 'hub.title': 'あなたのグループ', 'hub.invite': '招待コード', 'hub.share': '共有', 'hub.members': 'メンバーと役割',
  'hub.vote': 'アクティビティに投票', 'hub.checklist': 'チェックリスト', 'hub.done': '{a}/{b} 完了', 'hub.addTask': 'タスクを追加',
  'hub.documents': '書類', 'hub.documentsSub': 'Wi-Fiや通信なしでも閲覧できます。', 'hub.offline': 'オフライン', 'hub.tools': 'この旅のツール',
  'auth.welcome': 'おかえりなさい！', 'auth.welcomeSub': 'ログインして旅の計画を続けましょう。', 'auth.email': 'メールアドレス', 'auth.password': 'パスワード',
  'auth.remember': 'ログイン状態を保持', 'auth.forgot': 'パスワードをお忘れですか？', 'auth.signIn': 'ログイン', 'auth.or': 'または',
  'auth.google': 'Googleで続行', 'auth.noAccount': 'アカウントをお持ちでないですか？', 'auth.signUp': '新規登録', 'auth.demo': 'デモアカウント',
  'auth.terms': '続行すると、利用規約とプライバシーポリシーに同意したことになります',
  'common.cancel': 'キャンセル', 'common.save': '保存', 'common.delete': '削除', 'common.done': '完了', 'common.next': '次へ', 'common.getStarted': 'はじめる',
  'common.skip': 'スキップ', 'common.back': '戻る', 'common.edit': '編集', 'common.close': '閉じる',
  'settings.title': '設定', 'settings.language': '言語', 'settings.currency': '自国通貨', 'settings.notifications': '予定のリマインダー',
  'settings.lead': '通知のタイミング', 'settings.textSize': '文字サイズ', 'settings.signOut': 'ログアウト',
};

const hi: Dict = {
  'nav.explore': 'होम', 'nav.itinerary': 'योजना', 'nav.budget': 'बजट', 'nav.chat': 'चैट', 'nav.profile': 'और',
  'greet.morning': 'सुप्रभात', 'greet.afternoon': 'नमस्ते', 'greet.evening': 'शुभ संध्या', 'greet.night': 'नमस्ते',
  'explore.title': 'अपनी परफ़ेक्ट\nमंज़िल खोजें', 'explore.subtitle': 'साथ मिलकर योजना बनाएँ, घूमें और यात्रा करें।', 'explore.search': 'शहर या जगह खोजें…',
  'explore.popular': 'लोकप्रिय जगहें', 'explore.featured': 'विशेष मंज़िल', 'explore.seeAll': 'सभी देखें', 'explore.matches': '{n} परिणाम',
  'explore.nearby': 'दायरे में आस-पास', 'explore.activeTrip': 'चालू यात्रा', 'explore.upcomingTrip': 'आगामी यात्रा', 'explore.pastTrip': 'पूरी हुई यात्रा',
  'explore.left': '{amount} बाकी', 'explore.members': '{n} सदस्य', 'explore.reminder': 'रिमाइंडर', 'explore.reminderSub': 'हम सबको {n} मिनट पहले याद दिलाएँगे।',
  'explore.nearestAirport': 'नज़दीकी हवाई अड्डा', 'explore.addToTrip': 'यात्रा में जोड़ें', 'explore.noResults': '“{q}” से कोई मंज़िल नहीं मिली', 'explore.tools': 'यात्रा टूल',
  'cat.all': 'सभी', 'cat.best': 'सर्वश्रेष्ठ', 'cat.adventure': 'रोमांच', 'cat.food': 'खाना', 'cat.culture': 'संस्कृति', 'cat.beach': 'समुद्र तट',
  'cat.nature': 'प्रकृति', 'cat.nightlife': 'नाइटलाइफ़', 'cat.sightseeing': 'दर्शनीय स्थल', 'cat.shopping': 'खरीदारी', 'cat.relax': 'आराम',
  'len.any': 'कोई भी अवधि', 'len.u1': '1 घंटे से कम', 'len.1-2': '1–2 घंटे', 'len.2-4': '2–4 घंटे', 'len.half': 'आधा दिन', 'len.full': 'पूरा दिन',
  'itin.members': '{n} सदस्य', 'itin.noActivities': 'अभी कोई गतिविधि नहीं', 'itin.noActivitiesSub': 'इस दिन की पहली गतिविधि जोड़ने के लिए\n+ बटन दबाएँ।',
  'itin.noMatch': 'इन फ़िल्टर से कोई गतिविधि नहीं मिली', 'itin.addActivity': 'गतिविधि जोड़ें', 'itin.flights': 'उड़ानें और आगमन',
  'itin.flightsSub': 'समय {zone} में दिखाया गया है। बदलने के लिए टैप करें।', 'itin.addFlight': 'मेरी उड़ान जोड़ें', 'itin.assigned': '{n} ज़िम्मेदार',
  'itin.confirmed': 'पक्का', 'itin.voting': 'वोटिंग', 'itin.pending': 'बाकी', 'itin.perPerson': '/व्यक्ति',
  'add.title': 'गतिविधि जोड़ें', 'add.subtitle': 'नई गतिविधि का विवरण भरें।', 'add.type': 'गतिविधि का प्रकार', 'add.time': 'समय', 'add.name': 'गतिविधि का नाम',
  'add.location': 'स्थान', 'add.duration': 'अवधि', 'add.cost': 'प्रति व्यक्ति खर्च (USD)', 'add.notes': 'नोट्स (वैकल्पिक)', 'add.notesPh': 'समूह के लिए नोट्स…',
  'add.vote': 'समूह वोट के लिए रखें', 'add.remind': 'शुरू होने से पहले सबको याद दिलाएँ', 'add.submit': 'यात्रा योजना में जोड़ें', 'add.save': 'बदलाव सहेजें',
  'budget.kicker': 'समूह खर्च', 'budget.title': 'बजट', 'budget.spent': 'अब तक खर्च', 'budget.of': '{total} में से · {per} प्रति व्यक्ति',
  'budget.used': 'समूह बजट का {pct}% इस्तेमाल', 'budget.breakdown': 'खर्च का ब्यौरा', 'budget.currency': 'मुद्रा', 'budget.expenses': 'खर्च', 'budget.cap': 'बजट सीमा',
  'budget.addExpense': 'खर्च जोड़ें', 'budget.whatFor': 'किस लिए?', 'budget.add': 'खर्च जोड़ें', 'budget.settle': 'हिसाब बराबर करें', 'budget.planner': 'दैनिक खर्च योजना', 'budget.paidBy': '{name} ने भुगतान किया',
  'chat.kicker': '{n} यात्री', 'chat.title': 'यात्रा चैट', 'chat.placeholder': 'अपने साथियों को संदेश भेजें…', 'chat.askAi': 'ट्रिप असिस्टेंट से पूछें…',
  'chat.assistant': 'ट्रिप असिस्टेंट', 'chat.typing': '{name} लिख रहे हैं…', 'chat.q.budget': 'बजट कैसा है?', 'chat.q.food': 'आस-पास अच्छा खाना?',
  'chat.q.weather': 'मौसम कैसा है?', 'chat.q.next': 'आगे क्या है?',
  'hub.kicker': 'ट्रिप हब', 'hub.title': 'आपका समूह', 'hub.invite': 'आमंत्रण कोड', 'hub.share': 'शेयर', 'hub.members': 'सदस्य और भूमिकाएँ',
  'hub.vote': 'गतिविधियों पर वोट करें', 'hub.checklist': 'चेकलिस्ट', 'hub.done': '{a}/{b} पूरे', 'hub.addTask': 'काम जोड़ें', 'hub.documents': 'दस्तावेज़',
  'hub.documentsSub': 'बिना वाई-फ़ाई या डेटा के भी पढ़ सकते हैं।', 'hub.offline': 'ऑफ़लाइन', 'hub.tools': 'इस यात्रा के लिए और',
  'auth.welcome': 'फिर से स्वागत है!', 'auth.welcomeSub': 'अपनी यात्रा की योजना जारी रखने के लिए साइन इन करें।', 'auth.email': 'ईमेल पता', 'auth.password': 'पासवर्ड',
  'auth.remember': 'मुझे याद रखें', 'auth.forgot': 'पासवर्ड भूल गए?', 'auth.signIn': 'साइन इन', 'auth.or': 'या इसके साथ जारी रखें',
  'auth.google': 'Google के साथ जारी रखें', 'auth.noAccount': 'खाता नहीं है?', 'auth.signUp': 'साइन अप', 'auth.demo': 'डेमो खाता',
  'auth.terms': 'जारी रखकर आप हमारी सेवा की शर्तों और गोपनीयता नीति से सहमत होते हैं',
  'common.cancel': 'रद्द करें', 'common.save': 'सहेजें', 'common.delete': 'हटाएँ', 'common.done': 'हो गया', 'common.next': 'आगे', 'common.getStarted': 'शुरू करें',
  'common.skip': 'छोड़ें', 'common.back': 'वापस', 'common.edit': 'बदलें', 'common.close': 'बंद करें',
  'settings.title': 'सेटिंग्स', 'settings.language': 'भाषा', 'settings.currency': 'घरेलू मुद्रा', 'settings.notifications': 'गतिविधि रिमाइंडर',
  'settings.lead': 'पहले याद दिलाएँ', 'settings.textSize': 'टेक्स्ट का आकार', 'settings.signOut': 'साइन आउट',
};

const zh: Dict = {
  'nav.explore': '首页', 'nav.itinerary': '计划', 'nav.budget': '预算', 'nav.chat': '聊天', 'nav.profile': '更多',
  'greet.morning': '早上好', 'greet.afternoon': '下午好', 'greet.evening': '晚上好', 'greet.night': '夜深了',
  'explore.title': '发现你的\n理想目的地', 'explore.subtitle': '一起计划、探索、出发。', 'explore.search': '搜索城市或目的地…',
  'explore.popular': '热门地点', 'explore.featured': '精选目的地', 'explore.seeAll': '查看全部', 'explore.matches': '{n} 个结果',
  'explore.nearby': '半径范围内的地点', 'explore.activeTrip': '进行中的旅行', 'explore.upcomingTrip': '即将出发', 'explore.pastTrip': '已完成的旅行',
  'explore.left': '剩余 {amount}', 'explore.members': '{n} 位成员', 'explore.reminder': '提醒', 'explore.reminderSub': '我们会提前 {n} 分钟通
```

```
知大家。',
  'explore.nearestAirport': '最近的机场', 'explore.addToTrip': '加入旅行', 'explore.noResults': '没有与“{q}”匹配的目的地', 'explore.tools': '旅行工具',
  'cat.all': '全部', 'cat.best': '最佳', 'cat.adventure': '探险', 'cat.food': '美食', 'cat.culture': '文化', 'cat.beach': '海滩', 'cat.nature': '自然',
  'cat.nightlife': '夜生活', 'cat.sightseeing': '观光', 'cat.shopping': '购物', 'cat.relax': '放松',
  'len.any': '不限时长', 'len.u1': '1 小时以内', 'len.1-2': '1–2 小时', 'len.2-4': '2–4 小时', 'len.half': '半天', 'len.full': '全天',
  'itin.members': '{n} 位成员', 'itin.noActivities': '还没有活动', 'itin.noActivitiesSub': '点击 + 按钮添加\n当天的第一个活动。', 'itin.noMatch': '没有符合筛选的活动',
  'itin.addActivity': '添加活动', 'itin.flights': '航班与抵达', 'itin.flightsSub': '时间以 {zone} 显示，点击切换时区。', 'itin.addFlight': '添加我的航班',
  'itin.assigned': '{n} 人负责', 'itin.confirmed': '已确认', 'itin.voting': '投票中', 'itin.pending': '待定', 'itin.perPerson': '/人',
  'add.title': '添加活动', 'add.subtitle': '填写新活动的详细信息。', 'add.type': '活动类型', 'add.time': '时间', 'add.name': '活动名称', 'add.location': '地点',
  'add.duration': '时长', 'add.cost': '人均费用 (USD)', 'add.notes': '备注（可选）', 'add.notesPh': '给大家的备注…', 'add.vote': '发起小组投票',
  'add.remind': '开始前提醒所有人', 'add.submit': '加入行程', 'add.save': '保存修改',
  'budget.kicker': '小组开销', 'budget.title': '预算', 'budget.spent': '目前已花费', 'budget.of': '共 {total} · 人均 {per}', 'budget.used': '已使用小组预算的 {pct}%',
  'budget.breakdown': '支出明细', 'budget.currency': '货币', 'budget.expenses': '支出', 'budget.cap': '预算上限', 'budget.addExpense': '添加支出',
  'budget.whatFor': '用途是什么？', 'budget.add': '添加支出', 'budget.settle': '结算', 'budget.planner': '每日花费规划', 'budget.paidBy': '{name} 支付',
  'chat.kicker': '{n} 位旅伴', 'chat.title': '旅行群聊', 'chat.placeholder': '给旅伴发消息…', 'chat.askAi': '问问旅行助手…', 'chat.assistant': '旅行助手',
  'chat.typing': '{name} 正在输入…', 'chat.q.budget': '预算怎么样？', 'chat.q.food': '附近有什么好吃的？', 'chat.q.weather': '天气如何？', 'chat.q.next': '接下来做什么？',
  'hub.kicker': '旅行中心', 'hub.title': '你的小组', 'hub.invite': '邀请码', 'hub.share': '分享', 'hub.members': '成员与分工', 'hub.vote': '为活动投票',
  'hub.checklist': '清单', 'hub.done': '已完成 {a}/{b}', 'hub.addTask': '添加任务', 'hub.documents': '文件', 'hub.documentsSub': '无网络时也能查看。',
  'hub.offline': '离线可用', 'hub.tools': '更多旅行工具',
  'auth.welcome': '欢迎回来！', 'auth.welcomeSub': '登录以继续规划你的旅程。', 'auth.email': '电子邮箱', 'auth.password': '密码', 'auth.remember': '记住我',
  'auth.forgot': '忘记密码？', 'auth.signIn': '登录', 'auth.or': '或使用以下方式', 'auth.google': '使用 Google 继续', 'auth.noAccount': '还没有账号？',
  'auth.signUp': '注册', 'auth.demo': '演示账号', 'auth.terms': '继续即表示你同意我们的服务条款和隐私政策',
  'common.cancel': '取消', 'common.save': '保存', 'common.delete': '删除', 'common.done': '完成', 'common.next': '下一步', 'common.getStarted': '开始使用',
  'common.skip': '跳过', 'common.back': '返回', 'common.edit': '编辑', 'common.close': '关闭',
  'settings.title': '设置', 'settings.language': '语言', 'settings.currency': '本国货币', 'settings.notifications': '活动提醒', 'settings.lead': '提前提醒',
  'settings.textSize': '文字大小', 'settings.signOut': '退出登录',
};

export const dictionaries: Record<LangCode, Dict> = { en, es, fr, ja, hi, zh };
export const english = en;
```

src/services/__tests__/assistant.test.ts

```
import { CLAUDE_MODEL, answerLocally, askClaude, type AssistantContext } from '../assistant';
import type { ChatMessage } from '@/store/types';
import { buildDemoTrip, DEMO_USER_ID } from '@/store/seed';
import { settlePlan } from '@/utils/money';

const trip = buildDemoTrip('upcoming');
const ctx: AssistantContext = { trip, userId: DEMO_USER_ID, rates: null, homeCurrency: 'USD' };
const ask = (q: string) => answerLocally(q, ctx);

describe('offline Trip Assistant', () => {
  it('answers budget questions from the real trip data', () => {
    const a = ask('How’s the budget?');
    expect(a).toMatch(/\$2,086 of \$4,000/);
    expect(a).toMatch(/per person/);
  });

  it('lists the fewest payments to settle up and matches the app’s own plan', () => {
    const plan = settlePlan(trip);
    const a = ask('who owes who?');
    expect(a).toContain(`${plan.length} transfer`);
    expect(a).toMatch(/→/);
  });

  it('converts currency with an honest label when offline', () => {
    const a = ask('convert 50 usd to eur');
    expect(a).toMatch(/\$50\.00 ≈ /);
    expect(a).toMatch(/offline reference rate/);
  });

  it('knows local emergency numbers', () => {
    expect(ask('emergency, I need an ambulance')).toMatch(/Police \d+/);
  });

  it('suggests places that are not already on the itinerary', () => {
    expect(ask('what should we do?')).toMatch(/Ideas not on your itinerary yet|already planned/);
  });
```

```
  it('says what it can do when it does not understand', () => {
    expect(ask('zzzz qqqq')).toMatch(/I can help with/);
  });

  it('greets by trip name', () => {
    expect(ask('hello')).toContain(trip.name);
  });
});

describe('Claude mode', () => {
  const realFetch = global.fetch;
  afterEach(() => {
    global.fetch = realFetch;
    jest.useRealTimers();
  });

  const reply = (body: unknown, status = 200) => jest.fn(async (_url: string, _init?: RequestInit) => ({ ok: status < 400, status, json:
async () => body }) as Response);
  const sentBody = (fetchMock: ReturnType<typeof reply>) => JSON.parse(fetchMock.mock.calls[0][1]?.body as string);

  it('sends the question, the recent chat and the trip to the Messages API', async () => {
    const fetchMock = reply({ stop_reason: 'end_turn', content: [{ type: 'text', text: 'Take the ferry from Amalfi.' }] });
    global.fetch = fetchMock as unknown as typeof fetch;
    const history: ChatMessage[] = [
      { id: 'a', authorId: 'assistant', text: 'old answer that starts the thread', ts: 1, ai: true },
      { id: 'b', authorId: DEMO_USER_ID, text: 'Where do we eat?', ts: 2 },
      { id: 'c', authorId: 'assistant', text: 'Try Da Vincenzo.', ts: 3, ai: true },
      { id: 'd', authorId: 'system', text: 'someone joined', ts: 4, kind: 'system' },
    ];
    const text = await askClaude('sk-ant-test', 'How do we get to Capri?', history, ctx);
    expect(text).toBe('Take the ferry from Amalfi.');

    const [url, init] = fetchMock.mock.calls[0];
    expect(url).toBe('https://api.anthropic.com/v1/messages');
    expect(init?.method).toBe('POST');
    expect(init?.headers).toMatchObject({ 'x-api-key': 'sk-ant-test', 'anthropic-version': '2023-06-01' });
    const body = sentBody(fetchMock);
    expect(body.model).toBe(CLAUDE_MODEL);
    expect(body.system).toContain(trip.name);
    // the thread must start with a user message, system notes are left out, and the question comes last
    expect(body.messages[0].role).toBe('user');
    expect(body.messages.map((m: { role: string }) => m.role)).toEqual(['user', 'assistant', 'user']);
    expect(body.messages[0].content).toMatch(/Maya Chen: Where do we eat\?/);
    expect(body.messages.at(-1)).toEqual({ role: 'user', content: 'How do we get to Capri?' });
    expect(JSON.stringify(body.messages)).not.toContain('someone joined');
  });

  it('only sends the last 12 messages of the chat', async () => {
    const fetchMock = reply({ content: [{ type: 'text', text: 'ok' }] });
    global.fetch = fetchMock as unknown as typeof fetch;
    const history: ChatMessage[] = Array.from({ length: 30 }, (_, i) => ({ id: `m${i}`, authorId: DEMO_USER_ID, text: `message ${i}`, t
s: i }));
    await askClaude('sk-ant-test', 'And now?', history, ctx);
    expect(sentBody(fetchMock).messages).toHaveLength(13);
    expect(sentBody(fetchMock).messages[0].content).toContain('message 18');
  });

  it('joins several text blocks and ignores the rest', async () => {
    global.fetch = reply({ content: [{ type: 'text', text: 'First.' }, { type: 'tool_use' }, { type: 'text', text: 'Second.' }] }) as un
known as typeof fetch;
    expect(await askClaude('k', 'q', [], ctx)).toBe('First.\nSecond.');
  });

  it('fails clearly when the key is wrong, the model refuses or the answer is empty', async () => {
    global.fetch = reply({ error: { message: 'invalid x-api-key' } }, 401) as unknown as typeof fetch;
    await expect(askClaude('bad', 'q', [], ctx)).rejects.toThrow('Claude 401');
    global.fetch = reply({ stop_reason: 'refusal', content: [] }) as unknown as typeof fetch;
    await expect(askClaude('k', 'q', [], ctx)).rejects.toThrow('refused');
    global.fetch = reply({ content: [{ type: 'text', text: '   ' }] }) as unknown as typeof fetch;
    await expect(askClaude('k', 'q', [], ctx)).rejects.toThrow('empty');
  });

  it('gives up after 45 seconds instead of waiting forever', async () => {
    jest.useFakeTimers();
    global.fetch = jest.fn((_url, init) => new Promise((_resolve, reject) => (init as RequestInit).signal?.addEventListener('abort', ()
=> reject(new Error('aborted'))))) as unknown as typeof fetch;
    const pending = askClaude('k', 'q', [], ctx);
    const outcome = expect(pending).rejects.toThrow('aborted');
    await jest.advanceTimersByTimeAsync(45_000);
    await outcome;
  });
});
```

src/services/__tests__/placeSearch.test.ts

```
import { boxAround, guessType, mergeHits, normalizeQuery, searchOnline, searchSaved, typeFromOsm, type PlaceHit } from '../placeSearch';

/** What Photon answers for "Da Vincenzo" near Positano (trimmed to the fields we read). */
const photon = (features: unknown[]): typeof fetch => (async () => ({ ok: true, status: 200, json: async () => ({ features }) })) as unk
nown as typeof fetch;
const feature = (props: Record<string, unknown>, lng = 14.4849, lat = 40.6277) => ({ geometry: { coordinates: [lng, lat] }, properties:
props });

const DA_VINCENZO = feature({ osm_type: 'N', osm_id: 1, osm_key: 'amenity', osm_value: 'restaurant', name: 'Da Vincenzo 1958', street:
'Via Pasitea', housenumber: '172', city: 'Positano', country: 'Italy' });
const BUS_STOP = feature({ osm_type: 'N', osm_id: 2, osm_key: 'highway', osm_value: 'bus_stop', name: 'Positano (Sponda)', city: 'Posita
no' });

describe('normalizeQuery', () => {
  it('drops leading articles and extra spaces', () => {
    expect(normalizeQuery('The museum')).toBe('museum');
    expect(normalizeQuery('  a   Pizza place ')).toBe('Pizza place');
    expect(normalizeQuery('An Evening Walk')).toBe('Evening Walk');
  });
  it('leaves nothing for a lone article', () => {
    expect(normalizeQuery('the')).toBe('');
    expect(normalizeQuery('  ')).toBe('');
  });
});

describe('searchSaved (the trip’s own places, no connection needed)', () => {
  it('finds a place by the start of its name, then by a word in it', () => {
    expect(searchSaved('da vinc', ['positano'])[0].name).toBe('Da Vincenzo');
    expect(searchSaved('assunta', ['positano'])[0].name).toBe('Santa Maria Assunta');
  });
  it('carries what the form needs: where it is, what kind it is, how long and how much', () => {
    const [hit] = searchSaved('Path of the Gods', ['positano']);
    expect(hit).toMatchObject({ source: 'saved', type: 'adventure', durationMin: 300, costPerPerson: 0, placeId: 'path-of-gods' });
    expect(hit.lat).toBeCloseTo(40.619, 2);
    expect(hit.detail).toContain('Positano');
  });
  it('returns nothing for one letter or no match, and never the same place twice', () => {
    expect(searchSaved('d', ['positano'])).toEqual([]);
    expect(searchSaved('zzzzqqqq', ['positano'])).toEqual([]);
    const twice = searchSaved('vincenzo', ['positano', 'positano']);
    expect(twice.filter((h) => h.name === 'Da Vincenzo')).toHaveLength(1);
  });
});

describe('searchOnline (OpenStreetMap through Photon)', () => {
  const center = { lat: 40.6287, lng: 14.4855 };

  it('turns features into places and leaves out streets and bus stops', async () => {
    const hits = await searchOnline('da vincenzo online', center, { fetchFn: photon([DA_VINCENZO, BUS_STOP]) });
    expect(hits).toHaveLength(1);
    expect(hits[0]).toMatchObject({ name: 'Da Vincenzo 1958', detail: 'Via Pasitea 172, Positano', type: 'food', source: 'online', lat:
40.6277, lng: 14.4849 });
  });

  it('looks around the trip first (rounded point and a box) and sends the words without the leading "the"', async () => {
    const urls: string[] = [];
    const fetchFn = (async (url: string) => {
      urls.push(url);
      return { ok: true, status: 200, json: async () => ({ features: [DA_VINCENZO] }) };
    }) as unknown as typeof fetch;
    await searchOnline('The Trattoria box', center, { fetchFn });
    expect(urls).toHaveLength(1);
    expect(urls[0]).toContain('q=Trattoria%20box');
    expect(urls[0]).toContain('lat=40.63&lon=14.49');
    expect(urls[0]).toMatch(/bbox=14\.\d+,40\.\d+,14\.\d+,40\.\d+/);
  });

  it('tries again anywhere when nothing is near the trip', async () => {
    const urls: string[] = [];
    const fetchFn = (async (url: string) => {
      urls.push(url);
      return { ok: true, status: 200, json: async () => ({ features: urls.length === 1 ? [] : [DA_VINCENZO] }) };
    }) as unknown as typeof fetch;
    const hits = await searchOnline('faraway fallback', center, { fetchFn });
    expect(urls).toHaveLength(2);
    expect(urls[0]).toContain('bbox=');
    expect(urls[1]).not.toContain('bbox=');
    expect(hits).toHaveLength(1);
  });

  it('searches everywhere when the trip has no known place', async () => {
    const urls: string[] = [];
    const fetchFn = (async (url: string) => {
      urls.push(url);
```

```
      return { ok: true, status: 200, json: async () => ({ features: [DA_VINCENZO] }) };
    }) as unknown as typeof fetch;
    await searchOnline('nowhere in particular', null, { fetchFn });
    expect(urls).toHaveLength(1);
    expect(urls[0]).not.toMatch(/lat=|bbox=/);
  });

  it('remembers an answer instead of asking again', async () => {
    const fetchFn = jest.fn(photon([DA_VINCENZO]));
    await searchOnline('remember me', center, { fetchFn });
    await searchOnline('Remember me', center, { fetchFn });
    expect(fetchFn).toHaveBeenCalledTimes(1);
  });

  it('rejects when the service is down or the phone is offline', async () => {
    const down = (async () => ({ ok: false, status: 503, json: async () => ({}) })) as unknown as typeof fetch;
    await expect(searchOnline('service down', center, { fetchFn: down })).rejects.toThrow(/503/);
    const offline = (async () => {
      throw new TypeError('Network request failed');
    }) as unknown as typeof fetch;
    await expect(searchOnline('no network', center, { fetchFn: offline })).rejects.toThrow(/Network/);
  });

  it('gives up on a service that never answers', async () => {
    const hang: typeof fetch = (_url, init) =>
      new Promise((_resolve, reject) => {
        init?.signal?.addEventListener('abort', () => reject(Object.assign(new Error('aborted'), { name: 'AbortError' })));
      });
    const started = Date.now();
    await expect(searchOnline('very slow', center, { fetchFn: hang, timeoutMs: 40 })).rejects.toThrow(/abort/);
    expect(Date.now() - started).toBeLessThan(1000);
  });

  it('stops when the caller cancels', async () => {
    const hang: typeof fetch = (_url, init) =>
      new Promise((_resolve, reject) => {
        init?.signal?.addEventListener('abort', () => reject(Object.assign(new Error('aborted'), { name: 'AbortError' })));
      });
    const stop = new AbortController();
    const pending = searchOnline('cancel me', center, { fetchFn: hang, signal: stop.signal });
    stop.abort();
    await expect(pending).rejects.toThrow(/abort/);
  });

  it('does not ask for one letter', async () => {
    const fetchFn = jest.fn(photon([]));
    expect(await searchOnline('d', center, { fetchFn })).toEqual([]);
    expect(await searchOnline('the', center, { fetchFn })).toEqual([]);
    expect(fetchFn).not.toHaveBeenCalled();
  });
});

describe('typeFromOsm and guessType', () => {
  it('maps OpenStreetMap tags to activity types', () => {
    expect(typeFromOsm('tourism', 'museum')).toBe('culture');
    expect(typeFromOsm('tourism', 'hotel')).toBe('stay');
    expect(typeFromOsm('amenity', 'restaurant')).toBe('food');
    expect(typeFromOsm('amenity', 'bar')).toBe('nightlife');
    expect(typeFromOsm('shop', 'gift')).toBe('shopping');
    expect(typeFromOsm('natural', 'beach')).toBe('sightseeing');
    expect(typeFromOsm('railway', 'station')).toBe('transport');
    expect(typeFromOsm('something', 'else')).toBe('sightseeing');
  });
  it('guesses the type of words typed by hand', () => {
    expect(guessType('Dinner at 7')).toBe('food');
    expect(guessType('Sunset aperitivo')).toBe('nightlife');
    expect(guessType('Pompeii museum visit')).toBe('culture');
    expect(guessType('Morning hike')).toBe('adventure');
    expect(guessType('Transfer to the airport')).toBe('transport');
    expect(guessType('Check-in')).toBe('stay');
    expect(guessType('Wander around')).toBe('sightseeing');
  });
});

describe('mergeHits and boxAround', () => {
  const hit = (id: string, name: string, lat: number, lng: number, source: PlaceHit['source']): PlaceHit => ({ id, name, detail: '', la
t, lng, type: 'food', source });
  it('shows saved places first and drops an online copy of the same place', () => {
    const saved = [hit('saved:1', 'Da Vincenzo', 40.6295, 14.488, 'saved')];
    const online = [hit('online:a', 'Da Vincenzo 1958', 40.6277, 14.4849, 'online'), hit('online:b', 'Max Restaurant', 40.6296, 14.486,
'online')];
    const merged = mergeHits(saved, online);
    expect(merged.map((h) => h.name)).toEqual(['Da Vincenzo', 'Max Restaurant']);
  });
  it('keeps two places with the same name when they are far apart', () => {
    const merged = mergeHits([hit('saved:1', 'Museum', 40.6, 14.4, 'saved')], [hit('online:a', 'Museum', 50.08, 14.43, 'online')]);
```

```
    expect(merged).toHaveLength(2);
  });
  it('shows at most seven', () => {
    const online = Array.from({ length: 12 }, (_, i) => hit(`online:${i}`, `Place ${i}`, 40 + i, 14, 'online'));
    expect(mergeHits([], online)).toHaveLength(7);
  });
  it('draws a box around the trip that is wider than it is tall in degrees at higher latitudes', () => {
    const [w, s, e, n] = boxAround({ lat: 60, lng: 10 }, 40);
    expect(n - s).toBeCloseTo(0.724, 2);
    expect(e - w).toBeGreaterThan(n - s);
    expect((w + e) / 2).toBeCloseTo(10, 5);
  });
});
```

src/services/__tests__/pretendFriends.test.ts

```
import { friendReply, planFriendReply } from '../pretendFriends';

describe('friendReply', () => {
  it('answers by topic', () => {
    expect(friendReply('anyone hungry for dinner?')).toMatch(/food/);
    expect(friendReply('what time is the ferry?')).toMatch(/assistant|vote/);
    expect(friendReply('running a bit late')).toMatch(/save you a seat/);
    expect(friendReply('sent you the pics')).toMatch(/photo dump/);
    expect(friendReply('I paid $40 for the taxi')).toMatch(/Budget/);
  });

  it('falls back to a short cheer, chosen by the random number', () => {
    expect(friendReply('see you there', () => 0)).toBe('Love it 🔥');
    expect(friendReply('see you there', () => 0.999)).toBe('Let’s goooo 🙌');
  });
});

describe('planFriendReply', () => {
  it('stays quiet when nobody is there to answer', () => {
    expect(planFriendReply([], () => 0)).toBeNull();
  });

  it('replies about 70% of the time, with a delay of a few seconds', () => {
    expect(planFriendReply(['a', 'b'], () => 0.71)).toBeNull();
    const plan = planFriendReply(['a', 'b'], () => 0.5);
    expect(plan?.friend).toBe('b');
    expect(plan?.delay).toBeGreaterThanOrEqual(2600);
    expect(plan?.delay).toBeLessThanOrEqual(3800);
  });
});
```

src/services/__tests__/triviaReminder.test.ts

```
import * as Notifications from 'expo-notifications';

import { TRIVIA_REMINDER } from '@/data/personal';

import { setTriviaReminder } from '../notifications';

jest.mock('expo-notifications', () => ({
  cancelScheduledNotificationAsync: jest.fn(async () => {}),
  scheduleNotificationAsync: jest.fn(async () => 'id'),
  getPermissionsAsync: jest.fn(async () => ({ granted: true })),
  requestPermissionsAsync: jest.fn(async () => ({ granted: true })),
  setNotificationHandler: jest.fn(),
  setNotificationChannelAsync: jest.fn(async () => {}),
  addNotificationResponseReceivedListener: jest.fn(),
  AndroidImportance: { HIGH: 4 },
  SchedulableTriggerInputTypes: { DAILY: 'daily', DATE: 'date', TIME_INTERVAL: 'timeInterval' },
}));
jest.mock('expo-router', () => ({ router: { push: jest.fn() } }));

const n = jest.mocked(Notifications);

describe('the daily trivia reminder', () => {
  beforeEach(() => jest.clearAllMocks());

  it('schedules one notification a day at the same time when it is switched on', async () => {
    expect(await setTriviaReminder(true)).toBe(true);
    expect(n.scheduleNotificationAsync).toHaveBeenCalledTimes(1);
    const arg = n.scheduleNotificationAsync.mock.calls[0][0];
    expect(arg.identifier).toBe(TRIVIA_REMINDER.id);
    expect(arg.content.title).toBe(TRIVIA_REMINDER.title);
    expect(arg.trigger).toMatchObject({ type: 'daily', hour: TRIVIA_REMINDER.hour, minute: TRIVIA_REMINDER.minute });
  });

  it('replaces the old one instead of piling up a second', async () => {
    await setTriviaReminder(true);
    expect(n.cancelScheduledNotificationAsync).toHaveBeenCalledWith(TRIVIA_REMINDER.id);
  });
```

```
  it('takes it off the calendar when it is switched off, without asking for permission', async () => {
    expect(await setTriviaReminder(false)).toBe(true);
    expect(n.cancelScheduledNotificationAsync).toHaveBeenCalledWith(TRIVIA_REMINDER.id);
    expect(n.scheduleNotificationAsync).not.toHaveBeenCalled();
    expect(n.requestPermissionsAsync).not.toHaveBeenCalled();
  });

  it('says no, and schedules nothing, when the phone refuses notifications', async () => {
    n.getPermissionsAsync.mockResolvedValueOnce({ granted: false } as never);
    n.requestPermissionsAsync.mockResolvedValueOnce({ granted: false } as never);
    expect(await setTriviaReminder(true)).toBe(false);
    expect(n.scheduleNotificationAsync).not.toHaveBeenCalled();
  });

  it('says no instead of crashing when scheduling fails', async () => {
    n.scheduleNotificationAsync.mockRejectedValueOnce(new Error('boom'));
    expect(await setTriviaReminder(true)).toBe(false);
  });
});
```

src/services/assistant.ts

```
/**
 * Trip Assistant.
 * 1. Offline engine (default): intent matching over the live trip data, so it
 *    answers accurately with no internet — important at venues with bad wifi.
 * 2. Claude mode (optional): when a user adds their own Anthropic API key in
 *    Settings, questions go to Claude with the trip as context. Any failure
 *    falls back to the offline engine.
 */
import { destinationById } from '@/data/destinations';
import { placesFor } from '@/data/places';
import { currencies, emergencyByCountry, phrasebook } from '@/data/reference';
import { balances, settlePlan, totalSpent } from '@/utils/money';
import type { ChatMessage, RatesSnapshot, Trip, WeatherSnapshot } from '@/store/types';
import { durationLabel, money } from '@/utils/format';
import { distanceKm, kmLabel, nearestAirports } from '@/utils/geo';
import { addDays, dayLabel, deviceTimeZone, formatInTz, hoursAhead, partsInTz, time12, todayInTz, tzAbbrev } from '@/utils/time';

import { convert } from './currency';
import { cToF, weatherInfo } from './weather';

export const CLAUDE_MODEL = 'claude-opus-5-5';

export interface AssistantContext {
  trip: Trip;
  userId: string;
  weather?: WeatherSnapshot;
  rates: RatesSnapshot | null;
  homeCurrency: string;
}

const first = (t: Trip, id: string) => t.members.find((m) => m.id === id)?.name.split(' ')[0] ?? 'Someone';

const CURRENCY_WORDS: Record<string, string> = {
  dollar: 'USD', dollars: 'USD', usd: 'USD', $: 'USD', yen: 'JPY', jpy: 'JPY', '¥': 'JPY', euro: 'EUR', euros: 'EUR', eur: 'EUR', '€':
'EUR',
  pound: 'GBP', pounds: 'GBP', gbp: 'GBP', '£': 'GBP', won: 'KRW', krw: 'KRW', baht: 'THB', thb: 'THB', rupee: 'INR', rupees: 'INR', in
r: 'INR',
  peso: 'MXN', pesos: 'MXN', mxn: 'MXN', franc: 'CHF', francs: 'CHF', chf: 'CHF', cad: 'CAD', aud: 'AUD', sgd: 'SGD', yuan: 'CNY', cny:
'CNY',
};

function currencyFrom(word: string | undefined): string | undefined {
  if (!word) return undefined;
  const w = word.toLowerCase();
  if (CURRENCY_WORDS[w]) return CURRENCY_WORDS[w];
  const up = w.toUpperCase();
  return currencies.some((c) => c.code === up) ? up : undefined;
}

export function answerLocally(question: string, ctx: AssistantContext): string {
  const q = question.toLowerCase().trim();
  const { trip } = ctx;
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const spent = totalSpent(trip);
  const per = spent / Math.max(1, trip.members.length);
  const has = (re: RegExp) => re.test(q);

  // Currency conversion: "convert 50 usd to jpy", "how much is 3000 yen"
  const conv = q.match(/([\d][\d,.]*)\s*([a-z$€£¥]{1,8})?\s*(?:to|in|into)?\s*([a-z$€£¥]{3,8})?/);
  if (has(/convert|exchange|how much is|in (yen|dollars|euros|usd|jpy|eur)|→/) && conv) {
    const amount = Number(conv[1].replace(/,/g, ''));
    const from = currencyFrom(conv[2]) ?? (has(/yen|jpy/) && !has(/to (yen|jpy)/) ? 'JPY' : 'USD');
```

```
    let to = currencyFrom(conv[3]) ?? (from === (dest?.currency ?? 'JPY') ? ctx.homeCurrency : dest?.currency ?? 'JPY');
    if (to === from) to = from === 'USD' ? dest?.currency ?? 'EUR' : 'USD';
    const out = convert(amount, from, to, ctx.rates);
    return `${money(amount, from, { decimals: true })} ≈ ${money(out, to, { decimals: true })} ${ctx.rates?.live ? '(live rate)' : '(off
line reference rate)'}. Tip: the Budget tab has a full converter with home/local view.`;
  }

  if (has(/owe|settle|split|pay (back|me)|who pays|balance/)) {
    const plan = settlePlan(trip);
    if (!plan.length) return 'Everyone is square right now — no one owes anything.';
    const lines = plan.slice(0, 5).map((p) => `• ${first(trip, p.from)} → ${first(trip, p.to)}: ${money(p.amount, 'USD', { decimals: tru
e })}`);
    const mine = balances(trip)[ctx.userId] ?? 0;
    return `Here's the simplest way to settle up (${plan.length} transfer${plan.length > 1 ? 's' : ''}):\n${lines.join('\n')}\n\nYou are
${mine >= 0 ? `owed ${money(mine)}` : `owing ${money(-mine)}`}. Tap "Settle up" in Budget to record payments.`;
  }

  if (has(/budget|spent|spend|money|afford|left|expens|cost so far/)) {
    const pct = Math.round((spent / Math.max(1, trip.budget)) * 100);
    const byCat: Record<string, number> = {};
    trip.expenses.filter((e) => !e.settlement).forEach((e) => (byCat[e.category] = (byCat[e.category] ?? 0) + e.amount));
    const top = Object.entries(byCat).sort((a, b) => b[1] - a[1])[0];
    const remaining = trip.budget - spent;
    const days = Math.max(1, Math.round((Date.parse(trip.endDate) - Date.parse(trip.startDate)) / 86400000) + 1);
    return `You've committed ${money(spent)} of ${money(trip.budget)} — about ${pct}% of the group budget (${money(per)} per person).
${top ? `Biggest category: ${top[0]} (${money(top[1])}). ` : ''}${remaining >= 0 ? `${money(remaining)} left ≈ ${money(remaining / days
/ trip.members.length)} per person per day.` : `You're ${money(-remaining)} over — Food is usually the easiest place to trim.`}`;
  }

  if (has(/weather|rain|temperature|forecast|umbrella|cold|hot|sunny|snow|degrees/)) {
    const w = ctx.weather;
    if (!w) return `I don't have a forecast for ${dest?.city ?? 'the destination'} yet — open the Weather screen while online and I'll r
emember it for offline use.`;
    const now = weatherInfo(w.current.code);
    const tomorrow = w.daily[1];
    const rainy = w.daily.slice(0, 3).some((d) => d.rain >= 50);
    return `${dest?.city}: ${w.current.temp}°C (${cToF(w.current.temp)}°F), ${now.label.toLowerCase()}. ${tomorrow ? `Tomorrow ${tomorro
w.min}–${tomorrow.max}°C, ${tomorrow.rain}% chance of rain. ` : ''}${rainy ? 'Pack an umbrella ☔ — rain is likely in the next few days.'
: 'Looks good for walking days. 👟'}${w.live ? '' : ' (estimated — connect to refresh)'}`;
  }

  if (has(/food|eat|restaurant|dinner|lunch|breakfast|ramen|sushi|hungry|snack|cafe|coffee/)) {
    const cheap = has(/cheap|budget|afford|under/);
    const center = dest ? { lat: dest.lat, lng: dest.lng } : null;
    const list = placesFor(trip.destinationId)
      .concat(trip.extraDestinationIds.flatMap((id) => placesFor(id)))
      .filter((p) => p.category === 'food' && (!cheap || p.cost <= 20))
      .map((p) => ({ p, km: center ? distanceKm(center, p) : 0 }))
      .sort((a, b) => b.p.rating - a.p.rating)
      .slice(0, 3);
    if (!list.length) return 'I couldn't find food spots for this destination yet — try the Nearby screen to widen the radius.';
    return `Top-rated food${cheap ? ' on a budget' : ''} near ${dest?.city}:\n${list
      .map(({ p, km }) => `• ${p.name} (${p.area}) — ★${p.rating}, ~${p.cost ? money(p.cost) : 'free'}/person, ${kmLabel(km)} from centr
e`)
      .join('\n')}\nWant one added to the itinerary? Open Nearby and tap "Add".`;
  }

  if (has(/flight|land|arriv|depart|airport|plane/)) {
    const arrivals = trip.flights.filter((f) => f.direction === 'arrival').sort((a, b) => a.arriveUtc - b.arriveUtc);
    const missing = trip.members.filter((m) => !trip.flights.some((f) => f.memberId === m.id)).map((m) => m.name.split(' ')[0]);
    const air = dest ? nearestAirports(dest, 1)[0] : undefined;
    const lines = arrivals.map((f) => `• ${first(trip, f.memberId)} — ${f.flightNo}, lands ${formatInTz(f.arriveUtc, tz)} ${tzAbbrev(t
z)}`);
    return `${lines.length ? `Arrivals:\n${lines.join('\n')}` : 'No flights added yet.'}${missing.length ? `\n${missing.join(', ')} stil
l need to add flights.` : ''}${air ? `\nNearest airport to ${dest?.city} centre: ${air.iata} (${air.name}).` : ''}`;
  }

  if (has(/time zone|timezone|what time|jet ?lag|time difference/)) {
    const home = deviceTimeZone();
    const diff = hoursAhead(tz, home);
    const p = partsInTz(Date.now(), tz);
    return `It's ${time12(p.hhmm)} in ${dest?.city} (${tzAbbrev(tz)}). That's ${diff === 0 ? 'the same as' : `${Math.abs(diff)}h ${diff
> 0 ? 'ahead of' : 'behind'}`} your phone's time zone. Itinerary times are always shown in local trip time.${Math.abs(diff) >= 6 ? ' Jet
-lag tip: get daylight on arrival day and avoid naps over 30 min.' : ''}`;
  }

  if (has(/how (do|to) (i |you |we )?say|translate|phrase|in japanese|in french|in spanish/)) {
    const book = dest ? phrasebook[dest.language] : undefined;
    if (!book) return 'Open the Phrasebook from the menu for common phrases with audio.';
    const hit = book.phrases.find((p) => q.includes(p.en.toLowerCase().replace(/[?!]/g, '').split(' / ')[0]));
    if (hit) return `"${hit.en}" in ${book.lang}: ${hit.local}${hit.roman ? ` (${hit.roman})` : ''}. Tap the speaker in the Phrasebook t
o hear it.`;
    return `Handy ${book.lang} phrases:\n${book.phrases.slice(0, 4).map((p) => `• ${p.en} — ${p.local}${p.roman ? ` (${p.roman})` : ''}
`).join('\n')}`;
  }
```

```
  if (has(/emergency|police|hospital|ambulance|lost passport|help me|doctor|stolen/)) {
    const em = emergencyByCountry[dest?.countryCode ?? ''] ?? emergencyByCountry.DEFAULT;
    return `In ${dest?.country ?? 'this country'}: Police ${em.police} · Ambulance ${em.ambulance} · Fire ${em.fire}.\n${em.tips[0] ??
''}\nThe Emergency screen works offline and can share your location by text.`;
  }

  if (has(/pack|bring|luggage|suitcase/)) {
    const left = trip.packing.filter((p) => !p.packed);
    const rainy = ctx.weather?.daily.slice(0, 5).some((d) => d.rain >= 50);
    return `${left.length ? `Still to pack (${left.length}): ${left.slice(0, 5).map((p) => p.text).join(', ')}${left.length > 5 ? '…' :
''}.` : 'Everything on the packing list is packed! 🎒'}${rainy ? ' Rain is forecast — add an umbrella.' : ''} ${dest?.countryCode === 'J
P' ? 'Japan uses type A plugs (100V) and is still cash-friendly — bring a coin purse.' : ''}`.trim();
  }

  if (has(/vote|poll|decide|undecided/)) {
    const open = trip.polls.filter((p) => !p.closed);
    const voting = trip.activities.filter((a) => a.status === 'voting');
    const pollLines = open.map((p) => {
      const leader = p.options.slice().sort((a, b) => b.votes.length - a.votes.length)[0];
      const voted = p.options.some((o) => o.votes.includes(ctx.userId));
      return `• “${p.question}” — leading: ${leader?.text ?? '—'}${voted ? '' : ' (you haven’t voted)'}`;
    });
    return `${pollLines.length ? `Open polls:\n${pollLines.join('\n')}` : 'No open polls.'}${voting.length ? `\nActivities waiting on vo
tes: ${voting.map((a) => a.title).join(', ')}.` : ''}`;
  }

  if (has(/task|checklist|to-?do|remaining|left to do/)) {
    const open = trip.tasks.filter((k) => !k.done);
    if (!open.length) return 'The checklist is all done ✅';
    return `${open.length} task${open.length > 1 ? 's' : ''} left:\n${open.map((k) => `• ${k.text}${k.assignee ? ` — ${first(trip, k.ass
ignee)}` : ''}`).join('\n')}`;
  }

  if (has(/who('s| is)|role|treasurer|navigator|photographer|members?/)) {
    return `Roles for ${trip.name}:\n${trip.members.map((m) => `• ${m.name.split(' ')[0]} — ${m.role}`).join('\n')}\nChange roles in Pro
file → Members & roles.`;
  }

  const dayMatch = q.match(/day\s*(\d{1,2})/);
  if (has(/next|today|tomorrow|schedule|itinerary|plan for|what('s| is) on/) || dayMatch) {
    const today = todayInTz(tz);
    let target = today;
    if (has(/tomorrow/)) target = addDays(today, 1);
    if (dayMatch) target = addDays(trip.startDate, Number(dayMatch[1]) - 1);
    if (has(/next/) && !dayMatch) {
      const nowP = partsInTz(Date.now(), tz);
      const upcoming = trip.activities
        .filter((a) => a.date > nowP.ymd || (a.date === nowP.ymd && a.time >= nowP.hhmm))
        .sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time))[0];
      if (!upcoming) return 'Nothing else is scheduled — time to plan something fun! Try “suggest something to do”.';
      return `Next up: ${upcoming.title} — ${dayLabel(upcoming.date)} at ${time12(upcoming.time)}, ${upcoming.location} (${durationLabel
(upcoming.durationMin)}). ${upcoming.reminder ? 'Reminder is on 🔔' : 'Turn on the bell to get a reminder.'}`;
    }
    if (target < trip.startDate) target = trip.startDate;
    const acts = trip.activities.filter((a) => a.date === target).sort((a, b) => a.time.localeCompare(b.time));
    if (!acts.length) return `${dayLabel(target)} is a free day so far. Want ideas? Ask “what should we do?”`;
    return `${dayLabel(target)}:\n${acts.map((a) => `• ${time12(a.time)} ${a.title}${a.status !== 'confirmed' ? ` (${a.status})` : ''}
`).join('\n')}`;
  }

  if (has(/what (should|can|could) we do|idea|suggest|recommend|bored|activity|things to do|adventure|museum|culture|nightlife|shoppin
g/)) {
    const planned = new Set(trip.activities.map((a) => a.placeId));
    const cat = has(/adventure|hike|outdoor/) ? 'adventure' : has(/museum|culture|temple|shrine/) ? 'culture' : has(/night|bar|karaoke/)
? 'nightlife' : has(/shop/) ? 'shopping' : undefined;
    const ideas = placesFor(trip.destinationId)
      .concat(trip.extraDestinationIds.flatMap((id) => placesFor(id)))
      .filter((p) => !planned.has(p.id) && (!cat || p.category === cat))
      .sort((a, b) => b.rating - a.rating)
      .slice(0, 3);
    if (!ideas.length) return 'You’ve already planned the top spots! Try the Recommendations survey for new destinations.';
    return `Ideas not on your itinerary yet:\n${ideas.map((p) => `• ${p.name} — ${p.category}, ${durationLabel(p.durationMin)}, ${p.cost
? `${money(p.cost)}/person` : 'free'} (★${p.rating})`).join('\n')}\nAdd one from the Nearby screen or put it to a vote.`;
  }

  if (has(/^(hi|hey|hello|yo|sup|good (morning|evening|afternoon))\b/)) {
    return `Hey! 👋 I’m your Trip Assistant for ${trip.name}. Ask me about the budget, weather, food nearby, flights, what’s next, or “c
onvert 50 USD to ${dest?.currency ?? 'EUR'}”.`;
  }

  return `I can help with: budget & who owes who, weather, food nearby, today’s/tomorrow’s plan, flights & arrivals, currency (“convert
20 USD”), time zones, phrases, packing and emergencies. Try one of the quick questions below!`;
}

function systemPrompt(ctx: AssistantContext): string {
  const { trip } = ctx;
```

```
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const name = (id: string) => trip.members.find((m) => m.id === id)?.name ?? id;
  const acts = trip.activities
    .slice()
    .sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time))
    .map((a) => `${a.date} ${a.time} | ${a.title} | ${a.location} | ${a.status} | $${a.costPerPerson}/pp | ${durationLabel(a.durationMi
n)}`)
    .join('\n');
  const flights = trip.flights.map((f) => `${name(f.memberId)} ${f.direction} ${f.flightNo} ${f.from}->${f.to} ${formatInTz(f.direction
=== 'arrival' ? f.arriveUtc : f.departUtc, tz)}`).join('\n');
  const w = ctx.weather;
  return [
    'You are the Trip Assistant inside TogetherWeGo, a group trip-planning app, replying in the group chat.',
    'Be friendly and concise: 1–4 short sentences or a short bullet list. Use the trip data below; do not invent bookings.',
    'You cannot edit the trip yourself — when someone wants a change, suggest it and point them to the right screen (Itinerary → Add Act
ivity, Budget → Add expense, Profile → polls/checklist).',
    '',
    `Trip: ${trip.name} — ${dest?.city}, ${dest?.country}; ${trip.startDate} to ${trip.endDate}; local time zone ${tz}; today (local)
${todayInTz(tz)}.`,
    `Members: ${trip.members.map((m) => `${m.name} (${m.role})`).join(', ')}.`,
    `Budget: spent $${Math.round(totalSpent(trip))} of $${trip.budget} cap. Expenses: ${trip.expenses.map((e) => `${e.title} $${e.amoun
t} (${e.category}, paid by ${name(e.paidBy)})`).join('; ')}.`,
    w ? `Weather now: ${w.current.temp}°C ${weatherInfo(w.current.code).label}; next days: ${w.daily.slice(0, 4).map((d) => `${d.date}
${d.min}-${d.max}°C rain ${d.rain}%`).join(', ')}.` : '',
    `Itinerary (local times):\n${acts || 'none yet'}`,
    flights ? `Flights:\n${flights}` : '',
    `Open tasks: ${trip.tasks.filter((k) => !k.done).map((k) => k.text).join(', ') || 'none'}.`,
  ]
    .filter(Boolean)
    .join('\n');
}

export async function askClaude(apiKey: string, question: string, history: ChatMessage[], ctx: AssistantContext): Promise<string> {
  // Plain fetch to the Messages API: the official SDK does not bundle for React Native.
  const messages: { role: 'user' | 'assistant'; content: string }[] = [];
  for (const m of history.filter((x) => x.kind !== 'system').slice(-12)) {
    if (m.ai) messages.push({ role: 'assistant', content: m.text });
    else messages.push({ role: 'user', content: `${ctx.trip.members.find((x) => x.id === m.authorId)?.name ?? 'Member'}: ${m.text}` });
  }
  while (messages.length && messages[0].role !== 'user') messages.shift();
  messages.push({ role: 'user', content: question });

  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), 45_000);
  try {
    const res = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'content-type': 'application/json', 'x-api-key': apiKey, 'anthropic-version': '2023-06-01' },
      body: JSON.stringify({ model: CLAUDE_MODEL, max_tokens: 1024, system: systemPrompt(ctx), messages }),
      signal: controller.signal,
    });
    if (!res.ok) throw new Error(`Claude ${res.status}`);
    const data = (await res.json()) as { stop_reason?: string; content?: { type: string; text?: string }[] };
    if (data.stop_reason === 'refusal') throw new Error('refused');
    const text = (data.content ?? []).flatMap((b) => (b.type === 'text' && b.text ? [b.text] : [])).join('\n').trim();
    if (!text) throw new Error('empty');
    return text;
  } finally {
    clearTimeout(timer);
  }
}
```

src/services/currency.ts

```
/**
 * Exchange rates from open.er-api.com (free, no key), cached for 6 hours and
 * persisted for offline use. Falls back to bundled reference rates.
 */
import { currencies } from '@/data/reference';
import { useAppStore } from '@/store/useAppStore';
import type { RatesSnapshot } from '@/store/types';

const CACHE_MS = 6 * 60 * 60 * 1000;

export function fallbackRates(): RatesSnapshot {
  const rates: Record<string, number> = {};
  currencies.forEach((c) => (rates[c.code] = c.perUsd));
  return { base: 'USD', rates, fetchedAt: 0, live: false };
}

export async function fetchRates(force = false): Promise<RatesSnapshot> {
  const cached = useAppStore.getState().rates;
  if (!force && cached?.live && Date.now() - cached.fetchedAt < CACHE_MS) return cached;
  try {
    const ctrl = new AbortController();
```

```
    const timer = setTimeout(() => ctrl.abort(), 8000);
    const res = await fetch('https://open.er-api.com/v6/latest/USD', { signal: ctrl.signal });
    clearTimeout(timer);
    const j = await res.json();
    if (j.result !== 'success' || !j.rates) throw new Error('bad response');
    const snap: RatesSnapshot = { base: 'USD', rates: j.rates, fetchedAt: Date.now(), live: true };
    useAppStore.getState().setRates(snap);
    return snap;
  } catch {
    return cached ?? fallbackRates();
  }
}

export function convert(amount: number, from: string, to: string, rates: RatesSnapshot | null): number {
  const r = rates?.rates ?? fallbackRates().rates;
  const f = r[from] ?? fallbackRates().rates[from] ?? 1;
  const t = r[to] ?? fallbackRates().rates[to] ?? 1;
  return (amount / f) * t;
}

export function useRates(): RatesSnapshot {
  return useAppStore((s) => s.rates) ?? fallbackRates();
}
```

src/services/files.ts

```
/**
 * Offline document vault. Picked files are copied into the app's private
 * document directory (native) so they stay readable with no connection.
 * On web, small files are kept as data URLs in local storage.
 */
import * as DocumentPicker from 'expo-document-picker';
import { Directory, File, Paths } from 'expo-file-system';
import * as ImagePicker from 'expo-image-picker';
import { Platform } from 'react-native';

export const MAX_WEB_BYTES = 2 * 1024 * 1024;
export const MAX_NATIVE_BYTES = 25 * 1024 * 1024;

export interface SavedFile {
  uri: string;
  name: string;
  size?: number;
  kind: 'pdf' | 'image' | 'file';
}

function kindOf(name: string, mime?: string | null): SavedFile['kind'] {
  if (mime?.startsWith('image/') || /\.(png|jpe?g|heic|webp|gif)$/i.test(name)) return 'image';
  if (mime === 'application/pdf' || /\.pdf$/i.test(name)) return 'pdf';
  return 'file';
}

async function toDataUrl(file: Blob): Promise<string> {
  return new Promise((resolve, reject) => {
    const r = new FileReader();
    r.onload = () => resolve(String(r.result));
    r.onerror = () => reject(r.error);
    r.readAsDataURL(file);
  });
}

async function persist(uri: string, name: string, webFile?: Blob | null): Promise<string> {
  if (Platform.OS === 'web') {
    if (webFile) return toDataUrl(webFile);
    return uri;
  }
  const dir = new Directory(Paths.document, 'vault');
  if (!dir.exists) dir.create({ intermediates: true, idempotent: true });
  const safe = `${Date.now()}-${name.replace(/[^\w.-]+/g, '_')}`;
  const dest = new File(dir, safe);
  await new File(uri).copy(dest);
  return dest.uri;
}

export class FileTooLargeError extends Error {}

/** Lets the user pick a PDF/image/any file and saves an offline copy. */
export async function pickDocument(): Promise<SavedFile | null> {
  const res = await DocumentPicker.getDocumentAsync({ copyToCacheDirectory: true, multiple: false, type: ['application/pdf', 'image/*',
'*/*'] });
  if (res.canceled || !res.assets?.[0]) return null;
  const a = res.assets[0];
  const limit = Platform.OS === 'web' ? MAX_WEB_BYTES : MAX_NATIVE_BYTES;
  if (a.size && a.size > limit) throw new FileTooLargeError(`Files must be under ${Math.round(limit / 1024 / 1024)} MB`);
  const uri = await persist(a.uri, a.name, (a as { file?: Blob }).file ?? null);
  return { uri, name: a.name, size: a.size, kind: kindOf(a.name, a.mimeType) };
```

```
}

/** Picks a photo from the library (or camera) and keeps an offline copy. */
export async function pickImage(fromCamera = false): Promise<SavedFile | null> {
  if (fromCamera) {
    const perm = await ImagePicker.requestCameraPermissionsAsync();
    if (!perm.granted) throw new Error('Camera permission is needed to take photos.');
  }
  const opts: ImagePicker.ImagePickerOptions = { mediaTypes: ['images'], quality: 0.7, allowsEditing: false };
  const res = fromCamera ? await ImagePicker.launchCameraAsync(opts) : await ImagePicker.launchImageLibraryAsync(opts);
  if (res.canceled || !res.assets?.[0]) return null;
  const a = res.assets[0];
  const name = a.fileName ?? `photo-${Date.now()}.jpg`;
  if (Platform.OS === 'web') {
    if (a.fileSize && a.fileSize > MAX_WEB_BYTES) throw new FileTooLargeError('Photos must be under 2 MB in the web preview');
    return { uri: a.uri, name, size: a.fileSize, kind: 'image' };
  }
  const uri = await persist(a.uri, name);
  return { uri, name, size: a.fileSize, kind: 'image' };
}

export function deleteLocalFile(uri?: string) {
  if (!uri || Platform.OS === 'web' || !uri.startsWith('file:')) return;
  try {
    const f = new File(uri);
    if (f.exists) f.delete();
  } catch {
    // already gone
  }
}

export function sizeLabel(bytes?: number) {
  if (!bytes) return '';
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${Math.round(bytes / 1024)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(1)} MB`;
}
```

src/services/location.ts

```
/** Device location with graceful fallbacks (permission denied, web, timeouts). */
import * as Location from 'expo-location';

import type { LatLng } from '@/utils/geo';

export interface LocationResult extends LatLng {
  source: 'device' | 'fallback';
}

/** The last real fix, reused for a couple of minutes so screens that ask often do not ask the phone every time. */
let recent: { at: number; fix: LocationResult } | null = null;

export async function getLocation(fallback: LatLng): Promise<LocationResult> {
  if (recent && Date.now() - recent.at < 2 * 60 * 1000) return recent.fix;
  try {
    const perm = await Location.requestForegroundPermissionsAsync();
    if (perm.status !== 'granted') return { ...fallback, source: 'fallback' };
    const last = await Location.getLastKnownPositionAsync().catch(() => null);
    const pos =
      last ??
      (await Promise.race([
        Location.getCurrentPositionAsync({ accuracy: Location.Accuracy.Balanced }),
        new Promise<null>((r) => setTimeout(() => r(null), 8000)),
      ]));
    if (!pos) return { ...fallback, source: 'fallback' };
    const fix: LocationResult = { lat: pos.coords.latitude, lng: pos.coords.longitude, source: 'device' };
    recent = { at: Date.now(), fix };
    return fix;
  } catch {
    return { ...fallback, source: 'fallback' };
  }
}
```

src/services/notifications.ts

```
/**
 * Live activity reminders.
 * - iOS / Android: real local notifications scheduled with expo-notifications
 *   (they fire even when the app is closed).
 * - Every platform: an in-app watcher adds due reminders to the notification
 *   centre and, on web, shows a toast + browser notification.
 */
import * as Notifications from 'expo-notifications';
import { router } from 'expo-router';
import { Platform } from 'react-native';
```

```
import { toast } from '@/components/ui/feedback';
import { destinationById } from '@/data/destinations';
import { TRIVIA_REMINDER } from '@/data/personal';
import { tripStatus, useAppStore } from '@/store/useAppStore';
import type { Activity, Trip } from '@/store/types';
import { time12, zonedToUtc } from '@/utils/time';

const isNative = Platform.OS !== 'web';
let initialised = false;

export function initNotifications() {
  if (initialised || !isNative) return;
  initialised = true;
  Notifications.setNotificationHandler({
    handleNotification: async () => ({ shouldShowBanner: true, shouldShowList: true, shouldPlaySound: true, shouldSetBadge: false }),
  });
  if (Platform.OS === 'android') {
    Notifications.setNotificationChannelAsync('reminders', {
      name: 'Activity reminders',
      importance: Notifications.AndroidImportance.HIGH,
      vibrationPattern: [0, 200, 120, 200],
      lightColor: '#FBC91B',
    }).catch(() => {});
  }
  Notifications.addNotificationResponseReceivedListener((resp) => {
    const route = resp.notification.request.content.data?.route;
    if (typeof route === 'string') setTimeout(() => router.push(route as never), 300);
  });
}

export async function ensurePermission(): Promise<boolean> {
  if (!isNative) {
    // Browsers may leave the prompt unanswered forever, so never block on it:
    // ask in the background and rely on in-app toasts until it's granted.
    if (typeof window !== 'undefined' && 'Notification' in window) {
      if (Notification.permission === 'granted') return true;
      if (Notification.permission === 'default') Notification.requestPermission().catch(() => {});
    }
    return false;
  }
  try {
    const cur = await Notifications.getPermissionsAsync();
    if (cur.granted) return true;
    const req = await Notifications.requestPermissionsAsync();
    return req.granted;
  } catch {
    return false;
  }
}

/** Puts the daily trivia nudge on the phone's calendar (or takes it off). A no-op on the web. */
async function scheduleTrivia(on: boolean): Promise<void> {
  if (!isNative) return;
  try {
    await Notifications.cancelScheduledNotificationAsync(TRIVIA_REMINDER.id);
  } catch {
    // it was not scheduled
  }
  if (!on) return;
  await Notifications.scheduleNotificationAsync({
    identifier: TRIVIA_REMINDER.id,
    content: { title: TRIVIA_REMINDER.title, body: TRIVIA_REMINDER.body, data: { route: '/(tabs)/explore' } },
    trigger: { type: Notifications.SchedulableTriggerInputTypes.DAILY, hour: TRIVIA_REMINDER.hour, minute: TRIVIA_REMINDER.minute, chann
elId: 'reminders' },
  });
}

/**
 * Switches the daily trivia reminder on or off: one local notification a day, at the same time every day.
 * Asks the phone for permission when turning it on and returns false if that was refused (or anything failed).
 * On the web there are no scheduled notifications, so it does nothing and says yes.
 */
export async function setTriviaReminder(on: boolean): Promise<boolean> {
  if (!isNative) return true;
  try {
    if (on && !(await ensurePermission())) return false;
    await scheduleTrivia(on);
    return true;
  } catch {
    return false;
  }
}

export function activityStartUtc(trip: Trip, a: Activity): number {
  const tz = destinationById(trip.destinationId)?.tz ?? 'UTC';
  return zonedToUtc(a.date, a.time, tz);
```

```
}

export function reminderBody(trip: Trip, a: Activity, lead: number) {
  return {
    title: `${a.title} in ${lead} min`,
    body: `${time12(a.time)} · ${a.location} — ${trip.name}`,
  };
}

/** Reschedules every reminder for the signed-in user's upcoming activities. */
export async function syncReminders(): Promise<number> {
  const s = useAppStore.getState();
  if (!isNative) return 0;
  try {
    await Notifications.cancelAllScheduledNotificationsAsync();
  } catch {
    return 0;
  }
  // Cancelling everything above also removed the daily trivia nudge, so put it back (it does not depend on the activity reminders).
  const granted = (await Notifications.getPermissionsAsync().catch(() => null))?.granted;
  await scheduleTrivia(!!granted && s.personal.triviaReminder).catch(() => {});
  if (!s.settings.notificationsEnabled || !s.session) return 0;
  if (!granted) return 0;
  const lead = s.settings.reminderLeadMin;
  const now = Date.now();
  const due: { at: number; trip: Trip; a: Activity }[] = [];
  for (const trip of s.trips) {
    if (!trip.members.some((m) => m.id === s.session?.userId) || tripStatus(trip) === 'completed') continue;
    for (const a of trip.activities) {
      if (!a.reminder) continue;
      const at = activityStartUtc(trip, a) - lead * 60000;
      if (at > now + 5000 && at < now + 45 * 86400000) due.push({ at, trip, a });
    }
  }
  due.sort((x, y) => x.at - y.at);
  for (const { at, trip, a } of due.slice(0, 48)) {
    const { title, body } = reminderBody(trip, a, lead);
    await Notifications.scheduleNotificationAsync({
      content: { title, body, data: { route: '/(tabs)/itinerary', tripId: trip.id, activityId: a.id } },
      trigger: { type: Notifications.SchedulableTriggerInputTypes.DATE, date: new Date(at), channelId: 'reminders' },
    }).catch(() => {});
  }
  return Math.min(due.length, 48);
}

function browserNotify(title: string, body: string) {
  try {
    if (!isNative && typeof window !== 'undefined' && 'Notification' in window && Notification.permission === 'granted') {
      new Notification(title, { body, icon: '/favicon.ico' });
    }
  } catch {
    // ignore
  }
}

/** Sends a sample reminder in a few seconds so the feature can be demoed live. */
export async function sendTestReminder(seconds = 5) {
  const s = useAppStore.getState();
  const trip = s.trips.find((x) => x.id === s.activeTripId);
  const a = trip?.activities.find((x) => x.reminder) ?? trip?.activities[0];
  const title = a ? `${a.title} starts soon` : 'Test reminder';
  const body = a && trip ? `${time12(a.time)} · ${a.location} — ${trip.name}` : 'Reminders are working!';
  await ensurePermission();
  if (isNative) {
    await Notifications.scheduleNotificationAsync({
      content: { title, body, data: { route: '/(tabs)/itinerary' } },
      trigger: { type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL, seconds, channelId: 'reminders' },
    }).catch(() => {});
  }
  setTimeout(() => {
    useAppStore.getState().pushNotification({ kind: 'reminder', title, body, tripId: trip?.id, route: '/(tabs)/itinerary' });
    if (!isNative) {
      toast(`${title} — ${body}`, { icon: 'alarm-outline', tone: 'info' });
      browserNotify(title, body);
    }
  }, seconds * 1000);
}

/** Called on an interval while the app is open: surfaces reminders that just came due. */
export function checkDueReminders() {
  const s = useAppStore.getState();
  if (!s.session || !s.settings.notificationsEnabled) return;
  const lead = s.settings.reminderLeadMin;
  const now = Date.now();
  for (const trip of s.trips) {
    if (!trip.members.some((m) => m.id === s.session?.userId)) continue;
```

```
    for (const a of trip.activities) {
      if (!a.reminder) continue;
      const start = activityStartUtc(trip, a);
      const fireAt = start - lead * 60000;
      const key = `${a.id}@${a.date}T${a.time}-${lead}`;
      if (now >= fireAt && now < start && !s.firedReminders[key]) {
        s.markReminderFired(key);
        const { title, body } = reminderBody(trip, a, Math.max(1, Math.round((start - now) / 60000)));
        s.pushNotification({ kind: 'reminder', title, body, tripId: trip.id, route: '/(tabs)/itinerary' });
        if (!isNative) {
          toast(`${title}`, { icon: 'alarm-outline', tone: 'info' });
          browserNotify(title, body);
        }
      }
    }
  }
}

/** The next upcoming reminder-enabled activity across the active trip (for the "live" banner). */
export function nextUpcoming(trip: Trip | undefined): { a: Activity; startUtc: number } | null {
  if (!trip) return null;
  const now = Date.now();
  let best: { a: Activity; startUtc: number } | null = null;
  for (const a of trip.activities) {
    const st = activityStartUtc(trip, a);
    if (st > now - 30 * 60000 && (!best || st < best.startUtc)) best = { a, startUtc: st };
  }
  return best;
}
```

src/services/placeSearch.ts

```
/**
 * Finds places for the "Where do you want to go?" box. Two sources:
 *  - the trip's own saved places: instant, works with no connection;
 *  - OpenStreetMap places from Photon (free, no account or key), searched around the trip first and anywhere if nothing is near.
 * Only the words you typed and a rounded map point (about 1 km) are sent. Distances are worked out on the phone.
 */
import { destinationById } from '@/data/destinations';
import { placeToActivityType, placesFor, type Place } from '@/data/places';
import type { ActivityType } from '@/store/types';
import { distanceKm, type LatLng } from '@/utils/geo';

export interface PlaceHit {
  id: string;
  name: string;
  /** Street or area, e.g. "Via Pasitea 172, Positano". */
  detail: string;
  lat: number;
  lng: number;
  type: ActivityType;
  source: 'saved' | 'online';
  /** Only for saved places: the id in our own data and the usual visit length and price. */
  placeId?: string;
  durationMin?: number;
  costPerPerson?: number;
}

export const PHOTON_URL = 'https://photon.komoot.io/api/';
export const MAX_HITS = 7;

/** "The museum" -> "museum": leading articles only get in the way of a search. */
export function normalizeQuery(text: string): string {
  const q = text.trim().replace(/\s+/g, ' ').replace(/^(the|a|an)\s+/i, '');
  return /^(the|a|an)$/i.test(q) ? '' : q;
}

const escapeRe = (s: string) => s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

function savedScore(p: Place, q: string): number {
  const name = p.name.toLowerCase();
  if (name.startsWith(q)) return 4;
  if (name.split(/[\s,\-—]+/).some((w) => w.startsWith(q))) return 3;
  if (name.includes(q)) return 2;
  if (p.area.toLowerCase().includes(q) || new RegExp(`\\b${escapeRe(q)}\\b`).test(p.blurb.toLowerCase())) return 1;
  return 0;
}

function savedHit(p: Place): PlaceHit {
  const city = destinationById(p.destinationId)?.city;
  return {
    id: `saved:${p.id}`,
    name: p.name,
    detail: [...new Set([p.area, city].filter((x): x is string => !!x))].join(', '),
    lat: p.lat,
    lng: p.lng,
```

```
    type: placeToActivityType[p.category],
    source: 'saved',
    placeId: p.id,
    durationMin: p.durationMin,
    costPerPerson: p.cost,
  };
}

/** The trip's saved places that match what was typed (best first). Instant and offline. */
export function searchSaved(query: string, destinationIds: string[]): PlaceHit[] {
  const q = normalizeQuery(query).toLowerCase();
  if (q.length < 2) return [];
  const places = new Map(destinationIds.flatMap((id) => placesFor(id)).map((p) => [p.id, p] as const));
  return [...places.values()]
    .map((p) => ({ p, score: savedScore(p, q) }))
    .filter((x) => x.score > 0)
    .sort((a, b) => b.score - a.score || b.p.rating - a.p.rating)
    .slice(0, 4)
    .map((x) => savedHit(x.p));
}

// ---- online

interface PhotonFeature {
  geometry?: { coordinates?: number[] };
  properties?: Record<string, unknown>;
}

/** Streets and bus stops match almost any word but are never a plan. */
const NOT_A_PLAN = new Set(['highway', 'public_transport', 'boundary', 'waterway', 'building', 'landuse']);

const str = (v: unknown) => (typeof v === 'string' || typeof v === 'number' ? String(v).trim() : '');

/** What kind of activity an OpenStreetMap place is, from its tag (for example tourism=museum). */
export function typeFromOsm(key: string, value: string): ActivityType {
  const v = value.toLowerCase();
  switch (key) {
    case 'amenity':
      if (/^(bar|pub|nightclub|biergarten|casino)$/.test(v)) return 'nightlife';
      if (/^(restaurant|cafe|fast_food|food_court|ice_cream|bakery)$/.test(v)) return 'food';
      if (/^(place_of_worship|theatre|arts_centre|library|cinema|townhall)$/.test(v)) return 'culture';
      return 'sightseeing';
    case 'tourism':
      if (/^(hotel|hostel|guest_house|apartment|motel|chalet|camp_site)$/.test(v)) return 'stay';
      if (/^(museum|gallery|artwork)$/.test(v)) return 'culture';
      return 'sightseeing';
    case 'historic':
      return 'culture';
    case 'shop':
    case 'craft':
      return v === 'bakery' || v === 'pastry' || v === 'confectionery' ? 'food' : 'shopping';
    case 'railway':
    case 'aeroway':
      return 'transport';
    case 'leisure':
      return /^(sports_centre|pitch|swimming_pool|marina|water_park)$/.test(v) ? 'adventure' : 'sightseeing';
    case 'natural':
      return /^(peak|cave_entrance|volcano|cliff)$/.test(v) ? 'adventure' : 'sightseeing';
    default:
      return 'sightseeing';
  }
}

function fromFeature(f: PhotonFeature): PlaceHit | null {
  const p = f.properties ?? {};
  const [lng, lat] = f.geometry?.coordinates ?? [];
  const name = str(p.name);
  const key = str(p.osm_key);
  if (!name || typeof lat !== 'number' || typeof lng !== 'number' || NOT_A_PLAN.has(key)) return null;
  const street = [str(p.street), str(p.housenumber)].filter(Boolean).join(' ');
  const town = str(p.city) || str(p.district) || str(p.county) || str(p.state);
  const parts = [...new Set([street, town].filter((x) => x && x !== name))];
  return {
    id: `online:${str(p.osm_type)}${str(p.osm_id) || `${lat.toFixed(5)},${lng.toFixed(5)}`}`,
    name,
    detail: (parts.length ? parts : [str(p.country)].filter(Boolean)).join(', '),
    lat,
    lng,
    type: typeFromOsm(key, str(p.osm_value)),
    source: 'online',
  };
}

/** [west, south, east, north] of a square around `c`, `radiusKm` out in every direction. */
export function boxAround(c: LatLng, radiusKm: number): [number, number, number, number] {
  const dLat = radiusKm / 110.57;
```

```
  const dLng = radiusKm / (111.32 * Math.max(0.2, Math.cos((c.lat * Math.PI) / 180)));
  return [c.lng - dLng, c.lat - dLat, c.lng + dLng, c.lat + dLat];
}

export interface OnlineOptions {
  signal?: AbortSignal;
  /** For tests. */
  fetchFn?: typeof fetch;
  /** How far around the trip to look first. */
  radiusKm?: number;
  timeoutMs?: number;
}

const r2 = (n: number) => Math.round(n * 100) / 100;
const cache = new Map<string, PlaceHit[]>();

async function ask(q: string, center: LatLng | null, boxed: boolean, o: OnlineOptions): Promise<PlaceHit[]> {
  const params = [`q=${encodeURIComponent(q)}`, 'limit=6', 'lang=en'];
  if (center) params.push(`lat=${r2(center.lat)}`, `lon=${r2(center.lng)}`);
  if (center && boxed) params.push(`bbox=${boxAround(center, o.radiusKm ?? 40).map((n) => n.toFixed(3)).join(',')}`);

  // one signal that stops on the caller's cancel or on our own time limit
  const stop = new AbortController();
  const cancel = () => stop.abort();
  if (o.signal?.aborted) cancel();
  o.signal?.addEventListener('abort', cancel);
  const timer = setTimeout(cancel, o.timeoutMs ?? 6000);
  try {
    const res = await (o.fetchFn ?? fetch)(`${PHOTON_URL}?${params.join('&')}`, { signal: stop.signal });
    if (!res.ok) throw new Error(`Place search answered ${res.status}`);
    const json = (await res.json()) as { features?: PhotonFeature[] };
    return (json.features ?? []).map(fromFeature).filter((h): h is PlaceHit => !!h);
  } finally {
    clearTimeout(timer);
    o.signal?.removeEventListener('abort', cancel);
  }
}

/**
 * Places from OpenStreetMap that match what was typed, looked for around `center` first and anywhere if none are there.
 * Rejects when there is no connection or the service is down.
 */
export async function searchOnline(query: string, center: LatLng | null, o: OnlineOptions = {}): Promise<PlaceHit[]> {
  const q = normalizeQuery(query);
  if (q.length < 2) return [];
  const key = `${q.toLowerCase()}|${center ? `${r2(center.lat)}|${r2(center.lng)}` : 'anywhere'}`;
  const known = cache.get(key);
  if (known) return known;
  const near = center ? await ask(q, center, true, o) : [];
  const hits = near.length ? near : await ask(q, center, false, o);
  cache.set(key, hits);
  if (cache.size > 60) cache.delete(cache.keys().next().value as string);
  return hits;
}

const plain = (s: string) => s.toLowerCase().replace(/[\s\-——.,'’"()]+/g, ' ').trim();

/** Saved places first, then the online ones, without showing the same place twice. */
export function mergeHits(saved: PlaceHit[], online: PlaceHit[]): PlaceHit[] {
  const same = (a: PlaceHit, b: PlaceHit) => {
    const x = plain(a.name);
    const y = plain(b.name);
    return distanceKm(a, b) < 0.4 && (x.includes(y) || y.includes(x));
  };
  const out = [...saved];
  for (const h of online) if (!out.some((o) => o.id === h.id || same(o, h))) out.push(h);
  return out.slice(0, MAX_HITS);
}

const GUESSES: [RegExp, ActivityType][] = [
  [/\b(flight|airport|train|bus|ferry|taxi|transfer|station|metro|drive|rental)\b/, 'transport'],
  [/\b(hotel|hostel|check[- ]?in|check[- ]?out|airbnb|apartment|resort)\b/, 'stay'],
  [/\b(dinner|lunch|breakfast|brunch|restaurant|pizza|pasta|cafe|café|coffee|gelato|ice cream|bakery|food|eat|tacos|sushi|ramen|bbq)\b/,
'food'],
  [/\b(bar|pub|club|drinks|cocktails?|aperitivo|nightlife|party|karaoke)\b/, 'nightlife'],
  [/\b(museum|gallery|church|cathedral|temple|shrine|palace|castle|ruins|monument|theat(er|re)|opera|historic)\b/, 'culture'],
  [/\b(hike|hiking|trail|climb|kayak|surf|dive|diving|snorkel|boat|bike|ski|zipline)\b/, 'adventure'],
  [/\b(shop|shopping|market|mall|boutique|souvenirs?|store)\b/, 'shopping'],
];

/** The likeliest activity type for words typed by hand ("dinner at 7" is food). */
export function guessType(text: string): ActivityType {
  const t = text.toLowerCase();
  return GUESSES.find(([re]) => re.test(t))?.[1] ?? 'sightseeing';
}
```

src/services/pretendFriends.ts

```
/**
 * The demo crew: friendly canned replies so a trip with no real friends on the other end still feels alive.
 * Never used in a trip that is shared across phones (the real friends answer there).
 */
export function friendReply(text: string, random: () => number = Math.random): string {
  const q = text.toLowerCase();
  if (/food|eat|dinner|lunch|ramen|sushi|hungry/.test(q)) return 'I’m always down for food 😋 count me in!';
  if (/\?$/.test(q.trim())) return 'Good question — let’s ask the assistant or put it to a vote 🗳️';
  if (/late|delay|wait/.test(q)) return 'No stress, we’ll save you a seat 🙌';
  if (/photo|pic|camera/.test(q)) return 'Drop them in the photo dump! 📸';
  if (/money|pay|owe|\$/.test(q)) return 'I’ll log it in Budget so we can settle up later 💸';
  const pool = ['Love it 🔥', 'Works for me 👍', 'Yesss can’t wait!! ✈️', 'Adding it to my calendar 📅', 'Haha same 😂', 'Let’s goooo
🙌'];
  return pool[Math.floor(random() * pool.length)];
}

/** Picks a random group member to reply (70% of the time) and a natural-feeling delay in milliseconds. */
export function planFriendReply<M>(others: M[], random: () => number = Math.random): { friend: M; delay: number } | null {
  if (!others.length || random() >= 0.7) return null;
  return { friend: others[Math.floor(random() * others.length)], delay: 2600 + random() * 1200 };
}
```

src/services/print.ts

```
/** Printable trip planner: builds a styled HTML document and prints / saves it as PDF. */
import * as Print from 'expo-print';
import * as Sharing from 'expo-sharing';
import { Platform } from 'react-native';

import { destinationById } from '@/data/destinations';
import { emergencyByCountry } from '@/data/reference';
import { totalSpent } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { durationLabel, money } from '@/utils/format';
import { dayLabel, formatInTz, rangeLabel, time12, tripDays, tzAbbrev, weekdayShort } from '@/utils/time';

const esc = (s: string) => s.replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[c]!);

export function plannerHtml(trip: Trip): string {
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const name = (id: string) => trip.members.find((m) => m.id === id)?.name ?? 'Member';
  const days = tripDays(trip.startDate, trip.endDate)
    .map((d, i) => {
      const acts = trip.activities.filter((a) => a.date === d).sort((a, b) => a.time.localeCompare(b.time));
      const rows = acts.length
        ? acts
            .map(
              (a) => `<tr><td class="time">${time12(a.time)}</td><td><b>${esc(a.title)}</b><div class="muted">${esc(a.location)} · ${dur
ationLabel(a.durationMin)}${a.costPerPerson ? ` · ${money(a.costPerPerson)}/person` : ''}</div>${a.notes ? `<div class="note">${esc(a.no
tes)}</div>` : ''}</td><td class="who">${a.assigned.map((x) => esc(name(x).split(' ')[0])).join(', ')}</td><td class="box">□</td></tr>`,
            )
            .join('')
        : '<tr><td colspan="4" class="muted">Free day — explore at your own pace</td></tr>';
      return `<section class="day"><h3>Day ${i + 1} · ${weekdayShort(d)} ${dayLabel(d)}</h3><table>${rows}</table></section>`;
    })
    .join('');
  const flights = trip.flights
    .slice()
    .sort((a, b) => a.arriveUtc - b.arriveUtc)
    .map((f) => `<tr><td>${esc(name(f.memberId))}</td><td>${esc(f.flightNo)} · ${f.from}→${f.to}</td><td>${f.direction === 'arrival' ?
'Arrives' : 'Departs'} ${formatInTz(f.direction === 'arrival' ? f.arriveUtc : f.departUtc, tz)}</td></tr>`)
    .join('');
  const em = emergencyByCountry[dest?.countryCode ?? ''] ?? emergencyByCountry.DEFAULT;
  const spent = totalSpent(trip);
  return `<!doctype html><html><head><meta charset="utf-8"><title>${esc(trip.name)} — Planner</title>
<style>
  *{box-sizing:border-box} body{font-family:-apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif;color:#1A1C17;margin:28px;font-siz
e:12.5px}
  header{display:flex;justify-content:space-between;align-items:flex-end;border-bottom:3px solid #1F4D25;padding-bottom:10px;margin-bott
om:16px}
  h1{margin:0;font-size:24px;color:#1F4D25} h2{font-size:15px;margin:18px 0 6px;color:#1F4D25} h3{font-size:13px;margin:14px 0 6px;backg
round:#EAF5E4;padding:6px 10px;border-radius:6px}
  table{width:100%;border-collapse:collapse} td{padding:6px 8px;border-bottom:1px solid #E7E9DF;vertical-align:top}
  .time{width:70px;font-weight:600;white-space:nowrap} .who{width:110px;color:#5F645A} .box{width:20px;font-size:15px}
  .muted{color:#7A7F73;font-size:11.5px} .note{margin-top:3px;font-style:italic;color:#5F645A}
  .grid{display:grid;grid-template-columns:repeat(3,1fr);gap:8px} .stat{background:#F4F4EE;border-radius:8px;padding:8px 10px}
  .stat b{display:block;font-size:16px;color:#1F4D25} .day{page-break-inside:avoid} footer{margin-top:18px;color:#959A8E;font-size:10.5p
x;text-align:center}
</style></head><body>
<header><div><h1>${esc(trip.name)}</h1><div class="muted">${esc(dest ? `${dest.city}, ${dest.country}` : '')} · ${rangeLabel(trip.startD
ate, trip.endDate)} · times in ${tzAbbrev(tz)}</div></div><div class="muted">Invite code <b>${trip.inviteCode}</b></div></header>
<div class="grid">
```

```
  <div class="stat">Travellers<b>${trip.members.length}</b>${trip.members.map((m) => esc(m.name.split(' ')[0]) + ' (' + esc(m.role) +
')').join(', ')}</div>
  <div class="stat">Budget<b>${money(spent)} / ${money(trip.budget)}</b>${money(trip.budget / Math.max(1, trip.members.length))} per per
son</div>
  <div class="stat">Emergency<b>Police ${em.police} · Ambulance ${em.ambulance}</b>${esc(em.tips[0] ?? '')}</div>
</div>
<h2>Day-by-day itinerary</h2>${days}
${flights ? `<h2>Flights & arrivals</h2><table>${flights}</table>` : ''}
<h2>Checklist</h2><table>${trip.tasks.map((k) => `<tr><td class="box">${k.done ? '☑' : '☐'}</td><td>${esc(k.text)}</td><td class="who">
${k.assignee ? esc(name(k.assignee).split(' ')[0]) : 'Everyone'}</td></tr>`).join('')}</table>
<footer>Printed from TogetherWeGo · Plan together, travel better</footer>
</body></html>`;
}

export async function printPlanner(trip: Trip) {
  const html = plannerHtml(trip);
  if (Platform.OS === 'web') {
    const w = window.open('', '_blank');
    if (!w) throw new Error('Pop-up blocked');
    w.document.write(html);
    w.document.close();
    w.focus();
    setTimeout(() => w.print(), 350);
    return;
  }
  await Print.printAsync({ html });
}

export async function sharePlannerPdf(trip: Trip) {
  if (Platform.OS === 'web') return printPlanner(trip);
  const { uri } = await Print.printToFileAsync({ html: plannerHtml(trip) });
  if (await Sharing.isAvailableAsync()) {
    await Sharing.shareAsync(uri, { mimeType: 'application/pdf', dialogTitle: `${trip.name} planner`, UTI: 'com.adobe.pdf' });
  }
}
```

src/services/secrets.ts

```
/**
 * Sensitive values (the optional Anthropic API key) live in the OS keychain /
 * keystore via expo-secure-store — never in the regular app database.
 * Web has no secure enclave, so the preview keeps it in session storage only.
 */
import * as SecureStore from 'expo-secure-store';
import { Platform } from 'react-native';

const KEY = 'twg_anthropic_key';

export async function getApiKey(): Promise<string | null> {
  try {
    if (Platform.OS === 'web') return typeof sessionStorage !== 'undefined' ? sessionStorage.getItem(KEY) : null;
    return await SecureStore.getItemAsync(KEY);
  } catch {
    return null;
  }
}

export async function setApiKey(value: string | null): Promise<void> {
  if (Platform.OS === 'web') {
    if (typeof sessionStorage === 'undefined') return;
    if (value) sessionStorage.setItem(KEY, value);
    else sessionStorage.removeItem(KEY);
    return;
  }
  if (value) await SecureStore.setItemAsync(KEY, value);
  else await SecureStore.deleteItemAsync(KEY);
}

export const looksLikeApiKey = (v: string) => /^sk-ant-[A-Za-z0-9_-]{20,}$/.test(v.trim());
```

src/services/share.ts

```
/**
 * Social media integration: native share sheet plus direct deep links into
 * WhatsApp, X, Facebook, Messenger, Telegram, Instagram, SMS and email.
 */
import { Asset } from 'expo-asset';
import * as Clipboard from 'expo-clipboard';
import * as Linking from 'expo-linking';
import * as Sharing from 'expo-sharing';
import type { RefObject } from 'react';
import { Platform, Share, type View } from 'react-native';
import { captureRef } from 'react-native-view-shot';

import { toast } from '@/components/ui/feedback';
import type { IconName } from '@/components/ui/bits';
```

```
import { photos, type PhotoKey } from '@/data/images';
import { buildInviteUrl, inviteText } from '@/sync/invite';
import { serverForInvites } from '@/sync/prefs';
import { useSyncStatus } from '@/sync/status';
import { useAppStore } from '@/store/useAppStore';

export type SocialTarget = 'whatsapp' | 'x' | 'facebook' | 'messenger' | 'telegram' | 'instagram' | 'sms' | 'email';

export const socialTargets: { key: SocialTarget; label: string; icon: IconName; color: string }[] = [
  { key: 'whatsapp', label: 'WhatsApp', icon: 'logo-whatsapp', color: '#25D366' },
  { key: 'instagram', label: 'Instagram', icon: 'logo-instagram', color: '#E1306C' },
  { key: 'x', label: 'X', icon: 'logo-twitter', color: '#111111' },
  { key: 'facebook', label: 'Facebook', icon: 'logo-facebook', color: '#1877F2' },
  { key: 'messenger', label: 'Messenger', icon: 'chatbubble-ellipses-outline', color: '#0084FF' },
  { key: 'telegram', label: 'Telegram', icon: 'paper-plane', color: '#229ED9' },
  { key: 'sms', label: 'Messages', icon: 'chatbox-ellipses-outline', color: '#34C759' },
  { key: 'email', label: 'Email', icon: 'mail-outline', color: '#5F645A' },
];

/** The sync server only goes into an invite when that trip is actually shared across phones. */
function serverFor(code: string): string | null {
  const trip = useAppStore.getState().trips.find((t) => t.inviteCode === code);
  return trip && useSyncStatus.getState().shared.includes(trip.id) ? serverForInvites() : null;
}

/** The link that opens TogetherWeGo on the join screen with the code (and the sync server, if the trip is shared) filled in. */
export function inviteLink(code: string): string {
  return buildInviteUrl(Linking.createURL('/join'), code, serverFor(code));
}

export function inviteMessage(tripName: string, code: string): string {
  return inviteText(tripName, code, inviteLink(code));
}

export async function copy(text: string, label = 'Copied to clipboard') {
  await Clipboard.setStringAsync(text);
  toast(label, { icon: 'copy-outline' });
}

/** Opens the OS share sheet (Instagram, Snapchat, iMessage, Slack… whatever is installed). */
export async function shareText(message: string, title = 'TogetherWeGo') {
  try {
    if (Platform.OS === 'web') {
      const nav = typeof navigator !== 'undefined' ? (navigator as Navigator & { share?: (d: object) => Promise<void> }) : undefined;
      if (nav?.share) {
        await nav.share({ title, text: message });
        return;
      }
      await copy(message, 'Share text copied — paste it anywhere');
      return;
    }
    await Share.share({ message, title });
  } catch {
    // user cancelled
  }
}

async function openFirst(urls: string[]) {
  for (const url of urls) {
    try {
      if (Platform.OS === 'web') {
        if (url.startsWith('http') || url.startsWith('mailto') || url.startsWith('sms')) {
          window.open(url, '_blank');
          return true;
        }
        continue;
      }
      if (url.startsWith('http') || (await Linking.canOpenURL(url))) {
        await Linking.openURL(url);
        return true;
      }
    } catch {
      // try next
    }
  }
  return false;
}

/** Shares straight into a specific social app, falling back to its web intent. */
export async function shareTo(target: SocialTarget, text: string, url?: string) {
  const t = encodeURIComponent(text);
  const u = encodeURIComponent(url ?? '');
  const map: Record<SocialTarget, string[]> = {
    whatsapp: [`whatsapp://send?text=${t}`, `https://wa.me/?text=${t}`],
    x: [`twitter://post?message=${t}`, `https://x.com/intent/tweet?text=${t}`],
    facebook: [`https://www.facebook.com/sharer/sharer.php?u=${u || encodeURIComponent('https://expo.dev')}&quote=${t}`],
```

```
    messenger: [`fb-messenger://share?link=${u}`, `https://www.messenger.com/`],
    telegram: [`tg://msg?text=${t}`, `https://t.me/share/url?url=${u}&text=${t}`],
    instagram: ['instagram://story-camera', 'instagram://app', 'https://www.instagram.com/'],
    sms: [Platform.OS === 'ios' ? `sms:&body=${t}` : `sms:?body=${t}`],
    email: [`mailto:?subject=${encodeURIComponent('Join our trip on TogetherWeGo')}&body=${t}`],
  };
  if (target === 'instagram') {
    // Instagram has no text-share URL: copy the caption first, then open the app.
    await Clipboard.setStringAsync(text);
    toast('Caption copied — paste it in Instagram', { icon: 'logo-instagram' });
  }
  const ok = await openFirst(map[target]);
  if (!ok) await shareText(text);
}

/** Shares a photo file (bundled demo photo or one from the camera roll). */
export async function sharePhoto(uri: string, caption: string) {
  try {
    let fileUri = uri;
    if (uri in photos) {
      const asset = Asset.fromModule(photos[uri as PhotoKey]);
      await asset.downloadAsync();
      fileUri = asset.localUri ?? asset.uri;
    }
    if (Platform.OS !== 'web' && (await Sharing.isAvailableAsync())) {
      await Sharing.shareAsync(fileUri, { dialogTitle: caption, mimeType: 'image/jpeg' });
      return;
    }
    await shareText(`${caption} — shared from TogetherWeGo`);
  } catch {
    await shareText(`${caption} — shared from TogetherWeGo`);
  }
}

/**
 * Turns something on screen (like the recap story card) into a picture and opens the phone's share sheet with it, so it
 * can go to Instagram, Snapchat, Messages or anywhere else that takes images. Falls back to plain text.
 */
export async function shareView(view: RefObject<View | null>, title: string, fallbackText: string) {
  try {
    if (Platform.OS === 'web' || !(await Sharing.isAvailableAsync())) return await shareText(fallbackText);
    const uri = await captureRef(view, { format: 'png', quality: 1, result: 'tmpfile' });
    await Sharing.shareAsync(uri, { mimeType: 'image/png', UTI: 'public.png', dialogTitle: title });
  } catch {
    await shareText(fallbackText);
  }
}
```

src/services/weather.ts

```
/**
 * Weather via Open-Meteo (free, no API key). Results are cached in the store
 * so the last forecast stays visible offline; if the network is down on first
 * load we fall back to a typical-climate estimate and label it as such.
 */
import { destinationById, type Destination } from '@/data/destinations';
import type { IconName } from '@/components/ui/bits';
import { useAppStore } from '@/store/useAppStore';
import type { WeatherSnapshot } from '@/store/types';
import { addDays, todayInTz } from '@/utils/time';

const CACHE_MS = 30 * 60 * 1000;

export function weatherInfo(code: number): { label: string; icon: IconName } {
  if (code === 0) return { label: 'Clear sky', icon: 'sunny' };
  if (code <= 2) return { label: 'Partly cloudy', icon: 'partly-sunny' };
  if (code === 3) return { label: 'Overcast', icon: 'cloud-outline' };
  if (code <= 48) return { label: 'Foggy', icon: 'cloud-outline' };
  if (code <= 57) return { label: 'Drizzle', icon: 'rainy-outline' };
  if (code <= 67) return { label: 'Rain', icon: 'rainy-outline' };
  if (code <= 77) return { label: 'Snow', icon: 'snow-outline' };
  if (code <= 82) return { label: 'Rain showers', icon: 'rainy-outline' };
  if (code <= 86) return { label: 'Snow showers', icon: 'snow-outline' };
  return { label: 'Thunderstorms', icon: 'thunderstorm-outline' };
}

function estimate(d: Destination): WeatherSnapshot {
  // Rough climate model: warmer near the equator, seasonal swing by hemisphere.
  const month = new Date().getUTCMonth();
  const north = d.lat >= 0;
  const season = Math.cos(((month - (north ? 6.5 : 0.5)) / 12) * 2 * Math.PI);
  const base = 27 - Math.abs(d.lat) * 0.42 + season * Math.min(12, Math.abs(d.lat) * 0.3);
  const today = todayInTz(d.tz);
  const codes = [1, 2, 3, 61, 2, 0, 80];
  return {
    destinationId: d.id,
```

```
    fetchedAt: Date.now(),
    live: false,
    current: { temp: Math.round(base), code: 2, wind: 11, humidity: 64 },
    daily: codes.map((code, i) => ({
      date: addDays(today, i),
      max: Math.round(base + 3 + ((i * 7) % 3)),
      min: Math.round(base - 5 + ((i * 5) % 3)),
      code,
      rain: code >= 61 ? 60 : code === 3 ? 25 : 10,
    })),
  };
}

export async function fetchWeather(destinationId: string, force = false): Promise<WeatherSnapshot | undefined> {
  const d = destinationById(destinationId);
  if (!d) return undefined;
  const cached = useAppStore.getState().weather[destinationId];
  if (!force && cached?.live && Date.now() - cached.fetchedAt < CACHE_MS) return cached;
  try {
    const url =
      `https://api.open-meteo.com/v1/forecast?latitude=${d.lat}&longitude=${d.lng}` +
      '&current=temperature_2m,relative_humidity_2m,weather_code,wind_speed_10m' +
      '&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto&forecast_days=7';
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 8000);
    const res = await fetch(url, { signal: ctrl.signal });
    clearTimeout(timer);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const j = await res.json();
    const snap: WeatherSnapshot = {
      destinationId,
      fetchedAt: Date.now(),
      live: true,
      current: {
        temp: Math.round(j.current.temperature_2m),
        code: j.current.weather_code,
        wind: Math.round(j.current.wind_speed_10m),
        humidity: Math.round(j.current.relative_humidity_2m),
      },
      daily: (j.daily.time as string[]).map((date, i) => ({
        date,
        max: Math.round(j.daily.temperature_2m_max[i]),
        min: Math.round(j.daily.temperature_2m_min[i]),
        code: j.daily.weather_code[i],
        rain: j.daily.precipitation_probability_max?.[i] ?? 0,
      })),
    };
    useAppStore.getState().setWeather(snap);
    return snap;
  } catch {
    if (cached) return cached;
    const est = estimate(d);
    useAppStore.getState().setWeather(est);
    return est;
  }
}

export function weatherSummary(w: WeatherSnapshot | undefined): string {
  if (!w) return 'Loading forecast…';
  const today = w.daily[0];
  const later = today && today.rain >= 50 ? ' · rain likely later' : today && today.rain >= 30 ? ' · chance of showers' : '';
  return `${weatherInfo(w.current.code).label.toLowerCase()}${later}`;
}

export const cToF = (c: number) => Math.round((c * 9) / 5 + 32);
```

src/store/__tests__/auth.test.ts

```
import { createHash } from 'crypto';
import { useAppStore } from '../useAppStore';

const reset = () => useAppStore.getState().resetEverything();

describe('accounts and sign-in', () => {
  beforeEach(reset);

  it('signs up, signs out and signs back in with the same password', async () => {
    const s = useAppStore.getState();
    expect(await s.signUp('Sam Okafor', 'sam@example.com', 'Passw0rd!x')).toEqual({ ok: true });
    const account = useAppStore.getState().accounts[0];
    expect(account.passwordHash).toMatch(/^pbkdf2-sha256\$/);
    expect(JSON.stringify(useAppStore.getState())).not.toContain('Passw0rd!x');

    useAppStore.getState().signOut();
    expect(useAppStore.getState().session).toBeNull();
    expect((await useAppStore.getState().signIn('SAM@example.com ', 'wrong-pass1', true)).ok).toBe(false);
```

```
    expect(await useAppStore.getState().signIn('SAM@example.com ', 'Passw0rd!x', true)).toEqual({ ok: true });
    expect(useAppStore.getState().session?.userId).toBe(account.id);
  });

  it('refuses a second account with the same email and a malformed email', async () => {
    await useAppStore.getState().signUp('A', 'a@example.com', 'Passw0rd!x');
    expect((await useAppStore.getState().signUp('B', 'A@example.com', 'Passw0rd!y')).ok).toBe(false);
    expect((await useAppStore.getState().signUp('B', 'not-an-email', 'Passw0rd!y')).ok).toBe(false);
    expect(useAppStore.getState().accounts).toHaveLength(1);
  });

  it('upgrades an old single-round SHA-256 account the first time it signs in', async () => {
    const salt = 'legacy-salt';
    const passwordHash = createHash('sha256').update(`${salt}:Passw0rd!x`).digest('hex');
    useAppStore.setState({ accounts: [{ id: 'u-old', name: 'Old User', email: 'old@example.com', passwordHash, salt, provider: 'passwor
d' }] });
    expect((await useAppStore.getState().signIn('old@example.com', 'bad-pass-1', false)).ok).toBe(false);
    expect(useAppStore.getState().accounts[0].passwordHash).toBe(passwordHash); // a failed sign-in changes nothing
    expect((await useAppStore.getState().signIn('old@example.com', 'Passw0rd!x', false)).ok).toBe(true);
    expect(useAppStore.getState().accounts[0].passwordHash).toMatch(/^pbkdf2-sha256\$/);
    useAppStore.getState().signOut();
    expect((await useAppStore.getState().signIn('old@example.com', 'Passw0rd!x', false)).ok).toBe(true);
  });

  it('resets a forgotten password', async () => {
    await useAppStore.getState().signUp('A', 'a@example.com', 'Passw0rd!x');
    useAppStore.getState().signOut();
    expect((await useAppStore.getState().resetPassword('nobody@example.com', 'Newpass1!x')).ok).toBe(false);
    expect((await useAppStore.getState().resetPassword('a@example.com', 'Newpass1!x')).ok).toBe(true);
    expect((await useAppStore.getState().signIn('a@example.com', 'Passw0rd!x', true)).ok).toBe(false);
    expect((await useAppStore.getState().signIn('a@example.com', 'Newpass1!x', true)).ok).toBe(true);
  });

  it('seeds the demo account and lets it sign in', async () => {
    await useAppStore.getState().ensureSeed();
    const s = useAppStore.getState();
    expect(s.trips.map((t) => t.name)).toContain('Amalfi Squad');
    expect(await s.signIn('maya.chen@togetherwego.app', 'TravelTogether2026!', true)).toEqual({ ok: true });
  });
});
```

src/store/__tests__/migrate.test.ts

```
import { defaultPersonal } from '../slices/app';
import { useAppStore } from '../useAppStore';

type Saved = { personal?: { sections: Record<string, boolean>; order?: string[]; layout?: string; effects?: string }; trips?: unknown[]
};
const migrate = (state: object, version: number) => useAppStore.persist.getOptions().migrate!(state, version) as Saved;

// What the Home sections looked like before version 4
const OLD_SECTIONS = { glance: true, budget: true, upnext: true, quick: true, forYou: true, nearby: false, discover: true };
const oldPersonal = (sections: Record<string, boolean>) => ({ ...defaultPersonal, sections });

describe('saved data from an older version', () => {
  it('starts new people on the Planner layout with the Nearby block off', () => {
    expect(defaultPersonal.layout).toBe('planner');
    expect(defaultPersonal.sections.nearby).toBe(false);
    expect(defaultPersonal.sections.upnext).toBe(true);
    expect(defaultPersonal.order[0]).toBe('upnext');
    expect(defaultPersonal.effects).toBe('lively');
  });

  it('switches Nearby off once for people who never changed the Home sections (version 2 data)', () => {
    const everythingOn = oldPersonal({ ...OLD_SECTIONS, nearby: true });
    expect(migrate({ personal: everythingOn, trips: [] }, 2).personal?.sections.nearby).toBe(false);
  });

  it('keeps what a person had switched on or off when Home gets its new blocks', () => {
    const custom = oldPersonal({ ...OLD_SECTIONS, budget: false, nearby: true, glance: false });
    const out = migrate({ personal: custom, trips: [] }, 3).personal!;
    expect(out.sections.budget).toBe(false);
    expect(out.sections.weather).toBe(false); // "glance" became "weather"
    expect(out.sections.nearby).toBe(true);
    expect(out.sections.crew).toBe(true); // new blocks start on
    expect(out.sections.play).toBe(true);
    expect('quick' in out.sections).toBe(false);
    expect('glance' in out.sections).toBe(false);
    expect(out.layout).toBe('custom');
    expect(out.order).toHaveLength(8);
  });

  it('gives people who never changed Home the Planner layout', () => {
    const out = migrate({ personal: oldPersonal(OLD_SECTIONS), trips: [] }, 3).personal!;
    expect(out.layout).toBe('planner');
    expect(out.effects).toBe('lively');
```

```
  });

  it('does nothing once the data is already on the current version', () => {
    const current = { ...defaultPersonal, sections: { ...defaultPersonal.sections, nearby: true } };
    expect(migrate({ personal: current }, 4).personal?.sections.nearby).toBe(true);
  });

  it('still re-seeds the sample trips for data from before version 2', () => {
    const out = migrate({ trips: [{ id: 'own-trip' }], personal: oldPersonal(OLD_SECTIONS) }, 1) as { trips: { id: string }[]; seeded?:
boolean };
    expect(out.seeded).toBe(false);
    expect(out.trips.map((t) => t.id)).toContain('own-trip');
  });
});

describe('daily trivia in the store', () => {
  beforeEach(() => useAppStore.setState({ trivia: { day: null, picked: null, streak: 0, lastCorrectDay: null } }));

  it('counts only the first answer of a day and builds a streak over days', () => {
    const { answerTrivia } = useAppStore.getState();
    answerTrivia('2026-10-23', 2, true);
    answerTrivia('2026-10-23', 0, false); // a second try the same day changes nothing
    expect(useAppStore.getState().trivia).toMatchObject({ day: '2026-10-23', picked: 2, streak: 1 });
    answerTrivia('2026-10-24', 1, true);
    expect(useAppStore.getState().trivia.streak).toBe(2);
    answerTrivia('2026-10-25', 3, false);
    expect(useAppStore.getState().trivia.streak).toBe(0);
  });
});
```

src/store/__tests__/trips.test.ts

```
import { balances, settlePlan, useAppStore } from '../useAppStore';

const s = () => useAppStore.getState();
const tripOf = (id: string) => s().trips.find((t) => t.id === id)!;

async function signedInUser(name = 'Test User', email = 't@example.com') {
  s().resetEverything();
  await s().signUp(name, email, 'Passw0rd!x');
  return s().session!.userId;
}

const newTrip = () => s().createTrip({ name: 'Rome Crew', destinationId: 'rome', startDate: '2026-09-10', endDate: '2026-09-16', budget:
3000 });

describe('trips', () => {
  it('creates a trip owned by the signed-in user', async () => {
    const me = await signedInUser();
    const id = newTrip();
    const t = tripOf(id);
    expect(s().activeTripId).toBe(id);
    expect(t.ownerId).toBe(me);
    expect(t.members).toEqual([expect.objectContaining({ id: me, role: 'Organizer' })]);
    expect(t.inviteCode).toMatch(/^[A-Z]{3}-\d{4}$/);
    expect(t.tasks).toHaveLength(3);
    expect(t.messages[0]).toMatchObject({ ai: true });
  });

  it('lets a second person join by invite code, once', async () => {
    await signedInUser('Maya', 'maya@example.com');
    const id = newTrip();
    const code = tripOf(id).inviteCode;
    s().signOut();
    await s().signUp('Sam', 'sam@example.com', 'Passw0rd!x');
    const sam = s().session!.userId;
    expect(s().joinTrip(code)).toEqual({ ok: true, tripId: id });
    expect(tripOf(id).members.map((m) => m.id)).toContain(sam);
    expect(tripOf(id).messages.at(-1)).toMatchObject({ kind: 'system' });
    const again = s().joinTrip(code);
    expect(again.ok).toBe(false);
    expect(s().joinTrip('ZZZ-0000')).toMatchObject({ ok: false });
  });

  it('can join a trip from the built-in directory', async () => {
    await signedInUser();
    const r = s().joinTrip('SEO-5521');
    expect(r.ok).toBe(true);
    expect(s().trips.some((t) => t.inviteCode === 'SEO-5521')).toBe(true);
  });

  it('edits, leaves and deletes trips, moving the active trip sensibly', async () => {
    const me = await signedInUser();
    const a = newTrip();
    const b = s().createTrip({ name: 'Second', destinationId: 'tokyo', startDate: '2026-10-01', endDate: '2026-10-05', budget: 1000 });
    expect(s().activeTripId).toBe(b);
```

```
    s().updateTrip(a, { name: 'Renamed', budget: 4200 });
    expect(tripOf(a)).toMatchObject({ name: 'Renamed', budget: 4200 });
    s().leaveTrip(b);
    expect(tripOf(b).members.some((m) => m.id === me)).toBe(false);
    expect(s().activeTripId).toBe(a);
    s().deleteTrip(a);
    expect(s().trips.find((t) => t.id === a)).toBeUndefined();
    expect(s().activeTripId).toBeNull();
  });

  it('regenerates the invite code', async () => {
    await signedInUser();
    const id = newTrip();
    const before = tripOf(id).inviteCode;
    s().regenerateInvite(id);
    expect(tripOf(id).inviteCode).toMatch(/^[A-Z]{3}-\d{4}$/);
    expect(tripOf(id).inviteCode).not.toBe(undefined);
    void before;
  });
});

describe('itinerary and voting', () => {
  const base = { date: '2026-09-11', time: '10:00', title: 'Colosseum', location: 'Rome', type: 'sightseeing' as const, durationMin: 12
0, costPerPerson: 20, status: 'voting' as const, assigned: [], reminder: false };

  it('adds, edits and deletes activities', async () => {
    const me = await signedInUser();
    const t = newTrip();
    const id = s().addActivity(t, { ...base });
    expect(tripOf(t).activities[0]).toMatchObject({ id, createdBy: me, votes: { [me]: 1 } });
    s().updateActivity(t, id, { title: 'Colosseum tour', time: '09:30' });
    expect(tripOf(t).activities[0]).toMatchObject({ title: 'Colosseum tour', time: '09:30' });
    s().toggleAssign(t, id, me);
    expect(tripOf(t).activities[0].assigned).toEqual([me]);
    s().toggleAssign(t, id, me);
    expect(tripOf(t).activities[0].assigned).toEqual([]);
    s().deleteActivity(t, id);
    expect(tripOf(t).activities).toHaveLength(0);
  });

  it('confirms an activity once more than half the group votes yes', async () => {
    const me = await signedInUser();
    const t = newTrip();
    s().addMember(t, 'Friend One');
    s().addMember(t, 'Friend Two');
    const friend = tripOf(t).members[1].id;
    const id = s().addActivity(t, { ...base, votes: { [friend]: 1 } });
    expect(tripOf(t).activities[0].status).toBe('voting'); // 1 of 3
    s().voteActivity(t, id, 1);
    expect(tripOf(t).activities[0].votes[me]).toBe(1);
    expect(tripOf(t).activities[0].status).toBe('confirmed'); // 2 of 3
    s().voteActivity(t, id, 1); // same vote again takes it back
    expect(tripOf(t).activities[0].votes[me]).toBeUndefined();
  });

  it('removing a member cleans up their assignments and votes', async () => {
    await signedInUser();
    const t = newTrip();
    s().addMember(t, 'Friend');
    const friend = tripOf(t).members[1].id;
    const id = s().addActivity(t, { ...base, assigned: [friend], votes: { [friend]: -1 } });
    s().addTask(t, 'Book tickets', friend);
    s().removeMember(t, friend);
    const a = tripOf(t).activities.find((x) => x.id === id)!;
    expect(a.assigned).toEqual([]);
    expect(a.votes[friend]).toBeUndefined();
    expect(tripOf(t).tasks.find((k) => k.text === 'Book tickets')!.assignee).toBeUndefined();
  });
});

describe('budget', () => {
  it('tracks expenses and settles up to zero', async () => {
    const me = await signedInUser();
    const t = newTrip();
    s().addMember(t, 'Friend');
    const friend = tripOf(t).members[1].id;
    s().addExpense(t, { title: 'Hotel', amount: 300, category: 'Stays', paidBy: me, splitBetween: [me, friend], date: '2026-09-10' });
    s().addExpense(t, { title: 'Dinner', amount: 80, category: 'Food', paidBy: friend, splitBetween: [me, friend], date: '2026-09-11'
});
    expect(tripOf(t).expenses[0].title).toBe('Dinner'); // newest first
    expect(balances(tripOf(t))).toEqual({ [me]: 110, [friend]: -110 });
    const [payment] = settlePlan(tripOf(t));
    expect(payment).toEqual({ from: friend, to: me, amount: 110 });
    s().settleUp(t, payment.from, payment.to, payment.amount);
    expect(tripOf(t).expenses[0]).toMatchObject({ settlement: true, paidBy: friend });
    expect(settlePlan(tripOf(t))).toEqual([]);
```

```
    const spend = tripOf(t).expenses.filter((e) => !e.settlement).length;
    expect(spend).toBe(2);
    s().deleteExpense(t, tripOf(t).expenses[0].id);
    expect(tripOf(t).expenses).toHaveLength(2);
  });
});

describe('collaboration features', () => {
  it('polls: vote, change vote, close, delete', async () => {
    const me = await signedInUser();
    const t = newTrip();
    s().createPoll(t, 'Which beach?', ['Ostia', 'Sperlonga']);
    const poll = tripOf(t).polls[0];
    s().votePoll(t, poll.id, poll.options[0].id);
    expect(tripOf(t).polls[0].options[0].votes).toEqual([me]);
    s().votePoll(t, poll.id, poll.options[1].id); // switching moves the vote
    expect(tripOf(t).polls[0].options.map((o) => o.votes.length)).toEqual([0, 1]);
    s().closePoll(t, poll.id);
    s().votePoll(t, poll.id, poll.options[0].id); // closed polls ignore votes
    expect(tripOf(t).polls[0].options[0].votes).toEqual([]);
    s().deletePoll(t, poll.id);
    expect(tripOf(t).polls).toHaveLength(0);
  });

  it('checklist, packing, docs and photos', async () => {
    const me = await signedInUser();
    const t = newTrip();
    const first = tripOf(t).tasks[0];
    s().toggleTask(t, first.id);
    expect(tripOf(t).tasks[0].done).toBe(true);
    s().assignTask(t, first.id, me);
    expect(tripOf(t).tasks[0].assignee).toBe(me);
    s().deleteTask(t, first.id);
    expect(tripOf(t).tasks).toHaveLength(2);

    s().addPackingItems(t, [{ text: 'Passport', category: 'Documents' }, { text: 'passport', category: 'Documents' }, { text: 'Charger',
category: 'Tech' }]);
    expect(tripOf(t).packing.map((p) => p.text)).toEqual(['Passport', 'Charger']); // duplicates are skipped
    s().togglePacked(t, tripOf(t).packing[0].id);
    s().assignPacking(t, tripOf(t).packing[1].id, me);
    expect(tripOf(t).packing[0].packed).toBe(true);
    expect(tripOf(t).packing[1].broughtBy).toBe(me);

    s().addDoc(t, { name: 'Ticket.pdf', kind: 'pdf', offline: true });
    expect(tripOf(t).docs[0]).toMatchObject({ name: 'Ticket.pdf', addedBy: me });
    s().addPhoto(t, { uri: 'file:///p.jpg', caption: 'Pasta' });
    const photo = tripOf(t).photos[0];
    s().toggleLikePhoto(t, photo.id);
    expect(tripOf(t).photos[0].likes).toEqual([me]);
    s().toggleLikePhoto(t, photo.id);
    expect(tripOf(t).photos[0].likes).toEqual([]);
    s().deletePhoto(t, photo.id);
    s().deleteDoc(t, tripOf(t).docs[0].id);
    expect(tripOf(t).photos).toHaveLength(0);
    expect(tripOf(t).docs).toHaveLength(0);
  });

  it('records which server file belongs to a photo, a document and a receipt', async () => {
    const me = await signedInUser();
    const t = newTrip();
    s().addPhoto(t, { uri: 'file:///p.jpg', caption: 'Pasta' });
    s().addDoc(t, { name: 'Ticket', kind: 'pdf', uri: 'file:///t.pdf', offline: true });
    s().addExpense(t, { title: 'Taxi', amount: 20, category: 'Transport', paidBy: me, splitBetween: [me], date: '2026-09-10', receipt:
'file:///r.jpg' });
    const [photo, doc, expense] = [tripOf(t).photos[0], tripOf(t).docs[0], tripOf(t).expenses[0]];
    s().attachBlob(t, 'photos', photo.id, 'aaaa1111_jpg');
    s().attachBlob(t, 'docs', doc.id, 'bbbb2222_pdf');
    s().attachBlob(t, 'expenses', expense.id, 'cccc3333_jpg');
    expect(tripOf(t).photos[0]).toMatchObject({ uri: 'file:///p.jpg', blob: 'aaaa1111_jpg' });
    expect(tripOf(t).docs[0]).toMatchObject({ uri: 'file:///t.pdf', blob: 'bbbb2222_pdf' });
    expect(tripOf(t).expenses[0]).toMatchObject({ receipt: 'file:///r.jpg', receiptBlob: 'cccc3333_jpg' });
    s().attachBlob(t, 'photos', 'no-such-photo', 'dddd4444_jpg'); // an item deleted meanwhile is ignored
    s().attachBlob('no-such-trip', 'photos', photo.id, 'eeee5555_jpg');
    expect(tripOf(t).photos).toHaveLength(1);
  });

  it('chat keeps only the latest 400 messages', async () => {
    const me = await signedInUser();
    const t = newTrip();
    for (let i = 0; i < 405; i++) s().sendMessage(t, { authorId: me, text: `m${i}` });
    const msgs = tripOf(t).messages;
    expect(msgs).toHaveLength(400);
    expect(msgs.at(-1)!.text).toBe('m404');
    s().clearChat(t);
    expect(tripOf(t).messages).toEqual([]);
  });
```

```
  it('changes roles', async () => {
    const me = await signedInUser();
    const t = newTrip();
    s().setRole(t, me, 'Treasurer');
    expect(tripOf(t).members[0].role).toBe('Treasurer');
  });
});

describe('sample data', () => {
  it('can be cleared without touching the person’s own trips, and does not come back', async () => {
    s().resetEverything();
    await s().ensureSeed();
    expect(s().trips.some((t) => t.id === 'trip-amalfi')).toBe(true);
    await s().signUp('Own Trip', 'own@example.com', 'Passw0rd!x');
    const mine = newTrip();
    s().clearDemoData();
    expect(s().trips.map((t) => t.id)).toEqual([mine]);
    expect(s().activeTripId).toBe(mine);
    expect(s().notifications.every((n) => !n.tripId || n.tripId === mine)).toBe(true);
    expect(s().bucket).toEqual([]);
    expect(s().seeded).toBe(true);
    await s().ensureSeed(); // already seeded: nothing returns
    expect(s().trips.map((t) => t.id)).toEqual([mine]);
    expect(s().accounts.some((a) => a.email === 'own@example.com')).toBe(true);
  });

  it('keeps bucket-list items the person added themselves', async () => {
    s().resetEverything();
    await s().ensureSeed();
    s().addBucket('Learn to surf in Portugal');
    s().clearDemoData();
    expect(s().bucket.map((b) => b.text)).toEqual(['Learn to surf in Portugal']);
  });
});

describe('app-level state', () => {
  it('bucket list, favourites, notifications and settings', async () => {
    await signedInUser();
    s().addBucket('See the aurora', 'iceland');
    expect(s().bucket[0]).toMatchObject({ text: 'See the aurora', done: false });
    s().toggleBucket(s().bucket[0].id);
    expect(s().bucket[0].done).toBe(true);
    s().deleteBucket(s().bucket[0].id);
    expect(s().bucket).toEqual([]);

    s().toggleFavorite('rome');
    expect(s().favorites).toContain('rome');
    s().toggleFavorite('rome');
    expect(s().favorites).not.toContain('rome');

    for (let i = 0; i < 70; i++) s().pushNotification({ kind: 'system', title: `n${i}`, body: '' });
    expect(s().notifications).toHaveLength(60);
    s().markKindRead('system');
    expect(s().notifications.every((n) => n.read)).toBe(true);
    s().clearNotifications();
    expect(s().notifications).toEqual([]);

    s().updateSettings({ language: 'es', textScale: 1.25 });
    expect(s().settings).toMatchObject({ language: 'es', textScale: 1.25, haptics: true });
    s().updatePersonal({ nickname: 'Mimi', interests: ['food'] });
    expect(s().personal).toMatchObject({ nickname: 'Mimi', interests: ['food'], travelStyle: 'balanced' });
  });

  it('keeps the session only when "remember me" was ticked', async () => {
    await signedInUser();
    const opts = useAppStore.persist.getOptions();
    const saved = opts.partialize!({ ...s(), session: { userId: 'u', remember: false } });
    expect((saved as { session: unknown }).session).toBeNull();
    const kept = opts.partialize!({ ...s(), session: { userId: 'u', remember: true } });
    expect((kept as { session: unknown }).session).toEqual({ userId: 'u', remember: true });
  });

  it('re-seeds the demo trip for a different stage of the trip', async () => {
    await s().ensureSeed();
    s().reseedDemo('active');
    expect(s().demoTimeline).toBe('active');
    expect(s().trips.filter((t) => t.id === 'trip-amalfi')).toHaveLength(1);
  });
});
```

src/store/seed.ts

```
/**
 * Demo content. Dates are generated relative to "today" so the sample trip is
 * always fresh — whether the app is opened today or at the competition months
```

```
 * later. Settings → Demo timeline can re-seed it as upcoming, in-progress or
 * completed to show the whole trip lifecycle.
 */
import { packingBase } from '@/data/reference';
import { uid } from '@/utils/format';
import { addDays, todayInTz, zonedToUtc } from '@/utils/time';

import type { Activity, Expense, Flight, Member, PackingItem, Photo, Trip } from './types';

export const DEMO_EMAIL = 'maya.chen@togetherwego.app';
export const DEMO_PASSWORD = 'TravelTogether2026!';
export const DEMO_USER_ID = 'u-maya';
/** The sample trips that come with the app (the fourth is from an older version). "Clear sample trips" removes exactly these. */
export const DEMO_TRIP_IDS = ['trip-amalfi', 'trip-bali', 'trip-bcn', 'trip-tokyo'];

export type DemoTimeline = 'upcoming' | 'active' | 'completed';

const TOKYO_TZ = 'Asia/Tokyo';
const AMALFI_TZ = 'Europe/Rome';

/** The 6 members of the Tokyo & Kyoto trip. */
export const demoMembers: Member[] = [
  { id: DEMO_USER_ID, name: 'Maya Chen', email: DEMO_EMAIL, avatar: 'avatar-maya', role: 'Navigator', homeCity: 'New York' },
  { id: 'm-diego', name: 'Diego Alvarez', avatar: 'avatar-diego', role: 'Food Lead', homeCity: 'New York' },
  { id: 'm-priya', name: 'Priya Patel', avatar: 'avatar-priya', role: 'Treasurer', homeCity: 'San Francisco' },
  { id: 'm-sam', name: 'Sam Okafor', avatar: 'avatar-sam', role: 'Photographer', homeCity: 'New York' },
  { id: 'm-ana', name: 'Ana Silva', avatar: 'avatar-ana', role: 'Planner', homeCity: 'Boston' },
  { id: 'm-noah', name: 'Noah Kim', avatar: 'avatar-noah', role: 'Activities Lead', homeCity: 'Chicago' },
];

const ALL = demoMembers.map((m) => m.id);
const votesFrom = (ids: string[], down: string[] = []): Record<string, 1 | -1> => {
  const v: Record<string, 1 | -1> = {};
  ids.forEach((id) => (v[id] = 1));
  down.forEach((id) => (v[id] = -1));
  return v;
};

function act(start: string, day: number, a: Omit<Activity, 'id' | 'date' | 'createdBy' | 'reminder'> & { reminder?: boolean }): Activity
{
  return { id: uid('a-'), date: addDays(start, day), createdBy: DEMO_USER_ID, reminder: a.reminder ?? a.status === 'confirmed', ...a };
}

function demoActivities(start: string): Activity[] {
  return [
    // Day 1
    act(start, 0, { time: '14:30', title: 'Arrive at Haneda Airport (HND)', location: 'Ōta, Tokyo', type: 'transport', durationMin: 150,
costPerPerson: 0, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-sam'], votes: votesFrom(ALL), lat: 35.5494, lng: 139.7798, notes: 'Me
et at the Keikyu Line ticket gates, Terminal 3 arrivals.' }),
    act(start, 0, { time: '17:30', title: 'Check-in — Shinjuku Grand Hotel', location: 'Shinjuku, Tokyo', type: 'stay', durationMin: 30,
costPerPerson: 180, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 35.6938, lng: 139.7034 }),
    act(start, 0, { time: '19:30', title: 'Welcome Dinner — Omoide Yokocho', location: 'Shinjuku, Tokyo', type: 'food', durationMin: 12
0, costPerPerson: 30, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(ALL.slice(0, 5)), lat: 35.6929, lng: 139.6996, placeI
d: 'omoide' }),
    // Day 2
    act(start, 1, { time: '08:00', title: 'Tsukiji Outer Market Breakfast', location: 'Tsukiji, Tokyo', type: 'food', durationMin: 90, c
ostPerPerson: 25, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(ALL.slice(0, 4)), lat: 35.6655, lng: 139.7707, placeId:
'tsukiji' }),
    act(start, 1, { time: '10:30', title: 'Senso-ji Temple & Asakusa', location: 'Asakusa, Tokyo', type: 'culture', durationMin: 120, co
stPerPerson: 0, status: 'voting', assigned: ['m-ana', 'm-noah'], votes: votesFrom(['m-ana', 'm-noah', 'm-priya'], ['m-sam']), lat: 35.71
48, lng: 139.7967, placeId: 'sensoji' }),
    act(start, 1, { time: '14:00', title: 'teamLab Planets Digital Art Museum', location: 'Toyosu, Tokyo', type: 'culture', durationMin:
150, costPerPerson: 30, status: 'pending', assigned: ['m-ana', 'm-priya', 'm-noah'], votes: votesFrom(ALL.slice(0, 5)), lat: 35.6492, ln
g: 139.7898, placeId: 'teamlab', notes: 'Wear shorts — you walk through knee-deep water!' }),
    act(start, 1, { time: '20:00', title: 'Shibuya Crossing & Rooftop Bar', location: 'Shibuya, Tokyo', type: 'nightlife', durationMin:
180, costPerPerson: 40, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(ALL), lat: 35.6595, lng: 139.7005, placeId: 'shibuya-
crossing' }),
    // Day 3
    act(start, 2, { time: '09:00', title: 'Meiji Jingu & Harajuku stroll', location: 'Harajuku, Tokyo', type: 'culture', durationMin: 15
0, costPerPerson: 0, status: 'confirmed', assigned: ['m-ana'], votes: votesFrom(ALL), lat: 35.6764, lng: 139.6993, placeId: 'meiji' }),
    act(start, 2, { time: '13:30', title: 'Akihabara arcades & retro games', location: 'Akihabara, Tokyo', type: 'shopping', durationMi
n: 120, costPerPerson: 15, status: 'confirmed', assigned: ['m-noah'], votes: votesFrom(ALL.slice(1)), lat: 35.6984, lng: 139.7731, place
Id: 'akihabara' }),
    act(start, 2, { time: '19:00', title: 'Karaoke night in Shibuya', location: 'Shibuya, Tokyo', type: 'nightlife', durationMin: 120, c
ostPerPerson: 20, status: 'voting', assigned: ['m-noah'], votes: votesFrom(['m-noah', 'm-diego', 'm-sam']), lat: 35.66, lng: 139.6985, p
laceId: 'karaoke' }),
    // Day 4
    act(start, 3, { time: '07:30', title: 'Day trip: Mt Fuji & Chureito Pagoda', location: 'Fujiyoshida, Yamanashi', type: 'adventure',
durationMin: 540, costPerPerson: 45, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-sam'], votes: votesFrom(ALL), lat: 35.5016, lng: 1
38.8013, notes: 'Bus from Shinjuku Expressway Bus Terminal, gate B.' }),
    // Day 5
    act(start, 4, { time: '08:30', title: 'Shinkansen to Kyoto', location: 'Tokyo Station', type: 'transport', durationMin: 135, costPer
Person: 0, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 35.6812, lng: 139.7671, notes: 'Covered by JR Pas
s. Sit on the right (seat E) for Fuji views.' }),
    act(start, 4, { time: '13:00', title: 'Check-in — Gion ryokan', location: 'Gion, Kyoto', type: 'stay', durationMin: 30, costPerPerso
n: 140, status: 'confirmed', assigned: ['m-priya'], votes: votesFrom(ALL), lat: 35.0037, lng: 135.7788 }),
```

```
    act(start, 4, { time: '17:00', title: 'Gion evening walk', location: 'Gion, Kyoto', type: 'culture', durationMin: 90, costPerPerson:
0, status: 'confirmed', assigned: ['m-ana'], votes: votesFrom(ALL.slice(0, 5)), lat: 35.0037, lng: 135.7788, placeId: 'gion' }),
    // Day 6
    act(start, 5, { time: '06:30', title: 'Fushimi Inari sunrise hike', location: 'Fushimi, Kyoto', type: 'culture', durationMin: 150, c
ostPerPerson: 0, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(ALL.slice(0, 4)), lat: 34.9671, lng: 135.7727, placeId: 'ina
ri' }),
    act(start, 5, { time: '11:00', title: 'Nishiki Market lunch crawl', location: 'Nakagyo, Kyoto', type: 'food', durationMin: 90, costP
erPerson: 20, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(ALL), lat: 35.005, lng: 135.7649, placeId: 'nishiki' }),
    act(start, 5, { time: '15:00', title: 'Tea ceremony experience', location: 'Higashiyama, Kyoto', type: 'culture', durationMin: 60, c
ostPerPerson: 40, status: 'voting', assigned: ['m-priya'], votes: votesFrom(['m-priya', 'm-ana'], ['m-noah']), lat: 34.9986, lng: 135.78
09, placeId: 'tea' }),
    // Day 7
    act(start, 6, { time: '07:30', title: 'Arashiyama Bamboo Grove', location: 'Arashiyama, Kyoto', type: 'sightseeing', durationMin: 6
0, costPerPerson: 0, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(ALL), lat: 35.017, lng: 135.6713, placeId: 'bamboo' }),
    act(start, 6, { time: '13:00', title: 'Hozugawa river boat ride', location: 'Kameoka, Kyoto', type: 'adventure', durationMin: 120, c
ostPerPerson: 35, status: 'pending', assigned: ['m-noah'], votes: votesFrom(['m-noah', 'm-diego', 'm-sam', DEMO_USER_ID]), lat: 35.0128,
lng: 135.6744, placeId: 'hozugawa' }),
    // Day 8 left empty on purpose (free day)
    // Day 9
    act(start, 8, { time: '10:00', title: 'Shinkansen back to Tokyo', location: 'Kyoto Station', type: 'transport', durationMin: 135, co
stPerPerson: 0, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 34.9858, lng: 135.7588 }),
    act(start, 8, { time: '19:00', title: 'Farewell dinner — Gonpachi', location: 'Nishi-Azabu, Tokyo', type: 'food', durationMin: 120,
costPerPerson: 35, status: 'voting', assigned: ['m-diego'], votes: votesFrom(['m-diego', 'm-ana', 'm-priya']), lat: 35.6563, lng: 139.72
4, placeId: 'gonpachi' }),
    // Day 10
    act(start, 9, { time: '06:30', title: 'Head to Haneda for departures', location: 'Haneda Airport, Tokyo', type: 'transport', duratio
nMin: 60, costPerPerson: 6, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 35.5494, lng: 139.7798 }),
  ];
}

interface Airfield {
  tz: string;
  code: string;
}
const HND: Airfield = { tz: TOKYO_TZ, code: 'HND' };
const NAP: Airfield = { tz: AMALFI_TZ, code: 'NAP' };

function flightPair(at: Airfield, memberId: string, airline: string, outNo: string, inNo: string, from: string, start: string, end: stri
ng, arr: string, dep: string, hoursOut: number, hoursBack: number, arrDayOffset = 0): Flight[] {
  const arriveUtc = zonedToUtc(addDays(start, arrDayOffset), arr, at.tz);
  const departUtc = zonedToUtc(end, dep, at.tz);
  const confirmation = 'TWG' + memberId.slice(-3).toUpperCase();
  return [
    { id: uid('f-'), memberId, airline, flightNo: outNo, from, to: at.code, departUtc: arriveUtc - hoursOut * 3600000, arriveUtc, direct
ion: 'arrival', seat: '32A', confirmation },
    { id: uid('f-'), memberId, airline, flightNo: inNo, from: at.code, to: from, departUtc, arriveUtc: departUtc + hoursBack * 3600000,
direction: 'departure', seat: '41C', confirmation },
  ];
}

function demoExpenses(start: string): Expense[] {
  const before = addDays(start, -20);
  return [
    { id: uid('e-'), title: 'Round-trip flights', amount: 1840, category: 'Flights', paidBy: 'm-priya', splitBetween: ALL, date: before
},
    { id: uid('e-'), title: 'Shinjuku apartment', amount: 960, category: 'Stays', paidBy: DEMO_USER_ID, splitBetween: ALL, date: addDays
(before, 2) },
    { id: uid('e-'), title: 'JR Rail passes', amount: 420, category: 'Transport', paidBy: 'm-sam', splitBetween: ALL, date: addDays(befo
re, 5) },
    { id: uid('e-'), title: 'Group dinners', amount: 265, category: 'Food', paidBy: 'm-diego', splitBetween: ALL, date: addDays(before,
8), original: { amount: 40386, currency: 'JPY' } },
    { id: uid('e-'), title: 'teamLab tickets', amount: 132, category: 'Activities', paidBy: 'm-ana', splitBetween: ALL, date: addDays(be
fore, 9) },
  ];
}

function demoPacking(): PackingItem[] {
  const done = new Set(['Passport', 'Phone charger', 'Universal adapter', 'Comfortable walking shoes']);
  return packingBase.flatMap((g) => g.items.map((text) => ({ id: uid('p-'), text, category: g.category, packed: done.has(text) })));
}

function demoPhotos(start: string, now: number): Photo[] {
  const list: [string, string, string, number][] = [
    ['shibuya', 'Shibuya after dark 🌃', 'm-sam', 1],
    ['ramen', 'Diego's 2am ramen find 🍜', 'm-diego', 0],
    ['fushimi-inari', '10,000 gates and counting ⛩️', 'm-sam', 5],
    ['lantern-alley', 'Lantern alley vibes', 'm-ana', 2],
    ['sushi', 'Omakase at the market', 'm-priya', 1],
    ['tokyo-skyline', 'View from the tower', DEMO_USER_ID, 2],
    ['fuji-blossom', 'Fuji finally showed up 🗻', 'm-noah', 3],
    ['omoide-yokocho', 'Yakitori alley', 'm-diego', 0],
    ['akihabara', 'Akiba arcade crew', 'm-noah', 2],
    ['kyoto-skyline', 'Kyoto from Kiyomizu', 'm-ana', 5],
  ];
  return list.map(([uri, caption, by, day], i) => ({
    id: uid('ph-'), uri, caption, by, day: addDays(start, day), ts: now - i * 3600000,
```

```
    likes: ALL.filter((_, j) => (i + j) % 3 !== 0),
  }));
}

export function inviteCodeFor(name: string): string {
  const letters = name.replace(/[^A-Za-z]/g, '').toUpperCase().padEnd(3, 'X').slice(0, 3);
  return `${letters}-${Math.floor(1000 + Math.random() * 9000)}`;
}

/** The big 10-day Tokyo & Kyoto trip — kept as a second trip so multi-trip management has something to show. */
export function buildTokyoTrip(now = Date.now()): Trip {
  const today = todayInTz(TOKYO_TZ);
  const start = addDays(today, 45);
  const end = addDays(start, 9);
  const flights = [
    ...flightPair(HND, DEMO_USER_ID, 'ANA', 'NH 175', 'NH 176', 'JFK', start, end, '14:20', '09:05', 14, 13),
    ...flightPair(HND, 'm-diego', 'Japan Airlines', 'JL 006', 'JL 005', 'JFK', start, end, '16:40', '11:30', 14, 13),
    ...flightPair(HND, 'm-priya', 'United', 'UA 837', 'UA 838', 'SFO', start, end, '07:15', '09:05', 11, 10, 1),
    ...flightPair(HND, 'm-sam', 'ANA', 'NH 175', 'NH 176', 'JFK', start, addDays(end, -1), '14:20', '18:00', 14, 13),
  ];
  const t = (mins: number) => now - mins * 60000;
  return {
    id: 'trip-tokyo',
    name: 'Tokyo & Kyoto Explorer',
    destinationId: 'tokyo',
    extraDestinationIds: ['kyoto', 'mount-fuji'],
    startDate: start,
    endDate: end,
    cover: 'tokyo',
    budget: 4200,
    inviteCode: 'TKY-2481',
    ownerId: DEMO_USER_ID,
    members: demoMembers.map((m) => ({ ...m })),
    activities: demoActivities(start),
    expenses: demoExpenses(start),
    flights,
    polls: [
      {
        id: uid('poll-'), question: 'Day 4 evening — what are we doing?', createdBy: 'm-noah', createdAt: t(600),
        options: [
          { id: 'o1', text: 'Robot show in Shinjuku', votes: ['m-noah'] },
          { id: 'o2', text: 'Sumida river cruise', votes: ['m-priya'] },
          { id: 'o3', text: 'Karaoke night', votes: ['m-diego', 'm-ana', 'm-sam'] },
        ],
      },
      {
        id: uid('poll-'), question: 'Farewell dinner: where should we go?', createdBy: 'm-diego', createdAt: t(300),
        options: [
          { id: 'o1', text: 'Gonpachi izakaya (Kill Bill vibes)', votes: ['m-diego', 'm-ana'] },
          { id: 'o2', text: 'Sushi omakase splurge', votes: ['m-priya'] },
          { id: 'o3', text: 'Ramen crawl in Ebisu', votes: ['m-noah'] },
        ],
      },
    ],
    tasks: [
      { id: uid('t-'), text: 'Passports valid 6+ months', done: true },
      { id: uid('t-'), text: 'Book airport transfer', done: false, assignee: DEMO_USER_ID },
      { id: uid('t-'), text: 'Travel insurance', done: false, assignee: 'm-priya' },
      { id: uid('t-'), text: 'Download offline maps', done: true, assignee: 'm-sam' },
      { id: uid('t-'), text: 'Pocket wifi pickup code', done: false, assignee: 'm-diego' },
    ],
    docs: [
      {
        id: uid('d-'), name: 'Flight tickets (all members)', kind: 'note', addedBy: 'm-priya', addedAt: t(4000), offline: true,
        note: 'Maya — ANA NH 175 · JFK → HND · Conf. TWGAYA\nSam — ANA NH 175 · JFK → HND · Conf. TWGSAM\nDiego — JAL JL 006 · JFK → HND · Conf. TWGEGO\nPriya — United UA 837 · SFO → HND · Conf. TWGIYA\n\nCheck-in opens 24h before departure. Bring the passport used for booking.',
      },
      {
        id: uid('d-'), name: 'Apartment booking', kind: 'note', addedBy: DEMO_USER_ID, addedAt: t(3800), offline: true,
        note: 'Shinjuku Grand Hotel — 3 rooms, 4 nights\nCheck-in from 15:00 · Check-out 11:00\nBooking ref: SGH-55120\n\nShow the taxi driver:\n新宿駅までお願いします (To Shinjuku Station, please)',
      },
      {
        id: uid('d-'), name: 'Passport scans', kind: 'note', addedBy: DEMO_USER_ID, addedAt: t(3600), offline: true,
        note: 'Encrypted copies stored on each member’s phone.\nIf a passport is lost: report at the nearest koban (police box), then contact your embassy.\nUS Embassy Tokyo: +81 3-3224-5000',
      },
      {
        id: uid('d-'), name: 'Emergency contacts', kind: 'note', addedBy: 'm-ana', addedAt: t(3000), offline: true,
        note: 'Police 110 · Ambulance/Fire 119\nJapan Visitor Hotline (24h, English): 050-3816-2787\nTravel insurance: TravelSafe +1 800 555 0199, policy TS-88213',
      },
    ],
    messages: [
      { id: uid('c-'), authorId: 'm-priya', text: 'JR passes are booked 🎉 picking them up at Haneda', ts: t(320) },
```

```
      { id: uid('c-'), authorId: 'm-ana', text: 'Can we lock in teamLab for Day 2 afternoon? Tickets sell out', ts: t(260) },
      { id: uid('c-'), authorId: 'm-noah', text: 'Voted! Also started a poll for Day 4 evening 👀', ts: t(200) },
      { id: uid('c-'), authorId: 'm-diego', text: 'Found a ramen spot 4 min from the apartment, open till 2am 🍜', ts: t(42) },
      { id: uid('c-'), authorId: DEMO_USER_ID, text: 'Adding it to Day 1 after Shibuya!', ts: t(40) },
      { id: uid('c-'), authorId: 'assistant', ai: true, aiSource: 'local', text: 'Heads up: Priya lands at 07:15 on Day 2 — the morning after everyone else. Want me to move Tsukiji breakfast to 09:30 so she can join?', ts: t(38) },
    ],
    photos: demoPhotos(start, now),
    packing: demoPacking(),
    completed: false,
    createdAt: t(60 * 24 * 30),
  };
}


// ---------------------------------------------------------------------------
// Amalfi Squad — the main demo trip (4 friends, 6 days, Positano)
// ---------------------------------------------------------------------------
const squad: Member[] = [
  { id: DEMO_USER_ID, name: 'Maya Chen', email: DEMO_EMAIL, avatar: 'avatar-maya', role: 'Navigator', homeCity: 'New York' },
  { id: 'm-diego', name: 'Diego Alvarez', avatar: 'avatar-diego', role: 'Food Lead', homeCity: 'New York' },
  { id: 'm-priya', name: 'Priya Patel', avatar: 'avatar-priya', role: 'Treasurer', homeCity: 'San Francisco' },
  { id: 'm-sam', name: 'Sam Okafor', avatar: 'avatar-sam', role: 'Photographer', homeCity: 'New York' },
];
const SQUAD = squad.map((m) => m.id);

function amalfiActivities(start: string): Activity[] {
  return [
    act(start, 0, { time: '15:40', title: 'Arrive at Naples Airport (NAP)', location: 'Naples Airport', type: 'transport', durationMin:
90, costPerPerson: 0, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(SQUAD), lat: 40.886, lng: 14.2908, notes: 'Private
transfer to Positano, about 1h 30m. Driver holds a TogetherWeGo sign.' }),
    act(start, 0, { time: '17:45', title: 'Check-in — Villa Margherita', location: 'Positano', type: 'stay', durationMin: 30, costPerPer
son: 410, status: 'confirmed', assigned: ['m-priya'], votes: votesFrom(SQUAD), lat: 40.6283, lng: 14.4853, notes: 'Keys in the lockbox,
code in the Docs folder.' }),
    act(start, 0, { time: '19:00', title: 'Dinner at Da Vincenzo', location: 'Viale Pasitea', type: 'food', durationMin: 120, costPerPer
son: 64, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(SQUAD), lat: 40.6295, lng: 14.488, placeId: 'da-vincenzo' }),
    act(start, 1, { time: '08:00', title: 'Path of the Gods hike', location: 'Bomerano', type: 'adventure', durationMin: 300, costPerPer
son: 0, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-sam'], votes: votesFrom([DEMO_USER_ID, 'm-diego', 'm-sam']), lat: 40.619, lng:
14.5176, placeId: 'path-of-gods', notes: 'Bring water, sun hat and sturdy shoes. Bus from Amalfi to Bomerano at 07:15.' }),
    act(start, 1, { time: '20:00', title: 'Sunset aperitivo at Le Sirenuse', location: 'Positano', type: 'nightlife', durationMin: 90, c
ostPerPerson: 40, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(SQUAD.slice(0, 3)), lat: 40.6293, lng: 14.4868, placeId: 's
unset-bar' }),
    act(start, 2, { time: '10:00', title: 'Boat tour to Capri', location: 'Positano Marina', type: 'adventure', durationMin: 540, costPe
rPerson: 140, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-priya'], votes: votesFrom([DEMO_USER_ID, 'm-priya']), lat: 40.6272, lng:
14.4868, placeId: 'capri-boat' }),
    act(start, 3, { time: '11:00', title: 'Beach day at Fornillo Beach', location: 'Fornillo Beach', type: 'sightseeing', durationMin: 1
80, costPerPerson: 35, status: 'voting', assigned: [], votes: {}, reminder: false, lat: 40.6255, lng: 14.4795, placeId: 'fornillo' }),
    act(start, 3, { time: '16:00', title: 'Limoncello tasting', location: 'Amalfi', type: 'food', durationMin: 45, costPerPerson: 23, st
atus: 'voting', assigned: ['m-priya'], votes: votesFrom(['m-priya']), reminder: false, lat: 40.634, lng: 14.6027, placeId: 'limoncello'
}),
    act(start, 4, { time: '10:00', title: 'Villa Rufolo gardens in Ravello', location: 'Ravello', type: 'sightseeing', durationMin: 150,
costPerPerson: 8, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(SQUAD), lat: 40.649, lng: 14.6118, placeId: 'villa-rufolo'
}),
    act(start, 4, { time: '19:30', title: 'Farewell dinner — Trattoria da Adolfo', location: 'Laurito Beach', type: 'food', durationMin:
150, costPerPerson: 55, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(SQUAD.slice(0, 3)), lat: 40.6224, lng: 14.4731, pla
ceId: 'trattoria-chitarrella' }),
    act(start, 5, { time: '14:30', title: 'Transfer to Naples Airport', location: 'Positano', type: 'transport', durationMin: 100, costP
erPerson: 35, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(SQUAD), lat: 40.628, lng: 14.485 }),
  ];
}

export function buildDemoTrip(timeline: DemoTimeline = 'upcoming', now = Date.now()): Trip {
  const today = todayInTz(AMALFI_TZ);
  const offset = timeline === 'upcoming' ? 3 : timeline === 'active' ? -1 : -14;
  const start = addDays(today, offset);
  const end = addDays(start, 5);
  const t = (mins: number) => now - mins * 60000;
  const before = addDays(start, -20);
  return {
    id: 'trip-amalfi',
    name: 'Amalfi Squad',
    destinationId: 'positano',
    extraDestinationIds: [],
    startDate: start,
    endDate: end,
    cover: 'amalfi',
    budget: 4000,
    inviteCode: 'AML-2026',
    ownerId: DEMO_USER_ID,
    members: squad.map((m) => ({ ...m })),
    activities: amalfiActivities(start),
    expenses: [
      { id: uid('e-'), title: 'Villa Margherita (6 nights)', amount: 1650, category: 'Stays', paidBy: 'm-priya', splitBetween: SQUAD, da
te: before },
      { id: uid('e-'), title: 'Airport transfers', amount: 140, category: 'Transport', paidBy: DEMO_USER_ID, splitBetween: SQUAD, date:
addDays(before, 4) },
```

```
      { id: uid('e-'), title: 'Scooters & ferry tickets', amount: 210, category: 'Transport', paidBy: 'm-sam', splitBetween: SQUAD, dat
e: addDays(before, 6) },
      { id: uid('e-'), title: 'Dinner deposits & groceries', amount: 86, category: 'Food', paidBy: 'm-diego', splitBetween: SQUAD, date:
addDays(before, 8), original: { amount: 74, currency: 'EUR' } },
    ],
    flights: [
      ...flightPair(NAP, DEMO_USER_ID, 'Delta', 'DL 184', 'DL 185', 'JFK', start, end, '15:40', '19:15', 9.5, 10),
      ...flightPair(NAP, 'm-priya', 'ITA Airways', 'AZ 609', 'AZ 610', 'JFK', start, end, '17:05', '21:30', 9.5, 10),
    ],
    polls: [
      {
        id: uid('poll-'), question: 'Beach day — which beach?', createdBy: 'm-sam', createdAt: t(600),
        options: [
          { id: 'o1', text: 'Fornillo (quiet, pebbles)', votes: ['m-sam', 'm-diego'] },
          { id: 'o2', text: 'Spiaggia Grande (lively)', votes: ['m-priya'] },
          { id: 'o3', text: 'Fiordo di Furore (jump in!)', votes: [DEMO_USER_ID] },
        ],
      },
    ],
    tasks: [
      { id: uid('t-'), text: 'Passports valid 6+ months', done: true },
      { id: uid('t-'), text: 'Book airport transfers', done: true, assignee: DEMO_USER_ID },
      { id: uid('t-'), text: 'Travel insurance', done: false, assignee: 'm-priya' },
      { id: uid('t-'), text: 'Rent scooters (licences!)', done: false, assignee: 'm-sam' },
      { id: uid('t-'), text: 'Reserve Da Vincenzo for 4', done: true, assignee: 'm-diego' },
      { id: uid('t-'), text: 'Download offline maps', done: false, assignee: DEMO_USER_ID },
    ],
    docs: [
      {
        id: uid('d-'), name: 'Flight tickets', kind: 'note', addedBy: 'm-priya', addedAt: t(4000), offline: true,
        note: 'Maya — Delta DL 184 · JFK → NAP · Conf. TWGAYA\nPriya — ITA Airways AZ 609 · JFK → NAP · Conf. TWGIYA\n\nCheck-in opens 2
4h before departure. Bring the passport used for booking.',
      },
      {
        id: uid('d-'), name: 'Villa Margherita booking', kind: 'note', addedBy: 'm-priya', addedAt: t(3800), offline: true,
        note: 'Villa Margherita, Via Chiesa Nuova, Positano\nCheck-in from 15:00 · Check-out 10:00\nBooking ref: VMG-48213\nDoor code ar
rives by message on the day.\n\nTaxi phrase:\nVilla Margherita, per favore (Villa Margherita, please)',
      },
      {
        id: uid('d-'), name: 'Passport scans', kind: 'note', addedBy: DEMO_USER_ID, addedAt: t(3600), offline: true,
        note: 'Copies are stored on each member's phone.\nIf a passport is lost: report it to the Carabinieri, then contact your embass
y.\nUS Consulate Naples: +39 081 583 8111',
      },
      {
        id: uid('d-'), name: 'Emergency contacts', kind: 'note', addedBy: 'm-diego', addedAt: t(3000), offline: true,
        note: 'EU emergency 112 · Ambulance 118 · Fire 115\nTravel insurance: TravelSafe +1 800 555 0199, policy TS-88213\nNearest hospi
tal: Ospedale Costa d'Amalfi, Castiglione di Ravello',
      },
    ],
    messages: [
      { id: uid('c-'), authorId: 'm-priya', text: 'Villa is booked! 🏡 Paid the deposit.', ts: t(320) },
      { id: uid('c-'), authorId: 'm-sam', text: 'I'll handle maps & the scooter rental.', ts: t(260) },
      { id: uid('c-'), authorId: 'm-diego', text: 'Dinner tonight at 7, I reserved for 4 🍝', ts: t(200) },
    ],
    photos: [
      { id: uid('ph-'), uri: 'amalfi', caption: 'First look at the coast 😍', by: 'm-sam', day: start, ts: t(120), likes: [DEMO_USER_ID,
'm-priya'] },
      { id: uid('ph-'), uri: 'cinque-terre', caption: 'Cinque Terre next time?', by: 'm-priya', day: start, ts: t(90), likes: ['m-dieg
o'] },
    ],
    packing: demoPacking(),
    completed: timeline === 'completed',
    createdAt: t(60 * 24 * 30),
  };
}

export function buildSecondaryTrips(now = Date.now()): Trip[] {
  const today = todayInTz('UTC');
  const baliStart = addDays(today, 120);
  const bcnStart = addDays(today, -160);
  const four = demoMembers.slice(0, 4).map((m) => ({ ...m }));
  return [
    buildTokyoTrip(now),
    {
      id: 'trip-bali', name: 'Bali Surf & Chill', destinationId: 'bali', extraDestinationIds: [], startDate: baliStart, endDate: addDays
(baliStart, 6),
      cover: 'bali', budget: 3000, inviteCode: 'BAL-7316', ownerId: 'm-priya', members: four,
      activities: [
        { id: uid('a-'), date: addDays(baliStart, 1), time: '04:00', title: 'Mount Batur sunrise trek', location: 'Kintamani, Bali', typ
e: 'adventure', durationMin: 360, costPerPerson: 45, status: 'voting', assigned: ['m-priya'], votes: votesFrom(['m-priya', DEMO_USER_I
D]), reminder: false, createdBy: 'm-priya' },
        { id: uid('a-'), date: addDays(baliStart, 2), time: '09:00', title: 'Surf lesson at Kuta', location: 'Kuta, Bali', type: 'advent
ure', durationMin: 150, costPerPerson: 30, status: 'pending', assigned: ['m-sam'], votes: votesFrom(['m-sam']), reminder: false, created
By: 'm-sam' },
      ],
      expenses: [{ id: uid('e-'), title: 'Villa deposit', amount: 600, category: 'Stays', paidBy: 'm-priya', splitBetween: four.map((m)
```

```
=> m.id), date: today }],
      flights: [], polls: [], tasks: [{ id: uid('t-'), text: 'Decide villa vs. hostel', done: false, assignee: 'm-priya' }], docs: [],
      messages: [{ id: uid('c-'), authorId: 'm-priya', text: 'Started planning Bali for next year 🌴 who's in?', ts: now - 86400000 }],
      photos: [], packing: [], createdAt: now - 86400000 * 2,
    },
    {
      id: 'trip-bcn', name: 'Barcelona Spring Break', destinationId: 'barcelona', extraDestinationIds: [], startDate: bcnStart, endDate:
addDays(bcnStart, 5),
      cover: 'barcelona', budget: 2400, inviteCode: 'BCN-4402', ownerId: DEMO_USER_ID, members: four, completed: true,
      activities: [
        { id: uid('a-'), date: addDays(bcnStart, 1), time: '10:00', title: 'Sagrada Família tour', location: 'Eixample', type: 'cultur
e', durationMin: 120, costPerPerson: 34, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(four.map((m) => m.id)), reminde
r: false, createdBy: DEMO_USER_ID, done: true },
        { id: uid('a-'), date: addDays(bcnStart, 2), time: '13:00', title: 'Tapas crawl in El Born', location: 'El Born', type: 'food',
durationMin: 180, costPerPerson: 40, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(four.map((m) => m.id)), reminder: fals
e, createdBy: 'm-diego', done: true },
        { id: uid('a-'), date: addDays(bcnStart, 3), time: '11:00', title: 'Barceloneta beach day', location: 'Barceloneta', type: 'sigh
tseeing', durationMin: 300, costPerPerson: 0, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(four.map((m) => m.id)), reminde
r: false, createdBy: 'm-sam', done: true },
      ],
      expenses: [
        { id: uid('e-'), title: 'Flights', amount: 1320, category: 'Flights', paidBy: DEMO_USER_ID, splitBetween: four.map((m) => m.id),
date: bcnStart },
        { id: uid('e-'), title: 'Airbnb Gothic Quarter', amount: 640, category: 'Stays', paidBy: 'm-priya', splitBetween: four.map((m) =
> m.id), date: bcnStart },
        { id: uid('e-'), title: 'Tapas & dinners', amount: 310, category: 'Food', paidBy: 'm-diego', splitBetween: four.map((m) => m.i
d), date: addDays(bcnStart, 2) },
      ],
      flights: [], polls: [], tasks: [], docs: [], messages: [], packing: [],
      photos: [
        { id: uid('ph-'), uri: 'barcelona', caption: 'Rooftop views', by: 'm-sam', likes: [DEMO_USER_ID], ts: now - 86400000 * 150, day:
addDays(bcnStart, 1) },
      ],
      createdAt: now - 86400000 * 200,
    },
  ];
}

/** Trips that exist "on the server" and can be joined with an invite code. */
export function directoryTrips(now = Date.now()): Trip[] {
  const start = addDays(todayInTz('UTC'), 45);
  const jordan: Member = { id: 'm-jordan', name: 'Jordan Lee', role: 'Organizer', homeCity: 'Philadelphia' };
  const kai: Member = { id: 'm-kai', name: 'Kai Thompson', role: 'Food Lead', homeCity: 'Philadelphia' };
  return [
    {
      id: 'trip-seoul', name: 'Seoul Food Crawl', destinationId: 'seoul', extraDestinationIds: [], startDate: start, endDate: addDays(st
art, 5),
      cover: 'seoul', budget: 2600, inviteCode: 'SEO-5521', ownerId: jordan.id, members: [jordan, kai],
      activities: [
        { id: uid('a-'), date: addDays(start, 1), time: '18:00', title: 'Gwangjang Market night food tour', location: 'Jongno, Seoul', t
ype: 'food', durationMin: 150, costPerPerson: 35, status: 'confirmed', assigned: [kai.id], votes: votesFrom([jordan.id, kai.id]), remind
er: true, createdBy: kai.id },
      ],
      expenses: [], flights: [], polls: [], tasks: [], docs: [], photos: [], packing: [],
      messages: [{ id: uid('c-'), authorId: jordan.id, text: 'Welcome to the Seoul crew! Drop your flight times when you book ✈️', ts: n
ow - 3600000 }],
      createdAt: now - 86400000,
    },
  ];
}

export { AMALFI_TZ as DEMO_TZ };
```

src/store/selectors.ts

```
/** Selectors and derived data (the "view-model" helpers screens use to read the store). */
import { destinationById } from '@/data/destinations';
import { daysBetween, todayInTz } from '@/utils/time';

import { useAppStore } from './store';
import type { Account, Member, Trip, TripStatus } from './types';

export function useCurrentUser(): Account | undefined {
  return useAppStore((s) => s.accounts.find((a) => a.id === s.session?.userId));
}

export function useActiveTrip(): Trip | undefined {
  return useAppStore((s) => {
    const uidNow = s.session?.userId;
    const t = s.trips.find((x) => x.id === s.activeTripId);
    if (t && (!uidNow || t.members.some((m) => m.id === uidNow))) return t;
    return undefined;
  });
}

export function useMyTrips(): Trip[] {
```

```
  const userId = useAppStore((s) => s.session?.userId);
  const trips = useAppStore((s) => s.trips);
  return trips.filter((t) => t.members.some((m) => m.id === userId));
}

export function tripStatus(t: Trip): TripStatus {
  if (t.completed) return 'completed';
  const dest = destinationById(t.destinationId);
  const today = todayInTz(dest?.tz ?? 'UTC');
  if (daysBetween(today, t.startDate) > 0) return 'planning';
  if (daysBetween(t.endDate, today) > 0) return 'completed';
  return 'active';
}

export function memberById(t: Trip | undefined, id: string): Member | undefined {
  return t?.members.find((m) => m.id === id);
}
```

src/store/slices/app.ts

```
/**
 * App-wide state: onboarding, settings, personalization, bucket list, favourites, notifications and cached weather/rates.
 */
import { layoutAnswers } from '@/data/layouts';
import { uid } from '@/utils/format';
import { nextStreak } from '@/utils/trivia';

import type { AppNotification, BucketItem, RatesSnapshot, Settings, WeatherSnapshot, Personal } from '../types';
import { initialAuth } from './auth';
import { initialDemo } from './demo';
import { initialTrips } from './trips';
import type { Slice } from './types';

export const defaultSettings: Settings = {
  language: 'en',
  homeCurrency: 'USD',
  notificationsEnabled: true,
  reminderLeadMin: 30,
  textScale: 1,
  showTripTime: true,
  aiMode: 'offline',
  haptics: true,
};

export const defaultPersonal: Personal = {
  done: false,
  nickname: '',
  emoji: '✈️',
  interests: [],
  travelStyle: 'balanced',
  budgetStyle: 'mid',
  ...layoutAnswers('planner'),
  effects: 'lively',
  triviaReminder: false,
  motion: true,
};

/** Today's trivia question: which day it was answered, what was picked, and the streak of days answered right in a row. */
export interface TriviaState {
  day: string | null;
  picked: number | null;
  streak: number;
  lastCorrectDay: string | null;
}

export interface AppSlice {
  hydrated: boolean;
  hasOnboarded: boolean;
  bucket: BucketItem[];
  favorites: string[];
  notifications: AppNotification[];
  settings: Settings;
  weather: Record<string, WeatherSnapshot>;
  rates: RatesSnapshot | null;
  quizBest: number;
  surveyResult: string[] | null;
  personal: Personal;
  trivia: TriviaState;
  restyled: boolean;
  firedReminders: Record<string, number>;
  setHydrated: () => void;
  completeOnboarding: () => void;
  addBucket: (text: string, destinationId?: string) => void;
  toggleBucket: (id: string) => void;
  deleteBucket: (id: string) => void;
  toggleFavorite: (destinationId: string) => void;
```

```
  pushNotification: (n: Omit<AppNotification, 'id' | 'ts' | 'read'>) => void;
  markAllRead: () => void;
  markKindRead: (kind: AppNotification['kind']) => void;
  clearNotifications: () => void;
  updateSettings: (patch: Partial<Settings>) => void;
  setWeather: (w: WeatherSnapshot) => void;
  setRates: (r: RatesSnapshot) => void;
  setQuizBest: (n: number) => void;
  setSurveyResult: (ids: string[] | null) => void;
  updatePersonal: (patch: Partial<Personal>) => void;
  /** Records the answer to the day's trivia question (only the first answer of a day counts). */
  answerTrivia: (day: string, picked: number, correct: boolean) => void;
  setRestyled: (v: boolean) => void;
  setToured: (v: boolean) => void;
  markReminderFired: (key: string) => void;
  resetEverything: () => void;
}

/** Starting values, also used by "reset everything". */
export const initialApp = {
  hydrated: false,
  hasOnboarded: false,
  bucket: [],
  favorites: ['tokyo', 'kyoto'],
  notifications: [],
  settings: defaultSettings,
  weather: {},
  rates: null,
  quizBest: 0,
  surveyResult: null,
  personal: defaultPersonal,
  trivia: { day: null, picked: null, streak: 0, lastCorrectDay: null } as TriviaState,
  restyled: false,
  firedReminders: {},
};

export const createAppSlice: Slice<AppSlice> = (set) => {
  return {
    ...initialApp,
    setHydrated: () => set({ hydrated: true }),

    completeOnboarding: () => set({ hasOnboarded: true }),

    addBucket: (text, destinationId) =>
      set((s) => {
        s.bucket.unshift({ id: uid('b-'), text: text.trim(), destinationId, done: false, addedAt: Date.now() });
      }),

    toggleBucket: (id) =>
      set((s) => {
        const b = s.bucket.find((x) => x.id === id);
        if (b) b.done = !b.done;
      }),

    deleteBucket: (id) =>
      set((s) => {
        s.bucket = s.bucket.filter((x) => x.id !== id);
      }),

    toggleFavorite: (destinationId) =>
      set((s) => {
        s.favorites = s.favorites.includes(destinationId)
          ? s.favorites.filter((x) => x !== destinationId)
          : [...s.favorites, destinationId];
      }),

    pushNotification: (n) =>
      set((s) => {
        s.notifications.unshift({ ...n, id: uid('n-'), ts: Date.now(), read: false });
        if (s.notifications.length > 60) s.notifications.length = 60;
      }),

    markAllRead: () =>
      set((s) => {
        s.notifications.forEach((n) => (n.read = true));
      }),

    markKindRead: (kind) =>
      set((s) => {
        s.notifications.forEach((n) => {
          if (n.kind === kind) n.read = true;
        });
      }),

    clearNotifications: () => set({ notifications: [] }),
```

```
    updateSettings: (patch) =>
      set((s) => {
        Object.assign(s.settings, patch);
      }),

    setWeather: (w) =>
      set((s) => {
        s.weather[w.destinationId] = w;
      }),

    setRates: (r) => set({ rates: r }),

    setQuizBest: (n) => set((s) => ({ quizBest: Math.max(s.quizBest, n) })),

    setSurveyResult: (ids) => set({ surveyResult: ids }),

    updatePersonal: (patch) =>
      set((s) => {
        Object.assign(s.personal, patch);
      }),

    answerTrivia: (day, picked, correct) =>
      set((s) => {
        if (s.trivia.day === day) return;
        s.trivia.streak = nextStreak(s.trivia.streak, s.trivia.lastCorrectDay, day, correct);
        if (correct) s.trivia.lastCorrectDay = day;
        s.trivia.day = day;
        s.trivia.picked = picked;
      }),

    setRestyled: (v) => set({ restyled: v }),

    setToured: (v) =>
      set((s) => {
        s.personal.toured = v;
      }),

    markReminderFired: (key) =>
      set((s) => {
        s.firedReminders[key] = Date.now();
      }),

    resetEverything: () =>
      set(() => ({ ...initialAuth,
        ...initialTrips,
        ...initialDemo,
        ...initialApp, hydrated: true, hasOnboarded: true })),
  };
};
```

src/store/slices/auth.ts

```
/**
 * Accounts and sign-in: who is using this phone.
 */
import * as Crypto from 'expo-crypto';
import { uid } from '@/utils/format';
import { EMAIL_RE } from '@/utils/validation';
import { hashPassword, verifyPassword } from '@/utils/password';

import type { Account } from '../types';
import type { Slice } from './types';

interface Session {
  userId: string;
  remember: boolean;
}

export type Result = { ok: true } | { ok: false; error: string };

export interface AuthSlice {
  session: Session | null;
  accounts: Account[];
  signIn: (email: string, password: string, remember: boolean) => Promise<Result>;
  signUp: (name: string, email: string, password: string) => Promise<Result>;
  signInWithGoogle: (email: string, name: string) => Promise<Result>;
  signOut: () => void;
  accountExists: (email: string) => boolean;
  resetPassword: (email: string, newPassword: string) => Promise<Result>;
  updateProfile: (patch: Partial<Pick<Account, 'name' | 'homeCity' | 'avatar'>>) => void;
}

/** Starting values, also used by "reset everything". */
export const initialAuth = {
  session: null,
  accounts: [],
```

```
};

export const createAuthSlice: Slice<AuthSlice> = (set, get) => {
  return {
    ...initialAuth,
    signIn: async (email, password, remember) => {
      const acct = get().accounts.find((a) => a.email.toLowerCase() === email.trim().toLowerCase());
      if (!acct) return { ok: false, error: 'No account found with that email. Try Sign Up instead.' };
      if (acct.provider === 'google' && !acct.passwordHash) return { ok: false, error: 'This account uses Google sign-in.' };
      const check = await verifyPassword(password, acct);
      if (!check.ok) return { ok: false, error: 'Incorrect password. Please try again.' };
      set((s) => {
        // Old accounts used a single SHA-256 round: quietly move them to the stronger hash.
        if (check.upgrade) {
          const mine = s.accounts.find((a) => a.id === acct.id);
          if (mine) mine.passwordHash = check.upgrade;
        }
        s.session = { userId: acct.id, remember };
        const mine = s.trips.find((t) => t.members.some((m) => m.id === acct.id));
        if (!s.trips.find((t) => t.id === s.activeTripId && t.members.some((m) => m.id === acct.id))) {
          s.activeTripId = mine?.id ?? null;
        }
      });
      return { ok: true };
    },

    signUp: async (name, email, password) => {
      const e = email.trim().toLowerCase();
      if (!EMAIL_RE.test(e)) return { ok: false, error: 'Enter a valid email address.' };
      if (get().accounts.some((a) => a.email.toLowerCase() === e)) {
        return { ok: false, error: 'An account with this email already exists. Sign in instead.' };
      }
      const salt = Crypto.randomUUID();
      const passwordHash = await hashPassword(password, salt);
      const id = uid('u-');
      set((s) => {
        s.accounts.push({ id, name: name.trim(), email: e, passwordHash, salt, provider: 'password' });
        s.session = { userId: id, remember: true };
        s.activeTripId = null;
      });
      return { ok: true };
    },

    signInWithGoogle: async (email, name) => {
      const e = email.trim().toLowerCase();
      let acct = get().accounts.find((a) => a.email.toLowerCase() === e);
      if (!acct) {
        acct = { id: uid('u-'), name, email: e, passwordHash: '', salt: '', provider: 'google' };
        const created = acct;
        set((s) => {
          s.accounts.push(created);
        });
      }
      const id = acct.id;
      set((s) => {
        s.session = { userId: id, remember: true };
        const mine = s.trips.find((t) => t.members.some((m) => m.id === id));
        s.activeTripId = mine?.id ?? null;
      });
      return { ok: true };
    },

    signOut: () => set({ session: null }),

    accountExists: (email) => get().accounts.some((a) => a.email.toLowerCase() === email.trim().toLowerCase()),

    resetPassword: async (email, newPassword) => {
      const e = email.trim().toLowerCase();
      if (!get().accounts.some((a) => a.email.toLowerCase() === e)) return { ok: false, error: 'No account found with that email.' };
      const salt = Crypto.randomUUID();
      const passwordHash = await hashPassword(newPassword, salt);
      set((s) => {
        const a = s.accounts.find((x) => x.email.toLowerCase() === e);
        if (a) {
          a.salt = salt;
          a.passwordHash = passwordHash;
          a.provider = 'password';
        }
      });
      return { ok: true };
    },

    updateProfile: (patch) =>
      set((s) => {
        const a = s.accounts.find((x) => x.id === s.session?.userId);
        if (!a) return;
```

```
        Object.assign(a, patch);
        for (const t of s.trips) {
          const m = t.members.find((x) => x.id === a.id);
          if (m) {
            if (patch.name) m.name = patch.name;
            if (patch.avatar !== undefined) m.avatar = patch.avatar;
            if (patch.homeCity !== undefined) m.homeCity = patch.homeCity;
          }
        }
      }),
  };
};
```

src/store/slices/budget.ts

```
/**
 * Money: expenses and settle-up payments. The maths itself lives in utils/money.ts.
 */
import { uid } from '@/utils/format';
import { todayInTz } from '@/utils/time';

import type { Expense } from '../types';
import { tripHelpers } from './helpers';
import type { Slice } from './types';

export interface BudgetSlice {
  addExpense: (tripId: string, e: Omit<Expense, 'id'>) => void;
  deleteExpense: (tripId: string, id: string) => void;
  settleUp: (tripId: string, from: string, to: string, amount: number) => void;
}

export const createBudgetSlice: Slice<BudgetSlice> = (set, get) => {
  const { withTrip } = tripHelpers(set, get);
  return {
    addExpense: (tripId, e) =>
      withTrip(tripId, (t) => {
        t.expenses.unshift({ ...e, id: uid('e-') });
      }),

    deleteExpense: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.expenses = t.expenses.filter((x) => x.id !== id);
      }),

    settleUp: (tripId, from, to, amount) =>
      withTrip(tripId, (t) => {
        const toName = t.members.find((m) => m.id === to)?.name.split(' ')[0] ?? 'member';
        t.expenses.unshift({
          id: uid('e-'), title: `Settle-up to ${toName}`, amount: Math.round(amount * 100) / 100, category: 'Other',
          paidBy: from, splitBetween: [to], date: todayInTz('UTC'), settlement: true,
        });
      }),
  };
};
```

src/store/slices/demo.ts

```
/**
 * Demo content: the sample trips and the "Demo timeline" switch (upcoming / in progress / finished).
 */
import * as Crypto from 'expo-crypto';

import { uid } from '@/utils/format';
import { hashPassword } from '@/utils/password';

import { DEMO_EMAIL, DEMO_PASSWORD, DEMO_TRIP_IDS, DEMO_USER_ID, buildDemoTrip, buildSecondaryTrips, type DemoTimeline } from '../seed';
import type { Account, AppNotification, BucketItem } from '../types';
import type { Slice } from './types';

/** The sample bucket-list items: text, destination, done. */
const BUCKET_SEED: [string, string, boolean][] = [
  ['See the northern lights in Iceland', 'iceland', false],
  ['Sunrise at Machu Picchu', 'machu-picchu', false],
  ['Eat street tacos in Mexico City', 'mexico-city', false],
  ['Watch the sunset in Oia, Santorini', 'santorini', false],
  ['Tapas crawl in Barcelona', 'barcelona', true],
];

function seedBucket(): BucketItem[] {
  const now = Date.now();
  return BUCKET_SEED.map(([text, destinationId, done], i) => ({ id: uid('b-'), text, destinationId, done, addedAt: now - (9 - i) * 1e7
}));
}

function seedNotifications(): AppNotification[] {
  const now = Date.now();
```

```
  return [
    { id: uid('n-'), kind: 'vote', title: 'New poll from Sam', body: 'Beach day — which beach? Cast your vote.', ts: now - 600 * 60000,
read: false, tripId: 'trip-amalfi', route: '/crew' },
    { id: uid('n-'), kind: 'expense', title: 'Diego added an expense', body: 'Dinner deposits & groceries · $86 split 4 ways', ts: now -
420 * 60000, read: false, tripId: 'trip-amalfi', route: '/(tabs)/budget' },
    { id: uid('n-'), kind: 'chat', title: 'Diego in Amalfi Squad', body: 'Dinner tonight at 7, I reserved for 4 🍝', ts: now - 42 * 6000
0, read: false, tripId: 'trip-amalfi', route: '/(tabs)/chat' },
  ];
}

export interface DemoSlice {
  demoTimeline: DemoTimeline;
  seeded: boolean;
  ensureSeed: () => Promise<void>;
  reseedDemo: (timeline: DemoTimeline) => void;
  /** Removes the sample trips, their notifications and the sample bucket list; accounts and your own trips stay. */
  clearDemoData: () => void;
}

/** Starting values, also used by "reset everything". */
export const initialDemo = {
  demoTimeline: 'upcoming' as DemoTimeline,
  seeded: false,
};

export const createDemoSlice: Slice<DemoSlice> = (set, get) => {
  return {
    ...initialDemo,
    ensureSeed: async () => {
      if (get().seeded) return;
      const salt = Crypto.randomUUID();
      const passwordHash = await hashPassword(DEMO_PASSWORD, salt);
      const demo: Account = { id: DEMO_USER_ID, name: 'Maya Chen', email: DEMO_EMAIL, passwordHash, salt, avatar: 'avatar-maya', homeCit
y: 'New York', provider: 'password' };
      set((s) => {
        if (s.seeded) return;
        s.accounts = [demo, ...s.accounts.filter((a) => a.email !== DEMO_EMAIL)];
        s.trips = [buildDemoTrip('upcoming'), ...buildSecondaryTrips(), ...s.trips];
        s.activeTripId = 'trip-amalfi';
        s.bucket = seedBucket();
        s.notifications = seedNotifications();
        s.seeded = true;
      });
    },

    reseedDemo: (timeline) =>
      set((s) => {
        const fresh = buildDemoTrip(timeline);
        s.trips = [fresh, ...s.trips.filter((t) => t.id !== fresh.id)];
        s.activeTripId = fresh.id;
        s.demoTimeline = timeline;
        s.firedReminders = {};
      }),

    clearDemoData: () =>
      set((s) => {
        s.trips = s.trips.filter((t) => !DEMO_TRIP_IDS.includes(t.id));
        s.activeTripId = s.trips.find((t) => t.members.some((m) => m.id === s.session?.userId))?.id ?? null;
        s.notifications = s.notifications.filter((n) => !n.tripId || !DEMO_TRIP_IDS.includes(n.tripId));
        const sample = new Set(BUCKET_SEED.map(([text]) => text));
        s.bucket = s.bucket.filter((b) => !sample.has(b.text));
        s.firedReminders = {};
      }),
  };
};
```

src/store/slices/files.ts

```
/**
 * Lets the sync code record which file on the sync server belongs to a photo, a document or a receipt.
 */
import { tripHelpers } from './helpers';
import type { Slice } from './types';

export type FileList = 'photos' | 'docs' | 'expenses';

export interface FilesSlice {
  attachBlob: (tripId: string, list: FileList, itemId: string, blobId: string) => void;
}

export const createFilesSlice: Slice<FilesSlice> = (set, get) => {
  const { withTrip } = tripHelpers(set, get);
  return {
    attachBlob: (tripId, list, itemId, blobId) =>
      withTrip(tripId, (t) => {
        if (list === 'photos') {
```

```
          const photo = t.photos.find((x) => x.id === itemId);
          if (photo) photo.blob = blobId;
        } else if (list === 'docs') {
          const doc = t.docs.find((x) => x.id === itemId);
          if (doc) doc.blob = blobId;
        } else {
          const expense = t.expenses.find((x) => x.id === itemId);
          if (expense) expense.receiptBlob = blobId;
        }
      }),
  };
};
```

src/store/slices/groupTools.ts

```
/**
 * Shared group tools: polls, the checklist, the packing list and the document vault.
 */
import { uid } from '@/utils/format';

import type { PackingItem, Poll, TripDoc } from '../types';
import { tripHelpers } from './helpers';
import type { Slice } from './types';

export interface GroupToolsSlice {
  createPoll: (tripId: string, question: string, options: string[]) => void;
  votePoll: (tripId: string, pollId: string, optionId: string) => void;
  closePoll: (tripId: string, pollId: string) => void;
  deletePoll: (tripId: string, pollId: string) => void;
  addTask: (tripId: string, text: string, assignee?: string) => void;
  toggleTask: (tripId: string, id: string) => void;
  deleteTask: (tripId: string, id: string) => void;
  assignTask: (tripId: string, id: string, memberId?: string) => void;
  addDoc: (tripId: string, d: Omit<TripDoc, 'id' | 'addedAt' | 'addedBy'>) => void;
  deleteDoc: (tripId: string, id: string) => void;
  togglePacked: (tripId: string, id: string) => void;
  assignPacking: (tripId: string, id: string, memberId?: string) => void;
  addPackingItems: (tripId: string, items: Omit<PackingItem, 'id' | 'packed'>[]) => void;
  deletePackingItem: (tripId: string, id: string) => void;
}

export const createGroupToolsSlice: Slice<GroupToolsSlice> = (set, get) => {
  const { withTrip, me } = tripHelpers(set, get);
  return {
    createPoll: (tripId, question, options) =>
      withTrip(tripId, (t) => {
        t.polls.unshift({
          id: uid('poll-'), question: question.trim(), createdBy: me(), createdAt: Date.now(),
          options: options.map((text, i) => ({ id: `o${i + 1}`, text: text.trim(), votes: [] })),
        } satisfies Poll);
        t.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `New poll: ${question.trim()}`, ts: Date.now() });
      }),

    votePoll: (tripId, pollId, optionId) =>
      withTrip(tripId, (t) => {
        const p = t.polls.find((x) => x.id === pollId);
        if (!p || p.closed) return;
        const user = me();
        const already = p.options.find((o) => o.id === optionId)?.votes.includes(user);
        p.options.forEach((o) => (o.votes = o.votes.filter((v) => v !== user)));
        if (!already) p.options.find((o) => o.id === optionId)?.votes.push(user);
      }),

    closePoll: (tripId, pollId) =>
      withTrip(tripId, (t) => {
        const p = t.polls.find((x) => x.id === pollId);
        if (p) p.closed = !p.closed;
      }),

    deletePoll: (tripId, pollId) =>
      withTrip(tripId, (t) => {
        t.polls = t.polls.filter((x) => x.id !== pollId);
      }),

    addTask: (tripId, text, assignee) =>
      withTrip(tripId, (t) => {
        t.tasks.push({ id: uid('t-'), text: text.trim(), done: false, assignee });
      }),

    toggleTask: (tripId, id) =>
      withTrip(tripId, (t) => {
        const k = t.tasks.find((x) => x.id === id);
        if (k) k.done = !k.done;
      }),

    deleteTask: (tripId, id) =>
```

```
      withTrip(tripId, (t) => {
        t.tasks = t.tasks.filter((x) => x.id !== id);
      }),

    assignTask: (tripId, id, memberId) =>
      withTrip(tripId, (t) => {
        const k = t.tasks.find((x) => x.id === id);
        if (k) k.assignee = memberId;
      }),

    addDoc: (tripId, d) =>
      withTrip(tripId, (t) => {
        t.docs.unshift({ ...d, id: uid('d-'), addedAt: Date.now(), addedBy: me() });
      }),

    deleteDoc: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.docs = t.docs.filter((x) => x.id !== id);
      }),

    togglePacked: (tripId, id) =>
      withTrip(tripId, (t) => {
        const p = t.packing.find((x) => x.id === id);
        if (p) p.packed = !p.packed;
      }),

    assignPacking: (tripId, id, memberId) =>
      withTrip(tripId, (t) => {
        const p = t.packing.find((x) => x.id === id);
        if (p) p.broughtBy = memberId;
      }),

    addPackingItems: (tripId, items) =>
      withTrip(tripId, (t) => {
        const have = new Set(t.packing.map((p) => p.text.toLowerCase()));
        items.forEach((i) => {
          const key = i.text.toLowerCase();
          if (have.has(key)) return;
          have.add(key); // also skips duplicates inside this same batch
          t.packing.push({ ...i, id: uid('p-'), packed: false });
        });
      }),

    deletePackingItem: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.packing = t.packing.filter((x) => x.id !== id);
      }),
  };
};
```

src/store/slices/helpers.ts

```
import { DEMO_USER_ID } from '../seed';
import type { Trip } from '../types';
import type { AppStore, SliceGet, SliceSet } from './types';

/** Shortcuts for every slice that edits a trip. */
export function tripHelpers(set: SliceSet, get: SliceGet) {
  /** Runs `fn` on the trip with this id. The trip is an immer draft, so it can be changed in place. */
  const withTrip = (tripId: string, fn: (t: Trip, s: AppStore) => void) =>
    set((s) => {
      const t = s.trips.find((x) => x.id === tripId);
      if (t) fn(t, s);
    });
  /** The signed-in user's id (the demo user when nobody is signed in, so the demo screens always work). */
  const me = () => get().session?.userId ?? DEMO_USER_ID;
  return { withTrip, me };
}
```

src/store/slices/itinerary.ts

```
/**
 * The day-by-day plan: activities (with voting) and flights.
 */
import { uid } from '@/utils/format';

import type { Activity, Flight } from '../types';
import { tripHelpers } from './helpers';
import type { Slice } from './types';

export interface ItinerarySlice {
  addActivity: (tripId: string, a: Omit<Activity, 'id' | 'createdBy' | 'votes'> & { votes?: Activity['votes'] }) => string;
  updateActivity: (tripId: string, id: string, patch: Partial<Activity>) => void;
  deleteActivity: (tripId: string, id: string) => void;
  voteActivity: (tripId: string, id: string, value: 1 | -1) => void;
  toggleAssign: (tripId: string, id: string, memberId: string) => void;
```

```
  addFlight: (tripId: string, f: Omit<Flight, 'id'>) => void;
  deleteFlight: (tripId: string, id: string) => void;
}

export const createItinerarySlice: Slice<ItinerarySlice> = (set, get) => {
  const { withTrip, me } = tripHelpers(set, get);
  return {
    addActivity: (tripId, a) => {
      const id = uid('a-');
      withTrip(tripId, (t) => {
        t.activities.push({ ...a, votes: a.votes ?? { [me()]: 1 }, id, createdBy: me() });
      });
      return id;
    },

    updateActivity: (tripId, id, patch) =>
      withTrip(tripId, (t) => {
        const a = t.activities.find((x) => x.id === id);
        if (a) Object.assign(a, patch);
      }),

    deleteActivity: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.activities = t.activities.filter((x) => x.id !== id);
      }),

    voteActivity: (tripId, id, value) =>
      withTrip(tripId, (t) => {
        const a = t.activities.find((x) => x.id === id);
        if (!a) return;
        const user = me();
        if (a.votes[user] === value) delete a.votes[user];
        else a.votes[user] = value;
        // Majority rule: a voting item is confirmed once more than half the group upvotes it.
        const ups = Object.values(a.votes).filter((v) => v === 1).length;
        if (a.status === 'voting' && ups > t.members.length / 2) a.status = 'confirmed';
      }),

    toggleAssign: (tripId, id, memberId) =>
      withTrip(tripId, (t) => {
        const a = t.activities.find((x) => x.id === id);
        if (!a) return;
        a.assigned = a.assigned.includes(memberId) ? a.assigned.filter((x) => x !== memberId) : [...a.assigned, memberId];
      }),

    addFlight: (tripId, f) =>
      withTrip(tripId, (t) => {
        t.flights.push({ ...f, id: uid('f-') });
      }),

    deleteFlight: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.flights = t.flights.filter((x) => x.id !== id);
      }),
  };
};
```

src/store/slices/social.ts

```
/**
 * Chat messages and the photo dump.
 */
import { uid } from '@/utils/format';

import type { ChatMessage, Photo } from '../types';
import { tripHelpers } from './helpers';
import type { Slice } from './types';

export interface SocialSlice {
  sendMessage: (tripId: string, msg: Omit<ChatMessage, 'id' | 'ts'> & { ts?: number }) => void;
  clearChat: (tripId: string) => void;
  addPhoto: (tripId: string, p: Omit<Photo, 'id' | 'ts' | 'likes' | 'by'>) => void;
  toggleLikePhoto: (tripId: string, id: string) => void;
  deletePhoto: (tripId: string, id: string) => void;
}

export const createSocialSlice: Slice<SocialSlice> = (set, get) => {
  const { withTrip, me } = tripHelpers(set, get);
  return {
    sendMessage: (tripId, msg) =>
      withTrip(tripId, (t) => {
        t.messages.push({ ...msg, id: uid('c-'), ts: msg.ts ?? Date.now() });
        if (t.messages.length > 400) t.messages.splice(0, t.messages.length - 400);
      }),

    clearChat: (tripId) =>
```

```
      withTrip(tripId, (t) => {
        t.messages = [];
      }),

    addPhoto: (tripId, p) =>
      withTrip(tripId, (t) => {
        t.photos.unshift({ ...p, id: uid('ph-'), ts: Date.now(), likes: [], by: me() });
      }),

    toggleLikePhoto: (tripId, id) =>
      withTrip(tripId, (t) => {
        const p = t.photos.find((x) => x.id === id);
        if (!p) return;
        const user = me();
        p.likes = p.likes.includes(user) ? p.likes.filter((x) => x !== user) : [...p.likes, user];
      }),

    deletePhoto: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.photos = t.photos.filter((x) => x.id !== id);
      }),
  };
};
```

src/store/slices/trips.ts

```
/**
 * The list of trips: create, join with an invite code, leave, delete, edit, and who is in each trip.
 */
import { destinationById } from '@/data/destinations';
import { uid } from '@/utils/format';
import type { Role } from '@/data/reference';

import { directoryTrips, inviteCodeFor } from '../seed';
import type { Trip } from '../types';
import { tripHelpers } from './helpers';
import type { Slice } from './types';

export interface TripsSlice {
  trips: Trip[];
  activeTripId: string | null;
  setActiveTrip: (id: string) => void;
  createTrip: (t: { name: string; destinationId: string; startDate: string; endDate: string; budget: number }) => string;
  joinTrip: (code: string) => { ok: true; tripId: string } | { ok: false; error: string };
  leaveTrip: (tripId: string) => void;
  deleteTrip: (tripId: string) => void;
  updateTrip: (tripId: string, patch: Partial<Pick<Trip, 'name' | 'startDate' | 'endDate' | 'budget' | 'completed' | 'cover' | 'destinat
ionId' | 'extraDestinationIds'>>) => void;
  regenerateInvite: (tripId: string) => void;
  addMember: (tripId: string, name: string, email?: string) => void;
  removeMember: (tripId: string, memberId: string) => void;
  setRole: (tripId: string, memberId: string, role: Role) => void;
}

/** Starting values, also used by "reset everything". */
export const initialTrips = {
  trips: [],
  activeTripId: null,
};

export const createTripsSlice: Slice<TripsSlice> = (set, get) => {
  const { withTrip, me } = tripHelpers(set, get);
  return {
    ...initialTrips,
    setActiveTrip: (id) => set({ activeTripId: id }),

    createTrip: ({ name, destinationId, startDate, endDate, budget }) => {
      const id = uid('trip-');
      const acct = get().accounts.find((a) => a.id === me());
      const dest = destinationById(destinationId);
      set((s) => {
        s.trips.unshift({
          id, name: name.trim(), destinationId, extraDestinationIds: [], startDate, endDate,
          cover: dest?.image ?? 'planning', budget, inviteCode: inviteCodeFor(dest?.city ?? name), ownerId: me(),
          members: [{ id: me(), name: acct?.name ?? 'You', email: acct?.email, avatar: acct?.avatar, role: 'Organizer', homeCity: acct?.
homeCity }],
          activities: [], expenses: [], flights: [], polls: [], tasks: [
            { id: uid('t-'), text: 'Invite your travel group', done: false, assignee: me() },
            { id: uid('t-'), text: 'Agree on a budget', done: false },
            { id: uid('t-'), text: 'Add everyone’s flights', done: false },
          ], docs: [], photos: [], packing: [],
          messages: [{ id: uid('c-'), authorId: 'assistant', ai: true, aiSource: 'local', ts: Date.now(), text: `Welcome to ${name.trim
()}! Share invite code to bring your group in, then ask me anything — budget, weather, food nearby or what to do next.` }],
          createdAt: Date.now(),
        });
        s.activeTripId = id;
```

```
      });
      return id;
    },

    joinTrip: (code) => {
      const c = code.trim().toUpperCase();
      const user = me();
      const acct = get().accounts.find((a) => a.id === user);
      const local = get().trips.find((t) => t.inviteCode === c);
      if (local) {
        if (local.members.some((m) => m.id === user)) {
          set({ activeTripId: local.id });
          return { ok: false, error: `You're already in "${local.name}".` };
        }
        set((s) => {
          const t = s.trips.find((x) => x.id === local.id)!;
          t.members.push({ id: user, name: acct?.name ?? 'New member', email: acct?.email, avatar: acct?.avatar, role: 'Member', homeCit
y: acct?.homeCity });
          t.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `${acct?.name ?? 'Someone'} joined the trip`, ts: D
ate.now() });
          s.activeTripId = t.id;
        });
        return { ok: true, tripId: local.id };
      }
      const remote = directoryTrips().find((t) => t.inviteCode === c);
      if (!remote) return { ok: false, error: 'No trip found with that code. Check with your friend and try again.' };
      set((s) => {
        remote.members.push({ id: user, name: acct?.name ?? 'New member', email: acct?.email, avatar: acct?.avatar, role: 'Member', home
City: acct?.homeCity });
        remote.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `${acct?.name ?? 'Someone'} joined the trip`, t
s: Date.now() });
        s.trips.unshift(remote);
        s.activeTripId = remote.id;
      });
      return { ok: true, tripId: remote.id };
    },

    leaveTrip: (tripId) =>
      set((s) => {
        const t = s.trips.find((x) => x.id === tripId);
        if (!t) return;
        t.members = t.members.filter((m) => m.id !== s.session?.userId);
        if (s.activeTripId === tripId) {
          s.activeTripId = s.trips.find((x) => x.id !== tripId && x.members.some((m) => m.id === s.session?.userId))?.id ?? null;
        }
      }),

    deleteTrip: (tripId) =>
      set((s) => {
        s.trips = s.trips.filter((t) => t.id !== tripId);
        if (s.activeTripId === tripId) {
          s.activeTripId = s.trips.find((x) => x.members.some((m) => m.id === s.session?.userId))?.id ?? null;
        }
      }),

    updateTrip: (tripId, patch) => withTrip(tripId, (t) => Object.assign(t, patch)),

    regenerateInvite: (tripId) =>
      withTrip(tripId, (t) => {
        t.inviteCode = inviteCodeFor(destinationById(t.destinationId)?.city ?? t.name);
      }),

    addMember: (tripId, name, email) =>
      withTrip(tripId, (t) => {
        t.members.push({ id: uid('m-'), name: name.trim(), email: email?.trim() || undefined, role: 'Member', invited: true });
        t.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `${name.trim()} was added to the trip`, ts: Date.now
() });
      }),

    removeMember: (tripId, memberId) =>
      withTrip(tripId, (t) => {
        t.members = t.members.filter((m) => m.id !== memberId);
        t.activities.forEach((a) => {
          a.assigned = a.assigned.filter((x) => x !== memberId);
          delete a.votes[memberId];
        });
        t.tasks.forEach((k) => {
          if (k.assignee === memberId) k.assignee = undefined;
        });
      }),

    setRole: (tripId, memberId, role) =>
      withTrip(tripId, (t) => {
        const m = t.members.find((x) => x.id === memberId);
        if (m) m.role = role;
      }),
```

```
  };
};
```

src/store/slices/types.ts

```
import type { StateCreator } from 'zustand';

import type { AppSlice } from './app';
import type { AuthSlice } from './auth';
import type { BudgetSlice } from './budget';
import type { DemoSlice } from './demo';
import type { FilesSlice } from './files';
import type { GroupToolsSlice } from './groupTools';
import type { ItinerarySlice } from './itinerary';
import type { SocialSlice } from './social';
import type { TripsSlice } from './trips';

/** Everything in the store: each slice contributes its own state and actions. */
export type AppStore = AuthSlice & TripsSlice & ItinerarySlice & BudgetSlice & GroupToolsSlice & SocialSlice & FilesSlice & DemoSlice &
AppSlice;

/** A slice is a function that gets `set` / `get` and returns its piece of state plus the actions that change it. */
export type Slice<T> = StateCreator<AppStore, [['zustand/persist', unknown], ['zustand/immer', never]], [], T>;
export type SliceSet = Parameters<Slice<object>>[0];
export type SliceGet = Parameters<Slice<object>>[1];
```

src/store/store.ts

```
/**
 * The one store for the whole app (the "model" in MVVM): zustand for state + actions, immer for safe
 * nested updates, and AsyncStorage persistence so everything keeps working offline.
 * The actions are grouped into slices (see ./slices); this file only puts them together.
 */
import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';
import { immer } from 'zustand/middleware/immer';

import { layoutAnswers, layoutOf } from '@/data/layouts';

import { DEMO_TRIP_IDS } from './seed';
import { createAppSlice } from './slices/app';
import { createAuthSlice } from './slices/auth';
import { createBudgetSlice } from './slices/budget';
import { createDemoSlice } from './slices/demo';
import { createFilesSlice } from './slices/files';
import { createGroupToolsSlice } from './slices/groupTools';
import { createItinerarySlice } from './slices/itinerary';
import { createSocialSlice } from './slices/social';
import { createTripsSlice } from './slices/trips';
import type { AppStore } from './slices/types';

export type { AppStore };

export const useAppStore = create<AppStore>()(
  persist(
    immer((...a) => ({
      ...createAuthSlice(...a),
      ...createTripsSlice(...a),
      ...createItinerarySlice(...a),
      ...createBudgetSlice(...a),
      ...createGroupToolsSlice(...a),
      ...createSocialSlice(...a),
      ...createFilesSlice(...a),
      ...createDemoSlice(...a),
      ...createAppSlice(...a),
    })),
    {
      name: 'togetherwego-v1',
      version: 4,
      // v2: "Amalfi Squad" replaced the Tokyo trip as the main demo — re-seed the demo trips once, keep the user's own.
      // v3: a calmer Home — the "Nearby radius" block starts switched off. Only people who never changed the Home sections get it switc
hed off.
      // v4: the new Home (Up next, Weather, Budget, Crew, Play...) with an order and a layout: "glance" became "weather", the quick til
es went away,
      //     and what the person had switched on or off is kept.
      migrate: (persisted, version) => {
        let old = persisted as Partial<AppStore>;
        if (version < 2) {
          old = { ...old, seeded: false, activeTripId: null, trips: (old.trips ?? []).filter((t) => !DEMO_TRIP_IDS.includes(t.id)) };
        }
        const sections = old.personal?.sections;
        if (version < 3 && old.personal && sections && Object.values(sections).every(Boolean)) {
          old = { ...old, personal: { ...old.personal, sections: { ...sections, nearby: false } } };
        }
        if (version < 4 && old.personal) {
```

```
          const { glance, budget, upnext, forYou, nearby, discover } = (old.personal.sections ?? {}) as Record<string, boolean | undefin
ed>;
          const planner = layoutAnswers('planner');
          const sections = { ...planner.sections, weather: glance ?? true, budget: budget ?? true, upnext: upnext ?? true, forYou: forYo
u ?? true, nearby: nearby ?? false, discover: discover ?? true };
          old = { ...old, personal: { ...old.personal, sections, order: planner.order, layout: layoutOf({ order: planner.order, sections
}), effects: 'lively', triviaReminder: false } };
        }
        return old as AppStore;
      },
      // New settings added in later versions fall back to their defaults instead of vanishing.
      merge: (persisted, current) => {
        const p = (persisted ?? {}) as Partial<AppStore>;
        return { ...current, ...p, personal: { ...current.personal, ...p.personal, sections: { ...current.personal.sections, ...p.person
al?.sections } } } as AppStore;
      },
      storage: createJSONStorage(() => AsyncStorage),
      partialize: (s) => {
        // Don't persist the session unless "Remember me" was ticked.
        const { hydrated, ...rest } = s;
        void hydrated;
        return { ...rest, session: s.session?.remember || s.restyled ? s.session : null };
      },
      onRehydrateStorage: () => (state) => {
        state?.setHydrated();
      },
    },
  ),
);
```

src/store/types.ts

```
import type { Role } from '@/data/reference';
import type { LangCode } from '@/i18n/strings';

export type ActivityType = 'transport' | 'stay' | 'food' | 'sightseeing' | 'adventure' | 'culture' | 'nightlife' | 'shopping';
export type ActivityStatus = 'confirmed' | 'voting' | 'pending';

export interface Member {
  id: string;
  name: string;
  email?: string;
  avatar?: string;
  role: Role;
  homeCity?: string;
  /** Members added by name only (no account yet). */
  invited?: boolean;
}

export interface Activity {
  id: string;
  date: string;
  time: string;
  title: string;
  location: string;
  type: ActivityType;
  durationMin: number;
  costPerPerson: number;
  status: ActivityStatus;
  assigned: string[];
  /** memberId → +1 (up) / -1 (down) */
  votes: Record<string, 1 | -1>;
  notes?: string;
  reminder: boolean;
  lat?: number;
  lng?: number;
  placeId?: string;
  createdBy: string;
  done?: boolean;
}

export type ExpenseCategory = 'Flights' | 'Stays' | 'Food' | 'Activities' | 'Transport' | 'Shopping' | 'Other';

export interface Expense {
  id: string;
  title: string;
  /** Always stored in the trip's base currency (USD by default). */
  amount: number;
  category: ExpenseCategory;
  paidBy: string;
  splitBetween: string[];
  date: string;
  /** Original amount when entered in a foreign currency. */
  original?: { amount: number; currency: string };
  settlement?: boolean;
  /** Photo uri of the receipt (on the phone that took it). */
```

```
  receipt?: string;
  /** The receipt's file on the sync server, so friends' phones can fetch it. */
  receiptBlob?: string;
}

export interface Flight {
  id: string;
  memberId: string;
  airline: string;
  flightNo: string;
  from: string;
  to: string;
  departUtc: number;
  arriveUtc: number;
  direction: 'arrival' | 'departure';
  seat?: string;
  confirmation?: string;
  ticketUri?: string;
  ticketName?: string;
}

export interface PollOption {
  id: string;
  text: string;
  votes: string[];
}

export interface Poll {
  id: string;
  question: string;
  options: PollOption[];
  createdBy: string;
  closed?: boolean;
  createdAt: number;
}

export interface Task {
  id: string;
  text: string;
  done: boolean;
  assignee?: string;
}

export interface TripDoc {
  id: string;
  name: string;
  kind: 'pdf' | 'image' | 'note' | 'file';
  uri?: string;
  /** The file on the sync server, so friends' phones can fetch it. */
  blob?: string;
  note?: string;
  size?: number;
  addedBy: string;
  addedAt: number;
  offline: boolean;
}

export interface ChatMessage {
  id: string;
  authorId: string;
  text: string;
  ts: number;
  kind?: 'text' | 'system';
  /** True when the message came from the AI trip assistant. */
  ai?: boolean;
  aiSource?: 'local' | 'claude';
  /** Which thread the message belongs to: the human crew chat (default) or the Scout AI thread. */
  channel?: 'crew' | 'ai';
}

export interface Photo {
  id: string;
  uri: string;
  /** The file on the sync server, so friends' phones can fetch it. */
  blob?: string;
  caption: string;
  by: string;
  likes: string[];
  ts: number;
  day?: string;
}

export interface PackingItem {
  id: string;
  text: string;
  category: string;
```

```
  packed: boolean;
  /** Member id of whoever is bringing it. */
  broughtBy?: string;
}

export type TripStatus = 'planning' | 'active' | 'completed';

export interface Trip {
  id: string;
  name: string;
  destinationId: string;
  extraDestinationIds: string[];
  startDate: string;
  endDate: string;
  cover: string;
  /** Group budget cap in USD. */
  budget: number;
  inviteCode: string;
  ownerId: string;
  members: Member[];
  activities: Activity[];
  expenses: Expense[];
  flights: Flight[];
  polls: Poll[];
  tasks: Task[];
  docs: TripDoc[];
  messages: ChatMessage[];
  photos: Photo[];
  packing: PackingItem[];
  /** Manual override: user marked the trip as finished. */
  completed?: boolean;
  createdAt: number;
}

export interface Account {
  id: string;
  name: string;
  email: string;
  passwordHash: string;
  salt: string;
  avatar?: string;
  homeCity?: string;
  provider: 'password' | 'google';
}

export interface BucketItem {
  id: string;
  text: string;
  destinationId?: string;
  done: boolean;
  addedAt: number;
}

export interface AppNotification {
  id: string;
  title: string;
  body: string;
  ts: number;
  read: boolean;
  kind: 'reminder' | 'chat' | 'vote' | 'expense' | 'system' | 'member' | 'photo';
  tripId?: string;
  route?: string;
}

export type TextScale = 1 | 1.12 | 1.25;

export interface Settings {
  language: LangCode;
  homeCurrency: string;
  notificationsEnabled: boolean;
  reminderLeadMin: number;
  textScale: TextScale;
  showTripTime: boolean;
  aiMode: 'offline' | 'claude';
  haptics: boolean;
}

/** Home-screen blocks the person can show, hide and put in their own order. */
export type HomeSection = 'upnext' | 'weather' | 'budget' | 'crew' | 'play' | 'forYou' | 'nearby' | 'discover';

/** A ready-made Home order (Planner, Money, Party), or 'custom' once the person has changed it by hand. */
export type HomeLayout = 'planner' | 'money' | 'party' | 'custom';

/** How much celebration there is: Chill = none, Lively = a little confetti, Max = a lot. */
export type Effects = 'chill' | 'lively' | 'max';
```

```
/** Answers from the "Make it yours" personalization survey. */
export interface Personal {
  done: boolean;
  nickname: string;
  emoji: string;
  interests: string[];
  travelStyle: 'chill' | 'balanced' | 'packed';
  budgetStyle: 'saver' | 'mid' | 'treat';
  sections: Record<HomeSection, boolean>;
  /** Top-to-bottom order of every Home block (hidden ones too, so they come back where they were). */
  order: HomeSection[];
  layout: HomeLayout;
  effects: Effects;
  /** A daily nudge to play today's trivia question. */
  triviaReminder: boolean;
  motion: boolean;
  /** True once the first-run guided tour has been seen. */
  toured?: boolean;
}

export interface WeatherSnapshot {
  destinationId: string;
  fetchedAt: number;
  current: { temp: number; code: number; wind: number; humidity: number };
  daily: { date: string; max: number; min: number; code: number; rain: number }[];
  live: boolean;
}

export interface RatesSnapshot {
  base: 'USD';
  rates: Record<string, number>;
  fetchedAt: number;
  live: boolean;
}
```

src/store/useAppStore.ts

```
/**
 * Public entry point for app state. Screens import from here:
 *   import { useAppStore, useActiveTrip, balances } from '@/store/useAppStore';
 * The pieces live in ./store (the store itself), ./slices (the actions), ./selectors (reading helpers)
 * and utils/money (expense maths).
 */
export { useAppStore, type AppStore } from './store';
export { memberById, tripStatus, useActiveTrip, useCurrentUser, useMyTrips } from './selectors';
export { defaultPersonal } from './slices/app';
export { balances, settlePlan, totalSpent } from '@/utils/money';
```

src/sync/__tests__/blobs.test.ts

```
import type { Trip } from '@/store/types';

import { SyncStore, SyncError } from '../../../server/store';
import { BlobSync, FileTransferError, MAX_SHARED_FILE_BYTES, assetRef, extensionOf, fileNameOf, isAssetRef, isLocalFile, makeBlobId, res
olveFile, type BlobFiles } from '../blobs';
import { SyncEngine } from '../engine';
import { FakeStore, MemoryStorage, fakeNow, makeTrip, memoryTransport, seeded } from '../testing';

type Server = InstanceType<typeof SyncStore>;

/** A phone's storage and its connection to the server's files, in memory. */
function fakeFiles(server: Server, name: string) {
  const disk = new Map<string, Buffer>();
  const files: BlobFiles & { offline: boolean; uploads: string[]; downloads: string[]; disk: Map<string, Buffer> } = {
    offline: false,
    uploads: [],
    downloads: [],
    disk,
    exists: async (uri) => disk.has(uri),
    size: async (uri) => disk.get(uri)?.length ?? null,
    async upload(uri, url) {
      if (files.offline) throw new FileTransferError('network', 'offline');
      const [, tripId, fileId] = /\/v1\/trips\/([^/]+)\/files\/([^/]+)$/.exec(url)!;
      files.uploads.push(fileId);
      try {
        await server.putBlob(tripId, fileId, disk.get(uri)!);
      } catch (e) {
        const status = e instanceof SyncError ? e.status : 500;
        throw new FileTransferError(status === 404 ? 'missing' : status === 400 || status === 413 ? 'rejected' : 'network', String(e));
      }
    },
    async download(url, fileName) {
      if (files.offline) throw new FileTransferError('network', 'offline');
      const [, tripId, fileId] = /\/v1\/trips\/([^/]+)\/files\/([^/]+)$/.exec(url)!;
      files.downloads.push(fileId);
      const bytes = await server.getBlob(tripId, fileId);
```

```
      if (!bytes) throw new FileTransferError('missing', 'no such file');
      const uri = `file:///${name}/shared/${fileName}`;
      disk.set(uri, bytes);
      return uri;
    },
  };
  return files;
}

// a pass that keeps failing schedules its own retries: stop them so the test run can finish
const started: { blobs: BlobSync }[] = [];
afterEach(() => started.splice(0).forEach((p) => p.blobs.stop()));

function phone(server: Server, userId: string, name: string, seed: number) {
  const store = new FakeStore();
  store.userId = userId;
  const transport = memoryTransport(server);
  const engine = new SyncEngine({ store, storage: new MemoryStorage(), getTransport: () => transport, pollWaitMs: 0, now: fakeNow(), ran
dom: seeded(seed).next });
  const files = fakeFiles(server, name);
  const saved = new Map<string, string>();
  const rejected = new Set<string>();
  const notices: string[] = [];
  const pending: number[] = [];
  const random = seeded(seed + 100).next;
  const blobs = new BlobSync({
    trips: () => store.trips,
    remoteId: (id) => engine.remoteId(id),
    baseUrl: () => 'http://sync.test',
    files,
    cache: { get: (id) => saved.get(id), set: (id, uri) => void saved.set(id, uri), isRejected: (k) => rejected.has(k), reject: (k) => v
oid rejected.add(k) },
    attach: (tripId, list, itemId, blobId) =>
      store.edit(tripId, (d) => {
        if (list === 'expenses') d.expenses.find((x) => x.id === itemId)!.receiptBlob = blobId;
        else (d[list].find((x) => x.id === itemId) as { blob?: string }).blob = blobId;
      }),
    newId: () => Math.floor(random() * 36 ** 8).toString(36).padStart(8, '0'),
    onNotice: (m) => notices.push(m),
    onPending: (n) => pending.push(n),
  });
  const created = { store, engine, files, blobs, saved, rejected, notices, pending, name };
  started.push(created);
  return created;
}

type Phone = ReturnType<typeof phone>;

const bytesOf = (text: string) => Buffer.from(text);

/** Ana has a trip with a local photo, receipt and (optionally) more; Ben joins it. */
async function setup(options: { server?: Server; tripEdit?: (t: Trip) => void } = {}) {
  const server = options.server ?? new SyncStore();
  const ana = phone(server, 'u-a', 'ana', 1);
  const ben = phone(server, 'u-b', 'ben', 2);
  const trip = makeTrip();
  trip.expenses[0].receipt = 'file:///ana/receipt.jpg';
  trip.photos[0].uri = 'file:///ana/photo.jpg';
  options.tripEdit?.(trip);
  ana.store.trips = [trip];
  ana.files.disk.set('file:///ana/photo.jpg', bytesOf('PHOTO-BYTES'));
  ana.files.disk.set('file:///ana/receipt.jpg', bytesOf('RECEIPT-BYTES'));
  await ana.engine.init();
  await ben.engine.init();
  await ana.engine.share('trip-1');
  await ana.engine.syncNow();
  await ben.engine.join('ROM-1234');
  return { server, ana, ben };
}

/** Everyone sends and fetches, files move, and the news about the files travels once more. */
async function settle(...phones: Phone[]) {
  for (let i = 0; i < 2; i++) {
    for (const p of phones) await p.engine.syncNow();
    for (const p of phones) await p.blobs.run();
  }
  for (const p of phones) await p.engine.syncNow();
}

const tripOf = (p: Phone) => p.store.getTrip('trip-1')!;

describe('helpers', () => {
  it('works out a file extension from an address', () => {
    expect(extensionOf('file:///a/Photo.JPG')).toBe('jpg');
    expect(extensionOf('file:///a/photo.jpeg?x=1#y')).toBe('jpg');
    expect(extensionOf('file:///a/ticket.pdf')).toBe('pdf');
```

```
    expect(extensionOf('')).toBe('png');
    expect(extensionOf('')).toBe('svg');
    expect(extensionOf('blob:http://localhost/1234-5678', 'jpg')).toBe('jpg');
    expect(extensionOf('file:///a/no-extension')).toBe('bin');
  });

  it('makes server ids that turn back into a usable file name', () => {
    const id = makeBlobId('k3j9x2m1', 'file:///a/trip-plan.PDF');
    expect(id).toBe('k3j9x2m1_pdf');
    expect(fileNameOf(id)).toBe('k3j9x2m1.pdf');
    expect(fileNameOf('plainid')).toBe('plainid');
  });

  it('tells real files from the pictures that ship with the app', () => {
    expect(isLocalFile('file:///a/b.jpg')).toBe(true);
    expect(isLocalFile('blob:http://x/1')).toBe(true);
    expect(isLocalFile('')).toBe(true);
    expect(isLocalFile('amalfi')).toBe(false);
    expect(isLocalFile('')).toBe(false);
    expect(isLocalFile(undefined)).toBe(false);
  });
});

describe('what to show for a file', () => {
  it('prefers this phone’s own file, then a bundled picture, then the downloaded copy', () => {
    expect(resolveFile('file:///mine.jpg', 'abc_jpg', 'file:///copy.jpg')).toEqual({ uri: 'file:///mine.jpg', loading: false });
    expect(resolveFile('', 'asset:amalfi', undefined)).toEqual({ uri: 'amalfi', loading: false });
    expect(resolveFile('', 'abc12345_jpg', 'file:///copy.jpg')).toEqual({ uri: 'file:///copy.jpg', loading: false });
    expect(resolveFile('', 'abc12345_jpg', undefined)).toEqual({ uri: undefined, loading: true });
    expect(resolveFile(undefined, undefined, undefined)).toEqual({ uri: undefined, loading: false });
    expect(isAssetRef(assetRef('amalfi'))).toBe(true);
    expect(isAssetRef('abc12345_jpg')).toBe(false);
  });
});

describe('sending and fetching files', () => {
  it('shares the pictures that come with the app by name instead of sending them', async () => {
    const { ana, ben } = await setup({
      tripEdit: (t) => {
        t.photos.push({ id: 'ph-demo', uri: 'amalfi', caption: 'Bundled', by: 'u-a', likes: [], ts: 6 });
      },
    });
    await settle(ana, ben);
    expect(tripOf(ana).photos.find((p) => p.id === 'ph-demo')?.blob).toBe('asset:amalfi');
    expect(tripOf(ben).photos.find((p) => p.id === 'ph-demo')?.blob).toBe('asset:amalfi');
    expect(ben.files.downloads).toHaveLength(2); // only the real photo and receipt were fetched
    expect(ana.files.uploads).toHaveLength(2);
  });

  it('uploads your photo and receipt and writes the server ids on them', async () => {
    const { server, ana } = await setup();
    await ana.blobs.run();
    const trip = tripOf(ana);
    expect(trip.photos[0].blob).toMatch(/^[a-z0-9]{8}_jpg$/);
    expect(trip.expenses[0].receiptBlob).toMatch(/^[a-z0-9]{8}_jpg$/);
    const remote = ana.engine.remoteId('trip-1')!;
    expect((await server.getBlob(remote, trip.photos[0].blob!))?.toString()).toBe('PHOTO-BYTES');
    expect((await server.getBlob(remote, trip.expenses[0].receiptBlob!))?.toString()).toBe('RECEIPT-BYTES');
    expect(ana.files.uploads).toHaveLength(2);
  });

  it('a friend’s phone downloads the files and keeps its own copy', async () => {
    const { ana, ben } = await setup();
    await settle(ana, ben);
    const photo = tripOf(ben).photos[0];
    expect(photo.blob).toBe(tripOf(ana).photos[0].blob);
    expect(photo.uri).toBe(''); // the path on Ana's phone means nothing here
    const copy = ben.saved.get(photo.blob!)!;
    expect(copy).toMatch(/^file:\/\/\/ben\/shared\/[a-z0-9]{8}\.jpg$/);
    expect(ben.files.disk.get(copy)?.toString()).toBe('PHOTO-BYTES');
    expect(ben.files.disk.get(ben.saved.get(tripOf(ben).expenses[0].receiptBlob!)!)?.toString()).toBe('RECEIPT-BYTES');
  });

  it('does not send a file twice or fetch it again', async () => {
    const { ana, ben } = await setup();
    await settle(ana, ben);
    await settle(ana, ben);
    await ana.blobs.run();
    await ben.blobs.run();
    expect(ana.files.uploads).toHaveLength(2);
    expect(ben.files.downloads).toHaveLength(2);
    expect(ana.files.downloads).toHaveLength(0); // the owner already has the files
  });

  it('two passes at the same time still upload once', async () => {
    const { ana } = await setup();
```

```
    await Promise.all([ana.blobs.run(), ana.blobs.run(), ana.blobs.run()]);
    expect(ana.files.uploads).toHaveLength(2);
  });

  it('leaves trips that are not shared alone', async () => {
    const { ana } = await setup({
      tripEdit: (t) => {
        t.photos.push({ id: 'ph-demo', uri: 'amalfi', caption: 'Bundled', by: 'u-a', likes: [], ts: 6 });
      },
    });
    ana.store.trips = [...ana.store.trips, { ...makeTrip(), id: 'trip-private', inviteCode: 'PRV-0001', photos: [{ id: 'x', uri: 'fil
e:///ana/photo.jpg', caption: '', by: 'u-a', likes: [], ts: 1 }] }];
    await ana.blobs.run();
    expect(ana.files.uploads).toHaveLength(2); // only the two files in the shared trip
    expect(ana.store.getTrip('trip-private')!.photos[0].blob).toBeUndefined();
  });

  it('handles documents and keeps their file type', async () => {
    const { ana, ben } = await setup({
      tripEdit: (t) => {
        t.docs.push({ id: 'd1', name: 'Hotel booking', kind: 'pdf', uri: 'file:///ana/booking.pdf', addedBy: 'u-a', addedAt: 3, offline:
true });
        t.docs.push({ id: 'd2', name: 'Packing tips', kind: 'note', note: 'Bring a hat', addedBy: 'u-a', addedAt: 4, offline: false });
      },
    });
    ana.files.disk.set('file:///ana/booking.pdf', bytesOf('PDF-BYTES'));
    await settle(ana, ben);
    const doc = tripOf(ben).docs.find((d) => d.id === 'd1')!;
    expect(doc.blob).toMatch(/_pdf$/);
    expect(ben.saved.get(doc.blob!)).toMatch(/\.pdf$/);
    expect(tripOf(ben).docs.find((d) => d.id === 'd2')?.blob).toBeUndefined(); // a note has no file
  });

  it('fetches the file again when the saved copy has disappeared', async () => {
    const { ana, ben } = await setup();
    await settle(ana, ben);
    const id = tripOf(ben).photos[0].blob!;
    ben.files.disk.delete(ben.saved.get(id)!); // the phone cleaned its storage
    await ben.blobs.run();
    expect(ben.files.downloads.filter((d) => d === id)).toHaveLength(2);
    expect(ben.files.disk.get(ben.saved.get(id)!)?.toString()).toBe('PHOTO-BYTES');
  });

  it('a friend who joins later also gets files that were added before', async () => {
    const { server, ana, ben } = await setup();
    await settle(ana, ben);
    const cara = phone(server, 'u-c', 'cara', 3);
    await cara.engine.init();
    await cara.engine.join('ROM-1234');
    await cara.blobs.run();
    expect(cara.saved.size).toBe(2);
  });
});

describe('when things go wrong', () => {
  it('keeps the file for later when there is no connection, then sends it', async () => {
    const { ana, ben } = await setup();
    ana.files.offline = true;
    await ana.blobs.run();
    expect(tripOf(ana).photos[0].blob).toBeUndefined();
    expect(ana.notices).toEqual([]); // being offline is not worth a message
    ana.files.offline = false;
    await settle(ana, ben);
    expect(tripOf(ben).photos[0].blob).toBeDefined();
    expect(ben.saved.size).toBe(2);
  });

  it('tries again by itself after a pause', async () => {
    jest.useFakeTimers();
    try {
      const { ana } = await setup();
      ana.files.offline = true;
      await ana.blobs.run(); // fails and schedules a retry
      expect(ana.files.uploads).toHaveLength(0);
      ana.files.offline = false;
      await jest.advanceTimersByTimeAsync(2900);
      expect(ana.files.uploads).toHaveLength(0); // still waiting
      await jest.advanceTimersByTimeAsync(300);
      expect(ana.files.uploads).toHaveLength(2);
      ana.blobs.stop();
    } finally {
      jest.useRealTimers();
    }
  });

  it('does not share a file that is too big, and says so once', async () => {
```

```
    const { ana } = await setup();
    ana.files.disk.set('file:///ana/photo.jpg', Buffer.alloc(MAX_SHARED_FILE_BYTES + 1024 * 1024));
    await ana.blobs.run();
    await ana.blobs.run();
    expect(ana.notices).toHaveLength(1);
    expect(ana.notices[0]).toMatch(/9\.0 MB/);
    expect(tripOf(ana).photos[0].blob).toBeUndefined();
    expect(tripOf(ana).expenses[0].receiptBlob).toBeDefined(); // the receipt still went
  });

  it('remembers a file the server refused instead of trying it forever', async () => {
    const server = new SyncStore({ maxBlobBytes: 5 });
    const { ana } = await setup({ server });
    await ana.blobs.run();
    await ana.blobs.run();
    expect(ana.notices.length).toBe(2); // one per file, each only once
    expect(ana.files.uploads).toHaveLength(2);
    expect(ana.rejected.size).toBe(2);
  });

  it('stops quietly when the file was deleted from the phone', async () => {
    const { ana } = await setup();
    ana.files.disk.delete('file:///ana/photo.jpg');
    await ana.blobs.run();
    expect(tripOf(ana).photos[0].blob).toBeUndefined();
    expect(ana.notices).toEqual([]);
    expect(tripOf(ana).expenses[0].receiptBlob).toBeDefined();
  });

  it('keeps going with other files when one is missing on the server', async () => {
    const { ana, ben } = await setup();
    await settle(ana, ben);
    // a second photo whose file the server never received, plus the earlier ones gone from Ben's phone
    ana.store.edit('trip-1', (d) => void d.photos.unshift({ id: 'ph-lost', uri: '', blob: 'lost0000_jpg', caption: 'Lost', by: 'u-a', li
kes: [], ts: 9 }));
    await ana.engine.syncNow();
    await ben.engine.syncNow();
    ben.files.disk.delete(ben.saved.get(tripOf(ben).expenses[0].receiptBlob!)!); // this one can be fetched again
    await ben.blobs.run();
    expect(ben.files.disk.get(ben.saved.get(tripOf(ben).expenses[0].receiptBlob!)!)?.toString()).toBe('RECEIPT-BYTES');
  });

  it('reports how many files are waiting', async () => {
    const { ana } = await setup();
    await ana.blobs.run();
    expect(ana.pending[0]).toBe(2);
    expect(ana.pending.at(-1)).toBe(0);
  });

  it('does nothing without a server address', async () => {
    const { ana } = await setup();
    const quiet = new BlobSync({ trips: () => ana.store.trips, remoteId: (id) => ana.engine.remoteId(id), baseUrl: () => null, files: an
a.files, cache: { get: () => undefined, set: () => undefined, isRejected: () => false, reject: () => undefined }, attach: () => undefine
d, newId: () => 'abcdefgh' });
    await quiet.run();
    expect(ana.files.uploads).toHaveLength(0);
  });
});
```

src/sync/__tests__/crdt.test.ts

```
import { produce } from 'immer';

import { buildDemoTrip } from '@/store/seed';
import type { Trip } from '@/store/types';

import { applyOps, diffTrip, materializeTrip, resolve, type Op, type Registers } from '../crdt';
import { HybridClock } from '../hlc';
import { mergeLocalOnly } from '../view';
import { Replica, fakeNow, makeTrip, seeded, sharedPart } from '../testing';

const json = <T,>(x: T): T => JSON.parse(JSON.stringify(x));

describe('describing and rebuilding a trip', () => {
  it('a rebuilt trip equals the original (shared parts)', () => {
    const trip = makeTrip();
    const clock = new HybridClock('a', fakeNow());
    const regs: Registers = {};
    applyOps(regs, diffTrip(undefined, trip, () => clock.next()));
    const rebuilt = materializeTrip(regs)!;
    expect(rebuilt).toEqual(sharedPart(trip));
  });

  it('works for the full demo trip (hundreds of items) and puts phone-only parts back', () => {
    const trip = json(buildDemoTrip('upcoming'));
    const clock = new HybridClock('a', fakeNow());
```

```
    const regs: Registers = {};
    applyOps(regs, diffTrip(undefined, trip, () => clock.next()));
    const rebuilt = materializeTrip(regs)!;
    expect(rebuilt).toEqual(sharedPart(trip));
    expect(json(mergeLocalOnly(rebuilt, trip))).toEqual(trip);
  });

  it('keeps the order lists had: newest last for itineraries, newest first for expenses', () => {
    const trip = produce(makeTrip(), (d) => {
      d.activities.push({ ...d.activities[0], id: 'a2', title: 'Forum' }, { ...d.activities[0], id: 'a3', title: 'Pantheon' });
      d.expenses.unshift({ ...d.expenses[0], id: 'e2', title: 'Dinner' }, { ...d.expenses[0], id: 'e3', title: 'Taxi' });
    });
    const clock = new HybridClock('a', fakeNow());
    const regs: Registers = {};
    applyOps(regs, diffTrip(undefined, trip, () => clock.next()));
    const rebuilt = materializeTrip(regs)!;
    expect(rebuilt.activities.map((a) => a.id)).toEqual(['a1', 'a2', 'a3']);
    expect(rebuilt.expenses.map((e) => e.id)).toEqual(['e2', 'e3', 'e1']); // the same order the array had
  });

  it('never shares private chat, emails or file paths', () => {
    const ops = diffTrip(undefined, makeTrip(), new HybridClock('a', fakeNow()).next.bind(new HybridClock('a', fakeNow())));
    const text = JSON.stringify(ops);
    expect(text).not.toContain('ana@example.com');
    expect(text).not.toContain('private answer');
    expect(text).not.toContain('file:///');
  });
});

describe('merging is order-proof and repeat-proof', () => {
  it('applying ops twice, or shuffled, gives the same registers', () => {
    const clock = new HybridClock('a', fakeNow());
    const ops = diffTrip(undefined, makeTrip(), () => clock.next());
    const once: Registers = {};
    applyOps(once, ops);
    const twice: Registers = {};
    applyOps(twice, ops);
    expect(applyOps(twice, ops)).toBe(false);
    const rnd = seeded(5);
    const shuffled: Registers = {};
    applyOps(shuffled, rnd.shuffle(ops));
    expect(twice).toEqual(once);
    expect(shuffled).toEqual(once);
  });

  it('ignores paths it does not know (changes from a newer app version)', () => {
    const regs: Registers = {};
    expect(applyOps(regs, [{ p: ['hologram', 'x'], v: 1, t: '1800000000000-0000-zzzz' }, { p: ['activities', 'a1', 'teleport'], v: 1, t:
'1800000000001-0000-zzzz' }])).toBe(false);
    expect(regs).toEqual({});
  });

  it('validates paths against the schema', () => {
    expect(resolve(['name'])).toEqual({ items: [], isItem: false });
    expect(resolve(['activities', 'a1'])).toEqual({ items: [['activities', 'a1']], isItem: true });
    expect(resolve(['activities', 'a1', 'votes', 'u-a'])?.isItem).toBe(false);
    expect(resolve(['polls', 'p1', 'options', 'o1', 'votes', 'u-b'])?.items).toHaveLength(2);
    expect(resolve(['activities', 'a1', 'votes'])).toBeNull(); // a map node alone is not a register
    expect(resolve(['activities'])).toBeNull();
    expect(resolve(['name', 'extra'])).toBeNull();
  });

  it('hides items that arrive without their required fields', () => {
    const clock = new HybridClock('a', fakeNow());
    const regs: Registers = {};
    applyOps(regs, diffTrip(undefined, makeTrip(), () => clock.next()));
    const t = clock.next();
    applyOps(regs, [{ p: ['tasks', 'partial', '_c'], v: 1, t }, { p: ['tasks', 'partial', 'done'], v: true, t: clock.next() }]);
    expect(materializeTrip(regs)!.tasks.map((k) => k.id)).toEqual(['t1']);
  });
});

describe('two phones editing at the same time', () => {
  const setup = () => {
    const now = fakeNow();
    const a = new Replica('aaaaaaaa', now, makeTrip());
    const b = new Replica('bbbbbbbb', now);
    b.receive(a.publish());
    return { a, b, now };
  };
  const swap = (a: Replica, b: Replica, opsA: Op[], opsB: Op[]) => {
    b.receive(opsA);
    a.receive(opsB);
  };

  it('keeps both new activities', () => {
```

```
  const { a, b } = setup();
  const oa = a.edit((d) => void d.activities.push({ ...d.activities[0], id: 'a-ana', title: 'Forum' }));
  const ob = b.edit((d) => void d.activities.push({ ...d.activities[0], id: 'a-ben', title: 'Pantheon' }));
  swap(a, b, oa, ob);
  expect(a.trip!.activities.map((x) => x.id)).toEqual(['a1', 'a-ana', 'a-ben']);
  expect(sharedPart(a.trip)).toEqual(sharedPart(b.trip));
});

it('merges edits to different fields of one activity', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void (d.activities[0].title = 'Colosseum tour'));
  const ob = b.edit((d) => void (d.activities[0].time = '09:00'));
  swap(a, b, oa, ob);
  for (const r of [a, b]) expect(r.trip!.activities[0]).toMatchObject({ title: 'Colosseum tour', time: '09:00' });
});

it('the later edit wins when both change the same field, on both phones', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void (d.name = 'Rome Crew!'));
  const ob = b.edit((d) => void (d.name = 'Roma Squad')); // made a moment later
  swap(a, b, oa, ob);
  expect(a.trip!.name).toBe('Roma Squad');
  expect(b.trip!.name).toBe('Roma Squad');
});

it('a delete beats a concurrent edit of the same item', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void (d.activities = []));
  const ob = b.edit((d) => void (d.activities[0].title = 'Edited while deleted'));
  swap(a, b, oa, ob);
  expect(a.trip!.activities).toEqual([]);
  expect(b.trip!.activities).toEqual([]);
});

it('votes from different people both count', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void (d.activities[0].votes['u-a'] = -1));
  const ob = b.edit((d) => void (d.activities[0].votes['u-b'] = 1));
  swap(a, b, oa, ob);
  expect(a.trip!.activities[0].votes).toEqual({ 'u-a': -1, 'u-b': 1 });
  expect(b.trip!.activities[0].votes).toEqual({ 'u-a': -1, 'u-b': 1 });
});

it('adds and removes inside lists of names independently', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void d.expenses[0].splitBetween.push('u-c'));
  const ob = b.edit((d) => void (d.expenses[0].splitBetween = d.expenses[0].splitBetween.filter((x) => x !== 'u-a')));
  swap(a, b, oa, ob);
  expect([...a.trip!.expenses[0].splitBetween].sort()).toEqual(['u-b', 'u-c']);
  expect([...b.trip!.expenses[0].splitBetween].sort()).toEqual(['u-b', 'u-c']);
});

it('poll votes (nested items) merge too', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void d.polls[0].options[0].votes.push('u-a'));
  const ob = b.edit((d) => void d.polls[0].options[1].votes.push('u-c'));
  swap(a, b, oa, ob);
  for (const r of [a, b]) expect(r.trip!.polls[0].options.map((o) => [...o.votes].sort())).toEqual([['u-a'], ['u-b', 'u-c']]);
});

it('chat messages from both phones appear in time order', () => {
  const { a, b } = setup();
  const oa = a.edit((d) => void d.messages.push({ id: 'm-a', authorId: 'u-a', text: 'first', ts: 2000 }));
  const ob = b.edit((d) => void d.messages.push({ id: 'm-b', authorId: 'u-b', text: 'second', ts: 2001 }));
  swap(a, b, oa, ob);
  expect(a.trip!.messages.filter((m) => !m.ai).map((m) => m.id)).toEqual(['m1', 'm-a', 'm-b']);
  expect(b.trip!.messages.map((m) => m.id)).toEqual(['m1', 'm-a', 'm-b']);
});

it('keeps the private Scout chat on its own phone', () => {
  const { a, b } = setup();
  expect(a.trip!.messages.map((m) => m.id)).toContain('ai1');
  expect(b.trip!.messages.map((m) => m.id)).not.toContain('ai1');
  a.edit((d) => void d.messages.push({ id: 'ai2', authorId: 'assistant', text: 'another', ts: 3, ai: true, channel: 'ai' }));
  expect(b.trip!.messages.map((m) => m.id)).not.toContain('ai2');
});

it('a person who leaves and rejoins comes back with fresh details only', () => {
  const { a, b } = setup();
  // Ben leaves (his member entry is deleted) ...
  const leave = b.edit((d) => void (d.members = d.members.filter((m) => m.id !== 'u-b')));
  a.receive(leave);
  expect(a.trip!.members.map((m) => m.id)).toEqual(['u-a']);
  // ... then rejoins with a different role and no home city.
  const rejoin = b.edit((d) => void d.members.push({ id: 'u-b', name: 'Ben R.', role: 'Photographer' }));
  a.receive(rejoin);
```

```
    expect(a.trip!.members.find((m) => m.id === 'u-b')).toEqual({ id: 'u-b', name: 'Ben R.', role: 'Photographer' });
  });

  it('removing a field removes it everywhere', () => {
    const { a, b } = setup();
    b.receive(a.edit((d) => void (d.activities[0].notes = 'Bring water')));
    expect(b.trip!.activities[0].notes).toBe('Bring water');
    b.receive(a.edit((d) => void delete d.activities[0].notes));
    expect(b.trip!.activities[0].notes).toBeUndefined();
  });

  it('a phone that was offline catches up from everything it missed', () => {
    const { a, b } = setup();
    const missed: Op[] = [];
    missed.push(...a.edit((d) => void d.tasks.push({ id: 't2', text: 'Pack', done: false })));
    missed.push(...a.edit((d) => void (d.tasks[0].done = true)));
    missed.push(...a.edit((d) => void d.messages.push({ id: 'm2', authorId: 'u-a', text: 'back online?', ts: 9 })));
    b.receive(missed);
    expect(sharedPart(b.trip)).toEqual(sharedPart(a.trip));
  });
});

describe('many phones, many random edits: everyone ends up identical', () => {
  const ids = ['u-a', 'u-b', 'u-c'];

  const mutations: ((d: Trip, rnd: ReturnType<typeof seeded>, n: number) => void)[] = [
    (d, r, n) => void d.activities.push({ ...makeTrip().activities[0], id: `a-${n}`, title: `Act ${n}`, status: r.pick(['voting', 'confi
rmed', 'pending'] as const) }),
    (d, r) => void (d.activities.length && (d.activities[r.int(d.activities.length)].title = `Retitled ${r.int(99)}`)),
    (d, r) => void (d.activities.length && (d.activities[r.int(d.activities.length)].votes[r.pick(ids)] = r.pick([1, -1] as const))),
    (d, r) => {
      if (!d.activities.length) return;
      const a = d.activities[r.int(d.activities.length)];
      delete a.votes[r.pick(ids)];
    },
    (d, r) => void (d.activities.length && d.activities.splice(r.int(d.activities.length), 1)),
    (d, r) => {
      if (!d.activities.length) return;
      const a = d.activities[r.int(d.activities.length)];
      const who = r.pick(ids);
      a.assigned = a.assigned.includes(who) ? a.assigned.filter((x) => x !== who) : [...a.assigned, who];
    },
    (d, r, n) => void d.expenses.unshift({ id: `e-${n}`, title: `Exp ${n}`, amount: r.int(5000) / 100, category: 'Food', paidBy: r.pick
(ids), splitBetween: ids.filter(() => r.next() < 0.7), date: '2026-09-11' }),
    (d, r) => void (d.expenses.length && d.expenses.splice(r.int(d.expenses.length), 1)),
    (d, r, n) => void d.messages.push({ id: `m-${n}`, authorId: r.pick(ids), text: `msg ${n}`, ts: n }),
    (d, r, n) => void d.tasks.push({ id: `t-${n}`, text: `Task ${n}`, done: false }),
    (d, r) => void (d.tasks.length && (d.tasks[r.int(d.tasks.length)].done = r.next() < 0.5)),
    (d, r) => void (d.tasks.length && d.tasks.splice(r.int(d.tasks.length), 1)),
    (d, r) => void (d.name = `Trip ${r.int(100)}`),
    (d, r) => void (d.budget = 1000 + r.int(5000)),
    (d, r) => {
      const o = d.polls[0]?.options[r.int(2)];
      if (!o) return;
      const who = r.pick(ids);
      o.votes = o.votes.includes(who) ? o.votes.filter((x) => x !== who) : [...o.votes, who];
    },
    (d, r, n) => void d.members.push({ id: `m-${n}`, name: `Member ${n}`, role: 'Member' }),
    (d, r) => void (d.members.length > 1 && d.members.splice(1 + r.int(d.members.length - 1), 1)),
    (d, r) => void (d.photos.length && (d.photos[0].likes = d.photos[0].likes.includes('u-c') ? [] : ['u-c'])),
  ];

  // More seeds on demand:  SYNC_SEEDS=200 npx jest src/sync
  it.each(Array.from({ length: Number(process.env.SYNC_SEEDS ?? 6) }, (_, i) => i + 1))('seed %i: 3 phones, 90 edits, shuffled delivery
with duplicates', (seed) => {
    const rnd = seeded(seed * 7919);
    const now = fakeNow();
    const phones = [new Replica('aaaaaaaa', now, makeTrip()), new Replica('bbbbbbbb', now), new Replica('cccccccc', now)];
    const baseline = phones[0].publish();
    phones[1].receive(baseline);
    phones[2].receive(baseline);

    const inbox: Op[][][] = phones.map(() => []);
    for (let n = 0; n < 90; n++) {
      const i = rnd.int(3);
      const ops = phones[i].edit((d) => rnd.pick(mutations)(d, rnd, n));
      if (ops.length) phones.forEach((_, j) => j !== i && inbox[j].push(ops));
      if (rnd.next() < 0.35) {
        // some phone gets whatever is waiting for it, in random order, sometimes twice
        const j = rnd.int(3);
        const batches = rnd.shuffle(inbox[j]);
        inbox[j] = [];
        batches.forEach((b) => {
          phones[j].receive(b);
          if (rnd.next() < 0.2) phones[j].receive(b);
        });
```

```
      }
    }
    phones.forEach((p, j) => {
      rnd.shuffle(inbox[j]).forEach((b) => p.receive(b));
    });

    const [p0, p1, p2] = phones;
    expect(p1.regs).toEqual(p0.regs);
    expect(p2.regs).toEqual(p0.regs);
    expect(sharedPart(p1.trip)).toEqual(sharedPart(p0.trip));
    expect(sharedPart(p2.trip)).toEqual(sharedPart(p0.trip));
    // and what each phone shows is exactly the merge of everything
    expect(sharedPart(p0.trip)).toEqual(json(materializeTrip(p0.regs)));
  });
});
```

src/sync/__tests__/e2e.test.ts

```
/** @jest-environment node */
import http from 'http';
import type { AddressInfo } from 'net';

import { createServer } from '../../../server/index';
import { SyncStore } from '../../../server/store';
import { SyncEngine } from '../engine';
import { FakeStore, MemoryStorage, makeTrip, seeded, sharedPart } from '../testing';
import { createHttpTransport, normalizeServerUrl } from '../transport';

/** Jest's React Native setup replaces the global fetch, so these tests bring a tiny real one built on Node's http. */
const nodeFetch = ((input: string, init: RequestInit = {}) =>
  new Promise((resolve, reject) => {
    const req = http.request(input, { method: init.method ?? 'GET', headers: init.headers as Record<string, string>, agent: false }, (re
s) => {
      let text = '';
      res.on('data', (c) => (text += c));
      res.on('end', () => resolve({ status: res.statusCode, json: async () => JSON.parse(text) }));
    });
    req.on('error', reject);
    init.signal?.addEventListener('abort', () => {
      req.destroy();
      reject(Object.assign(new Error('aborted'), { name: 'AbortError' }));
    });
    if (init.body) req.write(init.body);
    req.end();
  })) as unknown as typeof fetch;

const until = async (check: () => boolean, ms = 3000) => {
  const start = Date.now();
  while (!check()) {
    if (Date.now() - start > ms) throw new Error('timed out waiting');
    await new Promise((r) => setTimeout(r, 15));
  }
};

describe('two phones talking to the real server over HTTP', () => {
  let server: ReturnType<typeof createServer>;
  let url: string;
  const engines: SyncEngine[] = [];

  beforeEach(async () => {
    server = createServer(new SyncStore());
    await new Promise<void>((r) => server.listen(0, '127.0.0.1', r));
    url = `http://127.0.0.1:${(server.address() as AddressInfo).port}`;
  });
  afterEach(async () => {
    engines.splice(0).forEach((e) => e.stop());
    await new Promise((r) => server.close(r));
  });

  function phone(userId: string, seed: number) {
    const store = new FakeStore();
    store.userId = userId;
    const engine = new SyncEngine({ store, storage: new MemoryStorage(), getTransport: () => createHttpTransport(url, nodeFetch), rando
m: seeded(seed).next, pollWaitMs: 5000 });
    engines.push(engine);
    return { store, engine };
  }

  it('shares, joins and keeps chatting live (long-polling)', async () => {
    const ana = phone('u-a', 1);
    const ben = phone('u-b', 2);
    ana.store.trips = [makeTrip()];
    await ana.engine.init();
    await ben.engine.init();
    await ana.engine.share('trip-1');
    ana.engine.start();
```

```
    await until(() => ana.engine.getStatus().pending === 0 && ana.engine.getStatus().online === true);

    expect((await ben.engine.join('ROM-1234'))?.name).toBe('Rome Crew');
    ben.engine.start();

    ana.store.edit('trip-1', (d) => void d.messages.push({ id: 'live1', authorId: 'u-a', text: 'Can you see this?', ts: 9000 }));
    await until(() => !!ben.store.getTrip('trip-1')?.messages.some((m) => m.id === 'live1'));

    ben.store.edit('trip-1', (d) => void (d.activities[0].votes['u-b'] = 1));
    await until(() => ana.store.getTrip('trip-1')?.activities[0].votes['u-b'] === 1);

    expect(sharedPart(ana.store.getTrip('trip-1'))).toEqual(sharedPart(ben.store.getTrip('trip-1')));
    await until(() => ana.engine.getStatus().peers === 2);
  });

  it('reports a clear error when the server is not there', async () => {
    const lonely = phone('u-a', 3);
    lonely.store.trips = [makeTrip()];
    await lonely.engine.init();
    await expect(lonely.engine.share('trip-1')).resolves.toBeUndefined();
    await new Promise((r) => server.close(r));
    server = createServer(new SyncStore()); // so afterEach can close something
    await new Promise<void>((r) => server.listen(0, '127.0.0.1', r));
    await lonely.engine.syncNow();
    expect(lonely.engine.getStatus()).toMatchObject({ online: false, error: 'Can’t reach the sync server' });
  });
});

describe('server addresses', () => {
  it('accepts the ways people type an address', () => {
    expect(normalizeServerUrl('192.168.1.20:8787')).toBe('http://192.168.1.20:8787');
    expect(normalizeServerUrl(' http://192.168.1.20:8787/ ')).toBe('http://192.168.1.20:8787');
    expect(normalizeServerUrl('https://sync.example.com')).toBe('https://sync.example.com');
    expect(normalizeServerUrl('localhost:8787/extra/path')).toBe('http://localhost:8787');
  });

  it('rejects things that are not addresses', () => {
    expect(normalizeServerUrl('')).toBeNull();
    expect(normalizeServerUrl('   ')).toBeNull();
    expect(normalizeServerUrl('http://')).toBeNull();
  });
});
```

src/sync/__tests__/engine.test.ts

```
import type { Trip } from '@/store/types';

import { SyncStore } from '../../../server/store';
import { CodeTakenError, SyncEngine } from '../engine';
import { FakeStore, MemoryStorage, fakeNow, makeTrip, memoryTransport, seeded, sharedPart } from '../testing';
import type { SyncTransport } from '../transport';

type Server = InstanceType<typeof SyncStore>;

function phone(server: Server, userId: string, seed = 1, now = fakeNow()) {
  const store = new FakeStore();
  store.userId = userId;
  const storage = new MemoryStorage();
  const transport = memoryTransport(server);
  const remote: { prev: Trip | undefined; next: Trip }[] = [];
  const engine = new SyncEngine({ store, storage, getTransport: () => transport, pollWaitMs: 0, now, random: seeded(seed).next, onRemote
Change: (prev, next) => remote.push({ prev, next }) });
  return { store, storage, transport, engine, remote, userId, now };
}

/** Everyone sends, then everyone fetches, twice, so changes travel all the way around. */
async function settle(...phones: ReturnType<typeof phone>[]) {
  for (let i = 0; i < 2; i++) for (const p of phones) await p.engine.syncNow();
}

async function sharedTrip() {
  const server = new SyncStore();
  const ana = phone(server, 'u-a', 1);
  const ben = phone(server, 'u-b', 2);
  ana.store.trips = [makeTrip()];
  await ana.engine.init();
  await ben.engine.init();
  await ana.engine.share('trip-1');
  await ana.engine.syncNow();
  const joined = await ben.engine.join('ROM-1234');
  return { server, ana, ben, joined };
}

describe('sharing and joining a trip', () => {
  it('a friend joins with the invite code and gets the same trip', async () => {
    const { ana, ben, joined } = await sharedTrip();
```

```
    expect(joined?.id).toBe('trip-1');
    expect(ben.store.trips).toHaveLength(1);
    expect(sharedPart(ben.store.getTrip('trip-1'))).toEqual(sharedPart(ana.store.getTrip('trip-1')));
    expect(ben.engine.isShared('trip-1')).toBe(true);
  });

  it('keeps each phone’s private parts private', async () => {
    const { ana, ben } = await sharedTrip();
    expect(ana.store.getTrip('trip-1')!.messages.map((m) => m.id)).toContain('ai1'); // Ana's Scout chat
    expect(ben.store.getTrip('trip-1')!.messages.map((m) => m.id)).not.toContain('ai1');
    expect(ben.store.getTrip('trip-1')!.members.find((m) => m.id === 'u-a')?.email).toBeUndefined();
    expect(ana.store.getTrip('trip-1')!.members.find((m) => m.id === 'u-a')?.email).toBe('ana@example.com');
  });

  it('an unknown code finds nothing', async () => {
    const { ben } = await sharedTrip();
    expect(await ben.engine.join('ZZZ-0000')).toBeNull();
  });

  it('refuses to share a second trip under a code another shared trip already uses', async () => {
    const { server } = await sharedTrip();
    const cara = phone(server, 'u-c', 3);
    cara.store.trips = [{ ...makeTrip(), id: 'trip-other' }];
    await cara.engine.init();
    await expect(cara.engine.share('trip-other')).rejects.toBeInstanceOf(CodeTakenError);
  });

  it('a big trip (700 chat messages) travels in pages', async () => {
    const server = new SyncStore();
    const ana = phone(server, 'u-a', 1);
    const ben = phone(server, 'u-b', 2);
    const trip = makeTrip();
    for (let i = 0; i < 700; i++) trip.messages.push({ id: `bulk${i}`, authorId: 'u-a', text: `message ${i}`, ts: 2000 + i });
    ana.store.trips = [trip];
    await ana.engine.init();
    await ben.engine.init();
    await ana.engine.share('trip-1');
    await ana.engine.syncNow();
    await ben.engine.join('ROM-1234');
    expect(ben.store.getTrip('trip-1')!.messages.filter((m) => m.id.startsWith('bulk'))).toHaveLength(700);
    expect(server.stats().ops).toBeGreaterThan(2500); // more than two pages of 1000 changes
  });
});

describe('working together', () => {
  it('chat, votes and expenses flow both ways', async () => {
    const { ana, ben } = await sharedTrip();
    ben.store.edit('trip-1', (d) => {
      d.messages.push({ id: 'm-ben', authorId: 'u-b', text: 'Hello from Ben', ts: 5000 });
      d.activities[0].votes['u-b'] = 1;
    });
    ana.store.edit('trip-1', (d) => {
      d.expenses.unshift({ id: 'e-ana', title: 'Pizza', amount: 40, category: 'Food', paidBy: 'u-a', splitBetween: ['u-a', 'u-b'], date:
'2026-09-11' });
    });
    await settle(ana, ben);
    for (const p of [ana, ben]) {
      const t = p.store.getTrip('trip-1')!;
      expect(t.messages.map((m) => m.id)).toContain('m-ben');
      expect(t.activities[0].votes).toEqual({ 'u-a': 1, 'u-b': 1 });
      expect(t.expenses[0].id).toBe('e-ana');
    }
    expect(sharedPart(ana.store.getTrip('trip-1'))).toEqual(sharedPart(ben.store.getTrip('trip-1')));
  });

  it('a new member joining shows up for everyone', async () => {
    const { server, ana, ben } = await sharedTrip();
    const cara = phone(server, 'u-c', 3);
    await cara.engine.init();
    await cara.engine.join('ROM-1234');
    cara.store.edit('trip-1', (d) => void d.members.push({ id: 'u-c', name: 'Cara', role: 'Member' }));
    await settle(ana, ben, cara);
    for (const p of [ana, ben, cara]) expect(p.store.getTrip('trip-1')!.members.map((m) => m.id)).toEqual(['u-a', 'u-b', 'u-c']);
  });

  it('does not send friends’ changes back to the server', async () => {
    const { ana, ben, server } = await sharedTrip();
    ben.store.edit('trip-1', (d) => void d.tasks.push({ id: 't9', text: 'Buy water', done: false }));
    await settle(ana, ben);
    const total = server.stats().ops;
    await settle(ana, ben);
    await settle(ana, ben);
    expect(server.stats().ops).toBe(total);
  });

  it('tells the app when friends changed something', async () => {
```

```
    const { ana, ben } = await sharedTrip();
    ben.store.edit('trip-1', (d) => void d.messages.push({ id: 'm-ben', authorId: 'u-b', text: 'Ping', ts: 6000 }));
    await settle(ana, ben);
    expect(ana.remote.length).toBeGreaterThan(0);
    const last = ana.remote.at(-1)!;
    expect(last.prev!.messages.some((m) => m.id === 'm-ben')).toBe(false);
    expect(last.next.messages.some((m) => m.id === 'm-ben')).toBe(true);
  });

  it('concurrent edits to different things both survive', async () => {
    const { ana, ben } = await sharedTrip();
    ana.store.edit('trip-1', (d) => void (d.activities[0].title = 'Colosseum at sunrise'));
    ben.store.edit('trip-1', (d) => void (d.activities[0].time = '07:30'));
    await settle(ana, ben);
    for (const p of [ana, ben]) expect(p.store.getTrip('trip-1')!.activities[0]).toMatchObject({ title: 'Colosseum at sunrise', time: '0
7:30' });
  });

  it('keeps private Scout messages off the server', async () => {
    const { ana, ben } = await sharedTrip();
    ana.store.edit('trip-1', (d) => void d.messages.push({ id: 'ai-new', authorId: 'assistant', text: 'secret', ts: 7000, ai: true, chan
nel: 'ai' }));
    await settle(ana, ben);
    expect(ben.store.getTrip('trip-1')!.messages.map((m) => m.id)).not.toContain('ai-new');
  });
});

describe('the live connection', () => {
  it('asks for a quick answer first, so a quiet connection shows as live right away', async () => {
    const server = new SyncStore();
    const ana = phone(server, 'u-a', 1);
    ana.store.trips = [makeTrip()];
    await ana.engine.init();
    await ana.engine.share('trip-1');
    await ana.engine.syncNow();

    const waits: number[] = [];
    const base = memoryTransport(server);
    const transport: SyncTransport = {
      ...base,
      // a long wait only ends when the engine stops (like a real long poll with nothing new to report)
      pull: (id, since, client, waitMs, signal) => {
        waits.push(waitMs);
        return waitMs > 0 ? new Promise((_resolve, reject) => signal?.addEventListener('abort', () => reject(new Error('aborted')))) : b
ase.pull(id, since, client, 0);
      },
    };
    const store = new FakeStore();
    store.userId = 'u-b';
    const engine = new SyncEngine({ store, storage: new MemoryStorage(), getTransport: () => transport, pollWaitMs: 5000, random: seeded
(2).next });
    await engine.init();
    engine.start();
    await engine.join('ROM-1234');
    await new Promise((resolve) => setTimeout(resolve, 20));
    expect(waits.slice(-2)).toEqual([0, 5000]);
    expect(engine.getStatus().online).toBe(true);
    engine.stop();
  });
});

describe('losing the connection and the app closing', () => {
  it('queues changes while offline and sends them when the connection returns', async () => {
    const { ana, ben } = await sharedTrip();
    ana.transport.offline = true;
    ana.store.edit('trip-1', (d) => void d.tasks.push({ id: 't-offline', text: 'Written offline', done: false }));
    await ana.engine.syncNow();
    expect(ana.engine.getStatus()).toMatchObject({ online: false, error: 'Can’t reach the sync server' });
    expect(ana.engine.getStatus().pending).toBeGreaterThan(0);
    await ben.engine.syncNow();
    expect(ben.store.getTrip('trip-1')!.tasks.map((t) => t.id)).not.toContain('t-offline');

    ana.transport.offline = false;
    await settle(ana, ben);
    expect(ana.engine.getStatus()).toMatchObject({ online: true, pending: 0, error: null });
    expect(ben.store.getTrip('trip-1')!.tasks.map((t) => t.id)).toContain('t-offline');
  });

  it('remembers where it was after the app restarts', async () => {
    const { server, ana, ben } = await sharedTrip();
    await ana.engine.saveNow();
    // "restart": a new engine with the same saved state and the same trips
    const again = phone(server, 'u-a', 9);
    again.store.trips = ana.store.trips;
    again.storage.saved = ana.storage.saved;
    await again.engine.init();
```

```
    expect(again.engine.isShared('trip-1')).toBe(true);
    expect(again.engine.nodeId).toBe(ana.engine.nodeId);
    const before = again.store.getTrip('trip-1');
    await again.engine.syncNow();
    expect(again.store.getTrip('trip-1')).toBe(before); // nothing new, nothing re-applied
    ben.store.edit('trip-1', (d) => void d.tasks.push({ id: 't-after', text: 'After restart', done: false }));
    await ben.engine.syncNow();
    await again.engine.syncNow();
    expect(again.store.getTrip('trip-1')!.tasks.map((t) => t.id)).toContain('t-after');
  });

  it('sends a change that was made while the app was closed', async () => {
    const { server, ana, ben } = await sharedTrip();
    await ana.engine.saveNow();
    const savedMeta = ana.storage.saved;
    // The change reaches the saved trips, but the app is killed before the sync engine hears about it.
    const edited = ana.store.trips.map((t) => ({ ...t, name: 'Renamed while closed' }));
    const again = phone(server, 'u-a', 9);
    again.store.trips = edited;
    again.storage.saved = savedMeta;
    await again.engine.init();
    await settle(again, ben);
    expect(ben.store.getTrip('trip-1')!.name).toBe('Renamed while closed');
  });

  it('stops syncing a trip that is deleted on this phone, and friends keep theirs', async () => {
    const { ana, ben, server } = await sharedTrip();
    ana.store.removeTrip('trip-1');
    await ana.engine.syncNow();
    expect(ana.engine.isShared('trip-1')).toBe(false);
    await ben.engine.syncNow();
    expect(ben.store.getTrip('trip-1')).toBeDefined();
    expect(server.stats().trips).toBe(1);
  });

  it('detaching leaves a normal local trip', async () => {
    const { ana, ben } = await sharedTrip();
    ana.engine.detach('trip-1');
    ana.store.edit('trip-1', (d) => void (d.name = 'Local only now'));
    await settle(ana, ben);
    expect(ben.store.getTrip('trip-1')!.name).toBe('Rome Crew');
    expect(ana.store.getTrip('trip-1')!.name).toBe('Local only now');
  });

  it('reports how many phones are in the trip', async () => {
    const { ana, ben } = await sharedTrip();
    await settle(ana, ben);
    expect(ana.engine.getStatus().peers).toBe(2);
    expect(ana.engine.getStatus().shared).toEqual(['trip-1']);
  });
});
```

src/sync/__tests__/invite.test.ts

```
import { buildInviteUrl, inviteText, parseInviteUrl } from '../invite';

describe('invite links', () => {
  it('survive the round trip for every kind of address the app can have', () => {
    for (const base of ['togetherwego://join', 'exp://192.168.1.20:8081/--/join', 'http://localhost:8081/join', 'https://example.org/ap
p/join']) {
      const link = buildInviteUrl(base, 'AML-2026', 'http://192.168.1.20:8787');
      expect(link.startsWith(base)).toBe(true);
      expect(parseInviteUrl(link)).toEqual({ code: 'AML-2026', server: 'http://192.168.1.20:8787' });
    }
  });

  it('leaves the server out when there is none', () => {
    const link = buildInviteUrl('togetherwego://join', 'SEO-5521');
    expect(link).toBe('togetherwego://join?code=SEO-5521');
    expect(parseInviteUrl(link)).toEqual({ code: 'SEO-5521', server: null });
  });

  it('adds to an address that already has a query', () => {
    expect(buildInviteUrl('https://x.org/join?lang=en', 'AML-2026')).toBe('https://x.org/join?lang=en&code=AML-2026');
    expect(parseInviteUrl('https://x.org/join?lang=en&code=AML-2026').code).toBe('AML-2026');
  });

  it('tidies codes typed in lower case or without the dash', () => {
    expect(parseInviteUrl('togetherwego://join?code=aml2026').code).toBe('AML-2026');
    expect(parseInviteUrl('togetherwego://join?code=aml-2026#section').code).toBe('AML-2026');
  });

  it('ignores anything that is not an invite', () => {
    expect(parseInviteUrl('')).toEqual({ code: null, server: null });
    expect(parseInviteUrl('https://example.org/')).toEqual({ code: null, server: null });
    expect(parseInviteUrl('togetherwego://join?code=hello')).toEqual({ code: null, server: null });
```

```
    expect(parseInviteUrl('togetherwego://join?code=AML-2026&server=not%20an%20address%20at%20all').server).toBeNull();
    expect(parseInviteUrl('togetherwego://join?code=%E0%A4%A').code).toBeNull(); // broken percent-encoding does not crash
  });

  it('writes a message that works even when the link does not open the app', () => {
    const text = inviteText('Amalfi Squad', 'AML-2026', 'togetherwego://join?code=AML-2026');
    expect(text).toContain('Amalfi Squad');
    expect(text).toContain('AML-2026');
    expect(text).toContain('togetherwego://join?code=AML-2026');
    expect(text).toContain('Join a trip');
  });
});
```

src/sync/__tests__/secureRandom.test.ts

```
import { newNodeId } from '../hlc';
import { secureRandom } from '../secureRandom';

describe('secureRandom', () => {
  it('gives numbers from 0 up to (not including) 1', () => {
    for (let i = 0; i < 1000; i++) {
      const n = secureRandom();
      expect(n).toBeGreaterThanOrEqual(0);
      expect(n).toBeLessThan(1);
    }
  });

  it('does not repeat itself, and makes distinct ids', () => {
    expect(new Set(Array.from({ length: 500 }, secureRandom)).size).toBe(500);
    const ids = new Set(Array.from({ length: 500 }, () => newNodeId(secureRandom)));
    expect(ids.size).toBe(500);
    for (const id of ids) expect(id).toMatch(/^[0-9a-z]{8}$/);
  });
});
```

src/sync/__tests__/transport.test.ts

```
import { NetworkError, ServerError, createHttpTransport, normalizeServerUrl } from '../transport';

/** A fetch that never answers on its own but stops when told to (like a wifi that drops packets). */
const hangingFetch: typeof fetch = (_url, init) =>
  new Promise((_resolve, reject) => {
    init?.signal?.addEventListener('abort', () => reject(Object.assign(new Error('aborted'), { name: 'AbortError' })));
  });

const jsonFetch = (status: number, body: unknown): typeof fetch => async () => ({ status, json: async () => body }) as Response;

// The phone's own URL class (React Native's) is a simple text matcher, much looser than Node's or a browser's.
// Run the address checks against both, so a typo is rejected on the phone too.
const PHONE_URL = jest.requireActual('react-native/Libraries/Blob/URL').URL as typeof URL;
describe.each([
  ['Node', URL],
  ['the phone', PHONE_URL],
])('normalizeServerUrl with the URL class of %s', (_name, UrlClass) => {
  const original = globalThis.URL;
  beforeAll(() => {
    globalThis.URL = UrlClass;
  });
  afterAll(() => {
    globalThis.URL = original;
  });

  it('adds http:// and drops paths and trailing slashes', () => {
    expect(normalizeServerUrl('192.168.1.20:8787')).toBe('http://192.168.1.20:8787');
    expect(normalizeServerUrl(' http://192.168.1.20:8787/ ')).toBe('http://192.168.1.20:8787');
    expect(normalizeServerUrl('https://sync.example.org/path?x=1')).toBe('https://sync.example.org');
    expect(normalizeServerUrl('HTTP://Laptop.local:8787')).toBe('http://laptop.local:8787');
    expect(normalizeServerUrl('localhost:8787')).toBe('http://localhost:8787');
  });

  it('rejects things that are not addresses', () => {
    for (const bad of ['', '   ', 'http://', 'not an address at all', 'my laptop', 'ftp://192.168.1.2', 'http://192.168.1.2 :8787', 'htt
p://-bad-.com', 'http://[::1]:8787', 'http://host:99999999']) {
      expect(normalizeServerUrl(bad)).toBeNull();
    }
  });
});

describe('normalizeServerUrl', () => {
  it('adds http:// and drops paths and trailing slashes', () => {
    expect(normalizeServerUrl('192.168.1.20:8787')).toBe('http://192.168.1.20:8787');
    expect(normalizeServerUrl(' http://192.168.1.20:8787/ ')).toBe('http://192.168.1.20:8787');
    expect(normalizeServerUrl('https://sync.example.org/path?x=1')).toBe('https://sync.example.org');
  });
  it('rejects empty and nonsense input', () => {
    expect(normalizeServerUrl('')).toBeNull();
```

```
    expect(normalizeServerUrl('   ')).toBeNull();
    expect(normalizeServerUrl('http://')).toBeNull();
  });
});

describe('http transport', () => {
  it('gives up on a server that never answers instead of waiting forever', async () => {
    const t = createHttpTransport('http://10.0.0.9:8787', hangingFetch, { timeoutMs: 40 });
    const started = Date.now();
    await expect(t.push('r1', 'AML-2026', 'phone001', [])).rejects.toThrow(NetworkError);
    await expect(t.pull('r1', 0, 'phone001', 0)).rejects.toThrow(/too long/);
    expect(Date.now() - started).toBeLessThan(1000);
  });

  it('looks up invite codes with a shorter limit than other requests', async () => {
    const t = createHttpTransport('http://10.0.0.9:8787', hangingFetch, { timeoutMs: 5000, lookupTimeoutMs: 40 });
    const started = Date.now();
    await expect(t.lookup('AML-2026')).rejects.toThrow(/too long/);
    expect(Date.now() - started).toBeLessThan(1000);
  });

  it('gives waiting requests extra time on top of the limit', async () => {
    const t = createHttpTransport('http://10.0.0.9:8787', hangingFetch, { timeoutMs: 20 });
    const started = Date.now();
    await expect(t.pull('r1', 0, 'me', 150)).rejects.toThrow(NetworkError);
    expect(Date.now() - started).toBeGreaterThanOrEqual(150);
  });

  it('lets the app cancel a request without calling it a network problem', async () => {
    const t = createHttpTransport('http://10.0.0.9:8787', hangingFetch, { timeoutMs: 5000 });
    const stop = new AbortController();
    const pending = t.pull('r1', 0, 'me', 1000, stop.signal);
    stop.abort();
    const error = await pending.catch((e: unknown) => e);
    expect(error).toBeInstanceOf(Error);
    expect(error).not.toBeInstanceOf(NetworkError);
  });

  it('turns a refused connection into a NetworkError', async () => {
    const refused: typeof fetch = async () => {
      throw new TypeError('Network request failed');
    };
    await expect(createHttpTransport('http://10.0.0.9:8787', refused).lookup('AML-2026')).rejects.toThrow(NetworkError);
  });

  it('treats 404 on a code lookup as "no such trip" and other errors as server errors', async () => {
    expect(await createHttpTransport('http://x:1', jsonFetch(404, { error: 'No trip with that code' })).lookup('ZZZ-0000')).toBeNull();
    expect(await createHttpTransport('http://x:1', jsonFetch(200, { tripId: 'rabc' })).lookup('AML-2026')).toBe('rabc');
    const err = await createHttpTransport('http://x:1', jsonFetch(429, { error: 'Too many tries. Wait a minute.' })).lookup('AML-2026').
catch((e: unknown) => e);
    expect(err).toBeInstanceOf(ServerError);
    expect((err as ServerError).status).toBe(429);
    expect((err as ServerError).message).toMatch(/Too many tries/);
  });
});
```

src/sync/actions.ts

```
/** The few things screens ask the sync engine to do: share a trip, join one, test the server. */
import { useAppStore } from '@/store/useAppStore';

import { useBlobCache } from './blobCache';
import { CodeTakenError } from './engine';
import { deleteSharedFiles } from './fileio';
import { SERVER_PORT, effectiveServerUrl } from './prefs';
import { syncReady } from './runtime';
import { NetworkError, ServerError, normalizeServerUrl } from './transport';

export type JoinOutcome = { ok: true; tripId: string; synced: boolean; offlineCopy: boolean } | { ok: false; error: string };

/**
 * Joins a trip by invite code. If a sync server is set, its shared trip is downloaded first, so everyone ends up
 * in the same trip. If the server is not reachable (or does not know the code) the code is tried on this phone,
 * which is how the demo trips and offline use keep working.
 */
export async function joinTripAnywhere(code: string): Promise<JoinOutcome> {
  let unreachable = false;
  if (effectiveServerUrl()) {
    try {
      const engine = await syncReady();
      if (await engine.join(code)) {
        const r = useAppStore.getState().joinTrip(code); // adds this person to the trip we just downloaded
        return r.ok ? { ok: true, tripId: r.tripId, synced: true, offlineCopy: false } : { ok: false, error: r.error };
      }
    } catch (e) {
      if (e instanceof ServerError && e.status === 429) return { ok: false, error: e.message };
```

```
      unreachable = e instanceof NetworkError;
    }
  }
  const r = useAppStore.getState().joinTrip(code);
  return r.ok ? { ok: true, tripId: r.tripId, synced: false, offlineCopy: unreachable } : { ok: false, error: r.error };
}

export type ShareOutcome = { ok: true } | { ok: false; reason: 'no-server' | 'code-taken' | 'network' | 'error'; message: string };

/** Starts sharing a trip with other phones. */
export async function shareTrip(tripId: string): Promise<ShareOutcome> {
  const url = effectiveServerUrl();
  if (!url) return { ok: false, reason: 'no-server', message: 'Set the sync server address first (Settings → Sync across phones).' };
  try {
    const engine = await syncReady();
    await engine.share(tripId);
    engine.start();
    return { ok: true };
  } catch (e) {
    if (e instanceof CodeTakenError) return { ok: false, reason: 'code-taken', message: e.message };
    if (e instanceof NetworkError) return { ok: false, reason: 'network', message: `Can’t reach the sync server at ${url}. Is it runnin
g, and are you on the same wifi?` };
    return { ok: false, reason: 'error', message: e instanceof Error ? e.message : 'Something went wrong' };
  }
}

/** Stops syncing a trip on this phone. It stays as a normal trip. */
export async function stopSharing(tripId: string): Promise<void> {
  (await syncReady()).detach(tripId);
}

/** Forgets all sync state (used when the app's data is reset). */
export async function resetSync(): Promise<void> {
  await (await syncReady()).reset();
  useBlobCache.getState().clear();
  deleteSharedFiles();
}

export type ConnectionTest = { ok: true; trips: number } | { ok: false; message: string };

/** Asks a server address "are you there?" with a short time limit. */
export async function testServer(input: string, fetchImpl: typeof fetch = fetch): Promise<ConnectionTest> {
  const url = normalizeServerUrl(input);
  if (!url) return { ok: false, message: `That doesn’t look like an address. Try something like 192.168.1.20:${SERVER_PORT}` };
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), 5000);
  try {
    const res = await fetchImpl(`${url}/health`, { signal: controller.signal });
    if (!res.ok) return { ok: false, message: `The address answered, but not like the sync server (error ${res.status}).` };
    const data = (await res.json()) as { ok?: boolean; trips?: number };
    return data.ok ? { ok: true, trips: data.trips ?? 0 } : { ok: false, message: 'The address answered, but it is not the TogetherWeGo
sync server.' };
  } catch {
    return { ok: false, message: `Couldn’t reach ${url}. Check that the server is running (npm run server) and that this phone is on the
same wifi.` };
  } finally {
    clearTimeout(timer);
  }
}
```

src/sync/adapter.ts

```
import { useAppStore } from '@/store/useAppStore';

import type { StoreAdapter } from './engine';

/** Lets the sync engine read and write trips in the app's store. */
export const storeAdapter: StoreAdapter = {
  getTrip: (id) => useAppStore.getState().trips.find((t) => t.id === id),
  getUserId: () => useAppStore.getState().session?.userId,
  setTrip: (trip) => {
    const { trips } = useAppStore.getState();
    useAppStore.setState({ trips: trips.map((t) => (t.id === trip.id ? trip : t)) });
  },
  addTrip: (trip) => {
    const { trips } = useAppStore.getState();
    useAppStore.setState({ trips: [trip, ...trips.filter((t) => t.id !== trip.id)], activeTripId: trip.id });
  },
  subscribe: (listener) => useAppStore.subscribe(() => listener()),
};
```

src/sync/blobCache.ts

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import { Platform } from 'react-native';
import { create } from 'zustand';
```

```
import { createJSONStorage, persist } from 'zustand/middleware';

import type { BlobCache } from './blobs';

interface BlobCacheState {
  /** Server file id -> where this phone keeps its copy. */
  files: Record<string, string>;
  /** Items that cannot be shared, so the app does not complain about them every time it opens. */
  rejected: Record<string, true>;
  setFile: (blobId: string, uri: string) => void;
  reject: (key: string) => void;
  clear: () => void;
}

/**
 * What this phone has downloaded. Saved between runs, except in the web preview, where a copy is a temporary
 * address that stops working when the page closes (so it is fetched again next time).
 */
export const useBlobCache = create<BlobCacheState>()(
  persist(
    (set) => ({
      files: {},
      rejected: {},
      setFile: (blobId, uri) => set((s) => ({ files: { ...s.files, [blobId]: uri } })),
      reject: (key) => set((s) => ({ rejected: { ...s.rejected, [key]: true } })),
      clear: () => set({ files: {}, rejected: {} }),
    }),
    {
      name: 'togetherwego-files',
      storage: createJSONStorage(() => AsyncStorage),
      partialize: (s) => ({ files: Platform.OS === 'web' ? {} : s.files, rejected: s.rejected }),
    },
  ),
);

/** The same cache, in the shape the file syncing code asks for. */
export const blobCache: BlobCache = {
  get: (blobId) => useBlobCache.getState().files[blobId],
  set: (blobId, uri) => useBlobCache.getState().setFile(blobId, uri),
  isRejected: (key) => !!useBlobCache.getState().rejected[key],
  reject: (key) => useBlobCache.getState().reject(key),
};

/** How many files are still waiting to be sent or fetched (for a small "syncing files" hint). */
export const useFileStatus = create<{ pending: number }>(() => ({ pending: 0 }));
```

src/sync/blobs.ts

```
/**
 * Files (photos, documents, receipts) travel through the sync server on their own, next to the trip's text changes.
 *
 *  - Your own new file is uploaded, then the server's id for it (`blob`) is written on the item. That small change
 *    syncs like any other, so friends' phones hear that a file exists.
 *  - A friend's item that has a `blob` but no file on this phone is downloaded and remembered in a cache, so it also
 *    opens with no connection later.
 *
 * A pass is safe to run at any time. Failures are retried later with a growing pause and never block the screens.
 */
import type { FileList } from '@/store/slices/files';
import type { Trip } from '@/store/types';

import { quiet } from './timers';

/** Matches the server's limit (server/index.js). Bigger files stay on the phone that has them. */
export const MAX_SHARED_FILE_BYTES = 8 * 1024 * 1024;

export type TransferFailure = 'network' | 'rejected' | 'missing';

/** What went wrong with an upload or download: no connection, the server said no, or it does not have the file. */
export class FileTransferError extends Error {
  constructor(
    readonly kind: TransferFailure,
    message: string,
  ) {
    super(message);
  }
}

/** Where files live on this phone and how they reach the server. The app has a native and a web version; tests use a fake. */
export interface BlobFiles {
  exists(uri: string): Promise<boolean>;
  /** Size in bytes, or null when there is no such file. */
  size(uri: string): Promise<number | null>;
  /** Sends the file to `url` (HTTP PUT with the raw bytes). Throws FileTransferError. */
  upload(uri: string, url: string): Promise<void>;
  /** Fetches `url` and keeps it as a file called `name`; returns where it is now. Throws FileTransferError. */
```

```
  download(url: string, name: string): Promise<string>;
}

/** This phone's memory of which downloaded file belongs to which blob. */
export interface BlobCache {
  get(blobId: string): string | undefined;
  set(blobId: string, uri: string): void;
  /** Items that cannot be shared (too big, file gone): remembered so the app does not complain every time it opens. */
  isRejected(key: string): boolean;
  reject(key: string): void;
}

/** For each list that holds files: the field with this phone's file path and the field with the server's id. */
export const FILE_FIELDS: Record<FileList, { local: 'uri' | 'receipt'; blob: 'blob' | 'receiptBlob' }> = {
  photos: { local: 'uri', blob: 'blob' },
  docs: { local: 'uri', blob: 'blob' },
  expenses: { local: 'receipt', blob: 'receiptBlob' },
};

const LISTS = Object.keys(FILE_FIELDS) as FileList[];

/** A real file on this phone, as opposed to the names of the pictures that come with the app ("amalfi"). */
export const isLocalFile = (uri: string | undefined): uri is string => !!uri && /^(file|content|blob|data):/i.test(uri);

/**
 * The pictures that come with the app are not files to send: every phone has them already. A photo that uses one is
 * shared by name, as `asset:amalfi`, in the same field that holds a server file id for real files.
 */
const ASSET_PREFIX = 'asset:';
export const isAssetRef = (blob: string | undefined): blob is string => !!blob && blob.startsWith(ASSET_PREFIX);
export const assetRef = (key: string) => `${ASSET_PREFIX}${key}`;

/**
 * What to show for a photo, document or receipt: this phone's own file, a bundled picture shared by name, the copy
 * downloaded from a friend, or (while that is on its way) nothing yet.
 */
export function resolveFile(local: string | undefined, blob: string | undefined, downloaded: string | undefined): { uri: string | undefi
ned; loading: boolean } {
  if (local) return { uri: local, loading: false };
  if (isAssetRef(blob)) return { uri: blob.slice(ASSET_PREFIX.length), loading: false };
  if (downloaded) return { uri: downloaded, loading: false };
  return { uri: undefined, loading: !!blob };
}

const MIME_EXT: Record<string, string> = { jpeg: 'jpg', 'svg+xml': 'svg', 'x-icon': 'ico' };

/** "photo.JPG?x=1" -> "jpg". A data: address names its type instead of a file. */
export function extensionOf(uri: string, hint = 'bin'): string {
  const data = /^data:[\w.+-]+\/([\w.+-]+)/i.exec(uri);
  const raw = (data ? data[1] : (/\.([A-Za-z0-9]{1,5})$/.exec(uri.split(/[?#]/)[0])?.[1] ?? hint)).toLowerCase();
  return MIME_EXT[raw] ?? (raw.replace(/[^a-z0-9]/g, '').slice(0, 5) || hint);
}

/** The server's id for a file. It ends in `_ext` so the downloaded copy gets a name that apps can open. */
export const makeBlobId = (random: string, uri: string, hint?: string) => `${random}_${extensionOf(uri, hint)}`;

/** "k3j9x2m1_pdf" -> "k3j9x2m1.pdf" */
export const fileNameOf = (blobId: string) => blobId.replace(/_([a-z0-9]{1,5})$/, '.$1');

const hintFor = (list: FileList, item: { kind?: string }) => (list === 'docs' ? (item.kind === 'pdf' ? 'pdf' : item.kind === 'image' ?
'jpg' : 'bin') : 'jpg');

interface Job {
  kind: 'upload' | 'download' | 'reference';
  tripId: string;
  remoteId: string;
  list: FileList;
  itemId: string;
  /** For an upload: this phone's file. For a reference: the bundled picture's name. */
  uri?: string;
  /** For a download: the server's id. */
  blobId?: string;
  hint: string;
}

export interface BlobSyncOptions {
  /** The trips in the app. */
  trips: () => Trip[];
  /** The id the server uses for a trip, or undefined when the trip is not shared. */
  remoteId: (tripId: string) => string | undefined;
  /** The sync server's address, or null when none is set. */
  baseUrl: () => string | null;
  files: BlobFiles;
  cache: BlobCache;
  /** Writes the server's id onto the item (so it syncs like any other change). */
  attach: (tripId: string, list: FileList, itemId: string, blobId: string) => void;
```

```
  newId: () => string;
  /** Something the person should hear about (a file that is too big to share). */
  onNotice?: (message: string) => void;
  /** How many files are still waiting (for a "syncing…" hint). */
  onPending?: (count: number) => void;
}

export class BlobSync {
  private timer: ReturnType<typeof setTimeout> | null = null;
  private running = false;
  private again = false;
  private pause = 0;
  private stopped = false;

  constructor(private readonly o: BlobSyncOptions) {}

  /** Look for files to send or fetch soon. Cheap to call after every change; calls close together become one pass. */
  schedule(delayMs = 500): void {
    if (this.stopped || this.timer) return;
    this.timer = quiet(
      setTimeout(() => {
        this.timer = null;
        void this.run();
      }, delayMs),
    );
  }

  stop(): void {
    this.stopped = true;
    if (this.timer) clearTimeout(this.timer);
    this.timer = null;
  }

  /** Does one full pass (or asks the running one to go round again). Resolves when it is done. */
  async run(): Promise<void> {
    if (this.stopped) return;
    if (this.running) {
      this.again = true;
      return;
    }
    this.running = true;
    try {
      let failed: boolean;
      do {
        this.again = false;
        failed = await this.pass();
      } while (this.again && !this.stopped);
      this.pause = failed ? Math.min(this.pause ? this.pause * 2 : 3000, 60000) : 0;
      if (failed) this.schedule(this.pause);
    } finally {
      this.running = false;
    }
  }

  private collect(): Job[] {
    const jobs: Job[] = [];
    for (const trip of this.o.trips()) {
      const remoteId = this.o.remoteId(trip.id);
      if (!remoteId) continue;
      for (const list of LISTS) {
        const { local, blob } = FILE_FIELDS[list];
        for (const item of trip[list] as unknown as Record<string, string | undefined>[]) {
          const uri = item[local];
          const blobId = item[blob];
          const base = { tripId: trip.id, remoteId, list, itemId: item.id as string, hint: hintFor(list, item as { kind?: string }) };
          if (isLocalFile(uri) && !blobId) jobs.push({ ...base, kind: 'upload', uri });
          else if (list === 'photos' && uri && !isLocalFile(uri) && !blobId) jobs.push({ ...base, kind: 'reference', uri });
          else if (blobId && !isLocalFile(uri) && !isAssetRef(blobId)) jobs.push({ ...base, kind: 'download', blobId });
        }
      }
    }
    return jobs;
  }

  /** Returns true when something should be tried again later. */
  private async pass(): Promise<boolean> {
    const base = this.o.baseUrl();
    if (!base) {
      this.o.onPending?.(0);
      return false;
    }
    const jobs = this.collect();
    for (const job of jobs.filter((j) => j.kind === 'reference')) this.o.attach(job.tripId, job.list, job.itemId, assetRef(job.uri as st
ring)); // nothing to send: just say which picture it is
    jobs.splice(0, jobs.length, ...jobs.filter((j) => j.kind !== 'reference'));
    this.o.onPending?.(jobs.length);
```

```
    let retry = false;
    for (const job of jobs) {
      if (this.stopped) return false;
      try {
        if (job.kind === 'upload') await this.upload(base, job);
        else await this.download(base, job);
      } catch (e) {
        retry = true;
        if (e instanceof FileTransferError && e.kind === 'network') break; // the server is out of reach: no point trying the rest now
      }
    }
    if (!retry) this.o.onPending?.(0);
    return retry;
  }

  private async upload(base: string, job: Job): Promise<void> {
    const key = `${job.tripId}/${job.list}/${job.itemId}`;
    if (this.o.cache.isRejected(key)) return;
    const uri = job.uri as string;
    const size = await this.o.files.size(uri);
    if (size === null) return this.o.cache.reject(key); // the file is gone from this phone: nothing to send
    if (size > MAX_SHARED_FILE_BYTES) return this.refuse(key, size);
    const blobId = makeBlobId(this.o.newId(), uri, job.hint);
    try {
      await this.o.files.upload(uri, `${base}/v1/trips/${job.remoteId}/files/${blobId}`);
    } catch (e) {
      if (e instanceof FileTransferError && e.kind === 'rejected') return this.refuse(key, size);
      throw e; // no connection, or the server has not heard of this trip yet: try again later
    }
    this.o.attach(job.tripId, job.list, job.itemId, blobId);
  }

  private refuse(key: string, size: number): void {
    this.o.cache.reject(key);
    this.o.onNotice?.(`That file is ${(size / 1024 / 1024).toFixed(1)} MB. Files over ${MAX_SHARED_FILE_BYTES / 1024 / 1024} MB stay on
your phone and are not shared with friends.`);
  }

  private async download(base: string, job: Job): Promise<void> {
    const blobId = job.blobId as string;
    const have = this.o.cache.get(blobId);
    if (have && (await this.o.files.exists(have))) return;
    const uri = await this.o.files.download(`${base}/v1/trips/${job.remoteId}/files/${blobId}`, fileNameOf(blobId));
    this.o.cache.set(blobId, uri);
  }
}
```

src/sync/crdt.ts

```
/**
 * The merge engine behind phone-to-phone sync.
 *
 *   diffTrip      : "what changed?"  Compares two versions of a trip and returns the small changes (ops) between them.
 *   applyOps      : "merge these"    Folds ops from any phone into the registers. Order and repeats do not matter.
 *   materializeTrip: "show me the trip" Rebuilds a Trip object from the registers.
 *
 * Rules that make every phone end up identical:
 *  1. Every op carries a unique, sortable stamp (see hlc.ts). For one register the op with the biggest stamp wins.
 *  2. Deleting an item is remembered (a tombstone). Changes older than the tombstone are ignored, so a delete beats a
 *     concurrent edit. Creating the item again later (a newer creation stamp) brings it back.
 *  3. Applying the same op twice, or in a different order, gives the same result.
 */
import type { Trip } from '@/store/types';

import { compareStamps } from './hlc';
import { CREATED, PATH_SEP, TRIP_SCHEMA, pathKey, type ItemSchema, type ListSchema } from './schema';

export type Json = string | number | boolean | null | Json[] | { [k: string]: Json };

/** One change. `p` is the path of the register, `v` the new value, `d` marks a delete, `t` is the stamp. */
export interface Op {
  p: string[];
  v?: Json;
  d?: 1;
  t: string;
}

/** What we remember about one register: the stamp of the change that wrote it, and its value (or that it was deleted). */
export interface Reg {
  t: string;
  v?: Json;
  d?: 1;
}
export type Registers = Record<string, Reg>;
```

```
// ------------------------------------------------------------------ diff

/** The ops that turn `prev` into `next`. Pass `undefined` as `prev` to describe a whole trip from scratch. */
export function diffTrip(prev: Trip | undefined, next: Trip, stamp: () => string): Op[] {
  const out: Op[] = [];
  diffItem([], prev, next, TRIP_SCHEMA, out, stamp);
  return out;
}

const setOp = (p: string[], v: Json, t: string): Op => ({ p, v, t });
const delOp = (p: string[], t: string): Op => ({ p, d: 1, t });

function diffItem(path: string[], prev: any, next: any, S: ItemSchema, out: Op[], stamp: () => string): void {
  for (const f of S.scalars) {
    const a = prev?.[f];
    const b = next[f];
    if (a !== b) out.push(b === undefined ? delOp([...path, f], stamp()) : setOp([...path, f], b, stamp()));
  }
  for (const f of S.json ?? []) {
    const a = prev?.[f];
    const b = next[f];
    if (a === b || (a !== undefined && b !== undefined && deepEqual(a, b))) continue;
    out.push(b === undefined ? delOp([...path, f], stamp()) : setOp([...path, f], b, stamp()));
  }
  for (const f of S.sets ?? []) {
    if (prev?.[f] === next[f]) continue;
    const before = new Set<string>(prev?.[f] ?? []);
    const after = new Set<string>(next[f] ?? []);
    for (const x of after) if (!before.has(x)) out.push(setOp([...path, f, x], 1, stamp()));
    for (const x of before) if (!after.has(x)) out.push(delOp([...path, f, x], stamp()));
  }
  for (const f of S.maps ?? []) {
    if (prev?.[f] === next[f]) continue;
    const a: Record<string, Json> = prev?.[f] ?? {};
    const b: Record<string, Json> = next[f] ?? {};
    for (const k of Object.keys(b)) if (a[k] !== b[k]) out.push(setOp([...path, f, k], b[k], stamp()));
    for (const k of Object.keys(a)) if (!(k in b)) out.push(delOp([...path, f, k], stamp()));
  }
  for (const [name, L] of Object.entries(S.lists ?? {})) diffList([...path, name], prev?.[name], next[name], L, out, stamp);
}

function diffList(path: string[], prevArr: any[] | undefined, nextArr: any[] | undefined, L: ListSchema, out: Op[], stamp: () => strin
g): void {
  if (prevArr === nextArr) return;
  const shared = (arr: any[] | undefined) => (arr ?? []).filter((x) => !L.share || L.share(x));
  const prevItems = new Map<string, any>(shared(prevArr).map((x) => [x.id, x]));
  const nextItems = shared(nextArr);
  const nextIds = new Set<string>(nextItems.map((x) => x.id));
  for (const id of prevItems.keys()) if (!nextIds.has(id)) out.push(delOp([...path, id], stamp()));
  // Oldest first, so creation stamps rise in the same order the items were made.
  const oldestFirst = L.order === 'asc' ? nextItems : [...nextItems].reverse();
  for (const item of oldestFirst) {
    const old = prevItems.get(item.id);
    if (!old) {
      out.push(setOp([...path, item.id, CREATED], 1, stamp()));
      diffItem([...path, item.id], undefined, item, L.item, out, stamp);
    } else if (old !== item) {
      diffItem([...path, item.id], old, item, L.item, out, stamp);
    }
  }
}

// ------------------------------------------------------------------ merge

/** Where a path points: the chain of items it sits inside, and whether it is an item itself. */
interface Resolved {
  items: string[][];
  isItem: boolean;
}

/** Checks a path against the schema. Unknown paths (from a newer version of the app) return null and are ignored. */
export function resolve(path: readonly string[], schema: ItemSchema = TRIP_SCHEMA, items: string[][] = [], at = 0): Resolved | null {
  const seg = path[at];
  if (seg === undefined) return null;
  const last = at === path.length - 1;
  if (schema.scalars.includes(seg) || schema.json?.includes(seg)) return last ? { items, isItem: false } : null;
  if (seg === CREATED && items.length > 0) return last ? { items, isItem: false } : null;
  if (schema.sets?.includes(seg) || schema.maps?.includes(seg)) return at === path.length - 2 ? { items, isItem: false } : null;
  const list = schema.lists?.[seg];
  if (list) {
    if (path.length < at + 2) return null;
    const itemPath = path.slice(0, at + 2) as string[];
    const chain = [...items, itemPath];
    if (path.length === at + 2) return { items: chain, isItem: true };
    return resolve(path, list.item, chain, at + 2);
  }
```

```
  return null;
}

/** Folds ops into the registers. Returns true if anything actually changed. */
export function applyOps(regs: Registers, ops: readonly Op[]): boolean {
  let changed = false;
  for (const op of ops) if (applyOp(regs, op)) changed = true;
  return changed;
}

function applyOp(regs: Registers, op: Op): boolean {
  const info = resolve(op.p);
  if (!info) return false;
  if (info.isItem && !op.d) return false; // an item node can only be deleted; it is created by its creation stamp
  // A change older than a delete of the item it belongs to is void.
  const owners = info.isItem ? info.items.slice(0, -1) : info.items;
  for (const itemPath of owners) {
    const tomb = regs[pathKey(itemPath)];
    if (tomb?.d && compareStamps(op.t, tomb.t) <= 0) return false;
  }
  const key = pathKey(op.p);
  const cur = regs[key];
  if (cur && compareStamps(op.t, cur.t) <= 0) return false;
  regs[key] = op.d ? { t: op.t, d: 1 } : { t: op.t, v: op.v };
  // Everything older that was inside a deleted item is void; dropping it keeps every phone's registers identical.
  if (info.isItem) forgetOlderBelow(regs, key, op.t);
  return true;
}

function forgetOlderBelow(regs: Registers, itemKey: string, t: string): void {
  const prefix = itemKey + PATH_SEP;
  for (const key of Object.keys(regs)) if (key.startsWith(prefix) && compareStamps(regs[key].t, t) < 0) delete regs[key];
}

// ---------------------------------------------------------------- rebuild

interface Node {
  reg?: Reg;
  kids: Map<string, Node>;
}

function buildTree(regs: Registers): Node {
  const root: Node = { kids: new Map() };
  for (const [key, reg] of Object.entries(regs)) {
    let n = root;
    for (const seg of key.split(PATH_SEP)) {
      let c = n.kids.get(seg);
      if (!c) {
        c = { kids: new Map() };
        n.kids.set(seg, c);
      }
      n = c;
    }
    n.reg = reg;
  }
  return root;
}

/** Rebuilds the shared part of a trip from the registers. Returns null if the trip is not complete yet. */
export function materializeTrip(regs: Registers): Trip | null {
  return readItem(TRIP_SCHEMA, buildTree(regs), '', undefined) as Trip | null;
}

/** `floor` is the stamp of the newest delete above this item: anything written before it is ignored. */
function readItem(S: ItemSchema, node: Node, floor: string, id: string | undefined): any | null {
  const alive = (r?: Reg): r is Reg => !!r && !r.d && compareStamps(r.t, floor) > 0;
  const obj: any = id === undefined ? {} : { id };
  for (const f of [...S.scalars, ...(S.json ?? [])]) {
    const r = node.kids.get(f)?.reg;
    if (alive(r)) obj[f] = r.v;
  }
  for (const f of S.sets ?? []) {
    const kids = [...(node.kids.get(f)?.kids ?? [])].filter(([, k]) => alive(k.reg));
    kids.sort((a, b) => compareStamps(a[1].reg!.t, b[1].reg!.t) || (a[0] < b[0] ? -1 : 1));
    obj[f] = kids.map(([k]) => k);
  }
  for (const f of S.maps ?? []) {
    obj[f] = Object.fromEntries([...(node.kids.get(f)?.kids ?? [])].filter(([, k]) => alive(k.reg)).map(([k, v]) => [k, v.reg!.v]));
  }
  for (const [name, L] of Object.entries(S.lists ?? {})) obj[name] = readList(L, node.kids.get(name), floor);
  for (const r of S.required ?? []) if (obj[r] === undefined) return null;
  return obj;
}

function readList(L: ListSchema, node: Node | undefined, floor: string): any[] {
  const found: { order: string; id: string; obj: any }[] = [];
```

```
  for (const [id, child] of node?.kids ?? []) {
    const created = child.kids.get(CREATED)?.reg;
    const tomb = child.reg?.d ? child.reg.t : '';
    const itemFloor = compareStamps(tomb, floor) > 0 ? tomb : floor;
    if (!created || created.d || compareStamps(created.t, itemFloor) <= 0) continue;
    const obj = readItem(L.item, child, itemFloor, id);
    if (obj) found.push({ order: created.t, id, obj });
  }
  found.sort((a, b) => compareStamps(a.order, b.order) || (a.id < b.id ? -1 : 1));
  const items = found.map((f) => f.obj);
  return L.order === 'desc' ? items.reverse() : items;
}

// ---------------------------------------------------------------- helpers

export function deepEqual(a: any, b: any): boolean {
  if (a === b) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Array.isArray(a) !== Array.isArray(b)) return false;
  const ka = Object.keys(a);
  const kb = Object.keys(b);
  if (ka.length !== kb.length) return false;
  return ka.every((k) => k in b && deepEqual(a[k], b[k]));
}
```

src/sync/engine.ts

```
/**
 * The sync engine: keeps a shared trip identical on every phone that has joined it.
 *
 *   phone edits the store ──► diff ──► ops ──► registers + outbox ──► server
 *   server ──► ops from friends ──► registers ──► rebuilt trip ──► store
 *
 * It knows nothing about React or the network library: the app store, the server connection and the place the
 * state is saved are all passed in, which is what lets the tests run two phones against one server in memory.
 */
import type { Trip } from '@/store/types';

import { applyOps, diffTrip, materializeTrip, type Op, type Registers } from './crdt';
import { HybridClock, newNodeId, type ClockState } from './hlc';
import { NetworkError, ServerError, type SyncTransport } from './transport';
import { quiet } from './timers';
import { toViewTrip } from './view';

/** What we remember about one shared trip. */
export interface TripSync {
  /** The trip's id on the server (long and random, so it works like a password). */
  remoteId: string;
  /** Everything we know about every shared field, with the stamp of its last change. */
  regs: Registers;
  /** How many of the server's changes we have already read. */
  cursor: number;
  /** Our own changes the server has not confirmed yet, in batches. */
  outbox: Op[][];
}

export interface SyncMeta {
  nodeId: string;
  clock?: ClockState;
  trips: Record<string, TripSync>;
}

/** How the engine reads and writes the app's trips. The real one wraps the zustand store. */
export interface StoreAdapter {
  getTrip(tripId: string): Trip | undefined;
  getUserId(): string | undefined;
  /** Replaces a trip that is already in the store. */
  setTrip(trip: Trip): void;
  /** Puts a trip that came from a friend into the store (replacing one with the same id) and makes it the active trip. */
  addTrip(trip: Trip): void;
  /** Calls the listener after any change to the trips. Returns a function that stops listening. */
  subscribe(listener: () => void): () => void;
}

export interface MetaStorage {
  load(): Promise<SyncMeta | null>;
  save(meta: SyncMeta): Promise<void>;
}

/** What the screens show about syncing. */
export interface SyncStatus {
  /** null until the first try, then whether the last conversation with the server worked. */
  online: boolean | null;
  /** Changes made here that the server has not confirmed. */
  pending: number;
  /** Phones talking to the server about our shared trips (including this one). */
```

```
  peers: number;
  lastSyncAt: number | null;
  error: string | null;
  /** Ids (in the app) of the trips being shared. */
  shared: string[];
}

export interface EngineOptions {
  store: StoreAdapter;
  storage: MetaStorage;
  /** The connection to the server, or null while no server address is set. Asked again every time it is needed. */
  getTransport: () => SyncTransport | null;
  /** Called after changes from friends were put into the store. */
  onRemoteChange?: (previous: Trip | undefined, next: Trip) => void;
  onStatus?: (status: SyncStatus) => void;
  now?: () => number;
  random?: () => number;
  /** How long to hold a request open waiting for news (ms). Tests use 0. */
  pollWaitMs?: number;
}

export class CodeTakenError extends Error {
  constructor(readonly code: string) {
    super(`The invite code ${code} is already used by another shared trip.`);
  }
}

interface Attached {
  sync: TripSync;
  /** The trip as it was when we last compared, so the next comparison finds only what changed since. */
  shadow: Trip | undefined;
  pushing: boolean;
  peers: number;
  retry: ReturnType<typeof setTimeout> | null;
  abort: AbortController | null;
  running: boolean;
  /** Ends a backoff wait early (used when stopping). */
  wake: (() => void) | null;
}

const BATCH = 400;
const chunk = <T>(items: T[], size: number): T[][] => Array.from({ length: Math.ceil(items.length / size) }, (_, i) => items.slice(i * s
ize, (i + 1) * size));

export class SyncEngine {
  private meta: SyncMeta = { nodeId: '', trips: {} };
  private clock!: HybridClock;
  private attached = new Map<string, Attached>();
  private unsubscribe: (() => void) | null = null;
  private applying = false;
  private running = false;
  private saveTimer: ReturnType<typeof setTimeout> | null = null;
  private status: SyncStatus = { online: null, pending: 0, peers: 0, lastSyncAt: null, error: null, shared: [] };

  constructor(private readonly o: EngineOptions) {}

  // ---------------------------------------------------------------- setup

  /** Loads what was saved, starts watching the store, and sends any change that was made while the app was closed. */
  async init(): Promise<void> {
    const saved = await this.o.storage.load();
    this.meta = saved ?? { nodeId: newNodeId(this.o.random), trips: {} };
    if (!this.meta.nodeId) this.meta.nodeId = newNodeId(this.o.random);
    this.clock = new HybridClock(this.meta.nodeId, this.o.now);
    this.clock.restore(this.meta.clock);
    for (const [tripId, sync] of Object.entries(this.meta.trips)) {
      this.attach(tripId, sync, materializeTrip(sync.regs) ? toViewTrip(materializeTrip(sync.regs)!, undefined) : undefined);
    }
    this.unsubscribe?.();
    this.unsubscribe = this.o.store.subscribe(() => this.onStoreChange());
    this.onStoreChange(); // catches changes made while the app was closed (they differ from the saved registers)
    this.publishStatus();
  }

  get nodeId(): string {
    return this.meta.nodeId;
  }

  isShared(tripId: string): boolean {
    return this.attached.has(tripId);
  }

  /** The id the server uses for a shared trip (undefined when the trip is not shared). */
  remoteId(tripId: string): string | undefined {
    return this.attached.get(tripId)?.sync.remoteId;
  }
```

```
  getStatus(): SyncStatus {
    return this.status;
  }

  /** Starts talking to the server: sending changes and listening for friends' changes. */
  start(): void {
    this.running = true;
    for (const [tripId, a] of this.attached) {
      this.schedulePush(tripId, 0);
      this.startPullLoop(tripId, a);
    }
  }

  stop(): void {
    this.running = false;
    for (const a of this.attached.values()) {
      a.abort?.abort();
      a.wake?.();
      if (a.retry) clearTimeout(a.retry);
      a.retry = null;
      a.running = false;
    }
    if (this.saveTimer) void this.saveNow(); // do not lose recent changes when the app goes to the background
  }

  /** Stops everything and forgets all sync state (used by "reset app data"). */
  async reset(): Promise<void> {
    this.stop();
    this.unsubscribe?.();
    this.unsubscribe = null;
    this.attached.clear();
    this.meta = { nodeId: newNodeId(this.o.random), trips: {} };
    await this.o.storage.save(this.meta);
    this.publishStatus();
  }

  // ------------------------------------------------------------------ sharing and joining

  /** Starts sharing a trip: describes the whole trip as changes and sends them. */
  async share(tripId: string): Promise<void> {
    if (this.attached.has(tripId)) return;
    const trip = this.o.store.getTrip(tripId);
    if (!trip) throw new Error('That trip no longer exists.');
    const transport = this.o.getTransport();
    if (!transport) throw new Error('Set the sync server address first.');
    const existing = await transport.lookup(trip.inviteCode);
    if (existing) throw new CodeTakenError(trip.inviteCode);
    const sync: TripSync = { remoteId: `r${newNodeId(this.o.random)}${newNodeId(this.o.random)}${newNodeId(this.o.random)}`, regs: {}, c
ursor: 0, outbox: [] };
    const ops = diffTrip(undefined, trip, () => this.clock.next());
    applyOps(sync.regs, ops);
    sync.outbox = chunk(ops, BATCH);
    this.meta.trips[tripId] = sync;
    this.attach(tripId, sync, trip);
    this.scheduleSave();
    this.publishStatus();
    if (this.running) {
      this.schedulePush(tripId, 0);
      this.startPullLoop(tripId, this.attached.get(tripId)!);
    }
  }

  /**
   * Joins a friend's shared trip using its invite code. Downloads the trip and puts it in the store.
   * Returns null when no trip has that code. Throws NetworkError when the server cannot be reached.
   */
  async join(code: string): Promise<Trip | null> {
    const transport = this.o.getTransport();
    if (!transport) throw new Error('Set the sync server address first.');
    const remoteId = await transport.lookup(code);
    if (!remoteId) return null;
    const already = [...this.attached.entries()].find(([, a]) => a.sync.remoteId === remoteId);
    if (already) return this.o.store.getTrip(already[0]) ?? null;
    const sync: TripSync = { remoteId, regs: {}, cursor: 0, outbox: [] };
    for (;;) {
      const page = await transport.pull(remoteId, sync.cursor, this.meta.nodeId, 0);
      page.ops.forEach((op) => this.clock.observe(op.t));
      applyOps(sync.regs, page.ops);
      sync.cursor = page.seq;
      if (sync.cursor >= page.total) break;
    }
    const shared = materializeTrip(sync.regs);
    if (!shared) throw new Error('That trip is not ready yet. Ask your friend to open the app and try again.');
    const trip = toViewTrip(shared, undefined);
    this.meta.trips[trip.id] = sync;
```

```
  this.attach(trip.id, sync, trip);
  this.applying = true;
  try {
    this.o.store.addTrip(trip);
  } finally {
    this.applying = false;
  }
  this.scheduleSave();
  this.publishStatus();
  if (this.running) {
    this.startPullLoop(trip.id, this.attached.get(trip.id)!);
  }
  return trip;
}

/** Stops syncing a trip on this phone. The trip stays here as a normal local trip. */
detach(tripId: string): void {
  const a = this.attached.get(tripId);
  if (!a) return;
  a.abort?.abort();
  a.wake?.();
  if (a.retry) clearTimeout(a.retry);
  a.running = false;
  this.attached.delete(tripId);
  delete this.meta.trips[tripId];
  this.scheduleSave();
  this.publishStatus();
}

// ---------------------------------------------------------------- local changes

private attach(tripId: string, sync: TripSync, shadow: Trip | undefined): void {
  this.attached.set(tripId, { sync, shadow, pushing: false, peers: 0, retry: null, abort: null, running: false, wake: null });
}

private onStoreChange(): void {
  if (this.applying) {
    // our own write of friends' changes: remember it as the new baseline instead of sending it back
    for (const [tripId, a] of this.attached) a.shadow = this.o.store.getTrip(tripId) ?? a.shadow;
    return;
  }
  let queued = false;
  for (const [tripId, a] of [...this.attached]) {
    const current = this.o.store.getTrip(tripId);
    if (!current) {
      this.detach(tripId); // the trip was deleted on this phone: stop syncing it, friends keep theirs
      continue;
    }
    if (current === a.shadow) continue;
    const ops = diffTrip(a.shadow, current, () => this.clock.next());
    a.shadow = current;
    if (!ops.length) continue;
    applyOps(a.sync.regs, ops);
    a.sync.outbox.push(...chunk(ops, BATCH));
    queued = true;
    this.schedulePush(tripId, 30);
  }
  if (queued) {
    this.scheduleSave();
    this.publishStatus();
  }
}

private schedulePush(tripId: string, delayMs: number): void {
  const a = this.attached.get(tripId);
  if (!a || !this.running || a.retry) return;
  a.retry = quiet(
    setTimeout(() => {
      a.retry = null;
      void this.push(tripId);
    }, delayMs),
  );
}

/** Sends our unconfirmed changes. Safe to call at any time; only one send per trip runs at once. */
async push(tripId: string): Promise<void> {
  const a = this.attached.get(tripId);
  const transport = this.o.getTransport();
  if (!a || a.pushing || !a.sync.outbox.length || !transport) return;
  a.pushing = true;
  try {
    while (a.sync.outbox.length) {
      const code = this.o.store.getTrip(tripId)?.inviteCode;
      await transport.push(a.sync.remoteId, code, this.meta.nodeId, a.sync.outbox[0]);
      a.sync.outbox.shift();
      this.scheduleSave();
```

```
    }
    this.setOnline(true, null);
  } catch (e) {
    this.setOnline(false, describe(e));
    if (this.running && !(e instanceof ServerError && e.status === 400)) this.schedulePush(tripId, 3000);
  } finally {
    a.pushing = false;
    this.publishStatus();
  }
}

// ---------------------------------------------------------------- changes from friends

/** Asks the server for new changes once and applies them. `waitMs` > 0 holds the request open for news. */
async pull(tripId: string, waitMs = 0, signal?: AbortSignal): Promise<number> {
  const a = this.attached.get(tripId);
  const transport = this.o.getTransport();
  if (!a || !transport) return 0;
  const page = await transport.pull(a.sync.remoteId, a.sync.cursor, this.meta.nodeId, waitMs, signal);
  if (!this.attached.has(tripId)) return 0;
  a.peers = page.peers;
  a.sync.cursor = page.seq;
  this.applyRemote(tripId, page.ops);
  this.scheduleSave();
  this.setOnline(true, null);
  this.publishStatus();
  return page.ops.length;
}

private applyRemote(tripId: string, ops: Op[]): void {
  const a = this.attached.get(tripId);
  if (!a || !ops.length) return;
  ops.forEach((op) => this.clock.observe(op.t));
  if (!applyOps(a.sync.regs, ops)) return; // nothing new (for example our own changes coming back)
  const shared = materializeTrip(a.sync.regs);
  if (!shared) return;
  const current = this.o.store.getTrip(tripId);
  const view = toViewTrip(shared, current);
  if (view === current) return;
  this.applying = true;
  try {
    this.o.store.setTrip(view);
  } finally {
    this.applying = false;
  }
  a.shadow = view;
  this.o.onRemoteChange?.(current, view);
}

private startPullLoop(tripId: string, a: Attached): void {
  if (a.running) return;
  a.running = true;
  void (async () => {
    let delay = 0;
    let quick = true; // the first answer after starting (or after a failure) is not held back, so "Live" shows at once
    while (this.running && a.running && this.attached.get(tripId) === a) {
      a.abort = new AbortController();
      try {
        await this.pull(tripId, quick ? 0 : (this.o.pollWaitMs ?? 20000), a.abort.signal);
        delay = 0;
        quick = false;
      } catch (e) {
        if (!this.running || !a.running || a.abort.signal.aborted) break;
        quick = true;
        this.setOnline(false, describe(e));
        this.publishStatus();
        delay = Math.min(delay ? delay * 2 : 1000, 15000);
        await new Promise<void>((resolve) => {
          const timer = quiet(setTimeout(resolve, delay));
          a.wake = () => {
            clearTimeout(timer);
            resolve();
          };
        });
        a.wake = null;
      }
    }
    a.running = false;
  })();
}

/** Sends and fetches once for every shared trip. Used by "Sync now" and by the tests. */
async syncNow(): Promise<void> {
  for (const tripId of [...this.attached.keys()]) {
    await this.push(tripId);
    try {
```

```
        await this.pull(tripId, 0);
      } catch (e) {
        this.setOnline(false, describe(e));
        this.publishStatus();
      }
    }
  }

  // ---------------------------------------------------------------- saving and status

  private scheduleSave(): void {
    if (this.saveTimer) return;
    this.saveTimer = quiet(
      setTimeout(() => {
        this.saveTimer = null;
        void this.saveNow();
      }, 800), // saving writes everything, so batch bursts of changes (the app also saves when it goes to the background)
    );
  }

  async saveNow(): Promise<void> {
    if (this.saveTimer) {
      clearTimeout(this.saveTimer);
      this.saveTimer = null;
    }
    this.meta.clock = this.clock.snapshot();
    await this.o.storage.save(this.meta);
  }

  private setOnline(online: boolean, error: string | null): void {
    this.status = { ...this.status, online, error, lastSyncAt: online ? (this.o.now ?? Date.now)() : this.status.lastSyncAt };
  }

  private publishStatus(): void {
    const all = [...this.attached.values()];
    this.status = {
      ...this.status,
      pending: all.reduce((n, a) => n + a.sync.outbox.reduce((m, b) => m + b.length, 0), 0),
      peers: all.reduce((n, a) => Math.max(n, a.peers), 0),
      shared: [...this.attached.keys()],
    };
    this.o.onStatus?.(this.status);
  }
}

function describe(e: unknown): string {
  if (e instanceof NetworkError) return 'Can’t reach the sync server';
  if (e instanceof ServerError) return e.message;
  return e instanceof Error ? e.message : 'Sync problem';
}
```

src/sync/fileio.ts

```
/** Moves files between this phone and the sync server: native files on iOS and Android, browser files on the web. */
import { Directory, File, Paths, UploadType } from 'expo-file-system';
import { Platform } from 'react-native';

import { FileTransferError, type BlobFiles } from './blobs';

/** Which kind of failure an HTTP status means. 5xx and anything else odd is treated like a lost connection: try again later. */
function failureFor(status: number, what: string): FileTransferError {
  if (status === 400 || status === 413) return new FileTransferError('rejected', `The server did not accept the file (${status})`);
  if (status === 404) return new FileTransferError('missing', `${what} is not on the server`);
  return new FileTransferError('network', `The server answered ${status}`);
}

const nativeFiles: BlobFiles = {
  async exists(uri) {
    try {
      return new File(uri).exists;
    } catch {
      return false;
    }
  },
  async size(uri) {
    try {
      const file = new File(uri);
      return file.exists ? file.size : null;
    } catch {
      return null;
    }
  },
  async upload(uri, url) {
    let status: number;
    try {
      const res = await new File(uri).upload(url, {
```

```
        httpMethod: 'PUT',
        uploadType: UploadType.BINARY_CONTENT,
        headers: { 'Content-Type': 'application/octet-stream' },
        sessionType: 'foreground',
      });
      status = res.status;
    } catch (e) {
      throw new FileTransferError('network', e instanceof Error ? e.message : 'Upload failed');
    }
    if (status < 200 || status >= 300) throw failureFor(status, 'The trip');
  },
  async download(url, name) {
    const dir = new Directory(Paths.document, 'shared');
    if (!dir.exists) dir.create({ intermediates: true, idempotent: true });
    try {
      const file = await File.downloadFileAsync(url, new File(dir, name), { idempotent: true });
      return file.uri;
    } catch (e) {
      const message = e instanceof Error ? e.message : 'Download failed';
      throw /\b404\b/.test(message) ? new FileTransferError('missing', 'That file is not on the server') : new FileTransferError('networ
k', message);
    }
  },
};

const webFiles: BlobFiles = {
  async exists(uri) {
    try {
      return (await fetch(uri)).ok;
    } catch {
      return false;
    }
  },
  async size(uri) {
    try {
      return (await (await fetch(uri)).blob()).size;
    } catch {
      return null;
    }
  },
  async upload(uri, url) {
    let res: Response;
    try {
      res = await fetch(url, { method: 'PUT', body: await (await fetch(uri)).blob() });
    } catch (e) {
      throw new FileTransferError('network', e instanceof Error ? e.message : 'Upload failed');
    }
    if (!res.ok) throw failureFor(res.status, 'The trip');
  },
  async download(url) {
    let res: Response;
    try {
      res = await fetch(url);
    } catch (e) {
      throw new FileTransferError('network', e instanceof Error ? e.message : 'Download failed');
    }
    if (!res.ok) throw failureFor(res.status, 'That file');
    return URL.createObjectURL(await res.blob());
  },
};

export const deviceFiles: BlobFiles = Platform.OS === 'web' ? webFiles : nativeFiles;

/** Deletes everything this phone downloaded from friends (used by "reset app data"). */
export function deleteSharedFiles(): void {
  if (Platform.OS === 'web') return;
  try {
    const dir = new Directory(Paths.document, 'shared');
    if (dir.exists) dir.delete();
  } catch {
    // nothing to delete
  }
}
```

src/sync/hlc.ts

```
/**
 * A hybrid logical clock. Every change gets a stamp that is unique and sortable as plain text:
 *   "<13-digit milliseconds>-<4-char counter>-<device id>"
 * Two phones with slightly different clocks still agree on the order of changes, because a phone that sees a
 * newer stamp from a friend moves its own clock forward before it makes the next change.
 */
export interface ClockState {
  ms: number;
  counter: number;
}
```

```
export class HybridClock {
  private last: ClockState = { ms: 0, counter: 0 };

  constructor(
    private readonly node: string,
    private readonly now: () => number = Date.now,
  ) {}

  /** A new stamp, always bigger than every stamp this clock has made or seen. */
  next(): string {
    const ms = Math.max(this.now(), this.last.ms);
    const counter = ms === this.last.ms ? this.last.counter + 1 : 0;
    this.last = { ms, counter };
    return format(ms, counter, this.node);
  }

  /** Call with every stamp received from another phone so our next stamp is newer. */
  observe(stamp: string): void {
    const parsed = parseStamp(stamp);
    if (!parsed) return;
    if (parsed.ms > this.last.ms || (parsed.ms === this.last.ms && parsed.counter > this.last.counter)) {
      this.last = { ms: parsed.ms, counter: parsed.counter };
    }
  }

  /** The latest stamp state, saved so the clock never goes backwards after a restart. */
  snapshot(): ClockState {
    return { ...this.last };
  }

  restore(state: ClockState | undefined): void {
    if (state && state.ms > this.last.ms) this.last = { ...state };
  }
}

function format(ms: number, counter: number, node: string): string {
  return `${String(ms).padStart(13, '0')}-${counter.toString(36).padStart(4, '0')}-${node}`;
}

export function parseStamp(stamp: string): { ms: number; counter: number; node: string } | null {
  const m = /^(\d{13})-([0-9a-z]{4})-([\w]+)$/.exec(stamp);
  return m ? { ms: Number(m[1]), counter: parseInt(m[2], 36), node: m[3] } : null;
}

/** Stamp comparison: -1, 0 or 1. Plain string order is the right order by design. */
export const compareStamps = (a: string, b: string) => (a < b ? -1 : a > b ? 1 : 0);

/** A short random device id (letters and numbers only). */
export function newNodeId(random: () => number = Math.random): string {
  let id = '';
  while (id.length < 8) id += Math.floor(random() * 36).toString(36);
  return id;
}
```

src/sync/invite.ts

```
/** The link and the message a friend gets when you invite them. Opening the link fills in the join screen. */
import { INVITE_RE, normalizeInvite } from '@/utils/validation';

import { normalizeServerUrl } from './transport';

/**
 * `base` is the app's own address for the join screen (togetherwego://join, or exp://…/--/join while developing).
 * The sync server is included when there is one, so a friend's phone does not need it typed in.
 */
export function buildInviteUrl(base: string, code: string, server?: string | null): string {
  const query = [`code=${encodeURIComponent(code)}`];
  if (server) query.push(`server=${encodeURIComponent(server)}`);
  return `${base}${base.includes('?') ? '&' : '?'}${query.join('&')}`;
}

const decode = (value: string) => {
  try {
    return decodeURIComponent(value.replace(/\+/g, ' '));
  } catch {
    return value;
  }
};

/** What an invite link or a scanned QR code says: a valid invite code and (if it names one) a sync server. */
export function parseInviteUrl(url: string): { code: string | null; server: string | null } {
  const query = url.trim().split('#')[0].split('?')[1] ?? '';
  const params = new Map<string, string>();
  for (const pair of query.split('&')) {
    const [key, value = ''] = pair.split('=');
```

```
    if (key) params.set(key, decode(value));
  }
  const code = normalizeInvite(params.get('code') ?? '');
  const server = normalizeServerUrl(params.get('server') ?? '');
  return { code: INVITE_RE.test(code) ? code : null, server };
}

/** A plain-words message for chat apps: the code, the link, and what to do if the link does not open the app. */
export function inviteText(tripName: string, code: string, link: string): string {
  return `Join my trip “${tripName}” on TogetherWeGo\nInvite code: ${code}\nOpen: ${link}\nOr open the app, tap Join a trip and type the
code.`;
}
```

src/sync/notify.ts

```
import type { Trip } from '@/store/types';
import { useAppStore } from '@/store/useAppStore';

const nameOf = (trip: Trip, id: string) => trip.members.find((m) => m.id === id)?.name.split(' ')[0] ?? 'Someone';

/**
 * Turns changes that arrived from friends into entries in the in-app notification list (and the unread dot on the
 * Chat tab): new chat messages, polls, expenses, photos and people joining.
 */
export function notifyRemoteChange(prev: Trip | undefined, next: Trip, me: string | undefined): void {
  if (!prev) return;
  const { pushNotification } = useAppStore.getState();
  const known = (list: { id: string }[]) => new Set(list.map((x) => x.id));

  const seenMessages = known(prev.messages);
  const messages = next.messages.filter((m) => !seenMessages.has(m.id) && m.authorId !== me && m.kind !== 'system' && !m.ai);
  if (messages.length === 1) {
    pushNotification({ kind: 'chat', title: `${nameOf(next, messages[0].authorId)} in ${next.name}`, body: messages[0].text, tripId: nex
t.id, route: '/(tabs)/chat' });
  } else if (messages.length > 1) {
    pushNotification({ kind: 'chat', title: `${messages.length} new messages in ${next.name}`, body: messages.at(-1)!.text, tripId: nex
t.id, route: '/(tabs)/chat' });
  }

  const seenPolls = known(prev.polls);
  for (const p of next.polls.filter((x) => !seenPolls.has(x.id) && x.createdBy !== me).slice(0, 2)) {
    pushNotification({ kind: 'vote', title: `New poll from ${nameOf(next, p.createdBy)}`, body: `${p.question} Cast your vote.`, tripId:
next.id, route: '/crew' });
  }

  const seenExpenses = known(prev.expenses);
  for (const e of next.expenses.filter((x) => !seenExpenses.has(x.id) && x.paidBy !== me && !x.settlement).slice(0, 2)) {
    pushNotification({ kind: 'expense', title: `${nameOf(next, e.paidBy)} added an expense`, body: `${e.title} · $${Math.round(e.amoun
t)} split ${e.splitBetween.length} ways`, tripId: next.id, route: '/(tabs)/budget' });
  }

  const seenPhotos = known(prev.photos);
  const photos = next.photos.filter((x) => !seenPhotos.has(x.id) && x.by !== me);
  if (photos.length) {
    const who = nameOf(next, photos[0].by);
    pushNotification({ kind: 'photo', title: photos.length === 1 ? `${who} added a photo` : `${photos.length} new photos in ${next.name}
`, body: photos[0].caption || 'Have a look in the photo dump', tripId: next.id, route: '/photos' });
  }

  const seenMembers = known(prev.members);
  for (const m of next.members.filter((x) => !seenMembers.has(x.id) && x.id !== me).slice(0, 2)) {
    pushNotification({ kind: 'member', title: `${m.name.split(' ')[0]} joined ${next.name}`, body: 'Say hi in the trip chat', tripId: ne
xt.id, route: '/(tabs)/chat' });
  }
}
```

src/sync/prefs.ts

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import Constants from 'expo-constants';
import { Platform } from 'react-native';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import { normalizeServerUrl } from './transport';

/** The port the sync server listens on (see server/index.js). */
export const SERVER_PORT = 8787;

interface SyncPrefs {
  /** What the user typed in Settings. Empty means "use the automatic guess". */
  serverUrl: string;
  setServerUrl: (url: string) => void;
}

export const useSyncPrefs = create<SyncPrefs>()(
```

```
  persist(
    (set) => ({
      serverUrl: '',
      setServerUrl: (serverUrl) => set({ serverUrl }),
    }),
    { name: 'togetherwego-sync-prefs', storage: createJSONStorage(() => AsyncStorage) },
  ),
);

const IPV4 = /^\d{1,3}(\.\d{1,3}){3}$/;

/**
 * While developing, the sync server runs on the same computer as Expo, so the phone can work out its address
 * from the Expo address it already used. Returns null when that is not possible (a published app, a tunnel...).
 */
export function guessServerUrl(): string | null {
  if (Platform.OS === 'web') {
    if (typeof window === 'undefined') return null;
    const { protocol, hostname } = window.location;
    return hostname === 'localhost' || IPV4.test(hostname) ? `${protocol}//${hostname}:${SERVER_PORT}` : null;
  }
  const host = Constants.expoConfig?.hostUri?.split(':')[0];
  return host && IPV4.test(host) ? `http://${host}:${SERVER_PORT}` : null;
}

/** The address the app will use: what the user typed, or else the automatic guess. */
export function effectiveServerUrl(): string | null {
  return normalizeServerUrl(useSyncPrefs.getState().serverUrl) ?? guessServerUrl();
}

/** The address to put in an invite. "localhost" only means something on the computer that runs the server, so it is left out. */
export function serverForInvites(): string | null {
  const url = effectiveServerUrl();
  return url && !/^https?:\/\/(localhost|127\.\d+\.\d+\.\d+)(:|$)/i.test(url) ? url : null;
}
```

src/sync/runtime.ts

```
/**
 * Starts the sync engine for the real app and wires it to the store, the saved state, the server address and
 * the app lifecycle (it pauses in the background and resumes when you come back).
 */
import { AppState } from 'react-native';

import { toast } from '@/components/ui/feedback';
import { useAppStore } from '@/store/useAppStore';

import { storeAdapter } from './adapter';
import { BlobSync } from './blobs';
import { blobCache, useFileStatus } from './blobCache';
import { SyncEngine } from './engine';
import { deviceFiles } from './fileio';
import { newNodeId } from './hlc';
import { notifyRemoteChange } from './notify';
import { effectiveServerUrl, useSyncPrefs } from './prefs';
import { secureRandom } from './secureRandom';
import { useSyncStatus } from './status';
import { asyncStorageMeta } from './storage';
import { createHttpTransport, type SyncTransport } from './transport';

let engine: SyncEngine | null = null;
let blobSync: BlobSync | null = null;
let ready: Promise<void> | null = null;
let cached: { url: string; transport: SyncTransport } | null = null;

function currentTransport(): SyncTransport | null {
  const url = effectiveServerUrl();
  if (!url) return null;
  if (cached?.url !== url) cached = { url, transport: createHttpTransport(url) };
  return cached.transport;
}

export function getEngine(): SyncEngine {
  engine ??= new SyncEngine({
    store: storeAdapter,
    storage: asyncStorageMeta,
    getTransport: currentTransport,
    random: secureRandom,
    onStatus: (status) => useSyncStatus.setState(status),
    onRemoteChange: (prev, next) => notifyRemoteChange(prev, next, useAppStore.getState().session?.userId),
  });
  return engine;
}

/** Sends your photos, documents and receipts to the server and fetches your friends' ones. */
export function getBlobSync(): BlobSync {
```

```
  blobSync ??= new BlobSync({
    trips: () => useAppStore.getState().trips,
    remoteId: (tripId) => getEngine().remoteId(tripId),
    baseUrl: effectiveServerUrl,
    files: deviceFiles,
    cache: blobCache,
    attach: (tripId, list, itemId, blobId) => useAppStore.getState().attachBlob(tripId, list, itemId, blobId),
    newId: () => newNodeId(secureRandom),
    onNotice: (message) => toast(message, { tone: 'warn', icon: 'alert-circle-outline' }),
    onPending: (pending) => useFileStatus.setState({ pending }),
  });
  return blobSync;
}

/** Call once when the app is ready. Safe to call again. */
export function startSync(): Promise<void> {
  ready ??= (async () => {
    const e = getEngine();
    await e.init();
    const files = getBlobSync();
    useAppStore.subscribe(() => files.schedule()); // a new photo here, or a friend's change that mentions a file
    useSyncPrefs.subscribe(() => files.schedule());
    useSyncStatus.subscribe((now, before) => {
      if (now.online && !before.online) files.schedule(0); // the connection came back
    });
    files.schedule(1000);
    if (AppState.currentState === 'active') e.start();
    AppState.addEventListener('change', (state) => {
      if (state === 'active') e.start();
      else if (state === 'background') e.stop();
    });
  })();
  return ready;
}

/** Waits for the engine to be loaded (used before sharing or joining). */
export async function syncReady(): Promise<SyncEngine> {
  await startSync();
  return getEngine();
}
```

src/sync/schema.ts

```
/**
 * What part of a trip is shared between phones, and how each part merges.
 *
 * A trip is treated as a tree of small "registers". Each register remembers the stamp of the change that last
 * wrote it, and the newest stamp always wins ("last writer wins"), so two phones that exchange their changes
 * end up identical no matter what order the changes arrive in.
 *
 *  - scalars : one value (title, date, amount...)
 *  - json    : one whole object that is replaced together (an expense's original currency amount)
 *  - sets    : lists of text values where only "is it in the list" matters (who is assigned, who votes)
 *  - maps    : name -> value (each member's vote on an activity)
 *  - lists   : lists of items with an id (activities, expenses, chat messages...), each an item with its own fields
 *
 * Left out on purpose: member emails (privacy), the private Scout AI chat, and the file paths of photos, documents,
 * receipts and tickets, because other phones cannot open a path into this phone. Photos, documents and receipts
 * instead share a `blob` id: the file itself travels through the server (see blobs.ts) and each phone keeps its
 * own copy. Flight tickets stay on the phone that added them (they hold personal details).
 */
export interface ItemSchema {
  /** Fields that hold one plain value. */
  scalars: readonly string[];
  /** Scalars that must be present for the item to count as complete. */
  required?: readonly string[];
  json?: readonly string[];
  sets?: readonly string[];
  maps?: readonly string[];
  lists?: Readonly<Record<string, ListSchema>>;
}

export interface ListSchema {
  item: ItemSchema;
  /** How the app keeps this list: 'asc' = newest last (push), 'desc' = newest first (unshift). */
  order: 'asc' | 'desc';
  /** Items this returns false for stay on this phone only. */
  share?: (item: any) => boolean;
}

/** Messages in the private Scout AI thread are never shared. */
export const isPrivateMessage = (m: { channel?: string }) => m.channel === 'ai';

const member: ItemSchema = { scalars: ['name', 'avatar', 'role', 'homeCity', 'invited'], required: ['name', 'role'] };

const activity: ItemSchema = {
```

```
  scalars: ['date', 'time', 'title', 'location', 'type', 'durationMin', 'costPerPerson', 'status', 'notes', 'reminder', 'lat', 'lng', 'p
laceId', 'createdBy', 'done'],
  required: ['date', 'time', 'title', 'type', 'status'],
  sets: ['assigned'],
  maps: ['votes'],
};

const expense: ItemSchema = {
  scalars: ['title', 'amount', 'category', 'paidBy', 'date', 'settlement', 'receiptBlob'],
  required: ['title', 'amount', 'paidBy'],
  json: ['original'],
  sets: ['splitBetween'],
};

const flight: ItemSchema = {
  scalars: ['memberId', 'airline', 'flightNo', 'from', 'to', 'departUtc', 'arriveUtc', 'direction', 'seat', 'confirmation'],
  required: ['memberId', 'flightNo', 'from', 'to', 'departUtc', 'arriveUtc', 'direction'],
};

const poll: ItemSchema = {
  scalars: ['question', 'createdBy', 'closed', 'createdAt'],
  required: ['question'],
  lists: { options: { item: { scalars: ['text'], required: ['text'], sets: ['votes'] }, order: 'asc' } },
};

const task: ItemSchema = { scalars: ['text', 'done', 'assignee'], required: ['text'] };
const doc: ItemSchema = { scalars: ['name', 'kind', 'note', 'size', 'addedBy', 'addedAt', 'offline', 'blob'], required: ['name', 'kind']
};
const message: ItemSchema = { scalars: ['authorId', 'text', 'ts', 'kind', 'ai', 'aiSource', 'channel'], required: ['authorId', 'text',
'ts'] };
const photo: ItemSchema = { scalars: ['caption', 'by', 'ts', 'day', 'blob'], required: ['by', 'ts'], sets: ['likes'] };
const packing: ItemSchema = { scalars: ['text', 'category', 'packed', 'broughtBy'], required: ['text'] };

export const TRIP_SCHEMA: ItemSchema = {
  scalars: ['id', 'name', 'destinationId', 'startDate', 'endDate', 'cover', 'budget', 'inviteCode', 'ownerId', 'completed', 'createdA
t'],
  required: ['id', 'name', 'destinationId', 'startDate', 'endDate', 'budget', 'inviteCode', 'ownerId'],
  sets: ['extraDestinationIds'],
  lists: {
    members: { item: member, order: 'asc' },
    activities: { item: activity, order: 'asc' },
    expenses: { item: expense, order: 'desc' },
    flights: { item: flight, order: 'asc' },
    polls: { item: poll, order: 'desc' },
    tasks: { item: task, order: 'asc' },
    docs: { item: doc, order: 'desc' },
    messages: { item: message, order: 'asc', share: (m) => !isPrivateMessage(m) },
    photos: { item: photo, order: 'desc' },
    packing: { item: packing, order: 'asc' },
  },
};

/** The register that records when an item was created (its stamp is also the item's place in the list). */
export const CREATED = '_c';
export const PATH_SEP = '\u0001';
export const pathKey = (p: readonly string[]) => p.join(PATH_SEP);
```

src/sync/secureRandom.ts

```
import * as Crypto from 'expo-crypto';

/**
 * A random number in [0, 1) from the phone's secure random generator. The ids that work like passwords (a trip's id on
 * the sync server, a file's id) come from this, not from Math.random, which is not meant to be unguessable.
 */
export function secureRandom(): number {
  const [a, b, c, d] = Crypto.getRandomBytes(4);
  return (((a << 24) | (b << 16) | (c << 8) | d) >>> 0) / 4294967296;
}
```

src/sync/status.ts

```
import { create } from 'zustand';

import type { SyncStatus } from './engine';

/** What the screens show about syncing. The engine writes to it; screens only read. */
export const useSyncStatus = create<SyncStatus>(() => ({ online: null, pending: 0, peers: 0, lastSyncAt: null, error: null, shared: []
}));
```

src/sync/storage.ts

```
import AsyncStorage from '@react-native-async-storage/async-storage';

import type { MetaStorage, SyncMeta } from './engine';
```

```
const KEY = 'togetherwego-sync-v1';

/** Keeps the sync engine's memory (what it has merged, what it has not sent yet) on the phone. */
export const asyncStorageMeta: MetaStorage = {
  async load() {
    try {
      const raw = await AsyncStorage.getItem(KEY);
      return raw ? (JSON.parse(raw) as SyncMeta) : null;
    } catch {
      return null;
    }
  },
  async save(meta) {
    await AsyncStorage.setItem(KEY, JSON.stringify(meta));
  },
};
```

src/sync/testing.ts

```
import { produce } from 'immer';

import type { Trip } from '@/store/types';

import { applyOps, diffTrip, materializeTrip, type Op, type Registers } from './crdt';
import { HybridClock } from './hlc';
import type { MetaStorage, StoreAdapter, SyncMeta } from './engine';
import { NetworkError, type SyncTransport } from './transport';
import { toViewTrip } from './view';

/** A tiny trip with a few of everything, for tests. */
export function makeTrip(): Trip {
  return {
    id: 'trip-1',
    name: 'Rome Crew',
    destinationId: 'rome',
    extraDestinationIds: [],
    startDate: '2026-09-10',
    endDate: '2026-09-16',
    cover: 'rome',
    budget: 3000,
    inviteCode: 'ROM-1234',
    ownerId: 'u-a',
    members: [
      { id: 'u-a', name: 'Ana', email: 'ana@example.com', role: 'Organizer' },
      { id: 'u-b', name: 'Ben', role: 'Member' },
    ],
    activities: [
      { id: 'a1', date: '2026-09-11', time: '10:00', title: 'Colosseum', location: 'Rome', type: 'sightseeing', durationMin: 120, costPe
rPerson: 20, status: 'voting', assigned: ['u-a'], votes: { 'u-a': 1 }, reminder: false, createdBy: 'u-a' },
    ],
    expenses: [{ id: 'e1', title: 'Hotel', amount: 300, category: 'Stays', paidBy: 'u-a', splitBetween: ['u-a', 'u-b'], date: '2026-09-1
0', receipt: 'file:///receipt.jpg' }],
    flights: [],
    polls: [{ id: 'p1', question: 'Beach?', createdBy: 'u-a', createdAt: 1, options: [{ id: 'o1', text: 'Ostia', votes: [] }, { id: 'o
2', text: 'Sperlonga', votes: ['u-b'] }] }],
    tasks: [{ id: 't1', text: 'Book tickets', done: false }],
    docs: [],
    messages: [
      { id: 'm1', authorId: 'u-a', text: 'Hi all', ts: 1000 },
      { id: 'ai1', authorId: 'assistant', text: 'private answer', ts: 1001, ai: true, channel: 'ai' },
    ],
    photos: [{ id: 'ph1', uri: 'file:///photo.jpg', caption: 'Pasta', by: 'u-a', likes: ['u-b'], ts: 5 }],
    packing: [],
    createdAt: 1,
  };
}

/** A fake clock shared by every phone in a test: each reading is one millisecond later. */
export function fakeNow(start = 1_800_000_000_000) {
  let t = start;
  return () => (t += 1);
}

/** One simulated phone: its own copy of the trip, its registers and its clock. */
export class Replica {
  regs: Registers = {};
  trip: Trip | undefined;
  clock: HybridClock;

  constructor(node: string, now: () => number, trip?: Trip) {
    this.clock = new HybridClock(node, now);
    this.trip = trip;
  }

  /** Makes a change the way a store action would, and returns the ops to send to the others. */
```

```
  edit(fn: (draft: Trip) => void): Op[] {
    const next = produce(this.trip!, fn);
    const ops = diffTrip(this.trip, next, () => this.clock.next());
    applyOps(this.regs, ops);
    this.trip = next;
    return ops;
  }

  /** Shares a trip for the first time: describes the whole trip as ops. */
  publish(): Op[] {
    const ops = diffTrip(undefined, this.trip!, () => this.clock.next());
    applyOps(this.regs, ops);
    return ops;
  }

  /** Receives ops from another phone. */
  receive(ops: readonly Op[]): void {
    ops.forEach((o) => this.clock.observe(o.t));
    if (applyOps(this.regs, ops)) {
      const shared = materializeTrip(this.regs);
      if (shared) this.trip = toViewTrip(shared, this.trip);
    }
  }
}

/** The trip without anything that stays on one phone, for comparing phones with each other. */
export function sharedPart(trip: Trip | undefined): unknown {
  if (!trip) return undefined;
  const t = JSON.parse(JSON.stringify(trip)) as Trip;
  t.messages = t.messages.filter((m) => m.channel !== 'ai');
  t.members.forEach((m) => delete m.email);
  t.photos.forEach((p) => delete (p as Partial<typeof p>).uri);
  t.docs.forEach((d) => delete d.uri);
  t.expenses.forEach((e) => delete e.receipt);
  t.flights.forEach((f) => {
    delete f.ticketUri;
    delete f.ticketName;
  });
  return t;
}

/** Small deterministic random numbers so a failing run can be repeated. */
export function seeded(seed: number) {
  let s = seed >>> 0;
  const next = () => {
    s = (Math.imul(s, 1664525) + 1013904223) >>> 0;
    return s / 4294967296;
  };
  return {
    next,
    int: (n: number) => Math.floor(next() * n),
    pick: <T,>(a: readonly T[]): T => a[Math.floor(next() * a.length)],
    shuffle: <T,>(a: readonly T[]): T[] => {
      const r = [...a];
      for (let i = r.length - 1; i > 0; i--) {
        const j = Math.floor(next() * (i + 1));
        [r[i], r[j]] = [r[j], r[i]];
      }
      return r;
    },
  };
}

// ---------------------------------------------------------------------- engine test doubles

/** An app store with just the trips: enough for the engine, and it notifies listeners like zustand does. */
export class FakeStore implements StoreAdapter {
  trips: Trip[] = [];
  userId: string | undefined = 'u-a';
  private listeners = new Set<() => void>();

  getTrip = (id: string) => this.trips.find((t) => t.id === id);
  getUserId = () => this.userId;
  setTrip = (trip: Trip) => {
    this.trips = this.trips.map((t) => (t.id === trip.id ? trip : t));
    this.emit();
  };
  addTrip = (trip: Trip) => {
    this.trips = [trip, ...this.trips.filter((t) => t.id !== trip.id)];
    this.emit();
  };
  removeTrip = (id: string) => {
    this.trips = this.trips.filter((t) => t.id !== id);
    this.emit();
  };
  subscribe = (l: () => void) => {
```

```
    this.listeners.add(l);
    return () => void this.listeners.delete(l);
  };
  private emit() {
    [...this.listeners].forEach((l) => l());
  }

  /** Changes a trip the way a store action would. */
  edit(tripId: string, fn: (d: Trip) => void) {
    this.trips = this.trips.map((t) => (t.id === tripId ? produce(t, fn) : t));
    this.emit();
  }
}

export class MemoryStorage implements MetaStorage {
  saved: string | null = null;
  async load(): Promise<SyncMeta | null> {
    return this.saved ? (JSON.parse(this.saved) as SyncMeta) : null;
  }
  async save(meta: SyncMeta): Promise<void> {
    this.saved = JSON.stringify(meta);
  }
}

/** Connects an engine straight to a server store in memory, and can be switched "offline". */
export function memoryTransport(server: {
  append: (...args: any[]) => { seq: number };
  since: (...args: any[]) => { ops: unknown[]; seq: number; total: number; peers: number };
  lookup: (code: string) => string | null;
}): SyncTransport & { offline: boolean; calls: { push: number; pull: number } } {
  const t = {
    offline: false,
    calls: { push: 0, pull: 0 },
    async lookup(code: string) {
      if (t.offline) throw new NetworkError('offline');
      return server.lookup(code);
    },
    async push(tripId: string, code: string | undefined, client: string, ops: Op[]) {
      if (t.offline) throw new NetworkError('offline');
      t.calls.push++;
      return server.append(tripId, code, client, JSON.parse(JSON.stringify(ops)));
    },
    async pull(tripId: string, since: number, client: string) {
      if (t.offline) throw new NetworkError('offline');
      t.calls.pull++;
      return server.since(tripId, since, client) as Awaited<ReturnType<SyncTransport['pull']>>;
    },
  };
  return t;
}
```

src/sync/timers.ts

```
/** In Node (the tests) a waiting timer would keep the program alive; in the app, timers have no such method. */
export function quiet<T>(timer: T): T {
  (timer as { unref?: () => void }).unref?.();
  return timer;
}
```

src/sync/transport.ts

```
/** Talking to the sync server over plain HTTP (see server/index.js). */
import type { Op } from './crdt';

export interface PullResult {
  ops: Op[];
  /** Position to ask from next time. */
  seq: number;
  /** How many changes the server holds in total (more than `seq` means there is another page). */
  total: number;
  /** How many phones talked to the server about this trip in the last minute. */
  peers: number;
}

export interface SyncTransport {
  /** Invite code to the trip's server id, or null when no trip has that code. */
  lookup(code: string): Promise<string | null>;
  push(tripId: string, code: string | undefined, client: string, ops: Op[]): Promise<{ seq: number }>;
  /** Fetches changes after `since`. With `waitMs` the server holds the request until something new arrives. */
  pull(tripId: string, since: number, client: string, waitMs: number, signal?: AbortSignal): Promise<PullResult>;
}

/** The server could not be reached (wrong address, wifi off, server not running). */
export class NetworkError extends Error {}
/** The server answered with an error. */
export class ServerError extends Error {
```

```
  constructor(
    readonly status: number,
    message: string,
  ) {
    super(message);
  }
}

const HOST = /^[A-Za-z0-9]([A-Za-z0-9.-]*[A-Za-z0-9])?$/;

/**
 * "192.168.1.5:8787" or "http://192.168.1.5:8787/" -> "http://192.168.1.5:8787". Returns null if it cannot be an address.
 * The phone's built-in URL class is much looser than a browser's (it accepts spaces in a host name), so the host is checked here too.
 */
export function normalizeServerUrl(input: string): string | null {
  let url = input.trim();
  if (!url) return null;
  url = /^[a-z][a-z0-9+.-]*:\/\//i.test(url) ? url.replace(/^[a-z][a-z0-9+.-]*:\/\//i, (scheme) => scheme.toLowerCase()) : `http://${ur
l}`;
  try {
    const parsed = new URL(url);
    if (!/^https?:$/i.test(parsed.protocol) || !HOST.test(parsed.hostname)) return null;
    if (!/^[A-Za-z0-9.-]+(:\d{1,5})?$/.test(parsed.host)) return null;
    return `${parsed.protocol.toLowerCase()}//${parsed.host.toLowerCase()}`;
  } catch {
    return null;
  }
}

export interface HttpTransportOptions {
  /** How long a request may take before we give up on the server (ms). Waiting requests get extra time on top. */
  timeoutMs?: number;
  /** Looking up an invite code is quick on a home network, so it gives up sooner (a server that is not there should not stall "Join").
*/
  lookupTimeoutMs?: number;
}

export function createHttpTransport(serverUrl: string, fetchImpl: typeof fetch = fetch, options: HttpTransportOptions = {}): SyncTranspo
rt {
  const base = serverUrl.replace(/\/+$/, '');
  const timeoutMs = options.timeoutMs ?? 8000;
  const lookupTimeoutMs = options.lookupTimeoutMs ?? 3000;

  /** One request with a time limit, so a wifi that silently drops packets cannot freeze "Join" for a minute. */
  async function request<T>(path: string, init?: RequestInit & { signal?: AbortSignal }, extraMs = 0, limitMs = timeoutMs): Promise<{ st
atus: number; data: T }> {
    const limit = new AbortController();
    const outer = init?.signal;
    const cancel = () => limit.abort();
    outer?.addEventListener('abort', cancel);
    const timer = setTimeout(cancel, limitMs + extraMs);
    try {
      let res: Response;
      try {
        res = await fetchImpl(base + path, { ...init, signal: limit.signal });
      } catch (e) {
        if (outer?.aborted) throw e; // the app cancelled this request on purpose
        throw new NetworkError(limit.signal.aborted ? 'The sync server took too long to answer' : e instanceof Error ? e.message : 'Netw
ork request failed');
      }
      let data: unknown = null;
      try {
        data = await res.json();
      } catch {
        // an empty or non-JSON body is handled below
      }
      return { status: res.status, data: data as T };
    } finally {
      clearTimeout(timer);
      outer?.removeEventListener('abort', cancel);
    }
  }

  const fail = (status: number, data: unknown): never => {
    const message = (data as { error?: string } | null)?.error ?? `Server answered ${status}`;
    throw new ServerError(status, message);
  };

  return {
    async lookup(code) {
      const { status, data } = await request<{ tripId: string }>(`/v1/codes/${encodeURIComponent(code)}`, undefined, 0, lookupTimeoutM
s);
      if (status === 404) return null;
      if (status !== 200) fail(status, data);
      return data.tripId;
    },
```

```
    async push(tripId, code, client, ops) {
      const { status, data } = await request<{ seq: number }>(`/v1/trips/${tripId}/ops`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ code, client, ops }),
      });
      if (status !== 200) fail(status, data);
      return data;
    },
    async pull(tripId, since, client, waitMs, signal) {
      const { status, data } = await request<PullResult>(`/v1/trips/${tripId}/ops?since=${since}&wait=${waitMs}&client=${client}`, { sig
nal }, waitMs);
      if (status !== 200) fail(status, data);
      return data;
    },
  };
}
```

src/sync/useFile.ts

```
import { useBlobCache } from './blobCache';
import { resolveFile } from './blobs';

export interface SharedFile {
  /** Where to open the file from, when this phone has it. */
  uri: string | undefined;
  /** True while a copy from a friend is on its way. */
  loading: boolean;
}

/**
 * The file to show for a photo, document or receipt: this phone's own file if it has one, otherwise the copy
 * that was downloaded from a friend through the sync server.
 */
export function useSharedFile(local: string | undefined, blob: string | undefined): SharedFile {
  const downloaded = useBlobCache((s) => (blob ? s.files[blob] : undefined));
  return resolveFile(local, blob, downloaded);
}
```

src/sync/view.ts

```
/**
 * Puts a shared trip together with the parts that only live on this phone, so screens see one ordinary Trip.
 *
 * Phone-only parts: the private Scout AI chat, member emails, and the files behind photos, documents, receipts and
 * tickets (other phones cannot open a path that points into this phone's storage).
 */
import type { Trip } from '@/store/types';

import { deepEqual } from './crdt';


/** Fields that stay on this phone, per list, and what other phones show when they do not have them. */
const LOCAL_FIELDS: Record<string, { fields: string[]; fallback?: Record<string, unknown> }> = {
  members: { fields: ['email'] },
  photos: { fields: ['uri'], fallback: { uri: '' } },
  docs: { fields: ['uri'] },
  expenses: { fields: ['receipt'] },
  flights: { fields: ['ticketUri', 'ticketName'] },
};

export function mergeLocalOnly(shared: Trip, current: Trip | undefined): Trip {
  const out: any = { ...shared };
  for (const [list, { fields, fallback }] of Object.entries(LOCAL_FIELDS)) {
    const mine = new Map<string, any>(((current as any)?.[list] ?? []).map((x: any) => [x.id, x]));
    out[list] = (shared as any)[list].map((item: any) => {
      const own = mine.get(item.id);
      let next = item;
      for (const f of fields) {
        const value = own?.[f] ?? fallback?.[f];
        if (value !== undefined && next[f] !== value) next = { ...next, [f]: value };
      }
      return next;
    });
  }
  const privateMessages = (current?.messages ?? []).filter((m) => m.channel === 'ai').sort((a, b) => a.ts - b.ts);
  if (privateMessages.length) out.messages = placePrivate(shared.messages, privateMessages);
  return out as Trip;
}

/** Slots the private messages in by time without reordering the shared ones (their order is the order they were sent). */
function placePrivate(shared: Trip['messages'], mine: Trip['messages']): Trip['messages'] {
  const out = [...shared];
  for (const m of mine) {
    let i = out.length;
    while (i > 0 && out[i - 1].ts > m.ts) i--;
```

```
    out.splice(i, 0, m);
  }
  return out;
}

/**
 * Keeps the old object for every part that did not change, so React only re-renders what is new.
 * Returns `prev` itself when nothing changed at all.
 */
export function reuseUnchanged(prev: Trip | undefined, next: Trip): Trip {
  if (!prev) return next;
  const out: any = { ...next };
  let same = true;
  for (const key of Object.keys(next) as (keyof Trip)[]) {
    const a: any = prev[key];
    const b: any = next[key];
    if (Array.isArray(b) && Array.isArray(a) && b.length && typeof b[0] === 'object') {
      const old = new Map<string, any>(a.map((x: any) => [x.id, x]));
      const items = b.map((item: any) => {
        const o = old.get(item.id);
        return o && deepEqual(o, item) ? o : item;
      });
      out[key] = items.length === a.length && items.every((x: any, i: number) => x === a[i]) ? a : items;
    } else if (deepEqual(a, b)) {
      out[key] = a;
    }
    if (out[key] !== a) same = false;
  }
  return same ? prev : (out as Trip);
}

/** What the store should hold for a shared trip after the registers changed. */
export function toViewTrip(shared: Trip, current: Trip | undefined): Trip {
  return reuseUnchanged(current, mergeLocalOnly(shared, current));
}
```

src/theme/__tests__/palettes.test.ts

```
import { contrast } from '@/utils/contrast';

import { LEGACY_THEMES, THEME_KEYS, buildPalette, resolveMode, themes, type Colors, type Mode } from '../palettes';

const MODES: Mode[] = ['light', 'dark'];
const AA = 4.5;
const combos = THEME_KEYS.flatMap((key) => MODES.map((mode) => ({ key, mode, c: buildPalette(key, mode) })));
const name = (x: { key: string; mode: string }) => `${x.key} / ${x.mode}`;

describe('theme colours', () => {
  it('has six themes, each with a name and a vibe', () => {
    expect(THEME_KEYS).toHaveLength(6);
    for (const k of THEME_KEYS) expect(themes[k].name && themes[k].vibe).toBeTruthy();
  });

  it.each(combos)('body text reads at 4.5:1 on every surface: $key / $mode', ({ c }) => {
    for (const surface of [c.bg, c.card, c.cardMuted, c.raised, c.bgAlt] as const) {
      expect(contrast(c.text, surface)).toBeGreaterThanOrEqual(AA);
      expect(contrast(c.textSecondary, surface)).toBeGreaterThanOrEqual(AA);
      expect(contrast(c.textMuted, surface)).toBeGreaterThanOrEqual(AA);
    }
  });

  it.each(combos)('the accent is readable as text and carries readable text: $key / $mode', ({ c }) => {
    for (const surface of [c.bg, c.card, c.raised] as const) expect(contrast(c.accentText, surface)).toBeGreaterThanOrEqual(AA);
    expect(contrast(c.onAccent, c.accent)).toBeGreaterThanOrEqual(AA);
    expect(contrast(c.onPrimary, c.primary)).toBeGreaterThanOrEqual(AA);
  });

  it.each(combos)('state colours read on their own tint and on the page: $key / $mode', ({ c }) => {
    const pairs: [keyof Colors, keyof Colors][] = [
      ['positive', 'positiveSoft'],
      ['warn', 'warnSoft'],
      ['red', 'redSoft'],
      ['orange', 'orangeSoft'],
      ['blue', 'blueSoft'],
      ['purple', 'purpleSoft'],
      ['teal', 'tealSoft'],
    ];
    for (const [fg, bg] of pairs) {
      expect(contrast(c[fg], c[bg])).toBeGreaterThanOrEqual(AA);
      expect(contrast(c[fg], c.bg)).toBeGreaterThanOrEqual(AA);
      expect(contrast(c[fg], c.card)).toBeGreaterThanOrEqual(AA);
    }
  });

  it('the dark mode lifts surfaces with lightness: page < card < raised, on #121212', () => {
    for (const k of THEME_KEYS) {
```

```
      const c = buildPalette(k, 'dark');
      expect(c.bg).toBe('#121212');
      const L = (hex: string) => parseInt(hex.slice(1, 3), 16);
      expect(L(c.bg)).toBeLessThan(L(c.card));
      expect(L(c.card)).toBeLessThan(L(c.raised));
    }
  });

  it('is printed in ink: text, outlines and main buttons are the same ink colour', () => {
    for (const x of combos) {
      expect(x.c.ink).toBe(x.c.text);
      expect(x.c.primary).toBe(x.c.ink);
      expect(contrast(x.c.ink, x.c.card)).toBeGreaterThanOrEqual(7);
    }
  });

  it('has no gradient wash: the page is one flat colour', () => {
    for (const x of combos) expect(x.c.washFrom === x.c.bg && x.c.washTo === x.c.bg).toBe(true);
  });

  it('puts the hard shadow plate in ink on paper and in the accent at night', () => {
    for (const k of THEME_KEYS) {
      expect(buildPalette(k, 'light').plate).toBe(buildPalette(k, 'light').ink);
      expect(buildPalette(k, 'dark').plate).toBe(buildPalette(k, 'dark').accent);
    }
  });

  it('desaturates the accent for the dark mode (the dark accent is lighter, never brighter-saturated)', () => {
    for (const k of THEME_KEYS) {
      if (k === 'classic') continue;
      const sat = (hex: string) => {
        const [r, g, b] = [1, 3, 5].map((i) => parseInt(hex.slice(i, i + 2), 16) / 255);
        const max = Math.max(r, g, b);
        const min = Math.min(r, g, b);
        return max === 0 ? 0 : (max - min) / max;
      };
      expect(sat(themes[k].dark.accent)).toBeLessThanOrEqual(sat(themes[k].light.accent) + 0.001);
    }
  });

  it('follows the phone only when asked to', () => {
    expect(resolveMode('system', true)).toBe('dark');
    expect(resolveMode('system', false)).toBe('light');
    expect(resolveMode('light', true)).toBe('light');
    expect(resolveMode('dark', false)).toBe('dark');
  });

  it('maps every look saved by earlier versions to a theme that exists', () => {
    for (const legacy of Object.values(LEGACY_THEMES)) expect(THEME_KEYS).toContain(legacy.theme);
  });

  it('names are unique, so the picker has no twins', () => {
    expect(new Set(THEME_KEYS.map((k) => themes[k].name)).size).toBe(THEME_KEYS.length);
    expect(combos.map(name)).toHaveLength(THEME_KEYS.length * 2);
  });
});
```

src/theme/index.ts

```
/**
 * TogetherWeGo design tokens.
 *
 * The look is a boarding pass printed on paper: a warm paper page, ink outlines on everything, a serif for
 * headlines, mono for the small labels and numbers, ONE flat accent per theme used like a highlighter, and a
 * hard offset "plate" under things you can press instead of a soft shadow. The colours themselves live in
 * ./palettes (pure data, unit-tested for contrast).
 *
 * The person picks a theme (accent) and a mode (light, dark, or follow the phone) in "Make it yours" or
 * Settings. Styles are created once when each module loads, so the saved choice is read synchronously
 * here, before any screen module evaluates, and the token objects below are final. Changing the look
 * then reloads the app (see applyAppearance).
 */
import { File, Paths } from 'expo-file-system';
import { Appearance as OsAppearance, DevSettings, Platform } from 'react-native';

import { LEGACY_THEMES, THEME_KEYS, buildPalette, categoryPalette, resolveMode, themes, type Colors, type Mode, type ModePref, type Them
eKey } from './palettes';

export { THEME_KEYS, buildPalette, resolveMode, themes };
export type { Colors, Mode, ModePref, ThemeKey };

export type CornerStyle = 'rounded' | 'bubbly' | 'sharp';

export const cornerStyles: Record<CornerStyle, { name: string; scale: number }> = {
  rounded: { name: 'Rounded', scale: 1 },
  bubbly: { name: 'Bubbly', scale: 1.4 },
```

```
  sharp: { name: 'Crisp', scale: 0.45 },
};

export interface Appearance {
  theme: ThemeKey;
  corners: CornerStyle;
  /** Light, dark, or follow the phone each time the app opens. */
  mode: ModePref;
}

const PREFS_KEY = 'togetherwego-appearance';
const DEFAULT_APPEARANCE: Appearance = { theme: 'amalfi', corners: 'rounded', mode: 'system' };

function prefsFile() {
  return new File(Paths.document, 'appearance.json');
}

const isMode = (m: unknown): m is ModePref => m === 'system' || m === 'light' || m === 'dark';

/** Turns whatever was saved (including looks from earlier versions of the app) into a valid look. */
export function parseAppearance(raw: string | null): Appearance {
  try {
    const parsed = raw ? (JSON.parse(raw) as { theme?: string; corners?: string; mode?: unknown }) : {};
    let theme: ThemeKey = DEFAULT_APPEARANCE.theme;
    let mode: ModePref = isMode(parsed.mode) ? parsed.mode : DEFAULT_APPEARANCE.mode;
    if (parsed.theme && (THEME_KEYS as string[]).includes(parsed.theme)) {
      theme = parsed.theme as ThemeKey;
      // the old Midnight was a dark theme, before light and dark became a separate choice
      if (parsed.theme === 'midnight' && !isMode(parsed.mode)) mode = 'dark';
    } else if (parsed.theme && parsed.theme in LEGACY_THEMES) {
      const legacy = LEGACY_THEMES[parsed.theme];
      theme = legacy.theme;
      if (!isMode(parsed.mode) && legacy.mode) mode = legacy.mode;
    }
    return { theme, corners: parsed.corners && parsed.corners in cornerStyles ? (parsed.corners as CornerStyle) : DEFAULT_APPEARANCE.cor
ners, mode };
  } catch {
    return DEFAULT_APPEARANCE;
  }
}

function readAppearance(): Appearance {
  try {
    let raw: string | null = null;
    if (Platform.OS === 'web') raw = typeof localStorage !== 'undefined' ? localStorage.getItem(PREFS_KEY) : null;
    else {
      const f = prefsFile();
      raw = f.exists ? f.textSync() : null;
    }
    return parseAppearance(raw);
  } catch {
    return DEFAULT_APPEARANCE;
  }
}

function writeAppearance(a: Appearance) {
  try {
    const raw = JSON.stringify(a);
    if (Platform.OS === 'web') localStorage.setItem(PREFS_KEY, raw);
    else prefsFile().write(raw);
  } catch {
    // Storage unavailable (private browsing etc.) — the choice lasts for this session only.
  }
}

const systemIsDark = () => {
  try {
    return OsAppearance.getColorScheme() === 'dark';
  } catch {
    return false;
  }
};

/** The look the app is currently drawn with (fixed until the next reload). */
export const appearance: Appearance = readAppearance();
/** `appearance.mode` with "follow the phone" worked out. */
export const resolvedMode: Mode = resolveMode(appearance.mode, systemIsDark());
export const isDark = resolvedMode === 'dark';

/** Every colour, for the chosen theme and mode. */
export const colors: Colors = buildPalette(appearance.theme, resolvedMode);

/** The colours of any theme and mode (for previews of a look that is not applied yet). */
export const paletteOf = (theme: ThemeKey, mode: ModePref): Colors => buildPalette(theme, resolveMode(mode, systemIsDark()));

/** Restarts the UI so every screen is drawn again with the current look. */
```

```
export function reloadForLook(): boolean {
  try {
    if (Platform.OS === 'web') {
      const home = (globalThis as { __TWG_HOME?: string }).__TWG_HOME;
      // Restart from the app's front door (not the page we're on).
      window.location.replace(home ?? `${window.location.origin}/`);
    } else DevSettings.reload('Applying your new look');
    return true;
  } catch {
    return false;
  }
}

/** Saves a new look and restarts the UI so every screen picks it up. Returns false if the restart did not work. */
export function applyAppearance(next: Appearance): boolean {
  writeAppearance(next);
  const same = next.theme === appearance.theme && next.corners === appearance.corners && resolveMode(next.mode, systemIsDark()) === reso
lvedMode;
  return same ? true : reloadForLook();
}

/** Category colours used by the spending breakdown and expense dots (data colours, so they stay distinct). */
export const categoryColors: Record<string, string> = categoryPalette(resolvedMode);

const cornerScale = cornerStyles[appearance.corners].scale;
const r = (n: number) => Math.round(n * cornerScale);
/**
 * Corner radii: 10 for buttons and inputs, 12 for cards and images, 16 for sheets and dialogs, 8 for chips and
 * tags (small rectangles, not pills). The older names map onto them.
 */
export const radius = {
  button: r(10),
  card: r(12),
  modal: r(16),
  /** Chips and tags are small rectangles, not pills: a pill shape is only for avatars and dots. */
  chip: r(8),
  xs: r(6),
  sm: r(8),
  md: r(10),
  lg: r(12),
  xl: r(14),
  pill: 999,
};

/** Outline widths: `line` is the ink outline on cards, buttons and inputs; `hair` divides things inside a card. */
export const line = 2;
export const hair = 1;
/** How far the hard plate under a pressable thing sticks out (right and down). */
export const plateOffset = 4;

/** A 4-point spacing scale. `section` is the gap between blocks on a page, bigger than any gap inside one (law of proximity). */
export const space = {
  xxs: 2,
  xs: 4,
  sm: 8,
  md: 12,
  lg: 16,
  xl: 20,
  xxl: 24,
  xxxl: 32,
  gutter: 20,
  section: 32,
} as const;

/** Smallest size of anything you tap, in points (Apple's guideline; Fitts's law). */
export const touch = { min: 44 } as const;

/** How long things take (ms): small elements 150 to 250, sheets 300 to 400. */
export const timing = { fast: 160, base: 240, sheet: 340, slow: 480, stagger: 50 } as const;

export const fonts = {
  regular: 'SpaceGrotesk_400Regular',
  medium: 'SpaceGrotesk_500Medium',
  semibold: 'SpaceGrotesk_600SemiBold',
  bold: 'SpaceGrotesk_700Bold',
  extrabold: 'SpaceGrotesk_700Bold',
  /** Heavy soft serif for page titles: the masthead of a travel magazine. */
  display: 'Fraunces_900Black',
  /** Serif for section headlines. */
  serif: 'Fraunces_700Bold',
  /** Mono for the small caps labels, times, codes and every number that matters: a ticket's print. */
  mono: 'SpaceMono_700Bold',
  monoRegular: 'SpaceMono_400Regular',
} as const;

/**
```

```
 * There are no soft shadows in this look (a blurry drop shadow is the first thing that makes an app look generated).
 * These stay as empty styles so older screens compile; depth comes from the hard plate (see ui/Plate).
 */
export const shadow = {} as object;
export const shadowStrong = {} as object;

/** Width of the app column on tablets / web so layouts stay phone-shaped. */
export const MAX_WIDTH = 520;
```

src/theme/palettes.ts

```
/**
 * Colour data for the app, with no React Native in it so tests can check it.
 *
 * The look is "a boarding pass printed on paper": a warm paper page, black ink for text and every outline,
 * and ONE flat accent colour per theme used like a highlighter or a sticker. There are no gradients and no
 * soft shadows; depth comes from a hard offset "plate" under the things you can press. The dark mode is a
 * night flight: #121212, cream ink, and the accent as the plate. Green, amber and red mean a state
 * (under budget, near, over; going, maybe, can't go) and are never used to decorate.
 *
 * A theme (accent) and a mode (light, dark, or follow the phone) are chosen separately.
 */
import { ensureContrast, mix, readableOn } from '@/utils/contrast';

export type ThemeKey = 'amalfi' | 'sunset' | 'lagoon' | 'rose' | 'midnight' | 'classic';
export type Mode = 'light' | 'dark';
export type ModePref = 'system' | Mode;

export interface Colors {
  /** The page: paper (light) or #121212 (dark). */
  bg: string;
  /** Wells, inputs and chips: one step away from the page. */
  bgAlt: string;
  /** Cards and tickets. */
  card: string;
  cardMuted: string;
  /** Sheets and dialogs. */
  raised: string;
  /** Hairline dividers inside a card. */
  border: string;
  borderStrong: string;
  /** The ink: text colour and the colour of every outline. */
  ink: string;
  /** The hard shadow under things you can press: ink in the light mode, the accent in the dark mode. */
  plate: string;

  /** Every main button: ink (cream in the dark mode). */
  primary: string;
  primaryDark: string;
  primaryPressed: string;
  primarySoft: string;
  primarySofter: string;
  primaryLine: string;
  navActive: string;
  onPrimary: string;

  /** The theme's one flat colour, for fills (the highlighter). */
  accent: string;
  accentSoft: string;
  /** The accent made dark (or light) enough to read as text. */
  accentText: string;
  /** Text on top of an accent fill. */
  onAccent: string;
  /** Kept so older screens compile; the new look has no gradient wash, so both equal the page colour. */
  washFrom: string;
  washTo: string;
  heroA: string;
  heroB: string;
  heroC: string;

  text: string;
  textSecondary: string;
  textMuted: string;

  /** States only: under budget / going / done. */
  positive: string;
  positiveSoft: string;
  /** States only: near the budget / maybe. */
  warn: string;
  warnSoft: string;
  /** States only: over budget / can't go / delete. */
  red: string;
  redSoft: string;

  orange: string;
  orangeSoft: string;
```

```
  blue: string;
  blueSoft: string;
  yellow: string;
  yellowSoft: string;
  purple: string;
  purpleSoft: string;
  teal: string;
  tealSoft: string;
  star: string;

  overlay: string;
  white: string;
  black: string;
}

interface ThemeDef {
  name: string;
  /** Unused by the new look (no emoji); kept so saved data and older code stay valid. */
  emoji: string;
  vibe: string;
  /** The mode the picker switches to along with this theme (the person can still change it). */
  mode?: Mode;
  light: { accent: string };
  dark: { accent: string };
}

export const themes: Record<ThemeKey, ThemeDef> = {
  amalfi: { name: 'Limoncello', emoji: '', vibe: 'Lemon on paper', light: { accent: '#FFD23F' }, dark: { accent: '#F6CF4D' } },
  sunset: { name: 'Tomato', emoji: '', vibe: 'Hot tomato red', light: { accent: '#FF5B37' }, dark: { accent: '#FF8461' } },
  lagoon: { name: 'Lagoon', emoji: '', vibe: 'Sea-glass green', light: { accent: '#25CDB8' }, dark: { accent: '#4FD8C8' } },
  rose: { name: 'Bubblegum', emoji: '', vibe: 'Loud pink', light: { accent: '#FF8DB8' }, dark: { accent: '#FF9EC4' } },
  midnight: { name: 'Grape', emoji: '', vibe: 'Violet, made for night', mode: 'dark', light: { accent: '#A394FF' }, dark: { accent: '#B7
A9FF' } },
  classic: { name: 'Ink', emoji: '', vibe: 'Only black and paper', light: { accent: '#16130E' }, dark: { accent: '#F1EADC' } },
};

/** The order the colour swatches are shown in. */
export const THEME_KEYS = Object.keys(themes) as ThemeKey[];

/** Saved looks from earlier versions of the app. */
export const LEGACY_THEMES: Record<string, { theme: ThemeKey; mode?: Mode }> = {
  forest: { theme: 'lagoon' },
  ocean: { theme: 'lagoon' },
  lavender: { theme: 'midnight', mode: 'light' },
  sakura: { theme: 'rose' },
};

const LIGHT = {
  bg: '#F4EFE4',
  bgAlt: '#ECE5D5',
  card: '#FFFDF8',
  cardMuted: '#F8F3E8',
  raised: '#FFFDF8',
  border: '#D6CCB6',
  borderStrong: '#B5AA90',
  ink: '#16130E',
  primary: '#16130E',
  primaryDark: '#000000',
  primaryPressed: '#2E2A22',
  primarySoft: '#E9E1CF',
  primarySofter: '#F1EBDC',
  primaryLine: '#D6CCB6',
  onPrimary: '#FFFDF8',
  text: '#16130E',
  textSecondary: '#4D463A',
  textMuted: '#6B6353',
  positive: '#16663A',
  positiveSoft: '#D9EED9',
  warn: '#7F4B00',
  warnSoft: '#FBE6B3',
  red: '#B3261E',
  redSoft: '#F8D8D2',
  orange: '#A0440A',
  orangeSoft: '#F6DCC5',
  blue: '#1B4FB0',
  blueSoft: '#D9E3F6',
  yellow: '#F2B33D',
  yellowSoft: '#FBEFC8',
  purple: '#5B3FC2',
  purpleSoft: '#E3DDF6',
  teal: '#0F6B62',
  tealSoft: '#D1ECE7',
  star: '#F5B301',
  overlay: 'rgba(22, 19, 14, 0.55)',
};
```

```
const DARK = {
  bg: '#121212',
  bgAlt: '#191816',
  card: '#1E1C19',
  cardMuted: '#232019',
  raised: '#2A2722',
  border: '#38342D',
  borderStrong: '#57514A',
  ink: '#F1EADC',
  primary: '#F1EADC',
  primaryDark: '#FFFFFF',
  primaryPressed: '#D5CDBC',
  primarySoft: '#2C2923',
  primarySofter: '#25221D',
  primaryLine: '#3E3A32',
  onPrimary: '#16130E',
  text: '#F1EADC',
  textSecondary: '#C4BCAA',
  textMuted: '#A39B89',
  positive: '#6EDC99',
  positiveSoft: '#163020',
  warn: '#FFC857',
  warnSoft: '#35290D',
  red: '#FF8F86',
  redSoft: '#3E1C19',
  orange: '#FFA062',
  orangeSoft: '#3B2515',
  blue: '#8DB8FF',
  blueSoft: '#1C2A45',
  yellow: '#FFD66B',
  yellowSoft: '#35290D',
  purple: '#B7A9FF',
  purpleSoft: '#2B2448',
  teal: '#5FD6C8',
  tealSoft: '#123430',
  star: '#FFC83D',
  overlay: 'rgba(0, 0, 0, 0.68)',
};

/** Every colour of one theme in one mode. */
export function buildPalette(key: ThemeKey, mode: Mode): Colors {
  const dark = mode === 'dark';
  const base = dark ? DARK : LIGHT;
  const accent = themes[key][dark ? 'dark' : 'light'].accent;
  const surfaces = [base.bg, base.card, base.cardMuted, base.raised, base.bgAlt];
  return {
    ...base,
    plate: dark ? accent : base.ink,
    navActive: accent,
    accent,
    accentSoft: mix(accent, base.bg, dark ? 0.78 : 0.7),
    accentText: ensureContrast(accent, surfaces, 4.5, dark ? '#FFFFFF' : '#000000'),
    onAccent: readableOn(accent, '#16130E', '#FFFDF8'),
    washFrom: base.bg,
    washTo: base.bg,
    heroA: base.ink,
    heroB: base.ink,
    heroC: accent,
    white: '#FFFFFF',
    black: '#000000',
  };
}

export function resolveMode(pref: ModePref, systemIsDark: boolean): Mode {
  return pref === 'system' ? (systemIsDark ? 'dark' : 'light') : pref;
}

/** Colours for the spending breakdown (data colours, so they stay distinct). */
export function categoryPalette(mode: Mode): Record<string, string> {
  return mode === 'dark'
    ? { Flights: '#B7A9FF', Stays: '#F1EADC', Food: '#F6CF4D', Activities: '#FFA062', Transport: '#8DB8FF', Shopping: '#FF9EC4', Other:
'#8E8776' }
    : { Flights: '#5B3FC2', Stays: '#16130E', Food: '#FFD23F', Activities: '#E8833A', Transport: '#1B4FB0', Shopping: '#E0568C', Other:
'#8E8776' };
}
```

src/utils/__tests__/guess.test.ts

```
import { guessExpenseCategory } from '../guess';

describe('guessExpenseCategory', () => {
  it.each([
    ['Taxi to the pier', 'Transport'],
    ['Gelato', 'Food'],
    ['Pizza & wine', 'Food'],
    ['Villa deposit', 'Stays'],
```

```
    ['Museum tickets', 'Activities'],
    ['Boat tour to Capri', 'Activities'],
    ['Delta flight', 'Flights'],
    ['Souvenirs for mum', 'Shopping'],
    ['Something else', 'Other'],
    ['', 'Other'],
  ])('%s -> %s', (text, expected) => {
    expect(guessExpenseCategory(text)).toBe(expected);
  });
});
```

src/utils/__tests__/money.test.ts

```
import type { Expense, Member, Trip } from '@/store/types';
import { balances, balancesCents, settlePlan, totalSpent, zeroSumGroups } from '../money';

const member = (id: string): Member => ({ id, name: id, role: 'Member' });
let n = 0;
const exp = (paidBy: string, amount: number, splitBetween: string[], extra: Partial<Expense> = {}): Expense => ({
  id: `e${n++}`, title: 'x', amount, category: 'Food', paidBy, splitBetween, date: '2026-01-01', ...extra,
});
const ledger = (ids: string[], expenses: Expense[]): Pick<Trip, 'members' | 'expenses'> => ({ members: ids.map(member), expenses });
const sum = (o: Record<string, number>) => Object.values(o).reduce((a, b) => a + b, 0);

/** Applies a settle-up plan as settlement expenses, the way the app records payments. */
const applyPlan = (t: Pick<Trip, 'members' | 'expenses'>) =>
  ({ ...t, expenses: [...t.expenses, ...settlePlan(t).map((p) => exp(p.from, p.amount, [p.to], { settlement: true }))] });

describe('balances', () => {
  it('credits the payer and charges everyone in the split', () => {
    const b = balances(ledger(['a', 'b', 'c'], [exp('a', 90, ['a', 'b', 'c'])]));
    expect(b).toEqual({ a: 60, b: -30, c: -30 });
  });

  it('gives the odd cents to the first people so the total stays exactly zero', () => {
    const b = balancesCents(ledger(['a', 'b', 'c'], [exp('a', 10, ['a', 'b', 'c'])]));
    expect(b).toEqual({ a: 666, b: -333, c: -333 }); // $10.00 / 3 = 3.34 + 3.33 + 3.33: a's own share is the 3.34
    expect(sum(b)).toBe(0);
  });

  it('ignores split members who are no longer in the trip', () => {
    const b = balances(ledger(['a', 'b'], [exp('a', 100, ['a', 'b', 'ghost'])]));
    expect(b).toEqual({ a: 50, b: -50 });
  });

  it('is exact for amounts that break floating point (0.1 + 0.2)', () => {
    const b = balancesCents(ledger(['a', 'b'], [exp('a', 0.1, ['b']), exp('a', 0.2, ['b'])]));
    expect(b).toEqual({ a: 30, b: -30 });
  });
});

describe('totalSpent', () => {
  it('does not count settle-up payments as spending', () => {
    const t = ledger(['a', 'b'], [exp('a', 40, ['a', 'b']), exp('b', 25, ['a'], { settlement: true })]);
    expect(totalSpent(t)).toBe(40);
  });
});

describe('settlePlan', () => {
  it('is empty when everyone is square', () => {
    expect(settlePlan(ledger(['a', 'b'], [exp('a', 20, ['a', 'b']), exp('b', 20, ['a', 'b'])]))).toEqual([]);
  });

  it('pays the right person the right amount', () => {
    const t = ledger(['a', 'b', 'c'], [exp('a', 90, ['a', 'b', 'c'])]);
    expect(settlePlan(t)).toEqual([{ from: 'b', to: 'a', amount: 30 }, { from: 'c', to: 'a', amount: 30 }]);
  });

  it('zeroes every balance once the plan is paid', () => {
    const t = ledger(['a', 'b', 'c', 'd'], [exp('a', 120, ['a', 'b', 'c', 'd']), exp('b', 45.5, ['b', 'c']), exp('d', 10, ['a', 'd'])]);
    const after = balances(applyPlan(t));
    Object.values(after).forEach((v) => expect(Math.abs(v)).toBeLessThan(0.51));
    expect(settlePlan(applyPlan(t))).toEqual([]);
  });

  it('finds fewer payments than the simple largest-first method when the group splits into pairs', () => {
    // Balances: p +6, q +5 are owed; r -3, s -3, u -5 owe. {p,r,s} and {q,u} each add up to zero,
    // so 3 payments are enough, while largest-first needs 4.
    const t = ledger(['p', 'q', 'r', 's', 'u'], [
      exp('p', 6, ['r', 's']), // p paid 6 for r and s -> r -3, s -3, p +6
      exp('q', 5, ['u']), //      q paid 5 for u      -> u -5, q +5
    ]);
    const plan = settlePlan(t);
    expect(plan).toHaveLength(3);
    expect(plan).toContainEqual({ from: 'u', to: 'q', amount: 5 });
  });
```

```
  it('never needs more than (people - 1) payments and always settles exactly', () => {
    let seed = 7;
    const rnd = () => ((seed = (seed * 1103515245 + 12345) & 0x7fffffff) / 0x7fffffff);
    for (let round = 0; round < 60; round++) {
      const size = 2 + Math.floor(rnd() * 7);
      const ids = Array.from({ length: size }, (_, i) => `m${i}`);
      const expenses = Array.from({ length: 1 + Math.floor(rnd() * 8) }, () => {
        const split = ids.filter(() => rnd() < 0.6);
        return exp(ids[Math.floor(rnd() * size)], Math.round(rnd() * 20000) / 100 + 1, split.length ? split : [ids[0]]);
      });
      const t = ledger(ids, expenses);
      const plan = settlePlan(t);
      expect(plan.length).toBeLessThanOrEqual(size - 1);
      Object.values(balances(applyPlan(t))).forEach((v) => expect(Math.abs(v)).toBeLessThan(0.51));
    }
  });
});

describe('zeroSumGroups', () => {
  it('splits into the most groups that each add up to zero', () => {
    expect(zeroSumGroups([5, -5, 3, -3]).map((g) => g.length).sort()).toEqual([2, 2]);
    expect(zeroSumGroups([6, 5, -3, -3, -5])).toHaveLength(2);
    expect(zeroSumGroups([4, 3, -5, -2])).toHaveLength(1);
    expect(zeroSumGroups([])).toEqual([]);
  });
});
```

src/utils/__tests__/password.test.ts

```
import { createHash } from 'crypto';
import { PBKDF2_ROUNDS, hashPassword, isLegacyHash, safeEqual, verifyPassword } from '../password';

describe('hashPassword', () => {
  it('matches the official PBKDF2-HMAC-SHA256 test vector', async () => {
    // From RFC 7914 section 11: P = "passwd", S = "salt", c = 1 (first 32 bytes of the 64-byte result)
    expect(await hashPassword('passwd', 'salt', 1)).toBe('pbkdf2-sha256$1$55ac046e56e3089fec1691c22544b605f94185216dde0465e68b9d57c20dac
bc');
  });

  it('is repeatable, and the salt changes the result', async () => {
    const a = await hashPassword('TravelTogether2026!', 'salt-one');
    expect(await hashPassword('TravelTogether2026!', 'salt-one')).toBe(a);
    expect(await hashPassword('TravelTogether2026!', 'salt-two')).not.toBe(a);
    expect(a).toMatch(new RegExp(`^pbkdf2-sha256\\$${PBKDF2_ROUNDS}\\$[0-9a-f]{64}$`));
  });

  it('never contains the password', async () => {
    expect(await hashPassword('hunter2hunter2', 's')).not.toContain('hunter2');
  });
});

describe('verifyPassword', () => {
  it('accepts the right password and rejects the wrong one', async () => {
    const account = { salt: 'abc', passwordHash: await hashPassword('Correct-Horse-9', 'abc') };
    expect(await verifyPassword('Correct-Horse-9', account)).toEqual({ ok: true });
    expect((await verifyPassword('correct-horse-9', account)).ok).toBe(false);
    expect((await verifyPassword('', account)).ok).toBe(false);
  });

  it('upgrades a version-1 hash (single SHA-256) after a correct sign-in', async () => {
    const legacy = createHash('sha256').update('old-salt:Passw0rd!').digest('hex');
    expect(isLegacyHash(legacy)).toBe(true);
    const account = { salt: 'old-salt', passwordHash: legacy };
    expect((await verifyPassword('wrong', account)).ok).toBe(false);
    const ok = await verifyPassword('Passw0rd!', account);
    expect(ok.ok).toBe(true);
    expect(ok.upgrade).toMatch(/^pbkdf2-sha256\$/);
    expect(await verifyPassword('Passw0rd!', { salt: 'old-salt', passwordHash: ok.upgrade! })).toEqual({ ok: true });
  });

  it('upgrades a hash made with fewer rounds', async () => {
    const weak = await hashPassword('Passw0rd!', 's', 1000);
    const r = await verifyPassword('Passw0rd!', { salt: 's', passwordHash: weak });
    expect(r.ok).toBe(true);
    expect(r.upgrade).toContain(`$${PBKDF2_ROUNDS}$`);
  });

  it('rejects damaged or unknown hash formats instead of crashing', async () => {
    for (const bad of ['', 'nonsense', 'pbkdf2-sha256$abc$00', 'pbkdf2-sha256$99999999$00', 'md5$1$00']) {
      expect((await verifyPassword('x', { salt: 's', passwordHash: bad })).ok).toBe(false);
    }
  });

  it('supports passwords with accents and emoji', async () => {
    const account = { salt: 's', passwordHash: await hashPassword('Contraseña✈️2026', 's') };
```

```
    expect((await verifyPassword('Contraseña✈️2026', account)).ok).toBe(true);
  });
});

describe('safeEqual', () => {
  it('compares strings', () => {
    expect(safeEqual('abc', 'abc')).toBe(true);
    expect(safeEqual('abc', 'abd')).toBe(false);
    expect(safeEqual('abc', 'abcd')).toBe(false);
  });
});
```

src/utils/__tests__/points.test.ts

```
import { crewStandings } from '../points';
import type { Trip } from '@/store/types';

const trip = {
  members: [
    { id: 'a', name: 'Ana', role: 'Member' },
    { id: 'b', name: 'Ben', role: 'Member' },
    { id: 'c', name: 'Cy', role: 'Member' },
  ],
  activities: [{ createdBy: 'a', votes: { a: 1, b: 1, c: -1 } }],
  tasks: [{ done: true, assignee: 'b' }, { done: false, assignee: 'c' }],
  photos: [{ by: 'b' }, { by: 'b' }],
  expenses: [{ paidBy: 'c' }, { paidBy: 'c', settlement: true }],
  messages: [{ authorId: 'a' }, { authorId: 'a', ai: true }, { authorId: 'c', kind: 'system' }],
} as unknown as Trip;

describe('crew points', () => {
  it('adds up what each person really did', () => {
    const s = Object.fromEntries(crewStandings(trip).map((x) => [x.memberId, x.points]));
    expect(s.a).toBe(3 + 1 + 1); // added an item, voted, one chat message (the AI and system lines do not count)
    expect(s.b).toBe(1 + 3 + 2 * 2); // voted, finished a job, two photos
    expect(s.c).toBe(1 + 2); // voted, logged one expense (the settlement does not count)
  });

  it('puts the person with the most points first, ties by name', () => {
    expect(crewStandings(trip).map((x) => x.name)).toEqual(['Ben', 'Ana', 'Cy']);
  });
});
```

src/utils/__tests__/qr.test.ts

```
import jsQR from 'jsqr';

import { qrMatrix, qrPath } from '../qr';

/** Draws the matrix the way a screen would (with the empty border a scanner needs) and reads it back with a QR decoder. */
function scan(matrix: boolean[][], scale = 6, quiet = 4): string | null {
  const size = (matrix.length + quiet * 2) * scale;
  const pixels = new Uint8ClampedArray(size * size * 4).fill(255);
  matrix.forEach((row, r) =>
    row.forEach((dark, c) => {
      if (!dark) return;
      for (let y = 0; y < scale; y++) {
        for (let x = 0; x < scale; x++) {
          const i = (((r + quiet) * scale + y) * size + (c + quiet) * scale + x) * 4;
          pixels[i] = pixels[i + 1] = pixels[i + 2] = 0;
        }
      }
    }),
  );
  return jsQR(pixels, size, size)?.data ?? null;
}

describe('QR codes', () => {
  it('can be read back by a scanner', () => {
    for (const text of [
      'AML-2026',
      'togetherwego://join?code=AML-2026',
      'exp://192.168.1.20:8081/--/join?code=AML-2026&server=http%3A%2F%2F192.168.1.20%3A8787',
      'Join my trip “Amalfi Squad” ✈️ — code AML-2026',
      'x'.repeat(180),
    ]) {
      expect(scan(qrMatrix(text))).toBe(text);
    }
  });

  it('is square, bigger for longer text, and has the three corner markers', () => {
    const short = qrMatrix('AML-2026');
    const long = qrMatrix('x'.repeat(180));
    expect(short.every((row) => row.length === short.length)).toBe(true);
    expect(long.length).toBeGreaterThan(short.length);
    const n = short.length;
```

```
    // the 7x7 finder pattern: dark ring, light ring, dark 3x3 centre — top-left, top-right, bottom-left
    for (const [r0, c0] of [[0, 0], [0, n - 7], [n - 7, 0]]) {
      expect(short[r0][c0]).toBe(true);
      expect(short[r0 + 1][c0 + 1]).toBe(false);
      expect(short[r0 + 3][c0 + 3]).toBe(true);
    }
  });

  it('draws the same squares as the grid says', () => {
    const matrix = qrMatrix('AML-2026');
    const path = qrPath(matrix);
    // every "M x y h len v1 h-len z" is a run of dark squares: counting their lengths gives the number of dark squares
    const dark = [...path.matchAll(/M(\d+) (\d+)h(\d+)v1h-\d+z/g)].reduce((sum, m) => sum + Number(m[3]), 0);
    expect(dark).toBe(matrix.flat().filter(Boolean).length);
    expect(qrPath([[false, false]])).toBe('');
    expect(qrPath([[true, true, false, true]])).toBe('M0 0h2v1h-2zM3 0h1v1h-1z');
  });
});
```

src/utils/__tests__/time.test.ts

```
import { addDays, daysBetween, formatInTz, hoursAhead, isValidYMD, partsInTz, tripDays, tzOffsetMinutes, zonedToUtc } from '../time';

const iso = (ms: number) => new Date(ms).toISOString();

describe('calendar dates', () => {
  it('adds days across months and leap years', () => {
    expect(addDays('2026-01-31', 1)).toBe('2026-02-01');
    expect(addDays('2028-02-28', 2)).toBe('2028-03-01');
    expect(addDays('2026-12-31', 1)).toBe('2027-01-01');
    expect(addDays('2026-03-01', -1)).toBe('2026-02-28');
  });

  it('counts days between two dates', () => {
    expect(daysBetween('2026-06-10', '2026-06-16')).toBe(6);
    expect(daysBetween('2026-06-16', '2026-06-10')).toBe(-6);
    expect(tripDays('2026-06-10', '2026-06-12')).toEqual(['2026-06-10', '2026-06-11', '2026-06-12']);
  });

  it('rejects dates that do not exist', () => {
    expect(isValidYMD('2026-02-30')).toBe(false);
    expect(isValidYMD('2026-13-01')).toBe(false);
    expect(isValidYMD('2026-6-1')).toBe(false);
    expect(isValidYMD('2028-02-29')).toBe(true);
  });
});

describe('time zones', () => {
  it('knows summer time and winter time in Rome', () => {
    expect(tzOffsetMinutes('Europe/Rome', Date.parse('2026-07-01T12:00:00Z'))).toBe(120);
    expect(tzOffsetMinutes('Europe/Rome', Date.parse('2026-01-15T12:00:00Z'))).toBe(60);
  });

  it('turns a wall-clock time in a city into the exact moment', () => {
    expect(iso(zonedToUtc('2026-07-01', '12:00', 'Europe/Rome'))).toBe('2026-07-01T10:00:00.000Z');
    expect(iso(zonedToUtc('2026-01-15', '12:00', 'Europe/Rome'))).toBe('2026-01-15T11:00:00.000Z');
    expect(iso(zonedToUtc('2026-06-10', '18:30', 'America/New_York'))).toBe('2026-06-10T22:30:00.000Z');
  });

  it('a New York to Rome flight converts to the right arrival day and duration', () => {
    const depart = zonedToUtc('2026-06-10', '18:30', 'America/New_York');
    const arrive = zonedToUtc('2026-06-11', '08:45', 'Europe/Rome');
    expect((arrive - depart) / 60000).toBe(8 * 60 + 15);
    expect(partsInTz(arrive, 'America/New_York')).toEqual({ ymd: '2026-06-11', hhmm: '02:45' });
    expect(formatInTz(arrive, 'Europe/Rome')).toMatch(/08:45$/);
  });

  it('round-trips wall time to UTC and back', () => {
    for (const tz of ['Asia/Tokyo', 'Australia/Sydney', 'America/Los_Angeles', 'Asia/Kolkata']) {
      const utc = zonedToUtc('2026-09-15', '07:05', tz);
      expect(partsInTz(utc, tz)).toEqual({ ymd: '2026-09-15', hhmm: '07:05' });
    }
  });

  it('hours ahead between cities', () => {
    const july = Date.parse('2026-07-01T12:00:00Z');
    expect(hoursAhead('Asia/Tokyo', 'America/New_York', july)).toBe(13);
    expect(hoursAhead('Europe/Rome', 'America/New_York', july)).toBe(6);
    expect(hoursAhead('Asia/Kolkata', 'UTC', july)).toBe(5.5);
  });
});
```

src/utils/__tests__/trivia.test.ts

```
import { quizQuestions } from '@/data/reference';
```

```
import { dailyQuestion, nextStreak } from '../trivia';

describe('daily trivia', () => {
  it('gives everyone the same question on the same day', () => {
    expect(dailyQuestion('2026-10-23')).toBe(dailyQuestion('2026-10-23'));
  });

  it('always picks a real question, and changes from day to day', () => {
    const days = Array.from({ length: 12 }, (_, i) => `2026-10-${String(10 + i).padStart(2, '0')}`);
    const picked = days.map((d) => dailyQuestion(d));
    for (const q of picked) expect(quizQuestions).toContain(q);
    expect(new Set(picked).size).toBeGreaterThan(3);
  });

  it('grows the streak on consecutive days and restarts it after a gap', () => {
    expect(nextStreak(0, null, '2026-10-23', true)).toBe(1);
    expect(nextStreak(1, '2026-10-23', '2026-10-24', true)).toBe(2);
    expect(nextStreak(5, '2026-10-20', '2026-10-24', true)).toBe(1);
  });

  it('ends the streak on a wrong answer', () => {
    expect(nextStreak(4, '2026-10-23', '2026-10-24', false)).toBe(0);
  });
});
```

src/utils/__tests__/validation.test.ts

```
import {
  firstError, maxLen, normalizeInvite, parseAmount, passwordStrength, required, validateAmount, validateEmail, validateFlightNumber,
  validateIata, validateInvite, validateName, validatePassword, validateTime, validateTripDates,
} from '../validation';

describe('syntax checks (is it shaped right?)', () => {
  it('emails', () => {
    expect(validateEmail('maya@mail.com')).toBeNull();
    expect(validateEmail('  maya@mail.com  ')).toBeNull();
    expect(validateEmail('')).toMatch(/Enter/);
    expect(validateEmail('maya@mail')).toMatch(/doesn’t look right/);
    expect(validateEmail('maya mail@x.com')).not.toBeNull();
  });

  it('passwords need 8+ characters with a letter and a number', () => {
    expect(validatePassword('TravelTogether2026!')).toBeNull();
    expect(validatePassword('short1')).toMatch(/8 characters/);
    expect(validatePassword('onlyletters')).toMatch(/letter and one number/);
    expect(validatePassword('12345678')).toMatch(/letter and one number/);
  });

  it('password strength rises with length and variety', () => {
    expect(passwordStrength('abc').score).toBe(0);
    expect(passwordStrength('abcdefgh1').score).toBeLessThan(passwordStrength('Abcdefgh1234!').score);
    expect(passwordStrength('Abcdefgh1234!').label).toBe('Very strong');
  });

  it('names accept accents and apostrophes but not digits or symbols', () => {
    expect(validateName('José O’Brien-Müller')).toBeNull();
    expect(validateName('A')).toMatch(/too short/);
    expect(validateName('R2D2')).toMatch(/only contain letters/);
    expect(validateName('x'.repeat(41))).toMatch(/under 40/);
  });

  it('times are 24-hour HH:MM', () => {
    expect(validateTime('09:30')).toBeNull();
    expect(validateTime('23:59')).toBeNull();
    expect(validateTime('24:00')).not.toBeNull();
    expect(validateTime('9:30')).not.toBeNull();
    expect(validateTime('12:60')).not.toBeNull();
  });

  it('invite codes are normalised then checked', () => {
    expect(normalizeInvite('aml2026')).toBe('AML-2026');
    expect(normalizeInvite('  aml-2026 ')).toBe('AML-2026');
    expect(validateInvite('AML-2026')).toBeNull();
    expect(validateInvite('AML2026')).toMatch(/ABC-1234/);
    expect(validateInvite('')).toMatch(/Enter the code/);
  });

  it('flight numbers', () => {
    expect(validateFlightNumber('NH 175')).toBeNull();
    expect(validateFlightNumber('ua837')).toBeNull();
    expect(validateFlightNumber('FLIGHT')).not.toBeNull();
  });
});

describe('meaning checks (does it make sense for a trip?)', () => {
  it('amounts: positive, sensible, plain numbers', () => {
```

```
    expect(parseAmount('1,200.50')).toBe(1200.5);
    expect(parseAmount('12.345')).toBeNull();
    expect(parseAmount('-5')).toBeNull();
    expect(parseAmount('12abc')).toBeNull();
    expect(validateAmount('0')).toMatch(/more than 0/);
    expect(validateAmount('2000000')).toMatch(/double-check/);
    expect(validateAmount('25.50')).toBeNull();
  });

  it('a trip cannot end before it starts, or run for months', () => {
    expect(validateTripDates('2026-06-10', '2026-06-16')).toEqual({});
    expect(validateTripDates('2026-06-10', '2026-06-09').end).toMatch(/on or after/);
    expect(validateTripDates('2026-06-10', '2026-09-01').end).toMatch(/at most 60 days/);
  });

  it('dates must exist on the calendar', () => {
    expect(validateTripDates('2026-02-30', '2026-03-02').start).toMatch(/valid start/);
    expect(validateTripDates('2028-02-29', '2028-03-02')).toEqual({}); // leap year
    expect(validateTripDates('2026-02-29', '2026-03-02').start).toBeDefined(); // not a leap year
  });

  it('airport codes must be real airports', () => {
    expect(validateIata('jfk', 'departure')).toBeNull();
    expect(validateIata('FCO', 'arrival')).toBeNull();
    expect(validateIata('XXX', 'arrival')).toMatch(/don’t recognise/);
    expect(validateIata('JF', 'arrival')).toMatch(/3 letters/);
    expect(validateIata('', 'arrival')).toMatch(/Enter the arrival/);
  });
});

describe('helpers', () => {
  it('firstError returns the first failing validator only', () => {
    expect(firstError('', required('a name'), maxLen(5))).toBe('Enter a name');
    expect(firstError('toolong', required('a name'), maxLen(5, 'Name'))).toBe('Name must be 5 characters or fewer');
    expect(firstError('ok', required('a name'), maxLen(5))).toBeNull();
  });
});
```

src/utils/contrast.ts

```
/** Colour maths for the theme: mixing colours and checking the WCAG contrast ratio (4.5:1 is the AA minimum for body text). */

export type Rgb = [number, number, number];

export function hexToRgb(hex: string): Rgb {
  const h = hex.replace('#', '');
  const full = h.length === 3 ? h.replace(/./g, (c) => c + c) : h;
  const n = parseInt(full, 16);
  return [(n >> 16) & 255, (n >> 8) & 255, n & 255];
}

export function rgbToHex([r, g, b]: Rgb): string {
  return '#' + [r, g, b].map((v) => Math.max(0, Math.min(255, Math.round(v))).toString(16).padStart(2, '0')).join('').toUpperCase();
}

/** `t` of the way from colour `a` to colour `b` (0 = a, 1 = b). */
export function mix(a: string, b: string, t: number): string {
  const [ar, ag, ab] = hexToRgb(a);
  const [br, bg, bb] = hexToRgb(b);
  return rgbToHex([ar + (br - ar) * t, ag + (bg - ag) * t, ab + (bb - ab) * t]);
}

/** Relative luminance (WCAG 2.x). */
export function luminance(hex: string): number {
  const [r, g, b] = hexToRgb(hex).map((v) => {
    const c = v / 255;
    return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
  });
  return 0.2126 * r + 0.7152 * g + 0.0722 * b;
}

/** Contrast ratio between two hex colours, from 1 (identical) to 21 (black on white). */
export function contrast(a: string, b: string): number {
  const la = luminance(a);
  const lb = luminance(b);
  return (Math.max(la, lb) + 0.05) / (Math.min(la, lb) + 0.05);
}

/** Black or white, whichever reads better on `bg`. */
export function readableOn(bg: string, dark = '#0A0A0A', light = '#FFFFFF'): string {
  return contrast(bg, dark) >= contrast(bg, light) ? dark : light;
}

/**
 * Moves `color` toward `toward` in small steps until it reaches `min` contrast on every colour in `on`.
 * Used to turn a vivid accent into one that is safe to use as text.
```

```
 */
export function ensureContrast(color: string, on: string[], min: number, toward: string): string {
  let c = color;
  for (let i = 0; i <= 40 && on.some((bg) => contrast(c, bg) < min); i++) c = mix(color, toward, (i + 1) / 40);
  return c;
}
```

src/utils/format.ts

```
import { currencyByCode } from '@/data/reference';

export function uid(prefix = ''): string {
  return `${prefix}${Date.now().toString(36)}${Math.random().toString(36).slice(2, 8)}`;
}

/** Formats money with the right symbol and sensible decimals for the currency. */
export function money(amount: number, code = 'USD', opts: { decimals?: boolean } = {}): string {
  const cur = currencyByCode(code);
  const bigUnit = ['JPY', 'KRW', 'IDR', 'VND', 'ISK'].includes(code);
  const decimals = opts.decimals && !bigUnit ? 2 : 0;
  const n = Math.abs(amount).toLocaleString('en-US', {
    minimumFractionDigits: decimals,
    maximumFractionDigits: decimals,
  });
  const sign = amount < 0 ? '−' : '';
  const sym = cur?.symbol ?? `${code} `;
  return sym.length > 2 ? `${sign}${n} ${code}` : `${sign}${sym}${n}`;
}

export function initials(name: string): string {
  const parts = name.trim().split(/\s+/).filter(Boolean);
  if (!parts.length) return '?';
  if (parts.length === 1) return parts[0].slice(0, 2).toUpperCase();
  return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase();
}

export function pluralize(n: number, word: string, plural = `${word}s`) {
  return `${n} ${n === 1 ? word : plural}`;
}

export function durationLabel(min: number): string {
  if (min < 60) return `${min}m`;
  const h = Math.floor(min / 60);
  const m = min % 60;
  return m ? `${h}h ${m}m` : `${h}h`;
}

export type LengthFilter = 'any' | 'u1' | '1-2' | '2-4' | 'half' | 'full';

export const LENGTH_FILTERS: { key: LengthFilter; labelKey: string }[] = [
  { key: 'any', labelKey: 'len.any' },
  { key: 'u1', labelKey: 'len.u1' },
  { key: '1-2', labelKey: 'len.1-2' },
  { key: '2-4', labelKey: 'len.2-4' },
  { key: 'half', labelKey: 'len.half' },
  { key: 'full', labelKey: 'len.full' },
];

/** Buckets match the brief: under 1h, 1–2h, 2–4h, half day (4–6h), full day (6h+). */
export function matchesLength(min: number, f: LengthFilter): boolean {
  switch (f) {
    case 'any':
      return true;
    case 'u1':
      return min < 60;
    case '1-2':
      return min >= 60 && min <= 120;
    case '2-4':
      return min > 120 && min <= 240;
    case 'half':
      return min > 240 && min <= 360;
    case 'full':
      return min > 360;
  }
}

export function lengthBucketLabel(min: number): string {
  if (min < 60) return 'Under 1 hour';
  if (min <= 120) return '1–2 hours';
  if (min <= 240) return '2–4 hours';
  if (min <= 360) return 'Half day';
  return 'Full day';
}

/** Short tag for activity cards: <1h, 1-2h, 2-4h, Half day, Full day. */
export function lengthTag(min: number): string {
```

```
  if (min < 60) return '<1h';
  if (min <= 120) return '1-2h';
  if (min <= 240) return '2-4h';
  if (min <= 360) return 'Half day';
  return 'Full day';
}

export function priceTier(cost: number): string {
  if (cost === 0) return 'Free';
  if (cost < 15) return '$';
  if (cost < 40) return '$$';
  return '$$$';
}

export function timeAgo(ts: number, now = Date.now()): string {
  const s = Math.max(0, Math.round((now - ts) / 1000));
  if (s < 45) return 'just now';
  const m = Math.round(s / 60);
  if (m < 60) return `${m}m ago`;
  const h = Math.round(m / 60);
  if (h < 24) return `${h}h ago`;
  const d = Math.round(h / 24);
  return `${d}d ago`;
}

export function countdownLabel(ms: number): string {
  if (ms <= 0) return 'now';
  const totalMin = Math.round(ms / 60000);
  const d = Math.floor(totalMin / 1440);
  const h = Math.floor((totalMin % 1440) / 60);
  const m = totalMin % 60;
  if (d > 0) return `${d}d ${h}h`;
  if (h > 0) return `${h}h ${m}m`;
  return `${m}m`;
}
```

src/utils/geo.ts

```
import { airports, type Airport } from '@/data/airports';

export interface LatLng {
  lat: number;
  lng: number;
}

const R = 6371;
const rad = (d: number) => (d * Math.PI) / 180;

export function distanceKm(a: LatLng, b: LatLng): number {
  const dLat = rad(b.lat - a.lat);
  const dLng = rad(b.lng - a.lng);
  const h = Math.sin(dLat / 2) ** 2 + Math.cos(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
}

export function bearingDeg(a: LatLng, b: LatLng): number {
  const y = Math.sin(rad(b.lng - a.lng)) * Math.cos(rad(b.lat));
  const x = Math.cos(rad(a.lat)) * Math.sin(rad(b.lat)) - Math.sin(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.cos(rad(b.lng - a.lng));
  return ((Math.atan2(y, x) * 180) / Math.PI + 360) % 360;
}

export function compassPoint(deg: number): string {
  const pts = ['N', 'NE', 'E', 'SE', 'S', 'SW', 'W', 'NW'];
  return pts[Math.round(deg / 45) % 8];
}

export function kmLabel(km: number): string {
  if (km < 1) return `${Math.round(km * 1000)} m`;
  if (km < 10) return `${km.toFixed(1)} km`;
  return `${Math.round(km).toLocaleString('en-US')} km`;
}

/** Rough door-to-door estimate for city driving / airport transfer. */
export function driveMinutes(km: number): number {
  return Math.max(5, Math.round((km / 38) * 60 + 8));
}

export function nearestAirports(from: LatLng, count = 5): (Airport & { km: number })[] {
  return airports
    .map((a) => ({ ...a, km: distanceKm(from, a) }))
    .sort((x, y) => x.km - y.km)
    .slice(0, count);
}

/** Projects a point into x/y kilometres east/north of `center` (equirectangular, fine for < 50 km). */
export function projectKm(center: LatLng, p: LatLng) {
```

```
  return {
    x: (p.lng - center.lng) * 111.32 * Math.cos(rad(center.lat)),
    y: (p.lat - center.lat) * 110.57,
  };
}
```

src/utils/guess.ts

```
import type { ExpenseCategory } from '@/store/types';

const GUESSES: [RegExp, ExpenseCategory][] = [
  [/\b(flight|flights|airline|baggage|luggage)\b/, 'Flights'],
  [/\b(hotel|hostel|villa|airbnb|apartment|rent|rooms?|accommodation|stay|resort)\b/, 'Stays'],
  [/\b(taxi|uber|lyft|train|bus|ferry|metro|fuel|gas|petrol|parking|toll|scooter|car hire|rental|transfer)\b/, 'Transport'],
  [/\b(ticket|tickets|tour|museum|entry|entrance|excursion|boat|show|concert|park|guide|activity)\b/, 'Activities'],
  [/\b(dinner|lunch|breakfast|brunch|pizza|pasta|gelato|coffee|cafe|café|restaurant|food|drinks?|snacks?|groceries|bar|beer|wine|tacos|s
ushi|ramen|bakery)\b/, 'Food'],
  [/\b(shop|shopping|souvenirs?|gift|gifts|clothes|market|store)\b/, 'Shopping'],
];

/** The likeliest expense category for words typed by hand ("Taxi to the pier" is Transport), or Other when nothing fits. */
export function guessExpenseCategory(text: string): ExpenseCategory {
  const t = text.toLowerCase();
  return GUESSES.find(([re]) => re.test(t))?.[1] ?? 'Other';
}
```

src/utils/money.ts

```
/**
 * Group-expense maths. Pure functions with no React or storage imports, so they are easy to test.
 * Everything is computed in whole cents internally: floating-point dollars drift (0.1 + 0.2 !== 0.3).
 */
import type { Trip } from '@/store/types';

type Ledger = Pick<Trip, 'members' | 'expenses'>;
export interface Transfer {
  from: string;
  to: string;
  /** Dollars, rounded to cents. */
  amount: number;
}

export const toCents = (dollars: number) => Math.round(dollars * 100);

/** Total spent by the group (settlement payments are not spending). */
export function totalSpent(t: Pick<Trip, 'expenses'>): number {
  return t.expenses.filter((e) => !e.settlement).reduce((sum, e) => sum + e.amount, 0);
}

/**
 * Net balance per member in cents: positive = is owed money, negative = owes money.
 * An expense is split equally; when it does not divide evenly the extra cents go to the first people in the
 * split, so the balances always add up to exactly zero.
 */
export function balancesCents(t: Ledger): Record<string, number> {
  const bal: Record<string, number> = {};
  t.members.forEach((m) => (bal[m.id] = 0));
  for (const e of t.expenses) {
    const split = e.splitBetween.filter((id) => id in bal);
    if (!split.length) continue;
    const cents = toCents(e.amount);
    const base = Math.floor(cents / split.length);
    const extra = cents - base * split.length;
    bal[e.paidBy] = (bal[e.paidBy] ?? 0) + cents;
    split.forEach((id, i) => (bal[id] -= base + (i < extra ? 1 : 0)));
  }
  return bal;
}

/** Same as {@link balancesCents} but in dollars, for display. */
export function balances(t: Ledger): Record<string, number> {
  return Object.fromEntries(Object.entries(balancesCents(t)).map(([id, c]) => [id, c / 100]));
}

const MAX_EXACT = 12; // 2^12 subsets is instant; beyond that we use the simple greedy plan
const DUST_CENTS = 50; // a balance under 50 cents counts as settled

/**
 * Fewest payments that settle everyone up.
 * The minimum number of payments is (people with a balance) minus (the most separate groups whose balances add
 * up to zero): each such group can settle among itself with one fewer payment than it has people.
 * We find that maximum by dynamic programming over subsets, then settle each group greedily.
 */
export function settlePlan(t: Ledger): Transfer[] {
  const people = Object.entries(balancesCents(t))
    .filter(([, c]) => c !== 0)
```

```
    .map(([id, c]) => ({ id, c }))
    .sort((a, b) => b.c - a.c || a.id.localeCompare(b.id));
  const groups = people.length <= MAX_EXACT ? zeroSumGroups(people.map((p) => p.c)) : [people.map((_, i) => i)];
  const plan: Transfer[] = [];
  for (const g of groups) plan.push(...greedyGroup(g.map((i) => people[i])));
  return plan.filter((p) => p.amount * 100 >= DUST_CENTS);
}

/** Splits balances (summing to 0) into the maximum number of groups that each sum to 0. Returns index groups. */
export function zeroSumGroups(values: number[]): number[][] {
  const n = values.length;
  if (n === 0) return [];
  const full = (1 << n) - 1;
  const sum = new Array<number>(1 << n).fill(0);
  for (let m = 1; m <= full; m++) {
    const low = m & -m;
    sum[m] = sum[m ^ low] + values[31 - Math.clz32(low)];
  }
  const best = new Array<number>(1 << n).fill(0);
  const prev = new Array<number>(1 << n).fill(0);
  for (let m = 1; m <= full; m++) {
    let bestPrev = 0;
    let bestVal = -1;
    for (let i = 0; i < n; i++) {
      if (!(m & (1 << i))) continue;
      const p = m ^ (1 << i);
      if (best[p] > bestVal) {
        bestVal = best[p];
        bestPrev = p;
      }
    }
    best[m] = bestVal + (sum[m] === 0 ? 1 : 0);
    prev[m] = bestPrev;
  }
  // Walk back from the full set: every mask on the path with a zero sum closes one group.
  const closers: number[] = [];
  for (let m = full; m !== 0; m = prev[m]) if (sum[m] === 0) closers.push(m);
  closers.reverse();
  const groups: number[][] = [];
  let before = 0;
  for (const m of closers) {
    const g = m & ~before;
    groups.push(Array.from({ length: n }, (_, i) => i).filter((i) => g & (1 << i)));
    before = m;
  }
  return groups;
}

/** Pairs the biggest debtor with the biggest creditor until everyone in the group is square. */
function greedyGroup(people: { id: string; c: number }[]): Transfer[] {
  const debtors = people.filter((p) => p.c < 0).map((p) => ({ id: p.id, v: -p.c })).sort((a, b) => b.v - a.v || a.id.localeCompare(b.i
d));
  const creditors = people.filter((p) => p.c > 0).map((p) => ({ id: p.id, v: p.c })).sort((a, b) => b.v - a.v || a.id.localeCompare(b.i
d));
  const out: Transfer[] = [];
  let i = 0;
  let j = 0;
  while (i < debtors.length && j < creditors.length) {
    const amt = Math.min(debtors[i].v, creditors[j].v);
    out.push({ from: debtors[i].id, to: creditors[j].id, amount: amt / 100 });
    debtors[i].v -= amt;
    creditors[j].v -= amt;
    if (debtors[i].v === 0) i++;
    if (creditors[j].v === 0) j++;
  }
  return out;
}
```

src/utils/password.ts

```
/**
 * Password hashing. Stored as `pbkdf2-sha256$<rounds>$<hex>`; the salt lives next to it on the account.
 *
 * Why not a bigger number of rounds? This runs in the phone's JavaScript engine, which is far slower than a
 * server, and the async version below yields to the UI so the screen never freezes. The round count is stored
 * in the hash, so it can be raised later and old hashes are upgraded on the next successful sign-in.
 * shortcut: a real service should hash on the server with Argon2id or bcrypt; this protects the on-device copy.
 */
import { pbkdf2Async } from '@noble/hashes/pbkdf2.js';
import { sha256 } from '@noble/hashes/sha2.js';
import { bytesToHex, utf8ToBytes } from '@noble/hashes/utils.js';

export const PBKDF2_ROUNDS = 10_000;
const PREFIX = 'pbkdf2-sha256';
const MAX_ROUNDS = 1_000_000; // refuse absurd values so a corrupted record cannot hang the app
```

```
/** Version 1 of the app stored a single round of SHA-256 over `salt:password` (64 hex characters). */
export const isLegacyHash = (stored: string) => /^[0-9a-f]{64}$/.test(stored);

export async function hashPassword(password: string, salt: string, rounds = PBKDF2_ROUNDS): Promise<string> {
  const key = await pbkdf2Async(sha256, password.normalize('NFKC'), salt, { c: rounds, dkLen: 32, asyncTick: 8 });
  return `${PREFIX}$${rounds}$${bytesToHex(key)}`;
}

/** Compares two strings in time that does not depend on where they first differ. */
export function safeEqual(a: string, b: string): boolean {
  if (a.length !== b.length) return false;
  let diff = 0;
  for (let i = 0; i < a.length; i++) diff |= a.charCodeAt(i) ^ b.charCodeAt(i);
  return diff === 0;
}

/**
 * Checks a password against a stored hash. When it is right but the stored hash is old or weak, `upgrade`
 * holds the new hash the caller should save.
 */
export async function verifyPassword(password: string, account: { passwordHash: string; salt: string }): Promise<{ ok: boolean; upgrad
e?: string }> {
  const { passwordHash, salt } = account;
  if (isLegacyHash(passwordHash)) {
    const legacy = bytesToHex(sha256(utf8ToBytes(`${salt}:${password}`)));
    return safeEqual(legacy, passwordHash) ? { ok: true, upgrade: await hashPassword(password, salt) } : { ok: false };
  }
  const [prefix, roundsText] = passwordHash.split('$');
  const rounds = Number(roundsText);
  if (prefix !== PREFIX || !Number.isInteger(rounds) || rounds < 1 || rounds > MAX_ROUNDS) return { ok: false };
  const actual = await hashPassword(password, salt, rounds);
  if (!safeEqual(actual, passwordHash)) return { ok: false };
  return rounds < PBKDF2_ROUNDS ? { ok: true, upgrade: await hashPassword(password, salt) } : { ok: true };
}
```

src/utils/points.ts

```
import type { Trip } from '@/store/types';

export interface Standing {
  memberId: string;
  name: string;
  avatar?: string;
  points: number;
}

/**
 * "Crew points": a friendly scoreboard built from what people really did on the trip, so it is always true and
 * needs no extra syncing. A vote is 1 point, a finished checklist job 3, a photo 2, an expense logged 2, a plan
 * item added 3, a chat message 1.
 */
export function crewStandings(trip: Trip): Standing[] {
  const points: Record<string, number> = Object.fromEntries(trip.members.map((m) => [m.id, 0]));
  const add = (id: string | undefined, n: number) => {
    if (id && id in points) points[id] += n;
  };
  for (const a of trip.activities) {
    add(a.createdBy, 3);
    for (const voter of Object.keys(a.votes)) add(voter, 1);
  }
  for (const t of trip.tasks) if (t.done) add(t.assignee, 3);
  for (const p of trip.photos) add(p.by, 2);
  for (const e of trip.expenses) if (!e.settlement) add(e.paidBy, 2);
  for (const m of trip.messages) if (!m.ai && m.kind !== 'system') add(m.authorId, 1);
  return trip.members
    .map((m) => ({ memberId: m.id, name: m.name, avatar: m.avatar, points: points[m.id] }))
    .sort((a, b) => b.points - a.points || a.name.localeCompare(b.name));
}
```

src/utils/qr.ts

```
/** QR codes for invite links: friends point their phone camera at the screen and the link opens TogetherWeGo. */
import qrcode from 'qrcode-generator';

/** The library writes one byte per letter, which breaks accents and emoji; links with them need UTF-8. */
function utf8Bytes(text: string): number[] {
  const out: number[] = [];
  for (const ch of text) {
    const cp = ch.codePointAt(0) as number;
    if (cp < 0x80) out.push(cp);
    else if (cp < 0x800) out.push(0xc0 | (cp >> 6), 0x80 | (cp & 63));
    else if (cp < 0x10000) out.push(0xe0 | (cp >> 12), 0x80 | ((cp >> 6) & 63), 0x80 | (cp & 63));
    else out.push(0xf0 | (cp >> 18), 0x80 | ((cp >> 12) & 63), 0x80 | ((cp >> 6) & 63), 0x80 | (cp & 63));
  }
  return out;
}
```

```
qrcode.stringToBytes = utf8Bytes;

/** The code as a grid of squares: true is dark. Medium error correction keeps it readable with a smudged screen. */
export function qrMatrix(text: string): boolean[][] {
  const qr = qrcode(0, 'M'); // 0 = pick the smallest size that fits
  qr.addData(text);
  qr.make();
  const n = qr.getModuleCount();
  return Array.from({ length: n }, (_, row) => Array.from({ length: n }, (_, col) => qr.isDark(row, col)));
}

/** One SVG path that covers all dark squares (each row is drawn as a few long rectangles, not hundreds of tiny ones). */
export function qrPath(matrix: boolean[][]): string {
  let d = '';
  matrix.forEach((row, y) => {
    let x = 0;
    while (x < row.length) {
      if (!row[x]) {
        x++;
        continue;
      }
      let run = 1;
      while (x + run < row.length && row[x + run]) run++;
      d += `M${x} ${y}h${run}v1h-${run}z`;
      x += run;
    }
  });
  return d;
}
```

src/utils/time.ts

```
/**
 * Date & time-zone helpers. Trip dates are stored as plain 'YYYY-MM-DD' strings
 * in the destination's local time, flights as absolute UTC timestamps. Uses Intl
 * when available and falls back to a fixed-offset table.
 */

const FALLBACK_OFFSETS: Record<string, number> = {
  'Asia/Tokyo': 540, 'Asia/Seoul': 540, 'Asia/Shanghai': 480, 'Asia/Hong_Kong': 480, 'Asia/Taipei': 480,
  'Asia/Singapore': 480, 'Asia/Makassar': 480, 'Asia/Kuala_Lumpur': 480, 'Asia/Manila': 480,
  'Asia/Bangkok': 420, 'Asia/Jakarta': 420, 'Asia/Ho_Chi_Minh': 420, 'Asia/Kolkata': 330,
  'Indian/Maldives': 300, 'Asia/Dubai': 240, 'Asia/Qatar': 180, 'Europe/Istanbul': 180,
  'Europe/London': 0, 'Europe/Dublin': 0, 'Europe/Lisbon': 0, 'Atlantic/Reykjavik': 0,
  'Europe/Paris': 60, 'Europe/Rome': 60, 'Europe/Madrid': 60, 'Europe/Berlin': 60, 'Europe/Amsterdam': 60,
  'Europe/Zurich': 60, 'Europe/Prague': 60, 'Europe/Vienna': 60, 'Africa/Casablanca': 60,
  'Europe/Athens': 120, 'Africa/Johannesburg': 120, 'Africa/Cairo': 120, 'Africa/Nairobi': 180,
  'Australia/Sydney': 600, 'Australia/Melbourne': 600, 'Pacific/Auckland': 720,
  'America/New_York': -300, 'America/Toronto': -300, 'America/Detroit': -300, 'America/Chicago': -360,
  'America/Denver': -420, 'America/Edmonton': -420, 'America/Phoenix': -420,
  'America/Los_Angeles': -480, 'America/Vancouver': -480, 'Pacific/Honolulu': -600,
  'America/Mexico_City': -360, 'America/Cancun': -300, 'America/Lima': -300, 'America/Bogota': -300,
  'America/Sao_Paulo': -180, 'America/Argentina/Buenos_Aires': -180, 'America/Santiago': -240, UTC: 0,
};

const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
const WEEKDAYS = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

export const YMD_RE = /^\d{4}-\d{2}-\d{2}$/;
export const HHMM_RE = /^([01]\d|2[0-3]):([0-5]\d)$/;

export function parseYMD(ymd: string) {
  const [y, m, d] = ymd.split('-').map(Number);
  return { y, m, d };
}

export function ymdFromUtcDate(date: Date): string {
  return `${date.getUTCFullYear()}-${String(date.getUTCMonth() + 1).padStart(2, '0')}-${String(date.getUTCDate()).padStart(2, '0')}`;
}

export function isValidYMD(ymd: string): boolean {
  if (!YMD_RE.test(ymd)) return false;
  const { y, m, d } = parseYMD(ymd);
  const dt = new Date(Date.UTC(y, m - 1, d));
  return dt.getUTCFullYear() === y && dt.getUTCMonth() === m - 1 && dt.getUTCDate() === d;
}

export function addDays(ymd: string, n: number): string {
  const { y, m, d } = parseYMD(ymd);
  return ymdFromUtcDate(new Date(Date.UTC(y, m - 1, d + n)));
}

export function daysBetween(a: string, b: string): number {
  const pa = parseYMD(a);
  const pb = parseYMD(b);
  return Math.round((Date.UTC(pb.y, pb.m - 1, pb.d) - Date.UTC(pa.y, pa.m - 1, pa.d)) / 86400000);
```

```
}

export function tripDays(start: string, end: string): string[] {
  const n = Math.max(0, daysBetween(start, end));
  return Array.from({ length: n + 1 }, (_, i) => addDays(start, i));
}

export function weekdayShort(ymd: string): string {
  const { y, m, d } = parseYMD(ymd);
  return WEEKDAYS[new Date(Date.UTC(y, m - 1, d)).getUTCDay()];
}

export function dayLabel(ymd: string): string {
  const { m, d } = parseYMD(ymd);
  return `${MONTHS[m - 1]} ${d}`;
}

export function rangeLabel(start: string, end: string): string {
  const a = parseYMD(start);
  const b = parseYMD(end);
  if (a.y === b.y) return `${MONTHS[a.m - 1]} ${a.d} – ${MONTHS[b.m - 1]} ${b.d}, ${b.y}`;
  return `${MONTHS[a.m - 1]} ${a.d}, ${a.y} – ${MONTHS[b.m - 1]} ${b.d}, ${b.y}`;
}

export function time12(hhmm: string): string {
  const [h, m] = hhmm.split(':').map(Number);
  const suffix = h >= 12 ? 'PM' : 'AM';
  const hr = h % 12 === 0 ? 12 : h % 12;
  return `${hr}:${String(m).padStart(2, '0')} ${suffix}`;
}

export function deviceTimeZone(): string {
  try {
    return Intl.DateTimeFormat().resolvedOptions().timeZone || 'UTC';
  } catch {
    return 'UTC';
  }
}

/** Minutes east of UTC for `tz` at the given instant. */
export function tzOffsetMinutes(tz: string, utcMs: number): number {
  try {
    const dtf = new Intl.DateTimeFormat('en-US', {
      timeZone: tz,
      hourCycle: 'h23',
      year: 'numeric',
      month: '2-digit',
      day: '2-digit',
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit',
    });
    const parts = dtf.formatToParts(new Date(utcMs));
    const get = (t: string) => Number(parts.find((p) => p.type === t)?.value);
    const asUtc = Date.UTC(get('year'), get('month') - 1, get('day'), get('hour') % 24, get('minute'), get('second'));
    const off = Math.round((asUtc - utcMs) / 60000);
    if (Number.isFinite(off)) return off;
  } catch {
    // fall through to table
  }
  if (tz === deviceTimeZone()) return -new Date(utcMs).getTimezoneOffset();
  return FALLBACK_OFFSETS[tz] ?? 0;
}

/** Converts a wall-clock time in `tz` to a UTC epoch (ms). */
export function zonedToUtc(ymd: string, hhmm: string, tz: string): number {
  const { y, m, d } = parseYMD(ymd);
  const [h, mi] = hhmm.split(':').map(Number);
  const guess = Date.UTC(y, m - 1, d, h, mi);
  const off1 = tzOffsetMinutes(tz, guess);
  let result = guess - off1 * 60000;
  const off2 = tzOffsetMinutes(tz, result);
  if (off2 !== off1) result = guess - off2 * 60000;
  return result;
}

/** Wall-clock parts of a UTC instant in a given zone. */
export function partsInTz(utcMs: number, tz: string) {
  const local = new Date(utcMs + tzOffsetMinutes(tz, utcMs) * 60000);
  return {
    ymd: ymdFromUtcDate(local),
    hhmm: `${String(local.getUTCHours()).padStart(2, '0')}:${String(local.getUTCMinutes()).padStart(2, '0')}`,
  };
}

export function formatInTz(utcMs: number, tz: string): string {
```

```
  const p = partsInTz(utcMs, tz);
  return `${dayLabel(p.ymd)} · ${p.hhmm}`;
}

export function utcOffsetLabel(tz: string, at = Date.now()): string {
  const off = tzOffsetMinutes(tz, at);
  const sign = off >= 0 ? '+' : '−';
  const h = Math.floor(Math.abs(off) / 60);
  const m = Math.abs(off) % 60;
  return `UTC${sign}${h}${m ? `:${String(m).padStart(2, '0')}` : ''}`;
}

const ABBREV: Record<string, string> = {
  'Asia/Tokyo': 'JST', 'Asia/Seoul': 'KST', 'Europe/London': 'GMT', 'Europe/Paris': 'CET',
  'America/New_York': 'ET', 'America/Chicago': 'CT', 'America/Denver': 'MT', 'America/Los_Angeles': 'PT',
  'Europe/Rome': 'CET', 'Europe/Madrid': 'CET', 'Europe/Berlin': 'CET', 'Europe/Amsterdam': 'CET', 'Europe/Prague': 'CET', 'Europe/Zuric
h': 'CET', 'Europe/Lisbon': 'WET',
  'Asia/Singapore': 'SGT', 'Asia/Bangkok': 'ICT', 'Australia/Sydney': 'AET', 'Asia/Kolkata': 'IST',
};

export function tzAbbrev(tz: string): string {
  const a = ABBREV[tz];
  if (!a) return tz.split('/').pop()?.replace(/_/g, ' ') ?? tz;
  // Summer time: Central European CET → CEST, UK GMT → BST, Western European WET → WEST.
  const summer = tzOffsetMinutes(tz, Date.now()) - tzOffsetMinutes('UTC', Date.now()) > ({ CET: 60, GMT: 0, WET: 0 }[a] ?? Infinity);
  return summer ? ({ CET: 'CEST', GMT: 'BST', WET: 'WEST' }[a] ?? a) : a;
}

/** Signed hour difference: positive when `tzA` is ahead of `tzB`. */
export function hoursAhead(tzA: string, tzB: string, at = Date.now()): number {
  return (tzOffsetMinutes(tzA, at) - tzOffsetMinutes(tzB, at)) / 60;
}

export function todayInTz(tz: string): string {
  return partsInTz(Date.now(), tz).ymd;
}

export function greetingFor(hour: number): string {
  if (hour < 5) return 'greet.night';
  if (hour < 12) return 'greet.morning';
  if (hour < 18) return 'greet.afternoon';
  return 'greet.evening';
}
```

src/utils/trivia.ts

```
import { quizQuestions, type QuizQuestion } from '@/data/reference';

import { addDays } from './time';

/** Today's question: the same one for everyone on a given day, a different one tomorrow. */
export function dailyQuestion(day: string, pool: QuizQuestion[] = quizQuestions): QuizQuestion {
  const n = day.split('').reduce((a, c) => (a * 31 + c.charCodeAt(0)) % 1000003, 7);
  return pool[n % pool.length];
}

/** The streak after answering on `day`: it grows when yesterday was also answered right, restarts at 1 otherwise, and a wrong answer en
ds it. */
export function nextStreak(streak: number, lastCorrectDay: string | null, day: string, correct: boolean): number {
  if (!correct) return 0;
  return lastCorrectDay === addDays(day, -1) ? streak + 1 : 1;
}
```

src/utils/validation.ts

```
/**
 * Input validation used across forms. Each validator returns an error message
 * (string) or null when the value is valid. Checks are both syntactic
 * (format) and semantic (does the value make sense for a trip?).
 */
import { airportByIata } from '@/data/airports';
import { HHMM_RE, daysBetween, isValidYMD } from './time';

export type Validator = (value: string) => string | null;

export const required =
  (label: string): Validator =>
  (v) =>
    v.trim() ? null : `Enter ${label}`;

export const maxLen =
  (n: number, label = 'This'): Validator =>
  (v) =>
    v.trim().length > n ? `${label} must be ${n} characters or fewer` : null;

export const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
```

```
export function validateEmail(v: string): string | null {
  if (!v.trim()) return 'Enter your email address';
  if (!EMAIL_RE.test(v.trim())) return 'That email doesn’t look right (e.g. name@mail.com)';
  return null;
}

export function validatePassword(v: string): string | null {
  if (!v) return 'Enter a password';
  if (v.length < 8) return 'Use at least 8 characters';
  if (!/[A-Za-z]/.test(v) || !/\d/.test(v)) return 'Include at least one letter and one number';
  return null;
}

export function passwordStrength(v: string): { score: 0 | 1 | 2 | 3 | 4; label: string } {
  let score = 0;
  if (v.length >= 8) score++;
  if (v.length >= 12) score++;
  if (/[A-Z]/.test(v) && /[a-z]/.test(v)) score++;
  if (/\d/.test(v) && /[^A-Za-z0-9]/.test(v)) score++;
  const labels = ['Too weak', 'Weak', 'Okay', 'Strong', 'Very strong'];
  return { score: score as 0 | 1 | 2 | 3 | 4, label: labels[score] };
}

export function validateName(v: string): string | null {
  const t = v.trim();
  if (!t) return 'Enter a name';
  if (t.length < 2) return 'Name is too short';
  if (t.length > 40) return 'Keep names under 40 characters';
  if (!/^[\p{L}\p{M}'’ .-]+$/u.test(t)) return 'Names can only contain letters, spaces, hyphens and apostrophes';
  return null;
}

/**
 * Parses a money amount. Accepts "12", "12.50", "1,200" — rejects negatives,
 * letters and absurd values.
 */
export function parseAmount(v: string): number | null {
  const cleaned = v.replace(/,/g, '').trim();
  if (!/^\d+(\.\d{1,2})?$/.test(cleaned)) return null;
  const n = Number(cleaned);
  return Number.isFinite(n) ? n : null;
}

export function validateAmount(v: string, { min = 0.01, max = 1_000_000, label = 'an amount' } = {}): string | null {
  if (!v.trim()) return `Enter ${label}`;
  const n = parseAmount(v);
  if (n === null) return 'Use numbers only, e.g. 25 or 25.50';
  if (n < min) return min > 0 ? `Amount must be more than 0` : 'Amount can’t be negative';
  if (n > max) return `That’s more than ${max.toLocaleString()} — double-check the amount`;
  return null;
}

export function validateTime(v: string): string | null {
  return HHMM_RE.test(v) ? null : 'Use 24-hour time, e.g. 09:30';
}

export function validateTripDates(start: string, end: string): { start?: string; end?: string } {
  const errs: { start?: string; end?: string } = {};
  if (!isValidYMD(start)) errs.start = 'Pick a valid start date';
  if (!isValidYMD(end)) errs.end = 'Pick a valid end date';
  if (!errs.start && !errs.end) {
    const len = daysBetween(start, end);
    if (len < 0) errs.end = 'End date must be on or after the start date';
    else if (len > 60) errs.end = 'Trips can be at most 60 days long';
  }
  return errs;
}

export const INVITE_RE = /^[A-Z]{3}-\d{4}$/;

export function normalizeInvite(v: string): string {
  const raw = v.toUpperCase().replace(/[^A-Z0-9]/g, '');
  if (raw.length <= 3) return raw;
  return `${raw.slice(0, 3)}-${raw.slice(3, 7)}`;
}

export function validateInvite(v: string): string | null {
  if (!v.trim()) return 'Enter the code your friend shared';
  if (!INVITE_RE.test(v)) return 'Codes look like ABC-1234';
  return null;
}

export const FLIGHT_RE = /^[A-Z0-9]{2}\s?\d{1,4}[A-Z]?$/;

export function validateFlightNumber(v: string): string | null {
```

```
  const t = v.trim().toUpperCase();
  if (!t) return 'Enter the flight number';
  if (!FLIGHT_RE.test(t)) return 'Flight numbers look like NH 175 or UA837';
  return null;
}

export function validateIata(v: string, label: string): string | null {
  const t = v.trim().toUpperCase();
  if (!t) return `Enter the ${label} airport code`;
  if (!/^[A-Z]{3}$/.test(t)) return 'Airport codes are 3 letters, e.g. JFK';
  if (!airportByIata(t)) return `We don’t recognise “${t}” — try a major airport code`;
  return null;
}

/** Runs validators in order and returns the first error. */
export function firstError(value: string, ...validators: Validator[]): string | null {
  for (const v of validators) {
    const e = v(value);
    if (e) return e;
  }
  return null;
}
```

server/__tests__/server.test.js

```
/** @jest-environment node */
const fs = require('fs');
const http = require('http');
const os = require('os');
const path = require('path');

const { createServer } = require('../index');
const { SyncStore, SyncError } = require('../store');

const TRIP = 'trip_abcdefghijklmnop1234';
const stamp = (n, node = 'aaaa1111') => `${String(1800000000000 + n)}-0000-${node}`;
const op = (n, extra = {}) => ({ p: ['name'], v: `Name ${n}`, t: stamp(n), ...extra });

describe('SyncStore', () => {
  it('keeps an ordered log and hands out everything after a position', () => {
    const s = new SyncStore();
    expect(s.append(TRIP, 'ROM-1234', 'phoneA01', [op(1), op(2)])).toEqual({ seq: 2 });
    expect(s.append(TRIP, 'ROM-1234', 'phoneB01', [op(3)])).toEqual({ seq: 3 });
    const all = s.since(TRIP, 0, 'phoneA01');
    expect(all.ops.map((o) => o.v)).toEqual(['Name 1', 'Name 2', 'Name 3']);
    expect(s.since(TRIP, 2, 'phoneA01').ops.map((o) => o.v)).toEqual(['Name 3']);
    expect(s.since(TRIP, 3, 'phoneA01').ops).toEqual([]);
    expect(s.since(TRIP, 0, 'phoneA01', 2).ops).toHaveLength(2); // paging
  });

  it('looks up a trip by invite code and follows a changed code', () => {
    const s = new SyncStore();
    s.append(TRIP, 'ROM-1234', 'phoneA01', [op(1)]);
    expect(s.lookup('ROM-1234')).toBe(TRIP);
    expect(s.lookup('XXX-0000')).toBeNull();
    expect(s.lookup('not a code')).toBeNull();
    s.append(TRIP, 'ROM-9999', 'phoneA01', [op(2)]);
    expect(s.lookup('ROM-1234')).toBeNull();
    expect(s.lookup('ROM-9999')).toBe(TRIP);
  });

  it('rejects anything that is not a well-formed change', () => {
    const s = new SyncStore();
    const bad = [
      [],
      [{}],
      [{ p: [], v: 1, t: stamp(1) }],
      [{ p: [1], v: 1, t: stamp(1) }],
      [{ p: ['a'], v: 1, t: 'nope' }],
      [{ p: ['a'], t: stamp(1) }], // no value and no delete marker
      [{ p: ['a'], v: 1, d: 1, t: stamp(1) }], // both
      [{ p: ['a'], d: 2, t: stamp(1) }],
      [{ p: Array(9).fill('x'), v: 1, t: stamp(1) }],
      [{ p: ['a'], v: 'x'.repeat(5000), t: stamp(1) }],
    ];
    for (const ops of bad) expect(() => s.append(TRIP, null, 'phoneA01', ops)).toThrow(SyncError);
    expect(() => s.append('short', null, 'phoneA01', [op(1)])).toThrow(/trip id/);
    expect(() => s.append(TRIP, 'bad code', 'phoneA01', [op(1)])).toThrow(/invite code/);
    expect(() => s.append(TRIP, null, '!', [op(1)])).toThrow(/device id/);
    expect(() => s.append(TRIP, null, 'phoneA01', Array.from({ length: 501 }, (_, i) => op(i)))).toThrow(/Too many/);
    expect(s.trips.size).toBe(0);
  });

  it('stores deletes without a value', () => {
    const s = new SyncStore();
    s.append(TRIP, null, 'phoneA01', [{ p: ['tasks', 't1'], d: 1, t: stamp(1) }]);
```

```
    expect(s.since(TRIP, 0).ops[0]).toEqual({ p: ['tasks', 't1'], d: 1, t: stamp(1) });
  });

  it('wakes a waiting phone as soon as a change arrives, and gives up after the wait time', async () => {
    const s = new SyncStore();
    s.append(TRIP, null, 'phoneA01', [op(1)]);
    const started = Date.now();
    const waiting = s.wait(TRIP, 1, 5000);
    setTimeout(() => s.append(TRIP, null, 'phoneB01', [op(2)]), 30);
    await waiting;
    expect(Date.now() - started).toBeLessThan(1500);
    const t0 = Date.now();
    await s.wait(TRIP, 2, 60);
    expect(Date.now() - t0).toBeGreaterThanOrEqual(50);
  });

  it('counts the phones seen recently', () => {
    let clock = 1_000_000;
    const s = new SyncStore({ now: () => clock });
    s.append(TRIP, null, 'phoneA01', [op(1)]);
    s.append(TRIP, null, 'phoneB01', [op(2)]);
    expect(s.since(TRIP, 0, 'phoneC01').peers).toBe(3);
    clock += 60000; // a minute later only the phone that just asked is still "online"
    expect(s.since(TRIP, 0, 'phoneC01').peers).toBe(1);
  });

  it('survives a restart', () => {
    const file = path.join(fs.mkdtempSync(path.join(os.tmpdir(), 'twg-')), 'sync.json');
    const a = new SyncStore({ file });
    a.append(TRIP, 'ROM-1234', 'phoneA01', [op(1), op(2)]);
    a.flush();
    const b = new SyncStore({ file });
    expect(b.since(TRIP, 0).ops).toHaveLength(2);
    expect(b.lookup('ROM-1234')).toBe(TRIP);
  });
});

describe('files (photos, documents, receipts)', () => {
  const FILE = 'photo_abc12345';
  const fresh = (options) => {
    const s = new SyncStore(options);
    s.append(TRIP, null, 'phoneA01', [op(1)]); // a trip has to exist first
    return s;
  };

  it('keeps a file for a trip and gives the same bytes back', async () => {
    const s = fresh();
    const bytes = Buffer.from([0, 1, 2, 250, 251, 252, 255]);
    expect(await s.putBlob(TRIP, FILE, bytes)).toEqual({ size: 7 });
    expect(await s.getBlob(TRIP, FILE)).toEqual(bytes);
    expect(await s.getBlob(TRIP, 'someone_else_1')).toBeNull();
    expect(s.stats().files).toBe(1);
  });

  it('refuses unknown trips, bad ids, empty files and files that are too large', async () => {
    const s = fresh({ maxBlobBytes: 100 });
    await expect(s.putBlob('trip_zzzzzzzzzzzzzzzz9999', FILE, Buffer.from('x'))).rejects.toMatchObject({ status: 404 });
    await expect(s.putBlob(TRIP, '../escape', Buffer.from('x'))).rejects.toMatchObject({ status: 400 });
    await expect(s.putBlob(TRIP, 'a b', Buffer.from('x'))).rejects.toMatchObject({ status: 400 });
    await expect(s.putBlob(TRIP, FILE, Buffer.alloc(0))).rejects.toMatchObject({ status: 400 });
    await expect(s.putBlob(TRIP, FILE, 'not bytes')).rejects.toMatchObject({ status: 400 });
    await expect(s.putBlob(TRIP, FILE, Buffer.alloc(101))).rejects.toMatchObject({ status: 413 });
    expect(await s.getBlob(TRIP, '../escape')).toBeNull();
    expect(s.stats().files).toBe(0);
  });

  it('stops one trip from filling the server, and lets a file be replaced by a smaller one', async () => {
    const s = fresh({ maxBlobBytesPerTrip: 100 });
    await s.putBlob(TRIP, 'file_one_1', Buffer.alloc(60));
    await expect(s.putBlob(TRIP, 'file_two_2', Buffer.alloc(60))).rejects.toMatchObject({ status: 413 });
    await s.putBlob(TRIP, 'file_one_1', Buffer.alloc(30)); // same id: replaces, frees 30
    await s.putBlob(TRIP, 'file_two_2', Buffer.alloc(60));
    expect(s.trips.get(TRIP).blobBytes).toBe(90);
  });

  it('counts two uploads at the same moment against the limit', async () => {
    const s = fresh({ maxBlobBytesPerTrip: 100 });
    const results = await Promise.allSettled([s.putBlob(TRIP, 'file_one_1', Buffer.alloc(70)), s.putBlob(TRIP, 'file_two_2', Buffer.allo
c(70))]);
    expect(results.filter((r) => r.status === 'fulfilled')).toHaveLength(1);
    expect(s.trips.get(TRIP).blobBytes).toBe(70);
  });

  it('keeps files across a restart and ignores half-finished uploads', async () => {
    const dir = fs.mkdtempSync(path.join(os.tmpdir(), 'twg-'));
    const file = path.join(dir, 'sync.json');
```

```
    const a = fresh({ file });
    await a.putBlob(TRIP, FILE, Buffer.from('hello'));
    fs.writeFileSync(path.join(dir, 'files', TRIP, 'half_done_1.tmp'), 'junk'); // an interrupted upload
    a.flush();
    const b = new SyncStore({ file });
    expect(b.stats().files).toBe(1);
    expect(b.trips.get(TRIP).blobBytes).toBe(5);
    expect((await b.getBlob(TRIP, FILE)).toString()).toBe('hello');
  });
});

describe('HTTP API', () => {
  let server;
  let base;

  const call = (method, pathname, body) =>
    new Promise((resolve, reject) => {
      const data = body === undefined ? undefined : typeof body === 'string' ? body : JSON.stringify(body);
      const req = http.request(base + pathname, { method, headers: data ? { 'Content-Type': 'application/json', 'Content-Length': Buffe
r.byteLength(data) } : {} }, (res) => {
        let text = '';
        res.on('data', (c) => (text += c));
        res.on('end', () => resolve({ status: res.statusCode, headers: res.headers, json: text && res.headers['content-type']?.includes
('json') ? JSON.parse(text) : text }));
      });
      req.on('error', reject);
      if (data) req.write(data);
      req.end();
    });

  const callRaw = (method, pathname, body) =>
    new Promise((resolve, reject) => {
      const req = http.request(base + pathname, { method, headers: body ? { 'Content-Type': 'application/octet-stream', 'Content-Lengt
h': body.length } : {} }, (res) => {
        const chunks = [];
        res.on('data', (c) => chunks.push(c));
        res.on('end', () => resolve({ status: res.statusCode, headers: res.headers, body: Buffer.concat(chunks) }));
      });
      req.on('error', reject);
      if (body) req.write(body);
      req.end();
    });

  beforeEach(async () => {
    server = createServer(new SyncStore());
    await new Promise((r) => server.listen(0, '127.0.0.1', r));
    base = `http://127.0.0.1:${server.address().port}`;
  });
  afterEach(() => new Promise((r) => server.close(r)));

  it('says it is alive', async () => {
    const r = await call('GET', '/health');
    expect(r.status).toBe(200);
    expect(r.json).toMatchObject({ ok: true, trips: 0 });
    expect((await call('GET', '/')).json).toContain('sync server is running');
  });

  it('sends and receives changes, and looks up a code', async () => {
    const post = await call('POST', `/v1/trips/${TRIP}/ops`, { code: 'ROM-1234', client: 'phoneA01', ops: [op(1), op(2)] });
    expect(post).toMatchObject({ status: 200, json: { seq: 2 } });
    const get = await call('GET', `/v1/trips/${TRIP}/ops?since=1&client=phoneB01`);
    expect(get.status).toBe(200);
    expect(get.json.ops).toHaveLength(1);
    expect(get.json).toMatchObject({ seq: 2, peers: 2 });
    expect(get.headers['access-control-allow-origin']).toBe('*');
    expect((await call('GET', '/v1/codes/ROM-1234')).json).toEqual({ tripId: TRIP });
    expect((await call('GET', '/v1/codes/NOP-0000')).status).toBe(404);
  });

  it('answers the browser pre-flight request', async () => {
    const r = await call('OPTIONS', `/v1/trips/${TRIP}/ops`);
    expect(r.status).toBe(204);
    expect(r.headers['access-control-allow-methods']).toContain('POST');
    expect(r.headers['access-control-allow-methods']).toContain('PUT');
  });

  it('long-polling returns as soon as another phone posts', async () => {
    await call('POST', `/v1/trips/${TRIP}/ops`, { client: 'phoneA01', ops: [op(1)] });
    const started = Date.now();
    const waiting = call('GET', `/v1/trips/${TRIP}/ops?since=1&wait=8000&client=phoneB01`);
    setTimeout(() => call('POST', `/v1/trips/${TRIP}/ops`, { client: 'phoneA01', ops: [op(2)] }), 40);
    const r = await waiting;
    expect(Date.now() - started).toBeLessThan(2000);
    expect(r.json.ops.map((o) => o.v)).toEqual(['Name 2']);
  });
```

```
  it('returns clear errors', async () => {
    expect((await call('POST', `/v1/trips/${TRIP}/ops`, '{not json')).status).toBe(400);
    expect((await call('POST', `/v1/trips/${TRIP}/ops`, { client: 'phoneA01', ops: [{ p: [] }] })).status).toBe(400);
    expect((await call('GET', `/v1/trips/${TRIP}/ops?since=0`)).status).toBe(404);
    expect((await call('GET', '/v1/trips/short/ops')).status).toBe(400);
    expect((await call('GET', '/nothing')).status).toBe(404);
    expect((await call('POST', `/v1/trips/${TRIP}/ops`, { client: 'phoneA01', ops: [{ p: ['x'], v: 'y'.repeat(1_100_000), t: stamp(1) }]
})).status).toBe(413);
  });


  it('sends and receives files', async () => {
    await call('POST', `/v1/trips/${TRIP}/ops`, { client: 'phoneA01', ops: [op(1)] });
    const bytes = Buffer.from(Array.from({ length: 5000 }, (_, i) => i % 256));
    const put = await callRaw('PUT', `/v1/trips/${TRIP}/files/photo_abc12345`, bytes);
    expect(put.status).toBe(200);
    expect(JSON.parse(put.body.toString())).toEqual({ size: 5000 });
    const get = await callRaw('GET', `/v1/trips/${TRIP}/files/photo_abc12345`);
    expect(get.status).toBe(200);
    expect(get.body.equals(bytes)).toBe(true);
    expect(get.headers['cache-control']).toContain('immutable');
    expect(get.headers['access-control-allow-origin']).toBe('*');
    expect((await call('GET', '/health')).json.files).toBe(1);
  });

  it('answers files that are missing, misplaced or too large with clear errors', async () => {
    await call('POST', `/v1/trips/${TRIP}/ops`, { client: 'phoneA01', ops: [op(1)] });
    expect((await callRaw('GET', `/v1/trips/${TRIP}/files/nothing_here_1`)).status).toBe(404);
    expect((await callRaw('PUT', `/v1/trips/trip_unknownunknown123456/files/photo_abc12345`, Buffer.from('x'))).status).toBe(404);
    expect((await callRaw('PUT', `/v1/trips/${TRIP}/files/bad!id`, Buffer.from('x'))).status).toBe(400);
    expect((await callRaw('PUT', `/v1/trips/${TRIP}/files/photo_abc12345`, Buffer.alloc(9 * 1024 * 1024, 1))).status).toBe(413);
  });

  it('limits how fast invite codes can be guessed', async () => {
    let last;
    for (let i = 0; i < 61; i++) last = await call('GET', '/v1/codes/AAA-0000');
    expect(last.status).toBe(429);
  });
});
```

server/index.js

```
/**
 * TogetherWeGo sync server.  Run it with:  npm run server
 *
 * It lets friends' phones share one trip: chat, itinerary, votes, expenses... Phones send small changes here and
 * ask for everyone else's. There are no accounts. A trip is protected by its long random id, which a phone only
 * learns by entering the trip's invite code, so only run this on a network you trust (home wifi or a hotspot).
 *
 *   POST /v1/trips/:id/ops      { code, client, ops }        send changes
 *   GET  /v1/trips/:id/ops      ?since=N&wait=ms&client=ID   get changes (waits for new ones if there are none)
 *   PUT  /v1/trips/:id/files/:fileId   (raw bytes)           send a photo, document or receipt
 *   GET  /v1/trips/:id/files/:fileId                         get it back
 *   GET  /v1/codes/:code                                     invite code -> trip id
 *   GET  /health                                             is the server alive?
 */
const http = require('http');
const os = require('os');
const path = require('path');

const { SyncStore, SyncError } = require('./store');

const MAX_BODY = 1_000_000;
const MAX_FILE = 8 * 1024 * 1024;
const MAX_WAIT_MS = 25000;
const LOOKUPS_PER_MINUTE = 60;

function createServer(store, options = {}) {
  const lookups = new Map(); // ip -> { count, resetAt }
  const now = options.now || Date.now;

  function tooManyLookups(ip) {
    const entry = lookups.get(ip);
    if (!entry || entry.resetAt < now()) {
      lookups.set(ip, { count: 1, resetAt: now() + 60000 });
      return false;
    }
    entry.count += 1;
    return entry.count > LOOKUPS_PER_MINUTE;
  }

  function send(res, status, body) {
    const text = JSON.stringify(body);
    res.writeHead(status, {
      'Content-Type': 'application/json',
      'Content-Length': Buffer.byteLength(text),
```

```
    'Cache-Control': 'no-store',
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Headers': 'content-type',
    'Access-Control-Allow-Methods': 'GET, POST, PUT, OPTIONS',
  });
  res.end(text);
}

function sendFile(res, bytes) {
  res.writeHead(200, {
    'Content-Type': 'application/octet-stream',
    'Content-Length': bytes.length,
    'Cache-Control': 'private, max-age=31536000, immutable', // a file never changes under its id
    'Access-Control-Allow-Origin': '*',
  });
  res.end(bytes);
}

/** The whole request body, or a 413 if it is bigger than `limit`. */
function readBody(req, limit) {
  return new Promise((resolve, reject) => {
    let size = 0;
    let tooBig = false;
    const chunks = [];
    req.on('data', (c) => {
      size += c.length;
      if (size > limit * 4) return req.destroy(); // far too much: stop listening
      if (size > limit) tooBig = true; // keep reading (and ignoring) so the 413 answer can still reach the sender
      else chunks.push(c);
    });
    req.on('end', () => (tooBig ? reject(new SyncError(413, 'Request too large')) : resolve(Buffer.concat(chunks))));
    req.on('error', reject);
    req.on('close', () => reject(new SyncError(400, 'The request was cut off')));
  });
}

async function readJson(req) {
  const raw = await readBody(req, MAX_BODY);
  try {
    return JSON.parse(raw.toString('utf8') || '{}');
  } catch {
    throw new SyncError(400, 'Body must be JSON');
  }
}

return http.createServer(async (req, res) => {
  try {
    const url = new URL(req.url, 'http://localhost');
    const parts = url.pathname.split('/').filter(Boolean);
    if (req.method === 'OPTIONS') return send(res, 204, {});

    if (req.method === 'GET' && url.pathname === '/health') return send(res, 200, { ok: true, ...store.stats() });
    if (req.method === 'GET' && url.pathname === '/') {
      res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
      return res.end('TogetherWeGo sync server is running.\nIn the app: Settings, Sync across phones, enter this address.\n');
    }

    if (req.method === 'GET' && parts[0] === 'v1' && parts[1] === 'codes' && parts.length === 3) {
      const ip = req.socket.remoteAddress || '';
      if (tooManyLookups(ip)) throw new SyncError(429, 'Too many tries. Wait a minute.');
      const tripId = store.lookup(decodeURIComponent(parts[2]));
      if (!tripId) throw new SyncError(404, 'No trip with that code');
      return send(res, 200, { tripId });
    }

    if (parts[0] === 'v1' && parts[1] === 'trips' && parts[3] === 'ops' && parts.length === 4) {
      const tripId = parts[2];
      if (req.method === 'POST') {
        const body = await readJson(req);
        return send(res, 200, store.append(tripId, body.code, body.client, body.ops));
      }
      if (req.method === 'GET') {
        const since = Number(url.searchParams.get('since')) || 0;
        const wait = Math.min(Number(url.searchParams.get('wait')) || 0, MAX_WAIT_MS);
        const waiting = store.wait(tripId, since, wait);
        req.on('close', () => waiting.cancel && waiting.cancel()); // the phone hung up: stop waiting
        await waiting;
        return send(res, 200, store.since(tripId, since, url.searchParams.get('client')));
      }
    }
    if (parts[0] === 'v1' && parts[1] === 'trips' && parts[3] === 'files' && parts.length === 5) {
      const [tripId, fileId] = [parts[2], parts[4]];
      if (req.method === 'PUT') {
        const bytes = await readBody(req, MAX_FILE);
        return send(res, 200, await store.putBlob(tripId, fileId, bytes));
      }
```

```
        if (req.method === 'GET') {
          const bytes = await store.getBlob(tripId, fileId);
          if (!bytes) throw new SyncError(404, 'No such file');
          return sendFile(res, bytes);
        }
      }
      throw new SyncError(404, 'Not found');
    } catch (e) {
      if (e instanceof SyncError) return send(res, e.status, { error: e.message });
      console.error(e);
      return send(res, 500, { error: 'Server error' });
    }
  });
}

function lanAddresses() {
  return Object.values(os.networkInterfaces())
    .flat()
    .filter((i) => i && i.family === 'IPv4' && !i.internal)
    .map((i) => i.address);
}

if (require.main === module) {
  const port = Number(process.env.PORT) || 8787;
  const file = process.env.SYNC_DATA || path.join(__dirname, 'data', 'sync.json');
  const store = new SyncStore({ file });
  const server = createServer(store);
  server.listen(port, '0.0.0.0', () => {
    console.log('\nTogetherWeGo sync server is running.\n');
    console.log('Phones on the same wifi can use any of these addresses:');
    for (const ip of lanAddresses()) console.log(`   http://${ip}:${port}`);
    console.log(`   http://localhost:${port}   (this computer only)\n`);
    console.log(`Saving to ${file}\nPress Ctrl+C to stop.\n`);
  });
  const stop = () => {
    store.flush();
    process.exit(0);
  };
  process.on('SIGINT', stop);
  process.on('SIGTERM', stop);
}

module.exports = { createServer, lanAddresses };
```

server/store.js

```
/**
 * The sync server's memory: for every shared trip, an ordered log of changes ("ops") that phones send and
 * fetch. The server never needs to understand a change; phones merge them (see src/sync/crdt.ts), so this stays
 * small. Plain JavaScript with no dependencies so it runs anywhere Node runs.
 */
const fs = require('fs');
const path = require('path');

const TRIP_ID = /^[A-Za-z0-9_-]{16,64}$/;
const CODE = /^[A-Z]{3}-\d{4}$/;
const STAMP = /^\d{13}-[0-9a-z]{4}-\w{1,16}$/;
const CLIENT = /^\w{4,32}$/;
const MAX_OPS_PER_REQUEST = 500;
const MAX_OP_BYTES = 4000;
const MAX_OPS_PER_TRIP = 50000;
const PEER_WINDOW_MS = 45000;
const BLOB_ID = /^[A-Za-z0-9_-]{8,40}$/;
const MAX_BLOB_BYTES = 8 * 1024 * 1024;
const MAX_BLOB_BYTES_PER_TRIP = 150 * 1024 * 1024;

class SyncError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}

/** Throws a SyncError for anything that is not a well-formed op. */
function checkOp(op) {
  if (!op || typeof op !== 'object') throw new SyncError(400, 'Each change must be an object');
  if (!Array.isArray(op.p) || op.p.length < 1 || op.p.length > 8 || op.p.some((s) => typeof s !== 'string' || s.length < 1 || s.length >
80)) {
    throw new SyncError(400, 'Bad change path');
  }
  if (typeof op.t !== 'string' || !STAMP.test(op.t)) throw new SyncError(400, 'Bad change stamp');
  if (op.d !== undefined && op.d !== 1) throw new SyncError(400, 'Bad delete marker');
  if (op.d === 1 && op.v !== undefined) throw new SyncError(400, 'A delete cannot carry a value');
  if (op.d !== 1 && op.v === undefined) throw new SyncError(400, 'A change needs a value');
  if (JSON.stringify(op).length > MAX_OP_BYTES) throw new SyncError(413, 'Change too large');
}
```

```
class SyncStore {
  /**
   * @param {{ file?: string, blobDir?: string, now?: () => number, maxBlobBytes?: number, maxBlobBytesPerTrip?: number }} [options]
   * Files (photos, documents, receipts) are kept next to the JSON file, or in memory when there is no file.
   */
  constructor(options = {}) {
    this.file = options.file;
    this.blobDir = options.blobDir || (options.file ? path.join(path.dirname(options.file), 'files') : null);
    this.limits = { blob: options.maxBlobBytes || MAX_BLOB_BYTES, perTrip: options.maxBlobBytesPerTrip || MAX_BLOB_BYTES_PER_TRIP };
    this.memBlobs = new Map();
    this.now = options.now || Date.now;
    /** @type {Map<string, { ops: object[], code: string | null, created: number, clients: Map<string, number>, blobs: Map<string, numbe
r>, blobBytes: number }>} */
    this.trips = new Map();
    /** @type {Map<string, string>} */
    this.codes = new Map();
    this.waiters = new Map();
    this.saveTimer = null;
    if (this.file) this.load();
  }

  trip(tripId, create) {
    if (!TRIP_ID.test(tripId)) throw new SyncError(400, 'Bad trip id');
    let t = this.trips.get(tripId);
    if (!t && create) {
      t = { ops: [], code: null, created: this.now(), clients: new Map(), blobs: new Map(), blobBytes: 0 };
      this.trips.set(tripId, t);
    }
    return t;
  }

  /** Adds changes from a phone. Returns the new log length (the "sequence number" phones remember). */
  append(tripId, code, client, ops) {
    if (!Array.isArray(ops) || ops.length === 0) throw new SyncError(400, 'Send at least one change');
    if (ops.length > MAX_OPS_PER_REQUEST) throw new SyncError(413, 'Too many changes in one request');
    if (!CLIENT.test(client || '')) throw new SyncError(400, 'Bad device id');
    if (code !== undefined && code !== null && !CODE.test(code)) throw new SyncError(400, 'Bad invite code');
    ops.forEach(checkOp);
    const t = this.trip(tripId, true);
    if (t.ops.length + ops.length > MAX_OPS_PER_TRIP) throw new SyncError(413, 'This trip has reached the server limit');
    for (const op of ops) t.ops.push({ p: op.p, t: op.t, ...(op.d ? { d: 1 } : { v: op.v }) });
    if (code && t.code !== code) {
      if (t.code) this.codes.delete(t.code);
      t.code = code;
      this.codes.set(code, tripId);
    }
    t.clients.set(client, this.now());
    this.wake(tripId);
    this.scheduleSave();
    return { seq: t.ops.length };
  }

  /** Changes after position `since`. */
  since(tripId, since, client, limit = 1000) {
    const t = this.trip(tripId, false);
    if (!t) throw new SyncError(404, 'Unknown trip');
    if (client && CLIENT.test(client)) t.clients.set(client, this.now());
    const from = Math.max(0, Math.min(Number(since) || 0, t.ops.length));
    return { ops: t.ops.slice(from, from + limit), seq: Math.min(t.ops.length, from + limit), total: t.ops.length, peers: this.peers(t)
};
  }

  /**
   * Waits until the trip has changes after `since` (or `ms` pass), so phones hear about changes almost instantly.
   * The returned promise has a `cancel()` for when the phone hangs up.
   */
  wait(tripId, since, ms) {
    const t = this.trip(tripId, false);
    if (!t) throw new SyncError(404, 'Unknown trip');
    if (t.ops.length > since || ms <= 0) return Promise.resolve();
    let finish;
    const promise = new Promise((resolve) => {
      const set = this.waiters.get(tripId) || new Set();
      this.waiters.set(tripId, set);
      const timer = setTimeout(() => finish(), ms);
      finish = () => {
        clearTimeout(timer);
        set.delete(finish);
        resolve();
      };
      set.add(finish);
    });
    promise.cancel = finish;
    return promise;
  }
```

```
  wake(tripId) {
    for (const done of [...(this.waiters.get(tripId) || [])]) done();
  }

  /** Keeps one file (a photo, document or receipt) for a trip. Sending the same id again replaces it. */
  async putBlob(tripId, blobId, bytes) {
    const t = this.trip(tripId, false);
    if (!t) throw new SyncError(404, 'Unknown trip');
    if (!BLOB_ID.test(blobId || '')) throw new SyncError(400, 'Bad file id');
    if (!Buffer.isBuffer(bytes) || bytes.length === 0) throw new SyncError(400, 'The file is empty');
    if (bytes.length > this.limits.blob) throw new SyncError(413, 'That file is too large to share');
    const before = t.blobs.get(blobId);
    if (t.blobBytes - (before || 0) + bytes.length > this.limits.perTrip) throw new SyncError(413, 'This trip has reached the server's f
ile limit');
    t.blobBytes += bytes.length - (before || 0); // counted first, so two uploads at once cannot both slip under the limit
    t.blobs.set(blobId, bytes.length);
    try {
      if (this.blobDir) {
        const dir = path.join(this.blobDir, tripId);
        await fs.promises.mkdir(dir, { recursive: true });
        const tmp = path.join(dir, `${blobId}.tmp`);
        await fs.promises.writeFile(tmp, bytes);
        await fs.promises.rename(tmp, path.join(dir, blobId));
      } else {
        this.memBlobs.set(`${tripId}/${blobId}`, bytes);
      }
    } catch (e) {
      t.blobBytes += (before || 0) - bytes.length;
      if (before) t.blobs.set(blobId, before);
      else t.blobs.delete(blobId);
      throw e;
    }
    return { size: bytes.length };
  }

  /** A stored file, or null when there is none with that id. */
  async getBlob(tripId, blobId) {
    const t = this.trip(tripId, false);
    if (!t) throw new SyncError(404, 'Unknown trip');
    if (!BLOB_ID.test(blobId || '') || !t.blobs.has(blobId)) return null;
    if (!this.blobDir) return this.memBlobs.get(`${tripId}/${blobId}`) || null;
    try {
      return await fs.promises.readFile(path.join(this.blobDir, tripId, blobId));
    } catch (e) {
      if (e.code === 'ENOENT') return null;
      throw e;
    }
  }

  lookup(code) {
    return CODE.test(code || '') ? this.codes.get(code) || null : null;
  }

  peers(t) {
    const cutoff = this.now() - PEER_WINDOW_MS;
    let n = 0;
    for (const seen of t.clients.values()) if (seen >= cutoff) n++;
    return n;
  }

  stats() {
    const all = [...this.trips.values()];
    return { trips: this.trips.size, ops: all.reduce((a, t) => a + t.ops.length, 0), files: all.reduce((a, t) => a + t.blobs.size, 0) };
  }

  // ---- saving to a plain JSON file, so a restart loses nothing ----
  load() {
    try {
      const data = JSON.parse(fs.readFileSync(this.file, 'utf8'));
      for (const [id, t] of Object.entries(data.trips || {})) {
        this.trips.set(id, { ops: t.ops, code: t.code, created: t.created, clients: new Map(), ...this.indexBlobs(id) });
        if (t.code) this.codes.set(t.code, id);
      }
    } catch (e) {
      if (e.code !== 'ENOENT') console.error('Could not read', this.file, e.message);
    }
  }

  /** Which files are on disk for a trip (after a restart), and how big they are. */
  indexBlobs(tripId) {
    const blobs = new Map();
    let blobBytes = 0;
    if (this.blobDir) {
      try {
        for (const name of fs.readdirSync(path.join(this.blobDir, tripId))) {
```

```
          if (!BLOB_ID.test(name)) continue; // leftovers of an interrupted upload end in .tmp
          const size = fs.statSync(path.join(this.blobDir, tripId, name)).size;
          blobs.set(name, size);
          blobBytes += size;
        }
      } catch (e) {
        if (e.code !== 'ENOENT') console.error('Could not read files of', tripId, e.message);
      }
    }
    return { blobs, blobBytes };
  }

  scheduleSave() {
    if (!this.file || this.saveTimer) return;
    this.saveTimer = setTimeout(() => {
      this.saveTimer = null;
      this.flush();
    }, 300);
    if (this.saveTimer.unref) this.saveTimer.unref();
  }

  flush() {
    if (!this.file) return;
    const data = { trips: {} };
    for (const [id, t] of this.trips) data.trips[id] = { ops: t.ops, code: t.code, created: t.created };
    fs.mkdirSync(path.dirname(this.file), { recursive: true });
    const tmp = `${this.file}.tmp`;
    fs.writeFileSync(tmp, JSON.stringify(data));
    fs.renameSync(tmp, this.file);
  }
}

module.exports = { SyncStore, SyncError, TRIP_ID, CODE, BLOB_ID };
```

e2e/README.md

```
# Browser tests

These tests open the **web build** of the app in a phone-sized browser window and use it like a person would. They sit next to the unit tests (`npm test`), which cover the logic without a screen.

```bash
npm install
npx playwright-core install chromium   # once: downloads a browser (or set E2E_CHROME to your Chrome)
npm run e2e                            # builds the web app, serves it, runs everything
npm run e2e -- --no-build              # reuse the last build
npm run e2e -- twophones overflow      # only some scripts
E2E_DIST=/tmp/mybuild E2E_PORT=8130 npm run e2e -- verify   # build into your own folder and port (to work next to someone else)
```

Script	What it checks
`verify.js`	Home, Plan filters and Lock in, Budget (expenses, receipt, settle up), Chat and Scout AI, the More menu
`verify2.js`	Crew roles, polls and checklist, flights and validation, packing, recap, language switch, trip phases, joining a trip
`verify3.js`	Sign-up, the personalisation survey, themes, the first-run tour
`verify4.js`	The 360° city views: spin, day and night, landmarks, city picker
`places.js`	The "Where do you want to go?" box: matches appear as you type, each with how far it is from you; pick one, type your own, no connection, location not shared
`motion.js`	The phone's *Reduce Motion* setting: nothing hidden, nothing moving, Settings says why
`overflow.js`	**Every screen** at 390 px, 320 px and with larger text: nothing may stick out sideways
`nesting.js`	**Every screen**: no button may sit inside another button (React's *"`<button>` cannot contain a nested `<button>`"* warning)
`targets.js`	**Every screen**: every control you tap is at least 44 x 44 points (Fitts's law)
`auth.js`	The screens before you sign in (first-run slides, sign-in, sign-up, reset password): tap sizes, overflow and nesting at three phone sizes, then each one used like a person: validation in words, the demo ticket, Join with a code, Google, reset
`shoot.js`	Not a test: takes pictures of screens in any theme and in light or dark, so the design can be looked at
`twophones.js`	Two phones and the real sync server: share, join from a QR code, chat, expenses, offline catch-up, restart, photo, receipt and PDF

Each script prints `PASS`/`FAIL` lines and `errors:` (browser console errors). `run.js` adds them up and exits with a non-zero code if anything failed.

The web build cannot show the native map, camera, notifications or file system; the on-phone parts of those features were built for iOS but not run on a physical phone.
```

e2e/auth.js

```
// The screens you see before you are signed in: the first-run slides, sign-in, sign-up and reset password. They cannot be reached once
// signed in (the router sends you home), so the sideways-overflow, tap-size and nesting checks that visit every screen miss them. This
// script opens them in a fresh browser, checks those three things at 390 px, 320 px and with larger text, and uses each one like a person.
const { BASE, launch, nestedControls } = require('./lib');
const results = [];
const check = (name, ok, extra = '') => results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`);
const vis = async (loc, t = 2500) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).catch(() => false);
```

```
const MIN = 44;

// every control you tap, smaller than 44 x 44 (same rule as targets.js)
const smallTargets = (p) =>
  p.evaluate((min) => {
    const SEL = 'button, a[href], [role=button], [role=link], [role=tab], [role=radio], [role=menuitem], input:not([type=hidden]):not([r
ole=switch]):not([type=checkbox]), textarea, select';
    const out = [];
    for (const el of document.querySelectorAll(SEL)) {
      const rc = el.getBoundingClientRect();
      const cs = getComputedStyle(el);
      if (rc.width < 1 || rc.height < 1 || cs.visibility === 'hidden' || cs.display === 'none' || Number(cs.opacity) === 0 || el.closest
('[aria-hidden=true]')) continue;
      if (Math.round(rc.width) >= min && Math.round(rc.height) >= min) continue;
      out.push(`"${(el.getAttribute('aria-label') || el.textContent || el.tagName).trim().slice(0, 30)}" ${Math.round(rc.width)}x${Math.
round(rc.height)}`);
    }
    return [...new Set(out)];
  }, MIN);

// anything wider than the screen (same idea as overflow.js)
const sticksOut = (p, vw) =>
  p.evaluate((w) => {
    const doc = document.scrollingElement || document.documentElement;
    const scrollers = [...document.querySelectorAll('*')].filter((el) => /(auto|scroll)/.test(getComputedStyle(el).overflowX));
    const inScroller = (el) => scrollers.some((s) => s !== el && s.contains(el));
    const clipped = (el) => { for (let n = el.parentElement; n; n = n.parentElement) { const cs = getComputedStyle(n); if ((cs.overflowX
=== 'hidden' || cs.overflow === 'hidden') && n.getBoundingClientRect().right <= w + 1) return true; } return false; };
    const out = [];
    for (const el of document.querySelectorAll('body *')) {
      const rc = el.getBoundingClientRect();
      if (rc.width < 2 || rc.right <= w + 1 || getComputedStyle(el).visibility === 'hidden' || inScroller(el) || clipped(el)) continue;
      out.push(`${el.tagName.toLowerCase()}[${(el.getAttribute('aria-label') || el.textContent || '').trim().slice(0, 24)}] right=${Mat
h.round(rc.right)}`);
      if (out.length >= 3) break;
    }
    return doc.scrollWidth > w + 1 ? [`page ${doc.scrollWidth} > ${w}`, ...out] : out;
  }, vw);

(async () => {
  const b = await launch();
  const open = async (viewport, scale = 1) => {
    const ctx = await b.newContext({ viewport, deviceScaleFactor: 2, isMobile: true, hasTouch: true });
    if (scale !== 1) await ctx.addInitScript((sc) => { try { const j = JSON.parse(localStorage.getItem('togetherwego-v1') || '{"state":
{"settings":{}},"version":0}'); j.state = j.state || {}; j.state.settings = { ...(j.state.settings || {}), textScale: sc }; localStorag
e.setItem('togetherwego-v1', JSON.stringify(j)); } catch {} }, scale);
    const p = await ctx.newPage();
    const errors = [];
    p.on('console', (m) => { if (m.type() === 'error' && !m.text().includes('ERR_CERT')) errors.push(m.text().slice(0, 300)); });
    p.on('pageerror', (e) => errors.push('PAGEERROR ' + e.message.slice(0, 300)));
    return { ctx, p, errors };
  };

  // ---------------------------------------------------------------- audit: sizes, overflow and nesting at three phone sizes
  for (const [label, viewport, scale] of [['390 px', { width: 390, height: 844 }, 1], ['320 px', { width: 320, height: 640 }, 1], ['360
px, larger text', { width: 360, height: 740 }, 1.25]]) {
    const { ctx, p } = await open(viewport, scale);
    const bad = [];
    const audit = async (where) => {
      const small = await smallTargets(p);
      const wide = await sticksOut(p, viewport.width);
      const nested = await nestedControls(p);
      if (small.length) bad.push(`${where}: small ${small.slice(0, 4).join(' | ')}`);
      if (wide.length) bad.push(`${where}: sticks out ${wide.join(' | ')}`);
      if (nested.length) bad.push(`${where}: nested ${nested.slice(0, 3).join(' | ')}`);
    };
    await p.goto(BASE + '/', { waitUntil: 'networkidle' }); await p.waitForTimeout(1800);
    await audit('first-run slide 1');
    await p.getByTestId('onboarding-next').click(); await p.waitForTimeout(1500); await audit('first-run slide 2');
    await p.getByTestId('onboarding-next').click(); await p.waitForTimeout(1500); await audit('first-run slide 3');
    await p.goto(BASE + '/sign-in', { waitUntil: 'networkidle' }); await p.waitForTimeout(1400);
    await audit('sign-in');
    await p.getByTestId('sign-in').click(); await p.waitForTimeout(400); await audit('sign-in with errors');
    await p.evaluate(() => { const el = [...document.querySelectorAll('*')].find((e) => /(auto|scroll)/.test(getComputedStyle(e).overflo
wY) && e.scrollHeight > e.clientHeight + 40); if (el) el.scrollTop = 99999; });
    await p.waitForTimeout(300); await audit('sign-in, lower part');
    await p.getByLabel('Join with a code').click(); await p.waitForTimeout(900); await audit('join with a code sheet'); await p.keyboar
d.press('Escape');
    await p.goto(BASE + '/sign-in', { waitUntil: 'networkidle' }); await p.waitForTimeout(1200);
    await p.getByRole('button', { name: 'Continue with Google' }).click(); await p.waitForTimeout(900); await audit('Google sheet');
    await p.goto(BASE + '/sign-up', { waitUntil: 'networkidle' }); await p.waitForTimeout(1200);
    await audit('sign-up');
    await p.getByRole('button', { name: 'Create account' }).click(); await p.waitForTimeout(400); await audit('sign-up with errors');
    await p.goto(BASE + '/forgot-password', { waitUntil: 'networkidle' }); await p.waitForTimeout(1200);
    await audit('reset password, step 1');
    await p.getByPlaceholder('you@email.com').fill('maya.chen@togetherwego.app');
```

```
    await p.getByRole('button', { name: 'Send reset code' }).click(); await p.waitForTimeout(700);
    await p.getByRole('button', { name: 'OK' }).click(); await p.waitForTimeout(400);
    await audit('reset password, step 2');
    check(`signed-out screens fit at ${label}: no control under ${MIN} x ${MIN}, nothing sticks out, no button inside a button`, bad.len
gth === 0, bad.slice(0, 4).join(' ;; '));
    await ctx.close();
  }

  // ---------------------------------------------------------------- first run
  {
    const { ctx, p, errors } = await open({ width: 390, height: 844 });
    await p.goto(BASE + '/', { waitUntil: 'networkidle' }); await p.waitForTimeout(1800);
    check('a first visit opens the first-run slides', await vis(p.getByRole('heading', { name: 'Plan together' })));
    check('there is exactly one Skip', (await p.getByRole('button', { name: 'Skip' }).count()) === 1);
    check('the step bar says where you are', await vis(p.getByRole('progressbar', { name: 'Step 1 of 3' })));
    await p.getByRole('button', { name: 'Next' }).click(); await p.waitForTimeout(900);
    check('Next shows the second idea', await vis(p.getByRole('heading', { name: 'Split the money' })) && await vis(p.getByRole('progres
sbar', { name: 'Step 2 of 3' })));
    await p.getByRole('button', { name: 'Next' }).click(); await p.waitForTimeout(900);
    check('the last slide says Get Started and hides Skip', await vis(p.getByRole('button', { name: 'Get Started' })) && (await p.getByR
ole('button', { name: 'Skip' }).count()) === 0);
    await p.getByRole('button', { name: 'Get Started' }).click(); await p.waitForTimeout(900);
    check('Get Started opens sign-in', await vis(p.getByRole('heading', { name: 'Welcome back' })));
    await p.goto(BASE + '/', { waitUntil: 'networkidle' }); await p.waitForTimeout(1500);
    check('after the slides, the app opens on sign-in', await vis(p.getByRole('heading', { name: 'Welcome back' })));
    check('first run and sign-in log no console errors', errors.length === 0, errors.join(' | '));
    await ctx.close();
  }

  // ---------------------------------------------------------------- sign-in
  {
    const { ctx, p, errors } = await open({ width: 390, height: 844 });
    await p.goto(BASE + '/sign-in', { waitUntil: 'networkidle' }); await p.waitForTimeout(1500);
    await p.getByTestId('sign-in').click(); await p.waitForTimeout(400);
    check('an empty form says what is missing, in words', await vis(p.getByText('Enter your email address')) && await vis(p.getByText('E
nter your password')));
    await p.getByTestId('email').fill('nobody@school.org'); await p.getByTestId('password').fill('Trip2027go');
    await p.getByTestId('sign-in').click(); await p.waitForTimeout(900);
    check('an unknown account gets a clear message with an icon', await vis(p.getByRole('alert')));
    await p.getByLabel('Use the demo account').click(); await p.waitForTimeout(500);
    check('the demo ticket fills the form', (await p.getByTestId('email').inputValue()) === 'maya.chen@togetherwego.app' && (await p.get
ByTestId('password').inputValue()).length > 8);
    check('the form error clears once you change something', !(await vis(p.getByText(/No account found/), 400)));
    check('the demo ticket has its own copy buttons (not inside another button)', (await p.getByLabel('Copy demo email').count()) === 1
&& (await p.getByLabel('Copy demo password').count()) === 1);
    // Join with a code: kept, and the join page opens with it after signing in
    await p.getByLabel('Join with a code').click(); await p.waitForTimeout(800);
    await p.getByTestId('join-code').fill('abc'); await p.getByTestId('join-code-continue').click(); await p.waitForTimeout(300);
    check('a code that is not a code is refused with words', await vis(p.getByText('Codes look like ABC-1234')));
    await p.getByTestId('join-code').fill('seo5521'); await p.getByTestId('join-code-continue').click(); await p.waitForTimeout(800);
    check('the invite code is kept on the sign-in screen', await vis(p.getByText('SEO-5521', { exact: true })) && await vis(p.getByLabel
('Remove the invite code')));
    await p.getByTestId('sign-in').click(); await p.waitForTimeout(3200);
    if (await p.getByRole('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(() =>
{});
    await p.waitForTimeout(800);
    check('after signing in, the join page opens with the code filled in', (await p.getByTestId('invite-code').inputValue().catch(() =>
'')) === 'SEO-5521');
    check('sign-in with an invite logs no console errors', errors.length === 0, errors.join(' | '));
    await ctx.close();
  }

  // ---------------------------------------------------------------- Google sheet
  {
    const { ctx, p } = await open({ width: 390, height: 844 });
    await p.goto(BASE + '/sign-in', { waitUntil: 'networkidle' }); await p.waitForTimeout(1500);
    await p.getByRole('button', { name: 'Continue with Google' }).click(); await p.waitForTimeout(800);
    check('the Google sheet lists Maya and "Use another account"', await vis(p.getByText('Maya Chen', { exact: true })) && await vis(p.g
etByLabel('Use another account')));
    await p.getByLabel('Use another account').click(); await p.waitForTimeout(400);
    await p.getByRole('button', { name: 'Continue', exact: true }).click(); await p.waitForTimeout(400);
    check('the Google form checks name and email in words', await vis(p.getByText('Enter a name')) && await vis(p.getByText('Enter your
email address')));
    await p.getByRole('button', { name: /Maya Chen, / }).click(); await p.waitForTimeout(3000);
    check('picking Maya signs in', await vis(p.getByRole('tab', { name: 'Home', exact: true }), 4000) || (await p.getByRole('button', {
name: 'Skip' }).count()) > 0);
    await ctx.close();
  }

  // ---------------------------------------------------------------- sign-up
  {
    const { ctx, p, errors } = await open({ width: 390, height: 844 });
    await p.goto(BASE + '/sign-up', { waitUntil: 'networkidle' }); await p.waitForTimeout(1300);
    await p.getByPlaceholder('e.g. Alex Rivera').fill('Jordan Smith');
    await p.getByPlaceholder('you@email.com').fill('nope'); await p.getByPlaceholder('you@email.com').blur(); await p.waitForTimeout(30
0);
```

```
    check('a bad email is marked when you leave the field (words, not just colour)', await vis(p.getByText(/doesn’t look right/)));
    await p.getByPlaceholder('you@email.com').fill('jordan@school.org'); await p.waitForTimeout(300);
    check('a good email gets a tick', await vis(p.getByLabel('Looks good')));
    await p.getByPlaceholder('At least 8 characters').first().fill('abc'); await p.waitForTimeout(300);
    check('the password bar says how strong it is', await vis(p.getByLabel(/Password strength: /)));
    await p.getByPlaceholder('At least 8 characters').first().fill('Trip2027go'); await p.getByPlaceholder('Re-enter password').fill('Tr
ip2027'); await p.getByPlaceholder('Re-enter password').blur(); await p.waitForTimeout(300);
    check('a password that does not match says so', await vis(p.getByText('Passwords don’t match')));
    await p.getByRole('button', { name: 'Create account' }).click(); await p.waitForTimeout(500);
    check('the terms box is required, in words', await vis(p.getByText('Please accept the terms to continue')));
    check('there is one checkbox and it is at least 44 high', (await p.getByRole('checkbox').count()) === 1 && (await p.getByRole('check
box').boundingBox()).height >= 44);
    await p.getByPlaceholder('Re-enter password').fill('Trip2027go'); await p.getByRole('checkbox').click(); await p.getByRole('button',
{ name: 'Create account' }).click(); await p.waitForTimeout(2500);
    check('a new account lands in Make it yours', await vis(p.getByRole('heading', { name: /make it yours/i })));
    check('sign-up logs no console errors', errors.length === 0, errors.join(' | '));
    await ctx.close();
  }

  // ---------------------------------------------------------------- an invited friend creates an account
  {
    const { ctx, p, errors } = await open({ width: 390, height: 844 });
    await p.goto(BASE + '/sign-in', { waitUntil: 'networkidle' }); await p.waitForTimeout(1500);
    await p.getByLabel('Join with a code').click(); await p.waitForTimeout(800);
    await p.getByTestId('join-code').fill('seo5521'); await p.getByTestId('join-code-continue').click(); await p.waitForTimeout(800);
    await p.getByRole('link', { name: 'Sign Up' }).click(); await p.waitForTimeout(1000);
    check('the invite code follows you to sign-up', await vis(p.getByText('SEO-5521', { exact: true })));
    await p.getByPlaceholder('e.g. Alex Rivera').fill('Casey Lee'); await p.getByPlaceholder('you@email.com').fill('casey@school.org');
    await p.getByPlaceholder('At least 8 characters').first().fill('Trip2027go'); await p.getByPlaceholder('Re-enter password').fill('Tr
ip2027go');
    await p.getByRole('checkbox').click(); await p.getByRole('button', { name: 'Create account' }).click(); await p.waitForTimeout(320
0);
    check('a new account with an invite lands on the join page with the code filled in', (await p.getByTestId('invite-code').inputValue
().catch(() => '')) === 'SEO-5521');
    check('an invited sign-up logs no console errors', errors.length === 0, errors.join(' | '));
    await ctx.close();
  }

  // ---------------------------------------------------------------- reset password
  {
    const { ctx, p, errors } = await open({ width: 390, height: 844 });
    await p.goto(BASE + '/sign-in', { waitUntil: 'networkidle' }); await p.waitForTimeout(1500);
    await p.getByRole('link', { name: 'Forgot password?' }).click(); await p.waitForTimeout(900);
    check('Forgot password opens the reset screen at step 1', await vis(p.getByRole('heading', { name: 'Reset password' })) && await vis
(p.getByRole('progressbar', { name: 'Step 1 of 2' })));
    await p.getByPlaceholder('you@email.com').last().fill('ghost@school.org'); await p.getByRole('button', { name: 'Send reset code' }).
click(); await p.waitForTimeout(500);
    check('an unknown email is told so', await vis(p.getByText(/couldn’t find an account/)));
    await p.getByPlaceholder('you@email.com').last().fill('maya.chen@togetherwego.app'); await p.getByRole('button', { name: 'Send reset
code' }).click(); await p.waitForTimeout(900);
    check('the demo build shows the code', await vis(p.getByText(/Demo build: your code is \d{6}/)));
    await p.getByRole('button', { name: 'OK' }).click(); await p.waitForTimeout(500);
    check('step 2 shows the code on screen too and the step bar moves', await vis(p.getByLabel(/^Fill in the demo code \d{6}$/)) && awai
t vis(p.getByRole('progressbar', { name: 'Step 2 of 2' })));
    await p.getByTestId('reset-code').fill('000000'); await p.getByTestId('reset-code').blur(); await p.waitForTimeout(300);
    check('a wrong code is refused in words', await vis(p.getByText('That code doesn’t match')));
    await p.getByLabel(/^Fill in the demo code \d{6}$/).click(); await p.waitForTimeout(300);
    await p.getByPlaceholder('At least 8 characters').fill('NewTrip2027'); await p.waitForTimeout(300);
    await p.getByRole('button', { name: 'Update password' }).click(); await p.waitForTimeout(1500);
    check('the new password is saved and you are back on sign-in', await vis(p.getByText(/Password updated/)) && await vis(p.getByTestId
('sign-in')));
    await p.getByTestId('email').fill('maya.chen@togetherwego.app'); await p.getByTestId('password').fill('NewTrip2027'); await p.getByT
estId('sign-in').click(); await p.waitForTimeout(3000);
    check('signing in with the new password works', await vis(p.getByRole('tab', { name: 'Home', exact: true }), 4000) || (await p.getBy
Role('button', { name: 'Skip' }).count()) > 0);
    check('reset logs no console errors', errors.length === 0, errors.join(' | '));
    await ctx.close();
  }

  console.log(results.join('\n'));
  await b.close();
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 600)); process.exit(1); });
```

e2e/lib.js

```
// Shared helpers for the browser tests: launch a browser, open the app as a phone-sized screen, sign in as the demo user.
const path = require('path');
const { chromium } = require('playwright-core');

/** Where the exported web app is being served (run.js starts it for you). */
exports.BASE = process.env.E2E_URL || 'http://localhost:8081';
exports.fixture = (name) => path.join(__dirname, 'fixtures', name);
exports.shot = (name) => path.join(__dirname, 'shots', name);
/** Set E2E_CHROME to a Chrome/Chromium path, or run `npx playwright-core install chromium` once. */
exports.launch = () => chromium.launch(process.env.E2E_CHROME ? { executablePath: process.env.E2E_CHROME } : {});
```

```
/** Every screen of the app (route paths). */
exports.ROUTES = ['', 'explore', 'itinerary', 'budget', 'chat', 'profile', 'about', 'airport', 'bucket-list', 'crew', 'currency', 'desti
nations', 'documents', 'emergency', 'flights', 'help', 'how-it-works', 'join', 'map', 'nearby', 'new-trip', 'notifications', 'packing',
'personalize', 'photos', 'phrasebook', 'quiz', 'recap', 'settings', 'trips', 'weather', 'destination/tokyo', 'sign-up', 'sign-in', 'surv
ey'];

/** Buttons and links that sit inside another button or link on the current screen, as `"outer" contains "inner"`. */
exports.nestedControls = (p) =>
  p.evaluate(() => {
    const SEL = 'button, a, [role=button], [role=link]';
    const name = (el) => (el.getAttribute('aria-label') || el.textContent || el.tagName).trim().slice(0, 40);
    const out = [];
    for (const inner of document.querySelectorAll(SEL)) {
      const outer = inner.parentElement && inner.parentElement.closest(SEL);
      if (outer) out.push(`"${name(outer)}" contains "${name(inner)}"`);
    }
    return [...new Set(out)];
  });

/**
 * `beforeSignIn(page)` runs on the sign-in screen, before the demo account is used.
 * `context` adds browser settings, for example a pretend location: `{ geolocation: { latitude, longitude }, permissions: ['geolocatio
n'] }`.
 */
exports.start = async ({ beforeSignIn, context } = {}) => {
  const b = await exports.launch();
  const ctx = await b.newContext({ viewport: { width: 390, height: 844 }, deviceScaleFactor: 2, isMobile: true, hasTouch: true, ...conte
xt });
  const p = await ctx.newPage();
  const errors = [];
  // a join that tries the (not running) sync server on :8787 and falls back to this phone logs one harmless "connection refused"
  p.on('console', (m) => { if (m.type() === 'error' && !m.text().includes('ERR_CERT') && !m.location().url.includes(':8787')) errors.pus
h(m.text().slice(0, 400)); });
  p.on('pageerror', (e) => errors.push('PAGEERROR ' + e.message.slice(0, 400)));
  await p.goto(exports.BASE + '/', { waitUntil: 'networkidle' });
  await p.waitForTimeout(2000);
  if (await p.getByRole('button', { name: 'Skip' }).count()) { await p.getByRole('button', { name: 'Skip' }).click(); await p.waitForTim
eout(1000); }
  if (beforeSignIn) await beforeSignIn(p);
  await p.getByLabel('Use the demo account').click(); await p.waitForTimeout(300);
  await p.getByTestId('sign-in').click(); await p.waitForTimeout(2500);
  if (!process.env.KEEP_TOUR && await p.getByRole('button', { name: 'Skip' }).count()) { await p.getByRole('button', { name: 'Skip' }).f
irst().click(); await p.waitForTimeout(600); }
  return { b, p, errors };
};
```

e2e/motion.js

```
const { BASE, launch } = require('./lib');
// The phone's "Reduce Motion" setting is respected: nothing hidden, nothing animating, and Settings says why.
const results = [];
const check = (name, ok, extra = '') => results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`);
const vis = async (loc, t = 3000) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);

async function open(b, reducedMotion) {
  const ctx = await b.newContext({ viewport: { width: 390, height: 844 }, isMobile: true, hasTouch: true, reducedMotion });
  const p = await ctx.newPage();
  await p.goto(BASE + '/', { waitUntil: 'networkidle' });
  await p.waitForTimeout(1500);
  if (await p.getByRole('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).click();
  await p.waitForTimeout(500);
  await p.getByLabel('Use the demo account').click();
  await p.getByTestId('sign-in').click();
  await p.waitForTimeout(2500);
  if (await p.getByRole('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(() =>
{});
  return p;
}
// The hero photo's transform as numbers: `scale` is the slow zoom, `shift` is how far the scroll has slid it (parallax).
const heroMotion = (p) =>
  p.getByTestId('hero-photo').evaluate((el) => {
    const t = getComputedStyle(el).transform;
    if (t === 'none') return { scale: 1, shift: 0 };
    const m = t.match(/matrix\(([^)]+)\)/)[1].split(',').map(Number);
    return { scale: m[0], shift: m[5] };
  });
const scrollHome = (p, y) =>
  p.evaluate((dy) => {
    const el = [...document.querySelectorAll('*')].find((e) => /(auto|scroll)/.test(getComputedStyle(e).overflowY) && e.scrollHeight >
e.clientHeight + 40);
    if (el) el.scrollBy(0, dy);
  }, y);
const go = async (p, r) => { await p.goto(BASE + '/' + r, { waitUntil: 'networkidle' }); await p.waitForTimeout(1500); if (await p.getBy
Role('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(() => {}); };
```

```
(async () => {
  const b = await launch();
  // ---- reduce motion ON
  const calm = await open(b, 'reduce');
  await go(calm, '');
  const headlineOpacity = await calm.getByRole('heading', { name: /to go|Living|Tomorrow|epic/ }).first().evaluate((el) => getComputedSt
yle(el.closest('[style*="opacity"]') || el).opacity).catch(() => 'n/a');
  check('home headline is fully visible with Reduce Motion on', headlineOpacity === '1' || headlineOpacity === 'n/a', headlineOpacity);
  check('home content shows', await vis(calm.getByText(/to go|Living|Tomorrow/)));
  // the photo does not zoom, and does not slide when the page scrolls
  const still0 = await heroMotion(calm);
  await calm.waitForTimeout(2500);
  await scrollHome(calm, 260); await calm.waitForTimeout(500);
  const still1 = await heroMotion(calm);
  check('home photo does not zoom with Reduce Motion on', still0.scale === 1 && still1.scale === 1, JSON.stringify([still0, still1]));
  check('home photo does not drift against the scroll with Reduce Motion on', still1.shift === 0, JSON.stringify(still1));
  await go(calm, 'settings');
  check('settings says why animations are off', await vis(calm.getByText(/Reduce Motion setting is on/)));
  await go(calm, 'panorama?city=paris');
  check('360 view starts paused', await vis(calm.getByLabel('Auto rotate').getByText('Paused')));
  const before = await calm.getByText(/360° ·/).first().textContent();
  await calm.waitForTimeout(4000);
  const after = await calm.getByText(/360° ·/).first().textContent();
  check('360 view does not turn by itself', before === after, `${before} -> ${after}`);
  await calm.getByLabel('Auto rotate').click();
  check('tapping Auto explains why it stays off', await vis(calm.getByText(/Animations are off/)));
  await calm.getByLabel('Look right').click();
  await calm.waitForTimeout(150);
  const turned = await calm.getByText(/360° ·/).first().textContent();
  check('the arrows still turn the view at once', turned !== before, `${before} -> ${turned}`);

  // ---- reduce motion OFF: the same screens animate
  const lively = await open(b, 'no-preference');
  await go(lively, '');
  const move0 = await heroMotion(lively);
  await lively.waitForTimeout(2500);
  await scrollHome(lively, 260); await lively.waitForTimeout(600);
  const move1 = await heroMotion(lively);
  check('home photo zooms slowly by default', move1.scale > move0.scale && move1.scale <= 1.08, JSON.stringify([move0, move1]));
  check('home photo drifts against the scroll by default', move1.shift > 0, JSON.stringify(move1));
  await go(lively, 'settings');
  check('settings offers animations normally', await vis(lively.getByText(/Flipping numbers, floating cards/)));
  await go(lively, 'panorama?city=paris');
  check('360 view auto-rotates by default', await vis(lively.getByLabel('Auto rotate').getByText('Auto')));
  console.log(results.join('\n'));
  await b.close();
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 500)); process.exit(1); });
```

e2e/nesting.js

```
// Looks for a button inside another button (or a link inside a link) on every screen. A browser lets you do it, React warns about it
// ("<button> cannot contain a nested <button>"), and keyboards and screen readers cannot use such a control properly.
const { BASE, ROUTES, start, nestedControls } = require('./lib');

(async () => {
  const failures = [];
  let screens = 0;
  const audit = async (page, where) => {
    screens++;
    for (const f of await nestedControls(page)) failures.push(`FAIL  ${where}: ${f}`);
  };

  const { b, p, errors } = await start({ beforeSignIn: (page) => audit(page, 'the sign-in screen') });
  const open = async (path) => {
    await p.goto(BASE + '/' + path, { waitUntil: 'load' });
    await p.waitForTimeout(1200);
  };

  for (const r of [...ROUTES, 'panorama']) {
    await open(r);
    await audit(p, '/' + r);
  }

  // screens that only show something after a tap (each step waits for what it expects, so a page that did not change is not audited by
mistake)
  const expect = (label) => p.getByLabel(label).first().waitFor({ state: 'visible', timeout: 5000 });
  await open('itinerary');
  await p.getByLabel(/, \d+ of \d+ in$/).first().click();
  await expect('Assign');
  await audit(p, 'an opened activity card');
  await p.getByRole('tab', { name: /Vote/ }).click();
  await expect(/^Lock in /);
  await audit(p, 'the Vote tab');
  await open('itinerary');
  await p.getByLabel('Add activity').first().click();
  await p.getByTestId('activity-name').fill('vincenzo');
```

```
  await expect(/^Da Vincenzo, /);
  await p.getByLabel('More options').click();
  await expect('Address (optional)');
  await audit(p, 'the Add Activity sheet with matches and more options');
  await open('itinerary');
  await p.getByLabel('Show filters').click();
  await audit(p, 'the Plan with filters open');
  await open('settings');
  await p.getByLabel('Advanced settings').click();
  await expect('Test connection');
  await audit(p, 'Settings with Advanced open');
  await open('settings');
  await p.getByLabel('Arrange blocks').click();
  await expect('Move Weather up');
  await audit(p, 'Settings with the Home blocks open');
  await open('personalize');
  await expect('Let’s go');
  await audit(p, 'Make it yours: welcome');
  for (const [button, where] of [['Let’s go', 'look'], ['Continue', 'interests'], ['Continue', 'layout'], ['Continue', 'vibe']]) {
    await p.getByRole('button', { name: button, exact: true }).click();
    await p.waitForTimeout(500);
    await audit(p, `Make it yours: ${where}`);
  }
  await p.getByLabel('Skip to the end').click();
  await expect('Apply my style');
  await audit(p, 'the last page of Make it yours');
  await open('survey');
  for (let i = 0; i < 5; i++) { await p.getByTestId('survey-option').first().click(); await p.waitForTimeout(250); }
  await p.getByText('Your matches').first().waitFor({ state: 'visible', timeout: 5000 });
  await audit(p, 'the survey matches');
  // Budget: the sheets and the folded tools
  await open('budget');
  await p.getByTestId('add-expense-open').click();
  await expect('Change who paid, the split or the category');
  await audit(p, 'the Add expense sheet');
  await p.getByLabel('Change who paid, the split or the category').click();
  await expect('Split just for me');
  await audit(p, 'the Add expense sheet with the details open');
  await open('budget');
  await p.getByRole('button', { name: 'Settle up', exact: true }).first().click();
  await expect('Mark paid');
  await audit(p, 'the Settle up sheet');
  await open('budget');
  await p.getByLabel(/^Villa Margherita/).click();
  await expect('Delete expense');
  await audit(p, 'an expense opened on Budget');
  await open('budget');
  await p.getByTestId('budget-cap-open').click();
  await expect('Save budget');
  await audit(p, 'the budget sheet');
  await open('budget');
  await p.getByLabel('More budget tools').click();
  await expect('Swap currencies');
  await audit(p, 'Budget with the converter and planner open');
  await p.getByLabel(/^Change from currency/).click();
  await expect('Search currency');
  await audit(p, 'the currency picker');

  console.log(failures.length ? failures.join('\n') : `PASS  no button or link sits inside another one (${screens} screens and states ch
ecked)`);

  // React itself only complains in a development build (point E2E_URL at `npx expo start --web`), so this part is skipped on the export
ed app
  if (await p.evaluate(() => typeof __DEV__ !== 'undefined' && __DEV__)) {
    const warned = errors.filter((e) => /cannot contain a nested|cannot be a descendant of/.test(e));
    console.log(warned.length ? warned.map((w) => `FAIL  React warned: ${w.replace(/\s+/g, ' ').slice(0, 160)}`).join('\n') : 'PASS  Rea
ct printed no invalid-nesting warnings');
  }

  // prove the check can fail: plant a button inside a button and look again
  await p.evaluate(() => {
    const outer = document.createElement('button'); outer.setAttribute('aria-label', 'planted outer');
    const inner = document.createElement('button'); inner.setAttribute('aria-label', 'planted inner');
    outer.appendChild(inner); document.body.appendChild(outer);
  });
  const seen = await nestedControls(p);
  console.log(`${seen.some((s) => s.includes('planted outer') && s.includes('planted inner')) ? 'PASS' : 'FAIL'}  the check notices a bu
tton planted inside a button`);
  await b.close();
})().catch((e) => { console.error('CRASH', e.message.slice(0, 500)); process.exit(1); });
```

e2e/overflow.js

```
// Visits every screen at two phone widths and reports anything that sticks out sideways (the "page scrolls horizontally" bug).
const { BASE, start, ROUTES: all } = require('./lib');
// E2E_ONLY=settings,profile checks just those screens (handy while working on one)
```

```
const routes = process.env.E2E_ONLY ? all.filter((r) => process.env.E2E_ONLY.split(',').includes(r)) : all;
const passes = [{ w: 390, scale: 1 }, { w: 320, scale: 1 }, { w: 360, scale: 1.25 }]; // the last one is the "larger text" setting
const SKIP = new Set(['panorama']); // the 360° scene is meant to be wider than the screen

(async () => {
  const { b, p } = await start();
  let bad = 0;
  for (const { w, scale } of passes) {
    await p.setViewportSize({ width: w, height: 760 });
    await p.evaluate((sc) => { const j = JSON.parse(localStorage.getItem('togetherwego-v1')); j.state.settings.textScale = sc; localStor
age.setItem('togetherwego-v1', JSON.stringify(j)); }, scale);
    for (const r of routes) {
      if (SKIP.has(r)) continue;
      await p.goto(BASE + '/' + r, { waitUntil: 'load' });
      await p.waitForTimeout(1300);
      if (await p.getByRole('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(()
=> {});
      const res = await p.evaluate((vw) => {
        // not window.innerWidth: a mobile browser widens its layout viewport to fit whatever sticks out
        const doc = document.scrollingElement || document.documentElement;
        const scrollers = [...document.querySelectorAll('*')].filter((el) => /(auto|scroll)/.test(getComputedStyle(el).overflowX));
        const inScroller = (el) => scrollers.some((s) => s !== el && s.contains(el));
        const clipped = (el) => { for (let n = el.parentElement; n; n = n.parentElement) { const cs = getComputedStyle(n); if ((cs.overf
lowX === 'hidden' || cs.overflow === 'hidden') && n.getBoundingClientRect().right <= vw + 1) return true; } return false; };
        const out = [];
        for (const el of document.querySelectorAll('body *')) {
          const rc = el.getBoundingClientRect();
          if (rc.width < 2 || rc.right <= vw + 1 || getComputedStyle(el).visibility === 'hidden' || inScroller(el) || clipped(el)) conti
nue;
          out.push(`${el.tagName.toLowerCase()}[${(el.getAttribute('aria-label') || el.textContent || '').trim().slice(0, 30)}] w=${Mat
h.round(rc.width)} right=${Math.round(rc.right)}`);
          if (out.length >= 3) break;
        }
        return { pageScroll: doc.scrollWidth > vw + 1, scrollWidth: doc.scrollWidth, vw, out };
      }, w);
      if (res.pageScroll || res.out.length) { bad++; console.log(`FAIL  ${w}px x${scale} /${r}  page ${res.scrollWidth}>${res.vw}: ${re
s.pageScroll}  ${res.out.join(' | ')}`); }
    }
  }
  console.log(bad ? `${bad} screens overflow` : 'no screen overflows at 390 px, 320 px or with larger text');
  await b.close();
})().catch((e) => { console.error('CRASH', e.message.slice(0, 500)); process.exit(1); });
```

e2e/places.js

```
// "Where do you want to go?": type a place, see matches with how far each is from you, pick one.
// The place-search service is answered by this script (no internet needed) and the browser is placed in New Jersey,
// so the distances are real arithmetic: Sorrento is about 6,700 km from there.
const { BASE, start } = require('./lib');
const results = [];
const check = (name, ok, extra = '') => { results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`); };
const vis = async (loc, t = 4000) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);

const NEW_JERSEY = { latitude: 40.7357, longitude: -74.1724 };
const feature = (props, lng, lat) => ({ type: 'Feature', geometry: { type: 'Point', coordinates: [lng, lat] }, properties: props });
const MUSEUM = feature({ osm_type: 'N', osm_id: 11, osm_key: 'tourism', osm_value: 'museum', name: 'Museo Correale di Terranova', stree
t: 'Via Correale', city: 'Sorrento' }, 14.3809, 40.6288);
const json = (features) => ({ status: 200, contentType: 'application/json', headers: { 'access-control-allow-origin': '*' }, body: JSON.
stringify({ type: 'FeatureCollection', features }) });

const openAdd = async (p) => {
  await p.goto(BASE + '/itinerary', { waitUntil: 'load' });
  await p.waitForTimeout(1500);
  await p.getByLabel('Add activity').first().click();
  await p.getByTestId('activity-name').waitFor({ state: 'visible', timeout: 4000 });
  await p.waitForTimeout(1500); // the phone's location arrives
};

(async () => {
  const { b, p } = await start({ context: { geolocation: NEW_JERSEY, permissions: ['geolocation'] } });
  const asked = [];
  await p.route('https://photon.komoot.io/**', (route) => {
    const url = new URL(route.request().url());
    asked.push(url);
    route.fulfill(json(url.searchParams.get('q').toLowerCase().includes('museum') ? [MUSEUM] : []));
  });

  // ---- an online match, with its distance from the phone
  await openAdd(p);
  await p.getByTestId('activity-name').fill('The museum');
  check('an online place appears below the box', await vis(p.getByLabel(/Museo Correale di Terranova/)));
  const row = await p.getByLabel(/Museo Correale di Terranova/).first().getAttribute('aria-label');
  check('it says how far it is from you', /7,\d{3} km from you/.test(row || ''), row);
  check('"The" is dropped from what is sent', asked.some((u) => u.searchParams.get('q') === 'museum'), asked.map((u) => u.search).join('
| '));
```

```
  const first = asked.find((u) => u.searchParams.get('q') === 'museum');
  check('it searches around the trip, with a rounded point and a box', !!first && first.searchParams.get('lat') === '40.63' && first.sea
rchParams.get('lon') === '14.49' && /^14\.\d+,40\.\d+,14\.\d+,40\.\d+$/.test(first.searchParams.get('bbox') || ''), first && first.searc
h);
  check('you can always use what you typed', await vis(p.getByLabel('Use The museum as typed')));

  // ---- a saved place shows at once and fills in the form
  await p.getByTestId('activity-name').fill('vincenzo');
  check('a saved place matches', await vis(p.getByLabel(/^Da Vincenzo, .*from you/)));
  await p.getByLabel(/^Da Vincenzo, .*from you/).first().click();
  check('picking a place shows its address and distance', await vis(p.getByText(/Viale Pasitea, Positano · [\d,]+ km from you/)));
  check('the box now holds the place name', (await p.getByTestId('activity-name').inputValue()) === 'Da Vincenzo');
  await p.getByTestId('add-activity-submit').click(); await p.waitForTimeout(900);
  check('the picked place is added to the plan', await vis(p.getByLabel(/^Da Vincenzo, /)));
  await p.getByLabel(/^Da Vincenzo, /).click(); await p.waitForTimeout(400); // open the new stop to reach Delete
  await p.getByLabel('Delete Da Vincenzo').click(); await p.waitForTimeout(400);
  await p.getByRole('button', { name: 'Delete', exact: true }).last().click(); await p.waitForTimeout(500);

  // ---- no connection: saved places still work and the box says so
  await p.unroute('https://photon.komoot.io/**');
  await p.route('https://photon.komoot.io/**', (route) => route.abort());
  await openAdd(p);
  await p.getByTestId('activity-name').fill('santa maria');
  check('offline: a saved place still shows', await vis(p.getByLabel(/^Santa Maria Assunta, /)));
  check('offline: the box says only saved places are shown', await vis(p.getByText(/No connection, so only saved places are shown/)));
  await p.getByTestId('activity-name').fill('Sunset picnic');
  await p.getByLabel('Use Sunset picnic as typed').click(); await p.waitForTimeout(300);
  check('using the typed words hides the list', !(await vis(p.getByLabel('Use Sunset picnic as typed'), 600)));
  await p.getByTestId('add-activity-submit').click(); await p.waitForTimeout(900);
  check('a place typed by hand is added', await vis(p.getByText('Sunset picnic')));
  await p.getByLabel(/^Sunset picnic, /).click(); await p.waitForTimeout(400);
  await p.getByLabel('Delete Sunset picnic').click(); await p.waitForTimeout(400);
  await p.getByRole('button', { name: 'Delete', exact: true }).last().click(); await p.waitForTimeout(500);
  await b.close();

  // ---- location not shared: distances start from the middle of the trip's city
  const denied = await start({ context: { permissions: [] } });
  await denied.p.route('https://photon.komoot.io/**', (route) => route.fulfill(json([])));
  await openAdd(denied.p);
  await denied.p.getByTestId('activity-name').fill('vincenzo');
  check('without location, distances start from the trip city', await vis(denied.p.getByLabel(/^Da Vincenzo, .*from Positano/), 12000));
  await denied.b.close();

  console.log(results.join('\n'));
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 600)); process.exit(1); });
```

e2e/run.js

```
// Runs the browser tests: exports the web app, serves it, runs each script, prints a summary.
//   node e2e/run.js                 build + run everything
//   node e2e/run.js --no-build      reuse the last build (e2e/.dist)
//   node e2e/run.js verify twophones   only these scripts
// Needs a browser: run `npx playwright-core install chromium` once, or set E2E_CHROME to a Chrome path.
const { spawn, spawnSync } = require('child_process');
const fs = require('fs');
const path = require('path');
const { serve } = require('./serve');

const root = path.join(__dirname, '..');
// E2E_DIST and E2E_PORT let several people (or agents) build and test side by side without sharing a folder or a port
const dist = process.env.E2E_DIST ? path.resolve(process.env.E2E_DIST) : path.join(__dirname, '.dist');
const port = Number(process.env.E2E_PORT) || 8081;
const args = process.argv.slice(2);
const wanted = args.filter((a) => !a.startsWith('--'));
const ALL = ['verify', 'verify2', 'verify3', 'verify4', 'places', 'motion', 'overflow', 'nesting', 'targets', 'auth', 'twophones'];
const LIMIT_MS = { overflow: 600000, targets: 400000, twophones: 400000 }; // anything else gets 5 minutes

function run(script) {
  return new Promise((resolve) => {
    const child = spawn(process.execPath, [path.join(__dirname, `${script}.js`)], { cwd: __dirname, env: { ...process.env, E2E_URL: `htt
p://localhost:${port}` } });
    let out = '';
    child.stdout.on('data', (d) => (out += d));
    child.stderr.on('data', (d) => (out += d));
    let timedOut = false;
    const timer = setTimeout(() => {
      timedOut = true;
      child.kill();
    }, LIMIT_MS[script] ?? 300000);
    child.on('close', () => {
      clearTimeout(timer);
      resolve(timedOut ? `${out}\nCRASH ${script} did not finish in time` : out);
    });
  });
}
```

```
(async () => {
  fs.mkdirSync(path.join(__dirname, 'shots'), { recursive: true });
  if (!args.includes('--no-build')) {
    console.log('Exporting the web app…');
    const npx = process.platform === 'win32' ? 'npx.cmd' : 'npx';
    const r = spawnSync(npx, ['expo', 'export', '--platform', 'web', '--output-dir', dist], { cwd: root, stdio: 'inherit', shell: proces
s.platform === 'win32' });
    if (r.status !== 0) process.exit(1);
  }
  const server = await serve(dist, port);
  let pass = 0;
  let fail = 0;
  let crashed = 0;
  for (const script of ALL.filter((s) => !wanted.length || wanted.includes(s))) {
    console.log(`\n=== ${script}`);
    const out = await run(script);
    console.log(out.trim());
    pass += (out.match(/^PASS /gm) || []).length;
    fail += (out.match(/^FAIL /gm) || []).length;
    crashed += /^CRASH/m.test(out) ? 1 : 0;
  }
  server.close();
  console.log(`\n${pass} passed, ${fail} failed${crashed ? `, ${crashed} script(s) crashed` : ''}`);
  process.exit(fail || crashed ? 1 : 0);
})();
```

e2e/serve.js

```
// A tiny static file server for the exported web app (every unknown path falls back to index.html, like a single-page app host).
const http = require('http');
const fs = require('fs');
const path = require('path');

const types = { '.html': 'text/html', '.js': 'application/javascript', '.css': 'text/css', '.png': 'image/png', '.jpg': 'image/jpeg', '.
ttf': 'font/ttf', '.json': 'application/json', '.ico': 'image/x-icon', '.svg': 'image/svg+xml' };

exports.serve = (root, port) =>
  new Promise((resolve) => {
    const server = http.createServer((req, res) => {
      let file = path.join(root, decodeURIComponent(req.url.split('?')[0]));
      if (!file.startsWith(root) || !fs.existsSync(file) || fs.statSync(file).isDirectory()) file = path.join(root, 'index.html');
      res.writeHead(200, { 'Content-Type': types[path.extname(file)] || 'application/octet-stream' });
      fs.createReadStream(file).pipe(res);
    });
    server.listen(port, () => resolve(server));
  });
```

e2e/shoot.js

```
// Takes pictures of screens so the design can be looked at (by a person or by Claude reading the PNGs).
//
//   node e2e/shoot.js --routes=explore,itinerary --modes=light,dark --themes=amalfi --out=/tmp/shots
//
//   --routes   screens to shoot, comma separated (default: the five tabs). "explore" is Home; "first" is the very first screen (use it
with --signed-out). Add :tap=Label to click a button first,
//              for example  itinerary:tap=Add activity   (the click is on whatever has that accessible name).
//   --themes   amalfi,sunset,lagoon,rose,midnight,classic (default amalfi)
//   --modes    light,dark (default light); the browser's own colour scheme is set to match, and the app is told to follow it
//   --out      folder for the PNGs (default e2e/shots)
//   --dist     the exported web app to serve (default e2e/.dist); build it with: npx expo export --platform web --output-dir e2e/.dist
//   --pages    shots per screen, scrolling down between them (default 1)
//   --width / --height   the phone, default 390 x 844
//   --scale    text size setting, 1 / 1.12 / 1.25 (default 1)
//   --keep-tour   leave the first-run tour open
//   --jpg         write small JPEG files (for documentation) instead of PNG
//   --signed-out  do not sign in: shoot the first-run slides and sign-in
const fs = require('fs');
const path = require('path');
const { chromium } = require('playwright-core');
const { serve } = require('./serve');

const arg = (name, fallback) => {
  const hit = process.argv.find((a) => a.startsWith(`--${name}=`));
  return hit ? hit.slice(name.length + 3) : process.argv.includes(`--${name}`) ? 'true' : fallback;
};
const list = (name, fallback) => String(arg(name, fallback)).split(',').filter(Boolean);

const dist = path.resolve(arg('dist', path.join(__dirname, '.dist')));
const out = path.resolve(arg('out', path.join(__dirname, 'shots')));
const routes = list('routes', 'explore,itinerary,budget,chat,profile');
const themes = list('themes', 'amalfi');
const modes = list('modes', 'light');
const pages = Number(arg('pages', '1'));
const width = Number(arg('width', '390'));
const height = Number(arg('height', '844'));
const scale = Number(arg('scale', '1'));
```

```
const jpg = process.argv.includes('--jpg');
const signedOut = process.argv.includes('--signed-out');

(async () => {
  fs.mkdirSync(out, { recursive: true });
  const server = await serve(dist, 0);
  const base = `http://localhost:${server.address().port}`;
  const browser = await chromium.launch(process.env.E2E_CHROME ? { executablePath: process.env.E2E_CHROME } : {});
  for (const theme of themes) {
    for (const mode of modes) {
      const ctx = await browser.newContext({ viewport: { width, height }, deviceScaleFactor: jpg ? 1.5 : 2, isMobile: true, hasTouch: tr
ue, colorScheme: mode });
      await ctx.addInitScript(([t, m]) => localStorage.setItem('togetherwego-appearance', JSON.stringify({ theme: t, corners: 'rounded',
mode: m })), [theme, mode]);
      const p = await ctx.newPage();
      const errors = [];
      p.on('console', (m) => m.type() === 'error' && !m.text().includes('ERR_CERT') && !m.location().url.includes(':8787') && errors.pus
h(m.text().slice(0, 300)));
      p.on('pageerror', (e) => errors.push('PAGEERROR ' + e.message.slice(0, 300)));
      await p.goto(base + '/', { waitUntil: 'networkidle' });
      await p.waitForTimeout(1800);
      if (!signedOut) {
        if (await p.getByRole('button', { name: 'Skip' }).count()) { await p.getByRole('button', { name: 'Skip' }).first().click(); awai
t p.waitForTimeout(800); }
        await p.getByLabel('Use the demo account').click();
        await p.getByTestId('sign-in').click();
        await p.waitForTimeout(2800);
        if (scale !== 1) await p.evaluate((sc) => { const j = JSON.parse(localStorage.getItem('togetherwego-v1')); j.state.settings.text
Scale = sc; localStorage.setItem('togetherwego-v1', JSON.stringify(j)); }, scale);
        if (!process.argv.includes('--keep-tour') && (await p.getByRole('button', { name: 'Skip' }).count())) { await p.getByRole('butto
n', { name: 'Skip' }).first().click(); await p.waitForTimeout(600); }
      }
      for (const spec of routes) {
        const [route, ...opts] = spec.split(':');
        await p.goto(route === 'first' ? `${base}/` : `${base}/${route}`, { waitUntil: 'load' });
        await p.waitForTimeout(1800);
        for (const o of opts) {
          if (o.startsWith('tap=')) { await p.getByLabel(o.slice(4)).first().click(); await p.waitForTimeout(900); }
        }
        const name = `${theme}-${mode}-${route.replace(/\//g, '_')}${opts.length ? '-' + opts.join('-').replace(/\W+/g, '_') : ''}`;
        for (let i = 0; i < pages; i++) {
          await p.screenshot(jpg ? { path: path.join(out, `${name}${pages > 1 ? '-' + (i + 1) : ''}.jpg`), type: 'jpeg', quality: 82 } :
{ path: path.join(out, `${name}${pages > 1 ? '-' + (i + 1) : ''}.png`) });
          if (i < pages - 1) {
            const moved = await p.evaluate(() => {
              const el = [...document.querySelectorAll('*')].find((e) => /(auto|scroll)/.test(getComputedStyle(e).overflowY) && e.scroll
Height > e.clientHeight + 40);
              if (!el) return false;
              el.scrollBy(0, el.clientHeight * 0.85);
              return true;
            });
            if (!moved) break;
            await p.waitForTimeout(600);
          }
        }
      }
      if (errors.length) console.log(`console errors for ${theme}/${mode}:\n  ` + [...new Set(errors)].join('\n  '));
      await ctx.close();
    }
  }
  await browser.close();
  server.close();
  console.log(`pictures are in ${out}`);
})().catch((e) => {
  console.error('CRASH', e.message.slice(0, 600));
  process.exit(1);
});
```

e2e/targets.js

```
// Fitts's law: everything you tap should be big enough to hit. Visits every screen and lists each control smaller than 44 x 44 points.
// (A phone's own switch keeps the size the platform gives it, so switches are not counted.)
const { BASE, start, ROUTES } = require('./lib');
const MIN = 44;
// the 360° screen is not in the list of every screen (the overflow test skips it) but its controls must be big enough too
const all = [...ROUTES, 'panorama'];
const routes = process.env.E2E_ONLY ? all.filter((r) => process.env.E2E_ONLY.split(',').includes(r)) : all;

const audit = (min) => {
  const SEL = 'button, a[href], [role=button], [role=link], [role=tab], [role=radio], [role=menuitem], input:not([type=hidden]):not([rol
e=switch]):not([type=checkbox]), textarea, select';
  const seen = new Set();
  const out = [];
  let count = 0;
  for (const el of document.querySelectorAll(SEL)) {
    const rc = el.getBoundingClientRect();
```

```
    const cs = getComputedStyle(el);
    if (rc.width < 1 || rc.height < 1 || cs.visibility === 'hidden' || cs.display === 'none' || Number(cs.opacity) === 0) continue;
    // a control inside a scroller that is clipped away entirely is not on screen
    if (el.closest('[aria-hidden=true]')) continue;
    count++;
    if (Math.round(rc.width) >= min && Math.round(rc.height) >= min) continue;
    const name = (el.getAttribute('aria-label') || el.textContent || el.tagName).trim().slice(0, 36);
    const key = `${name}|${Math.round(rc.width)}x${Math.round(rc.height)}`;
    if (seen.has(key)) continue;
    seen.add(key);
    out.push(`"${name}" is ${Math.round(rc.width)} x ${Math.round(rc.height)}`);
  }
  return { count, out };
};

(async () => {
  const { b, p } = await start();
  let bad = 0;
  let total = 0;
  for (const r of routes) {
    await p.goto(BASE + '/' + r, { waitUntil: 'load' });
    await p.waitForTimeout(1300);
    if (await p.getByRole('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(() =>
{});
    const res = await p.evaluate(audit, MIN);
    total += res.count;
    if (res.out.length) {
      bad++;
      console.log(`FAIL  /${r}  ${res.out.slice(0, 6).join(' | ')}${res.out.length > 6 ? ` (+${res.out.length - 6} more)` : ''}`);
    }
  }
  console.log(bad ? `${bad} screens have controls smaller than ${MIN} x ${MIN}` : `PASS  all ${total} controls on ${routes.length} scree
ns are at least ${MIN} x ${MIN}`);
  await b.close();
})().catch((e) => { console.error('CRASH', e.message.slice(0, 500)); process.exit(1); });
```

e2e/twophones.js

```
// Two separate "phones" (two browser profiles) + one real sync server on :8787.
const { spawn } = require('child_process');
const fs = require('fs');
const path = require('path');
const { BASE, fixture, shot, start } = require('./lib');
const jsQR = require('jsqr');

const results = [];
const check = (name, ok, extra = '') => results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`);
const vis = async (loc, t = 6000) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);
const gone = async (loc, t = 800) => !(await vis(loc, t));
const ROOT = path.join(__dirname, '..');
const DATA = '/tmp/twophones-sync.json';
let server;
const url = (r = '') => BASE + '/' + r;
/** With DOCS_SHOTS=1 the pictures used in the README are rewritten (docs/screenshots). */
const docShot = (page, name) => (process.env.DOCS_SHOTS ? page.screenshot({ path: path.join(ROOT, 'docs', 'screenshots', name), type: 'j
peg', quality: 80 }) : null);

const go = async (p, r) => {
  await p.goto(url(r), { waitUntil: 'load' }); // not 'networkidle': a synced trip always has one long-poll request open
  await p.waitForTimeout(2200);
  if (await p.getByRole('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(() =>
{});
};

async function health() {
  try {
    const r = await fetch('http://localhost:8787/health');
    return r.ok ? await r.json() : null;
  } catch {
    return null;
  }
}

/** Reads the QR code that is showing on a page the way a phone camera would: draw the SVG, decode the pixels. */
async function decodeQr(p) {
  const pixels = await p.evaluate(async () => {
    const svg = document.querySelector('svg[aria-label^="QR code"]');
    if (!svg) return null;
    const img = new Image();
    img.src = '' + btoa(new XMLSerializer().serializeToString(svg));
    await img.decode();
    const canvas = document.createElement('canvas');
    canvas.width = canvas.height = 600;
    const ctx = canvas.getContext('2d');
    ctx.fillStyle = '#fff';
```

```
    ctx.fillRect(0, 0, 600, 600);
    ctx.drawImage(img, 0, 0, 600, 600);
    return Array.from(ctx.getImageData(0, 0, 600, 600).data);
  });
  return pixels ? jsQR(Uint8ClampedArray.from(pixels), 600, 600)?.data ?? null : null;
}

async function say(p, text) {
  await p.getByTestId('chat-input').fill(text);
  await p.getByTestId('chat-send').click();
  await p.waitForTimeout(300);
}

(async () => {
  if (await health()) throw new Error('Something is already listening on :8787 - stop it first so the test uses a fresh server');
  try { fs.unlinkSync(DATA); } catch {}
  server = spawn('node', ['server/index.js'], { cwd: ROOT, env: { ...process.env, PORT: '8787', SYNC_DATA: DATA }, stdio: 'ignore' });
  for (let i = 0; i < 30 && !(await health()); i++) await new Promise((r) => setTimeout(r, 200));
  check('sync server is up', !!(await health()));

  // ---------------------------------------------------------------- phone A: the demo account, shares the trip
  const A = await start(); // signs in as the demo account
  const pa = A.p;
  const errorsA = A.errors;
  await go(pa, 'crew');
  check('A: crew has the sync card', await vis(pa.getByText('Sync across phones')));
  await pa.getByRole('button', { name: 'Share this trip across phones' }).click();
  check('A: sharing toast', await vis(pa.getByText(/Sharing! Friends can now join/)));
  check('A: badge goes Live', await vis(pa.getByText(/^Live/)));
  check('server now holds one trip', (await health())?.trips === 1);

  // ---------------------------------------------------------------- phone B: a brand-new account
  const ctxB = await A.b.newContext({ viewport: { width: 390, height: 844 }, deviceScaleFactor: 2, isMobile: true, hasTouch: true });
  const pb = await ctxB.newPage();
  const errorsB = [];
  pb.on('console', (m) => { if (m.type() === 'error') errorsB.push(m.text().slice(0, 300)); });
  pb.on('pageerror', (e) => errorsB.push('PAGEERROR ' + e.message.slice(0, 300)));
  await pb.goto(url(), { waitUntil: 'load' });
  await pb.waitForTimeout(2000);
  await pb.getByRole('button', { name: 'Skip' }).click();
  await pb.waitForTimeout(700);
  await pb.getByRole('link', { name: 'Sign Up' }).click();
  await pb.waitForTimeout(900);
  await pb.getByPlaceholder('e.g. Alex Rivera').fill('Jordan Smith');
  await pb.getByPlaceholder('you@email.com').fill('jordan.two@school.org');
  await pb.getByPlaceholder('At least 8 characters').fill('Trip2027go');
  await pb.getByPlaceholder('Re-enter password').fill('Trip2027go');
  await pb.getByRole('checkbox').click();
  await pb.getByRole('button', { name: 'Create account' }).click();
  await pb.waitForTimeout(2500);
  await pb.getByLabel('Skip to the end').click().catch(() => {});
  await pb.waitForTimeout(500);
  await pb.getByRole('button', { name: 'Apply my style' }).click().catch(() => {});
  await pb.waitForTimeout(2500);
  check('B: has no trip yet', await vis(pb.getByText(/Plan your first group trip/), 4000));

  // A shows the invite QR code; B "scans" it (same decoding a phone camera does) and opens the link
  await pa.getByTestId('invite-qr').click();
  await pa.waitForTimeout(800);
  const link = await decodeQr(pa);
  check('A: the QR code decodes to the invite link', link === `${BASE}/join?code=AML-2026`, String(link));
  await docShot(pa, '23-invite-qr.jpg');
  await pa.keyboard.press('Escape');

  // wrong code first
  await go(pb, 'join');
  await pb.getByTestId('invite-code').fill('ZZZ-9999');
  await pb.getByTestId('join-submit').click();
  check('B: unknown code says no', await vis(pb.getByText(/No trip found with that code/)));
  // an invite that names a sync server: the person is told before it is used
  await pb.goto(`${link}&server=${encodeURIComponent('http://192.168.1.20:8787')}`, { waitUntil: 'load' });
  await pb.waitForTimeout(1500);
  check('B: the link fills in the code', (await pb.getByTestId('invite-code').inputValue()) === 'AML-2026');
  check('B: a named sync server is announced', await vis(pb.getByText(/sync server at 192\.168\.1\.20:8787/)));
  await pb.goto(link, { waitUntil: 'load' });
  await pb.waitForTimeout(1500);
  check('B: opening the link fills in the code', (await pb.getByTestId('invite-code').inputValue()) === 'AML-2026');
  await pb.getByTestId('join-submit').click();
  check('B: join toast says it syncs live', await vis(pb.getByText(/now syncs live/)));
  await pb.waitForTimeout(1200);

  // B sees A's trip
  await go(pb, 'chat');
  check('B: sees A’s earlier chat messages', await vis(pb.getByText('Villa is booked! 🏡 Paid the deposit.')));
  check('B: its own join message/member is there', await vis(pb.getByText(/Jordan/)));
  if (process.env.DEBUG) {
```

```
    for (const t of [0, 4000, 12000]) {
      await pb.waitForTimeout(t);
      const badge = async (p) => (await p.locator('body').innerText()).split('\n').filter((l) => /Live|Offline|Connecting|phones/.test
(l)).join(' | ');
      console.log(`t+${t}: A[${await badge(pa)}] B[${await badge(pb)}]`);
    }
  }
  check('B: Live · 2 phones', await vis(pb.getByText(/Live · 2 phones/)));
  check('A: Live · 2 phones', await (async () => { await go(pa, 'chat'); return vis(pa.getByText(/Live · 2 phones/)); })());

  // ------------------------------------------------------------------ live chat both ways
  await say(pa, 'Hello from phone A');
  check('chat A→B', await vis(pb.getByText('Hello from phone A')));
  await say(pb, 'Hi from phone B');
  check('chat B→A', await vis(pa.getByText('Hi from phone B')));
  check('no fake friend replies in a synced trip', await gone(pa.getByText(/typing/i), 1500));

  // ------------------------------------------------------------------ expense B→A
  await go(pb, 'budget');
  await pb.getByTestId('add-expense-open').click();
  await pb.waitForTimeout(600);
  await pb.getByTestId('expense-title').fill('Taxi to the pier');
  await pb.getByTestId('expense-amount').fill('48');
  await pb.getByTestId('add-expense').click();
  await pb.waitForTimeout(800);
  await go(pa, 'budget');
  check('expense B→A', await vis(pa.getByText('Taxi to the pier')));

  // ------------------------------------------------------------------ files travel too: photo, receipt, document
  const choose = async (p, locator, file) => {
    const [fc] = await Promise.all([p.waitForEvent('filechooser', { timeout: 5000 }).catch(() => null), locator.click()]);
    if (fc) await fc.setFiles(file);
    return !!fc;
  };
  const until = async (fn, ms = 15000) => { const end = Date.now() + ms; while (Date.now() < end) { if (await fn().catch(() => false)) r
eturn true; await new Promise((r) => setTimeout(r, 300)); } return false; };

  await go(pa, 'photos');
  check('A: file chooser opens for photos', await choose(pa, pa.getByRole('button', { name: /Add photos/ }), fixture('receipt.jpg')));
  await pa.waitForTimeout(800);
  await pa.getByPlaceholder('Say something about it…').fill('Sunset from the villa');
  await pa.getByRole('button', { name: /Post to photo dump/ }).click();
  check('A: photo posted', await vis(pa.getByLabel('Sunset from the villa')));
  check('server received the photo file', await until(async () => (await health())?.files >= 1));
  await go(pb, 'photos');
  check('B: sees the photo tile', await vis(pb.getByLabel('Sunset from the villa')));
  check('B: the picture itself arrives', await until(async () => (await pb.locator('[aria-label="Sunset from the villa"] img').count())
> 0));
  await pb.screenshot({ path: shot('two-B-photos.png') });
  await docShot(pb, '24-sync-photos.jpg');

  await go(pb, 'budget');
  await pb.getByTestId('add-expense-open').click();
  await pb.waitForTimeout(600);
  await pb.getByTestId('expense-title').fill('Pier ferry');
  await pb.getByTestId('expense-amount').fill('36');
  check('B: file chooser opens for a receipt', await choose(pb, pb.getByRole('button', { name: /Attach receipt/ }), fixture('receipt.jp
g')));
  await pb.waitForTimeout(900);
  await pb.getByTestId('add-expense').click();
  await pb.waitForTimeout(800);
  check('server received the receipt too', await until(async () => (await health())?.files >= 2));
  await go(pa, 'budget');
  await pa.getByText('Pier ferry').first().click();
  const gotReceipt = await until(async () => (await pa.locator('[aria-label="Enlarge receipt"] img').count()) > 0);
  check('A: receipt photo arrives', gotReceipt);
  if (!gotReceipt) console.log('DEBUG has-receipt icons:', await pa.getByLabel('Has receipt').count(), '\n', (await pa.locator('body').i
nnerText()).replace(/\n+/g, ' | ').slice(-900));
  await pa.screenshot({ path: shot('two-A-receipt.png') });

  await go(pa, 'documents');
  await pa.getByRole('button', { name: 'Add a document' }).click(); await pa.waitForTimeout(700); // the add button opens a sheet with F
ile, Photo and Note
  check('A: file chooser opens for a document', await choose(pa, pa.getByRole('button', { name: 'File', exact: true }), fixture('tiny.pd
f')));
  check('A: document saved', await vis(pa.getByText(/Saved to the vault/)));
  check('server received the document', await until(async () => (await health())?.files >= 3));
  await go(pb, 'documents');
  check('B: sees the document', await vis(pb.getByLabel('Open tiny')));
  check('B: its pill says Offline once the file arrived', await until(async () => (await pb.getByLabel('Open tiny').getByText('Offlin
e').count()) > 0));
  await pb.getByLabel('Open tiny').click();
  check('B: can open or share the file', await vis(pb.getByRole('button', { name: 'Open / share file' })));
  await pb.keyboard.press('Escape');

  // ------------------------------------------------------------------ A goes offline, writes, restarts, comes back
```

```
  await go(pa, 'chat');
  await go(pb, 'chat');
  await A.b.contexts()[0].setOffline(true);
  await say(pa, 'Written with no signal');
  check('A: offline pill with 1 waiting', await vis(pa.getByText(/Offline · \d+ waiting|Offline/), 12000));
  check('B does not have it yet', await gone(pb.getByText('Written with no signal'), 1500));
  await pa.waitForTimeout(1200); // let the debounced save reach storage
  await A.b.contexts()[0].setOffline(false);
  check('B gets it after A is back', await vis(pb.getByText('Written with no signal'), 25000));
  check('A badge is Live again', await vis(pa.getByText(/^Live/), 15000));

  // ---------------------------------------------------------------- B closes the app (reload) and writes while closed state is restore
d
  await pb.reload({ waitUntil: 'load' });
  await pb.waitForTimeout(2500);
  await go(pb, 'chat');
  check('B after restart: still synced, same chat', await vis(pb.getByText('Hello from phone A')));
  await say(pa, 'Message while B restarted');
  check('B after restart receives live', await vis(pb.getByText('Message while B restarted'), 12000));

  // ---------------------------------------------------------------- screenshots
  await go(pa, 'chat');
  await pa.waitForTimeout(500);
  await pa.screenshot({ path: shot('two-A-chat.png') });
  await docShot(pa, '22-sync-chat.jpg');
  await pb.screenshot({ path: shot('two-B-chat.png') });
  await go(pa, 'crew');
  await pa.screenshot({ path: shot('two-A-crew.png') });

  console.log(results.join('\n'));
  const e = [...errorsA, ...errorsB].filter((m) => !/ERR_INTERNET_DISCONNECTED|ERR_NETWORK_CHANGED|Failed to load resource/.test(m));
  console.log('errors:', e.length ? e.join('\n') : 'none');
  await A.b.close();
  server.kill('SIGTERM');
})().catch((e) => {
  server?.kill('SIGTERM');
  console.log(results.join('\n'));
  console.error('CRASH', String(e.stack || e).slice(0, 900));
  process.exit(1);
});
```

e2e/verify.js

```
const { fixture, start } = require('./lib');
const results = [];
const check = (name, ok, extra = '') => { results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`); };
const vis = async (loc, t = 2500) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);
const tab = (p, n) => p.getByRole('tab', { name: n, exact: true }).click().then(() => p.waitForTimeout(1100));
(async () => {
  const { b, p, errors } = await start();
  // ---- Home
  // the ticket: "3" on the flip board with "Days to go" beside it; the header reads as one sentence
  check('home ticket countdown words', await vis(p.getByText('Days to go')));
  check('home countdown heading "3 days to go"', await vis(p.getByRole('heading', { name: '3 days to go' })));
  check('home up next ticket', await vis(p.getByText('Arrive at Naples Airport (NAP)')));
  check('home up next countdown stamp', await vis(p.getByLabel(/^in \d+d \d+h$/i)));
  check('home up next maps button', await vis(p.getByLabel('Open in maps')));
  check('home weather board', await vis(p.getByLabel(/(Typical weather|Forecast) in Positano\. Open the weather/)));
  check('home budget amount', await vis(p.getByText('$2,086')));
  check('home budget state stamp', await vis(p.getByLabel('On track', { exact: true })));
  check('home vote row says 2 things', await vis(p.getByLabel(/2 things to vote on/)));
  check('home crew count', await vis(p.getByLabel(/4 going\. Open the crew/)));
  await p.getByLabel(/Showing (night|day)/).click(); await p.waitForTimeout(500);
  check('day/night toggle flips', await vis(p.getByLabel(/Showing (night|day)\. Tap for/)));
  await p.getByLabel(/Invite code AML-2026/).click(); await p.waitForTimeout(500);
  check('invite code copy toast', await vis(p.getByText(/Invite code AML-2026 copied/)));
  await p.getByRole('button', { name: 'Invite', exact: true }).click(); await p.waitForTimeout(500);
  check('crew Invite button copies the code', await vis(p.getByText(/Invite code AML-2026 copied/)));
  // today's trivia: the question changes every day, so answer the first option and look for the verdict and the fact
  await p.getByLabel(/^A\. /).first().scrollIntoViewIfNeeded(); await p.waitForTimeout(300);
  await p.getByLabel(/^A\. /).first().click(); await p.waitForTimeout(700);
  check('trivia shows a verdict', await vis(p.getByText(/^(Correct|Not quite)$/)));
  check('trivia marks the right answer', await vis(p.getByLabel(/\. (Correct|The right answer)$/)));
  check('trivia locks the other answers', await p.getByLabel(/^B\. /).first().isDisabled().catch(() => false));
  // the podium opens the crew; the vote card takes a vote and says so (a no, so the Plan checks below still see 0 of 4)
  check('home podium', await vis(p.getByLabel(/^Crew points\. 1st/)));
  await p.getByLabel('Not for me', { exact: true }).scrollIntoViewIfNeeded(); await p.waitForTimeout(300);
  check('home vote card shows an activity', await vis(p.getByText(/yes votes to lock it in/)));
  await p.getByLabel('Not for me', { exact: true }).click(); await p.waitForTimeout(600);
  check('home vote counted stamp', await vis(p.getByLabel('Vote counted', { exact: true })));
  await p.getByLabel('Customize this page').scrollIntoViewIfNeeded(); await p.waitForTimeout(200);
  check('home customize link', await vis(p.getByLabel('Customize this page')));
  // ---- Plan
  await tab(p, 'Plan');
  check('plan lists confirmed dinner', await vis(p.getByText('Dinner at Da Vincenzo')));
```

```
  check('plan hides voting items', !(await vis(p.getByText('Limoncello tasting'), 600)));
  // a day with nothing confirmed (day 4 only has ideas up for a vote) is an invitation with one Add button, and the floating Add steps
aside
  await p.getByRole('tab', { name: /^Day 4,/ }).click(); await p.waitForTimeout(500);
  check('an empty day invites you to add something', await vis(p.getByText('Nothing planned for Day 4')));
  check('an empty day says ideas wait on the Vote tab', await vis(p.getByText(/waiting on the Vote tab/)));
  check('only one Add button on an empty day', (await p.getByLabel('Add activity').filter({ visible: true }).count()) === 1);
  await p.getByRole('tab', { name: 'All days' }).click(); await p.waitForTimeout(400);
  await p.getByLabel('Show filters').click(); await p.waitForTimeout(300); // the kind and length filters start folded away
  await p.getByRole('button', { name: /Food$/ }).first().click().catch(() => {}); await p.waitForTimeout(400);
  check('category filter hides hike', !(await vis(p.getByText('Path of the Gods hike'), 600)));
  await p.getByRole('button', { name: 'All', exact: true }).first().click(); await p.waitForTimeout(300);
  await p.getByRole('button', { name: /^Under 1 hour/ }).click(); await p.waitForTimeout(400);
  check('time filter under 1 hour keeps the 30-minute check-in', await vis(p.getByText('Check-in — Villa Margherita')));
  check('time filter under 1 hour hides the 2-hour dinner', !(await vis(p.getByText('Dinner at Da Vincenzo'), 600)));
  await p.getByRole('button', { name: /Any length/ }).click(); await p.waitForTimeout(300);
  await p.getByRole('tab', { name: /Vote/ }).click(); await p.waitForTimeout(600);
  check('vote tab lists limoncello', await vis(p.getByText('Limoncello tasting')));
  await p.getByRole('button', { name: /Lock in Limoncello/ }).click(); await p.waitForTimeout(500);
  check('lock in toast', await vis(p.getByText(/Locked in: Limoncello/)));
  check('vote count drops to 1', await vis(p.getByRole('tab', { name: /Vote \(1\)/ })));
  await p.getByLabel(/I’m in\. 0 of 4/).first().click(); await p.waitForTimeout(400);
  check('vote pill toggles to 1/4', await vis(p.getByLabel(/I’m in\. 1 of 4/)));
  await p.getByRole('tab', { name: 'Itinerary' }).click(); await p.waitForTimeout(500);
  check('locked item now in itinerary', await vis(p.getByText('Limoncello tasting')));
  // add activity validation
  await p.getByLabel('Add activity').first().click(); await p.waitForTimeout(800);
  await p.getByTestId('add-activity-submit').click(); await p.waitForTimeout(400);
  check('add-activity validation errors', await vis(p.getByText(/Enter a place/)));
  await p.getByTestId('activity-name').fill('Gelato run');
  // closing the sheet by accident keeps what was typed (a draft); it is there again when the sheet reopens
  await p.getByLabel('Close', { exact: true }).last().click({ position: { x: 20, y: 20 } }); await p.waitForTimeout(700);
  await p.getByLabel('Add activity').first().click(); await p.waitForTimeout(800);
  check('a half-typed activity survives closing the sheet', (await p.getByTestId('activity-name').inputValue()) === 'Gelato run');
  await p.getByTestId('add-activity-submit').click(); await p.waitForTimeout(1000);
  check('activity added', await vis(p.getByText('Gelato run')));
  await p.getByLabel('Add activity').first().click(); await p.waitForTimeout(800);
  check('the draft is cleared once the activity is added', (await p.getByTestId('activity-name').inputValue()) === '');
  await p.getByLabel('Close', { exact: true }).last().click({ position: { x: 20, y: 20 } }); await p.waitForTimeout(700);
  // Delete lives in the opened details of a stop (away from the reminder bell), so open the stop first
  await p.getByLabel(/^Gelato run, /).click(); await p.waitForTimeout(400);
  check('opened stop offers Assign, Directions, Done, Edit and Delete', await vis(p.getByLabel('Assign')) && await vis(p.getByLabel('Dir
ections')) && await vis(p.getByLabel('Edit')) && await vis(p.getByLabel('Delete Gelato run')));
  await p.getByLabel('Delete Gelato run').click(); await p.waitForTimeout(500);
  await p.getByRole('button', { name: 'Delete', exact: true }).last().click(); await p.waitForTimeout(600);
  check('activity deleted', !(await vis(p.getByText('Gelato run'), 600)));
  // reminder bell
  await p.getByLabel(/Turn reminder (on|off)/).first().click().catch(() => {}); await p.waitForTimeout(500);
  check('reminder toast', await vis(p.getByText(/Reminder (on|turned off)/)));
  // ---- Budget
  await tab(p, 'Budget');
  // the summary: money left as the big number on the ink pass, the stamp in words, per person and daily allowance in the stub
  check('budget left to spend', await vis(p.getByTestId('budget-left').filter({ hasText: '$1,914' })));
  check('budget spent line with percent', await vis(p.getByText(/Spent \$2,086 · 52%/i)));
  check('budget per person', await vis(p.getByText('$522')));
  check('budget daily allowance', await vis(p.getByText('$319')));
  // who owes who: a signed amount with a word, and a Settle up button
  check('who owes who: you owe', await vis(p.getByText(/owes \$382/)));
  check('who owes who: treasurer gets back', await vis(p.getByText(/gets back \$1,129/)));
  check('who owes who: row reads as a sentence', await vis(p.getByLabel('Priya gets back $1,129')));
  // expenses: grouped under day headings (Today / Yesterday / "Sun · Oct 11"), each a receipt line
  check('expenses grouped under day headings', await vis(p.getByText(/^(Today|Yesterday|[A-Z][a-z]{2}) · [A-Z][a-z]{2} \d{1,2}$/)));
  check('expense receipt line', await vis(p.getByLabel(/^Villa Margherita \(6 nights\), \$1,650$/)));
  await p.getByLabel(/^Villa Margherita/).click(); await p.waitForTimeout(700);
  check('expense details open', await vis(p.getByRole('button', { name: 'Delete expense' })));
  await p.keyboard.press('Escape'); await p.waitForTimeout(500);
  // changing the budget from the number itself
  await p.getByTestId('budget-cap-open').click(); await p.waitForTimeout(600);
  await p.getByTestId('budget-cap').fill('abc'); await p.getByTestId('budget-cap-save').click(); await p.waitForTimeout(300);
  check('budget cap validation', await vis(p.getByText(/Use numbers only|Enter a budget/)));
  await p.getByTestId('budget-cap').fill('4500'); await p.getByTestId('budget-cap-save').click(); await p.waitForTimeout(700);
  check('budget cap saved', await vis(p.getByText(/Budget set to \$4,500/)) && await vis(p.getByTestId('budget-left').filter({ hasText:
'$2,414' })));
  await p.getByTestId('budget-cap-open').click(); await p.waitForTimeout(600);
  await p.getByTestId('budget-cap').fill('4000'); await p.getByTestId('budget-cap-save').click(); await p.waitForTimeout(700);
  await p.getByRole('tab', { name: /Local/ }).click(); await p.waitForTimeout(500);
  check('local view shows euros', await vis(p.getByText(/€/)));
  await p.getByRole('tab', { name: /^Home ·/ }).click();
  await p.getByTestId('add-expense-open').click(); await p.waitForTimeout(700);
  await p.getByTestId('add-expense').click(); await p.waitForTimeout(400);
  check('expense validation', await vis(p.getByText(/what it was for|Enter an amount|amount/i)));
  await p.getByTestId('expense-title').fill('Gelato');
  // a half-typed expense survives closing the sheet
  await p.keyboard.press('Escape'); await p.waitForTimeout(600);
  await p.getByTestId('add-expense-open').click(); await p.waitForTimeout(700);
  check('half-typed expense is kept (draft)', (await p.getByTestId('expense-title').inputValue().catch(() => '')) === 'Gelato');
```

```
  check('category guessed from the words as one chip', await vis(p.getByRole('button', { name: 'Food', exact: true })));
  await p.getByTestId('expense-amount').fill('12.5');
  const [fc] = await Promise.all([p.waitForEvent('filechooser', { timeout: 4000 }).catch(() => null), p.getByRole('button', { name: /Att
ach receipt/ }).click()]);
  if (fc) { await fc.setFiles(fixture('receipt.jpg')); await p.waitForTimeout(900); }
  check('receipt attach thumbnail', !!fc && await vis(p.getByLabel(/receipt/i), 1500));
  await p.getByTestId('add-expense').click(); await p.waitForTimeout(800);
  check('expense added toast', await vis(p.getByText(/Added \$13|Added \$12/)));
  check('draft is cleared after adding', await (async () => { await p.getByTestId('add-expense-open').click(); await p.waitForTimeout(70
0); const v = await p.getByTestId('expense-title').inputValue().catch(() => 'x'); await p.keyboard.press('Escape'); await p.waitForTimeo
ut(500); return v === ''; })());
  check('new expense is under Today', await vis(p.getByText(/^Today · /)));
  await p.getByLabel('Settle up').first().click().catch(async () => { await p.getByRole('button', { name: /Settle up/ }).first().click
(); }); await p.waitForTimeout(600);
  check('settle sheet lists payments', await vis(p.getByText(/pays/)));
  check('settle sheet explains the fewest payments', await vis(p.getByText(/the fewest possible/)));
  await p.getByRole('button', { name: 'Mark paid' }).first().click(); await p.waitForTimeout(500);
  check('settle recorded', await vis(p.getByText(/Payment recorded/)));
  // the rest of the payments: the last one celebrates and stamps SETTLED (the sheet closes itself)
  for (let i = 0; i < 6 && await vis(p.getByRole('button', { name: 'Mark paid' }), 700); i++) { await p.getByRole('button', { name: 'Mar
k paid' }).first().click(); await p.waitForTimeout(700); }
  check('all settled celebration', await vis(p.getByText('All settled up')));
  check('settled stamp', await vis(p.getByLabel('Settled', { exact: true })));
  check('settle button gone once square', !(await vis(p.getByRole('button', { name: 'Settle up', exact: true }), 600)));
  await p.keyboard.press('Escape');
  // ---- Chat
  await tab(p, 'Chat');
  await p.getByTestId('chat-input').fill('Anyone up for gelato?'); await p.getByTestId('chat-send').click(); await p.waitForTimeout(50
0);
  check('crew message sent', await vis(p.getByText('Anyone up for gelato?')));
  check('chat shows a day heading and a clock time', await vis(p.getByText('Today', { exact: true })) && await vis(p.getByText(/^\d{1,
2}:\d{2} (AM|PM)$/)));
  check('the send button is a real 44 point target', await p.getByTestId('chat-send').evaluate((el) => { const r = el.getBoundingClientR
ect(); return r.width >= 44 && r.height >= 44; }));
  await p.getByRole('tab', { name: 'Scout AI' }).click(); await p.waitForTimeout(500);
  await p.getByTestId('chat-input').fill('Who owes who?'); await p.getByTestId('chat-send').click(); await p.waitForTimeout(2200);
  check('scout ai answers offline', await vis(p.getByText(/owes|gets|square/i)));
  check('ai message not in crew tab', await (async () => { await p.getByRole('tab', { name: 'Crew' }).click(); await p.waitForTimeout(50
0); return !(await vis(p.getByText('Who owes who?'), 600)); })());
  // ---- More
  await tab(p, 'More');
  for (const [label, expect] of [['Crew & roles', /Invite code|CREW/i], ['Flights & arrivals', /Flights/i], ['Packing list', /Packing/
i], ['Trip recap', /Recap/i]]) {
    await p.getByRole('button', { name: new RegExp(label) }).first().click(); await p.waitForTimeout(1000);
    check(`more → ${label}`, await vis(p.getByText(expect)));
    await p.goBack(); await p.waitForTimeout(900);
    if (!(await vis(p.getByText('Maya Chen'), 800))) { await tab(p, 'More'); }
  }
  // the boarding pass at the top (My trips is its stub) and the four groups of tags
  check('more → boarding pass and groups', (await vis(p.getByText('Maya Chen'))) && (await vis(p.getByRole('button', { name: 'My trips'
}))) && (await vis(p.getByRole('heading', { name: 'Around town' }))) && (await vis(p.getByRole('heading', { name: 'The app' }))));
  for (const [label, expect] of [['My trips', /Trips|Join|Create/i], ['Weather', /Right now|Loading/i], ['Phrasebook', /phrases/i], ['Em
ergency', /Call in an emergency/i], ['Help & FAQ', /The journey/i]]) {
    await p.getByRole('button', { name: label, exact: true }).first().click(); await p.waitForTimeout(1000);
    check(`more → ${label}`, await vis(p.getByText(expect)));
    await p.goBack(); await p.waitForTimeout(900);
    if (!(await vis(p.getByText('Maya Chen'), 800))) { await tab(p, 'More'); }
  }
  console.log(results.join('\n'));
  console.log('errors:', errors.length ? errors.join('\n') : 'none');
  await b.close();
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 500)); process.exit(1); });
```

e2e/verify2.js

```
const { BASE, start } = require('./lib');
const results = [];
const check = (name, ok, extra = '') => { results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`); };
const vis = async (loc, t = 2500) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);
const go = async (p, r) => { await p.goto(BASE + '/' + r, { waitUntil: 'networkidle' }); await p.waitForTimeout(1500); if (await p.getBy
Role('button', { name: 'Skip' }).count()) await p.getByRole('button', { name: 'Skip' }).first().click().catch(() => {}); };
(async () => {
  let { b, p, errors } = await start();
  // ---- Crew hub
  await go(p, 'crew');
  // the invitation: a ticket with the code in print, the QR button and one black Invite friends button
  check('crew invitation shows the code', await vis(p.getByLabel('Invite code AML-2026', { exact: true })));
  check('crew invitation has the QR button and Invite friends', await vis(p.getByTestId('invite-qr')) && await vis(p.getByRole('button',
{ name: 'Invite friends' })));
  await p.getByRole('button', { name: 'Invite friends' }).click(); await p.waitForTimeout(600);
  check('Invite friends opens the share sheet', await vis(p.getByText('WhatsApp')) && await vis(p.getByRole('button', { name: 'Copy', ex
act: true })));
  await p.keyboard.press('Escape'); await p.waitForTimeout(400);
  check('members are stamped GOING', (await p.getByLabel('Going', { exact: true }).filter({ visible: true }).count()) >= 4);
  check('the green dot marks you as here now', await vis(p.getByLabel('Maya Chen').first()));
```

```
  check('checklist shows progress', await vis(p.getByLabel(/^\d+ of \d+ done$/)));
  const before = await p.getByLabel(/^\d+ of \d+ done$/).first().getAttribute('aria-label');
  await p.getByRole('checkbox', { name: 'Travel insurance' }).click(); await p.waitForTimeout(500);
  const after = await p.getByLabel(/^\d+ of \d+ done$/).first().getAttribute('aria-label');
  check('ticking a task moves the count and the bar', before !== after, `${before} -> ${after}`);
  await p.getByRole('checkbox', { name: 'Travel insurance' }).click(); await p.waitForTimeout(300);
  check('polls start OPEN', await vis(p.getByLabel('Open', { exact: true })));
  await p.getByLabel(/Diego Alvarez, Food Lead/).click(); await p.waitForTimeout(600);
  await p.getByRole('button', { name: 'Navigator' }).last().click().catch(() => {}); await p.waitForTimeout(500);
  check('role change toast', await vis(p.getByText(/Diego is now Navigator/)));
  await p.keyboard.press('Escape'); await p.waitForTimeout(300);
  await p.getByLabel(/Fornillo \(quiet, pebbles\)/).click().catch(() => {}); await p.waitForTimeout(500);
  check('poll vote registers', await vis(p.getByLabel(/Fornillo \(quiet, pebbles\), .*your vote/)));
  await p.getByRole('button', { name: 'Close poll' }).first().click(); await p.waitForTimeout(700);
  check('closing a poll with a winner stamps it LOCKED IN', await vis(p.getByLabel('Locked in', { exact: true })));
  check('closing a poll celebrates the winner', await vis(p.getByText(/Locked in: Fornillo/)));
  await p.getByRole('button', { name: 'Reopen' }).first().click(); await p.waitForTimeout(500);
  check('a reopened poll is OPEN again', await vis(p.getByLabel('Open', { exact: true })));
  await p.getByLabel('Add task').click(); await p.waitForTimeout(400);
  check('empty task error', await vis(p.getByText(/enter|task|empty|required/i)));
  await p.getByLabel('New task').fill('Buy sunscreen'); await p.getByLabel('Add task').click(); await p.waitForTimeout(500);
  check('task added', await vis(p.getByText('Buy sunscreen')));
  await p.getByRole('button', { name: /New code/ }).click(); await p.waitForTimeout(500);
  await p.getByRole('button', { name: /^(Create|New code|Confirm|Yes)/ }).last().click().catch(() => {}); await p.waitForTimeout(600);
  // add member validation
  await p.getByText('+ Add').first().click(); await p.waitForTimeout(500);
  await p.getByRole('button', { name: 'Add member' }).click(); await p.waitForTimeout(400);
  check('add member validation', await vis(p.getByText(/name/i).filter({ hasText: /enter|required|letters/i })) || await vis(p.getByText
(/Enter .*name|required/i)));
  await p.getByLabel('Name', { exact: true }).fill('Jules Moreau'); await p.getByRole('button', { name: 'Add member' }).click(); await
p.waitForTimeout(700);
  check('someone added by name is stamped INVITED', await vis(p.getByLabel('Invited', { exact: true })));
  await p.getByText('+ Add').first().click(); await p.waitForTimeout(500);
  await p.getByLabel('Name', { exact: true }).fill('A'); await p.keyboard.press('Escape'); await p.waitForTimeout(500);
  await p.getByText('+ Add').first().click(); await p.waitForTimeout(500);
  check('a half-typed member name survives closing the sheet', (await p.getByLabel('Name', { exact: true }).inputValue()) === 'A');
  await p.getByLabel('Name', { exact: true }).fill(''); await p.keyboard.press('Escape');
  // ---- Flights
  await go(p, 'flights');
  check('flights Sam shows no flight', await vis(p.getByText('No flight added yet')));
  await p.getByTestId('add-flight').click(); await p.waitForTimeout(700);
  await p.getByRole('button', { name: 'Save flight' }).click(); await p.waitForTimeout(400);
  check('flight validation', await vis(p.getByText(/required|enter|airline/i)));
  await p.getByPlaceholder('ANA').fill('Delta'); await p.getByPlaceholder('NH 175').fill('DL 99'); await p.getByPlaceholder('JFK').fill
('XXX');
  await p.getByRole('button', { name: 'Save flight' }).click(); await p.waitForTimeout(400);
  check('unknown IATA rejected', await vis(p.getByText(/airport|IATA|recogni/i)));
  await p.getByPlaceholder('JFK').fill('BOS');
  await p.getByRole('button', { name: 'Save flight' }).click(); await p.waitForTimeout(900);
  check('flight saved', await vis(p.getByText(/Flight saved/)) || await vis(p.getByText('Delta DL 99')));
  // ---- Packing
  await go(p, 'packing');
  await p.getByRole('button', { name: /Split the rest fairly/ }).click(); await p.waitForTimeout(700);
  check('packing split toast', await vis(p.getByText(/Assigned \d+ item/)));
  await p.getByRole('tab', { name: 'Mine' }).click(); await p.waitForTimeout(500);
  check('mine filter shows my items', await vis(p.getByText(/Me$/)));
  // ---- Recap
  await go(p, 'recap');
  check('recap story card', await vis(p.getByLabel(/Story card/)));
  await p.getByRole('button', { name: /Share story card/ }).click(); await p.waitForTimeout(700);
  check('share sheet opens', await vis(p.getByText(/WhatsApp/)));
  await p.keyboard.press('Escape');
  // ---- How it works + help
  await go(p, 'how-it-works');
  check('how it works rubric map', await vis(p.getByText(/Rubric/i)));
  // ---- Language
  await go(p, 'settings');
  await p.getByRole('button', { name: /Español/ }).click(); await p.waitForTimeout(700);
  await go(p, ''); // settings has no tab bar: look at the tabs on Home
  await p.getByRole('tab', { name: 'Inicio' }).waitFor({ timeout: 3000 }).then(() => check('language switches tabs', true)).catch(() =>
check('language switches tabs', false));
  await go(p, 'settings');
  await p.getByRole('button', { name: /English/ }).click().catch(() => {}); await p.waitForTimeout(600);
  // ---- Settings: look and feel, and the Home layout
  await go(p, 'settings');
  check('Look and feel offers the six themes', (await p.getByRole('radio', { name: / theme$/ }).count()) === 6);
  check('no Apply look button until something is changed', !(await vis(p.getByRole('button', { name: 'Apply look' }), 600)));
  await p.getByRole('radio', { name: 'Tomato theme' }).click(); await p.waitForTimeout(300);
  check('choosing a theme offers Apply look', await vis(p.getByRole('button', { name: 'Apply look' })));
  await p.getByRole('radio', { name: 'Limoncello theme' }).click(); await p.waitForTimeout(300);
  check('choosing the current theme again takes Apply look away', !(await vis(p.getByRole('button', { name: 'Apply look' }), 600)));
  await p.getByRole('button', { name: 'Chill', exact: true }).click(); await p.waitForTimeout(300);
  check('Effects Chill is chosen at once', (await p.getByRole('button', { name: 'Chill', exact: true }).getAttribute('aria-selected')) =
== 'true');
  await p.getByRole('button', { name: 'Lively', exact: true }).click(); await p.waitForTimeout(300);
  await p.getByRole('button', { name: 'Money', exact: true }).click(); await p.waitForTimeout(300);
```

```
  check('a Home layout preset is chosen', (await p.getByRole('button', { name: 'Money', exact: true }).getAttribute('aria-selected')) ==
= 'true');
  await p.getByLabel('Arrange blocks').click(); await p.waitForTimeout(300);
  await p.getByLabel('Move Weather up').click(); await p.waitForTimeout(300);
  check('moving a block by hand makes the layout Custom', await vis(p.getByLabel('Custom', { exact: true })));
  await p.getByLabel('Show Weather on Home').click({ force: true }).catch(() => {}); await p.waitForTimeout(300);
  await p.getByRole('button', { name: 'Planner', exact: true }).click(); await p.waitForTimeout(300);
  check('picking a preset starts over', (await p.getByRole('button', { name: 'Planner', exact: true }).getAttribute('aria-selected')) ==
= 'true');
  check('Make it yours again is in Settings', await vis(p.getByLabel('Make it yours again')));
  // ---- Demo timeline: in progress -> Today card
  await go(p, 'settings');
  await p.getByLabel('Advanced settings').click(); await p.waitForTimeout(400); // the demo timeline is folded under Advanced
  await p.mouse.move(195, 500); await p.mouse.wheel(0, 1500); await p.waitForTimeout(500);
  await p.getByRole('button', { name: /in progress/i }).first().click().catch(() => {}); await p.waitForTimeout(500);
  await p.getByRole('button', { name: /Rebuild|Reset|Confirm|Apply/ }).last().click().catch(() => {}); await p.waitForTimeout(800);
  await go(p, '');
  check('active trip headline', await vis(p.getByRole('heading', { name: /^(Day \d of 6|Today is the first day of your 6 day trip)$/
})));
  check('today card on home', await vis(p.getByText(/Today · Day \d of 6/)));
  // ---- Join
  await go(p, 'join');
  check('join shows the demo ticket', await vis(p.getByText('Seoul Food Crawl')));
  await p.getByLabel('Use demo code SEO-5521').click(); await p.waitForTimeout(300);
  check('the demo ticket fills in its code', (await p.getByTestId('invite-code').inputValue()) === 'SEO-5521');
  await p.getByTestId('invite-code').fill('ZZZ-9999'); await p.getByTestId('join-submit').click(); await p.waitForTimeout(800);
  check('an unknown code is explained in words', await vis(p.getByText(/No trip found with that code/)));
  await p.getByRole('button', { name: /^Join/ }).first().click().catch(() => {});
  await p.getByPlaceholder(/ABC|TKY|code/i).first().fill('SEO-5521').catch(() => {});
  await p.getByRole('button', { name: /Join trip|Join$/ }).last().click().catch(() => {}); await p.waitForTimeout(1200);
  check('join SEO-5521', await vis(p.getByText(/Seoul/)));
  // ---- New trip (a short form with one Create button), the Trips list, and the invitation-style empty states of a new trip
  await go(p, 'new-trip');
  await p.getByRole('button', { name: 'Create trip' }).click(); await p.waitForTimeout(500);
  check('new trip asks for a name and a budget', await vis(p.getByText('Enter a trip name')) && await vis(p.getByText(/Enter a group bud
get/)));
  await p.getByLabel('Search destinations').fill('Lisbon'); await p.waitForTimeout(300);
  check('destination search narrows the photo tiles', !(await vis(p.getByLabel(/^Tokyo, Japan$/), 500)));
  await p.getByLabel('Search destinations').fill('');
  await p.getByLabel('Trip name').fill('Pier Pressure'); await p.getByLabel('Group budget (USD)').fill('900');
  await p.getByRole('button', { name: 'Create trip' }).click(); await p.waitForTimeout(1200);
  check('new trip is created and opens the crew page', await vis(p.getByText('Trip created. Share the invite code with your group')) &&
await vis(p.getByRole('heading', { name: 'The crew' })));
  check('a new trip has an empty polls invitation', await vis(p.getByText('No polls yet')));
  await go(p, 'trips');
  check('trips list stamps the current trip', await vis(p.getByLabel('Pier Pressure, current trip')) && await vis(p.getByLabel('Curren
t', { exact: true })));
  check('trips list has Join with code and New trip', await vis(p.getByRole('button', { name: 'Join with code' })) && await vis(p.getByL
abel('New trip').first()));
  console.log(results.join('\n'));
  console.log('errors:', errors.length ? errors.join('\n') : 'none');
  await b.close();
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 600)); process.exit(1); });
```

e2e/verify3.js

```
const { BASE, launch, shot } = require('./lib');
const results = [];
const check = (name, ok, extra = '') => { results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`); };
const vis = async (loc, t = 2500) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);
(async () => {
  const b = await launch();
  const ctx = await b.newContext({ viewport: { width: 390, height: 844 }, isMobile: true, hasTouch: true });
  const p = await ctx.newPage(); const errs = []; p.on('pageerror', (e) => errs.push(e.message));
  await p.goto(BASE + '/', { waitUntil: 'networkidle' }); await p.waitForTimeout(2000);
  await p.getByRole('button', { name: 'Skip' }).click(); await p.waitForTimeout(700);
  // sign-up → survey prompt
  await p.getByRole('link', { name: 'Sign Up' }).click(); await p.waitForTimeout(900);
  await p.getByPlaceholder('e.g. Alex Rivera').fill('Jordan Smith'); await p.getByPlaceholder('you@email.com').fill('jordan@school.or
g');
  await p.getByPlaceholder('At least 8 characters').fill('Trip2027go'); await p.getByPlaceholder('Re-enter password').fill('Trip2027g
o');
  await p.getByRole('checkbox').click(); await p.getByRole('button', { name: 'Create account' }).click(); await p.waitForTimeout(2500);
  check('new account lands in Make it yours', await vis(p.getByRole('heading', { name: /make it yours/i })));
  await p.getByRole('button', { name: 'Let’s go' }).click(); await p.waitForTimeout(600);
  // the look step: picking a theme repaints this screen at once (the accent of the step bar and the button plate change)
  await p.getByRole('radio', { name: 'Tomato theme' }).click(); await p.waitForTimeout(500);
  check('the Look step marks the chosen theme', (await p.getByRole('radio', { name: 'Tomato theme' }).getAttribute('aria-checked')) ===
'true');
  await p.getByRole('radio', { name: 'Grape theme' }).click(); await p.waitForTimeout(400);
  check('Grape switches the mode to Dark', (await p.getByRole('radio', { name: 'Dark' }).getAttribute('aria-checked')) === 'true');
  await p.getByRole('radio', { name: 'Light' }).click(); await p.getByRole('radio', { name: 'Tomato theme' }).click(); await p.waitForTi
meout(300);
  await p.getByRole('button', { name: 'Continue' }).click(); await p.waitForTimeout(500);
  // picking no interest never blocks
```

```
  await p.getByRole('button', { name: 'Continue' }).click(); await p.waitForTimeout(500);
  check('no interest picked still moves on to the layout step', await vis(p.getByRole('radio', { name: 'Money layout' })));
  await p.getByRole('radio', { name: 'Party layout' }).click(); await p.waitForTimeout(300);
  check('a layout can be chosen', (await p.getByRole('radio', { name: 'Party layout' }).getAttribute('aria-checked')) === 'true');
  await p.getByRole('button', { name: 'Back', exact: true }).click(); await p.waitForTimeout(500);
  await p.getByRole('checkbox', { name: 'Beaches' }).click();
  await p.getByLabel('Skip to the end').click(); await p.waitForTimeout(600);
  // the last page lists every answer; tapping a row opens that step, and "Back to summary" (or the back arrow) returns
  check('last page lists the answers as tappable rows', await vis(p.getByLabel(/Change interests, now Beaches/)));
  check('the layout chosen on the way is listed', await vis(p.getByLabel(/Change home layout, now Party/)));
  await p.getByLabel('Change interests').click(); await p.waitForTimeout(500);
  check('a summary row opens its step', await vis(p.getByRole('checkbox', { name: 'Nightlife' })));
  await p.getByRole('checkbox', { name: 'Nightlife' }).click();
  await p.getByRole('button', { name: 'Back to summary' }).click(); await p.waitForTimeout(600);
  check('"Back to summary" returns with the change applied', await vis(p.getByLabel(/Change interests, now Nightlife, Beaches/)));
  await p.getByLabel('Change look').click(); await p.waitForTimeout(500);
  await p.getByRole('button', { name: 'Back', exact: true }).click(); await p.waitForTimeout(600);
  check('the back arrow also returns to the summary', await vis(p.getByLabel(/Change look, now Tomato/)));
  await p.getByRole('button', { name: 'Apply my style' }).click();
  check('a short "Building your trip HQ" moment plays', await vis(p.getByRole('heading', { name: /Building your trip HQ/ }), 2500));
  await p.waitForTimeout(5500);
  const saved = await p.evaluate(() => localStorage.getItem('togetherwego-appearance'));
  check('theme saved', /sunset/.test(saved || ''), saved);
  check('still signed in after reload (no trip card)', await vis(p.getByText(/Plan your first group trip/)));
  check('no-trip home has create/join', await vis(p.getByRole('button', { name: /Create trip/ })));
  await p.close();
  // tour for the demo user
  const p2 = await (await b.newContext({ viewport: { width: 390, height: 844 }, isMobile: true, hasTouch: true })).newPage();
  await p2.goto(BASE + '/', { waitUntil: 'networkidle' }); await p2.waitForTimeout(2000);
  await p2.getByRole('button', { name: 'Skip' }).click(); await p2.waitForTimeout(700);
  await p2.getByLabel('Use the demo account').click(); await p2.getByTestId('sign-in').click(); await p2.waitForTimeout(2500);
  check('guided tour shows on first run', await vis(p2.getByText(/at a glance/i)));
  for (let i = 0; i < 4; i++) { await p2.getByRole('button', { name: 'Next' }).last().click(); await p2.waitForTimeout(450); }
  check('tour last step', await vis(p2.getByRole('button', { name: 'Got it' })));
  await p2.getByRole('button', { name: 'Got it' }).click(); await p2.waitForTimeout(700);
  check('tour closes', !(await vis(p2.getByText(/at a glance/i), 500)));
  await p2.reload({ waitUntil: 'networkidle' }); await p2.waitForTimeout(2500);
  check('tour does not return', !(await vis(p2.getByRole('button', { name: 'Got it' }), 800)));
  // Settings can put on a new look too: pick, Apply look, and the app restarts in it without signing anyone out
  await p2.goto(BASE + '/settings', { waitUntil: 'load' }); await p2.waitForTimeout(1500);
  await p2.getByRole('radio', { name: 'Tomato theme' }).click(); await p2.waitForTimeout(300);
  await p2.getByRole('button', { name: 'Apply look' }).click(); await p2.waitForTimeout(4500);
  check('Apply look in Settings saves the new theme', /sunset/.test((await p2.evaluate(() => localStorage.getItem('togetherwego-appearan
ce'))) || ''));
  check('the app is back on Home, still signed in, after the restart', await vis(p2.getByRole('tab', { name: 'Home', exact: true })));
  // midnight theme screenshot set
  await p2.evaluate(() => localStorage.setItem('togetherwego-appearance', JSON.stringify({ theme: 'midnight', corners: 'bubbly' })));
  await p2.reload({ waitUntil: 'networkidle' }); await p2.waitForTimeout(2500);
  await p2.screenshot({ path: shot('v3-midnight-home.png') });
  for (const t of ['Plan', 'Budget', 'Chat', 'More']) { await p2.getByRole('tab', { name: t, exact: true }).click(); await p2.waitForTim
eout(1100); await p2.screenshot({ path: shot(`v3-midnight-${t}.png`) }); }
  console.log(results.join('\n')); console.log('errors:', errs.length ? errs.join('\n') : 'none');
  await b.close();
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 600)); process.exit(1); });
```

e2e/verify4.js

```
const { BASE, start } = require('./lib');
const results = [];
const check = (name, ok, extra = '') => results.push(`${ok ? 'PASS' : 'FAIL'}  ${name}${ok ? '' : '  ' + extra}`);
const vis = async (loc, t = 2500) => loc.filter({ visible: true }).first().waitFor({ state: 'visible', timeout: t }).then(() => true).ca
tch(() => false);
const heading = async (p) => Number((await p.getByText(/360° ·/).first().innerText()).match(/(\d+)°$/)[1]);
(async () => {
  const { b, p, errors } = await start();
  // Home chip opens the trip's city
  await p.getByLabel('Open the 360 degree city view').click(); await p.waitForTimeout(2000);
  check('home 360° chip opens Positano', await vis(p.getByRole('heading', { name: /Positano/i })));
  check('night/day toggle exists', await vis(p.getByLabel(/(Night|Day) view\./)));
  // day/night toggle flips
  const before = await p.getByLabel(/(Night|Day) view\./).getAttribute('aria-label');
  await p.getByLabel(/(Night|Day) view\./).click(); await p.waitForTimeout(500);
  const after = await p.getByLabel(/(Night|Day) view\./).getAttribute('aria-label');
  check('toggle flips day/night', before !== after, `${before} -> ${after}`);
  // city picker
  await p.getByLabel('Paris 360 degree view').click(); await p.waitForTimeout(1500);
  check('city picker → Paris', await vis(p.getByRole('heading', { name: /Paris/i })));
  check('landmark callout shows Eiffel fact', await vis(p.getByText(/330 m tall/)));
  // arrows change heading, auto pause
  await p.getByLabel(/Auto rotate/).click();
  const h0 = await heading(p);
  await p.getByLabel('Look right').click(); await p.waitForTimeout(1200);
  const h1 = await heading(p);
  check('arrow turns the view', h1 !== h0, `${h0} -> ${h1}`);
  // drag
  await p.mouse.move(150, 300); await p.mouse.down(); await p.mouse.move(300, 300, { steps: 6 }); await p.mouse.up(); await p.waitForTim
```

```
eout(600);
  check('drag turns the view', (await heading(p)) !== h1);
  // landmark jump
  await p.getByLabel('Look at Arc de Triomphe').click(); await p.waitForTimeout(1500);
  check('jump to landmark shows its fact', await vis(p.getByText(/Napoleon in 1806/)));
  // full wrap: 360° of arrows returns near start heading
  const start0 = await heading(p);
  for (let i = 0; i < 12; i++) { await p.getByLabel('Look right').click(); await p.waitForTimeout(900); }
  const end0 = await heading(p);
  const diff = Math.abs(((end0 - start0 + 540) % 360) - 180);
  check('360° wraps around (≈ full circle after 12 turns)', diff <= 25, `${start0} -> ${end0} (off by ${diff}°)`);
  console.log(`  (12 arrow turns: ${start0}° -> ${end0}°, off by ${diff}°)`);
  // destination page button
  await p.goto(BASE + '/destination/tokyo', { waitUntil: 'networkidle' }); await p.waitForTimeout(1800);
  await p.getByRole('button', { name: /360° view/ }).click(); await p.waitForTimeout(1500);
  check('destination page opens Tokyo 360', await vis(p.getByRole('heading', { name: /Tokyo/i })));
  // More menu entry
  await p.goto(BASE + '/profile', { waitUntil: 'networkidle' }); await p.waitForTimeout(1500);
  await p.getByRole('button', { name: /360° City Views/ }).first().click(); await p.waitForTimeout(1500);
  check('More → 360° City Views', await vis(p.getByLabel(/360 degree view/)));
  console.log(results.join('\n')); console.log('errors:', errors.length ? errors.join('\n') : 'none');
  await b.close();
})().catch((e) => { console.log(results.join('\n')); console.error('CRASH', e.message.slice(0, 500)); process.exit(1); });
```

scripts/source-pdf.js

```
// Prints all hand-written source code into docs/TogetherWeGo-source-code.pdf.
//   npm run source-pdf        (needs a browser: `npx playwright-core install chromium`, or set E2E_CHROME)
const fs = require('fs');
const path = require('path');
const { chromium } = require('playwright-core');

const root = path.join(__dirname, '..');
const SKIP_DIRS = new Set(['node_modules', '.git', '.expo', 'dist', '.dist', 'shots', 'data', 'assets', 'screenshots', 'fixtures']);
const KEEP = /\.(tsx?|jsx?|json|md)$/;
const ROOT_FILES = ['README.md', 'ATTRIBUTIONS.md', 'package.json', 'app.json', 'eas.json', 'tsconfig.json', 'eslint.config.js', 'jest.c
onfig.js', 'jest.setup.js'];
const FOLDERS = ['src', 'server', 'e2e', 'scripts', 'docs'];

function walk(dir, out = []) {
  for (const name of fs.readdirSync(dir).sort()) {
    const full = path.join(dir, name);
    if (fs.statSync(full).isDirectory()) {
      if (!SKIP_DIRS.has(name)) walk(full, out);
    } else if (KEEP.test(name)) {
      out.push(full);
    }
  }
  return out;
}

const files = [...ROOT_FILES.map((f) => path.join(root, f)), ...FOLDERS.flatMap((d) => walk(path.join(root, d)))].filter((f) => fs.exist
sSync(f));
const esc = (s) => s.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
const rel = (f) => path.relative(root, f).split(path.sep).join('/');
const lines = files.reduce((n, f) => n + fs.readFileSync(f, 'utf8').split('\n').length, 0);

const html = `<html><head><meta charset="utf-8"><style>
@page{size:Letter;margin:14mm 10mm}body{font-family:Helvetica,Arial,sans-serif;font-size:10pt}h1{font-size:22pt}
h2{font-size:10.5pt;background:#222;color:#fff;padding:3px 6px;margin:0 0 4px;page-break-after:avoid}
pre{font-family:DejaVu Sans Mono,Menlo,Consolas,monospace;font-size:6.6pt;line-height:1.25;white-space:pre-wrap;word-break:break-all;mar
gin:0 0 10px}
.f{margin-top:6px}.toc div{font-size:8.5pt;line-height:1.5}
</style></head><body>
<h1>TogetherWeGo: complete source code</h1>
<p>FBLA Mobile Application Development 2026-27, &ldquo;Together We Go: Group Trip Planner&rdquo;. Expo SDK 57 / React Native / TypeScrip
t, with a small Node sync server. ${files.length} files, ${lines.toLocaleString('en-US')} lines. Photos and fonts are listed in ATTRIBUT
IONS.md and are not printed.</p>
<div class="toc">${files.map((f, i) => `<div>${i + 1}. ${esc(rel(f))}</div>`).join('')}</div>
${files.map((f) => `<div class="f" style="page-break-before:auto"><h2>${esc(rel(f))}</h2><pre>${esc(fs.readFileSync(f, 'utf8'))}</pre></
div>`).join('\n')}
</body></html>`;

(async () => {
  const browser = await chromium.launch(process.env.E2E_CHROME ? { executablePath: process.env.E2E_CHROME } : {});
  const page = await browser.newPage();
  await page.setContent(html, { waitUntil: 'load' });
  const out = path.join(root, 'docs', 'TogetherWeGo-source-code.pdf');
  await page.pdf({
    path: out,
    format: 'Letter',
    printBackground: true,
    displayHeaderFooter: true,
    headerTemplate: '<span></span>',
    footerTemplate: '<div style="font-size:7px;width:100%;text-align:center">TogetherWeGo source code - page <span class="pageNumber"></
span> of <span class="totalPages"></span></div>',
```

```
    margin: { top: '14mm', bottom: '14mm', left: '10mm', right: '10mm' },
  });
  await browser.close();
  console.log(`Wrote ${rel(out)} (${files.length} files, ${lines} lines)`);
})();
```

docs/BUILDING.md

```
# Running TogetherWeGo without Expo Go

The competition rules ask for a solution that runs on its own. Expo Go is a development tool: it needs your laptop's dev server running and reachable. This page lists the ways to get a standalone copy and what each one costs.

| Goal | Way | Cost |
|---|---|---|
| Android phone, no laptop needed | Build an **`.apk`** with EAS (below) | Free Expo account |
| iPhone, no laptop needed | EAS build for iOS or TestFlight | **Apple Developer Program** (paid, yearly) |
| iPhone, laptop running | Expo Go with the laptop on the same wifi or a phone hotspot (`npx expo start`, or `--tunnel` if the wifi blocks devices from seeing each other). SDK 57 is not in Apple's App Store: build your own Expo Go with `npx eas-cli@latest go` (see the README) | Apple Developer Program for that TestFlight build, and the laptop must stay running |
| Any phone or laptop browser | Web export (below) | Free, fewer features |

## Android `.apk` with EAS (about 15 minutes, mostly waiting)

```bash
npm install
npx eas-cli@latest login            # free account at expo.dev
npx eas-cli@latest init             # links this folder to your account (writes a project id into app.json)
npx eas-cli@latest build -p android --profile preview
```

When the build finishes (about 10 to 20 minutes in the cloud queue) EAS prints a link to the `.apk`. Open it on the phone, allow installing from the browser, and install. The app then runs with **no laptop, no dev server and no internet**. The first build asks to create a signing key: say yes.

You can also build on your own computer with Android Studio installed: `npx expo run:android`.

iPhone

An installable iPhone build always needs an Apple Developer account:

```bash
npx eas-cli@latest device:create     # registers a phone (internal distribution)
npx eas-cli@latest build -p ios --profile preview
```

Without that account, use the web build on the iPhone (below) or an Android phone with Expo Go. Tip for the competition: start Expo on the laptop **before** the set-up window, join a phone hotspot with both devices, and keep the screenshots in `docs/screenshots/` as a backup.

Web build

```bash
npx expo export --platform web      # writes a static site to dist/
```

Host the `dist/` folder anywhere that serves static files. On an iPhone, *Share → Add to Home Screen* gives it an icon. The web build runs the whole planner, but not the things that need a phone's hardware: notifications, the native map (it falls back to OpenStreetMap), the camera QR scanner, haptics and speech on some browsers.

Sync from an installed copy

The sync server speaks plain HTTP on your own network. The build settings allow that (`app.json`: Android `usesCleartextTraffic`, iOS `NSAllowsLocalNetworking`), and the first time the app talks to the server iOS asks for local-network access. An installed copy cannot guess the server's address the way Expo Go does, so either:

* open an invite link or scan an invite QR code that carries the address, or
* type the address once in *Settings → Advanced → Sync across phones* (the server prints it when it starts).

Windows tips

* If PowerShell says *running scripts is disabled on this system*, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or use **Command Prompt** instead.
* The first time `npm run server` or `npx expo start` runs, Windows Firewall asks whether Node.js may use the network. Choose **Allow** for private networks. Without it phones cannot reach ports 8081 (Expo) or 8787 (sync).
* The phone and the laptop must be on the same network, and some school or venue wifi blocks devices from talking to each other. A phone hotspot always works.

Things to know

* The native map on a standalone Android build needs a Google Maps API key (`android.config.googleMaps.apiKey` in `app.json`). Expo Go already includes one. The offline radar map needs nothing.
* Live weather and exchange rates need internet; offline the app shows the last saved data and says so.
* `npx expo prebuild` generates the native projects if you want to open them in Xcode or Android Studio. The generated folders are not committed.
```

docs/DEMO_SCRIPT.md

```
# 7-minute presentation script

Built around the rating sheet so every scored item gets shown. Times are cumulative. Split the speaking parts between teammates (all mem
bers must present).

**Before the 3-minute set-up ends** (do not talk to the judges while you do it):

* Laptop: slides on screen, and a second terminal running `npm run server`. Put the laptop and **both phones on a phone hotspot** (venue
wifi often stops devices seeing each other). That is exactly three devices.
* Phone A: signed in as the demo account, *Settings → Advanced → Demo timeline* on **"Starts in 3 days"**, notifications allowed, volume
up (phrasebook audio). *Settings → Advanced → Sync across phones → Test connection* says **Connected**.
* Phone B: signed in with a second account (Sign Up with any name, skip the survey), on the *Join a trip* screen.
* Backup if the network fails: skip the two-phone minute and show `docs/screenshots/`.

| Time | Show | Say (key points) | Rating-sheet item |
|---|---|---|---|
| 0:00 | Title / app icon | The problem: group trips scattered across chats, spreadsheets and screenshots. TogetherWeGo puts plan, mone
y, people and paperwork in one place that works offline. | Addresses the prompt |
| 0:25 | Planning doc (personas, journey, MoSCoW) | How we planned: personas → user journey → feature priorities → navigation map → data
model → test log. | Planning process |
| 0:50 | Sign-in → tap Demo Account → Sign In; then a validation error on sign-up | Expo SDK 57 / React Native for iOS **and** Android.
Passwords are salted and stretched (PBKDF2). Every form checks format *and* meaning. | Platform · Input validation · Data handling |
| 1:10 | Home: hero photo, ☀️/🌙 chip, countdown, code pill, weather + local time, budget, Up next | The hero follows the local time in
Positano. Everything you need before a trip is on one screen. | UX · Functionality |
| 1:35 | Home → 360° chip → spin Positano, switch to Paris, 🌙 (lights come on), tap a landmark chip | Any city in 360°; at night the wi
ndows, landmarks and cars light up. Drawn in code, so it works offline. | Innovation · UX |
| 1:50 | More → Crew & roles → **Share** → WhatsApp; change a role; vote in a poll | Invite codes, roles, polls and a shared checklist.
One tap shares into social apps; the invite is also a QR code (we do not ask you to scan it). | **Social media integration** · Collabora
tion |
| 2:20 | Crew & roles → **Share this trip across phones**; phone B: type the code → Join. Send a chat message from each phone, add an ex
pense on B, add a photo on A | Two real phones, one server we run. Changes arrive within a second; the badge says *Live · 2 phones*. Men
tion airplane mode: changes wait and catch up. | Collaboration · Communication · Innovation |
| 3:00 | Plan: filters (Food, Under 1 hour), expand a card → 🔔; Vote tab → **Lock in** | Time and category filters. Votes show who is i
n; the organizer locks in the winner and the reminder turns on. | Scheduling · Collaboration |
| 3:30 | ➕ Add Activity → submit empty (errors) → type `museum` in *Where do you want to go?* → matches with distance from you → pick o
ne → overlap warning | Syntactic and semantic validation (it even warns about schedule clashes). Search is instant for saved places and
live for the rest (OpenStreetMap). | Input validation · UX |
| 3:50 | More → Flights & arrivals → tap the time-zone line | Flights are stored as exact instants, so arrivals convert between Rome tim
e and your time. | Scheduling / time zones |
| 4:15 | Budget: donut, Local · EUR, ➕ add a € expense + **receipt photo**, Who owes who → Settle up | Money is kept in whole cents; th
e settle-up finds the fewest payments; the receipt is attached (and reaches the other phone). | Budgeting |
| 4:55 | Chat → Crew, then **Scout AI** → "How's the budget?" | The group chat stays human; Scout answers from our real trip data, **off
line**. Optional Claude mode with a key in the secure keychain. | Innovation |
| 5:20 | Airplane mode → Offline Docs, Trip Map → Offline radar, Packing → Split the rest fairly | Built for bad wifi: documents, map, p
hrasebook and emergency info work offline. Everyone sees who is bringing what. | Data handling · Organization |
| 5:45 | Settings → Language → 日本語 → back; text size; mention Reduce Motion | 6 languages, adjustable text, screen-reader labels, and
the phone's Reduce Motion setting is respected. | Accessibility |
| 6:00 | Home → Customize this page → Make it yours → pick Tomato or Grape + a few interests + a Home layout → Build my trip HQ | Every
user makes the app theirs: six themes in light or dark, interests drive "Picked for you", three Home layouts. The reveal ends with confe
tti. | Innovation · UX |
| 6:15 | Settings → Advanced → Demo timeline "In progress" → Home "Today" card; then "Just finished" → Recap story card | The full lifec
ycle: planning → travelling → recap with stats, balances and a shareable story card. | Addresses all parts |
| 6:35 | More → How it works; then the repo: README, `docs/`, tests | Layered MVVM, a sync engine with its own tests, strict TypeScript,
0 lint errors; unit tests and browser tests; every library, API and photo documented. | Code quality · Architecture · Documentation |

**Protocol reminders** (full checklist in `docs/RATING_SHEET.md`): max 3 devices, no external speakers, no power, never ask judges to sc
an a QR code or click a link, leave nothing behind, stay silent during set-up.

## Likely questions: short answers

- **How do friends' phones stay in sync?** Every phone keeps its own full copy and turns each edit into a tiny stamped change. A small s
erver we run passes the changes between phones, and each field takes its newest change, so all phones end up identical whatever the orde
r. Changes made offline wait in a queue and are sent when the connection is back.
- **How do you know the merge is correct?** We wrote a test that makes three simulated phones edit at random (add, change, delete, vote)
and swap changes in random orders, 150 times; they always end identical. There are also tests with the real server over real HTTP.
- **Why not Firebase?** It needs an account and keys that a school project cannot safely hold, and it does not give us offline merging f
or free. Our server is about 150 lines, has no dependencies and is part of the repo. A real release would put the same protocol behind H
TTPS and accounts.
- **Is the sync secure?** A trip is protected by a 24-character random id that only the invite code reveals, code look-ups are rate limi
ted, and private things (your Scout chat, people's emails, flight tickets) never leave your phone. It is plain HTTP, so it is for a netw
ork you trust. That is a stated limit, not a surprise.
- **What if the venue wifi blocks it?** We use a phone hotspot. Without a network each phone still works fully on its own.
- **What did you test?** Unit tests for the money maths (exact settle-up), validation, time zones, password hashing against the official
test vector, the store, the assistant and the whole sync stack; browser tests of every main flow, a sideways-overflow audit of every scr
een, and two phones syncing. We have **not** yet tapped the camera QR scanner, on-device file upload and "share as picture" on a physica
l phone.
- **Why Expo / React Native?** One TypeScript codebase for iOS and Android, with native modules for notifications, maps, camera, files a
nd secure storage.
- **How is data protected?** Salted PBKDF2-HMAC-SHA256 password hashes (stretched, upgraded automatically), optional session persistenc
e, API key in Keychain/Keystore, files in app-private storage, private data kept off the sync server.
- **What makes it accessible?** Labels and roles on every control, adjustable text size, colour contrast, haptics toggle, Reduce Motion
respected, 6 languages, clear inline errors.
- **What happens with no internet?** Everything keeps working from local storage; weather and exchange rates show the last saved data (o
r an estimate) and say so.
```

- **How does the AI work offline?** It is our own intent-matching engine over the trip data (budget maths, settle-up algorithm, weather cache, place database). Claude is optional for open-ended questions.
- **Can I try to break it?** Yes: `docs/JUDGE_CHECKLIST.md` lists things to try and what should happen.

docs/DESIGN.md

TogetherWeGo design system

The idea: a boarding pass printed on paper. The app looks like the things you carry on a trip: tickets, stamps, a departures board, a passport. A warm paper page, black ink for every word and outline, one flat accent colour used like a highlighter, and a hard "plate" of ink under the things you can press. Nothing glows, blurs or fades. That restraint is what makes it look designed instead of generated.

The second idea is **quiet base, loud moments**: the frame is calm, and colour, motion and celebration are kept for the moments that matter (the countdown, a vote locking in, a quiz win). Each screen has at most one loud thing.

The colours live in `src/theme/palettes.ts` (plain data, unit-tested), the other tokens in `src/theme/index.ts`, the building blocks in `src/components/ui/`.

1. Look

Colour: a theme (accent) and a mode (light, dark, or follow the phone)

Part	Light (paper)	Dark (night flight)
Page	`#F4EFE4`	`#121212`
Card	`#FFFDF8`	`#1E1C19`
Sheet	`#FFFDF8`	`#2A2722` (height = a lighter surface)
Ink: text, every outline, main buttons	`#16130E`	`#F1EADC` (cream)
Secondary, muted text	`#4D463A`, `#6B6353`	`#C4BCAA`, `#A39B89`
Plate (the hard shadow)	ink	the accent

Six themes, each adding **one flat accent** (used as a fill, never as a gradient):

Theme	Light	Dark
Limoncello (default)	`#FFD23F`	`#F6CF4D`
Tomato	`#FF5B37`	`#FF8461`
Lagoon	`#25CDB8`	`#4FD8C8`
Bubblegum	`#FF8DB8`	`#FF9EC4`
Grape (picking it switches to dark)	`#A394FF`	`#B7A9FF`
Ink	black	cream

Rules:

* The accent is a **highlighter**: selected chips, the tab sticker, the plate under the main button in the light mode, the one stamp that matters, a bar that fills. Text on it is ink (`onAccent`). As text on paper use `accentText`, the accent darkened until it passes 4.5:1.
* **No gradients** (a scrim on a photo, so type stays readable, is the only exception) and **no soft or blurred shadows**.
* **Green, amber and red mean a state** (under budget, near, over; going, maybe, can't go; delete) and are never decoration. A state also gets an icon or a word, never colour alone.
* **No emoji as decoration.** Icons are Ionicons line icons. Emoji appear only where a person typed them (a nickname, a message).
* Every text pair is checked by `src/theme/__tests__/palettes.test.ts`: body text reaches 4.5:1 (WCAG AA) on every surface in all 6 themes in both modes.

Type: three voices (`T` in `src/components/ui/T.tsx`)

Voice	Font	Variants	Use
Serif	Fraunces, heavy	`display` 40, `headline` 26	Page titles and section titles. Sentence case. Never for paragraphs, never below 26 px.
Mono	Space Mono bold	`hero` 72, `numeral` 34, `numeralSm` 22, `kicker` 11 (caps)	Every number that matters (days, money, times, scores) and the small caps labels (DAY 02 / 05, POSITANO, WHO PAID). The print on a ticket.
Grotesk	Space Grotesk	`h1` 26, `h2` 21, `h3` 18, `title` 16, `body` 16, `bodySm` 15, `small` 14, `caption` 12.5, `micro` 11	Everything you read.

Shape, depth and space

* **Outline:** every card, button, input and chip has a 2 px ink outline (`line`). Dividers *inside* a card are 1 px (`hair`). Section titles sit over a 2 px rule (`Rule`).
* **Radius:** 10 for buttons and inputs, 12 for cards and images, 16 for sheets and dialogs, 8 for chips and tags. Round shapes are only for avatars, dots and the notches of a ticket. *Corner style* in Settings scales them.
* **Plate:** pressable things and the one standing card per screen have a solid plate 4 px right and down (`Plate`, `PlatePress`). Pressing slides the face onto the plate. Flat cards have no plate.
* **Space:** a 4-point scale (`space`). The gap **between** blocks (`space.section`, 32) is always bigger than the gap **inside** one (8 to 16).
* **Touch:** everything you tap is at least 44 x 44 points (`touch.min`) in its real size. Buttons are 44, 48 or 56 high.

Motifs (use these, do not invent new ones)

Motif	Component	Where
Ticket with a perforated stub and punched edges	`Ticket`	The trip on Home, Up next, flights, the invitation, the recap
Stamp: a short word in mono caps in an outline, slightly crooked	`Stamp`	Every status: GOING, LOCKED IN, SETTLED, OVER BUDGET, NEW BEST. It can slam in (`slam`).
Departures board digits that flip	`SplitFlap`	The countdown (days to go)

Plate under a button or card	`Plate`, `PlatePress`	Buttons, the floating add button, the standing card
Rule (solid) and **Perforation** (dashed)	`Rule`, `Perforation`	Under titles, between a ticket's halves
Sticker (accent block with an outline)	tab bar, `Fab`, chosen `Chip`	The thing that is selected or main

Motion (`src/components/ui/motion.tsx`)

* Pressing a plated thing slides it onto its plate (70 ms) and springs back; anything else squeezes to 96%. Small things take 150 to 250 ms, sheets 340 ms.
* Headlines slide up word by word (`StaggerText`); the countdown flips (`SplitFlap`); the photo hero zooms slowly and drifts against the scroll.
* **Everything that moves is gated** by `useMotion()`: it stops when *Animations* is off or the phone's *Reduce Motion* is on. Replace movement with a fade, never remove content.
* **Celebrations are rare and chosen** (`celebrate()`): a quiz win, an activity or poll locking in, a settled budget, "1 day to go", the end of *Make it yours*. The haptic always ticks; confetti follows the *Effects* setting (Chill none, Lively some, Max lots).
* **Loading** shows a `Skeleton` the size of the real thing. **Empty** screens are invitations (`EmptyState`): a line icon in a dashed ring, a serif prompt, one button.

2. Structure

* **Five tabs:** Home, Plan, Budget, Chat, More. Home first and More last, the ends that are easiest to find. A sticker slides to where you are.
* **Home**, in the order of the chosen layout (*Planner*, *Money* or *Party*; any block can be hidden or moved in Settings): the boarding pass (photo, days to go, crew, invite code), Up next, Weather, Group budget, Your crew, Play (today's trivia, the crew-points podium, something to vote on), then Picked for you and Discover.
* **Make it yours** is short: Welcome, Look (theme and mode), Interests, Home layout, Vibe (effects and a trivia reminder), then a reveal. One question per screen, every step skippable, every answer changes the app you can see, all editable later in Settings, with *Make it yours again*.
* **One main action per screen** in the thumb zone: the accent **+** button at the bottom right (`Fab`) or a black button at the bottom of a sheet.

3. The Laws of UX, and how the app uses them

Law	In plain words	What the app does
Hick's law	More choices, slower decisions	Five tabs; More is 4 groups of big tiles; settings are a few cards with the rest under *Advanced*; Add activity asks for a place first and hides the rest under *More options*; Home shows the blocks you chose.
Fitts's law	Big, close targets are easy	Every tap target is at least 44 pt (`e2e/targets.js` checks every screen); the main action sits in the thumb zone; destructive buttons are never next to the main one.
Jakob's law	People expect what other apps do	Bottom tabs, a back arrow, pull to refresh, bottom sheets, a share sheet, standard switches, a keyboard that does not cover the field.
Miller's law, **chunking**	Short lists are easier to hold	Lists are grouped under headings; the crew shows 4 faces and "+3"; sheets group fields in 3 to 5.
Proximity, **common region**	Things that belong together sit together	32 between blocks, 8 to 16 inside; each idea sits in its own outlined card.
Similarity	Things that look alike behave alike	One button style per kind of action; chips always choose; stamps are always labels; one icon family.
Prägnanz	People read the simplest shape	Flat paper, one outline weight, three radii, no gradients, no shadows.
Von Restorff effect	The odd one out is remembered	The accent and the one black button are the only loud things on a screen, so they are what you notice.
Aesthetic-usability effect	Beautiful feels easier	One idea (a boarding pass), applied everywhere, with real photos and a systematic type scale.
Doherty threshold	Answer within 400 ms	Optimistic updates, skeletons, saved places shown at once while web results load, 70 to 250 ms transitions.
Goal-gradient, **Zeigarnik**	Progress pulls you to finish	The step bar in *Make it yours*, the budget bar, packing and checklist progress, a countdown.
Peak-end rule	The high point and the end are remembered	Confetti and a stamp slam at the moments that matter, a "Building your trip HQ" reveal at the end of the survey, a trip recap you can share.
Serial position effect	First and last items stick	Home first and More last in the tab bar; the main action first in a list of actions.
Tesler's law	The app, not the user, absorbs complexity	Categories are guessed from the words, distances are worked out for you, currencies are converted, time zones are shown for you.
Postel's law	Be liberal in what you accept	"the museum" finds "Museum"; dates and times are checked kindly; half-typed forms are kept (`useDraft`).
Mental model, **paradox of the active user**	People start using it at once	A demo account, a short tour instead of a manual, familiar words.
Cognitive load, **selective attention**	Less to take in at once	Progressive disclosure (*More options*, *Advanced*, *Filter*), one question per screen, a calm base so the few bright things are seen.
Choice overload	Too many options freeze people	Presets first (three Home layouts, six themes), custom second.

4. Accessibility

* **Contrast:** unit-tested. **Text size:** the Settings switch scales text; the serif and mono voices grow less so layouts hold. **Reduce Motion** is respected and nothing is hidden when motion stops.
* **Targets:** at least 44 pt; `e2e/targets.js` fails on anything smaller.
* **Screen readers:** every control has a name; the countdown reads as a sentence; decorative marks are hidden.
* **Colour is never the only signal.**

5. Checking the design

```bash
npm test                        # palettes, contrast, layouts, confetti, migrations ...
npm run e2e                     # every screen at 3 sizes, no nested buttons, tap-target sizes, reduced motion ...
node e2e/shoot.js --modes=light,dark --themes=amalfi,midnight --routes=explore,itinerary   # pictures to look at
```

Before adding or changing a screen: use `Screen` and `Header`, the `T` variants and the tokens (no loose hex colours or font sizes),

one `Fab` or one primary `Button`, `Stamp` for statuses, an `EmptyState`, a `Skeleton` for anything that loads, `celebrate()` only for a real achievement, no emoji, no gradient, no soft shadow. Then look at it in the light and the dark mode.

docs/JUDGE_CHECKLIST.md

```
# Try to break it: a checklist for judges and teammates

The sample trip is tidy on purpose. These checks use the app the way a real person (or a grumpy judge) would, so you can see that nothin
g is hard-wired. Each line says what *should* happen.

## Start from nothing
- [ ] **Create a new account** (Sign Up) with your own name. You land in a short survey, then an empty Home that says *Plan your first g
roup trip*. There is no sample data.
- [ ] With the demo account: *Settings → Advanced → Clear the sample trips*. The app is empty again. *Reset app data* brings the sample
trips back.
- [ ] **Create a trip** from scratch (any city, dates, budget). Add an activity, a poll, an expense, a task, a flight.

## Type the wrong thing
- [ ] Sign up with `abc`, then `a@b`, then a 5-letter password: each gets a plain message under the field.
- [ ] Add an expense of `-5`, `abc`, `0`, `1000001`: refused with a clear message. `12.5` and `1,200` work.
- [ ] Join a trip with `ZZZ-9999`, `abc`, an empty code: refused. Join a trip you are already in: refused.
- [ ] Add a flight with an unknown airport (`XXX`), identical from/to airports, or arrival before departure: refused.
- [ ] Add an activity that overlaps another: you get a warning.
- [ ] Add an activity with nothing typed: *Enter a place*. Type `museum` or `pizza` in **Where do you want to go?**: matches appear belo
w the box, each with how far it is from you. Pick one and its address, kind, length and price fill in; or tap *Use “…”* to add your own
words.
- [ ] Type a name with an accent or apostrophe (`Zoë`, `O’Brien`): accepted.

## Break the connection
- [ ] Turn on **airplane mode**. Plan, vote, add expenses, open documents and the offline map, ask Scout. Weather and rates show saved d
ata and say so. The place search still matches the trip's saved places and says *No connection, so only saved places are shown*.
- [ ] With sync on (see below) make a change offline, then turn the connection back on: it reaches the other phone. The badge says *Offl
ine · N waiting* and then *Live*.

## Two phones
- [ ] Start the server (`npm run server`). Phone A: *Crew & roles → Share this trip across phones*. Phone B: *Join a trip*, type the cod
e. Both show **Live · 2 phones**.
- [ ] Send a chat message, cast a vote, add an expense, add a photo, a receipt, a PDF on one phone: each shows up on the other.
- [ ] Your **Scout AI** chat and other people's emails do not appear on the other phone.

## Accessibility and comfort
- [ ] *Settings → Text size → Extra large*: nothing is cut off or sideways-scrolling.
- [ ] Turn on the phone's **Reduce Motion**: the 360° view stops auto-rotating, entrances do not animate, Settings explains why.
- [ ] Switch language to 日本語, then back.
- [ ] Use a screen reader (VoiceOver / TalkBack): every button has a label.

## Look under the bonnet
- [ ] `npm test` and `npm run e2e` (see `e2e/README.md`): unit and browser tests pass.
- [ ] *More → How it works*: architecture, sync, security, validation, rubric map.
- [ ] `docs/RATING_SHEET.md`: every line of the rating sheet and where it is shown, including the weak spots.
```

docs/PLANNING.md

```
# TogetherWeGo — Planning documents

These are the planning artifacts behind the app (rating sheet: *Planning Process — tangible planning documents*).

## 1. Problem statement
Group trips fall apart in group chats: plans live in five apps, nobody knows who paid for what, flights land at different times in diffe
rent time zones, and the one person with the booking PDF has no signal. **TogetherWeGo puts the whole trip — plan, money, people and pap
erwork — in one shared place that still works offline.**

## 2. Personas
| Persona | Needs | Features that serve them |
|---|---|---|
| **Maya, 17 — the organizer** | Get six friends to agree, keep everyone on schedule | Invite codes, roles, polls & activity voting, liv
e reminders, printable planner |
| **Priya, 17 — the treasurer** | Track a shared budget fairly, settle up without awkwardness | Budget cap, multi-currency expenses, spl
its, fewest-payment settle-up |
| **Diego, 16 — the foodie** | Find great food nearby within budget | Nearby radius + price filter, budget finds, Trip Assistant “best f
ood nearby?” |
| **Mr. Kim — a parent chaperone** | Know where everyone is, keep documents & emergency info handy | Flights & arrivals board, offline d
ocument vault, emergency numbers, location sharing |

## 3. User journey
```mermaid
flowchart LR
  A[Onboarding] --> B[Sign in / Sign up]
  B --> C{Has a trip?}
  C -- no --> D[Create trip<br/>or Join with code]
  C -- yes --> E[Home]
  D --> F[Invite group<br/>share code to socials]
  F --> G[Assign roles]
  G --> H[Build itinerary<br/>vote & confirm]
  H --> I[Add flights<br/>time-zone board]
  I --> J[Budget & expenses]
```
```

```
  J --> K[Travel: reminders,<br/>chat + assistant, map,<br/>photo dump]
  K --> L[Mark complete →<br/>Recap & settle up]
```

4. Navigation map
```mermaid
flowchart TB
  Root[Root Stack<br/>Stack.Protected] --> Auth[Onboarding · Sign in · Sign up · Reset password]
  Root --> Tabs[Bottom tabs]
  Tabs --> T1[Home]
  Tabs --> T2[Plan + Vote]
  Tabs --> T3[Budget]
  Tabs --> T4[Chat]
  Tabs --> T5[More menu]
  Root --> More[Stack screens]
  More --> M1[Destination · All destinations · Nearby · Map]
  More --> M2[Flights · Weather · Currency · Nearest airport]
  More --> M3[Photo dump · Documents · Packing · Bucket list]
  More --> M4[Quiz · Survey · Recap · Emergency · Phrasebook]
  More --> M5[My trips · New/Edit trip · Join · Notifications · Settings · Help · About]
  More --> M6[Make it yours — personalization survey]
  More --> M7[Crew & roles · How it works]
  More --> M8[360° City Views — day / night panoramas]
```

5. Architecture
```mermaid
flowchart LR
  subgraph View[Views — src/app + src/components]
    S[Screens] --> UI[UI kit: Button, Sheet, Chip, Input, Slider…]
  end
  subgraph VM[View-model — selector hooks]
    H[useActiveTrip · useCurrentUser · tripStatus · balances · settlePlan]
  end
  subgraph Model[Model — src/store]
    Z[(Zustand + Immer store)] --> P[(AsyncStorage<br/>offline persistence)]
  end
  subgraph Svc[Services — src/services]
    W[weather] & C[currency] & N[notifications] & AI[assistant] & F[files] & PR[print] & SH[share] & L[location] & SE[secrets] & PS[plac
e search]
  end
  subgraph Sync[Sync — src/sync + server]
    E[Sync engine<br/>diff · merge · outbox] --> SV[(Sync server<br/>Node, runs on your computer)]
  end
  S --> H --> Z
  S -- actions --> Z
  S --> Svc
  Z <--> E
  W --> OM[(Open-Meteo)]
  C --> ER[(ExchangeRate API)]
  AI -. optional .-> CL[(Claude API)]
  PS -. optional .-> PH[(Photon / OpenStreetMap)]
  SE --> KC[(Keychain / Keystore)]
```

6. Data model
```mermaid
erDiagram
  ACCOUNT ||--o{ MEMBER : "is"
  TRIP ||--|{ MEMBER : has
  TRIP ||--o{ ACTIVITY : schedules
  TRIP ||--o{ EXPENSE : tracks
  TRIP ||--o{ FLIGHT : lists
  TRIP ||--o{ POLL : runs
  TRIP ||--o{ TASK : checklist
  TRIP ||--o{ DOC : vault
  TRIP ||--o{ MESSAGE : chat
  TRIP ||--o{ PHOTO : "photo dump"
  TRIP ||--o{ PACKING_ITEM : packs
  ACTIVITY }o--o{ MEMBER : "assigned / votes"
  EXPENSE }o--|| MEMBER : "paid by"
  EXPENSE }o--|{ MEMBER : "split between"
  FLIGHT }o--|| MEMBER : "belongs to"
```
Key decisions:
- **Dates vs. instants.** Itinerary times are wall-clock times in the destination’s zone (`2026-10-06` + `14:30`), because that is how p
eople plan. Flights are stored as UTC instants so they convert correctly to any time zone.
- **Money in one base currency.** Expenses are stored in USD (with the original amount kept) so totals and splits are always comparable;
views convert to home or local currency.
- **Majority voting.** A “voting” activity becomes “confirmed” when more than half of members vote yes.
- **Exact money.** Splits and balances are calculated in whole cents, and the fewest-payments settle-up is solved exactly for groups up
to 12 people, so the books always balance to the cent.
- **Passwords.** Salted PBKDF2-HMAC-SHA256, the round count stored with each hash so it can be raised later; older hashes are upgraded t
he next time someone signs in.
- **Sync.** Every field of a shared trip is a small register stamped by a hybrid logical clock; the newest stamp wins, so phones that ha
ve exchanged their changes hold identical trips whatever the order. Private data (Scout AI chat, other people's emails, flight tickets)

is never sent. Details in `docs/SYNC.md`.

7. Feature priorities (MoSCoW)
Must	Should	Could	Won’t (this version)
Trips + invite codes, roles, itinerary, voting, budget & splits, chat, flights & time zones, reminders, offline storage, validation	AI assistant, currency converter, weather, documents vault, map, nearby radius & filters, printable planner, social sharing	Quiz, survey recommendations, phrasebook TTS, recap, packing suggestions, emergency screen, 6 languages, personalization (themes, interests, home layout), animations, **live sync across phones** through a self-hosted server, invite QR codes	A hosted cloud service, accounts on a server, real Google OAuth, payments

8. Accessibility & UX rationale
- Reference-matched visual language: cream canvas, forest-green primary, soft-green highlights, pill navigation — calm and readable outdoors.
- Every icon-only control has an `accessibilityLabel`; toggles expose checked/selected state; sliders are adjustable with screen readers.
- Text size setting (Default / Large / Extra large) on top of the phone’s own font scaling.
- Personalization: 6 themes in light, dark or follow-my-phone, three Home layouts, adjustable corner shapes, a celebration setting (Chill, Lively, Max) and an *Animations* switch for people sensitive to motion. The look is documented in `docs/DESIGN.md`.
- Errors appear inline under the field in red with a plain-language fix (“Codes look like ABC-1234”).
- Destructive actions always confirm; toasts confirm every successful action.
- **Kept simple.** Each form asks for the few things that matter and folds the rest under *More options* (adding an activity: where, which day, what time). The kind of activity and of expense is guessed from the words typed. *More* is one grid of big buttons, *Settings* keeps the technical switches under *Advanced*, and the Plan's kind and length filters open from one *Filter* button.
- **Place search privacy.** Typing in *Where do you want to go?* sends the words (and a map point rounded to about 1 km, around the trip or the phone when it is at the trip) to Photon, a free OpenStreetMap search. Nothing else is sent, there is no account, and the distances are worked out on the phone. With no connection the trip's saved places still match.

9. Test log

9a. Automated (`npm test`, runs in about 5 seconds)
Area	What the tests prove
Money (`src/utils/__tests__/money.test.ts`)	Cents never drift (even 0.1 + 0.2), splits add up to the total with the odd cents handed out fairly, settle-up leaves every balance at zero, never needs more than (people − 1) payments on random groups, and finds fewer payments than the simple largest-first method when a group splits into pairs
Validation, time zones, passwords	Email/name/amount/date/invite/flight/IATA rules, daylight-saving edges, PBKDF2 against the RFC 7914 test vector, old hashes upgrade
Store (`src/store/__tests__`)	Sign-up/sign-in/reset, trips, joining, itinerary and voting, budget, polls, packing, photos, docs, attaching files, clearing sample data
Assistant	Local answers (budget, who owes who, currency, emergency numbers, places) and the Claude request: what is sent, the last 12 messages only, wrong key, refusal, empty answer, the 45-second limit
Sync (`src/sync/__tests__`, `server/__tests__`)	Merge convergence on 150 random three-phone runs, sharing and joining, offline queue, restart, private data, files, the real server over real HTTP, time limits, address parsing
Place search and guesses (`src/services/__tests__/placeSearch.test.ts`)	Saved places match by name and area, Photon answers become places (streets and bus stops dropped), the search looks around the trip first and anywhere if nothing is near, rounded point and box in the request, caching, time limit, cancel, merging without duplicates, activity and expense type guessed from words
360° view on the phone's native animation driver (`src/components/panorama/__tests__`)	The screen does not crash when day/night and position share a view with looping lights (it did, on a real phone, before this test)
Invites and QR	Link round trip for every address form, QR codes that a decoder reads back
Home ranking, pretend friends, Reduce Motion	The "Picked for you" ranking, canned replies, the motion rule

9b. Automated browser tests (`npm run e2e`, see `e2e/README.md`)
Eight scripts drive the web build in a phone-sized window: every main flow, the personalisation survey (including tapping a summary row to change an answer), the 360° views, Reduce Motion, a **sideways-overflow audit of every screen at 390 px, 320 px and with larger text**, a **no-button-inside-a-button audit** (the web warning *"`<button>` cannot contain a nested `<button>`"*), and a **two-phone sync run** against the real server (share, join, chat, expenses, offline catch-up, restart, a photo, a receipt and a PDF).

9c. Manual + earlier browser runs
Area	Checks
Auth	Empty-field errors, wrong password, sign-up validation & strength meter, reset-code flow, old password rejected after reset
Plan	Add-activity validation, overlap warning, vote → auto-confirm, **Lock in** (organizer only), reminder toggle, edit, delete, category + time filters
Budget	Receipt attach, expense validation, who-owes-who, settle up, home/local currency view
360° views	Drag/arrows/auto-rotate wrap past 0°, day/night toggle, landmark jump + facts, city picker, entry points (Home, destination, More)
Extras	Day/night hero toggle, Scout AI tab separate from the crew chat, packing assignment + fair split, first-run tour, story card
Budget	Amount validation, ¥ expense converted to USD, settle-up records payment, budget-cap validation
Collaboration	Join with empty/unknown/valid code, add-member validation (name, email), polls (vote, close), checklist add + empty-task error
Flights	Missing-field errors, unknown IATA rejected, flight saved, time-zone toggle
Personalization	Survey requires ≥1 interest, nickname length check, theme/corner choice survives restart, sign-in kept across the theme reload, Explore sections hide/show, “Picked for you” ranking
Platform	`tsc --noEmit` (strict) ✔, ESLint 0 problems ✔, iOS and Android bundles compile ✔, `expo prebuild` produces the native projects with the local-network and camera settings ✔
Not run on a physical phone	On-device file upload/download, the camera QR scanner and "share as picture" (they are native-only; everything around them is covered above)

docs/RATING_SHEET.md

Rating sheet map: where each point is earned

The FBLA 2026-27 *Mobile Application Development* rating sheet has 16 scored lines (110 points). This page lists every line, what we show the judges, and where to find it. It also lists the honest weak spots.

Design and code quality (25)

Line	Pts	Evidence	Where
Planning process	10	Personas, user journey, navigation map, MoSCoW priorities, data model, manual test log, all in one planning document. Terms used: persona, user journey, MoSCoW, MVP, MVVM, offline-first, one-way data flow.	`docs/PLANNING.md`
Classes, modules, components	5	UI kit of reusable components (`Screen`, `Card`, `Button`, `Press`, `Segmented`, `Donut`), trip widgets, one component per file, services for every outside system. The store is split into slices (`auth`, `trips`, `itinerary`, `budget`, `groupTools`, `social`, `files`, `demo`, `app`) and every large screen is a short composition of components (Home, Crew, Settings, Chat, Photos, Flights, Budget, Survey). No screen file is over 300 lines.	`src/components/ui/`, `src/components/home/`, `src/components/crew/`, `src/store/slices/`, `src/services/`
Architectural patterns	5	Layered MVVM: screens (views) read selector hooks and call store actions; services wrap APIs and device features; one-way data flow; Expo Router file-based navigation with `Stack.Protected` guards; platform-specific files (`TripMap.tsx` / `TripMap.web.tsx`). A separate **sync engine** that knows nothing about React or the network (store, storage and transport are passed in, so it is tested with two phones in memory), plus a small Node server.	`src/store/`, `src/app/`, `src/sync/`, `server/`, *More → How it works*
Innovation and creativity	5	**Live sync across phones** with a merge rule that gives identical trips whatever order changes arrive in (offline queue, photos and receipts included), 360° city views with day/night and night lights, skyline hero that follows the city's local time, offline Scout AI answering from trip data, "Lock in" voting, 7-step personalization survey with live themes, offline radar map, invite QR codes.	`src/sync/`, `src/app/panorama.tsx`, `src/services/assistant.ts`, `src/app/personalize.tsx`

User experience (20)

Line	Pts	Evidence	Where
UX design: user journey, rationale, accessibility	5	Journey map and rationale in the planning doc. Accessibility: labels and roles on every control, adjustable text size, high-contrast palette, haptics toggle, "Animations" off switch that also follows the phone's **Reduce Motion** setting, 6 languages. A browser audit checks that no screen scrolls sideways at 390 px, 320 px or with larger text. A written design system (a boarding pass printed on paper) with a table of how each Law of UX is applied, text contrast checked by unit tests in all 6 themes and both light and dark modes, and a browser test that every control is at least 44 x 44 points.	`docs/PLANNING.md`, *Settings*, `e2e/overflow.js`, `docs/DESIGN.md`, `src/theme/__tests__/palettes.test.ts`, `e2e/targets.js`
Intuitive UI or clear instructions	5	First-run guided tour, a 9-step *How to use* page with an FAQ, inline hints, empty states that say what to do next.	`src/components/GuidedTour.tsx`, `src/app/help.tsx`
Icons and graphics consistent	5	Custom app icon (a take-off plane on an ink tile with a lemon corner) reused as the in-app logo on sign-in, onboarding, tour and recap; one icon set and one palette throughout.	`assets/images/icon.png`, `src/components/Logo.tsx`
Input validation (syntax **and** meaning)	5	Syntax: email, invite-code format (`AML-2026`), time `HH:MM`, flight number, IATA code, amount parsing. Meaning: trip end after start, amount between limits, activity time-clash warning, duplicate email is refused, joining a trip you are already in is refused.	`src/utils/validation.ts`

Application functionality (25)

Line	Pts	Evidence	Where
Addresses all parts of the prompt	10	The topic names collaboration, communication, scheduling, budgeting, organization and multi-user trip management. Each is a feature set, and the mapping is explained inside the app.	README section 2, *More → How it works → Rubric map*
Social media integration	5	Direct deep links into WhatsApp, X, Facebook, Messenger, Telegram, Instagram, SMS and email, plus the native share sheet. Photos and the **recap story card share as pictures**. Invites are a code, a link and a **QR code** (scan it inside the app or with a camera); the link opens the Join screen with the code filled in, and also carries the sync server's address. Invite link and QR round trips are unit tested. (Logging in with or posting to a social network from inside the app needs OAuth credentials and app review, which is out of reach for a student project.)	`src/services/share.ts`, `src/sync/invite.ts`, `src/components/crew/InviteQrSheet.tsx`, *Recap*
Data handling and storage	5	Zustand store persisted to local storage with versioned migrations (integrity); passwords salted and stretched with PBKDF2, session off when "Remember me" is off, API key in the Keychain/Keystore (protection); money kept in whole cents so totals never drift; export a trip, clear the sample data and reset all data (control); whole app works offline; shared trips are kept identical across phones and private data (Scout chat, emails, tickets) never leaves the phone.	`src/store/`, `src/sync/`, `src/services/secrets.ts`, `src/utils/password.ts`, *Settings*
Documentation and copyright	5	README, planning doc (with a test log), sync design, build guide, judge checklist and attributions: every library with its licence, every photo with its Unsplash source URL, fonts under OFL, no logos or trademarks.	`README.md`, `docs/`, `ATTRIBUTIONS.md`

Evidence that it works

* `npm test`: unit tests for money, validation, time zones, passwords, the store, the assistant, invites, QR codes and the whole sync stack (merge convergence on 150 random three-phone runs; the real server over HTTP).
* `npm run e2e`: browser tests of every main flow, a sideways-overflow audit of every screen, Reduce Motion, and two phones syncing through the real server.
* `docs/JUDGE_CHECKLIST.md`: things to try (wrong input, airplane mode, two phones, large text) and what should happen.

Presentation delivery (30)

Line	Pts	Prepared	Where
Organized, clearly stated	10	A timed 7-minute script built around this very sheet, with a speaker split.	`docs/DEMO_SCRIPT.md`
Confidence, body language, eye contact, voice	10	Rehearse with the table below. Judges mark this by watching you, not the app.	n/a
Answering questions	10	Short answers to the likely questions (sync, security, offline, why React Native, accessibility).	`docs/DEMO_SCRIPT.md`

Presentation protocols (10): the checklist

- [ ] At most **3 devices** (e.g. phone + laptop for slides + a tablet). Screens laptop-sized or smaller.
- [ ] **No external speakers**: audio (phrasebook voice) comes from the phone itself.
- [ ] **No power outlet**: charge everything beforehand.
- [ ] **Links and QR codes**: you may show them, but judges must never scan or click them. Do not make the demo depend on a QR code or l

ink (for the two-phone demo type the invite code on the second phone).
- [ ] **Two-phone sync demo**: phone A, phone B and the laptop (which shows the slides *and* runs `npm run server`) are exactly 3 device
s. Put all three on a **phone hotspot** (venue wifi often blocks devices from seeing each other), start the server before set-up ends an
d check *Settings → Sync across phones → Test connection* on both phones.
- [ ] Nothing left with the judges afterwards. Pack up printouts.
- [ ] No food or animals. Topic stays "Together We Go: Group Trip Planner".
- [ ] Do not talk to the judges during the 3-minute set-up.
- [ ] Every teammate presents a part.
- [ ] Venue wifi may fail: use *Settings → Advanced → Demo timeline*, airplane-mode offline demo, and the screenshots in `docs/screensho
ts/` as a backup.

Honest weak spots (so you are not surprised in Q&A)

1. **Standalone run.** The rules say the solution must run standalone. Expo Go needs your laptop's dev server, so build an installed cop
y before the event (`docs/BUILDING.md`): `npx eas-cli@latest build -p android --profile preview` gives an `.apk` (free Expo account). iP
hone standalone needs an Apple Developer account. **No installed build was made while writing this**: it needs an Expo account that only
the team can create. The build configuration was checked with `expo prebuild`.
2. **Sync needs a server you run.** Phones share a trip through `npm run server` on a computer they can reach. There is no hosted servic
e and no accounts, and the traffic is plain HTTP, so it belongs on a network you trust. Say so before a judge asks (`docs/SYNC.md`, "Lim
its").
3. **Not run on a physical phone.** The on-device file upload and download, the camera QR scanner and "share as picture" are native-onl
y. They compile into the iPhone bundle and the logic around them is tested, but nobody has tapped them on a phone yet. Try them once bef
ore the event.
4. **Same-field edits.** If two people change the *same* field at the same instant, the later change wins and the other is lost. Chat, v
otes, expenses and photos are separate items, so they never clash.
5. **Passwords** use salted PBKDF2-HMAC-SHA256 with 10,000 rounds (a server would use 600,000 or Argon2id). The count is a judgement cal
l for a phone's JavaScript engine: it takes about 20 ms in Node, and we expect well under a second on a phone, but we have **not timed i
t on a phone**. The work is split into 8 ms slices so the screen stays responsive, and the round count is stored in each hash so it can
be raised later. Accounts live on the phone, not on a server.
6. **Live weather and exchange rates** need internet. Offline, the app shows the last saved data and labels it.
7. **"Continue with Google"** is a simulated account chooser and says so on screen.

docs/SYNC.md

How phones stay in sync

Friends can each plan on their own phone and see the same trip: chat, votes, the itinerary, expenses, the checklist, photos, documents a
nd receipts. This page explains how it works, what is private, how it was tested and where it stops.

Try it (two minutes)

1. On the computer that runs the app, open a **second terminal** in the project folder and start the sync server:

   ```bash
   npm run server
   ```

   It prints addresses such as `http://192.168.1.20:8787`. On Windows, click **Allow access** (private networks) when the firewall asks.

2. Start the app as usual (`npx expo start`). While developing, phones **find the server by themselves** (they use the same computer add
ress that Expo Go used). If a phone cannot, enter the address in **Settings → Advanced → Sync across phones** and tap *Test connection*.

3. Phone A: **More → Crew & roles → Share this trip across phones**.
4. Phone B: **Join a trip**, type the invite code (or scan the QR code from *Crew & roles → QR code*; the QR code and invite link carry
the server's address once the trip is shared, and the phone asks before using it). The trip appears with its chat, plan and photos, and
the badge says **Live · 2 phones**.

Everything either phone changes shows up on the other within a second or two. Turn on airplane mode, make changes, turn it off: they cat
ch up.

> Rules of the competition: judges must not scan QR codes or click links, so during the presentation type the code on the second phone i
nstead.

What is shared and what is not

Shared with the group	Stays on your phone
Chat messages, polls and votes, the checklist, packing list	Your **Scout AI** conversation
Itinerary, activity votes, flights (without ticket files)	Other people's **email addresses**
Expenses and settle-up, members and roles	**Flight ticket files** (they hold personal details)
Photos, documents and receipt photos (as files)	Your password, settings, language, theme

The pieces

```
 phone A                                                  phone B
┌──────────────────┐                                    ┌──────────────────┐
│ app store (Zustand)│                                  │ app store (Zustand)│
│        │ diff    │                                    │       ▲ rebuild  │
│        ▼         │  ops   ┌──────────────┐  ops       │       │          │
│  sync engine ──────────────▶ sync server ──────────────▶ sync engine      │
│  registers + outbox│ ◀──────  (Node, no  │ ◀──────── registers+outbox│
└──────────────────┘  long   │  accounts)   │  long      └──────────────────┘
                      poll   │ log per trip │  poll
                             │ files per trip│
                             └──────────────┘
```

* `src/sync/schema.ts` says which parts of a trip are shared and how each part merges.
* `src/sync/crdt.ts` turns "what changed" into small stamped changes (ops) and merges ops into **registers**.
* `src/sync/engine.ts` watches the store, queues ops, sends them, fetches friends' ops, and writes the merged trip back into the store. It knows nothing about React or the network library, which is why the tests can run two phones against one server in memory.
* `src/sync/transport.ts` is the HTTP client (every request has a time limit). `server/` is the server.
* `src/sync/blobs.ts` moves photos, documents and receipts (see below).

The merge rule

Every field of the trip is a **register** that remembers the stamp of the change that last wrote it. **The newest stamp wins**, field by field. Stamps are hybrid logical clocks (`<time>-<counter>-<phone>`), so they stay in order even when two phones' clocks disagree a little.

* Two people editing *different* things both win (one renames an activity, the other changes its time).
* Two people editing the *same* field: the later change wins, on every phone.
* Lists are lists of items with ids. Adding an item never conflicts. A delete leaves a small tombstone so a late change to a deleted item cannot bring it back.
* Votes and "who is assigned" are sets or maps, so two people voting at the same moment both count.

Because the rule is deterministic, phones that have seen the same changes hold the same trip, **no matter in what order the changes arrived**. The tests check this with random runs (see below).

Offline and restarts

Changes go into an **outbox** that is saved on the phone. If the server is out of reach the badge says *Offline · 3 waiting* and the app keeps working. When the connection returns the outbox is sent. If the app is closed before that, the unsent change is found again on the next start (the engine compares the saved trip with what it last sent).

Files

Photos, documents and receipts are too big to be changes. The phone that has the file uploads it (`PUT /v1/trips/:id/files/:fileId`) and then writes the server's file id on the item, which syncs like any other change. Friends' phones see the id, download the file into the app's private storage and show it; after that it opens offline too. Pictures that ship with the app are shared by name (`asset:amalfi`) instead of being uploaded. A file over 8 MB stays on the phone that has it, and that person is told once.

The server

`server/index.js` is about 150 lines of Node with no dependencies. It keeps, for every shared trip, an append-only log of ops and a folder of files, in `server/data/` (a JSON file and a `files/` folder). It never has to understand an op.

Request	What it does
`POST /v1/trips/:id/ops`	send changes (up to 500, 1 MB)
`GET /v1/trips/:id/ops?since=N&wait=ms`	get changes after N; if there are none, **wait** up to 25 s for one (long polling), so news arrives at once
`PUT` / `GET /v1/trips/:id/files/:fileId`	send / fetch a file (8 MB each, 150 MB per trip)
`GET /v1/codes/:code`	invite code to trip id (60 tries a minute per address)
`GET /health`	is it alive, how many trips

Security model (read this before you rely on it). There are no accounts. A trip is protected by its **id**: 24 random characters (about 124 bits) from the phone's secure random generator. A phone only learns the id by entering the invite code, and code look-ups are rate limited. Traffic is plain HTTP, so **only run the server on a network you trust** (home wifi, a phone hotspot), not on a public network. Anyone who can read the network could read the trip. A production version would use HTTPS and real accounts.

What was tested

* `src/sync/__tests__/crdt.test.ts`: the merge rule. **150 random runs** where three phones make random edits (add, change, delete, vote) and exchange changes in random orders always end with identical trips.
* `src/sync/__tests__/engine.test.ts`: sharing, joining, edits both ways, offline queueing, restart, a change made while the app was closed, private data staying private, paging a trip of 700 messages, an unknown code, a code that is already taken.
* `src/sync/__tests__/e2e.test.ts`: the same engines talking to the **real server over real HTTP**, including the long-poll wake-up.
* `src/sync/__tests__/blobs.test.ts`: uploading, downloading, retrying, too-big files, deleted files, bundled pictures.
* `src/sync/__tests__/transport.test.ts`: time limits, cancelling, error kinds, and server-address checks against **both** Node's `URL` and the phone's looser `URL` class.
* `server/__tests__/server.test.js`: the store and the HTTP API, including limits and restarts.
* Browser test (`e2e/twophones.js`): two separate browser profiles and one real server. Phone A shares, phone B joins from a decoded QR link, chat and expenses travel both ways, A goes offline and comes back, B restarts, and a photo, a receipt and a PDF reach the other phone.

Limits (the honest list)

* The server is a program you run on your own computer; there is no hosted service. Phones need to reach it (same wifi, hotspot, or the address typed in Settings).
* Files uploaded to the server are not deleted when the item is deleted (the 150 MB per trip limit keeps this bounded).
* The first version of "last writer wins" means two people editing the *same* field at the same moment lose one of the edits (the later one stays). Chat messages, votes and expenses are separate items, so they never clash.
* The on-device file upload and download code, the camera QR scanner and "share as picture" use the phone's native modules. They build into the iPhone bundle and are covered by the web tests of everything around them, but they were **not run on a physical phone** while this was written.